

STAR RX72N - rx_nanopb Library
1.0.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 Data Structure Index	7
2.1 Data Structures	7
3 File Index	9
3.1 File List	9
4 Directory Documentation	11
4.1 libs/rx_nanopb/nanopb/extra Directory Reference	11
4.2 libs/rx_nanopb/inc/gen Directory Reference	11
4.3 libs/rx_nanopb/inc/gen/google Directory Reference	12
4.4 libs/rx_nanopb/inc Directory Reference	12
4.5 libs Directory Reference	13
4.6 libs/rx_nanopb/nanopb Directory Reference	13
4.7 libs/rx_nanopb/nanopb/spm_headers/nanopb Directory Reference	14
4.8 libs/rx_nanopb/nanopb/spm-test/objc Directory Reference	14
4.9 libs/rx_nanopb/inc/gen/google/protobuf Directory Reference	15
4.10 libs/rx_nanopb Directory Reference	15
4.11 libs/rx_nanopb/nanopb/spm-test Directory Reference	16
4.12 libs/rx_nanopb/nanopb/spm_headers Directory Reference	16
4.13 libs/rx_nanopb/src Directory Reference	17
4.14 libs/rx_nanopb/inc/gen/star Directory Reference	17
4.15 libs/rx_nanopb/inc/gen/star/v1 Directory Reference	18
5 Data Structure Documentation	19
5.1 pb_callback_s.funcs Union Reference	19
5.1.1 Detailed Description	19
5.1.2 Field Documentation	19
5.1.2.1 decode [1/2]	19
5.1.2.2 decode [2/2]	19
5.1.2.3 encode [1/2]	19
5.1.2.4 encode [2/2]	19
5.2 pb_callback_t Struct Reference	20
5.2.1 Detailed Description	20
5.2.2 Field Documentation	20
5.2.2.1 arg	20
5.2.2.2 [union] [1/2]	20
5.2.2.3 [union] [2/2]	21
5.3 pb_extension_type_t Struct Reference	21
5.3.1 Detailed Description	21
5.3.2 Field Documentation	21
5.3.2.1 arg	21
5.3.2.2 decode	22

5.3.2.3 encode	22
5.4 pb_istream_t Struct Reference	22
5.4.1 Detailed Description	22
5.4.2 Field Documentation	23
5.4.2.1 bytes_left	23
5.4.2.2 callback	23
5.4.2.3 errmsg	23
5.4.2.4 state	23
5.5 pb_msgdesc_t Struct Reference	23
5.5.1 Detailed Description	24
5.5.2 Field Documentation	24
5.5.2.1 default_value	24
5.5.2.2 field_callback	24
5.5.2.3 field_count	24
5.5.2.4 field_info	24
5.5.2.5 largest_tag	24
5.5.2.6 required_field_count	25
5.5.2.7 submsg_info	25
5.6 pb_ostream_t Struct Reference	25
5.6.1 Detailed Description	25
5.6.2 Field Documentation	26
5.6.2.1 bytes_written	26
5.6.2.2 callback	26
5.6.2.3 errmsg	26
5.6.2.4 max_size	26
5.6.2.5 state	26
5.7 star_v1_STAREnvelope Struct Reference	26
5.7.1 Detailed Description	27
5.7.2 Field Documentation	27
5.7.2.1 [union]	27
5.7.2.2 seq	27
5.7.2.3 ts_ms	27
5.7.2.4 which_payload	27
5.8 star_v1_WireMessage Struct Reference	28
5.8.1 Detailed Description	28
5.8.2 Field Documentation	28
5.8.2.1 [union]	28
5.8.2.2 which_payload	28
6 File Documentation	29
6.1 libs/rx_nanopb/inc/gen/google/protobuf/duration.pb.c File Reference	29
6.2 duration.pb.c	29
6.3 libs/rx_nanopb/inc/gen/google/protobuf/duration.pb.h File Reference	30

6.3.1 Data Structure Documentation	31
6.3.1.1 struct google_protobuf_Duration	31
6.3.2 Macro Definition Documentation	31
6.3.2.1 google_protobuf_Duration_CALLBACK	31
6.3.2.2 google_protobuf_Duration_DEFAULT	31
6.3.2.3 google_protobuf_Duration_FIELDLIST	31
6.3.2.4 google_protobuf_Duration_fields	32
6.3.2.5 google_protobuf_Duration_init_default	32
6.3.2.6 google_protobuf_Duration_init_zero	32
6.3.2.7 google_protobuf_Duration_nanos_tag	32
6.3.2.8 google_protobuf_Duration_seconds_tag	32
6.3.2.9 google_protobuf_Duration_size	32
6.3.2.10 GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_DURATION_PB_H_MAX_SIZE	33
6.3.3 Variable Documentation	33
6.3.3.1 google_protobuf_Duration_msg	33
6.4 duration.pb.h	33
6.5 libs/rx_nanopb/inc/gen/google/protobuf/timestamp.pb.c File Reference	34
6.6 timestamp.pb.c	35
6.7 libs/rx_nanopb/inc/gen/google/protobuf/timestamp.pb.h File Reference	35
6.7.1 Data Structure Documentation	36
6.7.1.1 struct google_protobuf_Timestamp	36
6.7.2 Macro Definition Documentation	37
6.7.2.1 GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_TIMESTAMP_PB_H_MAX_SIZE	37
6.7.2.2 google_protobuf_Timestamp_CALLBACK	37
6.7.2.3 google_protobuf_Timestamp_DEFAULT	37
6.7.2.4 google_protobuf_Timestamp_FIELDLIST	37
6.7.2.5 google_protobuf_Timestamp_fields	37
6.7.2.6 google_protobuf_Timestamp_init_default	38
6.7.2.7 google_protobuf_Timestamp_init_zero	38
6.7.2.8 google_protobuf_Timestamp_nanos_tag	38
6.7.2.9 google_protobuf_Timestamp_seconds_tag	38
6.7.2.10 google_protobuf_Timestamp_size	38
6.7.3 Variable Documentation	38
6.7.3.1 google_protobuf_Timestamp_msg	38
6.8 timestamp.pb.h	39
6.9 libs/rx_nanopb/inc/gen/star/v1/common.pb.c File Reference	40
6.10 common.pb.c	41
6.11 libs/rx_nanopb/inc/gen/star/v1/common.pb.h File Reference	41
6.11.1 Data Structure Documentation	44
6.11.1.1 struct star_v1_RequestHeader	44
6.11.1.2 struct star_v1_ResponseHeader	45
6.11.1.3 struct star_v1_Vector3	45
6.11.1.4 struct star_v1_Quaternion	46

6.11.1.5 struct star_v1_SE2Pose	47
6.11.1.6 struct star_v1_SE2Velocity	47
6.11.2 Macro Definition Documentation	48
6.11.2.1 _star_v1_Status_ARRAYSIZE	48
6.11.2.2 _star_v1_Status_MAX	48
6.11.2.3 _star_v1_Status_MIN	48
6.11.2.4 star_v1_Quaternion_CALLBACK	48
6.11.2.5 star_v1_Quaternion_DEFAULT	48
6.11.2.6 star_v1_Quaternion_FIELDLIST	48
6.11.2.7 star_v1_Quaternion_fields	49
6.11.2.8 star_v1_Quaternion_init_default	49
6.11.2.9 star_v1_Quaternion_init_zero	49
6.11.2.10 star_v1_Quaternion_size	49
6.11.2.11 star_v1_Quaternion_w_tag	49
6.11.2.12 star_v1_Quaternion_x_tag	49
6.11.2.13 star_v1_Quaternion_y_tag	49
6.11.2.14 star_v1_Quaternion_z_tag	49
6.11.2.15 star_v1_RequestHeader_CALLBACK	50
6.11.2.16 star_v1_RequestHeader_client_timestamp_MSGTYPE	50
6.11.2.17 star_v1_RequestHeader_client_timestamp_tag	50
6.11.2.18 star_v1_RequestHeader_client_version_tag	50
6.11.2.19 star_v1_RequestHeader_DEFAULT	50
6.11.2.20 star_v1_RequestHeader_FIELDLIST	50
6.11.2.21 star_v1_RequestHeader_fields	51
6.11.2.22 star_v1_RequestHeader_init_default	51
6.11.2.23 star_v1_RequestHeader_init_zero	51
6.11.2.24 star_v1_RequestHeader_request_id_tag	51
6.11.2.25 star_v1_RequestHeader_size	51
6.11.2.26 star_v1_RequestHeader_timeout_MSGTYPE	51
6.11.2.27 star_v1_RequestHeader_timeout_tag	51
6.11.2.28 star_v1_ResponseHeader_CALLBACK	51
6.11.2.29 star_v1_ResponseHeader_DEFAULT	52
6.11.2.30 star_v1_ResponseHeader_error_message_tag	52
6.11.2.31 star_v1_ResponseHeader_FIELDLIST	52
6.11.2.32 star_v1_ResponseHeader_fields	52
6.11.2.33 star_v1_ResponseHeader_init_default	52
6.11.2.34 star_v1_ResponseHeader_init_zero	52
6.11.2.35 star_v1_ResponseHeader_latency_us_tag	53
6.11.2.36 star_v1_ResponseHeader_request_id_tag	53
6.11.2.37 star_v1_ResponseHeader_server_timestamp_MSGTYPE	53
6.11.2.38 star_v1_ResponseHeader_server_timestamp_tag	53
6.11.2.39 star_v1_ResponseHeader_size	53
6.11.2.40 star_v1_ResponseHeader_status_ENUMTYPE	53

6.11.2.41	star_v1_ResponseHeader_status_tag	53
6.11.2.42	star_v1_SE2Pose_angle_rad_tag	53
6.11.2.43	star_v1_SE2Pose_CALLBACK	54
6.11.2.44	star_v1_SE2Pose_DEFAULT	54
6.11.2.45	star_v1_SE2Pose_FIELDLIST	54
6.11.2.46	star_v1_SE2Pose_fields	54
6.11.2.47	star_v1_SE2Pose_init_default	54
6.11.2.48	star_v1_SE2Pose_init_zero	54
6.11.2.49	star_v1_SE2Pose_size	54
6.11.2.50	star_v1_SE2Pose_x_m_tag	55
6.11.2.51	star_v1_SE2Pose_y_m_tag	55
6.11.2.52	star_v1_SE2Velocity_CALLBACK	55
6.11.2.53	star_v1_SE2Velocity_DEFAULT	55
6.11.2.54	star_v1_SE2Velocity_FIELDLIST	55
6.11.2.55	star_v1_SE2Velocity_fields	55
6.11.2.56	star_v1_SE2Velocity_init_default	55
6.11.2.57	star_v1_SE2Velocity_init_zero	56
6.11.2.58	star_v1_SE2Velocity_omega_rad_per_s_tag	56
6.11.2.59	star_v1_SE2Velocity_size	56
6.11.2.60	star_v1_SE2Velocity_vx_mps_tag	56
6.11.2.61	star_v1_SE2Velocity_vy_mps_tag	56
6.11.2.62	STAR_V1_STAR_V1_COMMON_PB_H_MAX_SIZE	56
6.11.2.63	star_v1_Vector3_CALLBACK	56
6.11.2.64	star_v1_Vector3_DEFAULT	56
6.11.2.65	star_v1_Vector3_FIELDLIST	57
6.11.2.66	star_v1_Vector3_fields	57
6.11.2.67	star_v1_Vector3_init_default	57
6.11.2.68	star_v1_Vector3_init_zero	57
6.11.2.69	star_v1_Vector3_size	57
6.11.2.70	star_v1_Vector3_x_tag	57
6.11.2.71	star_v1_Vector3_y_tag	57
6.11.2.72	star_v1_Vector3_z_tag	58
6.11.3	Enumeration Type Documentation	58
6.11.3.1	star_v1_Status	58
6.11.4	Variable Documentation	58
6.11.4.1	star_v1_Quaternion_msg	58
6.11.4.2	star_v1_RequestHeader_msg	58
6.11.4.3	star_v1_ResponseHeader_msg	59
6.11.4.4	star_v1_SE2Pose_msg	59
6.11.4.5	star_v1_SE2Velocity_msg	59
6.11.4.6	star_v1_Vector3_msg	59
6.12	common.pb.h	59
6.13	libs/rx_nanopb/inc/gen/star/v1/configuration.pb.c File Reference	62

6.14 configuration.pb.c	63
6.15 libs/rx_nanopb/inc/gen/star/v1/configuration.pb.h File Reference	64
6.15.1 Data Structure Documentation	72
6.15.1.1 struct star_v1_GetConfigurationRequest	72
6.15.1.2 struct star_v1_ResetToDefaultsRequest	73
6.15.1.3 struct star_v1_SaveConfigurationRequest	73
6.15.1.4 struct star_v1_SaveConfigurationResponse	74
6.15.1.5 struct star_v1_GetMotorPidConfigRequest	74
6.15.1.6 struct star_v1_GetMotorPidConfigResponse	75
6.15.1.7 struct star_v1_SetMotorPidConfigRequest	75
6.15.1.8 struct star_v1_RetransmitConfig	76
6.15.1.9 struct star_v1_SetRetransmitConfigRequest	77
6.15.1.10 struct star_v1_SetRetransmitConfigResponse	77
6.15.1.11 struct star_v1_MotorPidConfiguration	78
6.15.1.12 struct star_v1_CurrentSensorCalibration	79
6.15.1.13 struct star_v1_TemperatureCalibration	79
6.15.1.14 struct star_v1_SafetyThresholds	80
6.15.1.15 struct star_v1_EncoderConfiguration	80
6.15.1.16 struct star_v1_TimingConfiguration	81
6.15.1.17 struct star_v1_SystemConfiguration	81
6.15.1.18 struct star_v1_GetConfigurationResponse	82
6.15.1.19 struct star_v1_SetConfigurationRequest	83
6.15.1.20 struct star_v1_ResetToDefaultsResponse	84
6.15.1.21 struct star_v1_ValidateConfigurationRequest	85
6.15.1.22 struct star_v1_ConfigValidationError	86
6.15.1.23 struct star_v1_ConfigValidationResult	87
6.15.1.24 struct star_v1_SetConfigurationResponse	87
6.15.1.25 struct star_v1_ValidateConfigurationResponse	88
6.15.1.26 struct star_v1_SetMotorPidConfigResponse	89
6.15.2 Macro Definition Documentation	89
6.15.2.1 _star_v1_ConfigErrorCode_ARRAYSIZE	89
6.15.2.2 _star_v1_ConfigErrorCode_MAX	89
6.15.2.3 _star_v1_ConfigErrorCode_MIN	89
6.15.2.4 _star_v1_ConfigValidationStatus_ARRAYSIZE	90
6.15.2.5 _star_v1_ConfigValidationStatus_MAX	90
6.15.2.6 _star_v1_ConfigValidationStatus_MIN	90
6.15.2.7 star_v1_ConfigValidationError_actual_value_tag	90
6.15.2.8 star_v1_ConfigValidationError_CALLBACK	90
6.15.2.9 star_v1_ConfigValidationError_DEFAULT	90
6.15.2.10 star_v1_ConfigValidationError_error_code_ENUMTYPE	90
6.15.2.11 star_v1_ConfigValidationError_error_code_tag	90
6.15.2.12 star_v1_ConfigValidationError_expected_constraint_tag	91
6.15.2.13 star_v1_ConfigValidationError_field_path_tag	91

6.15.2.14	star_v1_ConfigValidationError_FIELDLIST	91
6.15.2.15	star_v1_ConfigValidationError_fields	91
6.15.2.16	star_v1_ConfigValidationError_init_default	91
6.15.2.17	star_v1_ConfigValidationError_init_zero	91
6.15.2.18	star_v1_ConfigValidationError_message_tag	92
6.15.2.19	star_v1_ConfigValidationError_size	92
6.15.2.20	star_v1_ConfigValidationResult_CALLBACK	92
6.15.2.21	star_v1_ConfigValidationResult_DEFAULT	92
6.15.2.22	star_v1_ConfigValidationResult_errors_MSGTYPE	92
6.15.2.23	star_v1_ConfigValidationResult_errors_tag	92
6.15.2.24	star_v1_ConfigValidationResult_FIELDLIST	92
6.15.2.25	star_v1_ConfigValidationResult_fields	93
6.15.2.26	star_v1_ConfigValidationResult_init_default	93
6.15.2.27	star_v1_ConfigValidationResult_init_zero	93
6.15.2.28	star_v1_ConfigValidationResult_size	93
6.15.2.29	star_v1_ConfigValidationResult_status_ENUMTYPE	93
6.15.2.30	star_v1_ConfigValidationResult_status_tag	93
6.15.2.31	star_v1_CurrentSensorCalibration_CALLBACK	93
6.15.2.32	star_v1_CurrentSensorCalibration_DEFAULT	94
6.15.2.33	star_v1_CurrentSensorCalibration_FIELDLIST	94
6.15.2.34	star_v1_CurrentSensorCalibration_fields	94
6.15.2.35	star_v1_CurrentSensorCalibration_init_default	94
6.15.2.36	star_v1_CurrentSensorCalibration_init_zero	94
6.15.2.37	star_v1_CurrentSensorCalibration_offset_ma_tag	94
6.15.2.38	star_v1_CurrentSensorCalibration_scale_factor_tag	94
6.15.2.39	star_v1_EncoderConfiguration_CALLBACK	95
6.15.2.40	star_v1_EncoderConfiguration_DEFAULT	95
6.15.2.41	star_v1_EncoderConfiguration_edges_per_revolution_tag	95
6.15.2.42	star_v1_EncoderConfiguration_FIELDLIST	95
6.15.2.43	star_v1_EncoderConfiguration_fields	95
6.15.2.44	star_v1_EncoderConfiguration_gear_ratio_tag	95
6.15.2.45	star_v1_EncoderConfiguration_init_default	95
6.15.2.46	star_v1_EncoderConfiguration_init_zero	96
6.15.2.47	star_v1_EncoderConfiguration_size	96
6.15.2.48	star_v1_EncoderConfiguration_wheel_diameter_m_tag	96
6.15.2.49	star_v1_GetConfigurationRequest_CALLBACK	96
6.15.2.50	star_v1_GetConfigurationRequest_DEFAULT	96
6.15.2.51	star_v1_GetConfigurationRequest_FIELDLIST	96
6.15.2.52	star_v1_GetConfigurationRequest_fields	96
6.15.2.53	star_v1_GetConfigurationRequest_header_MSGTYPE	97
6.15.2.54	star_v1_GetConfigurationRequest_header_tag	97
6.15.2.55	star_v1_GetConfigurationRequest_init_default	97
6.15.2.56	star_v1_GetConfigurationRequest_init_zero	97

6.15.2.57	star_v1_GetConfigurationRequest_size	97
6.15.2.58	star_v1_GetConfigurationResponse_CALLBACK	97
6.15.2.59	star_v1_GetConfigurationResponse_configuration_MSGTYPE	97
6.15.2.60	star_v1_GetConfigurationResponse_configuration_tag	97
6.15.2.61	star_v1_GetConfigurationResponse_DEFAULT	98
6.15.2.62	star_v1_GetConfigurationResponse_FIELDLIST	98
6.15.2.63	star_v1_GetConfigurationResponse_fields	98
6.15.2.64	star_v1_GetConfigurationResponse_header_MSGTYPE	98
6.15.2.65	star_v1_GetConfigurationResponse_header_tag	98
6.15.2.66	star_v1_GetConfigurationResponse_init_default	98
6.15.2.67	star_v1_GetConfigurationResponse_init_zero	98
6.15.2.68	star_v1_GetMotorPidConfigRequest_CALLBACK	99
6.15.2.69	star_v1_GetMotorPidConfigRequest_DEFAULT	99
6.15.2.70	star_v1_GetMotorPidConfigRequest_FIELDLIST	99
6.15.2.71	star_v1_GetMotorPidConfigRequest_fields	99
6.15.2.72	star_v1_GetMotorPidConfigRequest_header_MSGTYPE	99
6.15.2.73	star_v1_GetMotorPidConfigRequest_header_tag	99
6.15.2.74	star_v1_GetMotorPidConfigRequest_init_default	99
6.15.2.75	star_v1_GetMotorPidConfigRequest_init_zero	100
6.15.2.76	star_v1_GetMotorPidConfigRequest_motor_id_tag	100
6.15.2.77	star_v1_GetMotorPidConfigRequest_size	100
6.15.2.78	star_v1_GetMotorPidConfigResponse_CALLBACK	100
6.15.2.79	star_v1_GetMotorPidConfigResponse_DEFAULT	100
6.15.2.80	star_v1_GetMotorPidConfigResponse_FIELDLIST	100
6.15.2.81	star_v1_GetMotorPidConfigResponse_fields	100
6.15.2.82	star_v1_GetMotorPidConfigResponse_header_MSGTYPE	101
6.15.2.83	star_v1_GetMotorPidConfigResponse_header_tag	101
6.15.2.84	star_v1_GetMotorPidConfigResponse_init_default	101
6.15.2.85	star_v1_GetMotorPidConfigResponse_init_zero	101
6.15.2.86	star_v1_GetMotorPidConfigResponse_motor_id_tag	101
6.15.2.87	star_v1_GetMotorPidConfigResponse_pid_config_MSGTYPE	101
6.15.2.88	star_v1_GetMotorPidConfigResponse_pid_config_tag	101
6.15.2.89	star_v1_GetMotorPidConfigResponse_size	102
6.15.2.90	star_v1_MotorPidConfiguration_CALLBACK	102
6.15.2.91	star_v1_MotorPidConfiguration_DEFAULT	102
6.15.2.92	star_v1_MotorPidConfiguration_FIELDLIST	102
6.15.2.93	star_v1_MotorPidConfiguration_fields	102
6.15.2.94	star_v1_MotorPidConfiguration_init_default	102
6.15.2.95	star_v1_MotorPidConfiguration_init_zero	102
6.15.2.96	star_v1_MotorPidConfiguration_motor_id_tag	103
6.15.2.97	star_v1_MotorPidConfiguration_pid_config_MSGTYPE	103
6.15.2.98	star_v1_MotorPidConfiguration_pid_config_tag	103
6.15.2.99	star_v1_MotorPidConfiguration_size	103

6.15.2.100	star_v1_ResetToDefaultsRequest_CALLBACK	103
6.15.2.101	star_v1_ResetToDefaultsRequest_DEFAULT	103
6.15.2.102	star_v1_ResetToDefaultsRequest_FIELDLIST	103
6.15.2.103	star_v1_ResetToDefaultsRequest_fields	104
6.15.2.104	star_v1_ResetToDefaultsRequest_header_MSGTYPE	104
6.15.2.105	star_v1_ResetToDefaultsRequest_header_tag	104
6.15.2.106	star_v1_ResetToDefaultsRequest_init_default	104
6.15.2.107	star_v1_ResetToDefaultsRequest_init_zero	104
6.15.2.108	star_v1_ResetToDefaultsRequest_persist_to_nvs_tag	104
6.15.2.109	star_v1_ResetToDefaultsRequest_size	104
6.15.2.110	star_v1_ResetToDefaultsResponse_CALLBACK	104
6.15.2.111	star_v1_ResetToDefaultsResponse_configuration_MSGTYPE	105
6.15.2.112	star_v1_ResetToDefaultsResponse_configuration_tag	105
6.15.2.113	star_v1_ResetToDefaultsResponse_DEFAULT	105
6.15.2.114	star_v1_ResetToDefaultsResponse_FIELDLIST	105
6.15.2.115	star_v1_ResetToDefaultsResponse_fields	105
6.15.2.116	star_v1_ResetToDefaultsResponse_header_MSGTYPE	105
6.15.2.117	star_v1_ResetToDefaultsResponse_header_tag	105
6.15.2.118	star_v1_ResetToDefaultsResponse_init_default	106
6.15.2.119	star_v1_ResetToDefaultsResponse_init_zero	106
6.15.2.120	star_v1_RetransmitConfig_ack_timeout_ms_tag	106
6.15.2.121	star_v1_RetransmitConfig_CALLBACK	106
6.15.2.122	star_v1_RetransmitConfig_DEFAULT	106
6.15.2.123	star_v1_RetransmitConfig_enabled_tag	106
6.15.2.124	star_v1_RetransmitConfig_FIELDLIST	106
6.15.2.125	star_v1_RetransmitConfig_fields	107
6.15.2.126	star_v1_RetransmitConfig_init_default	107
6.15.2.127	star_v1_RetransmitConfig_init_zero	107
6.15.2.128	star_v1_RetransmitConfig_max_backoff_ms_tag	107
6.15.2.129	star_v1_RetransmitConfig_max_retries_tag	107
6.15.2.130	star_v1_RetransmitConfig_size	107
6.15.2.131	star_v1_SafetyThresholds_CALLBACK	107
6.15.2.132	star_v1_SafetyThresholds_cell_imbalance_threshold_mv_tag	107
6.15.2.133	star_v1_SafetyThresholds_DEFAULT	108
6.15.2.134	star_v1_SafetyThresholds_FIELDLIST	108
6.15.2.135	star_v1_SafetyThresholds_fields	108
6.15.2.136	star_v1_SafetyThresholds_init_default	108
6.15.2.137	star_v1_SafetyThresholds_init_zero	108
6.15.2.138	star_v1_SafetyThresholds_overcurrent_threshold_ma_tag	108
6.15.2.139	star_v1_SafetyThresholds_size	108
6.15.2.140	star_v1_SafetyThresholds_thermal_shutdown_deci_celsius_tag	109
6.15.2.141	star_v1_SaveConfigurationRequest_CALLBACK	109
6.15.2.142	star_v1_SaveConfigurationRequest_DEFAULT	109

6.15.2.143	star_v1_SaveConfigurationRequest_FIELDLIST	109
6.15.2.144	star_v1_SaveConfigurationRequest_fields	109
6.15.2.145	star_v1_SaveConfigurationRequest_header_MSGTYPE	109
6.15.2.146	star_v1_SaveConfigurationRequest_header_tag	109
6.15.2.147	star_v1_SaveConfigurationRequest_init_default	110
6.15.2.148	star_v1_SaveConfigurationRequest_init_zero	110
6.15.2.149	star_v1_SaveConfigurationRequest_size	110
6.15.2.150	star_v1_SaveConfigurationResponse_CALLBACK	110
6.15.2.151	star_v1_SaveConfigurationResponse_config_crc_tag	110
6.15.2.152	star_v1_SaveConfigurationResponse_DEFAULT	110
6.15.2.153	star_v1_SaveConfigurationResponse_FIELDLIST	110
6.15.2.154	star_v1_SaveConfigurationResponse_fields	111
6.15.2.155	star_v1_SaveConfigurationResponse_header_MSGTYPE	111
6.15.2.156	star_v1_SaveConfigurationResponse_header_tag	111
6.15.2.157	star_v1_SaveConfigurationResponse_init_default	111
6.15.2.158	star_v1_SaveConfigurationResponse_init_zero	111
6.15.2.159	star_v1_SaveConfigurationResponse_saved_tag	111
6.15.2.160	star_v1_SaveConfigurationResponse_size	111
6.15.2.161	star_v1_SetConfigurationRequest_CALLBACK	111
6.15.2.162	star_v1_SetConfigurationRequest_configuration_MSGTYPE	112
6.15.2.163	star_v1_SetConfigurationRequest_configuration_tag	112
6.15.2.164	star_v1_SetConfigurationRequest_DEFAULT	112
6.15.2.165	star_v1_SetConfigurationRequest_FIELDLIST	112
6.15.2.166	star_v1_SetConfigurationRequest_fields	112
6.15.2.167	star_v1_SetConfigurationRequest_header_MSGTYPE	112
6.15.2.168	star_v1_SetConfigurationRequest_header_tag	112
6.15.2.169	star_v1_SetConfigurationRequest_init_default	113
6.15.2.170	star_v1_SetConfigurationRequest_init_zero	113
6.15.2.171	star_v1_SetConfigurationRequest_persist_to_nvs_tag	113
6.15.2.172	star_v1_SetConfigurationResponse_CALLBACK	113
6.15.2.173	star_v1_SetConfigurationResponse_DEFAULT	113
6.15.2.174	star_v1_SetConfigurationResponse_FIELDLIST	113
6.15.2.175	star_v1_SetConfigurationResponse_fields	114
6.15.2.176	star_v1_SetConfigurationResponse_header_MSGTYPE	114
6.15.2.177	star_v1_SetConfigurationResponse_header_tag	114
6.15.2.178	star_v1_SetConfigurationResponse_init_default	114
6.15.2.179	star_v1_SetConfigurationResponse_init_zero	114
6.15.2.180	star_v1_SetConfigurationResponse_size	114
6.15.2.181	star_v1_SetConfigurationResponse_validation_result_MSGTYPE	114
6.15.2.182	star_v1_SetConfigurationResponse_validation_result_tag	114
6.15.2.183	star_v1_SetMotorPidConfigRequest_CALLBACK	115
6.15.2.184	star_v1_SetMotorPidConfigRequest_DEFAULT	115
6.15.2.185	star_v1_SetMotorPidConfigRequest_FIELDLIST	115

6.15.2.186	star_v1_SetMotorPidConfigRequest_fields	115
6.15.2.187	star_v1_SetMotorPidConfigRequest_header_MSGTYPE	115
6.15.2.188	star_v1_SetMotorPidConfigRequest_header_tag	115
6.15.2.189	star_v1_SetMotorPidConfigRequest_init_default	116
6.15.2.190	star_v1_SetMotorPidConfigRequest_init_zero	116
6.15.2.191	star_v1_SetMotorPidConfigRequest_motor_id_tag	116
6.15.2.192	star_v1_SetMotorPidConfigRequest_persist_to_nvs_tag	116
6.15.2.193	star_v1_SetMotorPidConfigRequest_pid_config_MSGTYPE	116
6.15.2.194	star_v1_SetMotorPidConfigRequest_pid_config_tag	116
6.15.2.195	star_v1_SetMotorPidConfigRequest_size	116
6.15.2.196	star_v1_SetMotorPidConfigResponse_CALLBACK	117
6.15.2.197	star_v1_SetMotorPidConfigResponse_DEFAULT	117
6.15.2.198	star_v1_SetMotorPidConfigResponse_FIELDLIST	117
6.15.2.199	star_v1_SetMotorPidConfigResponse_fields	117
6.15.2.200	star_v1_SetMotorPidConfigResponse_header_MSGTYPE	117
6.15.2.201	star_v1_SetMotorPidConfigResponse_header_tag	117
6.15.2.202	star_v1_SetMotorPidConfigResponse_init_default	117
6.15.2.203	star_v1_SetMotorPidConfigResponse_init_zero	118
6.15.2.204	star_v1_SetMotorPidConfigResponse_size	118
6.15.2.205	star_v1_SetMotorPidConfigResponse_validation_result_MSGTYPE	118
6.15.2.206	star_v1_SetMotorPidConfigResponse_validation_result_tag	118
6.15.2.207	star_v1_SetRetransmitConfigRequest_CALLBACK	118
6.15.2.208	star_v1_SetRetransmitConfigRequest_DEFAULT	118
6.15.2.209	star_v1_SetRetransmitConfigRequest_FIELDLIST	118
6.15.2.210	star_v1_SetRetransmitConfigRequest_fields	119
6.15.2.211	star_v1_SetRetransmitConfigRequest_header_MSGTYPE	119
6.15.2.212	star_v1_SetRetransmitConfigRequest_header_tag	119
6.15.2.213	star_v1_SetRetransmitConfigRequest_init_default	119
6.15.2.214	star_v1_SetRetransmitConfigRequest_init_zero	119
6.15.2.215	star_v1_SetRetransmitConfigRequest_retransmit_config_MSGTYPE	119
6.15.2.216	star_v1_SetRetransmitConfigRequest_retransmit_config_tag	119
6.15.2.217	star_v1_SetRetransmitConfigRequest_size	120
6.15.2.218	star_v1_SetRetransmitConfigResponse_CALLBACK	120
6.15.2.219	star_v1_SetRetransmitConfigResponse_DEFAULT	120
6.15.2.220	star_v1_SetRetransmitConfigResponse_FIELDLIST	120
6.15.2.221	star_v1_SetRetransmitConfigResponse_fields	120
6.15.2.222	star_v1_SetRetransmitConfigResponse_header_MSGTYPE	120
6.15.2.223	star_v1_SetRetransmitConfigResponse_header_tag	120
6.15.2.224	star_v1_SetRetransmitConfigResponse_init_default	121
6.15.2.225	star_v1_SetRetransmitConfigResponse_init_zero	121
6.15.2.226	star_v1_SetRetransmitConfigResponse_size	121
6.15.2.227	STAR_V1_STAR_V1_CONFIGURATION_PB_H_MAX_SIZE	121
6.15.2.228	star_v1_SystemConfiguration_CALLBACK	121

6.15.2.229	star_v1_SystemConfiguration_config_crc_tag	121
6.15.2.230	star_v1_SystemConfiguration_config_version_tag	121
6.15.2.231	star_v1_SystemConfiguration_current_calibration_MSGTYPE	121
6.15.2.232	star_v1_SystemConfiguration_current_calibration_tag	122
6.15.2.233	star_v1_SystemConfiguration_DEFAULT	122
6.15.2.234	star_v1_SystemConfiguration_encoder_config_MSGTYPE	122
6.15.2.235	star_v1_SystemConfiguration_encoder_config_tag	122
6.15.2.236	star_v1_SystemConfiguration_FIELDLIST	122
6.15.2.237	star_v1_SystemConfiguration_fields	122
6.15.2.238	star_v1_SystemConfiguration_init_default	123
6.15.2.239	star_v1_SystemConfiguration_init_zero	123
6.15.2.240	star_v1_SystemConfiguration_motor_configs_MSGTYPE	123
6.15.2.241	star_v1_SystemConfiguration_motor_configs_tag	123
6.15.2.242	star_v1_SystemConfiguration_safety_thresholds_MSGTYPE	123
6.15.2.243	star_v1_SystemConfiguration_safety_thresholds_tag	123
6.15.2.244	star_v1_SystemConfiguration_temperature_calibration_MSGTYPE	123
6.15.2.245	star_v1_SystemConfiguration_temperature_calibration_tag	124
6.15.2.246	star_v1_SystemConfiguration_timing_config_MSGTYPE	124
6.15.2.247	star_v1_SystemConfiguration_timing_config_tag	124
6.15.2.248	star_v1_TemperatureCalibration_CALLBACK	124
6.15.2.249	star_v1_TemperatureCalibration_DEFAULT	124
6.15.2.250	star_v1_TemperatureCalibration_FIELDLIST	124
6.15.2.251	star_v1_TemperatureCalibration_fields	124
6.15.2.252	star_v1_TemperatureCalibration_init_default	125
6.15.2.253	star_v1_TemperatureCalibration_init_zero	125
6.15.2.254	star_v1_TemperatureCalibration_offset_celsius_tag	125
6.15.2.255	star_v1_TemperatureCalibration_size	125
6.15.2.256	star_v1_TimingConfiguration_CALLBACK	125
6.15.2.257	star_v1_TimingConfiguration_communication_timeout_ms_tag	125
6.15.2.258	star_v1_TimingConfiguration_DEFAULT	125
6.15.2.259	star_v1_TimingConfiguration_FIELDLIST	126
6.15.2.260	star_v1_TimingConfiguration_fields	126
6.15.2.261	star_v1_TimingConfiguration_init_default	126
6.15.2.262	star_v1_TimingConfiguration_init_zero	126
6.15.2.263	star_v1_TimingConfiguration_motor_control_period_ms_tag	126
6.15.2.264	star_v1_TimingConfiguration_size	126
6.15.2.265	star_v1_TimingConfiguration_telemetry_period_ms_tag	126
6.15.2.266	star_v1_ValidateConfigurationRequest_CALLBACK	127
6.15.2.267	star_v1_ValidateConfigurationRequest_configuration_MSGTYPE	127
6.15.2.268	star_v1_ValidateConfigurationRequest_configuration_tag	127
6.15.2.269	star_v1_ValidateConfigurationRequest_DEFAULT	127
6.15.2.270	star_v1_ValidateConfigurationRequest_FIELDLIST	127
6.15.2.271	star_v1_ValidateConfigurationRequest_fields	127

6.15.2.272	star_v1_ValidateConfigurationRequest_header_MSGTYPE	127
6.15.2.273	star_v1_ValidateConfigurationRequest_header_tag	128
6.15.2.274	star_v1_ValidateConfigurationRequest_init_default	128
6.15.2.275	star_v1_ValidateConfigurationRequest_init_zero	128
6.15.2.276	star_v1_ValidateConfigurationResponse_CALLBACK	128
6.15.2.277	star_v1_ValidateConfigurationResponse_DEFAULT	128
6.15.2.278	star_v1_ValidateConfigurationResponse_FIELDLIST	128
6.15.2.279	star_v1_ValidateConfigurationResponse_fields	128
6.15.2.280	star_v1_ValidateConfigurationResponse_header_MSGTYPE	129
6.15.2.281	star_v1_ValidateConfigurationResponse_header_tag	129
6.15.2.282	star_v1_ValidateConfigurationResponse_init_default	129
6.15.2.283	star_v1_ValidateConfigurationResponse_init_zero	129
6.15.2.284	star_v1_ValidateConfigurationResponse_size	129
6.15.2.285	star_v1_ValidateConfigurationResponse_validation_result_MSGTYPE	129
6.15.2.286	star_v1_ValidateConfigurationResponse_validation_result_tag	129
6.15.3	Enumeration Type Documentation	130
6.15.3.1	star_v1_ConfigErrorCode	130
6.15.3.2	star_v1_ConfigValidationStatus	130
6.15.4	Variable Documentation	131
6.15.4.1	star_v1_ConfigValidationError_msg	131
6.15.4.2	star_v1_ConfigValidationResult_msg	131
6.15.4.3	star_v1_CurrentSensorCalibration_msg	131
6.15.4.4	star_v1_EncoderConfiguration_msg	131
6.15.4.5	star_v1_GetConfigurationRequest_msg	131
6.15.4.6	star_v1_GetConfigurationResponse_msg	131
6.15.4.7	star_v1_GetMotorPidConfigRequest_msg	131
6.15.4.8	star_v1_GetMotorPidConfigResponse_msg	131
6.15.4.9	star_v1_MotorPidConfiguration_msg	132
6.15.4.10	star_v1_ResetToDefaultsRequest_msg	132
6.15.4.11	star_v1_ResetToDefaultsResponse_msg	132
6.15.4.12	star_v1_RetransmitConfig_msg	132
6.15.4.13	star_v1_SafetyThresholds_msg	132
6.15.4.14	star_v1_SaveConfigurationRequest_msg	132
6.15.4.15	star_v1_SaveConfigurationResponse_msg	132
6.15.4.16	star_v1_SetConfigurationRequest_msg	132
6.15.4.17	star_v1_SetConfigurationResponse_msg	132
6.15.4.18	star_v1_SetMotorPidConfigRequest_msg	132
6.15.4.19	star_v1_SetMotorPidConfigResponse_msg	133
6.15.4.20	star_v1_SetRetransmitConfigRequest_msg	133
6.15.4.21	star_v1_SetRetransmitConfigResponse_msg	133
6.15.4.22	star_v1_SystemConfiguration_msg	133
6.15.4.23	star_v1_TemperatureCalibration_msg	133
6.15.4.24	star_v1_TimingConfiguration_msg	133

6.15.4.25	star_v1_ValidateConfigurationRequest_msg	133
6.15.4.26	star_v1_ValidateConfigurationResponse_msg	133
6.16	configuration.pb.h	134
6.17	libs/rx_nanopb/inc/gen/star/v1/controller.pb.c File Reference	144
6.18	controller.pb.c	144
6.19	libs/rx_nanopb/inc/gen/star/v1/controller.pb.h File Reference	144
6.19.1	Data Structure Documentation	146
6.19.1.1	struct star_v1_ControllerState	146
6.19.2	Macro Definition Documentation	146
6.19.2.1	star_v1_ControllerState_angular_vel_tag	146
6.19.2.2	star_v1_ControllerState_CALLBACK	146
6.19.2.3	star_v1_ControllerState_debug_tag	146
6.19.2.4	star_v1_ControllerState_DEFAULT	147
6.19.2.5	star_v1_ControllerState_FIELDLIST	147
6.19.2.6	star_v1_ControllerState_fields	147
6.19.2.7	star_v1_ControllerState_init_default	147
6.19.2.8	star_v1_ControllerState_init_zero	147
6.19.2.9	star_v1_ControllerState_linear_vel_tag	147
6.19.2.10	star_v1_ControllerState_size	148
6.19.2.11	star_v1_ControllerState_timestamp_ms_tag	148
6.19.2.12	STAR_V1_STAR_V1_CONTROLLER_PB_H_MAX_SIZE	148
6.19.3	Variable Documentation	148
6.19.3.1	star_v1_ControllerState_msg	148
6.20	controller.pb.h	148
6.21	libs/rx_nanopb/inc/gen/star/v1/diagnostics.pb.c File Reference	149
6.22	diagnostics.pb.c	149
6.23	libs/rx_nanopb/inc/gen/star/v1/diagnostics.pb.h File Reference	150
6.23.1	Data Structure Documentation	152
6.23.1.1	struct star_v1_TransportHealthReport	152
6.23.1.2	struct star_v1_TransportDiagnostics	153
6.23.2	Macro Definition Documentation	153
6.23.2.1	_star_v1_TransportType_ARRAYSIZE	153
6.23.2.2	_star_v1_TransportType_MAX	154
6.23.2.3	_star_v1_TransportType_MIN	154
6.23.2.4	STAR_V1_STAR_V1_DIAGNOSTICS_PB_H_MAX_SIZE	154
6.23.2.5	star_v1_TransportDiagnostics_active_transport_tag	154
6.23.2.6	star_v1_TransportDiagnostics_CALLBACK	154
6.23.2.7	star_v1_TransportDiagnostics_collected_at_MSGTYPE	154
6.23.2.8	star_v1_TransportDiagnostics_collected_at_tag	154
6.23.2.9	star_v1_TransportDiagnostics_DEFAULT	154
6.23.2.10	star_v1_TransportDiagnostics_FIELDLIST	155
6.23.2.11	star_v1_TransportDiagnostics_fields	155
6.23.2.12	star_v1_TransportDiagnostics_init_default	155

6.23.2.13	star_v1_TransportDiagnostics_init_zero	155
6.23.2.14	star_v1_TransportDiagnostics_size	155
6.23.2.15	star_v1_TransportDiagnostics_switch_count_tag	155
6.23.2.16	star_v1_TransportDiagnostics_transports_MSGTYPE	156
6.23.2.17	star_v1_TransportDiagnostics_transports_tag	156
6.23.2.18	star_v1_TransportHealthReport_avg_latency_MSGTYPE	156
6.23.2.19	star_v1_TransportHealthReport_avg_latency_tag	156
6.23.2.20	star_v1_TransportHealthReport_CALLBACK	156
6.23.2.21	star_v1_TransportHealthReport_consecutive_failures_tag	156
6.23.2.22	star_v1_TransportHealthReport_DEFAULT	156
6.23.2.23	star_v1_TransportHealthReport_FIELDLIST	157
6.23.2.24	star_v1_TransportHealthReport_fields	157
6.23.2.25	star_v1_TransportHealthReport_init_default	157
6.23.2.26	star_v1_TransportHealthReport_init_zero	157
6.23.2.27	star_v1_TransportHealthReport_is_active_tag	158
6.23.2.28	star_v1_TransportHealthReport_is_healthy_tag	158
6.23.2.29	star_v1_TransportHealthReport_last_failure_MSGTYPE	158
6.23.2.30	star_v1_TransportHealthReport_last_failure_tag	158
6.23.2.31	star_v1_TransportHealthReport_last_recovery_MSGTYPE	158
6.23.2.32	star_v1_TransportHealthReport_last_recovery_tag	158
6.23.2.33	star_v1_TransportHealthReport_last_success_MSGTYPE	158
6.23.2.34	star_v1_TransportHealthReport_last_success_tag	158
6.23.2.35	star_v1_TransportHealthReport_packet_loss_rate_tag	159
6.23.2.36	star_v1_TransportHealthReport_size	159
6.23.2.37	star_v1_TransportHealthReport_total_errors_tag	159
6.23.2.38	star_v1_TransportHealthReport_total_received_tag	159
6.23.2.39	star_v1_TransportHealthReport_total_sent_tag	159
6.23.2.40	star_v1_TransportHealthReport_transport_name_tag	159
6.23.2.41	star_v1_TransportHealthReport_transport_type_ENUMTYPE	159
6.23.2.42	star_v1_TransportHealthReport_transport_type_tag	159
6.23.3	Enumeration Type Documentation	160
6.23.3.1	star_v1_TransportType	160
6.23.4	Variable Documentation	160
6.23.4.1	star_v1_TransportDiagnostics_msg	160
6.23.4.2	star_v1_TransportHealthReport_msg	160
6.24	diagnostics.pb.h	160
6.25	libs/rx_nanopb/inc/gen/star/v1/firmware_update.pb.c File Reference	162
6.26	firmware_update.pb.c	163
6.27	libs/rx_nanopb/inc/gen/star/v1/firmware_update.pb.h File Reference	164
6.27.1	Data Structure Documentation	172
6.27.1.1	struct star_v1_BeginUpdateRequest	172
6.27.1.2	struct star_v1_FirmwareChunk	173
6.27.1.3	struct star_v1_WriteChunkRequest	173

6.27.1.4 struct star_v1_FinalizeUpdateRequest	174
6.27.1.5 struct star_v1_AbortUpdateRequest	174
6.27.1.6 struct star_v1_AbortUpdateResponse	175
6.27.1.7 struct star_v1_GetUpdateProgressRequest	175
6.27.1.8 struct star_v1_StreamUpdateProgressRequest	176
6.27.1.9 struct star_v1_FirmwareUpdateProgress	176
6.27.1.10 struct star_v1_WriteChunkResponse	177
6.27.1.11 struct star_v1_GetUpdateProgressResponse	178
6.27.1.12 struct star_v1_RebootRequest	178
6.27.1.13 struct star_v1_RebootResponse	179
6.27.1.14 struct star_v1_RollbackRequest	180
6.27.1.15 struct star_v1_RollbackResponse	180
6.27.1.16 struct star_v1_MarkValidRequest	181
6.27.1.17 struct star_v1_MarkValidResponse	181
6.27.1.18 struct star_v1_GetFirmwareInfoRequest	182
6.27.1.19 struct star_v1_FirmwareInfo	182
6.27.1.20 struct star_v1_GetFirmwareInfoResponse	183
6.27.1.21 struct star_v1_FirmwareUpdateError	184
6.27.1.22 struct star_v1_BeginUpdateResponse	184
6.27.1.23 struct star_v1_StreamChunksResponse	185
6.27.1.24 struct star_v1_FinalizeUpdateResponse	186
6.27.2 Macro Definition Documentation	186
6.27.2.1 _star_v1_FirmwareUpdateErrorCode_ARRAYSIZE	186
6.27.2.2 _star_v1_FirmwareUpdateErrorCode_MAX	186
6.27.2.3 _star_v1_FirmwareUpdateErrorCode_MIN	186
6.27.2.4 _star_v1_FirmwareUpdateState_ARRAYSIZE	187
6.27.2.5 _star_v1_FirmwareUpdateState_MAX	187
6.27.2.6 _star_v1_FirmwareUpdateState_MIN	187
6.27.2.7 star_v1_AbortUpdateRequest_CALLBACK	187
6.27.2.8 star_v1_AbortUpdateRequest_DEFAULT	187
6.27.2.9 star_v1_AbortUpdateRequest_FIELDLIST	187
6.27.2.10 star_v1_AbortUpdateRequest_fields	187
6.27.2.11 star_v1_AbortUpdateRequest_header_MSGTYPE	188
6.27.2.12 star_v1_AbortUpdateRequest_header_tag	188
6.27.2.13 star_v1_AbortUpdateRequest_init_default	188
6.27.2.14 star_v1_AbortUpdateRequest_init_zero	188
6.27.2.15 star_v1_AbortUpdateRequest_reason_tag	188
6.27.2.16 star_v1_AbortUpdateRequest_size	188
6.27.2.17 star_v1_AbortUpdateResponse_aborted_tag	188
6.27.2.18 star_v1_AbortUpdateResponse_CALLBACK	188
6.27.2.19 star_v1_AbortUpdateResponse_DEFAULT	189
6.27.2.20 star_v1_AbortUpdateResponse_FIELDLIST	189
6.27.2.21 star_v1_AbortUpdateResponse_fields	189

6.27.2.22	star_v1_AbortUpdateResponse_header_MSGTYPE	189
6.27.2.23	star_v1_AbortUpdateResponse_header_tag	189
6.27.2.24	star_v1_AbortUpdateResponse_init_default	189
6.27.2.25	star_v1_AbortUpdateResponse_init_zero	189
6.27.2.26	star_v1_AbortUpdateResponse_size	190
6.27.2.27	star_v1_BeginUpdateRequest_CALLBACK	190
6.27.2.28	star_v1_BeginUpdateRequest_DEFAULT	190
6.27.2.29	star_v1_BeginUpdateRequest_expected_crc32_tag	190
6.27.2.30	star_v1_BeginUpdateRequest_expected_version_tag	190
6.27.2.31	star_v1_BeginUpdateRequest_FIELDLIST	190
6.27.2.32	star_v1_BeginUpdateRequest_fields	191
6.27.2.33	star_v1_BeginUpdateRequest_firmware_size_tag	191
6.27.2.34	star_v1_BeginUpdateRequest_header_MSGTYPE	191
6.27.2.35	star_v1_BeginUpdateRequest_header_tag	191
6.27.2.36	star_v1_BeginUpdateRequest_init_default	191
6.27.2.37	star_v1_BeginUpdateRequest_init_zero	191
6.27.2.38	star_v1_BeginUpdateRequest_size	191
6.27.2.39	star_v1_BeginUpdateResponse_accepted_tag	191
6.27.2.40	star_v1_BeginUpdateResponse_CALLBACK	192
6.27.2.41	star_v1_BeginUpdateResponse_DEFAULT	192
6.27.2.42	star_v1_BeginUpdateResponse_error_MSGTYPE	192
6.27.2.43	star_v1_BeginUpdateResponse_error_tag	192
6.27.2.44	star_v1_BeginUpdateResponse_FIELDLIST	192
6.27.2.45	star_v1_BeginUpdateResponse_fields	192
6.27.2.46	star_v1_BeginUpdateResponse_header_MSGTYPE	193
6.27.2.47	star_v1_BeginUpdateResponse_header_tag	193
6.27.2.48	star_v1_BeginUpdateResponse_init_default	193
6.27.2.49	star_v1_BeginUpdateResponse_init_zero	193
6.27.2.50	star_v1_BeginUpdateResponse_max_chunk_size_tag	193
6.27.2.51	star_v1_BeginUpdateResponse_session_id_tag	193
6.27.2.52	star_v1_BeginUpdateResponse_size	193
6.27.2.53	star_v1_FinalizeUpdateRequest_CALLBACK	194
6.27.2.54	star_v1_FinalizeUpdateRequest_DEFAULT	194
6.27.2.55	star_v1_FinalizeUpdateRequest_FIELDLIST	194
6.27.2.56	star_v1_FinalizeUpdateRequest_fields	194
6.27.2.57	star_v1_FinalizeUpdateRequest_header_MSGTYPE	194
6.27.2.58	star_v1_FinalizeUpdateRequest_header_tag	194
6.27.2.59	star_v1_FinalizeUpdateRequest_init_default	194
6.27.2.60	star_v1_FinalizeUpdateRequest_init_zero	195
6.27.2.61	star_v1_FinalizeUpdateRequest_size	195
6.27.2.62	star_v1_FinalizeUpdateResponse_CALLBACK	195
6.27.2.63	star_v1_FinalizeUpdateResponse_DEFAULT	195
6.27.2.64	star_v1_FinalizeUpdateResponse_error_MSGTYPE	195

6.27.2.65	star_v1_FinalizeUpdateResponse_error_tag	195
6.27.2.66	star_v1_FinalizeUpdateResponse_FIELDLIST	195
6.27.2.67	star_v1_FinalizeUpdateResponse_fields	196
6.27.2.68	star_v1_FinalizeUpdateResponse_header_MSGTYPE	196
6.27.2.69	star_v1_FinalizeUpdateResponse_header_tag	196
6.27.2.70	star_v1_FinalizeUpdateResponse_init_default	196
6.27.2.71	star_v1_FinalizeUpdateResponse_init_zero	196
6.27.2.72	star_v1_FinalizeUpdateResponse_ready_to_reboot_tag	196
6.27.2.73	star_v1_FinalizeUpdateResponse_size	196
6.27.2.74	star_v1_FinalizeUpdateResponse_validated_tag	197
6.27.2.75	star_v1_FirmwareChunk_CALLBACK	197
6.27.2.76	star_v1_FirmwareChunk_chunk_crc32_tag	197
6.27.2.77	star_v1_FirmwareChunk_data_tag	197
6.27.2.78	star_v1_FirmwareChunk_DEFAULT	197
6.27.2.79	star_v1_FirmwareChunk_FIELDLIST	197
6.27.2.80	star_v1_FirmwareChunk_fields	198
6.27.2.81	star_v1_FirmwareChunk_init_default	198
6.27.2.82	star_v1_FirmwareChunk_init_zero	198
6.27.2.83	star_v1_FirmwareChunk_offset_tag	198
6.27.2.84	star_v1_FirmwareChunk_sequence_tag	198
6.27.2.85	star_v1_FirmwareChunk_size	198
6.27.2.86	star_v1_FirmwareInfo_build_date_tag	198
6.27.2.87	star_v1_FirmwareInfo_CALLBACK	198
6.27.2.88	star_v1_FirmwareInfo_chip_target_tag	199
6.27.2.89	star_v1_FirmwareInfo_crc32_tag	199
6.27.2.90	star_v1_FirmwareInfo_DEFAULT	199
6.27.2.91	star_v1_FirmwareInfo_FIELDLIST	199
6.27.2.92	star_v1_FirmwareInfo_fields	199
6.27.2.93	star_v1_FirmwareInfo_git_hash_tag	199
6.27.2.94	star_v1_FirmwareInfo_idf_version_tag	200
6.27.2.95	star_v1_FirmwareInfo_init_default	200
6.27.2.96	star_v1_FirmwareInfo_init_zero	200
6.27.2.97	star_v1_FirmwareInfo_size	200
6.27.2.98	star_v1_FirmwareInfo_size_tag	200
6.27.2.99	star_v1_FirmwareInfo_version_tag	200
6.27.2.100	star_v1_FirmwareUpdateError_CALLBACK	200
6.27.2.101	star_v1_FirmwareUpdateError_code_ENUMTYPE	200
6.27.2.102	star_v1_FirmwareUpdateError_code_tag	201
6.27.2.103	star_v1_FirmwareUpdateError_DEFAULT	201
6.27.2.104	star_v1_FirmwareUpdateError_details_tag	201
6.27.2.105	star_v1_FirmwareUpdateError_error_at_offset_tag	201
6.27.2.106	star_v1_FirmwareUpdateError_error_at_sequence_tag	201
6.27.2.107	star_v1_FirmwareUpdateError_FIELDLIST	201

6.27.2.108	star_v1_FirmwareUpdateError_fields	202
6.27.2.109	star_v1_FirmwareUpdateError_init_default	202
6.27.2.110	star_v1_FirmwareUpdateError_init_zero	202
6.27.2.111	star_v1_FirmwareUpdateError_message_tag	202
6.27.2.112	star_v1_FirmwareUpdateError_size	202
6.27.2.113	star_v1_FirmwareUpdateProgress_bytes_received_tag	202
6.27.2.114	star_v1_FirmwareUpdateProgress_CALLBACK	202
6.27.2.115	star_v1_FirmwareUpdateProgress_DEFAULT	202
6.27.2.116	star_v1_FirmwareUpdateProgress_estimated_seconds_remaining_tag	203
6.27.2.117	star_v1_FirmwareUpdateProgress_FIELDLIST	203
6.27.2.118	star_v1_FirmwareUpdateProgress_fields	203
6.27.2.119	star_v1_FirmwareUpdateProgress_init_default	203
6.27.2.120	star_v1_FirmwareUpdateProgress_init_zero	203
6.27.2.121	star_v1_FirmwareUpdateProgress_last_sequence_tag	203
6.27.2.122	star_v1_FirmwareUpdateProgress_progress_percent_tag	204
6.27.2.123	star_v1_FirmwareUpdateProgress_running_crc32_tag	204
6.27.2.124	star_v1_FirmwareUpdateProgress_size	204
6.27.2.125	star_v1_FirmwareUpdateProgress_state_ENUMTYPE	204
6.27.2.126	star_v1_FirmwareUpdateProgress_state_tag	204
6.27.2.127	star_v1_FirmwareUpdateProgress_total_size_tag	204
6.27.2.128	star_v1_FirmwareUpdateProgress_transfer_speed_bps_tag	204
6.27.2.129	star_v1_GetFirmwareInfoRequest_CALLBACK	204
6.27.2.130	star_v1_GetFirmwareInfoRequest_DEFAULT	205
6.27.2.131	star_v1_GetFirmwareInfoRequest_FIELDLIST	205
6.27.2.132	star_v1_GetFirmwareInfoRequest_fields	205
6.27.2.133	star_v1_GetFirmwareInfoRequest_header_MSGTYPE	205
6.27.2.134	star_v1_GetFirmwareInfoRequest_header_tag	205
6.27.2.135	star_v1_GetFirmwareInfoRequest_init_default	205
6.27.2.136	star_v1_GetFirmwareInfoRequest_init_zero	205
6.27.2.137	star_v1_GetFirmwareInfoRequest_size	206
6.27.2.138	star_v1_GetFirmwareInfoResponse_CALLBACK	206
6.27.2.139	star_v1_GetFirmwareInfoResponse_current_firmware_MSGTYPE	206
6.27.2.140	star_v1_GetFirmwareInfoResponse_current_firmware_tag	206
6.27.2.141	star_v1_GetFirmwareInfoResponse_DEFAULT	206
6.27.2.142	star_v1_GetFirmwareInfoResponse_FIELDLIST	206
6.27.2.143	star_v1_GetFirmwareInfoResponse_fields	207
6.27.2.144	star_v1_GetFirmwareInfoResponse_header_MSGTYPE	207
6.27.2.145	star_v1_GetFirmwareInfoResponse_header_tag	207
6.27.2.146	star_v1_GetFirmwareInfoResponse_init_default	207
6.27.2.147	star_v1_GetFirmwareInfoResponse_init_zero	207
6.27.2.148	star_v1_GetFirmwareInfoResponse_is_ota_boot_tag	207
6.27.2.149	star_v1_GetFirmwareInfoResponse_is_valid_tag	207
6.27.2.150	star_v1_GetFirmwareInfoResponse_previous_firmware_MSGTYPE	208

6.27.2.151	star_v1_GetFirmwareInfoResponse_previous_firmware_tag	208
6.27.2.152	star_v1_GetFirmwareInfoResponse_size	208
6.27.2.153	star_v1_GetUpdateProgressRequest_CALLBACK	208
6.27.2.154	star_v1_GetUpdateProgressRequest_DEFAULT	208
6.27.2.155	star_v1_GetUpdateProgressRequest_FIELDLIST	208
6.27.2.156	star_v1_GetUpdateProgressRequest_fields	208
6.27.2.157	star_v1_GetUpdateProgressRequest_header_MSGTYPE	209
6.27.2.158	star_v1_GetUpdateProgressRequest_header_tag	209
6.27.2.159	star_v1_GetUpdateProgressRequest_init_default	209
6.27.2.160	star_v1_GetUpdateProgressRequest_init_zero	209
6.27.2.161	star_v1_GetUpdateProgressRequest_size	209
6.27.2.162	star_v1_GetUpdateProgressResponse_CALLBACK	209
6.27.2.163	star_v1_GetUpdateProgressResponse_DEFAULT	209
6.27.2.164	star_v1_GetUpdateProgressResponse_FIELDLIST	210
6.27.2.165	star_v1_GetUpdateProgressResponse_fields	210
6.27.2.166	star_v1_GetUpdateProgressResponse_header_MSGTYPE	210
6.27.2.167	star_v1_GetUpdateProgressResponse_header_tag	210
6.27.2.168	star_v1_GetUpdateProgressResponse_init_default	210
6.27.2.169	star_v1_GetUpdateProgressResponse_init_zero	210
6.27.2.170	star_v1_GetUpdateProgressResponse_progress_MSGTYPE	210
6.27.2.171	star_v1_GetUpdateProgressResponse_progress_tag	211
6.27.2.172	star_v1_GetUpdateProgressResponse_size	211
6.27.2.173	star_v1_MarkValidRequest_CALLBACK	211
6.27.2.174	star_v1_MarkValidRequest_DEFAULT	211
6.27.2.175	star_v1_MarkValidRequest_FIELDLIST	211
6.27.2.176	star_v1_MarkValidRequest_fields	211
6.27.2.177	star_v1_MarkValidRequest_header_MSGTYPE	211
6.27.2.178	star_v1_MarkValidRequest_header_tag	212
6.27.2.179	star_v1_MarkValidRequest_init_default	212
6.27.2.180	star_v1_MarkValidRequest_init_zero	212
6.27.2.181	star_v1_MarkValidRequest_size	212
6.27.2.182	star_v1_MarkValidResponse_CALLBACK	212
6.27.2.183	star_v1_MarkValidResponse_DEFAULT	212
6.27.2.184	star_v1_MarkValidResponse_FIELDLIST	212
6.27.2.185	star_v1_MarkValidResponse_fields	213
6.27.2.186	star_v1_MarkValidResponse_header_MSGTYPE	213
6.27.2.187	star_v1_MarkValidResponse_header_tag	213
6.27.2.188	star_v1_MarkValidResponse_init_default	213
6.27.2.189	star_v1_MarkValidResponse_init_zero	213
6.27.2.190	star_v1_MarkValidResponse_marked_valid_tag	213
6.27.2.191	star_v1_MarkValidResponse_size	213
6.27.2.192	star_v1_RebootRequest_CALLBACK	213
6.27.2.193	star_v1_RebootRequest_DEFAULT	214

6.27.2.194	star_v1_RebootRequest_delay_ms_tag	214
6.27.2.195	star_v1_RebootRequest_FIELDLIST	214
6.27.2.196	star_v1_RebootRequest_fields	214
6.27.2.197	star_v1_RebootRequest_header_MSGTYPE	214
6.27.2.198	star_v1_RebootRequest_header_tag	214
6.27.2.199	star_v1_RebootRequest_init_default	214
6.27.2.200	star_v1_RebootRequest_init_zero	215
6.27.2.201	star_v1_RebootRequest_size	215
6.27.2.202	star_v1_RebootResponse_CALLBACK	215
6.27.2.203	star_v1_RebootResponse_DEFAULT	215
6.27.2.204	star_v1_RebootResponse_FIELDLIST	215
6.27.2.205	star_v1_RebootResponse_fields	215
6.27.2.206	star_v1_RebootResponse_header_MSGTYPE	215
6.27.2.207	star_v1_RebootResponse_header_tag	216
6.27.2.208	star_v1_RebootResponse_init_default	216
6.27.2.209	star_v1_RebootResponse_init_zero	216
6.27.2.210	star_v1_RebootResponse_reboot_delay_ms_tag	216
6.27.2.211	star_v1_RebootResponse_rebooting_tag	216
6.27.2.212	star_v1_RebootResponse_size	216
6.27.2.213	star_v1_RollbackRequest_CALLBACK	216
6.27.2.214	star_v1_RollbackRequest_DEFAULT	216
6.27.2.215	star_v1_RollbackRequest_FIELDLIST	217
6.27.2.216	star_v1_RollbackRequest_fields	217
6.27.2.217	star_v1_RollbackRequest_header_MSGTYPE	217
6.27.2.218	star_v1_RollbackRequest_header_tag	217
6.27.2.219	star_v1_RollbackRequest_init_default	217
6.27.2.220	star_v1_RollbackRequest_init_zero	217
6.27.2.221	star_v1_RollbackRequest_size	217
6.27.2.222	star_v1_RollbackResponse_CALLBACK	218
6.27.2.223	star_v1_RollbackResponse_DEFAULT	218
6.27.2.224	star_v1_RollbackResponse_FIELDLIST	218
6.27.2.225	star_v1_RollbackResponse_fields	218
6.27.2.226	star_v1_RollbackResponse_header_MSGTYPE	218
6.27.2.227	star_v1_RollbackResponse_header_tag	218
6.27.2.228	star_v1_RollbackResponse_init_default	218
6.27.2.229	star_v1_RollbackResponse_init_zero	219
6.27.2.230	star_v1_RollbackResponse_previous_version_tag	219
6.27.2.231	star_v1_RollbackResponse_rolling_back_tag	219
6.27.2.232	star_v1_RollbackResponse_size	219
6.27.2.233	STAR_V1_STAR_V1_FIRMWARE_UPDATE_PB_H_MAX_SIZE	219
6.27.2.234	star_v1_StreamChunksResponse_CALLBACK	219
6.27.2.235	star_v1_StreamChunksResponse_chunks_received_tag	219
6.27.2.236	star_v1_StreamChunksResponse_DEFAULT	219

6.27.2.237	star_v1_StreamChunksResponse_error_MSGTYPE	220
6.27.2.238	star_v1_StreamChunksResponse_error_tag	220
6.27.2.239	star_v1_StreamChunksResponse_FIELDLIST	220
6.27.2.240	star_v1_StreamChunksResponse_fields	220
6.27.2.241	star_v1_StreamChunksResponse_header_MSGTYPE	220
6.27.2.242	star_v1_StreamChunksResponse_header_tag	220
6.27.2.243	star_v1_StreamChunksResponse_init_default	221
6.27.2.244	star_v1_StreamChunksResponse_init_zero	221
6.27.2.245	star_v1_StreamChunksResponse_progress_MSGTYPE	221
6.27.2.246	star_v1_StreamChunksResponse_progress_tag	221
6.27.2.247	star_v1_StreamChunksResponse_size	221
6.27.2.248	star_v1_StreamUpdateProgressRequest_CALLBACK	221
6.27.2.249	star_v1_StreamUpdateProgressRequest_DEFAULT	221
6.27.2.250	star_v1_StreamUpdateProgressRequest_FIELDLIST	222
6.27.2.251	star_v1_StreamUpdateProgressRequest_fields	222
6.27.2.252	star_v1_StreamUpdateProgressRequest_header_MSGTYPE	222
6.27.2.253	star_v1_StreamUpdateProgressRequest_header_tag	222
6.27.2.254	star_v1_StreamUpdateProgressRequest_init_default	222
6.27.2.255	star_v1_StreamUpdateProgressRequest_init_zero	222
6.27.2.256	star_v1_StreamUpdateProgressRequest_size	222
6.27.2.257	star_v1_StreamUpdateProgressRequest_update_interval_ms_tag	223
6.27.2.258	star_v1_WriteChunkRequest_CALLBACK	223
6.27.2.259	star_v1_WriteChunkRequest_chunk_MSGTYPE	223
6.27.2.260	star_v1_WriteChunkRequest_chunk_tag	223
6.27.2.261	star_v1_WriteChunkRequest_DEFAULT	223
6.27.2.262	star_v1_WriteChunkRequest_FIELDLIST	223
6.27.2.263	star_v1_WriteChunkRequest_fields	223
6.27.2.264	star_v1_WriteChunkRequest_header_MSGTYPE	224
6.27.2.265	star_v1_WriteChunkRequest_header_tag	224
6.27.2.266	star_v1_WriteChunkRequest_init_default	224
6.27.2.267	star_v1_WriteChunkRequest_init_zero	224
6.27.2.268	star_v1_WriteChunkRequest_size	224
6.27.2.269	star_v1_WriteChunkResponse_CALLBACK	224
6.27.2.270	star_v1_WriteChunkResponse_DEFAULT	224
6.27.2.271	star_v1_WriteChunkResponse_FIELDLIST	225
6.27.2.272	star_v1_WriteChunkResponse_fields	225
6.27.2.273	star_v1_WriteChunkResponse_header_MSGTYPE	225
6.27.2.274	star_v1_WriteChunkResponse_header_tag	225
6.27.2.275	star_v1_WriteChunkResponse_init_default	225
6.27.2.276	star_v1_WriteChunkResponse_init_zero	225
6.27.2.277	star_v1_WriteChunkResponse_progress_MSGTYPE	225
6.27.2.278	star_v1_WriteChunkResponse_progress_tag	226
6.27.2.279	star_v1_WriteChunkResponse_size	226

6.27.2.280 star_v1_WriteChunkResponse_success_tag	226
6.27.3 Enumeration Type Documentation	226
6.27.3.1 star_v1_FirmwareUpdateErrorCode	226
6.27.3.2 star_v1_FirmwareUpdateState	227
6.27.4 Function Documentation	227
6.27.4.1 PB_BYTES_ARRAY_T()	227
6.27.5 Variable Documentation	227
6.27.5.1 star_v1_AbortUpdateRequest_msg	227
6.27.5.2 star_v1_AbortUpdateResponse_msg	228
6.27.5.3 star_v1_BeginUpdateRequest_msg	228
6.27.5.4 star_v1_BeginUpdateResponse_msg	228
6.27.5.5 star_v1_FinalizeUpdateRequest_msg	228
6.27.5.6 star_v1_FinalizeUpdateResponse_msg	228
6.27.5.7 star_v1_FirmwareChunk_msg	228
6.27.5.8 star_v1_FirmwareInfo_msg	228
6.27.5.9 star_v1_FirmwareUpdateError_msg	228
6.27.5.10 star_v1_FirmwareUpdateProgress_msg	228
6.27.5.11 star_v1_GetFirmwareInfoRequest_msg	228
6.27.5.12 star_v1_GetFirmwareInfoResponse_msg	229
6.27.5.13 star_v1_GetUpdateProgressRequest_msg	229
6.27.5.14 star_v1_GetUpdateProgressResponse_msg	229
6.27.5.15 star_v1_MarkValidRequest_msg	229
6.27.5.16 star_v1_MarkValidResponse_msg	229
6.27.5.17 star_v1_RebootRequest_msg	229
6.27.5.18 star_v1_RebootResponse_msg	229
6.27.5.19 star_v1_RollbackRequest_msg	229
6.27.5.20 star_v1_RollbackResponse_msg	229
6.27.5.21 star_v1_StreamChunksResponse_msg	229
6.27.5.22 star_v1_StreamUpdateProgressRequest_msg	230
6.27.5.23 star_v1_WriteChunkRequest_msg	230
6.27.5.24 star_v1_WriteChunkResponse_msg	230
6.28 firmware_update.pb.h	230
6.29 libs/rx_nanopb/inc/gen/star/v1/gateway_service.pb.c File Reference	239
6.30 gateway_service.pb.c	240
6.31 libs/rx_nanopb/inc/gen/star/v1/gateway_service.pb.h File Reference	240
6.31.1 Data Structure Documentation	243
6.31.1.1 struct star_v1_ForwardTelemetryRequest	243
6.31.1.2 struct star_v1_ForwardTelemetryResponse	245
6.31.1.3 struct star_v1_GetTeleopCommandRequest	245
6.31.1.4 struct star_v1_GetTeleopCommandResponse	246
6.31.1.5 struct star_v1_SetPidGainsRequest	247
6.31.1.6 struct star_v1_SetPidGainsResponse	247
6.31.2 Macro Definition Documentation	248

6.31.2.1	star_v1_ForwardTelemetryRequest_CALLBACK	248
6.31.2.2	star_v1_ForwardTelemetryRequest_DEFAULT	248
6.31.2.3	star_v1_ForwardTelemetryRequest_FIELDLIST	248
6.31.2.4	star_v1_ForwardTelemetryRequest_fields	248
6.31.2.5	star_v1_ForwardTelemetryRequest_header_MSGTYPE	248
6.31.2.6	star_v1_ForwardTelemetryRequest_header_tag	249
6.31.2.7	star_v1_ForwardTelemetryRequest_init_default	249
6.31.2.8	star_v1_ForwardTelemetryRequest_init_zero	249
6.31.2.9	star_v1_ForwardTelemetryRequest_lidar_scan_MSGTYPE	249
6.31.2.10	star_v1_ForwardTelemetryRequest_lidar_scan_tag	249
6.31.2.11	star_v1_ForwardTelemetryRequest_motor_status_MSGTYPE	249
6.31.2.12	star_v1_ForwardTelemetryRequest_motor_status_tag	249
6.31.2.13	star_v1_ForwardTelemetryRequest_odometry_MSGTYPE	250
6.31.2.14	star_v1_ForwardTelemetryRequest_odometry_tag	250
6.31.2.15	star_v1_ForwardTelemetryRequest_size	250
6.31.2.16	star_v1_ForwardTelemetryRequest_system_status_MSGTYPE	250
6.31.2.17	star_v1_ForwardTelemetryRequest_system_status_tag	250
6.31.2.18	star_v1_ForwardTelemetryRequest_telemetry_MSGTYPE	250
6.31.2.19	star_v1_ForwardTelemetryRequest_telemetry_tag	250
6.31.2.20	star_v1_ForwardTelemetryResponse_active_clients_tag	250
6.31.2.21	star_v1_ForwardTelemetryResponse_cached_tag	251
6.31.2.22	star_v1_ForwardTelemetryResponse_CALLBACK	251
6.31.2.23	star_v1_ForwardTelemetryResponse_DEFAULT	251
6.31.2.24	star_v1_ForwardTelemetryResponse_FIELDLIST	251
6.31.2.25	star_v1_ForwardTelemetryResponse_fields	251
6.31.2.26	star_v1_ForwardTelemetryResponse_header_MSGTYPE	251
6.31.2.27	star_v1_ForwardTelemetryResponse_header_tag	251
6.31.2.28	star_v1_ForwardTelemetryResponse_init_default	252
6.31.2.29	star_v1_ForwardTelemetryResponse_init_zero	252
6.31.2.30	star_v1_ForwardTelemetryResponse_size	252
6.31.2.31	star_v1_GetTeleopCommandRequest_CALLBACK	252
6.31.2.32	star_v1_GetTeleopCommandRequest_DEFAULT	252
6.31.2.33	star_v1_GetTeleopCommandRequest_FIELDLIST	252
6.31.2.34	star_v1_GetTeleopCommandRequest_fields	252
6.31.2.35	star_v1_GetTeleopCommandRequest_header_MSGTYPE	253
6.31.2.36	star_v1_GetTeleopCommandRequest_header_tag	253
6.31.2.37	star_v1_GetTeleopCommandRequest_init_default	253
6.31.2.38	star_v1_GetTeleopCommandRequest_init_zero	253
6.31.2.39	star_v1_GetTeleopCommandRequest_size	253
6.31.2.40	star_v1_GetTeleopCommandResponse_CALLBACK	253
6.31.2.41	star_v1_GetTeleopCommandResponse_command_age_ms_tag	253
6.31.2.42	star_v1_GetTeleopCommandResponse_command_available_tag	253
6.31.2.43	star_v1_GetTeleopCommandResponse_command_MSGTYPE	254

6.31.2.44	star_v1_GetTeleopCommandResponse_command_tag	254
6.31.2.45	star_v1_GetTeleopCommandResponse_DEFAULT	254
6.31.2.46	star_v1_GetTeleopCommandResponse_FIELDLIST	254
6.31.2.47	star_v1_GetTeleopCommandResponse_fields	254
6.31.2.48	star_v1_GetTeleopCommandResponse_header_MSGTYPE	254
6.31.2.49	star_v1_GetTeleopCommandResponse_header_tag	255
6.31.2.50	star_v1_GetTeleopCommandResponse_init_default	255
6.31.2.51	star_v1_GetTeleopCommandResponse_init_zero	255
6.31.2.52	star_v1_GetTeleopCommandResponse_size	255
6.31.2.53	star_v1_SetPidGainsRequest_CALLBACK	255
6.31.2.54	star_v1_SetPidGainsRequest_DEFAULT	255
6.31.2.55	star_v1_SetPidGainsRequest_FIELDLIST	255
6.31.2.56	star_v1_SetPidGainsRequest_fields	256
6.31.2.57	star_v1_SetPidGainsRequest_header_MSGTYPE	256
6.31.2.58	star_v1_SetPidGainsRequest_header_tag	256
6.31.2.59	star_v1_SetPidGainsRequest_init_default	256
6.31.2.60	star_v1_SetPidGainsRequest_init_zero	256
6.31.2.61	star_v1_SetPidGainsRequest_motor_id_tag	256
6.31.2.62	star_v1_SetPidGainsRequest_pid_config_MSGTYPE	256
6.31.2.63	star_v1_SetPidGainsRequest_pid_config_tag	257
6.31.2.64	star_v1_SetPidGainsRequest_size	257
6.31.2.65	star_v1_SetPidGainsResponse_CALLBACK	257
6.31.2.66	star_v1_SetPidGainsResponse_DEFAULT	257
6.31.2.67	star_v1_SetPidGainsResponse_FIELDLIST	257
6.31.2.68	star_v1_SetPidGainsResponse_fields	257
6.31.2.69	star_v1_SetPidGainsResponse_header_MSGTYPE	258
6.31.2.70	star_v1_SetPidGainsResponse_header_tag	258
6.31.2.71	star_v1_SetPidGainsResponse_init_default	258
6.31.2.72	star_v1_SetPidGainsResponse_init_zero	258
6.31.2.73	star_v1_SetPidGainsResponse_message_tag	258
6.31.2.74	star_v1_SetPidGainsResponse_success_tag	258
6.31.2.75	STAR_V1_STAR_V1_GATEWAY_SERVICE_PB_H_MAX_SIZE	258
6.31.3	Variable Documentation	258
6.31.3.1	star_v1_ForwardTelemetryRequest_msg	258
6.31.3.2	star_v1_ForwardTelemetryResponse_msg	259
6.31.3.3	star_v1_GetTeleopCommandRequest_msg	259
6.31.3.4	star_v1_GetTeleopCommandResponse_msg	259
6.31.3.5	star_v1_SetPidGainsRequest_msg	259
6.31.3.6	star_v1_SetPidGainsResponse_msg	259
6.32	gateway_service.pb.h	259
6.33	libs/rx_nanopb/inc/gen/star/v1/motor_control.pb.c File Reference	262
6.34	motor_control.pb.c	262
6.35	libs/rx_nanopb/inc/gen/star/v1/motor_control.pb.h File Reference	263

6.35.1 Data Structure Documentation	268
6.35.1.1 struct star_v1_VelocityCommand	268
6.35.1.2 struct star_v1_SetVelocityRequest	269
6.35.1.3 struct star_v1_EmergencyStopRequest	269
6.35.1.4 struct star_v1_EmergencyStopResponse	270
6.35.1.5 struct star_v1_SetMotorPowerResponse	270
6.35.1.6 struct star_v1_MotorPowerCommand	271
6.35.1.7 struct star_v1_SetMotorPowerRequest	271
6.35.1.8 struct star_v1_StreamEncodersRequest	272
6.35.1.9 struct star_v1_EncoderData	272
6.35.1.10 struct star_v1_MotorStatus	273
6.35.1.11 struct star_v1_SetVelocityResponse	273
6.35.1.12 struct star_v1_PidConfig	274
6.35.2 Macro Definition Documentation	275
6.35.2.1 _star_v1_MotorState_ARRAYSIZE	275
6.35.2.2 _star_v1_MotorState_MAX	275
6.35.2.3 _star_v1_MotorState_MIN	275
6.35.2.4 star_v1_EmergencyStopRequest_CALLBACK	275
6.35.2.5 star_v1_EmergencyStopRequest_DEFAULT	275
6.35.2.6 star_v1_EmergencyStopRequest_FIELDLIST	276
6.35.2.7 star_v1_EmergencyStopRequest_fields	276
6.35.2.8 star_v1_EmergencyStopRequest_header_MSGTYPE	276
6.35.2.9 star_v1_EmergencyStopRequest_header_tag	276
6.35.2.10 star_v1_EmergencyStopRequest_init_default	276
6.35.2.11 star_v1_EmergencyStopRequest_init_zero	276
6.35.2.12 star_v1_EmergencyStopRequest_reason_tag	277
6.35.2.13 star_v1_EmergencyStopRequest_size	277
6.35.2.14 star_v1_EmergencyStopResponse_CALLBACK	277
6.35.2.15 star_v1_EmergencyStopResponse_DEFAULT	277
6.35.2.16 star_v1_EmergencyStopResponse_estop_engaged_tag	277
6.35.2.17 star_v1_EmergencyStopResponse_FIELDLIST	277
6.35.2.18 star_v1_EmergencyStopResponse_fields	277
6.35.2.19 star_v1_EmergencyStopResponse_header_MSGTYPE	278
6.35.2.20 star_v1_EmergencyStopResponse_header_tag	278
6.35.2.21 star_v1_EmergencyStopResponse_init_default	278
6.35.2.22 star_v1_EmergencyStopResponse_init_zero	278
6.35.2.23 star_v1_EmergencyStopResponse_size	278
6.35.2.24 star_v1_EncoderData_CALLBACK	278
6.35.2.25 star_v1_EncoderData_DEFAULT	278
6.35.2.26 star_v1_EncoderData_FIELDLIST	279
6.35.2.27 star_v1_EncoderData_fields	279
6.35.2.28 star_v1_EncoderData_init_default	279
6.35.2.29 star_v1_EncoderData_init_zero	279

6.35.2.30	star_v1_EncoderData_motor_id_tag	279
6.35.2.31	star_v1_EncoderData_size	279
6.35.2.32	star_v1_EncoderData_ticks_tag	280
6.35.2.33	star_v1_EncoderData_timestamp_us_tag	280
6.35.2.34	star_v1_EncoderData_velocity_mps_tag	280
6.35.2.35	star_v1_MotorPowerCommand_CALLBACK	280
6.35.2.36	star_v1_MotorPowerCommand_DEFAULT	280
6.35.2.37	star_v1_MotorPowerCommand_duty_cycle_percent_tag	280
6.35.2.38	star_v1_MotorPowerCommand_FIELDLIST	280
6.35.2.39	star_v1_MotorPowerCommand_fields	281
6.35.2.40	star_v1_MotorPowerCommand_init_default	281
6.35.2.41	star_v1_MotorPowerCommand_init_zero	281
6.35.2.42	star_v1_MotorPowerCommand_motor_id_tag	281
6.35.2.43	star_v1_MotorPowerCommand_size	281
6.35.2.44	star_v1_MotorStatus_CALLBACK	281
6.35.2.45	star_v1_MotorStatus_current_ma_tag	281
6.35.2.46	star_v1_MotorStatus_DEFAULT	281
6.35.2.47	star_v1_MotorStatus_duty_cycle_percent_tag	282
6.35.2.48	star_v1_MotorStatus_fault_flags_tag	282
6.35.2.49	star_v1_MotorStatus_FIELDLIST	282
6.35.2.50	star_v1_MotorStatus_fields	282
6.35.2.51	star_v1_MotorStatus_init_default	282
6.35.2.52	star_v1_MotorStatus_init_zero	282
6.35.2.53	star_v1_MotorStatus_motor_id_tag	283
6.35.2.54	star_v1_MotorStatus_size	283
6.35.2.55	star_v1_MotorStatus_state_ENUMTYPE	283
6.35.2.56	star_v1_MotorStatus_state_tag	283
6.35.2.57	star_v1_MotorStatus_target_velocity_mps_tag	283
6.35.2.58	star_v1_MotorStatus_temperature_celsius_tag	283
6.35.2.59	star_v1_MotorStatus_velocity_mps_tag	283
6.35.2.60	star_v1_PidConfig_CALLBACK	283
6.35.2.61	star_v1_PidConfig_DEFAULT	284
6.35.2.62	star_v1_PidConfig_FIELDLIST	284
6.35.2.63	star_v1_PidConfig_fields	284
6.35.2.64	star_v1_PidConfig_init_default	284
6.35.2.65	star_v1_PidConfig_init_zero	284
6.35.2.66	star_v1_PidConfig_integral_max_tag	284
6.35.2.67	star_v1_PidConfig_integral_min_tag	285
6.35.2.68	star_v1_PidConfig_kd_tag	285
6.35.2.69	star_v1_PidConfig_ki_tag	285
6.35.2.70	star_v1_PidConfig_kp_tag	285
6.35.2.71	star_v1_PidConfig_output_max_percent_tag	285
6.35.2.72	star_v1_PidConfig_output_min_percent_tag	285

6.35.2.73	star_v1_PidConfig_size	285
6.35.2.74	star_v1_SetMotorPowerRequest_CALLBACK	285
6.35.2.75	star_v1_SetMotorPowerRequest_command_MSGTYPE	286
6.35.2.76	star_v1_SetMotorPowerRequest_command_tag	286
6.35.2.77	star_v1_SetMotorPowerRequest_DEFAULT	286
6.35.2.78	star_v1_SetMotorPowerRequest_FIELDLIST	286
6.35.2.79	star_v1_SetMotorPowerRequest_fields	286
6.35.2.80	star_v1_SetMotorPowerRequest_header_MSGTYPE	286
6.35.2.81	star_v1_SetMotorPowerRequest_header_tag	286
6.35.2.82	star_v1_SetMotorPowerRequest_init_default	287
6.35.2.83	star_v1_SetMotorPowerRequest_init_zero	287
6.35.2.84	star_v1_SetMotorPowerRequest_size	287
6.35.2.85	star_v1_SetMotorPowerResponse_CALLBACK	287
6.35.2.86	star_v1_SetMotorPowerResponse_DEFAULT	287
6.35.2.87	star_v1_SetMotorPowerResponse_FIELDLIST	287
6.35.2.88	star_v1_SetMotorPowerResponse_fields	287
6.35.2.89	star_v1_SetMotorPowerResponse_header_MSGTYPE	288
6.35.2.90	star_v1_SetMotorPowerResponse_header_tag	288
6.35.2.91	star_v1_SetMotorPowerResponse_init_default	288
6.35.2.92	star_v1_SetMotorPowerResponse_init_zero	288
6.35.2.93	star_v1_SetMotorPowerResponse_size	288
6.35.2.94	star_v1_SetVelocityRequest_CALLBACK	288
6.35.2.95	star_v1_SetVelocityRequest_command_MSGTYPE	288
6.35.2.96	star_v1_SetVelocityRequest_command_tag	288
6.35.2.97	star_v1_SetVelocityRequest_DEFAULT	289
6.35.2.98	star_v1_SetVelocityRequest_FIELDLIST	289
6.35.2.99	star_v1_SetVelocityRequest_fields	289
6.35.2.100	star_v1_SetVelocityRequest_header_MSGTYPE	289
6.35.2.101	star_v1_SetVelocityRequest_header_tag	289
6.35.2.102	star_v1_SetVelocityRequest_init_default	289
6.35.2.103	star_v1_SetVelocityRequest_init_zero	290
6.35.2.104	star_v1_SetVelocityRequest_size	290
6.35.2.105	star_v1_SetVelocityResponse_CALLBACK	290
6.35.2.106	star_v1_SetVelocityResponse_DEFAULT	290
6.35.2.107	star_v1_SetVelocityResponse_FIELDLIST	290
6.35.2.108	star_v1_SetVelocityResponse_fields	290
6.35.2.109	star_v1_SetVelocityResponse_header_MSGTYPE	291
6.35.2.110	star_v1_SetVelocityResponse_header_tag	291
6.35.2.111	star_v1_SetVelocityResponse_init_default	291
6.35.2.112	star_v1_SetVelocityResponse_init_zero	291
6.35.2.113	star_v1_SetVelocityResponse_motor_status_MSGTYPE	291
6.35.2.114	star_v1_SetVelocityResponse_motor_status_tag	291
6.35.2.115	star_v1_SetVelocityResponse_size	291

6.35.2.116	STAR_V1_STAR_V1_MOTOR_CONTROL_PB_H_MAX_SIZE	292
6.35.2.117	star_v1_StreamEncodersRequest_CALLBACK	292
6.35.2.118	star_v1_StreamEncodersRequest_DEFAULT	292
6.35.2.119	star_v1_StreamEncodersRequest_FIELDLIST	292
6.35.2.120	star_v1_StreamEncodersRequest_fields	292
6.35.2.121	star_v1_StreamEncodersRequest_header_MSGTYPE	292
6.35.2.122	star_v1_StreamEncodersRequest_header_tag	292
6.35.2.123	star_v1_StreamEncodersRequest_init_default	293
6.35.2.124	star_v1_StreamEncodersRequest_init_zero	293
6.35.2.125	star_v1_StreamEncodersRequest_rate_hz_tag	293
6.35.2.126	star_v1_StreamEncodersRequest_size	293
6.35.2.127	star_v1_VelocityCommand_back_left_velocity_mps_tag	293
6.35.2.128	star_v1_VelocityCommand_back_right_velocity_mps_tag	293
6.35.2.129	star_v1_VelocityCommand_CALLBACK	293
6.35.2.130	star_v1_VelocityCommand_DEFAULT	293
6.35.2.131	star_v1_VelocityCommand_FIELDLIST	294
6.35.2.132	star_v1_VelocityCommand_fields	294
6.35.2.133	star_v1_VelocityCommand_front_left_velocity_mps_tag	294
6.35.2.134	star_v1_VelocityCommand_front_right_velocity_mps_tag	294
6.35.2.135	star_v1_VelocityCommand_init_default	294
6.35.2.136	star_v1_VelocityCommand_init_zero	294
6.35.2.137	star_v1_VelocityCommand_sequence_tag	295
6.35.2.138	star_v1_VelocityCommand_size	295
6.35.2.139	star_v1_VelocityCommand_timestamp_us_tag	295
6.35.3	Enumeration Type Documentation	295
6.35.3.1	star_v1_MotorState	295
6.35.4	Variable Documentation	295
6.35.4.1	star_v1_EmergencyStopRequest_msg	295
6.35.4.2	star_v1_EmergencyStopResponse_msg	296
6.35.4.3	star_v1_EncoderData_msg	296
6.35.4.4	star_v1_MotorPowerCommand_msg	296
6.35.4.5	star_v1_MotorStatus_msg	296
6.35.4.6	star_v1_PidConfig_msg	296
6.35.4.7	star_v1_SetMotorPowerRequest_msg	296
6.35.4.8	star_v1_SetMotorPowerResponse_msg	296
6.35.4.9	star_v1_SetVelocityRequest_msg	296
6.35.4.10	star_v1_SetVelocityResponse_msg	296
6.35.4.11	star_v1_StreamEncodersRequest_msg	296
6.35.4.12	star_v1_VelocityCommand_msg	297
6.36	motor_control.pb.h	297
6.37	libs/rx_nanopb/inc/gen/star/v1/telemetry.pb.c File Reference	302
6.38	telemetry.pb.c	302
6.39	libs/rx_nanopb/inc/gen/star/v1/telemetry.pb.h File Reference	303

6.39.1 Data Structure Documentation	309
6.39.1.1 struct star_v1_GetTelemetryRequest	309
6.39.1.2 struct star_v1_StreamTelemetryRequest	309
6.39.1.3 struct star_v1_GetSystemStatusRequest	310
6.39.1.4 struct star_v1_ImuData	311
6.39.1.5 struct star_v1_BaroData	312
6.39.1.6 struct star_v1_ObstacleData	312
6.39.1.7 struct star_v1_GpsData	313
6.39.1.8 struct star_v1_TelemetryData	313
6.39.1.9 struct star_v1_GetTelemetryResponse	315
6.39.1.10 struct star_v1_SystemStatus	316
6.39.1.11 struct star_v1_GetSystemStatusResponse	317
6.39.2 Macro Definition Documentation	318
6.39.2.1 _star_v1_ConnectionStatus_ARRAYSIZE	318
6.39.2.2 _star_v1_ConnectionStatus_MAX	318
6.39.2.3 _star_v1_ConnectionStatus_MIN	318
6.39.2.4 _star_v1_EstopReason_ARRAYSIZE	318
6.39.2.5 _star_v1_EstopReason_MAX	318
6.39.2.6 _star_v1_EstopReason_MIN	318
6.39.2.7 _star_v1_GpsFix_ARRAYSIZE	318
6.39.2.8 _star_v1_GpsFix_MAX	319
6.39.2.9 _star_v1_GpsFix_MIN	319
6.39.2.10 _star_v1_RobotMode_ARRAYSIZE	319
6.39.2.11 _star_v1_RobotMode_MAX	319
6.39.2.12 _star_v1_RobotMode_MIN	319
6.39.2.13 star_v1_BaroData_CALLBACK	319
6.39.2.14 star_v1_BaroData_DEFAULT	319
6.39.2.15 star_v1_BaroData_FIELDLIST	320
6.39.2.16 star_v1_BaroData_fields	320
6.39.2.17 star_v1_BaroData_init_default	320
6.39.2.18 star_v1_BaroData_init_zero	320
6.39.2.19 star_v1_BaroData_pressure_pa_tag	320
6.39.2.20 star_v1_BaroData_size	320
6.39.2.21 star_v1_BaroData_temperature_celsius_tag	320
6.39.2.22 star_v1_GetSystemStatusRequest_CALLBACK	321
6.39.2.23 star_v1_GetSystemStatusRequest_DEFAULT	321
6.39.2.24 star_v1_GetSystemStatusRequest_FIELDLIST	321
6.39.2.25 star_v1_GetSystemStatusRequest_fields	321
6.39.2.26 star_v1_GetSystemStatusRequest_header_MSGTYPE	321
6.39.2.27 star_v1_GetSystemStatusRequest_header_tag	321
6.39.2.28 star_v1_GetSystemStatusRequest_init_default	321
6.39.2.29 star_v1_GetSystemStatusRequest_init_zero	322
6.39.2.30 star_v1_GetSystemStatusRequest_size	322

6.39.2.31	star_v1_GetSystemStatusResponse_CALLBACK	322
6.39.2.32	star_v1_GetSystemStatusResponse_DEFAULT	322
6.39.2.33	star_v1_GetSystemStatusResponse_FIELDLIST	322
6.39.2.34	star_v1_GetSystemStatusResponse_fields	322
6.39.2.35	star_v1_GetSystemStatusResponse_header_MSGTYPE	322
6.39.2.36	star_v1_GetSystemStatusResponse_header_tag	323
6.39.2.37	star_v1_GetSystemStatusResponse_init_default	323
6.39.2.38	star_v1_GetSystemStatusResponse_init_zero	323
6.39.2.39	star_v1_GetSystemStatusResponse_size	323
6.39.2.40	star_v1_GetSystemStatusResponse_status_MSGTYPE	323
6.39.2.41	star_v1_GetSystemStatusResponse_status_tag	323
6.39.2.42	star_v1_GetTelemetryRequest_CALLBACK	323
6.39.2.43	star_v1_GetTelemetryRequest_DEFAULT	323
6.39.2.44	star_v1_GetTelemetryRequest_FIELDLIST	324
6.39.2.45	star_v1_GetTelemetryRequest_fields	324
6.39.2.46	star_v1_GetTelemetryRequest_header_MSGTYPE	324
6.39.2.47	star_v1_GetTelemetryRequest_header_tag	324
6.39.2.48	star_v1_GetTelemetryRequest_init_default	324
6.39.2.49	star_v1_GetTelemetryRequest_init_zero	324
6.39.2.50	star_v1_GetTelemetryRequest_size	324
6.39.2.51	star_v1_GetTelemetryResponse_CALLBACK	325
6.39.2.52	star_v1_GetTelemetryResponse_DEFAULT	325
6.39.2.53	star_v1_GetTelemetryResponse_FIELDLIST	325
6.39.2.54	star_v1_GetTelemetryResponse_fields	325
6.39.2.55	star_v1_GetTelemetryResponse_header_MSGTYPE	325
6.39.2.56	star_v1_GetTelemetryResponse_header_tag	325
6.39.2.57	star_v1_GetTelemetryResponse_init_default	325
6.39.2.58	star_v1_GetTelemetryResponse_init_zero	326
6.39.2.59	star_v1_GetTelemetryResponse_size	326
6.39.2.60	star_v1_GetTelemetryResponse_telemetry_MSGTYPE	326
6.39.2.61	star_v1_GetTelemetryResponse_telemetry_tag	326
6.39.2.62	star_v1_GpsData_accuracy_m_tag	326
6.39.2.63	star_v1_GpsData_altitude_m_tag	326
6.39.2.64	star_v1_GpsData_CALLBACK	326
6.39.2.65	star_v1_GpsData_DEFAULT	326
6.39.2.66	star_v1_GpsData_FIELDLIST	327
6.39.2.67	star_v1_GpsData_fields	327
6.39.2.68	star_v1_GpsData_fix_type_ENUMTYPE	327
6.39.2.69	star_v1_GpsData_fix_type_tag	327
6.39.2.70	star_v1_GpsData_init_default	327
6.39.2.71	star_v1_GpsData_init_zero	327
6.39.2.72	star_v1_GpsData_latitude_deg_tag	328
6.39.2.73	star_v1_GpsData_longitude_deg_tag	328

6.39.2.74	star_v1_GpsData_satellites_tag	328
6.39.2.75	star_v1_GpsData_size	328
6.39.2.76	star_v1_ImuData_accel_x_mps2_tag	328
6.39.2.77	star_v1_ImuData_accel_y_mps2_tag	328
6.39.2.78	star_v1_ImuData_accel_z_mps2_tag	328
6.39.2.79	star_v1_ImuData_calibration_status_tag	328
6.39.2.80	star_v1_ImuData_CALLBACK	329
6.39.2.81	star_v1_ImuData_DEFAULT	329
6.39.2.82	star_v1_ImuData_FIELDLIST	329
6.39.2.83	star_v1_ImuData_fields	329
6.39.2.84	star_v1_ImuData_gyro_x_rad_per_s_tag	330
6.39.2.85	star_v1_ImuData_gyro_y_rad_per_s_tag	330
6.39.2.86	star_v1_ImuData_gyro_z_rad_per_s_tag	330
6.39.2.87	star_v1_ImuData_heading_rad_tag	330
6.39.2.88	star_v1_ImuData_init_default	330
6.39.2.89	star_v1_ImuData_init_zero	330
6.39.2.90	star_v1_ImuData_pitch_rad_tag	330
6.39.2.91	star_v1_ImuData_quat_w_tag	330
6.39.2.92	star_v1_ImuData_quat_x_tag	331
6.39.2.93	star_v1_ImuData_quat_y_tag	331
6.39.2.94	star_v1_ImuData_quat_z_tag	331
6.39.2.95	star_v1_ImuData_roll_rad_tag	331
6.39.2.96	star_v1_ImuData_size	331
6.39.2.97	star_v1_ImuData_temperature_celsius_tag	331
6.39.2.98	star_v1_ImuData_yaw_rad_tag	331
6.39.2.99	star_v1_ObstacleData_any_obstacle_tag	331
6.39.2.100	star_v1_ObstacleData_CALLBACK	332
6.39.2.101	star_v1_ObstacleData_DEFAULT	332
6.39.2.102	star_v1_ObstacleData_detected_mask_tag	332
6.39.2.103	star_v1_ObstacleData_distance_back_left_m_tag	332
6.39.2.104	star_v1_ObstacleData_distance_back_right_m_tag	332
6.39.2.105	star_v1_ObstacleData_distance_front_left_m_tag	332
6.39.2.106	star_v1_ObstacleData_distance_front_right_m_tag	332
6.39.2.107	star_v1_ObstacleData_FIELDLIST	333
6.39.2.108	star_v1_ObstacleData_fields	333
6.39.2.109	star_v1_ObstacleData_init_default	333
6.39.2.110	star_v1_ObstacleData_init_zero	333
6.39.2.111	star_v1_ObstacleData_size	333
6.39.2.112	STAR_V1_STAR_V1_TELEMETRY_PB_H_MAX_SIZE	333
6.39.2.113	star_v1_StreamTelemetryRequest_CALLBACK	334
6.39.2.114	star_v1_StreamTelemetryRequest_DEFAULT	334
6.39.2.115	star_v1_StreamTelemetryRequest_FIELDLIST	334
6.39.2.116	star_v1_StreamTelemetryRequest_fields	334

6.39.2.117	star_v1_StreamTelemetryRequest_fields_tag	334
6.39.2.118	star_v1_StreamTelemetryRequest_header_MSGTYPE	334
6.39.2.119	star_v1_StreamTelemetryRequest_header_tag	334
6.39.2.120	star_v1_StreamTelemetryRequest_init_default	335
6.39.2.121	star_v1_StreamTelemetryRequest_init_zero	335
6.39.2.122	star_v1_StreamTelemetryRequest_rate_hz_tag	335
6.39.2.123	star_v1_StreamTelemetryRequest_size	335
6.39.2.124	star_v1_SystemStatus_CALLBACK	335
6.39.2.125	star_v1_SystemStatus_connection_status_ENUMTYPE	335
6.39.2.126	star_v1_SystemStatus_connection_status_tag	335
6.39.2.127	star_v1_SystemStatus_DEFAULT	336
6.39.2.128	star_v1_SystemStatus_FIELDLIST	336
6.39.2.129	star_v1_SystemStatus_fields	336
6.39.2.130	star_v1_SystemStatus_firmware_version_tag	336
6.39.2.131	star_v1_SystemStatus_free_heap_bytes_tag	336
6.39.2.132	star_v1_SystemStatus_init_default	336
6.39.2.133	star_v1_SystemStatus_init_zero	337
6.39.2.134	star_v1_SystemStatus_lidar_connected_tag	337
6.39.2.135	star_v1_SystemStatus_mode_ENUMTYPE	337
6.39.2.136	star_v1_SystemStatus_mode_tag	337
6.39.2.137	star_v1_SystemStatus_ros_connected_tag	337
6.39.2.138	star_v1_SystemStatus_rx72n_connected_tag	337
6.39.2.139	star_v1_SystemStatus_size	337
6.39.2.140	star_v1_SystemStatus_uptime_s_tag	337
6.39.2.141	star_v1_TelemetryData_baro_MSGTYPE	338
6.39.2.142	star_v1_TelemetryData_baro_tag	338
6.39.2.143	star_v1_TelemetryData_CALLBACK	338
6.39.2.144	star_v1_TelemetryData_cpu_usage_percent_tag	338
6.39.2.145	star_v1_TelemetryData_DEFAULT	338
6.39.2.146	star_v1_TelemetryData_emergency_stop_tag	338
6.39.2.147	star_v1_TelemetryData_encoder_back_left_MSGTYPE	338
6.39.2.148	star_v1_TelemetryData_encoder_back_left_tag	338
6.39.2.149	star_v1_TelemetryData_encoder_back_right_MSGTYPE	339
6.39.2.150	star_v1_TelemetryData_encoder_back_right_tag	339
6.39.2.151	star_v1_TelemetryData_encoder_front_left_MSGTYPE	339
6.39.2.152	star_v1_TelemetryData_encoder_front_left_tag	339
6.39.2.153	star_v1_TelemetryData_encoder_front_right_MSGTYPE	339
6.39.2.154	star_v1_TelemetryData_encoder_front_right_tag	339
6.39.2.155	star_v1_TelemetryData_estop_reason_ENUMTYPE	339
6.39.2.156	star_v1_TelemetryData_estop_reason_tag	339
6.39.2.157	star_v1_TelemetryData_fault_flags_tag	340
6.39.2.158	star_v1_TelemetryData_FIELDLIST	340
6.39.2.159	star_v1_TelemetryData_fields	340

6.39.2.160	star_v1_TelemetryData_frame_sequence_tag	340
6.39.2.161	star_v1_TelemetryData_gps_MSGTYPE	341
6.39.2.162	star_v1_TelemetryData_gps_tag	341
6.39.2.163	star_v1_TelemetryData_imu_MSGTYPE	341
6.39.2.164	star_v1_TelemetryData_imu_tag	341
6.39.2.165	star_v1_TelemetryData_init_default	341
6.39.2.166	star_v1_TelemetryData_init_zero	341
6.39.2.167	star_v1_TelemetryData_motor_load_percent_tag	341
6.39.2.168	star_v1_TelemetryData_obstacle_MSGTYPE	342
6.39.2.169	star_v1_TelemetryData_obstacle_tag	342
6.39.2.170	star_v1_TelemetryData_size	342
6.39.2.171	star_v1_TelemetryData_temperature_celsius_tag	342
6.39.2.172	star_v1_TelemetryData_timestamp_MSGTYPE	342
6.39.2.173	star_v1_TelemetryData_timestamp_tag	342
6.39.2.174	star_v1_TelemetryData_timestamp_us_tag	342
6.39.2.175	star_v1_TelemetryData_wifi_signal_dbm_tag	342
6.39.3	Enumeration Type Documentation	343
6.39.3.1	star_v1_ConnectionStatus	343
6.39.3.2	star_v1_EstopReason	343
6.39.3.3	star_v1_GpsFix	344
6.39.3.4	star_v1_RobotMode	344
6.39.4	Variable Documentation	345
6.39.4.1	star_v1_BaroData_msg	345
6.39.4.2	star_v1_GetSystemStatusRequest_msg	345
6.39.4.3	star_v1_GetSystemStatusResponse_msg	345
6.39.4.4	star_v1_GetTelemetryRequest_msg	345
6.39.4.5	star_v1_GetTelemetryResponse_msg	345
6.39.4.6	star_v1_GpsData_msg	345
6.39.4.7	star_v1_ImuData_msg	345
6.39.4.8	star_v1_ObstacleData_msg	345
6.39.4.9	star_v1_StreamTelemetryRequest_msg	345
6.39.4.10	star_v1_SystemStatus_msg	345
6.39.4.11	star_v1_TelemetryData_msg	346
6.40	telemetry.pb.h	346
6.41	libs/rx_nanopb/inc/gen/star/v1/ui.pb.c File Reference	353
6.42	ui.pb.c	354
6.43	libs/rx_nanopb/inc/gen/star/v1/ui.pb.h File Reference	354
6.43.1	Data Structure Documentation	359
6.43.1.1	struct star_v1_MotorStatusList	359
6.43.1.2	struct star_v1_OdometryData	359
6.43.1.3	struct star_v1_LidarScan	360
6.43.1.4	struct star_v1_Alert	361
6.43.1.5	struct star_v1_EStopCommand	361

6.43.1.6 union star_v1_STAREnvelope.payload	362
6.43.2 Macro Definition Documentation	362
6.43.2.1 _star_v1_AlertLevel_ARRAYSIZE	362
6.43.2.2 _star_v1_AlertLevel_MAX	362
6.43.2.3 _star_v1_AlertLevel_MIN	362
6.43.2.4 star_v1_Alert_CALLBACK	362
6.43.2.5 star_v1_Alert_code_tag	363
6.43.2.6 star_v1_Alert_DEFAULT	363
6.43.2.7 star_v1_Alert_FIELDLIST	363
6.43.2.8 star_v1_Alert_fields	363
6.43.2.9 star_v1_Alert_init_default	363
6.43.2.10 star_v1_Alert_init_zero	363
6.43.2.11 star_v1_Alert_level_ENUMTYPE	364
6.43.2.12 star_v1_Alert_level_tag	364
6.43.2.13 star_v1_Alert_message_tag	364
6.43.2.14 star_v1_Alert_size	364
6.43.2.15 star_v1_Alert_source_tag	364
6.43.2.16 star_v1_Alert_timestamp_us_tag	364
6.43.2.17 star_v1_EStopCommand_active_tag	364
6.43.2.18 star_v1_EStopCommand_CALLBACK	364
6.43.2.19 star_v1_EStopCommand_DEFAULT	365
6.43.2.20 star_v1_EStopCommand_FIELDLIST	365
6.43.2.21 star_v1_EStopCommand_fields	365
6.43.2.22 star_v1_EStopCommand_init_default	365
6.43.2.23 star_v1_EStopCommand_init_zero	365
6.43.2.24 star_v1_EStopCommand_reason_tag	365
6.43.2.25 star_v1_EStopCommand_size	365
6.43.2.26 star_v1_EStopCommand_timestamp_us_tag	366
6.43.2.27 star_v1_LidarScan_angle_rad_tag	366
6.43.2.28 star_v1_LidarScan_CALLBACK	366
6.43.2.29 star_v1_LidarScan_DEFAULT	366
6.43.2.30 star_v1_LidarScan_FIELDLIST	366
6.43.2.31 star_v1_LidarScan_fields	366
6.43.2.32 star_v1_LidarScan_init_default	367
6.43.2.33 star_v1_LidarScan_init_zero	367
6.43.2.34 star_v1_LidarScan_intensity_tag	368
6.43.2.35 star_v1_LidarScan_range_m_tag	368
6.43.2.36 star_v1_LidarScan_size	368
6.43.2.37 star_v1_LidarScan_timestamp_us_tag	368
6.43.2.38 star_v1_MotorStatusList_CALLBACK	368
6.43.2.39 star_v1_MotorStatusList_DEFAULT	369
6.43.2.40 star_v1_MotorStatusList_FIELDLIST	369
6.43.2.41 star_v1_MotorStatusList_fields	369

6.43.2.42	star_v1_MotorStatusList_init_default	369
6.43.2.43	star_v1_MotorStatusList_init_zero	369
6.43.2.44	star_v1_MotorStatusList_motors_MSGTYPE	369
6.43.2.45	star_v1_MotorStatusList_motors_tag	369
6.43.2.46	star_v1_MotorStatusList_size	370
6.43.2.47	star_v1_OdometryData_angular_velocity_rad_per_s_tag	370
6.43.2.48	star_v1_OdometryData_CALLBACK	370
6.43.2.49	star_v1_OdometryData_DEFAULT	370
6.43.2.50	star_v1_OdometryData_FIELDLIST	370
6.43.2.51	star_v1_OdometryData_fields	370
6.43.2.52	star_v1_OdometryData_init_default	371
6.43.2.53	star_v1_OdometryData_init_zero	371
6.43.2.54	star_v1_OdometryData_linear_velocity_mps_tag	371
6.43.2.55	star_v1_OdometryData_size	371
6.43.2.56	star_v1_OdometryData_theta_rad_tag	371
6.43.2.57	star_v1_OdometryData_timestamp_us_tag	371
6.43.2.58	star_v1_OdometryData_x_m_tag	371
6.43.2.59	star_v1_OdometryData_y_m_tag	371
6.43.2.60	STAR_V1_STAR_V1_UI_PB_H_MAX_SIZE	372
6.43.2.61	star_v1_STAREnvelope_alert_tag	372
6.43.2.62	star_v1_STAREnvelope_CALLBACK	372
6.43.2.63	star_v1_STAREnvelope_controller_tag	372
6.43.2.64	star_v1_STAREnvelope_DEFAULT	372
6.43.2.65	star_v1_STAREnvelope_estop_tag	372
6.43.2.66	star_v1_STAREnvelope_FIELDLIST	372
6.43.2.67	star_v1_STAREnvelope_fields	373
6.43.2.68	star_v1_STAREnvelope_init_default	373
6.43.2.69	star_v1_STAREnvelope_init_zero	373
6.43.2.70	star_v1_STAREnvelope_lidar_tag	373
6.43.2.71	star_v1_STAREnvelope_motors_tag	373
6.43.2.72	star_v1_STAREnvelope_odometry_tag	373
6.43.2.73	star_v1_STAREnvelope_payload_alert_MSGTYPE	373
6.43.2.74	star_v1_STAREnvelope_payload_controller_MSGTYPE	373
6.43.2.75	star_v1_STAREnvelope_payload_estop_MSGTYPE	374
6.43.2.76	star_v1_STAREnvelope_payload_lidar_MSGTYPE	374
6.43.2.77	star_v1_STAREnvelope_payload_motors_MSGTYPE	374
6.43.2.78	star_v1_STAREnvelope_payload_odometry_MSGTYPE	374
6.43.2.79	star_v1_STAREnvelope_payload_system_MSGTYPE	374
6.43.2.80	star_v1_STAREnvelope_payload_telemetry_MSGTYPE	374
6.43.2.81	star_v1_STAREnvelope_seq_tag	374
6.43.2.82	star_v1_STAREnvelope_size	374
6.43.2.83	star_v1_STAREnvelope_system_tag	375
6.43.2.84	star_v1_STAREnvelope_telemetry_tag	375

6.43.2.85	star_v1_STAREnvelope_ts_ms_tag	375
6.43.3	Enumeration Type Documentation	375
6.43.3.1	star_v1_AlertLevel	375
6.43.4	Variable Documentation	375
6.43.4.1	star_v1_Alert_msg	375
6.43.4.2	star_v1_EStopCommand_msg	376
6.43.4.3	star_v1_LidarScan_msg	376
6.43.4.4	star_v1_MotorStatusList_msg	376
6.43.4.5	star_v1_OdometryData_msg	376
6.43.4.6	star_v1_STAREnvelope_msg	376
6.44	ui.pb.h	376
6.45	libs/rx_nanopb/inc/gen/star/v1/wire.pb.c File Reference	380
6.46	wire.pb.c	381
6.47	libs/rx_nanopb/inc/gen/star/v1/wire.pb.h File Reference	381
6.47.1	Data Structure Documentation	383
6.47.1.1	struct star_v1_EmergencyStopCommand	383
6.47.1.2	union star_v1_WireMessage_payload	384
6.47.2	Macro Definition Documentation	384
6.47.2.1	star_v1_EmergencyStopCommand_CALLBACK	384
6.47.2.2	star_v1_EmergencyStopCommand_DEFAULT	384
6.47.2.3	star_v1_EmergencyStopCommand_engage_hardware_stop_tag	384
6.47.2.4	star_v1_EmergencyStopCommand_FIELDLIST	384
6.47.2.5	star_v1_EmergencyStopCommand_fields	385
6.47.2.6	star_v1_EmergencyStopCommand_init_default	385
6.47.2.7	star_v1_EmergencyStopCommand_init_zero	385
6.47.2.8	star_v1_EmergencyStopCommand_reason_tag	385
6.47.2.9	star_v1_EmergencyStopCommand_size	385
6.47.2.10	star_v1_EmergencyStopCommand_timestamp_us_tag	385
6.47.2.11	STAR_V1_STAR_V1_WIRE_PB_H_MAX_SIZE	385
6.47.2.12	star_v1_WireMessage_CALLBACK	385
6.47.2.13	star_v1_WireMessage_DEFAULT	386
6.47.2.14	star_v1_WireMessage_emergency_stop_command_tag	386
6.47.2.15	star_v1_WireMessage_encoder_data_tag	386
6.47.2.16	star_v1_WireMessage_FIELDLIST	386
6.47.2.17	star_v1_WireMessage_fields	386
6.47.2.18	star_v1_WireMessage_init_default	387
6.47.2.19	star_v1_WireMessage_init_zero	387
6.47.2.20	star_v1_WireMessage_motor_power_command_tag	387
6.47.2.21	star_v1_WireMessage_payload_emergency_stop_command_MSGTYPE	387
6.47.2.22	star_v1_WireMessage_payload_encoder_data_MSGTYPE	387
6.47.2.23	star_v1_WireMessage_payload_motor_power_command_MSGTYPE	387
6.47.2.24	star_v1_WireMessage_payload_pid_config_MSGTYPE	387
6.47.2.25	star_v1_WireMessage_payload_retransmit_config_MSGTYPE	387

6.47.2.26	star_v1_WireMessage_payload_telemetry_data_MSGTYPE	388
6.47.2.27	star_v1_WireMessage_payload_transport_diagnostics_MSGTYPE . . .	388
6.47.2.28	star_v1_WireMessage_payload_velocity_command_MSGTYPE	388
6.47.2.29	star_v1_WireMessage_pid_config_tag	388
6.47.2.30	star_v1_WireMessage_retransmit_config_tag	388
6.47.2.31	star_v1_WireMessage_size	388
6.47.2.32	star_v1_WireMessage_telemetry_data_tag	388
6.47.2.33	star_v1_WireMessage_transport_diagnostics_tag	389
6.47.2.34	star_v1_WireMessage_velocity_command_tag	389
6.47.3	Variable Documentation	389
6.47.3.1	star_v1_EmergencyStopCommand_msg	389
6.47.3.2	star_v1_WireMessage_msg	389
6.48	wire.pb.h	389
6.49	libs/rx_nanopb/inc/rx_nanopb.h File Reference	391
6.49.1	Detailed Description	393
6.49.2	Data Structure Documentation	397
6.49.2.1	struct rx_velocity_command_params_t	397
6.49.2.2	struct rx_velocity_diff_drive_params_t	398
6.49.3	Function Documentation	399
6.49.3.1	rx_nanopb_create_response_header()	399
6.49.3.2	rx_nanopb_create_velocity_command()	402
6.49.3.3	rx_nanopb_create_velocity_command_diff_drive()	405
6.49.3.4	rx_nanopb_decode_estop_request()	408
6.49.3.5	rx_nanopb_decode_pid_gains_request()	412
6.49.3.6	rx_nanopb_decode_retransmit_config_request()	417
6.49.3.7	rx_nanopb_decode_velocity_request()	419
6.49.3.8	rx_nanopb_encode_estop_response()	423
6.49.3.9	rx_nanopb_encode_pid_gains_response()	425
6.49.3.10	rx_nanopb_encode_telemetry()	428
6.49.3.11	rx_nanopb_encode_velocity_request()	432
6.49.3.12	rx_nanopb_encode_velocity_response()	436
6.49.3.13	rx_nanopb_init()	439
6.49.3.14	rx_nanopb_test_reset_state()	443
6.49.4	Variable Documentation	445
6.49.4.1	s_nanopb_buffer_size	445
6.50	rx_nanopb.h	446
6.51	libs/rx_nanopb/nanopb/extra/pb_syshdr.h File Reference	458
6.51.1	Macro Definition Documentation	458
6.51.1.1	CHAR_BIT	458
6.51.1.2	false	458
6.51.1.3	NULL	459
6.51.1.4	offsetof	459
6.51.1.5	true	459

6.51.2 Typedef Documentation	459
6.51.2.1 bool	459
6.51.2.2 int16_t	459
6.51.2.3 int32_t	459
6.51.2.4 int64_t	460
6.51.2.5 int8_t	460
6.51.2.6 int_least16_t	460
6.51.2.7 int_least8_t	460
6.51.2.8 size_t	460
6.51.2.9 uint16_t	460
6.51.2.10 uint32_t	460
6.51.2.11 uint64_t	460
6.51.2.12 uint8_t	461
6.51.2.13 uint_fast8_t	461
6.51.2.14 uint_least16_t	461
6.51.2.15 uint_least8_t	461
6.51.3 Function Documentation	461
6.51.3.1 memcpy()	461
6.51.3.2 memset()	462
6.51.3.3 strlen()	463
6.52 pb_syshdr.h	463
6.53 libs/rx_nanopb/nanopb/pb_common.c File Reference	464
6.53.1 Function Documentation	465
6.53.1.1 advance_iterator()	465
6.53.1.2 load_descriptor_values()	466
6.53.1.3 pb_const_cast()	468
6.53.1.4 pb_default_field_callback()	469
6.53.1.5 pb_field_iter_begin()	469
6.53.1.6 pb_field_iter_begin_const()	470
6.53.1.7 pb_field_iter_begin_extension()	471
6.53.1.8 pb_field_iter_begin_extension_const()	472
6.53.1.9 pb_field_iter_find()	472
6.53.1.10 pb_field_iter_find_extension()	474
6.53.1.11 pb_field_iter_next()	475
6.54 pb_common.c	475
6.55 libs/rx_nanopb/nanopb/pb_common.h File Reference	480
6.55.1 Function Documentation	481
6.55.1.1 pb_field_iter_begin()	481
6.55.1.2 pb_field_iter_begin_const()	481
6.55.1.3 pb_field_iter_begin_extension()	481
6.55.1.4 pb_field_iter_begin_extension_const()	482
6.55.1.5 pb_field_iter_find()	482
6.55.1.6 pb_field_iter_find_extension()	483

6.55.1.7 pb_field_iter_next()	483
6.56 pb_common.h	483
6.57 libs/rx_nanopb/nanopb/spm_headers/nanopb/pb_common.h File Reference	484
6.57.1 Function Documentation	485
6.57.1.1 pb_field_iter_begin()	485
6.57.1.2 pb_field_iter_begin_const()	486
6.57.1.3 pb_field_iter_begin_extension()	487
6.57.1.4 pb_field_iter_begin_extension_const()	488
6.57.1.5 pb_field_iter_find()	488
6.57.1.6 pb_field_iter_find_extension()	490
6.57.1.7 pb_field_iter_next()	491
6.58 pb_common.h	491
6.59 libs/rx_nanopb/nanopb/spm_headers/pb_common.h File Reference	492
6.60 pb_common.h	492
6.61 libs/rx_nanopb/nanopb/pb_decode.c File Reference	493
6.61.1 Data Structure Documentation	494
6.61.1.1 struct pb_fields_seen_t	494
6.61.2 Macro Definition Documentation	495
6.61.2.1 checkreturn	495
6.61.2.2 pb_int64_t	495
6.61.2.3 pb_uint64_t	495
6.61.3 Function Documentation	496
6.61.3.1 buf_read()	496
6.61.3.2 decode_basic_field()	497
6.61.3.3 decode_callback_field()	498
6.61.3.4 decode_extension()	500
6.61.3.5 decode_field()	501
6.61.3.6 decode_pointer_field()	502
6.61.3.7 decode_static_field()	504
6.61.3.8 default_extension_decoder()	506
6.61.3.9 pb_close_string_substream()	507
6.61.3.10 pb_dec_bool()	508
6.61.3.11 pb_dec_bytes()	509
6.61.3.12 pb_dec_fixed_length_bytes()	510
6.61.3.13 pb_dec_string()	511
6.61.3.14 pb_dec_submessage()	512
6.61.3.15 pb_dec_varint()	513
6.61.3.16 pb_decode()	514
6.61.3.17 pb_decode_bool()	516
6.61.3.18 pb_decode_ex()	516
6.61.3.19 pb_decode_fixed32()	518
6.61.3.20 pb_decode_fixed64()	518
6.61.3.21 pb_decode_inner()	519

6.61.3.22	pb_decode_svarint()	522
6.61.3.23	pb_decode_tag()	523
6.61.3.24	pb_decode_varint()	524
6.61.3.25	pb_decode_varint32()	525
6.61.3.26	pb_field_set_to_default()	526
6.61.3.27	pb_istream_from_buffer()	528
6.61.3.28	pb_make_string_substream()	529
6.61.3.29	pb_message_set_to_defaults()	529
6.61.3.30	pb_read()	531
6.61.3.31	pb_readbyte()	532
6.61.3.32	pb_release()	533
6.61.3.33	pb_skip_field()	533
6.61.3.34	pb_skip_string()	534
6.61.3.35	pb_skip_varint()	535
6.61.3.36	read_raw_value()	535
6.62	pb_decode.c	536
6.63	libs/rx_nanopb/nanopb/pb_encode.c File Reference	557
6.63.1	Macro Definition Documentation	558
6.63.1.1	checkreturn	558
6.63.1.2	pb_int64_t	558
6.63.1.3	pb_uint64_t	558
6.63.2	Function Documentation	559
6.63.2.1	buf_write()	559
6.63.2.2	default_extension_encoder()	559
6.63.2.3	encode_array()	560
6.63.2.4	encode_basic_field()	562
6.63.2.5	encode_callback_field()	564
6.63.2.6	encode_extension_field()	564
6.63.2.7	encode_field()	565
6.63.2.8	pb_check_proto3_default_value()	567
6.63.2.9	pb_enc_bool()	569
6.63.2.10	pb_enc_bytes()	569
6.63.2.11	pb_enc_fixed()	570
6.63.2.12	pb_enc_fixed_length_bytes()	571
6.63.2.13	pb_enc_string()	572
6.63.2.14	pb_enc_submessage()	573
6.63.2.15	pb_enc_varint()	574
6.63.2.16	pb_encode()	575
6.63.2.17	pb_encode_ex()	576
6.63.2.18	pb_encode_fixed32()	577
6.63.2.19	pb_encode_fixed64()	577
6.63.2.20	pb_encode_string()	578
6.63.2.21	pb_encode_submessage()	579

6.63.2.22	pb_encode_svarint()	580
6.63.2.23	pb_encode_tag()	581
6.63.2.24	pb_encode_tag_for_field()	582
6.63.2.25	pb_encode_varint()	583
6.63.2.26	pb_encode_varint_32()	584
6.63.2.27	pb_get_encoded_size()	585
6.63.2.28	pb_ostream_from_buffer()	585
6.63.2.29	pb_write()	586
6.63.2.30	safe_read_bool()	587
6.64	pb_encode.c	588
6.65	libs/rx_nanopb/nanopb/spm-test/objc/c-header.c File Reference	599
6.66	c-header.c	600
6.67	libs/rx_nanopb/nanopb/pb_decode.h File Reference	600
6.67.1	Macro Definition Documentation	601
6.67.1.1	PB_DECODE_DELIMITED	601
6.67.1.2	pb_decode_delimited	601
6.67.1.3	pb_decode_delimited_noinit	601
6.67.1.4	PB_DECODE_NOINIT	602
6.67.1.5	pb_decode_noinit	602
6.67.1.6	PB_DECODE_NULLTERMINATED	602
6.67.1.7	pb_decode_nullterminated	602
6.67.1.8	PB_ISTREAM_EMPTY	602
6.67.2	Function Documentation	603
6.67.2.1	pb_close_string_substream()	603
6.67.2.2	pb_decode()	603
6.67.2.3	pb_decode_bool()	603
6.67.2.4	pb_decode_ex()	604
6.67.2.5	pb_decode_fixed32()	604
6.67.2.6	pb_decode_fixed64()	604
6.67.2.7	pb_decode_svarint()	605
6.67.2.8	pb_decode_tag()	605
6.67.2.9	pb_decode_varint()	606
6.67.2.10	pb_decode_varint32()	606
6.67.2.11	pb_istream_from_buffer()	607
6.67.2.12	pb_make_string_substream()	607
6.67.2.13	pb_read()	608
6.67.2.14	pb_release()	608
6.67.2.15	pb_skip_field()	609
6.68	pb_decode.h	609
6.69	libs/rx_nanopb/nanopb/spm_headers/nanopb/pb_decode.h File Reference	611
6.69.1	Macro Definition Documentation	613
6.69.1.1	PB_DECODE_DELIMITED	613
6.69.1.2	pb_decode_delimited	613

6.69.1.3	pb_decode_delimited_noinit	613
6.69.1.4	PB_DECODE_NOINIT	613
6.69.1.5	pb_decode_noinit	614
6.69.1.6	PB_DECODE_NULLTERMINATED	614
6.69.1.7	pb_decode_nullterminated	614
6.69.1.8	PB_ISTREAM_EMPTY	614
6.69.2	Function Documentation	614
6.69.2.1	pb_close_string_substream()	614
6.69.2.2	pb_decode()	615
6.69.2.3	pb_decode_bool()	616
6.69.2.4	pb_decode_ex()	617
6.69.2.5	pb_decode_fixed32()	618
6.69.2.6	pb_decode_fixed64()	619
6.69.2.7	pb_decode_svarint()	620
6.69.2.8	pb_decode_tag()	621
6.69.2.9	pb_decode_varint()	622
6.69.2.10	pb_decode_varint32()	622
6.69.2.11	pb_istream_from_buffer()	624
6.69.2.12	pb_make_string_substream()	625
6.69.2.13	pb_read()	625
6.69.2.14	pb_release()	627
6.69.2.15	pb_skip_field()	627
6.70	pb_decode.h	628
6.71	libs/rx_nanopb/nanopb/spm_headers/pb_decode.h File Reference	630
6.72	pb_decode.h	631
6.73	libs/rx_nanopb/nanopb/pb_encode.h File Reference	631
6.73.1	Macro Definition Documentation	632
6.73.1.1	PB_ENCODE_DELIMITED	632
6.73.1.2	pb_encode_delimited	633
6.73.1.3	PB_ENCODE_NULLTERMINATED	633
6.73.1.4	pb_encode_nullterminated	633
6.73.1.5	PB_OSTREAM_SIZING	633
6.73.2	Function Documentation	634
6.73.2.1	pb_encode()	634
6.73.2.2	pb_encode_ex()	634
6.73.2.3	pb_encode_fixed32()	634
6.73.2.4	pb_encode_fixed64()	635
6.73.2.5	pb_encode_string()	635
6.73.2.6	pb_encode_submessage()	635
6.73.2.7	pb_encode_svarint()	636
6.73.2.8	pb_encode_tag()	636
6.73.2.9	pb_encode_tag_for_field()	637
6.73.2.10	pb_encode_varint()	637

6.73.2.11 pb_get_encoded_size()	638
6.73.2.12 pb_ostream_from_buffer()	638
6.73.2.13 pb_write()	638
6.74 pb_encode.h	639
6.75 libs/rx_nanopb/nanopb/spm_headers/nanopb/pb_encode.h File Reference	641
6.75.1 Macro Definition Documentation	642
6.75.1.1 PB_ENCODE_DELIMITED	642
6.75.1.2 pb_encode_delimited	642
6.75.1.3 PB_ENCODE_NULLTERMINATED	643
6.75.1.4 pb_encode_nullterminated	643
6.75.1.5 PB_OSTREAM_SIZING	643
6.75.2 Function Documentation	643
6.75.2.1 pb_encode()	643
6.75.2.2 pb_encode_ex()	644
6.75.2.3 pb_encode_fixed32()	645
6.75.2.4 pb_encode_fixed64()	646
6.75.2.5 pb_encode_string()	646
6.75.2.6 pb_encode_submessage()	647
6.75.2.7 pb_encode_svarint()	648
6.75.2.8 pb_encode_tag()	649
6.75.2.9 pb_encode_tag_for_field()	650
6.75.2.10 pb_encode_varint()	651
6.75.2.11 pb_get_encoded_size()	652
6.75.2.12 pb_ostream_from_buffer()	652
6.75.2.13 pb_write()	653
6.76 pb_encode.h	654
6.77 libs/rx_nanopb/nanopb/spm_headers/pb_encode.h File Reference	656
6.78 pb_encode.h	657
6.79 libs/rx_nanopb/nanopb/pb.h File Reference	657
6.79.1 Data Structure Documentation	663
6.79.1.1 struct pb_field_iter_s	663
6.79.1.2 struct pb_bytes_array_s	663
6.79.1.3 struct pb_extension_s	664
6.79.2 Macro Definition Documentation	664
6.79.2.1 NANOPB_VERSION	664
6.79.2.2 PB_ARRAY_SIZE_CALLBACK	665
6.79.2.3 PB_ARRAY_SIZE_POINTER	665
6.79.2.4 PB_ARRAY_SIZE_STATIC	665
6.79.2.5 pb_arraysize	665
6.79.2.6 PB_AS_PB_HTYPE_FIXARRAY	666
6.79.2.7 PB_AS_PB_HTYPE_ONEOF	666
6.79.2.8 PB_AS_PB_HTYPE_OPTIONAL	666
6.79.2.9 PB_AS_PB_HTYPE_REPEATED	666

6.79.2.10 PB_AS_PB_HTYPE_REQUIRED	666
6.79.2.11 PB_AS_PB_HTYPE_SINGULAR	667
6.79.2.12 PB_AS_PTR_PB_HTYPE_FIXARRAY	667
6.79.2.13 PB_AS_PTR_PB_HTYPE_ONEOF	667
6.79.2.14 PB_AS_PTR_PB_HTYPE_OPTIONAL	667
6.79.2.15 PB_AS_PTR_PB_HTYPE_REPEATED	667
6.79.2.16 PB_AS_PTR_PB_HTYPE_REQUIRED	668
6.79.2.17 PB_AS_PTR_PB_HTYPE_SINGULAR	668
6.79.2.18 PB_ATYPE	668
6.79.2.19 PB_ATYPE_CALLBACK	668
6.79.2.20 PB_ATYPE_MASK	668
6.79.2.21 PB_ATYPE_POINTER	669
6.79.2.22 PB_ATYPE_STATIC	669
6.79.2.23 PB_BIND	669
6.79.2.24 PB_BYTES_ARRAY_T	670
6.79.2.25 PB_BYTES_ARRAY_T_ALLOCSIZE	670
6.79.2.26 PB_DATA_OFFSET_CALLBACK	670
6.79.2.27 PB_DATA_OFFSET_POINTER	670
6.79.2.28 PB_DATA_OFFSET_STATIC	670
6.79.2.29 PB_DATA_SIZE_CALLBACK	671
6.79.2.30 PB_DATA_SIZE_POINTER	671
6.79.2.31 PB_DATA_SIZE_STATIC	671
6.79.2.32 pb_delta	671
6.79.2.33 PB_DO_PB_HTYPE_FIXARRAY	672
6.79.2.34 PB_DO_PB_HTYPE_ONEOF	672
6.79.2.35 PB_DO_PB_HTYPE_OPTIONAL	672
6.79.2.36 PB_DO_PB_HTYPE_REPEATED	672
6.79.2.37 PB_DO_PB_HTYPE_REQUIRED	672
6.79.2.38 PB_DO_PB_HTYPE_SINGULAR	673
6.79.2.39 PB_DS_CB_PB_HTYPE_FIXARRAY	673
6.79.2.40 PB_DS_CB_PB_HTYPE_ONEOF	673
6.79.2.41 PB_DS_CB_PB_HTYPE_OPTIONAL	673
6.79.2.42 PB_DS_CB_PB_HTYPE_REPEATED	673
6.79.2.43 PB_DS_CB_PB_HTYPE_REQUIRED	674
6.79.2.44 PB_DS_CB_PB_HTYPE_SINGULAR	674
6.79.2.45 PB_DS_PB_HTYPE_FIXARRAY	674
6.79.2.46 PB_DS_PB_HTYPE_ONEOF	674
6.79.2.47 PB_DS_PB_HTYPE_OPTIONAL	674
6.79.2.48 PB_DS_PB_HTYPE_REPEATED	675
6.79.2.49 PB_DS_PB_HTYPE_REQUIRED	675
6.79.2.50 PB_DS_PB_HTYPE_SINGULAR	675
6.79.2.51 PB_DS_PTR_PB_HTYPE_FIXARRAY	675
6.79.2.52 PB_DS_PTR_PB_HTYPE_ONEOF	675

6.79.2.53 PB_DS_PTR_PB_HTYPE_OPTIONAL	676
6.79.2.54 PB_DS_PTR_PB_HTYPE_REPEATED	676
6.79.2.55 PB_DS_PTR_PB_HTYPE_REQUIRED	676
6.79.2.56 PB_DS_PTR_PB_HTYPE_SINGULAR	676
6.79.2.57 PB_EXPAND	676
6.79.2.58 pb_extension_init_zero	677
6.79.2.59 PB_FI_WIDTH_PB_ATYPE_CALLBACK	677
6.79.2.60 PB_FI_WIDTH_PB_ATYPE_POINTER	677
6.79.2.61 PB_FI_WIDTH_PB_ATYPE_STATIC	677
6.79.2.62 PB_FI_WIDTH_PB_HTYPE_FIXARRAY	677
6.79.2.63 PB_FI_WIDTH_PB_HTYPE_ONEOF	678
6.79.2.64 PB_FI_WIDTH_PB_HTYPE_OPTIONAL	678
6.79.2.65 PB_FI_WIDTH_PB_HTYPE_REPEATED	678
6.79.2.66 PB_FI_WIDTH_PB_HTYPE_REQUIRED	678
6.79.2.67 PB_FI_WIDTH_PB_HTYPE_SINGULAR	678
6.79.2.68 PB_FI_WIDTH_PB_LTYPE_BOOL	679
6.79.2.69 PB_FI_WIDTH_PB_LTYPE_BYTES	679
6.79.2.70 PB_FI_WIDTH_PB_LTYPE_DOUBLE	679
6.79.2.71 PB_FI_WIDTH_PB_LTYPE_ENUM	679
6.79.2.72 PB_FI_WIDTH_PB_LTYPE_EXTENSION	679
6.79.2.73 PB_FI_WIDTH_PB_LTYPE_FIXED32	679
6.79.2.74 PB_FI_WIDTH_PB_LTYPE_FIXED64	679
6.79.2.75 PB_FI_WIDTH_PB_LTYPE_FIXED_LENGTH_BYTES	679
6.79.2.76 PB_FI_WIDTH_PB_LTYPE_FLOAT	680
6.79.2.77 PB_FI_WIDTH_PB_LTYPE_INT32	680
6.79.2.78 PB_FI_WIDTH_PB_LTYPE_INT64	680
6.79.2.79 PB_FI_WIDTH_PB_LTYPE_MESSAGE	680
6.79.2.80 PB_FI_WIDTH_PB_LTYPE_MSG_W_CB	680
6.79.2.81 PB_FI_WIDTH_PB_LTYPE_SF32	680
6.79.2.82 PB_FI_WIDTH_PB_LTYPE_SF64	680
6.79.2.83 PB_FI_WIDTH_PB_LTYPE_SINT32	680
6.79.2.84 PB_FI_WIDTH_PB_LTYPE_SINT64	681
6.79.2.85 PB_FI_WIDTH_PB_LTYPE_STRING	681
6.79.2.86 PB_FI_WIDTH_PB_LTYPE_UENUM	681
6.79.2.87 PB_FI_WIDTH_PB_LTYPE_UINT32	681
6.79.2.88 PB_FI_WIDTH_PB_LTYPE_UINT64	681
6.79.2.89 PB_FIELDINFO_1	681
6.79.2.90 PB_FIELDINFO_2	682
6.79.2.91 PB_FIELDINFO_4	682
6.79.2.92 PB_FIELDINFO_8	682
6.79.2.93 PB_FIELDINFO_ASSERT_1	683
6.79.2.94 PB_FIELDINFO_ASSERT_2	683
6.79.2.95 PB_FIELDINFO_ASSERT_4	683

6.79.2.96 PB_FIELDINFO_ASSERT_8	684
6.79.2.97 PB_FIELDINFO_ASSERT_AUTO2	684
6.79.2.98 PB_FIELDINFO_ASSERT_AUTO3	684
6.79.2.99 PB_FIELDINFO_AUTO2	685
6.79.2.100 PB_FIELDINFO_AUTO3	685
6.79.2.101 PB_FIELDINFO_WIDTH_AUTO	685
6.79.2.102 PB_FITS	685
6.79.2.103 PB_GEN_FIELD_COUNT	686
6.79.2.104 PB_GEN_FIELD_INFO_1	686
6.79.2.105 PB_GEN_FIELD_INFO_2	686
6.79.2.106 PB_GEN_FIELD_INFO_4	687
6.79.2.107 PB_GEN_FIELD_INFO_8	687
6.79.2.108 PB_GEN_FIELD_INFO_ASSERT_1	687
6.79.2.109 PB_GEN_FIELD_INFO_ASSERT_2	688
6.79.2.110 PB_GEN_FIELD_INFO_ASSERT_4	688
6.79.2.111 PB_GEN_FIELD_INFO_ASSERT_8	689
6.79.2.112 PB_GEN_FIELD_INFO_ASSERT_AUTO	689
6.79.2.113 PB_GEN_FIELD_INFO_AUTO	689
6.79.2.114 PB_GEN_LARGEST_TAG	690
6.79.2.115 PB_GEN_REQ_FIELD_COUNT	690
6.79.2.116 PB_GEN_SUBMSG_INFO	691
6.79.2.117 PB_GET_ERROR	691
6.79.2.118 PB_HTYPE	691
6.79.2.119 PB_HTYPE_FIXARRAY	691
6.79.2.120 PB_HTYPE_MASK	691
6.79.2.121 PB_HTYPE_ONEOF	692
6.79.2.122 PB_HTYPE_OPTIONAL	692
6.79.2.123 PB_HTYPE_REPEATED	692
6.79.2.124 PB_HTYPE_REQUIRED	692
6.79.2.125 PB_HTYPE_SINGULAR	692
6.79.2.126 PB_LTYPE	692
6.79.2.127 PB_LTYPE_BOOL	693
6.79.2.128 PB_LTYPE_BYTES	693
6.79.2.129 PB_LTYPE_EXTENSION	693
6.79.2.130 PB_LTYPE_FIXED32	693
6.79.2.131 PB_LTYPE_FIXED64	693
6.79.2.132 PB_LTYPE_FIXED_LENGTH_BYTES	693
6.79.2.133 PB_LTYPE_IS_SUBMSG	694
6.79.2.134 PB_LTYPE_LAST_PACKABLE	694
6.79.2.135 PB_LTYPE_MAP_BOOL	694
6.79.2.136 PB_LTYPE_MAP_BYTES	694
6.79.2.137 PB_LTYPE_MAP_DOUBLE	694
6.79.2.138 PB_LTYPE_MAP_ENUM	694

6.79.2.139 PB_LTYPE_MAP_EXTENSION	695
6.79.2.140 PB_LTYPE_MAP_FIXED32	695
6.79.2.141 PB_LTYPE_MAP_FIXED64	695
6.79.2.142 PB_LTYPE_MAP_FIXED_LENGTH_BYTES	695
6.79.2.143 PB_LTYPE_MAP_FLOAT	695
6.79.2.144 PB_LTYPE_MAP_INT32	695
6.79.2.145 PB_LTYPE_MAP_INT64	695
6.79.2.146 PB_LTYPE_MAP_MESSAGE	695
6.79.2.147 PB_LTYPE_MAP_MSG_W_CB	696
6.79.2.148 PB_LTYPE_MAP_SFIXED32	696
6.79.2.149 PB_LTYPE_MAP_SFIXED64	696
6.79.2.150 PB_LTYPE_MAP_SINT32	696
6.79.2.151 PB_LTYPE_MAP_SINT64	696
6.79.2.152 PB_LTYPE_MAP_STRING	696
6.79.2.153 PB_LTYPE_MAP_UENUM	696
6.79.2.154 PB_LTYPE_MAP_UINT32	696
6.79.2.155 PB_LTYPE_MAP_UINT64	697
6.79.2.156 PB_LTYPE_MASK	697
6.79.2.157 PB_LTYPE_STRING	697
6.79.2.158 PB_LTYPE_SUBMESSAGE	697
6.79.2.159 PB_LTYPE_SUBMSG_W_CB	697
6.79.2.160 PB_LTYPE_SVARINT	697
6.79.2.161 PB_LTYPE_UVARINT	698
6.79.2.162 PB_LTYPE_VARINT	698
6.79.2.163 PB_LTYPES_COUNT	698
6.79.2.164 PB_MAX_REQUIRED_FIELDS	698
6.79.2.165 pb_membersize	698
6.79.2.166 pb_noinline	698
6.79.2.167 PB_ONEOF_NAME	699
6.79.2.168 PB_ONEOF_NAME_FULL	699
6.79.2.169 PB_ONEOF_NAME_MEMBER	699
6.79.2.170 PB_ONEOF_NAME_UNION	699
6.79.2.171 pb_packed	699
6.79.2.172 PB_PACKED_STRUCT_END	700
6.79.2.173 PB_PACKED_STRUCT_START	700
6.79.2.174 PB_PROGMEM	700
6.79.2.175 PB_PROGMEM_READU32	700
6.79.2.176 PB_PROTO_HEADER_VERSION	700
6.79.2.177 PB_RETURN_ERROR	700
6.79.2.178 PB_SET_ERROR	701
6.79.2.179 PB_SI_PB_LTYPE_BOOL	701
6.79.2.180 PB_SI_PB_LTYPE_BYTES	701
6.79.2.181 PB_SI_PB_LTYPE_DOUBLE	701

6.79.2.182 PB_SI_PB_LTYPE_ENUM	701
6.79.2.183 PB_SI_PB_LTYPE_EXTENSION	701
6.79.2.184 PB_SI_PB_LTYPE_FIXED32	702
6.79.2.185 PB_SI_PB_LTYPE_FIXED64	702
6.79.2.186 PB_SI_PB_LTYPE_FIXED_LENGTH_BYTES	702
6.79.2.187 PB_SI_PB_LTYPE_FLOAT	702
6.79.2.188 PB_SI_PB_LTYPE_INT32	702
6.79.2.189 PB_SI_PB_LTYPE_INT64	702
6.79.2.190 PB_SI_PB_LTYPE_MESSAGE	702
6.79.2.191 PB_SI_PB_LTYPE_MSG_W_CB	703
6.79.2.192 PB_SI_PB_LTYPE_SFIXED32	703
6.79.2.193 PB_SI_PB_LTYPE_SFIXED64	703
6.79.2.194 PB_SI_PB_LTYPE_SINT32	703
6.79.2.195 PB_SI_PB_LTYPE_SINT64	703
6.79.2.196 PB_SI_PB_LTYPE_STRING	703
6.79.2.197 PB_SI_PB_LTYPE_UENUM	704
6.79.2.198 PB_SI_PB_LTYPE_UINT32	704
6.79.2.199 PB_SI_PB_LTYPE_UINT64	704
6.79.2.200 PB_SIZE_MAX	704
6.79.2.201 PB_SIZE_OFFSET_CALLBACK	704
6.79.2.202 PB_SIZE_OFFSET_POINTER	704
6.79.2.203 PB_SIZE_OFFSET_STATIC	705
6.79.2.204 PB_SO_CB_PB_HTYPE_FIXARRAY	705
6.79.2.205 PB_SO_CB_PB_HTYPE_ONEOF	705
6.79.2.206 PB_SO_CB_PB_HTYPE_OPTIONAL	705
6.79.2.207 PB_SO_CB_PB_HTYPE_REPEATED	705
6.79.2.208 PB_SO_CB_PB_HTYPE_REQUIRED	706
6.79.2.209 PB_SO_CB_PB_HTYPE_SINGULAR	706
6.79.2.210 PB_SO_PB_HTYPE_FIXARRAY	706
6.79.2.211 PB_SO_PB_HTYPE_ONEOF	706
6.79.2.212 PB_SO_PB_HTYPE_ONEOF2	706
6.79.2.213 PB_SO_PB_HTYPE_ONEOF3	707
6.79.2.214 PB_SO_PB_HTYPE_OPTIONAL	707
6.79.2.215 PB_SO_PB_HTYPE_REPEATED	707
6.79.2.216 PB_SO_PB_HTYPE_REQUIRED	707
6.79.2.217 PB_SO_PB_HTYPE_SINGULAR	707
6.79.2.218 PB_SO_PTR_PB_HTYPE_FIXARRAY	708
6.79.2.219 PB_SO_PTR_PB_HTYPE_ONEOF	708
6.79.2.220 PB_SO_PTR_PB_HTYPE_OPTIONAL	708
6.79.2.221 PB_SO_PTR_PB_HTYPE_REPEATED	708
6.79.2.222 PB_SO_PTR_PB_HTYPE_REQUIRED	708
6.79.2.223 PB_SO_PTR_PB_HTYPE_SINGULAR	709
6.79.2.224 PB_STATIC_ASSERT	709

6.79.2.225	PB_SUBMSG_DESCRIPTOR	709
6.79.2.226	PB_SUBMSG_INFO_FIXARRAY	709
6.79.2.227	PB_SUBMSG_INFO_ONEOF	709
6.79.2.228	PB_SUBMSG_INFO_ONEOF2	710
6.79.2.229	PB_SUBMSG_INFO_ONEOF3	710
6.79.2.230	PB_SUBMSG_INFO_OPTIONAL	710
6.79.2.231	PB_SUBMSG_INFO_REPEATED	710
6.79.2.232	PB_SUBMSG_INFO_REQUIRED	711
6.79.2.233	PB_SUBMSG_INFO_SINGULAR	711
6.79.2.234	PB_UNUSED	711
6.79.3	Typedef Documentation	711
6.79.3.1	pb_byte_t	711
6.79.3.2	pb_field_t	711
6.79.3.3	pb_size_t	712
6.79.3.4	pb_ssize_t	712
6.79.3.5	pb_type_t	712
6.79.4	Enumeration Type Documentation	712
6.79.4.1	pb_wire_type_t	712
6.79.5	Function Documentation	713
6.79.5.1	pb_default_field_callback()	713
6.80	pb.h	713
6.81	libs/rx_nanopb/nanopb/spm_headers/nanopb/pb.h File Reference	725
6.81.1	Data Structure Documentation	730
6.81.1.1	struct pb_field_iter_s	730
6.81.1.2	struct pb_bytes_array_s	731
6.81.1.3	struct pb_extension_s	731
6.81.2	Macro Definition Documentation	732
6.81.2.1	NANOPB_VERSION	732
6.81.2.2	PB_ARRAY_SIZE_CALLBACK	732
6.81.2.3	PB_ARRAY_SIZE_POINTER	732
6.81.2.4	PB_ARRAY_SIZE_STATIC	733
6.81.2.5	pb_arraysize	733
6.81.2.6	PB_AS_PB_HTYPE_FIXARRAY	733
6.81.2.7	PB_AS_PB_HTYPE_ONEOF	733
6.81.2.8	PB_AS_PB_HTYPE_OPTIONAL	733
6.81.2.9	PB_AS_PB_HTYPE_REPEATED	734
6.81.2.10	PB_AS_PB_HTYPE_REQUIRED	734
6.81.2.11	PB_AS_PB_HTYPE_SINGULAR	734
6.81.2.12	PB_AS_PTR_PB_HTYPE_FIXARRAY	734
6.81.2.13	PB_AS_PTR_PB_HTYPE_ONEOF	734
6.81.2.14	PB_AS_PTR_PB_HTYPE_OPTIONAL	735
6.81.2.15	PB_AS_PTR_PB_HTYPE_REPEATED	735
6.81.2.16	PB_AS_PTR_PB_HTYPE_REQUIRED	735

6.81.2.17 PB_AS_PTR_PB_HTYPE_SINGULAR	735
6.81.2.18 PB_ATYPE	735
6.81.2.19 PB_ATYPE_CALLBACK	736
6.81.2.20 PB_ATYPE_MASK	736
6.81.2.21 PB_ATYPE_POINTER	736
6.81.2.22 PB_ATYPE_STATIC	736
6.81.2.23 PB_BIND	736
6.81.2.24 PB_BYTES_ARRAY_T	737
6.81.2.25 PB_BYTES_ARRAY_T_ALLOCSIZE	737
6.81.2.26 PB_DATA_OFFSET_CALLBACK	737
6.81.2.27 PB_DATA_OFFSET_POINTER	737
6.81.2.28 PB_DATA_OFFSET_STATIC	738
6.81.2.29 PB_DATA_SIZE_CALLBACK	738
6.81.2.30 PB_DATA_SIZE_POINTER	738
6.81.2.31 PB_DATA_SIZE_STATIC	738
6.81.2.32 pb_delta	739
6.81.2.33 PB_DO_PB_HTYPE_FIXARRAY	739
6.81.2.34 PB_DO_PB_HTYPE_ONEOF	739
6.81.2.35 PB_DO_PB_HTYPE_OPTIONAL	739
6.81.2.36 PB_DO_PB_HTYPE_REPEATED	739
6.81.2.37 PB_DO_PB_HTYPE_REQUIRED	740
6.81.2.38 PB_DO_PB_HTYPE_SINGULAR	740
6.81.2.39 PB_DS_CB_PB_HTYPE_FIXARRAY	740
6.81.2.40 PB_DS_CB_PB_HTYPE_ONEOF	740
6.81.2.41 PB_DS_CB_PB_HTYPE_OPTIONAL	740
6.81.2.42 PB_DS_CB_PB_HTYPE_REPEATED	741
6.81.2.43 PB_DS_CB_PB_HTYPE_REQUIRED	741
6.81.2.44 PB_DS_CB_PB_HTYPE_SINGULAR	741
6.81.2.45 PB_DS_PB_HTYPE_FIXARRAY	741
6.81.2.46 PB_DS_PB_HTYPE_ONEOF	741
6.81.2.47 PB_DS_PB_HTYPE_OPTIONAL	742
6.81.2.48 PB_DS_PB_HTYPE_REPEATED	742
6.81.2.49 PB_DS_PB_HTYPE_REQUIRED	742
6.81.2.50 PB_DS_PB_HTYPE_SINGULAR	742
6.81.2.51 PB_DS_PTR_PB_HTYPE_FIXARRAY	742
6.81.2.52 PB_DS_PTR_PB_HTYPE_ONEOF	743
6.81.2.53 PB_DS_PTR_PB_HTYPE_OPTIONAL	743
6.81.2.54 PB_DS_PTR_PB_HTYPE_REPEATED	743
6.81.2.55 PB_DS_PTR_PB_HTYPE_REQUIRED	743
6.81.2.56 PB_DS_PTR_PB_HTYPE_SINGULAR	743
6.81.2.57 PB_EXPAND	744
6.81.2.58 pb_extension_init_zero	744
6.81.2.59 PB_FI_WIDTH_PB_ATYPE_CALLBACK	744

6.81.2.60 PB_FI_WIDTH_PB_ATYPE_POINTER	744
6.81.2.61 PB_FI_WIDTH_PB_ATYPE_STATIC	744
6.81.2.62 PB_FI_WIDTH_PB_HTYPE_FIXARRAY	745
6.81.2.63 PB_FI_WIDTH_PB_HTYPE_ONEOF	745
6.81.2.64 PB_FI_WIDTH_PB_HTYPE_OPTIONAL	745
6.81.2.65 PB_FI_WIDTH_PB_HTYPE_REPEATED	745
6.81.2.66 PB_FI_WIDTH_PB_HTYPE_REQUIRED	745
6.81.2.67 PB_FI_WIDTH_PB_HTYPE_SINGULAR	746
6.81.2.68 PB_FI_WIDTH_PB_LTYPE_BOOL	746
6.81.2.69 PB_FI_WIDTH_PB_LTYPE_BYTES	746
6.81.2.70 PB_FI_WIDTH_PB_LTYPE_DOUBLE	746
6.81.2.71 PB_FI_WIDTH_PB_LTYPE_ENUM	746
6.81.2.72 PB_FI_WIDTH_PB_LTYPE_EXTENSION	746
6.81.2.73 PB_FI_WIDTH_PB_LTYPE_FIXED32	746
6.81.2.74 PB_FI_WIDTH_PB_LTYPE_FIXED64	747
6.81.2.75 PB_FI_WIDTH_PB_LTYPE_FIXED_LENGTH_BYTES	747
6.81.2.76 PB_FI_WIDTH_PB_LTYPE_FLOAT	747
6.81.2.77 PB_FI_WIDTH_PB_LTYPE_INT32	747
6.81.2.78 PB_FI_WIDTH_PB_LTYPE_INT64	747
6.81.2.79 PB_FI_WIDTH_PB_LTYPE_MESSAGE	747
6.81.2.80 PB_FI_WIDTH_PB_LTYPE_MSG_W_CB	747
6.81.2.81 PB_FI_WIDTH_PB_LTYPE_SFIXED32	747
6.81.2.82 PB_FI_WIDTH_PB_LTYPE_SFIXED64	748
6.81.2.83 PB_FI_WIDTH_PB_LTYPE_SINT32	748
6.81.2.84 PB_FI_WIDTH_PB_LTYPE_SINT64	748
6.81.2.85 PB_FI_WIDTH_PB_LTYPE_STRING	748
6.81.2.86 PB_FI_WIDTH_PB_LTYPE_UENUM	748
6.81.2.87 PB_FI_WIDTH_PB_LTYPE_UINT32	748
6.81.2.88 PB_FI_WIDTH_PB_LTYPE_UINT64	748
6.81.2.89 PB_FIELDINFO_1	749
6.81.2.90 PB_FIELDINFO_2	749
6.81.2.91 PB_FIELDINFO_4	749
6.81.2.92 PB_FIELDINFO_8	750
6.81.2.93 PB_FIELDINFO_ASSERT_1	750
6.81.2.94 PB_FIELDINFO_ASSERT_2	750
6.81.2.95 PB_FIELDINFO_ASSERT_4	751
6.81.2.96 PB_FIELDINFO_ASSERT_8	751
6.81.2.97 PB_FIELDINFO_ASSERT_AUTO2	751
6.81.2.98 PB_FIELDINFO_ASSERT_AUTO3	752
6.81.2.99 PB_FIELDINFO_AUTO2	752
6.81.2.100 PB_FIELDINFO_AUTO3	752
6.81.2.101 PB_FIELDINFO_WIDTH_AUTO	753
6.81.2.102 PB_FITS	753

6.81.2.103 PB_GEN_FIELD_COUNT	753
6.81.2.104 PB_GEN_FIELD_INFO_1	753
6.81.2.105 PB_GEN_FIELD_INFO_2	754
6.81.2.106 PB_GEN_FIELD_INFO_4	754
6.81.2.107 PB_GEN_FIELD_INFO_8	754
6.81.2.108 PB_GEN_FIELD_INFO_ASSERT_1	755
6.81.2.109 PB_GEN_FIELD_INFO_ASSERT_2	755
6.81.2.110 PB_GEN_FIELD_INFO_ASSERT_4	756
6.81.2.111 PB_GEN_FIELD_INFO_ASSERT_8	756
6.81.2.112 PB_GEN_FIELD_INFO_ASSERT_AUTO	756
6.81.2.113 PB_GEN_FIELD_INFO_AUTO	757
6.81.2.114 PB_GEN_LARGEST_TAG	757
6.81.2.115 PB_GEN_REQ_FIELD_COUNT	758
6.81.2.116 PB_GEN_SUBMSG_INFO	758
6.81.2.117 PB_GET_ERROR	758
6.81.2.118 PB_HTYPE	758
6.81.2.119 PB_HTYPE_FIXARRAY	759
6.81.2.120 PB_HTYPE_MASK	759
6.81.2.121 PB_HTYPE_ONEOF	759
6.81.2.122 PB_HTYPE_OPTIONAL	759
6.81.2.123 PB_HTYPE_REPEATED	759
6.81.2.124 PB_HTYPE_REQUIRED	759
6.81.2.125 PB_HTYPE_SINGULAR	759
6.81.2.126 PB_LTYPE	760
6.81.2.127 PB_LTYPE_BOOL	760
6.81.2.128 PB_LTYPE_BYTES	760
6.81.2.129 PB_LTYPE_EXTENSION	760
6.81.2.130 PB_LTYPE_FIXED32	760
6.81.2.131 PB_LTYPE_FIXED64	760
6.81.2.132 PB_LTYPE_FIXED_LENGTH_BYTES	760
6.81.2.133 PB_LTYPE_IS_SUBMSG	761
6.81.2.134 PB_LTYPE_LAST_PACKABLE	761
6.81.2.135 PB_LTYPE_MAP_BOOL	761
6.81.2.136 PB_LTYPE_MAP_BYTES	761
6.81.2.137 PB_LTYPE_MAP_DOUBLE	761
6.81.2.138 PB_LTYPE_MAP_ENUM	761
6.81.2.139 PB_LTYPE_MAP_EXTENSION	761
6.81.2.140 PB_LTYPE_MAP_FIXED32	762
6.81.2.141 PB_LTYPE_MAP_FIXED64	762
6.81.2.142 PB_LTYPE_MAP_FIXED_LENGTH_BYTES	762
6.81.2.143 PB_LTYPE_MAP_FLOAT	762
6.81.2.144 PB_LTYPE_MAP_INT32	762
6.81.2.145 PB_LTYPE_MAP_INT64	762

6.81.2.146 PB_LTYPE_MAP_MESSAGE	762
6.81.2.147 PB_LTYPE_MAP_MSG_W_CB	762
6.81.2.148 PB_LTYPE_MAP_SFIXED32	763
6.81.2.149 PB_LTYPE_MAP_SFIXED64	763
6.81.2.150 PB_LTYPE_MAP_SINT32	763
6.81.2.151 PB_LTYPE_MAP_SINT64	763
6.81.2.152 PB_LTYPE_MAP_STRING	763
6.81.2.153 PB_LTYPE_MAP_UENUM	763
6.81.2.154 PB_LTYPE_MAP_UINT32	763
6.81.2.155 PB_LTYPE_MAP_UINT64	763
6.81.2.156 PB_LTYPE_MASK	764
6.81.2.157 PB_LTYPE_STRING	764
6.81.2.158 PB_LTYPE_SUBMESSAGE	764
6.81.2.159 PB_LTYPE_SUBMSG_W_CB	764
6.81.2.160 PB_LTYPE_SVARINT	764
6.81.2.161 PB_LTYPE_UVARINT	764
6.81.2.162 PB_LTYPE_VARINT	764
6.81.2.163 PB_LTYPES_COUNT	764
6.81.2.164 PB_MAX_REQUIRED_FIELDS	765
6.81.2.165 pb_membersize	765
6.81.2.166 pb_noinline	765
6.81.2.167 PB_ONEOF_NAME	765
6.81.2.168 PB_ONEOF_NAME_FULL	765
6.81.2.169 PB_ONEOF_NAME_MEMBER	766
6.81.2.170 PB_ONEOF_NAME_UNION	766
6.81.2.171 pb_packed	766
6.81.2.172 PB_PACKED_STRUCT_END	766
6.81.2.173 PB_PACKED_STRUCT_START	766
6.81.2.174 PB_PROGMEM	766
6.81.2.175 PB_PROGMEM_READU32	767
6.81.2.176 PB_PROTO_HEADER_VERSION	767
6.81.2.177 PB_RETURN_ERROR	767
6.81.2.178 PB_SET_ERROR	767
6.81.2.179 PB_SI_PB_LTYPE_BOOL	767
6.81.2.180 PB_SI_PB_LTYPE_BYTES	767
6.81.2.181 PB_SI_PB_LTYPE_DOUBLE	768
6.81.2.182 PB_SI_PB_LTYPE_ENUM	768
6.81.2.183 PB_SI_PB_LTYPE_EXTENSION	768
6.81.2.184 PB_SI_PB_LTYPE_FIXED32	768
6.81.2.185 PB_SI_PB_LTYPE_FIXED64	768
6.81.2.186 PB_SI_PB_LTYPE_FIXED_LENGTH_BYTES	768
6.81.2.187 PB_SI_PB_LTYPE_FLOAT	768
6.81.2.188 PB_SI_PB_LTYPE_INT32	769

6.81.2.189 PB_SI_PB_LTYPE_INT64	769
6.81.2.190 PB_SI_PB_LTYPE_MESSAGE	769
6.81.2.191 PB_SI_PB_LTYPE_MSG_W_CB	769
6.81.2.192 PB_SI_PB_LTYPE_SF_FIXED32	769
6.81.2.193 PB_SI_PB_LTYPE_SF_FIXED64	769
6.81.2.194 PB_SI_PB_LTYPE_SINT32	770
6.81.2.195 PB_SI_PB_LTYPE_SINT64	770
6.81.2.196 PB_SI_PB_LTYPE_STRING	770
6.81.2.197 PB_SI_PB_LTYPE_UENUM	770
6.81.2.198 PB_SI_PB_LTYPE_UINT32	770
6.81.2.199 PB_SI_PB_LTYPE_UINT64	770
6.81.2.200 PB_SIZE_MAX	770
6.81.2.201 PB_SIZE_OFFSET_CALLBACK	771
6.81.2.202 PB_SIZE_OFFSET_POINTER	771
6.81.2.203 PB_SIZE_OFFSET_STATIC	771
6.81.2.204 PB_SO_CB_PB_HTYPE_FIXARRAY	771
6.81.2.205 PB_SO_CB_PB_HTYPE_ONEOF	772
6.81.2.206 PB_SO_CB_PB_HTYPE_OPTIONAL	772
6.81.2.207 PB_SO_CB_PB_HTYPE_REPEATED	772
6.81.2.208 PB_SO_CB_PB_HTYPE_REQUIRED	772
6.81.2.209 PB_SO_CB_PB_HTYPE_SINGULAR	772
6.81.2.210 PB_SO_PB_HTYPE_FIXARRAY	773
6.81.2.211 PB_SO_PB_HTYPE_ONEOF	773
6.81.2.212 PB_SO_PB_HTYPE_ONEOF2	773
6.81.2.213 PB_SO_PB_HTYPE_ONEOF3	773
6.81.2.214 PB_SO_PB_HTYPE_OPTIONAL	773
6.81.2.215 PB_SO_PB_HTYPE_REPEATED	774
6.81.2.216 PB_SO_PB_HTYPE_REQUIRED	774
6.81.2.217 PB_SO_PB_HTYPE_SINGULAR	774
6.81.2.218 PB_SO_PTR_PB_HTYPE_FIXARRAY	774
6.81.2.219 PB_SO_PTR_PB_HTYPE_ONEOF	774
6.81.2.220 PB_SO_PTR_PB_HTYPE_OPTIONAL	775
6.81.2.221 PB_SO_PTR_PB_HTYPE_REPEATED	775
6.81.2.222 PB_SO_PTR_PB_HTYPE_REQUIRED	775
6.81.2.223 PB_SO_PTR_PB_HTYPE_SINGULAR	775
6.81.2.224 PB_STATIC_ASSERT	775
6.81.2.225 PB_SUBMSG_DESCRIPTOR	776
6.81.2.226 PB_SUBMSG_INFO_FIXARRAY	776
6.81.2.227 PB_SUBMSG_INFO_ONEOF	776
6.81.2.228 PB_SUBMSG_INFO_ONEOF2	776
6.81.2.229 PB_SUBMSG_INFO_ONEOF3	777
6.81.2.230 PB_SUBMSG_INFO_OPTIONAL	777
6.81.2.231 PB_SUBMSG_INFO_REPEATED	777

6.81.2.232 PB_SUBMSG_INFO_REQUIRED	777
6.81.2.233 PB_SUBMSG_INFO_SINGULAR	778
6.81.2.234 PB_UNUSED	778
6.81.3 Enumeration Type Documentation	778
6.81.3.1 pb_wire_type_t	778
6.81.4 Function Documentation	779
6.81.4.1 pb_default_field_callback()	779
6.82 pb.h	779
6.83 libs/rx_nanopb/nanopb/spm_headers/pb.h File Reference	790
6.84 pb.h	791
6.85 libs/rx_nanopb/src/rx_nanopb.c File Reference	791
6.85.1 Detailed Description	793
6.85.2 Function Documentation	796
6.85.2.1 rx_nanopb_create_response_header()	796
6.85.2.2 rx_nanopb_create_velocity_command()	798
6.85.2.3 rx_nanopb_create_velocity_command_diff_drive()	800
6.85.2.4 rx_nanopb_decode_estop_request()	801
6.85.2.5 rx_nanopb_decode_pid_gains_request()	804
6.85.2.6 rx_nanopb_decode_retransmit_config_request()	807
6.85.2.7 rx_nanopb_decode_velocity_request()	809
6.85.2.8 rx_nanopb_encode_estop_response()	811
6.85.2.9 rx_nanopb_encode_pid_gains_response()	813
6.85.2.10 rx_nanopb_encode_telemetry()	814
6.85.2.11 rx_nanopb_encode_velocity_request()	817
6.85.2.12 rx_nanopb_encode_velocity_response()	819
6.85.2.13 rx_nanopb_init()	821
6.85.2.14 rx_nanopb_test_reset_state()	824
6.85.3 Variable Documentation	826
6.85.3.1 k_velocity_mps_max	826
6.85.3.2 k_velocity_mps_min	827
6.85.3.3 s_initialized	827
6.86 rx_nanopb.c	828

Chapter 1

Directory Hierarchy

1.1 Directories

extra	11
pb_syshdr.h	458
gen	11
google	12
protobuf	15
duration.pb.c	29
duration.pb.h	30
timestamp.pb.c	34
timestamp.pb.h	35
star	17
v1	18
common.pb.c	40
common.pb.h	41
configuration.pb.c	62
configuration.pb.h	64
controller.pb.c	144
controller.pb.h	144
diagnostics.pb.c	149
diagnostics.pb.h	150
firmware_update.pb.c	162
firmware_update.pb.h	164
gateway_service.pb.c	239
gateway_service.pb.h	240
motor_control.pb.c	262
motor_control.pb.h	263
telemetry.pb.c	302
telemetry.pb.h	303
ui.pb.c	353
ui.pb.h	354
wire.pb.c	380
wire.pb.h	381
google	12
protobuf	15
duration.pb.c	29
duration.pb.h	30
timestamp.pb.c	34
timestamp.pb.h	35

inc	12
gen	11
google	12
protobuf	15
duration.pb.c	29
duration.pb.h	30
timestamp.pb.c	34
timestamp.pb.h	35
star	17
v1	18
common.pb.c	40
common.pb.h	41
configuration.pb.c	62
configuration.pb.h	64
controller.pb.c	144
controller.pb.h	144
diagnostics.pb.c	149
diagnostics.pb.h	150
firmware_update.pb.c	162
firmware_update.pb.h	164
gateway_service.pb.c	239
gateway_service.pb.h	240
motor_control.pb.c	262
motor_control.pb.h	263
telemetry.pb.c	302
telemetry.pb.h	303
ui.pb.c	353
ui.pb.h	354
wire.pb.c	380
wire.pb.h	381
rx_nanopb.h	391
libs	13
rx_nanopb	15
inc	12
gen	11
google	12
protobuf	15
duration.pb.c	29
duration.pb.h	30
timestamp.pb.c	34
timestamp.pb.h	35
star	17
v1	18
common.pb.c	40
common.pb.h	41
configuration.pb.c	62
configuration.pb.h	64
controller.pb.c	144
controller.pb.h	144
diagnostics.pb.c	149
diagnostics.pb.h	150
firmware_update.pb.c	162
firmware_update.pb.h	164
gateway_service.pb.c	239
gateway_service.pb.h	240
motor_control.pb.c	262
motor_control.pb.h	263

telemetry.pb.c	302
telemetry.pb.h	303
ui.pb.c	353
ui.pb.h	354
wire.pb.c	380
wire.pb.h	381
rx_nanopb.h	391
nanopb	13
extra	11
pb_syshdr.h	458
spm-test	16
objc	14
c-header.c	599
spm_headers	16
nanopb	14
pb.h	725
pb_common.h	484
pb_decode.h	611
pb_encode.h	641
pb.h	790
pb_common.h	492
pb_decode.h	630
pb_encode.h	656
pb.h	657
pb_common.c	464
pb_common.h	480
pb_decode.c	493
pb_decode.h	600
pb_encode.c	557
pb_encode.h	631
src	17
rx_nanopb.c	791
nanopb	13
extra	11
pb_syshdr.h	458
spm-test	16
objc	14
c-header.c	599
spm_headers	16
nanopb	14
pb.h	725
pb_common.h	484
pb_decode.h	611
pb_encode.h	641
pb.h	790
pb_common.h	492
pb_decode.h	630
pb_encode.h	656
pb.h	657
pb_common.c	464
pb_common.h	480
pb_decode.c	493
pb_decode.h	600
pb_encode.c	557
pb_encode.h	631
nanopb	14

pb.h	725
pb_common.h	484
pb_decode.h	611
pb_encode.h	641
objc	14
c-header.c	599
protobuf	15
duration.pb.c	29
duration.pb.h	30
timestamp.pb.c	34
timestamp.pb.h	35
rx_nanopb	15
inc	12
gen	11
google	12
protobuf	15
duration.pb.c	29
duration.pb.h	30
timestamp.pb.c	34
timestamp.pb.h	35
star	17
v1	18
common.pb.c	40
common.pb.h	41
configuration.pb.c	62
configuration.pb.h	64
controller.pb.c	144
controller.pb.h	144
diagnostics.pb.c	149
diagnostics.pb.h	150
firmware_update.pb.c	162
firmware_update.pb.h	164
gateway_service.pb.c	239
gateway_service.pb.h	240
motor_control.pb.c	262
motor_control.pb.h	263
telemetry.pb.c	302
telemetry.pb.h	303
ui.pb.c	353
ui.pb.h	354
wire.pb.c	380
wire.pb.h	381
rx_nanopb.h	391
nanopb	13
extra	11
pb_syshdr.h	458
spm-test	16
objc	14
c-header.c	599
spm_headers	16
nanopb	14
pb.h	725
pb_common.h	484
pb_decode.h	611
pb_encode.h	641
pb.h	790

pb_common.h	492
pb_decode.h	630
pb_encode.h	656
pb.h	657
pb_common.c	464
pb_common.h	480
pb_decode.c	493
pb_decode.h	600
pb_encode.c	557
pb_encode.h	631
src	17
rx_nanopb.c	791
spm-test	16
objc	14
c-header.c	599
spm_headers	16
nanopb	14
pb.h	725
pb_common.h	484
pb_decode.h	611
pb_encode.h	641
pb.h	790
pb_common.h	492
pb_decode.h	630
pb_encode.h	656
src	17
rx_nanopb.c	791
star	17
v1	18
common.pb.c	40
common.pb.h	41
configuration.pb.c	62
configuration.pb.h	64
controller.pb.c	144
controller.pb.h	144
diagnostics.pb.c	149
diagnostics.pb.h	150
firmware_update.pb.c	162
firmware_update.pb.h	164
gateway_service.pb.c	239
gateway_service.pb.h	240
motor_control.pb.c	262
motor_control.pb.h	263
telemetry.pb.c	302
telemetry.pb.h	303
ui.pb.c	353
ui.pb.h	354
wire.pb.c	380
wire.pb.h	381
v1	18
common.pb.c	40
common.pb.h	41
configuration.pb.c	62
configuration.pb.h	64
controller.pb.c	144
controller.pb.h	144

diagnostics.pb.c	149
diagnostics.pb.h	150
firmware_update.pb.c	162
firmware_update.pb.h	164
gateway_service.pb.c	239
gateway_service.pb.h	240
motor_control.pb.c	262
motor_control.pb.h	263
telemetry.pb.c	302
telemetry.pb.h	303
ui.pb.c	353
ui.pb.h	354
wire.pb.c	380
wire.pb.h	381

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

pb_callback_s.funcs	19
pb_callback_t	20
pb_extension_type_t	21
pb_istream_t	22
pb_msgdesc_t	23
pb_ostream_t	25
star_v1_STAREnvelope	26
star_v1_WireMessage	28

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

libs/rx_nanopb/inc/rx_nanopb.h	
Nanopb Integration Wrapper for RX72N	391
libs/rx_nanopb/inc/gen/google/protobuf/duration.pb.c	29
libs/rx_nanopb/inc/gen/google/protobuf/duration.pb.h	30
libs/rx_nanopb/inc/gen/google/protobuf/timestamp.pb.c	34
libs/rx_nanopb/inc/gen/google/protobuf/timestamp.pb.h	35
libs/rx_nanopb/inc/gen/star/v1/common.pb.c	40
libs/rx_nanopb/inc/gen/star/v1/common.pb.h	41
libs/rx_nanopb/inc/gen/star/v1/configuration.pb.c	62
libs/rx_nanopb/inc/gen/star/v1/configuration.pb.h	64
libs/rx_nanopb/inc/gen/star/v1/controller.pb.c	144
libs/rx_nanopb/inc/gen/star/v1/controller.pb.h	144
libs/rx_nanopb/inc/gen/star/v1/diagnostics.pb.c	149
libs/rx_nanopb/inc/gen/star/v1/diagnostics.pb.h	150
libs/rx_nanopb/inc/gen/star/v1/firmware_update.pb.c	162
libs/rx_nanopb/inc/gen/star/v1/firmware_update.pb.h	164
libs/rx_nanopb/inc/gen/star/v1/gateway_service.pb.c	239
libs/rx_nanopb/inc/gen/star/v1/gateway_service.pb.h	240
libs/rx_nanopb/inc/gen/star/v1/motor_control.pb.c	262
libs/rx_nanopb/inc/gen/star/v1/motor_control.pb.h	263
libs/rx_nanopb/inc/gen/star/v1/telemetry.pb.c	302
libs/rx_nanopb/inc/gen/star/v1/telemetry.pb.h	303
libs/rx_nanopb/inc/gen/star/v1/ui.pb.c	353
libs/rx_nanopb/inc/gen/star/v1/ui.pb.h	354
libs/rx_nanopb/inc/gen/star/v1/wire.pb.c	380
libs/rx_nanopb/inc/gen/star/v1/wire.pb.h	381
libs/rx_nanopb/nanopb/pb.h	657
libs/rx_nanopb/nanopb/pb_common.c	464
libs/rx_nanopb/nanopb/pb_common.h	480
libs/rx_nanopb/nanopb/pb_decode.c	493
libs/rx_nanopb/nanopb/pb_decode.h	600
libs/rx_nanopb/nanopb/pb_encode.c	557
libs/rx_nanopb/nanopb/pb_encode.h	631
libs/rx_nanopb/nanopb/extra/pb_syshdr.h	458
libs/rx_nanopb/nanopb/spm-test/objc/c-header.c	599

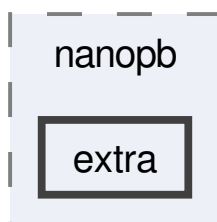
libs/rx_nanopb/nanopb/spm_headers/ pb.h	790
libs/rx_nanopb/nanopb/spm_headers/ pb_common.h	492
libs/rx_nanopb/nanopb/spm_headers/ pb_decode.h	630
libs/rx_nanopb/nanopb/spm_headers/ pb_encode.h	656
libs/rx_nanopb/nanopb/spm_headers/nanopb/ pb.h	725
libs/rx_nanopb/nanopb/spm_headers/nanopb/ pb_common.h	484
libs/rx_nanopb/nanopb/spm_headers/nanopb/ pb_decode.h	611
libs/rx_nanopb/nanopb/spm_headers/nanopb/ pb_encode.h	641
libs/rx_nanopb/src/ rx_nanopb.c	
Nanopb Integration Wrapper Implementation for RX72N	791

Chapter 4

Directory Documentation

4.1 libs/rx_nanopb/nanopb/extra Directory Reference

Directory dependency graph for extra:

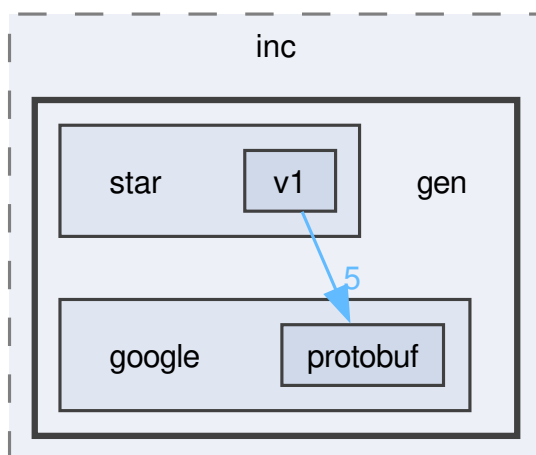


Files

- file [pb_syshdr.h](#)

4.2 libs/rx_nanopb/inc/gen Directory Reference

Directory dependency graph for gen:

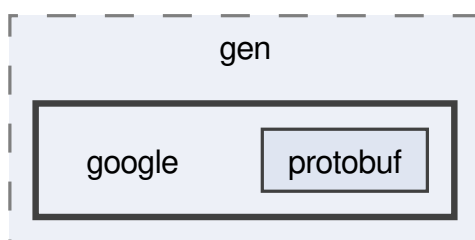


Directories

- directory [google](#)
- directory [star](#)

4.3 libs/rx_nanopb/inc/gen/google Directory Reference

Directory dependency graph for google:

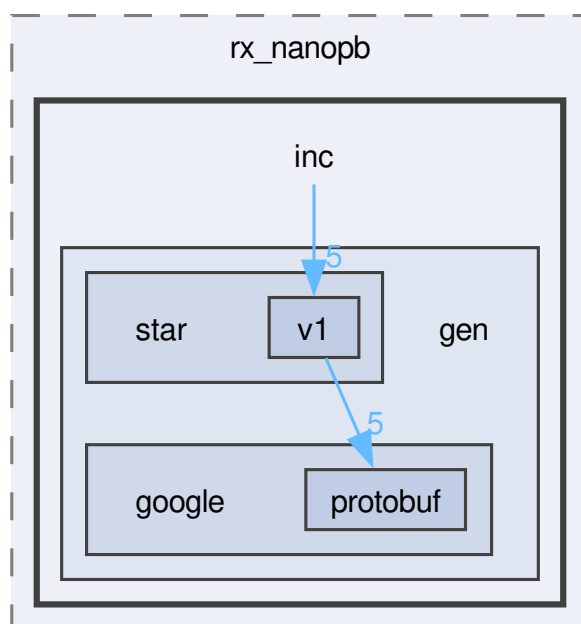


Directories

- directory [protobuf](#)

4.4 libs/rx_nanopb/inc Directory Reference

Directory dependency graph for inc:



Directories

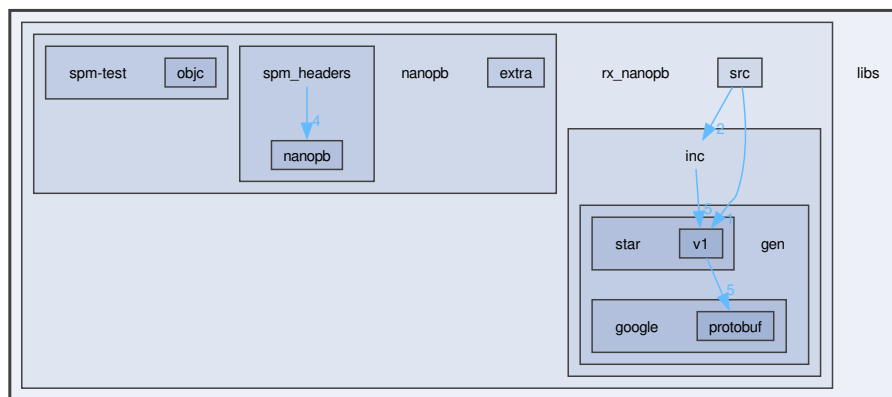
- directory [gen](#)

Files

- file [rx_nanopb.h](#)
nanopb Integration Wrapper for RX72N

4.5 libs Directory Reference

Directory dependency graph for libs:

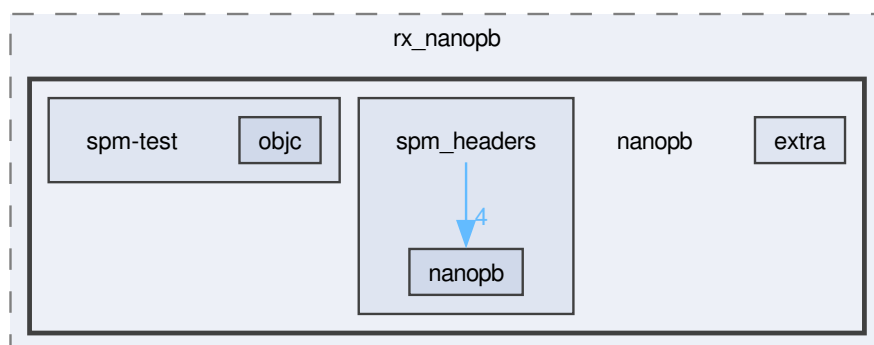


Directories

- directory [rx_nanopb](#)

4.6 libs/rx_nanopb/nanopb Directory Reference

Directory dependency graph for nanopb:



Directories

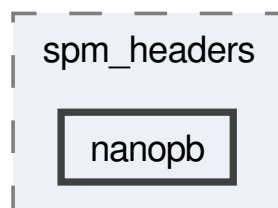
- directory [extra](#)
- directory [spm-test](#)
- directory [spm_headers](#)

Files

- file [pb.h](#)
- file [pb_common.c](#)
- file [pb_common.h](#)
- file [pb_decode.c](#)
- file [pb_decode.h](#)
- file [pb_encode.c](#)
- file [pb_encode.h](#)

4.7 libs/rx_nanopb/nanopb/spm_headers/nanopb Directory Reference

Directory dependency graph for nanopb:

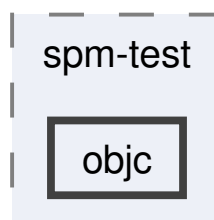


Files

- file [pb.h](#)
- file [pb_common.h](#)
- file [pb_decode.h](#)
- file [pb_encode.h](#)

4.8 libs/rx_nanopb/nanopb/spm-test/objc Directory Reference

Directory dependency graph for objc:

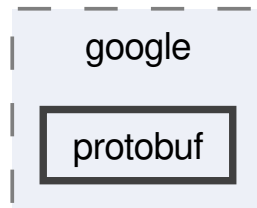


Files

- file [c-header.c](#)

4.9 libs/rx_nanopb/inc/gen/google/protobuf Directory Reference

Directory dependency graph for protobuf:

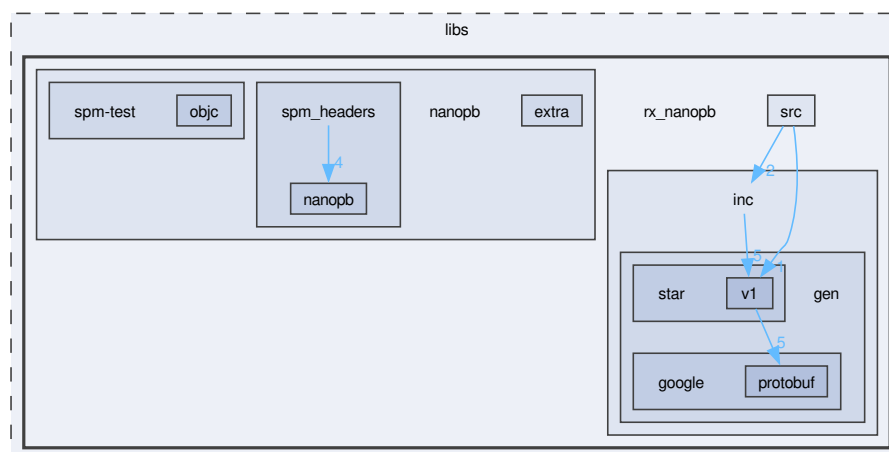


Files

- file [duration.pb.c](#)
- file [duration.pb.h](#)
- file [timestamp.pb.c](#)
- file [timestamp.pb.h](#)

4.10 libs/rx_nanopb Directory Reference

Directory dependency graph for rx_nanopb:

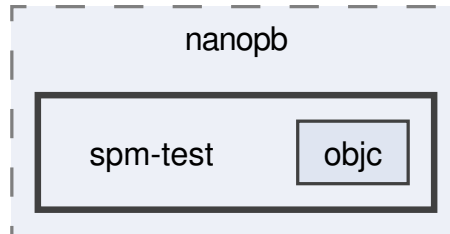


Directories

- directory [inc](#)
- directory [nanopb](#)
- directory [src](#)

4.11 libs/rx_nanopb/nanopb/spm-test Directory Reference

Directory dependency graph for spm-test:

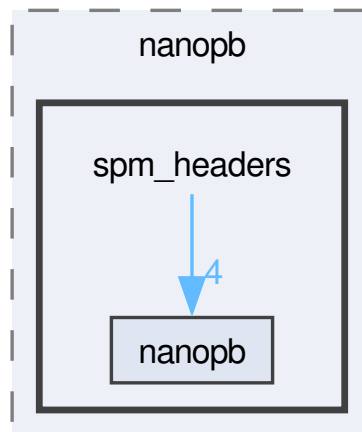


Directories

- directory [objc](#)

4.12 libs/rx_nanopb/nanopb/spm_headers Directory Reference

Directory dependency graph for spm_headers:



Directories

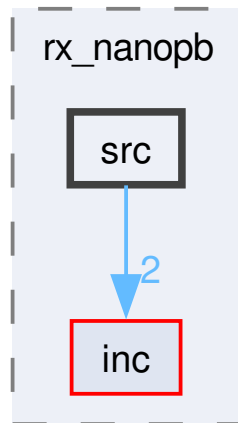
- directory [nanopb](#)

Files

- file [pb.h](#)
- file [pb_common.h](#)
- file [pb_decode.h](#)
- file [pb_encode.h](#)

4.13 libs/rx_nanopb/src Directory Reference

Directory dependency graph for src:

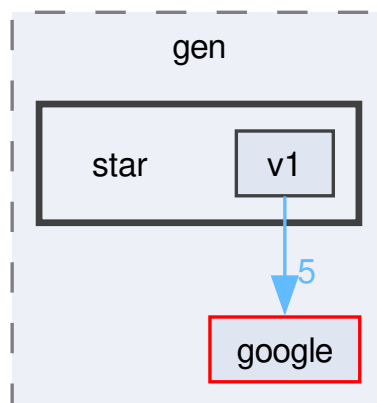


Files

- file [rx_nanopb.c](#)
nanopb Integration Wrapper Implementation for RX72N

4.14 libs/rx_nanopb/inc/gen/star Directory Reference

Directory dependency graph for star:

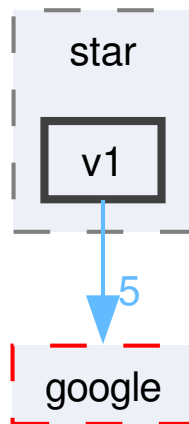


Directories

- directory [v1](#)

4.15 libs/rx_nanopb/inc/gen/star/v1 Directory Reference

Directory dependency graph for v1:



Files

- file [common.pb.c](#)
- file [common.pb.h](#)
- file [configuration.pb.c](#)
- file [configuration.pb.h](#)
- file [controller.pb.c](#)
- file [controller.pb.h](#)
- file [diagnostics.pb.c](#)
- file [diagnostics.pb.h](#)
- file [firmware_update.pb.c](#)
- file [firmware_update.pb.h](#)
- file [gateway_service.pb.c](#)
- file [gateway_service.pb.h](#)
- file [motor_control.pb.c](#)
- file [motor_control.pb.h](#)
- file [telemetry.pb.c](#)
- file [telemetry.pb.h](#)
- file [ui.pb.c](#)
- file [ui.pb.h](#)
- file [wire.pb.c](#)
- file [wire.pb.h](#)

Chapter 5

Data Structure Documentation

5.1 pb_callback_s.funcs Union Reference

Data Fields

- [bool\(* decode\)](#)(pb_istream_t *stream, const [pb_field_t](#) *field, void **arg)
- [bool\(* encode\)](#)(pb_ostream_t *stream, const [pb_field_t](#) *field, void *const *arg)
- [bool\(* decode\)](#)(pb_istream_t *stream, const [pb_field_t](#) *field, void **arg)
- [bool\(* encode\)](#)(pb_ostream_t *stream, const [pb_field_t](#) *field, void *const *arg)

5.1.1 Detailed Description

Definition at line [434](#) of file [pb.h](#).

5.1.2 Field Documentation

5.1.2.1 decode [1/2]

5.1.2.2 decode [2/2]

5.1.2.3 encode [1/2]

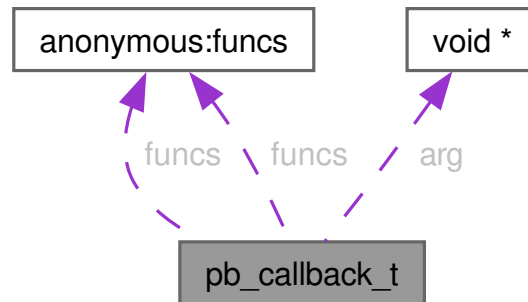
5.1.2.4 encode [2/2]

The documentation for this union was generated from the following files:

5.2 pb_callback_t Struct Reference

```
#include <pb.h>
```

Collaboration diagram for pb_callback_t:



Data Fields

- union {
 bool(* decode)(pb_istream_t *stream, const pb_field_t *field, void **arg)
 bool(* encode)(pb_ostream_t *stream, const pb_field_t *field, void *const *arg)
 } funcs
- void * arg
- union {
 bool(* decode)(pb_istream_t *stream, const pb_field_t *field, void **arg)
 bool(* encode)(pb_ostream_t *stream, const pb_field_t *field, void *const *arg)
 } funcs

5.2.1 Detailed Description

Definition at line 430 of file [pb.h](#).

5.2.2 Field Documentation

5.2.2.1 arg

void * pb_callback_t::arg

Definition at line 440 of file [pb.h](#).

5.2.2.2 [union] [1/2]

union { ... } pb_callback_t::funcs

5.2.2.3 [union] [2/2]

```
union { ... } pb_callback_t::funcs
```

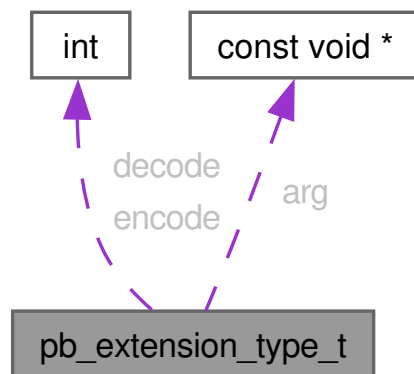
The documentation for this struct was generated from the following files:

- [libs/rx_nanopb/nanopb/pb.h](#)
- [libs/rx_nanopb/nanopb/spm_headers/nanopb/pb.h](#)

5.3 pb_extension_type_t Struct Reference

```
#include <pb.h>
```

Collaboration diagram for pb_extension_type_t:



Data Fields

- [bool\(* decode\)](#)(pb_istream_t *stream, pb_extension_t *extension, [uint32_t](#) tag, [pb_wire_type_t](#) wire_type)
- [bool\(* encode\)](#)(pb_ostream_t *stream, const pb_extension_t *extension)
- [const void * arg](#)

5.3.1 Detailed Description

Definition at line [462](#) of file [pb.h](#).

5.3.2 Field Documentation

5.3.2.1 arg

```
const void * pb_extension_type_t::arg
```

Definition at line [481](#) of file [pb.h](#).

5.3.2.2 decode

```
bool(* pb_extension_type_t::decode)(pb_istream_t *stream, pb_extension_t *extension, uint32_t tag, pb_wire_type_t wire_type)
```

Definition at line 469 of file [pb.h](#).

5.3.2.3 encode

```
bool(* pb_extension_type_t::encode)(pb_ostream_t *stream, const pb_extension_t *extension)
```

Definition at line 478 of file [pb.h](#).

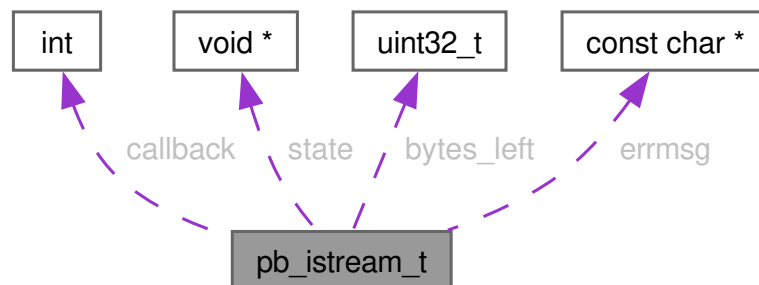
The documentation for this struct was generated from the following files:

- [libs/rx_nanopb/nanopb/pb.h](#)
- [libs/rx_nanopb/nanopb/spm_headers/nanopb/pb.h](#)

5.4 pb_istream_t Struct Reference

```
#include <pb_decode.h>
```

Collaboration diagram for pb_istream_t:



Data Fields

- [bool\(* callback\)](#)(pb_istream_t *stream, [pb_byte_t](#) *buf, [size_t](#) count)
- void * [state](#)
- [size_t](#) [bytes_left](#)
- const char * [errmsg](#)

5.4.1 Detailed Description

Definition at line 28 of file [pb_decode.h](#).

5.4.2 Field Documentation

5.4.2.1 bytes_left

`size_t pb_istream_t::bytes_left`

Definition at line 51 of file [pb_decode.h](#).

5.4.2.2 callback

`bool(* pb_istream_t::callback)(pb_istream_t *stream, pb_byte_t *buf, size_t count)`

Definition at line 37 of file [pb_decode.h](#).

5.4.2.3 errmsg

`const char * pb_istream_t::errmsg`

Definition at line 55 of file [pb_decode.h](#).

5.4.2.4 state

`void * pb_istream_t::state`

Definition at line 44 of file [pb_decode.h](#).

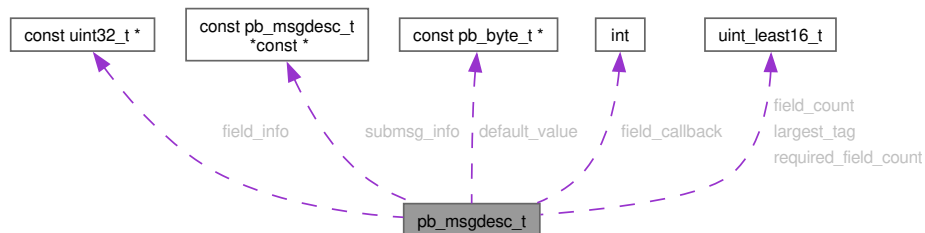
The documentation for this struct was generated from the following files:

- [libs/rx_nanopb/nanopb/pb_decode.h](#)
- [libs/rx_nanopb/nanopb/spm_headers/nanopb/pb_decode.h](#)

5.5 pb_msgdesc_t Struct Reference

`#include <pb.h>`

Collaboration diagram for `pb_msgdesc_t`:



Data Fields

- const [uint32_t](#) * [field_info](#)
- const [pb_msgdesc_t](#) *const * [submsg_info](#)
- const [pb_byte_t](#) * [default_value](#)
- [bool](#)(* [field_callback](#))(pb_istream_t *istream, pb_ostream_t *ostream, const pb_field_iter_t *field)
- [pb_size_t](#) [field_count](#)
- [pb_size_t](#) [required_field_count](#)
- [pb_size_t](#) [largest_tag](#)

5.5.1 Detailed Description

Definition at line [350](#) of file [pb.h](#).

5.5.2 Field Documentation

5.5.2.1 default_value

const [pb_byte_t](#) * [pb_msgdesc_t::default_value](#)

Definition at line [353](#) of file [pb.h](#).

5.5.2.2 field_callback

[bool](#)(* [pb_msgdesc_t::field_callback](#))(pb_istream_t *istream, pb_ostream_t *ostream, const pb_field_iter_t *field)

Definition at line [355](#) of file [pb.h](#).

5.5.2.3 field_count

[pb_size_t](#) [pb_msgdesc_t::field_count](#)

Definition at line [357](#) of file [pb.h](#).

5.5.2.4 field_info

const [uint32_t](#) * [pb_msgdesc_t::field_info](#)

Definition at line [351](#) of file [pb.h](#).

5.5.2.5 largest_tag

[pb_size_t](#) [pb_msgdesc_t::largest_tag](#)

Definition at line [359](#) of file [pb.h](#).

5.5.2.6 required_field_count

[pb_size_t](#) pb_msgdesc_t::required_field_count

Definition at line 358 of file [pb.h](#).

5.5.2.7 submsg_info

const pb_msgdesc_t *const * pb_msgdesc_t::submsg_info

Definition at line 352 of file [pb.h](#).

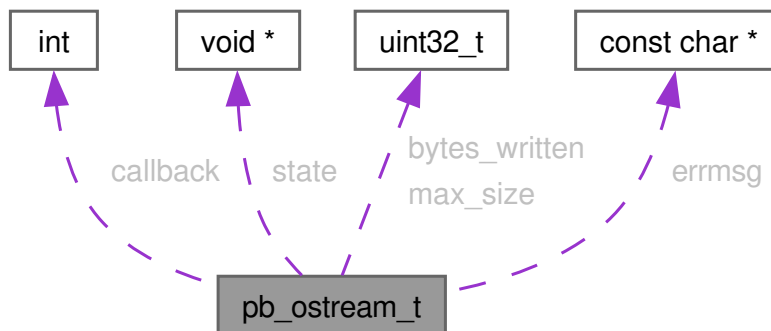
The documentation for this struct was generated from the following files:

- [libs/rx_nanopb/nanopb/pb.h](#)
- [libs/rx_nanopb/nanopb/spm_headers/nanopb/pb.h](#)

5.6 pb_ostream_t Struct Reference

```
#include <pb_encode.h>
```

Collaboration diagram for pb_ostream_t:



Data Fields

- [bool](#)(* [callback](#))(pb_ostream_t *stream, const [pb_byte_t](#) *buf, [size_t](#) count)
- void * [state](#)
- [size_t](#) [max_size](#)
- [size_t](#) [bytes_written](#)
- const char * [errmsg](#)

5.6.1 Detailed Description

Definition at line 27 of file [pb_encode.h](#).

5.6.2 Field Documentation

5.6.2.1 bytes_written

[size_t](#) pb_ostream_t::bytes_written

Definition at line 51 of file [pb_encode.h](#).

5.6.2.2 callback

[bool](#)(* pb_ostream_t::callback)(pb_ostream_t *stream, const [pb_byte_t](#) *buf, [size_t](#) count)

Definition at line 38 of file [pb_encode.h](#).

5.6.2.3 errmsg

const char * pb_ostream_t::errmsg

Definition at line 55 of file [pb_encode.h](#).

5.6.2.4 max_size

[size_t](#) pb_ostream_t::max_size

Definition at line 48 of file [pb_encode.h](#).

5.6.2.5 state

void * pb_ostream_t::state

Definition at line 45 of file [pb_encode.h](#).

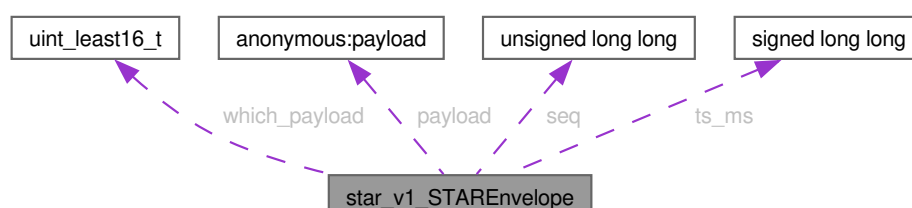
The documentation for this struct was generated from the following files:

- [libs/rx_nanopb/nanopb/pb_encode.h](#)
- [libs/rx_nanopb/nanopb/spm_headers/nanopb/pb_encode.h](#)

5.7 star_v1_STAREnvelope Struct Reference

#include <ui.pb.h>

Collaboration diagram for star_v1_STAREnvelope:



Data Fields

- [pb_size_t](#) [which_payload](#)
- [union](#) {
 - [star_v1_TelemetryData](#) [telemetry](#)
 - [star_v1_MotorStatusList](#) [motors](#)
 - [star_v1_OdometryData](#) [odometry](#)
 - [star_v1_LidarScan](#) [lidar](#)
 - [star_v1_Alert](#) [alert](#)
 - [star_v1_SystemStatus](#) [system](#)
 - [star_v1_ControllerState](#) [controller](#)
 - [star_v1_EStopCommand](#) [estop](#)
- [uint64_t](#) [seq](#)
- [int64_t](#) [ts_ms](#)

5.7.1 Detailed Description

Definition at line 104 of file [ui.pb.h](#).

5.7.2 Field Documentation

5.7.2.1 [union]

[union](#) { ... } [star_v1_STAREnvelope::payload](#)

5.7.2.2 seq

[uint64_t](#) [star_v1_STAREnvelope::seq](#)

Definition at line 125 of file [ui.pb.h](#).

5.7.2.3 ts_ms

[int64_t](#) [star_v1_STAREnvelope::ts_ms](#)

Definition at line 127 of file [ui.pb.h](#).

5.7.2.4 which_payload

[pb_size_t](#) [star_v1_STAREnvelope::which_payload](#)

Definition at line 105 of file [ui.pb.h](#).

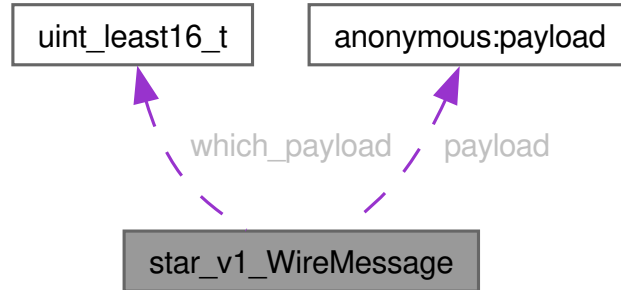
The documentation for this struct was generated from the following file:

- [libs/rx_nanopb/inc/gen/star/v1/ui.pb.h](#)

5.8 star_v1_WireMessage Struct Reference

```
#include <wire.pb.h>
```

Collaboration diagram for star_v1_WireMessage:



Data Fields

- [pb_size_t](#) [which_payload](#)
- union {
 - [star_v1_VelocityCommand](#) [velocity_command](#)
 - [star_v1_EmergencyStopCommand](#) [emergency_stop_command](#)
 - [star_v1_MotorPowerCommand](#) [motor_power_command](#)
 - [star_v1_TelemetryData](#) [telemetry_data](#)
 - [star_v1_EncoderData](#) [encoder_data](#)
 - [star_v1_PidConfig](#) [pid_config](#)
 - [star_v1_RetransmitConfig](#) [retransmit_config](#)
 - [star_v1_TransportDiagnostics](#) [transport_diagnostics](#)
- } [payload](#)

5.8.1 Detailed Description

Definition at line 54 of file [wire.pb.h](#).

5.8.2 Field Documentation

5.8.2.1 [union]

union { ... } [star_v1_WireMessage::payload](#)

Referenced by [rx_nanopb_encode_telemetry\(\)](#).

5.8.2.2 which_payload

[pb_size_t](#) [star_v1_WireMessage::which_payload](#)

Definition at line 55 of file [wire.pb.h](#).

Referenced by [rx_nanopb_encode_telemetry\(\)](#).

The documentation for this struct was generated from the following file:

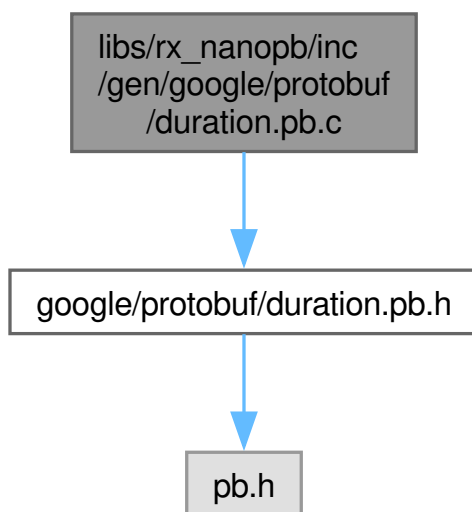
- [libs/rx_nanopb/inc/gen/star/v1/wire.pb.h](#)

Chapter 6

File Documentation

6.1 libs/rx_nanopb/inc/gen/google/protobuf/duration.pb.c File Reference

#include "google/protobuf/duration.pb.h"
Include dependency graph for duration.pb.c:



6.2 duration.pb.c

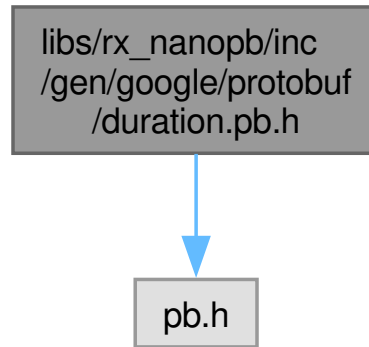
[Go to the documentation of this file.](#)

```
00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "google/protobuf/duration.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40
00006 #error Regenerate this file with the current version of nanopb generator.
00007 #endif
00008
00009 PB_BIND(google_protobuf_Duration, google_protobuf_Duration, AUTO)
```

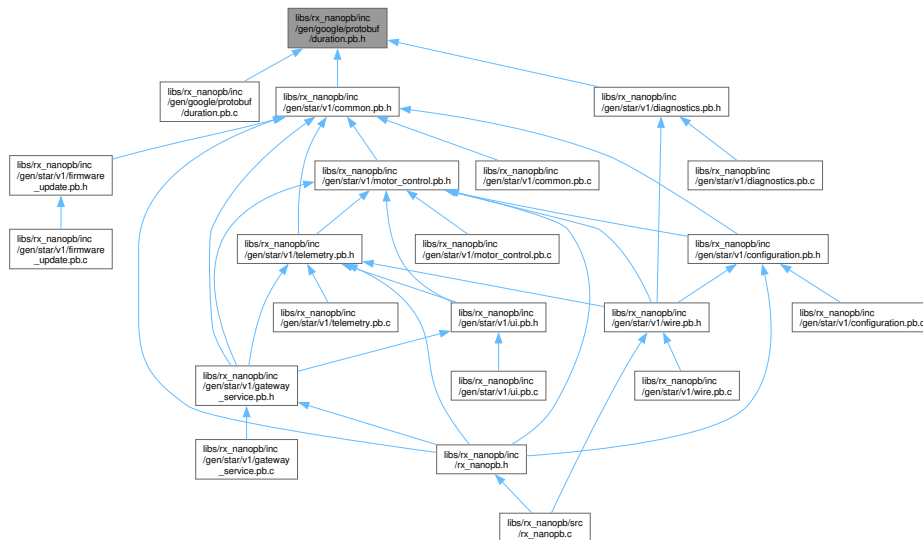
6.3 libs/rx_nanopb/inc/gen/google/protobuf/duration.pb.h File Reference

```
#include <pb.h>
```

Include dependency graph for duration.pb.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [google_protobuf_Duration](#)

Macros

- `#define google_protobuf_Duration_init_default`
- `#define google_protobuf_Duration_init_zero`
- `#define google_protobuf_Duration_seconds_tag 1`
- `#define google_protobuf_Duration_nanos_tag 2`
- `#define google_protobuf_Duration_FIELDLIST(X, a)`
- `#define google_protobuf_Duration_CALLBACK nullptr`
- `#define google_protobuf_Duration_DEFAULT nullptr`
- `#define google_protobuf_Duration_fields &google_protobuf_Duration_msg`
- `#define GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_DURATION_PB_H_MAX_SIZE google_protobuf_`
- `#define google_protobuf_Duration_size 22`

Variables

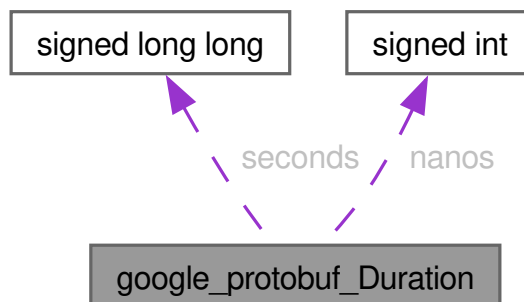
- const pb_msgdesc_t [google_protobuf_Duration_msg](#)

6.3.1 Data Structure Documentation

6.3.1.1 struct google_protobuf_Duration

Definition at line 71 of file [duration.pb.h](#).

Collaboration diagram for google_protobuf_Duration:



Data Fields

int32_t	nanos	
int64_t	seconds	

6.3.2 Macro Definition Documentation

6.3.2.1 google_protobuf_Duration_CALLBACK

```
#define google_protobuf_Duration_CALLBACK nullptr
```

Definition at line 107 of file [duration.pb.h](#).

6.3.2.2 google_protobuf_Duration_DEFAULT

```
#define google_protobuf_Duration_DEFAULT nullptr
```

Definition at line 108 of file [duration.pb.h](#).

6.3.2.3 google_protobuf_Duration_FIELDLIST

```
#define google_protobuf_Duration_FIELDLIST(
    X,
    a)
```

Value:

```
X(a, STATIC, SINGULAR, INT64, seconds, 1)
X(a, STATIC, SINGULAR, INT32, nanos, 2)
```

Definition at line 104 of file [duration.pb.h](#).

```
00104 #define google_protobuf_Duration_FIELDLIST(X, a)
00105     X(a, STATIC, SINGULAR, INT64, seconds, 1)
00106     X(a, STATIC, SINGULAR, INT32, nanos, 2)
```

6.3.2.4 google_protobuf_Duration_fields

```
#define google_protobuf_Duration_fields &google_protobuf_Duration_msg
```

Definition at line 113 of file [duration.pb.h](#).

6.3.2.5 google_protobuf_Duration_init_default

```
#define google_protobuf_Duration_init_default
```

Value:

```
{
  0, 0
}
```

Definition at line 90 of file [duration.pb.h](#).

```
00090 #define google_protobuf_Duration_init_default
00091 {
00092     0, 0
00093 }
```

6.3.2.6 google_protobuf_Duration_init_zero

```
#define google_protobuf_Duration_init_zero
```

Value:

```
{
  0, 0
}
```

Definition at line 94 of file [duration.pb.h](#).

```
00094 #define google_protobuf_Duration_init_zero
00095 {
00096     0, 0
00097 }
```

6.3.2.7 google_protobuf_Duration_nanos_tag

```
#define google_protobuf_Duration_nanos_tag 2
```

Definition at line 101 of file [duration.pb.h](#).

6.3.2.8 google_protobuf_Duration_seconds_tag

```
#define google_protobuf_Duration_seconds_tag 1
```

Definition at line 100 of file [duration.pb.h](#).

6.3.2.9 google_protobuf_Duration_size

```
#define google_protobuf_Duration_size 22
```

Definition at line 117 of file [duration.pb.h](#).

6.3.2.10 GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_DURATION_PB_H_MAX_SIZE

```
#define GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_DURATION_PB_H_MAX_SIZE google\_protobuf\_Duration\_size
```

Definition at line 116 of file [duration.pb.h](#).

6.3.3 Variable Documentation

6.3.3.1 google_protobuf_Duration_msg

```
const pb_msgdesc_t google_protobuf_Duration_msg [extern]
```

6.4 duration.pb.h

[Go to the documentation of this file.](#)

```
00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_DURATION_PB_H_INCLUDED
00005 #define PB_GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_DURATION_PB_H_INCLUDED
00006 #include <pb.h>
00007
00008 #if PB_PROTO_HEADER_VERSION != 40
00009 #error Regenerate this file with the current version of nanopb generator.
00010 #endif
00011
00012 /* Struct definitions */
00013 /* A Duration represents a signed, fixed-length span of time represented
00014 as a count of seconds and fractions of seconds at nanosecond
00015 resolution. It is independent of any calendar and concepts like "day"
00016 or "month". It is related to Timestamp in that the difference between
00017 two Timestamp values is a Duration and it can be added or subtracted
00018 from a Timestamp. Range is approximately +/-10,000 years.
00019
00020 # Examples
00021
00022 Example 1: Compute Duration from two Timestamps in pseudo code.
00023
00024     Timestamp start = ...;
00025     Timestamp end = ...;
00026     Duration duration = ...;
00027
00028     duration.seconds = end.seconds - start.seconds;
00029     duration.nanos = end.nanos - start.nanos;
00030
00031     if (duration.seconds < 0 && duration.nanos > 0) {
00032         duration.seconds += 1;
00033         duration.nanos -= 1000000000;
00034     } else if (duration.seconds > 0 && duration.nanos < 0) {
00035         duration.seconds -= 1;
00036         duration.nanos += 1000000000;
00037     }
00038
00039 Example 2: Compute Timestamp from Timestamp + Duration in pseudo code.
00040
00041     Timestamp start = ...;
00042     Duration duration = ...;
00043     Timestamp end = ...;
00044
00045     end.seconds = start.seconds + duration.seconds;
00046     end.nanos = start.nanos + duration.nanos;
00047
00048     if (end.nanos < 0) {
00049         end.seconds -= 1;
00050         end.nanos += 1000000000;
00051     } else if (end.nanos >= 1000000000) {
00052         end.seconds += 1;
00053         end.nanos -= 1000000000;
00054     }
00055
00056 Example 3: Compute Duration from datetime.timedelta in Python.
00057
```

```

00058     td = datetime.timedelta(days=3, minutes=10)
00059     duration = Duration()
00060     duration.FromTimedelta(td)
00061
00062     # JSON Mapping
00063
00064     In JSON format, the Duration type is encoded as a string rather than an
00065     object, where the string ends in the suffix "s" (indicating seconds) and
00066     is preceded by the number of seconds, with nanoseconds expressed as
00067     fractional seconds. For example, 3 seconds with 0 nanoseconds should be
00068     encoded in JSON format as "3s", while 3 seconds and 1 nanosecond should
00069     be expressed in JSON format as "3.000000001s", and 3 seconds and 1
00070     microsecond should be expressed in JSON format as "3.000001s". */
00071     typedef struct _google_protobuf_Duration {
00072     /* Signed seconds of the span of time. Must be from -315,576,000,000
00073     to +315,576,000,000 inclusive. Note: these bounds are computed from:
00074     60 sec/min * 60 min/hr * 24 hr/day * 365.25 days/year * 10000 years */
00075     int64_t seconds;
00076     /* Signed fractions of a second at nanosecond resolution of the span
00077     of time. Durations less than one second are represented with a 0
00078     `seconds` field and a positive or negative `nanos` field. For durations
00079     of one second or more, a non-zero value for the `nanos` field must be
00080     of the same sign as the `seconds` field. Must be from -999,999,999
00081     to +999,999,999 inclusive. */
00082     int32_t nanos;
00083     } google_protobuf_Duration;
00084
00085     #ifdef __cplusplus
00086     extern "C" {
00087     #endif
00088
00089     /* Initializer values for message structs */
00090     #define google_protobuf_Duration_init_default
00091     {
00092         0, 0
00093     }
00094     #define google_protobuf_Duration_init_zero
00095     {
00096         0, 0
00097     }
00098
00099     /* Field tags (for use in manual encoding/decoding) */
00100     #define google_protobuf_Duration_seconds_tag 1
00101     #define google_protobuf_Duration_nanos_tag 2
00102
00103     /* Struct field encoding specification for nanopb */
00104     #define google_protobuf_Duration_FIELDLIST(X, a)
00105     X(a, STATIC, SINGULAR, INT64, seconds, 1)
00106     X(a, STATIC, SINGULAR, INT32, nanos, 2)
00107     #define google_protobuf_Duration_CALLBACK nullptr
00108     #define google_protobuf_Duration_DEFAULT nullptr
00109
00110     extern const pb_msgdesc_t google_protobuf_Duration_msg;
00111
00112     /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00113     #define google_protobuf_Duration_fields &google_protobuf_Duration_msg
00114
00115     /* Maximum encoded size of messages (where known) */
00116     #define GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_DURATION_PB_H_MAX_SIZE
00117     google_protobuf_Duration_size
00118     #define google_protobuf_Duration_size 22
00119
00120     #ifdef __cplusplus
00121     } /* extern "C" */
00122     #endif
00123     #endif

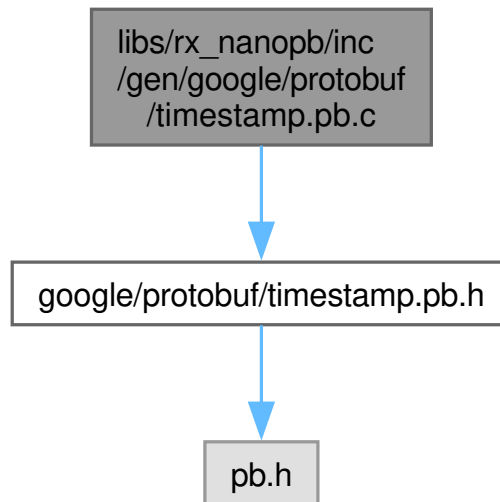
```

6.5 libs/rx_nanopb/inc/gen/google/protobuf/timestamp.pb.c File Reference

```

#include "google/protobuf/timestamp.pb.h"
Include dependency graph for timestamp.pb.c:

```



6.6 timestamp.pb.c

[Go to the documentation of this file.](#)

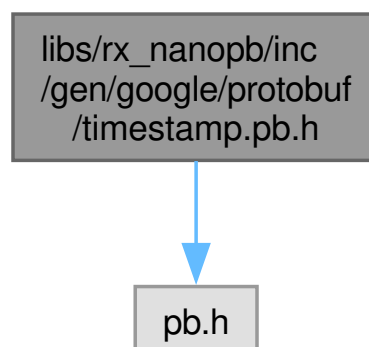
```

00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "google/protobuf/timestamp.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40
00006 #error Regenerate this file with the current version of nanopb generator.
00007 #endif
00008
00009 PB_BIND(google_protobuf_Timestamp, google_protobuf_Timestamp, AUTO)
  
```

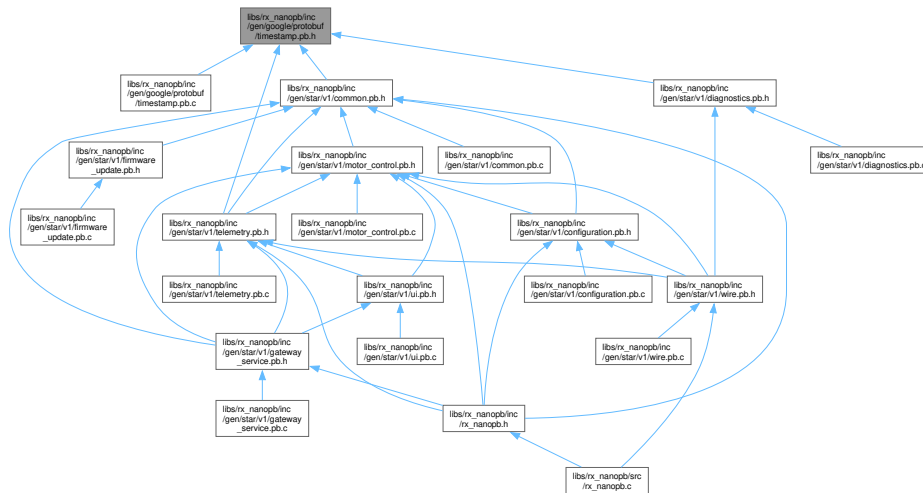
6.7 libs/rx_nanopb/inc/gen/google/protobuf/timestamp.pb.h File Reference

```
#include <pb.h>
```

Include dependency graph for timestamp.pb.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [google_protobuf_Timestamp](#)

Macros

- `#define google_protobuf_Timestamp_init_default`
- `#define google_protobuf_Timestamp_init_zero`
- `#define google_protobuf_Timestamp_seconds_tag 1`
- `#define google_protobuf_Timestamp_nanos_tag 2`
- `#define google_protobuf_Timestamp_FIELDLIST(X, a)`
- `#define google_protobuf_Timestamp_CALLBACK nullptr`
- `#define google_protobuf_Timestamp_DEFAULT nullptr`
- `#define google_protobuf_Timestamp_fields &google_protobuf_Timestamp_msg`
- `#define GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_TIMESTAMP_PB_H_MAX_SIZE google_protobuf_Timestamp_msg`
- `#define google_protobuf_Timestamp_size 22`

Variables

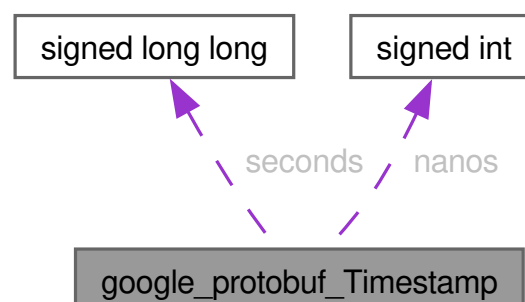
- `const pb_msgdesc_t google_protobuf_Timestamp_msg`

6.7.1 Data Structure Documentation

6.7.1.1 struct google_protobuf_Timestamp

Definition at line 102 of file [timestamp.pb.h](#).

Collaboration diagram for `google_protobuf_Timestamp`:



Data Fields

int32_t	nanos	
int64_t	seconds	

6.7.2 Macro Definition Documentation

6.7.2.1 GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_TIMESTAMP_PB_H_MAX_SIZE

```
#define GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_TIMESTAMP_PB_H_MAX_SIZE google\_protobuf\_Timestamp\_size
```

Definition at line [146](#) of file [timestamp.pb.h](#).

6.7.2.2 google_protobuf_Timestamp_CALLBACK

```
#define google_protobuf_Timestamp_CALLBACK nullptr
```

Definition at line [137](#) of file [timestamp.pb.h](#).

6.7.2.3 google_protobuf_Timestamp_DEFAULT

```
#define google_protobuf_Timestamp_DEFAULT nullptr
```

Definition at line [138](#) of file [timestamp.pb.h](#).

6.7.2.4 google_protobuf_Timestamp_FIELDLIST

```
#define google_protobuf_Timestamp_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, INT64, seconds, 1) \
X(a, STATIC, SINGULAR, INT32, nanos, 2)
```

Definition at line [134](#) of file [timestamp.pb.h](#).

```
00134 #define google_protobuf_Timestamp_FIELDLIST(X, a) \
00135     X(a, STATIC, SINGULAR, INT64, seconds, 1) \
00136     X(a, STATIC, SINGULAR, INT32, nanos, 2)
```

6.7.2.5 google_protobuf_Timestamp_fields

```
#define google_protobuf_Timestamp_fields &google\_protobuf\_Timestamp\_msg
```

Definition at line [143](#) of file [timestamp.pb.h](#).

6.7.2.6 google_protobuf_Timestamp_init_default

```
#define google_protobuf_Timestamp_init_default
```

Value:

```
{
  0, 0
}
```

Definition at line 120 of file [timestamp.pb.h](#).

```
00120 #define google_protobuf_Timestamp_init_default
00121 {
00122     0, 0
00123 }
```

6.7.2.7 google_protobuf_Timestamp_init_zero

```
#define google_protobuf_Timestamp_init_zero
```

Value:

```
{
  0, 0
}
```

Definition at line 124 of file [timestamp.pb.h](#).

```
00124 #define google_protobuf_Timestamp_init_zero
00125 {
00126     0, 0
00127 }
```

6.7.2.8 google_protobuf_Timestamp_nanos_tag

```
#define google_protobuf_Timestamp_nanos_tag 2
```

Definition at line 131 of file [timestamp.pb.h](#).

6.7.2.9 google_protobuf_Timestamp_seconds_tag

```
#define google_protobuf_Timestamp_seconds_tag 1
```

Definition at line 130 of file [timestamp.pb.h](#).

6.7.2.10 google_protobuf_Timestamp_size

```
#define google_protobuf_Timestamp_size 22
```

Definition at line 147 of file [timestamp.pb.h](#).

6.7.3 Variable Documentation

6.7.3.1 google_protobuf_Timestamp_msg

```
const pb_msgdesc_t google_protobuf_Timestamp_msg [extern]
```


6.8 timestamp.pb.h

[Go to the documentation of this file.](#)

```

00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_TIMESTAMP_PB_H_INCLUDED
00005 #define PB_GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_TIMESTAMP_PB_H_INCLUDED
00006 #include <pb.h>
00007
00008 #if PB_PROTO_HEADER_VERSION != 40
00009 #error Regenerate this file with the current version of nanopb generator.
00010 #endif
00011
00012 /* Struct definitions */
00013 /* A Timestamp represents a point in time independent of any time zone or local
00014 calendar, encoded as a count of seconds and fractions of seconds at
00015 nanosecond resolution. The count is relative to an epoch at UTC midnight on
00016 January 1, 1970, in the proleptic Gregorian calendar which extends the
00017 Gregorian calendar backwards to year one.
00018
00019 All minutes are 60 seconds long. Leap seconds are "smeared" so that no leap
00020 second table is needed for interpretation, using a [24-hour linear
00021 smear](https://developers.google.com/time/smear).
00022
00023 The range is from 0001-01-01T00:00:00Z to 9999-12-31T23:59:59.999999999Z. By
00024 restricting to that range, we ensure that we can convert to and from [RFC
00025 3339](https://www.ietf.org/rfc/rfc3339.txt) date strings.
00026
00027 # Examples
00028
00029 Example 1: Compute Timestamp from POSIX `time()`.
00030
00031     Timestamp timestamp;
00032     timestamp.set_seconds(time(NULL));
00033     timestamp.set_nanos(0);
00034
00035 Example 2: Compute Timestamp from POSIX `gettimeofday()`.
00036
00037     struct timeval tv;
00038     gettimeofday(&tv, NULL);
00039
00040     Timestamp timestamp;
00041     timestamp.set_seconds(tv.tv_sec);
00042     timestamp.set_nanos(tv.tv_usec * 1000);
00043
00044 Example 3: Compute Timestamp from Win32 `GetSystemTimeAsFileTime()`.
00045
00046     FILETIME ft;
00047     GetSystemTimeAsFileTime(&ft);
00048     UINT64 ticks = (((UINT64)ft.dwHighDateTime) « 32) | ft.dwLowDateTime;
00049
00050     // A Windows tick is 100 nanoseconds. Windows epoch 1601-01-01T00:00:00Z
00051     // is 11644473600 seconds before Unix epoch 1970-01-01T00:00:00Z.
00052     Timestamp timestamp;
00053     timestamp.set_seconds((INT64) ((ticks / 10000000) - 11644473600LL));
00054     timestamp.set_nanos((INT32) ((ticks % 10000000) * 100));
00055
00056 Example 4: Compute Timestamp from Java `System.currentTimeMillis()`.
00057
00058     long millis = System.currentTimeMillis();
00059
00060     Timestamp timestamp = Timestamp.newBuilder().setSeconds(millis / 1000)
00061         .setNanos((int) ((millis % 1000) * 1000000)).build();
00062
00063 Example 5: Compute Timestamp from Java `Instant.now()`.
00064
00065     Instant now = Instant.now();
00066
00067     Timestamp timestamp =
00068         Timestamp.newBuilder().setSeconds(now.getEpochSecond())
00069         .setNanos(now.getNano()).build();
00070
00071 Example 6: Compute Timestamp from current time in Python.
00072
00073     timestamp = Timestamp()
00074     timestamp.GetCurrentTime()
00075
00076 # JSON Mapping
00077
00078 In JSON format, the Timestamp type is encoded as a string in the
00079 [RFC 3339](https://www.ietf.org/rfc/rfc3339.txt) format. That is, the
00080 format is "{year}-{month}-{day}T{hour}:{min}:{sec}[.{frac_sec}]Z"
00081 where {year} is always expressed using four digits while {month}, {day},
00082 {hour}, {min}, and {sec} are zero-padded to two digits each. The fractional

```

```

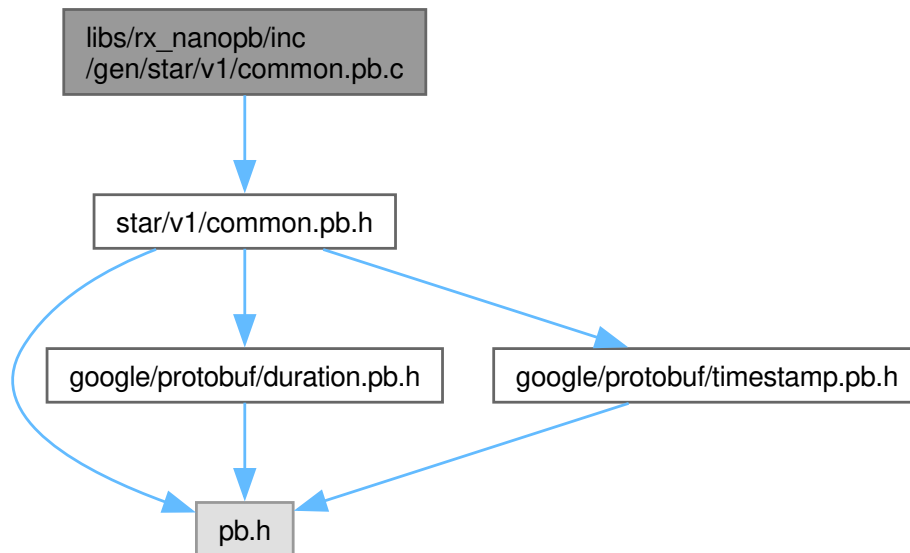
00083 seconds, which can go up to 9 digits (i.e. up to 1 nanosecond resolution),
00084 are optional. The "Z" suffix indicates the timezone ("UTC"); the timezone
00085 is required. A ProtoJSON serializer should always use UTC (as indicated by
00086 "Z") when printing the Timestamp type and a ProtoJSON parser should be
00087 able to accept both UTC and other timezones (as indicated by an offset).
00088
00089 For example, "2017-01-15T01:30:15.01Z" encodes 15.01 seconds past
00090 01:30 UTC on January 15, 2017.
00091
00092 In JavaScript, one can convert a Date object to this format using the
00093 standard
00094 [toISOString()](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toISOString)
00095 method. In Python, a standard `datetime.datetime` object can be converted
00096 to this format using
00097 [strftime](https://docs.python.org/2/library/time.html#time.strftime) with
00098 the time format spec '%Y-%m-%dT%H:%M:%S.%fZ'. Likewise, in Java, one can use
00099 the Joda Time's [ISODateTimeFormat.dateTime()](
00100 http://joda-time.sourceforge.net/apidocs/org/joda/time/format/ISODateTimeFormat.html#dateTime()
00101 ) to obtain a formatter capable of generating timestamps in this format. */
00102 typedef struct _google_protobuf_Timestamp {
00103     /* Represents seconds of UTC time since Unix epoch 1970-01-01T00:00:00Z. Must
00104     be between -62135596800 and 253402300799 inclusive (which corresponds to
00105     0001-01-01T00:00:00Z to 9999-12-31T23:59:59Z). */
00106     int64_t seconds;
00107     /* Non-negative fractions of a second at nanosecond resolution. This field is
00108     the nanosecond portion of the duration, not an alternative to seconds.
00109     Negative second values with fractions must still have non-negative nanos
00110     values that count forward in time. Must be between 0 and 999,999,999
00111     inclusive. */
00112     int32_t nanos;
00113 } google_protobuf_Timestamp;
00114
00115 #ifdef __cplusplus
00116 extern "C" {
00117 #endif
00118
00119 /* Initializer values for message structs */
00120 #define google_protobuf_Timestamp_init_default
00121 {
00122     0, 0
00123 }
00124 #define google_protobuf_Timestamp_init_zero
00125 {
00126     0, 0
00127 }
00128
00129 /* Field tags (for use in manual encoding/decoding) */
00130 #define google_protobuf_Timestamp_seconds_tag 1
00131 #define google_protobuf_Timestamp_nanos_tag 2
00132
00133 /* Struct field encoding specification for nanopb */
00134 #define google_protobuf_Timestamp_FIELDLIST(X, a)
00135 X(a, STATIC, SINGULAR, INT64, seconds, 1)
00136 X(a, STATIC, SINGULAR, INT32, nanos, 2)
00137 #define google_protobuf_Timestamp_CALLBACK nullptr
00138 #define google_protobuf_Timestamp_DEFAULT nullptr
00139
00140 extern const pb_msgdesc_t google_protobuf_Timestamp_msg;
00141
00142 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00143 #define google_protobuf_Timestamp_fields &google_protobuf_Timestamp_msg
00144
00145 /* Maximum encoded size of messages (where known) */
00146 #define GOOGLE_PROTOBUF_GOOGLE_PROTOBUF_TIMESTAMP_PB_H_MAX_SIZE
00147 google_protobuf_Timestamp_size
00148 #define google_protobuf_Timestamp_size 22
00149
00150 #ifdef __cplusplus
00151 } /* extern "C" */
00152 #endif
00153 #endif

```

6.9 libs/rx_nanopb/inc/gen/star/v1/common.pb.c File Reference

```
#include "star/v1/common.pb.h"
```

Include dependency graph for common.pb.c:



6.10 common.pb.c

[Go to the documentation of this file.](#)

```

00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "star/v1/common.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40
00006 #error Regenerate this file with the current version of nanopb generator.
00007 #endif
00008
00009 PB_BIND(star_v1_RequestHeader, star_v1_RequestHeader, AUTO)
00010
00011
00012 PB_BIND(star_v1_ResponseHeader, star_v1_ResponseHeader, 2)
00013
00014
00015 PB_BIND(star_v1_Vector3, star_v1_Vector3, AUTO)
00016
00017
00018 PB_BIND(star_v1_Quaternion, star_v1_Quaternion, AUTO)
00019
00020
00021 PB_BIND(star_v1_SE2Pose, star_v1_SE2Pose, AUTO)
00022
00023
00024 PB_BIND(star_v1_SE2Velocity, star_v1_SE2Velocity, AUTO)
00025
00026
00027
00028
00029
00030 #ifndef PB_CONVERT_DOUBLE_FLOAT
00031 /* On some platforms (such as AVR), double is really float.
00032 * To be able to encode/decode double on these platforms, you need.
00033 * to define PB_CONVERT_DOUBLE_FLOAT in pb.h or compiler command line.
00034 */
00035 PB_STATIC_ASSERT(sizeof(double) == 8, DOUBLE_MUST_BE_8_BYTES)
00036 #endif
00037

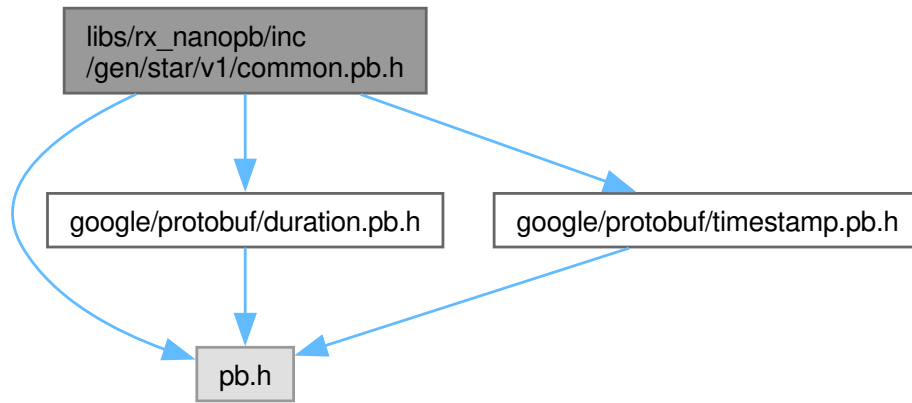
```

6.11 libs/rx_nanopb/inc/gen/star/v1/common.pb.h File Reference

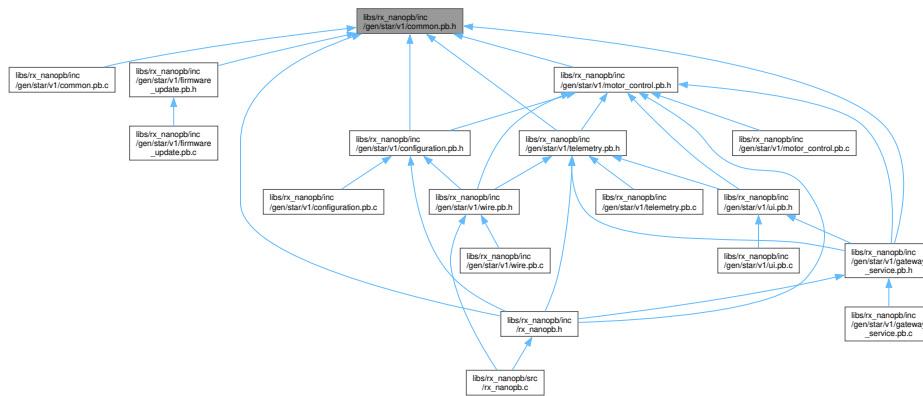
```

#include <pb.h>
#include "google/protobuf/duration.pb.h"
#include "google/protobuf/timestamp.pb.h"
Include dependency graph for common.pb.h:

```



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [star_v1_RequestHeader](#)
- struct [star_v1_ResponseHeader](#)
- struct [star_v1_Vector3](#)
- struct [star_v1_Quaternion](#)
- struct [star_v1_SE2Pose](#)
- struct [star_v1_SE2Velocity](#)

Macros

- [#define _star_v1_Status_MIN star_v1_Status_STATUS_UNKNOWN](#)
- [#define _star_v1_Status_MAX star_v1_Status_STATUS_ESTOP_ACTIVE](#)
- [#define _star_v1_Status_ARRAYSIZE \(\(star_v1_Status\)\(star_v1_Status_STATUS_ESTOP_ACTIVE+1\)\)](#)
- [#define star_v1_ResponseHeader_status_ENUMTYPE star_v1_Status](#)
- [#define star_v1_RequestHeader_init_default { "", false, google_protobuf_Timestamp_init_default, "", false, google_protobuf_Duration_init_default }](#)
- [#define star_v1_ResponseHeader_init_default { "", false, google_protobuf_Timestamp_init_default, _star_v1_Status_MIN, "", 0 }](#)
- [#define star_v1_Vector3_init_default { 0, 0, 0 }](#)
- [#define star_v1_Quaternion_init_default { 0, 0, 0, 0 }](#)
- [#define star_v1_SE2Pose_init_default { 0, 0, 0 }](#)
- [#define star_v1_SE2Velocity_init_default { 0, 0, 0 }](#)

- `#define star_v1_RequestHeader_init_zero { "", false, google_protobuf_Timestamp_init_zero, "", false, google_protobuf_Duration_init_zero }`
- `#define star_v1_ResponseHeader_init_zero { "", false, google_protobuf_Timestamp_init_zero, star_v1_Status_MIN, "", 0 }`
- `#define star_v1_Vector3_init_zero { 0, 0, 0 }`
- `#define star_v1_Quaternion_init_zero { 0, 0, 0, 0 }`
- `#define star_v1_SE2Pose_init_zero { 0, 0, 0 }`
- `#define star_v1_SE2Velocity_init_zero { 0, 0, 0 }`
- `#define star_v1_RequestHeader_request_id_tag 1`
- `#define star_v1_RequestHeader_client_timestamp_tag 2`
- `#define star_v1_RequestHeader_client_version_tag 3`
- `#define star_v1_RequestHeader_timeout_tag 4`
- `#define star_v1_ResponseHeader_request_id_tag 1`
- `#define star_v1_ResponseHeader_server_timestamp_tag 2`
- `#define star_v1_ResponseHeader_status_tag 3`
- `#define star_v1_ResponseHeader_error_message_tag 4`
- `#define star_v1_ResponseHeader_latency_us_tag 5`
- `#define star_v1_Vector3_x_tag 1`
- `#define star_v1_Vector3_y_tag 2`
- `#define star_v1_Vector3_z_tag 3`
- `#define star_v1_Quaternion_w_tag 1`
- `#define star_v1_Quaternion_x_tag 2`
- `#define star_v1_Quaternion_y_tag 3`
- `#define star_v1_Quaternion_z_tag 4`
- `#define star_v1_SE2Pose_x_m_tag 1`
- `#define star_v1_SE2Pose_y_m_tag 2`
- `#define star_v1_SE2Pose_angle_rad_tag 3`
- `#define star_v1_SE2Velocity_vx_mps_tag 1`
- `#define star_v1_SE2Velocity_vy_mps_tag 2`
- `#define star_v1_SE2Velocity_omega_rad_per_s_tag 3`
- `#define star_v1_RequestHeader_FIELDLIST(X, a)`
- `#define star_v1_RequestHeader_CALLBACK NULL`
- `#define star_v1_RequestHeader_DEFAULT NULL`
- `#define star_v1_RequestHeader_client_timestamp_MSGTYPE google_protobuf_Timestamp`
- `#define star_v1_RequestHeader_timeout_MSGTYPE google_protobuf_Duration`
- `#define star_v1_ResponseHeader_FIELDLIST(X, a)`
- `#define star_v1_ResponseHeader_CALLBACK NULL`
- `#define star_v1_ResponseHeader_DEFAULT NULL`
- `#define star_v1_ResponseHeader_server_timestamp_MSGTYPE google_protobuf_Timestamp`
- `#define star_v1_Vector3_FIELDLIST(X, a)`
- `#define star_v1_Vector3_CALLBACK NULL`
- `#define star_v1_Vector3_DEFAULT NULL`
- `#define star_v1_Quaternion_FIELDLIST(X, a)`
- `#define star_v1_Quaternion_CALLBACK NULL`
- `#define star_v1_Quaternion_DEFAULT NULL`
- `#define star_v1_SE2Pose_FIELDLIST(X, a)`
- `#define star_v1_SE2Pose_CALLBACK NULL`
- `#define star_v1_SE2Pose_DEFAULT NULL`
- `#define star_v1_SE2Velocity_FIELDLIST(X, a)`
- `#define star_v1_SE2Velocity_CALLBACK NULL`
- `#define star_v1_SE2Velocity_DEFAULT NULL`
- `#define star_v1_RequestHeader_fields &star_v1_RequestHeader_msg`
- `#define star_v1_ResponseHeader_fields &star_v1_ResponseHeader_msg`
- `#define star_v1_Vector3_fields &star_v1_Vector3_msg`
- `#define star_v1_Quaternion_fields &star_v1_Quaternion_msg`

- `#define star_v1_SE2Pose_fields &star_v1_SE2Pose_msg`
- `#define star_v1_SE2Velocity_fields &star_v1_SE2Velocity_msg`
- `#define STAR_V1_STAR_V1_COMMON_PB_H_MAX_SIZE star_v1_ResponseHeader_size`
- `#define star_v1_Quaternion_size 36`
- `#define star_v1_RequestHeader_size 146`
- `#define star_v1_ResponseHeader_size 360`
- `#define star_v1_SE2Pose_size 27`
- `#define star_v1_SE2Velocity_size 27`
- `#define star_v1_Vector3_size 27`

Enumerations

- `enum star_v1_Status {`
`star_v1_Status_STATUS_UNKNOWN = 0 , star_v1_Status_STATUS_OK = 1 , star_v1_Status_STATUS_IN`
`= 2 , star_v1_Status_STATUS_INTERNAL_ERROR = 3 ,`
`star_v1_Status_STATUS_NOT_FOUND = 4 , star_v1_Status_STATUS_TIMEOUT = 5 ,`
`star_v1_Status_STATUS_INVALID_STATE = 6 , star_v1_Status_STATUS_ESTOP_ACTIVE`
`= 7 }`

Variables

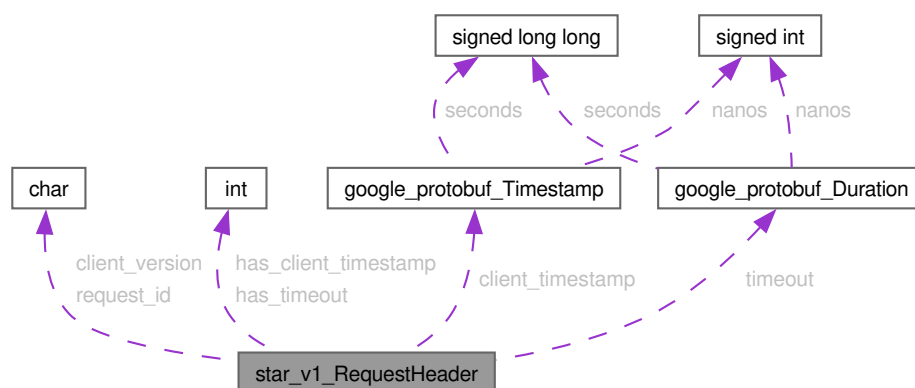
- `const pb_msgdesc_t star_v1_RequestHeader_msg`
- `const pb_msgdesc_t star_v1_ResponseHeader_msg`
- `const pb_msgdesc_t star_v1_Vector3_msg`
- `const pb_msgdesc_t star_v1_Quaternion_msg`
- `const pb_msgdesc_t star_v1_SE2Pose_msg`
- `const pb_msgdesc_t star_v1_SE2Velocity_msg`

6.11.1 Data Structure Documentation

6.11.1.1 struct star_v1_RequestHeader

Definition at line 39 of file [common.pb.h](#).

Collaboration diagram for `star_v1_RequestHeader`:



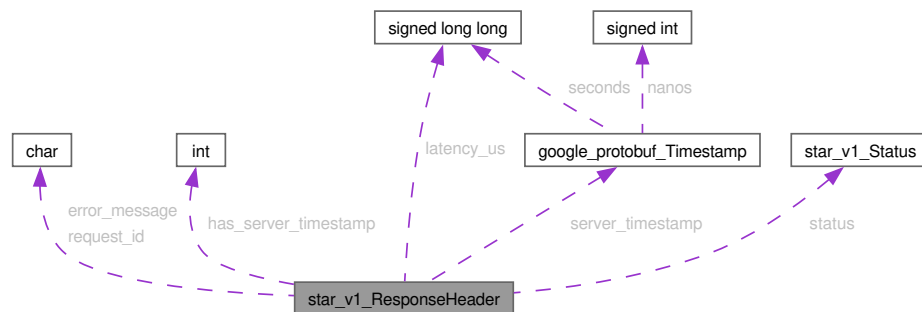
Data Fields

google_protobuf_Timestamp	client_timestamp	
char	client_version[32]	
bool	has_client_timestamp	
bool	has_timeout	
char	request_id[64]	
google_protobuf_Duration	timeout	

6.11.1.2 struct star_v1_ResponseHeader

Definition at line 55 of file [common.pb.h](#).

Collaboration diagram for star_v1_ResponseHeader:



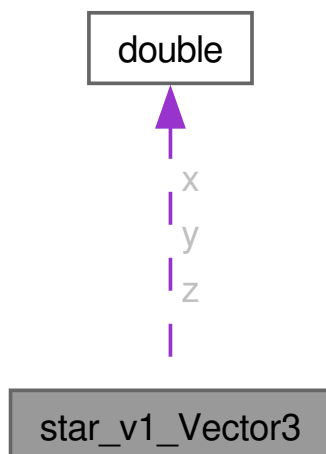
Data Fields

char	error_message[256]	
bool	has_server_timestamp	
int64_t	latency_us	
char	request_id[64]	
google_protobuf_Timestamp	server_timestamp	
star_v1_Status	status	

6.11.1.3 struct star_v1_Vector3

Definition at line 71 of file [common.pb.h](#).

Collaboration diagram for star_v1_Vector3:



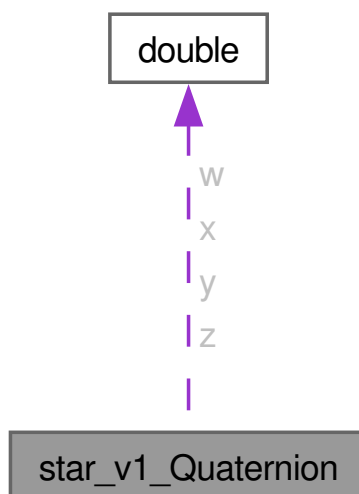
Data Fields

double	x	
double	y	
double	z	

6.11.1.4 struct star_v1_Quaternion

Definition at line 82 of file [common.pb.h](#).

Collaboration diagram for star_v1_Quaternion:



Data Fields

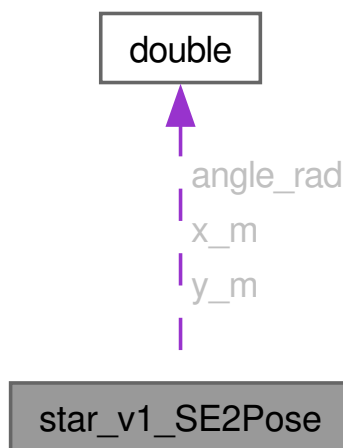
double	w	
double	x	
double	y	

double	z	
--------	---	--

6.11.1.5 struct star_v1_SE2Pose

Definition at line 95 of file [common.pb.h](#).

Collaboration diagram for star_v1_SE2Pose:



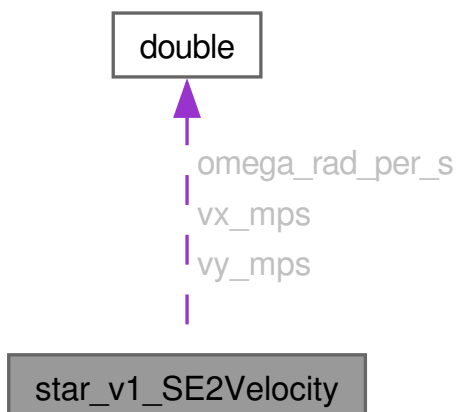
Data Fields

double	angle_rad	
double	x_m	
double	y_m	

6.11.1.6 struct star_v1_SE2Velocity

Definition at line 107 of file [common.pb.h](#).

Collaboration diagram for star_v1_SE2Velocity:



Data Fields

double	omega_rad_per_s	
double	vx_mps	
double	vy_mps	

6.11.2 Macro Definition Documentation

6.11.2.1 __star_v1_Status_ARRAYSIZE

```
#define __star_v1_Status_ARRAYSIZE ((star_v1_Status)(star_v1_Status_STATUS_ESTOP_ACTIVE+1))
```

Definition at line 125 of file [common.pb.h](#).

6.11.2.2 __star_v1_Status_MAX

```
#define __star_v1_Status_MAX star_v1_Status_STATUS_ESTOP_ACTIVE
```

Definition at line 124 of file [common.pb.h](#).

6.11.2.3 __star_v1_Status_MIN

```
#define __star_v1_Status_MIN star_v1_Status_STATUS_UNKNOWN
```

Definition at line 123 of file [common.pb.h](#).

6.11.2.4 star_v1_Quaternion_CALLBACK

```
#define star_v1_Quaternion_CALLBACK NULL
```

Definition at line 206 of file [common.pb.h](#).

6.11.2.5 star_v1_Quaternion_DEFAULT

```
#define star_v1_Quaternion_DEFAULT NULL
```

Definition at line 207 of file [common.pb.h](#).

6.11.2.6 star_v1_Quaternion_FIELDLIST

```
#define star_v1_Quaternion_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, DOUBLE, w, 1) \
X(a, STATIC, SINGULAR, DOUBLE, x, 2) \
X(a, STATIC, SINGULAR, DOUBLE, y, 3) \
X(a, STATIC, SINGULAR, DOUBLE, z, 4)
```

Definition at line 201 of file [common.pb.h](#).

```
00201 #define star_v1_Quaternion_FIELDLIST(X, a) \
00202 X(a, STATIC, SINGULAR, DOUBLE, w, 1) \
00203 X(a, STATIC, SINGULAR, DOUBLE, x, 2) \
00204 X(a, STATIC, SINGULAR, DOUBLE, y, 3) \
00205 X(a, STATIC, SINGULAR, DOUBLE, z, 4)
```

6.11.2.7 star_v1_Quaternion_fields

```
#define star_v1_Quaternion_fields &star_v1_Quaternion_msg
```

Definition at line 234 of file [common.pb.h](#).

6.11.2.8 star_v1_Quaternion_init_default

```
#define star_v1_Quaternion_init_default {0, 0, 0, 0}
```

Definition at line 139 of file [common.pb.h](#).

6.11.2.9 star_v1_Quaternion_init_zero

```
#define star_v1_Quaternion_init_zero {0, 0, 0, 0}
```

Definition at line 145 of file [common.pb.h](#).

6.11.2.10 star_v1_Quaternion_size

```
#define star_v1_Quaternion_size 36
```

Definition at line 240 of file [common.pb.h](#).

6.11.2.11 star_v1_Quaternion_w_tag

```
#define star_v1_Quaternion_w_tag 1
```

Definition at line 162 of file [common.pb.h](#).

6.11.2.12 star_v1_Quaternion_x_tag

```
#define star_v1_Quaternion_x_tag 2
```

Definition at line 163 of file [common.pb.h](#).

6.11.2.13 star_v1_Quaternion_y_tag

```
#define star_v1_Quaternion_y_tag 3
```

Definition at line 164 of file [common.pb.h](#).

6.11.2.14 star_v1_Quaternion_z_tag

```
#define star_v1_Quaternion_z_tag 4
```

Definition at line 165 of file [common.pb.h](#).

6.11.2.15 star_v1_RequestHeader_CALLBACK

```
#define star_v1_RequestHeader_CALLBACK NULL
```

Definition at line 179 of file [common.pb.h](#).

6.11.2.16 star_v1_RequestHeader_client_timestamp_MSGTYPE

```
#define star_v1_RequestHeader_client_timestamp_MSGTYPE google_protobuf_Timestamp
```

Definition at line 181 of file [common.pb.h](#).

6.11.2.17 star_v1_RequestHeader_client_timestamp_tag

```
#define star_v1_RequestHeader_client_timestamp_tag 2
```

Definition at line 151 of file [common.pb.h](#).

6.11.2.18 star_v1_RequestHeader_client_version_tag

```
#define star_v1_RequestHeader_client_version_tag 3
```

Definition at line 152 of file [common.pb.h](#).

6.11.2.19 star_v1_RequestHeader_DEFAULT

```
#define star_v1_RequestHeader_DEFAULT NULL
```

Definition at line 180 of file [common.pb.h](#).

6.11.2.20 star_v1_RequestHeader_FIELDLIST

```
#define star_v1_RequestHeader_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, STRING, request_id, 1) \  
X(a, STATIC, OPTIONAL, MESSAGE, client_timestamp, 2) \  
X(a, STATIC, SINGULAR, STRING, client_version, 3) \  
X(a, STATIC, OPTIONAL, MESSAGE, timeout, 4)
```

Definition at line 174 of file [common.pb.h](#).

```
00174 #define star_v1_RequestHeader_FIELDLIST(X, a) \  
00175 X(a, STATIC, SINGULAR, STRING, request_id, 1) \  
00176 X(a, STATIC, OPTIONAL, MESSAGE, client_timestamp, 2) \  
00177 X(a, STATIC, SINGULAR, STRING, client_version, 3) \  
00178 X(a, STATIC, OPTIONAL, MESSAGE, timeout, 4)
```

6.11.2.21 star_v1_RequestHeader_fields

```
#define star_v1_RequestHeader_fields &star_v1_RequestHeader_msg
```

Definition at line 231 of file [common.pb.h](#).

6.11.2.22 star_v1_RequestHeader_init_default

```
#define star_v1_RequestHeader_init_default { "", false, google_protobuf_Timestamp_init_default, "", false, google_protobuf_Duration_init_default }
```

Definition at line 136 of file [common.pb.h](#).

6.11.2.23 star_v1_RequestHeader_init_zero

```
#define star_v1_RequestHeader_init_zero { "", false, google_protobuf_Timestamp_init_zero, "", false, google_protobuf_Duration_init_zero }
```

Definition at line 142 of file [common.pb.h](#).

6.11.2.24 star_v1_RequestHeader_request_id_tag

```
#define star_v1_RequestHeader_request_id_tag 1
```

Definition at line 150 of file [common.pb.h](#).

6.11.2.25 star_v1_RequestHeader_size

```
#define star_v1_RequestHeader_size 146
```

Definition at line 241 of file [common.pb.h](#).

6.11.2.26 star_v1_RequestHeader_timeout_MSGTYPE

```
#define star_v1_RequestHeader_timeout_MSGTYPE google_protobuf_Duration
```

Definition at line 182 of file [common.pb.h](#).

6.11.2.27 star_v1_RequestHeader_timeout_tag

```
#define star_v1_RequestHeader_timeout_tag 4
```

Definition at line 153 of file [common.pb.h](#).

6.11.2.28 star_v1_ResponseHeader_CALLBACK

```
#define star_v1_ResponseHeader_CALLBACK NULL
```

Definition at line 190 of file [common.pb.h](#).

6.11.2.29 `star_v1_ResponseHeader_DEFAULT`

```
#define star_v1_ResponseHeader_DEFAULT NULL
```

Definition at line 191 of file [common.pb.h](#).

6.11.2.30 `star_v1_ResponseHeader_error_message_tag`

```
#define star_v1_ResponseHeader_error_message_tag 4
```

Definition at line 157 of file [common.pb.h](#).

6.11.2.31 `star_v1_ResponseHeader_FIELDLIST`

```
#define star_v1_ResponseHeader_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, STRING, request_id, 1) \  
X(a, STATIC, OPTIONAL, MESSAGE, server_timestamp, 2) \  
X(a, STATIC, SINGULAR, UENUM, status, 3) \  
X(a, STATIC, SINGULAR, STRING, error_message, 4) \  
X(a, STATIC, SINGULAR, INT64, latency_us, 5)
```

Definition at line 184 of file [common.pb.h](#).

```
00184 #define star_v1_ResponseHeader_FIELDLIST(X, a) \  
00185 X(a, STATIC, SINGULAR, STRING, request_id, 1) \  
00186 X(a, STATIC, OPTIONAL, MESSAGE, server_timestamp, 2) \  
00187 X(a, STATIC, SINGULAR, UENUM, status, 3) \  
00188 X(a, STATIC, SINGULAR, STRING, error_message, 4) \  
00189 X(a, STATIC, SINGULAR, INT64, latency_us, 5)
```

6.11.2.32 `star_v1_ResponseHeader_fields`

```
#define star_v1_ResponseHeader_fields &star_v1_ResponseHeader_msg
```

Definition at line 232 of file [common.pb.h](#).

6.11.2.33 `star_v1_ResponseHeader_init_default`

```
#define star_v1_ResponseHeader_init_default {"", false, google_protobuf_Timestamp_init_default, _star_v1_Status_MIN,  
"", 0}
```

Definition at line 137 of file [common.pb.h](#).

6.11.2.34 `star_v1_ResponseHeader_init_zero`

```
#define star_v1_ResponseHeader_init_zero {"", false, google_protobuf_Timestamp_init_zero, _star_v1_Status_MIN,  
"", 0}
```

Definition at line 143 of file [common.pb.h](#).

Referenced by [rx_nanopb_create_response_header\(\)](#).

6.11.2.35 star_v1_ResponseHeader_latency_us_tag

```
#define star_v1_ResponseHeader_latency_us_tag 5
```

Definition at line 158 of file [common.pb.h](#).

6.11.2.36 star_v1_ResponseHeader_request_id_tag

```
#define star_v1_ResponseHeader_request_id_tag 1
```

Definition at line 154 of file [common.pb.h](#).

6.11.2.37 star_v1_ResponseHeader_server_timestamp_MSGTYPE

```
#define star_v1_ResponseHeader_server_timestamp_MSGTYPE google_protobuf_Timestamp
```

Definition at line 192 of file [common.pb.h](#).

6.11.2.38 star_v1_ResponseHeader_server_timestamp_tag

```
#define star_v1_ResponseHeader_server_timestamp_tag 2
```

Definition at line 155 of file [common.pb.h](#).

6.11.2.39 star_v1_ResponseHeader_size

```
#define star_v1_ResponseHeader_size 360
```

Definition at line 242 of file [common.pb.h](#).

6.11.2.40 star_v1_ResponseHeader_status_ENUMTYPE

```
#define star_v1_ResponseHeader_status_ENUMTYPE star_v1_Status
```

Definition at line 128 of file [common.pb.h](#).

6.11.2.41 star_v1_ResponseHeader_status_tag

```
#define star_v1_ResponseHeader_status_tag 3
```

Definition at line 156 of file [common.pb.h](#).

6.11.2.42 star_v1_SE2Pose_angle_rad_tag

```
#define star_v1_SE2Pose_angle_rad_tag 3
```

Definition at line 168 of file [common.pb.h](#).

6.11.2.43 `star_v1_SE2Pose_CALLBACK`

```
#define star_v1_SE2Pose_CALLBACK NULL
```

Definition at line 213 of file [common.pb.h](#).

6.11.2.44 `star_v1_SE2Pose_DEFAULT`

```
#define star_v1_SE2Pose_DEFAULT NULL
```

Definition at line 214 of file [common.pb.h](#).

6.11.2.45 `star_v1_SE2Pose_FIELDLIST`

```
#define star_v1_SE2Pose_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, DOUBLE, x_m, 1) \  
X(a, STATIC, SINGULAR, DOUBLE, y_m, 2) \  
X(a, STATIC, SINGULAR, DOUBLE, angle_rad, 3)
```

Definition at line 209 of file [common.pb.h](#).

```
00209 #define star_v1_SE2Pose_FIELDLIST(X, a) \  
00210 X(a, STATIC, SINGULAR, DOUBLE, x_m, 1) \  
00211 X(a, STATIC, SINGULAR, DOUBLE, y_m, 2) \  
00212 X(a, STATIC, SINGULAR, DOUBLE, angle_rad, 3)
```

6.11.2.46 `star_v1_SE2Pose_fields`

```
#define star_v1_SE2Pose_fields &star_v1_SE2Pose_msg
```

Definition at line 235 of file [common.pb.h](#).

6.11.2.47 `star_v1_SE2Pose_init_default`

```
#define star_v1_SE2Pose_init_default {0, 0, 0}
```

Definition at line 140 of file [common.pb.h](#).

6.11.2.48 `star_v1_SE2Pose_init_zero`

```
#define star_v1_SE2Pose_init_zero {0, 0, 0}
```

Definition at line 146 of file [common.pb.h](#).

6.11.2.49 `star_v1_SE2Pose_size`

```
#define star_v1_SE2Pose_size 27
```

Definition at line 243 of file [common.pb.h](#).

6.11.2.50 star_v1_SE2Pose_x_m_tag

```
#define star_v1_SE2Pose_x_m_tag 1
```

Definition at line 166 of file [common.pb.h](#).

6.11.2.51 star_v1_SE2Pose_y_m_tag

```
#define star_v1_SE2Pose_y_m_tag 2
```

Definition at line 167 of file [common.pb.h](#).

6.11.2.52 star_v1_SE2Velocity_CALLBACK

```
#define star_v1_SE2Velocity_CALLBACK NULL
```

Definition at line 220 of file [common.pb.h](#).

6.11.2.53 star_v1_SE2Velocity_DEFAULT

```
#define star_v1_SE2Velocity_DEFAULT NULL
```

Definition at line 221 of file [common.pb.h](#).

6.11.2.54 star_v1_SE2Velocity_FIELDLIST

```
#define star_v1_SE2Velocity_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, DOUBLE, vx_mps, 1) \  
X(a, STATIC, SINGULAR, DOUBLE, vy_mps, 2) \  
X(a, STATIC, SINGULAR, DOUBLE, omega_rad_per_s, 3)
```

Definition at line 216 of file [common.pb.h](#).

```
00216 #define star_v1_SE2Velocity_FIELDLIST(X, a) \  
00217 X(a, STATIC, SINGULAR, DOUBLE, vx_mps, 1) \  
00218 X(a, STATIC, SINGULAR, DOUBLE, vy_mps, 2) \  
00219 X(a, STATIC, SINGULAR, DOUBLE, omega_rad_per_s, 3)
```

6.11.2.55 star_v1_SE2Velocity_fields

```
#define star_v1_SE2Velocity_fields &star_v1_SE2Velocity_msg
```

Definition at line 236 of file [common.pb.h](#).

6.11.2.56 star_v1_SE2Velocity_init_default

```
#define star_v1_SE2Velocity_init_default {0, 0, 0}
```

Definition at line 141 of file [common.pb.h](#).

6.11.2.57 `star_v1_SE2Velocity_init_zero`

```
#define star_v1_SE2Velocity_init_zero {0, 0, 0}
```

Definition at line 147 of file [common.pb.h](#).

6.11.2.58 `star_v1_SE2Velocity_omega_rad_per_s_tag`

```
#define star_v1_SE2Velocity_omega_rad_per_s_tag 3
```

Definition at line 171 of file [common.pb.h](#).

6.11.2.59 `star_v1_SE2Velocity_size`

```
#define star_v1_SE2Velocity_size 27
```

Definition at line 244 of file [common.pb.h](#).

6.11.2.60 `star_v1_SE2Velocity_vx_mps_tag`

```
#define star_v1_SE2Velocity_vx_mps_tag 1
```

Definition at line 169 of file [common.pb.h](#).

6.11.2.61 `star_v1_SE2Velocity_vy_mps_tag`

```
#define star_v1_SE2Velocity_vy_mps_tag 2
```

Definition at line 170 of file [common.pb.h](#).

6.11.2.62 `STAR_V1_STAR_V1_COMMON_PB_H_MAX_SIZE`

```
#define STAR_V1_STAR_V1_COMMON_PB_H_MAX_SIZE star_v1_ResponseHeader_size
```

Definition at line 239 of file [common.pb.h](#).

6.11.2.63 `star_v1_Vector3_CALLBACK`

```
#define star_v1_Vector3_CALLBACK NULL
```

Definition at line 198 of file [common.pb.h](#).

6.11.2.64 `star_v1_Vector3_DEFAULT`

```
#define star_v1_Vector3_DEFAULT NULL
```

Definition at line 199 of file [common.pb.h](#).

6.11.2.65 star_v1_Vector3_FIELDLIST

```
#define star_v1_Vector3_FIELDLIST(
    X,
    a)
```

Value:

```
X(a, STATIC, SINGULAR, DOUBLE, x,      1) \
X(a, STATIC, SINGULAR, DOUBLE, y,      2) \
X(a, STATIC, SINGULAR, DOUBLE, z,      3)
```

Definition at line 194 of file [common.pb.h](#).

```
00194 #define star_v1_Vector3_FIELDLIST(X, a) \
00195 X(a, STATIC, SINGULAR, DOUBLE, x,      1) \
00196 X(a, STATIC, SINGULAR, DOUBLE, y,      2) \
00197 X(a, STATIC, SINGULAR, DOUBLE, z,      3)
```

6.11.2.66 star_v1_Vector3_fields

```
#define star_v1_Vector3_fields &star_v1_Vector3_msg
```

Definition at line 233 of file [common.pb.h](#).

6.11.2.67 star_v1_Vector3_init_default

```
#define star_v1_Vector3_init_default {0, 0, 0}
```

Definition at line 138 of file [common.pb.h](#).

6.11.2.68 star_v1_Vector3_init_zero

```
#define star_v1_Vector3_init_zero {0, 0, 0}
```

Definition at line 144 of file [common.pb.h](#).

6.11.2.69 star_v1_Vector3_size

```
#define star_v1_Vector3_size 27
```

Definition at line 245 of file [common.pb.h](#).

6.11.2.70 star_v1_Vector3_x_tag

```
#define star_v1_Vector3_x_tag 1
```

Definition at line 159 of file [common.pb.h](#).

6.11.2.71 star_v1_Vector3_y_tag

```
#define star_v1_Vector3_y_tag 2
```

Definition at line 160 of file [common.pb.h](#).

6.11.2.72 star_v1_Vector3_z_tag

```
#define star_v1_Vector3_z_tag 3
```

Definition at line 161 of file [common.pb.h](#).

6.11.3 Enumeration Type Documentation

6.11.3.1 star_v1_Status

```
enum star_v1_Status
```

Enumerator

star_v1_Status_STATUS_UNKNOWN	
star_v1_Status_STATUS_OK	
star_v1_Status_STATUS_INVALID_REQUEST	
star_v1_Status_STATUS_INTERNAL_ERROR	
star_v1_Status_STATUS_NOT_FOUND	
star_v1_Status_STATUS_TIMEOUT	
star_v1_Status_STATUS_INVALID_STATE	
star_v1_Status_STATUS_ESTOP_ACTIVE	

Definition at line 17 of file [common.pb.h](#).

```
00017     {
00018     /* Unknown or unspecified status. Treat as error. */
00019     star_v1_Status_STATUS_UNKNOWN = 0,
00020     /* Request processed successfully. */
00021     star_v1_Status_STATUS_OK = 1,
00022     /* Request contained invalid parameters. */
00023     star_v1_Status_STATUS_INVALID_REQUEST = 2,
00024     /* Internal server error occurred. */
00025     star_v1_Status_STATUS_INTERNAL_ERROR = 3,
00026     /* Requested resource not found. */
00027     star_v1_Status_STATUS_NOT_FOUND = 4,
00028     /* Operation timed out. */
00029     star_v1_Status_STATUS_TIMEOUT = 5,
00030     /* Robot is in an invalid state for this operation. */
00031     star_v1_Status_STATUS_INVALID_STATE = 6,
00032     /* Emergency stop is active. */
00033     star_v1_Status_STATUS_ESTOP_ACTIVE = 7
00034 } star_v1_Status;
```

6.11.4 Variable Documentation

6.11.4.1 star_v1_Quaternion_msg

```
const pb_msgdesc_t star_v1_Quaternion_msg [extern]
```

6.11.4.2 star_v1_RequestHeader_msg

```
const pb_msgdesc_t star_v1_RequestHeader_msg [extern]
```

6.11.4.3 star_v1_ResponseHeader_msg

```
const pb_msgdesc_t star_v1_ResponseHeader_msg [extern]
```

6.11.4.4 star_v1_SE2Pose_msg

```
const pb_msgdesc_t star_v1_SE2Pose_msg [extern]
```

6.11.4.5 star_v1_SE2Velocity_msg

```
const pb_msgdesc_t star_v1_SE2Velocity_msg [extern]
```

6.11.4.6 star_v1_Vector3_msg

```
const pb_msgdesc_t star_v1_Vector3_msg [extern]
```

6.12 common.pb.h

[Go to the documentation of this file.](#)

```
00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_STAR_V1_STAR_V1_COMMON_PB_H_INCLUDED
00005 #define PB_STAR_V1_STAR_V1_COMMON_PB_H_INCLUDED
00006 #include <pb.h>
00007 #include "google/protobuf/duration.pb.h"
00008 #include "google/protobuf/timestamp.pb.h"
00009
00010 #if PB_PROTO_HEADER_VERSION != 40
00011 #error Regenerate this file with the current version of nanopb generator.
00012 #endif
00013
00014 /* Enum definitions */
00015 /* Response status codes following Boston Dynamics pattern.
00016 Zero value is STATUS_UNKNOWN per style guide. */
00017 typedef enum _star_v1_Status {
00018     /* Unknown or unspecified status. Treat as error. */
00019     star_v1_Status_STATUS_UNKNOWN = 0,
00020     /* Request processed successfully. */
00021     star_v1_Status_STATUS_OK = 1,
00022     /* Request contained invalid parameters. */
00023     star_v1_Status_STATUS_INVALID_REQUEST = 2,
00024     /* Internal server error occurred. */
00025     star_v1_Status_STATUS_INTERNAL_ERROR = 3,
00026     /* Requested resource not found. */
00027     star_v1_Status_STATUS_NOT_FOUND = 4,
00028     /* Operation timed out. */
00029     star_v1_Status_STATUS_TIMEOUT = 5,
00030     /* Robot is in an invalid state for this operation. */
00031     star_v1_Status_STATUS_INVALID_STATE = 6,
00032     /* Emergency stop is active. */
00033     star_v1_Status_STATUS_ESTOP_ACTIVE = 7
00034 } star_v1_Status;
00035
00036 /* Struct definitions */
00037 /* Standard request header included in all requests.
00038 Provides tracing, versioning, and client identification. */
00039 typedef struct _star_v1_RequestHeader {
00040     /* Client-generated request identifier for tracing.
00041     Should be unique per request (UUID recommended). */
00042     char request_id[64];
00043     /* Client timestamp when request was created. */
00044     bool has_client_timestamp;
00045     google_protobuf_Timestamp client_timestamp;
00046     /* Client software version string (e.g., "1.0.0"). */
00047     char client_version[32];
```

```

00048  /* Optional deadline for request processing. */
00049  bool has_timeout;
00050  google_protobuf_Duration timeout;
00051 } star_v1_RequestHeader;
00052
00053 /* Standard response header included in all responses.
00054 Provides status, timing, and error information. */
00055 typedef struct _star_v1_ResponseHeader {
00056  /* Echo of the request_id from RequestHeader. */
00057  char request_id[64];
00058  /* Server timestamp when response was generated. */
00059  bool has_server_timestamp;
00060  google_protobuf_Timestamp server_timestamp;
00061  /* Processing status. */
00062  star_v1_Status status;
00063  /* Human-readable error message if status is not STATUS_OK. */
00064  char error_message[256];
00065  /* Round-trip latency in microseconds. */
00066  int64_t latency_us;
00067 } star_v1_ResponseHeader;
00068
00069 /* 3D vector with MKS units.
00070 Usage context determines units (meters for position, m/s for velocity). */
00071 typedef struct _star_v1_Vector3 {
00072  /* X component. */
00073  double x;
00074  /* Y component. */
00075  double y;
00076  /* Z component. */
00077  double z;
00078 } star_v1_Vector3;
00079
00080 /* Quaternion for orientation representation.
00081 Normalized quaternion (w, x, y, z) with w as scalar component. */
00082 typedef struct _star_v1_Quaternion {
00083  /* W component (scalar part). */
00084  double w;
00085  /* X component. */
00086  double x;
00087  /* Y component. */
00088  double y;
00089  /* Z component. */
00090  double z;
00091 } star_v1_Quaternion;
00092
00093 /* SE2 pose for 2D navigation (position + heading).
00094 Uses MKS units: meters for position, radians for angle. */
00095 typedef struct _star_v1_SE2Pose {
00096  /* X position in meters. */
00097  double x_m;
00098  /* Y position in meters. */
00099  double y_m;
00100  /* Heading angle in radians.
00101 Range: -pi to pi, 0 = facing positive X axis. */
00102  double angle_rad;
00103 } star_v1_SE2Pose;
00104
00105 /* SE2 velocity for 2D motion.
00106 Uses MKS units: m/s for linear, rad/s for angular. */
00107 typedef struct _star_v1_SE2Velocity {
00108  /* Linear velocity X in meters per second. */
00109  double vx_mps;
00110  /* Linear velocity Y in meters per second. */
00111  double vy_mps;
00112  /* Angular velocity in radians per second.
00113 Positive = counter-clockwise. */
00114  double omega_rad_per_s;
00115 } star_v1_SE2Velocity;
00116
00117
00118 #ifdef __cplusplus
00119 extern "C" {
00120 #endif
00121
00122 /* Helper constants for enums */
00123 #define _star_v1_Status_MIN star_v1_Status_STATUS_UNKNOWN
00124 #define _star_v1_Status_MAX star_v1_Status_STATUS_STOP_ACTIVE
00125 #define _star_v1_Status_ARRAYSIZE ((star_v1_Status)(star_v1_Status_STATUS_STOP_ACTIVE+1))
00126
00127
00128 #define star_v1_ResponseHeader_status_ENUMTYPE star_v1_Status
00129
00130
00131
00132
00133
00134

```

```

00135 /* Initializer values for message structs */
00136 #define star_v1_RequestHeader_init_default {"", false, google_protobuf_Timestamp_init_default, "", false,
google_protobuf_Duration_init_default}
00137 #define star_v1_ResponseHeader_init_default {"", false, google_protobuf_Timestamp_init_default,
star_v1_Status_MIN, "", 0}
00138 #define star_v1_Vector3_init_default {0, 0, 0}
00139 #define star_v1_Quaternion_init_default {0, 0, 0, 0}
00140 #define star_v1_SE2Pose_init_default {0, 0, 0}
00141 #define star_v1_SE2Velocity_init_default {0, 0, 0}
00142 #define star_v1_RequestHeader_init_zero {"", false, google_protobuf_Timestamp_init_zero, "", false,
google_protobuf_Duration_init_zero}
00143 #define star_v1_ResponseHeader_init_zero {"", false, google_protobuf_Timestamp_init_zero,
star_v1_Status_MIN, "", 0}
00144 #define star_v1_Vector3_init_zero {0, 0, 0}
00145 #define star_v1_Quaternion_init_zero {0, 0, 0, 0}
00146 #define star_v1_SE2Pose_init_zero {0, 0, 0}
00147 #define star_v1_SE2Velocity_init_zero {0, 0, 0}
00148
00149 /* Field tags (for use in manual encoding/decoding) */
00150 #define star_v1_RequestHeader_request_id_tag 1
00151 #define star_v1_RequestHeader_client_timestamp_tag 2
00152 #define star_v1_RequestHeader_client_version_tag 3
00153 #define star_v1_RequestHeader_timeout_tag 4
00154 #define star_v1_ResponseHeader_request_id_tag 1
00155 #define star_v1_ResponseHeader_server_timestamp_tag 2
00156 #define star_v1_ResponseHeader_status_tag 3
00157 #define star_v1_ResponseHeader_error_message_tag 4
00158 #define star_v1_ResponseHeader_latency_us_tag 5
00159 #define star_v1_Vector3_x_tag 1
00160 #define star_v1_Vector3_y_tag 2
00161 #define star_v1_Vector3_z_tag 3
00162 #define star_v1_Quaternion_w_tag 1
00163 #define star_v1_Quaternion_x_tag 2
00164 #define star_v1_Quaternion_y_tag 3
00165 #define star_v1_Quaternion_z_tag 4
00166 #define star_v1_SE2Pose_x_m_tag 1
00167 #define star_v1_SE2Pose_y_m_tag 2
00168 #define star_v1_SE2Pose_angle_rad_tag 3
00169 #define star_v1_SE2Velocity_vx_mps_tag 1
00170 #define star_v1_SE2Velocity_vy_mps_tag 2
00171 #define star_v1_SE2Velocity_omega_rad_per_s_tag 3
00172
00173 /* Struct field encoding specification for nanopb */
00174 #define star_v1_RequestHeader_FIELDLIST(X, a) \
00175 X(a, STATIC, SINGULAR, STRING, request_id, 1) \
00176 X(a, STATIC, OPTIONAL, MESSAGE, client_timestamp, 2) \
00177 X(a, STATIC, SINGULAR, STRING, client_version, 3) \
00178 X(a, STATIC, OPTIONAL, MESSAGE, timeout, 4)
00179 #define star_v1_RequestHeader_CALLBACK NULL
00180 #define star_v1_RequestHeader_DEFAULT NULL
00181 #define star_v1_RequestHeader_client_timestamp_MSGTYPE google_protobuf_Timestamp
00182 #define star_v1_RequestHeader_timeout_MSGTYPE google_protobuf_Duration
00183
00184 #define star_v1_ResponseHeader_FIELDLIST(X, a) \
00185 X(a, STATIC, SINGULAR, STRING, request_id, 1) \
00186 X(a, STATIC, OPTIONAL, MESSAGE, server_timestamp, 2) \
00187 X(a, STATIC, SINGULAR, UENUM, status, 3) \
00188 X(a, STATIC, SINGULAR, STRING, error_message, 4) \
00189 X(a, STATIC, SINGULAR, INT64, latency_us, 5)
00190 #define star_v1_ResponseHeader_CALLBACK NULL
00191 #define star_v1_ResponseHeader_DEFAULT NULL
00192 #define star_v1_ResponseHeader_server_timestamp_MSGTYPE google_protobuf_Timestamp
00193
00194 #define star_v1_Vector3_FIELDLIST(X, a) \
00195 X(a, STATIC, SINGULAR, DOUBLE, x, 1) \
00196 X(a, STATIC, SINGULAR, DOUBLE, y, 2) \
00197 X(a, STATIC, SINGULAR, DOUBLE, z, 3)
00198 #define star_v1_Vector3_CALLBACK NULL
00199 #define star_v1_Vector3_DEFAULT NULL
00200
00201 #define star_v1_Quaternion_FIELDLIST(X, a) \
00202 X(a, STATIC, SINGULAR, DOUBLE, w, 1) \
00203 X(a, STATIC, SINGULAR, DOUBLE, x, 2) \
00204 X(a, STATIC, SINGULAR, DOUBLE, y, 3) \
00205 X(a, STATIC, SINGULAR, DOUBLE, z, 4)
00206 #define star_v1_Quaternion_CALLBACK NULL
00207 #define star_v1_Quaternion_DEFAULT NULL
00208
00209 #define star_v1_SE2Pose_FIELDLIST(X, a) \
00210 X(a, STATIC, SINGULAR, DOUBLE, x_m, 1) \
00211 X(a, STATIC, SINGULAR, DOUBLE, y_m, 2) \
00212 X(a, STATIC, SINGULAR, DOUBLE, angle_rad, 3)
00213 #define star_v1_SE2Pose_CALLBACK NULL
00214 #define star_v1_SE2Pose_DEFAULT NULL
00215
00216 #define star_v1_SE2Velocity_FIELDLIST(X, a) \
00217 X(a, STATIC, SINGULAR, DOUBLE, vx_mps, 1) \

```

```

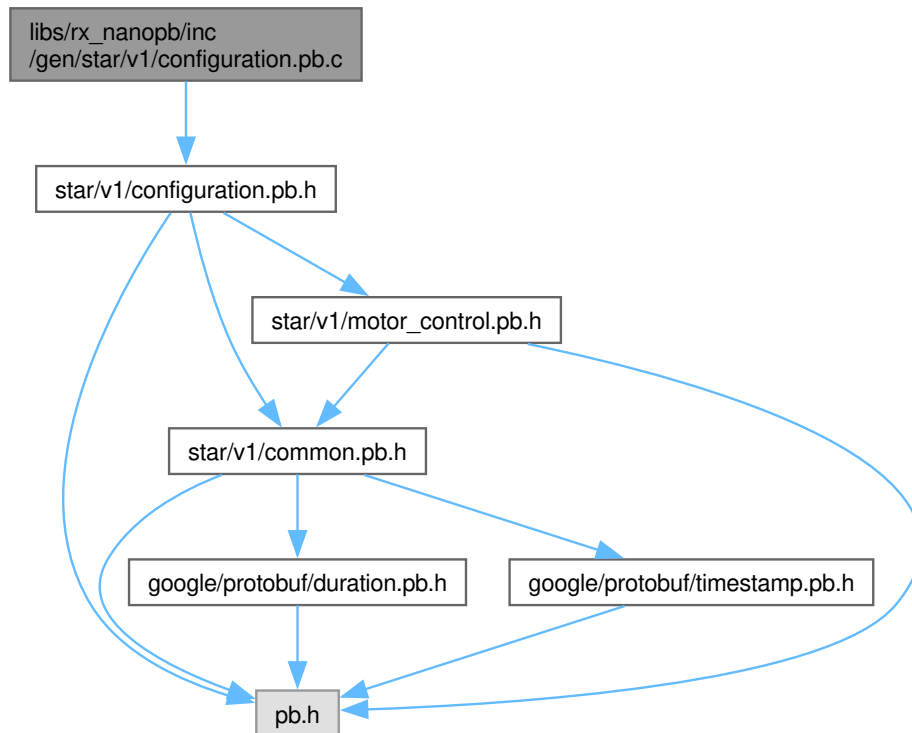
00218 X(a, STATIC, SINGULAR, DOUBLE, vy_mps, 2) \
00219 X(a, STATIC, SINGULAR, DOUBLE, omega_rad_per_s, 3)
00220 #define star_v1_SE2Velocity_CALLBACK NULL
00221 #define star_v1_SE2Velocity_DEFAULT NULL
00222
00223 extern const pb_msgdesc_t star_v1_RequestHeader_msg;
00224 extern const pb_msgdesc_t star_v1_ResponseHeader_msg;
00225 extern const pb_msgdesc_t star_v1_Vector3_msg;
00226 extern const pb_msgdesc_t star_v1_Quaternion_msg;
00227 extern const pb_msgdesc_t star_v1_SE2Pose_msg;
00228 extern const pb_msgdesc_t star_v1_SE2Velocity_msg;
00229
00230 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00231 #define star_v1_RequestHeader_fields &star_v1_RequestHeader_msg
00232 #define star_v1_ResponseHeader_fields &star_v1_ResponseHeader_msg
00233 #define star_v1_Vector3_fields &star_v1_Vector3_msg
00234 #define star_v1_Quaternion_fields &star_v1_Quaternion_msg
00235 #define star_v1_SE2Pose_fields &star_v1_SE2Pose_msg
00236 #define star_v1_SE2Velocity_fields &star_v1_SE2Velocity_msg
00237
00238 /* Maximum encoded size of messages (where known) */
00239 #define STAR_V1_STAR_V1_COMMON_PB_H_MAX_SIZE star_v1_ResponseHeader_size
00240 #define star_v1_Quaternion_size 36
00241 #define star_v1_RequestHeader_size 146
00242 #define star_v1_ResponseHeader_size 360
00243 #define star_v1_SE2Pose_size 27
00244 #define star_v1_SE2Velocity_size 27
00245 #define star_v1_Vector3_size 27
00246
00247 #ifdef __cplusplus
00248 } /* extern "C" */
00249 #endif
00250
00251 #endif

```

6.13 libs/rx_nanopb/inc/gen/star/v1/configuration.pb.c File Reference

#include "star/v1/configuration.pb.h"

Include dependency graph for configuration.pb.c:



6.14 configuration.pb.c

[Go to the documentation of this file.](#)

```

00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "star/v1/configuration.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40
00006 #error Regenerate this file with the current version of nanopb generator.
00007 #endif
00008
00009 PB_BIND(star_v1_GetConfigurationRequest, star_v1_GetConfigurationRequest, AUTO)
00010
00011
00012 PB_BIND(star_v1_GetConfigurationResponse, star_v1_GetConfigurationResponse, 2)
00013
00014
00015 PB_BIND(star_v1_SetConfigurationRequest, star_v1_SetConfigurationRequest, 2)
00016
00017
00018 PB_BIND(star_v1_SetConfigurationResponse, star_v1_SetConfigurationResponse, 4)
00019
00020
00021 PB_BIND(star_v1_ResetToDefaultsRequest, star_v1_ResetToDefaultsRequest, AUTO)
00022
00023
00024 PB_BIND(star_v1_ResetToDefaultsResponse, star_v1_ResetToDefaultsResponse, 2)
00025
00026
00027 PB_BIND(star_v1_ValidateConfigurationRequest, star_v1_ValidateConfigurationRequest, 2)
00028
00029
00030 PB_BIND(star_v1_ValidateConfigurationResponse, star_v1_ValidateConfigurationResponse, 4)
00031
00032
00033 PB_BIND(star_v1_SaveConfigurationRequest, star_v1_SaveConfigurationRequest, AUTO)
00034
00035
00036 PB_BIND(star_v1_SaveConfigurationResponse, star_v1_SaveConfigurationResponse, 2)
00037
00038
00039 PB_BIND(star_v1_GetMotorPidConfigRequest, star_v1_GetMotorPidConfigRequest, AUTO)
00040
00041
00042 PB_BIND(star_v1_GetMotorPidConfigResponse, star_v1_GetMotorPidConfigResponse, 2)
00043
00044
00045 PB_BIND(star_v1_SetMotorPidConfigRequest, star_v1_SetMotorPidConfigRequest, AUTO)
00046
00047
00048 PB_BIND(star_v1_SetMotorPidConfigResponse, star_v1_SetMotorPidConfigResponse, 4)
00049
00050
00051 PB_BIND(star_v1_RetransmitConfig, star_v1_RetransmitConfig, AUTO)
00052
00053
00054 PB_BIND(star_v1_SetRetransmitConfigRequest, star_v1_SetRetransmitConfigRequest, AUTO)
00055
00056
00057 PB_BIND(star_v1_SetRetransmitConfigResponse, star_v1_SetRetransmitConfigResponse, 2)
00058
00059
00060 PB_BIND(star_v1_SystemConfiguration, star_v1_SystemConfiguration, 2)
00061
00062
00063 PB_BIND(star_v1_MotorPidConfiguration, star_v1_MotorPidConfiguration, AUTO)
00064
00065
00066 PB_BIND(star_v1_CurrentSensorCalibration, star_v1_CurrentSensorCalibration, AUTO)
00067
00068
00069 PB_BIND(star_v1_TemperatureCalibration, star_v1_TemperatureCalibration, AUTO)
00070
00071
00072 PB_BIND(star_v1_SafetyThresholds, star_v1_SafetyThresholds, AUTO)
00073
00074
00075 PB_BIND(star_v1_EncoderConfiguration, star_v1_EncoderConfiguration, AUTO)
00076
00077
00078 PB_BIND(star_v1_TimingConfiguration, star_v1_TimingConfiguration, AUTO)
00079
00080
00081 PB_BIND(star_v1_ConfigValidationResult, star_v1_ConfigValidationResult, 4)
00082

```

```

00083
00084 PB_BIND(star_v1_ConfigValidationError, star_v1_ConfigValidationError, 2)
00085
00086
00087
00088
00089
00090
00091
00092 #ifndef PB_CONVERT_DOUBLE_FLOAT
00093 /* On some platforms (such as AVR), double is really float.
00094  * To be able to encode/decode double on these platforms, you need.
00095  * to define PB_CONVERT_DOUBLE_FLOAT in pb.h or compiler command line.
00096  */
00097 PB_STATIC_ASSERT(sizeof(double) == 8, DOUBLE_MUST_BE_8_BYTES)
00098 #endif
00099

```

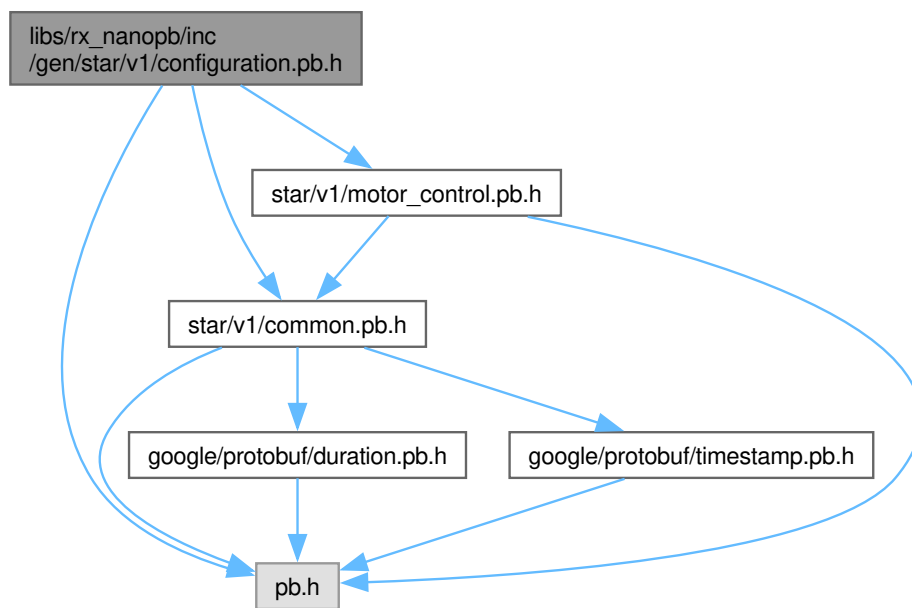
6.15 libs/rx_nanopb/inc/gen/star/v1/configuration.pb.h File Reference

```

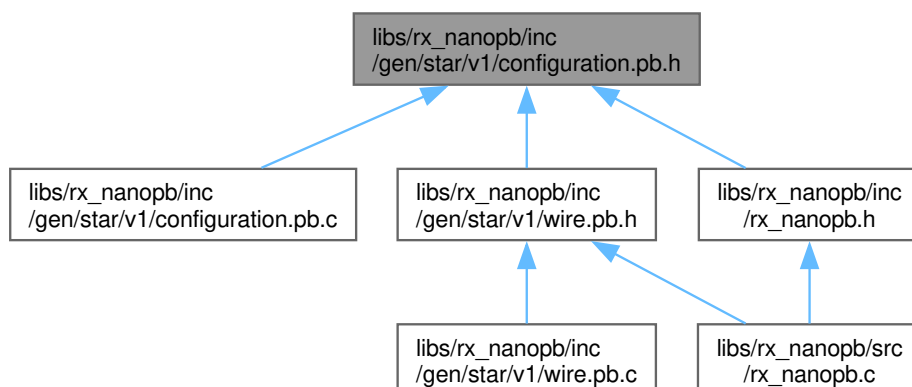
#include <pb.h>
#include "star/v1/common.pb.h"
#include "star/v1/motor_control.pb.h"

```

Include dependency graph for configuration.pb.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [star_v1_GetConfigurationRequest](#)
- struct [star_v1_ResetToDefaultsRequest](#)
- struct [star_v1_SaveConfigurationRequest](#)
- struct [star_v1_SaveConfigurationResponse](#)
- struct [star_v1_GetMotorPidConfigRequest](#)
- struct [star_v1_GetMotorPidConfigResponse](#)
- struct [star_v1_SetMotorPidConfigRequest](#)
- struct [star_v1_RetransmitConfig](#)
- struct [star_v1_SetRetransmitConfigRequest](#)
- struct [star_v1_SetRetransmitConfigResponse](#)
- struct [star_v1_MotorPidConfiguration](#)
- struct [star_v1_CurrentSensorCalibration](#)
- struct [star_v1_TemperatureCalibration](#)
- struct [star_v1_SafetyThresholds](#)
- struct [star_v1_EncoderConfiguration](#)
- struct [star_v1_TimingConfiguration](#)
- struct [star_v1_SystemConfiguration](#)
- struct [star_v1_GetConfigurationResponse](#)
- struct [star_v1_SetConfigurationRequest](#)
- struct [star_v1_ResetToDefaultsResponse](#)
- struct [star_v1_ValidateConfigurationRequest](#)
- struct [star_v1_ConfigValidationResult](#)
- struct [star_v1_SetConfigurationResponse](#)
- struct [star_v1_ValidateConfigurationResponse](#)
- struct [star_v1_SetMotorPidConfigResponse](#)

Macros

- `#define star_v1_ConfigValidationStatus_MIN star_v1_ConfigValidationStatus_CONFIG_VALIDATION_ST`
- `#define star_v1_ConfigValidationStatus_MAX star_v1_ConfigValidationStatus_CONFIG_VALIDATION_ST`
- `#define star_v1_ConfigValidationStatus_ARRAYSIZE ((star_v1_ConfigValidationStatus)(star_v1_ConfigVali`
- `#define star_v1_ConfigErrorCode_MIN star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_UNKNOWN`
- `#define star_v1_ConfigErrorCode_MAX star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_STORAGE`
- `#define star_v1_ConfigErrorCode_ARRAYSIZE ((star_v1_ConfigErrorCode)(star_v1_ConfigErrorCode_CO`
- `#define star_v1_ConfigValidationResult_status_ENUMTYPE star_v1_ConfigValidationStatus`
- `#define star_v1_ConfigValidationResult_error_code_ENUMTYPE star_v1_ConfigErrorCode`
- `#define star_v1_GetConfigurationRequest_init_default {false, star_v1_RequestHeader_init_default}`
- `#define star_v1_GetConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default,`
`false, star_v1_SystemConfiguration_init_default}`
- `#define star_v1_SetConfigurationRequest_init_default {false, star_v1_RequestHeader_init_default,`
`false, star_v1_SystemConfiguration_init_default, 0}`
- `#define star_v1_SetConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default,`
`false, star_v1_ConfigValidationResult_init_default}`
- `#define star_v1_ResetToDefaultsRequest_init_default {false, star_v1_RequestHeader_init_default,`
`0}`
- `#define star_v1_ResetToDefaultsResponse_init_default {false, star_v1_ResponseHeader_init_default,`
`false, star_v1_SystemConfiguration_init_default}`
- `#define star_v1_ValidateConfigurationRequest_init_default {false, star_v1_RequestHeader_init_default,`
`false, star_v1_SystemConfiguration_init_default}`
- `#define star_v1_ValidateConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default,`
`false, star_v1_ConfigValidationResult_init_default}`

- `#define star_v1_SaveConfigurationRequest_init_default {false, star_v1_RequestHeader_init_default}`
- `#define star_v1_SaveConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0}`
- `#define star_v1_GetMotorPidConfigRequest_init_default {false, star_v1_RequestHeader_init_default, 0}`
- `#define star_v1_GetMotorPidConfigResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, false, star_v1_PidConfig_init_default}`
- `#define star_v1_SetMotorPidConfigRequest_init_default {false, star_v1_RequestHeader_init_default, 0, false, star_v1_PidConfig_init_default, 0}`
- `#define star_v1_SetMotorPidConfigResponse_init_default {false, star_v1_ResponseHeader_init_default, false, star_v1_ConfigValidationResult_init_default}`
- `#define star_v1_RetransmitConfig_init_default {0, 0, 0, 0}`
- `#define star_v1_SetRetransmitConfigRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_RetransmitConfig_init_default}`
- `#define star_v1_SetRetransmitConfigResponse_init_default {false, star_v1_ResponseHeader_init_default}`
- `#define star_v1_SystemConfiguration_init_default {0, {star_v1_MotorPidConfiguration_init_default, star_v1_MotorPidConfiguration_init_default, star_v1_MotorPidConfiguration_init_default}, false, star_v1_CurrentSensorCalibration_init_default, false, star_v1_TemperatureCalibration_init_default, false, star_v1_SafetyThresholds_init_default, false, star_v1_EncoderConfiguration_init_default, false, star_v1_TimingConfiguration_init_default, 0, 0}`
- `#define star_v1_MotorPidConfiguration_init_default {0, false, star_v1_PidConfig_init_default}`
- `#define star_v1_CurrentSensorCalibration_init_default {{{NULL}, NULL}, {{{NULL}, NULL}}}`
- `#define star_v1_TemperatureCalibration_init_default {0}`
- `#define star_v1_SafetyThresholds_init_default {0, 0, 0}`
- `#define star_v1_EncoderConfiguration_init_default {0, 0, 0}`
- `#define star_v1_TimingConfiguration_init_default {0, 0, 0}`
- `#define star_v1_ConfigValidationResult_init_default {star_v1_ConfigValidationStatus_MIN, 0, {star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default}}`
- `#define star_v1_ConfigValidationError_init_default {"" ,star_v1_ConfigErrorCode_MIN, "", "", ""}`
- `#define star_v1_GetConfigurationRequest_init_zero {false, star_v1_RequestHeader_init_zero}`
- `#define star_v1_GetConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_SystemConfiguration_init_zero}`
- `#define star_v1_SetConfigurationRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_SystemConfiguration_init_zero, 0}`
- `#define star_v1_SetConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_ConfigValidationResult_init_zero}`
- `#define star_v1_ResetToDefaultsRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}`
- `#define star_v1_ResetToDefaultsResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_SystemConfiguration_init_zero}`
- `#define star_v1_ValidateConfigurationRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_SystemConfiguration_init_zero}`
- `#define star_v1_ValidateConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_ConfigValidationResult_init_zero}`
- `#define star_v1_SaveConfigurationRequest_init_zero {false, star_v1_RequestHeader_init_zero}`
- `#define star_v1_SaveConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0}`
- `#define star_v1_GetMotorPidConfigRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}`
- `#define star_v1_GetMotorPidConfigResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, false, star_v1_PidConfig_init_zero}`

- ```

#define star_v1_SetMotorPidConfigRequest_init_zero {false, star_v1_RequestHeader_init_zero,
0, false, star_v1_PidConfig_init_zero, 0}
• #define star_v1_SetMotorPidConfigResponse_init_zero {false, star_v1_ResponseHeader_init_zero,
false, star_v1_ConfigValidationResult_init_zero}
• #define star_v1_RetransmitConfig_init_zero {0, 0, 0, 0}
• #define star_v1_SetRetransmitConfigRequest_init_zero {false, star_v1_RequestHeader_init_zero,
false, star_v1_RetransmitConfig_init_zero}
• #define star_v1_SetRetransmitConfigResponse_init_zero {false, star_v1_ResponseHeader_init_zero}
• #define star_v1_SystemConfiguration_init_zero {0, {star_v1_MotorPidConfiguration_init_zero,
star_v1_MotorPidConfiguration_init_zero, star_v1_MotorPidConfiguration_init_zero, star_v1_MotorPidConfiguration_init_zero,
false, star_v1_CurrentSensorCalibration_init_zero, false, star_v1_TemperatureCalibration_init_zero,
false, star_v1_SafetyThresholds_init_zero, false, star_v1_EncoderConfiguration_init_zero,
false, star_v1_TimingConfiguration_init_zero, 0, 0}
• #define star_v1_MotorPidConfiguration_init_zero {0, false, star_v1_PidConfig_init_zero}
• #define star_v1_CurrentSensorCalibration_init_zero {{{NULL}, NULL}, {{{NULL}, NULL}}}
• #define star_v1_TemperatureCalibration_init_zero {0}
• #define star_v1_SafetyThresholds_init_zero {0, 0, 0}
• #define star_v1_EncoderConfiguration_init_zero {0, 0, 0}
• #define star_v1_TimingConfiguration_init_zero {0, 0, 0}
• #define star_v1_ConfigValidationResult_init_zero {_star_v1_ConfigValidationStatus_MIN,
0, {star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero,
star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero,
star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero}
• #define star_v1_ConfigValidationError_init_zero {"", _star_v1_ConfigErrorCode_MIN, "",
"", ""}
• #define star_v1_GetConfigurationRequest_header_tag 1
• #define star_v1_ResetToDefaultsRequest_header_tag 1
• #define star_v1_ResetToDefaultsRequest_persist_to_nvs_tag 2
• #define star_v1_SaveConfigurationRequest_header_tag 1
• #define star_v1_SaveConfigurationResponse_header_tag 1
• #define star_v1_SaveConfigurationResponse_saved_tag 2
• #define star_v1_SaveConfigurationResponse_config_crc_tag 3
• #define star_v1_GetMotorPidConfigRequest_header_tag 1
• #define star_v1_GetMotorPidConfigRequest_motor_id_tag 2
• #define star_v1_GetMotorPidConfigResponse_header_tag 1
• #define star_v1_GetMotorPidConfigResponse_motor_id_tag 2
• #define star_v1_GetMotorPidConfigResponse_pid_config_tag 3
• #define star_v1_SetMotorPidConfigRequest_header_tag 1
• #define star_v1_SetMotorPidConfigRequest_motor_id_tag 2
• #define star_v1_SetMotorPidConfigRequest_pid_config_tag 3
• #define star_v1_SetMotorPidConfigRequest_persist_to_nvs_tag 4
• #define star_v1_RetransmitConfig_enabled_tag 1
• #define star_v1_RetransmitConfig_max_retries_tag 2
• #define star_v1_RetransmitConfig_ack_timeout_ms_tag 3
• #define star_v1_RetransmitConfig_max_backoff_ms_tag 4
• #define star_v1_SetRetransmitConfigRequest_header_tag 1
• #define star_v1_SetRetransmitConfigRequest_retransmit_config_tag 2
• #define star_v1_SetRetransmitConfigResponse_header_tag 1
• #define star_v1_MotorPidConfiguration_motor_id_tag 1
• #define star_v1_MotorPidConfiguration_pid_config_tag 2
• #define star_v1_CurrentSensorCalibration_offset_ma_tag 1
• #define star_v1_CurrentSensorCalibration_scale_factor_tag 2
• #define star_v1_TemperatureCalibration_offset_celsius_tag 1
• #define star_v1_SafetyThresholds_overcurrent_threshold_ma_tag 1
• #define star_v1_SafetyThresholds_thermal_shutdown_deci_celsius_tag 2

```

- `#define star_v1_SafetyThresholds_cell_imbalance_threshold_mv_tag 3`
- `#define star_v1_EncoderConfiguration_edges_per_revolution_tag 1`
- `#define star_v1_EncoderConfiguration_wheel_diameter_m_tag 2`
- `#define star_v1_EncoderConfiguration_gear_ratio_tag 3`
- `#define star_v1_TimingConfiguration_motor_control_period_ms_tag 1`
- `#define star_v1_TimingConfiguration_telemetry_period_ms_tag 2`
- `#define star_v1_TimingConfiguration_communication_timeout_ms_tag 3`
- `#define star_v1_SystemConfiguration_motor_configs_tag 1`
- `#define star_v1_SystemConfiguration_current_calibration_tag 2`
- `#define star_v1_SystemConfiguration_temperature_calibration_tag 3`
- `#define star_v1_SystemConfiguration_safety_thresholds_tag 4`
- `#define star_v1_SystemConfiguration_encoder_config_tag 5`
- `#define star_v1_SystemConfiguration_timing_config_tag 6`
- `#define star_v1_SystemConfiguration_config_version_tag 7`
- `#define star_v1_SystemConfiguration_config_crc_tag 8`
- `#define star_v1_GetConfigurationResponse_header_tag 1`
- `#define star_v1_GetConfigurationResponse_configuration_tag 2`
- `#define star_v1_SetConfigurationRequest_header_tag 1`
- `#define star_v1_SetConfigurationRequest_configuration_tag 2`
- `#define star_v1_SetConfigurationRequest_persist_to_nvs_tag 3`
- `#define star_v1_ResetToDefaultsResponse_header_tag 1`
- `#define star_v1_ResetToDefaultsResponse_configuration_tag 2`
- `#define star_v1_ValidateConfigurationRequest_header_tag 1`
- `#define star_v1_ValidateConfigurationRequest_configuration_tag 2`
- `#define star_v1_ConfigValidationError_field_path_tag 1`
- `#define star_v1_ConfigValidationError_error_code_tag 2`
- `#define star_v1_ConfigValidationError_message_tag 3`
- `#define star_v1_ConfigValidationError_actual_value_tag 4`
- `#define star_v1_ConfigValidationError_expected_constraint_tag 5`
- `#define star_v1_ConfigValidationResult_status_tag 1`
- `#define star_v1_ConfigValidationResult_errors_tag 2`
- `#define star_v1_SetConfigurationResponse_header_tag 1`
- `#define star_v1_SetConfigurationResponse_validation_result_tag 2`
- `#define star_v1_ValidateConfigurationResponse_header_tag 1`
- `#define star_v1_ValidateConfigurationResponse_validation_result_tag 2`
- `#define star_v1_SetMotorPidConfigResponse_header_tag 1`
- `#define star_v1_SetMotorPidConfigResponse_validation_result_tag 2`
- `#define star_v1_GetConfigurationRequest_FIELDLIST(X, a)`
- `#define star_v1_GetConfigurationRequest_CALLBACK NULL`
- `#define star_v1_GetConfigurationRequest_DEFAULT NULL`
- `#define star_v1_GetConfigurationRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_GetConfigurationResponse_FIELDLIST(X, a)`
- `#define star_v1_GetConfigurationResponse_CALLBACK NULL`
- `#define star_v1_GetConfigurationResponse_DEFAULT NULL`
- `#define star_v1_GetConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_GetConfigurationResponse_configuration_MSGTYPE star_v1_SystemConfiguration`
- `#define star_v1_SetConfigurationRequest_FIELDLIST(X, a)`
- `#define star_v1_SetConfigurationRequest_CALLBACK NULL`
- `#define star_v1_SetConfigurationRequest_DEFAULT NULL`
- `#define star_v1_SetConfigurationRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_SetConfigurationRequest_configuration_MSGTYPE star_v1_SystemConfiguration`
- `#define star_v1_SetConfigurationResponse_FIELDLIST(X, a)`
- `#define star_v1_SetConfigurationResponse_CALLBACK NULL`
- `#define star_v1_SetConfigurationResponse_DEFAULT NULL`
- `#define star_v1_SetConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader`



- `#define star_v1_SetConfigurationResponse_validation_result_MSGTYPE star_v1_ConfigValidationResult`
- `#define star_v1_ResetToDefaultsRequest_FIELDLIST(X, a)`
- `#define star_v1_ResetToDefaultsRequest_CALLBACK NULL`
- `#define star_v1_ResetToDefaultsRequest_DEFAULT NULL`
- `#define star_v1_ResetToDefaultsRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_ResetToDefaultsResponse_FIELDLIST(X, a)`
- `#define star_v1_ResetToDefaultsResponse_CALLBACK NULL`
- `#define star_v1_ResetToDefaultsResponse_DEFAULT NULL`
- `#define star_v1_ResetToDefaultsResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_ResetToDefaultsResponse_configuration_MSGTYPE star_v1_SystemConfiguration`
- `#define star_v1_ValidateConfigurationRequest_FIELDLIST(X, a)`
- `#define star_v1_ValidateConfigurationRequest_CALLBACK NULL`
- `#define star_v1_ValidateConfigurationRequest_DEFAULT NULL`
- `#define star_v1_ValidateConfigurationRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_ValidateConfigurationRequest_configuration_MSGTYPE star_v1_SystemConfiguration`
- `#define star_v1_ValidateConfigurationResponse_FIELDLIST(X, a)`
- `#define star_v1_ValidateConfigurationResponse_CALLBACK NULL`
- `#define star_v1_ValidateConfigurationResponse_DEFAULT NULL`
- `#define star_v1_ValidateConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_ValidateConfigurationResponse_validation_result_MSGTYPE star_v1_ConfigValidationResult`
- `#define star_v1_SaveConfigurationRequest_FIELDLIST(X, a)`
- `#define star_v1_SaveConfigurationRequest_CALLBACK NULL`
- `#define star_v1_SaveConfigurationRequest_DEFAULT NULL`
- `#define star_v1_SaveConfigurationRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_SaveConfigurationResponse_FIELDLIST(X, a)`
- `#define star_v1_SaveConfigurationResponse_CALLBACK NULL`
- `#define star_v1_SaveConfigurationResponse_DEFAULT NULL`
- `#define star_v1_SaveConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_GetMotorPidConfigRequest_FIELDLIST(X, a)`
- `#define star_v1_GetMotorPidConfigRequest_CALLBACK NULL`
- `#define star_v1_GetMotorPidConfigRequest_DEFAULT NULL`
- `#define star_v1_GetMotorPidConfigRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_GetMotorPidConfigResponse_FIELDLIST(X, a)`
- `#define star_v1_GetMotorPidConfigResponse_CALLBACK NULL`
- `#define star_v1_GetMotorPidConfigResponse_DEFAULT NULL`
- `#define star_v1_GetMotorPidConfigResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_GetMotorPidConfigResponse_pid_config_MSGTYPE star_v1_PidConfig`
- `#define star_v1_SetMotorPidConfigRequest_FIELDLIST(X, a)`
- `#define star_v1_SetMotorPidConfigRequest_CALLBACK NULL`
- `#define star_v1_SetMotorPidConfigRequest_DEFAULT NULL`
- `#define star_v1_SetMotorPidConfigRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_SetMotorPidConfigRequest_pid_config_MSGTYPE star_v1_PidConfig`
- `#define star_v1_SetMotorPidConfigResponse_FIELDLIST(X, a)`
- `#define star_v1_SetMotorPidConfigResponse_CALLBACK NULL`
- `#define star_v1_SetMotorPidConfigResponse_DEFAULT NULL`
- `#define star_v1_SetMotorPidConfigResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_SetMotorPidConfigResponse_validation_result_MSGTYPE star_v1_ConfigValidationResult`
- `#define star_v1_RetransmitConfig_FIELDLIST(X, a)`
- `#define star_v1_RetransmitConfig_CALLBACK NULL`
- `#define star_v1_RetransmitConfig_DEFAULT NULL`
- `#define star_v1_SetRetransmitConfigRequest_FIELDLIST(X, a)`
- `#define star_v1_SetRetransmitConfigRequest_CALLBACK NULL`
- `#define star_v1_SetRetransmitConfigRequest_DEFAULT NULL`
- `#define star_v1_SetRetransmitConfigRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_SetRetransmitConfigRequest_retransmit_config_MSGTYPE star_v1_RetransmitConfig`

- `#define star_v1_SetRetransmitConfigResponse_FIELDLIST(X, a)`
- `#define star_v1_SetRetransmitConfigResponse_CALLBACK NULL`
- `#define star_v1_SetRetransmitConfigResponse_DEFAULT NULL`
- `#define star_v1_SetRetransmitConfigResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_SystemConfiguration_FIELDLIST(X, a)`
- `#define star_v1_SystemConfiguration_CALLBACK NULL`
- `#define star_v1_SystemConfiguration_DEFAULT NULL`
- `#define star_v1_SystemConfiguration_motor_configs_MSGTYPE star_v1_MotorPidConfiguration`
- `#define star_v1_SystemConfiguration_current_calibration_MSGTYPE star_v1_CurrentSensorCalibration`
- `#define star_v1_SystemConfiguration_temperature_calibration_MSGTYPE star_v1_TemperatureCalibration`
- `#define star_v1_SystemConfiguration_safety_thresholds_MSGTYPE star_v1_SafetyThresholds`
- `#define star_v1_SystemConfiguration_encoder_config_MSGTYPE star_v1_EncoderConfiguration`
- `#define star_v1_SystemConfiguration_timing_config_MSGTYPE star_v1_TimingConfiguration`
- `#define star_v1_MotorPidConfiguration_FIELDLIST(X, a)`
- `#define star_v1_MotorPidConfiguration_CALLBACK NULL`
- `#define star_v1_MotorPidConfiguration_DEFAULT NULL`
- `#define star_v1_MotorPidConfiguration_pid_config_MSGTYPE star_v1_PidConfig`
- `#define star_v1_CurrentSensorCalibration_FIELDLIST(X, a)`
- `#define star_v1_CurrentSensorCalibration_CALLBACK pb_default_field_callback`
- `#define star_v1_CurrentSensorCalibration_DEFAULT NULL`
- `#define star_v1_TemperatureCalibration_FIELDLIST(X, a)`
- `#define star_v1_TemperatureCalibration_CALLBACK NULL`
- `#define star_v1_TemperatureCalibration_DEFAULT NULL`
- `#define star_v1_SafetyThresholds_FIELDLIST(X, a)`
- `#define star_v1_SafetyThresholds_CALLBACK NULL`
- `#define star_v1_SafetyThresholds_DEFAULT NULL`
- `#define star_v1_EncoderConfiguration_FIELDLIST(X, a)`
- `#define star_v1_EncoderConfiguration_CALLBACK NULL`
- `#define star_v1_EncoderConfiguration_DEFAULT NULL`
- `#define star_v1_TimingConfiguration_FIELDLIST(X, a)`
- `#define star_v1_TimingConfiguration_CALLBACK NULL`
- `#define star_v1_TimingConfiguration_DEFAULT NULL`
- `#define star_v1_ConfigValidationResult_FIELDLIST(X, a)`
- `#define star_v1_ConfigValidationResult_CALLBACK NULL`
- `#define star_v1_ConfigValidationResult_DEFAULT NULL`
- `#define star_v1_ConfigValidationResult_errors_MSGTYPE star_v1_ConfigValidationError`
- `#define star_v1_ConfigValidationError_FIELDLIST(X, a)`
- `#define star_v1_ConfigValidationError_CALLBACK NULL`
- `#define star_v1_ConfigValidationError_DEFAULT NULL`
- `#define star_v1_GetConfigurationRequest_fields &star_v1_GetConfigurationRequest_msg`
- `#define star_v1_GetConfigurationResponse_fields &star_v1_GetConfigurationResponse_msg`
- `#define star_v1_SetConfigurationRequest_fields &star_v1_SetConfigurationRequest_msg`
- `#define star_v1_SetConfigurationResponse_fields &star_v1_SetConfigurationResponse_msg`
- `#define star_v1_ResetToDefaultsRequest_fields &star_v1_ResetToDefaultsRequest_msg`
- `#define star_v1_ResetToDefaultsResponse_fields &star_v1_ResetToDefaultsResponse_msg`
- `#define star_v1_ValidateConfigurationRequest_fields &star_v1_ValidateConfigurationRequest_msg`
- `#define star_v1_ValidateConfigurationResponse_fields &star_v1_ValidateConfigurationResponse_msg`
- `#define star_v1_SaveConfigurationRequest_fields &star_v1_SaveConfigurationRequest_msg`
- `#define star_v1_SaveConfigurationResponse_fields &star_v1_SaveConfigurationResponse_msg`
- `#define star_v1_GetMotorPidConfigRequest_fields &star_v1_GetMotorPidConfigRequest_msg`
- `#define star_v1_GetMotorPidConfigResponse_fields &star_v1_GetMotorPidConfigResponse_msg`
- `#define star_v1_SetMotorPidConfigRequest_fields &star_v1_SetMotorPidConfigRequest_msg`
- `#define star_v1_SetMotorPidConfigResponse_fields &star_v1_SetMotorPidConfigResponse_msg`
- `#define star_v1_RetransmitConfig_fields &star_v1_RetransmitConfig_msg`
- `#define star_v1_SetRetransmitConfigRequest_fields &star_v1_SetRetransmitConfigRequest_msg`



- `#define star_v1_SetRetransmitConfigResponse_fields &star_v1_SetRetransmitConfigResponse_msg`
- `#define star_v1_SystemConfiguration_fields &star_v1_SystemConfiguration_msg`
- `#define star_v1_MotorPidConfiguration_fields &star_v1_MotorPidConfiguration_msg`
- `#define star_v1_CurrentSensorCalibration_fields &star_v1_CurrentSensorCalibration_msg`
- `#define star_v1_TemperatureCalibration_fields &star_v1_TemperatureCalibration_msg`
- `#define star_v1_SafetyThresholds_fields &star_v1_SafetyThresholds_msg`
- `#define star_v1_EncoderConfiguration_fields &star_v1_EncoderConfiguration_msg`
- `#define star_v1_TimingConfiguration_fields &star_v1_TimingConfiguration_msg`
- `#define star_v1_ConfigValidationResult_fields &star_v1_ConfigValidationResult_msg`
- `#define star_v1_ConfigValidationError_fields &star_v1_ConfigValidationError_msg`
- `#define STAR_V1_STAR_V1_CONFIGURATION_PB_H_MAX_SIZE star_v1_SetConfigurationResponse_s`
- `#define star_v1_ConfigValidationError_size 585`
- `#define star_v1_ConfigValidationResult_size 4706`
- `#define star_v1_EncoderConfiguration_size 24`
- `#define star_v1_GetConfigurationRequest_size 149`
- `#define star_v1_GetMotorPidConfigRequest_size 160`
- `#define star_v1_GetMotorPidConfigResponse_size 439`
- `#define star_v1_MotorPidConfiguration_size 76`
- `#define star_v1_ResetToDefaultsRequest_size 151`
- `#define star_v1_RetransmitConfig_size 20`
- `#define star_v1_SafetyThresholds_size 23`
- `#define star_v1_SaveConfigurationRequest_size 149`
- `#define star_v1_SaveConfigurationResponse_size 371`
- `#define star_v1_SetConfigurationResponse_size 5072`
- `#define star_v1_SetMotorPidConfigRequest_size 227`
- `#define star_v1_SetMotorPidConfigResponse_size 5072`
- `#define star_v1_SetRetransmitConfigRequest_size 171`
- `#define star_v1_SetRetransmitConfigResponse_size 363`
- `#define star_v1_TemperatureCalibration_size 9`
- `#define star_v1_TimingConfiguration_size 18`
- `#define star_v1_ValidateConfigurationResponse_size 5072`

## Enumerations

- `enum star_v1_ConfigValidationStatus {`  
`star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_UNKNOWN = 0 ,`  
`star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_VALID = 1 , star_v1_ConfigValidationSta`  
`= 2 , star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_WARNING = 3 ,`  
`star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_CRC_MISMATCH = 4 ,`  
`star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_VERSION_MISMATCH`  
`= 5 }`
- `enum star_v1_ConfigErrorCode {`  
`star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_UNKNOWN = 0 , star_v1_ConfigErrorCode_CONFIG`  
`= 1 , star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_VALUE_TOO_HIGH = 2 ,`  
`star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_INVALID_NUMBER = 3 ,`  
`star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_MISSING_FIELD = 4 , star_v1_ConfigErrorCode_CON`  
`= 5 , star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_CRC_FAILED = 6 , star_v1_ConfigErrorCode_CO`  
`= 7 ,`  
`star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_STORAGE_ERROR = 8 }`

## Variables

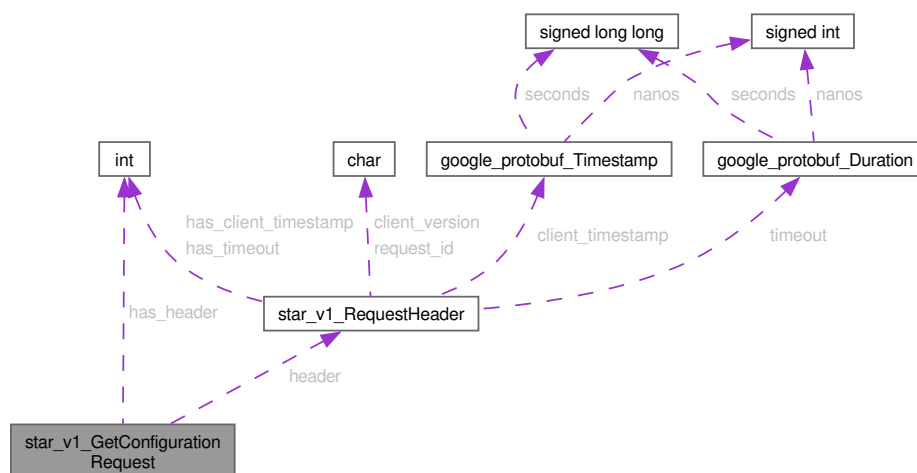
- `const pb_msgdesc_t star_v1_GetConfigurationRequest_msg`
- `const pb_msgdesc_t star_v1_GetConfigurationResponse_msg`
- `const pb_msgdesc_t star_v1_SetConfigurationRequest_msg`
- `const pb_msgdesc_t star_v1_SetConfigurationResponse_msg`
- `const pb_msgdesc_t star_v1_ResetToDefaultsRequest_msg`
- `const pb_msgdesc_t star_v1_ResetToDefaultsResponse_msg`
- `const pb_msgdesc_t star_v1_ValidateConfigurationRequest_msg`
- `const pb_msgdesc_t star_v1_ValidateConfigurationResponse_msg`
- `const pb_msgdesc_t star_v1_SaveConfigurationRequest_msg`
- `const pb_msgdesc_t star_v1_SaveConfigurationResponse_msg`
- `const pb_msgdesc_t star_v1_GetMotorPidConfigRequest_msg`
- `const pb_msgdesc_t star_v1_GetMotorPidConfigResponse_msg`
- `const pb_msgdesc_t star_v1_SetMotorPidConfigRequest_msg`
- `const pb_msgdesc_t star_v1_SetMotorPidConfigResponse_msg`
- `const pb_msgdesc_t star_v1_RetransmitConfig_msg`
- `const pb_msgdesc_t star_v1_SetRetransmitConfigRequest_msg`
- `const pb_msgdesc_t star_v1_SetRetransmitConfigResponse_msg`
- `const pb_msgdesc_t star_v1_SystemConfiguration_msg`
- `const pb_msgdesc_t star_v1_MotorPidConfiguration_msg`
- `const pb_msgdesc_t star_v1_CurrentSensorCalibration_msg`
- `const pb_msgdesc_t star_v1_TemperatureCalibration_msg`
- `const pb_msgdesc_t star_v1_SafetyThresholds_msg`
- `const pb_msgdesc_t star_v1_EncoderConfiguration_msg`
- `const pb_msgdesc_t star_v1_TimingConfiguration_msg`
- `const pb_msgdesc_t star_v1_ConfigValidationResult_msg`
- `const pb_msgdesc_t star_v1_ConfigValidationError_msg`

## 6.15.1 Data Structure Documentation

## 6.15.1.1 struct star\_v1\_GetConfigurationRequest

Definition at line 55 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_GetConfigurationRequest:



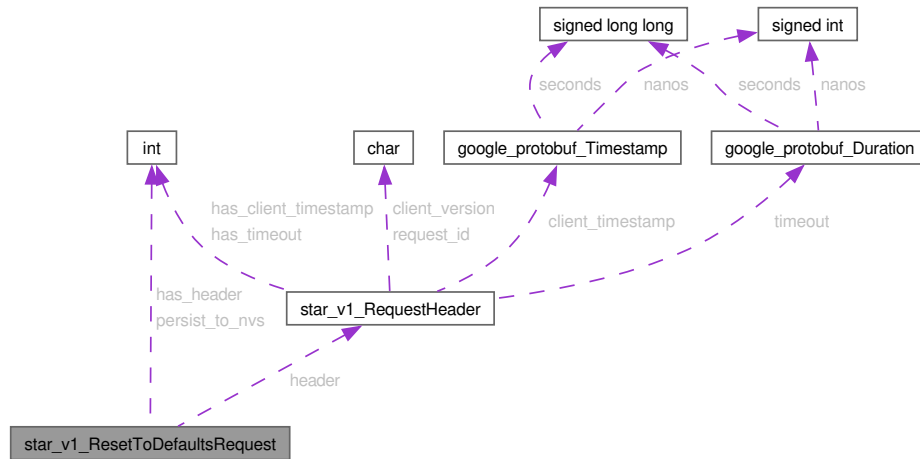
## Data Fields

|                                       |            |  |
|---------------------------------------|------------|--|
| <a href="#">bool</a>                  | has_header |  |
| <a href="#">star_v1_RequestHeader</a> | header     |  |

## 6.15.1.2 struct star\_v1\_ResetToDefaultsRequest

Definition at line 62 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_ResetToDefaultsRequest:



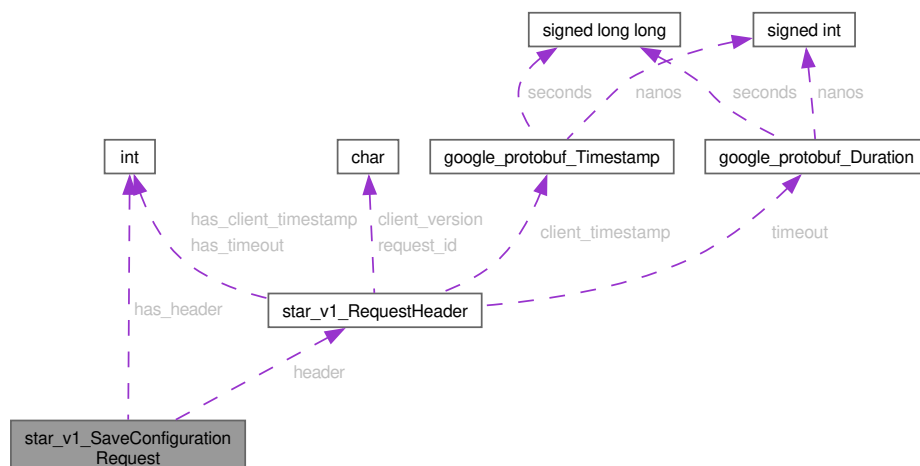
## Data Fields

|                                       |                |  |
|---------------------------------------|----------------|--|
| <a href="#">bool</a>                  | has_header     |  |
| <a href="#">star_v1_RequestHeader</a> | header         |  |
| <a href="#">bool</a>                  | persist_to_nvs |  |

## 6.15.1.3 struct star\_v1\_SaveConfigurationRequest

Definition at line 71 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_SaveConfigurationRequest:



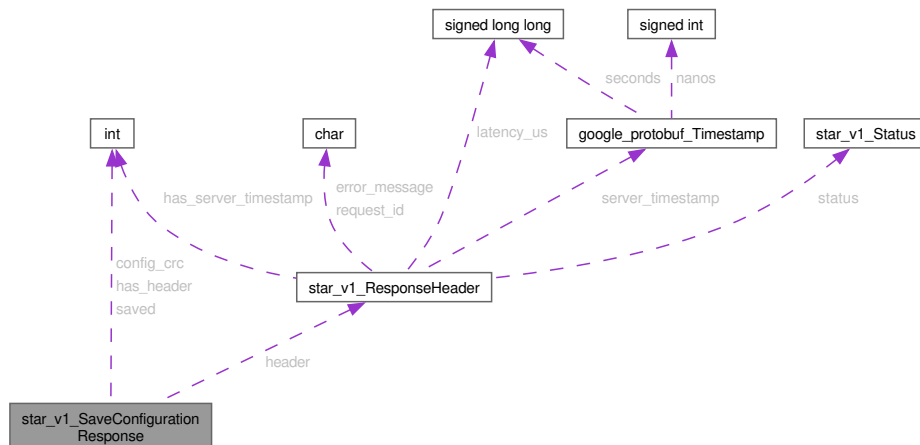
## Data Fields

|                                       |            |  |
|---------------------------------------|------------|--|
| <a href="#">bool</a>                  | has_header |  |
| <a href="#">star_v1_RequestHeader</a> | header     |  |

## 6.15.1.4 struct star\_v1\_SaveConfigurationResponse

Definition at line 78 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_SaveConfigurationResponse:



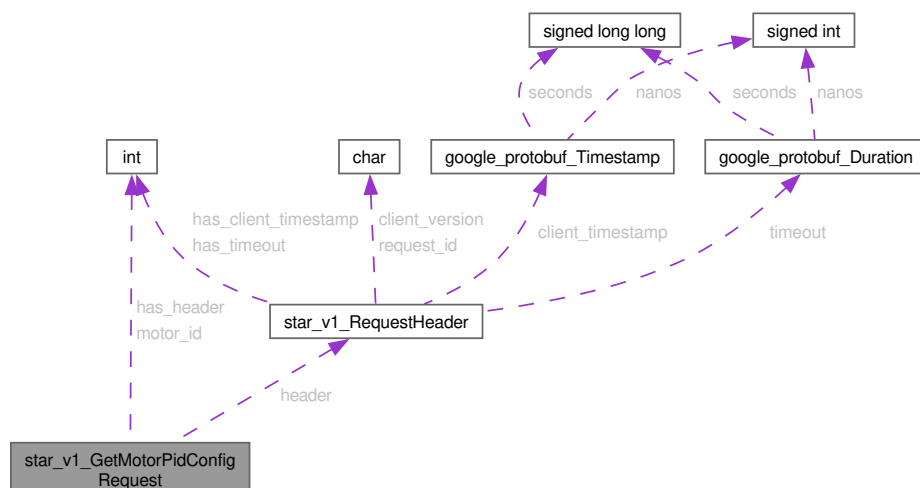
## Data Fields

|                                        |            |  |
|----------------------------------------|------------|--|
| <a href="#">uint32_t</a>               | config_crc |  |
| <a href="#">bool</a>                   | has_header |  |
| <a href="#">star_v1_ResponseHeader</a> | header     |  |
| <a href="#">bool</a>                   | saved      |  |

## 6.15.1.5 struct star\_v1\_GetMotorPidConfigRequest

Definition at line 89 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_GetMotorPidConfigRequest:



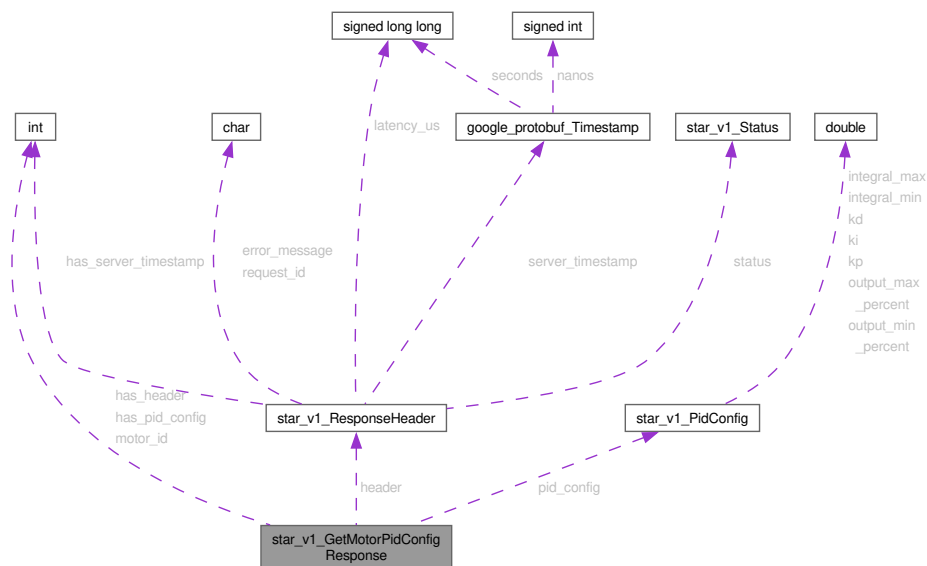
## Data Fields

|                                       |            |  |
|---------------------------------------|------------|--|
| <a href="#">bool</a>                  | has_header |  |
| <a href="#">star_v1_RequestHeader</a> | header     |  |
| <a href="#">int32_t</a>               | motor_id   |  |

## 6.15.1.6 struct star\_v1\_GetMotorPidConfigResponse

Definition at line 98 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_GetMotorPidConfigResponse:



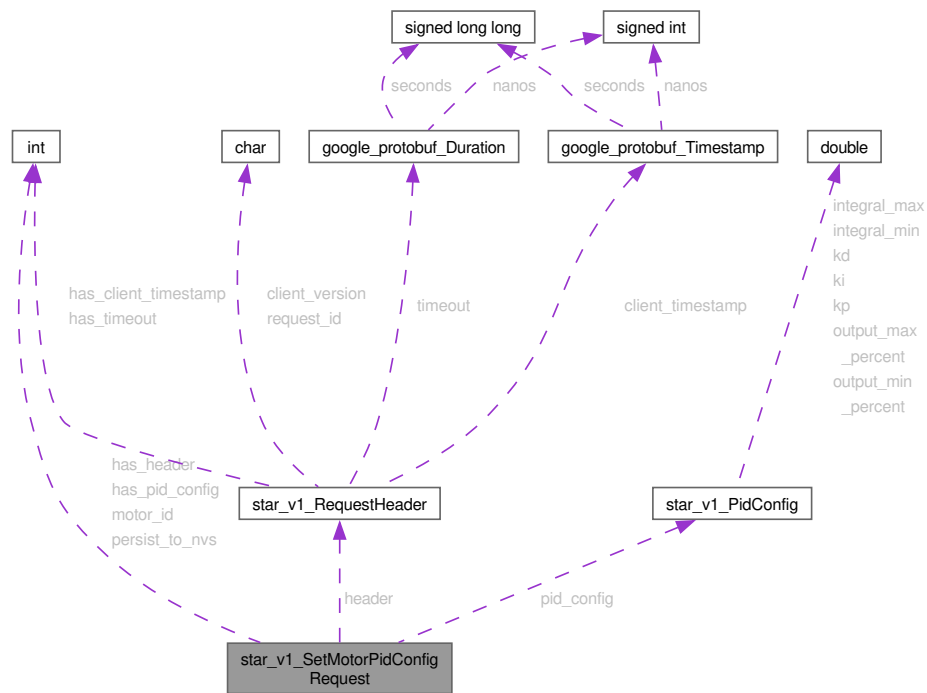
## Data Fields

|                                        |                |  |
|----------------------------------------|----------------|--|
| <a href="#">bool</a>                   | has_header     |  |
| <a href="#">bool</a>                   | has_pid_config |  |
| <a href="#">star_v1_ResponseHeader</a> | header         |  |
| <a href="#">int32_t</a>                | motor_id       |  |
| <a href="#">star_v1_PidConfig</a>      | pid_config     |  |

## 6.15.1.7 struct star\_v1\_SetMotorPidConfigRequest

Definition at line 110 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_SetMotorPidConfigRequest:



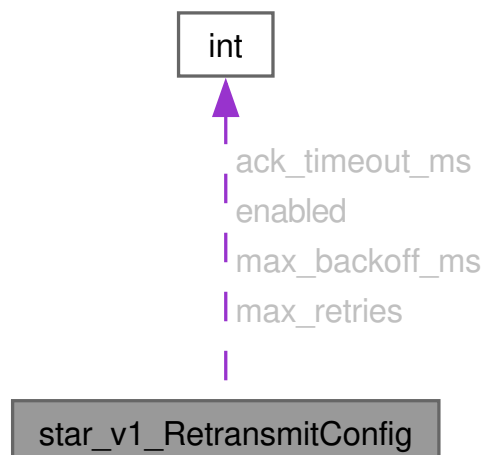
## Data Fields

|                                       |                |  |
|---------------------------------------|----------------|--|
| <a href="#">bool</a>                  | has_header     |  |
| <a href="#">bool</a>                  | has_pid_config |  |
| <a href="#">star_v1_RequestHeader</a> | header         |  |
| <a href="#">int32_t</a>               | motor_id       |  |
| <a href="#">bool</a>                  | persist_to_nvs |  |
| <a href="#">star_v1_PidConfig</a>     | pid_config     |  |

## 6.15.1.8 struct star\_v1\_RetransmitConfig

Definition at line 124 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_RetransmitConfig:



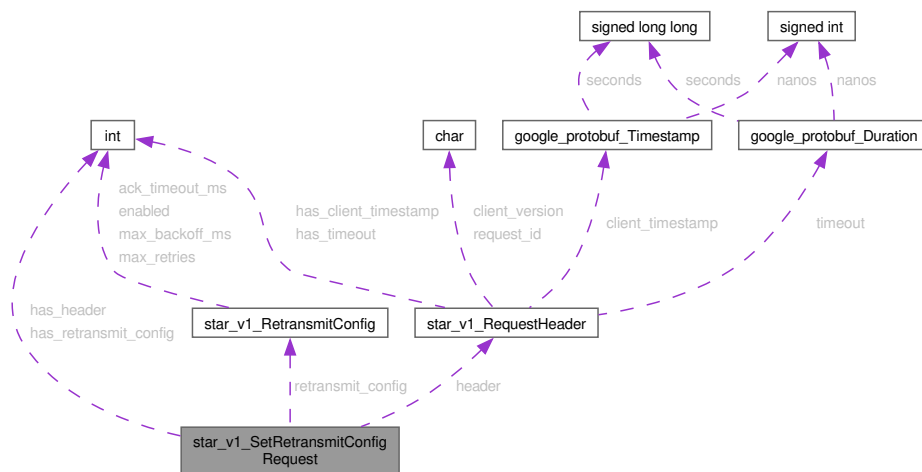
## Data Fields

|                          |                |  |
|--------------------------|----------------|--|
| <a href="#">uint32_t</a> | ack_timeout_ms |  |
| <a href="#">bool</a>     | enabled        |  |
| <a href="#">uint32_t</a> | max_backoff_ms |  |
| <a href="#">uint32_t</a> | max_retries    |  |

## 6.15.1.9 struct star\_v1\_SetRetransmitConfigRequest

Definition at line 139 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_SetRetransmitConfigRequest:



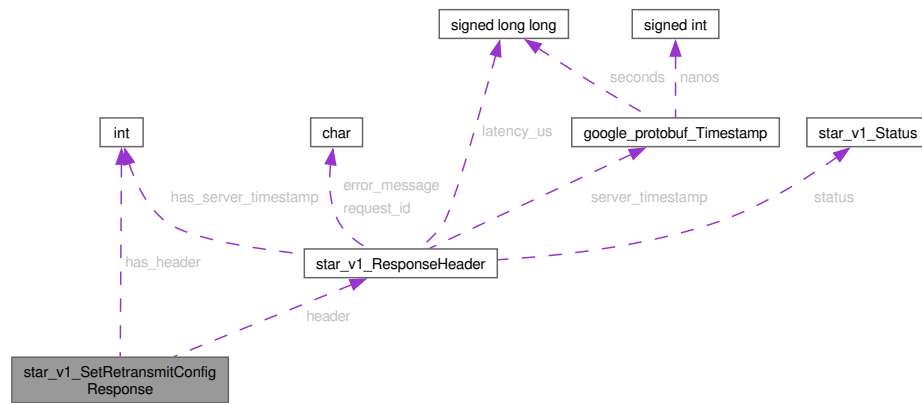
## Data Fields

|                                          |                       |  |
|------------------------------------------|-----------------------|--|
| <a href="#">bool</a>                     | has_header            |  |
| <a href="#">bool</a>                     | has_retransmit_config |  |
| <a href="#">star_v1_RequestHeader</a>    | header                |  |
| <a href="#">star_v1_RetransmitConfig</a> | retransmit_config     |  |

## 6.15.1.10 struct star\_v1\_SetRetransmitConfigResponse

Definition at line 149 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_SetRetransmitConfigResponse:



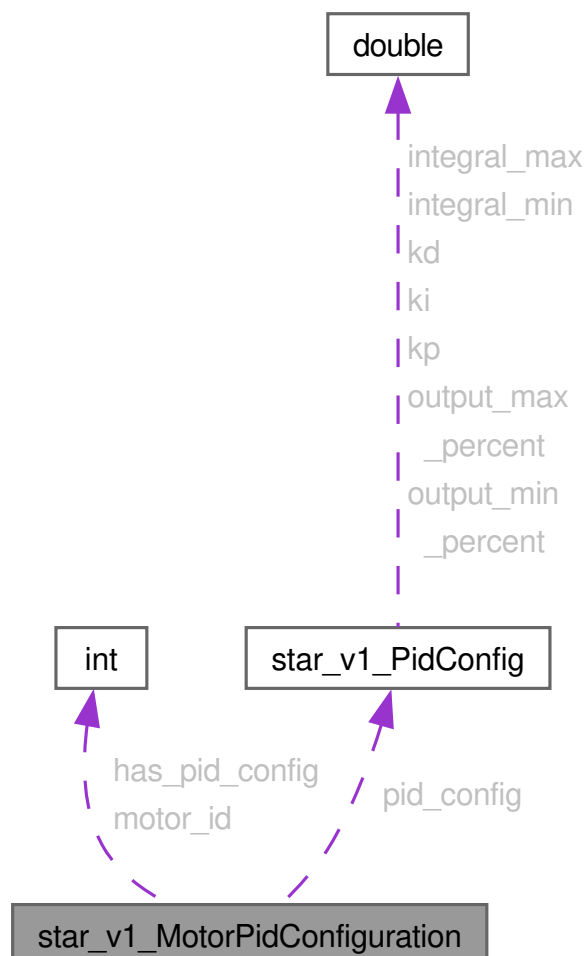
## Data Fields

|                                     |                         |  |
|-------------------------------------|-------------------------|--|
| <code>bool</code>                   | <code>has_header</code> |  |
| <code>star_v1_ResponseHeader</code> | <code>header</code>     |  |

6.15.1.11 struct `star_v1_MotorPidConfiguration`

Definition at line 160 of file [configuration.pb.h](#).

Collaboration diagram for `star_v1_MotorPidConfiguration`:





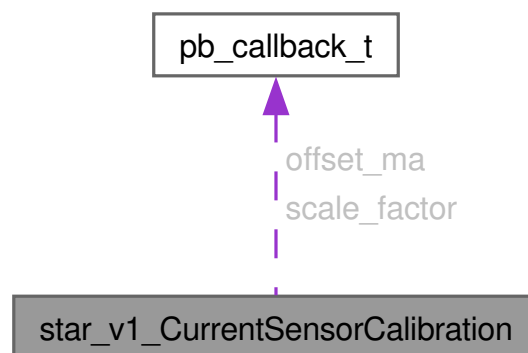
## Data Fields

|                                   |                |  |
|-----------------------------------|----------------|--|
| <a href="#">bool</a>              | has_pid_config |  |
| <a href="#">int32_t</a>           | motor_id       |  |
| <a href="#">star_v1_PidConfig</a> | pid_config     |  |

## 6.15.1.12 struct star\_v1\_CurrentSensorCalibration

Definition at line 172 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_CurrentSensorCalibration:



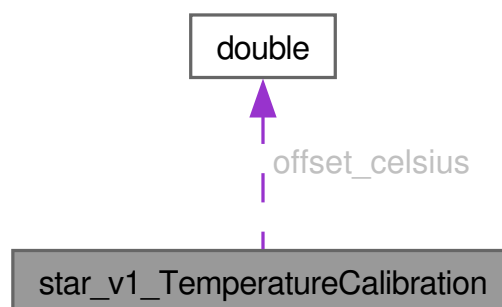
## Data Fields

|                               |              |  |
|-------------------------------|--------------|--|
| <a href="#">pb_callback_t</a> | offset_ma    |  |
| <a href="#">pb_callback_t</a> | scale_factor |  |

## 6.15.1.13 struct star\_v1\_TemperatureCalibration

Definition at line 183 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_TemperatureCalibration:



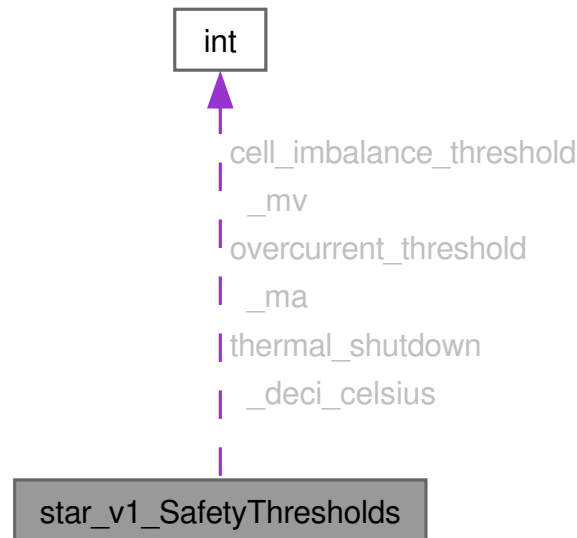
## Data Fields

|        |                |  |
|--------|----------------|--|
| double | offset_celsius |  |
|--------|----------------|--|

## 6.15.1.14 struct star\_v1\_SafetyThresholds

Definition at line 190 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_SafetyThresholds:



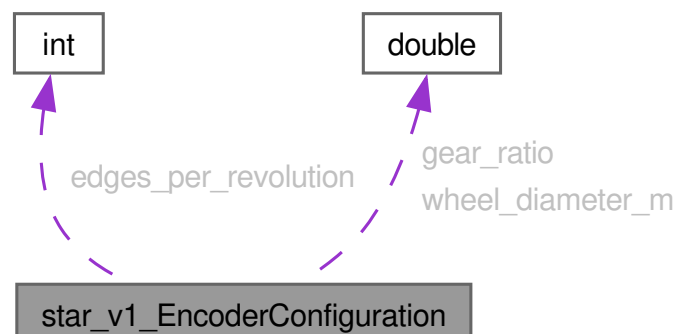
## Data Fields

|                          |                                            |  |
|--------------------------|--------------------------------------------|--|
| <a href="#">uint32_t</a> | <code>cell_imbalance_threshold_mv</code>   |  |
| <a href="#">uint32_t</a> | <code>overcurrent_threshold_ma</code>      |  |
| <a href="#">int32_t</a>  | <code>thermal_shutdown_deci_celsius</code> |  |

## 6.15.1.15 struct star\_v1\_EncoderConfiguration

Definition at line 205 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_EncoderConfiguration:



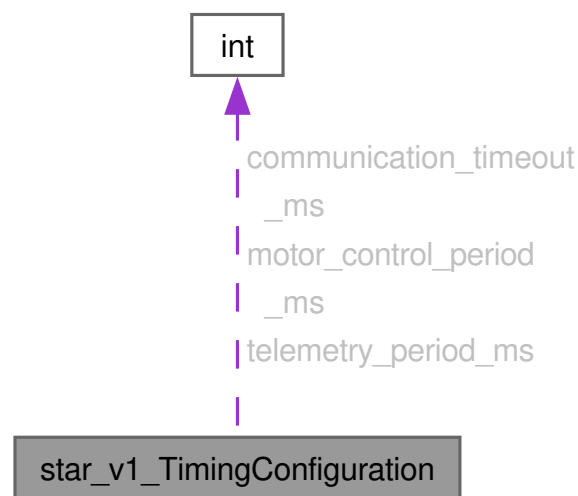
## Data Fields

|                          |                      |  |
|--------------------------|----------------------|--|
| <a href="#">uint32_t</a> | edges_per_revolution |  |
| double                   | gear_ratio           |  |
| double                   | wheel_diameter_m     |  |

## 6.15.1.16 struct star\_v1\_TimingConfiguration

Definition at line 217 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_TimingConfiguration:



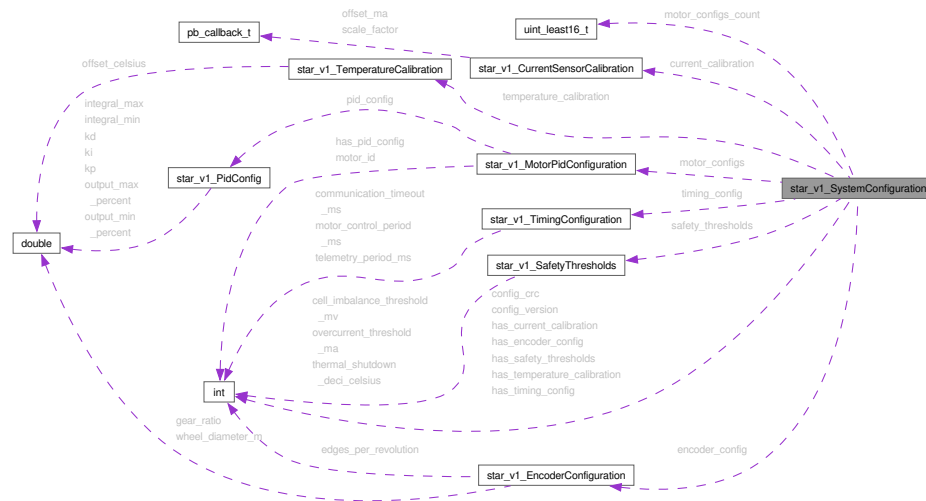
## Data Fields

|                          |                          |  |
|--------------------------|--------------------------|--|
| <a href="#">uint32_t</a> | communication_timeout_ms |  |
| <a href="#">uint32_t</a> | motor_control_period_ms  |  |
| <a href="#">uint32_t</a> | telemetry_period_ms      |  |

## 6.15.1.17 struct star\_v1\_SystemConfiguration

Definition at line 233 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_SystemConfiguration:



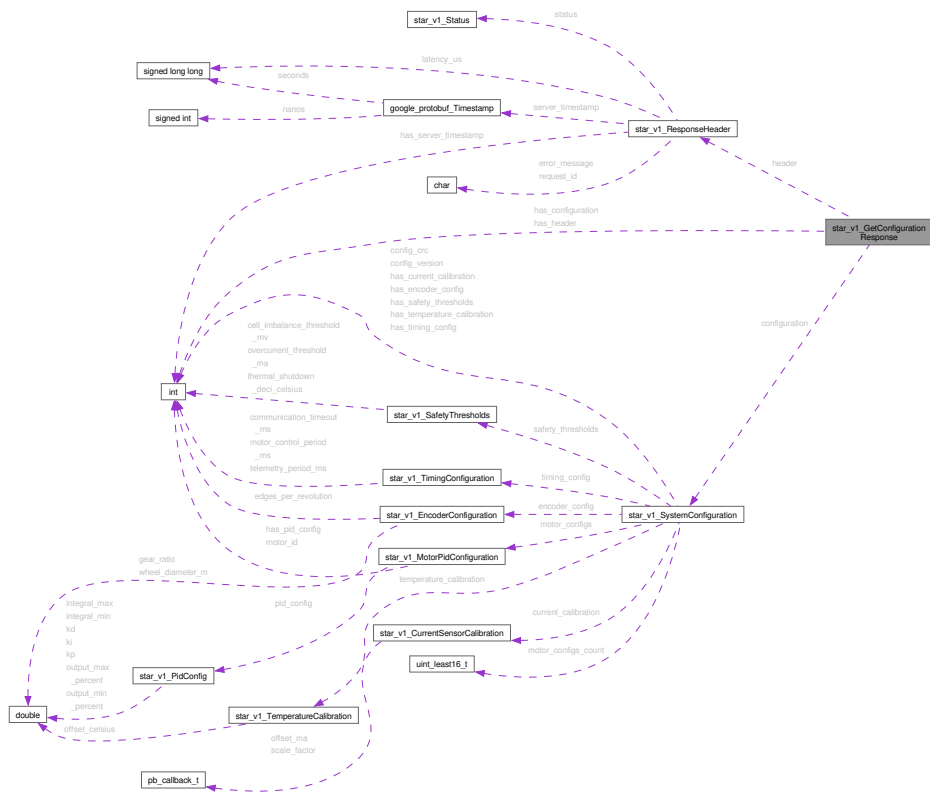
## Data Fields

|                                                  |                             |  |
|--------------------------------------------------|-----------------------------|--|
| <a href="#">uint32_t</a>                         | config_crc                  |  |
| <a href="#">uint32_t</a>                         | config_version              |  |
| <a href="#">star_v1_CurrentSensorCalibration</a> | current_calibration         |  |
| <a href="#">star_v1_EncoderConfiguration</a>     | encoder_config              |  |
| <a href="#">bool</a>                             | has_current_calibration     |  |
| <a href="#">bool</a>                             | has_encoder_config          |  |
| <a href="#">bool</a>                             | has_safety_thresholds       |  |
| <a href="#">bool</a>                             | has_temperature_calibration |  |
| <a href="#">bool</a>                             | has_timing_config           |  |
| <a href="#">star_v1_MotorPidConfiguration</a>    | motor_configs[4]            |  |
| <a href="#">pb_size_t</a>                        | motor_configs_count         |  |
| <a href="#">star_v1_SafetyThresholds</a>         | safety_thresholds           |  |
| <a href="#">star_v1_TemperatureCalibration</a>   | temperature_calibration     |  |
| <a href="#">star_v1_TimingConfiguration</a>      | timing_config               |  |

### 6.15.1.18 struct star\_v1\_GetConfigurationResponse

Definition at line 262 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_GetConfigurationResponse:



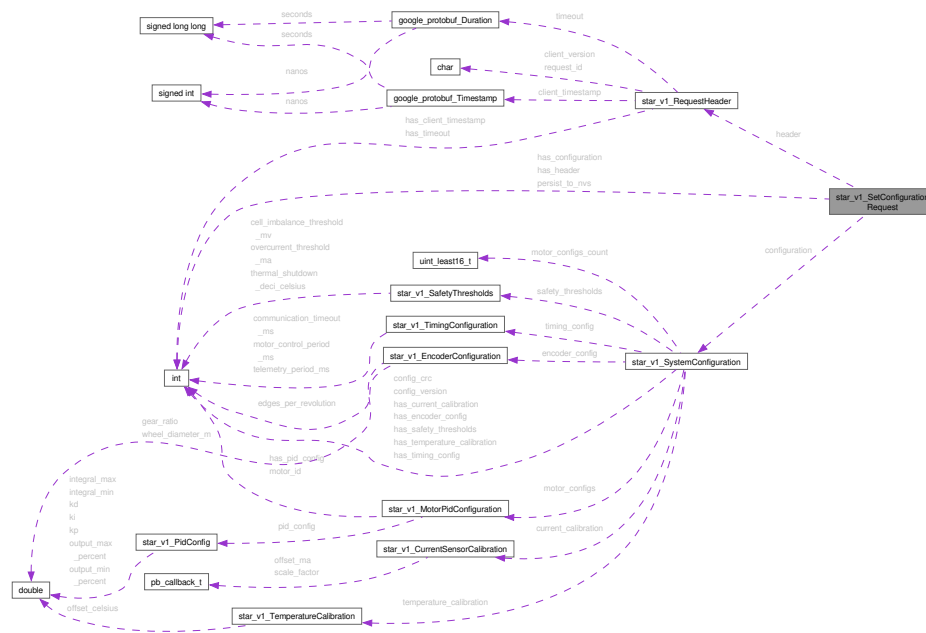
## Data Fields

|                                             |                   |  |
|---------------------------------------------|-------------------|--|
| <a href="#">star_v1_SystemConfiguration</a> | configuration     |  |
| <a href="#">bool</a>                        | has_configuration |  |
| <a href="#">bool</a>                        | has_header        |  |
| <a href="#">star_v1_ResponseHeader</a>      | header            |  |

## 6.15.1.19 struct star\_v1\_SetConfigurationRequest

Definition at line 272 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_SetConfigurationRequest:



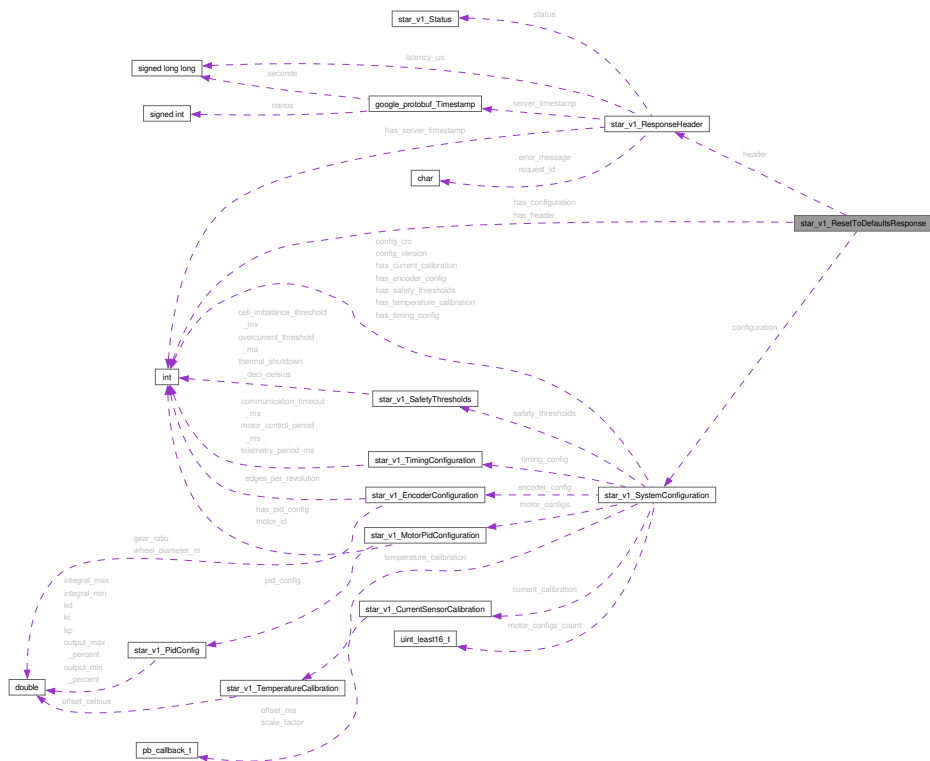
## Data Fields

|                                             |                   |  |
|---------------------------------------------|-------------------|--|
| <a href="#">star_v1_SystemConfiguration</a> | configuration     |  |
| <a href="#">bool</a>                        | has_configuration |  |
| <a href="#">bool</a>                        | has_header        |  |
| <a href="#">star_v1_RequestHeader</a>       | header            |  |
| <a href="#">bool</a>                        | persist_to_nvs    |  |

### 6.15.1.20 struct star\_v1\_ResetToDefaultsResponse

Definition at line 285 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_ResetToDefaultsResponse:



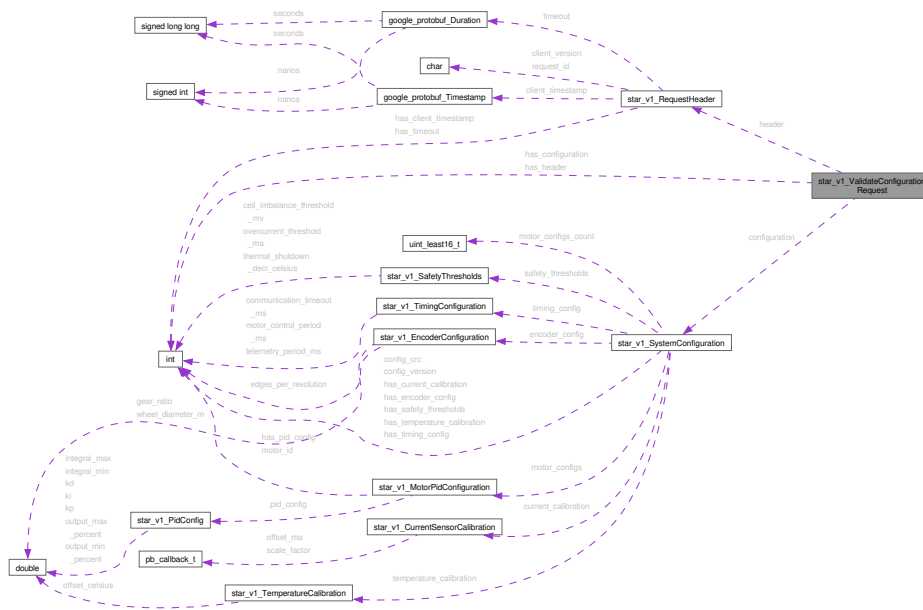
## Data Fields

|                                             |                   |  |
|---------------------------------------------|-------------------|--|
| <a href="#">star_v1_SystemConfiguration</a> | configuration     |  |
| <a href="#">bool</a>                        | has_configuration |  |
| <a href="#">bool</a>                        | has_header        |  |
| <a href="#">star_v1_ResponseHeader</a>      | header            |  |

## 6.15.1.21 struct star\_v1\_ValidateConfigurationRequest

Definition at line 295 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_ValidateConfigurationRequest:



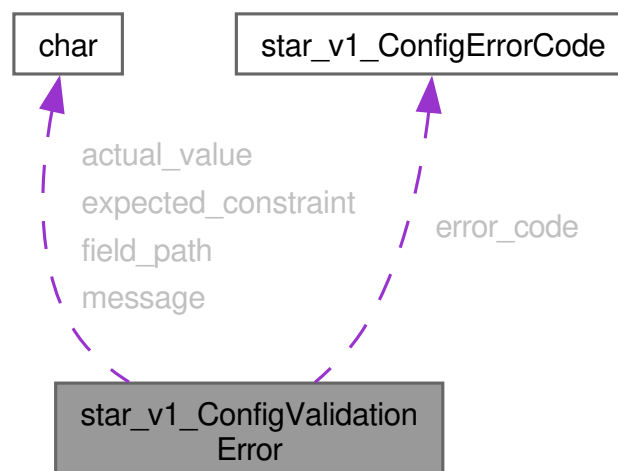
## Data Fields

|                                             |                   |  |
|---------------------------------------------|-------------------|--|
| <a href="#">star_v1_SystemConfiguration</a> | configuration     |  |
| <a href="#">bool</a>                        | has_configuration |  |
| <a href="#">bool</a>                        | has_header        |  |
| <a href="#">star_v1_RequestHeader</a>       | header            |  |

### 6.15.1.22 struct star\_v1\_ConfigValidationError

Definition at line 305 of file [configuration.pb.h](#).

Collaboration diagram for [star\\_v1\\_ConfigValidationError](#):





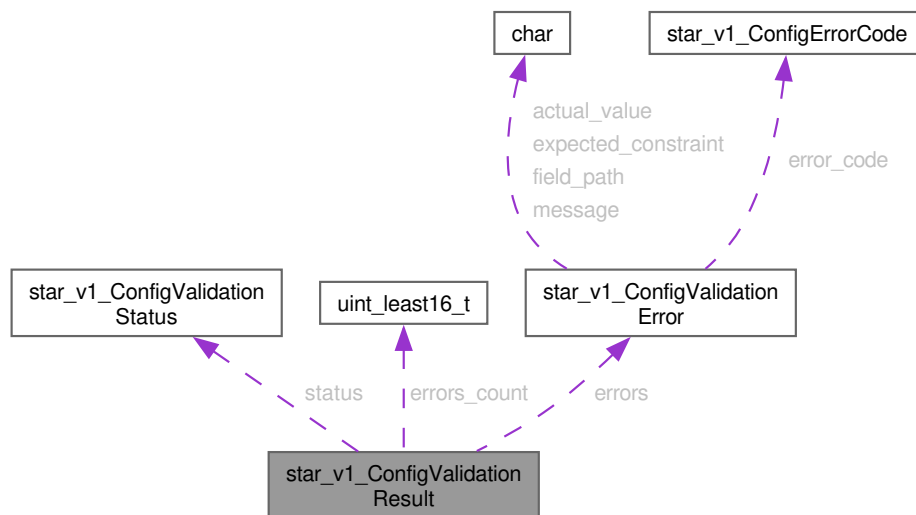
## Data Fields

|                                         |      |                          |  |
|-----------------------------------------|------|--------------------------|--|
|                                         | char | actual_value[64]         |  |
| <a href="#">star_v1_ConfigErrorCode</a> |      | error_code               |  |
|                                         | char | expected_constraint[128] |  |
|                                         | char | field_path[128]          |  |
|                                         | char | message[256]             |  |

## 6.15.1.23 struct star\_v1\_ConfigValidationResult

Definition at line 319 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_ConfigValidationResult:



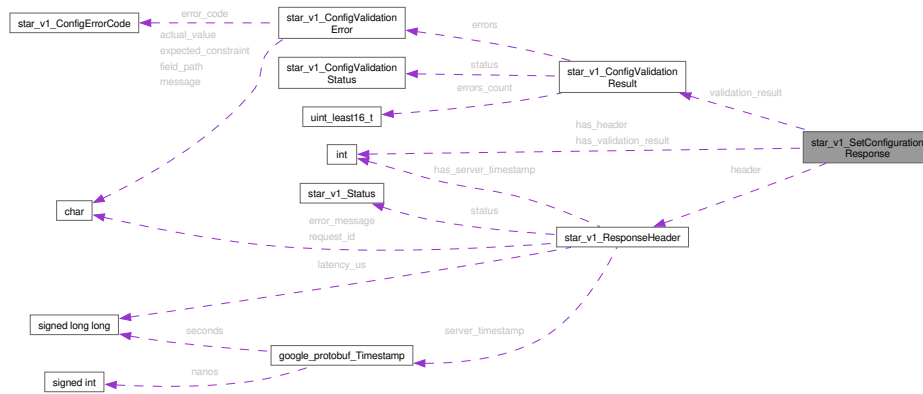
## Data Fields

|                                                |              |  |
|------------------------------------------------|--------------|--|
| <a href="#">star_v1_ConfigValidationError</a>  | errors[8]    |  |
| <a href="#">pb_size_t</a>                      | errors_count |  |
| <a href="#">star_v1_ConfigValidationStatus</a> | status       |  |

## 6.15.1.24 struct star\_v1\_SetConfigurationResponse

Definition at line 328 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_SetConfigurationResponse:



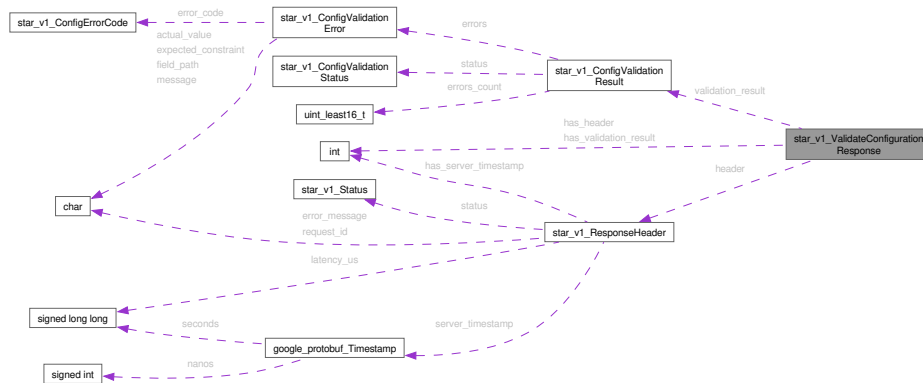
## Data Fields

|                                                |                      |                                       |  |
|------------------------------------------------|----------------------|---------------------------------------|--|
|                                                | <a href="#">bool</a> | <a href="#">has_header</a>            |  |
|                                                | <a href="#">bool</a> | <a href="#">has_validation_result</a> |  |
| <a href="#">star_v1_ResponseHeader</a>         |                      | <a href="#">header</a>                |  |
| <a href="#">star_v1_ConfigValidationResult</a> |                      | <a href="#">validation_result</a>     |  |

## 6.15.1.25 struct star\_v1\_ValidateConfigurationResponse

Definition at line 338 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_ValidateConfigurationResponse:



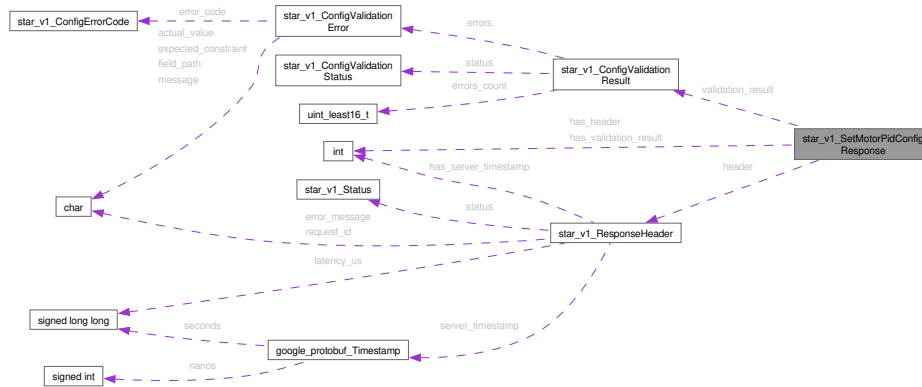
## Data Fields

|                                                |                      |                                       |  |
|------------------------------------------------|----------------------|---------------------------------------|--|
|                                                | <a href="#">bool</a> | <a href="#">has_header</a>            |  |
|                                                | <a href="#">bool</a> | <a href="#">has_validation_result</a> |  |
| <a href="#">star_v1_ResponseHeader</a>         |                      | <a href="#">header</a>                |  |
| <a href="#">star_v1_ConfigValidationResult</a> |                      | <a href="#">validation_result</a>     |  |

## 6.15.1.26 struct star\_v1\_SetMotorPidConfigResponse

Definition at line 348 of file [configuration.pb.h](#).

Collaboration diagram for star\_v1\_SetMotorPidConfigResponse:



## Data Fields

|                                                |                       |  |
|------------------------------------------------|-----------------------|--|
| <a href="#">bool</a>                           | has_header            |  |
| <a href="#">bool</a>                           | has_validation_result |  |
| <a href="#">star_v1_ResponseHeader</a>         | header                |  |
| <a href="#">star_v1_ConfigValidationResult</a> | validation_result     |  |

## 6.15.2 Macro Definition Documentation

## 6.15.2.1 \_\_star\_v1\_ConfigErrorCode\_ARRAYSIZE

```
#define __star_v1_ConfigErrorCode_ARRAYSIZE ((star_v1_ConfigErrorCode)(star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_STORAGE_ERROR))
```

Definition at line 369 of file [configuration.pb.h](#).

## 6.15.2.2 \_\_star\_v1\_ConfigErrorCode\_MAX

```
#define __star_v1_ConfigErrorCode_MAX star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_STORAGE_ERROR
```

Definition at line 368 of file [configuration.pb.h](#).

## 6.15.2.3 \_\_star\_v1\_ConfigErrorCode\_MIN

```
#define __star_v1_ConfigErrorCode_MIN star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_UNKNOWN
```

Definition at line 367 of file [configuration.pb.h](#).

#### 6.15.2.4 \_\_star\_v1\_ConfigValidationStatus\_\_ARRAYSIZE

```
#define __star_v1_ConfigValidationStatus__ARRAYSIZE ((star_v1_ConfigValidationStatus)(star_v1_ConfigValidationStatus_CONFIG_V
```

Definition at line 365 of file [configuration.pb.h](#).

#### 6.15.2.5 \_\_star\_v1\_ConfigValidationStatus\_\_MAX

```
#define __star_v1_ConfigValidationStatus__MAX star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_VERSION_MISM
```

Definition at line 364 of file [configuration.pb.h](#).

#### 6.15.2.6 \_\_star\_v1\_ConfigValidationStatus\_\_MIN

```
#define __star_v1_ConfigValidationStatus__MIN star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_UNKNOWN
```

Definition at line 363 of file [configuration.pb.h](#).

#### 6.15.2.7 star\_v1\_ConfigValidationError\_\_actual\_value\_tag

```
#define star_v1_ConfigValidationError__actual_value_tag 4
```

Definition at line 512 of file [configuration.pb.h](#).

#### 6.15.2.8 star\_v1\_ConfigValidationError\_\_CALLBACK

```
#define star_v1_ConfigValidationError__CALLBACK NULL
```

Definition at line 726 of file [configuration.pb.h](#).

#### 6.15.2.9 star\_v1\_ConfigValidationError\_\_DEFAULT

```
#define star_v1_ConfigValidationError__DEFAULT NULL
```

Definition at line 727 of file [configuration.pb.h](#).

#### 6.15.2.10 star\_v1\_ConfigValidationError\_\_error\_code\_\_ENUMTYPE

```
#define star_v1_ConfigValidationError__error_code__ENUMTYPE star_v1_ConfigErrorCode
```

Definition at line 397 of file [configuration.pb.h](#).

#### 6.15.2.11 star\_v1\_ConfigValidationError\_\_error\_code\_tag

```
#define star_v1_ConfigValidationError__error_code_tag 2
```

Definition at line 510 of file [configuration.pb.h](#).

## 6.15.2.12 star\_v1\_ConfigValidationError\_expected\_constraint\_tag

```
#define star_v1_ConfigValidationError_expected_constraint_tag 5
```

Definition at line 513 of file [configuration.pb.h](#).

## 6.15.2.13 star\_v1\_ConfigValidationError\_field\_path\_tag

```
#define star_v1_ConfigValidationError_field_path_tag 1
```

Definition at line 509 of file [configuration.pb.h](#).

## 6.15.2.14 star\_v1\_ConfigValidationError\_FIELDLIST

```
#define star_v1_ConfigValidationError_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, STRING, field_path, 1) \
X(a, STATIC, SINGULAR, UENUM, error_code, 2) \
X(a, STATIC, SINGULAR, STRING, message, 3) \
X(a, STATIC, SINGULAR, STRING, actual_value, 4) \
X(a, STATIC, SINGULAR, STRING, expected_constraint, 5)
```

Definition at line 720 of file [configuration.pb.h](#).

```
00720 #define star_v1_ConfigValidationError_FIELDLIST(X, a) \
00721 X(a, STATIC, SINGULAR, STRING, field_path, 1) \
00722 X(a, STATIC, SINGULAR, UENUM, error_code, 2) \
00723 X(a, STATIC, SINGULAR, STRING, message, 3) \
00724 X(a, STATIC, SINGULAR, STRING, actual_value, 4) \
00725 X(a, STATIC, SINGULAR, STRING, expected_constraint, 5)
```

## 6.15.2.15 star\_v1\_ConfigValidationError\_fields

```
#define star_v1_ConfigValidationError_fields &star_v1_ConfigValidationError_msg
```

Definition at line 782 of file [configuration.pb.h](#).

## 6.15.2.16 star\_v1\_ConfigValidationError\_init\_default

```
#define star_v1_ConfigValidationError_init_default {"", _star_v1_ConfigErrorCode_MIN, "", "", ""}
```

Definition at line 426 of file [configuration.pb.h](#).

## 6.15.2.17 star\_v1\_ConfigValidationError\_init\_zero

```
#define star_v1_ConfigValidationError_init_zero {"", _star_v1_ConfigErrorCode_MIN, "", "", ""}
```

Definition at line 452 of file [configuration.pb.h](#).

6.15.2.18 `star_v1_ConfigValidationError_message_tag`

```
#define star_v1_ConfigValidationError_message_tag 3
```

Definition at line 511 of file [configuration.pb.h](#).

6.15.2.19 `star_v1_ConfigValidationError_size`

```
#define star_v1_ConfigValidationError_size 585
```

Definition at line 792 of file [configuration.pb.h](#).

6.15.2.20 `star_v1_ConfigValidationResult_CALLBACK`

```
#define star_v1_ConfigValidationResult_CALLBACK NULL
```

Definition at line 716 of file [configuration.pb.h](#).

6.15.2.21 `star_v1_ConfigValidationResult_DEFAULT`

```
#define star_v1_ConfigValidationResult_DEFAULT NULL
```

Definition at line 717 of file [configuration.pb.h](#).

6.15.2.22 `star_v1_ConfigValidationResult_errors_MSGTYPE`

```
#define star_v1_ConfigValidationResult_errors_MSGTYPE star_v1_ConfigValidationError
```

Definition at line 718 of file [configuration.pb.h](#).

6.15.2.23 `star_v1_ConfigValidationResult_errors_tag`

```
#define star_v1_ConfigValidationResult_errors_tag 2
```

Definition at line 515 of file [configuration.pb.h](#).

6.15.2.24 `star_v1_ConfigValidationResult_FIELDLIST`

```
#define star_v1_ConfigValidationResult_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, UENUM, status, 1) \
X(a, STATIC, REPEATED, MESSAGE, errors, 2)
```

Definition at line 713 of file [configuration.pb.h](#).

```
00713 #define star_v1_ConfigValidationResult_FIELDLIST(X, a) \
00714 X(a, STATIC, SINGULAR, UENUM, status, 1) \
00715 X(a, STATIC, REPEATED, MESSAGE, errors, 2)
```

## 6.15.2.25 star\_v1\_ConfigValidationResult\_fields

```
#define star_v1_ConfigValidationResult_fields &star_v1_ConfigValidationResult_msg
```

Definition at line 781 of file [configuration.pb.h](#).

## 6.15.2.26 star\_v1\_ConfigValidationResult\_init\_default

```
#define star_v1_ConfigValidationResult_init_default {__star_v1_ConfigValidationStatus_MIN, 0, {star_v1_ConfigValidationError_init_d
star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default,
star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default,
star_v1_ConfigValidationError_init_default}}
```

Definition at line 425 of file [configuration.pb.h](#).

## 6.15.2.27 star\_v1\_ConfigValidationResult\_init\_zero

```
#define star_v1_ConfigValidationResult_init_zero {__star_v1_ConfigValidationStatus_MIN, 0, {star_v1_ConfigValidationError_init_zero
star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero,
star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero,
star_v1_ConfigValidationError_init_zero}}
```

Definition at line 451 of file [configuration.pb.h](#).

## 6.15.2.28 star\_v1\_ConfigValidationResult\_size

```
#define star_v1_ConfigValidationResult_size 4706
```

Definition at line 793 of file [configuration.pb.h](#).

## 6.15.2.29 star\_v1\_ConfigValidationResult\_status\_ENUMTYPE

```
#define star_v1_ConfigValidationResult_status_ENUMTYPE star_v1_ConfigValidationStatus
```

Definition at line 395 of file [configuration.pb.h](#).

## 6.15.2.30 star\_v1\_ConfigValidationResult\_status\_tag

```
#define star_v1_ConfigValidationResult_status_tag 1
```

Definition at line 514 of file [configuration.pb.h](#).

## 6.15.2.31 star\_v1\_CurrentSensorCalibration\_CALLBACK

```
#define star_v1_CurrentSensorCalibration_CALLBACK pb_default_field_callback
```

Definition at line 684 of file [configuration.pb.h](#).

6.15.2.32 `star_v1_CurrentSensorCalibration_DEFAULT`

```
#define star_v1_CurrentSensorCalibration_DEFAULT NULL
```

Definition at line 685 of file [configuration.pb.h](#).

6.15.2.33 `star_v1_CurrentSensorCalibration_FIELDLIST`

```
#define star_v1_CurrentSensorCalibration_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, CALLBACK, REPEATED, DOUBLE, offset_ma, 1) \
X(a, CALLBACK, REPEATED, DOUBLE, scale_factor, 2)
```

Definition at line 681 of file [configuration.pb.h](#).

```
00681 #define star_v1_CurrentSensorCalibration_FIELDLIST(X, a) \
00682 X(a, CALLBACK, REPEATED, DOUBLE, offset_ma, 1) \
00683 X(a, CALLBACK, REPEATED, DOUBLE, scale_factor, 2)
```

6.15.2.34 `star_v1_CurrentSensorCalibration_fields`

```
#define star_v1_CurrentSensorCalibration_fields &star_v1_CurrentSensorCalibration_msg
```

Definition at line 776 of file [configuration.pb.h](#).

6.15.2.35 `star_v1_CurrentSensorCalibration_init_default`

```
#define star_v1_CurrentSensorCalibration_init_default {{{NULL}, NULL}, {{{NULL}, NULL}}}
```

Definition at line 420 of file [configuration.pb.h](#).

6.15.2.36 `star_v1_CurrentSensorCalibration_init_zero`

```
#define star_v1_CurrentSensorCalibration_init_zero {{{NULL}, NULL}, {{{NULL}, NULL}}}
```

Definition at line 446 of file [configuration.pb.h](#).

6.15.2.37 `star_v1_CurrentSensorCalibration_offset_ma_tag`

```
#define star_v1_CurrentSensorCalibration_offset_ma_tag 1
```

Definition at line 480 of file [configuration.pb.h](#).

6.15.2.38 `star_v1_CurrentSensorCalibration_scale_factor_tag`

```
#define star_v1_CurrentSensorCalibration_scale_factor_tag 2
```

Definition at line 481 of file [configuration.pb.h](#).



## 6.15.2.39 star\_v1\_EncoderConfiguration\_CALLBACK

```
#define star_v1_EncoderConfiguration_CALLBACK NULL
```

Definition at line 703 of file [configuration.pb.h](#).

## 6.15.2.40 star\_v1\_EncoderConfiguration\_DEFAULT

```
#define star_v1_EncoderConfiguration_DEFAULT NULL
```

Definition at line 704 of file [configuration.pb.h](#).

## 6.15.2.41 star\_v1\_EncoderConfiguration\_edges\_per\_revolution\_tag

```
#define star_v1_EncoderConfiguration_edges_per_revolution_tag 1
```

Definition at line 486 of file [configuration.pb.h](#).

## 6.15.2.42 star\_v1\_EncoderConfiguration\_FIELDLIST

```
#define star_v1_EncoderConfiguration_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, UINT32, edges_per_revolution, 1) \
X(a, STATIC, SINGULAR, DOUBLE, wheel_diameter_m, 2) \
X(a, STATIC, SINGULAR, DOUBLE, gear_ratio, 3)
```

Definition at line 699 of file [configuration.pb.h](#).

```
00699 #define star_v1_EncoderConfiguration_FIELDLIST(X, a) \
00700 X(a, STATIC, SINGULAR, UINT32, edges_per_revolution, 1) \
00701 X(a, STATIC, SINGULAR, DOUBLE, wheel_diameter_m, 2) \
00702 X(a, STATIC, SINGULAR, DOUBLE, gear_ratio, 3)
```

## 6.15.2.43 star\_v1\_EncoderConfiguration\_fields

```
#define star_v1_EncoderConfiguration_fields &star_v1_EncoderConfiguration_msg
```

Definition at line 779 of file [configuration.pb.h](#).

## 6.15.2.44 star\_v1\_EncoderConfiguration\_gear\_ratio\_tag

```
#define star_v1_EncoderConfiguration_gear_ratio_tag 3
```

Definition at line 488 of file [configuration.pb.h](#).

## 6.15.2.45 star\_v1\_EncoderConfiguration\_init\_default

```
#define star_v1_EncoderConfiguration_init_default {0, 0, 0}
```

Definition at line 423 of file [configuration.pb.h](#).

6.15.2.46 `star_v1_EncoderConfiguration_init_zero`

```
#define star_v1_EncoderConfiguration_init_zero {0, 0, 0}
```

Definition at line 449 of file [configuration.pb.h](#).

6.15.2.47 `star_v1_EncoderConfiguration_size`

```
#define star_v1_EncoderConfiguration_size 24
```

Definition at line 794 of file [configuration.pb.h](#).

6.15.2.48 `star_v1_EncoderConfiguration_wheel_diameter_m_tag`

```
#define star_v1_EncoderConfiguration_wheel_diameter_m_tag 2
```

Definition at line 487 of file [configuration.pb.h](#).

6.15.2.49 `star_v1_GetConfigurationRequest_CALLBACK`

```
#define star_v1_GetConfigurationRequest_CALLBACK NULL
```

Definition at line 526 of file [configuration.pb.h](#).

6.15.2.50 `star_v1_GetConfigurationRequest_DEFAULT`

```
#define star_v1_GetConfigurationRequest_DEFAULT NULL
```

Definition at line 527 of file [configuration.pb.h](#).

6.15.2.51 `star_v1_GetConfigurationRequest_FIELDLIST`

```
#define star_v1_GetConfigurationRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

Definition at line 524 of file [configuration.pb.h](#).

```
00524 #define star_v1_GetConfigurationRequest_FIELDLIST(X, a) \
00525 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

6.15.2.52 `star_v1_GetConfigurationRequest_fields`

```
#define star_v1_GetConfigurationRequest_fields &star_v1_GetConfigurationRequest_msg
```

Definition at line 757 of file [configuration.pb.h](#).

6.15.2.53 `star_v1_GetConfigurationRequest_header_MSGTYPE`

```
#define star_v1_GetConfigurationRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 528 of file [configuration.pb.h](#).

6.15.2.54 `star_v1_GetConfigurationRequest_header_tag`

```
#define star_v1_GetConfigurationRequest_header_tag 1
```

Definition at line 455 of file [configuration.pb.h](#).

6.15.2.55 `star_v1_GetConfigurationRequest_init_default`

```
#define star_v1_GetConfigurationRequest_init_default {false, star_v1_RequestHeader_init_default}
```

Definition at line 401 of file [configuration.pb.h](#).

6.15.2.56 `star_v1_GetConfigurationRequest_init_zero`

```
#define star_v1_GetConfigurationRequest_init_zero {false, star_v1_RequestHeader_init_zero}
```

Definition at line 427 of file [configuration.pb.h](#).

6.15.2.57 `star_v1_GetConfigurationRequest_size`

```
#define star_v1_GetConfigurationRequest_size 149
```

Definition at line 795 of file [configuration.pb.h](#).

6.15.2.58 `star_v1_GetConfigurationResponse_CALLBACK`

```
#define star_v1_GetConfigurationResponse_CALLBACK NULL
```

Definition at line 533 of file [configuration.pb.h](#).

6.15.2.59 `star_v1_GetConfigurationResponse_configuration_MSGTYPE`

```
#define star_v1_GetConfigurationResponse_configuration_MSGTYPE star_v1_SystemConfiguration
```

Definition at line 536 of file [configuration.pb.h](#).

6.15.2.60 `star_v1_GetConfigurationResponse_configuration_tag`

```
#define star_v1_GetConfigurationResponse_configuration_tag 2
```

Definition at line 501 of file [configuration.pb.h](#).

6.15.2.61 `star_v1_GetConfigurationResponse_DEFAULT`

```
#define star_v1_GetConfigurationResponse_DEFAULT NULL
```

Definition at line 534 of file [configuration.pb.h](#).

6.15.2.62 `star_v1_GetConfigurationResponse_FIELDLIST`

```
#define star_v1_GetConfigurationResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2)
```

Definition at line 530 of file [configuration.pb.h](#).

```
00530 #define star_v1_GetConfigurationResponse_FIELDLIST(X, a) \
00531 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00532 X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2)
```

6.15.2.63 `star_v1_GetConfigurationResponse_fields`

```
#define star_v1_GetConfigurationResponse_fields &star_v1_GetConfigurationResponse_msg
```

Definition at line 758 of file [configuration.pb.h](#).

6.15.2.64 `star_v1_GetConfigurationResponse_header_MSGTYPE`

```
#define star_v1_GetConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 535 of file [configuration.pb.h](#).

6.15.2.65 `star_v1_GetConfigurationResponse_header_tag`

```
#define star_v1_GetConfigurationResponse_header_tag 1
```

Definition at line 500 of file [configuration.pb.h](#).

6.15.2.66 `star_v1_GetConfigurationResponse_init_default`

```
#define star_v1_GetConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
star_v1_SystemConfiguration_init_default}
```

Definition at line 402 of file [configuration.pb.h](#).

6.15.2.67 `star_v1_GetConfigurationResponse_init_zero`

```
#define star_v1_GetConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_SystemConfiguration_init
```

Definition at line 428 of file [configuration.pb.h](#).

## 6.15.2.68 star\_v1\_GetMotorPidConfigRequest\_\_CALLBACK

```
#define star_v1_GetMotorPidConfigRequest__CALLBACK NULL
```

Definition at line 603 of file [configuration.pb.h](#).

## 6.15.2.69 star\_v1\_GetMotorPidConfigRequest\_\_DEFAULT

```
#define star_v1_GetMotorPidConfigRequest__DEFAULT NULL
```

Definition at line 604 of file [configuration.pb.h](#).

## 6.15.2.70 star\_v1\_GetMotorPidConfigRequest\_\_FIELDLIST

```
#define star_v1_GetMotorPidConfigRequest__FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, INT32, motor_id, 2)
```

Definition at line 600 of file [configuration.pb.h](#).

```
00600 #define star_v1_GetMotorPidConfigRequest__FIELDLIST(X, a) \
00601 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00602 X(a, STATIC, SINGULAR, INT32, motor_id, 2)
```

## 6.15.2.71 star\_v1\_GetMotorPidConfigRequest\_\_fields

```
#define star_v1_GetMotorPidConfigRequest__fields &star_v1_GetMotorPidConfigRequest__msg
```

Definition at line 767 of file [configuration.pb.h](#).

## 6.15.2.72 star\_v1\_GetMotorPidConfigRequest\_\_header\_\_MSGTYPE

```
#define star_v1_GetMotorPidConfigRequest__header__MSGTYPE star_v1_RequestHeader
```

Definition at line 605 of file [configuration.pb.h](#).

## 6.15.2.73 star\_v1\_GetMotorPidConfigRequest\_\_header\_\_tag

```
#define star_v1_GetMotorPidConfigRequest__header__tag 1
```

Definition at line 462 of file [configuration.pb.h](#).

## 6.15.2.74 star\_v1\_GetMotorPidConfigRequest\_\_init\_\_default

```
#define star_v1_GetMotorPidConfigRequest__init__default {false, star_v1_RequestHeader__init__default, 0}
```

Definition at line 411 of file [configuration.pb.h](#).

6.15.2.75 `star_v1_GetMotorPidConfigRequest_init_zero`

```
#define star_v1_GetMotorPidConfigRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}
```

Definition at line 437 of file [configuration.pb.h](#).

6.15.2.76 `star_v1_GetMotorPidConfigRequest_motor_id_tag`

```
#define star_v1_GetMotorPidConfigRequest_motor_id_tag 2
```

Definition at line 463 of file [configuration.pb.h](#).

6.15.2.77 `star_v1_GetMotorPidConfigRequest_size`

```
#define star_v1_GetMotorPidConfigRequest_size 160
```

Definition at line 796 of file [configuration.pb.h](#).

6.15.2.78 `star_v1_GetMotorPidConfigResponse_CALLBACK`

```
#define star_v1_GetMotorPidConfigResponse_CALLBACK NULL
```

Definition at line 611 of file [configuration.pb.h](#).

6.15.2.79 `star_v1_GetMotorPidConfigResponse_DEFAULT`

```
#define star_v1_GetMotorPidConfigResponse_DEFAULT NULL
```

Definition at line 612 of file [configuration.pb.h](#).

6.15.2.80 `star_v1_GetMotorPidConfigResponse_FIELDLIST`

```
#define star_v1_GetMotorPidConfigResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, INT32, motor_id, 2) \
X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 3)
```

Definition at line 607 of file [configuration.pb.h](#).

```
00607 #define star_v1_GetMotorPidConfigResponse_FIELDLIST(X, a) \
00608 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00609 X(a, STATIC, SINGULAR, INT32, motor_id, 2) \
00610 X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 3)
```

6.15.2.81 `star_v1_GetMotorPidConfigResponse_fields`

```
#define star_v1_GetMotorPidConfigResponse_fields &star_v1_GetMotorPidConfigResponse_msg
```

Definition at line 768 of file [configuration.pb.h](#).

## 6.15.2.82 star\_v1\_GetMotorPidConfigResponse\_header\_MSGTYPE

```
#define star_v1_GetMotorPidConfigResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 613 of file [configuration.pb.h](#).

## 6.15.2.83 star\_v1\_GetMotorPidConfigResponse\_header\_tag

```
#define star_v1_GetMotorPidConfigResponse_header_tag 1
```

Definition at line 464 of file [configuration.pb.h](#).

## 6.15.2.84 star\_v1\_GetMotorPidConfigResponse\_init\_default

```
#define star_v1_GetMotorPidConfigResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, false, star_v1_PidConfig_init_default}
```

Definition at line 412 of file [configuration.pb.h](#).

## 6.15.2.85 star\_v1\_GetMotorPidConfigResponse\_init\_zero

```
#define star_v1_GetMotorPidConfigResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, false, star_v1_PidConfig_init_zero}
```

Definition at line 438 of file [configuration.pb.h](#).

## 6.15.2.86 star\_v1\_GetMotorPidConfigResponse\_motor\_id\_tag

```
#define star_v1_GetMotorPidConfigResponse_motor_id_tag 2
```

Definition at line 465 of file [configuration.pb.h](#).

## 6.15.2.87 star\_v1\_GetMotorPidConfigResponse\_pid\_config\_MSGTYPE

```
#define star_v1_GetMotorPidConfigResponse_pid_config_MSGTYPE star_v1_PidConfig
```

Definition at line 614 of file [configuration.pb.h](#).

## 6.15.2.88 star\_v1\_GetMotorPidConfigResponse\_pid\_config\_tag

```
#define star_v1_GetMotorPidConfigResponse_pid_config_tag 3
```

Definition at line 466 of file [configuration.pb.h](#).

6.15.2.89 `star_v1_GetMotorPidConfigResponse_size`

```
#define star_v1_GetMotorPidConfigResponse_size 439
```

Definition at line 797 of file [configuration.pb.h](#).

6.15.2.90 `star_v1_MotorPidConfiguration_CALLBACK`

```
#define star_v1_MotorPidConfiguration_CALLBACK NULL
```

Definition at line 677 of file [configuration.pb.h](#).

6.15.2.91 `star_v1_MotorPidConfiguration_DEFAULT`

```
#define star_v1_MotorPidConfiguration_DEFAULT NULL
```

Definition at line 678 of file [configuration.pb.h](#).

6.15.2.92 `star_v1_MotorPidConfiguration_FIELDLIST`

```
#define star_v1_MotorPidConfiguration_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, INT32, motor_id, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 2)
```

Definition at line 674 of file [configuration.pb.h](#).

```
00674 #define star_v1_MotorPidConfiguration_FIELDLIST(X, a) \
00675 X(a, STATIC, SINGULAR, INT32, motor_id, 1) \
00676 X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 2)
```

6.15.2.93 `star_v1_MotorPidConfiguration_fields`

```
#define star_v1_MotorPidConfiguration_fields &star_v1_MotorPidConfiguration_msg
```

Definition at line 775 of file [configuration.pb.h](#).

6.15.2.94 `star_v1_MotorPidConfiguration_init_default`

```
#define star_v1_MotorPidConfiguration_init_default {0, false, star_v1_PidConfig_init_default}
```

Definition at line 419 of file [configuration.pb.h](#).

6.15.2.95 `star_v1_MotorPidConfiguration_init_zero`

```
#define star_v1_MotorPidConfiguration_init_zero {0, false, star_v1_PidConfig_init_zero}
```

Definition at line 445 of file [configuration.pb.h](#).



## 6.15.2.96 star\_v1\_MotorPidConfiguration\_motor\_id\_tag

```
#define star_v1_MotorPidConfiguration_motor_id_tag 1
```

Definition at line 478 of file [configuration.pb.h](#).

## 6.15.2.97 star\_v1\_MotorPidConfiguration\_pid\_config\_MSGTYPE

```
#define star_v1_MotorPidConfiguration_pid_config_MSGTYPE star_v1_PidConfig
```

Definition at line 679 of file [configuration.pb.h](#).

## 6.15.2.98 star\_v1\_MotorPidConfiguration\_pid\_config\_tag

```
#define star_v1_MotorPidConfiguration_pid_config_tag 2
```

Definition at line 479 of file [configuration.pb.h](#).

## 6.15.2.99 star\_v1\_MotorPidConfiguration\_size

```
#define star_v1_MotorPidConfiguration_size 76
```

Definition at line 798 of file [configuration.pb.h](#).

## 6.15.2.100 star\_v1\_ResetToDefaultsRequest\_CALLBACK

```
#define star_v1_ResetToDefaultsRequest_CALLBACK NULL
```

Definition at line 558 of file [configuration.pb.h](#).

## 6.15.2.101 star\_v1\_ResetToDefaultsRequest\_DEFAULT

```
#define star_v1_ResetToDefaultsRequest_DEFAULT NULL
```

Definition at line 559 of file [configuration.pb.h](#).

## 6.15.2.102 star\_v1\_ResetToDefaultsRequest\_FIELDLIST

```
#define star_v1_ResetToDefaultsRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, BOOL, persist_to_nvs, 2)
```

Definition at line 555 of file [configuration.pb.h](#).

```
00555 #define star_v1_ResetToDefaultsRequest_FIELDLIST(X, a) \
00556 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00557 X(a, STATIC, SINGULAR, BOOL, persist_to_nvs, 2)
```

#### 6.15.2.103 star\_v1\_ResetToDefaultsRequest\_fields

```
#define star_v1_ResetToDefaultsRequest_fields &star_v1_ResetToDefaultsRequest_msg
```

Definition at line 761 of file [configuration.pb.h](#).

#### 6.15.2.104 star\_v1\_ResetToDefaultsRequest\_header\_MSGTYPE

```
#define star_v1_ResetToDefaultsRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 560 of file [configuration.pb.h](#).

#### 6.15.2.105 star\_v1\_ResetToDefaultsRequest\_header\_tag

```
#define star_v1_ResetToDefaultsRequest_header_tag 1
```

Definition at line 456 of file [configuration.pb.h](#).

#### 6.15.2.106 star\_v1\_ResetToDefaultsRequest\_init\_default

```
#define star_v1_ResetToDefaultsRequest_init_default {false, star_v1_RequestHeader_init_default, 0}
```

Definition at line 405 of file [configuration.pb.h](#).

#### 6.15.2.107 star\_v1\_ResetToDefaultsRequest\_init\_zero

```
#define star_v1_ResetToDefaultsRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}
```

Definition at line 431 of file [configuration.pb.h](#).

#### 6.15.2.108 star\_v1\_ResetToDefaultsRequest\_persist\_to\_nvs\_tag

```
#define star_v1_ResetToDefaultsRequest_persist_to_nvs_tag 2
```

Definition at line 457 of file [configuration.pb.h](#).

#### 6.15.2.109 star\_v1\_ResetToDefaultsRequest\_size

```
#define star_v1_ResetToDefaultsRequest_size 151
```

Definition at line 799 of file [configuration.pb.h](#).

#### 6.15.2.110 star\_v1\_ResetToDefaultsResponse\_CALLBACK

```
#define star_v1_ResetToDefaultsResponse_CALLBACK NULL
```

Definition at line 565 of file [configuration.pb.h](#).

## 6.15.2.111 star\_v1\_ResetToDefaultsResponse\_configuration\_MSGTYPE

```
#define star_v1_ResetToDefaultsResponse_configuration_MSGTYPE star_v1_SystemConfiguration
```

Definition at line 568 of file [configuration.pb.h](#).

## 6.15.2.112 star\_v1\_ResetToDefaultsResponse\_configuration\_tag

```
#define star_v1_ResetToDefaultsResponse_configuration_tag 2
```

Definition at line 506 of file [configuration.pb.h](#).

## 6.15.2.113 star\_v1\_ResetToDefaultsResponse\_DEFAULT

```
#define star_v1_ResetToDefaultsResponse_DEFAULT NULL
```

Definition at line 566 of file [configuration.pb.h](#).

## 6.15.2.114 star\_v1\_ResetToDefaultsResponse\_FIELDLIST

```
#define star_v1_ResetToDefaultsResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2)
```

Definition at line 562 of file [configuration.pb.h](#).

```
00562 #define star_v1_ResetToDefaultsResponse_FIELDLIST(X, a) \
00563 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00564 X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2)
```

## 6.15.2.115 star\_v1\_ResetToDefaultsResponse\_fields

```
#define star_v1_ResetToDefaultsResponse_fields &star_v1_ResetToDefaultsResponse_msg
```

Definition at line 762 of file [configuration.pb.h](#).

## 6.15.2.116 star\_v1\_ResetToDefaultsResponse\_header\_MSGTYPE

```
#define star_v1_ResetToDefaultsResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 567 of file [configuration.pb.h](#).

## 6.15.2.117 star\_v1\_ResetToDefaultsResponse\_header\_tag

```
#define star_v1_ResetToDefaultsResponse_header_tag 1
```

Definition at line 505 of file [configuration.pb.h](#).

## 6.15.2.118 star\_v1\_ResetToDefaultsResponse\_\_init\_\_default

```
#define star_v1_ResetToDefaultsResponse__init__default {false, star_v1_ResponseHeader__init__default, false,
star_v1_SystemConfiguration__init__default}
```

Definition at line 406 of file [configuration.pb.h](#).

## 6.15.2.119 star\_v1\_ResetToDefaultsResponse\_\_init\_\_zero

```
#define star_v1_ResetToDefaultsResponse__init__zero {false, star_v1_ResponseHeader__init__zero, false, star_v1_SystemConfiguration__init__zero}
```

Definition at line 432 of file [configuration.pb.h](#).

## 6.15.2.120 star\_v1\_RetransmitConfig\_ack\_timeout\_ms\_tag

```
#define star_v1_RetransmitConfig_ack_timeout_ms_tag 3
```

Definition at line 473 of file [configuration.pb.h](#).

## 6.15.2.121 star\_v1\_RetransmitConfig\_CALLBACK

```
#define star_v1_RetransmitConfig_CALLBACK NULL
```

Definition at line 639 of file [configuration.pb.h](#).

## 6.15.2.122 star\_v1\_RetransmitConfig\_DEFAULT

```
#define star_v1_RetransmitConfig_DEFAULT NULL
```

Definition at line 640 of file [configuration.pb.h](#).

## 6.15.2.123 star\_v1\_RetransmitConfig\_enabled\_tag

```
#define star_v1_RetransmitConfig_enabled_tag 1
```

Definition at line 471 of file [configuration.pb.h](#).

## 6.15.2.124 star\_v1\_RetransmitConfig\_FIELDLIST

```
#define star_v1_RetransmitConfig_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, BOOL, enabled, 1) \
X(a, STATIC, SINGULAR, UINT32, max_retries, 2) \
X(a, STATIC, SINGULAR, UINT32, ack_timeout_ms, 3) \
X(a, STATIC, SINGULAR, UINT32, max_backoff_ms, 4)
```

Definition at line 634 of file [configuration.pb.h](#).

```
00634 #define star_v1_RetransmitConfig_FIELDLIST(X, a) \
00635 X(a, STATIC, SINGULAR, BOOL, enabled, 1) \
00636 X(a, STATIC, SINGULAR, UINT32, max_retries, 2) \
00637 X(a, STATIC, SINGULAR, UINT32, ack_timeout_ms, 3) \
00638 X(a, STATIC, SINGULAR, UINT32, max_backoff_ms, 4)
```

## 6.15.2.125 star\_v1\_RetransmitConfig\_fields

```
#define star_v1_RetransmitConfig_fields &star_v1_RetransmitConfig_msg
```

Definition at line 771 of file [configuration.pb.h](#).

## 6.15.2.126 star\_v1\_RetransmitConfig\_init\_default

```
#define star_v1_RetransmitConfig_init_default {0, 0, 0, 0}
```

Definition at line 415 of file [configuration.pb.h](#).

## 6.15.2.127 star\_v1\_RetransmitConfig\_init\_zero

```
#define star_v1_RetransmitConfig_init_zero {0, 0, 0, 0}
```

Definition at line 441 of file [configuration.pb.h](#).

## 6.15.2.128 star\_v1\_RetransmitConfig\_max\_backoff\_ms\_tag

```
#define star_v1_RetransmitConfig_max_backoff_ms_tag 4
```

Definition at line 474 of file [configuration.pb.h](#).

## 6.15.2.129 star\_v1\_RetransmitConfig\_max\_retries\_tag

```
#define star_v1_RetransmitConfig_max_retries_tag 2
```

Definition at line 472 of file [configuration.pb.h](#).

## 6.15.2.130 star\_v1\_RetransmitConfig\_size

```
#define star_v1_RetransmitConfig_size 20
```

Definition at line 800 of file [configuration.pb.h](#).

## 6.15.2.131 star\_v1\_SafetyThresholds\_CALLBACK

```
#define star_v1_SafetyThresholds_CALLBACK NULL
```

Definition at line 696 of file [configuration.pb.h](#).

## 6.15.2.132 star\_v1\_SafetyThresholds\_cell\_imbalance\_threshold\_mv\_tag

```
#define star_v1_SafetyThresholds_cell_imbalance_threshold_mv_tag 3
```

Definition at line 485 of file [configuration.pb.h](#).

## 6.15.2.133 star\_v1\_SafetyThresholds\_DEFAULT

```
#define star_v1_SafetyThresholds_DEFAULT NULL
```

Definition at line 697 of file [configuration.pb.h](#).

## 6.15.2.134 star\_v1\_SafetyThresholds\_FIELDLIST

```
#define star_v1_SafetyThresholds_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, UINT32, overcurrent_threshold_ma, 1) \
X(a, STATIC, SINGULAR, INT32, thermal_shutdown_deci_celsius, 2) \
X(a, STATIC, SINGULAR, UINT32, cell_imbalance_threshold_mv, 3)
```

Definition at line 692 of file [configuration.pb.h](#).

```
00692 #define star_v1_SafetyThresholds_FIELDLIST(X, a) \
00693 X(a, STATIC, SINGULAR, UINT32, overcurrent_threshold_ma, 1) \
00694 X(a, STATIC, SINGULAR, INT32, thermal_shutdown_deci_celsius, 2) \
00695 X(a, STATIC, SINGULAR, UINT32, cell_imbalance_threshold_mv, 3)
```

## 6.15.2.135 star\_v1\_SafetyThresholds\_fields

```
#define star_v1_SafetyThresholds_fields &star_v1_SafetyThresholds_msg
```

Definition at line 778 of file [configuration.pb.h](#).

## 6.15.2.136 star\_v1\_SafetyThresholds\_init\_default

```
#define star_v1_SafetyThresholds_init_default {0, 0, 0}
```

Definition at line 422 of file [configuration.pb.h](#).

## 6.15.2.137 star\_v1\_SafetyThresholds\_init\_zero

```
#define star_v1_SafetyThresholds_init_zero {0, 0, 0}
```

Definition at line 448 of file [configuration.pb.h](#).

## 6.15.2.138 star\_v1\_SafetyThresholds\_overcurrent\_threshold\_ma\_tag

```
#define star_v1_SafetyThresholds_overcurrent_threshold_ma_tag 1
```

Definition at line 483 of file [configuration.pb.h](#).

## 6.15.2.139 star\_v1\_SafetyThresholds\_size

```
#define star_v1_SafetyThresholds_size 23
```

Definition at line 801 of file [configuration.pb.h](#).

## 6.15.2.140 star\_v1\_SafetyThresholds\_thermal\_shutdown\_deci\_celsius\_tag

```
#define star_v1_SafetyThresholds_thermal_shutdown_deci_celsius_tag 2
```

Definition at line 484 of file [configuration.pb.h](#).

## 6.15.2.141 star\_v1\_SaveConfigurationRequest\_CALLBACK

```
#define star_v1_SaveConfigurationRequest_CALLBACK NULL
```

Definition at line 588 of file [configuration.pb.h](#).

## 6.15.2.142 star\_v1\_SaveConfigurationRequest\_DEFAULT

```
#define star_v1_SaveConfigurationRequest_DEFAULT NULL
```

Definition at line 589 of file [configuration.pb.h](#).

## 6.15.2.143 star\_v1\_SaveConfigurationRequest\_FIELDLIST

```
#define star_v1_SaveConfigurationRequest_FIELDLIST(
 X,
 a)
```

Value:

X(a, STATIC, OPTIONAL, MESSAGE, header, 1)

Definition at line 586 of file [configuration.pb.h](#).

```
00586 #define star_v1_SaveConfigurationRequest_FIELDLIST(X, a) \
00587 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

## 6.15.2.144 star\_v1\_SaveConfigurationRequest\_fields

```
#define star_v1_SaveConfigurationRequest_fields &star_v1_SaveConfigurationRequest_msg
```

Definition at line 765 of file [configuration.pb.h](#).

## 6.15.2.145 star\_v1\_SaveConfigurationRequest\_header\_MSGTYPE

```
#define star_v1_SaveConfigurationRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 590 of file [configuration.pb.h](#).

## 6.15.2.146 star\_v1\_SaveConfigurationRequest\_header\_tag

```
#define star_v1_SaveConfigurationRequest_header_tag 1
```

Definition at line 458 of file [configuration.pb.h](#).

6.15.2.147 `star_v1_SaveConfigurationRequest_init_default`

```
#define star_v1_SaveConfigurationRequest_init_default {false, star_v1_RequestHeader_init_default}
```

Definition at line 409 of file [configuration.pb.h](#).

6.15.2.148 `star_v1_SaveConfigurationRequest_init_zero`

```
#define star_v1_SaveConfigurationRequest_init_zero {false, star_v1_RequestHeader_init_zero}
```

Definition at line 435 of file [configuration.pb.h](#).

6.15.2.149 `star_v1_SaveConfigurationRequest_size`

```
#define star_v1_SaveConfigurationRequest_size 149
```

Definition at line 802 of file [configuration.pb.h](#).

6.15.2.150 `star_v1_SaveConfigurationResponse_CALLBACK`

```
#define star_v1_SaveConfigurationResponse_CALLBACK NULL
```

Definition at line 596 of file [configuration.pb.h](#).

6.15.2.151 `star_v1_SaveConfigurationResponse_config_crc_tag`

```
#define star_v1_SaveConfigurationResponse_config_crc_tag 3
```

Definition at line 461 of file [configuration.pb.h](#).

6.15.2.152 `star_v1_SaveConfigurationResponse_DEFAULT`

```
#define star_v1_SaveConfigurationResponse_DEFAULT NULL
```

Definition at line 597 of file [configuration.pb.h](#).

6.15.2.153 `star_v1_SaveConfigurationResponse_FIELDLIST`

```
#define star_v1_SaveConfigurationResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, BOOL, saved, 2) \
X(a, STATIC, SINGULAR, UINT32, config_crc, 3)
```

Definition at line 592 of file [configuration.pb.h](#).

```
00592 #define star_v1_SaveConfigurationResponse_FIELDLIST(X, a) \
00593 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00594 X(a, STATIC, SINGULAR, BOOL, saved, 2) \
00595 X(a, STATIC, SINGULAR, UINT32, config_crc, 3)
```



## 6.15.2.154 star\_v1\_SaveConfigurationResponse\_fields

```
#define star_v1_SaveConfigurationResponse_fields &star_v1_SaveConfigurationResponse_msg
```

Definition at line 766 of file [configuration.pb.h](#).

## 6.15.2.155 star\_v1\_SaveConfigurationResponse\_header\_MSGTYPE

```
#define star_v1_SaveConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 598 of file [configuration.pb.h](#).

## 6.15.2.156 star\_v1\_SaveConfigurationResponse\_header\_tag

```
#define star_v1_SaveConfigurationResponse_header_tag 1
```

Definition at line 459 of file [configuration.pb.h](#).

## 6.15.2.157 star\_v1\_SaveConfigurationResponse\_init\_default

```
#define star_v1_SaveConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0}
```

Definition at line 410 of file [configuration.pb.h](#).

## 6.15.2.158 star\_v1\_SaveConfigurationResponse\_init\_zero

```
#define star_v1_SaveConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0}
```

Definition at line 436 of file [configuration.pb.h](#).

## 6.15.2.159 star\_v1\_SaveConfigurationResponse\_saved\_tag

```
#define star_v1_SaveConfigurationResponse_saved_tag 2
```

Definition at line 460 of file [configuration.pb.h](#).

## 6.15.2.160 star\_v1\_SaveConfigurationResponse\_size

```
#define star_v1_SaveConfigurationResponse_size 371
```

Definition at line 803 of file [configuration.pb.h](#).

## 6.15.2.161 star\_v1\_SetConfigurationRequest\_CALLBACK

```
#define star_v1_SetConfigurationRequest_CALLBACK NULL
```

Definition at line 542 of file [configuration.pb.h](#).

6.15.2.162 `star_v1_SetConfigurationRequest_configuration_MSGTYPE`

```
#define star_v1_SetConfigurationRequest_configuration_MSGTYPE star_v1_SystemConfiguration
```

Definition at line [545](#) of file [configuration.pb.h](#).

6.15.2.163 `star_v1_SetConfigurationRequest_configuration_tag`

```
#define star_v1_SetConfigurationRequest_configuration_tag 2
```

Definition at line [503](#) of file [configuration.pb.h](#).

6.15.2.164 `star_v1_SetConfigurationRequest_DEFAULT`

```
#define star_v1_SetConfigurationRequest_DEFAULT NULL
```

Definition at line [543](#) of file [configuration.pb.h](#).

6.15.2.165 `star_v1_SetConfigurationRequest_FIELDLIST`

```
#define star_v1_SetConfigurationRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2) \
X(a, STATIC, SINGULAR, BOOL, persist_to_nvs, 3)
```

Definition at line [538](#) of file [configuration.pb.h](#).

```
00538 #define star_v1_SetConfigurationRequest_FIELDLIST(X, a) \
00539 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00540 X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2) \
00541 X(a, STATIC, SINGULAR, BOOL, persist_to_nvs, 3)
```

6.15.2.166 `star_v1_SetConfigurationRequest_fields`

```
#define star_v1_SetConfigurationRequest_fields &star_v1_SetConfigurationRequest_msg
```

Definition at line [759](#) of file [configuration.pb.h](#).

6.15.2.167 `star_v1_SetConfigurationRequest_header_MSGTYPE`

```
#define star_v1_SetConfigurationRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line [544](#) of file [configuration.pb.h](#).

6.15.2.168 `star_v1_SetConfigurationRequest_header_tag`

```
#define star_v1_SetConfigurationRequest_header_tag 1
```

Definition at line [502](#) of file [configuration.pb.h](#).

## 6.15.2.169 star\_v1\_SetConfigurationRequest\_init\_default

```
#define star_v1_SetConfigurationRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_SystemConfiguration_init_z
0}
```

Definition at line 403 of file [configuration.pb.h](#).

## 6.15.2.170 star\_v1\_SetConfigurationRequest\_init\_zero

```
#define star_v1_SetConfigurationRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_SystemConfiguration_init_z
0}
```

Definition at line 429 of file [configuration.pb.h](#).

## 6.15.2.171 star\_v1\_SetConfigurationRequest\_persist\_to\_nvs\_tag

```
#define star_v1_SetConfigurationRequest_persist_to_nvs_tag 3
```

Definition at line 504 of file [configuration.pb.h](#).

## 6.15.2.172 star\_v1\_SetConfigurationResponse\_CALLBACK

```
#define star_v1_SetConfigurationResponse_CALLBACK NULL
```

Definition at line 550 of file [configuration.pb.h](#).

## 6.15.2.173 star\_v1\_SetConfigurationResponse\_DEFAULT

```
#define star_v1_SetConfigurationResponse_DEFAULT NULL
```

Definition at line 551 of file [configuration.pb.h](#).

## 6.15.2.174 star\_v1\_SetConfigurationResponse\_FIELDLIST

```
#define star_v1_SetConfigurationResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, validation_result, 2)
```

Definition at line 547 of file [configuration.pb.h](#).

```
00547 #define star_v1_SetConfigurationResponse_FIELDLIST(X, a) \
00548 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00549 X(a, STATIC, OPTIONAL, MESSAGE, validation_result, 2)
```

#### 6.15.2.175 star\_v1\_SetConfigurationResponse\_fields

```
#define star_v1_SetConfigurationResponse_fields &star_v1_SetConfigurationResponse_msg
```

Definition at line 760 of file [configuration.pb.h](#).

#### 6.15.2.176 star\_v1\_SetConfigurationResponse\_header\_MSGTYPE

```
#define star_v1_SetConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 552 of file [configuration.pb.h](#).

#### 6.15.2.177 star\_v1\_SetConfigurationResponse\_header\_tag

```
#define star_v1_SetConfigurationResponse_header_tag 1
```

Definition at line 516 of file [configuration.pb.h](#).

#### 6.15.2.178 star\_v1\_SetConfigurationResponse\_init\_default

```
#define star_v1_SetConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
star_v1_ConfigValidationResult_init_default}
```

Definition at line 404 of file [configuration.pb.h](#).

#### 6.15.2.179 star\_v1\_SetConfigurationResponse\_init\_zero

```
#define star_v1_SetConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_ConfigValidationResult_in
```

Definition at line 430 of file [configuration.pb.h](#).

#### 6.15.2.180 star\_v1\_SetConfigurationResponse\_size

```
#define star_v1_SetConfigurationResponse_size 5072
```

Definition at line 804 of file [configuration.pb.h](#).

#### 6.15.2.181 star\_v1\_SetConfigurationResponse\_validation\_result\_MSGTYPE

```
#define star_v1_SetConfigurationResponse_validation_result_MSGTYPE star_v1_ConfigValidationResult
```

Definition at line 553 of file [configuration.pb.h](#).

#### 6.15.2.182 star\_v1\_SetConfigurationResponse\_validation\_result\_tag

```
#define star_v1_SetConfigurationResponse_validation_result_tag 2
```

Definition at line 517 of file [configuration.pb.h](#).

## 6.15.2.183 star\_v1\_SetMotorPidConfigRequest\_CALLBACK

```
#define star_v1_SetMotorPidConfigRequest_CALLBACK NULL
```

Definition at line 621 of file [configuration.pb.h](#).

## 6.15.2.184 star\_v1\_SetMotorPidConfigRequest\_DEFAULT

```
#define star_v1_SetMotorPidConfigRequest_DEFAULT NULL
```

Definition at line 622 of file [configuration.pb.h](#).

## 6.15.2.185 star\_v1\_SetMotorPidConfigRequest\_FIELDLIST

```
#define star_v1_SetMotorPidConfigRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, INT32, motor_id, 2) \
X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 3) \
X(a, STATIC, SINGULAR, BOOL, persist_to_nvs, 4)
```

Definition at line 616 of file [configuration.pb.h](#).

```
00616 #define star_v1_SetMotorPidConfigRequest_FIELDLIST(X, a) \
00617 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00618 X(a, STATIC, SINGULAR, INT32, motor_id, 2) \
00619 X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 3) \
00620 X(a, STATIC, SINGULAR, BOOL, persist_to_nvs, 4)
```

## 6.15.2.186 star\_v1\_SetMotorPidConfigRequest\_fields

```
#define star_v1_SetMotorPidConfigRequest_fields &star_v1_SetMotorPidConfigRequest_msg
```

Definition at line 769 of file [configuration.pb.h](#).

## 6.15.2.187 star\_v1\_SetMotorPidConfigRequest\_header\_MSGTYPE

```
#define star_v1_SetMotorPidConfigRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 623 of file [configuration.pb.h](#).

## 6.15.2.188 star\_v1\_SetMotorPidConfigRequest\_header\_tag

```
#define star_v1_SetMotorPidConfigRequest_header_tag 1
```

Definition at line 467 of file [configuration.pb.h](#).

## 6.15.2.189 star\_v1\_SetMotorPidConfigRequest\_init\_default

```
#define star_v1_SetMotorPidConfigRequest_init_default {false, star_v1_RequestHeader_init_default, 0, false, star_v1_PidConfig_init_default, 0}
```

Definition at line 413 of file [configuration.pb.h](#).

## 6.15.2.190 star\_v1\_SetMotorPidConfigRequest\_init\_zero

```
#define star_v1_SetMotorPidConfigRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0, false, star_v1_PidConfig_init_zero, 0}
```

Definition at line 439 of file [configuration.pb.h](#).

## 6.15.2.191 star\_v1\_SetMotorPidConfigRequest\_motor\_id\_tag

```
#define star_v1_SetMotorPidConfigRequest_motor_id_tag 2
```

Definition at line 468 of file [configuration.pb.h](#).

## 6.15.2.192 star\_v1\_SetMotorPidConfigRequest\_persist\_to\_nvs\_tag

```
#define star_v1_SetMotorPidConfigRequest_persist_to_nvs_tag 4
```

Definition at line 470 of file [configuration.pb.h](#).

## 6.15.2.193 star\_v1\_SetMotorPidConfigRequest\_pid\_config\_MSGTYPE

```
#define star_v1_SetMotorPidConfigRequest_pid_config_MSGTYPE star_v1_PidConfig
```

Definition at line 624 of file [configuration.pb.h](#).

## 6.15.2.194 star\_v1\_SetMotorPidConfigRequest\_pid\_config\_tag

```
#define star_v1_SetMotorPidConfigRequest_pid_config_tag 3
```

Definition at line 469 of file [configuration.pb.h](#).

## 6.15.2.195 star\_v1\_SetMotorPidConfigRequest\_size

```
#define star_v1_SetMotorPidConfigRequest_size 227
```

Definition at line 805 of file [configuration.pb.h](#).

## 6.15.2.196 star\_v1\_SetMotorPidConfigResponse\_CALLBACK

```
#define star_v1_SetMotorPidConfigResponse_CALLBACK NULL
```

Definition at line 629 of file [configuration.pb.h](#).

## 6.15.2.197 star\_v1\_SetMotorPidConfigResponse\_DEFAULT

```
#define star_v1_SetMotorPidConfigResponse_DEFAULT NULL
```

Definition at line 630 of file [configuration.pb.h](#).

## 6.15.2.198 star\_v1\_SetMotorPidConfigResponse\_FIELDLIST

```
#define star_v1_SetMotorPidConfigResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, validation_result, 2)
```

Definition at line 626 of file [configuration.pb.h](#).

```
00626 #define star_v1_SetMotorPidConfigResponse_FIELDLIST(X, a) \
00627 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00628 X(a, STATIC, OPTIONAL, MESSAGE, validation_result, 2)
```

## 6.15.2.199 star\_v1\_SetMotorPidConfigResponse\_fields

```
#define star_v1_SetMotorPidConfigResponse_fields &star_v1_SetMotorPidConfigResponse_msg
```

Definition at line 770 of file [configuration.pb.h](#).

## 6.15.2.200 star\_v1\_SetMotorPidConfigResponse\_header\_MSGTYPE

```
#define star_v1_SetMotorPidConfigResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 631 of file [configuration.pb.h](#).

## 6.15.2.201 star\_v1\_SetMotorPidConfigResponse\_header\_tag

```
#define star_v1_SetMotorPidConfigResponse_header_tag 1
```

Definition at line 520 of file [configuration.pb.h](#).

## 6.15.2.202 star\_v1\_SetMotorPidConfigResponse\_init\_default

```
#define star_v1_SetMotorPidConfigResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
star_v1_ConfigValidationResult_init_default}
```

Definition at line 414 of file [configuration.pb.h](#).

6.15.2.203 `star_v1_SetMotorPidConfigResponse_init_zero`

```
#define star_v1_SetMotorPidConfigResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_ConfigValidationResult
```

Definition at line 440 of file [configuration.pb.h](#).

6.15.2.204 `star_v1_SetMotorPidConfigResponse_size`

```
#define star_v1_SetMotorPidConfigResponse_size 5072
```

Definition at line 806 of file [configuration.pb.h](#).

6.15.2.205 `star_v1_SetMotorPidConfigResponse_validation_result_MSGTYPE`

```
#define star_v1_SetMotorPidConfigResponse_validation_result_MSGTYPE star_v1_ConfigValidationResult
```

Definition at line 632 of file [configuration.pb.h](#).

6.15.2.206 `star_v1_SetMotorPidConfigResponse_validation_result_tag`

```
#define star_v1_SetMotorPidConfigResponse_validation_result_tag 2
```

Definition at line 521 of file [configuration.pb.h](#).

6.15.2.207 `star_v1_SetRetransmitConfigRequest_CALLBACK`

```
#define star_v1_SetRetransmitConfigRequest_CALLBACK NULL
```

Definition at line 645 of file [configuration.pb.h](#).

6.15.2.208 `star_v1_SetRetransmitConfigRequest_DEFAULT`

```
#define star_v1_SetRetransmitConfigRequest_DEFAULT NULL
```

Definition at line 646 of file [configuration.pb.h](#).

6.15.2.209 `star_v1_SetRetransmitConfigRequest_FIELDLIST`

```
#define star_v1_SetRetransmitConfigRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, retransmit_config, 2)
```

Definition at line 642 of file [configuration.pb.h](#).

```
00642 #define star_v1_SetRetransmitConfigRequest_FIELDLIST(X, a) \
00643 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00644 X(a, STATIC, OPTIONAL, MESSAGE, retransmit_config, 2)
```



## 6.15.2.210 star\_v1\_SetRetransmitConfigRequest\_fields

```
#define star_v1_SetRetransmitConfigRequest_fields &star_v1_SetRetransmitConfigRequest_msg
```

Definition at line 772 of file [configuration.pb.h](#).

Referenced by [rx\\_nanopb\\_decode\\_retransmit\\_config\\_request\(\)](#).

## 6.15.2.211 star\_v1\_SetRetransmitConfigRequest\_header\_MSGTYPE

```
#define star_v1_SetRetransmitConfigRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 647 of file [configuration.pb.h](#).

## 6.15.2.212 star\_v1\_SetRetransmitConfigRequest\_header\_tag

```
#define star_v1_SetRetransmitConfigRequest_header_tag 1
```

Definition at line 475 of file [configuration.pb.h](#).

## 6.15.2.213 star\_v1\_SetRetransmitConfigRequest\_init\_default

```
#define star_v1_SetRetransmitConfigRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_RetransmitConfig_init_default}
```

Definition at line 416 of file [configuration.pb.h](#).

## 6.15.2.214 star\_v1\_SetRetransmitConfigRequest\_init\_zero

```
#define star_v1_SetRetransmitConfigRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_RetransmitConfig_init_zero}
```

Definition at line 442 of file [configuration.pb.h](#).

Referenced by [rx\\_nanopb\\_decode\\_retransmit\\_config\\_request\(\)](#).

## 6.15.2.215 star\_v1\_SetRetransmitConfigRequest\_retransmit\_config\_MSGTYPE

```
#define star_v1_SetRetransmitConfigRequest_retransmit_config_MSGTYPE star_v1_RetransmitConfig
```

Definition at line 648 of file [configuration.pb.h](#).

## 6.15.2.216 star\_v1\_SetRetransmitConfigRequest\_retransmit\_config\_tag

```
#define star_v1_SetRetransmitConfigRequest_retransmit_config_tag 2
```

Definition at line 476 of file [configuration.pb.h](#).

6.15.2.217 `star_v1_SetRetransmitConfigRequest_size`

```
#define star_v1_SetRetransmitConfigRequest_size 171
```

Definition at line 807 of file [configuration.pb.h](#).

6.15.2.218 `star_v1_SetRetransmitConfigResponse_CALLBACK`

```
#define star_v1_SetRetransmitConfigResponse_CALLBACK NULL
```

Definition at line 652 of file [configuration.pb.h](#).

6.15.2.219 `star_v1_SetRetransmitConfigResponse_DEFAULT`

```
#define star_v1_SetRetransmitConfigResponse_DEFAULT NULL
```

Definition at line 653 of file [configuration.pb.h](#).

6.15.2.220 `star_v1_SetRetransmitConfigResponse_FIELDLIST`

```
#define star_v1_SetRetransmitConfigResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

Definition at line 650 of file [configuration.pb.h](#).

```
00650 #define star_v1_SetRetransmitConfigResponse_FIELDLIST(X, a) \
00651 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

6.15.2.221 `star_v1_SetRetransmitConfigResponse_fields`

```
#define star_v1_SetRetransmitConfigResponse_fields &star_v1_SetRetransmitConfigResponse_msg
```

Definition at line 773 of file [configuration.pb.h](#).

6.15.2.222 `star_v1_SetRetransmitConfigResponse_header_MSGTYPE`

```
#define star_v1_SetRetransmitConfigResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 654 of file [configuration.pb.h](#).

6.15.2.223 `star_v1_SetRetransmitConfigResponse_header_tag`

```
#define star_v1_SetRetransmitConfigResponse_header_tag 1
```

Definition at line 477 of file [configuration.pb.h](#).

## 6.15.2.224 star\_v1\_SetRetransmitConfigResponse\_init\_default

```
#define star_v1_SetRetransmitConfigResponse_init_default {false, star_v1_ResponseHeader_init_default}
```

Definition at line 417 of file [configuration.pb.h](#).

## 6.15.2.225 star\_v1\_SetRetransmitConfigResponse\_init\_zero

```
#define star_v1_SetRetransmitConfigResponse_init_zero {false, star_v1_ResponseHeader_init_zero}
```

Definition at line 443 of file [configuration.pb.h](#).

## 6.15.2.226 star\_v1\_SetRetransmitConfigResponse\_size

```
#define star_v1_SetRetransmitConfigResponse_size 363
```

Definition at line 808 of file [configuration.pb.h](#).

## 6.15.2.227 STAR\_V1\_STAR\_V1\_CONFIGURATION\_PB\_H\_MAX\_SIZE

```
#define STAR_V1_STAR_V1_CONFIGURATION_PB_H_MAX_SIZE star_v1_SetConfigurationResponse_size
```

Definition at line 791 of file [configuration.pb.h](#).

## 6.15.2.228 star\_v1\_SystemConfiguration\_CALLBACK

```
#define star_v1_SystemConfiguration_CALLBACK NULL
```

Definition at line 665 of file [configuration.pb.h](#).

## 6.15.2.229 star\_v1\_SystemConfiguration\_config\_crc\_tag

```
#define star_v1_SystemConfiguration_config_crc_tag 8
```

Definition at line 499 of file [configuration.pb.h](#).

## 6.15.2.230 star\_v1\_SystemConfiguration\_config\_version\_tag

```
#define star_v1_SystemConfiguration_config_version_tag 7
```

Definition at line 498 of file [configuration.pb.h](#).

## 6.15.2.231 star\_v1\_SystemConfiguration\_current\_calibration\_MSGTYPE

```
#define star_v1_SystemConfiguration_current_calibration_MSGTYPE star_v1_CurrentSensorCalibration
```

Definition at line 668 of file [configuration.pb.h](#).

6.15.2.232 `star_v1_SystemConfiguration_current_calibration_tag`

```
#define star_v1_SystemConfiguration_current_calibration_tag 2
```

Definition at line 493 of file [configuration.pb.h](#).

6.15.2.233 `star_v1_SystemConfiguration_DEFAULT`

```
#define star_v1_SystemConfiguration_DEFAULT NULL
```

Definition at line 666 of file [configuration.pb.h](#).

6.15.2.234 `star_v1_SystemConfiguration_encoder_config_MSGTYPE`

```
#define star_v1_SystemConfiguration_encoder_config_MSGTYPE star_v1_EncoderConfiguration
```

Definition at line 671 of file [configuration.pb.h](#).

6.15.2.235 `star_v1_SystemConfiguration_encoder_config_tag`

```
#define star_v1_SystemConfiguration_encoder_config_tag 5
```

Definition at line 496 of file [configuration.pb.h](#).

6.15.2.236 `star_v1_SystemConfiguration_FIELDLIST`

```
#define star_v1_SystemConfiguration_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, REPEATED, MESSAGE, motor_configs, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, current_calibration, 2) \
X(a, STATIC, OPTIONAL, MESSAGE, temperature_calibration, 3) \
X(a, STATIC, OPTIONAL, MESSAGE, safety_thresholds, 4) \
X(a, STATIC, OPTIONAL, MESSAGE, encoder_config, 5) \
X(a, STATIC, OPTIONAL, MESSAGE, timing_config, 6) \
X(a, STATIC, SINGULAR, UINT32, config_version, 7) \
X(a, STATIC, SINGULAR, UINT32, config_crc, 8)
```

Definition at line 656 of file [configuration.pb.h](#).

```
00656 #define star_v1_SystemConfiguration_FIELDLIST(X, a) \
00657 X(a, STATIC, REPEATED, MESSAGE, motor_configs, 1) \
00658 X(a, STATIC, OPTIONAL, MESSAGE, current_calibration, 2) \
00659 X(a, STATIC, OPTIONAL, MESSAGE, temperature_calibration, 3) \
00660 X(a, STATIC, OPTIONAL, MESSAGE, safety_thresholds, 4) \
00661 X(a, STATIC, OPTIONAL, MESSAGE, encoder_config, 5) \
00662 X(a, STATIC, OPTIONAL, MESSAGE, timing_config, 6) \
00663 X(a, STATIC, SINGULAR, UINT32, config_version, 7) \
00664 X(a, STATIC, SINGULAR, UINT32, config_crc, 8)
```

6.15.2.237 `star_v1_SystemConfiguration_fields`

```
#define star_v1_SystemConfiguration_fields &star_v1_SystemConfiguration_msg
```

Definition at line 774 of file [configuration.pb.h](#).

## 6.15.2.238 star\_v1\_SystemConfiguration\_init\_default

```
#define star_v1_SystemConfiguration_init_default {0, {star_v1_MotorPidConfiguration_init_default, star_v1_MotorPidConfiguration_init_default, star_v1_MotorPidConfiguration_init_default, star_v1_MotorPidConfiguration_init_default}, false, star_v1_CurrentSensorCalibration_init_default, false, star_v1_TemperatureCalibration_init_default, false, star_v1_SafetyThresholds_init_default, false, star_v1_EncoderConfiguration_init_default, false, star_v1_TimingConfiguration_init_default, 0, 0}
```

Definition at line 418 of file [configuration.pb.h](#).

## 6.15.2.239 star\_v1\_SystemConfiguration\_init\_zero

```
#define star_v1_SystemConfiguration_init_zero {0, {star_v1_MotorPidConfiguration_init_zero, star_v1_MotorPidConfiguration_init_zero, star_v1_MotorPidConfiguration_init_zero, star_v1_MotorPidConfiguration_init_zero}, false, star_v1_CurrentSensorCalibration_init_zero, false, star_v1_TemperatureCalibration_init_zero, false, star_v1_SafetyThresholds_init_zero, false, star_v1_EncoderConfiguration_init_zero, false, star_v1_TimingConfiguration_init_zero, 0, 0}
```

Definition at line 444 of file [configuration.pb.h](#).

## 6.15.2.240 star\_v1\_SystemConfiguration\_motor\_configs\_MSGTYPE

```
#define star_v1_SystemConfiguration_motor_configs_MSGTYPE star_v1_MotorPidConfiguration
```

Definition at line 667 of file [configuration.pb.h](#).

## 6.15.2.241 star\_v1\_SystemConfiguration\_motor\_configs\_tag

```
#define star_v1_SystemConfiguration_motor_configs_tag 1
```

Definition at line 492 of file [configuration.pb.h](#).

## 6.15.2.242 star\_v1\_SystemConfiguration\_safety\_thresholds\_MSGTYPE

```
#define star_v1_SystemConfiguration_safety_thresholds_MSGTYPE star_v1_SafetyThresholds
```

Definition at line 670 of file [configuration.pb.h](#).

## 6.15.2.243 star\_v1\_SystemConfiguration\_safety\_thresholds\_tag

```
#define star_v1_SystemConfiguration_safety_thresholds_tag 4
```

Definition at line 495 of file [configuration.pb.h](#).

## 6.15.2.244 star\_v1\_SystemConfiguration\_temperature\_calibration\_MSGTYPE

```
#define star_v1_SystemConfiguration_temperature_calibration_MSGTYPE star_v1_TemperatureCalibration
```

Definition at line 669 of file [configuration.pb.h](#).

6.15.2.245 `star_v1_SystemConfiguration_temperature_calibration_tag`

```
#define star_v1_SystemConfiguration_temperature_calibration_tag 3
```

Definition at line 494 of file [configuration.pb.h](#).

6.15.2.246 `star_v1_SystemConfiguration_timing_config_MSGTYPE`

```
#define star_v1_SystemConfiguration_timing_config_MSGTYPE star_v1_TimingConfiguration
```

Definition at line 672 of file [configuration.pb.h](#).

6.15.2.247 `star_v1_SystemConfiguration_timing_config_tag`

```
#define star_v1_SystemConfiguration_timing_config_tag 6
```

Definition at line 497 of file [configuration.pb.h](#).

6.15.2.248 `star_v1_TemperatureCalibration_CALLBACK`

```
#define star_v1_TemperatureCalibration_CALLBACK NULL
```

Definition at line 689 of file [configuration.pb.h](#).

6.15.2.249 `star_v1_TemperatureCalibration_DEFAULT`

```
#define star_v1_TemperatureCalibration_DEFAULT NULL
```

Definition at line 690 of file [configuration.pb.h](#).

6.15.2.250 `star_v1_TemperatureCalibration_FIELDLIST`

```
#define star_v1_TemperatureCalibration_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, DOUBLE, offset_celsius, 1)
```

Definition at line 687 of file [configuration.pb.h](#).

```
00687 #define star_v1_TemperatureCalibration_FIELDLIST(X, a) \
00688 X(a, STATIC, SINGULAR, DOUBLE, offset_celsius, 1)
```

6.15.2.251 `star_v1_TemperatureCalibration_fields`

```
#define star_v1_TemperatureCalibration_fields &star_v1_TemperatureCalibration_msg
```

Definition at line 777 of file [configuration.pb.h](#).

## 6.15.2.252 star\_v1\_TemperatureCalibration\_init\_default

```
#define star_v1_TemperatureCalibration_init_default {0}
```

Definition at line 421 of file [configuration.pb.h](#).

## 6.15.2.253 star\_v1\_TemperatureCalibration\_init\_zero

```
#define star_v1_TemperatureCalibration_init_zero {0}
```

Definition at line 447 of file [configuration.pb.h](#).

## 6.15.2.254 star\_v1\_TemperatureCalibration\_offset\_celsius\_tag

```
#define star_v1_TemperatureCalibration_offset_celsius_tag 1
```

Definition at line 482 of file [configuration.pb.h](#).

## 6.15.2.255 star\_v1\_TemperatureCalibration\_size

```
#define star_v1_TemperatureCalibration_size 9
```

Definition at line 809 of file [configuration.pb.h](#).

## 6.15.2.256 star\_v1\_TimingConfiguration\_CALLBACK

```
#define star_v1_TimingConfiguration_CALLBACK NULL
```

Definition at line 710 of file [configuration.pb.h](#).

## 6.15.2.257 star\_v1\_TimingConfiguration\_communication\_timeout\_ms\_tag

```
#define star_v1_TimingConfiguration_communication_timeout_ms_tag 3
```

Definition at line 491 of file [configuration.pb.h](#).

## 6.15.2.258 star\_v1\_TimingConfiguration\_DEFAULT

```
#define star_v1_TimingConfiguration_DEFAULT NULL
```

Definition at line 711 of file [configuration.pb.h](#).

## 6.15.2.259 star\_v1\_TimingConfiguration\_FIELDLIST

```
#define star_v1_TimingConfiguration_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, UINT32, motor_control_period_ms, 1) \
X(a, STATIC, SINGULAR, UINT32, telemetry_period_ms, 2) \
X(a, STATIC, SINGULAR, UINT32, communication_timeout_ms, 3)
```

Definition at line 706 of file [configuration.pb.h](#).

```
00706 #define star_v1_TimingConfiguration_FIELDLIST(X, a) \
00707 X(a, STATIC, SINGULAR, UINT32, motor_control_period_ms, 1) \
00708 X(a, STATIC, SINGULAR, UINT32, telemetry_period_ms, 2) \
00709 X(a, STATIC, SINGULAR, UINT32, communication_timeout_ms, 3)
```

## 6.15.2.260 star\_v1\_TimingConfiguration\_fields

```
#define star_v1_TimingConfiguration_fields &star_v1_TimingConfiguration_msg
```

Definition at line 780 of file [configuration.pb.h](#).

## 6.15.2.261 star\_v1\_TimingConfiguration\_init\_default

```
#define star_v1_TimingConfiguration_init_default {0, 0, 0}
```

Definition at line 424 of file [configuration.pb.h](#).

## 6.15.2.262 star\_v1\_TimingConfiguration\_init\_zero

```
#define star_v1_TimingConfiguration_init_zero {0, 0, 0}
```

Definition at line 450 of file [configuration.pb.h](#).

## 6.15.2.263 star\_v1\_TimingConfiguration\_motor\_control\_period\_ms\_tag

```
#define star_v1_TimingConfiguration_motor_control_period_ms_tag 1
```

Definition at line 489 of file [configuration.pb.h](#).

## 6.15.2.264 star\_v1\_TimingConfiguration\_size

```
#define star_v1_TimingConfiguration_size 18
```

Definition at line 810 of file [configuration.pb.h](#).

## 6.15.2.265 star\_v1\_TimingConfiguration\_telemetry\_period\_ms\_tag

```
#define star_v1_TimingConfiguration_telemetry_period_ms_tag 2
```

Definition at line 490 of file [configuration.pb.h](#).



## 6.15.2.266 star\_v1\_ValidateConfigurationRequest\_CALLBACK

```
#define star_v1_ValidateConfigurationRequest_CALLBACK NULL
```

Definition at line 573 of file [configuration.pb.h](#).

## 6.15.2.267 star\_v1\_ValidateConfigurationRequest\_configuration\_MSGTYPE

```
#define star_v1_ValidateConfigurationRequest_configuration_MSGTYPE star_v1_SystemConfiguration
```

Definition at line 576 of file [configuration.pb.h](#).

## 6.15.2.268 star\_v1\_ValidateConfigurationRequest\_configuration\_tag

```
#define star_v1_ValidateConfigurationRequest_configuration_tag 2
```

Definition at line 508 of file [configuration.pb.h](#).

## 6.15.2.269 star\_v1\_ValidateConfigurationRequest\_DEFAULT

```
#define star_v1_ValidateConfigurationRequest_DEFAULT NULL
```

Definition at line 574 of file [configuration.pb.h](#).

## 6.15.2.270 star\_v1\_ValidateConfigurationRequest\_FIELDLIST

```
#define star_v1_ValidateConfigurationRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2)
```

Definition at line 570 of file [configuration.pb.h](#).

```
00570 #define star_v1_ValidateConfigurationRequest_FIELDLIST(X, a) \
00571 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00572 X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2)
```

## 6.15.2.271 star\_v1\_ValidateConfigurationRequest\_fields

```
#define star_v1_ValidateConfigurationRequest_fields &star_v1_ValidateConfigurationRequest_msg
```

Definition at line 763 of file [configuration.pb.h](#).

## 6.15.2.272 star\_v1\_ValidateConfigurationRequest\_header\_MSGTYPE

```
#define star_v1_ValidateConfigurationRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 575 of file [configuration.pb.h](#).

6.15.2.273 `star_v1__ValidateConfigurationRequest_header_tag`

```
#define star_v1__ValidateConfigurationRequest_header_tag 1
```

Definition at line 507 of file [configuration.pb.h](#).

6.15.2.274 `star_v1__ValidateConfigurationRequest_init_default`

```
#define star_v1__ValidateConfigurationRequest_init_default {false, star_v1__RequestHeader_init_default, false,
star_v1__SystemConfiguration_init_default}
```

Definition at line 407 of file [configuration.pb.h](#).

6.15.2.275 `star_v1__ValidateConfigurationRequest_init_zero`

```
#define star_v1__ValidateConfigurationRequest_init_zero {false, star_v1__RequestHeader_init_zero, false, star_v1__SystemConfiguration_init_zero}
```

Definition at line 433 of file [configuration.pb.h](#).

6.15.2.276 `star_v1__ValidateConfigurationResponse_CALLBACK`

```
#define star_v1__ValidateConfigurationResponse_CALLBACK NULL
```

Definition at line 581 of file [configuration.pb.h](#).

6.15.2.277 `star_v1__ValidateConfigurationResponse_DEFAULT`

```
#define star_v1__ValidateConfigurationResponse_DEFAULT NULL
```

Definition at line 582 of file [configuration.pb.h](#).

6.15.2.278 `star_v1__ValidateConfigurationResponse_FIELDLIST`

```
#define star_v1__ValidateConfigurationResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, validation_result, 2)
```

Definition at line 578 of file [configuration.pb.h](#).

```
00578 #define star_v1__ValidateConfigurationResponse_FIELDLIST(X, a) \
00579 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00580 X(a, STATIC, OPTIONAL, MESSAGE, validation_result, 2)
```

6.15.2.279 `star_v1__ValidateConfigurationResponse_fields`

```
#define star_v1__ValidateConfigurationResponse_fields &star_v1__ValidateConfigurationResponse_msg
```

Definition at line 764 of file [configuration.pb.h](#).

## 6.15.2.280 star\_v1\_ValidateConfigurationResponse\_header\_MSGTYPE

```
#define star_v1_ValidateConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 583 of file [configuration.pb.h](#).

## 6.15.2.281 star\_v1\_ValidateConfigurationResponse\_header\_tag

```
#define star_v1_ValidateConfigurationResponse_header_tag 1
```

Definition at line 518 of file [configuration.pb.h](#).

## 6.15.2.282 star\_v1\_ValidateConfigurationResponse\_init\_default

```
#define star_v1_ValidateConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default, false, star_v1_ConfigValidationResult_init_default}
```

Definition at line 408 of file [configuration.pb.h](#).

## 6.15.2.283 star\_v1\_ValidateConfigurationResponse\_init\_zero

```
#define star_v1_ValidateConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_ConfigValidationResult_init_zero}
```

Definition at line 434 of file [configuration.pb.h](#).

## 6.15.2.284 star\_v1\_ValidateConfigurationResponse\_size

```
#define star_v1_ValidateConfigurationResponse_size 5072
```

Definition at line 811 of file [configuration.pb.h](#).

## 6.15.2.285 star\_v1\_ValidateConfigurationResponse\_validation\_result\_MSGTYPE

```
#define star_v1_ValidateConfigurationResponse_validation_result_MSGTYPE star_v1_ConfigValidationResult
```

Definition at line 584 of file [configuration.pb.h](#).

## 6.15.2.286 star\_v1\_ValidateConfigurationResponse\_validation\_result\_tag

```
#define star_v1_ValidateConfigurationResponse_validation_result_tag 2
```

Definition at line 519 of file [configuration.pb.h](#).

### 6.15.3 Enumeration Type Documentation

#### 6.15.3.1 `star_v1_ConfigErrorCode`

enum [star\\_v1\\_ConfigErrorCode](#)

Enumerator

|                                                                            |  |
|----------------------------------------------------------------------------|--|
| <code>star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_UNKNOWN</code>             |  |
| <code>star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_VALUE_TOO_LOW</code>       |  |
| <code>star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_VALUE_TOO_HIGH</code>      |  |
| <code>star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_INVALID_NUMBER</code>      |  |
| <code>star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_MISSING_FIELD</code>       |  |
| <code>star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_WRONG_ARRAY_SIZE</code>    |  |
| <code>star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_CRC_FAILED</code>          |  |
| <code>star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_UNSUPPORTED_VERSION</code> |  |
| <code>star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_STORAGE_ERROR</code>       |  |

Definition at line 32 of file [configuration.pb.h](#).

```

00032 {
00033 /* Unknown error. */
00034 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_UNKNOWN = 0,
00035 /* Value is below minimum allowed. */
00036 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_VALUE_TOO_LOW = 1,
00037 /* Value is above maximum allowed. */
00038 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_VALUE_TOO_HIGH = 2,
00039 /* Value is NaN or infinity. */
00040 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_INVALID_NUMBER = 3,
00041 /* Required field is missing. */
00042 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_MISSING_FIELD = 4,
00043 /* Array has wrong number of elements. */
00044 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_WRONG_ARRAY_SIZE = 5,
00045 /* CRC checksum failed. */
00046 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_CRC_FAILED = 6,
00047 /* Version is not supported. */
00048 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_UNSUPPORTED_VERSION = 7,
00049 /* NVS storage error. */
00050 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_STORAGE_ERROR = 8
00051 } star_v1_ConfigErrorCode;
```

#### 6.15.3.2 `star_v1_ConfigValidationStatus`

enum [star\\_v1\\_ConfigValidationStatus](#)

Enumerator

|                                                                                       |  |
|---------------------------------------------------------------------------------------|--|
| <code>star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_UNKNOWN</code>          |  |
| <code>star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_VALID</code>            |  |
| <code>star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_INVALID</code>          |  |
| <code>star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_WARNING</code>          |  |
| <code>star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_CRC_MISMATCH</code>     |  |
| <code>star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_VERSION_MISMATCH</code> |  |

Definition at line 16 of file [configuration.pb.h](#).

```

00016 {
```

```

00017 /* Unknown validation status. */
00018 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_UNKNOWN = 0,
00019 /* Configuration is valid and can be applied. */
00020 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_VALID = 1,
00021 /* Configuration has invalid parameters. */
00022 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_INVALID = 2,
00023 /* Configuration has warnings but can be applied. */
00024 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_WARNING = 3,
00025 /* CRC checksum mismatch. */
00026 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_CRC_MISMATCH = 4,
00027 /* Configuration version incompatible. */
00028 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_VERSION_MISMATCH = 5
00029 } star_v1_ConfigValidationStatus;

```

## 6.15.4 Variable Documentation

### 6.15.4.1 star\_v1\_ConfigValidationError\_msg

const pb\_msgdesc\_t star\_v1\_ConfigValidationError\_msg [extern]

### 6.15.4.2 star\_v1\_ConfigValidationResult\_msg

const pb\_msgdesc\_t star\_v1\_ConfigValidationResult\_msg [extern]

### 6.15.4.3 star\_v1\_CurrentSensorCalibration\_msg

const pb\_msgdesc\_t star\_v1\_CurrentSensorCalibration\_msg [extern]

### 6.15.4.4 star\_v1\_EncoderConfiguration\_msg

const pb\_msgdesc\_t star\_v1\_EncoderConfiguration\_msg [extern]

### 6.15.4.5 star\_v1\_GetConfigurationRequest\_msg

const pb\_msgdesc\_t star\_v1\_GetConfigurationRequest\_msg [extern]

### 6.15.4.6 star\_v1\_GetConfigurationResponse\_msg

const pb\_msgdesc\_t star\_v1\_GetConfigurationResponse\_msg [extern]

### 6.15.4.7 star\_v1\_GetMotorPidConfigRequest\_msg

const pb\_msgdesc\_t star\_v1\_GetMotorPidConfigRequest\_msg [extern]

### 6.15.4.8 star\_v1\_GetMotorPidConfigResponse\_msg

const pb\_msgdesc\_t star\_v1\_GetMotorPidConfigResponse\_msg [extern]

#### 6.15.4.9 star\_v1\_MotorPidConfiguration\_msg

const pb\_msgdesc\_t star\_v1\_MotorPidConfiguration\_msg [extern]

#### 6.15.4.10 star\_v1\_ResetToDefaultsRequest\_msg

const pb\_msgdesc\_t star\_v1\_ResetToDefaultsRequest\_msg [extern]

#### 6.15.4.11 star\_v1\_ResetToDefaultsResponse\_msg

const pb\_msgdesc\_t star\_v1\_ResetToDefaultsResponse\_msg [extern]

#### 6.15.4.12 star\_v1\_RetransmitConfig\_msg

const pb\_msgdesc\_t star\_v1\_RetransmitConfig\_msg [extern]

#### 6.15.4.13 star\_v1\_SafetyThresholds\_msg

const pb\_msgdesc\_t star\_v1\_SafetyThresholds\_msg [extern]

#### 6.15.4.14 star\_v1\_SaveConfigurationRequest\_msg

const pb\_msgdesc\_t star\_v1\_SaveConfigurationRequest\_msg [extern]

#### 6.15.4.15 star\_v1\_SaveConfigurationResponse\_msg

const pb\_msgdesc\_t star\_v1\_SaveConfigurationResponse\_msg [extern]

#### 6.15.4.16 star\_v1\_SetConfigurationRequest\_msg

const pb\_msgdesc\_t star\_v1\_SetConfigurationRequest\_msg [extern]

#### 6.15.4.17 star\_v1\_SetConfigurationResponse\_msg

const pb\_msgdesc\_t star\_v1\_SetConfigurationResponse\_msg [extern]

#### 6.15.4.18 star\_v1\_SetMotorPidConfigRequest\_msg

const pb\_msgdesc\_t star\_v1\_SetMotorPidConfigRequest\_msg [extern]

#### 6.15.4.19 star\_v1\_SetMotorPidConfigResponse\_msg

const pb\_msgdesc\_t star\_v1\_SetMotorPidConfigResponse\_msg [extern]

#### 6.15.4.20 star\_v1\_SetRetransmitConfigRequest\_msg

const pb\_msgdesc\_t star\_v1\_SetRetransmitConfigRequest\_msg [extern]

#### 6.15.4.21 star\_v1\_SetRetransmitConfigResponse\_msg

const pb\_msgdesc\_t star\_v1\_SetRetransmitConfigResponse\_msg [extern]

#### 6.15.4.22 star\_v1\_SystemConfiguration\_msg

const pb\_msgdesc\_t star\_v1\_SystemConfiguration\_msg [extern]

#### 6.15.4.23 star\_v1\_TemperatureCalibration\_msg

const pb\_msgdesc\_t star\_v1\_TemperatureCalibration\_msg [extern]

#### 6.15.4.24 star\_v1\_TimingConfiguration\_msg

const pb\_msgdesc\_t star\_v1\_TimingConfiguration\_msg [extern]

#### 6.15.4.25 star\_v1\_ValidateConfigurationRequest\_msg

const pb\_msgdesc\_t star\_v1\_ValidateConfigurationRequest\_msg [extern]

#### 6.15.4.26 star\_v1\_ValidateConfigurationResponse\_msg

const pb\_msgdesc\_t star\_v1\_ValidateConfigurationResponse\_msg [extern]

## 6.16 configuration.pb.h

[Go to the documentation of this file.](#)

```

00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_STAR_V1_STAR_V1_CONFIGURATION_PB_H_INCLUDED
00005 #define PB_STAR_V1_STAR_V1_CONFIGURATION_PB_H_INCLUDED
00006 #include <pb.h>
00007 #include "star/v1/common.pb.h"
00008 #include "star/v1/motor_control.pb.h"
00009
00010 #if PB_PROTO_HEADER_VERSION != 40
00011 #error Regenerate this file with the current version of nanopb generator.
00012 #endif
00013
00014 /* Enum definitions */
00015 /* Configuration validation status. */
00016 typedef enum _star_v1_ConfigValidationStatus {
00017 /* Unknown validation status. */
00018 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_UNKNOWN = 0,
00019 /* Configuration is valid and can be applied. */
00020 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_VALID = 1,
00021 /* Configuration has invalid parameters. */
00022 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_INVALID = 2,
00023 /* Configuration has warnings but can be applied. */
00024 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_WARNING = 3,
00025 /* CRC checksum mismatch. */
00026 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_CRC_MISMATCH = 4,
00027 /* Configuration version incompatible. */
00028 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_VERSION_MISMATCH = 5
00029 } star_v1_ConfigValidationStatus;
00030
00031 /* Configuration error codes. */
00032 typedef enum _star_v1_ConfigErrorCode {
00033 /* Unknown error. */
00034 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_UNKNOWN = 0,
00035 /* Value is below minimum allowed. */
00036 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_VALUE_TOO_LOW = 1,
00037 /* Value is above maximum allowed. */
00038 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_VALUE_TOO_HIGH = 2,
00039 /* Value is NaN or infinity. */
00040 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_INVALID_NUMBER = 3,
00041 /* Required field is missing. */
00042 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_MISSING_FIELD = 4,
00043 /* Array has wrong number of elements. */
00044 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_WRONG_ARRAY_SIZE = 5,
00045 /* CRC checksum failed. */
00046 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_CRC_FAILED = 6,
00047 /* Version is not supported. */
00048 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_UNSUPPORTED_VERSION = 7,
00049 /* NVS storage error. */
00050 star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_STORAGE_ERROR = 8
00051 } star_v1_ConfigErrorCode;
00052
00053 /* Struct definitions */
00054 /* Request for current configuration. */
00055 typedef struct _star_v1_GetConfigurationRequest {
00056 /* Standard request header. */
00057 bool has_header;
00058 star_v1_RequestHeader header;
00059 } star_v1_GetConfigurationRequest;
00060
00061 /* Request to reset configuration. */
00062 typedef struct _star_v1_ResetToDefaultsRequest {
00063 /* Standard request header. */
00064 bool has_header;
00065 star_v1_RequestHeader header;
00066 /* If true, automatically save defaults to NVS. */
00067 bool persist_to_nvs;
00068 } star_v1_ResetToDefaultsRequest;
00069
00070 /* Request to save configuration to NVS. */
00071 typedef struct _star_v1_SaveConfigurationRequest {
00072 /* Standard request header. */
00073 bool has_header;
00074 star_v1_RequestHeader header;
00075 } star_v1_SaveConfigurationRequest;
00076
00077 /* Response to save configuration. */
00078 typedef struct _star_v1_SaveConfigurationResponse {
00079 /* Standard response header with status. */
00080 bool has_header;
00081 star_v1_ResponseHeader header;
00082 /* True if save was successful. */

```



```

00083 bool saved;
00084 /* Updated CRC-32 checksum after save. */
00085 uint32_t config_crc;
00086 } star_v1_SaveConfigurationResponse;
00087
00088 /* Request for motor PID configuration. */
00089 typedef struct _star_v1_GetMotorPidConfigRequest {
00090 /* Standard request header. */
00091 bool has_header;
00092 star_v1_RequestHeader header;
00093 /* Motor index (0-3 for 4-motor system). */
00094 int32_t motor_id;
00095 } star_v1_GetMotorPidConfigRequest;
00096
00097 /* Response with motor PID configuration. */
00098 typedef struct _star_v1_GetMotorPidConfigResponse {
00099 /* Standard response header with status. */
00100 bool has_header;
00101 star_v1_ResponseHeader header;
00102 /* Motor index. */
00103 int32_t motor_id;
00104 /* PID configuration for this motor. */
00105 bool has_pid_config;
00106 star_v1_PidConfig pid_config;
00107 } star_v1_GetMotorPidConfigResponse;
00108
00109 /* Request to set motor PID configuration. */
00110 typedef struct _star_v1_SetMotorPidConfigRequest {
00111 /* Standard request header. */
00112 bool has_header;
00113 star_v1_RequestHeader header;
00114 /* Motor index (0-3 for 4-motor system). */
00115 int32_t motor_id;
00116 /* PID configuration to apply. */
00117 bool has_pid_config;
00118 star_v1_PidConfig pid_config;
00119 /* If true, automatically save to NVS. */
00120 bool persist_to_nvs;
00121 } star_v1_SetMotorPidConfigRequest;
00122
00123 /* Retransmit configuration for SPI communication layer. */
00124 typedef struct _star_v1_RetransmitConfig {
00125 /* Enable automatic retransmission. */
00126 bool enabled;
00127 /* Maximum retransmit attempts before giving up.
00128 Valid range: 1-10. Default: 3. */
00129 uint32_t max_retries;
00130 /* Initial ACK timeout in milliseconds.
00131 Valid range: 10-1000. Default: 50. */
00132 uint32_t ack_timeout_ms;
00133 /* Maximum backoff cap in milliseconds.
00134 Valid range: 50-5000. Default: 400. */
00135 uint32_t max_backoff_ms;
00136 } star_v1_RetransmitConfig;
00137
00138 /* Request to set retransmit configuration (RPI5 -> RX72N). */
00139 typedef struct _star_v1_SetRetransmitConfigRequest {
00140 /* Standard request header. */
00141 bool has_header;
00142 star_v1_RequestHeader header;
00143 /* Retransmit configuration to apply. */
00144 bool has_retransmit_config;
00145 star_v1_RetransmitConfig retransmit_config;
00146 } star_v1_SetRetransmitConfigRequest;
00147
00148 /* Response to set retransmit configuration. */
00149 typedef struct _star_v1_SetRetransmitConfigResponse {
00150 /* Standard response header with status. */
00151 bool has_header;
00152 star_v1_ResponseHeader header;
00153 } star_v1_SetRetransmitConfigResponse;
00154
00155 /* Motor PID configuration for a single motor.
00156 Note: motor_id is included for self-documentation and validation,
00157 even though array index in SystemConfiguration.motor_configs also
00158 identifies the motor. This enables server-side validation and
00159 clearer debugging of serialized messages. */
00160 typedef struct _star_v1_MotorPidConfiguration {
00161 /* Motor index (0-3).
00162 Should match array index in SystemConfiguration.motor_configs. */
00163 int32_t motor_id;
00164 /* PID gains configuration.
00165 Reuses PidConfig from motor_control.proto. */
00166 bool has_pid_config;
00167 star_v1_PidConfig pid_config;
00168 } star_v1_MotorPidConfiguration;
00169

```

```

00170 /* Current sensor calibration data.
00171 Applied to all motor current sense readings. */
00172 typedef struct _star_v1_CurrentSensorCalibration {
00173 /* ADC offset when motor stopped, in milliamps.
00174 One per motor (max 4). */
00175 pb_callback_t offset_ma;
00176 /* Scaling factor adjustment for each motor.
00177 Multiplied with raw ADC reading.
00178 One per motor (max 4). */
00179 pb_callback_t scale_factor;
00180 } star_v1_CurrentSensorCalibration;
00181
00182 /* Temperature sensor calibration. */
00183 typedef struct _star_v1_TemperatureCalibration {
00184 /* Temperature offset correction in Celsius.
00185 Added to raw temperature reading. */
00186 double offset_celsius;
00187 } star_v1_TemperatureCalibration;
00188
00189 /* Safety thresholds for motor protection. */
00190 typedef struct _star_v1_SafetyThresholds {
00191 /* Motor overcurrent limit in milliamps.
00192 Motors disabled if current limit exceeds this.
00193 Valid range: 1000-20000 mA. */
00194 uint32_t overcurrent_threshold_ma;
00195 /* Emergency temperature cutoff in deci-celsius (0.1C units).
00196 System enters safe mode above this temperature.
00197 Valid range: 500-1000 (50.0-100.0 C). */
00198 int32_t thermal_shutdown_deci_celsius;
00199 /* Motor phase imbalance threshold in millivolts.
00200 Valid range: 50-500 mV. */
00201 uint32_t cell_imbalance_threshold_mv;
00202 } star_v1_SafetyThresholds;
00203
00204 /* Encoder configuration parameters. */
00205 typedef struct _star_v1_EncoderConfiguration {
00206 /* Encoder edges per motor revolution.
00207 Used for velocity calculation.
00208 Common values: 500, 1000, 2000, 4000. */
00209 uint32_t edges_per_revolution;
00210 /* Wheel diameter in meters (for linear velocity). */
00211 double wheel_diameter_m;
00212 /* Gear ratio (motor revolutions per wheel revolution). */
00213 double gear_ratio;
00214 } star_v1_EncoderConfiguration;
00215
00216 /* Timing configuration for control loops and communication. */
00217 typedef struct _star_v1_TimingConfiguration {
00218 /* Motor control loop period in milliseconds.
00219 Lower = faster response, higher CPU usage.
00220 Valid range: 1-100 ms. Typical: 10 ms (100 Hz). */
00221 uint32_t motor_control_period_ms;
00222 /* Telemetry reporting period in milliseconds.
00223 Valid range: 10-1000 ms. Typical: 100 ms (10 Hz). */
00224 uint32_t telemetry_period_ms;
00225 /* RPi5 communication watchdog timeout in milliseconds.
00226 Motors stopped if no command received within timeout.
00227 Valid range: 100-5000 ms. Typical: 500 ms. */
00228 uint32_t communication_timeout_ms;
00229 } star_v1_TimingConfiguration;
00230
00231 /* Complete system configuration.
00232 Maps to star_config_t structure. */
00233 typedef struct _star_v1_SystemConfiguration {
00234 /* Motor PID configuration for all motors (max 4 motors).
00235 Array index corresponds to motor ID (0-3). */
00236 pb_size_t motor_configs_count;
00237 star_v1_MotorPidConfiguration motor_configs[4];
00238 /* Current sensor calibration for all motors. */
00239 bool has_current_calibration;
00240 star_v1_CurrentSensorCalibration current_calibration;
00241 /* Temperature sensor calibration. */
00242 bool has_temperature_calibration;
00243 star_v1_TemperatureCalibration temperature_calibration;
00244 /* Safety thresholds. */
00245 bool has_safety_thresholds;
00246 star_v1_SafetyThresholds safety_thresholds;
00247 /* Encoder configuration. */
00248 bool has_encoder_config;
00249 star_v1_EncoderConfiguration encoder_config;
00250 /* Timing parameters. */
00251 bool has_timing_config;
00252 star_v1_TimingConfiguration timing_config;
00253 /* Configuration schema version.
00254 Used for migration between firmware versions. */
00255 uint32_t config_version;
00256 /* CRC-32 checksum for integrity validation.

```

```

00257 Computed over all fields except this one. */
00258 uint32_t config_crc;
00259 } star_v1_SystemConfiguration;
00260
00261 /* Response with current configuration. */
00262 typedef struct _star_v1_GetConfigurationResponse {
00263 /* Standard response header with status. */
00264 bool has_header;
00265 star_v1_ResponseHeader header;
00266 /* Current system configuration. */
00267 bool has_configuration;
00268 star_v1_SystemConfiguration configuration;
00269 } star_v1_GetConfigurationResponse;
00270
00271 /* Request to set configuration. */
00272 typedef struct _star_v1_SetConfigurationRequest {
00273 /* Standard request header. */
00274 bool has_header;
00275 star_v1_RequestHeader header;
00276 /* Configuration to apply. */
00277 bool has_configuration;
00278 star_v1_SystemConfiguration configuration;
00279 /* If true, automatically save to NVS after validation.
00280 If false, configuration is only held in memory. */
00281 bool persist_to_nvs;
00282 } star_v1_SetConfigurationRequest;
00283
00284 /* Response to reset configuration. */
00285 typedef struct _star_v1_ResetToDefaultsResponse {
00286 /* Standard response header with status. */
00287 bool has_header;
00288 star_v1_ResponseHeader header;
00289 /* Default configuration that was applied. */
00290 bool has_configuration;
00291 star_v1_SystemConfiguration configuration;
00292 } star_v1_ResetToDefaultsResponse;
00293
00294 /* Request to validate configuration. */
00295 typedef struct _star_v1_ValidateConfigurationRequest {
00296 /* Standard request header. */
00297 bool has_header;
00298 star_v1_RequestHeader header;
00299 /* Configuration to validate. */
00300 bool has_configuration;
00301 star_v1_SystemConfiguration configuration;
00302 } star_v1_ValidateConfigurationRequest;
00303
00304 /* Individual configuration validation error. */
00305 typedef struct _star_v1_ConfigValidationError {
00306 /* Field that failed validation (dot notation, e.g., "motor_configs[0].pid_config.kp"). */
00307 char field_path[128];
00308 /* Error code for programmatic handling. */
00309 star_v1_ConfigErrorCode error_code;
00310 /* Human-readable error message. */
00311 char message[256];
00312 /* Actual value that failed validation. */
00313 char actual_value[64];
00314 /* Expected range or constraint description. */
00315 char expected_constraint[128];
00316 } star_v1_ConfigValidationError;
00317
00318 /* Configuration validation result. */
00319 typedef struct _star_v1_ConfigValidationResult {
00320 /* Overall validation status. */
00321 star_v1_ConfigValidationStatus status;
00322 /* List of validation errors (empty if valid). */
00323 pb_size_t errors_count;
00324 star_v1_ConfigValidationError errors[8];
00325 } star_v1_ConfigValidationResult;
00326
00327 /* Response to set configuration. */
00328 typedef struct _star_v1_SetConfigurationResponse {
00329 /* Standard response header with status. */
00330 bool has_header;
00331 star_v1_ResponseHeader header;
00332 /* Validation result. */
00333 bool has_validation_result;
00334 star_v1_ConfigValidationResult validation_result;
00335 } star_v1_SetConfigurationResponse;
00336
00337 /* Response with validation results. */
00338 typedef struct _star_v1_ValidateConfigurationResponse {
00339 /* Standard response header with status. */
00340 bool has_header;
00341 star_v1_ResponseHeader header;
00342 /* Detailed validation result. */
00343 bool has_validation_result;

```

```

00344 star_v1_ConfigValidationResult validation_result;
00345 } star_v1_ValidateConfigurationResponse;
00346
00347 /* Response to set motor PID configuration. */
00348 typedef struct _star_v1_SetMotorPidConfigResponse {
00349 /* Standard response header with status. */
00350 bool has_header;
00351 star_v1_ResponseHeader header;
00352 /* Validation result. */
00353 bool has_validation_result;
00354 star_v1_ConfigValidationResult validation_result;
00355 } star_v1_SetMotorPidConfigResponse;
00356
00357
00358 #ifdef __cplusplus
00359 extern "C" {
00360 #endif
00361
00362 /* Helper constants for enums */
00363 #define _star_v1_ConfigValidationStatus_MIN
00364 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_UNKNOWN
00365 #define _star_v1_ConfigValidationStatus_MAX
00366 star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_VERSION_MISMATCH
00367 #define _star_v1_ConfigValidationStatus_ARRAYSIZE
00368 ((star_v1_ConfigValidationStatus)(star_v1_ConfigValidationStatus_CONFIG_VALIDATION_STATUS_VERSION_MISMATCH+1))
00369
00370 #define _star_v1_ConfigErrorCode_MIN star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_UNKNOWN
00371 #define _star_v1_ConfigErrorCode_MAX star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_STORAGE_ERROR
00372 #define _star_v1_ConfigErrorCode_ARRAYSIZE
00373 ((star_v1_ConfigErrorCode)(star_v1_ConfigErrorCode_CONFIG_ERROR_CODE_STORAGE_ERROR+1))
00374
00375
00376
00377
00378
00379
00380
00381
00382
00383
00384
00385
00386
00387
00388
00389
00390
00391
00392
00393
00394
00395 #define star_v1_ConfigValidationResult_status_ENUMTYPE star_v1_ConfigValidationStatus
00396
00397 #define star_v1_ConfigValidationResult_error_code_ENUMTYPE star_v1_ConfigErrorCode
00398
00399
00400 /* Initializer values for message structs */
00401 #define star_v1_GetConfigurationRequest_init_default {false, star_v1_RequestHeader_init_default}
00402 #define star_v1_GetConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
00403 star_v1_SystemConfiguration_init_default}
00404 #define star_v1_SetConfigurationRequest_init_default {false, star_v1_RequestHeader_init_default, false,
00405 star_v1_SystemConfiguration_init_default, 0}
00406 #define star_v1_SetConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
00407 star_v1_ConfigValidationResult_init_default}
00408 #define star_v1_ResetToDefaultsRequest_init_default {false, star_v1_RequestHeader_init_default, 0}
00409 #define star_v1_ResetToDefaultsResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
00410 star_v1_SystemConfiguration_init_default}
00411 #define star_v1_ValidateConfigurationRequest_init_default {false, star_v1_RequestHeader_init_default, false,
00412 star_v1_SystemConfiguration_init_default}
00413 #define star_v1_ValidateConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
00414 star_v1_ConfigValidationResult_init_default}
00415 #define star_v1_SaveConfigurationRequest_init_default {false, star_v1_RequestHeader_init_default}
00416 #define star_v1_SaveConfigurationResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0}
00417 #define star_v1_GetMotorPidConfigRequest_init_default {false, star_v1_RequestHeader_init_default, 0}
00418 #define star_v1_GetMotorPidConfigResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, false,
00419 star_v1_PidConfig_init_default}
00420 #define star_v1_SetMotorPidConfigRequest_init_default {false, star_v1_RequestHeader_init_default, 0, false,
00421 star_v1_PidConfig_init_default, 0}
00422 #define star_v1_SetMotorPidConfigResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
00423 star_v1_ConfigValidationResult_init_default}
00424 #define star_v1_RetransmitConfig_init_default {0, 0, 0, 0}
00425 #define star_v1_SetRetransmitConfigRequest_init_default {false, star_v1_RequestHeader_init_default, false,
00426 star_v1_RetransmitConfig_init_default}

```

```

00417 #define star_v1_SetRetransmitConfigResponse_init_default {false, star_v1_ResponseHeader_init_default}
00418 #define star_v1_SystemConfiguration_init_default {0, {star_v1_MotorPidConfiguration_init_default,
star_v1_MotorPidConfiguration_init_default, star_v1_MotorPidConfiguration_init_default,
star_v1_MotorPidConfiguration_init_default}, false, star_v1_CurrentSensorCalibration_init_default, false,
star_v1_TemperatureCalibration_init_default, false, star_v1_SafetyThresholds_init_default, false,
star_v1_EncoderConfiguration_init_default, false, star_v1_TimingConfiguration_init_default, 0, 0}
00419 #define star_v1_MotorPidConfiguration_init_default {0, false, star_v1_PidConfig_init_default}
00420 #define star_v1_CurrentSensorCalibration_init_default {{{NULL}, NULL}, {{{NULL}, NULL}}}
00421 #define star_v1_TemperatureCalibration_init_default {0}
00422 #define star_v1_SafetyThresholds_init_default {0, 0, 0}
00423 #define star_v1_EncoderConfiguration_init_default {0, 0, 0}
00424 #define star_v1_TimingConfiguration_init_default {0, 0, 0}
00425 #define star_v1_ConfigValidationResult_init_default {_star_v1_ConfigValidationStatus_MIN, 0,
{star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default,
star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default,
star_v1_ConfigValidationError_init_default, star_v1_ConfigValidationError_init_default}}
00426 #define star_v1_ConfigValidationError_init_default {"", _star_v1_ConfigErrorCode_MIN, "", "", ""}
00427 #define star_v1_GetConfigurationRequest_init_zero {false, star_v1_RequestHeader_init_zero}
00428 #define star_v1_GetConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false,
star_v1_SystemConfiguration_init_zero}
00429 #define star_v1_SetConfigurationRequest_init_zero {false, star_v1_RequestHeader_init_zero, false,
star_v1_SystemConfiguration_init_zero, 0}
00430 #define star_v1_SetConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false,
star_v1_ConfigValidationResult_init_zero}
00431 #define star_v1_ResetToDefaultsRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}
00432 #define star_v1_ResetToDefaultsResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false,
star_v1_SystemConfiguration_init_zero}
00433 #define star_v1_ValidateConfigurationRequest_init_zero {false, star_v1_RequestHeader_init_zero, false,
star_v1_SystemConfiguration_init_zero}
00434 #define star_v1_ValidateConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false,
star_v1_ConfigValidationResult_init_zero}
00435 #define star_v1_SaveConfigurationRequest_init_zero {false, star_v1_RequestHeader_init_zero}
00436 #define star_v1_SaveConfigurationResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0}
00437 #define star_v1_GetMotorPidConfigRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}
00438 #define star_v1_GetMotorPidConfigResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, false,
star_v1_PidConfig_init_zero}
00439 #define star_v1_SetMotorPidConfigRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0, false,
star_v1_PidConfig_init_zero, 0}
00440 #define star_v1_SetMotorPidConfigResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false,
star_v1_ConfigValidationResult_init_zero}
00441 #define star_v1_RetransmitConfig_init_zero {0, 0, 0, 0}
00442 #define star_v1_SetRetransmitConfigRequest_init_zero {false, star_v1_RequestHeader_init_zero, false,
star_v1_RetransmitConfig_init_zero}
00443 #define star_v1_SetRetransmitConfigResponse_init_zero {false, star_v1_ResponseHeader_init_zero}
00444 #define star_v1_SystemConfiguration_init_zero {0, {star_v1_MotorPidConfiguration_init_zero,
star_v1_MotorPidConfiguration_init_zero, star_v1_MotorPidConfiguration_init_zero,
star_v1_MotorPidConfiguration_init_zero}, false, star_v1_CurrentSensorCalibration_init_zero, false,
star_v1_TemperatureCalibration_init_zero, false, star_v1_SafetyThresholds_init_zero, false,
star_v1_EncoderConfiguration_init_zero, false, star_v1_TimingConfiguration_init_zero, 0, 0}
00445 #define star_v1_MotorPidConfiguration_init_zero {0, false, star_v1_PidConfig_init_zero}
00446 #define star_v1_CurrentSensorCalibration_init_zero {{{NULL}, NULL}, {{{NULL}, NULL}}}
00447 #define star_v1_TemperatureCalibration_init_zero {0}
00448 #define star_v1_SafetyThresholds_init_zero {0, 0, 0}
00449 #define star_v1_EncoderConfiguration_init_zero {0, 0, 0}
00450 #define star_v1_TimingConfiguration_init_zero {0, 0, 0}
00451 #define star_v1_ConfigValidationResult_init_zero {_star_v1_ConfigValidationStatus_MIN, 0,
{star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero,
star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero,
star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero},
star_v1_ConfigValidationError_init_zero, star_v1_ConfigValidationError_init_zero}
00452 #define star_v1_ConfigValidationError_init_zero {"", _star_v1_ConfigErrorCode_MIN, "", "", ""}
00453
00454 /* Field tags (for use in manual encoding/decoding) */
00455 #define star_v1_GetConfigurationRequest_header_tag 1
00456 #define star_v1_ResetToDefaultsRequest_header_tag 1
00457 #define star_v1_ResetToDefaultsRequest_persist_to_nvs_tag 2
00458 #define star_v1_SaveConfigurationRequest_header_tag 1
00459 #define star_v1_SaveConfigurationResponse_header_tag 1
00460 #define star_v1_SaveConfigurationResponse_saved_tag 2
00461 #define star_v1_SaveConfigurationResponse_config_crc_tag 3
00462 #define star_v1_GetMotorPidConfigRequest_header_tag 1
00463 #define star_v1_GetMotorPidConfigRequest_motor_id_tag 2
00464 #define star_v1_GetMotorPidConfigResponse_header_tag 1
00465 #define star_v1_GetMotorPidConfigResponse_motor_id_tag 2
00466 #define star_v1_GetMotorPidConfigResponse_pid_config_tag 3
00467 #define star_v1_SetMotorPidConfigRequest_header_tag 1
00468 #define star_v1_SetMotorPidConfigRequest_motor_id_tag 2
00469 #define star_v1_SetMotorPidConfigRequest_pid_config_tag 3
00470 #define star_v1_SetMotorPidConfigRequest_persist_to_nvs_tag 4
00471 #define star_v1_RetransmitConfig_enabled_tag 1
00472 #define star_v1_RetransmitConfig_max_retries_tag 2
00473 #define star_v1_RetransmitConfig_ack_timeout_ms_tag 3
00474 #define star_v1_RetransmitConfig_max_backoff_ms_tag 4
00475 #define star_v1_SetRetransmitConfigRequest_header_tag 1
00476 #define star_v1_SetRetransmitConfigRequest_retransmit_config_tag 2
00477 #define star_v1_SetRetransmitConfigResponse_header_tag 1

```



```

00478 #define star_v1_MotorPidConfiguration_motor_id_tag 1
00479 #define star_v1_MotorPidConfiguration_pid_config_tag 2
00480 #define star_v1_CurrentSensorCalibration_offset_ma_tag 1
00481 #define star_v1_CurrentSensorCalibration_scale_factor_tag 2
00482 #define star_v1_TemperatureCalibration_offset_celsius_tag 1
00483 #define star_v1_SafetyThresholds_overcurrent_threshold_ma_tag 1
00484 #define star_v1_SafetyThresholds_thermal_shutdown_deci_celsius_tag 2
00485 #define star_v1_SafetyThresholds_cell_imbalance_threshold_mv_tag 3
00486 #define star_v1_EncoderConfiguration_edges_per_revolution_tag 1
00487 #define star_v1_EncoderConfiguration_wheel_diameter_m_tag 2
00488 #define star_v1_EncoderConfiguration_gear_ratio_tag 3
00489 #define star_v1_TimingConfiguration_motor_control_period_ms_tag 1
00490 #define star_v1_TimingConfiguration_telemetry_period_ms_tag 2
00491 #define star_v1_TimingConfiguration_communication_timeout_ms_tag 3
00492 #define star_v1_SystemConfiguration_motor_configs_tag 1
00493 #define star_v1_SystemConfiguration_current_calibration_tag 2
00494 #define star_v1_SystemConfiguration_temperature_calibration_tag 3
00495 #define star_v1_SystemConfiguration_safety_thresholds_tag 4
00496 #define star_v1_SystemConfiguration_encoder_config_tag 5
00497 #define star_v1_SystemConfiguration_timing_config_tag 6
00498 #define star_v1_SystemConfiguration_config_version_tag 7
00499 #define star_v1_SystemConfiguration_config_crc_tag 8
00500 #define star_v1_GetConfigurationResponse_header_tag 1
00501 #define star_v1_GetConfigurationResponse_configuration_tag 2
00502 #define star_v1_SetConfigurationRequest_header_tag 1
00503 #define star_v1_SetConfigurationRequest_configuration_tag 2
00504 #define star_v1_SetConfigurationRequest_persist_to_nvs_tag 3
00505 #define star_v1_ResetToDefaultsResponse_header_tag 1
00506 #define star_v1_ResetToDefaultsResponse_configuration_tag 2
00507 #define star_v1_ValidateConfigurationRequest_header_tag 1
00508 #define star_v1_ValidateConfigurationRequest_configuration_tag 2
00509 #define star_v1_ConfigValidationErrorResponse_field_path_tag 1
00510 #define star_v1_ConfigValidationErrorResponse_error_code_tag 2
00511 #define star_v1_ConfigValidationErrorResponse_message_tag 3
00512 #define star_v1_ConfigValidationErrorResponse_actual_value_tag 4
00513 #define star_v1_ConfigValidationErrorResponse_expected_constraint_tag 5
00514 #define star_v1_ConfigValidationResult_status_tag 1
00515 #define star_v1_ConfigValidationResult_errors_tag 2
00516 #define star_v1_SetConfigurationResponse_header_tag 1
00517 #define star_v1_SetConfigurationResponse_validation_result_tag 2
00518 #define star_v1_ValidateConfigurationResponse_header_tag 1
00519 #define star_v1_ValidateConfigurationResponse_validation_result_tag 2
00520 #define star_v1_SetMotorPidConfigResponse_header_tag 1
00521 #define star_v1_SetMotorPidConfigResponse_validation_result_tag 2
00522
00523 /* Struct field encoding specification for nanopb */
00524 #define star_v1_GetConfigurationRequest_FIELDLIST(X, a) \
00525 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00526 #define star_v1_GetConfigurationRequest_CALLBACK NULL
00527 #define star_v1_GetConfigurationRequest_DEFAULT NULL
00528 #define star_v1_GetConfigurationRequest_header_MSGTYPE star_v1_RequestHeader
00529
00530 #define star_v1_GetConfigurationResponse_FIELDLIST(X, a) \
00531 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00532 X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2) \
00533 #define star_v1_GetConfigurationResponse_CALLBACK NULL
00534 #define star_v1_GetConfigurationResponse_DEFAULT NULL
00535 #define star_v1_GetConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader
00536 #define star_v1_GetConfigurationResponse_configuration_MSGTYPE star_v1_SystemConfiguration
00537
00538 #define star_v1_SetConfigurationRequest_FIELDLIST(X, a) \
00539 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00540 X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2) \
00541 X(a, STATIC, SINGULAR, BOOL, persist_to_nvs, 3) \
00542 #define star_v1_SetConfigurationRequest_CALLBACK NULL
00543 #define star_v1_SetConfigurationRequest_DEFAULT NULL
00544 #define star_v1_SetConfigurationRequest_header_MSGTYPE star_v1_RequestHeader
00545 #define star_v1_SetConfigurationRequest_configuration_MSGTYPE star_v1_SystemConfiguration
00546
00547 #define star_v1_SetConfigurationResponse_FIELDLIST(X, a) \
00548 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00549 X(a, STATIC, OPTIONAL, MESSAGE, validation_result, 2) \
00550 #define star_v1_SetConfigurationResponse_CALLBACK NULL
00551 #define star_v1_SetConfigurationResponse_DEFAULT NULL
00552 #define star_v1_SetConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader
00553 #define star_v1_SetConfigurationResponse_validation_result_MSGTYPE star_v1_ConfigValidationResult
00554
00555 #define star_v1_ResetToDefaultsRequest_FIELDLIST(X, a) \
00556 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00557 X(a, STATIC, SINGULAR, BOOL, persist_to_nvs, 2) \
00558 #define star_v1_ResetToDefaultsRequest_CALLBACK NULL
00559 #define star_v1_ResetToDefaultsRequest_DEFAULT NULL
00560 #define star_v1_ResetToDefaultsRequest_header_MSGTYPE star_v1_RequestHeader
00561
00562 #define star_v1_ResetToDefaultsResponse_FIELDLIST(X, a) \
00563 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00564 X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2)

```

```

00565 #define star_v1_ResetToDefaultsResponse_CALLBACK NULL
00566 #define star_v1_ResetToDefaultsResponse_DEFAULT NULL
00567 #define star_v1_ResetToDefaultsResponse_header_MSGTYPE star_v1_ResponseHeader
00568 #define star_v1_ResetToDefaultsResponse_configuration_MSGTYPE star_v1_SystemConfiguration
00569
00570 #define star_v1_ValidateConfigurationRequest_FIELDLIST(X, a) \
00571 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00572 X(a, STATIC, OPTIONAL, MESSAGE, configuration, 2)
00573 #define star_v1_ValidateConfigurationRequest_CALLBACK NULL
00574 #define star_v1_ValidateConfigurationRequest_DEFAULT NULL
00575 #define star_v1_ValidateConfigurationRequest_header_MSGTYPE star_v1_RequestHeader
00576 #define star_v1_ValidateConfigurationRequest_configuration_MSGTYPE star_v1_SystemConfiguration
00577
00578 #define star_v1_ValidateConfigurationResponse_FIELDLIST(X, a) \
00579 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00580 X(a, STATIC, OPTIONAL, MESSAGE, validation_result, 2)
00581 #define star_v1_ValidateConfigurationResponse_CALLBACK NULL
00582 #define star_v1_ValidateConfigurationResponse_DEFAULT NULL
00583 #define star_v1_ValidateConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader
00584 #define star_v1_ValidateConfigurationResponse_validation_result_MSGTYPE star_v1_ConfigValidationResult
00585
00586 #define star_v1_SaveConfigurationRequest_FIELDLIST(X, a) \
00587 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
00588 #define star_v1_SaveConfigurationRequest_CALLBACK NULL
00589 #define star_v1_SaveConfigurationRequest_DEFAULT NULL
00590 #define star_v1_SaveConfigurationRequest_header_MSGTYPE star_v1_RequestHeader
00591
00592 #define star_v1_SaveConfigurationResponse_FIELDLIST(X, a) \
00593 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00594 X(a, STATIC, SINGULAR, BOOL, saved, 2) \
00595 X(a, STATIC, SINGULAR, UINT32, config_crc, 3)
00596 #define star_v1_SaveConfigurationResponse_CALLBACK NULL
00597 #define star_v1_SaveConfigurationResponse_DEFAULT NULL
00598 #define star_v1_SaveConfigurationResponse_header_MSGTYPE star_v1_ResponseHeader
00599
00600 #define star_v1_GetMotorPidConfigRequest_FIELDLIST(X, a) \
00601 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00602 X(a, STATIC, SINGULAR, INT32, motor_id, 2)
00603 #define star_v1_GetMotorPidConfigRequest_CALLBACK NULL
00604 #define star_v1_GetMotorPidConfigRequest_DEFAULT NULL
00605 #define star_v1_GetMotorPidConfigRequest_header_MSGTYPE star_v1_RequestHeader
00606
00607 #define star_v1_GetMotorPidConfigResponse_FIELDLIST(X, a) \
00608 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00609 X(a, STATIC, SINGULAR, INT32, motor_id, 2) \
00610 X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 3)
00611 #define star_v1_GetMotorPidConfigResponse_CALLBACK NULL
00612 #define star_v1_GetMotorPidConfigResponse_DEFAULT NULL
00613 #define star_v1_GetMotorPidConfigResponse_header_MSGTYPE star_v1_ResponseHeader
00614 #define star_v1_GetMotorPidConfigResponse_pid_config_MSGTYPE star_v1_PidConfig
00615
00616 #define star_v1_SetMotorPidConfigRequest_FIELDLIST(X, a) \
00617 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00618 X(a, STATIC, SINGULAR, INT32, motor_id, 2) \
00619 X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 3) \
00620 X(a, STATIC, SINGULAR, BOOL, persist_to_nvs, 4)
00621 #define star_v1_SetMotorPidConfigRequest_CALLBACK NULL
00622 #define star_v1_SetMotorPidConfigRequest_DEFAULT NULL
00623 #define star_v1_SetMotorPidConfigRequest_header_MSGTYPE star_v1_RequestHeader
00624 #define star_v1_SetMotorPidConfigRequest_pid_config_MSGTYPE star_v1_PidConfig
00625
00626 #define star_v1_SetMotorPidConfigResponse_FIELDLIST(X, a) \
00627 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00628 X(a, STATIC, OPTIONAL, MESSAGE, validation_result, 2)
00629 #define star_v1_SetMotorPidConfigResponse_CALLBACK NULL
00630 #define star_v1_SetMotorPidConfigResponse_DEFAULT NULL
00631 #define star_v1_SetMotorPidConfigResponse_header_MSGTYPE star_v1_ResponseHeader
00632 #define star_v1_SetMotorPidConfigResponse_validation_result_MSGTYPE star_v1_ConfigValidationResult
00633
00634 #define star_v1_RetransmitConfig_FIELDLIST(X, a) \
00635 X(a, STATIC, SINGULAR, BOOL, enabled, 1) \
00636 X(a, STATIC, SINGULAR, UINT32, max_retries, 2) \
00637 X(a, STATIC, SINGULAR, UINT32, ack_timeout_ms, 3) \
00638 X(a, STATIC, SINGULAR, UINT32, max_backoff_ms, 4)
00639 #define star_v1_RetransmitConfig_CALLBACK NULL
00640 #define star_v1_RetransmitConfig_DEFAULT NULL
00641
00642 #define star_v1_SetRetransmitConfigRequest_FIELDLIST(X, a) \
00643 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00644 X(a, STATIC, OPTIONAL, MESSAGE, retransmit_config, 2)
00645 #define star_v1_SetRetransmitConfigRequest_CALLBACK NULL
00646 #define star_v1_SetRetransmitConfigRequest_DEFAULT NULL
00647 #define star_v1_SetRetransmitConfigRequest_header_MSGTYPE star_v1_RequestHeader
00648 #define star_v1_SetRetransmitConfigRequest_retransmit_config_MSGTYPE star_v1_RetransmitConfig
00649
00650 #define star_v1_SetRetransmitConfigResponse_FIELDLIST(X, a) \
00651 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)

```

```

00652 #define star_v1_SetRetransmitConfigResponse_CALLBACK NULL
00653 #define star_v1_SetRetransmitConfigResponse_DEFAULT NULL
00654 #define star_v1_SetRetransmitConfigResponse_header_MSGTYPE star_v1_ResponseHeader
00655
00656 #define star_v1_SystemConfiguration_FIELDLIST(X, a) \
00657 X(a, STATIC, REPEATED, MESSAGE, motor_configs, 1) \
00658 X(a, STATIC, OPTIONAL, MESSAGE, current_calibration, 2) \
00659 X(a, STATIC, OPTIONAL, MESSAGE, temperature_calibration, 3) \
00660 X(a, STATIC, OPTIONAL, MESSAGE, safety_thresholds, 4) \
00661 X(a, STATIC, OPTIONAL, MESSAGE, encoder_config, 5) \
00662 X(a, STATIC, OPTIONAL, MESSAGE, timing_config, 6) \
00663 X(a, STATIC, SINGULAR, UINT32, config_version, 7) \
00664 X(a, STATIC, SINGULAR, UINT32, config_crc, 8)
00665 #define star_v1_SystemConfiguration_CALLBACK NULL
00666 #define star_v1_SystemConfiguration_DEFAULT NULL
00667 #define star_v1_SystemConfiguration_motor_configs_MSGTYPE star_v1_MotorPidConfiguration
00668 #define star_v1_SystemConfiguration_current_calibration_MSGTYPE star_v1_CurrentSensorCalibration
00669 #define star_v1_SystemConfiguration_temperature_calibration_MSGTYPE star_v1_TemperatureCalibration
00670 #define star_v1_SystemConfiguration_safety_thresholds_MSGTYPE star_v1_SafetyThresholds
00671 #define star_v1_SystemConfiguration_encoder_config_MSGTYPE star_v1_EncoderConfiguration
00672 #define star_v1_SystemConfiguration_timing_config_MSGTYPE star_v1_TimingConfiguration
00673
00674 #define star_v1_MotorPidConfiguration_FIELDLIST(X, a) \
00675 X(a, STATIC, SINGULAR, INT32, motor_id, 1) \
00676 X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 2)
00677 #define star_v1_MotorPidConfiguration_CALLBACK NULL
00678 #define star_v1_MotorPidConfiguration_DEFAULT NULL
00679 #define star_v1_MotorPidConfiguration_pid_config_MSGTYPE star_v1_PidConfig
00680
00681 #define star_v1_CurrentSensorCalibration_FIELDLIST(X, a) \
00682 X(a, CALLBACK, REPEATED, DOUBLE, offset_ma, 1) \
00683 X(a, CALLBACK, REPEATED, DOUBLE, scale_factor, 2)
00684 #define star_v1_CurrentSensorCalibration_CALLBACK pb_default_field_callback
00685 #define star_v1_CurrentSensorCalibration_DEFAULT NULL
00686
00687 #define star_v1_TemperatureCalibration_FIELDLIST(X, a) \
00688 X(a, STATIC, SINGULAR, DOUBLE, offset_celsius, 1)
00689 #define star_v1_TemperatureCalibration_CALLBACK NULL
00690 #define star_v1_TemperatureCalibration_DEFAULT NULL
00691
00692 #define star_v1_SafetyThresholds_FIELDLIST(X, a) \
00693 X(a, STATIC, SINGULAR, UINT32, overcurrent_threshold_ma, 1) \
00694 X(a, STATIC, SINGULAR, INT32, thermal_shutdown_deci_celsius, 2) \
00695 X(a, STATIC, SINGULAR, UINT32, cell_imbalance_threshold_mv, 3)
00696 #define star_v1_SafetyThresholds_CALLBACK NULL
00697 #define star_v1_SafetyThresholds_DEFAULT NULL
00698
00699 #define star_v1_EncoderConfiguration_FIELDLIST(X, a) \
00700 X(a, STATIC, SINGULAR, UINT32, edges_per_revolution, 1) \
00701 X(a, STATIC, SINGULAR, DOUBLE, wheel_diameter_m, 2) \
00702 X(a, STATIC, SINGULAR, DOUBLE, gear_ratio, 3)
00703 #define star_v1_EncoderConfiguration_CALLBACK NULL
00704 #define star_v1_EncoderConfiguration_DEFAULT NULL
00705
00706 #define star_v1_TimingConfiguration_FIELDLIST(X, a) \
00707 X(a, STATIC, SINGULAR, UINT32, motor_control_period_ms, 1) \
00708 X(a, STATIC, SINGULAR, UINT32, telemetry_period_ms, 2) \
00709 X(a, STATIC, SINGULAR, UINT32, communication_timeout_ms, 3)
00710 #define star_v1_TimingConfiguration_CALLBACK NULL
00711 #define star_v1_TimingConfiguration_DEFAULT NULL
00712
00713 #define star_v1_ConfigValidationResult_FIELDLIST(X, a) \
00714 X(a, STATIC, SINGULAR, UENUM, status, 1) \
00715 X(a, STATIC, REPEATED, MESSAGE, errors, 2)
00716 #define star_v1_ConfigValidationResult_CALLBACK NULL
00717 #define star_v1_ConfigValidationResult_DEFAULT NULL
00718 #define star_v1_ConfigValidationResult_errors_MSGTYPE star_v1_ConfigValidationError
00719
00720 #define star_v1_ConfigValidationError_FIELDLIST(X, a) \
00721 X(a, STATIC, SINGULAR, STRING, field_path, 1) \
00722 X(a, STATIC, SINGULAR, UENUM, error_code, 2) \
00723 X(a, STATIC, SINGULAR, STRING, message, 3) \
00724 X(a, STATIC, SINGULAR, STRING, actual_value, 4) \
00725 X(a, STATIC, SINGULAR, STRING, expected_constraint, 5)
00726 #define star_v1_ConfigValidationError_CALLBACK NULL
00727 #define star_v1_ConfigValidationError_DEFAULT NULL
00728
00729 extern const pb_msgdesc_t star_v1_GetConfigurationRequest_msg;
00730 extern const pb_msgdesc_t star_v1_GetConfigurationResponse_msg;
00731 extern const pb_msgdesc_t star_v1_SetConfigurationRequest_msg;
00732 extern const pb_msgdesc_t star_v1_SetConfigurationResponse_msg;
00733 extern const pb_msgdesc_t star_v1_ResetToDefaultsRequest_msg;
00734 extern const pb_msgdesc_t star_v1_ResetToDefaultsResponse_msg;
00735 extern const pb_msgdesc_t star_v1_ValidateConfigurationRequest_msg;
00736 extern const pb_msgdesc_t star_v1_ValidateConfigurationResponse_msg;
00737 extern const pb_msgdesc_t star_v1_SaveConfigurationRequest_msg;
00738 extern const pb_msgdesc_t star_v1_SaveConfigurationResponse_msg;

```



```

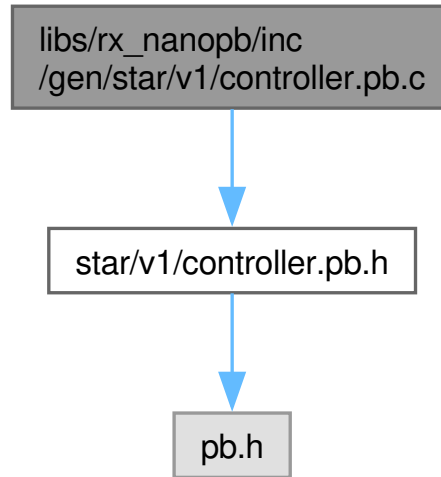
00739 extern const pb_msgdesc_t star_v1_GetMotorPidConfigRequest_msg;
00740 extern const pb_msgdesc_t star_v1_GetMotorPidConfigResponse_msg;
00741 extern const pb_msgdesc_t star_v1_SetMotorPidConfigRequest_msg;
00742 extern const pb_msgdesc_t star_v1_SetMotorPidConfigResponse_msg;
00743 extern const pb_msgdesc_t star_v1_RetransmitConfig_msg;
00744 extern const pb_msgdesc_t star_v1_SetRetransmitConfigRequest_msg;
00745 extern const pb_msgdesc_t star_v1_SetRetransmitConfigResponse_msg;
00746 extern const pb_msgdesc_t star_v1_SystemConfiguration_msg;
00747 extern const pb_msgdesc_t star_v1_MotorPidConfiguration_msg;
00748 extern const pb_msgdesc_t star_v1_CurrentSensorCalibration_msg;
00749 extern const pb_msgdesc_t star_v1_TemperatureCalibration_msg;
00750 extern const pb_msgdesc_t star_v1_SafetyThresholds_msg;
00751 extern const pb_msgdesc_t star_v1_EncoderConfiguration_msg;
00752 extern const pb_msgdesc_t star_v1_TimingConfiguration_msg;
00753 extern const pb_msgdesc_t star_v1_ConfigValidationResult_msg;
00754 extern const pb_msgdesc_t star_v1_ConfigValidationErrorResponse_msg;
00755
00756 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00757 #define star_v1_GetConfigurationRequest_fields &star_v1_GetConfigurationRequest_msg
00758 #define star_v1_GetConfigurationResponse_fields &star_v1_GetConfigurationResponse_msg
00759 #define star_v1_SetConfigurationRequest_fields &star_v1_SetConfigurationRequest_msg
00760 #define star_v1_SetConfigurationResponse_fields &star_v1_SetConfigurationResponse_msg
00761 #define star_v1_ResetToDefaultsRequest_fields &star_v1_ResetToDefaultsRequest_msg
00762 #define star_v1_ResetToDefaultsResponse_fields &star_v1_ResetToDefaultsResponse_msg
00763 #define star_v1_ValidateConfigurationRequest_fields &star_v1_ValidateConfigurationRequest_msg
00764 #define star_v1_ValidateConfigurationResponse_fields &star_v1_ValidateConfigurationResponse_msg
00765 #define star_v1_SaveConfigurationRequest_fields &star_v1_SaveConfigurationRequest_msg
00766 #define star_v1_SaveConfigurationResponse_fields &star_v1_SaveConfigurationResponse_msg
00767 #define star_v1_GetMotorPidConfigRequest_fields &star_v1_GetMotorPidConfigRequest_msg
00768 #define star_v1_GetMotorPidConfigResponse_fields &star_v1_GetMotorPidConfigResponse_msg
00769 #define star_v1_SetMotorPidConfigRequest_fields &star_v1_SetMotorPidConfigRequest_msg
00770 #define star_v1_SetMotorPidConfigResponse_fields &star_v1_SetMotorPidConfigResponse_msg
00771 #define star_v1_RetransmitConfig_fields &star_v1_RetransmitConfig_msg
00772 #define star_v1_SetRetransmitConfigRequest_fields &star_v1_SetRetransmitConfigRequest_msg
00773 #define star_v1_SetRetransmitConfigResponse_fields &star_v1_SetRetransmitConfigResponse_msg
00774 #define star_v1_SystemConfiguration_fields &star_v1_SystemConfiguration_msg
00775 #define star_v1_MotorPidConfiguration_fields &star_v1_MotorPidConfiguration_msg
00776 #define star_v1_CurrentSensorCalibration_fields &star_v1_CurrentSensorCalibration_msg
00777 #define star_v1_TemperatureCalibration_fields &star_v1_TemperatureCalibration_msg
00778 #define star_v1_SafetyThresholds_fields &star_v1_SafetyThresholds_msg
00779 #define star_v1_EncoderConfiguration_fields &star_v1_EncoderConfiguration_msg
00780 #define star_v1_TimingConfiguration_fields &star_v1_TimingConfiguration_msg
00781 #define star_v1_ConfigValidationResult_fields &star_v1_ConfigValidationResult_msg
00782 #define star_v1_ConfigValidationErrorResponse_fields &star_v1_ConfigValidationErrorResponse_msg
00783
00784 /* Maximum encoded size of messages (where known) */
00785 /* star_v1_GetConfigurationResponse_size depends on runtime parameters */
00786 /* star_v1_SetConfigurationRequest_size depends on runtime parameters */
00787 /* star_v1_ResetToDefaultsResponse_size depends on runtime parameters */
00788 /* star_v1_ValidateConfigurationRequest_size depends on runtime parameters */
00789 /* star_v1_SystemConfiguration_size depends on runtime parameters */
00790 /* star_v1_CurrentSensorCalibration_size depends on runtime parameters */
00791 #define STAR_V1_STAR_V1_CONFIGURATION_PB_H_MAX_SIZE star_v1_SetConfigurationResponse_size
00792 #define star_v1_ConfigValidationErrorResponse_size 585
00793 #define star_v1_ConfigValidationResult_size 4706
00794 #define star_v1_EncoderConfiguration_size 24
00795 #define star_v1_GetConfigurationRequest_size 149
00796 #define star_v1_GetMotorPidConfigRequest_size 160
00797 #define star_v1_GetMotorPidConfigResponse_size 439
00798 #define star_v1_MotorPidConfiguration_size 76
00799 #define star_v1_ResetToDefaultsRequest_size 151
00800 #define star_v1_RetransmitConfig_size 20
00801 #define star_v1_SafetyThresholds_size 23
00802 #define star_v1_SaveConfigurationRequest_size 149
00803 #define star_v1_SaveConfigurationResponse_size 371
00804 #define star_v1_SetConfigurationResponse_size 5072
00805 #define star_v1_SetMotorPidConfigRequest_size 227
00806 #define star_v1_SetMotorPidConfigResponse_size 5072
00807 #define star_v1_SetRetransmitConfigRequest_size 171
00808 #define star_v1_SetRetransmitConfigResponse_size 363
00809 #define star_v1_TemperatureCalibration_size 9
00810 #define star_v1_TimingConfiguration_size 18
00811 #define star_v1_ValidateConfigurationResponse_size 5072
00812
00813 #ifdef __cplusplus
00814 } /* extern "C" */
00815 #endif
00816
00817 #endif

```

## 6.17 libs/rx\_nanopb/inc/gen/star/v1/controller.pb.c File Reference

```
#include "star/v1/controller.pb.h"
```

Include dependency graph for controller.pb.c:



## 6.18 controller.pb.c

[Go to the documentation of this file.](#)

```

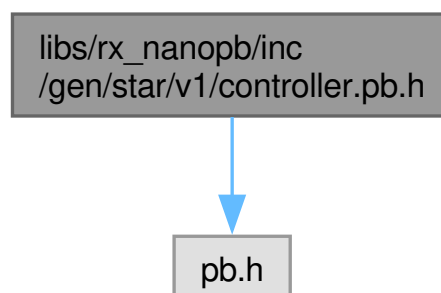
00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "star/v1/controller.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40
00006 #error Regenerate this file with the current version of nanopb generator.
00007 #endif
00008
00009 PB_BIND(star_v1_ControllerState, star_v1_ControllerState, AUTO)
00010
00011
00012

```

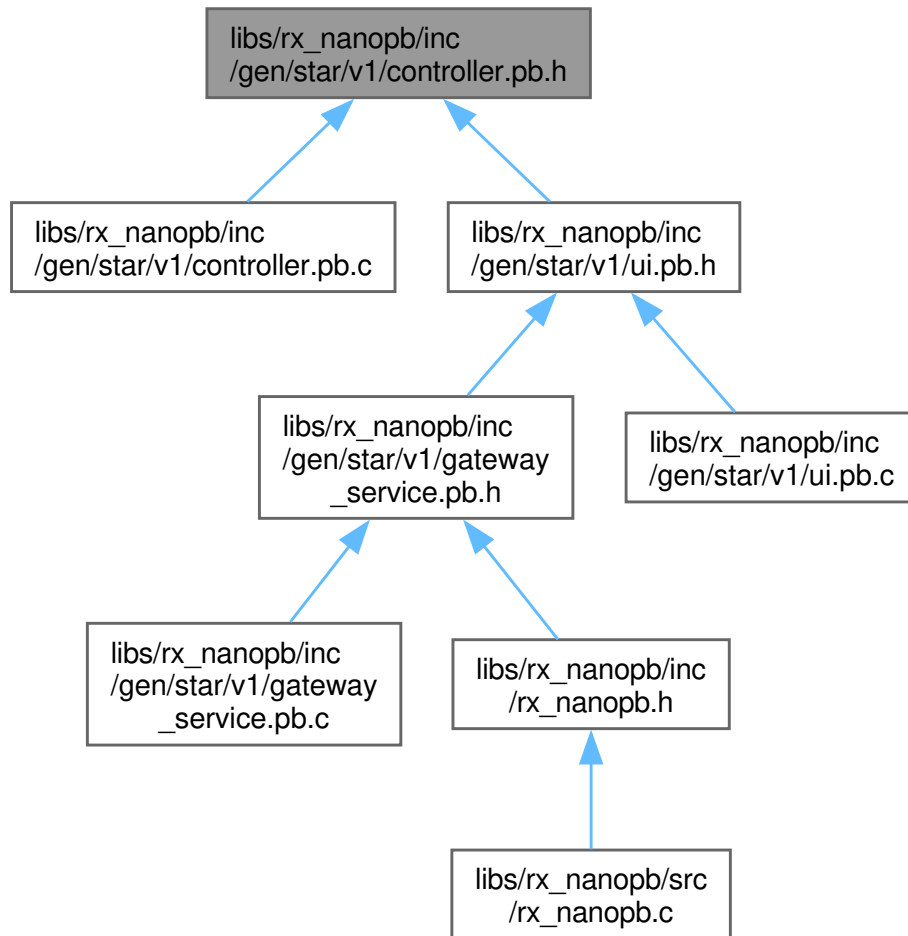
## 6.19 libs/rx\_nanopb/inc/gen/star/v1/controller.pb.h File Reference

```
#include <pb.h>
```

Include dependency graph for controller.pb.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct `star_v1_ControllerState`

## Macros

- `#define star_v1_ControllerState_init_default {0, 0, 0, 0}`
- `#define star_v1_ControllerState_init_zero {0, 0, 0, 0}`
- `#define star_v1_ControllerState_linear_vel_tag 1`
- `#define star_v1_ControllerState_angular_vel_tag 2`
- `#define star_v1_ControllerState_timestamp_ms_tag 3`
- `#define star_v1_ControllerState_debug_tag 4`
- `#define star_v1_ControllerState_FIELDLIST(X, a)`
- `#define star_v1_ControllerState_CALLBACK NULL`
- `#define star_v1_ControllerState_DEFAULT NULL`
- `#define star_v1_ControllerState_fields &star_v1_ControllerState_msg`
- `#define STAR_V1_STAR_V1_CONTROLLER_PB_H_MAX_SIZE star_v1_ControllerState_size`
- `#define star_v1_ControllerState_size 23`

## Variables

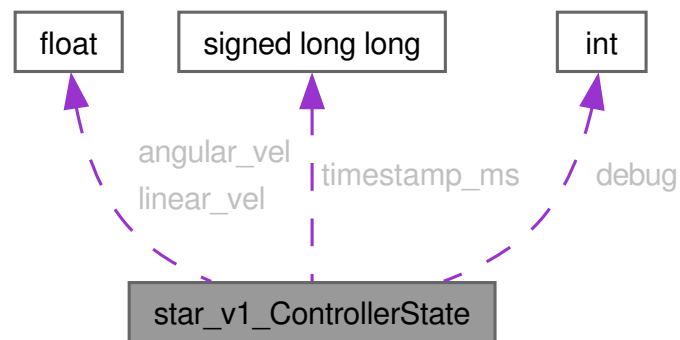
- `const pb_msgdesc_t star_v1_ControllerState_msg`

## 6.19.1 Data Structure Documentation

### 6.19.1.1 struct star\_v1\_ControllerState

Definition at line 14 of file [controller.pb.h](#).

Collaboration diagram for star\_v1\_ControllerState:



Data Fields

|         |              |  |
|---------|--------------|--|
| float   | angular_vel  |  |
| bool    | debug        |  |
| float   | linear_vel   |  |
| int64_t | timestamp_ms |  |

## 6.19.2 Macro Definition Documentation

### 6.19.2.1 star\_v1\_ControllerState\_angular\_vel\_tag

```
#define star_v1_ControllerState_angular_vel_tag 2
```

Definition at line 38 of file [controller.pb.h](#).

### 6.19.2.2 star\_v1\_ControllerState\_CALLBACK

```
#define star_v1_ControllerState_CALLBACK NULL
```

Definition at line 48 of file [controller.pb.h](#).

### 6.19.2.3 star\_v1\_ControllerState\_debug\_tag

```
#define star_v1_ControllerState_debug_tag 4
```

Definition at line 40 of file [controller.pb.h](#).

#### 6.19.2.4 star\_v1\_ControllerState\_DEFAULT

```
#define star_v1_ControllerState_DEFAULT NULL
```

Definition at line 49 of file [controller.pb.h](#).

#### 6.19.2.5 star\_v1\_ControllerState\_FIELDLIST

```
#define star_v1_ControllerState_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, FLOAT, linear_vel, 1) \
X(a, STATIC, SINGULAR, FLOAT, angular_vel, 2) \
X(a, STATIC, SINGULAR, INT64, timestamp_ms, 3) \
X(a, STATIC, SINGULAR, BOOL, debug, 4)
```

Definition at line 43 of file [controller.pb.h](#).

```
00043 #define star_v1_ControllerState_FIELDLIST(X, a) \
00044 X(a, STATIC, SINGULAR, FLOAT, linear_vel, 1) \
00045 X(a, STATIC, SINGULAR, FLOAT, angular_vel, 2) \
00046 X(a, STATIC, SINGULAR, INT64, timestamp_ms, 3) \
00047 X(a, STATIC, SINGULAR, BOOL, debug, 4)
```

#### 6.19.2.6 star\_v1\_ControllerState\_fields

```
#define star_v1_ControllerState_fields &star_v1_ControllerState_msg
```

Definition at line 54 of file [controller.pb.h](#).

#### 6.19.2.7 star\_v1\_ControllerState\_init\_default

```
#define star_v1_ControllerState_init_default {0, 0, 0, 0}
```

Definition at line 33 of file [controller.pb.h](#).

#### 6.19.2.8 star\_v1\_ControllerState\_init\_zero

```
#define star_v1_ControllerState_init_zero {0, 0, 0, 0}
```

Definition at line 34 of file [controller.pb.h](#).

#### 6.19.2.9 star\_v1\_ControllerState\_linear\_vel\_tag

```
#define star_v1_ControllerState_linear_vel_tag 1
```

Definition at line 37 of file [controller.pb.h](#).

6.19.2.10 `star_v1_ControllerState_size`

```
#define star_v1_ControllerState_size 23
```

Definition at line 58 of file [controller.pb.h](#).

6.19.2.11 `star_v1_ControllerState_timestamp_ms_tag`

```
#define star_v1_ControllerState_timestamp_ms_tag 3
```

Definition at line 39 of file [controller.pb.h](#).

6.19.2.12 `STAR_V1_STAR_V1_CONTROLLER_PB_H_MAX_SIZE`

```
#define STAR_V1_STAR_V1_CONTROLLER_PB_H_MAX_SIZE star_v1_ControllerState_size
```

Definition at line 57 of file [controller.pb.h](#).

## 6.19.3 Variable Documentation

6.19.3.1 `star_v1_ControllerState_msg`

```
const pb_msgdesc_t star_v1_ControllerState_msg [extern]
```

6.20 `controller.pb.h`

[Go to the documentation of this file.](#)

```
00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_STAR_V1_STAR_V1_CONTROLLER_PB_H_INCLUDED
00005 #define PB_STAR_V1_STAR_V1_CONTROLLER_PB_H_INCLUDED
00006 #include <pb.h>
00007
00008 #if PB_PROTO_HEADER_VERSION != 40
00009 #error Regenerate this file with the current version of nanopb generator.
00010 #endif
00011
00012 /* Struct definitions */
00013 /* ControllerState represents the state of a gamepad controller for direct robot control. */
00014 typedef struct _star_v1_ControllerState {
00015 /* Linear velocity (v) from Left Stick Y-axis. Range: [-1.0, 1.0] */
00016 float linear_vel;
00017 /* Angular velocity (omega) from Left Stick X-axis. Range: [-1.0, 1.0] */
00018 float angular_vel;
00019 /* Timestamp in milliseconds (e.g., since epoch or monotonic boot time)
00020 Used for jitter buffer and out-of-order packet handling.
00021 Project MKS-suffix mandate: numeric fields name the unit; ms here. */
00022 int64_t timestamp_ms;
00023 /* Enable verbose debug logging on the gateway */
00024 bool debug;
00025 } star_v1_ControllerState;
00026
00027
00028 #ifdef __cplusplus
00029 extern "C" {
00030 #endif
00031
00032 /* Initializer values for message structs */
00033 #define star_v1_ControllerState_init_default {0, 0, 0, 0}
00034 #define star_v1_ControllerState_init_zero {0, 0, 0, 0}
```

```

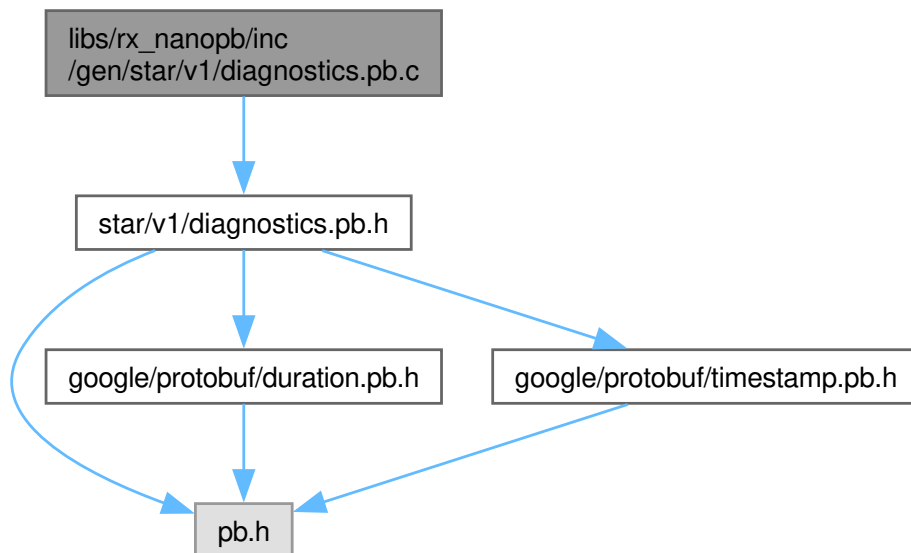
00035
00036 /* Field tags (for use in manual encoding/decoding) */
00037 #define star_v1_ControllerState_linear_vel_tag 1
00038 #define star_v1_ControllerState_angular_vel_tag 2
00039 #define star_v1_ControllerState_timestamp_ms_tag 3
00040 #define star_v1_ControllerState_debug_tag 4
00041
00042 /* Struct field encoding specification for nanopb */
00043 #define star_v1_ControllerState_FIELDLIST(X, a) \
00044 X(a, STATIC, SINGULAR, FLOAT, linear_vel, 1) \
00045 X(a, STATIC, SINGULAR, FLOAT, angular_vel, 2) \
00046 X(a, STATIC, SINGULAR, INT64, timestamp_ms, 3) \
00047 X(a, STATIC, SINGULAR, BOOL, debug, 4) \
00048 #define star_v1_ControllerState_CALLBACK NULL
00049 #define star_v1_ControllerState_DEFAULT NULL
00050
00051 extern const pb_msgdesc_t star_v1_ControllerState_msg;
00052
00053 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00054 #define star_v1_ControllerState_fields &star_v1_ControllerState_msg
00055
00056 /* Maximum encoded size of messages (where known) */
00057 #define STAR_V1_STAR_V1_CONTROLLER_PB_H_MAX_SIZE star_v1_ControllerState_size
00058 #define star_v1_ControllerState_size 23
00059
00060 #ifdef __cplusplus
00061 } /* extern "C" */
00062 #endif
00063
00064 #endif

```

## 6.21 libs/rx\_nanopb/inc/gen/star/v1/diagnostics.pb.c File Reference

#include "star/v1/diagnostics.pb.h"

Include dependency graph for diagnostics.pb.c:



## 6.22 diagnostics.pb.c

[Go to the documentation of this file.](#)

```

00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "star/v1/diagnostics.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40
00006 #error Regenerate this file with the current version of nanopb generator.

```

```

00007 #endif
00008
00009 PB_BIND(star_v1_TransportHealthReport, star_v1_TransportHealthReport, AUTO)
00010
00011
00012 PB_BIND(star_v1_TransportDiagnostics, star_v1_TransportDiagnostics, 2)
00013
00014
00015
00016
00017

```

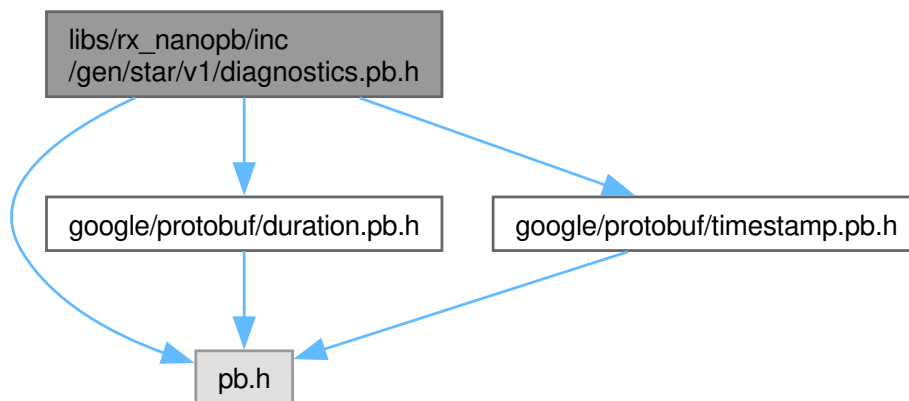
## 6.23 libs/rx\_nanopb/inc/gen/star/v1/diagnostics.pb.h File Reference

```

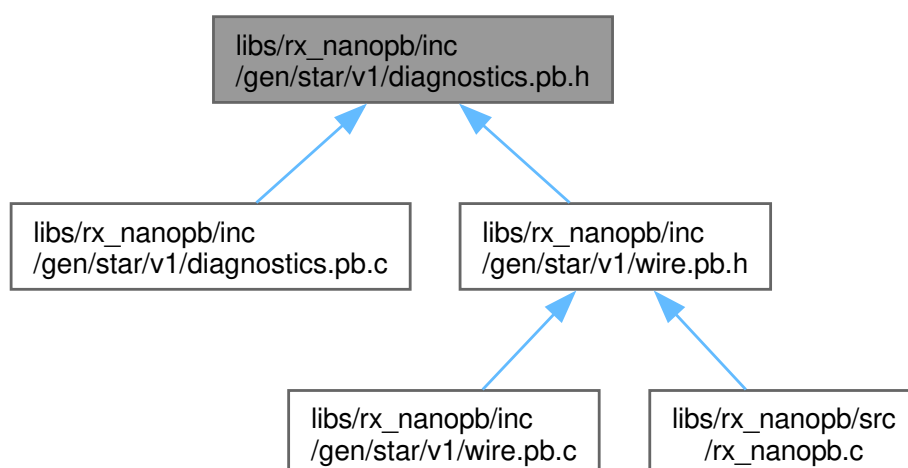
#include <pb.h>
#include "google/protobuf/duration.pb.h"
#include "google/protobuf/timestamp.pb.h"

```

Include dependency graph for diagnostics.pb.h:



This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [star\\_v1\\_TransportHealthReport](#)
- struct [star\\_v1\\_TransportDiagnostics](#)



## Macros

- `#define _star_v1_TransportType_MIN star_v1_TransportType_TRANSPORT_TYPE_UNKNOWN`
- `#define _star_v1_TransportType_MAX star_v1_TransportType_TRANSPORT_TYPE_SPI`
- `#define _star_v1_TransportType_ARRAYSIZE ((star_v1_TransportType)(star_v1_TransportType_TRANSP`
- `#define star_v1_TransportHealthReport_transport_type_ENUMTYPE star_v1_TransportType`
- `#define star_v1_TransportHealthReport_init_default {"" , _star_v1_TransportType_MIN, 0, 0, 0, 0, 0, 0, false, google_protobuf_Duration_init_default, 0, false, google_protobuf_Timestamp_init_default, false, google_protobuf_Timestamp_init_default, false, google_protobuf_Timestamp_init_default}`
- `#define star_v1_TransportDiagnostics_init_default {0, {star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default}, "" , 0, false, google_protobuf_Timestamp_init_default}`
- `#define star_v1_TransportHealthReport_init_zero {"" , _star_v1_TransportType_MIN, 0, 0, 0, 0, 0, 0, false, google_protobuf_Duration_init_zero, 0, false, google_protobuf_Timestamp_init_zero, false, google_protobuf_Timestamp_init_zero, false, google_protobuf_Timestamp_init_zero}`
- `#define star_v1_TransportDiagnostics_init_zero {0, {star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero}, "" , 0, false, google_protobuf_Timestamp_init_zero}`
- `#define star_v1_TransportHealthReport_transport_name_tag 1`
- `#define star_v1_TransportHealthReport_transport_type_tag 2`
- `#define star_v1_TransportHealthReport_is_active_tag 3`
- `#define star_v1_TransportHealthReport_is_healthy_tag 4`
- `#define star_v1_TransportHealthReport_total_sent_tag 10`
- `#define star_v1_TransportHealthReport_total_received_tag 11`
- `#define star_v1_TransportHealthReport_total_errors_tag 12`
- `#define star_v1_TransportHealthReport_consecutive_failures_tag 13`
- `#define star_v1_TransportHealthReport_avg_latency_tag 14`
- `#define star_v1_TransportHealthReport_packet_loss_rate_tag 15`
- `#define star_v1_TransportHealthReport_last_success_tag 20`
- `#define star_v1_TransportHealthReport_last_failure_tag 21`
- `#define star_v1_TransportHealthReport_last_recovery_tag 22`
- `#define star_v1_TransportDiagnostics_transports_tag 1`
- `#define star_v1_TransportDiagnostics_active_transport_tag 2`
- `#define star_v1_TransportDiagnostics_switch_count_tag 3`
- `#define star_v1_TransportDiagnostics_collected_at_tag 4`
- `#define star_v1_TransportHealthReport_FIELDLIST(X, a)`
- `#define star_v1_TransportHealthReport_CALLBACK NULL`
- `#define star_v1_TransportHealthReport_DEFAULT NULL`
- `#define star_v1_TransportHealthReport_avg_latency_MSGTYPE google_protobuf_Duration`
- `#define star_v1_TransportHealthReport_last_success_MSGTYPE google_protobuf_Timestamp`
- `#define star_v1_TransportHealthReport_last_failure_MSGTYPE google_protobuf_Timestamp`
- `#define star_v1_TransportHealthReport_last_recovery_MSGTYPE google_protobuf_Timestamp`
- `#define star_v1_TransportDiagnostics_FIELDLIST(X, a)`
- `#define star_v1_TransportDiagnostics_CALLBACK NULL`
- `#define star_v1_TransportDiagnostics_DEFAULT NULL`
- `#define star_v1_TransportDiagnostics_transports_MSGTYPE star_v1_TransportHealthReport`
- `#define star_v1_TransportDiagnostics_collected_at_MSGTYPE google_protobuf_Timestamp`
- `#define star_v1_TransportHealthReport_fields &star_v1_TransportHealthReport_msg`
- `#define star_v1_TransportDiagnostics_fields &star_v1_TransportDiagnostics_msg`
- `#define STAR_V1_STAR_V1_DIAGNOSTICS_PB_H_MAX_SIZE star_v1_TransportDiagnostics_size`
- `#define star_v1_TransportDiagnostics_size 1399`
- `#define star_v1_TransportHealthReport_size 166`

## Enumerations

- enum [star\\_v1\\_TransportType](#) { [star\\_v1\\_TransportType\\_TRANSPORT\\_TYPE\\_UNKNOWN](#) = 0 , [star\\_v1\\_TransportType\\_TRANSPORT\\_TYPE\\_USB\\_CDC](#) = 1 , [star\\_v1\\_TransportType\\_TRANSPORT\\_TYPE\\_USB\\_CDC](#) = 2 }

## Variables

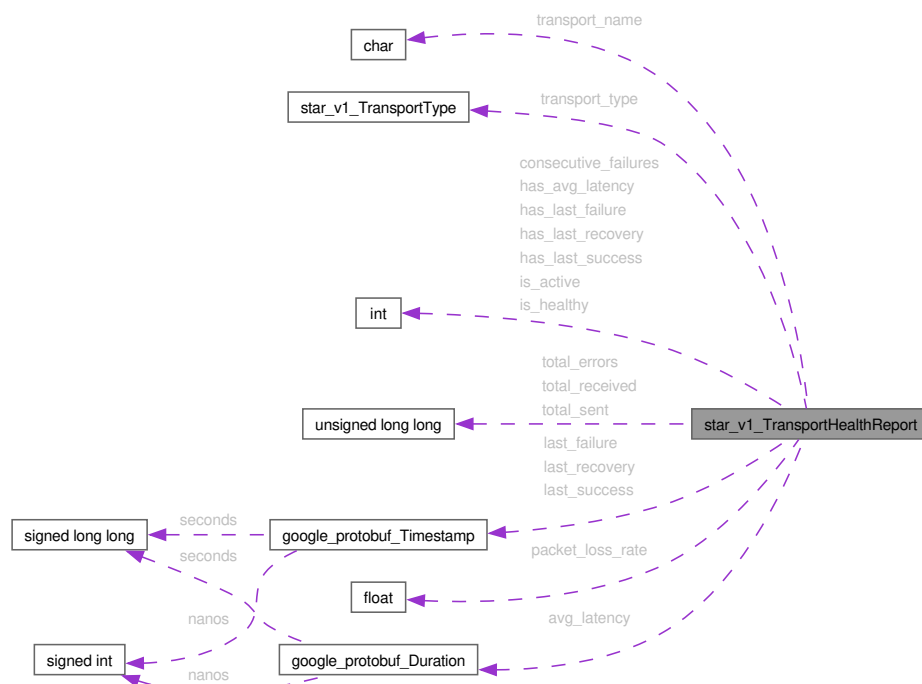
- const pb\_msgdesc\_t [star\\_v1\\_TransportHealthReport\\_msg](#)
- const pb\_msgdesc\_t [star\\_v1\\_TransportDiagnostics\\_msg](#)

## 6.23.1 Data Structure Documentation

### 6.23.1.1 struct [star\\_v1\\_TransportHealthReport](#)

Definition at line 28 of file [diagnostics.pb.h](#).

Collaboration diagram for [star\\_v1\\_TransportHealthReport](#):



## Data Fields

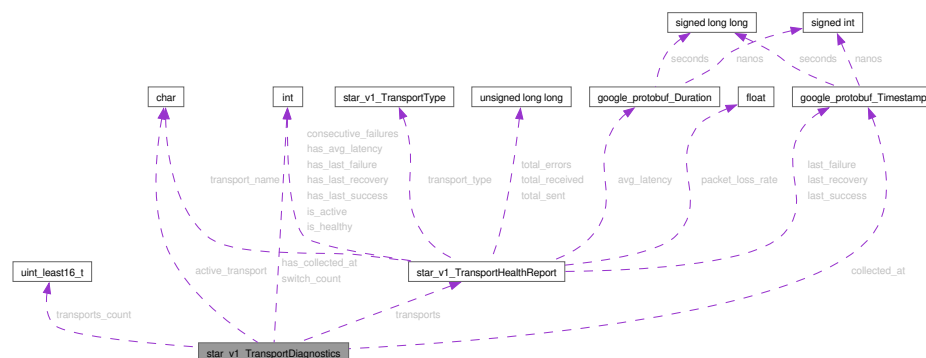
|                                          |                      |  |
|------------------------------------------|----------------------|--|
| <a href="#">google_protobuf_Duration</a> | avg_latency          |  |
| <a href="#">uint32_t</a>                 | consecutive_failures |  |
| <a href="#">bool</a>                     | has_avg_latency      |  |
| <a href="#">bool</a>                     | has_last_failure     |  |
| <a href="#">bool</a>                     | has_last_recovery    |  |
| <a href="#">bool</a>                     | has_last_success     |  |

|                           |                    |  |
|---------------------------|--------------------|--|
| bool                      | is_active          |  |
| bool                      | is_healthy         |  |
| google_protobuf_Timestamp | last_failure       |  |
| google_protobuf_Timestamp | last_recovery      |  |
| google_protobuf_Timestamp | last_success       |  |
| float                     | packet_loss_rate   |  |
| uint64_t                  | total_errors       |  |
| uint64_t                  | total_received     |  |
| uint64_t                  | total_sent         |  |
| char                      | transport_name[16] |  |
| star_v1_TransportType     | transport_type     |  |

### 6.23.1.2 struct star\_v1\_TransportDiagnostics

Definition at line 63 of file [diagnostics.pb.h](#).

Collaboration diagram for star\_v1\_TransportDiagnostics:



### Data Fields

|                               |                      |  |
|-------------------------------|----------------------|--|
| char                          | active_transport[16] |  |
| google_protobuf_Timestamp     | collected_at         |  |
| bool                          | has_collected_at     |  |
| uint32_t                      | switch_count         |  |
| star_v1_TransportHealthReport | transports[8]        |  |
| pb_size_t                     | transports_count     |  |

## 6.23.2 Macro Definition Documentation

### 6.23.2.1 \_star\_v1\_TransportType\_ARRAYSIZE

```
#define _star_v1_TransportType_ARRAYSIZE ((star_v1_TransportType)(star_v1_TransportType_TRANSPORT_TYPE_SPI+1))
```

Definition at line 84 of file [diagnostics.pb.h](#).

### 6.23.2.2 \_\_star\_v1\_TransportType\_MAX

```
#define __star_v1_TransportType_MAX star_v1_TransportType_TRANSPORT_TYPE_SPI
```

Definition at line 83 of file [diagnostics.pb.h](#).

### 6.23.2.3 \_\_star\_v1\_TransportType\_MIN

```
#define __star_v1_TransportType_MIN star_v1_TransportType_TRANSPORT_TYPE_UNKNOWN
```

Definition at line 82 of file [diagnostics.pb.h](#).

### 6.23.2.4 STAR\_V1\_STAR\_V1\_DIAGNOSTICS\_PB\_H\_MAX\_SIZE

```
#define STAR_V1_STAR_V1_DIAGNOSTICS_PB_H_MAX_SIZE star_v1_TransportDiagnostics_size
```

Definition at line 155 of file [diagnostics.pb.h](#).

### 6.23.2.5 star\_v1\_TransportDiagnostics\_active\_transport\_tag

```
#define star_v1_TransportDiagnostics_active_transport_tag 2
```

Definition at line 111 of file [diagnostics.pb.h](#).

### 6.23.2.6 star\_v1\_TransportDiagnostics\_CALLBACK

```
#define star_v1_TransportDiagnostics_CALLBACK NULL
```

Definition at line 142 of file [diagnostics.pb.h](#).

### 6.23.2.7 star\_v1\_TransportDiagnostics\_collected\_at\_MSGTYPE

```
#define star_v1_TransportDiagnostics_collected_at_MSGTYPE google_protobuf_Timestamp
```

Definition at line 145 of file [diagnostics.pb.h](#).

### 6.23.2.8 star\_v1\_TransportDiagnostics\_collected\_at\_tag

```
#define star_v1_TransportDiagnostics_collected_at_tag 4
```

Definition at line 113 of file [diagnostics.pb.h](#).

### 6.23.2.9 star\_v1\_TransportDiagnostics\_DEFAULT

```
#define star_v1_TransportDiagnostics_DEFAULT NULL
```

Definition at line 143 of file [diagnostics.pb.h](#).

## 6.23.2.10 star\_v1\_TransportDiagnostics\_FIELDLIST

```
#define star_v1_TransportDiagnostics_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, REPEATED, MESSAGE, transports, 1) \
X(a, STATIC, SINGULAR, STRING, active_transport, 2) \
X(a, STATIC, SINGULAR, UINT32, switch_count, 3) \
X(a, STATIC, OPTIONAL, MESSAGE, collected_at, 4)
```

Definition at line 137 of file [diagnostics.pb.h](#).

```
00137 #define star_v1_TransportDiagnostics_FIELDLIST(X, a) \
00138 X(a, STATIC, REPEATED, MESSAGE, transports, 1) \
00139 X(a, STATIC, SINGULAR, STRING, active_transport, 2) \
00140 X(a, STATIC, SINGULAR, UINT32, switch_count, 3) \
00141 X(a, STATIC, OPTIONAL, MESSAGE, collected_at, 4)
```

## 6.23.2.11 star\_v1\_TransportDiagnostics\_fields

```
#define star_v1_TransportDiagnostics_fields &star_v1_TransportDiagnostics_msg
```

Definition at line 152 of file [diagnostics.pb.h](#).

## 6.23.2.12 star\_v1\_TransportDiagnostics\_init\_default

```
#define star_v1_TransportDiagnostics_init_default {0, {star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default}, 0, false, google_protobuf_Timestamp_init_default}
```

Definition at line 92 of file [diagnostics.pb.h](#).

## 6.23.2.13 star\_v1\_TransportDiagnostics\_init\_zero

```
#define star_v1_TransportDiagnostics_init_zero {0, {star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero}, 0, false, google_protobuf_Timestamp_init_zero}
```

Definition at line 94 of file [diagnostics.pb.h](#).

## 6.23.2.14 star\_v1\_TransportDiagnostics\_size

```
#define star_v1_TransportDiagnostics_size 1399
```

Definition at line 156 of file [diagnostics.pb.h](#).

## 6.23.2.15 star\_v1\_TransportDiagnostics\_switch\_count\_tag

```
#define star_v1_TransportDiagnostics_switch_count_tag 3
```

Definition at line 112 of file [diagnostics.pb.h](#).

#### 6.23.2.16 `star_v1_TransportDiagnostics_transports_MSGTYPE`

```
#define star_v1_TransportDiagnostics_transports_MSGTYPE star_v1_TransportHealthReport
```

Definition at line [144](#) of file [diagnostics.pb.h](#).

#### 6.23.2.17 `star_v1_TransportDiagnostics_transports_tag`

```
#define star_v1_TransportDiagnostics_transports_tag 1
```

Definition at line [110](#) of file [diagnostics.pb.h](#).

#### 6.23.2.18 `star_v1_TransportHealthReport_avg_latency_MSGTYPE`

```
#define star_v1_TransportHealthReport_avg_latency_MSGTYPE google_protobuf_Duration
```

Definition at line [132](#) of file [diagnostics.pb.h](#).

#### 6.23.2.19 `star_v1_TransportHealthReport_avg_latency_tag`

```
#define star_v1_TransportHealthReport_avg_latency_tag 14
```

Definition at line [105](#) of file [diagnostics.pb.h](#).

#### 6.23.2.20 `star_v1_TransportHealthReport_CALLBACK`

```
#define star_v1_TransportHealthReport_CALLBACK NULL
```

Definition at line [130](#) of file [diagnostics.pb.h](#).

#### 6.23.2.21 `star_v1_TransportHealthReport_consecutive_failures_tag`

```
#define star_v1_TransportHealthReport_consecutive_failures_tag 13
```

Definition at line [104](#) of file [diagnostics.pb.h](#).

#### 6.23.2.22 `star_v1_TransportHealthReport_DEFAULT`

```
#define star_v1_TransportHealthReport_DEFAULT NULL
```

Definition at line [131](#) of file [diagnostics.pb.h](#).

## 6.23.2.23 star\_v1\_TransportHealthReport\_FIELDLIST

```
#define star_v1_TransportHealthReport_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, STRING, transport_name, 1) \
X(a, STATIC, SINGULAR, UENUM, transport_type, 2) \
X(a, STATIC, SINGULAR, BOOL, is_active, 3) \
X(a, STATIC, SINGULAR, BOOL, is_healthy, 4) \
X(a, STATIC, SINGULAR, UINT64, total_sent, 10) \
X(a, STATIC, SINGULAR, UINT64, total_received, 11) \
X(a, STATIC, SINGULAR, UINT64, total_errors, 12) \
X(a, STATIC, SINGULAR, UINT32, consecutive_failures, 13) \
X(a, STATIC, OPTIONAL, MESSAGE, avg_latency, 14) \
X(a, STATIC, SINGULAR, FLOAT, packet_loss_rate, 15) \
X(a, STATIC, OPTIONAL, MESSAGE, last_success, 20) \
X(a, STATIC, OPTIONAL, MESSAGE, last_failure, 21) \
X(a, STATIC, OPTIONAL, MESSAGE, last_recovery, 22)
```

Definition at line 116 of file [diagnostics.pb.h](#).

```
00116 #define star_v1_TransportHealthReport_FIELDLIST(X, a) \
00117 X(a, STATIC, SINGULAR, STRING, transport_name, 1) \
00118 X(a, STATIC, SINGULAR, UENUM, transport_type, 2) \
00119 X(a, STATIC, SINGULAR, BOOL, is_active, 3) \
00120 X(a, STATIC, SINGULAR, BOOL, is_healthy, 4) \
00121 X(a, STATIC, SINGULAR, UINT64, total_sent, 10) \
00122 X(a, STATIC, SINGULAR, UINT64, total_received, 11) \
00123 X(a, STATIC, SINGULAR, UINT64, total_errors, 12) \
00124 X(a, STATIC, SINGULAR, UINT32, consecutive_failures, 13) \
00125 X(a, STATIC, OPTIONAL, MESSAGE, avg_latency, 14) \
00126 X(a, STATIC, SINGULAR, FLOAT, packet_loss_rate, 15) \
00127 X(a, STATIC, OPTIONAL, MESSAGE, last_success, 20) \
00128 X(a, STATIC, OPTIONAL, MESSAGE, last_failure, 21) \
00129 X(a, STATIC, OPTIONAL, MESSAGE, last_recovery, 22)
```

## 6.23.2.24 star\_v1\_TransportHealthReport\_fields

```
#define star_v1_TransportHealthReport_fields &star_v1_TransportHealthReport_msg
```

Definition at line 151 of file [diagnostics.pb.h](#).

## 6.23.2.25 star\_v1\_TransportHealthReport\_init\_default

```
#define star_v1_TransportHealthReport_init_default {", _star_v1_TransportType_MIN, 0, 0, 0, 0, 0, 0, false,
google_protobuf_Duration_init_default, 0, false, google_protobuf_Timestamp_init_default, false, google_protobuf_Timestamp_init_default,
false, google_protobuf_Timestamp_init_default}
```

Definition at line 91 of file [diagnostics.pb.h](#).

## 6.23.2.26 star\_v1\_TransportHealthReport\_init\_zero

```
#define star_v1_TransportHealthReport_init_zero {", _star_v1_TransportType_MIN, 0, 0, 0, 0, 0, 0, false,
google_protobuf_Duration_init_zero, 0, false, google_protobuf_Timestamp_init_zero, false, google_protobuf_Timestamp_init_zero,
false, google_protobuf_Timestamp_init_zero}
```

Definition at line 93 of file [diagnostics.pb.h](#).

#### 6.23.2.27 `star_v1_TransportHealthReport_is_active_tag`

```
#define star_v1_TransportHealthReport_is_active_tag 3
```

Definition at line 99 of file [diagnostics.pb.h](#).

#### 6.23.2.28 `star_v1_TransportHealthReport_is_healthy_tag`

```
#define star_v1_TransportHealthReport_is_healthy_tag 4
```

Definition at line 100 of file [diagnostics.pb.h](#).

#### 6.23.2.29 `star_v1_TransportHealthReport_last_failure_MSGTYPE`

```
#define star_v1_TransportHealthReport_last_failure_MSGTYPE google_protobuf_Timestamp
```

Definition at line 134 of file [diagnostics.pb.h](#).

#### 6.23.2.30 `star_v1_TransportHealthReport_last_failure_tag`

```
#define star_v1_TransportHealthReport_last_failure_tag 21
```

Definition at line 108 of file [diagnostics.pb.h](#).

#### 6.23.2.31 `star_v1_TransportHealthReport_last_recovery_MSGTYPE`

```
#define star_v1_TransportHealthReport_last_recovery_MSGTYPE google_protobuf_Timestamp
```

Definition at line 135 of file [diagnostics.pb.h](#).

#### 6.23.2.32 `star_v1_TransportHealthReport_last_recovery_tag`

```
#define star_v1_TransportHealthReport_last_recovery_tag 22
```

Definition at line 109 of file [diagnostics.pb.h](#).

#### 6.23.2.33 `star_v1_TransportHealthReport_last_success_MSGTYPE`

```
#define star_v1_TransportHealthReport_last_success_MSGTYPE google_protobuf_Timestamp
```

Definition at line 133 of file [diagnostics.pb.h](#).

#### 6.23.2.34 `star_v1_TransportHealthReport_last_success_tag`

```
#define star_v1_TransportHealthReport_last_success_tag 20
```

Definition at line 107 of file [diagnostics.pb.h](#).



6.23.2.35 `star_v1_TransportHealthReport_packet_loss_rate_tag`

```
#define star_v1_TransportHealthReport_packet_loss_rate_tag 15
```

Definition at line 106 of file [diagnostics.pb.h](#).

6.23.2.36 `star_v1_TransportHealthReport_size`

```
#define star_v1_TransportHealthReport_size 166
```

Definition at line 157 of file [diagnostics.pb.h](#).

6.23.2.37 `star_v1_TransportHealthReport_total_errors_tag`

```
#define star_v1_TransportHealthReport_total_errors_tag 12
```

Definition at line 103 of file [diagnostics.pb.h](#).

6.23.2.38 `star_v1_TransportHealthReport_total_received_tag`

```
#define star_v1_TransportHealthReport_total_received_tag 11
```

Definition at line 102 of file [diagnostics.pb.h](#).

6.23.2.39 `star_v1_TransportHealthReport_total_sent_tag`

```
#define star_v1_TransportHealthReport_total_sent_tag 10
```

Definition at line 101 of file [diagnostics.pb.h](#).

6.23.2.40 `star_v1_TransportHealthReport_transport_name_tag`

```
#define star_v1_TransportHealthReport_transport_name_tag 1
```

Definition at line 97 of file [diagnostics.pb.h](#).

6.23.2.41 `star_v1_TransportHealthReport_transport_type_ENUMTYPE`

```
#define star_v1_TransportHealthReport_transport_type_ENUMTYPE star_v1_TransportType
```

Definition at line 86 of file [diagnostics.pb.h](#).

6.23.2.42 `star_v1_TransportHealthReport_transport_type_tag`

```
#define star_v1_TransportHealthReport_transport_type_tag 2
```

Definition at line 98 of file [diagnostics.pb.h](#).

### 6.23.3 Enumeration Type Documentation

#### 6.23.3.1 star\_v1\_TransportType

enum [star\\_v1\\_TransportType](#)

Enumerator

|                                              |  |
|----------------------------------------------|--|
| star_v1_TransportType_TRANSPORT_TYPE_UNKNOWN |  |
| star_v1_TransportType_TRANSPORT_TYPE_USB_CDC |  |
| star_v1_TransportType_TRANSPORT_TYPE_SPI     |  |

Definition at line 16 of file [diagnostics.pb.h](#).

```

00016 {
00017 /* Unknown or unspecified transport. */
00018 star_v1_TransportType_TRANSPORT_TYPE_UNKNOWN = 0,
00019 /* USB CDC (ACM) serial transport. */
00020 star_v1_TransportType_TRANSPORT_TYPE_USB_CDC = 1,
00021 /* SPI full-duplex transport. */
00022 star_v1_TransportType_TRANSPORT_TYPE_SPI = 2
00023 } star_v1_TransportType;
```

### 6.23.4 Variable Documentation

#### 6.23.4.1 star\_v1\_TransportDiagnostics\_msg

const pb\_msgdesc\_t star\_v1\_TransportDiagnostics\_msg [extern]

#### 6.23.4.2 star\_v1\_TransportHealthReport\_msg

const pb\_msgdesc\_t star\_v1\_TransportHealthReport\_msg [extern]

## 6.24 diagnostics.pb.h

[Go to the documentation of this file.](#)

```

00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_STAR_V1_STAR_V1_DIAGNOSTICS_PB_H_INCLUDED
00005 #define PB_STAR_V1_STAR_V1_DIAGNOSTICS_PB_H_INCLUDED
00006 #include <pb.h>
00007 #include "google/protobuf/duration.pb.h"
00008 #include "google/protobuf/timestamp.pb.h"
00009
00010 #if PB_PROTO_HEADER_VERSION != 40
00011 #error Regenerate this file with the current version of nanopb generator.
00012 #endif
00013
00014 /* Enum definitions */
00015 /* TransportType identifies the physical transport. */
00016 typedef enum _star_v1_TransportType {
00017 /* Unknown or unspecified transport. */
00018 star_v1_TransportType_TRANSPORT_TYPE_UNKNOWN = 0,
00019 /* USB CDC (ACM) serial transport. */
00020 star_v1_TransportType_TRANSPORT_TYPE_USB_CDC = 1,
00021 /* SPI full-duplex transport. */
00022 star_v1_TransportType_TRANSPORT_TYPE_SPI = 2
00023 } star_v1_TransportType;
00024
00025 /* Struct definitions */
```

```

00026 /* TransportHealthReport contains health metrics for a single transport.
00027 Sent periodically by the gateway or on-demand for diagnostics. */
00028 typedef struct _star_v1_TransportHealthReport {
00029 /* Transport identifier (e.g., "usb", "spi"). */
00030 char transport_name[16];
00031 /* Physical transport type. */
00032 star_v1_TransportType transport_type;
00033 /* Whether this transport is currently the active (selected) transport. */
00034 bool is_active;
00035 /* Whether this transport is considered healthy. */
00036 bool is_healthy;
00037 /* Total frames sent since last reset. */
00038 uint64_t total_sent;
00039 /* Total frames received since last reset. */
00040 uint64_t total_received;
00041 /* Total errors encountered since last reset. */
00042 uint64_t total_errors;
00043 /* Current consecutive failure count (resets on success). */
00044 uint32_t consecutive_failures;
00045 /* Average round-trip latency. */
00046 bool has_avg_latency;
00047 google_protobuf_Duration avg_latency;
00048 /* Packet loss rate (0.0 to 1.0) over sliding window. */
00049 float packet_loss_rate;
00050 /* Timestamp of last successful operation. */
00051 bool has_last_success;
00052 google_protobuf_Timestamp last_success;
00053 /* Timestamp of last failure. */
00054 bool has_last_failure;
00055 google_protobuf_Timestamp last_failure;
00056 /* Timestamp of last recovery (unhealthy -> healthy transition). */
00057 bool has_last_recovery;
00058 google_protobuf_Timestamp last_recovery;
00059 } star_v1_TransportHealthReport;
00060
00061 /* TransportDiagnostics contains health reports for all transports.
00062 This is the top-level diagnostics message sent over the wire. */
00063 typedef struct _star_v1_TransportDiagnostics {
00064 /* Health reports for each registered transport. */
00065 pb_size_t transports_count;
00066 star_v1_TransportHealthReport transports[8];
00067 /* Name of the currently active transport. */
00068 char active_transport[16];
00069 /* Total number of transport switches since startup. */
00070 uint32_t switch_count;
00071 /* Timestamp when diagnostics were collected. */
00072 bool has_collected_at;
00073 google_protobuf_Timestamp collected_at;
00074 } star_v1_TransportDiagnostics;
00075
00076
00077 #ifdef __cplusplus
00078 extern "C" {
00079 #endif
00080
00081 /* Helper constants for enums */
00082 #define _star_v1_TransportType_MIN star_v1_TransportType_TRANSPORT_TYPE_UNKNOWN
00083 #define _star_v1_TransportType_MAX star_v1_TransportType_TRANSPORT_TYPE_SPI
00084 #define _star_v1_TransportType_ARRAYSIZE
00085 ((star_v1_TransportType)(star_v1_TransportType_TRANSPORT_TYPE_SPI+1))
00086
00087 #define star_v1_TransportHealthReport_transport_type_ENUMTYPE star_v1_TransportType
00088
00089
00090 /* Initializer values for message structs */
00091 #define star_v1_TransportHealthReport_init_default {"", _star_v1_TransportType_MIN, 0, 0, 0, 0, 0, false,
00092 google_protobuf_Duration_init_default, 0, false, google_protobuf_Timestamp_init_default, false,
00093 google_protobuf_Timestamp_init_default, false, google_protobuf_Timestamp_init_default}
00094 #define star_v1_TransportDiagnostics_init_default {0, {star_v1_TransportHealthReport_init_default,
00095 star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default,
00096 star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default,
00097 star_v1_TransportHealthReport_init_default, star_v1_TransportHealthReport_init_default,
00098 star_v1_TransportHealthReport_init_default}, "", 0, false, google_protobuf_Timestamp_init_default}
00099 #define star_v1_TransportHealthReport_init_zero {"", _star_v1_TransportType_MIN, 0, 0, 0, 0, 0, false,
00100 google_protobuf_Duration_init_zero, 0, false, google_protobuf_Timestamp_init_zero, false,
00101 google_protobuf_Timestamp_init_zero, false, google_protobuf_Timestamp_init_zero}
00102 #define star_v1_TransportDiagnostics_init_zero {0, {star_v1_TransportHealthReport_init_zero,
00103 star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero,
00104 star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero,
00105 star_v1_TransportHealthReport_init_zero, star_v1_TransportHealthReport_init_zero,
00106 star_v1_TransportHealthReport_init_zero}, "", 0, false, google_protobuf_Timestamp_init_zero}
00107
00108
00109 /* Field tags (for use in manual encoding/decoding) */
00110 #define star_v1_TransportHealthReport_transport_name_tag 1
00111 #define star_v1_TransportHealthReport_transport_type_tag 2
00112 #define star_v1_TransportHealthReport_is_active_tag 3

```

```

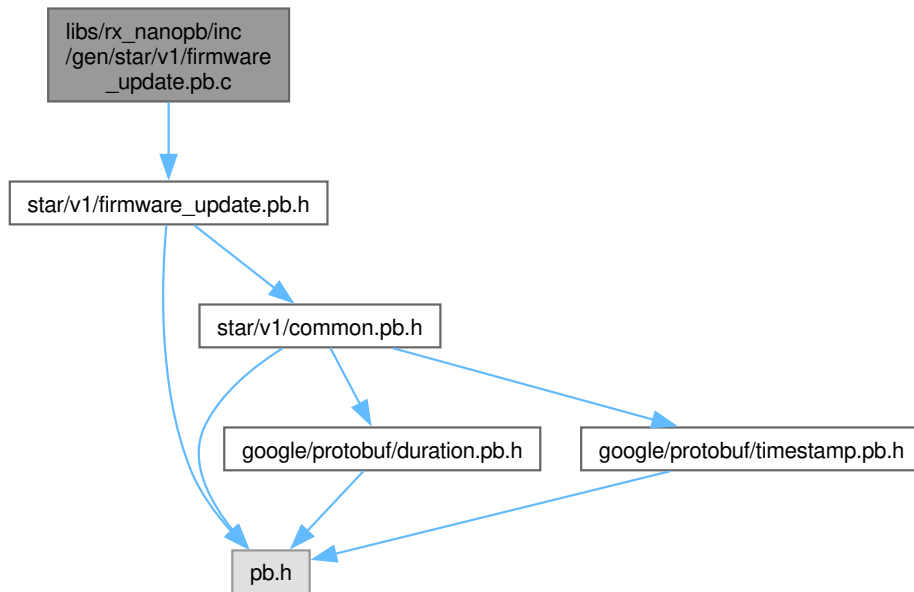
00100 #define star_v1_TransportHealthReport_is_healthy_tag 4
00101 #define star_v1_TransportHealthReport_total_sent_tag 10
00102 #define star_v1_TransportHealthReport_total_received_tag 11
00103 #define star_v1_TransportHealthReport_total_errors_tag 12
00104 #define star_v1_TransportHealthReport_consecutive_failures_tag 13
00105 #define star_v1_TransportHealthReport_avg_latency_tag 14
00106 #define star_v1_TransportHealthReport_packet_loss_rate_tag 15
00107 #define star_v1_TransportHealthReport_last_success_tag 20
00108 #define star_v1_TransportHealthReport_last_failure_tag 21
00109 #define star_v1_TransportHealthReport_last_recovery_tag 22
00110 #define star_v1_TransportDiagnostics_transports_tag 1
00111 #define star_v1_TransportDiagnostics_active_transport_tag 2
00112 #define star_v1_TransportDiagnostics_switch_count_tag 3
00113 #define star_v1_TransportDiagnostics_collected_at_tag 4
00114
00115 /* Struct field encoding specification for nanopb */
00116 #define star_v1_TransportHealthReport_FIELDLIST(X, a) \
00117 X(a, STATIC, SINGULAR, STRING, transport_name, 1) \
00118 X(a, STATIC, SINGULAR, UENUM, transport_type, 2) \
00119 X(a, STATIC, SINGULAR, BOOL, is_active, 3) \
00120 X(a, STATIC, SINGULAR, BOOL, is_healthy, 4) \
00121 X(a, STATIC, SINGULAR, UINT64, total_sent, 10) \
00122 X(a, STATIC, SINGULAR, UINT64, total_received, 11) \
00123 X(a, STATIC, SINGULAR, UINT64, total_errors, 12) \
00124 X(a, STATIC, SINGULAR, UINT32, consecutive_failures, 13) \
00125 X(a, STATIC, OPTIONAL, MESSAGE, avg_latency, 14) \
00126 X(a, STATIC, SINGULAR, FLOAT, packet_loss_rate, 15) \
00127 X(a, STATIC, OPTIONAL, MESSAGE, last_success, 20) \
00128 X(a, STATIC, OPTIONAL, MESSAGE, last_failure, 21) \
00129 X(a, STATIC, OPTIONAL, MESSAGE, last_recovery, 22)
00130 #define star_v1_TransportHealthReport_CALLBACK NULL
00131 #define star_v1_TransportHealthReport_DEFAULT NULL
00132 #define star_v1_TransportHealthReport_avg_latency_MSGTYPE google_protobuf_Duration
00133 #define star_v1_TransportHealthReport_last_success_MSGTYPE google_protobuf_Timestamp
00134 #define star_v1_TransportHealthReport_last_failure_MSGTYPE google_protobuf_Timestamp
00135 #define star_v1_TransportHealthReport_last_recovery_MSGTYPE google_protobuf_Timestamp
00136
00137 #define star_v1_TransportDiagnostics_FIELDLIST(X, a) \
00138 X(a, STATIC, REPEATED, MESSAGE, transports, 1) \
00139 X(a, STATIC, SINGULAR, STRING, active_transport, 2) \
00140 X(a, STATIC, SINGULAR, UINT32, switch_count, 3) \
00141 X(a, STATIC, OPTIONAL, MESSAGE, collected_at, 4)
00142 #define star_v1_TransportDiagnostics_CALLBACK NULL
00143 #define star_v1_TransportDiagnostics_DEFAULT NULL
00144 #define star_v1_TransportDiagnostics_transports_MSGTYPE star_v1_TransportHealthReport
00145 #define star_v1_TransportDiagnostics_collected_at_MSGTYPE google_protobuf_Timestamp
00146
00147 extern const pb_msgdesc_t star_v1_TransportHealthReport_msg;
00148 extern const pb_msgdesc_t star_v1_TransportDiagnostics_msg;
00149
00150 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00151 #define star_v1_TransportHealthReport_fields &star_v1_TransportHealthReport_msg
00152 #define star_v1_TransportDiagnostics_fields &star_v1_TransportDiagnostics_msg
00153
00154 /* Maximum encoded size of messages (where known) */
00155 #define STAR_V1_STAR_V1_DIAGNOSTICS_PB_H_MAX_SIZE star_v1_TransportDiagnostics_size
00156 #define star_v1_TransportDiagnostics_size 1399
00157 #define star_v1_TransportHealthReport_size 166
00158
00159 #ifdef __cplusplus
00160 } /* extern "C" */
00161 #endif
00162
00163 #endif

```

## 6.25 libs/rx\_nanopb/inc/gen/star/v1/firmware\_update.pb.c File Reference

#include "star/v1/firmware\_update.pb.h"

Include dependency graph for firmware\_update.pb.c:



## 6.26 firmware\_update.pb.c

[Go to the documentation of this file.](#)

```

00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "star/v1/firmware_update.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40
00006 #error Regenerate this file with the current version of nanopb generator.
00007 #endif
00008
00009 PB_BIND(star_v1_BeginUpdateRequest, star_v1_BeginUpdateRequest, AUTO)
00010
00011
00012 PB_BIND(star_v1_BeginUpdateResponse, star_v1_BeginUpdateResponse, 2)
00013
00014
00015 PB_BIND(star_v1_WriteChunkRequest, star_v1_WriteChunkRequest, 2)
00016
00017
00018 PB_BIND(star_v1_WriteChunkResponse, star_v1_WriteChunkResponse, 2)
00019
00020
00021 PB_BIND(star_v1_FirmwareChunk, star_v1_FirmwareChunk, 2)
00022
00023
00024 PB_BIND(star_v1_StreamChunksResponse, star_v1_StreamChunksResponse, 2)
00025
00026
00027 PB_BIND(star_v1_FinalizeUpdateRequest, star_v1_FinalizeUpdateRequest, AUTO)
00028
00029
00030 PB_BIND(star_v1_FinalizeUpdateResponse, star_v1_FinalizeUpdateResponse, 2)
00031
00032
00033 PB_BIND(star_v1_AbortUpdateRequest, star_v1_AbortUpdateRequest, 2)
00034
00035
00036 PB_BIND(star_v1_AbortUpdateResponse, star_v1_AbortUpdateResponse, 2)
00037
00038
00039 PB_BIND(star_v1_GetUpdateProgressRequest, star_v1_GetUpdateProgressRequest, AUTO)
00040
00041
00042 PB_BIND(star_v1_GetUpdateProgressResponse, star_v1_GetUpdateProgressResponse, 2)
00043
00044
00045 PB_BIND(star_v1_StreamUpdateProgressRequest, star_v1_StreamUpdateProgressRequest, AUTO)
00046
00047
00048 PB_BIND(star_v1_FirmwareUpdateProgress, star_v1_FirmwareUpdateProgress, AUTO)
00049

```

```

00050
00051 PB_BIND(star_v1_RebootRequest, star_v1_RebootRequest, AUTO)
00052
00053
00054 PB_BIND(star_v1_RebootResponse, star_v1_RebootResponse, 2)
00055
00056
00057 PB_BIND(star_v1_RollbackRequest, star_v1_RollbackRequest, AUTO)
00058
00059
00060 PB_BIND(star_v1_RollbackResponse, star_v1_RollbackResponse, 2)
00061
00062
00063 PB_BIND(star_v1_MarkValidRequest, star_v1_MarkValidRequest, AUTO)
00064
00065
00066 PB_BIND(star_v1_MarkValidResponse, star_v1_MarkValidResponse, 2)
00067
00068
00069 PB_BIND(star_v1_GetFirmwareInfoRequest, star_v1_GetFirmwareInfoRequest, AUTO)
00070
00071
00072 PB_BIND(star_v1_GetFirmwareInfoResponse, star_v1_GetFirmwareInfoResponse, 2)
00073
00074
00075 PB_BIND(star_v1_FirmwareInfo, star_v1_FirmwareInfo, AUTO)
00076
00077
00078 PB_BIND(star_v1_FirmwareUpdateError, star_v1_FirmwareUpdateError, 2)
00079
00080
00081
00082
00083
00084
00085

```

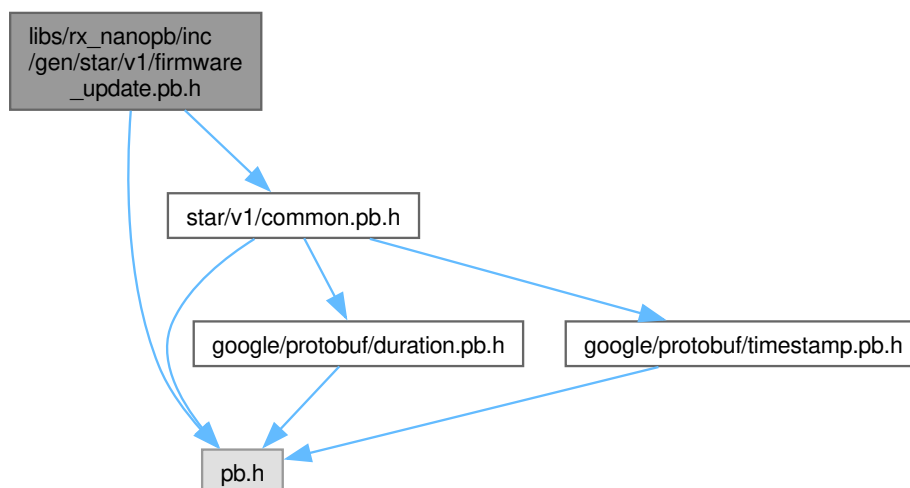
## 6.27 libs/rx\_nanopb/inc/gen/star/v1/firmware\_update.pb.h File Reference

```

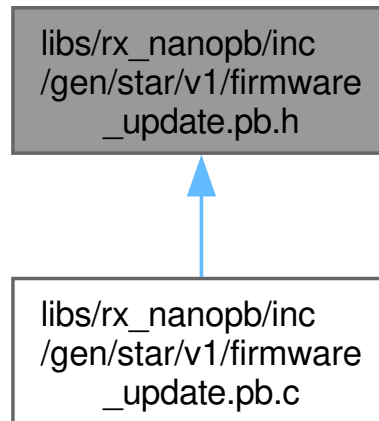
#include <pb.h>
#include "star/v1/common.pb.h"

```

Include dependency graph for firmware\_update.pb.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [star\\_v1\\_BeginUpdateRequest](#)
- struct [star\\_v1\\_FirmwareChunk](#)
- struct [star\\_v1\\_WriteChunkRequest](#)
- struct [star\\_v1\\_FinalizeUpdateRequest](#)
- struct [star\\_v1\\_AbortUpdateRequest](#)
- struct [star\\_v1\\_AbortUpdateResponse](#)
- struct [star\\_v1\\_GetUpdateProgressRequest](#)
- struct [star\\_v1\\_StreamUpdateProgressRequest](#)
- struct [star\\_v1\\_FirmwareUpdateProgress](#)
- struct [star\\_v1\\_WriteChunkResponse](#)
- struct [star\\_v1\\_GetUpdateProgressResponse](#)
- struct [star\\_v1\\_RebootRequest](#)
- struct [star\\_v1\\_RebootResponse](#)
- struct [star\\_v1\\_RollbackRequest](#)
- struct [star\\_v1\\_RollbackResponse](#)
- struct [star\\_v1\\_MarkValidRequest](#)
- struct [star\\_v1\\_MarkValidResponse](#)
- struct [star\\_v1\\_GetFirmwareInfoRequest](#)
- struct [star\\_v1\\_FirmwareInfo](#)
- struct [star\\_v1\\_GetFirmwareInfoResponse](#)
- struct [star\\_v1\\_FirmwareUpdateError](#)
- struct [star\\_v1\\_BeginUpdateResponse](#)
- struct [star\\_v1\\_StreamChunksResponse](#)
- struct [star\\_v1\\_FinalizeUpdateResponse](#)

## Macros

- `#define star_v1_FirmwareUpdateState_MIN star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_MIN`
- `#define star_v1_FirmwareUpdateState_MAX star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_MAX`
- `#define star_v1_FirmwareUpdateState_ARRAYSIZE ((star_v1_FirmwareUpdateState)(star_v1_FirmwareUpdateState_MAX - star_v1_FirmwareUpdateState_MIN + 1))`
- `#define star_v1_FirmwareUpdateErrorCode_MIN star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_MIN`
- `#define star_v1_FirmwareUpdateErrorCode_MAX star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_MAX`
- `#define star_v1_FirmwareUpdateErrorCode_ARRAYSIZE ((star_v1_FirmwareUpdateErrorCode)(star_v1_FirmwareUpdateErrorCode_MAX - star_v1_FirmwareUpdateErrorCode_MIN + 1))`
- `#define star_v1_FirmwareUpdateProgress_state_ENUMTYPE star_v1_FirmwareUpdateState`
- `#define star_v1_FirmwareUpdateError_code_ENUMTYPE star_v1_FirmwareUpdateErrorCode`
- `#define star_v1_BeginUpdateRequest_init_default {false, star_v1_RequestHeader_init_default, 0, "", 0}`

- `#define star_v1_BeginUpdateResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0, "", false, star_v1_FirmwareUpdateError_init_default}`
- `#define star_v1_WriteChunkRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_FirmwareChunk_init_default}`
- `#define star_v1_WriteChunkResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, false, star_v1_FirmwareUpdateProgress_init_default}`
- `#define star_v1_FirmwareChunk_init_default {{0, {0}}, 0, 0, 0}`
- `#define star_v1_StreamChunksResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, false, star_v1_FirmwareUpdateProgress_init_default, false, star_v1_FirmwareUpdateError_init_default}`
- `#define star_v1_FinalizeUpdateRequest_init_default {false, star_v1_RequestHeader_init_default}`
- `#define star_v1_FinalizeUpdateResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0, false, star_v1_FirmwareUpdateError_init_default}`
- `#define star_v1_AbortUpdateRequest_init_default {false, star_v1_RequestHeader_init_default, ""}`
- `#define star_v1_AbortUpdateResponse_init_default {false, star_v1_ResponseHeader_init_default, 0}`
- `#define star_v1_GetUpdateProgressRequest_init_default {false, star_v1_RequestHeader_init_default}`
- `#define star_v1_GetUpdateProgressResponse_init_default {false, star_v1_ResponseHeader_init_default, false, star_v1_FirmwareUpdateProgress_init_default}`
- `#define star_v1_StreamUpdateProgressRequest_init_default {false, star_v1_RequestHeader_init_default, 0}`
- `#define star_v1_FirmwareUpdateProgress_init_default {0, 0, 0, 0, _star_v1_FirmwareUpdateState_MIN, 0, 0, 0}`
- `#define star_v1_RebootRequest_init_default {false, star_v1_RequestHeader_init_default, 0}`
- `#define star_v1_RebootResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0}`
- `#define star_v1_RollbackRequest_init_default {false, star_v1_RequestHeader_init_default}`
- `#define star_v1_RollbackResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, ""}`
- `#define star_v1_MarkValidRequest_init_default {false, star_v1_RequestHeader_init_default}`
- `#define star_v1_MarkValidResponse_init_default {false, star_v1_ResponseHeader_init_default, 0}`
- `#define star_v1_GetFirmwareInfoRequest_init_default {false, star_v1_RequestHeader_init_default}`
- `#define star_v1_GetFirmwareInfoResponse_init_default {false, star_v1_ResponseHeader_init_default, false, star_v1_FirmwareInfo_init_default, false, star_v1_FirmwareInfo_init_default, 0, 0}`
- `#define star_v1_FirmwareInfo_init_default {"", "", "", 0, 0, "", ""}`
- `#define star_v1_FirmwareUpdateError_init_default {star_v1_FirmwareUpdateErrorCode_MIN, "", "", 0, 0}`
- `#define star_v1_BeginUpdateRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0, "", 0}`
- `#define star_v1_BeginUpdateResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0, "", false, star_v1_FirmwareUpdateError_init_zero}`
- `#define star_v1_WriteChunkRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_FirmwareChunk_init_zero}`
- `#define star_v1_WriteChunkResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, false, star_v1_FirmwareUpdateProgress_init_zero}`
- `#define star_v1_FirmwareChunk_init_zero {{0, {0}}, 0, 0, 0}`
- `#define star_v1_StreamChunksResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, false, star_v1_FirmwareUpdateProgress_init_zero, false, star_v1_FirmwareUpdateError_init_zero}`
- `#define star_v1_FinalizeUpdateRequest_init_zero {false, star_v1_RequestHeader_init_zero}`
- `#define star_v1_FinalizeUpdateResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0, false, star_v1_FirmwareUpdateError_init_zero}`
- `#define star_v1_AbortUpdateRequest_init_zero {false, star_v1_RequestHeader_init_zero, ""}`
- `#define star_v1_AbortUpdateResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0}`
- `#define star_v1_GetUpdateProgressRequest_init_zero {false, star_v1_RequestHeader_init_zero}`



- `#define star_v1_GetUpdateProgressResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_FirmwareUpdateProgress_init_zero}`
- `#define star_v1_StreamUpdateProgressRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}`
- `#define star_v1_FirmwareUpdateProgress_init_zero {0, 0, 0, 0, _star_v1_FirmwareUpdateState_MIN, 0, 0, 0}`
- `#define star_v1_RebootRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}`
- `#define star_v1_RebootResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0}`
- `#define star_v1_RollbackRequest_init_zero {false, star_v1_RequestHeader_init_zero}`
- `#define star_v1_RollbackResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, ""}`
- `#define star_v1_MarkValidRequest_init_zero {false, star_v1_RequestHeader_init_zero}`
- `#define star_v1_MarkValidResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0}`
- `#define star_v1_GetFirmwareInfoRequest_init_zero {false, star_v1_RequestHeader_init_zero}`
- `#define star_v1_GetFirmwareInfoResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_FirmwareInfo_init_zero, false, star_v1_FirmwareInfo_init_zero, 0, 0}`
- `#define star_v1_FirmwareInfo_init_zero { "", "", "", 0, 0, "", "" }`
- `#define star_v1_FirmwareUpdateError_init_zero { _star_v1_FirmwareUpdateErrorCode_MIN, "", "", 0, 0 }`
- `#define star_v1_BeginUpdateRequest_header_tag 1`
- `#define star_v1_BeginUpdateRequest_firmware_size_tag 2`
- `#define star_v1_BeginUpdateRequest_expected_version_tag 3`
- `#define star_v1_BeginUpdateRequest_expected_crc32_tag 4`
- `#define star_v1_FirmwareChunk_data_tag 1`
- `#define star_v1_FirmwareChunk_sequence_tag 2`
- `#define star_v1_FirmwareChunk_offset_tag 3`
- `#define star_v1_FirmwareChunk_chunk_crc32_tag 4`
- `#define star_v1_WriteChunkRequest_header_tag 1`
- `#define star_v1_WriteChunkRequest_chunk_tag 2`
- `#define star_v1_FinalizeUpdateRequest_header_tag 1`
- `#define star_v1_AbortUpdateRequest_header_tag 1`
- `#define star_v1_AbortUpdateRequest_reason_tag 2`
- `#define star_v1_AbortUpdateResponse_header_tag 1`
- `#define star_v1_AbortUpdateResponse_aborted_tag 2`
- `#define star_v1_GetUpdateProgressRequest_header_tag 1`
- `#define star_v1_StreamUpdateProgressRequest_header_tag 1`
- `#define star_v1_StreamUpdateProgressRequest_update_interval_ms_tag 2`
- `#define star_v1_FirmwareUpdateProgress_total_size_tag 1`
- `#define star_v1_FirmwareUpdateProgress_bytes_received_tag 2`
- `#define star_v1_FirmwareUpdateProgress_progress_percent_tag 3`
- `#define star_v1_FirmwareUpdateProgress_running_crc32_tag 4`
- `#define star_v1_FirmwareUpdateProgress_state_tag 5`
- `#define star_v1_FirmwareUpdateProgress_estimated_seconds_remaining_tag 6`
- `#define star_v1_FirmwareUpdateProgress_transfer_speed_bps_tag 7`
- `#define star_v1_FirmwareUpdateProgress_last_sequence_tag 8`
- `#define star_v1_WriteChunkResponse_header_tag 1`
- `#define star_v1_WriteChunkResponse_success_tag 2`
- `#define star_v1_WriteChunkResponse_progress_tag 3`
- `#define star_v1_GetUpdateProgressResponse_header_tag 1`
- `#define star_v1_GetUpdateProgressResponse_progress_tag 2`
- `#define star_v1_RebootRequest_header_tag 1`
- `#define star_v1_RebootRequest_delay_ms_tag 2`
- `#define star_v1_RebootResponse_header_tag 1`
- `#define star_v1_RebootResponse_rebooting_tag 2`
- `#define star_v1_RebootResponse_reboot_delay_ms_tag 3`
- `#define star_v1_RollbackRequest_header_tag 1`

- `#define star_v1_RollbackResponse_header_tag 1`
- `#define star_v1_RollbackResponse_rolling_back_tag 2`
- `#define star_v1_RollbackResponse_previous_version_tag 3`
- `#define star_v1_MarkValidRequest_header_tag 1`
- `#define star_v1_MarkValidResponse_header_tag 1`
- `#define star_v1_MarkValidResponse_marked_valid_tag 2`
- `#define star_v1_GetFirmwareInfoRequest_header_tag 1`
- `#define star_v1_FirmwareInfo_version_tag 1`
- `#define star_v1_FirmwareInfo_git_hash_tag 2`
- `#define star_v1_FirmwareInfo_build_date_tag 3`
- `#define star_v1_FirmwareInfo_size_tag 4`
- `#define star_v1_FirmwareInfo_crc32_tag 5`
- `#define star_v1_FirmwareInfo_idf_version_tag 6`
- `#define star_v1_FirmwareInfo_chip_target_tag 7`
- `#define star_v1_GetFirmwareInfoResponse_header_tag 1`
- `#define star_v1_GetFirmwareInfoResponse_current_firmware_tag 2`
- `#define star_v1_GetFirmwareInfoResponse_previous_firmware_tag 3`
- `#define star_v1_GetFirmwareInfoResponse_is_ota_boot_tag 4`
- `#define star_v1_GetFirmwareInfoResponse_is_valid_tag 5`
- `#define star_v1_FirmwareUpdateError_code_tag 1`
- `#define star_v1_FirmwareUpdateError_message_tag 2`
- `#define star_v1_FirmwareUpdateError_details_tag 3`
- `#define star_v1_FirmwareUpdateError_error_at_sequence_tag 4`
- `#define star_v1_FirmwareUpdateError_error_at_offset_tag 5`
- `#define star_v1_BeginUpdateResponse_header_tag 1`
- `#define star_v1_BeginUpdateResponse_accepted_tag 2`
- `#define star_v1_BeginUpdateResponse_max_chunk_size_tag 3`
- `#define star_v1_BeginUpdateResponse_session_id_tag 4`
- `#define star_v1_BeginUpdateResponse_error_tag 5`
- `#define star_v1_StreamChunksResponse_header_tag 1`
- `#define star_v1_StreamChunksResponse_chunks_received_tag 2`
- `#define star_v1_StreamChunksResponse_progress_tag 3`
- `#define star_v1_StreamChunksResponse_error_tag 4`
- `#define star_v1_FinalizeUpdateResponse_header_tag 1`
- `#define star_v1_FinalizeUpdateResponse_validated_tag 2`
- `#define star_v1_FinalizeUpdateResponse_ready_to_reboot_tag 3`
- `#define star_v1_FinalizeUpdateResponse_error_tag 4`
- `#define star_v1_BeginUpdateRequest_FIELDLIST(X, a)`
- `#define star_v1_BeginUpdateRequest_CALLBACK NULL`
- `#define star_v1_BeginUpdateRequest_DEFAULT NULL`
- `#define star_v1_BeginUpdateRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_BeginUpdateResponse_FIELDLIST(X, a)`
- `#define star_v1_BeginUpdateResponse_CALLBACK NULL`
- `#define star_v1_BeginUpdateResponse_DEFAULT NULL`
- `#define star_v1_BeginUpdateResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_BeginUpdateResponse_error_MSGTYPE star_v1_FirmwareUpdateError`
- `#define star_v1_WriteChunkRequest_FIELDLIST(X, a)`
- `#define star_v1_WriteChunkRequest_CALLBACK NULL`
- `#define star_v1_WriteChunkRequest_DEFAULT NULL`
- `#define star_v1_WriteChunkRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_WriteChunkRequest_chunk_MSGTYPE star_v1_FirmwareChunk`
- `#define star_v1_WriteChunkResponse_FIELDLIST(X, a)`
- `#define star_v1_WriteChunkResponse_CALLBACK NULL`
- `#define star_v1_WriteChunkResponse_DEFAULT NULL`
- `#define star_v1_WriteChunkResponse_header_MSGTYPE star_v1_ResponseHeader`

- `#define star_v1_WriteChunkResponse_progress_MSGTYPE star_v1_FirmwareUpdateProgress`
- `#define star_v1_FirmwareChunk_FIELDLIST(X, a)`
- `#define star_v1_FirmwareChunk_CALLBACK NULL`
- `#define star_v1_FirmwareChunk_DEFAULT NULL`
- `#define star_v1_StreamChunksResponse_FIELDLIST(X, a)`
- `#define star_v1_StreamChunksResponse_CALLBACK NULL`
- `#define star_v1_StreamChunksResponse_DEFAULT NULL`
- `#define star_v1_StreamChunksResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_StreamChunksResponse_progress_MSGTYPE star_v1_FirmwareUpdateProgress`
- `#define star_v1_StreamChunksResponse_error_MSGTYPE star_v1_FirmwareUpdateError`
- `#define star_v1_FinalizeUpdateRequest_FIELDLIST(X, a)`
- `#define star_v1_FinalizeUpdateRequest_CALLBACK NULL`
- `#define star_v1_FinalizeUpdateRequest_DEFAULT NULL`
- `#define star_v1_FinalizeUpdateRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_FinalizeUpdateResponse_FIELDLIST(X, a)`
- `#define star_v1_FinalizeUpdateResponse_CALLBACK NULL`
- `#define star_v1_FinalizeUpdateResponse_DEFAULT NULL`
- `#define star_v1_FinalizeUpdateResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_FinalizeUpdateResponse_error_MSGTYPE star_v1_FirmwareUpdateError`
- `#define star_v1_AbortUpdateRequest_FIELDLIST(X, a)`
- `#define star_v1_AbortUpdateRequest_CALLBACK NULL`
- `#define star_v1_AbortUpdateRequest_DEFAULT NULL`
- `#define star_v1_AbortUpdateRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_AbortUpdateResponse_FIELDLIST(X, a)`
- `#define star_v1_AbortUpdateResponse_CALLBACK NULL`
- `#define star_v1_AbortUpdateResponse_DEFAULT NULL`
- `#define star_v1_AbortUpdateResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_GetUpdateProgressRequest_FIELDLIST(X, a)`
- `#define star_v1_GetUpdateProgressRequest_CALLBACK NULL`
- `#define star_v1_GetUpdateProgressRequest_DEFAULT NULL`
- `#define star_v1_GetUpdateProgressRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_GetUpdateProgressResponse_FIELDLIST(X, a)`
- `#define star_v1_GetUpdateProgressResponse_CALLBACK NULL`
- `#define star_v1_GetUpdateProgressResponse_DEFAULT NULL`
- `#define star_v1_GetUpdateProgressResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_GetUpdateProgressResponse_progress_MSGTYPE star_v1_FirmwareUpdateProgress`
- `#define star_v1_StreamUpdateProgressRequest_FIELDLIST(X, a)`
- `#define star_v1_StreamUpdateProgressRequest_CALLBACK NULL`
- `#define star_v1_StreamUpdateProgressRequest_DEFAULT NULL`
- `#define star_v1_StreamUpdateProgressRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_FirmwareUpdateProgress_FIELDLIST(X, a)`
- `#define star_v1_FirmwareUpdateProgress_CALLBACK NULL`
- `#define star_v1_FirmwareUpdateProgress_DEFAULT NULL`
- `#define star_v1_RebootRequest_FIELDLIST(X, a)`
- `#define star_v1_RebootRequest_CALLBACK NULL`
- `#define star_v1_RebootRequest_DEFAULT NULL`
- `#define star_v1_RebootRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_RebootResponse_FIELDLIST(X, a)`
- `#define star_v1_RebootResponse_CALLBACK NULL`
- `#define star_v1_RebootResponse_DEFAULT NULL`
- `#define star_v1_RebootResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_RollbackRequest_FIELDLIST(X, a)`
- `#define star_v1_RollbackRequest_CALLBACK NULL`
- `#define star_v1_RollbackRequest_DEFAULT NULL`
- `#define star_v1_RollbackRequest_header_MSGTYPE star_v1_RequestHeader`

- `#define star_v1_RollbackResponse_FIELDLIST(X, a)`
- `#define star_v1_RollbackResponse_CALLBACK NULL`
- `#define star_v1_RollbackResponse_DEFAULT NULL`
- `#define star_v1_RollbackResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_MarkValidRequest_FIELDLIST(X, a)`
- `#define star_v1_MarkValidRequest_CALLBACK NULL`
- `#define star_v1_MarkValidRequest_DEFAULT NULL`
- `#define star_v1_MarkValidRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_MarkValidResponse_FIELDLIST(X, a)`
- `#define star_v1_MarkValidResponse_CALLBACK NULL`
- `#define star_v1_MarkValidResponse_DEFAULT NULL`
- `#define star_v1_MarkValidResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_GetFirmwareInfoRequest_FIELDLIST(X, a)`
- `#define star_v1_GetFirmwareInfoRequest_CALLBACK NULL`
- `#define star_v1_GetFirmwareInfoRequest_DEFAULT NULL`
- `#define star_v1_GetFirmwareInfoRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_GetFirmwareInfoResponse_FIELDLIST(X, a)`
- `#define star_v1_GetFirmwareInfoResponse_CALLBACK NULL`
- `#define star_v1_GetFirmwareInfoResponse_DEFAULT NULL`
- `#define star_v1_GetFirmwareInfoResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_GetFirmwareInfoResponse_current_firmware_MSGTYPE star_v1_FirmwareInfo`
- `#define star_v1_GetFirmwareInfoResponse_previous_firmware_MSGTYPE star_v1_FirmwareInfo`
- `#define star_v1_FirmwareInfo_FIELDLIST(X, a)`
- `#define star_v1_FirmwareInfo_CALLBACK NULL`
- `#define star_v1_FirmwareInfo_DEFAULT NULL`
- `#define star_v1_FirmwareUpdateError_FIELDLIST(X, a)`
- `#define star_v1_FirmwareUpdateError_CALLBACK NULL`
- `#define star_v1_FirmwareUpdateError_DEFAULT NULL`
- `#define star_v1_BeginUpdateRequest_fields &star_v1_BeginUpdateRequest_msg`
- `#define star_v1_BeginUpdateResponse_fields &star_v1_BeginUpdateResponse_msg`
- `#define star_v1_WriteChunkRequest_fields &star_v1_WriteChunkRequest_msg`
- `#define star_v1_WriteChunkResponse_fields &star_v1_WriteChunkResponse_msg`
- `#define star_v1_FirmwareChunk_fields &star_v1_FirmwareChunk_msg`
- `#define star_v1_StreamChunksResponse_fields &star_v1_StreamChunksResponse_msg`
- `#define star_v1_FinalizeUpdateRequest_fields &star_v1_FinalizeUpdateRequest_msg`
- `#define star_v1_FinalizeUpdateResponse_fields &star_v1_FinalizeUpdateResponse_msg`
- `#define star_v1_AbortUpdateRequest_fields &star_v1_AbortUpdateRequest_msg`
- `#define star_v1_AbortUpdateResponse_fields &star_v1_AbortUpdateResponse_msg`
- `#define star_v1_GetUpdateProgressRequest_fields &star_v1_GetUpdateProgressRequest_msg`
- `#define star_v1_GetUpdateProgressResponse_fields &star_v1_GetUpdateProgressResponse_msg`
- `#define star_v1_StreamUpdateProgressRequest_fields &star_v1_StreamUpdateProgressRequest_msg`
- `#define star_v1_FirmwareUpdateProgress_fields &star_v1_FirmwareUpdateProgress_msg`
- `#define star_v1_RebootRequest_fields &star_v1_RebootRequest_msg`
- `#define star_v1_RebootResponse_fields &star_v1_RebootResponse_msg`
- `#define star_v1_RollbackRequest_fields &star_v1_RollbackRequest_msg`
- `#define star_v1_RollbackResponse_fields &star_v1_RollbackResponse_msg`
- `#define star_v1_MarkValidRequest_fields &star_v1_MarkValidRequest_msg`
- `#define star_v1_MarkValidResponse_fields &star_v1_MarkValidResponse_msg`
- `#define star_v1_GetFirmwareInfoRequest_fields &star_v1_GetFirmwareInfoRequest_msg`
- `#define star_v1_GetFirmwareInfoResponse_fields &star_v1_GetFirmwareInfoResponse_msg`
- `#define star_v1_FirmwareInfo_fields &star_v1_FirmwareInfo_msg`
- `#define star_v1_FirmwareUpdateError_fields &star_v1_FirmwareUpdateError_msg`
- `#define STAR_V1_STAR_V1_FIRMWARE_UPDATE_PB_H_MAX_SIZE star_v1_WriteChunkRequest_size`
- `#define star_v1_AbortUpdateRequest_size 279`
- `#define star_v1_AbortUpdateResponse_size 365`

- `#define star_v1_BeginUpdateRequest_size` 194
- `#define star_v1_BeginUpdateResponse_size` 713
- `#define star_v1_FinalizeUpdateRequest_size` 149
- `#define star_v1_FinalizeUpdateResponse_size` 644
- `#define star_v1_FirmwareChunk_size` 1045
- `#define star_v1_FirmwareInfo_size` 145
- `#define star_v1_FirmwareUpdateError_size` 274
- `#define star_v1_FirmwareUpdateProgress_size` 54
- `#define star_v1_GetFirmwareInfoRequest_size` 149
- `#define star_v1_GetFirmwareInfoResponse_size` 663
- `#define star_v1_GetUpdateProgressRequest_size` 149
- `#define star_v1_GetUpdateProgressResponse_size` 419
- `#define star_v1_MarkValidRequest_size` 149
- `#define star_v1_MarkValidResponse_size` 365
- `#define star_v1_RebootRequest_size` 155
- `#define star_v1_RebootResponse_size` 371
- `#define star_v1_RollbackRequest_size` 149
- `#define star_v1_RollbackResponse_size` 398
- `#define star_v1_StreamChunksResponse_size` 702
- `#define star_v1_StreamUpdateProgressRequest_size` 155
- `#define star_v1_WriteChunkRequest_size` 1197
- `#define star_v1_WriteChunkResponse_size` 421

## Enumerations

- `enum star_v1_FirmwareUpdateState {`  
`star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_UNKNOWN = 0 , star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_RECEIVING = 2 ,`  
`star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_VALIDATING = 3 ,`  
`star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_COMPLETE = 4 , star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_ERROR = 5 }`
- `enum star_v1_FirmwareUpdateErrorCode {`  
`star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_UNKNOWN = 0 , star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_INVALID_SIZE = 1 ,`  
`star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_ALREADY_IN_PROGRESS = 2 , star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NO_PARTITION = 3 ,`  
`star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NOT_STARTED = 4 , star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_SIZE_EXCEEDED = 5 ,`  
`star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_INCOMPLETE = 6 , star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_CRC_FAILED = 7 ,`  
`star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_VALIDATION_FAILED = 8 , star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_WRITE_FAILED = 9 ,`  
`star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_SEQUENCE_ERROR = 10 , star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_TIMEOUT = 11 ,`  
`star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NO_SPACE = 12 }`

## Functions

- `typedef PB_BYTES_ARRAY_T (1024) star_v1_FirmwareChunk_data_t`

## Variables

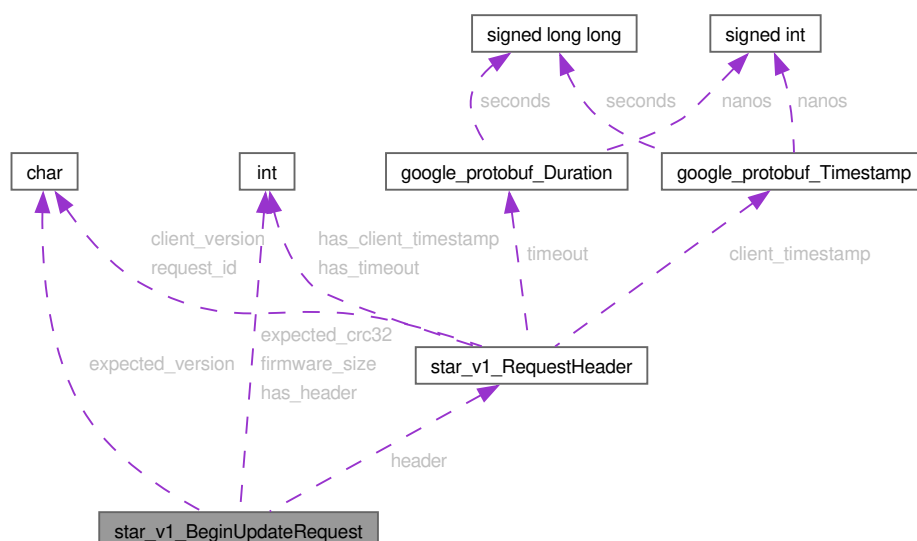
- `const pb_msgdesc_t star_v1_BeginUpdateRequest_msg`
- `const pb_msgdesc_t star_v1_BeginUpdateResponse_msg`
- `const pb_msgdesc_t star_v1_WriteChunkRequest_msg`
- `const pb_msgdesc_t star_v1_WriteChunkResponse_msg`
- `const pb_msgdesc_t star_v1_FirmwareChunk_msg`
- `const pb_msgdesc_t star_v1_StreamChunksResponse_msg`
- `const pb_msgdesc_t star_v1_FinalizeUpdateRequest_msg`
- `const pb_msgdesc_t star_v1_FinalizeUpdateResponse_msg`
- `const pb_msgdesc_t star_v1_AbortUpdateRequest_msg`
- `const pb_msgdesc_t star_v1_AbortUpdateResponse_msg`
- `const pb_msgdesc_t star_v1_GetUpdateProgressRequest_msg`
- `const pb_msgdesc_t star_v1_GetUpdateProgressResponse_msg`
- `const pb_msgdesc_t star_v1_StreamUpdateProgressRequest_msg`
- `const pb_msgdesc_t star_v1_FirmwareUpdateProgress_msg`
- `const pb_msgdesc_t star_v1_RebootRequest_msg`
- `const pb_msgdesc_t star_v1_RebootResponse_msg`
- `const pb_msgdesc_t star_v1_RollbackRequest_msg`
- `const pb_msgdesc_t star_v1_RollbackResponse_msg`
- `const pb_msgdesc_t star_v1_MarkValidRequest_msg`
- `const pb_msgdesc_t star_v1_MarkValidResponse_msg`
- `const pb_msgdesc_t star_v1_GetFirmwareInfoRequest_msg`
- `const pb_msgdesc_t star_v1_GetFirmwareInfoResponse_msg`
- `const pb_msgdesc_t star_v1_FirmwareInfo_msg`
- `const pb_msgdesc_t star_v1_FirmwareUpdateError_msg`

## 6.27.1 Data Structure Documentation

## 6.27.1.1 struct star\_v1\_BeginUpdateRequest

Definition at line 63 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_BeginUpdateRequest:



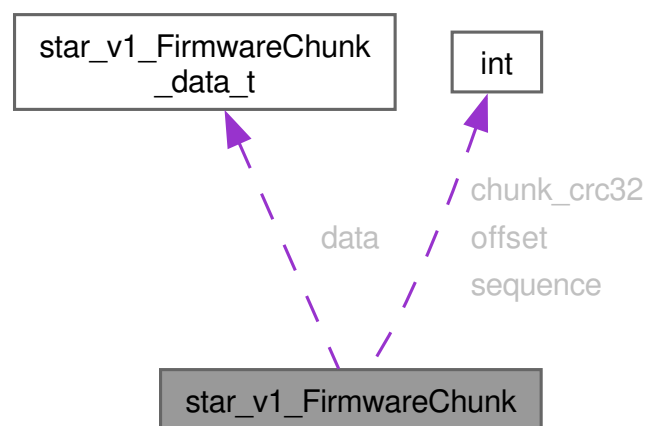
## Data Fields

|                                       |                      |  |
|---------------------------------------|----------------------|--|
| <a href="#">uint32_t</a>              | expected_crc32       |  |
| char                                  | expected_version[32] |  |
| <a href="#">uint32_t</a>              | firmware_size        |  |
| <a href="#">bool</a>                  | has_header           |  |
| <a href="#">star_v1_RequestHeader</a> | header               |  |

## 6.27.1.2 struct star\_v1\_FirmwareChunk

Definition at line 77 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_FirmwareChunk:



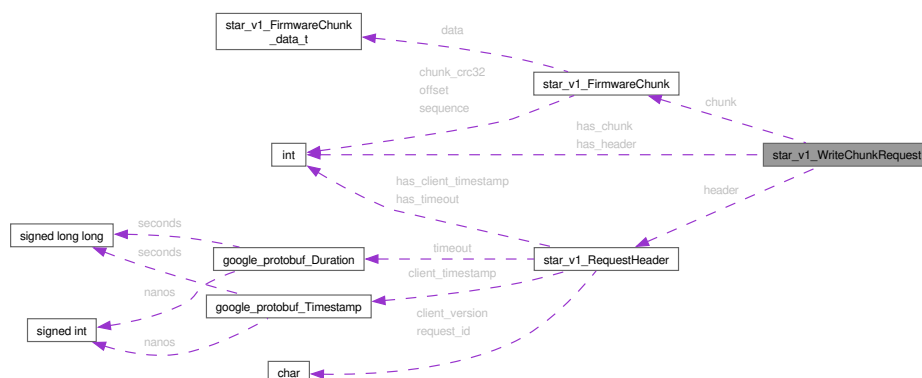
## Data Fields

|                                           |             |  |
|-------------------------------------------|-------------|--|
| <a href="#">uint32_t</a>                  | chunk_crc32 |  |
| <code>star_v1_FirmwareChunk_data_t</code> | data        |  |
| <a href="#">uint32_t</a>                  | offset      |  |
| <a href="#">uint32_t</a>                  | sequence    |  |

## 6.27.1.3 struct star\_v1\_WriteChunkRequest

Definition at line 91 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_WriteChunkRequest:





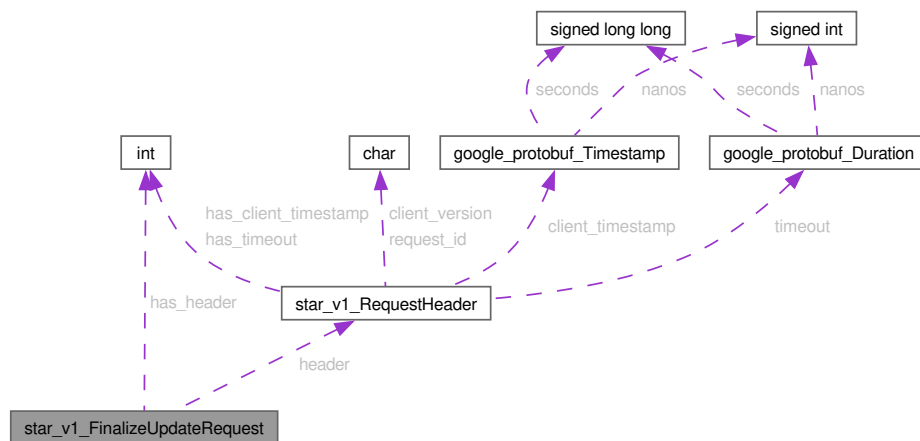
## Data Fields

|                                       |            |  |
|---------------------------------------|------------|--|
| <a href="#">star_v1_FirmwareChunk</a> | chunk      |  |
| <a href="#">bool</a>                  | has_chunk  |  |
| <a href="#">bool</a>                  | has_header |  |
| <a href="#">star_v1_RequestHeader</a> | header     |  |

## 6.27.1.4 struct star\_v1\_FinalizeUpdateRequest

Definition at line 101 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_FinalizeUpdateRequest:



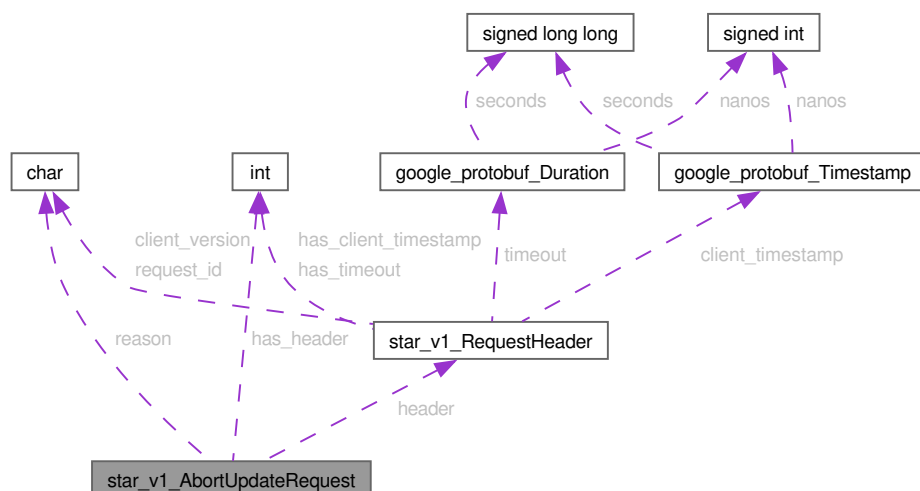
## Data Fields

|                                       |            |  |
|---------------------------------------|------------|--|
| <a href="#">bool</a>                  | has_header |  |
| <a href="#">star_v1_RequestHeader</a> | header     |  |

## 6.27.1.5 struct star\_v1\_AbortUpdateRequest

Definition at line 108 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_AbortUpdateRequest:





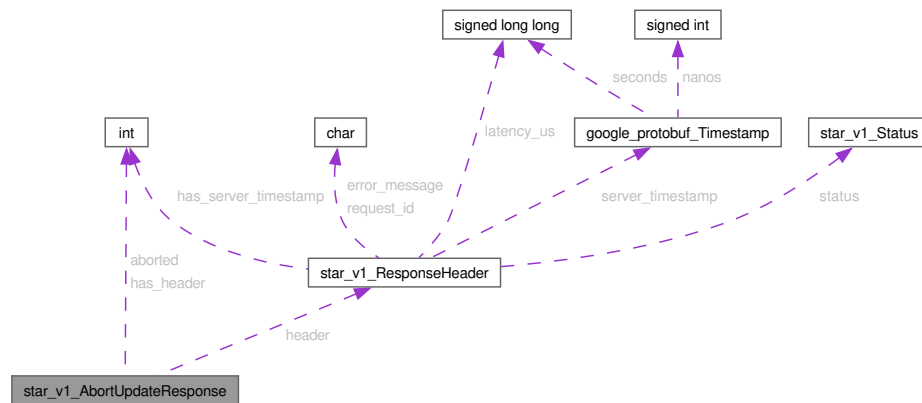
## Data Fields

|                       |             |  |
|-----------------------|-------------|--|
| bool                  | has_header  |  |
| star_v1_RequestHeader | header      |  |
| char                  | reason[128] |  |

## 6.27.1.6 struct star\_v1\_AbortUpdateResponse

Definition at line 117 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_AbortUpdateResponse:



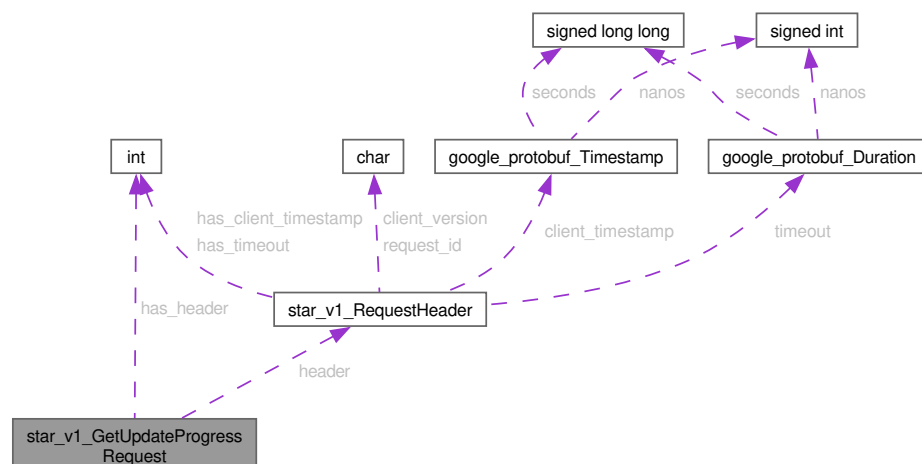
## Data Fields

|                        |            |  |
|------------------------|------------|--|
| bool                   | aborted    |  |
| bool                   | has_header |  |
| star_v1_ResponseHeader | header     |  |

## 6.27.1.7 struct star\_v1\_GetUpdateProgressRequest

Definition at line 126 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_GetUpdateProgressRequest:



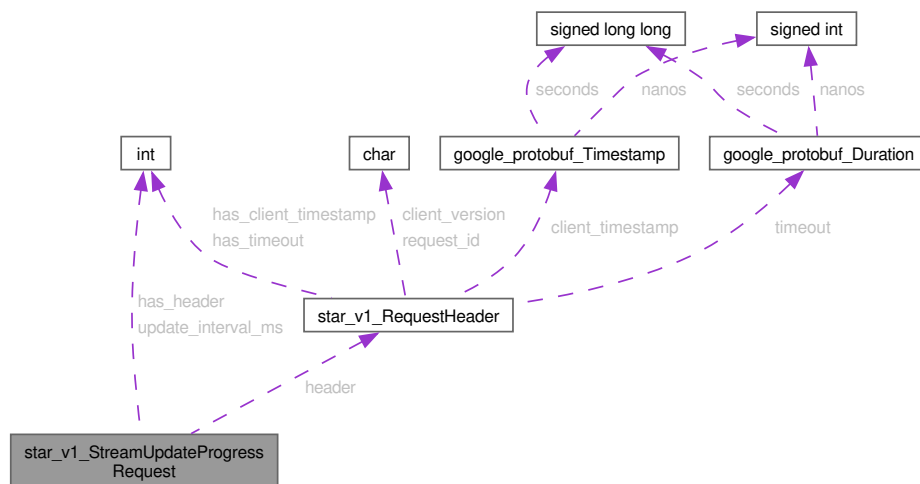
## Data Fields

|                                       |            |  |
|---------------------------------------|------------|--|
| <a href="#">bool</a>                  | has_header |  |
| <a href="#">star_v1_RequestHeader</a> | header     |  |

## 6.27.1.8 struct star\_v1\_StreamUpdateProgressRequest

Definition at line 133 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_StreamUpdateProgressRequest:



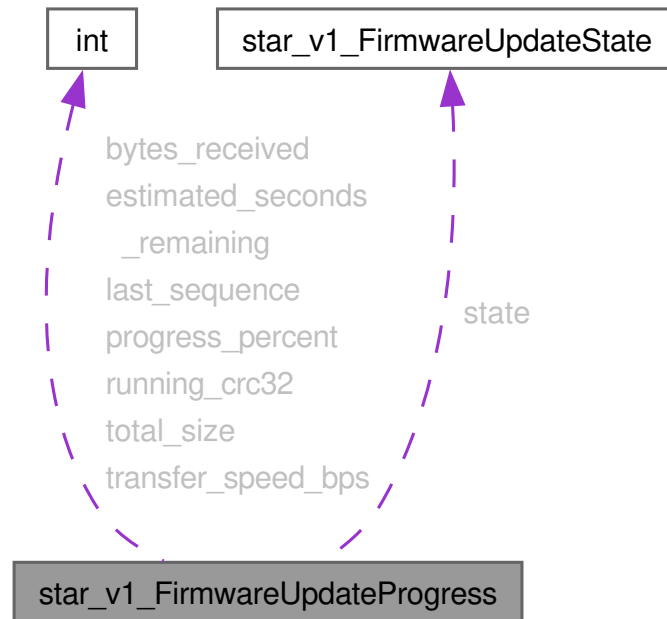
## Data Fields

|                                       |                    |  |
|---------------------------------------|--------------------|--|
| <a href="#">bool</a>                  | has_header         |  |
| <a href="#">star_v1_RequestHeader</a> | header             |  |
| <a href="#">uint32_t</a>              | update_interval_ms |  |

## 6.27.1.9 struct star\_v1\_FirmwareUpdateProgress

Definition at line 144 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_FirmwareUpdateProgress:



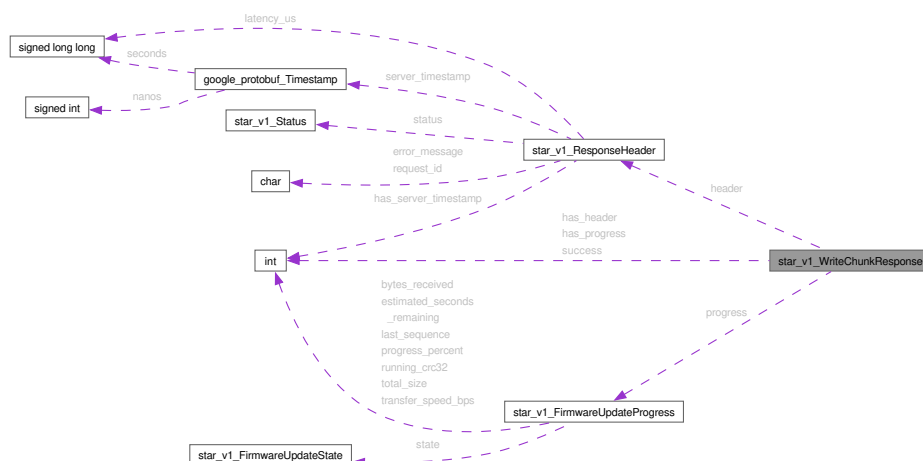
## Data Fields

|                                             |                                          |  |
|---------------------------------------------|------------------------------------------|--|
| <a href="#">uint32_t</a>                    | <code>bytes_received</code>              |  |
| <a href="#">int32_t</a>                     | <code>estimated_seconds_remaining</code> |  |
| <a href="#">uint32_t</a>                    | <code>last_sequence</code>               |  |
| <a href="#">int32_t</a>                     | <code>progress_percent</code>            |  |
| <a href="#">uint32_t</a>                    | <code>running_crc32</code>               |  |
| <a href="#">star_v1_FirmwareUpdateState</a> | <code>state</code>                       |  |
| <a href="#">uint32_t</a>                    | <code>total_size</code>                  |  |
| <a href="#">uint32_t</a>                    | <code>transfer_speed_bps</code>          |  |

6.27.1.10 struct `star_v1_WriteChunkResponse`

Definition at line 165 of file [firmware\\_update.pb.h](#).

Collaboration diagram for `star_v1_WriteChunkResponse`:



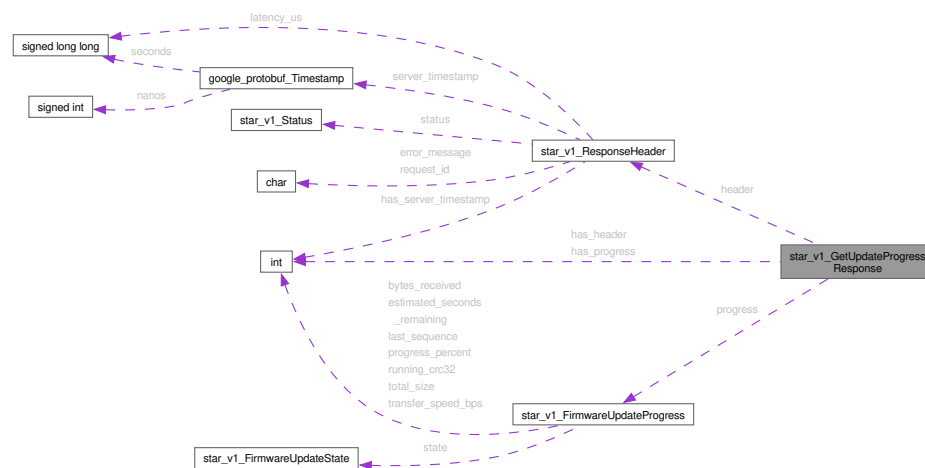
## Data Fields

|  |                                                |              |  |
|--|------------------------------------------------|--------------|--|
|  | <a href="#">bool</a>                           | has_header   |  |
|  | <a href="#">bool</a>                           | has_progress |  |
|  | <a href="#">star_v1_ResponseHeader</a>         | header       |  |
|  | <a href="#">star_v1_FirmwareUpdateProgress</a> | progress     |  |
|  | <a href="#">bool</a>                           | success      |  |

## 6.27.1.11 struct star\_v1\_GetUpdateProgressResponse

Definition at line 177 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_GetUpdateProgressResponse:



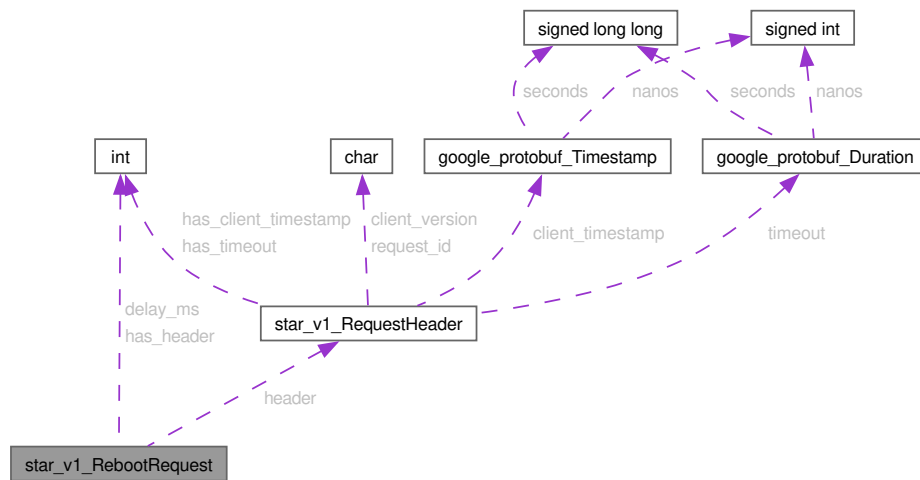
## Data Fields

|  |                                                |              |  |
|--|------------------------------------------------|--------------|--|
|  | <a href="#">bool</a>                           | has_header   |  |
|  | <a href="#">bool</a>                           | has_progress |  |
|  | <a href="#">star_v1_ResponseHeader</a>         | header       |  |
|  | <a href="#">star_v1_FirmwareUpdateProgress</a> | progress     |  |

## 6.27.1.12 struct star\_v1\_RebootRequest

Definition at line 187 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_RebootRequest:



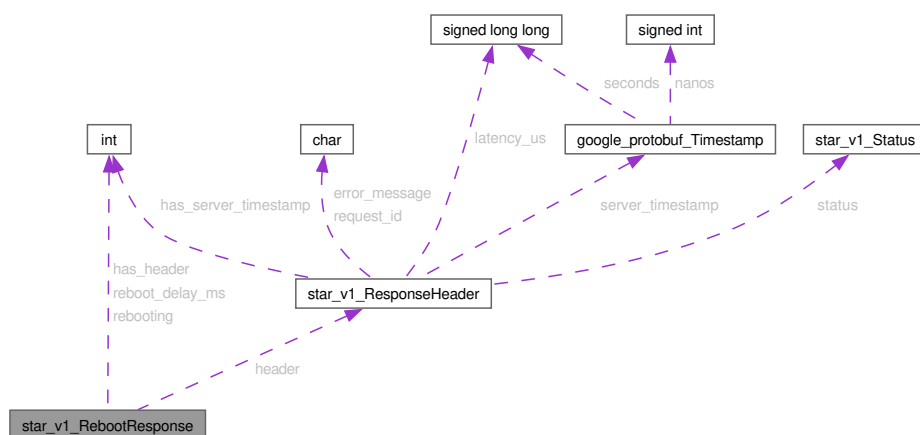
## Data Fields

|                                       |                         |  |
|---------------------------------------|-------------------------|--|
| <a href="#">uint32_t</a>              | <code>delay_ms</code>   |  |
| <a href="#">bool</a>                  | <code>has_header</code> |  |
| <a href="#">star_v1_RequestHeader</a> | <code>header</code>     |  |

6.27.1.13 struct `star_v1_RebootResponse`

Definition at line 198 of file [firmware\\_update.pb.h](#).

Collaboration diagram for `star_v1_RebootResponse`:



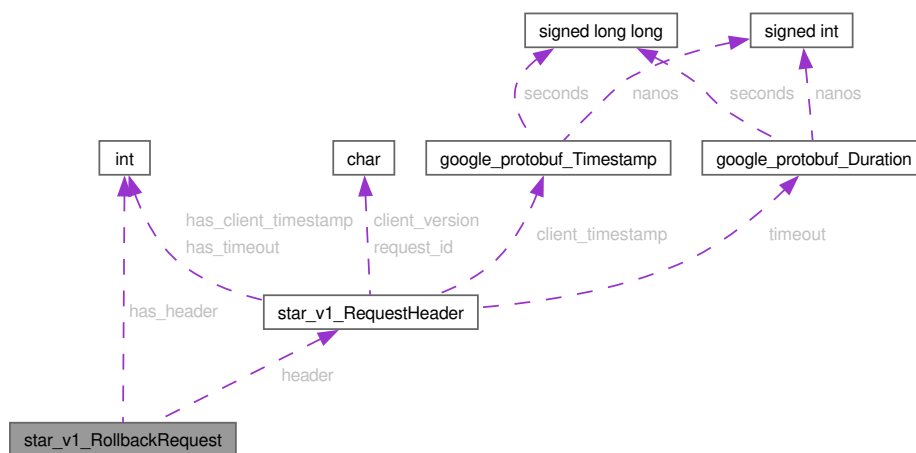
## Data Fields

|                                        |                              |  |
|----------------------------------------|------------------------------|--|
| <a href="#">bool</a>                   | <code>has_header</code>      |  |
| <a href="#">star_v1_ResponseHeader</a> | <code>header</code>          |  |
| <a href="#">uint32_t</a>               | <code>reboot_delay_ms</code> |  |
| <a href="#">bool</a>                   | <code>rebooting</code>       |  |

## 6.27.1.14 struct star\_v1\_RollbackRequest

Definition at line 209 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_RollbackRequest:



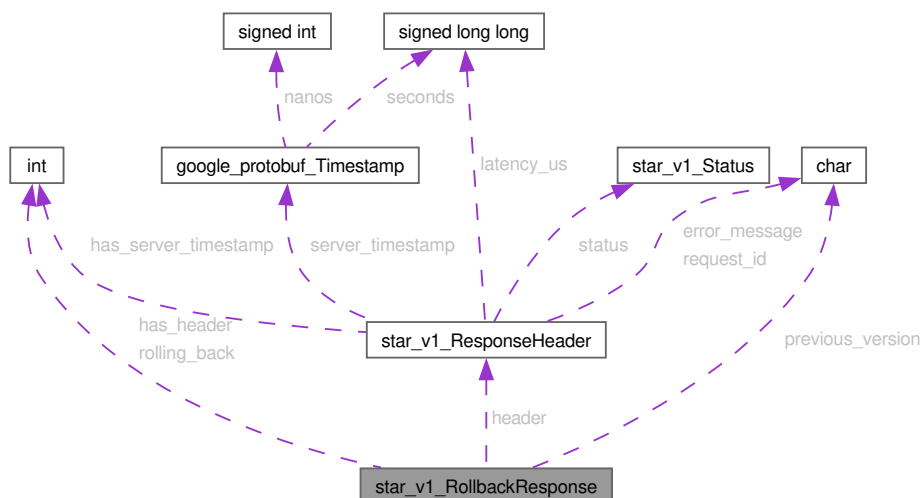
## Data Fields

|                                       |            |  |
|---------------------------------------|------------|--|
| <a href="#">bool</a>                  | has_header |  |
| <a href="#">star_v1_RequestHeader</a> | header     |  |

## 6.27.1.15 struct star\_v1\_RollbackResponse

Definition at line 216 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_RollbackResponse:



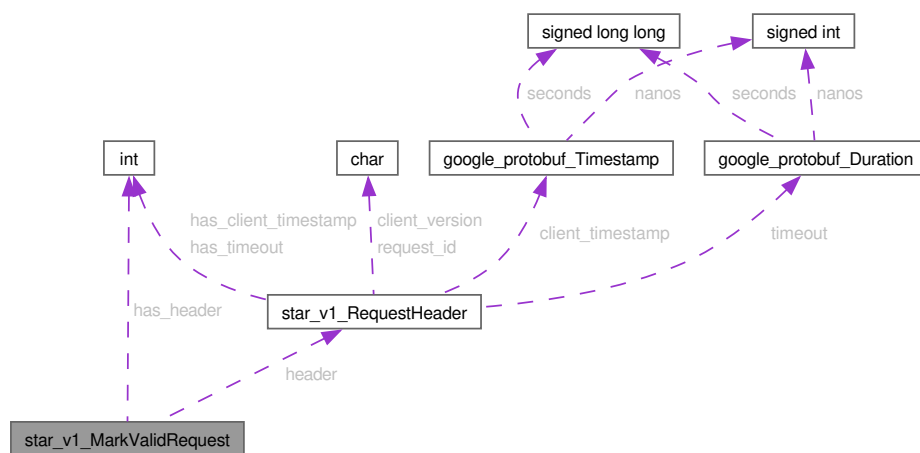
## Data Fields

|                                        |                      |  |
|----------------------------------------|----------------------|--|
| <a href="#">bool</a>                   | has_header           |  |
| <a href="#">star_v1_ResponseHeader</a> | header               |  |
| char                                   | previous_version[32] |  |
| <a href="#">bool</a>                   | rolling_back         |  |

## 6.27.1.16 struct star\_v1\_MarkValidRequest

Definition at line 227 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_MarkValidRequest:



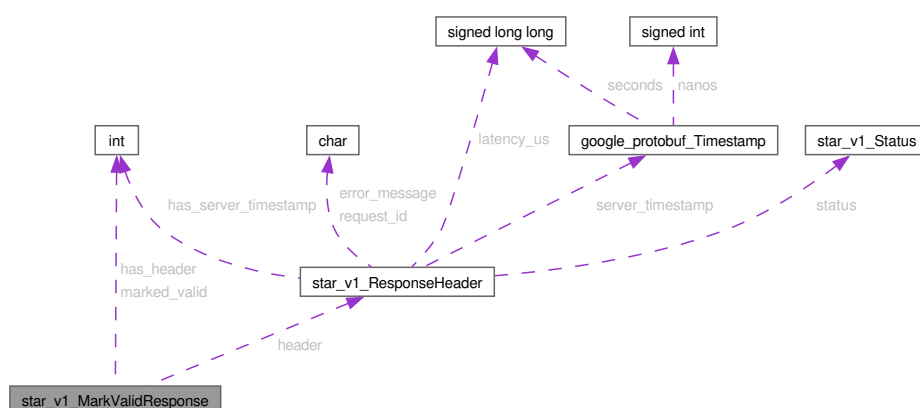
## Data Fields

|                                       |            |  |
|---------------------------------------|------------|--|
| <a href="#">bool</a>                  | has_header |  |
| <a href="#">star_v1_RequestHeader</a> | header     |  |

## 6.27.1.17 struct star\_v1\_MarkValidResponse

Definition at line 234 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_MarkValidResponse:



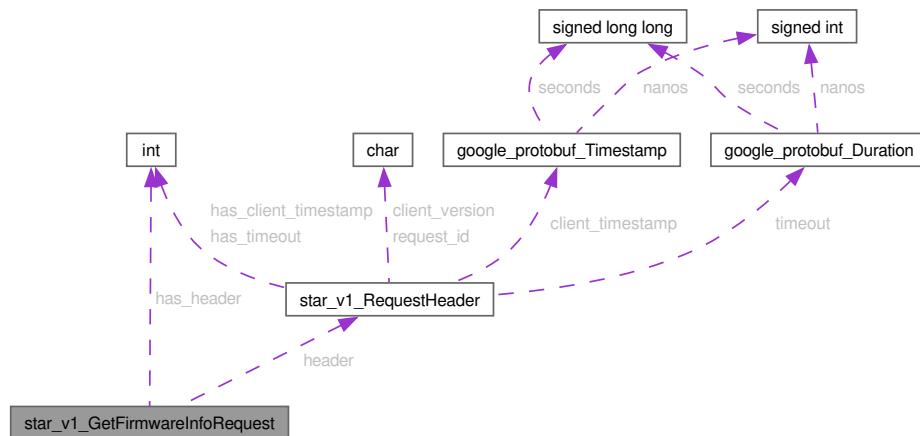
## Data Fields

|                        |      |              |  |
|------------------------|------|--------------|--|
|                        | bool | has_header   |  |
| star_v1_ResponseHeader |      | header       |  |
|                        | bool | marked_valid |  |

## 6.27.1.18 struct star\_v1\_GetFirmwareInfoRequest

Definition at line 243 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_GetFirmwareInfoRequest:



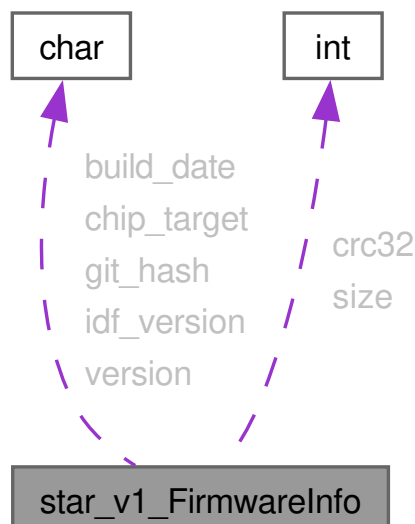
## Data Fields

|                       |      |            |  |
|-----------------------|------|------------|--|
|                       | bool | has_header |  |
| star_v1_RequestHeader |      | header     |  |

## 6.27.1.19 struct star\_v1\_FirmwareInfo

Definition at line 250 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_FirmwareInfo:





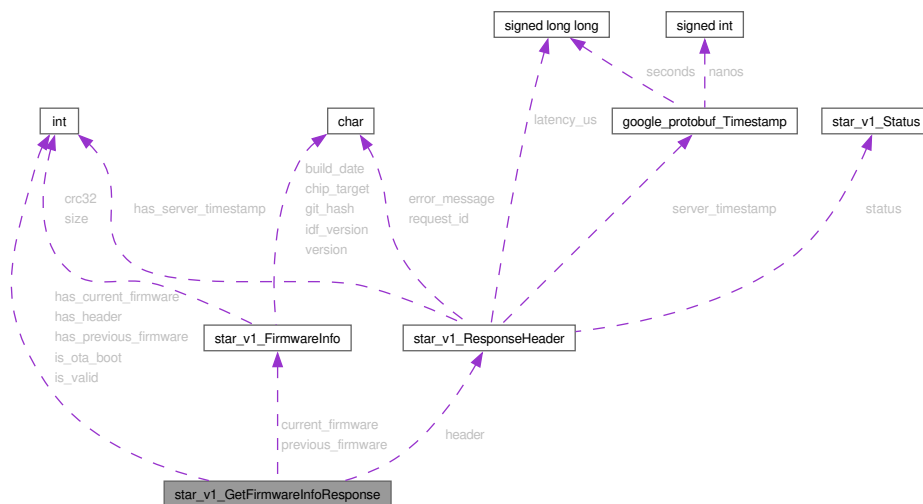
## Data Fields

|                          |                 |  |
|--------------------------|-----------------|--|
| char                     | build_date[32]  |  |
| char                     | chip_target[16] |  |
| <a href="#">uint32_t</a> | crc32           |  |
| char                     | git_hash[16]    |  |
| char                     | idf_version[32] |  |
| <a href="#">uint32_t</a> | size            |  |
| char                     | version[32]     |  |

## 6.27.1.20 struct star\_v1\_GetFirmwareInfoResponse

Definition at line 268 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_GetFirmwareInfoResponse:



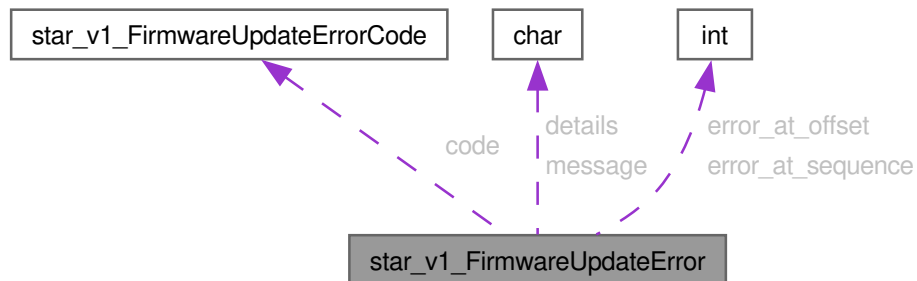
## Data Fields

|                                        |                       |  |
|----------------------------------------|-----------------------|--|
| <a href="#">star_v1_FirmwareInfo</a>   | current_firmware      |  |
| <a href="#">bool</a>                   | has_current_firmware  |  |
| <a href="#">bool</a>                   | has_header            |  |
| <a href="#">bool</a>                   | has_previous_firmware |  |
| <a href="#">star_v1_ResponseHeader</a> | header                |  |
| <a href="#">bool</a>                   | is_ota_boot           |  |
| <a href="#">bool</a>                   | is_valid              |  |
| <a href="#">star_v1_FirmwareInfo</a>   | previous_firmware     |  |

## 6.27.1.21 struct star\_v1\_FirmwareUpdateError

Definition at line 286 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_FirmwareUpdateError:



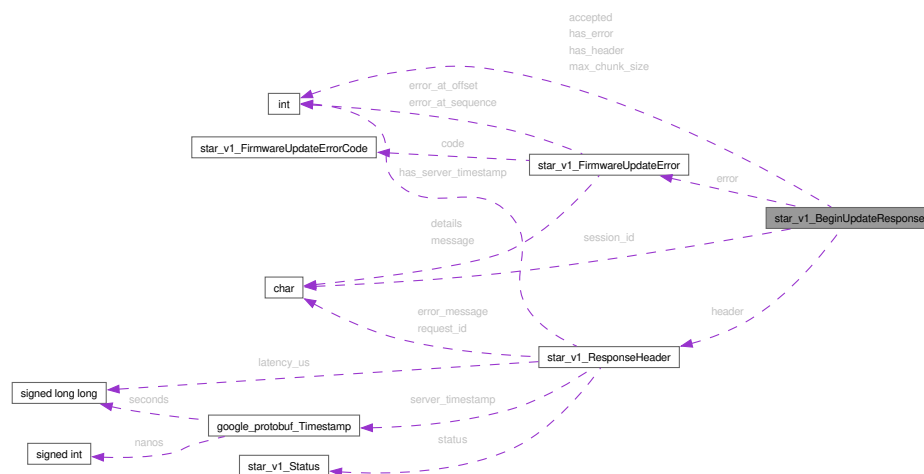
## Data Fields

|                                                 |                   |  |
|-------------------------------------------------|-------------------|--|
| <a href="#">star_v1_FirmwareUpdateErrorCode</a> | code              |  |
| char                                            | details[128]      |  |
| <a href="#">uint32_t</a>                        | error_at_offset   |  |
| <a href="#">uint32_t</a>                        | error_at_sequence |  |
| char                                            | message[128]      |  |

## 6.27.1.22 struct star\_v1\_BeginUpdateResponse

Definition at line 300 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_BeginUpdateResponse:



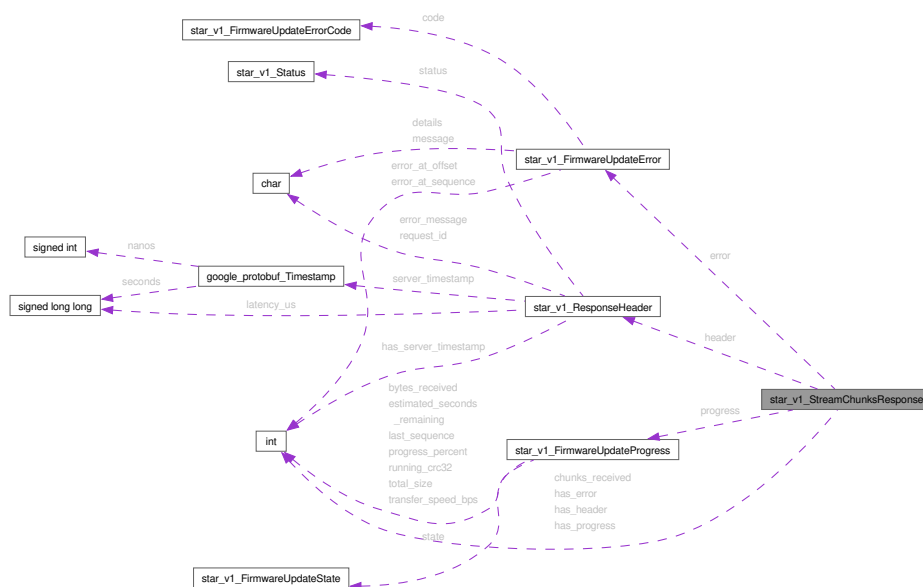
## Data Fields

|                             |                |  |
|-----------------------------|----------------|--|
| bool                        | accepted       |  |
| star_v1_FirmwareUpdateError | error          |  |
| bool                        | has_error      |  |
| bool                        | has_header     |  |
| star_v1_ResponseHeader      | header         |  |
| uint32_t                    | max_chunk_size |  |
| char                        | session_id[64] |  |

## 6.27.1.23 struct star\_v1\_StreamChunksResponse

Definition at line 316 of file [firmware\\_update.pb.h](#).

Collaboration diagram for star\_v1\_StreamChunksResponse:



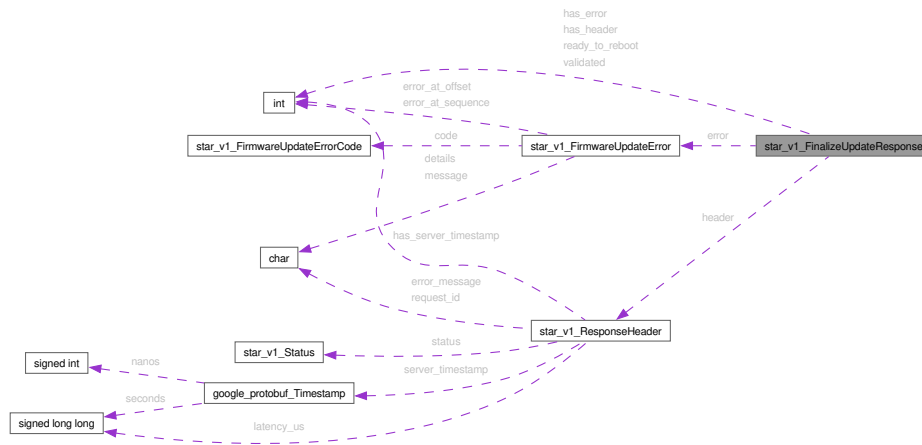
## Data Fields

|                                |                 |  |
|--------------------------------|-----------------|--|
| uint32_t                       | chunks_received |  |
| star_v1_FirmwareUpdateError    | error           |  |
| bool                           | has_error       |  |
| bool                           | has_header      |  |
| bool                           | has_progress    |  |
| star_v1_ResponseHeader         | header          |  |
| star_v1_FirmwareUpdateProgress | progress        |  |

### 6.27.1.24 struct star\_v1\_FinalizeUpdateResponse

Definition at line 331 of file [firmware\\_\\_update.pb.h](#).

Collaboration diagram for star\_v1\_FinalizeUpdateResponse:



#### Data Fields

|                                             |                 |  |
|---------------------------------------------|-----------------|--|
| <a href="#">star_v1_FirmwareUpdateError</a> | error           |  |
| <a href="#">bool</a>                        | has_error       |  |
| <a href="#">bool</a>                        | has_header      |  |
| <a href="#">star_v1_ResponseHeader</a>      | header          |  |
| <a href="#">bool</a>                        | ready_to_reboot |  |
| <a href="#">bool</a>                        | validated       |  |

## 6.27.2 Macro Definition Documentation

### 6.27.2.1 \_\_star\_v1\_FirmwareUpdateErrorCode\_ARRAYSIZE

```
#define __star_v1_FirmwareUpdateErrorCode_ARRAYSIZE ((star_v1_FirmwareUpdateErrorCode)(star_v1_FirmwareUpdateErrorCode_NO_ERROR))
```

Definition at line 356 of file [firmware\\_\\_update.pb.h](#).

### 6.27.2.2 \_\_star\_v1\_FirmwareUpdateErrorCode\_MAX

```
#define __star_v1_FirmwareUpdateErrorCode_MAX star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NO_ERROR
```

Definition at line 355 of file [firmware\\_\\_update.pb.h](#).

### 6.27.2.3 \_\_star\_v1\_FirmwareUpdateErrorCode\_MIN

```
#define __star_v1_FirmwareUpdateErrorCode_MIN star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_UNKNOWN
```

Definition at line 354 of file [firmware\\_\\_update.pb.h](#).

6.27.2.4 `_star_v1_FirmwareUpdateState_ARRAYSIZE`

```
#define _star_v1_FirmwareUpdateState_ARRAYSIZE ((star_v1_FirmwareUpdateState)(star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_ERROR))
```

Definition at line 352 of file [firmware\\_update.pb.h](#).

6.27.2.5 `_star_v1_FirmwareUpdateState_MAX`

```
#define _star_v1_FirmwareUpdateState_MAX star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_ERROR
```

Definition at line 351 of file [firmware\\_update.pb.h](#).

6.27.2.6 `_star_v1_FirmwareUpdateState_MIN`

```
#define _star_v1_FirmwareUpdateState_MIN star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_UNKNOWN
```

Definition at line 350 of file [firmware\\_update.pb.h](#).

6.27.2.7 `star_v1_AbortUpdateRequest_CALLBACK`

```
#define star_v1_AbortUpdateRequest_CALLBACK NULL
```

Definition at line 587 of file [firmware\\_update.pb.h](#).

6.27.2.8 `star_v1_AbortUpdateRequest_DEFAULT`

```
#define star_v1_AbortUpdateRequest_DEFAULT NULL
```

Definition at line 588 of file [firmware\\_update.pb.h](#).

6.27.2.9 `star_v1_AbortUpdateRequest_FIELDLIST`

```
#define star_v1_AbortUpdateRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, STRING, reason, 2)
```

Definition at line 584 of file [firmware\\_update.pb.h](#).

```
00584 #define star_v1_AbortUpdateRequest_FIELDLIST(X, a) \
00585 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00586 X(a, STATIC, SINGULAR, STRING, reason, 2)
```

6.27.2.10 `star_v1_AbortUpdateRequest_fields`

```
#define star_v1_AbortUpdateRequest_fields &star_v1_AbortUpdateRequest_msg
```

Definition at line 745 of file [firmware\\_update.pb.h](#).

#### 6.27.2.11 star\_v1\_AbortUpdateRequest\_header\_MSGTYPE

```
#define star_v1_AbortUpdateRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 589 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.12 star\_v1\_AbortUpdateRequest\_header\_tag

```
#define star_v1_AbortUpdateRequest_header_tag 1
```

Definition at line 447 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.13 star\_v1\_AbortUpdateRequest\_init\_default

```
#define star_v1_AbortUpdateRequest_init_default {false, star_v1_RequestHeader_init_default, ""}
```

Definition at line 394 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.14 star\_v1\_AbortUpdateRequest\_init\_zero

```
#define star_v1_AbortUpdateRequest_init_zero {false, star_v1_RequestHeader_init_zero, ""}
```

Definition at line 418 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.15 star\_v1\_AbortUpdateRequest\_reason\_tag

```
#define star_v1_AbortUpdateRequest_reason_tag 2
```

Definition at line 448 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.16 star\_v1\_AbortUpdateRequest\_size

```
#define star_v1_AbortUpdateRequest_size 279
```

Definition at line 764 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.17 star\_v1\_AbortUpdateResponse\_aborted\_tag

```
#define star_v1_AbortUpdateResponse_aborted_tag 2
```

Definition at line 450 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.18 star\_v1\_AbortUpdateResponse\_CALLBACK

```
#define star_v1_AbortUpdateResponse_CALLBACK NULL
```

Definition at line 594 of file [firmware\\_\\_update.pb.h](#).

## 6.27.2.19 star\_v1\_AbortUpdateResponse\_DEFAULT

```
#define star_v1_AbortUpdateResponse_DEFAULT NULL
```

Definition at line 595 of file [firmware\\_update.pb.h](#).

## 6.27.2.20 star\_v1\_AbortUpdateResponse\_FIELDLIST

```
#define star_v1_AbortUpdateResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, BOOL, aborted, 2)
```

Definition at line 591 of file [firmware\\_update.pb.h](#).

```
00591 #define star_v1_AbortUpdateResponse_FIELDLIST(X, a) \
00592 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00593 X(a, STATIC, SINGULAR, BOOL, aborted, 2)
```

## 6.27.2.21 star\_v1\_AbortUpdateResponse\_fields

```
#define star_v1_AbortUpdateResponse_fields &star_v1_AbortUpdateResponse_msg
```

Definition at line 746 of file [firmware\\_update.pb.h](#).

## 6.27.2.22 star\_v1\_AbortUpdateResponse\_header\_MSGTYPE

```
#define star_v1_AbortUpdateResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 596 of file [firmware\\_update.pb.h](#).

## 6.27.2.23 star\_v1\_AbortUpdateResponse\_header\_tag

```
#define star_v1_AbortUpdateResponse_header_tag 1
```

Definition at line 449 of file [firmware\\_update.pb.h](#).

## 6.27.2.24 star\_v1\_AbortUpdateResponse\_init\_default

```
#define star_v1_AbortUpdateResponse_init_default {false, star_v1_ResponseHeader_init_default, 0}
```

Definition at line 395 of file [firmware\\_update.pb.h](#).

## 6.27.2.25 star\_v1\_AbortUpdateResponse\_init\_zero

```
#define star_v1_AbortUpdateResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0}
```

Definition at line 419 of file [firmware\\_update.pb.h](#).

6.27.2.26 `star_v1_AbortUpdateResponse_size`

```
#define star_v1_AbortUpdateResponse_size 365
```

Definition at line 765 of file [firmware\\_update.pb.h](#).

6.27.2.27 `star_v1_BeginUpdateRequest_CALLBACK`

```
#define star_v1_BeginUpdateRequest_CALLBACK NULL
```

Definition at line 517 of file [firmware\\_update.pb.h](#).

6.27.2.28 `star_v1_BeginUpdateRequest_DEFAULT`

```
#define star_v1_BeginUpdateRequest_DEFAULT NULL
```

Definition at line 518 of file [firmware\\_update.pb.h](#).

6.27.2.29 `star_v1_BeginUpdateRequest_expected_crc32_tag`

```
#define star_v1_BeginUpdateRequest_expected_crc32_tag 4
```

Definition at line 439 of file [firmware\\_update.pb.h](#).

6.27.2.30 `star_v1_BeginUpdateRequest_expected_version_tag`

```
#define star_v1_BeginUpdateRequest_expected_version_tag 3
```

Definition at line 438 of file [firmware\\_update.pb.h](#).

6.27.2.31 `star_v1_BeginUpdateRequest_FIELDLIST`

```
#define star_v1_BeginUpdateRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, UINT32, firmware_size, 2) \
X(a, STATIC, SINGULAR, STRING, expected_version, 3) \
X(a, STATIC, SINGULAR, UINT32, expected_crc32, 4)
```

Definition at line 512 of file [firmware\\_update.pb.h](#).

```
00512 #define star_v1_BeginUpdateRequest_FIELDLIST(X, a) \
00513 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00514 X(a, STATIC, SINGULAR, UINT32, firmware_size, 2) \
00515 X(a, STATIC, SINGULAR, STRING, expected_version, 3) \
00516 X(a, STATIC, SINGULAR, UINT32, expected_crc32, 4)
```



#### 6.27.2.32 star\_v1\_BeginUpdateRequest\_fields

```
#define star_v1_BeginUpdateRequest_fields &star_v1_BeginUpdateRequest_msg
```

Definition at line 737 of file [firmware\\_update.pb.h](#).

#### 6.27.2.33 star\_v1\_BeginUpdateRequest\_firmware\_size\_tag

```
#define star_v1_BeginUpdateRequest_firmware_size_tag 2
```

Definition at line 437 of file [firmware\\_update.pb.h](#).

#### 6.27.2.34 star\_v1\_BeginUpdateRequest\_header\_MSGTYPE

```
#define star_v1_BeginUpdateRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 519 of file [firmware\\_update.pb.h](#).

#### 6.27.2.35 star\_v1\_BeginUpdateRequest\_header\_tag

```
#define star_v1_BeginUpdateRequest_header_tag 1
```

Definition at line 436 of file [firmware\\_update.pb.h](#).

#### 6.27.2.36 star\_v1\_BeginUpdateRequest\_init\_default

```
#define star_v1_BeginUpdateRequest_init_default {false, star_v1_RequestHeader_init_default, 0, "", 0}
```

Definition at line 386 of file [firmware\\_update.pb.h](#).

#### 6.27.2.37 star\_v1\_BeginUpdateRequest\_init\_zero

```
#define star_v1_BeginUpdateRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0, "", 0}
```

Definition at line 410 of file [firmware\\_update.pb.h](#).

#### 6.27.2.38 star\_v1\_BeginUpdateRequest\_size

```
#define star_v1_BeginUpdateRequest_size 194
```

Definition at line 766 of file [firmware\\_update.pb.h](#).

#### 6.27.2.39 star\_v1\_BeginUpdateResponse\_accepted\_tag

```
#define star_v1_BeginUpdateResponse_accepted_tag 2
```

Definition at line 498 of file [firmware\\_update.pb.h](#).

## 6.27.2.40 star\_v1\_BeginUpdateResponse\_CALLBACK

```
#define star_v1_BeginUpdateResponse_CALLBACK NULL
```

Definition at line 527 of file [firmware\\_update.pb.h](#).

## 6.27.2.41 star\_v1\_BeginUpdateResponse\_DEFAULT

```
#define star_v1_BeginUpdateResponse_DEFAULT NULL
```

Definition at line 528 of file [firmware\\_update.pb.h](#).

## 6.27.2.42 star\_v1\_BeginUpdateResponse\_error\_MSGTYPE

```
#define star_v1_BeginUpdateResponse_error_MSGTYPE star_v1_FirmwareUpdateError
```

Definition at line 530 of file [firmware\\_update.pb.h](#).

## 6.27.2.43 star\_v1\_BeginUpdateResponse\_error\_tag

```
#define star_v1_BeginUpdateResponse_error_tag 5
```

Definition at line 501 of file [firmware\\_update.pb.h](#).

## 6.27.2.44 star\_v1\_BeginUpdateResponse\_FIELDLIST

```
#define star_v1_BeginUpdateResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, BOOL, accepted, 2) \
X(a, STATIC, SINGULAR, UINT32, max_chunk_size, 3) \
X(a, STATIC, SINGULAR, STRING, session_id, 4) \
X(a, STATIC, OPTIONAL, MESSAGE, error, 5)
```

Definition at line 521 of file [firmware\\_update.pb.h](#).

```
00521 #define star_v1_BeginUpdateResponse_FIELDLIST(X, a) \
00522 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00523 X(a, STATIC, SINGULAR, BOOL, accepted, 2) \
00524 X(a, STATIC, SINGULAR, UINT32, max_chunk_size, 3) \
00525 X(a, STATIC, SINGULAR, STRING, session_id, 4) \
00526 X(a, STATIC, OPTIONAL, MESSAGE, error, 5)
```

## 6.27.2.45 star\_v1\_BeginUpdateResponse\_fields

```
#define star_v1_BeginUpdateResponse_fields &star_v1_BeginUpdateResponse_msg
```

Definition at line 738 of file [firmware\\_update.pb.h](#).

## 6.27.2.46 star\_v1\_BeginUpdateResponse\_header\_MSGTYPE

```
#define star_v1_BeginUpdateResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 529 of file [firmware\\_update.pb.h](#).

## 6.27.2.47 star\_v1\_BeginUpdateResponse\_header\_tag

```
#define star_v1_BeginUpdateResponse_header_tag 1
```

Definition at line 497 of file [firmware\\_update.pb.h](#).

## 6.27.2.48 star\_v1\_BeginUpdateResponse\_init\_default

```
#define star_v1_BeginUpdateResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0, "", false, star_v1_FirmwareUpdateError_init_default}
```

Definition at line 387 of file [firmware\\_update.pb.h](#).

## 6.27.2.49 star\_v1\_BeginUpdateResponse\_init\_zero

```
#define star_v1_BeginUpdateResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0, "", false, star_v1_FirmwareUpdateError_init_zero}
```

Definition at line 411 of file [firmware\\_update.pb.h](#).

## 6.27.2.50 star\_v1\_BeginUpdateResponse\_max\_chunk\_size\_tag

```
#define star_v1_BeginUpdateResponse_max_chunk_size_tag 3
```

Definition at line 499 of file [firmware\\_update.pb.h](#).

## 6.27.2.51 star\_v1\_BeginUpdateResponse\_session\_id\_tag

```
#define star_v1_BeginUpdateResponse_session_id_tag 4
```

Definition at line 500 of file [firmware\\_update.pb.h](#).

## 6.27.2.52 star\_v1\_BeginUpdateResponse\_size

```
#define star_v1_BeginUpdateResponse_size 713
```

Definition at line 767 of file [firmware\\_update.pb.h](#).

6.27.2.53 `star_v1_FinalizeUpdateRequest_CALLBACK`

```
#define star_v1_FinalizeUpdateRequest_CALLBACK NULL
```

Definition at line 570 of file [firmware\\_\\_update.pb.h](#).

6.27.2.54 `star_v1_FinalizeUpdateRequest_DEFAULT`

```
#define star_v1_FinalizeUpdateRequest_DEFAULT NULL
```

Definition at line 571 of file [firmware\\_\\_update.pb.h](#).

6.27.2.55 `star_v1_FinalizeUpdateRequest_FIELDLIST`

```
#define star_v1_FinalizeUpdateRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

Definition at line 568 of file [firmware\\_\\_update.pb.h](#).

```
00568 #define star_v1_FinalizeUpdateRequest_FIELDLIST(X, a) \
00569 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

6.27.2.56 `star_v1_FinalizeUpdateRequest_fields`

```
#define star_v1_FinalizeUpdateRequest_fields &star_v1_FinalizeUpdateRequest_msg
```

Definition at line 743 of file [firmware\\_\\_update.pb.h](#).

6.27.2.57 `star_v1_FinalizeUpdateRequest_header_MSGTYPE`

```
#define star_v1_FinalizeUpdateRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 572 of file [firmware\\_\\_update.pb.h](#).

6.27.2.58 `star_v1_FinalizeUpdateRequest_header_tag`

```
#define star_v1_FinalizeUpdateRequest_header_tag 1
```

Definition at line 446 of file [firmware\\_\\_update.pb.h](#).

6.27.2.59 `star_v1_FinalizeUpdateRequest_init_default`

```
#define star_v1_FinalizeUpdateRequest_init_default {false, star_v1_RequestHeader_init_default}
```

Definition at line 392 of file [firmware\\_\\_update.pb.h](#).

## 6.27.2.60 star\_v1\_FinalizeUpdateRequest\_init\_zero

```
#define star_v1_FinalizeUpdateRequest_init_zero {false, star_v1_RequestHeader_init_zero}
```

Definition at line 416 of file [firmware\\_update.pb.h](#).

## 6.27.2.61 star\_v1\_FinalizeUpdateRequest\_size

```
#define star_v1_FinalizeUpdateRequest_size 149
```

Definition at line 768 of file [firmware\\_update.pb.h](#).

## 6.27.2.62 star\_v1\_FinalizeUpdateResponse\_CALLBACK

```
#define star_v1_FinalizeUpdateResponse_CALLBACK NULL
```

Definition at line 579 of file [firmware\\_update.pb.h](#).

## 6.27.2.63 star\_v1\_FinalizeUpdateResponse\_DEFAULT

```
#define star_v1_FinalizeUpdateResponse_DEFAULT NULL
```

Definition at line 580 of file [firmware\\_update.pb.h](#).

## 6.27.2.64 star\_v1\_FinalizeUpdateResponse\_error\_MSGTYPE

```
#define star_v1_FinalizeUpdateResponse_error_MSGTYPE star_v1_FirmwareUpdateError
```

Definition at line 582 of file [firmware\\_update.pb.h](#).

## 6.27.2.65 star\_v1\_FinalizeUpdateResponse\_error\_tag

```
#define star_v1_FinalizeUpdateResponse_error_tag 4
```

Definition at line 509 of file [firmware\\_update.pb.h](#).

## 6.27.2.66 star\_v1\_FinalizeUpdateResponse\_FIELDLIST

```
#define star_v1_FinalizeUpdateResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, BOOL, validated, 2) \
X(a, STATIC, SINGULAR, BOOL, ready_to_reboot, 3) \
X(a, STATIC, OPTIONAL, MESSAGE, error, 4)
```

Definition at line 574 of file [firmware\\_update.pb.h](#).

```
00574 #define star_v1_FinalizeUpdateResponse_FIELDLIST(X, a) \
00575 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00576 X(a, STATIC, SINGULAR, BOOL, validated, 2) \
00577 X(a, STATIC, SINGULAR, BOOL, ready_to_reboot, 3) \
00578 X(a, STATIC, OPTIONAL, MESSAGE, error, 4)
```

#### 6.27.2.67 `star_v1_FinalizeUpdateResponse_fields`

```
#define star_v1_FinalizeUpdateResponse_fields &star_v1_FinalizeUpdateResponse_msg
```

Definition at line 744 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.68 `star_v1_FinalizeUpdateResponse_header_MSGTYPE`

```
#define star_v1_FinalizeUpdateResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 581 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.69 `star_v1_FinalizeUpdateResponse_header_tag`

```
#define star_v1_FinalizeUpdateResponse_header_tag 1
```

Definition at line 506 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.70 `star_v1_FinalizeUpdateResponse_init_default`

```
#define star_v1_FinalizeUpdateResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0, false, star_v1_FirmwareUpdateError_init_default}
```

Definition at line 393 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.71 `star_v1_FinalizeUpdateResponse_init_zero`

```
#define star_v1_FinalizeUpdateResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0, false, star_v1_FirmwareUpdateError_init_zero}
```

Definition at line 417 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.72 `star_v1_FinalizeUpdateResponse_ready_to_reboot_tag`

```
#define star_v1_FinalizeUpdateResponse_ready_to_reboot_tag 3
```

Definition at line 508 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.73 `star_v1_FinalizeUpdateResponse_size`

```
#define star_v1_FinalizeUpdateResponse_size 644
```

Definition at line 769 of file [firmware\\_\\_update.pb.h](#).

## 6.27.2.74 star\_v1\_FinalizeUpdateResponse\_validated\_tag

```
#define star_v1_FinalizeUpdateResponse_validated_tag 2
```

Definition at line 507 of file [firmware\\_update.pb.h](#).

## 6.27.2.75 star\_v1\_FirmwareChunk\_CALLBACK

```
#define star_v1_FirmwareChunk_CALLBACK NULL
```

Definition at line 554 of file [firmware\\_update.pb.h](#).

## 6.27.2.76 star\_v1\_FirmwareChunk\_chunk\_crc32\_tag

```
#define star_v1_FirmwareChunk_chunk_crc32_tag 4
```

Definition at line 443 of file [firmware\\_update.pb.h](#).

## 6.27.2.77 star\_v1\_FirmwareChunk\_data\_tag

```
#define star_v1_FirmwareChunk_data_tag 1
```

Definition at line 440 of file [firmware\\_update.pb.h](#).

## 6.27.2.78 star\_v1\_FirmwareChunk\_DEFAULT

```
#define star_v1_FirmwareChunk_DEFAULT NULL
```

Definition at line 555 of file [firmware\\_update.pb.h](#).

## 6.27.2.79 star\_v1\_FirmwareChunk\_FIELDLIST

```
#define star_v1_FirmwareChunk_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, BYTES, data, 1) \
X(a, STATIC, SINGULAR, UINT32, sequence, 2) \
X(a, STATIC, SINGULAR, UINT32, offset, 3) \
X(a, STATIC, SINGULAR, UINT32, chunk_crc32, 4)
```

Definition at line 549 of file [firmware\\_update.pb.h](#).

```
00549 #define star_v1_FirmwareChunk_FIELDLIST(X, a) \
00550 X(a, STATIC, SINGULAR, BYTES, data, 1) \
00551 X(a, STATIC, SINGULAR, UINT32, sequence, 2) \
00552 X(a, STATIC, SINGULAR, UINT32, offset, 3) \
00553 X(a, STATIC, SINGULAR, UINT32, chunk_crc32, 4)
```

#### 6.27.2.80 star\_v1\_FirmwareChunk\_fields

```
#define star_v1_FirmwareChunk_fields &star_v1_FirmwareChunk_msg
```

Definition at line 741 of file [firmware\\_update.pb.h](#).

#### 6.27.2.81 star\_v1\_FirmwareChunk\_init\_default

```
#define star_v1_FirmwareChunk_init_default {{0, {0}}, 0, 0, 0}
```

Definition at line 390 of file [firmware\\_update.pb.h](#).

#### 6.27.2.82 star\_v1\_FirmwareChunk\_init\_zero

```
#define star_v1_FirmwareChunk_init_zero {{0, {0}}, 0, 0, 0}
```

Definition at line 414 of file [firmware\\_update.pb.h](#).

#### 6.27.2.83 star\_v1\_FirmwareChunk\_offset\_tag

```
#define star_v1_FirmwareChunk_offset_tag 3
```

Definition at line 442 of file [firmware\\_update.pb.h](#).

#### 6.27.2.84 star\_v1\_FirmwareChunk\_sequence\_tag

```
#define star_v1_FirmwareChunk_sequence_tag 2
```

Definition at line 441 of file [firmware\\_update.pb.h](#).

#### 6.27.2.85 star\_v1\_FirmwareChunk\_size

```
#define star_v1_FirmwareChunk_size 1045
```

Definition at line 770 of file [firmware\\_update.pb.h](#).

#### 6.27.2.86 star\_v1\_FirmwareInfo\_build\_date\_tag

```
#define star_v1_FirmwareInfo_build_date_tag 3
```

Definition at line 482 of file [firmware\\_update.pb.h](#).

#### 6.27.2.87 star\_v1\_FirmwareInfo\_CALLBACK

```
#define star_v1_FirmwareInfo_CALLBACK NULL
```

Definition at line 699 of file [firmware\\_update.pb.h](#).



## 6.27.2.88 star\_v1\_FirmwareInfo\_chip\_target\_tag

```
#define star_v1_FirmwareInfo_chip_target_tag 7
```

Definition at line 486 of file [firmware\\_update.pb.h](#).

## 6.27.2.89 star\_v1\_FirmwareInfo\_crc32\_tag

```
#define star_v1_FirmwareInfo_crc32_tag 5
```

Definition at line 484 of file [firmware\\_update.pb.h](#).

## 6.27.2.90 star\_v1\_FirmwareInfo\_DEFAULT

```
#define star_v1_FirmwareInfo_DEFAULT NULL
```

Definition at line 700 of file [firmware\\_update.pb.h](#).

## 6.27.2.91 star\_v1\_FirmwareInfo\_FIELDLIST

```
#define star_v1_FirmwareInfo_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, STRING, version, 1) \
X(a, STATIC, SINGULAR, STRING, git_hash, 2) \
X(a, STATIC, SINGULAR, STRING, build_date, 3) \
X(a, STATIC, SINGULAR, UINT32, size, 4) \
X(a, STATIC, SINGULAR, UINT32, crc32, 5) \
X(a, STATIC, SINGULAR, STRING, idf_version, 6) \
X(a, STATIC, SINGULAR, STRING, chip_target, 7)
```

Definition at line 691 of file [firmware\\_update.pb.h](#).

```
00691 #define star_v1_FirmwareInfo_FIELDLIST(X, a) \
00692 X(a, STATIC, SINGULAR, STRING, version, 1) \
00693 X(a, STATIC, SINGULAR, STRING, git_hash, 2) \
00694 X(a, STATIC, SINGULAR, STRING, build_date, 3) \
00695 X(a, STATIC, SINGULAR, UINT32, size, 4) \
00696 X(a, STATIC, SINGULAR, UINT32, crc32, 5) \
00697 X(a, STATIC, SINGULAR, STRING, idf_version, 6) \
00698 X(a, STATIC, SINGULAR, STRING, chip_target, 7)
```

## 6.27.2.92 star\_v1\_FirmwareInfo\_fields

```
#define star_v1_FirmwareInfo_fields &star_v1_FirmwareInfo_msg
```

Definition at line 759 of file [firmware\\_update.pb.h](#).

## 6.27.2.93 star\_v1\_FirmwareInfo\_git\_hash\_tag

```
#define star_v1_FirmwareInfo_git_hash_tag 2
```

Definition at line 481 of file [firmware\\_update.pb.h](#).

6.27.2.94 `star_v1_FirmwareInfo_idf_version_tag`

```
#define star_v1_FirmwareInfo_idf_version_tag 6
```

Definition at line 485 of file [firmware\\_update.pb.h](#).

6.27.2.95 `star_v1_FirmwareInfo_init_default`

```
#define star_v1_FirmwareInfo_init_default { "", "", "", 0, 0, "", "" }
```

Definition at line 408 of file [firmware\\_update.pb.h](#).

6.27.2.96 `star_v1_FirmwareInfo_init_zero`

```
#define star_v1_FirmwareInfo_init_zero { "", "", "", 0, 0, "", "" }
```

Definition at line 432 of file [firmware\\_update.pb.h](#).

6.27.2.97 `star_v1_FirmwareInfo_size`

```
#define star_v1_FirmwareInfo_size 145
```

Definition at line 771 of file [firmware\\_update.pb.h](#).

6.27.2.98 `star_v1_FirmwareInfo_size_tag`

```
#define star_v1_FirmwareInfo_size_tag 4
```

Definition at line 483 of file [firmware\\_update.pb.h](#).

6.27.2.99 `star_v1_FirmwareInfo_version_tag`

```
#define star_v1_FirmwareInfo_version_tag 1
```

Definition at line 480 of file [firmware\\_update.pb.h](#).

6.27.2.100 `star_v1_FirmwareUpdateError_CALLBACK`

```
#define star_v1_FirmwareUpdateError_CALLBACK NULL
```

Definition at line 708 of file [firmware\\_update.pb.h](#).

6.27.2.101 `star_v1_FirmwareUpdateError_code_ENUMTYPE`

```
#define star_v1_FirmwareUpdateError_code_ENUMTYPE star_v1_FirmwareUpdateErrorCode
```

Definition at line 382 of file [firmware\\_update.pb.h](#).

## 6.27.2.102 star\_v1\_FirmwareUpdateError\_code\_tag

```
#define star_v1_FirmwareUpdateError_code_tag 1
```

Definition at line 492 of file [firmware\\_update.pb.h](#).

## 6.27.2.103 star\_v1\_FirmwareUpdateError\_DEFAULT

```
#define star_v1_FirmwareUpdateError_DEFAULT NULL
```

Definition at line 709 of file [firmware\\_update.pb.h](#).

## 6.27.2.104 star\_v1\_FirmwareUpdateError\_details\_tag

```
#define star_v1_FirmwareUpdateError_details_tag 3
```

Definition at line 494 of file [firmware\\_update.pb.h](#).

## 6.27.2.105 star\_v1\_FirmwareUpdateError\_error\_at\_offset\_tag

```
#define star_v1_FirmwareUpdateError_error_at_offset_tag 5
```

Definition at line 496 of file [firmware\\_update.pb.h](#).

## 6.27.2.106 star\_v1\_FirmwareUpdateError\_error\_at\_sequence\_tag

```
#define star_v1_FirmwareUpdateError_error_at_sequence_tag 4
```

Definition at line 495 of file [firmware\\_update.pb.h](#).

## 6.27.2.107 star\_v1\_FirmwareUpdateError\_FIELDLIST

```
#define star_v1_FirmwareUpdateError_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, UENUM, code, 1) \
X(a, STATIC, SINGULAR, STRING, message, 2) \
X(a, STATIC, SINGULAR, STRING, details, 3) \
X(a, STATIC, SINGULAR, UINT32, error_at_sequence, 4) \
X(a, STATIC, SINGULAR, UINT32, error_at_offset, 5)
```

Definition at line 702 of file [firmware\\_update.pb.h](#).

```
00702 #define star_v1_FirmwareUpdateError_FIELDLIST(X, a) \
00703 X(a, STATIC, SINGULAR, UENUM, code, 1) \
00704 X(a, STATIC, SINGULAR, STRING, message, 2) \
00705 X(a, STATIC, SINGULAR, STRING, details, 3) \
00706 X(a, STATIC, SINGULAR, UINT32, error_at_sequence, 4) \
00707 X(a, STATIC, SINGULAR, UINT32, error_at_offset, 5)
```

#### 6.27.2.108 star\_v1\_FirmwareUpdateError\_fields

```
#define star_v1_FirmwareUpdateError_fields &star_v1_FirmwareUpdateError_msg
```

Definition at line 760 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.109 star\_v1\_FirmwareUpdateError\_init\_default

```
#define star_v1_FirmwareUpdateError_init_default {_star_v1_FirmwareUpdateErrorCode_MIN, "", "", 0, 0}
```

Definition at line 409 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.110 star\_v1\_FirmwareUpdateError\_init\_zero

```
#define star_v1_FirmwareUpdateError_init_zero {_star_v1_FirmwareUpdateErrorCode_MIN, "", "", 0, 0}
```

Definition at line 433 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.111 star\_v1\_FirmwareUpdateError\_message\_tag

```
#define star_v1_FirmwareUpdateError_message_tag 2
```

Definition at line 493 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.112 star\_v1\_FirmwareUpdateError\_size

```
#define star_v1_FirmwareUpdateError_size 274
```

Definition at line 772 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.113 star\_v1\_FirmwareUpdateProgress\_bytes\_received\_tag

```
#define star_v1_FirmwareUpdateProgress_bytes_received_tag 2
```

Definition at line 455 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.114 star\_v1\_FirmwareUpdateProgress\_CALLBACK

```
#define star_v1_FirmwareUpdateProgress_CALLBACK NULL
```

Definition at line 628 of file [firmware\\_\\_update.pb.h](#).

#### 6.27.2.115 star\_v1\_FirmwareUpdateProgress\_DEFAULT

```
#define star_v1_FirmwareUpdateProgress_DEFAULT NULL
```

Definition at line 629 of file [firmware\\_\\_update.pb.h](#).

## 6.27.2.116 star\_v1\_FirmwareUpdateProgress\_estimated\_seconds\_remaining\_tag

```
#define star_v1_FirmwareUpdateProgress_estimated_seconds_remaining_tag 6
```

Definition at line 459 of file [firmware\\_update.pb.h](#).

## 6.27.2.117 star\_v1\_FirmwareUpdateProgress\_FIELDLIST

```
#define star_v1_FirmwareUpdateProgress_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, SINGULAR, UINT32, total_size, 1) \
X(a, STATIC, SINGULAR, UINT32, bytes_received, 2) \
X(a, STATIC, SINGULAR, INT32, progress_percent, 3) \
X(a, STATIC, SINGULAR, UINT32, running_crc32, 4) \
X(a, STATIC, SINGULAR, UENUM, state, 5) \
X(a, STATIC, SINGULAR, INT32, estimated_seconds_remaining, 6) \
X(a, STATIC, SINGULAR, UINT32, transfer_speed_bps, 7) \
X(a, STATIC, SINGULAR, UINT32, last_sequence, 8)
```

Definition at line 619 of file [firmware\\_update.pb.h](#).

```
00619 #define star_v1_FirmwareUpdateProgress_FIELDLIST(X, a) \
00620 X(a, STATIC, SINGULAR, UINT32, total_size, 1) \
00621 X(a, STATIC, SINGULAR, UINT32, bytes_received, 2) \
00622 X(a, STATIC, SINGULAR, INT32, progress_percent, 3) \
00623 X(a, STATIC, SINGULAR, UINT32, running_crc32, 4) \
00624 X(a, STATIC, SINGULAR, UENUM, state, 5) \
00625 X(a, STATIC, SINGULAR, INT32, estimated_seconds_remaining, 6) \
00626 X(a, STATIC, SINGULAR, UINT32, transfer_speed_bps, 7) \
00627 X(a, STATIC, SINGULAR, UINT32, last_sequence, 8)
```

## 6.27.2.118 star\_v1\_FirmwareUpdateProgress\_fields

```
#define star_v1_FirmwareUpdateProgress_fields &star_v1_FirmwareUpdateProgress_msg
```

Definition at line 750 of file [firmware\\_update.pb.h](#).

## 6.27.2.119 star\_v1\_FirmwareUpdateProgress\_init\_default

```
#define star_v1_FirmwareUpdateProgress_init_default {0, 0, 0, 0, __star_v1_FirmwareUpdateState_MIN, 0, 0, 0}
```

Definition at line 399 of file [firmware\\_update.pb.h](#).

## 6.27.2.120 star\_v1\_FirmwareUpdateProgress\_init\_zero

```
#define star_v1_FirmwareUpdateProgress_init_zero {0, 0, 0, 0, __star_v1_FirmwareUpdateState_MIN, 0, 0, 0}
```

Definition at line 423 of file [firmware\\_update.pb.h](#).

## 6.27.2.121 star\_v1\_FirmwareUpdateProgress\_last\_sequence\_tag

```
#define star_v1_FirmwareUpdateProgress_last_sequence_tag 8
```

Definition at line 461 of file [firmware\\_update.pb.h](#).

6.27.2.122 `star_v1_FirmwareUpdateProgress_progress_percent_tag`

```
#define star_v1_FirmwareUpdateProgress_progress_percent_tag 3
```

Definition at line 456 of file [firmware\\_update.pb.h](#).

6.27.2.123 `star_v1_FirmwareUpdateProgress_running_crc32_tag`

```
#define star_v1_FirmwareUpdateProgress_running_crc32_tag 4
```

Definition at line 457 of file [firmware\\_update.pb.h](#).

6.27.2.124 `star_v1_FirmwareUpdateProgress_size`

```
#define star_v1_FirmwareUpdateProgress_size 54
```

Definition at line 773 of file [firmware\\_update.pb.h](#).

6.27.2.125 `star_v1_FirmwareUpdateProgress_state_ENUMTYPE`

```
#define star_v1_FirmwareUpdateProgress_state_ENUMTYPE star_v1_FirmwareUpdateState
```

Definition at line 371 of file [firmware\\_update.pb.h](#).

6.27.2.126 `star_v1_FirmwareUpdateProgress_state_tag`

```
#define star_v1_FirmwareUpdateProgress_state_tag 5
```

Definition at line 458 of file [firmware\\_update.pb.h](#).

6.27.2.127 `star_v1_FirmwareUpdateProgress_total_size_tag`

```
#define star_v1_FirmwareUpdateProgress_total_size_tag 1
```

Definition at line 454 of file [firmware\\_update.pb.h](#).

6.27.2.128 `star_v1_FirmwareUpdateProgress_transfer_speed_bps_tag`

```
#define star_v1_FirmwareUpdateProgress_transfer_speed_bps_tag 7
```

Definition at line 460 of file [firmware\\_update.pb.h](#).

6.27.2.129 `star_v1_GetFirmwareInfoRequest_CALLBACK`

```
#define star_v1_GetFirmwareInfoRequest_CALLBACK NULL
```

Definition at line 675 of file [firmware\\_update.pb.h](#).

## 6.27.2.130 star\_v1\_GetFirmwareInfoRequest\_DEFAULT

```
#define star_v1_GetFirmwareInfoRequest_DEFAULT NULL
```

Definition at line 676 of file [firmware\\_update.pb.h](#).

## 6.27.2.131 star\_v1\_GetFirmwareInfoRequest\_FIELDLIST

```
#define star_v1_GetFirmwareInfoRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

Definition at line 673 of file [firmware\\_update.pb.h](#).

```
00673 #define star_v1_GetFirmwareInfoRequest_FIELDLIST(X, a) \
00674 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

## 6.27.2.132 star\_v1\_GetFirmwareInfoRequest\_fields

```
#define star_v1_GetFirmwareInfoRequest_fields &star_v1_GetFirmwareInfoRequest_msg
```

Definition at line 757 of file [firmware\\_update.pb.h](#).

## 6.27.2.133 star\_v1\_GetFirmwareInfoRequest\_header\_MSGTYPE

```
#define star_v1_GetFirmwareInfoRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 677 of file [firmware\\_update.pb.h](#).

## 6.27.2.134 star\_v1\_GetFirmwareInfoRequest\_header\_tag

```
#define star_v1_GetFirmwareInfoRequest_header_tag 1
```

Definition at line 479 of file [firmware\\_update.pb.h](#).

## 6.27.2.135 star\_v1\_GetFirmwareInfoRequest\_init\_default

```
#define star_v1_GetFirmwareInfoRequest_init_default {false, star_v1_RequestHeader_init_default}
```

Definition at line 406 of file [firmware\\_update.pb.h](#).

## 6.27.2.136 star\_v1\_GetFirmwareInfoRequest\_init\_zero

```
#define star_v1_GetFirmwareInfoRequest_init_zero {false, star_v1_RequestHeader_init_zero}
```

Definition at line 430 of file [firmware\\_update.pb.h](#).

## 6.27.2.137 star\_v1\_GetFirmwareInfoRequest\_size

```
#define star_v1_GetFirmwareInfoRequest_size 149
```

Definition at line 774 of file [firmware\\_update.pb.h](#).

## 6.27.2.138 star\_v1\_GetFirmwareInfoResponse\_CALLBACK

```
#define star_v1_GetFirmwareInfoResponse_CALLBACK NULL
```

Definition at line 685 of file [firmware\\_update.pb.h](#).

## 6.27.2.139 star\_v1\_GetFirmwareInfoResponse\_current\_firmware\_MSGTYPE

```
#define star_v1_GetFirmwareInfoResponse_current_firmware_MSGTYPE star_v1_FirmwareInfo
```

Definition at line 688 of file [firmware\\_update.pb.h](#).

## 6.27.2.140 star\_v1\_GetFirmwareInfoResponse\_current\_firmware\_tag

```
#define star_v1_GetFirmwareInfoResponse_current_firmware_tag 2
```

Definition at line 488 of file [firmware\\_update.pb.h](#).

## 6.27.2.141 star\_v1\_GetFirmwareInfoResponse\_DEFAULT

```
#define star_v1_GetFirmwareInfoResponse_DEFAULT NULL
```

Definition at line 686 of file [firmware\\_update.pb.h](#).

## 6.27.2.142 star\_v1\_GetFirmwareInfoResponse\_FIELDLIST

```
#define star_v1_GetFirmwareInfoResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, current_firmware, 2) \
X(a, STATIC, OPTIONAL, MESSAGE, previous_firmware, 3) \
X(a, STATIC, SINGULAR, BOOL, is_ota_boot, 4) \
X(a, STATIC, SINGULAR, BOOL, is_valid, 5)
```

Definition at line 679 of file [firmware\\_update.pb.h](#).

```
00679 #define star_v1_GetFirmwareInfoResponse_FIELDLIST(X, a) \
00680 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00681 X(a, STATIC, OPTIONAL, MESSAGE, current_firmware, 2) \
00682 X(a, STATIC, OPTIONAL, MESSAGE, previous_firmware, 3) \
00683 X(a, STATIC, SINGULAR, BOOL, is_ota_boot, 4) \
00684 X(a, STATIC, SINGULAR, BOOL, is_valid, 5)
```



## 6.27.2.143 star\_v1\_GetFirmwareInfoResponse\_fields

```
#define star_v1_GetFirmwareInfoResponse_fields &star_v1_GetFirmwareInfoResponse_msg
```

Definition at line 758 of file [firmware\\_update.pb.h](#).

## 6.27.2.144 star\_v1\_GetFirmwareInfoResponse\_header\_MSGTYPE

```
#define star_v1_GetFirmwareInfoResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 687 of file [firmware\\_update.pb.h](#).

## 6.27.2.145 star\_v1\_GetFirmwareInfoResponse\_header\_tag

```
#define star_v1_GetFirmwareInfoResponse_header_tag 1
```

Definition at line 487 of file [firmware\\_update.pb.h](#).

## 6.27.2.146 star\_v1\_GetFirmwareInfoResponse\_init\_default

```
#define star_v1_GetFirmwareInfoResponse_init_default {false, star_v1_ResponseHeader_init_default, false, star_v1_FirmwareInfo_init_default, false, star_v1_FirmwareInfo_init_default, 0, 0}
```

Definition at line 407 of file [firmware\\_update.pb.h](#).

## 6.27.2.147 star\_v1\_GetFirmwareInfoResponse\_init\_zero

```
#define star_v1_GetFirmwareInfoResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_FirmwareInfo_init_zero, false, star_v1_FirmwareInfo_init_zero, 0, 0}
```

Definition at line 431 of file [firmware\\_update.pb.h](#).

## 6.27.2.148 star\_v1\_GetFirmwareInfoResponse\_is\_ota\_boot\_tag

```
#define star_v1_GetFirmwareInfoResponse_is_ota_boot_tag 4
```

Definition at line 490 of file [firmware\\_update.pb.h](#).

## 6.27.2.149 star\_v1\_GetFirmwareInfoResponse\_is\_valid\_tag

```
#define star_v1_GetFirmwareInfoResponse_is_valid_tag 5
```

Definition at line 491 of file [firmware\\_update.pb.h](#).

6.27.2.150 `star_v1_GetFirmwareInfoResponse_previous_firmware_MSGTYPE`

```
#define star_v1_GetFirmwareInfoResponse_previous_firmware_MSGTYPE star_v1_FirmwareInfo
```

Definition at line 689 of file [firmware\\_\\_update.pb.h](#).

6.27.2.151 `star_v1_GetFirmwareInfoResponse_previous_firmware_tag`

```
#define star_v1_GetFirmwareInfoResponse_previous_firmware_tag 3
```

Definition at line 489 of file [firmware\\_\\_update.pb.h](#).

6.27.2.152 `star_v1_GetFirmwareInfoResponse_size`

```
#define star_v1_GetFirmwareInfoResponse_size 663
```

Definition at line 775 of file [firmware\\_\\_update.pb.h](#).

6.27.2.153 `star_v1_GetUpdateProgressRequest_CALLBACK`

```
#define star_v1_GetUpdateProgressRequest_CALLBACK NULL
```

Definition at line 600 of file [firmware\\_\\_update.pb.h](#).

6.27.2.154 `star_v1_GetUpdateProgressRequest_DEFAULT`

```
#define star_v1_GetUpdateProgressRequest_DEFAULT NULL
```

Definition at line 601 of file [firmware\\_\\_update.pb.h](#).

6.27.2.155 `star_v1_GetUpdateProgressRequest_FIELDLIST`

```
#define star_v1_GetUpdateProgressRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

Definition at line 598 of file [firmware\\_\\_update.pb.h](#).

```
00598 #define star_v1_GetUpdateProgressRequest_FIELDLIST(X, a) \
00599 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

6.27.2.156 `star_v1_GetUpdateProgressRequest_fields`

```
#define star_v1_GetUpdateProgressRequest_fields &star_v1_GetUpdateProgressRequest_msg
```

Definition at line 747 of file [firmware\\_\\_update.pb.h](#).

## 6.27.2.157 star\_v1\_GetUpdateProgressRequest\_header\_MSGTYPE

```
#define star_v1_GetUpdateProgressRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 602 of file [firmware\\_update.pb.h](#).

## 6.27.2.158 star\_v1\_GetUpdateProgressRequest\_header\_tag

```
#define star_v1_GetUpdateProgressRequest_header_tag 1
```

Definition at line 451 of file [firmware\\_update.pb.h](#).

## 6.27.2.159 star\_v1\_GetUpdateProgressRequest\_init\_default

```
#define star_v1_GetUpdateProgressRequest_init_default {false, star_v1_RequestHeader_init_default}
```

Definition at line 396 of file [firmware\\_update.pb.h](#).

## 6.27.2.160 star\_v1\_GetUpdateProgressRequest\_init\_zero

```
#define star_v1_GetUpdateProgressRequest_init_zero {false, star_v1_RequestHeader_init_zero}
```

Definition at line 420 of file [firmware\\_update.pb.h](#).

## 6.27.2.161 star\_v1\_GetUpdateProgressRequest\_size

```
#define star_v1_GetUpdateProgressRequest_size 149
```

Definition at line 776 of file [firmware\\_update.pb.h](#).

## 6.27.2.162 star\_v1\_GetUpdateProgressResponse\_CALLBACK

```
#define star_v1_GetUpdateProgressResponse_CALLBACK NULL
```

Definition at line 607 of file [firmware\\_update.pb.h](#).

## 6.27.2.163 star\_v1\_GetUpdateProgressResponse\_DEFAULT

```
#define star_v1_GetUpdateProgressResponse_DEFAULT NULL
```

Definition at line 608 of file [firmware\\_update.pb.h](#).

## 6.27.2.164 star\_v1\_GetUpdateProgressResponse\_FIELDLIST

```
#define star_v1_GetUpdateProgressResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, progress, 2)
```

Definition at line 604 of file [firmware\\_update.pb.h](#).

```
00604 #define star_v1_GetUpdateProgressResponse_FIELDLIST(X, a) \
00605 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00606 X(a, STATIC, OPTIONAL, MESSAGE, progress, 2)
```

## 6.27.2.165 star\_v1\_GetUpdateProgressResponse\_fields

```
#define star_v1_GetUpdateProgressResponse_fields &star_v1_GetUpdateProgressResponse_msg
```

Definition at line 748 of file [firmware\\_update.pb.h](#).

## 6.27.2.166 star\_v1\_GetUpdateProgressResponse\_header\_MSGTYPE

```
#define star_v1_GetUpdateProgressResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 609 of file [firmware\\_update.pb.h](#).

## 6.27.2.167 star\_v1\_GetUpdateProgressResponse\_header\_tag

```
#define star_v1_GetUpdateProgressResponse_header_tag 1
```

Definition at line 465 of file [firmware\\_update.pb.h](#).

## 6.27.2.168 star\_v1\_GetUpdateProgressResponse\_init\_default

```
#define star_v1_GetUpdateProgressResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
star_v1_FirmwareUpdateProgress_init_default}
```

Definition at line 397 of file [firmware\\_update.pb.h](#).

## 6.27.2.169 star\_v1\_GetUpdateProgressResponse\_init\_zero

```
#define star_v1_GetUpdateProgressResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_FirmwareUpdateProgress_init_zero}
```

Definition at line 421 of file [firmware\\_update.pb.h](#).

## 6.27.2.170 star\_v1\_GetUpdateProgressResponse\_progress\_MSGTYPE

```
#define star_v1_GetUpdateProgressResponse_progress_MSGTYPE star_v1_FirmwareUpdateProgress
```

Definition at line 610 of file [firmware\\_update.pb.h](#).

## 6.27.2.171 star\_v1\_GetUpdateProgressResponse\_progress\_tag

```
#define star_v1_GetUpdateProgressResponse_progress_tag 2
```

Definition at line 466 of file [firmware\\_update.pb.h](#).

## 6.27.2.172 star\_v1\_GetUpdateProgressResponse\_size

```
#define star_v1_GetUpdateProgressResponse_size 419
```

Definition at line 777 of file [firmware\\_update.pb.h](#).

## 6.27.2.173 star\_v1\_MarkValidRequest\_CALLBACK

```
#define star_v1_MarkValidRequest_CALLBACK NULL
```

Definition at line 662 of file [firmware\\_update.pb.h](#).

## 6.27.2.174 star\_v1\_MarkValidRequest\_DEFAULT

```
#define star_v1_MarkValidRequest_DEFAULT NULL
```

Definition at line 663 of file [firmware\\_update.pb.h](#).

## 6.27.2.175 star\_v1\_MarkValidRequest\_FIELDLIST

```
#define star_v1_MarkValidRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

Definition at line 660 of file [firmware\\_update.pb.h](#).

```
00660 #define star_v1_MarkValidRequest_FIELDLIST(X, a) \
00661 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

## 6.27.2.176 star\_v1\_MarkValidRequest\_fields

```
#define star_v1_MarkValidRequest_fields &star_v1_MarkValidRequest_msg
```

Definition at line 755 of file [firmware\\_update.pb.h](#).

## 6.27.2.177 star\_v1\_MarkValidRequest\_header\_MSGTYPE

```
#define star_v1_MarkValidRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 664 of file [firmware\\_update.pb.h](#).

6.27.2.178 `star_v1_MarkValidRequest_header_tag`

```
#define star_v1_MarkValidRequest_header_tag 1
```

Definition at line 476 of file [firmware\\_\\_update.pb.h](#).

6.27.2.179 `star_v1_MarkValidRequest_init_default`

```
#define star_v1_MarkValidRequest_init_default {false, star_v1_RequestHeader_init_default}
```

Definition at line 404 of file [firmware\\_\\_update.pb.h](#).

6.27.2.180 `star_v1_MarkValidRequest_init_zero`

```
#define star_v1_MarkValidRequest_init_zero {false, star_v1_RequestHeader_init_zero}
```

Definition at line 428 of file [firmware\\_\\_update.pb.h](#).

6.27.2.181 `star_v1_MarkValidRequest_size`

```
#define star_v1_MarkValidRequest_size 149
```

Definition at line 778 of file [firmware\\_\\_update.pb.h](#).

6.27.2.182 `star_v1_MarkValidResponse_CALLBACK`

```
#define star_v1_MarkValidResponse_CALLBACK NULL
```

Definition at line 669 of file [firmware\\_\\_update.pb.h](#).

6.27.2.183 `star_v1_MarkValidResponse_DEFAULT`

```
#define star_v1_MarkValidResponse_DEFAULT NULL
```

Definition at line 670 of file [firmware\\_\\_update.pb.h](#).

6.27.2.184 `star_v1_MarkValidResponse_FIELDLIST`

```
#define star_v1_MarkValidResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, BOOL, marked_valid, 2)
```

Definition at line 666 of file [firmware\\_\\_update.pb.h](#).

```
00666 #define star_v1_MarkValidResponse_FIELDLIST(X, a) \
00667 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00668 X(a, STATIC, SINGULAR, BOOL, marked_valid, 2)
```

## 6.27.2.185 star\_v1\_MarkValidResponse\_fields

```
#define star_v1_MarkValidResponse_fields &star_v1_MarkValidResponse_msg
```

Definition at line 756 of file [firmware\\_update.pb.h](#).

## 6.27.2.186 star\_v1\_MarkValidResponse\_header\_MSGTYPE

```
#define star_v1_MarkValidResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 671 of file [firmware\\_update.pb.h](#).

## 6.27.2.187 star\_v1\_MarkValidResponse\_header\_tag

```
#define star_v1_MarkValidResponse_header_tag 1
```

Definition at line 477 of file [firmware\\_update.pb.h](#).

## 6.27.2.188 star\_v1\_MarkValidResponse\_init\_default

```
#define star_v1_MarkValidResponse_init_default {false, star_v1_ResponseHeader_init_default, 0}
```

Definition at line 405 of file [firmware\\_update.pb.h](#).

## 6.27.2.189 star\_v1\_MarkValidResponse\_init\_zero

```
#define star_v1_MarkValidResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0}
```

Definition at line 429 of file [firmware\\_update.pb.h](#).

## 6.27.2.190 star\_v1\_MarkValidResponse\_marked\_valid\_tag

```
#define star_v1_MarkValidResponse_marked_valid_tag 2
```

Definition at line 478 of file [firmware\\_update.pb.h](#).

## 6.27.2.191 star\_v1\_MarkValidResponse\_size

```
#define star_v1_MarkValidResponse_size 365
```

Definition at line 779 of file [firmware\\_update.pb.h](#).

## 6.27.2.192 star\_v1\_RebootRequest\_CALLBACK

```
#define star_v1_RebootRequest_CALLBACK NULL
```

Definition at line 634 of file [firmware\\_update.pb.h](#).

## 6.27.2.193 star\_v1\_RebootRequest\_DEFAULT

```
#define star_v1_RebootRequest_DEFAULT NULL
```

Definition at line 635 of file [firmware\\_update.pb.h](#).

## 6.27.2.194 star\_v1\_RebootRequest\_delay\_ms\_tag

```
#define star_v1_RebootRequest_delay_ms_tag 2
```

Definition at line 468 of file [firmware\\_update.pb.h](#).

## 6.27.2.195 star\_v1\_RebootRequest\_FIELDLIST

```
#define star_v1_RebootRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, UINT32, delay_ms, 2)
```

Definition at line 631 of file [firmware\\_update.pb.h](#).

```
00631 #define star_v1_RebootRequest_FIELDLIST(X, a) \
00632 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00633 X(a, STATIC, SINGULAR, UINT32, delay_ms, 2)
```

## 6.27.2.196 star\_v1\_RebootRequest\_fields

```
#define star_v1_RebootRequest_fields &star_v1_RebootRequest_msg
```

Definition at line 751 of file [firmware\\_update.pb.h](#).

## 6.27.2.197 star\_v1\_RebootRequest\_header\_MSGTYPE

```
#define star_v1_RebootRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 636 of file [firmware\\_update.pb.h](#).

## 6.27.2.198 star\_v1\_RebootRequest\_header\_tag

```
#define star_v1_RebootRequest_header_tag 1
```

Definition at line 467 of file [firmware\\_update.pb.h](#).

## 6.27.2.199 star\_v1\_RebootRequest\_init\_default

```
#define star_v1_RebootRequest_init_default {false, star_v1_RequestHeader_init_default, 0}
```

Definition at line 400 of file [firmware\\_update.pb.h](#).



## 6.27.2.200 star\_v1\_RebootRequest\_init\_zero

```
#define star_v1_RebootRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}
```

Definition at line 424 of file [firmware\\_update.pb.h](#).

## 6.27.2.201 star\_v1\_RebootRequest\_size

```
#define star_v1_RebootRequest_size 155
```

Definition at line 780 of file [firmware\\_update.pb.h](#).

## 6.27.2.202 star\_v1\_RebootResponse\_CALLBACK

```
#define star_v1_RebootResponse_CALLBACK NULL
```

Definition at line 642 of file [firmware\\_update.pb.h](#).

## 6.27.2.203 star\_v1\_RebootResponse\_DEFAULT

```
#define star_v1_RebootResponse_DEFAULT NULL
```

Definition at line 643 of file [firmware\\_update.pb.h](#).

## 6.27.2.204 star\_v1\_RebootResponse\_FIELDLIST

```
#define star_v1_RebootResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, BOOL, rebooting, 2) \
X(a, STATIC, SINGULAR, UINT32, reboot_delay_ms, 3)
```

Definition at line 638 of file [firmware\\_update.pb.h](#).

```
00638 #define star_v1_RebootResponse_FIELDLIST(X, a) \
00639 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00640 X(a, STATIC, SINGULAR, BOOL, rebooting, 2) \
00641 X(a, STATIC, SINGULAR, UINT32, reboot_delay_ms, 3)
```

## 6.27.2.205 star\_v1\_RebootResponse\_fields

```
#define star_v1_RebootResponse_fields &star_v1_RebootResponse_msg
```

Definition at line 752 of file [firmware\\_update.pb.h](#).

## 6.27.2.206 star\_v1\_RebootResponse\_header\_MSGTYPE

```
#define star_v1_RebootResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 644 of file [firmware\\_update.pb.h](#).

6.27.2.207 `star_v1_RebootResponse_header_tag`

```
#define star_v1_RebootResponse_header_tag 1
```

Definition at line 469 of file [firmware\\_update.pb.h](#).

6.27.2.208 `star_v1_RebootResponse_init_default`

```
#define star_v1_RebootResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0}
```

Definition at line 401 of file [firmware\\_update.pb.h](#).

6.27.2.209 `star_v1_RebootResponse_init_zero`

```
#define star_v1_RebootResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0}
```

Definition at line 425 of file [firmware\\_update.pb.h](#).

6.27.2.210 `star_v1_RebootResponse_reboot_delay_ms_tag`

```
#define star_v1_RebootResponse_reboot_delay_ms_tag 3
```

Definition at line 471 of file [firmware\\_update.pb.h](#).

6.27.2.211 `star_v1_RebootResponse_rebooting_tag`

```
#define star_v1_RebootResponse_rebooting_tag 2
```

Definition at line 470 of file [firmware\\_update.pb.h](#).

6.27.2.212 `star_v1_RebootResponse_size`

```
#define star_v1_RebootResponse_size 371
```

Definition at line 781 of file [firmware\\_update.pb.h](#).

6.27.2.213 `star_v1_RollbackRequest_CALLBACK`

```
#define star_v1_RollbackRequest_CALLBACK NULL
```

Definition at line 648 of file [firmware\\_update.pb.h](#).

6.27.2.214 `star_v1_RollbackRequest_DEFAULT`

```
#define star_v1_RollbackRequest_DEFAULT NULL
```

Definition at line 649 of file [firmware\\_update.pb.h](#).

## 6.27.2.215 star\_v1\_RollbackRequest\_FIELDLIST

```
#define star_v1_RollbackRequest_FIELDLIST(
 X,
 a)
```

Value:

X(a, STATIC, OPTIONAL, MESSAGE, header, 1)

Definition at line 646 of file [firmware\\_update.pb.h](#).

```
00646 #define star_v1_RollbackRequest_FIELDLIST(X, a) \
00647 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

## 6.27.2.216 star\_v1\_RollbackRequest\_fields

```
#define star_v1_RollbackRequest_fields &star_v1_RollbackRequest_msg
```

Definition at line 753 of file [firmware\\_update.pb.h](#).

## 6.27.2.217 star\_v1\_RollbackRequest\_header\_MSGTYPE

```
#define star_v1_RollbackRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 650 of file [firmware\\_update.pb.h](#).

## 6.27.2.218 star\_v1\_RollbackRequest\_header\_tag

```
#define star_v1_RollbackRequest_header_tag 1
```

Definition at line 472 of file [firmware\\_update.pb.h](#).

## 6.27.2.219 star\_v1\_RollbackRequest\_init\_default

```
#define star_v1_RollbackRequest_init_default {false, star_v1_RequestHeader_init_default}
```

Definition at line 402 of file [firmware\\_update.pb.h](#).

## 6.27.2.220 star\_v1\_RollbackRequest\_init\_zero

```
#define star_v1_RollbackRequest_init_zero {false, star_v1_RequestHeader_init_zero}
```

Definition at line 426 of file [firmware\\_update.pb.h](#).

## 6.27.2.221 star\_v1\_RollbackRequest\_size

```
#define star_v1_RollbackRequest_size 149
```

Definition at line 782 of file [firmware\\_update.pb.h](#).

6.27.2.222 `star_v1_RollbackResponse_CALLBACK`

```
#define star_v1_RollbackResponse_CALLBACK NULL
```

Definition at line 656 of file [firmware\\_update.pb.h](#).

6.27.2.223 `star_v1_RollbackResponse_DEFAULT`

```
#define star_v1_RollbackResponse_DEFAULT NULL
```

Definition at line 657 of file [firmware\\_update.pb.h](#).

6.27.2.224 `star_v1_RollbackResponse_FIELDLIST`

```
#define star_v1_RollbackResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, BOOL, rolling_back, 2) \
X(a, STATIC, SINGULAR, STRING, previous_version, 3)
```

Definition at line 652 of file [firmware\\_update.pb.h](#).

```
00652 #define star_v1_RollbackResponse_FIELDLIST(X, a) \
00653 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00654 X(a, STATIC, SINGULAR, BOOL, rolling_back, 2) \
00655 X(a, STATIC, SINGULAR, STRING, previous_version, 3)
```

6.27.2.225 `star_v1_RollbackResponse_fields`

```
#define star_v1_RollbackResponse_fields &star_v1_RollbackResponse_msg
```

Definition at line 754 of file [firmware\\_update.pb.h](#).

6.27.2.226 `star_v1_RollbackResponse_header_MSGTYPE`

```
#define star_v1_RollbackResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 658 of file [firmware\\_update.pb.h](#).

6.27.2.227 `star_v1_RollbackResponse_header_tag`

```
#define star_v1_RollbackResponse_header_tag 1
```

Definition at line 473 of file [firmware\\_update.pb.h](#).

6.27.2.228 `star_v1_RollbackResponse_init_default`

```
#define star_v1_RollbackResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, ""}
```

Definition at line 403 of file [firmware\\_update.pb.h](#).

6.27.2.229 `star_v1_RollbackResponse_init_zero`

```
#define star_v1_RollbackResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, ""}
```

Definition at line 427 of file [firmware\\_update.pb.h](#).

6.27.2.230 `star_v1_RollbackResponse_previous_version_tag`

```
#define star_v1_RollbackResponse_previous_version_tag 3
```

Definition at line 475 of file [firmware\\_update.pb.h](#).

6.27.2.231 `star_v1_RollbackResponse_rolling_back_tag`

```
#define star_v1_RollbackResponse_rolling_back_tag 2
```

Definition at line 474 of file [firmware\\_update.pb.h](#).

6.27.2.232 `star_v1_RollbackResponse_size`

```
#define star_v1_RollbackResponse_size 398
```

Definition at line 783 of file [firmware\\_update.pb.h](#).

6.27.2.233 `STAR_V1_STAR_V1_FIRMWARE_UPDATE_PB_H_MAX_SIZE`

```
#define STAR_V1_STAR_V1_FIRMWARE_UPDATE_PB_H_MAX_SIZE star_v1_WriteChunkRequest_size
```

Definition at line 763 of file [firmware\\_update.pb.h](#).

6.27.2.234 `star_v1_StreamChunksResponse_CALLBACK`

```
#define star_v1_StreamChunksResponse_CALLBACK NULL
```

Definition at line 562 of file [firmware\\_update.pb.h](#).

6.27.2.235 `star_v1_StreamChunksResponse_chunks_received_tag`

```
#define star_v1_StreamChunksResponse_chunks_received_tag 2
```

Definition at line 503 of file [firmware\\_update.pb.h](#).

6.27.2.236 `star_v1_StreamChunksResponse_DEFAULT`

```
#define star_v1_StreamChunksResponse_DEFAULT NULL
```

Definition at line 563 of file [firmware\\_update.pb.h](#).

6.27.2.237 `star_v1_StreamChunksResponse_error_MSGTYPE`

```
#define star_v1_StreamChunksResponse_error_MSGTYPE star_v1_FirmwareUpdateError
```

Definition at line 566 of file [firmware\\_update.pb.h](#).

6.27.2.238 `star_v1_StreamChunksResponse_error_tag`

```
#define star_v1_StreamChunksResponse_error_tag 4
```

Definition at line 505 of file [firmware\\_update.pb.h](#).

6.27.2.239 `star_v1_StreamChunksResponse_FIELDLIST`

```
#define star_v1_StreamChunksResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, UINT32, chunks_received, 2) \
X(a, STATIC, OPTIONAL, MESSAGE, progress, 3) \
X(a, STATIC, OPTIONAL, MESSAGE, error, 4)
```

Definition at line 557 of file [firmware\\_update.pb.h](#).

```
00557 #define star_v1_StreamChunksResponse_FIELDLIST(X, a) \
00558 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00559 X(a, STATIC, SINGULAR, UINT32, chunks_received, 2) \
00560 X(a, STATIC, OPTIONAL, MESSAGE, progress, 3) \
00561 X(a, STATIC, OPTIONAL, MESSAGE, error, 4)
```

6.27.2.240 `star_v1_StreamChunksResponse_fields`

```
#define star_v1_StreamChunksResponse_fields &star_v1_StreamChunksResponse_msg
```

Definition at line 742 of file [firmware\\_update.pb.h](#).

6.27.2.241 `star_v1_StreamChunksResponse_header_MSGTYPE`

```
#define star_v1_StreamChunksResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 564 of file [firmware\\_update.pb.h](#).

6.27.2.242 `star_v1_StreamChunksResponse_header_tag`

```
#define star_v1_StreamChunksResponse_header_tag 1
```

Definition at line 502 of file [firmware\\_update.pb.h](#).

6.27.2.243 `star_v1_StreamChunksResponse_init_default`

```
#define star_v1_StreamChunksResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, false, star_v1_FirmwareUpdateProgress_init_default, false, star_v1_FirmwareUpdateError_init_default}
```

Definition at line 391 of file [firmware\\_update.pb.h](#).

6.27.2.244 `star_v1_StreamChunksResponse_init_zero`

```
#define star_v1_StreamChunksResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, false, star_v1_FirmwareUpdateProgress_init_zero, false, star_v1_FirmwareUpdateError_init_zero}
```

Definition at line 415 of file [firmware\\_update.pb.h](#).

6.27.2.245 `star_v1_StreamChunksResponse_progress_MSGTYPE`

```
#define star_v1_StreamChunksResponse_progress_MSGTYPE star_v1_FirmwareUpdateProgress
```

Definition at line 565 of file [firmware\\_update.pb.h](#).

6.27.2.246 `star_v1_StreamChunksResponse_progress_tag`

```
#define star_v1_StreamChunksResponse_progress_tag 3
```

Definition at line 504 of file [firmware\\_update.pb.h](#).

6.27.2.247 `star_v1_StreamChunksResponse_size`

```
#define star_v1_StreamChunksResponse_size 702
```

Definition at line 784 of file [firmware\\_update.pb.h](#).

6.27.2.248 `star_v1_StreamUpdateProgressRequest_CALLBACK`

```
#define star_v1_StreamUpdateProgressRequest_CALLBACK NULL
```

Definition at line 615 of file [firmware\\_update.pb.h](#).

6.27.2.249 `star_v1_StreamUpdateProgressRequest_DEFAULT`

```
#define star_v1_StreamUpdateProgressRequest_DEFAULT NULL
```

Definition at line 616 of file [firmware\\_update.pb.h](#).

## 6.27.2.250 star\_v1\_StreamUpdateProgressRequest\_\_FIELDLIST

```
#define star_v1_StreamUpdateProgressRequest__FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, UINT32, update_interval_ms, 2)
```

Definition at line 612 of file [firmware\\_\\_update.pb.h](#).

```
00612 #define star_v1_StreamUpdateProgressRequest__FIELDLIST(X, a) \
00613 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00614 X(a, STATIC, SINGULAR, UINT32, update_interval_ms, 2)
```

## 6.27.2.251 star\_v1\_StreamUpdateProgressRequest\_\_fields

```
#define star_v1_StreamUpdateProgressRequest__fields &star_v1_StreamUpdateProgressRequest__msg
```

Definition at line 749 of file [firmware\\_\\_update.pb.h](#).

## 6.27.2.252 star\_v1\_StreamUpdateProgressRequest\_\_header\_\_MSGTYPE

```
#define star_v1_StreamUpdateProgressRequest__header__MSGTYPE star_v1_RequestHeader
```

Definition at line 617 of file [firmware\\_\\_update.pb.h](#).

## 6.27.2.253 star\_v1\_StreamUpdateProgressRequest\_\_header\_\_tag

```
#define star_v1_StreamUpdateProgressRequest__header__tag 1
```

Definition at line 452 of file [firmware\\_\\_update.pb.h](#).

## 6.27.2.254 star\_v1\_StreamUpdateProgressRequest\_\_init\_\_default

```
#define star_v1_StreamUpdateProgressRequest__init__default {false, star_v1_RequestHeader__init__default, 0}
```

Definition at line 398 of file [firmware\\_\\_update.pb.h](#).

## 6.27.2.255 star\_v1\_StreamUpdateProgressRequest\_\_init\_\_zero

```
#define star_v1_StreamUpdateProgressRequest__init__zero {false, star_v1_RequestHeader__init__zero, 0}
```

Definition at line 422 of file [firmware\\_\\_update.pb.h](#).

## 6.27.2.256 star\_v1\_StreamUpdateProgressRequest\_\_size

```
#define star_v1_StreamUpdateProgressRequest__size 155
```

Definition at line 785 of file [firmware\\_\\_update.pb.h](#).



6.27.2.257 `star_v1_StreamUpdateProgressRequest_update_interval_ms_tag`

```
#define star_v1_StreamUpdateProgressRequest_update_interval_ms_tag 2
```

Definition at line 453 of file [firmware\\_update.pb.h](#).

6.27.2.258 `star_v1_WriteChunkRequest_CALLBACK`

```
#define star_v1_WriteChunkRequest_CALLBACK NULL
```

Definition at line 535 of file [firmware\\_update.pb.h](#).

6.27.2.259 `star_v1_WriteChunkRequest_chunk_MSGTYPE`

```
#define star_v1_WriteChunkRequest_chunk_MSGTYPE star_v1_FirmwareChunk
```

Definition at line 538 of file [firmware\\_update.pb.h](#).

6.27.2.260 `star_v1_WriteChunkRequest_chunk_tag`

```
#define star_v1_WriteChunkRequest_chunk_tag 2
```

Definition at line 445 of file [firmware\\_update.pb.h](#).

6.27.2.261 `star_v1_WriteChunkRequest_DEFAULT`

```
#define star_v1_WriteChunkRequest_DEFAULT NULL
```

Definition at line 536 of file [firmware\\_update.pb.h](#).

6.27.2.262 `star_v1_WriteChunkRequest_FIELDLIST`

```
#define star_v1_WriteChunkRequest_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, chunk, 2)
```

Definition at line 532 of file [firmware\\_update.pb.h](#).

```
00532 #define star_v1_WriteChunkRequest_FIELDLIST(X, a) \
00533 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00534 X(a, STATIC, OPTIONAL, MESSAGE, chunk, 2)
```

6.27.2.263 `star_v1_WriteChunkRequest_fields`

```
#define star_v1_WriteChunkRequest_fields &star_v1_WriteChunkRequest_msg
```

Definition at line 739 of file [firmware\\_update.pb.h](#).

6.27.2.264 `star_v1_WriteChunkRequest_header_MSGTYPE`

```
#define star_v1_WriteChunkRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 537 of file [firmware\\_\\_update.pb.h](#).

6.27.2.265 `star_v1_WriteChunkRequest_header_tag`

```
#define star_v1_WriteChunkRequest_header_tag 1
```

Definition at line 444 of file [firmware\\_\\_update.pb.h](#).

6.27.2.266 `star_v1_WriteChunkRequest_init_default`

```
#define star_v1_WriteChunkRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_FirmwareChunk_init_default}
```

Definition at line 388 of file [firmware\\_\\_update.pb.h](#).

6.27.2.267 `star_v1_WriteChunkRequest_init_zero`

```
#define star_v1_WriteChunkRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_FirmwareChunk_init_zero}
```

Definition at line 412 of file [firmware\\_\\_update.pb.h](#).

6.27.2.268 `star_v1_WriteChunkRequest_size`

```
#define star_v1_WriteChunkRequest_size 1197
```

Definition at line 786 of file [firmware\\_\\_update.pb.h](#).

6.27.2.269 `star_v1_WriteChunkResponse_CALLBACK`

```
#define star_v1_WriteChunkResponse_CALLBACK NULL
```

Definition at line 544 of file [firmware\\_\\_update.pb.h](#).

6.27.2.270 `star_v1_WriteChunkResponse_DEFAULT`

```
#define star_v1_WriteChunkResponse_DEFAULT NULL
```

Definition at line 545 of file [firmware\\_\\_update.pb.h](#).

## 6.27.2.271 star\_v1\_WriteChunkResponse\_FIELDLIST

```
#define star_v1_WriteChunkResponse_FIELDLIST(
 X,
 a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, BOOL, success, 2) \
X(a, STATIC, OPTIONAL, MESSAGE, progress, 3)
```

Definition at line 540 of file [firmware\\_update.pb.h](#).

```
00540 #define star_v1_WriteChunkResponse_FIELDLIST(X, a) \
00541 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00542 X(a, STATIC, SINGULAR, BOOL, success, 2) \
00543 X(a, STATIC, OPTIONAL, MESSAGE, progress, 3)
```

## 6.27.2.272 star\_v1\_WriteChunkResponse\_fields

```
#define star_v1_WriteChunkResponse_fields &star_v1_WriteChunkResponse_msg
```

Definition at line 740 of file [firmware\\_update.pb.h](#).

## 6.27.2.273 star\_v1\_WriteChunkResponse\_header\_MSGTYPE

```
#define star_v1_WriteChunkResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 546 of file [firmware\\_update.pb.h](#).

## 6.27.2.274 star\_v1\_WriteChunkResponse\_header\_tag

```
#define star_v1_WriteChunkResponse_header_tag 1
```

Definition at line 462 of file [firmware\\_update.pb.h](#).

## 6.27.2.275 star\_v1\_WriteChunkResponse\_init\_default

```
#define star_v1_WriteChunkResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, false, star_v1_FirmwareUpdateProgress_in
```

Definition at line 389 of file [firmware\\_update.pb.h](#).

## 6.27.2.276 star\_v1\_WriteChunkResponse\_init\_zero

```
#define star_v1_WriteChunkResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, false, star_v1_FirmwareUpdateProgress_in
```

Definition at line 413 of file [firmware\\_update.pb.h](#).

## 6.27.2.277 star\_v1\_WriteChunkResponse\_progress\_MSGTYPE

```
#define star_v1_WriteChunkResponse_progress_MSGTYPE star_v1_FirmwareUpdateProgress
```

Definition at line 547 of file [firmware\\_update.pb.h](#).

6.27.2.278 `star_v1_WriteChunkResponse_progress_tag`

```
#define star_v1_WriteChunkResponse_progress_tag 3
```

Definition at line 464 of file [firmware\\_update.pb.h](#).

6.27.2.279 `star_v1_WriteChunkResponse_size`

```
#define star_v1_WriteChunkResponse_size 421
```

Definition at line 787 of file [firmware\\_update.pb.h](#).

6.27.2.280 `star_v1_WriteChunkResponse_success_tag`

```
#define star_v1_WriteChunkResponse_success_tag 2
```

Definition at line 463 of file [firmware\\_update.pb.h](#).

## 6.27.3 Enumeration Type Documentation

6.27.3.1 `star_v1_FirmwareUpdateErrorCode`

```
enum star_v1_FirmwareUpdateErrorCode
```

Enumerator

|                                                                                             |
|---------------------------------------------------------------------------------------------|
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_UNKNOWN</code>             |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_INVALID_SIZE</code>        |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_ALREADY_IN_PROGRESS</code> |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NO_PARTITION</code>        |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NOT_STARTED</code>         |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_SIZE_EXCEEDED</code>       |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_INCOMPLETE</code>          |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_CRC_FAILED</code>          |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_VALIDATION_FAILED</code>   |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_WRITE_FAILED</code>        |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_SEQUENCE_ERROR</code>      |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_TIMEOUT</code>             |
| <code>star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NO_SPACE</code>            |

Definition at line 32 of file [firmware\\_update.pb.h](#).

```
00032 {
00033 /* Unknown error. */
00034 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_UNKNOWN = 0,
00035 /* Invalid firmware size. */
00036 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_INVALID_SIZE = 1,
00037 /* OTA already in progress. */
00038 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_ALREADY_IN_PROGRESS = 2,
00039 /* OTA partition not found. */
```

```

00040 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NO_PARTITION = 3,
00041 /* OTA not started. */
00042 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NOT_STARTED = 4,
00043 /* Size exceeds expected. */
00044 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_SIZE_EXCEEDED = 5,
00045 /* Incomplete transfer. */
00046 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_INCOMPLETE = 6,
00047 /* CRC validation failed. */
00048 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_CRC_FAILED = 7,
00049 /* Firmware validation failed. */
00050 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_VALIDATION_FAILED = 8,
00051 /* Flash write failed. */
00052 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_WRITE_FAILED = 9,
00053 /* Chunk sequence error. */
00054 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_SEQUENCE_ERROR = 10,
00055 /* Timeout during transfer. */
00056 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_TIMEOUT = 11,
00057 /* Insufficient storage space. */
00058 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NO_SPACE = 12
00059 } star_v1_FirmwareUpdateErrorCode;

```

### 6.27.3.2 star\_v1\_FirmwareUpdateState

enum [star\\_v1\\_FirmwareUpdateState](#)

Enumerator

|                                                              |  |
|--------------------------------------------------------------|--|
| star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_UNKNOWN    |  |
| star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_IDLE       |  |
| star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_RECEIVING  |  |
| star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_VALIDATING |  |
| star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_COMPLETE   |  |
| star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_ERROR      |  |

Definition at line 16 of file [firmware\\_update.pb.h](#).

```

00016 {
00017 /* Unknown state. */
00018 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_UNKNOWN = 0,
00019 /* No OTA in progress. */
00020 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_IDLE = 1,
00021 /* Receiving firmware data. */
00022 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_RECEIVING = 2,
00023 /* Validating received firmware. */
00024 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_VALIDATING = 3,
00025 /* OTA complete, ready to reboot. */
00026 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_COMPLETE = 4,
00027 /* Error occurred during OTA. */
00028 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_ERROR = 5
00029 } star_v1_FirmwareUpdateState;

```

## 6.27.4 Function Documentation

### 6.27.4.1 PB\_BYTES\_ARRAY\_T()

```

typedef PB_BYTES_ARRAY_T (
 1024)

```

## 6.27.5 Variable Documentation

### 6.27.5.1 star\_v1\_AbortUpdateRequest\_msg

const pb\_msgdesc\_t star\_v1\_AbortUpdateRequest\_msg [extern]

#### 6.27.5.2 star\_v1\_AbortUpdateResponse\_msg

const pb\_msgdesc\_t star\_v1\_AbortUpdateResponse\_msg [extern]

#### 6.27.5.3 star\_v1\_BeginUpdateRequest\_msg

const pb\_msgdesc\_t star\_v1\_BeginUpdateRequest\_msg [extern]

#### 6.27.5.4 star\_v1\_BeginUpdateResponse\_msg

const pb\_msgdesc\_t star\_v1\_BeginUpdateResponse\_msg [extern]

#### 6.27.5.5 star\_v1\_FinalizeUpdateRequest\_msg

const pb\_msgdesc\_t star\_v1\_FinalizeUpdateRequest\_msg [extern]

#### 6.27.5.6 star\_v1\_FinalizeUpdateResponse\_msg

const pb\_msgdesc\_t star\_v1\_FinalizeUpdateResponse\_msg [extern]

#### 6.27.5.7 star\_v1\_FirmwareChunk\_msg

const pb\_msgdesc\_t star\_v1\_FirmwareChunk\_msg [extern]

#### 6.27.5.8 star\_v1\_FirmwareInfo\_msg

const pb\_msgdesc\_t star\_v1\_FirmwareInfo\_msg [extern]

#### 6.27.5.9 star\_v1\_FirmwareUpdateError\_msg

const pb\_msgdesc\_t star\_v1\_FirmwareUpdateError\_msg [extern]

#### 6.27.5.10 star\_v1\_FirmwareUpdateProgress\_msg

const pb\_msgdesc\_t star\_v1\_FirmwareUpdateProgress\_msg [extern]

#### 6.27.5.11 star\_v1\_GetFirmwareInfoRequest\_msg

const pb\_msgdesc\_t star\_v1\_GetFirmwareInfoRequest\_msg [extern]

#### 6.27.5.12 star\_v1\_GetFirmwareInfoResponse\_msg

const pb\_msgdesc\_t star\_v1\_GetFirmwareInfoResponse\_msg [extern]

#### 6.27.5.13 star\_v1\_GetUpdateProgressRequest\_msg

const pb\_msgdesc\_t star\_v1\_GetUpdateProgressRequest\_msg [extern]

#### 6.27.5.14 star\_v1\_GetUpdateProgressResponse\_msg

const pb\_msgdesc\_t star\_v1\_GetUpdateProgressResponse\_msg [extern]

#### 6.27.5.15 star\_v1\_MarkValidRequest\_msg

const pb\_msgdesc\_t star\_v1\_MarkValidRequest\_msg [extern]

#### 6.27.5.16 star\_v1\_MarkValidResponse\_msg

const pb\_msgdesc\_t star\_v1\_MarkValidResponse\_msg [extern]

#### 6.27.5.17 star\_v1\_RebootRequest\_msg

const pb\_msgdesc\_t star\_v1\_RebootRequest\_msg [extern]

#### 6.27.5.18 star\_v1\_RebootResponse\_msg

const pb\_msgdesc\_t star\_v1\_RebootResponse\_msg [extern]

#### 6.27.5.19 star\_v1\_RollbackRequest\_msg

const pb\_msgdesc\_t star\_v1\_RollbackRequest\_msg [extern]

#### 6.27.5.20 star\_v1\_RollbackResponse\_msg

const pb\_msgdesc\_t star\_v1\_RollbackResponse\_msg [extern]

#### 6.27.5.21 star\_v1\_StreamChunksResponse\_msg

const pb\_msgdesc\_t star\_v1\_StreamChunksResponse\_msg [extern]

## 6.27.5.22 star\_v1\_StreamUpdateProgressRequest\_msg

```
const pb_msgdesc_t star_v1_StreamUpdateProgressRequest_msg [extern]
```

## 6.27.5.23 star\_v1\_WriteChunkRequest\_msg

```
const pb_msgdesc_t star_v1_WriteChunkRequest_msg [extern]
```

## 6.27.5.24 star\_v1\_WriteChunkResponse\_msg

```
const pb_msgdesc_t star_v1_WriteChunkResponse_msg [extern]
```

## 6.28 firmware\_update.pb.h

[Go to the documentation of this file.](#)

```
00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_STAR_V1_STAR_V1_FIRMWARE_UPDATE_PB_H_INCLUDED
00005 #define PB_STAR_V1_STAR_V1_FIRMWARE_UPDATE_PB_H_INCLUDED
00006 #include <pb.h>
00007 #include "star/v1/common.pb.h"
00008
00009 #if PB_PROTO_HEADER_VERSION != 40
00010 #error Regenerate this file with the current version of nanopb generator.
00011 #endif
00012
00013 /* Enum definitions */
00014 /* Firmware update state.
00015 Maps to star_ota_state_t. */
00016 typedef enum star_v1_FirmwareUpdateState {
00017 /* Unknown state. */
00018 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_UNKNOWN = 0,
00019 /* No OTA in progress. */
00020 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_IDLE = 1,
00021 /* Receiving firmware data. */
00022 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_RECEIVING = 2,
00023 /* Validating received firmware. */
00024 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_VALIDATING = 3,
00025 /* OTA complete, ready to reboot. */
00026 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_COMPLETE = 4,
00027 /* Error occurred during OTA. */
00028 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_ERROR = 5
00029 } star_v1_FirmwareUpdateState;
00030
00031 /* Firmware update error codes. */
00032 typedef enum star_v1_FirmwareUpdateErrorCode {
00033 /* Unknown error. */
00034 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_UNKNOWN = 0,
00035 /* Invalid firmware size. */
00036 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_INVALID_SIZE = 1,
00037 /* OTA already in progress. */
00038 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_ALREADY_IN_PROGRESS = 2,
00039 /* OTA partition not found. */
00040 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NO_PARTITION = 3,
00041 /* OTA not started. */
00042 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NOT_STARTED = 4,
00043 /* Size exceeds expected. */
00044 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_SIZE_EXCEEDED = 5,
00045 /* Incomplete transfer. */
00046 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_INCOMPLETE = 6,
00047 /* CRC validation failed. */
00048 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_CRC_FAILED = 7,
00049 /* Firmware validation failed. */
00050 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_VALIDATION_FAILED = 8,
00051 /* Flash write failed. */
00052 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_WRITE_FAILED = 9,
00053 /* Chunk sequence error. */
00054 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_SEQUENCE_ERROR = 10,
00055 /* Timeout during transfer. */

```



```

00056 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_TIMEOUT = 11,
00057 /* Insufficient storage space. */
00058 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NO_SPACE = 12
00059 } star_v1_FirmwareUpdateErrorCode;
00060
00061 /* Struct definitions */
00062 /* Request to begin OTA update. */
00063 typedef struct _star_v1_BeginUpdateRequest {
00064 /* Standard request header. */
00065 bool has_header;
00066 star_v1_RequestHeader header;
00067 /* Expected firmware size in bytes. */
00068 uint32_t firmware_size;
00069 /* Expected firmware version (for validation). */
00070 char expected_version[32];
00071 /* Expected CRC-32 checksum of complete firmware. */
00072 uint32_t expected_crc32;
00073 } star_v1_BeginUpdateRequest;
00074
00075 typedef PB_BYTES_ARRAY_T(1024) star_v1_FirmwareChunk_data_t;
00076 /* Firmware data chunk. */
00077 typedef struct _star_v1_FirmwareChunk {
00078 /* Raw firmware data bytes.
00079 Recommended size: 1024 bytes (1KB). */
00080 star_v1_FirmwareChunk_data_t data;
00081 /* Chunk sequence number (0-indexed).
00082 Used for ordering and deduplication. */
00083 uint32_t sequence;
00084 /* Byte offset in firmware image. */
00085 uint32_t offset;
00086 /* CRC-32 of this chunk (optional, for validation). */
00087 uint32_t chunk_crc32;
00088 } star_v1_FirmwareChunk;
00089
00090 /* Request to write a firmware chunk. */
00091 typedef struct _star_v1_WriteChunkRequest {
00092 /* Standard request header. */
00093 bool has_header;
00094 star_v1_RequestHeader header;
00095 /* Firmware chunk data. */
00096 bool has_chunk;
00097 star_v1_FirmwareChunk chunk;
00098 } star_v1_WriteChunkRequest;
00099
00100 /* Request to finalize update. */
00101 typedef struct _star_v1_FinalizeUpdateRequest {
00102 /* Standard request header. */
00103 bool has_header;
00104 star_v1_RequestHeader header;
00105 } star_v1_FinalizeUpdateRequest;
00106
00107 /* Request to abort update. */
00108 typedef struct _star_v1_AbortUpdateRequest {
00109 /* Standard request header. */
00110 bool has_header;
00111 star_v1_RequestHeader header;
00112 /* Reason for abort (for logging). */
00113 char reason[128];
00114 } star_v1_AbortUpdateRequest;
00115
00116 /* Response to abort request. */
00117 typedef struct _star_v1_AbortUpdateResponse {
00118 /* Standard response header with status. */
00119 bool has_header;
00120 star_v1_ResponseHeader header;
00121 /* True if abort was successful. */
00122 bool aborted;
00123 } star_v1_AbortUpdateResponse;
00124
00125 /* Request to get update progress. */
00126 typedef struct _star_v1_GetUpdateProgressRequest {
00127 /* Standard request header. */
00128 bool has_header;
00129 star_v1_RequestHeader header;
00130 } star_v1_GetUpdateProgressRequest;
00131
00132 /* Request to stream update progress. */
00133 typedef struct _star_v1_StreamUpdateProgressRequest {
00134 /* Standard request header. */
00135 bool has_header;
00136 star_v1_RequestHeader header;
00137 /* Update interval in milliseconds.
00138 Valid range: 100-5000 ms. */
00139 uint32_t update_interval_ms;
00140 } star_v1_StreamUpdateProgressRequest;
00141
00142 /* Firmware update progress information.

```

```

00143 Maps to star_ota_stats_t. */
00144 typedef struct _star_v1_FirmwareUpdateProgress {
00145 /* Total firmware size in bytes. */
00146 uint32_t total_size;
00147 /* Bytes received so far. */
00148 uint32_t bytes_received;
00149 /* Progress percentage (0-100). */
00150 int32_t progress_percent;
00151 /* Running CRC-32 checksum of received data. */
00152 uint32_t running_crc32;
00153 /* Current update state. */
00154 star_v1_FirmwareUpdateState state;
00155 /* Estimated time remaining in seconds.
00156 -1 if cannot be calculated. */
00157 int32_t estimated_seconds_remaining;
00158 /* Transfer speed in bytes per second. */
00159 uint32_t transfer_speed_bps;
00160 /* Last chunk sequence number received. */
00161 uint32_t last_sequence;
00162 } star_v1_FirmwareUpdateProgress;
00163
00164 /* Response to write chunk request. */
00165 typedef struct _star_v1_WriteChunkResponse {
00166 /* Standard response header with status. */
00167 bool has_header;
00168 star_v1_ResponseHeader header;
00169 /* True if chunk was written successfully. */
00170 bool success;
00171 /* Current progress after this chunk. */
00172 bool has_progress;
00173 star_v1_FirmwareUpdateProgress progress;
00174 } star_v1_WriteChunkResponse;
00175
00176 /* Response with update progress. */
00177 typedef struct _star_v1_GetUpdateProgressResponse {
00178 /* Standard response header with status. */
00179 bool has_header;
00180 star_v1_ResponseHeader header;
00181 /* Current update progress. */
00182 bool has_progress;
00183 star_v1_FirmwareUpdateProgress progress;
00184 } star_v1_GetUpdateProgressResponse;
00185
00186 /* Request to reboot. */
00187 typedef struct _star_v1_RebootRequest {
00188 /* Standard request header. */
00189 bool has_header;
00190 star_v1_RequestHeader header;
00191 /* Delay before reboot in milliseconds.
00192 Allows time for response to be sent.
00193 Valid range: 0-5000 ms. Default: 1000 ms. */
00194 uint32_t delay_ms;
00195 } star_v1_RebootRequest;
00196
00197 /* Response to reboot request. */
00198 typedef struct _star_v1_RebootResponse {
00199 /* Standard response header with status. */
00200 bool has_header;
00201 star_v1_ResponseHeader header;
00202 /* True if reboot will occur. */
00203 bool rebooting;
00204 /* Delay before reboot in milliseconds. */
00205 uint32_t reboot_delay_ms;
00206 } star_v1_RebootResponse;
00207
00208 /* Request to rollback firmware. */
00209 typedef struct _star_v1_RollbackRequest {
00210 /* Standard request header. */
00211 bool has_header;
00212 star_v1_RequestHeader header;
00213 } star_v1_RollbackRequest;
00214
00215 /* Response to rollback request. */
00216 typedef struct _star_v1_RollbackResponse {
00217 /* Standard response header with status. */
00218 bool has_header;
00219 star_v1_ResponseHeader header;
00220 /* True if rollback will occur. */
00221 bool rolling_back;
00222 /* Previous firmware version being restored. */
00223 char previous_version[32];
00224 } star_v1_RollbackResponse;
00225
00226 /* Request to mark firmware valid. */
00227 typedef struct _star_v1_MarkValidRequest {
00228 /* Standard request header. */
00229 bool has_header;

```

```

00230 star_v1_RequestHeader header;
00231 } star_v1_MarkValidRequest;
00232
00233 /* Response to mark valid request. */
00234 typedef struct _star_v1_MarkValidResponse {
00235 /* Standard response header with status. */
00236 bool has_header;
00237 star_v1_ResponseHeader header;
00238 /* True if firmware marked as valid. */
00239 bool marked_valid;
00240 } star_v1_MarkValidResponse;
00241
00242 /* Request for firmware information. */
00243 typedef struct _star_v1_GetFirmwareInfoRequest {
00244 /* Standard request header. */
00245 bool has_header;
00246 star_v1_RequestHeader header;
00247 } star_v1_GetFirmwareInfoRequest;
00248
00249 /* Firmware version and metadata. */
00250 typedef struct _star_v1_FirmwareInfo {
00251 /* Firmware version string (e.g., "1.0.0"). */
00252 char version[32];
00253 /* Git commit hash (short form). */
00254 char git_hash[16];
00255 /* Build date in ISO 8601 format. */
00256 char build_date[32];
00257 /* Firmware size in bytes. */
00258 uint32_t size;
00259 /* Firmware CRC-32 checksum. */
00260 uint32_t crc32;
00261 /* IDF version used to build. */
00262 char idf_version[32];
00263 /* Target chip (e.g., "esp32s3"). */
00264 char chip_target[16];
00265 } star_v1_FirmwareInfo;
00266
00267 /* Response with firmware information. */
00268 typedef struct _star_v1_GetFirmwareInfoResponse {
00269 /* Standard response header with status. */
00270 bool has_header;
00271 star_v1_ResponseHeader header;
00272 /* Current running firmware info. */
00273 bool has_current_firmware;
00274 star_v1_FirmwareInfo current_firmware;
00275 /* Previous firmware info (for rollback), if available.
00276 May be absent on devices that have never been updated. */
00277 bool has_previous_firmware;
00278 star_v1_FirmwareInfo previous_firmware;
00279 /* True if running from OTA partition. */
00280 bool is_ota_boot;
00281 /* True if firmware is marked valid. */
00282 bool is_valid;
00283 } star_v1_GetFirmwareInfoResponse;
00284
00285 /* Detailed firmware update error. */
00286 typedef struct _star_v1_FirmwareUpdateError {
00287 /* Error code for programmatic handling. */
00288 star_v1_FirmwareUpdateErrorCode code;
00289 /* Human-readable error message. */
00290 char message[128];
00291 /* Additional error details (ESP error code, etc.). */
00292 char details[128];
00293 /* Chunk sequence number where error occurred (if applicable). */
00294 uint32_t error_at_sequence;
00295 /* Byte offset where error occurred (if applicable). */
00296 uint32_t error_at_offset;
00297 } star_v1_FirmwareUpdateError;
00298
00299 /* Response to begin update request. */
00300 typedef struct _star_v1_BeginUpdateResponse {
00301 /* Standard response header with status. */
00302 bool has_header;
00303 star_v1_ResponseHeader header;
00304 /* True if update can begin. */
00305 bool accepted;
00306 /* Maximum chunk size supported (in bytes). */
00307 uint32_t max_chunk_size;
00308 /* Update session ID for tracking. */
00309 char session_id[64];
00310 /* Detailed error if not accepted. */
00311 bool has_error;
00312 star_v1_FirmwareUpdateError error;
00313 } star_v1_BeginUpdateResponse;
00314
00315 /* Response to streaming chunks. */
00316 typedef struct _star_v1_StreamChunksResponse {

```

```

00317 /* Standard response header with status. */
00318 bool has_header;
00319 star_v1_ResponseHeader header;
00320 /* Total chunks received. */
00321 uint32_t chunks_received;
00322 /* Final progress after streaming. */
00323 bool has_progress;
00324 star_v1_FirmwareUpdateProgress progress;
00325 /* Error if streaming failed. */
00326 bool has_error;
00327 star_v1_FirmwareUpdateError error;
00328 } star_v1_StreamChunksResponse;
00329
00330 /* Response to finalize request. */
00331 typedef struct _star_v1_FinalizeUpdateResponse {
00332 /* Standard response header with status. */
00333 bool has_header;
00334 star_v1_ResponseHeader header;
00335 /* True if firmware validation passed. */
00336 bool validated;
00337 /* True if ready to reboot. */
00338 bool ready_to_reboot;
00339 /* Validation error if failed. */
00340 bool has_error;
00341 star_v1_FirmwareUpdateError error;
00342 } star_v1_FinalizeUpdateResponse;
00343
00344
00345 #ifdef __cplusplus
00346 extern "C" {
00347 #endif
00348
00349 /* Helper constants for enums */
00350 #define _star_v1_FirmwareUpdateState_MIN
00351 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_UNKNOWN
00352 #define _star_v1_FirmwareUpdateState_MAX
00353 star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_ERROR
00354 #define _star_v1_FirmwareUpdateState_ARRAYSIZE
00355 ((star_v1_FirmwareUpdateState)(star_v1_FirmwareUpdateState_FIRMWARE_UPDATE_STATE_ERROR+1))
00356
00357 #define _star_v1_FirmwareUpdateErrorCode_MIN
00358 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_UNKNOWN
00359 #define _star_v1_FirmwareUpdateErrorCode_MAX
00360 star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NO_SPACE
00361 #define _star_v1_FirmwareUpdateErrorCode_ARRAYSIZE
00362 ((star_v1_FirmwareUpdateErrorCode)(star_v1_FirmwareUpdateErrorCode_FIRMWARE_UPDATE_ERROR_CODE_NO_SPACE+1))
00363
00364
00365
00366
00367
00368
00369
00370
00371 #define star_v1_FirmwareUpdateProgress_state_ENUMTYPE star_v1_FirmwareUpdateState
00372
00373
00374
00375
00376
00377
00378
00379
00380
00381
00382 #define star_v1_FirmwareUpdateError_code_ENUMTYPE star_v1_FirmwareUpdateErrorCode
00383
00384
00385 /* Initializer values for message structs */
00386 #define star_v1_BeginUpdateRequest_init_default {false, star_v1_RequestHeader_init_default, 0, "", 0}
00387 #define star_v1_BeginUpdateResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0, "", false,
00388 star_v1_FirmwareUpdateError_init_default}
00389 #define star_v1_WriteChunkRequest_init_default {false, star_v1_RequestHeader_init_default, false,
00390 star_v1_FirmwareChunk_init_default}
00391 #define star_v1_WriteChunkResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, false,
00392 star_v1_FirmwareUpdateProgress_init_default}
00393 #define star_v1_FirmwareChunk_init_default {{0, {0}}, 0, 0, 0}
00394 #define star_v1_StreamChunksResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, false,
00395 star_v1_FirmwareUpdateProgress_init_default, false, star_v1_FirmwareUpdateError_init_default}
00396 #define star_v1_FinalizeUpdateRequest_init_default {false, star_v1_RequestHeader_init_default}
00397 #define star_v1_FinalizeUpdateResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0, false,

```

```

 star_v1_FirmwareUpdateError_init_default}
00394 #define star_v1_AbortUpdateRequest_init_default {false, star_v1_RequestHeader_init_default, ""}
00395 #define star_v1_AbortUpdateResponse_init_default {false, star_v1_ResponseHeader_init_default, 0}
00396 #define star_v1_GetUpdateProgressRequest_init_default {false, star_v1_RequestHeader_init_default}
00397 #define star_v1_GetUpdateProgressResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
 star_v1_FirmwareUpdateProgress_init_default}
00398 #define star_v1_StreamUpdateProgressRequest_init_default {false, star_v1_RequestHeader_init_default, 0}
00399 #define star_v1_FirmwareUpdateProgress_init_default {0, 0, 0, 0, _star_v1_FirmwareUpdateState_MIN, 0, 0, 0}
00400 #define star_v1_RebootRequest_init_default {false, star_v1_RequestHeader_init_default, 0}
00401 #define star_v1_RebootResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0}
00402 #define star_v1_RollbackRequest_init_default {false, star_v1_RequestHeader_init_default}
00403 #define star_v1_RollbackResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, ""}
00404 #define star_v1_MarkValidRequest_init_default {false, star_v1_RequestHeader_init_default}
00405 #define star_v1_MarkValidResponse_init_default {false, star_v1_ResponseHeader_init_default, 0}
00406 #define star_v1_GetFirmwareInfoRequest_init_default {false, star_v1_RequestHeader_init_default}
00407 #define star_v1_GetFirmwareInfoResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
 star_v1_FirmwareInfo_init_default, false, star_v1_FirmwareInfo_init_default, 0, 0}
00408 #define star_v1_FirmwareInfo_init_default {"", "", "", 0, 0, "", ""}
00409 #define star_v1_FirmwareUpdateError_init_default {_star_v1_FirmwareUpdateErrorCode_MIN, "", "", 0, 0}
00410 #define star_v1_BeginUpdateRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0, "", 0}
00411 #define star_v1_BeginUpdateResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0, "", false,
 star_v1_FirmwareUpdateError_init_zero}
00412 #define star_v1_WriteChunkRequest_init_zero {false, star_v1_RequestHeader_init_zero, false,
 star_v1_FirmwareChunk_init_zero}
00413 #define star_v1_WriteChunkResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, false,
 star_v1_FirmwareUpdateProgress_init_zero}
00414 #define star_v1_FirmwareChunk_init_zero {{0, {0}}, 0, 0, 0}
00415 #define star_v1_StreamChunksResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, false,
 star_v1_FirmwareUpdateProgress_init_zero, false, star_v1_FirmwareUpdateError_init_zero}
00416 #define star_v1_FinalizeUpdateRequest_init_zero {false, star_v1_RequestHeader_init_zero}
00417 #define star_v1_FinalizeUpdateResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0, false,
 star_v1_FirmwareUpdateError_init_zero}
00418 #define star_v1_AbortUpdateRequest_init_zero {false, star_v1_RequestHeader_init_zero, ""}
00419 #define star_v1_AbortUpdateResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0}
00420 #define star_v1_GetUpdateProgressRequest_init_zero {false, star_v1_RequestHeader_init_zero}
00421 #define star_v1_GetUpdateProgressResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false,
 star_v1_FirmwareUpdateProgress_init_zero}
00422 #define star_v1_StreamUpdateProgressRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}
00423 #define star_v1_FirmwareUpdateProgress_init_zero {0, 0, 0, 0, _star_v1_FirmwareUpdateState_MIN, 0, 0, 0}
00424 #define star_v1_RebootRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}
00425 #define star_v1_RebootResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0}
00426 #define star_v1_RollbackRequest_init_zero {false, star_v1_RequestHeader_init_zero}
00427 #define star_v1_RollbackResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, ""}
00428 #define star_v1_MarkValidRequest_init_zero {false, star_v1_RequestHeader_init_zero}
00429 #define star_v1_MarkValidResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0}
00430 #define star_v1_GetFirmwareInfoRequest_init_zero {false, star_v1_RequestHeader_init_zero}
00431 #define star_v1_GetFirmwareInfoResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false,
 star_v1_FirmwareInfo_init_zero, false, star_v1_FirmwareInfo_init_zero, 0, 0}
00432 #define star_v1_FirmwareInfo_init_zero {"", "", "", 0, 0, "", ""}
00433 #define star_v1_FirmwareUpdateError_init_zero {_star_v1_FirmwareUpdateErrorCode_MIN, "", "", 0, 0}
00434
00435 /* Field tags (for use in manual encoding/decoding) */
00436 #define star_v1_BeginUpdateRequest_header_tag 1
00437 #define star_v1_BeginUpdateRequest_firmware_size_tag 2
00438 #define star_v1_BeginUpdateRequest_expected_version_tag 3
00439 #define star_v1_BeginUpdateRequest_expected_crc32_tag 4
00440 #define star_v1_FirmwareChunk_data_tag 1
00441 #define star_v1_FirmwareChunk_sequence_tag 2
00442 #define star_v1_FirmwareChunk_offset_tag 3
00443 #define star_v1_FirmwareChunk_chunk_crc32_tag 4
00444 #define star_v1_WriteChunkRequest_header_tag 1
00445 #define star_v1_WriteChunkRequest_chunk_tag 2
00446 #define star_v1_FinalizeUpdateRequest_header_tag 1
00447 #define star_v1_AbortUpdateRequest_header_tag 1
00448 #define star_v1_AbortUpdateRequest_reason_tag 2
00449 #define star_v1_AbortUpdateResponse_header_tag 1
00450 #define star_v1_AbortUpdateResponse_aborted_tag 2
00451 #define star_v1_GetUpdateProgressRequest_header_tag 1
00452 #define star_v1_StreamUpdateProgressRequest_header_tag 1
00453 #define star_v1_StreamUpdateProgressRequest_update_interval_ms_tag 2
00454 #define star_v1_FirmwareUpdateProgress_total_size_tag 1
00455 #define star_v1_FirmwareUpdateProgress_bytes_received_tag 2
00456 #define star_v1_FirmwareUpdateProgress_progress_percent_tag 3
00457 #define star_v1_FirmwareUpdateProgress_running_crc32_tag 4
00458 #define star_v1_FirmwareUpdateProgress_state_tag 5
00459 #define star_v1_FirmwareUpdateProgress_estimated_seconds_remaining_tag 6
00460 #define star_v1_FirmwareUpdateProgress_transfer_speed_bps_tag 7
00461 #define star_v1_FirmwareUpdateProgress_last_sequence_tag 8
00462 #define star_v1_WriteChunkResponse_header_tag 1
00463 #define star_v1_WriteChunkResponse_success_tag 2
00464 #define star_v1_WriteChunkResponse_progress_tag 3
00465 #define star_v1_GetUpdateProgressResponse_header_tag 1
00466 #define star_v1_GetUpdateProgressResponse_progress_tag 2
00467 #define star_v1_RebootRequest_header_tag 1
00468 #define star_v1_RebootRequest_delay_ms_tag 2
00469 #define star_v1_RebootResponse_header_tag 1
00470 #define star_v1_RebootResponse_rebooting_tag 2

```

```

00471 #define star_v1_RebootResponse_reboot_delay_ms_tag 3
00472 #define star_v1_RollbackRequest_header_tag 1
00473 #define star_v1_RollbackResponse_header_tag 1
00474 #define star_v1_RollbackResponse_rolling_back_tag 2
00475 #define star_v1_RollbackResponse_previous_version_tag 3
00476 #define star_v1_MarkValidRequest_header_tag 1
00477 #define star_v1_MarkValidResponse_header_tag 1
00478 #define star_v1_MarkValidResponse_marked_valid_tag 2
00479 #define star_v1_GetFirmwareInfoRequest_header_tag 1
00480 #define star_v1_FirmwareInfo_version_tag 1
00481 #define star_v1_FirmwareInfo_git_hash_tag 2
00482 #define star_v1_FirmwareInfo_build_date_tag 3
00483 #define star_v1_FirmwareInfo_size_tag 4
00484 #define star_v1_FirmwareInfo_crc32_tag 5
00485 #define star_v1_FirmwareInfo_idf_version_tag 6
00486 #define star_v1_FirmwareInfo_chip_target_tag 7
00487 #define star_v1_GetFirmwareInfoResponse_header_tag 1
00488 #define star_v1_GetFirmwareInfoResponse_current_firmware_tag 2
00489 #define star_v1_GetFirmwareInfoResponse_previous_firmware_tag 3
00490 #define star_v1_GetFirmwareInfoResponse_is_ota_boot_tag 4
00491 #define star_v1_GetFirmwareInfoResponse_is_valid_tag 5
00492 #define star_v1_FirmwareUpdateError_code_tag 1
00493 #define star_v1_FirmwareUpdateError_message_tag 2
00494 #define star_v1_FirmwareUpdateError_details_tag 3
00495 #define star_v1_FirmwareUpdateError_error_at_sequence_tag 4
00496 #define star_v1_FirmwareUpdateError_error_at_offset_tag 5
00497 #define star_v1_BeginUpdateResponse_header_tag 1
00498 #define star_v1_BeginUpdateResponse_accepted_tag 2
00499 #define star_v1_BeginUpdateResponse_max_chunk_size_tag 3
00500 #define star_v1_BeginUpdateResponse_session_id_tag 4
00501 #define star_v1_BeginUpdateResponse_error_tag 5
00502 #define star_v1_StreamChunksResponse_header_tag 1
00503 #define star_v1_StreamChunksResponse_chunks_received_tag 2
00504 #define star_v1_StreamChunksResponse_progress_tag 3
00505 #define star_v1_StreamChunksResponse_error_tag 4
00506 #define star_v1_FinalizeUpdateResponse_header_tag 1
00507 #define star_v1_FinalizeUpdateResponse_validated_tag 2
00508 #define star_v1_FinalizeUpdateResponse_ready_to_reboot_tag 3
00509 #define star_v1_FinalizeUpdateResponse_error_tag 4
00510
00511 /* Struct field encoding specification for nanopb */
00512 #define star_v1_BeginUpdateRequest_FIELDLIST(X, a) \
00513 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00514 X(a, STATIC, SINGULAR, UINT32, firmware_size, 2) \
00515 X(a, STATIC, SINGULAR, STRING, expected_version, 3) \
00516 X(a, STATIC, SINGULAR, UINT32, expected_crc32, 4)
00517 #define star_v1_BeginUpdateRequest_CALLBACK NULL
00518 #define star_v1_BeginUpdateRequest_DEFAULT NULL
00519 #define star_v1_BeginUpdateRequest_header_MSGTYPE star_v1_RequestHeader
00520
00521 #define star_v1_BeginUpdateResponse_FIELDLIST(X, a) \
00522 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00523 X(a, STATIC, SINGULAR, BOOL, accepted, 2) \
00524 X(a, STATIC, SINGULAR, UINT32, max_chunk_size, 3) \
00525 X(a, STATIC, SINGULAR, STRING, session_id, 4) \
00526 X(a, STATIC, OPTIONAL, MESSAGE, error, 5)
00527 #define star_v1_BeginUpdateResponse_CALLBACK NULL
00528 #define star_v1_BeginUpdateResponse_DEFAULT NULL
00529 #define star_v1_BeginUpdateResponse_header_MSGTYPE star_v1_ResponseHeader
00530 #define star_v1_BeginUpdateResponse_error_MSGTYPE star_v1_FirmwareUpdateError
00531
00532 #define star_v1_WriteChunkRequest_FIELDLIST(X, a) \
00533 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00534 X(a, STATIC, OPTIONAL, MESSAGE, chunk, 2)
00535 #define star_v1_WriteChunkRequest_CALLBACK NULL
00536 #define star_v1_WriteChunkRequest_DEFAULT NULL
00537 #define star_v1_WriteChunkRequest_header_MSGTYPE star_v1_RequestHeader
00538 #define star_v1_WriteChunkRequest_chunk_MSGTYPE star_v1_FirmwareChunk
00539
00540 #define star_v1_WriteChunkResponse_FIELDLIST(X, a) \
00541 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00542 X(a, STATIC, SINGULAR, BOOL, success, 2) \
00543 X(a, STATIC, OPTIONAL, MESSAGE, progress, 3)
00544 #define star_v1_WriteChunkResponse_CALLBACK NULL
00545 #define star_v1_WriteChunkResponse_DEFAULT NULL
00546 #define star_v1_WriteChunkResponse_header_MSGTYPE star_v1_ResponseHeader
00547 #define star_v1_WriteChunkResponse_progress_MSGTYPE star_v1_FirmwareUpdateProgress
00548
00549 #define star_v1_FirmwareChunk_FIELDLIST(X, a) \
00550 X(a, STATIC, SINGULAR, BYTES, data, 1) \
00551 X(a, STATIC, SINGULAR, UINT32, sequence, 2) \
00552 X(a, STATIC, SINGULAR, UINT32, offset, 3) \
00553 X(a, STATIC, SINGULAR, UINT32, chunk_crc32, 4)
00554 #define star_v1_FirmwareChunk_CALLBACK NULL
00555 #define star_v1_FirmwareChunk_DEFAULT NULL
00556
00557 #define star_v1_StreamChunksResponse_FIELDLIST(X, a) \

```



```

00558 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00559 X(a, STATIC, SINGULAR, UINT32, chunks_received, 2) \
00560 X(a, STATIC, OPTIONAL, MESSAGE, progress, 3) \
00561 X(a, STATIC, OPTIONAL, MESSAGE, error, 4) \
00562 #define star_v1_StreamChunksResponse_CALLBACK NULL
00563 #define star_v1_StreamChunksResponse_DEFAULT NULL
00564 #define star_v1_StreamChunksResponse_header_MSGTYPE star_v1_ResponseHeader
00565 #define star_v1_StreamChunksResponse_progress_MSGTYPE star_v1_FirmwareUpdateProgress
00566 #define star_v1_StreamChunksResponse_error_MSGTYPE star_v1_FirmwareUpdateError
00567
00568 #define star_v1_FinalizeUpdateRequest_FIELDLIST(X, a) \
00569 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00570 #define star_v1_FinalizeUpdateRequest_CALLBACK NULL
00571 #define star_v1_FinalizeUpdateRequest_DEFAULT NULL
00572 #define star_v1_FinalizeUpdateRequest_header_MSGTYPE star_v1_RequestHeader
00573
00574 #define star_v1_FinalizeUpdateResponse_FIELDLIST(X, a) \
00575 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00576 X(a, STATIC, SINGULAR, BOOL, validated, 2) \
00577 X(a, STATIC, SINGULAR, BOOL, ready_to_reboot, 3) \
00578 X(a, STATIC, OPTIONAL, MESSAGE, error, 4) \
00579 #define star_v1_FinalizeUpdateResponse_CALLBACK NULL
00580 #define star_v1_FinalizeUpdateResponse_DEFAULT NULL
00581 #define star_v1_FinalizeUpdateResponse_header_MSGTYPE star_v1_ResponseHeader
00582 #define star_v1_FinalizeUpdateResponse_error_MSGTYPE star_v1_FirmwareUpdateError
00583
00584 #define star_v1_AbortUpdateRequest_FIELDLIST(X, a) \
00585 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00586 X(a, STATIC, SINGULAR, STRING, reason, 2) \
00587 #define star_v1_AbortUpdateRequest_CALLBACK NULL
00588 #define star_v1_AbortUpdateRequest_DEFAULT NULL
00589 #define star_v1_AbortUpdateRequest_header_MSGTYPE star_v1_RequestHeader
00590
00591 #define star_v1_AbortUpdateResponse_FIELDLIST(X, a) \
00592 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00593 X(a, STATIC, SINGULAR, BOOL, aborted, 2) \
00594 #define star_v1_AbortUpdateResponse_CALLBACK NULL
00595 #define star_v1_AbortUpdateResponse_DEFAULT NULL
00596 #define star_v1_AbortUpdateResponse_header_MSGTYPE star_v1_ResponseHeader
00597
00598 #define star_v1_GetUpdateProgressRequest_FIELDLIST(X, a) \
00599 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00600 #define star_v1_GetUpdateProgressRequest_CALLBACK NULL
00601 #define star_v1_GetUpdateProgressRequest_DEFAULT NULL
00602 #define star_v1_GetUpdateProgressRequest_header_MSGTYPE star_v1_RequestHeader
00603
00604 #define star_v1_GetUpdateProgressResponse_FIELDLIST(X, a) \
00605 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00606 X(a, STATIC, OPTIONAL, MESSAGE, progress, 2) \
00607 #define star_v1_GetUpdateProgressResponse_CALLBACK NULL
00608 #define star_v1_GetUpdateProgressResponse_DEFAULT NULL
00609 #define star_v1_GetUpdateProgressResponse_header_MSGTYPE star_v1_ResponseHeader
00610 #define star_v1_GetUpdateProgressResponse_progress_MSGTYPE star_v1_FirmwareUpdateProgress
00611
00612 #define star_v1_StreamUpdateProgressRequest_FIELDLIST(X, a) \
00613 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00614 X(a, STATIC, SINGULAR, UINT32, update_interval_ms, 2) \
00615 #define star_v1_StreamUpdateProgressRequest_CALLBACK NULL
00616 #define star_v1_StreamUpdateProgressRequest_DEFAULT NULL
00617 #define star_v1_StreamUpdateProgressRequest_header_MSGTYPE star_v1_RequestHeader
00618
00619 #define star_v1_FirmwareUpdateProgress_FIELDLIST(X, a) \
00620 X(a, STATIC, SINGULAR, UINT32, total_size, 1) \
00621 X(a, STATIC, SINGULAR, UINT32, bytes_received, 2) \
00622 X(a, STATIC, SINGULAR, INT32, progress_percent, 3) \
00623 X(a, STATIC, SINGULAR, UINT32, running_crc32, 4) \
00624 X(a, STATIC, SINGULAR, UENUM, state, 5) \
00625 X(a, STATIC, SINGULAR, INT32, estimated_seconds_remaining, 6) \
00626 X(a, STATIC, SINGULAR, UINT32, transfer_speed_bps, 7) \
00627 X(a, STATIC, SINGULAR, UINT32, last_sequence, 8) \
00628 #define star_v1_FirmwareUpdateProgress_CALLBACK NULL
00629 #define star_v1_FirmwareUpdateProgress_DEFAULT NULL
00630
00631 #define star_v1_RebootRequest_FIELDLIST(X, a) \
00632 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00633 X(a, STATIC, SINGULAR, UINT32, delay_ms, 2) \
00634 #define star_v1_RebootRequest_CALLBACK NULL
00635 #define star_v1_RebootRequest_DEFAULT NULL
00636 #define star_v1_RebootRequest_header_MSGTYPE star_v1_RequestHeader
00637
00638 #define star_v1_RebootResponse_FIELDLIST(X, a) \
00639 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00640 X(a, STATIC, SINGULAR, BOOL, rebooting, 2) \
00641 X(a, STATIC, SINGULAR, UINT32, reboot_delay_ms, 3) \
00642 #define star_v1_RebootResponse_CALLBACK NULL
00643 #define star_v1_RebootResponse_DEFAULT NULL
00644 #define star_v1_RebootResponse_header_MSGTYPE star_v1_ResponseHeader

```

```

00645
00646 #define star_v1_RollbackRequest_FIELDLIST(X, a) \
00647 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00648 #define star_v1_RollbackRequest_CALLBACK NULL
00649 #define star_v1_RollbackRequest_DEFAULT NULL
00650 #define star_v1_RollbackRequest_header_MSGTYPE star_v1_RequestHeader
00651
00652 #define star_v1_RollbackResponse_FIELDLIST(X, a) \
00653 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00654 X(a, STATIC, SINGULAR, BOOL, rolling_back, 2) \
00655 X(a, STATIC, SINGULAR, STRING, previous_version, 3) \
00656 #define star_v1_RollbackResponse_CALLBACK NULL
00657 #define star_v1_RollbackResponse_DEFAULT NULL
00658 #define star_v1_RollbackResponse_header_MSGTYPE star_v1_ResponseHeader
00659
00660 #define star_v1_MarkValidRequest_FIELDLIST(X, a) \
00661 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00662 #define star_v1_MarkValidRequest_CALLBACK NULL
00663 #define star_v1_MarkValidRequest_DEFAULT NULL
00664 #define star_v1_MarkValidRequest_header_MSGTYPE star_v1_RequestHeader
00665
00666 #define star_v1_MarkValidResponse_FIELDLIST(X, a) \
00667 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00668 X(a, STATIC, SINGULAR, BOOL, marked_valid, 2) \
00669 #define star_v1_MarkValidResponse_CALLBACK NULL
00670 #define star_v1_MarkValidResponse_DEFAULT NULL
00671 #define star_v1_MarkValidResponse_header_MSGTYPE star_v1_ResponseHeader
00672
00673 #define star_v1_GetFirmwareInfoRequest_FIELDLIST(X, a) \
00674 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00675 #define star_v1_GetFirmwareInfoRequest_CALLBACK NULL
00676 #define star_v1_GetFirmwareInfoRequest_DEFAULT NULL
00677 #define star_v1_GetFirmwareInfoRequest_header_MSGTYPE star_v1_RequestHeader
00678
00679 #define star_v1_GetFirmwareInfoResponse_FIELDLIST(X, a) \
00680 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00681 X(a, STATIC, OPTIONAL, MESSAGE, current_firmware, 2) \
00682 X(a, STATIC, OPTIONAL, MESSAGE, previous_firmware, 3) \
00683 X(a, STATIC, SINGULAR, BOOL, is_ota_boot, 4) \
00684 X(a, STATIC, SINGULAR, BOOL, is_valid, 5) \
00685 #define star_v1_GetFirmwareInfoResponse_CALLBACK NULL
00686 #define star_v1_GetFirmwareInfoResponse_DEFAULT NULL
00687 #define star_v1_GetFirmwareInfoResponse_header_MSGTYPE star_v1_ResponseHeader
00688 #define star_v1_GetFirmwareInfoResponse_current_firmware_MSGTYPE star_v1_FirmwareInfo
00689 #define star_v1_GetFirmwareInfoResponse_previous_firmware_MSGTYPE star_v1_FirmwareInfo
00690
00691 #define star_v1_FirmwareInfo_FIELDLIST(X, a) \
00692 X(a, STATIC, SINGULAR, STRING, version, 1) \
00693 X(a, STATIC, SINGULAR, STRING, git_hash, 2) \
00694 X(a, STATIC, SINGULAR, STRING, build_date, 3) \
00695 X(a, STATIC, SINGULAR, UINT32, size, 4) \
00696 X(a, STATIC, SINGULAR, UINT32, crc32, 5) \
00697 X(a, STATIC, SINGULAR, STRING, idf_version, 6) \
00698 X(a, STATIC, SINGULAR, STRING, chip_target, 7) \
00699 #define star_v1_FirmwareInfo_CALLBACK NULL
00700 #define star_v1_FirmwareInfo_DEFAULT NULL
00701
00702 #define star_v1_FirmwareUpdateError_FIELDLIST(X, a) \
00703 X(a, STATIC, SINGULAR, UENUM, code, 1) \
00704 X(a, STATIC, SINGULAR, STRING, message, 2) \
00705 X(a, STATIC, SINGULAR, STRING, details, 3) \
00706 X(a, STATIC, SINGULAR, UINT32, error_at_sequence, 4) \
00707 X(a, STATIC, SINGULAR, UINT32, error_at_offset, 5) \
00708 #define star_v1_FirmwareUpdateError_CALLBACK NULL
00709 #define star_v1_FirmwareUpdateError_DEFAULT NULL
00710
00711 extern const pb_msgdesc_t star_v1_BeginUpdateRequest_msg;
00712 extern const pb_msgdesc_t star_v1_BeginUpdateResponse_msg;
00713 extern const pb_msgdesc_t star_v1_WriteChunkRequest_msg;
00714 extern const pb_msgdesc_t star_v1_WriteChunkResponse_msg;
00715 extern const pb_msgdesc_t star_v1_FirmwareChunk_msg;
00716 extern const pb_msgdesc_t star_v1_StreamChunksResponse_msg;
00717 extern const pb_msgdesc_t star_v1_FinalizeUpdateRequest_msg;
00718 extern const pb_msgdesc_t star_v1_FinalizeUpdateResponse_msg;
00719 extern const pb_msgdesc_t star_v1_AbortUpdateRequest_msg;
00720 extern const pb_msgdesc_t star_v1_AbortUpdateResponse_msg;
00721 extern const pb_msgdesc_t star_v1_GetUpdateProgressRequest_msg;
00722 extern const pb_msgdesc_t star_v1_GetUpdateProgressResponse_msg;
00723 extern const pb_msgdesc_t star_v1_StreamUpdateProgressRequest_msg;
00724 extern const pb_msgdesc_t star_v1_FirmwareUpdateProgress_msg;
00725 extern const pb_msgdesc_t star_v1_RebootRequest_msg;
00726 extern const pb_msgdesc_t star_v1_RebootResponse_msg;
00727 extern const pb_msgdesc_t star_v1_RollbackRequest_msg;
00728 extern const pb_msgdesc_t star_v1_RollbackResponse_msg;
00729 extern const pb_msgdesc_t star_v1_MarkValidRequest_msg;
00730 extern const pb_msgdesc_t star_v1_MarkValidResponse_msg;
00731 extern const pb_msgdesc_t star_v1_GetFirmwareInfoRequest_msg;

```



```

00732 extern const pb_msgdesc_t star_v1_GetFirmwareInfoResponse_msg;
00733 extern const pb_msgdesc_t star_v1_FirmwareInfo_msg;
00734 extern const pb_msgdesc_t star_v1_FirmwareUpdateError_msg;
00735
00736 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00737 #define star_v1_BeginUpdateRequest_fields &star_v1_BeginUpdateRequest_msg
00738 #define star_v1_BeginUpdateResponse_fields &star_v1_BeginUpdateResponse_msg
00739 #define star_v1_WriteChunkRequest_fields &star_v1_WriteChunkRequest_msg
00740 #define star_v1_WriteChunkResponse_fields &star_v1_WriteChunkResponse_msg
00741 #define star_v1_FirmwareChunk_fields &star_v1_FirmwareChunk_msg
00742 #define star_v1_StreamChunksResponse_fields &star_v1_StreamChunksResponse_msg
00743 #define star_v1_FinalizeUpdateRequest_fields &star_v1_FinalizeUpdateRequest_msg
00744 #define star_v1_FinalizeUpdateResponse_fields &star_v1_FinalizeUpdateResponse_msg
00745 #define star_v1_AbortUpdateRequest_fields &star_v1_AbortUpdateRequest_msg
00746 #define star_v1_AbortUpdateResponse_fields &star_v1_AbortUpdateResponse_msg
00747 #define star_v1_GetUpdateProgressRequest_fields &star_v1_GetUpdateProgressRequest_msg
00748 #define star_v1_GetUpdateProgressResponse_fields &star_v1_GetUpdateProgressResponse_msg
00749 #define star_v1_StreamUpdateProgressRequest_fields &star_v1_StreamUpdateProgressRequest_msg
00750 #define star_v1_FirmwareUpdateProgress_fields &star_v1_FirmwareUpdateProgress_msg
00751 #define star_v1_RebootRequest_fields &star_v1_RebootRequest_msg
00752 #define star_v1_RebootResponse_fields &star_v1_RebootResponse_msg
00753 #define star_v1_RollbackRequest_fields &star_v1_RollbackRequest_msg
00754 #define star_v1_RollbackResponse_fields &star_v1_RollbackResponse_msg
00755 #define star_v1_MarkValidRequest_fields &star_v1_MarkValidRequest_msg
00756 #define star_v1_MarkValidResponse_fields &star_v1_MarkValidResponse_msg
00757 #define star_v1_GetFirmwareInfoRequest_fields &star_v1_GetFirmwareInfoRequest_msg
00758 #define star_v1_GetFirmwareInfoResponse_fields &star_v1_GetFirmwareInfoResponse_msg
00759 #define star_v1_FirmwareInfo_fields &star_v1_FirmwareInfo_msg
00760 #define star_v1_FirmwareUpdateError_fields &star_v1_FirmwareUpdateError_msg
00761
00762 /* Maximum encoded size of messages (where known) */
00763 #define STAR_V1_STAR_V1_FIRMWARE_UPDATE_PB_H_MAX_SIZE star_v1_WriteChunkRequest_size
00764 #define star_v1_AbortUpdateRequest_size 279
00765 #define star_v1_AbortUpdateResponse_size 365
00766 #define star_v1_BeginUpdateRequest_size 194
00767 #define star_v1_BeginUpdateResponse_size 713
00768 #define star_v1_FinalizeUpdateRequest_size 149
00769 #define star_v1_FinalizeUpdateResponse_size 644
00770 #define star_v1_FirmwareChunk_size 1045
00771 #define star_v1_FirmwareInfo_size 145
00772 #define star_v1_FirmwareUpdateError_size 274
00773 #define star_v1_FirmwareUpdateProgress_size 54
00774 #define star_v1_GetFirmwareInfoRequest_size 149
00775 #define star_v1_GetFirmwareInfoResponse_size 663
00776 #define star_v1_GetUpdateProgressRequest_size 149
00777 #define star_v1_GetUpdateProgressResponse_size 419
00778 #define star_v1_MarkValidRequest_size 149
00779 #define star_v1_MarkValidResponse_size 365
00780 #define star_v1_RebootRequest_size 155
00781 #define star_v1_RebootResponse_size 371
00782 #define star_v1_RollbackRequest_size 149
00783 #define star_v1_RollbackResponse_size 398
00784 #define star_v1_StreamChunksResponse_size 702
00785 #define star_v1_StreamUpdateProgressRequest_size 155
00786 #define star_v1_WriteChunkRequest_size 1197
00787 #define star_v1_WriteChunkResponse_size 421
00788
00789 #ifdef __cplusplus
00790 } /* extern "C" */
00791 #endif
00792
00793 #endif

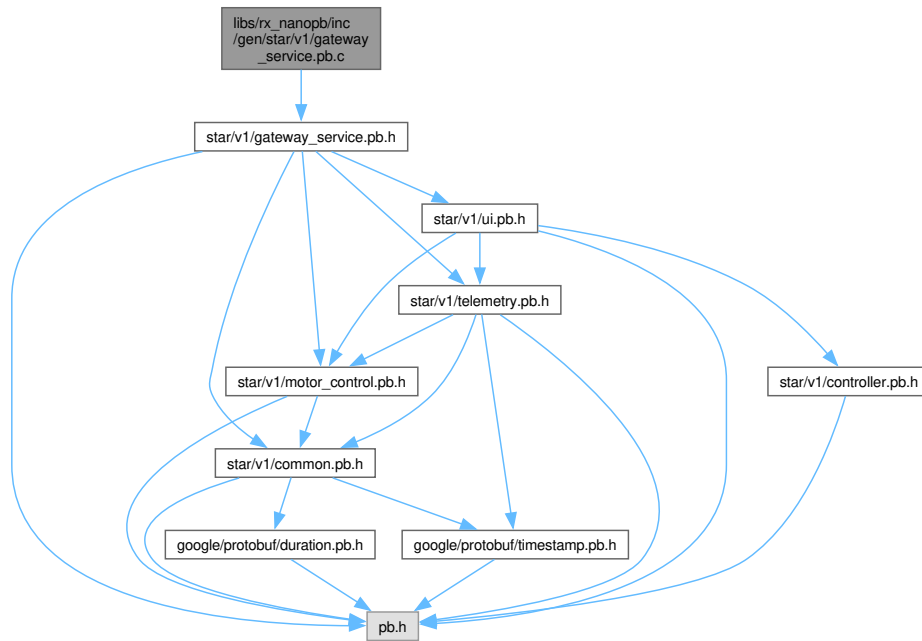
```

## 6.29 libs/rx\_nanopb/inc/gen/star/v1/gateway\_service.pb.c File Reference

```

#include "star/v1/gateway_service.pb.h"
Include dependency graph for gateway_service.pb.c:

```



## 6.30 gateway\_service.pb.c

[Go to the documentation of this file.](#)

```

00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "star/v1/gateway_service.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40
00006 #error Regenerate this file with the current version of nanopb generator.
00007 #endif
00008
00009 PB_BIND(star_v1_ForumTelemetryRequest, star_v1_ForumTelemetryRequest, 4)
00010
00011
00012 PB_BIND(star_v1_ForumTelemetryResponse, star_v1_ForumTelemetryResponse, 2)
00013
00014
00015 PB_BIND(star_v1_GetTeleopCommandRequest, star_v1_GetTeleopCommandRequest, AUTO)
00016
00017
00018 PB_BIND(star_v1_GetTeleopCommandResponse, star_v1_GetTeleopCommandResponse, 2)
00019
00020
00021 PB_BIND(star_v1_SetPidGainsRequest, star_v1_SetPidGainsRequest, AUTO)
00022
00023
00024 PB_BIND(star_v1_SetPidGainsResponse, star_v1_SetPidGainsResponse, 2)
00025
00026
00027

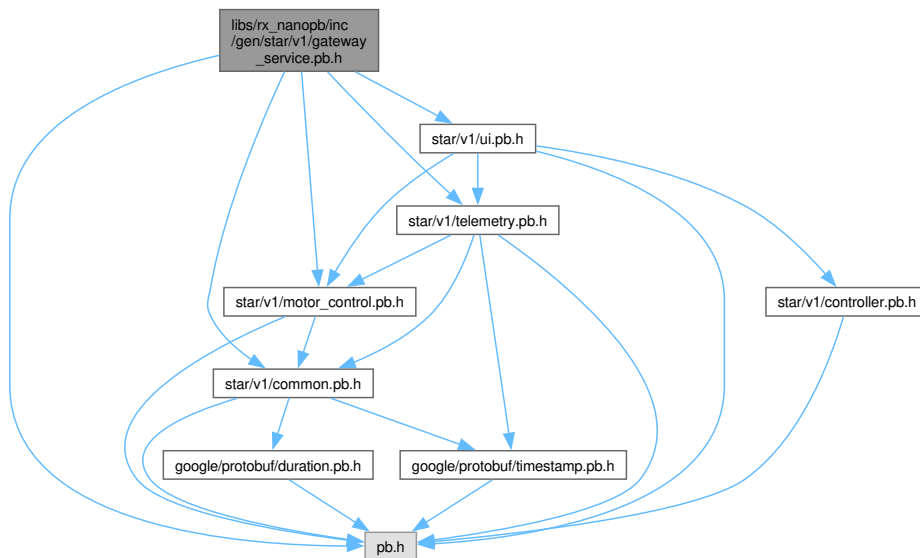
```

## 6.31 libs/rx\_nanopb/inc/gen/star/v1/gateway\_service.pb.h File Reference

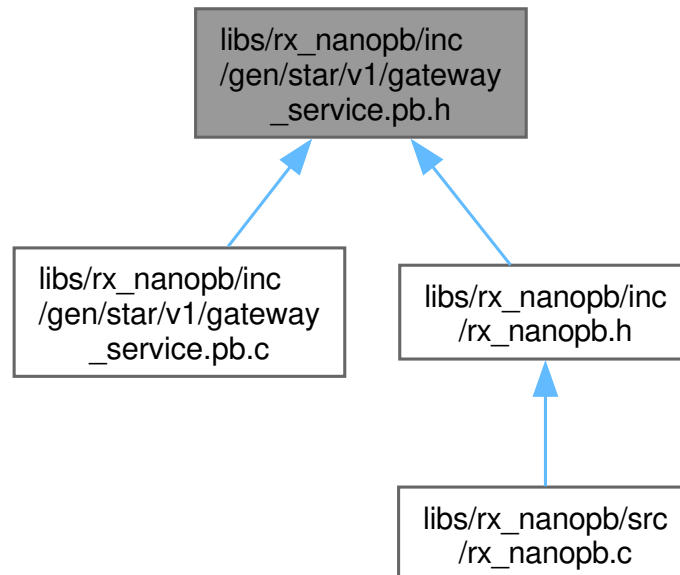
```

#include <pb.h>
#include "star/v1/common.pb.h"
#include "star/v1/motor_control.pb.h"
#include "star/v1/telemetry.pb.h"
#include "star/v1/ui.pb.h"
Include dependency graph for gateway_service.pb.h:

```



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [star\\_v1\\_ForwardTelemetryRequest](#)
- struct [star\\_v1\\_ForwardTelemetryResponse](#)
- struct [star\\_v1\\_GetTeleopCommandRequest](#)
- struct [star\\_v1\\_GetTeleopCommandResponse](#)
- struct [star\\_v1\\_SetPidGainsRequest](#)
- struct [star\\_v1\\_SetPidGainsResponse](#)

## Macros

- ```
#define star_v1_ForwardTelemetryRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_SystemStatus_init_default, false, star_v1_TelemetryData_init_default, 0, {star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default}, false, star_v1_OdometryData_init_default, false, star_v1_LidarScan_init_default}
```

- `#define star_v1_ForwardTelemetryResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0}`
- `#define star_v1_GetTeleopCommandRequest_init_default {false, star_v1_RequestHeader_init_default}`
- `#define star_v1_GetTeleopCommandResponse_init_default {false, star_v1_ResponseHeader_init_default, false, star_v1_VelocityCommand_init_default, 0, 0}`
- `#define star_v1_SetPidGainsRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_PidConfig_init_default, 0}`
- `#define star_v1_SetPidGainsResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, {{NULL}, NULL}}`
- `#define star_v1_ForwardTelemetryRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_SystemStatus_init_zero, false, star_v1_TelemetryData_init_zero, 0, {star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero}, false, star_v1_OdometryData_init_zero, false, star_v1_LidarScan_init_zero}`
- `#define star_v1_ForwardTelemetryResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0}`
- `#define star_v1_GetTeleopCommandRequest_init_zero {false, star_v1_RequestHeader_init_zero}`
- `#define star_v1_GetTeleopCommandResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_VelocityCommand_init_zero, 0, 0}`
- `#define star_v1_SetPidGainsRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_PidConfig_init_zero, 0}`
- `#define star_v1_SetPidGainsResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, {{NULL}, NULL}}`
- `#define star_v1_ForwardTelemetryRequest_header_tag 1`
- `#define star_v1_ForwardTelemetryRequest_system_status_tag 2`
- `#define star_v1_ForwardTelemetryRequest_telemetry_tag 4`
- `#define star_v1_ForwardTelemetryRequest_motor_status_tag 5`
- `#define star_v1_ForwardTelemetryRequest_odometry_tag 6`
- `#define star_v1_ForwardTelemetryRequest_lidar_scan_tag 7`
- `#define star_v1_ForwardTelemetryResponse_header_tag 1`
- `#define star_v1_ForwardTelemetryResponse_cached_tag 2`
- `#define star_v1_ForwardTelemetryResponse_active_clients_tag 3`
- `#define star_v1_GetTeleopCommandRequest_header_tag 1`
- `#define star_v1_GetTeleopCommandResponse_header_tag 1`
- `#define star_v1_GetTeleopCommandResponse_command_tag 2`
- `#define star_v1_GetTeleopCommandResponse_command_available_tag 3`
- `#define star_v1_GetTeleopCommandResponse_command_age_ms_tag 4`
- `#define star_v1_SetPidGainsRequest_header_tag 1`
- `#define star_v1_SetPidGainsRequest_pid_config_tag 2`
- `#define star_v1_SetPidGainsRequest_motor_id_tag 3`
- `#define star_v1_SetPidGainsResponse_header_tag 1`
- `#define star_v1_SetPidGainsResponse_success_tag 2`
- `#define star_v1_SetPidGainsResponse_message_tag 3`
- `#define star_v1_ForwardTelemetryRequest_FIELDLIST(X, a)`
- `#define star_v1_ForwardTelemetryRequest_CALLBACK NULL`
- `#define star_v1_ForwardTelemetryRequest_DEFAULT NULL`
- `#define star_v1_ForwardTelemetryRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_ForwardTelemetryRequest_system_status_MSGTYPE star_v1_SystemStatus`
- `#define star_v1_ForwardTelemetryRequest_telemetry_MSGTYPE star_v1_TelemetryData`
- `#define star_v1_ForwardTelemetryRequest_motor_status_MSGTYPE star_v1_MotorStatus`
- `#define star_v1_ForwardTelemetryRequest_odometry_MSGTYPE star_v1_OdometryData`
- `#define star_v1_ForwardTelemetryRequest_lidar_scan_MSGTYPE star_v1_LidarScan`
- `#define star_v1_ForwardTelemetryResponse_FIELDLIST(X, a)`
- `#define star_v1_ForwardTelemetryResponse_CALLBACK NULL`
- `#define star_v1_ForwardTelemetryResponse_DEFAULT NULL`
- `#define star_v1_ForwardTelemetryResponse_header_MSGTYPE star_v1_ResponseHeader`

- `#define star_v1_GetTeleopCommandRequest_FIELDLIST(X, a)`
- `#define star_v1_GetTeleopCommandRequest_CALLBACK NULL`
- `#define star_v1_GetTeleopCommandRequest_DEFAULT NULL`
- `#define star_v1_GetTeleopCommandRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_GetTeleopCommandResponse_FIELDLIST(X, a)`
- `#define star_v1_GetTeleopCommandResponse_CALLBACK NULL`
- `#define star_v1_GetTeleopCommandResponse_DEFAULT NULL`
- `#define star_v1_GetTeleopCommandResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_GetTeleopCommandResponse_command_MSGTYPE star_v1_VelocityCommand`
- `#define star_v1_SetPidGainsRequest_FIELDLIST(X, a)`
- `#define star_v1_SetPidGainsRequest_CALLBACK NULL`
- `#define star_v1_SetPidGainsRequest_DEFAULT NULL`
- `#define star_v1_SetPidGainsRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_SetPidGainsRequest_pid_config_MSGTYPE star_v1_PidConfig`
- `#define star_v1_SetPidGainsResponse_FIELDLIST(X, a)`
- `#define star_v1_SetPidGainsResponse_CALLBACK pb_default_field_callback`
- `#define star_v1_SetPidGainsResponse_DEFAULT NULL`
- `#define star_v1_SetPidGainsResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_ForwardTelemetryRequest_fields &star_v1_ForwardTelemetryRequest_msg`
- `#define star_v1_ForwardTelemetryResponse_fields &star_v1_ForwardTelemetryResponse_msg`
- `#define star_v1_GetTeleopCommandRequest_fields &star_v1_GetTeleopCommandRequest_msg`
- `#define star_v1_GetTeleopCommandResponse_fields &star_v1_GetTeleopCommandResponse_msg`
- `#define star_v1_SetPidGainsRequest_fields &star_v1_SetPidGainsRequest_msg`
- `#define star_v1_SetPidGainsResponse_fields &star_v1_SetPidGainsResponse_msg`
- `#define STAR_V1_STAR_V1_GATEWAY_SERVICE_PB_H_MAX_SIZE star_v1_ForwardTelemetryRequest`
- `#define star_v1_ForwardTelemetryRequest_size 8565`
- `#define star_v1_ForwardTelemetryResponse_size 376`
- `#define star_v1_GetTeleopCommandRequest_size 149`
- `#define star_v1_GetTeleopCommandResponse_size 431`
- `#define star_v1_SetPidGainsRequest_size 225`

Variables

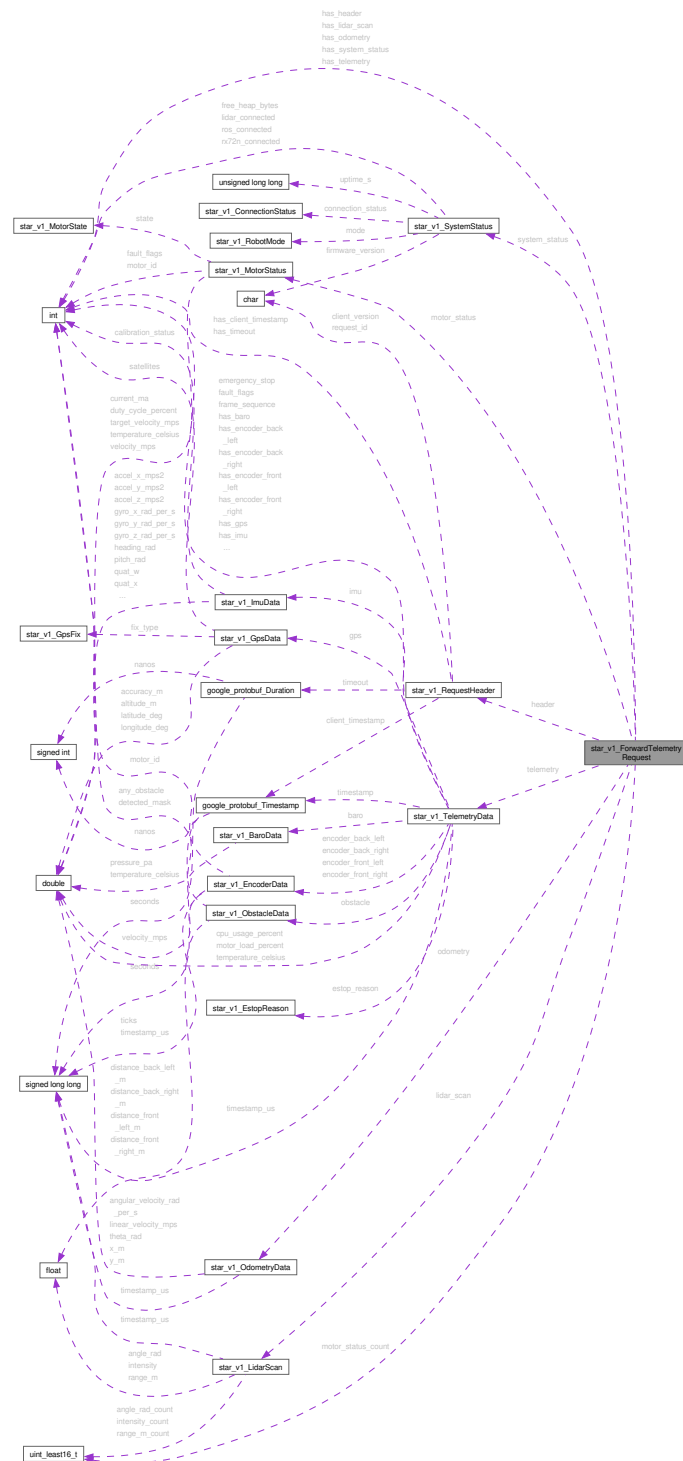
- `const pb_msgdesc_t star_v1_ForwardTelemetryRequest_msg`
- `const pb_msgdesc_t star_v1_ForwardTelemetryResponse_msg`
- `const pb_msgdesc_t star_v1_GetTeleopCommandRequest_msg`
- `const pb_msgdesc_t star_v1_GetTeleopCommandResponse_msg`
- `const pb_msgdesc_t star_v1_SetPidGainsRequest_msg`
- `const pb_msgdesc_t star_v1_SetPidGainsResponse_msg`

6.31.1 Data Structure Documentation

6.31.1.1 struct star_v1_ForwardTelemetryRequest

Definition at line 18 of file [gateway_service.pb.h](#).

Collaboration diagram for `star_v1_ForwardTelemetryRequest`:



Data Fields

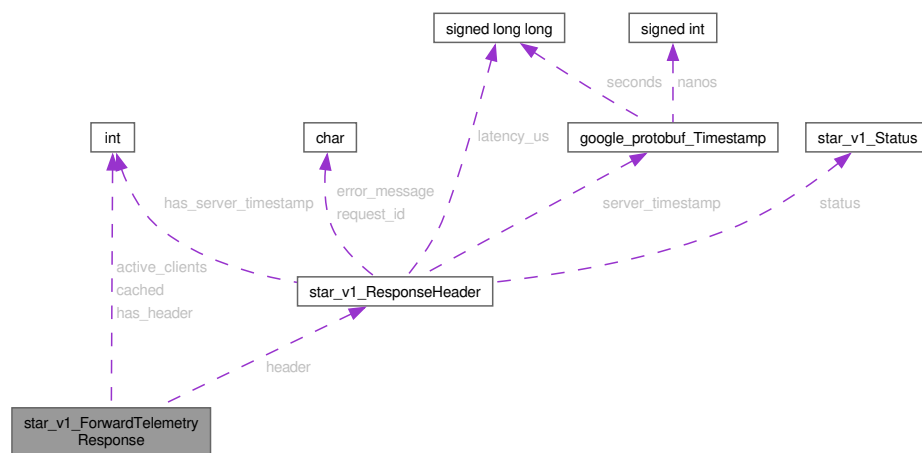
bool	has_header	
bool	has_lidar_scan	
bool	has_odometry	
bool	has_system_status	
bool	has_telemetry	
star_v1_RequestHeader	header	

star_v1_LidarScan	lidar_scan	
star_v1_MotorStatus	motor_status[4]	
pb_size_t	motor_status_count	
star_v1_OdometryData	odometry	
star_v1_SystemStatus	system_status	
star_v1_TelemetryData	telemetry	

6.31.1.2 struct star_v1_ForwardTelemetryResponse

Definition at line 52 of file [gateway_service.pb.h](#).

Collaboration diagram for star_v1_ForwardTelemetryResponse:



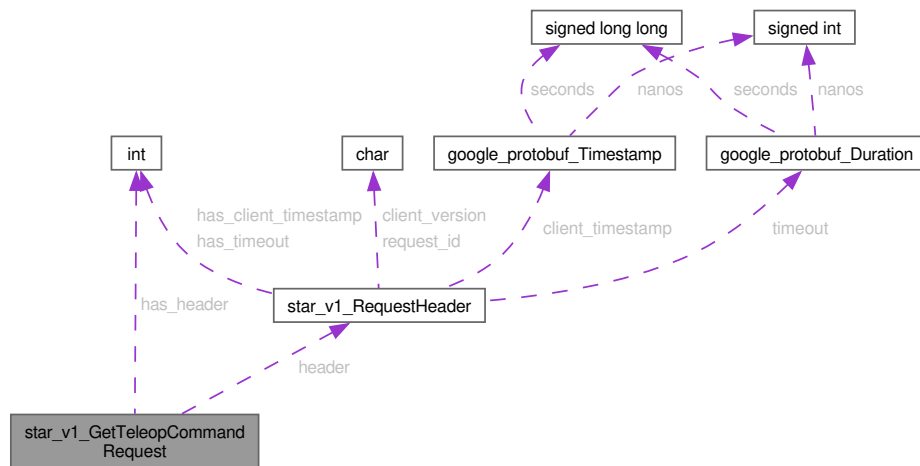
Data Fields

int32_t	active_clients	
bool	cached	
bool	has_header	
star_v1_ResponseHeader	header	

6.31.1.3 struct star_v1_GetTeleopCommandRequest

Definition at line 63 of file [gateway_service.pb.h](#).

Collaboration diagram for star_v1_GetTeleopCommandRequest:



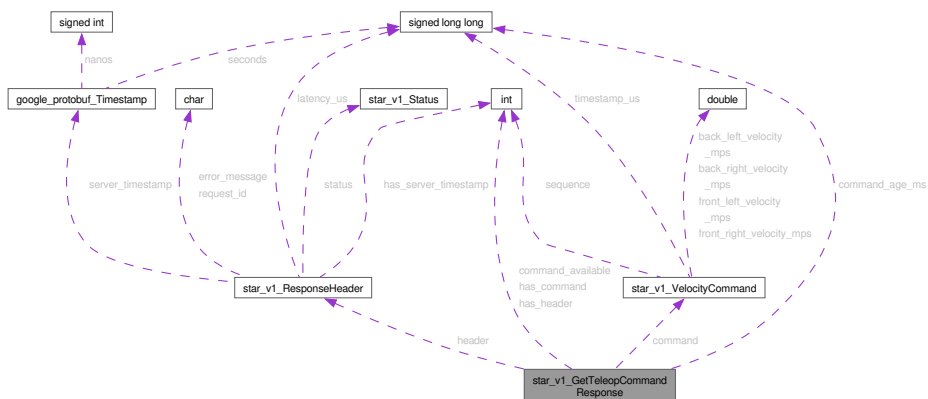
Data Fields

bool	has_header	
star_v1_RequestHeader	header	

6.31.1.4 struct star_v1_GetTeleopCommandResponse

Definition at line 70 of file [gateway_service.pb.h](#).

Collaboration diagram for star_v1_GetTeleopCommandResponse:



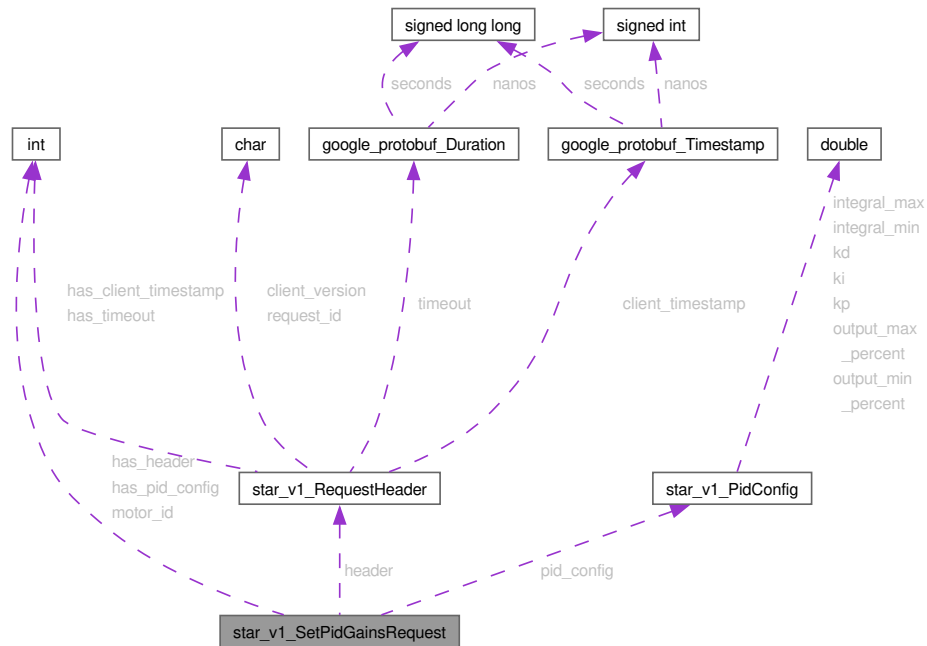
Data Fields

star_v1_VelocityCommand	command	
int64_t	command_age_ms	
bool	command_available	
bool	has_command	
bool	has_header	
star_v1_ResponseHeader	header	

6.31.1.5 struct star_v1_SetPidGainsRequest

Definition at line 86 of file [gateway_service.pb.h](#).

Collaboration diagram for star_v1_SetPidGainsRequest:



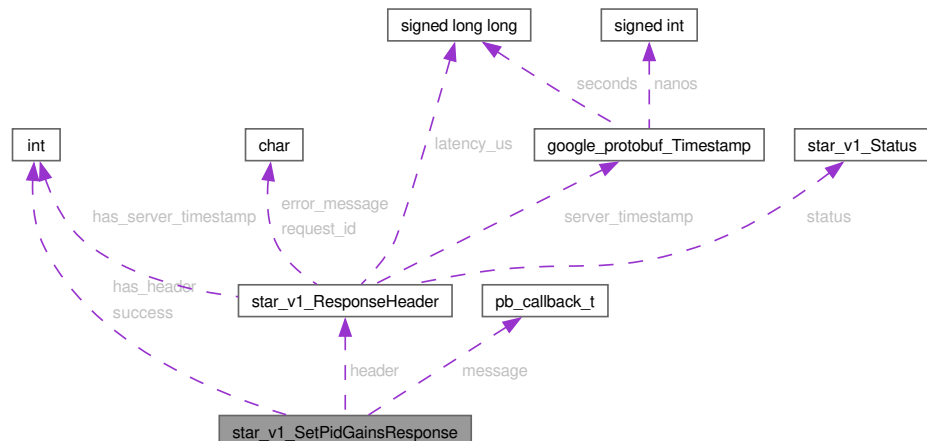
Data Fields

bool	<code>has_header</code>	
bool	<code>has_pid_config</code>	
star_v1_RequestHeader	<code>header</code>	
int32_t	<code>motor_id</code>	
star_v1_PidConfig	<code>pid_config</code>	

6.31.1.6 struct star_v1_SetPidGainsResponse

Definition at line 98 of file [gateway_service.pb.h](#).

Collaboration diagram for star_v1_SetPidGainsResponse:



Data Fields

bool	has_header	
star_v1_ResponseHeader	header	
pb_callback_t	message	
bool	success	

6.31.2 Macro Definition Documentation

6.31.2.1 star_v1_ForwardTelemetryRequest_CALLBACK

```
#define star_v1_ForwardTelemetryRequest_CALLBACK NULL
```

Definition at line 157 of file [gateway_service.pb.h](#).

6.31.2.2 star_v1_ForwardTelemetryRequest_DEFAULT

```
#define star_v1_ForwardTelemetryRequest_DEFAULT NULL
```

Definition at line 158 of file [gateway_service.pb.h](#).

6.31.2.3 star_v1_ForwardTelemetryRequest_FIELDLIST

```
#define star_v1_ForwardTelemetryRequest_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
X(a, STATIC, OPTIONAL, MESSAGE, system_status, 2) \  
X(a, STATIC, OPTIONAL, MESSAGE, telemetry, 4) \  
X(a, STATIC, REPEATED, MESSAGE, motor_status, 5) \  
X(a, STATIC, OPTIONAL, MESSAGE, odometry, 6) \  
X(a, STATIC, OPTIONAL, MESSAGE, lidar_scan, 7)
```

Definition at line 150 of file [gateway_service.pb.h](#).

```
00150 #define star_v1_ForwardTelemetryRequest_FIELDLIST(X, a) \  
00151 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
00152 X(a, STATIC, OPTIONAL, MESSAGE, system_status, 2) \  
00153 X(a, STATIC, OPTIONAL, MESSAGE, telemetry, 4) \  
00154 X(a, STATIC, REPEATED, MESSAGE, motor_status, 5) \  
00155 X(a, STATIC, OPTIONAL, MESSAGE, odometry, 6) \  
00156 X(a, STATIC, OPTIONAL, MESSAGE, lidar_scan, 7)
```

6.31.2.4 star_v1_ForwardTelemetryRequest_fields

```
#define star_v1_ForwardTelemetryRequest_fields &star_v1_ForwardTelemetryRequest_msg
```

Definition at line 215 of file [gateway_service.pb.h](#).

6.31.2.5 star_v1_ForwardTelemetryRequest_header_MSGTYPE

```
#define star_v1_ForwardTelemetryRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 159 of file [gateway_service.pb.h](#).

6.31.2.6 star_v1_ForwardTelemetryRequest_header_tag

```
#define star_v1_ForwardTelemetryRequest_header_tag 1
```

Definition at line 128 of file [gateway_service.pb.h](#).

6.31.2.7 star_v1_ForwardTelemetryRequest_init_default

```
#define star_v1_ForwardTelemetryRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_SystemStatus_init_d  
false, star_v1_TelemetryData_init_default, 0, {star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default,  
star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default}, false, star_v1_OdometryData_init_default,  
false, star_v1_LidarScan_init_default}
```

Definition at line 114 of file [gateway_service.pb.h](#).

6.31.2.8 star_v1_ForwardTelemetryRequest_init_zero

```
#define star_v1_ForwardTelemetryRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_SystemStatus_init_zero,  
false, star_v1_TelemetryData_init_zero, 0, {star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero,  
star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero}, false, star_v1_OdometryData_init_zero, false,  
star_v1_LidarScan_init_zero}
```

Definition at line 120 of file [gateway_service.pb.h](#).

6.31.2.9 star_v1_ForwardTelemetryRequest_lidar_scan_MSGTYPE

```
#define star_v1_ForwardTelemetryRequest_lidar_scan_MSGTYPE star_v1_LidarScan
```

Definition at line 164 of file [gateway_service.pb.h](#).

6.31.2.10 star_v1_ForwardTelemetryRequest_lidar_scan_tag

```
#define star_v1_ForwardTelemetryRequest_lidar_scan_tag 7
```

Definition at line 133 of file [gateway_service.pb.h](#).

6.31.2.11 star_v1_ForwardTelemetryRequest_motor_status_MSGTYPE

```
#define star_v1_ForwardTelemetryRequest_motor_status_MSGTYPE star_v1_MotorStatus
```

Definition at line 162 of file [gateway_service.pb.h](#).

6.31.2.12 star_v1_ForwardTelemetryRequest_motor_status_tag

```
#define star_v1_ForwardTelemetryRequest_motor_status_tag 5
```

Definition at line 131 of file [gateway_service.pb.h](#).

6.31.2.13 star_v1_ForwardTelemetryRequest__odometry__MSGTYPE

```
#define star_v1_ForwardTelemetryRequest__odometry__MSGTYPE star\_v1\_OdometryData
```

Definition at line [163](#) of file [gateway_service.pb.h](#).

6.31.2.14 star_v1_ForwardTelemetryRequest__odometry__tag

```
#define star_v1_ForwardTelemetryRequest__odometry__tag 6
```

Definition at line [132](#) of file [gateway_service.pb.h](#).

6.31.2.15 star_v1_ForwardTelemetryRequest__size

```
#define star_v1_ForwardTelemetryRequest__size 8565
```

Definition at line [225](#) of file [gateway_service.pb.h](#).

6.31.2.16 star_v1_ForwardTelemetryRequest__system__status__MSGTYPE

```
#define star_v1_ForwardTelemetryRequest__system__status__MSGTYPE star\_v1\_SystemStatus
```

Definition at line [160](#) of file [gateway_service.pb.h](#).

6.31.2.17 star_v1_ForwardTelemetryRequest__system__status__tag

```
#define star_v1_ForwardTelemetryRequest__system__status__tag 2
```

Definition at line [129](#) of file [gateway_service.pb.h](#).

6.31.2.18 star_v1_ForwardTelemetryRequest__telemetry__MSGTYPE

```
#define star_v1_ForwardTelemetryRequest__telemetry__MSGTYPE star\_v1\_TelemetryData
```

Definition at line [161](#) of file [gateway_service.pb.h](#).

6.31.2.19 star_v1_ForwardTelemetryRequest__telemetry__tag

```
#define star_v1_ForwardTelemetryRequest__telemetry__tag 4
```

Definition at line [130](#) of file [gateway_service.pb.h](#).

6.31.2.20 star_v1_ForwardTelemetryResponse__active__clients__tag

```
#define star_v1_ForwardTelemetryResponse__active__clients__tag 3
```

Definition at line [136](#) of file [gateway_service.pb.h](#).

6.31.2.21 `star_v1_FowardTelemetryResponse_cached_tag`

```
#define star_v1_FowardTelemetryResponse_cached_tag 2
```

Definition at line 135 of file [gateway_service.pb.h](#).

6.31.2.22 `star_v1_FowardTelemetryResponse_CALLBACK`

```
#define star_v1_FowardTelemetryResponse_CALLBACK NULL
```

Definition at line 170 of file [gateway_service.pb.h](#).

6.31.2.23 `star_v1_FowardTelemetryResponse_DEFAULT`

```
#define star_v1_FowardTelemetryResponse_DEFAULT NULL
```

Definition at line 171 of file [gateway_service.pb.h](#).

6.31.2.24 `star_v1_FowardTelemetryResponse_FIELDLIST`

```
#define star_v1_FowardTelemetryResponse_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
X(a, STATIC, SINGULAR, BOOL, cached, 2) \  
X(a, STATIC, SINGULAR, INT32, active_clients, 3)
```

Definition at line 166 of file [gateway_service.pb.h](#).

```
00166 #define star_v1_FowardTelemetryResponse_FIELDLIST(X, a) \  
00167 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
00168 X(a, STATIC, SINGULAR, BOOL, cached, 2) \  
00169 X(a, STATIC, SINGULAR, INT32, active_clients, 3)
```

6.31.2.25 `star_v1_FowardTelemetryResponse_fields`

```
#define star_v1_FowardTelemetryResponse_fields &star_v1_FowardTelemetryResponse_msg
```

Definition at line 216 of file [gateway_service.pb.h](#).

6.31.2.26 `star_v1_FowardTelemetryResponse_header_MSGTYPE`

```
#define star_v1_FowardTelemetryResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 172 of file [gateway_service.pb.h](#).

6.31.2.27 `star_v1_FowardTelemetryResponse_header_tag`

```
#define star_v1_FowardTelemetryResponse_header_tag 1
```

Definition at line 134 of file [gateway_service.pb.h](#).

6.31.2.28 `star_v1_FowardTelemetryResponse_init_default`

```
#define star_v1_FowardTelemetryResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0}
```

Definition at line 115 of file [gateway_service.pb.h](#).

6.31.2.29 `star_v1_FowardTelemetryResponse_init_zero`

```
#define star_v1_FowardTelemetryResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0}
```

Definition at line 121 of file [gateway_service.pb.h](#).

6.31.2.30 `star_v1_FowardTelemetryResponse_size`

```
#define star_v1_FowardTelemetryResponse_size 376
```

Definition at line 226 of file [gateway_service.pb.h](#).

6.31.2.31 `star_v1_GetTeleopCommandRequest_CALLBACK`

```
#define star_v1_GetTeleopCommandRequest_CALLBACK NULL
```

Definition at line 176 of file [gateway_service.pb.h](#).

6.31.2.32 `star_v1_GetTeleopCommandRequest_DEFAULT`

```
#define star_v1_GetTeleopCommandRequest_DEFAULT NULL
```

Definition at line 177 of file [gateway_service.pb.h](#).

6.31.2.33 `star_v1_GetTeleopCommandRequest_FIELDLIST`

```
#define star_v1_GetTeleopCommandRequest_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

Definition at line 174 of file [gateway_service.pb.h](#).

```
00174 #define star_v1_GetTeleopCommandRequest_FIELDLIST(X, a) \  
00175 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

6.31.2.34 `star_v1_GetTeleopCommandRequest_fields`

```
#define star_v1_GetTeleopCommandRequest_fields &star_v1_GetTeleopCommandRequest_msg
```

Definition at line 217 of file [gateway_service.pb.h](#).

6.31.2.35 star_v1_GetTeleopCommandRequest_header_MSGTYPE

```
#define star_v1_GetTeleopCommandRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 178 of file [gateway_service.pb.h](#).

6.31.2.36 star_v1_GetTeleopCommandRequest_header_tag

```
#define star_v1_GetTeleopCommandRequest_header_tag 1
```

Definition at line 137 of file [gateway_service.pb.h](#).

6.31.2.37 star_v1_GetTeleopCommandRequest_init_default

```
#define star_v1_GetTeleopCommandRequest_init_default {false, star_v1_RequestHeader_init_default}
```

Definition at line 116 of file [gateway_service.pb.h](#).

6.31.2.38 star_v1_GetTeleopCommandRequest_init_zero

```
#define star_v1_GetTeleopCommandRequest_init_zero {false, star_v1_RequestHeader_init_zero}
```

Definition at line 122 of file [gateway_service.pb.h](#).

6.31.2.39 star_v1_GetTeleopCommandRequest_size

```
#define star_v1_GetTeleopCommandRequest_size 149
```

Definition at line 227 of file [gateway_service.pb.h](#).

6.31.2.40 star_v1_GetTeleopCommandResponse_CALLBACK

```
#define star_v1_GetTeleopCommandResponse_CALLBACK NULL
```

Definition at line 185 of file [gateway_service.pb.h](#).

6.31.2.41 star_v1_GetTeleopCommandResponse_command_age_ms_tag

```
#define star_v1_GetTeleopCommandResponse_command_age_ms_tag 4
```

Definition at line 141 of file [gateway_service.pb.h](#).

6.31.2.42 star_v1_GetTeleopCommandResponse_command_available_tag

```
#define star_v1_GetTeleopCommandResponse_command_available_tag 3
```

Definition at line 140 of file [gateway_service.pb.h](#).

6.31.2.43 `star_v1_GetTeleopCommandResponse_command_MSGTYPE`

```
#define star_v1_GetTeleopCommandResponse_command_MSGTYPE star_v1_VelocityCommand
```

Definition at line 188 of file [gateway_service.pb.h](#).

6.31.2.44 `star_v1_GetTeleopCommandResponse_command_tag`

```
#define star_v1_GetTeleopCommandResponse_command_tag 2
```

Definition at line 139 of file [gateway_service.pb.h](#).

6.31.2.45 `star_v1_GetTeleopCommandResponse_DEFAULT`

```
#define star_v1_GetTeleopCommandResponse_DEFAULT NULL
```

Definition at line 186 of file [gateway_service.pb.h](#).

6.31.2.46 `star_v1_GetTeleopCommandResponse_FIELDLIST`

```
#define star_v1_GetTeleopCommandResponse_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header,      1) \
X(a, STATIC, OPTIONAL, MESSAGE, command,     2) \
X(a, STATIC, SINGULAR, BOOL,   command_available, 3) \
X(a, STATIC, SINGULAR, INT64,   command_age_ms,  4)
```

Definition at line 180 of file [gateway_service.pb.h](#).

```
00180 #define star_v1_GetTeleopCommandResponse_FIELDLIST(X, a) \
00181 X(a, STATIC, OPTIONAL, MESSAGE, header,      1) \
00182 X(a, STATIC, OPTIONAL, MESSAGE, command,     2) \
00183 X(a, STATIC, SINGULAR, BOOL,   command_available, 3) \
00184 X(a, STATIC, SINGULAR, INT64,   command_age_ms,  4)
```

6.31.2.47 `star_v1_GetTeleopCommandResponse_fields`

```
#define star_v1_GetTeleopCommandResponse_fields &star_v1_GetTeleopCommandResponse_msg
```

Definition at line 218 of file [gateway_service.pb.h](#).

6.31.2.48 `star_v1_GetTeleopCommandResponse_header_MSGTYPE`

```
#define star_v1_GetTeleopCommandResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 187 of file [gateway_service.pb.h](#).

6.31.2.49 star_v1_GetTeleopCommandResponse_header_tag

```
#define star_v1_GetTeleopCommandResponse_header_tag 1
```

Definition at line 138 of file [gateway_service.pb.h](#).

6.31.2.50 star_v1_GetTeleopCommandResponse_init_default

```
#define star_v1_GetTeleopCommandResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
star_v1_VelocityCommand_init_default, 0, 0}
```

Definition at line 117 of file [gateway_service.pb.h](#).

6.31.2.51 star_v1_GetTeleopCommandResponse_init_zero

```
#define star_v1_GetTeleopCommandResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_VelocityCommand_init_zero, 0, 0}
```

Definition at line 123 of file [gateway_service.pb.h](#).

6.31.2.52 star_v1_GetTeleopCommandResponse_size

```
#define star_v1_GetTeleopCommandResponse_size 431
```

Definition at line 228 of file [gateway_service.pb.h](#).

6.31.2.53 star_v1_SetPidGainsRequest_CALLBACK

```
#define star_v1_SetPidGainsRequest_CALLBACK NULL
```

Definition at line 194 of file [gateway_service.pb.h](#).

6.31.2.54 star_v1_SetPidGainsRequest_DEFAULT

```
#define star_v1_SetPidGainsRequest_DEFAULT NULL
```

Definition at line 195 of file [gateway_service.pb.h](#).

6.31.2.55 star_v1_SetPidGainsRequest_FIELDLIST

```
#define star_v1_SetPidGainsRequest_FIELDLIST(
    X,
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 2) \
X(a, STATIC, SINGULAR, INT32, motor_id, 3)
```

Definition at line 190 of file [gateway_service.pb.h](#).

```
00190 #define star_v1_SetPidGainsRequest_FIELDLIST(X, a) \
00191 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00192 X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 2) \
00193 X(a, STATIC, SINGULAR, INT32, motor_id, 3)
```

6.31.2.56 star_v1_SetPidGainsRequest_fields

```
#define star_v1_SetPidGainsRequest_fields &star_v1_SetPidGainsRequest_msg
```

Definition at line 219 of file [gateway_service.pb.h](#).

Referenced by [rx_nanopb_decode_pid_gains_request\(\)](#).

6.31.2.57 star_v1_SetPidGainsRequest_header_MSGTYPE

```
#define star_v1_SetPidGainsRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 196 of file [gateway_service.pb.h](#).

6.31.2.58 star_v1_SetPidGainsRequest_header_tag

```
#define star_v1_SetPidGainsRequest_header_tag 1
```

Definition at line 142 of file [gateway_service.pb.h](#).

6.31.2.59 star_v1_SetPidGainsRequest_init_default

```
#define star_v1_SetPidGainsRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_PidConfig_init_default, 0}
```

Definition at line 118 of file [gateway_service.pb.h](#).

6.31.2.60 star_v1_SetPidGainsRequest_init_zero

```
#define star_v1_SetPidGainsRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_PidConfig_init_zero, 0}
```

Definition at line 124 of file [gateway_service.pb.h](#).

Referenced by [rx_nanopb_decode_pid_gains_request\(\)](#).

6.31.2.61 star_v1_SetPidGainsRequest_motor_id_tag

```
#define star_v1_SetPidGainsRequest_motor_id_tag 3
```

Definition at line 144 of file [gateway_service.pb.h](#).

6.31.2.62 star_v1_SetPidGainsRequest_pid_config_MSGTYPE

```
#define star_v1_SetPidGainsRequest_pid_config_MSGTYPE star_v1_PidConfig
```

Definition at line 197 of file [gateway_service.pb.h](#).

6.31.2.63 star_v1_SetPidGainsRequest_pid_config_tag

```
#define star_v1_SetPidGainsRequest_pid_config_tag 2
```

Definition at line 143 of file [gateway_service.pb.h](#).

6.31.2.64 star_v1_SetPidGainsRequest_size

```
#define star_v1_SetPidGainsRequest_size 225
```

Definition at line 229 of file [gateway_service.pb.h](#).

6.31.2.65 star_v1_SetPidGainsResponse_CALLBACK

```
#define star_v1_SetPidGainsResponse_CALLBACK pb_default_field_callback
```

Definition at line 203 of file [gateway_service.pb.h](#).

6.31.2.66 star_v1_SetPidGainsResponse_DEFAULT

```
#define star_v1_SetPidGainsResponse_DEFAULT NULL
```

Definition at line 204 of file [gateway_service.pb.h](#).

6.31.2.67 star_v1_SetPidGainsResponse_FIELDLIST

```
#define star_v1_SetPidGainsResponse_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header,      1) \  
X(a, STATIC, SINGULAR, BOOL, success,        2) \  
X(a, CALLBACK, SINGULAR, STRING, message,    3)
```

Definition at line 199 of file [gateway_service.pb.h](#).

```
00199 #define star_v1_SetPidGainsResponse_FIELDLIST(X, a) \  
00200 X(a, STATIC, OPTIONAL, MESSAGE, header,      1) \  
00201 X(a, STATIC, SINGULAR, BOOL, success,        2) \  
00202 X(a, CALLBACK, SINGULAR, STRING, message,    3)
```

6.31.2.68 star_v1_SetPidGainsResponse_fields

```
#define star_v1_SetPidGainsResponse_fields &star_v1_SetPidGainsResponse_msg
```

Definition at line 220 of file [gateway_service.pb.h](#).

Referenced by [rx_nanopb_encode_pid_gains_response\(\)](#).

6.31.2.69 `star_v1_SetPidGainsResponse_header_MSGTYPE`

```
#define star_v1_SetPidGainsResponse_header_MSGTYPE star\_v1\_ResponseHeader
```

Definition at line [205](#) of file [gateway_service.pb.h](#).

6.31.2.70 `star_v1_SetPidGainsResponse_header_tag`

```
#define star_v1_SetPidGainsResponse_header_tag 1
```

Definition at line [145](#) of file [gateway_service.pb.h](#).

6.31.2.71 `star_v1_SetPidGainsResponse_init_default`

```
#define star_v1_SetPidGainsResponse_init_default {false, star\_v1\_ResponseHeader\_init\_default, 0, {{NULL},  
NULL}}
```

Definition at line [119](#) of file [gateway_service.pb.h](#).

6.31.2.72 `star_v1_SetPidGainsResponse_init_zero`

```
#define star_v1_SetPidGainsResponse_init_zero {false, star\_v1\_ResponseHeader\_init\_zero, 0, {{NULL}, NULL}}
```

Definition at line [125](#) of file [gateway_service.pb.h](#).

6.31.2.73 `star_v1_SetPidGainsResponse_message_tag`

```
#define star_v1_SetPidGainsResponse_message_tag 3
```

Definition at line [147](#) of file [gateway_service.pb.h](#).

6.31.2.74 `star_v1_SetPidGainsResponse_success_tag`

```
#define star_v1_SetPidGainsResponse_success_tag 2
```

Definition at line [146](#) of file [gateway_service.pb.h](#).

6.31.2.75 `STAR_V1_STAR_V1_GATEWAY_SERVICE_PB_H_MAX_SIZE`

```
#define STAR_V1_STAR_V1_GATEWAY_SERVICE_PB_H_MAX_SIZE star\_v1\_ForwardTelemetryRequest\_size
```

Definition at line [224](#) of file [gateway_service.pb.h](#).

6.31.3 Variable Documentation

6.31.3.1 `star_v1_ForwardTelemetryRequest_msg`

```
const pb_msgdesc_t star_v1_ForwardTelemetryRequest_msg [extern]
```

6.31.3.2 star_v1_ForwardTelemetryResponse_msg

```
const pb_msgdesc_t star_v1_ForwardTelemetryResponse_msg [extern]
```

6.31.3.3 star_v1_GetTeleopCommandRequest_msg

```
const pb_msgdesc_t star_v1_GetTeleopCommandRequest_msg [extern]
```

6.31.3.4 star_v1_GetTeleopCommandResponse_msg

```
const pb_msgdesc_t star_v1_GetTeleopCommandResponse_msg [extern]
```

6.31.3.5 star_v1_SetPidGainsRequest_msg

```
const pb_msgdesc_t star_v1_SetPidGainsRequest_msg [extern]
```

6.31.3.6 star_v1_SetPidGainsResponse_msg

```
const pb_msgdesc_t star_v1_SetPidGainsResponse_msg [extern]
```

6.32 gateway_service.pb.h

[Go to the documentation of this file.](#)

```
00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_STAR_V1_STAR_V1_GATEWAY_SERVICE_PB_H_INCLUDED
00005 #define PB_STAR_V1_STAR_V1_GATEWAY_SERVICE_PB_H_INCLUDED
00006 #include <pb.h>
00007 #include "star/v1/common.pb.h"
00008 #include "star/v1/motor_control.pb.h"
00009 #include "star/v1/telemetry.pb.h"
00010 #include "star/v1/ui.pb.h"
00011
00012 #if PB_PROTO_HEADER_VERSION != 40
00013 #error Regenerate this file with the current version of nanopb generator.
00014 #endif
00015
00016 /* Struct definitions */
00017 /* Request to forward telemetry from ROS2 to Gateway. */
00018 typedef struct _star_v1_ForwardTelemetryRequest {
00019     /* Standard request header. */
00020     bool has_header;
00021     star_v1_RequestHeader header;
00022     /* Robot system status (mode, connections, uptime).
00023 Forwarded from /robot_status ROS2 topic. */
00024     bool has_system_status;
00025     star_v1_SystemStatus system_status;
00026     /* Optional: Additional telemetry data (IMU, GPS, etc.). */
00027     bool has_telemetry;
00028     star_v1_TelemetryData telemetry;
00029     /* Per-motor status for all drive motors.
00030 Forwarded from /motor_status ROS2 topic.
00031 nanopb: max_count:4 (4-motor drive configuration; defined in gateway_service.options). */
00032     pb_size_t motor_status_count;
00033     star_v1_MotorStatus motor_status[4];
00034     /* Robot pose and velocity from wheel odometry.
00035 Forwarded from /odometry ROS2 topic.
00036 nanopb: OdometryData contains only fixed-width scalar fields (double, int64);
00037 no repeated fields or variable-length strings, so no max_size/max_count
00038 constraints are needed beyond those enforced by the message field types. */
00039     bool has_odometry;
```

```

00040     star_v1_OdometryData odometry;
00041     /* Latest LiDAR scan from RPLiDAR C1.
00042     Forwarded from /scan ROS2 topic.
00043     nanopb: LidarScan.angle_rad, LidarScan.range_m, and LidarScan.intensity are
00044     bounded by max_count:500 (RPLiDAR C1 ~500 samples per revolution at
00045     10 Hz scan mode -- 5 kHz sample rate / 10 Hz = 500 samples).
00046     Defined in ui.options and duplicated in gateway_service.options. */
00047     bool has_lidar_scan;
00048     star_v1_LidarScan lidar_scan;
00049 } star_v1_ForwardTelemetryRequest;
00050
00051 /* Response to telemetry forwarding. */
00052 typedef struct _star_v1_ForwardTelemetryResponse {
00053     /* Standard response header. */
00054     bool has_header;
00055     star_v1_ResponseHeader header;
00056     /* True if telemetry was successfully cached for UI streaming. */
00057     bool cached;
00058     /* Number of active UI WebSocket clients receiving this telemetry. */
00059     int32_t active_clients;
00060 } star_v1_ForwardTelemetryResponse;
00061
00062 /* Request for latest teleop command from Gateway. */
00063 typedef struct _star_v1_GetTeleopCommandRequest {
00064     /* Standard request header. */
00065     bool has_header;
00066     star_v1_RequestHeader header;
00067 } star_v1_GetTeleopCommandRequest;
00068
00069 /* Response with latest teleop command. */
00070 typedef struct _star_v1_GetTeleopCommandResponse {
00071     /* Standard response header. */
00072     bool has_header;
00073     star_v1_ResponseHeader header;
00074     /* Latest velocity command from UI.
00075     If no command received yet, contains zero velocities. */
00076     bool has_command;
00077     star_v1_VelocityCommand command;
00078     /* True if a valid command is available (not just zeros). */
00079     bool command_available;
00080     /* Age of command in milliseconds (for staleness checking).
00081     ROS2 should reject commands older than 500ms. */
00082     int64_t command_age_ms;
00083 } star_v1_GetTeleopCommandResponse;
00084
00085 /* Request to set PID gains for motor velocity control. */
00086 typedef struct _star_v1_SetPidGainsRequest {
00087     /* Standard request header. */
00088     bool has_header;
00089     star_v1_RequestHeader header;
00090     /* PID configuration to apply. */
00091     bool has_pid_config;
00092     star_v1_PidConfig pid_config;
00093     /* Motor ID to configure (0 = left, 1 = right, -1 = both). */
00094     int32_t motor_id;
00095 } star_v1_SetPidGainsRequest;
00096
00097 /* Response to PID gains update. */
00098 typedef struct _star_v1_SetPidGainsResponse {
00099     /* Standard response header. */
00100     bool has_header;
00101     star_v1_ResponseHeader header;
00102     /* True if gains were successfully applied to motor controller. */
00103     bool success;
00104     /* Human-readable message (e.g., "PID gains updated for both motors"). */
00105     pb_callback_t message;
00106 } star_v1_SetPidGainsResponse;
00107
00108
00109 #ifdef __cplusplus
00110 extern "C" {
00111 #endif
00112
00113 /* Initializer values for message structs */
00114 #define star_v1_ForwardTelemetryRequest_init_default {false, star_v1_RequestHeader_init_default, false,
    star_v1_SystemStatus_init_default, false, star_v1_TelemetryData_init_default, 0,
    {star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default,
    star_v1_MotorStatus_init_default}, false, star_v1_OdometryData_init_default, false,
    star_v1_LidarScan_init_default}
00115 #define star_v1_ForwardTelemetryResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, 0}
00116 #define star_v1_GetTeleopCommandRequest_init_default {false, star_v1_RequestHeader_init_default}
00117 #define star_v1_GetTeleopCommandResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
    star_v1_VelocityCommand_init_default, 0, 0}
00118 #define star_v1_SetPidGainsRequest_init_default {false, star_v1_RequestHeader_init_default, false,
    star_v1_PidConfig_init_default, 0}
00119 #define star_v1_SetPidGainsResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, {{NULL},
    NULL}}

```

```

00120 #define star_v1_ForwardTelemetryRequest_init_zero {false, star_v1_RequestHeader_init_zero, false,
    star_v1_SystemStatus_init_zero, false, star_v1_TelemetryData_init_zero, 0, {star_v1_MotorStatus_init_zero,
    star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero}, false,
    star_v1_OdometryData_init_zero, false, star_v1_LidarScan_init_zero}
00121 #define star_v1_ForwardTelemetryResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, 0}
00122 #define star_v1_GetTeleopCommandRequest_init_zero {false, star_v1_RequestHeader_init_zero}
00123 #define star_v1_GetTeleopCommandResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false,
    star_v1_VelocityCommand_init_zero, 0, 0}
00124 #define star_v1_SetPidGainsRequest_init_zero {false, star_v1_RequestHeader_init_zero, false,
    star_v1_PidConfig_init_zero, 0}
00125 #define star_v1_SetPidGainsResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, {{NULL}, NULL}}
00126
00127 /* Field tags (for use in manual encoding/decoding) */
00128 #define star_v1_ForwardTelemetryRequest_header_tag 1
00129 #define star_v1_ForwardTelemetryRequest_system_status_tag 2
00130 #define star_v1_ForwardTelemetryRequest_telemetry_tag 4
00131 #define star_v1_ForwardTelemetryRequest_motor_status_tag 5
00132 #define star_v1_ForwardTelemetryRequest_odometry_tag 6
00133 #define star_v1_ForwardTelemetryRequest_lidar_scan_tag 7
00134 #define star_v1_ForwardTelemetryResponse_header_tag 1
00135 #define star_v1_ForwardTelemetryResponse_cached_tag 2
00136 #define star_v1_ForwardTelemetryResponse_active_clients_tag 3
00137 #define star_v1_GetTeleopCommandRequest_header_tag 1
00138 #define star_v1_GetTeleopCommandResponse_header_tag 1
00139 #define star_v1_GetTeleopCommandResponse_command_tag 2
00140 #define star_v1_GetTeleopCommandResponse_command_available_tag 3
00141 #define star_v1_GetTeleopCommandResponse_command_age_ms_tag 4
00142 #define star_v1_SetPidGainsRequest_header_tag 1
00143 #define star_v1_SetPidGainsRequest_pid_config_tag 2
00144 #define star_v1_SetPidGainsRequest_motor_id_tag 3
00145 #define star_v1_SetPidGainsResponse_header_tag 1
00146 #define star_v1_SetPidGainsResponse_success_tag 2
00147 #define star_v1_SetPidGainsResponse_message_tag 3
00148
00149 /* Struct field encoding specification for nanopb */
00150 #define star_v1_ForwardTelemetryRequest_FIELDLIST(X, a) \
00151 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00152 X(a, STATIC, OPTIONAL, MESSAGE, system_status, 2) \
00153 X(a, STATIC, OPTIONAL, MESSAGE, telemetry, 4) \
00154 X(a, STATIC, REPEATED, MESSAGE, motor_status, 5) \
00155 X(a, STATIC, OPTIONAL, MESSAGE, odometry, 6) \
00156 X(a, STATIC, OPTIONAL, MESSAGE, lidar_scan, 7)
00157 #define star_v1_ForwardTelemetryRequest_CALLBACK NULL
00158 #define star_v1_ForwardTelemetryRequest_DEFAULT NULL
00159 #define star_v1_ForwardTelemetryRequest_header_MSGTYPE star_v1_RequestHeader
00160 #define star_v1_ForwardTelemetryRequest_system_status_MSGTYPE star_v1_SystemStatus
00161 #define star_v1_ForwardTelemetryRequest_telemetry_MSGTYPE star_v1_TelemetryData
00162 #define star_v1_ForwardTelemetryRequest_motor_status_MSGTYPE star_v1_MotorStatus
00163 #define star_v1_ForwardTelemetryRequest_odometry_MSGTYPE star_v1_OdometryData
00164 #define star_v1_ForwardTelemetryRequest_lidar_scan_MSGTYPE star_v1_LidarScan
00165
00166 #define star_v1_ForwardTelemetryResponse_FIELDLIST(X, a) \
00167 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00168 X(a, STATIC, SINGULAR, BOOL, cached, 2) \
00169 X(a, STATIC, SINGULAR, INT32, active_clients, 3)
00170 #define star_v1_ForwardTelemetryResponse_CALLBACK NULL
00171 #define star_v1_ForwardTelemetryResponse_DEFAULT NULL
00172 #define star_v1_ForwardTelemetryResponse_header_MSGTYPE star_v1_ResponseHeader
00173
00174 #define star_v1_GetTeleopCommandRequest_FIELDLIST(X, a) \
00175 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
00176 #define star_v1_GetTeleopCommandRequest_CALLBACK NULL
00177 #define star_v1_GetTeleopCommandRequest_DEFAULT NULL
00178 #define star_v1_GetTeleopCommandRequest_header_MSGTYPE star_v1_RequestHeader
00179
00180 #define star_v1_GetTeleopCommandResponse_FIELDLIST(X, a) \
00181 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00182 X(a, STATIC, OPTIONAL, MESSAGE, command, 2) \
00183 X(a, STATIC, SINGULAR, BOOL, command_available, 3) \
00184 X(a, STATIC, SINGULAR, INT64, command_age_ms, 4)
00185 #define star_v1_GetTeleopCommandResponse_CALLBACK NULL
00186 #define star_v1_GetTeleopCommandResponse_DEFAULT NULL
00187 #define star_v1_GetTeleopCommandResponse_header_MSGTYPE star_v1_ResponseHeader
00188 #define star_v1_GetTeleopCommandResponse_command_MSGTYPE star_v1_VelocityCommand
00189
00190 #define star_v1_SetPidGainsRequest_FIELDLIST(X, a) \
00191 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00192 X(a, STATIC, OPTIONAL, MESSAGE, pid_config, 2) \
00193 X(a, STATIC, SINGULAR, INT32, motor_id, 3)
00194 #define star_v1_SetPidGainsRequest_CALLBACK NULL
00195 #define star_v1_SetPidGainsRequest_DEFAULT NULL
00196 #define star_v1_SetPidGainsRequest_header_MSGTYPE star_v1_RequestHeader
00197 #define star_v1_SetPidGainsRequest_pid_config_MSGTYPE star_v1_PidConfig
00198
00199 #define star_v1_SetPidGainsResponse_FIELDLIST(X, a) \
00200 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00201 X(a, STATIC, SINGULAR, BOOL, success, 2) \

```

```

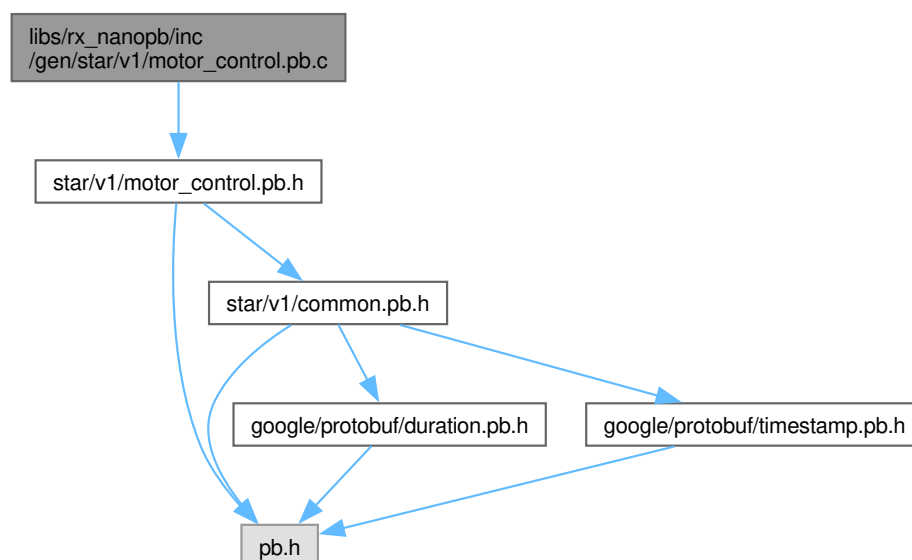
00202 X(a, CALLBACK, SINGULAR, STRING, message, 3)
00203 #define star_v1_SetPidGainsResponse_CALLBACK pb_default_field_callback
00204 #define star_v1_SetPidGainsResponse_DEFAULT NULL
00205 #define star_v1_SetPidGainsResponse_header_MSGTYPE star_v1_ResponseHeader
00206
00207 extern const pb_msgdesc_t star_v1_ForwardTelemetryRequest_msg;
00208 extern const pb_msgdesc_t star_v1_ForwardTelemetryResponse_msg;
00209 extern const pb_msgdesc_t star_v1_GetTeleopCommandRequest_msg;
00210 extern const pb_msgdesc_t star_v1_GetTeleopCommandResponse_msg;
00211 extern const pb_msgdesc_t star_v1_SetPidGainsRequest_msg;
00212 extern const pb_msgdesc_t star_v1_SetPidGainsResponse_msg;
00213
00214 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00215 #define star_v1_ForwardTelemetryRequest_fields &star_v1_ForwardTelemetryRequest_msg
00216 #define star_v1_ForwardTelemetryResponse_fields &star_v1_ForwardTelemetryResponse_msg
00217 #define star_v1_GetTeleopCommandRequest_fields &star_v1_GetTeleopCommandRequest_msg
00218 #define star_v1_GetTeleopCommandResponse_fields &star_v1_GetTeleopCommandResponse_msg
00219 #define star_v1_SetPidGainsRequest_fields &star_v1_SetPidGainsRequest_msg
00220 #define star_v1_SetPidGainsResponse_fields &star_v1_SetPidGainsResponse_msg
00221
00222 /* Maximum encoded size of messages (where known) */
00223 /* star_v1_SetPidGainsResponse_size depends on runtime parameters */
00224 #define STAR_V1_STAR_V1_GATEWAY_SERVICE_PB_H_MAX_SIZE star_v1_ForwardTelemetryRequest_size
00225 #define star_v1_ForwardTelemetryRequest_size 8565
00226 #define star_v1_ForwardTelemetryResponse_size 376
00227 #define star_v1_GetTeleopCommandRequest_size 149
00228 #define star_v1_GetTeleopCommandResponse_size 431
00229 #define star_v1_SetPidGainsRequest_size 225
00230
00231 #ifdef __cplusplus
00232 } /* extern "C" */
00233 #endif
00234
00235 #endif

```

6.33 libs/rx_nanopb/inc/gen/star/v1/motor_control.pb.c File Reference

#include "star/v1/motor_control.pb.h"

Include dependency graph for motor_control.pb.c:



6.34 motor_control.pb.c

[Go to the documentation of this file.](#)


```

00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "star/v1/motor_control.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40
00006 #error Regenerate this file with the current version of nanopb generator.
00007 #endif
00008
00009 PB_BIND(star_v1_SetVelocityRequest, star_v1_SetVelocityRequest, AUTO)
00010
00011
00012 PB_BIND(star_v1_SetVelocityResponse, star_v1_SetVelocityResponse, 2)
00013
00014
00015 PB_BIND(star_v1_VelocityCommand, star_v1_VelocityCommand, AUTO)
00016
00017
00018 PB_BIND(star_v1_EmergencyStopRequest, star_v1_EmergencyStopRequest, 2)
00019
00020
00021 PB_BIND(star_v1_EmergencyStopResponse, star_v1_EmergencyStopResponse, 2)
00022
00023
00024 PB_BIND(star_v1_SetMotorPowerRequest, star_v1_SetMotorPowerRequest, AUTO)
00025
00026
00027 PB_BIND(star_v1_SetMotorPowerResponse, star_v1_SetMotorPowerResponse, 2)
00028
00029
00030 PB_BIND(star_v1_MotorPowerCommand, star_v1_MotorPowerCommand, AUTO)
00031
00032
00033 PB_BIND(star_v1_StreamEncodersRequest, star_v1_StreamEncodersRequest, AUTO)
00034
00035
00036 PB_BIND(star_v1_EncoderData, star_v1_EncoderData, AUTO)
00037
00038
00039 PB_BIND(star_v1_MotorStatus, star_v1_MotorStatus, AUTO)
00040
00041
00042 PB_BIND(star_v1_PidConfig, star_v1_PidConfig, AUTO)
00043
00044
00045
00046
00047
00048 #ifndef PB_CONVERT_DOUBLE_FLOAT
00049 /* On some platforms (such as AVR), double is really float.
00050 * To be able to encode/decode double on these platforms, you need.
00051 * to define PB_CONVERT_DOUBLE_FLOAT in pb.h or compiler command line.
00052 */
00053 PB_STATIC_ASSERT(sizeof(double) == 8, DOUBLE_MUST_BE_8_BYTES)
00054 #endif
00055

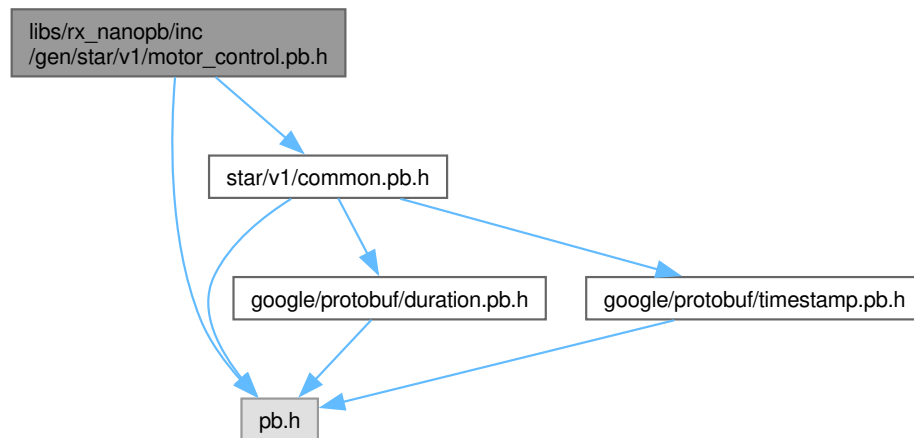
```

6.35 libs/rx_nanopb/inc/gen/star/v1/motor_control.pb.h File Reference

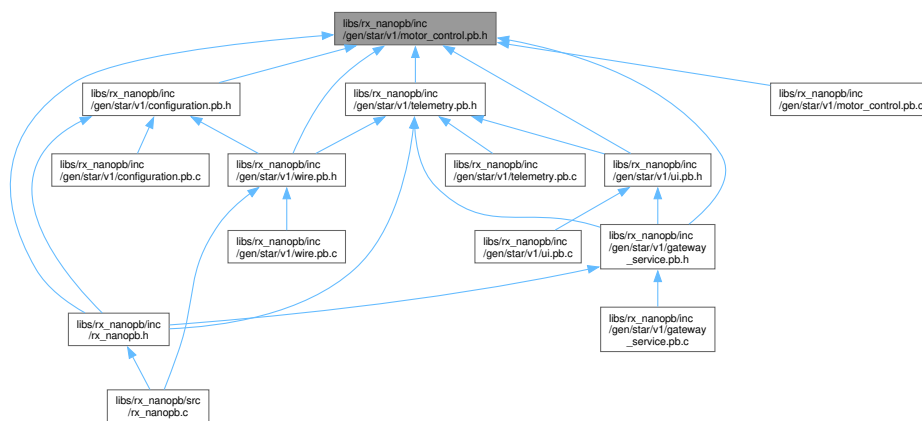
```

#include <pb.h>
#include "star/v1/common.pb.h"
Include dependency graph for motor_control.pb.h:

```



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `star_v1_VelocityCommand`
- struct `star_v1_SetVelocityRequest`
- struct `star_v1_EmergencyStopRequest`
- struct `star_v1_EmergencyStopResponse`
- struct `star_v1_SetMotorPowerResponse`
- struct `star_v1_MotorPowerCommand`
- struct `star_v1_SetMotorPowerRequest`
- struct `star_v1_StreamEncodersRequest`
- struct `star_v1_EncoderData`
- struct `star_v1_MotorStatus`
- struct `star_v1_SetVelocityResponse`
- struct `star_v1_PidConfig`

Macros

- `#define _star_v1_MotorState_MIN star_v1_MotorState_MOTOR_STATE_UNKNOWN`
- `#define _star_v1_MotorState_MAX star_v1_MotorState_MOTOR_STATE_ESTOP`
- `#define _star_v1_MotorState_ARRAYSIZE ((star_v1_MotorState)(star_v1_MotorState_MOTOR_STATE_MAX + 1))`
- `#define star_v1_MotorStatus_state_ENUMTYPE star_v1_MotorState`

- `#define star_v1_SetVelocityRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_VelocityCommand_init_default}`
- `#define star_v1_SetVelocityResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, {star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default}}`
- `#define star_v1_VelocityCommand_init_default {0, 0, 0, 0, 0, 0}`
- `#define star_v1_EmergencyStopRequest_init_default {false, star_v1_RequestHeader_init_default, ""}`
- `#define star_v1_EmergencyStopResponse_init_default {false, star_v1_ResponseHeader_init_default, 0}`
- `#define star_v1_SetMotorPowerRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_MotorPowerCommand_init_default}`
- `#define star_v1_SetMotorPowerResponse_init_default {false, star_v1_ResponseHeader_init_default}`
- `#define star_v1_MotorPowerCommand_init_default {0, 0}`
- `#define star_v1_StreamEncodersRequest_init_default {false, star_v1_RequestHeader_init_default, 0}`
- `#define star_v1_EncoderData_init_default {0, 0, 0, 0}`
- `#define star_v1_MotorStatus_init_default {0, 0, 0, 0, 0, 0, 0, _star_v1_MotorState_MIN}`
- `#define star_v1_PidConfig_init_default {0, 0, 0, 0, 0, 0, 0}`
- `#define star_v1_SetVelocityRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_VelocityCommand_init_zero}`
- `#define star_v1_SetVelocityResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, {star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero}}`
- `#define star_v1_VelocityCommand_init_zero {0, 0, 0, 0, 0, 0}`
- `#define star_v1_EmergencyStopRequest_init_zero {false, star_v1_RequestHeader_init_zero, ""}`
- `#define star_v1_EmergencyStopResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0}`
- `#define star_v1_SetMotorPowerRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_MotorPowerCommand_init_zero}`
- `#define star_v1_SetMotorPowerResponse_init_zero {false, star_v1_ResponseHeader_init_zero}`
- `#define star_v1_MotorPowerCommand_init_zero {0, 0}`
- `#define star_v1_StreamEncodersRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}`
- `#define star_v1_EncoderData_init_zero {0, 0, 0, 0}`
- `#define star_v1_MotorStatus_init_zero {0, 0, 0, 0, 0, 0, 0, _star_v1_MotorState_MIN}`
- `#define star_v1_PidConfig_init_zero {0, 0, 0, 0, 0, 0, 0}`
- `#define star_v1_VelocityCommand_front_left_velocity_mps_tag 1`
- `#define star_v1_VelocityCommand_front_right_velocity_mps_tag 2`
- `#define star_v1_VelocityCommand_sequence_tag 3`
- `#define star_v1_VelocityCommand_timestamp_us_tag 4`
- `#define star_v1_VelocityCommand_back_left_velocity_mps_tag 5`
- `#define star_v1_VelocityCommand_back_right_velocity_mps_tag 6`
- `#define star_v1_SetVelocityRequest_header_tag 1`
- `#define star_v1_SetVelocityRequest_command_tag 2`
- `#define star_v1_EmergencyStopRequest_header_tag 1`
- `#define star_v1_EmergencyStopRequest_reason_tag 2`
- `#define star_v1_EmergencyStopResponse_header_tag 1`
- `#define star_v1_EmergencyStopResponse_estop_engaged_tag 2`
- `#define star_v1_SetMotorPowerResponse_header_tag 1`
- `#define star_v1_MotorPowerCommand_motor_id_tag 1`
- `#define star_v1_MotorPowerCommand_duty_cycle_percent_tag 2`
- `#define star_v1_SetMotorPowerRequest_header_tag 1`
- `#define star_v1_SetMotorPowerRequest_command_tag 2`
- `#define star_v1_StreamEncodersRequest_header_tag 1`
- `#define star_v1_StreamEncodersRequest_rate_hz_tag 2`

- `#define star_v1_EncoderData_motor_id_tag 1`
- `#define star_v1_EncoderData_ticks_tag 2`
- `#define star_v1_EncoderData_velocity_mps_tag 3`
- `#define star_v1_EncoderData_timestamp_us_tag 4`
- `#define star_v1_MotorStatus_motor_id_tag 1`
- `#define star_v1_MotorStatus_duty_cycle_percent_tag 2`
- `#define star_v1_MotorStatus_velocity_mps_tag 3`
- `#define star_v1_MotorStatus_target_velocity_mps_tag 4`
- `#define star_v1_MotorStatus_temperature_celsius_tag 5`
- `#define star_v1_MotorStatus_current_ma_tag 6`
- `#define star_v1_MotorStatus_fault_flags_tag 7`
- `#define star_v1_MotorStatus_state_tag 8`
- `#define star_v1_SetVelocityResponse_header_tag 1`
- `#define star_v1_SetVelocityResponse_motor_status_tag 2`
- `#define star_v1_PidConfig_kp_tag 1`
- `#define star_v1_PidConfig_ki_tag 2`
- `#define star_v1_PidConfig_kd_tag 3`
- `#define star_v1_PidConfig_output_min_percent_tag 4`
- `#define star_v1_PidConfig_output_max_percent_tag 5`
- `#define star_v1_PidConfig_integral_min_tag 6`
- `#define star_v1_PidConfig_integral_max_tag 7`
- `#define star_v1_SetVelocityRequest_FIELDLIST(X, a)`
- `#define star_v1_SetVelocityRequest_CALLBACK NULL`
- `#define star_v1_SetVelocityRequest_DEFAULT NULL`
- `#define star_v1_SetVelocityRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_SetVelocityRequest_command_MSGTYPE star_v1_VelocityCommand`
- `#define star_v1_SetVelocityResponse_FIELDLIST(X, a)`
- `#define star_v1_SetVelocityResponse_CALLBACK NULL`
- `#define star_v1_SetVelocityResponse_DEFAULT NULL`
- `#define star_v1_SetVelocityResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_SetVelocityResponse_motor_status_MSGTYPE star_v1_MotorStatus`
- `#define star_v1_VelocityCommand_FIELDLIST(X, a)`
- `#define star_v1_VelocityCommand_CALLBACK NULL`
- `#define star_v1_VelocityCommand_DEFAULT NULL`
- `#define star_v1_EmergencyStopRequest_FIELDLIST(X, a)`
- `#define star_v1_EmergencyStopRequest_CALLBACK NULL`
- `#define star_v1_EmergencyStopRequest_DEFAULT NULL`
- `#define star_v1_EmergencyStopRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_EmergencyStopResponse_FIELDLIST(X, a)`
- `#define star_v1_EmergencyStopResponse_CALLBACK NULL`
- `#define star_v1_EmergencyStopResponse_DEFAULT NULL`
- `#define star_v1_EmergencyStopResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_SetMotorPowerRequest_FIELDLIST(X, a)`
- `#define star_v1_SetMotorPowerRequest_CALLBACK NULL`
- `#define star_v1_SetMotorPowerRequest_DEFAULT NULL`
- `#define star_v1_SetMotorPowerRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_SetMotorPowerRequest_command_MSGTYPE star_v1_MotorPowerCommand`
- `#define star_v1_SetMotorPowerResponse_FIELDLIST(X, a)`
- `#define star_v1_SetMotorPowerResponse_CALLBACK NULL`
- `#define star_v1_SetMotorPowerResponse_DEFAULT NULL`
- `#define star_v1_SetMotorPowerResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_MotorPowerCommand_FIELDLIST(X, a)`
- `#define star_v1_MotorPowerCommand_CALLBACK NULL`
- `#define star_v1_MotorPowerCommand_DEFAULT NULL`
- `#define star_v1_StreamEncodersRequest_FIELDLIST(X, a)`

- `#define star_v1_StreamEncodersRequest_CALLBACK NULL`
- `#define star_v1_StreamEncodersRequest_DEFAULT NULL`
- `#define star_v1_StreamEncodersRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_EncoderData_FIELDLIST(X, a)`
- `#define star_v1_EncoderData_CALLBACK NULL`
- `#define star_v1_EncoderData_DEFAULT NULL`
- `#define star_v1_MotorStatus_FIELDLIST(X, a)`
- `#define star_v1_MotorStatus_CALLBACK NULL`
- `#define star_v1_MotorStatus_DEFAULT NULL`
- `#define star_v1_PidConfig_FIELDLIST(X, a)`
- `#define star_v1_PidConfig_CALLBACK NULL`
- `#define star_v1_PidConfig_DEFAULT NULL`
- `#define star_v1_SetVelocityRequest_fields &star_v1_SetVelocityRequest_msg`
- `#define star_v1_SetVelocityResponse_fields &star_v1_SetVelocityResponse_msg`
- `#define star_v1_VelocityCommand_fields &star_v1_VelocityCommand_msg`
- `#define star_v1_EmergencyStopRequest_fields &star_v1_EmergencyStopRequest_msg`
- `#define star_v1_EmergencyStopResponse_fields &star_v1_EmergencyStopResponse_msg`
- `#define star_v1_SetMotorPowerRequest_fields &star_v1_SetMotorPowerRequest_msg`
- `#define star_v1_SetMotorPowerResponse_fields &star_v1_SetMotorPowerResponse_msg`
- `#define star_v1_MotorPowerCommand_fields &star_v1_MotorPowerCommand_msg`
- `#define star_v1_StreamEncodersRequest_fields &star_v1_StreamEncodersRequest_msg`
- `#define star_v1_EncoderData_fields &star_v1_EncoderData_msg`
- `#define star_v1_MotorStatus_fields &star_v1_MotorStatus_msg`
- `#define star_v1_PidConfig_fields &star_v1_PidConfig_msg`
- `#define STAR_V1_STAR_V1_MOTOR_CONTROL_PB_H_MAX_SIZE star_v1_SetVelocityResponse_size`
- `#define star_v1_EmergencyStopRequest_size 279`
- `#define star_v1_EmergencyStopResponse_size 365`
- `#define star_v1_EncoderData_size 42`
- `#define star_v1_MotorPowerCommand_size 20`
- `#define star_v1_MotorStatus_size 64`
- `#define star_v1_PidConfig_size 63`
- `#define star_v1_SetMotorPowerRequest_size 171`
- `#define star_v1_SetMotorPowerResponse_size 363`
- `#define star_v1_SetVelocityRequest_size 204`
- `#define star_v1_SetVelocityResponse_size 495`
- `#define star_v1_StreamEncodersRequest_size 160`
- `#define star_v1_VelocityCommand_size 53`

Enumerations

- `enum star_v1_MotorState {`
`star_v1_MotorState_MOTOR_STATE_UNKNOWN = 0, star_v1_MotorState_MOTOR_STATE_IDLE`
`= 1, star_v1_MotorState_MOTOR_STATE_RUNNING = 2, star_v1_MotorState_MOTOR_STATE_FAULT`
`= 3,`
`star_v1_MotorState_MOTOR_STATE_ESTOP = 4 }`

Variables

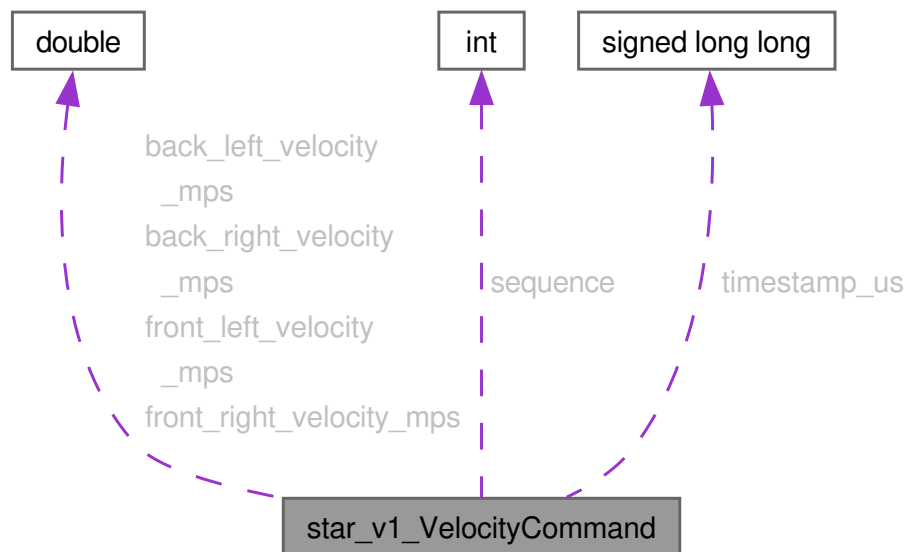
- `const pb_msgdesc_t star_v1_SetVelocityRequest_msg`
- `const pb_msgdesc_t star_v1_SetVelocityResponse_msg`
- `const pb_msgdesc_t star_v1_VelocityCommand_msg`
- `const pb_msgdesc_t star_v1_EmergencyStopRequest_msg`
- `const pb_msgdesc_t star_v1_EmergencyStopResponse_msg`
- `const pb_msgdesc_t star_v1_SetMotorPowerRequest_msg`
- `const pb_msgdesc_t star_v1_SetMotorPowerResponse_msg`
- `const pb_msgdesc_t star_v1_MotorPowerCommand_msg`
- `const pb_msgdesc_t star_v1_StreamEncodersRequest_msg`
- `const pb_msgdesc_t star_v1_EncoderData_msg`
- `const pb_msgdesc_t star_v1_MotorStatus_msg`
- `const pb_msgdesc_t star_v1_PidConfig_msg`

6.35.1 Data Structure Documentation

6.35.1.1 struct star_v1_VelocityCommand

Definition at line 45 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_VelocityCommand:



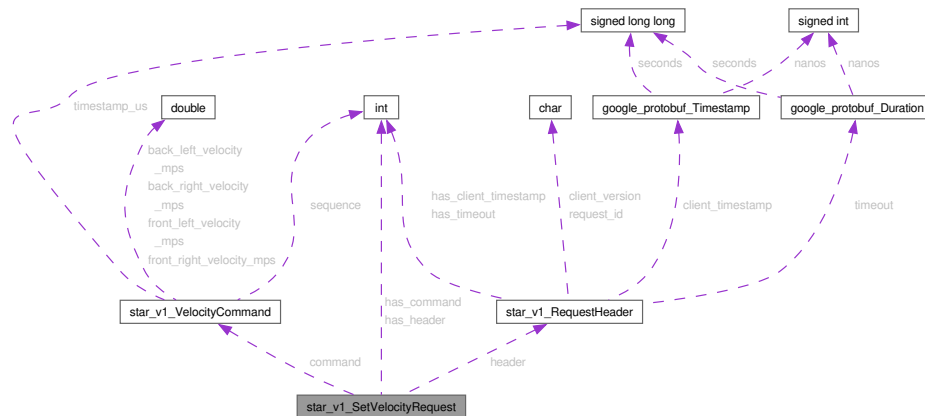
Data Fields

double	back_left_velocity_mps	
double	back_right_velocity_mps	
double	front_left_velocity_mps	
double	front_right_velocity_mps	
uint32_t	sequence	
int64_t	timestamp_us	

6.35.1.2 struct star_v1_SetVelocityRequest

Definition at line 69 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_SetVelocityRequest:



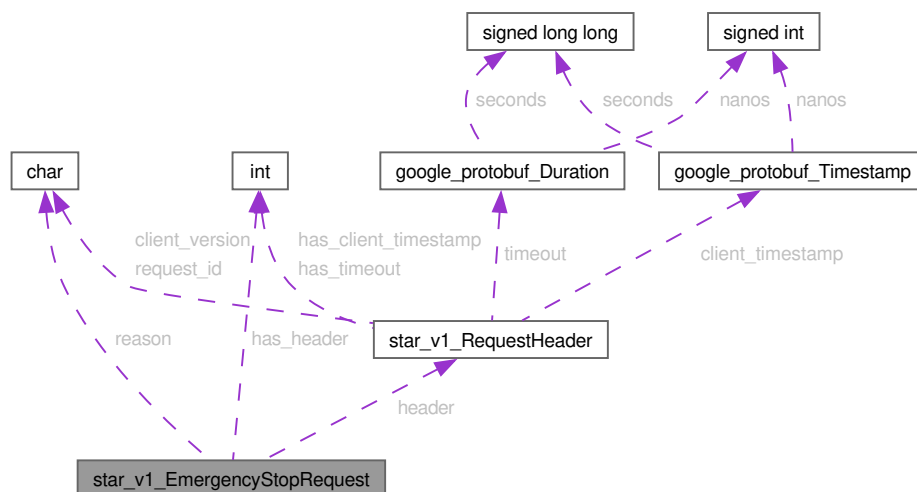
Data Fields

star_v1_VelocityCommand	command	
bool	has_command	
bool	has_header	
star_v1_RequestHeader	header	

6.35.1.3 struct star_v1_EmergencyStopRequest

Definition at line 79 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_EmergencyStopRequest:



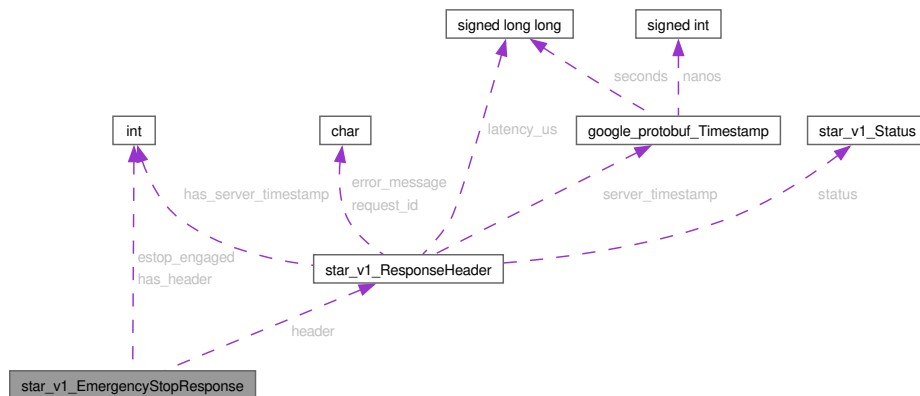
Data Fields

bool	has_header	
star_v1_RequestHeader	header	
char	reason[128]	

6.35.1.4 struct star_v1_EmergencyStopResponse

Definition at line 88 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_EmergencyStopResponse:



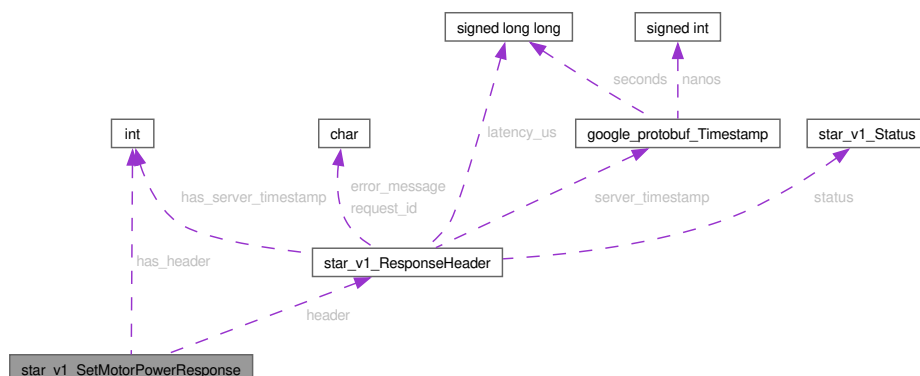
Data Fields

bool	estop_engaged	
bool	has_header	
star_v1_ResponseHeader	header	

6.35.1.5 struct star_v1_SetMotorPowerResponse

Definition at line 97 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_SetMotorPowerResponse:



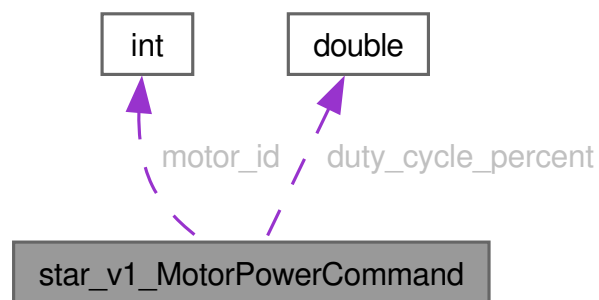
Data Fields

bool	has_header	
star_v1_ResponseHeader	header	

6.35.1.6 struct star_v1_MotorPowerCommand

Definition at line 105 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_MotorPowerCommand:



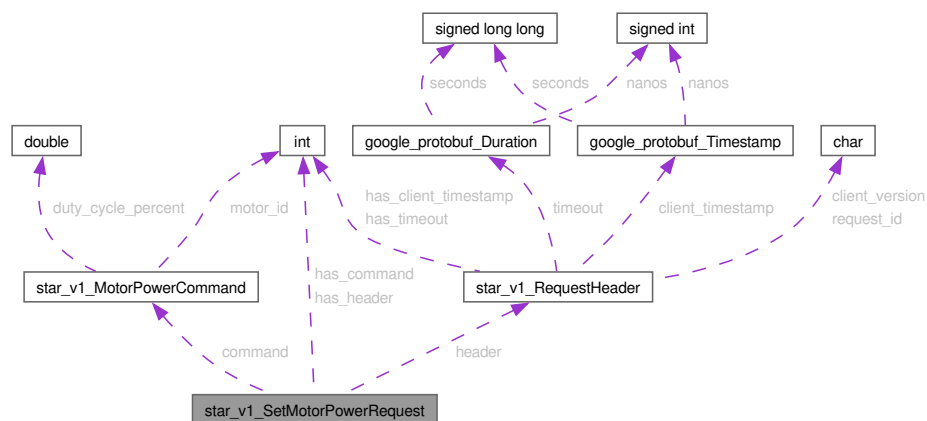
Data Fields

double	duty_cycle_percent	
int32_t	motor_id	

6.35.1.7 struct star_v1_SetMotorPowerRequest

Definition at line 115 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_SetMotorPowerRequest:



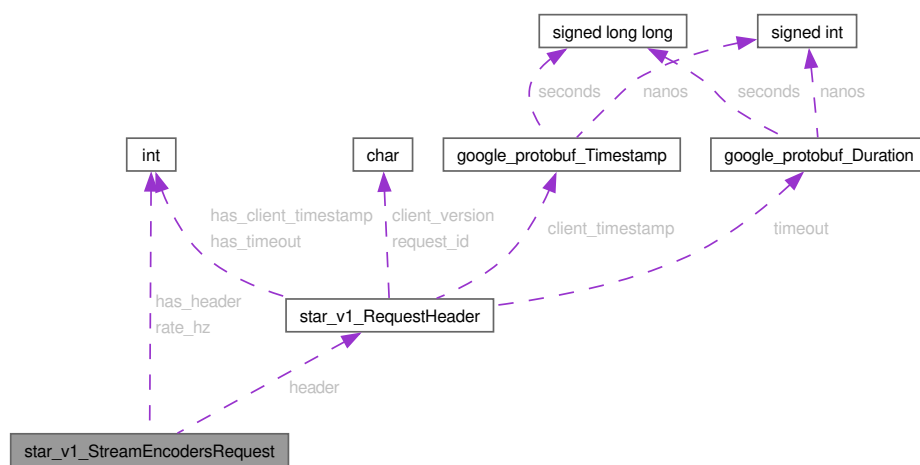
Data Fields

star_v1_MotorPowerCommand	command	
bool	has_command	
bool	has_header	
star_v1_RequestHeader	header	

6.35.1.8 struct star_v1_StreamEncodersRequest

Definition at line 125 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_StreamEncodersRequest:



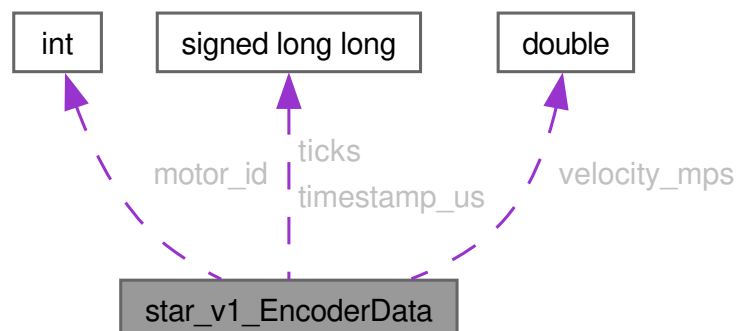
Data Fields

bool	has_header	
star_v1_RequestHeader	header	
int32_t	rate_hz	

6.35.1.9 struct star_v1_EncoderData

Definition at line 136 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_EncoderData:



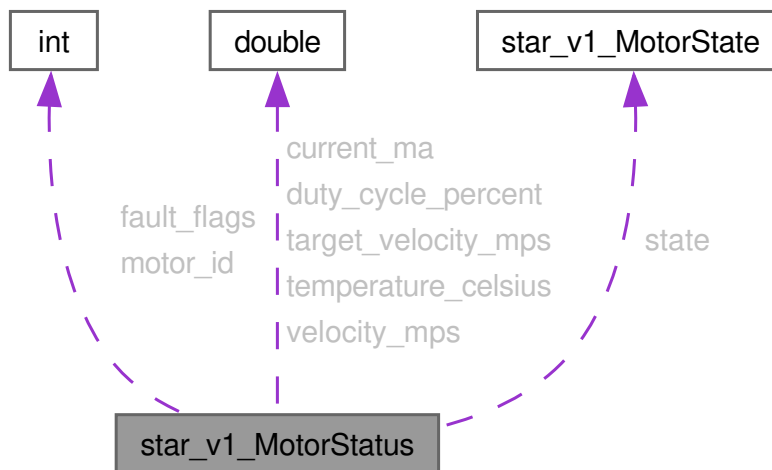
Data Fields

int32_t	motor_id	
int64_t	ticks	
int64_t	timestamp_us	
double	velocity_mps	

6.35.1.10 struct star_v1_MotorStatus

Definition at line 149 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_MotorStatus:



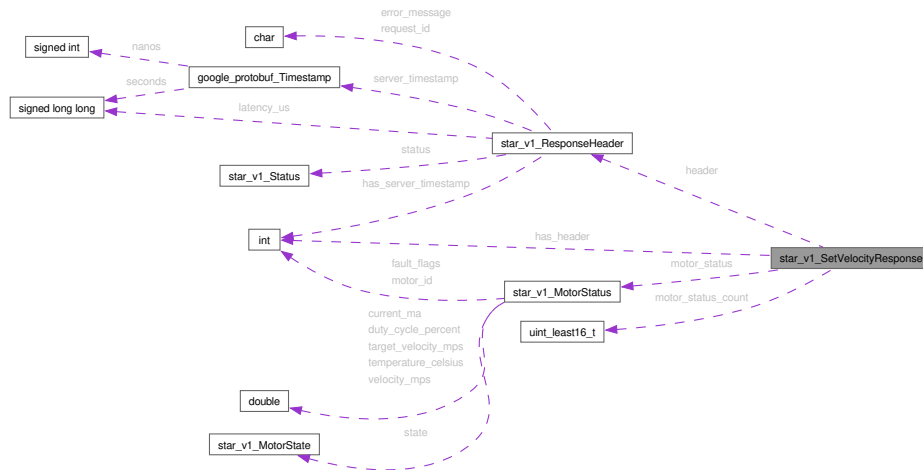
Data Fields

double	current_ma	
double	duty_cycle_percent	
uint32_t	fault_flags	
int32_t	motor_id	
star_v1_MotorState	state	
double	target_velocity_mps	
double	temperature_celsius	
double	velocity_mps	

6.35.1.11 struct star_v1_SetVelocityResponse

Definition at line 171 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_SetVelocityResponse:



Data Fields

<code>bool</code>	<code>has_header</code>	
<code>star_v1_ResponseHeader</code>	<code>header</code>	
<code>star_v1_MotorStatus</code>	<code>motor_status[2]</code>	
<code>pb_size_t</code>	<code>motor_status_count</code>	

6.35.1.12 struct star_v1_PidConfig

Definition at line 182 of file [motor_control.pb.h](#).

Collaboration diagram for star_v1_PidConfig:



Data Fields

double	integral_max	
double	integral_min	
double	kd	
double	ki	
double	kp	
double	output_max_percent	
double	output_min_percent	

6.35.2 Macro Definition Documentation

6.35.2.1 __star_v1_MotorState_ARRAYSIZE

```
#define __star_v1_MotorState_ARRAYSIZE ((star_v1_MotorState)(star_v1_MotorState_MOTOR_STATE_ESTOP+1))
```

Definition at line 207 of file [motor_control.pb.h](#).

6.35.2.2 __star_v1_MotorState_MAX

```
#define __star_v1_MotorState_MAX star_v1_MotorState_MOTOR_STATE_ESTOP
```

Definition at line 206 of file [motor_control.pb.h](#).

6.35.2.3 __star_v1_MotorState_MIN

```
#define __star_v1_MotorState_MIN star_v1_MotorState_MOTOR_STATE_UNKNOWN
```

Definition at line 205 of file [motor_control.pb.h](#).

6.35.2.4 star_v1_EmergencyStopRequest_CALLBACK

```
#define star_v1_EmergencyStopRequest_CALLBACK NULL
```

Definition at line 321 of file [motor_control.pb.h](#).

6.35.2.5 star_v1_EmergencyStopRequest_DEFAULT

```
#define star_v1_EmergencyStopRequest_DEFAULT NULL
```

Definition at line 322 of file [motor_control.pb.h](#).

6.35.2.6 star_v1_EmergencyStopRequest_FIELDLIST

```
#define star_v1_EmergencyStopRequest_FIELDLIST(
    X,
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
X(a, STATIC, SINGULAR, STRING, reason, 2)
```

Definition at line 318 of file [motor_control.pb.h](#).

```
00318 #define star_v1_EmergencyStopRequest_FIELDLIST(X, a) \
00319 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00320 X(a, STATIC, SINGULAR, STRING, reason, 2)
```

6.35.2.7 star_v1_EmergencyStopRequest_fields

```
#define star_v1_EmergencyStopRequest_fields &star_v1_EmergencyStopRequest_msg
```

Definition at line 407 of file [motor_control.pb.h](#).

Referenced by [rx_nanopb_decode_estop_request\(\)](#).

6.35.2.8 star_v1_EmergencyStopRequest_header_MSGTYPE

```
#define star_v1_EmergencyStopRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 323 of file [motor_control.pb.h](#).

6.35.2.9 star_v1_EmergencyStopRequest_header_tag

```
#define star_v1_EmergencyStopRequest_header_tag 1
```

Definition at line 258 of file [motor_control.pb.h](#).

6.35.2.10 star_v1_EmergencyStopRequest_init_default

```
#define star_v1_EmergencyStopRequest_init_default {false, star_v1_RequestHeader_init_default, ""}
```

Definition at line 227 of file [motor_control.pb.h](#).

6.35.2.11 star_v1_EmergencyStopRequest_init_zero

```
#define star_v1_EmergencyStopRequest_init_zero {false, star_v1_RequestHeader_init_zero, ""}
```

Definition at line 239 of file [motor_control.pb.h](#).

Referenced by [rx_nanopb_decode_estop_request\(\)](#).

6.35.2.12 star_v1_EmergencyStopRequest_reason_tag

```
#define star_v1_EmergencyStopRequest_reason_tag 2
```

Definition at line 259 of file [motor_control.pb.h](#).

6.35.2.13 star_v1_EmergencyStopRequest_size

```
#define star_v1_EmergencyStopRequest_size 279
```

Definition at line 419 of file [motor_control.pb.h](#).

6.35.2.14 star_v1_EmergencyStopResponse_CALLBACK

```
#define star_v1_EmergencyStopResponse_CALLBACK NULL
```

Definition at line 328 of file [motor_control.pb.h](#).

6.35.2.15 star_v1_EmergencyStopResponse_DEFAULT

```
#define star_v1_EmergencyStopResponse_DEFAULT NULL
```

Definition at line 329 of file [motor_control.pb.h](#).

6.35.2.16 star_v1_EmergencyStopResponse_estop_engaged_tag

```
#define star_v1_EmergencyStopResponse_estop_engaged_tag 2
```

Definition at line 261 of file [motor_control.pb.h](#).

6.35.2.17 star_v1_EmergencyStopResponse_FIELDLIST

```
#define star_v1_EmergencyStopResponse_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
X(a, STATIC, SINGULAR, BOOL, estop_engaged, 2)
```

Definition at line 325 of file [motor_control.pb.h](#).

```
00325 #define star_v1_EmergencyStopResponse_FIELDLIST(X, a) \  
00326 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
00327 X(a, STATIC, SINGULAR, BOOL, estop_engaged, 2)
```

6.35.2.18 star_v1_EmergencyStopResponse_fields

```
#define star_v1_EmergencyStopResponse_fields &star_v1_EmergencyStopResponse_msg
```

Definition at line 408 of file [motor_control.pb.h](#).

Referenced by [rx_nanopb_encode_estop_response\(\)](#).

6.35.2.19 star_v1_EmergencyStopResponse_header_MSGTYPE

```
#define star_v1_EmergencyStopResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 330 of file [motor_control.pb.h](#).

6.35.2.20 star_v1_EmergencyStopResponse_header_tag

```
#define star_v1_EmergencyStopResponse_header_tag 1
```

Definition at line 260 of file [motor_control.pb.h](#).

6.35.2.21 star_v1_EmergencyStopResponse_init_default

```
#define star_v1_EmergencyStopResponse_init_default {false, star_v1_ResponseHeader_init_default, 0}
```

Definition at line 228 of file [motor_control.pb.h](#).

6.35.2.22 star_v1_EmergencyStopResponse_init_zero

```
#define star_v1_EmergencyStopResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0}
```

Definition at line 240 of file [motor_control.pb.h](#).

6.35.2.23 star_v1_EmergencyStopResponse_size

```
#define star_v1_EmergencyStopResponse_size 365
```

Definition at line 420 of file [motor_control.pb.h](#).

6.35.2.24 star_v1_EncoderData_CALLBACK

```
#define star_v1_EncoderData_CALLBACK NULL
```

Definition at line 364 of file [motor_control.pb.h](#).

6.35.2.25 star_v1_EncoderData_DEFAULT

```
#define star_v1_EncoderData_DEFAULT NULL
```

Definition at line 365 of file [motor_control.pb.h](#).

6.35.2.26 star_v1_EncoderData_FIELDLIST

```
#define star_v1_EncoderData_FIELDLIST(
    X,
    a)
```

Value:

```
X(a, STATIC, SINGULAR, INT32, motor_id, 1) \
X(a, STATIC, SINGULAR, INT64, ticks, 2) \
X(a, STATIC, SINGULAR, DOUBLE, velocity_mps, 3) \
X(a, STATIC, SINGULAR, INT64, timestamp_us, 4)
```

Definition at line 359 of file [motor_control.pb.h](#).

```
00359 #define star_v1_EncoderData_FIELDLIST(X, a) \
00360 X(a, STATIC, SINGULAR, INT32, motor_id, 1) \
00361 X(a, STATIC, SINGULAR, INT64, ticks, 2) \
00362 X(a, STATIC, SINGULAR, DOUBLE, velocity_mps, 3) \
00363 X(a, STATIC, SINGULAR, INT64, timestamp_us, 4)
```

6.35.2.27 star_v1_EncoderData_fields

```
#define star_v1_EncoderData_fields &star_v1_EncoderData_msg
```

Definition at line 413 of file [motor_control.pb.h](#).

6.35.2.28 star_v1_EncoderData_init_default

```
#define star_v1_EncoderData_init_default {0, 0, 0, 0}
```

Definition at line 233 of file [motor_control.pb.h](#).

6.35.2.29 star_v1_EncoderData_init_zero

```
#define star_v1_EncoderData_init_zero {0, 0, 0, 0}
```

Definition at line 245 of file [motor_control.pb.h](#).

6.35.2.30 star_v1_EncoderData_motor_id_tag

```
#define star_v1_EncoderData_motor_id_tag 1
```

Definition at line 269 of file [motor_control.pb.h](#).

6.35.2.31 star_v1_EncoderData_size

```
#define star_v1_EncoderData_size 42
```

Definition at line 421 of file [motor_control.pb.h](#).

6.35.2.32 `star_v1_EncoderData_ticks_tag`

```
#define star_v1_EncoderData_ticks_tag 2
```

Definition at line 270 of file [motor_control.pb.h](#).

6.35.2.33 `star_v1_EncoderData_timestamp_us_tag`

```
#define star_v1_EncoderData_timestamp_us_tag 4
```

Definition at line 272 of file [motor_control.pb.h](#).

6.35.2.34 `star_v1_EncoderData_velocity_mps_tag`

```
#define star_v1_EncoderData_velocity_mps_tag 3
```

Definition at line 271 of file [motor_control.pb.h](#).

6.35.2.35 `star_v1_MotorPowerCommand_CALLBACK`

```
#define star_v1_MotorPowerCommand_CALLBACK NULL
```

Definition at line 349 of file [motor_control.pb.h](#).

6.35.2.36 `star_v1_MotorPowerCommand_DEFAULT`

```
#define star_v1_MotorPowerCommand_DEFAULT NULL
```

Definition at line 350 of file [motor_control.pb.h](#).

6.35.2.37 `star_v1_MotorPowerCommand_duty_cycle_percent_tag`

```
#define star_v1_MotorPowerCommand_duty_cycle_percent_tag 2
```

Definition at line 264 of file [motor_control.pb.h](#).

6.35.2.38 `star_v1_MotorPowerCommand_FIELDLIST`

```
#define star_v1_MotorPowerCommand_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, INT32, motor_id, 1) \
X(a, STATIC, SINGULAR, DOUBLE, duty_cycle_percent, 2)
```

Definition at line 346 of file [motor_control.pb.h](#).

```
00346 #define star_v1_MotorPowerCommand_FIELDLIST(X, a) \
00347 X(a, STATIC, SINGULAR, INT32, motor_id, 1) \
00348 X(a, STATIC, SINGULAR, DOUBLE, duty_cycle_percent, 2)
```

6.35.2.39 star_v1_MotorPowerCommand_fields

```
#define star_v1_MotorPowerCommand_fields &star_v1_MotorPowerCommand_msg
```

Definition at line 411 of file [motor_control.pb.h](#).

6.35.2.40 star_v1_MotorPowerCommand_init_default

```
#define star_v1_MotorPowerCommand_init_default {0, 0}
```

Definition at line 231 of file [motor_control.pb.h](#).

6.35.2.41 star_v1_MotorPowerCommand_init_zero

```
#define star_v1_MotorPowerCommand_init_zero {0, 0}
```

Definition at line 243 of file [motor_control.pb.h](#).

6.35.2.42 star_v1_MotorPowerCommand_motor_id_tag

```
#define star_v1_MotorPowerCommand_motor_id_tag 1
```

Definition at line 263 of file [motor_control.pb.h](#).

6.35.2.43 star_v1_MotorPowerCommand_size

```
#define star_v1_MotorPowerCommand_size 20
```

Definition at line 422 of file [motor_control.pb.h](#).

6.35.2.44 star_v1_MotorStatus_CALLBACK

```
#define star_v1_MotorStatus_CALLBACK NULL
```

Definition at line 376 of file [motor_control.pb.h](#).

6.35.2.45 star_v1_MotorStatus_current_ma_tag

```
#define star_v1_MotorStatus_current_ma_tag 6
```

Definition at line 278 of file [motor_control.pb.h](#).

6.35.2.46 star_v1_MotorStatus_DEFAULT

```
#define star_v1_MotorStatus_DEFAULT NULL
```

Definition at line 377 of file [motor_control.pb.h](#).

6.35.2.47 `star_v1_MotorStatus_duty_cycle_percent_tag`

```
#define star_v1_MotorStatus_duty_cycle_percent_tag 2
```

Definition at line 274 of file [motor_control.pb.h](#).

6.35.2.48 `star_v1_MotorStatus_fault_flags_tag`

```
#define star_v1_MotorStatus_fault_flags_tag 7
```

Definition at line 279 of file [motor_control.pb.h](#).

6.35.2.49 `star_v1_MotorStatus_FIELDLIST`

```
#define star_v1_MotorStatus_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, INT32, motor_id, 1) \  
X(a, STATIC, SINGULAR, DOUBLE, duty_cycle_percent, 2) \  
X(a, STATIC, SINGULAR, DOUBLE, velocity_mps, 3) \  
X(a, STATIC, SINGULAR, DOUBLE, target_velocity_mps, 4) \  
X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 5) \  
X(a, STATIC, SINGULAR, DOUBLE, current_ma, 6) \  
X(a, STATIC, SINGULAR, UINT32, fault_flags, 7) \  
X(a, STATIC, SINGULAR, UENUM, state, 8)
```

Definition at line 367 of file [motor_control.pb.h](#).

```
00367 #define star_v1_MotorStatus_FIELDLIST(X, a) \  
00368 X(a, STATIC, SINGULAR, INT32, motor_id, 1) \  
00369 X(a, STATIC, SINGULAR, DOUBLE, duty_cycle_percent, 2) \  
00370 X(a, STATIC, SINGULAR, DOUBLE, velocity_mps, 3) \  
00371 X(a, STATIC, SINGULAR, DOUBLE, target_velocity_mps, 4) \  
00372 X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 5) \  
00373 X(a, STATIC, SINGULAR, DOUBLE, current_ma, 6) \  
00374 X(a, STATIC, SINGULAR, UINT32, fault_flags, 7) \  
00375 X(a, STATIC, SINGULAR, UENUM, state, 8)
```

6.35.2.50 `star_v1_MotorStatus_fields`

```
#define star_v1_MotorStatus_fields &star_v1_MotorStatus_msg
```

Definition at line 414 of file [motor_control.pb.h](#).

6.35.2.51 `star_v1_MotorStatus_init_default`

```
#define star_v1_MotorStatus_init_default {0, 0, 0, 0, 0, 0, 0, 0, __star_v1_MotorState_MIN}
```

Definition at line 234 of file [motor_control.pb.h](#).

6.35.2.52 `star_v1_MotorStatus_init_zero`

```
#define star_v1_MotorStatus_init_zero {0, 0, 0, 0, 0, 0, 0, 0, __star_v1_MotorState_MIN}
```

Definition at line 246 of file [motor_control.pb.h](#).

6.35.2.53 `star_v1_MotorStatus_motor_id_tag`

```
#define star_v1_MotorStatus_motor_id_tag 1
```

Definition at line 273 of file [motor_control.pb.h](#).

6.35.2.54 `star_v1_MotorStatus_size`

```
#define star_v1_MotorStatus_size 64
```

Definition at line 423 of file [motor_control.pb.h](#).

6.35.2.55 `star_v1_MotorStatus_state_ENUMTYPE`

```
#define star_v1_MotorStatus_state_ENUMTYPE star_v1_MotorState
```

Definition at line 219 of file [motor_control.pb.h](#).

6.35.2.56 `star_v1_MotorStatus_state_tag`

```
#define star_v1_MotorStatus_state_tag 8
```

Definition at line 280 of file [motor_control.pb.h](#).

6.35.2.57 `star_v1_MotorStatus_target_velocity_mps_tag`

```
#define star_v1_MotorStatus_target_velocity_mps_tag 4
```

Definition at line 276 of file [motor_control.pb.h](#).

6.35.2.58 `star_v1_MotorStatus_temperature_celsius_tag`

```
#define star_v1_MotorStatus_temperature_celsius_tag 5
```

Definition at line 277 of file [motor_control.pb.h](#).

6.35.2.59 `star_v1_MotorStatus_velocity_mps_tag`

```
#define star_v1_MotorStatus_velocity_mps_tag 3
```

Definition at line 275 of file [motor_control.pb.h](#).

6.35.2.60 `star_v1_PidConfig_CALLBACK`

```
#define star_v1_PidConfig_CALLBACK NULL
```

Definition at line 387 of file [motor_control.pb.h](#).

6.35.2.61 star_v1_PidConfig_DEFAULT

```
#define star_v1_PidConfig_DEFAULT NULL
```

Definition at line 388 of file [motor_control.pb.h](#).

6.35.2.62 star_v1_PidConfig_FIELDLIST

```
#define star_v1_PidConfig_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, DOUBLE, kp, 1) \  
X(a, STATIC, SINGULAR, DOUBLE, ki, 2) \  
X(a, STATIC, SINGULAR, DOUBLE, kd, 3) \  
X(a, STATIC, SINGULAR, DOUBLE, output_min_percent, 4) \  
X(a, STATIC, SINGULAR, DOUBLE, output_max_percent, 5) \  
X(a, STATIC, SINGULAR, DOUBLE, integral_min, 6) \  
X(a, STATIC, SINGULAR, DOUBLE, integral_max, 7)
```

Definition at line 379 of file [motor_control.pb.h](#).

```
00379 #define star_v1_PidConfig_FIELDLIST(X, a) \  
00380 X(a, STATIC, SINGULAR, DOUBLE, kp, 1) \  
00381 X(a, STATIC, SINGULAR, DOUBLE, ki, 2) \  
00382 X(a, STATIC, SINGULAR, DOUBLE, kd, 3) \  
00383 X(a, STATIC, SINGULAR, DOUBLE, output_min_percent, 4) \  
00384 X(a, STATIC, SINGULAR, DOUBLE, output_max_percent, 5) \  
00385 X(a, STATIC, SINGULAR, DOUBLE, integral_min, 6) \  
00386 X(a, STATIC, SINGULAR, DOUBLE, integral_max, 7)
```

6.35.2.63 star_v1_PidConfig_fields

```
#define star_v1_PidConfig_fields &star_v1_PidConfig_msg
```

Definition at line 415 of file [motor_control.pb.h](#).

6.35.2.64 star_v1_PidConfig_init_default

```
#define star_v1_PidConfig_init_default {0, 0, 0, 0, 0, 0, 0}
```

Definition at line 235 of file [motor_control.pb.h](#).

6.35.2.65 star_v1_PidConfig_init_zero

```
#define star_v1_PidConfig_init_zero {0, 0, 0, 0, 0, 0, 0}
```

Definition at line 247 of file [motor_control.pb.h](#).

6.35.2.66 star_v1_PidConfig_integral_max_tag

```
#define star_v1_PidConfig_integral_max_tag 7
```

Definition at line 289 of file [motor_control.pb.h](#).

6.35.2.67 star_v1_PidConfig_integral_min_tag

```
#define star_v1_PidConfig_integral_min_tag 6
```

Definition at line 288 of file [motor_control.pb.h](#).

6.35.2.68 star_v1_PidConfig_kd_tag

```
#define star_v1_PidConfig_kd_tag 3
```

Definition at line 285 of file [motor_control.pb.h](#).

6.35.2.69 star_v1_PidConfig_ki_tag

```
#define star_v1_PidConfig_ki_tag 2
```

Definition at line 284 of file [motor_control.pb.h](#).

6.35.2.70 star_v1_PidConfig_kp_tag

```
#define star_v1_PidConfig_kp_tag 1
```

Definition at line 283 of file [motor_control.pb.h](#).

6.35.2.71 star_v1_PidConfig_output_max_percent_tag

```
#define star_v1_PidConfig_output_max_percent_tag 5
```

Definition at line 287 of file [motor_control.pb.h](#).

6.35.2.72 star_v1_PidConfig_output_min_percent_tag

```
#define star_v1_PidConfig_output_min_percent_tag 4
```

Definition at line 286 of file [motor_control.pb.h](#).

6.35.2.73 star_v1_PidConfig_size

```
#define star_v1_PidConfig_size 63
```

Definition at line 424 of file [motor_control.pb.h](#).

6.35.2.74 star_v1_SetMotorPowerRequest_CALLBACK

```
#define star_v1_SetMotorPowerRequest_CALLBACK NULL
```

Definition at line 335 of file [motor_control.pb.h](#).

6.35.2.75 `star_v1_SetMotorPowerRequest_command_MSGTYPE`

```
#define star_v1_SetMotorPowerRequest_command_MSGTYPE star_v1_MotorPowerCommand
```

Definition at line 338 of file [motor_control.pb.h](#).

6.35.2.76 `star_v1_SetMotorPowerRequest_command_tag`

```
#define star_v1_SetMotorPowerRequest_command_tag 2
```

Definition at line 266 of file [motor_control.pb.h](#).

6.35.2.77 `star_v1_SetMotorPowerRequest_DEFAULT`

```
#define star_v1_SetMotorPowerRequest_DEFAULT NULL
```

Definition at line 336 of file [motor_control.pb.h](#).

6.35.2.78 `star_v1_SetMotorPowerRequest_FIELDLIST`

```
#define star_v1_SetMotorPowerRequest_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
X(a, STATIC, OPTIONAL, MESSAGE, command, 2)
```

Definition at line 332 of file [motor_control.pb.h](#).

```
00332 #define star_v1_SetMotorPowerRequest_FIELDLIST(X, a) \  
00333 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
00334 X(a, STATIC, OPTIONAL, MESSAGE, command, 2)
```

6.35.2.79 `star_v1_SetMotorPowerRequest_fields`

```
#define star_v1_SetMotorPowerRequest_fields &star_v1_SetMotorPowerRequest_msg
```

Definition at line 409 of file [motor_control.pb.h](#).

6.35.2.80 `star_v1_SetMotorPowerRequest_header_MSGTYPE`

```
#define star_v1_SetMotorPowerRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 337 of file [motor_control.pb.h](#).

6.35.2.81 `star_v1_SetMotorPowerRequest_header_tag`

```
#define star_v1_SetMotorPowerRequest_header_tag 1
```

Definition at line 265 of file [motor_control.pb.h](#).

6.35.2.82 star_v1_SetMotorPowerRequest_init_default

```
#define star_v1_SetMotorPowerRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_MotorPowerCommand_init_default}
```

Definition at line 229 of file [motor_control.pb.h](#).

6.35.2.83 star_v1_SetMotorPowerRequest_init_zero

```
#define star_v1_SetMotorPowerRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_MotorPowerCommand_init_zero}
```

Definition at line 241 of file [motor_control.pb.h](#).

6.35.2.84 star_v1_SetMotorPowerRequest_size

```
#define star_v1_SetMotorPowerRequest_size 171
```

Definition at line 425 of file [motor_control.pb.h](#).

6.35.2.85 star_v1_SetMotorPowerResponse_CALLBACK

```
#define star_v1_SetMotorPowerResponse_CALLBACK NULL
```

Definition at line 342 of file [motor_control.pb.h](#).

6.35.2.86 star_v1_SetMotorPowerResponse_DEFAULT

```
#define star_v1_SetMotorPowerResponse_DEFAULT NULL
```

Definition at line 343 of file [motor_control.pb.h](#).

6.35.2.87 star_v1_SetMotorPowerResponse_FIELDLIST

```
#define star_v1_SetMotorPowerResponse_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

Definition at line 340 of file [motor_control.pb.h](#).

```
00340 #define star_v1_SetMotorPowerResponse_FIELDLIST(X, a) \
00341 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

6.35.2.88 star_v1_SetMotorPowerResponse_fields

```
#define star_v1_SetMotorPowerResponse_fields &star_v1_SetMotorPowerResponse_msg
```

Definition at line 410 of file [motor_control.pb.h](#).

6.35.2.89 star_v1_SetMotorPowerResponse_header_MSGTYPE

```
#define star_v1_SetMotorPowerResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 344 of file [motor_control.pb.h](#).

6.35.2.90 star_v1_SetMotorPowerResponse_header_tag

```
#define star_v1_SetMotorPowerResponse_header_tag 1
```

Definition at line 262 of file [motor_control.pb.h](#).

6.35.2.91 star_v1_SetMotorPowerResponse_init_default

```
#define star_v1_SetMotorPowerResponse_init_default {false, star_v1_ResponseHeader_init_default}
```

Definition at line 230 of file [motor_control.pb.h](#).

6.35.2.92 star_v1_SetMotorPowerResponse_init_zero

```
#define star_v1_SetMotorPowerResponse_init_zero {false, star_v1_ResponseHeader_init_zero}
```

Definition at line 242 of file [motor_control.pb.h](#).

6.35.2.93 star_v1_SetMotorPowerResponse_size

```
#define star_v1_SetMotorPowerResponse_size 363
```

Definition at line 426 of file [motor_control.pb.h](#).

6.35.2.94 star_v1_SetVelocityRequest_CALLBACK

```
#define star_v1_SetVelocityRequest_CALLBACK NULL
```

Definition at line 295 of file [motor_control.pb.h](#).

6.35.2.95 star_v1_SetVelocityRequest_command_MSGTYPE

```
#define star_v1_SetVelocityRequest_command_MSGTYPE star_v1_VelocityCommand
```

Definition at line 298 of file [motor_control.pb.h](#).

6.35.2.96 star_v1_SetVelocityRequest_command_tag

```
#define star_v1_SetVelocityRequest_command_tag 2
```

Definition at line 257 of file [motor_control.pb.h](#).

6.35.2.97 star_v1_SetVelocityRequest_DEFAULT

```
#define star_v1_SetVelocityRequest_DEFAULT NULL
```

Definition at line 296 of file [motor_control.pb.h](#).

6.35.2.98 star_v1_SetVelocityRequest_FIELDLIST

```
#define star_v1_SetVelocityRequest_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
X(a, STATIC, OPTIONAL, MESSAGE, command, 2)
```

Definition at line 292 of file [motor_control.pb.h](#).

```
00292 #define star_v1_SetVelocityRequest_FIELDLIST(X, a) \  
00293 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
00294 X(a, STATIC, OPTIONAL, MESSAGE, command, 2)
```

6.35.2.99 star_v1_SetVelocityRequest_fields

```
#define star_v1_SetVelocityRequest_fields &star_v1_SetVelocityRequest_msg
```

Definition at line 404 of file [motor_control.pb.h](#).

Referenced by [rx_nanopb_decode_velocity_request\(\)](#), and [rx_nanopb_encode_velocity_request\(\)](#).

6.35.2.100 star_v1_SetVelocityRequest_header_MSGTYPE

```
#define star_v1_SetVelocityRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 297 of file [motor_control.pb.h](#).

6.35.2.101 star_v1_SetVelocityRequest_header_tag

```
#define star_v1_SetVelocityRequest_header_tag 1
```

Definition at line 256 of file [motor_control.pb.h](#).

6.35.2.102 star_v1_SetVelocityRequest_init_default

```
#define star_v1_SetVelocityRequest_init_default {false, star_v1_RequestHeader_init_default, false, star_v1_VelocityCommand_init_def
```

Definition at line 224 of file [motor_control.pb.h](#).

6.35.2.103 `star_v1_SetVelocityRequest_init_zero`

```
#define star_v1_SetVelocityRequest_init_zero {false, star_v1_RequestHeader_init_zero, false, star_v1_VelocityCommand_init_zero}
```

Definition at line 236 of file [motor_control.pb.h](#).

Referenced by [rx_nanopb_decode_velocity_request\(\)](#).

6.35.2.104 `star_v1_SetVelocityRequest_size`

```
#define star_v1_SetVelocityRequest_size 204
```

Definition at line 427 of file [motor_control.pb.h](#).

6.35.2.105 `star_v1_SetVelocityResponse_CALLBACK`

```
#define star_v1_SetVelocityResponse_CALLBACK NULL
```

Definition at line 303 of file [motor_control.pb.h](#).

6.35.2.106 `star_v1_SetVelocityResponse_DEFAULT`

```
#define star_v1_SetVelocityResponse_DEFAULT NULL
```

Definition at line 304 of file [motor_control.pb.h](#).

6.35.2.107 `star_v1_SetVelocityResponse_FIELDLIST`

```
#define star_v1_SetVelocityResponse_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC,  OPTIONAL, MESSAGE, header,      1) \  
X(a, STATIC,  REPEATED, MESSAGE, motor_status, 2)
```

Definition at line 300 of file [motor_control.pb.h](#).

```
00300 #define star_v1_SetVelocityResponse_FIELDLIST(X, a) \  
00301 X(a, STATIC,  OPTIONAL, MESSAGE, header,      1) \  
00302 X(a, STATIC,  REPEATED, MESSAGE, motor_status, 2)
```

6.35.2.108 `star_v1_SetVelocityResponse_fields`

```
#define star_v1_SetVelocityResponse_fields &star_v1_SetVelocityResponse_msg
```

Definition at line 405 of file [motor_control.pb.h](#).

Referenced by [rx_nanopb_encode_velocity_response\(\)](#).

6.35.2.109 star_v1_SetVelocityResponse_header_MSGTYPE

```
#define star_v1_SetVelocityResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 305 of file [motor_control.pb.h](#).

6.35.2.110 star_v1_SetVelocityResponse_header_tag

```
#define star_v1_SetVelocityResponse_header_tag 1
```

Definition at line 281 of file [motor_control.pb.h](#).

6.35.2.111 star_v1_SetVelocityResponse_init_default

```
#define star_v1_SetVelocityResponse_init_default {false, star_v1_ResponseHeader_init_default, 0, {star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default}}
```

Definition at line 225 of file [motor_control.pb.h](#).

6.35.2.112 star_v1_SetVelocityResponse_init_zero

```
#define star_v1_SetVelocityResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0, {star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero}}
```

Definition at line 237 of file [motor_control.pb.h](#).

6.35.2.113 star_v1_SetVelocityResponse_motor_status_MSGTYPE

```
#define star_v1_SetVelocityResponse_motor_status_MSGTYPE star_v1_MotorStatus
```

Definition at line 306 of file [motor_control.pb.h](#).

6.35.2.114 star_v1_SetVelocityResponse_motor_status_tag

```
#define star_v1_SetVelocityResponse_motor_status_tag 2
```

Definition at line 282 of file [motor_control.pb.h](#).

6.35.2.115 star_v1_SetVelocityResponse_size

```
#define star_v1_SetVelocityResponse_size 495
```

Definition at line 428 of file [motor_control.pb.h](#).

6.35.2.116 STAR_V1_STAR_V1_MOTOR_CONTROL_PB_H_MAX_SIZE

```
#define STAR_V1_STAR_V1_MOTOR_CONTROL_PB_H_MAX_SIZE star\_v1\_SetVelocityResponse\_size
```

Definition at line [418](#) of file [motor_control.pb.h](#).

6.35.2.117 star_v1_StreamEncodersRequest_CALLBACK

```
#define star_v1_StreamEncodersRequest_CALLBACK NULL
```

Definition at line [355](#) of file [motor_control.pb.h](#).

6.35.2.118 star_v1_StreamEncodersRequest_DEFAULT

```
#define star_v1_StreamEncodersRequest_DEFAULT NULL
```

Definition at line [356](#) of file [motor_control.pb.h](#).

6.35.2.119 star_v1_StreamEncodersRequest_FIELDLIST

```
#define star_v1_StreamEncodersRequest_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
X(a, STATIC, SINGULAR, INT32, rate_hz, 2)
```

Definition at line [352](#) of file [motor_control.pb.h](#).

```
00352 #define star_v1_StreamEncodersRequest_FIELDLIST(X, a) \  
00353 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
00354 X(a, STATIC, SINGULAR, INT32, rate_hz, 2)
```

6.35.2.120 star_v1_StreamEncodersRequest_fields

```
#define star_v1_StreamEncodersRequest_fields &star\_v1\_StreamEncodersRequest\_msg
```

Definition at line [412](#) of file [motor_control.pb.h](#).

6.35.2.121 star_v1_StreamEncodersRequest_header_MSGTYPE

```
#define star_v1_StreamEncodersRequest_header_MSGTYPE star\_v1\_RequestHeader
```

Definition at line [357](#) of file [motor_control.pb.h](#).

6.35.2.122 star_v1_StreamEncodersRequest_header_tag

```
#define star_v1_StreamEncodersRequest_header_tag 1
```

Definition at line [267](#) of file [motor_control.pb.h](#).

6.35.2.123 star_v1_StreamEncodersRequest_init_default

```
#define star_v1_StreamEncodersRequest_init_default {false, star_v1_RequestHeader_init_default, 0}
```

Definition at line 232 of file [motor_control.pb.h](#).

6.35.2.124 star_v1_StreamEncodersRequest_init_zero

```
#define star_v1_StreamEncodersRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}
```

Definition at line 244 of file [motor_control.pb.h](#).

6.35.2.125 star_v1_StreamEncodersRequest_rate_hz_tag

```
#define star_v1_StreamEncodersRequest_rate_hz_tag 2
```

Definition at line 268 of file [motor_control.pb.h](#).

6.35.2.126 star_v1_StreamEncodersRequest_size

```
#define star_v1_StreamEncodersRequest_size 160
```

Definition at line 429 of file [motor_control.pb.h](#).

6.35.2.127 star_v1_VelocityCommand_back_left_velocity_mps_tag

```
#define star_v1_VelocityCommand_back_left_velocity_mps_tag 5
```

Definition at line 254 of file [motor_control.pb.h](#).

6.35.2.128 star_v1_VelocityCommand_back_right_velocity_mps_tag

```
#define star_v1_VelocityCommand_back_right_velocity_mps_tag 6
```

Definition at line 255 of file [motor_control.pb.h](#).

6.35.2.129 star_v1_VelocityCommand_CALLBACK

```
#define star_v1_VelocityCommand_CALLBACK NULL
```

Definition at line 315 of file [motor_control.pb.h](#).

6.35.2.130 star_v1_VelocityCommand_DEFAULT

```
#define star_v1_VelocityCommand_DEFAULT NULL
```

Definition at line 316 of file [motor_control.pb.h](#).

6.35.2.131 star_v1_VelocityCommand_FIELDLIST

```
#define star_v1_VelocityCommand_FIELDLIST(
    X,
    a)
```

Value:

```
X(a, STATIC, SINGULAR, DOUBLE, front_left_velocity_mps, 1) \
X(a, STATIC, SINGULAR, DOUBLE, front_right_velocity_mps, 2) \
X(a, STATIC, SINGULAR, UINT32, sequence, 3) \
X(a, STATIC, SINGULAR, INT64, timestamp_us, 4) \
X(a, STATIC, SINGULAR, DOUBLE, back_left_velocity_mps, 5) \
X(a, STATIC, SINGULAR, DOUBLE, back_right_velocity_mps, 6)
```

Definition at line 308 of file [motor_control.pb.h](#).

```
00308 #define star_v1_VelocityCommand_FIELDLIST(X, a) \
00309 X(a, STATIC, SINGULAR, DOUBLE, front_left_velocity_mps, 1) \
00310 X(a, STATIC, SINGULAR, DOUBLE, front_right_velocity_mps, 2) \
00311 X(a, STATIC, SINGULAR, UINT32, sequence, 3) \
00312 X(a, STATIC, SINGULAR, INT64, timestamp_us, 4) \
00313 X(a, STATIC, SINGULAR, DOUBLE, back_left_velocity_mps, 5) \
00314 X(a, STATIC, SINGULAR, DOUBLE, back_right_velocity_mps, 6)
```

6.35.2.132 star_v1_VelocityCommand_fields

```
#define star_v1_VelocityCommand_fields &star_v1_VelocityCommand_msg
```

Definition at line 406 of file [motor_control.pb.h](#).

6.35.2.133 star_v1_VelocityCommand_front_left_velocity_mps_tag

```
#define star_v1_VelocityCommand_front_left_velocity_mps_tag 1
```

Definition at line 250 of file [motor_control.pb.h](#).

6.35.2.134 star_v1_VelocityCommand_front_right_velocity_mps_tag

```
#define star_v1_VelocityCommand_front_right_velocity_mps_tag 2
```

Definition at line 251 of file [motor_control.pb.h](#).

6.35.2.135 star_v1_VelocityCommand_init_default

```
#define star_v1_VelocityCommand_init_default {0, 0, 0, 0, 0, 0}
```

Definition at line 226 of file [motor_control.pb.h](#).

6.35.2.136 star_v1_VelocityCommand_init_zero

```
#define star_v1_VelocityCommand_init_zero {0, 0, 0, 0, 0, 0}
```

Definition at line 238 of file [motor_control.pb.h](#).

Referenced by [rx_nanopb_create_velocity_command\(\)](#).

6.35.2.137 star_v1_VelocityCommand_sequence_tag

```
#define star_v1_VelocityCommand_sequence_tag 3
```

Definition at line 252 of file [motor_control.pb.h](#).

6.35.2.138 star_v1_VelocityCommand_size

```
#define star_v1_VelocityCommand_size 53
```

Definition at line 430 of file [motor_control.pb.h](#).

6.35.2.139 star_v1_VelocityCommand_timestamp_us_tag

```
#define star_v1_VelocityCommand_timestamp_us_tag 4
```

Definition at line 253 of file [motor_control.pb.h](#).

6.35.3 Enumeration Type Documentation

6.35.3.1 star_v1_MotorState

```
enum star\_v1\_MotorState
```

Enumerator

star_v1_MotorState_MOTOR_STATE_UNKNOWN	
star_v1_MotorState_MOTOR_STATE_IDLE	
star_v1_MotorState_MOTOR_STATE_RUNNING	
star_v1_MotorState_MOTOR_STATE_FAULT	
star_v1_MotorState_MOTOR_STATE_ESTOP	

Definition at line 15 of file [motor_control.pb.h](#).

```
00015 {
00016     /* Unknown state. */
00017     star_v1_MotorState_MOTOR_STATE_UNKNOWN = 0,
00018     /* Motor is idle (not commanded). */
00019     star_v1_MotorState_MOTOR_STATE_IDLE = 1,
00020     /* Motor is running under PID control. */
00021     star_v1_MotorState_MOTOR_STATE_RUNNING = 2,
00022     /* Motor is in fault condition. */
00023     star_v1_MotorState_MOTOR_STATE_FAULT = 3,
00024     /* Motor is in emergency stop. */
00025     star_v1_MotorState_MOTOR_STATE_ESTOP = 4
00026 } star_v1_MotorState;
```

6.35.4 Variable Documentation

6.35.4.1 star_v1_EmergencyStopRequest_msg

```
const pb_msgdesc_t star_v1_EmergencyStopRequest_msg [extern]
```

6.35.4.2 star_v1_EmergencyStopResponse_msg

const pb_msgdesc_t star_v1_EmergencyStopResponse_msg [extern]

6.35.4.3 star_v1_EncoderData_msg

const pb_msgdesc_t star_v1_EncoderData_msg [extern]

6.35.4.4 star_v1_MotorPowerCommand_msg

const pb_msgdesc_t star_v1_MotorPowerCommand_msg [extern]

6.35.4.5 star_v1_MotorStatus_msg

const pb_msgdesc_t star_v1_MotorStatus_msg [extern]

6.35.4.6 star_v1_PidConfig_msg

const pb_msgdesc_t star_v1_PidConfig_msg [extern]

6.35.4.7 star_v1_SetMotorPowerRequest_msg

const pb_msgdesc_t star_v1_SetMotorPowerRequest_msg [extern]

6.35.4.8 star_v1_SetMotorPowerResponse_msg

const pb_msgdesc_t star_v1_SetMotorPowerResponse_msg [extern]

6.35.4.9 star_v1_SetVelocityRequest_msg

const pb_msgdesc_t star_v1_SetVelocityRequest_msg [extern]

6.35.4.10 star_v1_SetVelocityResponse_msg

const pb_msgdesc_t star_v1_SetVelocityResponse_msg [extern]

6.35.4.11 star_v1_StreamEncodersRequest_msg

const pb_msgdesc_t star_v1_StreamEncodersRequest_msg [extern]

6.35.4.12 star_v1_VelocityCommand_msg

```
const pb_msgdesc_t star_v1_VelocityCommand_msg [extern]
```

6.36 motor_control.pb.h

[Go to the documentation of this file.](#)

```
00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_STAR_V1_STAR_V1_MOTOR_CONTROL_PB_H_INCLUDED
00005 #define PB_STAR_V1_STAR_V1_MOTOR_CONTROL_PB_H_INCLUDED
00006 #include <pb.h>
00007 #include "star/v1/common.pb.h"
00008
00009 #if PB_PROTO_HEADER_VERSION != 40
00010 #error Regenerate this file with the current version of nanopb generator.
00011 #endif
00012
00013 /* Enum definitions */
00014 /* Motor operating state. */
00015 typedef enum _star_v1_MotorState {
00016     /* Unknown state. */
00017     star_v1_MotorState_MOTOR_STATE_UNKNOWN = 0,
00018     /* Motor is idle (not commanded). */
00019     star_v1_MotorState_MOTOR_STATE_IDLE = 1,
00020     /* Motor is running under PID control. */
00021     star_v1_MotorState_MOTOR_STATE_RUNNING = 2,
00022     /* Motor is in fault condition. */
00023     star_v1_MotorState_MOTOR_STATE_FAULT = 3,
00024     /* Motor is in emergency stop. */
00025     star_v1_MotorState_MOTOR_STATE_ESTOP = 4
00026 } star_v1_MotorState;
00027
00028 /* Struct definitions */
00029 /* Velocity command for 4-motor independent control.
00030  Uses MKS units: meters per second.
00031
00032 Motor Layout (top view):
00033   Front
00034   FL  FR
00035   BL  BR
00036   Back
00037
00038 WARNING: Firmware Update Required
00039 When regenerating firmware code, ensure mapping logic in comm_manager.c
00040 matches these fields to the correct motor indices:
00041   front_left  (1) -> velocity_mps[0]
00042   front_right (2) -> velocity_mps[1]
00043   back_left   (5) -> velocity_mps[2]
00044   back_right  (6) -> velocity_mps[3] */
00045 typedef struct _star_v1_VelocityCommand {
00046     /* Front left motor velocity in meters per second.
00047     Valid range: -2.0 to 2.0 m/s.
00048     Positive = forward, negative = reverse. */
00049     double front_left_velocity_mps;
00050     /* Front right motor velocity in meters per second.
00051     Valid range: -2.0 to 2.0 m/s.
00052     Positive = forward, negative = reverse. */
00053     double front_right_velocity_mps;
00054     /* Command sequence number for ordering and deduplication. */
00055     uint32_t sequence;
00056     /* Client timestamp in microseconds for latency tracking. */
00057     int64_t timestamp_us;
00058     /* Back left motor velocity in meters per second.
00059     Valid range: -2.0 to 2.0 m/s.
00060     Positive = forward, negative = reverse. */
00061     double back_left_velocity_mps;
00062     /* Back right motor velocity in meters per second.
00063     Valid range: -2.0 to 2.0 m/s.
00064     Positive = forward, negative = reverse. */
00065     double back_right_velocity_mps;
00066 } star_v1_VelocityCommand;
00067
00068 /* Request to set wheel velocities. */
00069 typedef struct _star_v1_SetVelocityRequest {
00070     /* Standard request header. */
00071     bool has_header;
00072     star_v1_RequestHeader header;
```

```

00073  /* Velocity command to execute. */
00074  bool has_command;
00075  star_v1_VelocityCommand command;
00076 } star_v1_SetVelocityRequest;
00077
00078 /* Emergency stop request. */
00079 typedef struct _star_v1_EmergencyStopRequest {
00080  /* Standard request header. */
00081  bool has_header;
00082  star_v1_RequestHeader header;
00083  /* Reason for emergency stop (for logging). */
00084  char reason[128];
00085 } star_v1_EmergencyStopRequest;
00086
00087 /* Emergency stop response. */
00088 typedef struct _star_v1_EmergencyStopResponse {
00089  /* Standard response header with status. */
00090  bool has_header;
00091  star_v1_ResponseHeader header;
00092  /* True if emergency stop was successfully engaged. */
00093  bool estop_engaged;
00094 } star_v1_EmergencyStopResponse;
00095
00096 /* Response to motor power command. */
00097 typedef struct _star_v1_SetMotorPowerResponse {
00098  /* Standard response header with status. */
00099  bool has_header;
00100  star_v1_ResponseHeader header;
00101 } star_v1_SetMotorPowerResponse;
00102
00103 /* Direct motor power control for individual motors.
00104  Bypasses PID controller - use for testing only. */
00105 typedef struct _star_v1_MotorPowerCommand {
00106  /* Motor index: 0-3 for 4 independent motors. */
00107  int32_t motor_id;
00108  /* Power level as duty cycle percentage.
00109  Valid range: -100.0 to 100.0 percent.
00110  Positive = forward, negative = reverse. */
00111  double duty_cycle_percent;
00112 } star_v1_MotorPowerCommand;
00113
00114 /* Request to set individual motor power. */
00115 typedef struct _star_v1_SetMotorPowerRequest {
00116  /* Standard request header. */
00117  bool has_header;
00118  star_v1_RequestHeader header;
00119  /* Motor power command. */
00120  bool has_command;
00121  star_v1_MotorPowerCommand command;
00122 } star_v1_SetMotorPowerRequest;
00123
00124 /* Request to stream encoder data. */
00125 typedef struct _star_v1_StreamEncodersRequest {
00126  /* Standard request header. */
00127  bool has_header;
00128  star_v1_RequestHeader header;
00129  /* Desired update rate in Hz.
00130  Valid range: 1 to 100 Hz. */
00131  int32_t rate_hz;
00132 } star_v1_StreamEncodersRequest;
00133
00134 /* Real-time encoder feedback from motor controller.
00135  Streamed at configured rate. */
00136 typedef struct _star_v1_EncoderData {
00137  /* Motor index: 0-3 for 4 independent motors. */
00138  int32_t motor_id;
00139  /* Absolute encoder position in ticks.
00140  341 PPR Hall encoder * 34.02:1 gear ratio = 1364 ticks per revolution. */
00141  int64_t ticks;
00142  /* Current velocity in meters per second. */
00143  double velocity_mps;
00144  /* Timestamp in microseconds since boot. */
00145  int64_t timestamp_us;
00146 } star_v1_EncoderData;
00147
00148 /* Motor status information. */
00149 typedef struct _star_v1_MotorStatus {
00150  /* Motor index: 0-3 for 4 independent motors. */
00151  int32_t motor_id;
00152  /* Current duty cycle percentage.
00153  Range: -100.0 to 100.0 percent. */
00154  double duty_cycle_percent;
00155  /* Current velocity in meters per second. */
00156  double velocity_mps;
00157  /* Target velocity in meters per second. */
00158  double target_velocity_mps;
00159  /* Motor temperature in Celsius (if available). */

```

```

00160     double temperature_celsius;
00161     /* Current draw in milliamps (if current sensing available). */
00162     double current_ma;
00163     /* Fault flags (bitfield).
00164     Bit 0: overcurrent, Bit 1: overtemperature, Bit 2: encoder fault. */
00165     uint32_t fault_flags;
00166     /* Current motor state. */
00167     star_v1_MotorState state;
00168 } star_v1_MotorStatus;
00169
00170 /* Response to velocity command. */
00171 typedef struct _star_v1_SetVelocityResponse {
00172     /* Standard response header with status. */
00173     bool has_header;
00174     star_v1_ResponseHeader header;
00175     /* Current motor status after command. */
00176     pb_size_t motor_status_count;
00177     star_v1_MotorStatus motor_status[2];
00178 } star_v1_SetVelocityResponse;
00179
00180 /* PID gains for motor velocity control.
00181 Matches ESP32 star_pid_config_t structure. */
00182 typedef struct _star_v1_PidConfig {
00183     /* Proportional gain. */
00184     double kp;
00185     /* Integral gain. */
00186     double ki;
00187     /* Derivative gain. */
00188     double kd;
00189     /* Output saturation minimum (duty cycle percent). */
00190     double output_min_percent;
00191     /* Output saturation maximum (duty cycle percent). */
00192     double output_max_percent;
00193     /* Anti-windup integral minimum. */
00194     double integral_min;
00195     /* Anti-windup integral maximum. */
00196     double integral_max;
00197 } star_v1_PidConfig;
00198
00199
00200 #ifdef __cplusplus
00201 extern "C" {
00202 #endif
00203
00204 /* Helper constants for enums */
00205 #define star_v1_MotorState_MIN star_v1_MotorState_MOTOR_STATE_UNKNOWN
00206 #define star_v1_MotorState_MAX star_v1_MotorState_MOTOR_STATE_ESTOP
00207 #define star_v1_MotorState_ARRAYSIZE
00208     ((star_v1_MotorState)(star_v1_MotorState_MOTOR_STATE_ESTOP+1))
00209
00210
00211
00212
00213
00214
00215
00216
00217
00218
00219 #define star_v1_MotorStatus_state_ENUMTYPE star_v1_MotorState
00220
00221
00222
00223 /* Initializer values for message structs */
00224 #define star_v1_SetVelocityRequest_init_default {false, star_v1_RequestHeader_init_default, false,
00225     star_v1_VelocityCommand_init_default}
00226 #define star_v1_SetVelocityResponse_init_default {false, star_v1_ResponseHeader_init_default, 0,
00227     {star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default}}
00228 #define star_v1_VelocityCommand_init_default {0, 0, 0, 0, 0, 0}
00229 #define star_v1_EmergencyStopRequest_init_default {false, star_v1_RequestHeader_init_default, ""}
00230 #define star_v1_EmergencyStopResponse_init_default {false, star_v1_ResponseHeader_init_default, 0}
00231 #define star_v1_SetMotorPowerRequest_init_default {false, star_v1_RequestHeader_init_default, false,
00232     star_v1_MotorPowerCommand_init_default}
00233 #define star_v1_SetMotorPowerResponse_init_default {false, star_v1_ResponseHeader_init_default}
00234 #define star_v1_MotorPowerCommand_init_default {0, 0}
00235 #define star_v1_StreamEncodersRequest_init_default {false, star_v1_RequestHeader_init_default, 0}
00236 #define star_v1_EncoderData_init_default {0, 0, 0, 0}
00237 #define star_v1_MotorStatus_init_default {0, 0, 0, 0, 0, 0, star_v1_MotorState_MIN}
00238 #define star_v1_PidConfig_init_default {0, 0, 0, 0, 0, 0}
00239 #define star_v1_SetVelocityRequest_init_zero {false, star_v1_RequestHeader_init_zero, false,
00240     star_v1_VelocityCommand_init_zero}
00241 #define star_v1_SetVelocityResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0,
00242     {star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero}}
00243 #define star_v1_VelocityCommand_init_zero {0, 0, 0, 0, 0, 0}
00244 #define star_v1_EmergencyStopRequest_init_zero {false, star_v1_RequestHeader_init_zero, ""}
00245 #define star_v1_EmergencyStopResponse_init_zero {false, star_v1_ResponseHeader_init_zero, 0}

```

```

00241 #define star_v1_SetMotorPowerRequest_init_zero {false, star_v1_RequestHeader_init_zero, false,
        star_v1_MotorPowerCommand_init_zero}
00242 #define star_v1_SetMotorPowerResponse_init_zero {false, star_v1_ResponseHeader_init_zero}
00243 #define star_v1_MotorPowerCommand_init_zero {0, 0}
00244 #define star_v1_StreamEncodersRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0}
00245 #define star_v1_EncoderData_init_zero {0, 0, 0, 0}
00246 #define star_v1_MotorStatus_init_zero {0, 0, 0, 0, 0, 0, 0, 0, _star_v1_MotorState_MIN}
00247 #define star_v1_PidConfig_init_zero {0, 0, 0, 0, 0, 0}
00248
00249 /* Field tags (for use in manual encoding/decoding) */
00250 #define star_v1_VelocityCommand_front_left_velocity_mps_tag 1
00251 #define star_v1_VelocityCommand_front_right_velocity_mps_tag 2
00252 #define star_v1_VelocityCommand_sequence_tag 3
00253 #define star_v1_VelocityCommand_timestamp_us_tag 4
00254 #define star_v1_VelocityCommand_back_left_velocity_mps_tag 5
00255 #define star_v1_VelocityCommand_back_right_velocity_mps_tag 6
00256 #define star_v1_SetVelocityRequest_header_tag 1
00257 #define star_v1_SetVelocityRequest_command_tag 2
00258 #define star_v1_EmergencyStopRequest_header_tag 1
00259 #define star_v1_EmergencyStopRequest_reason_tag 2
00260 #define star_v1_EmergencyStopResponse_header_tag 1
00261 #define star_v1_EmergencyStopResponse_estop_engaged_tag 2
00262 #define star_v1_SetMotorPowerResponse_header_tag 1
00263 #define star_v1_MotorPowerCommand_motor_id_tag 1
00264 #define star_v1_MotorPowerCommand_duty_cycle_percent_tag 2
00265 #define star_v1_SetMotorPowerRequest_header_tag 1
00266 #define star_v1_SetMotorPowerRequest_command_tag 2
00267 #define star_v1_StreamEncodersRequest_header_tag 1
00268 #define star_v1_StreamEncodersRequest_rate_hz_tag 2
00269 #define star_v1_EncoderData_motor_id_tag 1
00270 #define star_v1_EncoderData_ticks_tag 2
00271 #define star_v1_EncoderData_velocity_mps_tag 3
00272 #define star_v1_EncoderData_timestamp_us_tag 4
00273 #define star_v1_MotorStatus_motor_id_tag 1
00274 #define star_v1_MotorStatus_duty_cycle_percent_tag 2
00275 #define star_v1_MotorStatus_velocity_mps_tag 3
00276 #define star_v1_MotorStatus_target_velocity_mps_tag 4
00277 #define star_v1_MotorStatus_temperature_celsius_tag 5
00278 #define star_v1_MotorStatus_current_ma_tag 6
00279 #define star_v1_MotorStatus_fault_flags_tag 7
00280 #define star_v1_MotorStatus_state_tag 8
00281 #define star_v1_SetVelocityResponse_header_tag 1
00282 #define star_v1_SetVelocityResponse_motor_status_tag 2
00283 #define star_v1_PidConfig_kp_tag 1
00284 #define star_v1_PidConfig_ki_tag 2
00285 #define star_v1_PidConfig_kd_tag 3
00286 #define star_v1_PidConfig_output_min_percent_tag 4
00287 #define star_v1_PidConfig_output_max_percent_tag 5
00288 #define star_v1_PidConfig_integral_min_tag 6
00289 #define star_v1_PidConfig_integral_max_tag 7
00290
00291 /* Struct field encoding specification for nanopb */
00292 #define star_v1_SetVelocityRequest_FIELDLIST(X, a) \
00293 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00294 X(a, STATIC, OPTIONAL, MESSAGE, command, 2)
00295 #define star_v1_SetVelocityRequest_CALLBACK NULL
00296 #define star_v1_SetVelocityRequest_DEFAULT NULL
00297 #define star_v1_SetVelocityRequest_header_MSGTYPE star_v1_RequestHeader
00298 #define star_v1_SetVelocityRequest_command_MSGTYPE star_v1_VelocityCommand
00299
00300 #define star_v1_SetVelocityResponse_FIELDLIST(X, a) \
00301 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00302 X(a, STATIC, REPEATED, MESSAGE, motor_status, 2)
00303 #define star_v1_SetVelocityResponse_CALLBACK NULL
00304 #define star_v1_SetVelocityResponse_DEFAULT NULL
00305 #define star_v1_SetVelocityResponse_header_MSGTYPE star_v1_ResponseHeader
00306 #define star_v1_SetVelocityResponse_motor_status_MSGTYPE star_v1_MotorStatus
00307
00308 #define star_v1_VelocityCommand_FIELDLIST(X, a) \
00309 X(a, STATIC, SINGULAR, DOUBLE, front_left_velocity_mps, 1) \
00310 X(a, STATIC, SINGULAR, DOUBLE, front_right_velocity_mps, 2) \
00311 X(a, STATIC, SINGULAR, UINT32, sequence, 3) \
00312 X(a, STATIC, SINGULAR, INT64, timestamp_us, 4) \
00313 X(a, STATIC, SINGULAR, DOUBLE, back_left_velocity_mps, 5) \
00314 X(a, STATIC, SINGULAR, DOUBLE, back_right_velocity_mps, 6)
00315 #define star_v1_VelocityCommand_CALLBACK NULL
00316 #define star_v1_VelocityCommand_DEFAULT NULL
00317
00318 #define star_v1_EmergencyStopRequest_FIELDLIST(X, a) \
00319 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00320 X(a, STATIC, SINGULAR, STRING, reason, 2)
00321 #define star_v1_EmergencyStopRequest_CALLBACK NULL
00322 #define star_v1_EmergencyStopRequest_DEFAULT NULL
00323 #define star_v1_EmergencyStopRequest_header_MSGTYPE star_v1_RequestHeader
00324
00325 #define star_v1_EmergencyStopResponse_FIELDLIST(X, a) \
00326 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \

```

```

00327 X(a, STATIC, SINGULAR, BOOL, estop_engaged, 2)
00328 #define star_v1_EmergencyStopResponse_CALLBACK NULL
00329 #define star_v1_EmergencyStopResponse_DEFAULT NULL
00330 #define star_v1_EmergencyStopResponse_header_MSGTYPE star_v1_ResponseHeader
00331
00332 #define star_v1_SetMotorPowerRequest_FIELDLIST(X, a) \
00333 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00334 X(a, STATIC, OPTIONAL, MESSAGE, command, 2)
00335 #define star_v1_SetMotorPowerRequest_CALLBACK NULL
00336 #define star_v1_SetMotorPowerRequest_DEFAULT NULL
00337 #define star_v1_SetMotorPowerRequest_header_MSGTYPE star_v1_RequestHeader
00338 #define star_v1_SetMotorPowerRequest_command_MSGTYPE star_v1_MotorPowerCommand
00339
00340 #define star_v1_SetMotorPowerResponse_FIELDLIST(X, a) \
00341 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00342 #define star_v1_SetMotorPowerResponse_CALLBACK NULL
00343 #define star_v1_SetMotorPowerResponse_DEFAULT NULL
00344 #define star_v1_SetMotorPowerResponse_header_MSGTYPE star_v1_ResponseHeader
00345
00346 #define star_v1_MotorPowerCommand_FIELDLIST(X, a) \
00347 X(a, STATIC, SINGULAR, INT32, motor_id, 1) \
00348 X(a, STATIC, SINGULAR, DOUBLE, duty_cycle_percent, 2)
00349 #define star_v1_MotorPowerCommand_CALLBACK NULL
00350 #define star_v1_MotorPowerCommand_DEFAULT NULL
00351
00352 #define star_v1_StreamEncodersRequest_FIELDLIST(X, a) \
00353 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00354 X(a, STATIC, SINGULAR, INT32, rate_hz, 2)
00355 #define star_v1_StreamEncodersRequest_CALLBACK NULL
00356 #define star_v1_StreamEncodersRequest_DEFAULT NULL
00357 #define star_v1_StreamEncodersRequest_header_MSGTYPE star_v1_RequestHeader
00358
00359 #define star_v1_EncoderData_FIELDLIST(X, a) \
00360 X(a, STATIC, SINGULAR, INT32, motor_id, 1) \
00361 X(a, STATIC, SINGULAR, INT64, ticks, 2) \
00362 X(a, STATIC, SINGULAR, DOUBLE, velocity_mps, 3) \
00363 X(a, STATIC, SINGULAR, INT64, timestamp_us, 4)
00364 #define star_v1_EncoderData_CALLBACK NULL
00365 #define star_v1_EncoderData_DEFAULT NULL
00366
00367 #define star_v1_MotorStatus_FIELDLIST(X, a) \
00368 X(a, STATIC, SINGULAR, INT32, motor_id, 1) \
00369 X(a, STATIC, SINGULAR, DOUBLE, duty_cycle_percent, 2) \
00370 X(a, STATIC, SINGULAR, DOUBLE, velocity_mps, 3) \
00371 X(a, STATIC, SINGULAR, DOUBLE, target_velocity_mps, 4) \
00372 X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 5) \
00373 X(a, STATIC, SINGULAR, DOUBLE, current_ma, 6) \
00374 X(a, STATIC, SINGULAR, UINT32, fault_flags, 7) \
00375 X(a, STATIC, SINGULAR, UENUM, state, 8)
00376 #define star_v1_MotorStatus_CALLBACK NULL
00377 #define star_v1_MotorStatus_DEFAULT NULL
00378
00379 #define star_v1_PidConfig_FIELDLIST(X, a) \
00380 X(a, STATIC, SINGULAR, DOUBLE, kp, 1) \
00381 X(a, STATIC, SINGULAR, DOUBLE, ki, 2) \
00382 X(a, STATIC, SINGULAR, DOUBLE, kd, 3) \
00383 X(a, STATIC, SINGULAR, DOUBLE, output_min_percent, 4) \
00384 X(a, STATIC, SINGULAR, DOUBLE, output_max_percent, 5) \
00385 X(a, STATIC, SINGULAR, DOUBLE, integral_min, 6) \
00386 X(a, STATIC, SINGULAR, DOUBLE, integral_max, 7)
00387 #define star_v1_PidConfig_CALLBACK NULL
00388 #define star_v1_PidConfig_DEFAULT NULL
00389
00390 extern const pb_msgdesc_t star_v1_SetVelocityRequest_msg;
00391 extern const pb_msgdesc_t star_v1_SetVelocityResponse_msg;
00392 extern const pb_msgdesc_t star_v1_VelocityCommand_msg;
00393 extern const pb_msgdesc_t star_v1_EmergencyStopRequest_msg;
00394 extern const pb_msgdesc_t star_v1_EmergencyStopResponse_msg;
00395 extern const pb_msgdesc_t star_v1_SetMotorPowerRequest_msg;
00396 extern const pb_msgdesc_t star_v1_SetMotorPowerResponse_msg;
00397 extern const pb_msgdesc_t star_v1_MotorPowerCommand_msg;
00398 extern const pb_msgdesc_t star_v1_StreamEncodersRequest_msg;
00399 extern const pb_msgdesc_t star_v1_EncoderData_msg;
00400 extern const pb_msgdesc_t star_v1_MotorStatus_msg;
00401 extern const pb_msgdesc_t star_v1_PidConfig_msg;
00402
00403 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00404 #define star_v1_SetVelocityRequest_fields &star_v1_SetVelocityRequest_msg
00405 #define star_v1_SetVelocityResponse_fields &star_v1_SetVelocityResponse_msg
00406 #define star_v1_VelocityCommand_fields &star_v1_VelocityCommand_msg
00407 #define star_v1_EmergencyStopRequest_fields &star_v1_EmergencyStopRequest_msg
00408 #define star_v1_EmergencyStopResponse_fields &star_v1_EmergencyStopResponse_msg
00409 #define star_v1_SetMotorPowerRequest_fields &star_v1_SetMotorPowerRequest_msg
00410 #define star_v1_SetMotorPowerResponse_fields &star_v1_SetMotorPowerResponse_msg
00411 #define star_v1_MotorPowerCommand_fields &star_v1_MotorPowerCommand_msg
00412 #define star_v1_StreamEncodersRequest_fields &star_v1_StreamEncodersRequest_msg
00413 #define star_v1_EncoderData_fields &star_v1_EncoderData_msg

```

```

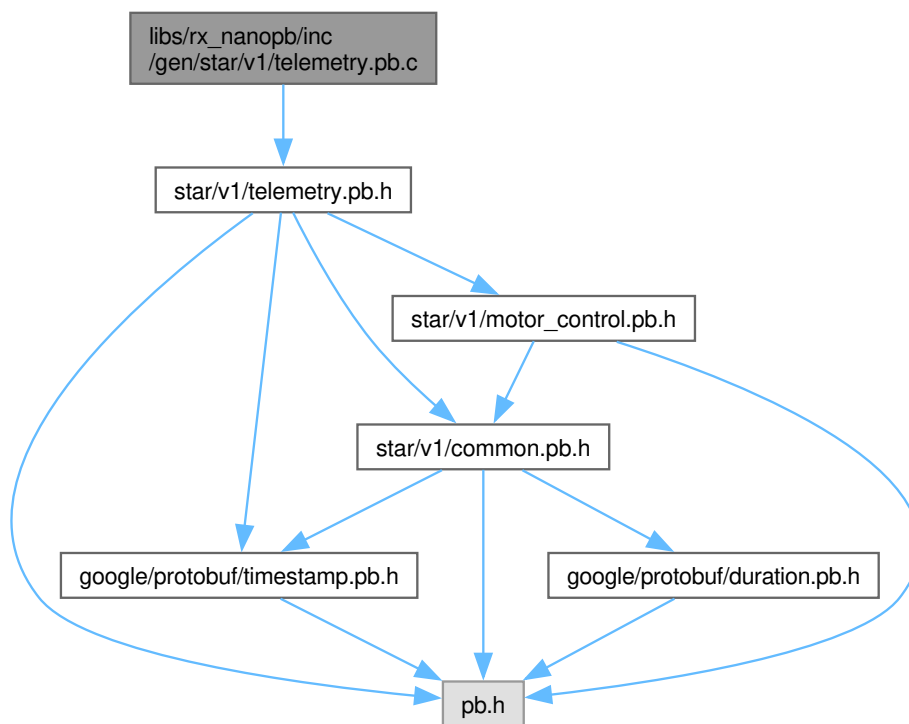
00414 #define star_v1_MotorStatus_fields &star_v1_MotorStatus_msg
00415 #define star_v1_PidConfig_fields &star_v1_PidConfig_msg
00416
00417 /* Maximum encoded size of messages (where known) */
00418 #define STAR_V1_STAR_V1_MOTOR_CONTROL_PB_H_MAX_SIZE star_v1_SetVelocityResponse_size
00419 #define star_v1_EmergencyStopRequest_size 279
00420 #define star_v1_EmergencyStopResponse_size 365
00421 #define star_v1_EncoderData_size 42
00422 #define star_v1_MotorPowerCommand_size 20
00423 #define star_v1_MotorStatus_size 64
00424 #define star_v1_PidConfig_size 63
00425 #define star_v1_SetMotorPowerRequest_size 171
00426 #define star_v1_SetMotorPowerResponse_size 363
00427 #define star_v1_SetVelocityRequest_size 204
00428 #define star_v1_SetVelocityResponse_size 495
00429 #define star_v1_StreamEncodersRequest_size 160
00430 #define star_v1_VelocityCommand_size 53
00431
00432 #ifdef __cplusplus
00433 } /* extern "C" */
00434 #endif
00435
00436 #endif

```

6.37 libs/rx_nanopb/inc/gen/star/v1/telemetry.pb.c File Reference

#include "star/v1/telemetry.pb.h"

Include dependency graph for telemetry.pb.c:



6.38 telemetry.pb.c

[Go to the documentation of this file.](#)

```

00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "star/v1/telemetry.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40

```



```

00006 #error Regenerate this file with the current version of nanopb generator.
00007 #endif
00008
00009 PB_BIND(star_v1_GetTelemetryRequest, star_v1_GetTelemetryRequest, AUTO)
00010
00011
00012 PB_BIND(star_v1_GetTelemetryResponse, star_v1_GetTelemetryResponse, 2)
00013
00014
00015 PB_BIND(star_v1_StreamTelemetryRequest, star_v1_StreamTelemetryRequest, 2)
00016
00017
00018 PB_BIND(star_v1_GetSystemStatusRequest, star_v1_GetSystemStatusRequest, AUTO)
00019
00020
00021 PB_BIND(star_v1_GetSystemStatusResponse, star_v1_GetSystemStatusResponse, 2)
00022
00023
00024 PB_BIND(star_v1_TelemetryData, star_v1_TelemetryData, 2)
00025
00026
00027 PB_BIND(star_v1_ImuData, star_v1_ImuData, AUTO)
00028
00029
00030 PB_BIND(star_v1_BaroData, star_v1_BaroData, AUTO)
00031
00032
00033 PB_BIND(star_v1_ObstacleData, star_v1_ObstacleData, AUTO)
00034
00035
00036 PB_BIND(star_v1_GpsData, star_v1_GpsData, AUTO)
00037
00038
00039 PB_BIND(star_v1_SystemStatus, star_v1_SystemStatus, AUTO)
00040
00041
00042
00043
00044
00045
00046
00047
00048
00049
00050
00051 #ifndef PB_CONVERT_DOUBLE_FLOAT
00052 /* On some platforms (such as AVR), double is really float.
00053  * To be able to encode/decode double on these platforms, you need.
00054  * to define PB_CONVERT_DOUBLE_FLOAT in pb.h or compiler command line.
00055  */
00056 PB_STATIC_ASSERT(sizeof(double) == 8, DOUBLE_MUST_BE_8_BYTES)
00057 #endif
00058

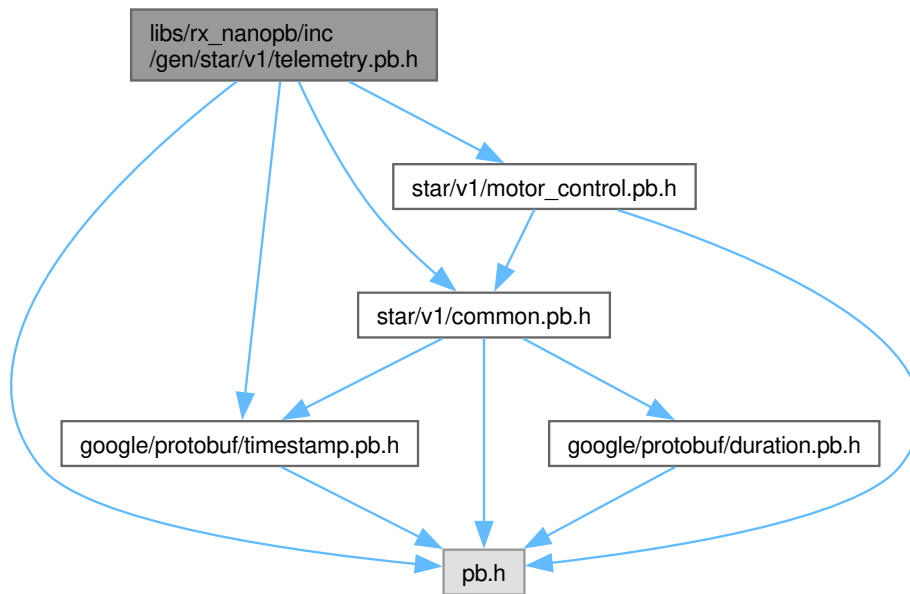
```

6.39 libs/rx_nanopb/inc/gen/star/v1/telemetry.pb.h File Reference

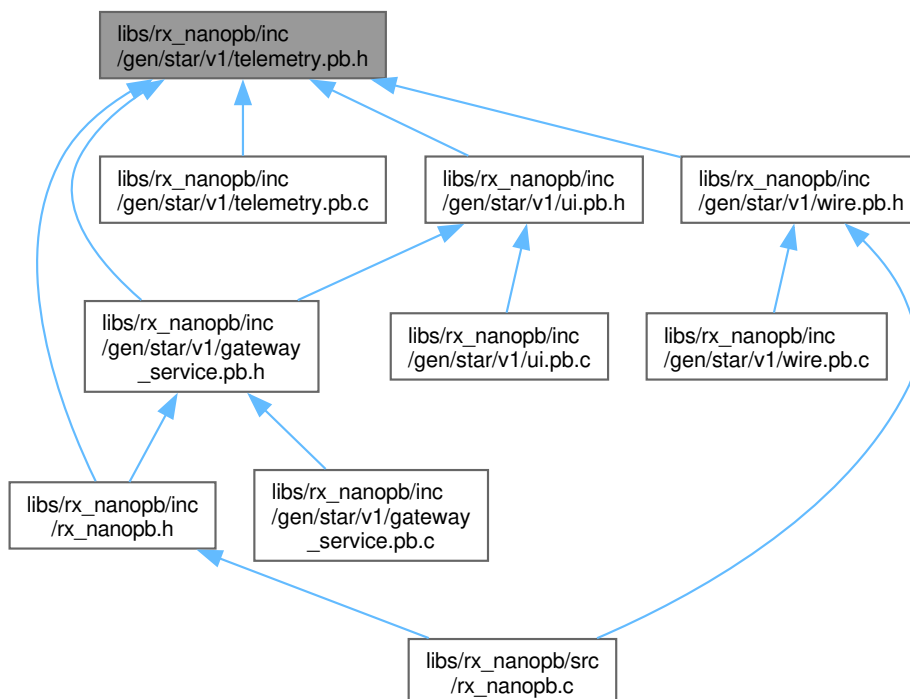
```

#include <pb.h>
#include "google/protobuf/timestamp.pb.h"
#include "star/v1/common.pb.h"
#include "star/v1/motor_control.pb.h"
Include dependency graph for telemetry.pb.h:

```



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `star_v1_GetTelemetryRequest`
- struct `star_v1_StreamTelemetryRequest`
- struct `star_v1_GetSystemStatusRequest`
- struct `star_v1_ImuData`
- struct `star_v1_BaroData`
- struct `star_v1_ObstacleData`
- struct `star_v1_GpsData`
- struct `star_v1_TelemetryData`
- struct `star_v1_GetTelemetryResponse`
- struct `star_v1_SystemStatus`
- struct `star_v1_GetSystemStatusResponse`

Macros

- `#define _star_v1_EstopReason_MIN star_v1_EstopReason_STOP_REASON_UNKNOWN`
- `#define _star_v1_EstopReason_MAX star_v1_EstopReason_STOP_REASON_OVERCURRENT`
- `#define _star_v1_EstopReason_ARRAYSIZE ((star_v1_EstopReason)(star_v1_EstopReason_STOP_REASON_UNKNOWN+1))`
- `#define _star_v1_GpsFix_MIN star_v1_GpsFix_GPS_FIX_UNKNOWN`
- `#define _star_v1_GpsFix_MAX star_v1_GpsFix_GPS_FIX_RTK_FIXED`
- `#define _star_v1_GpsFix_ARRAYSIZE ((star_v1_GpsFix)(star_v1_GpsFix_GPS_FIX_RTK_FIXED+1))`
- `#define _star_v1_ConnectionStatus_MIN star_v1_ConnectionStatus_CONNECTION_STATUS_UNKNOWN`
- `#define _star_v1_ConnectionStatus_MAX star_v1_ConnectionStatus_CONNECTION_STATUS_ERROR`
- `#define _star_v1_ConnectionStatus_ARRAYSIZE ((star_v1_ConnectionStatus)(star_v1_ConnectionStatus_CONNECTION_STATUS_ERROR+1))`
- `#define _star_v1_RobotMode_MIN star_v1_RobotMode_ROBOT_MODE_UNKNOWN`
- `#define _star_v1_RobotMode_MAX star_v1_RobotMode_ROBOT_MODE_EMERGENCY_STOP`
- `#define _star_v1_RobotMode_ARRAYSIZE ((star_v1_RobotMode)(star_v1_RobotMode_ROBOT_MODE_EMERGENCY_STOP+1))`
- `#define star_v1_TelemetryData_estop_reason_ENUMTYPE star_v1_EstopReason`
- `#define star_v1_GpsData_fix_type_ENUMTYPE star_v1_GpsFix`
- `#define star_v1_SystemStatus_connection_status_ENUMTYPE star_v1_ConnectionStatus`
- `#define star_v1_SystemStatus_mode_ENUMTYPE star_v1_RobotMode`
- `#define star_v1_GetTelemetryRequest_init_default {false, star_v1_RequestHeader_init_default}`
- `#define star_v1_GetTelemetryResponse_init_default {false, star_v1_ResponseHeader_init_default, false, star_v1_TelemetryData_init_default}`
- `#define star_v1_StreamTelemetryRequest_init_default {false, star_v1_RequestHeader_init_default, 0, 0, {"", "", "", "", "", "", "", "", "", "", "", "", "", "", ""}}`
- `#define star_v1_GetSystemStatusRequest_init_default {false, star_v1_RequestHeader_init_default}`
- `#define star_v1_GetSystemStatusResponse_init_default {false, star_v1_ResponseHeader_init_default, false, star_v1_SystemStatus_init_default}`
- `#define star_v1_TelemetryData_init_default {false, star_v1_ImuData_init_default, false, star_v1_GpsData_init_default, false, star_v1_ObstacleData_init_default, 0, 0, 0, 0, false, google_protobuf_Timestamp_init_default, _star_v1_EstopReason_MIN, 0, 0, false, star_v1_BaroData_init_default, 0, false, star_v1_EncoderData_init_default, false, star_v1_EncoderData_init_default, 0}`
- `#define star_v1_ImuData_init_default {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}`
- `#define star_v1_BaroData_init_default {0, 0}`
- `#define star_v1_ObstacleData_init_default {0, 0, 0, 0, 0, 0}`
- `#define star_v1_GpsData_init_default {0, 0, 0, 0, 0, _star_v1_GpsFix_MIN}`
- `#define star_v1_SystemStatus_init_default {_star_v1_ConnectionStatus_MIN, _star_v1_RobotMode_MIN, 0, 0, 0, "", 0, 0}`
- `#define star_v1_GetTelemetryRequest_init_zero {false, star_v1_RequestHeader_init_zero}`
- `#define star_v1_GetTelemetryResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_TelemetryData_init_zero}`
- `#define star_v1_StreamTelemetryRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0, 0, {"", "", "", "", "", "", "", "", "", "", "", "", "", "", ""}}`
- `#define star_v1_GetSystemStatusRequest_init_zero {false, star_v1_RequestHeader_init_zero}`
- `#define star_v1_GetSystemStatusResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_SystemStatus_init_zero}`
- `#define star_v1_TelemetryData_init_zero {false, star_v1_ImuData_init_zero, false, star_v1_GpsData_init_zero, false, star_v1_ObstacleData_init_zero, 0, 0, 0, 0, false, google_protobuf_Timestamp_init_zero, _star_v1_EstopReason_MIN, 0, 0, false, star_v1_BaroData_init_zero, 0, false, star_v1_EncoderData_init_zero, false, star_v1_EncoderData_init_zero, false, star_v1_EncoderData_init_zero, 0}`
- `#define star_v1_ImuData_init_zero {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}`
- `#define star_v1_BaroData_init_zero {0, 0}`
- `#define star_v1_ObstacleData_init_zero {0, 0, 0, 0, 0, 0}`
- `#define star_v1_GpsData_init_zero {0, 0, 0, 0, 0, _star_v1_GpsFix_MIN}`
- `#define star_v1_SystemStatus_init_zero {_star_v1_ConnectionStatus_MIN, _star_v1_RobotMode_MIN, 0, 0, 0, "", 0, 0}`

- `#define star_v1_GetTelemetryRequest_header_tag 1`
- `#define star_v1_StreamTelemetryRequest_header_tag 1`
- `#define star_v1_StreamTelemetryRequest_rate_hz_tag 2`
- `#define star_v1_StreamTelemetryRequest_fields_tag 3`
- `#define star_v1_GetSystemStatusRequest_header_tag 1`
- `#define star_v1_ImuData_pitch_rad_tag 1`
- `#define star_v1_ImuData_roll_rad_tag 2`
- `#define star_v1_ImuData_yaw_rad_tag 3`
- `#define star_v1_ImuData_accel_x_mps2_tag 4`
- `#define star_v1_ImuData_accel_y_mps2_tag 5`
- `#define star_v1_ImuData_accel_z_mps2_tag 6`
- `#define star_v1_ImuData_gyro_x_rad_per_s_tag 7`
- `#define star_v1_ImuData_gyro_y_rad_per_s_tag 8`
- `#define star_v1_ImuData_gyro_z_rad_per_s_tag 9`
- `#define star_v1_ImuData_heading_rad_tag 10`
- `#define star_v1_ImuData_quat_w_tag 11`
- `#define star_v1_ImuData_quat_x_tag 12`
- `#define star_v1_ImuData_quat_y_tag 13`
- `#define star_v1_ImuData_quat_z_tag 14`
- `#define star_v1_ImuData_calibration_status_tag 15`
- `#define star_v1_ImuData_temperature_celsius_tag 16`
- `#define star_v1_BaroData_temperature_celsius_tag 1`
- `#define star_v1_BaroData_pressure_pa_tag 2`
- `#define star_v1_ObstacleData_distance_front_left_m_tag 1`
- `#define star_v1_ObstacleData_distance_front_right_m_tag 2`
- `#define star_v1_ObstacleData_distance_back_left_m_tag 3`
- `#define star_v1_ObstacleData_distance_back_right_m_tag 4`
- `#define star_v1_ObstacleData_any_obstacle_tag 5`
- `#define star_v1_ObstacleData_detected_mask_tag 6`
- `#define star_v1_GpsData_latitude_deg_tag 1`
- `#define star_v1_GpsData_longitude_deg_tag 2`
- `#define star_v1_GpsData_altitude_m_tag 3`
- `#define star_v1_GpsData_accuracy_m_tag 4`
- `#define star_v1_GpsData_satellites_tag 5`
- `#define star_v1_GpsData_fix_type_tag 6`
- `#define star_v1_TelemetryData_imu_tag 1`
- `#define star_v1_TelemetryData_gps_tag 2`
- `#define star_v1_TelemetryData_obstacle_tag 3`
- `#define star_v1_TelemetryData_wifi_signal_dbm_tag 4`
- `#define star_v1_TelemetryData_cpu_usage_percent_tag 5`
- `#define star_v1_TelemetryData_temperature_celsius_tag 6`
- `#define star_v1_TelemetryData_motor_load_percent_tag 7`
- `#define star_v1_TelemetryData_timestamp_tag 8`
- `#define star_v1_TelemetryData_estop_reason_tag 9`
- `#define star_v1_TelemetryData_emergency_stop_tag 10`
- `#define star_v1_TelemetryData_fault_flags_tag 11`
- `#define star_v1_TelemetryData_baro_tag 12`
- `#define star_v1_TelemetryData_timestamp_us_tag 14`
- `#define star_v1_TelemetryData_encoder_front_left_tag 15`
- `#define star_v1_TelemetryData_encoder_front_right_tag 16`
- `#define star_v1_TelemetryData_encoder_back_left_tag 17`
- `#define star_v1_TelemetryData_encoder_back_right_tag 18`
- `#define star_v1_TelemetryData_frame_sequence_tag 19`
- `#define star_v1_GetTelemetryResponse_header_tag 1`
- `#define star_v1_GetTelemetryResponse_telemetry_tag 2`

- `#define star_v1_SystemStatus_connection_status_tag 1`
- `#define star_v1_SystemStatus_mode_tag 2`
- `#define star_v1_SystemStatus_rx72n_connected_tag 3`
- `#define star_v1_SystemStatus_lidar_connected_tag 4`
- `#define star_v1_SystemStatus_ros_connected_tag 5`
- `#define star_v1_SystemStatus_firmware_version_tag 6`
- `#define star_v1_SystemStatus_uptime_s_tag 7`
- `#define star_v1_SystemStatus_free_heap_bytes_tag 8`
- `#define star_v1_GetSystemStatusResponse_header_tag 1`
- `#define star_v1_GetSystemStatusResponse_status_tag 2`
- `#define star_v1_GetTelemetryRequest_FIELDLIST(X, a)`
- `#define star_v1_GetTelemetryRequest_CALLBACK NULL`
- `#define star_v1_GetTelemetryRequest_DEFAULT NULL`
- `#define star_v1_GetTelemetryRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_GetTelemetryResponse_FIELDLIST(X, a)`
- `#define star_v1_GetTelemetryResponse_CALLBACK NULL`
- `#define star_v1_GetTelemetryResponse_DEFAULT NULL`
- `#define star_v1_GetTelemetryResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_GetTelemetryResponse_telemetry_MSGTYPE star_v1_TelemetryData`
- `#define star_v1_StreamTelemetryRequest_FIELDLIST(X, a)`
- `#define star_v1_StreamTelemetryRequest_CALLBACK NULL`
- `#define star_v1_StreamTelemetryRequest_DEFAULT NULL`
- `#define star_v1_StreamTelemetryRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_GetSystemStatusRequest_FIELDLIST(X, a)`
- `#define star_v1_GetSystemStatusRequest_CALLBACK NULL`
- `#define star_v1_GetSystemStatusRequest_DEFAULT NULL`
- `#define star_v1_GetSystemStatusRequest_header_MSGTYPE star_v1_RequestHeader`
- `#define star_v1_GetSystemStatusResponse_FIELDLIST(X, a)`
- `#define star_v1_GetSystemStatusResponse_CALLBACK NULL`
- `#define star_v1_GetSystemStatusResponse_DEFAULT NULL`
- `#define star_v1_GetSystemStatusResponse_header_MSGTYPE star_v1_ResponseHeader`
- `#define star_v1_GetSystemStatusResponse_status_MSGTYPE star_v1_SystemStatus`
- `#define star_v1_TelemetryData_FIELDLIST(X, a)`
- `#define star_v1_TelemetryData_CALLBACK NULL`
- `#define star_v1_TelemetryData_DEFAULT NULL`
- `#define star_v1_TelemetryData_imu_MSGTYPE star_v1_ImuData`
- `#define star_v1_TelemetryData_gps_MSGTYPE star_v1_GpsData`
- `#define star_v1_TelemetryData_obstacle_MSGTYPE star_v1_ObstacleData`
- `#define star_v1_TelemetryData_timestamp_MSGTYPE google_protobuf_Timestamp`
- `#define star_v1_TelemetryData_baro_MSGTYPE star_v1_BaroData`
- `#define star_v1_TelemetryData_encoder_front_left_MSGTYPE star_v1_EncoderData`
- `#define star_v1_TelemetryData_encoder_front_right_MSGTYPE star_v1_EncoderData`
- `#define star_v1_TelemetryData_encoder_back_left_MSGTYPE star_v1_EncoderData`
- `#define star_v1_TelemetryData_encoder_back_right_MSGTYPE star_v1_EncoderData`
- `#define star_v1_ImuData_FIELDLIST(X, a)`
- `#define star_v1_ImuData_CALLBACK NULL`
- `#define star_v1_ImuData_DEFAULT NULL`
- `#define star_v1_BaroData_FIELDLIST(X, a)`
- `#define star_v1_BaroData_CALLBACK NULL`
- `#define star_v1_BaroData_DEFAULT NULL`
- `#define star_v1_ObstacleData_FIELDLIST(X, a)`
- `#define star_v1_ObstacleData_CALLBACK NULL`
- `#define star_v1_ObstacleData_DEFAULT NULL`
- `#define star_v1_GpsData_FIELDLIST(X, a)`
- `#define star_v1_GpsData_CALLBACK NULL`

- `#define star_v1_GpsData_DEFAULT NULL`
- `#define star_v1_SystemStatus_FIELDLIST(X, a)`
- `#define star_v1_SystemStatus_CALLBACK NULL`
- `#define star_v1_SystemStatus_DEFAULT NULL`
- `#define star_v1_GetTelemetryRequest_fields &star_v1_GetTelemetryRequest_msg`
- `#define star_v1_GetTelemetryResponse_fields &star_v1_GetTelemetryResponse_msg`
- `#define star_v1_StreamTelemetryRequest_fields &star_v1_StreamTelemetryRequest_msg`
- `#define star_v1_GetSystemStatusRequest_fields &star_v1_GetSystemStatusRequest_msg`
- `#define star_v1_GetSystemStatusResponse_fields &star_v1_GetSystemStatusResponse_msg`
- `#define star_v1_TelemetryData_fields &star_v1_TelemetryData_msg`
- `#define star_v1_ImuData_fields &star_v1_ImuData_msg`
- `#define star_v1_BaroData_fields &star_v1_BaroData_msg`
- `#define star_v1_ObstacleData_fields &star_v1_ObstacleData_msg`
- `#define star_v1_GpsData_fields &star_v1_GpsData_msg`
- `#define star_v1_SystemStatus_fields &star_v1_SystemStatus_msg`
- `#define STAR_V1_STAR_V1_TELEMETRY_PB_H_MAX_SIZE star_v1_GetTelemetryResponse_size`
- `#define star_v1_BaroData_size 18`
- `#define star_v1_GetSystemStatusRequest_size 149`
- `#define star_v1_GetSystemStatusResponse_size 425`
- `#define star_v1_GetTelemetryRequest_size 149`
- `#define star_v1_GetTelemetryResponse_size 881`
- `#define star_v1_GpsData_size 49`
- `#define star_v1_ImuData_size 142`
- `#define star_v1_ObstacleData_size 28`
- `#define star_v1_StreamTelemetryRequest_size 688`
- `#define star_v1_SystemStatus_size 60`
- `#define star_v1_TelemetryData_size 515`

Enumerations

- `enum star_v1_EstopReason {`
`star_v1_EstopReason_ESTOP_REASON_UNKNOWN = 0 , star_v1_EstopReason_ESTOP_REASON_MANUAL = 1 , star_v1_EstopReason_ESTOP_REASON_FAULT = 2 , star_v1_EstopReason_ESTOP_REASON_COMING_ONLINE = 3 ,`
`star_v1_EstopReason_ESTOP_REASON_OBSTACLE = 4 , star_v1_EstopReason_ESTOP_REASON_OVERHEATING = 5 }`
- `enum star_v1_GpsFix {`
`star_v1_GpsFix_GPS_FIX_UNKNOWN = 0 , star_v1_GpsFix_GPS_FIX_NONE = 1 ,`
`star_v1_GpsFix_GPS_FIX_2D = 2 , star_v1_GpsFix_GPS_FIX_3D = 3 ,`
`star_v1_GpsFix_GPS_FIX_DGPS = 4 , star_v1_GpsFix_GPS_FIX_RTK_FLOAT = 5 ,`
`star_v1_GpsFix_GPS_FIX_RTK_FIXED = 6 }`
- `enum star_v1_ConnectionStatus {`
`star_v1_ConnectionStatus_CONNECTION_STATUS_UNKNOWN = 0 , star_v1_ConnectionStatus_CONNECTION_STATUS_CONNECTED = 1 , star_v1_ConnectionStatus_CONNECTION_STATUS_DISCONNECTED = 2 ,`
`star_v1_ConnectionStatus_CONNECTION_STATUS_CONNECTING = 3 ,`
`star_v1_ConnectionStatus_CONNECTION_STATUS_ERROR = 4 }`
- `enum star_v1_RobotMode {`
`star_v1_RobotMode_ROBOT_MODE_UNKNOWN = 0 , star_v1_RobotMode_ROBOT_MODE_IDLE = 1 , star_v1_RobotMode_ROBOT_MODE_MANUAL = 2 , star_v1_RobotMode_ROBOT_MODE_AUTONOMOUS = 3 ,`
`star_v1_RobotMode_ROBOT_MODE_MAPPING = 4 , star_v1_RobotMode_ROBOT_MODE_EMERGENCY_STOP = 5 }`

Variables

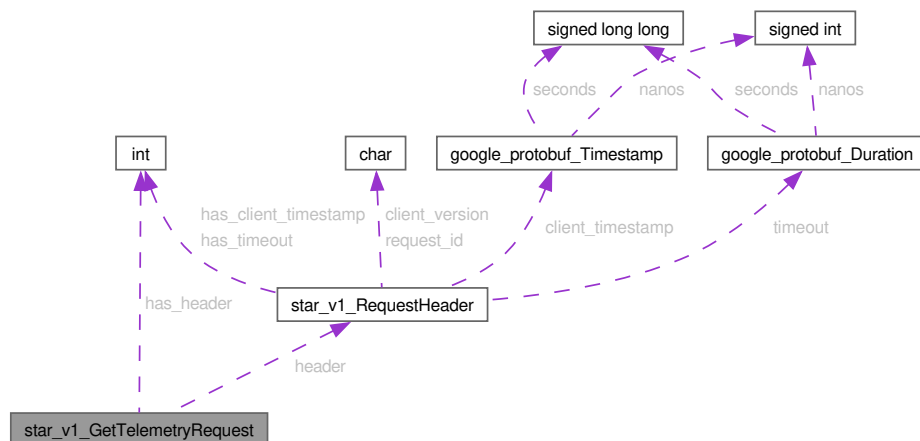
- const pb_msgdesc_t [star_v1_GetTelemetryRequest_msg](#)
- const pb_msgdesc_t [star_v1_GetTelemetryResponse_msg](#)
- const pb_msgdesc_t [star_v1_StreamTelemetryRequest_msg](#)
- const pb_msgdesc_t [star_v1_GetSystemStatusRequest_msg](#)
- const pb_msgdesc_t [star_v1_GetSystemStatusResponse_msg](#)
- const pb_msgdesc_t [star_v1_TelemetryData_msg](#)
- const pb_msgdesc_t [star_v1_ImuData_msg](#)
- const pb_msgdesc_t [star_v1_BaroData_msg](#)
- const pb_msgdesc_t [star_v1_ObstacleData_msg](#)
- const pb_msgdesc_t [star_v1_GpsData_msg](#)
- const pb_msgdesc_t [star_v1_SystemStatus_msg](#)

6.39.1 Data Structure Documentation

6.39.1.1 struct star_v1_GetTelemetryRequest

Definition at line 83 of file [telemetry.pb.h](#).

Collaboration diagram for star_v1_GetTelemetryRequest:



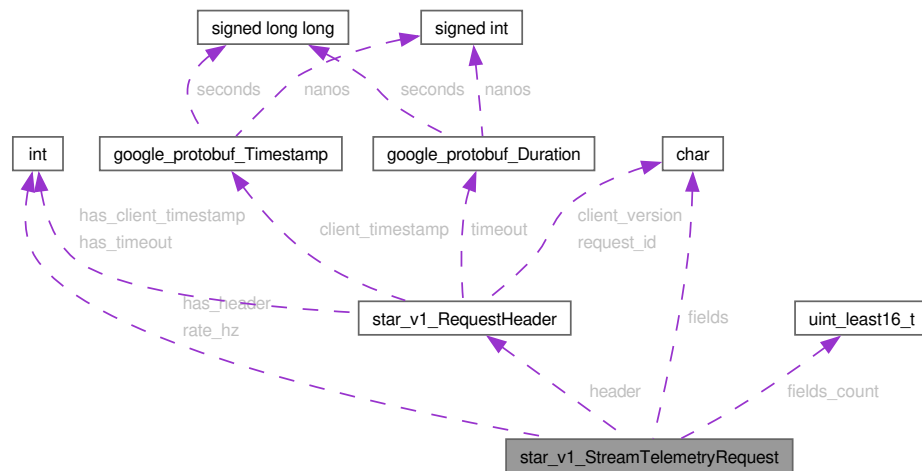
Data Fields

bool	has_header	
star_v1_RequestHeader	header	

6.39.1.2 struct star_v1_StreamTelemetryRequest

Definition at line 90 of file [telemetry.pb.h](#).

Collaboration diagram for star_v1_StreamTelemetryRequest:



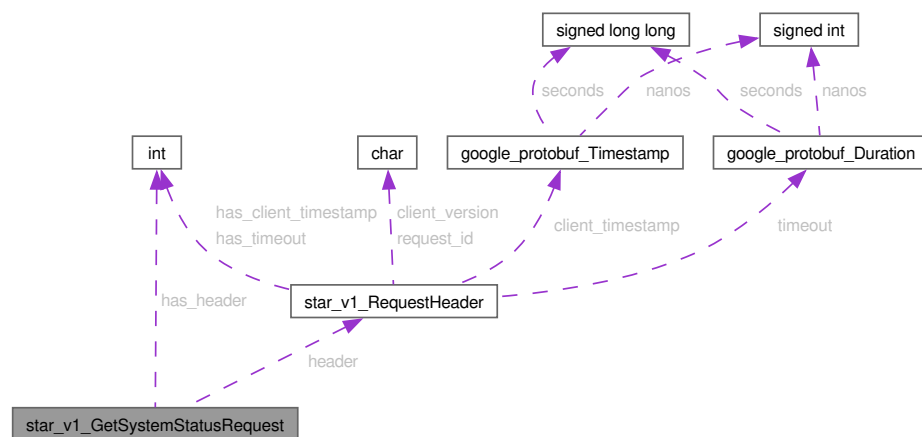
Data Fields

char	fields[16][32]	
pb_size_t	fields_count	
bool	has_header	
star_v1_RequestHeader	header	
int32_t	rate_hz	

6.39.1.3 struct star_v1_GetSystemStatusRequest

Definition at line 103 of file [telemetry.pb.h](#).

Collaboration diagram for star_v1_GetSystemStatusRequest:



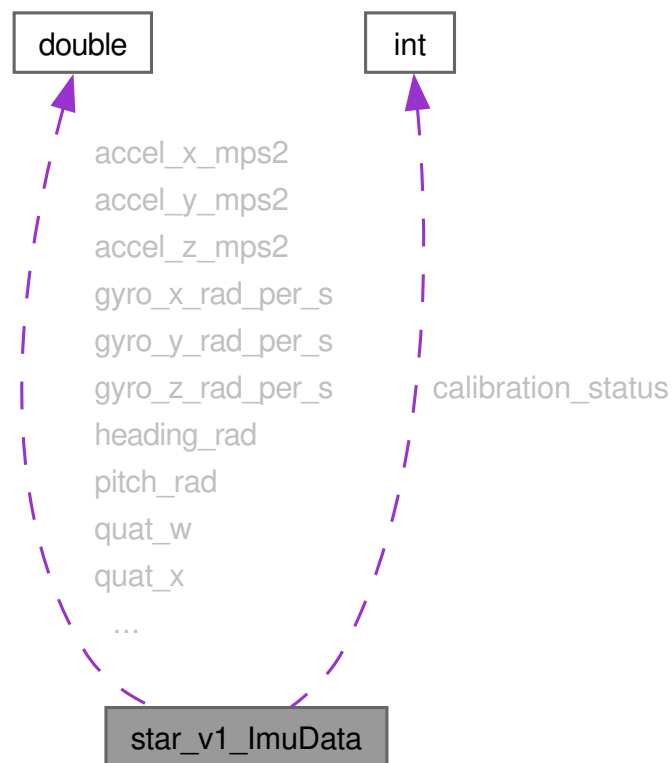
Data Fields

bool	has_header	
star_v1_RequestHeader	header	

6.39.1.4 struct star_v1_ImuData

Definition at line 112 of file [telemetry.pb.h](#).

Collaboration diagram for star_v1_ImuData:



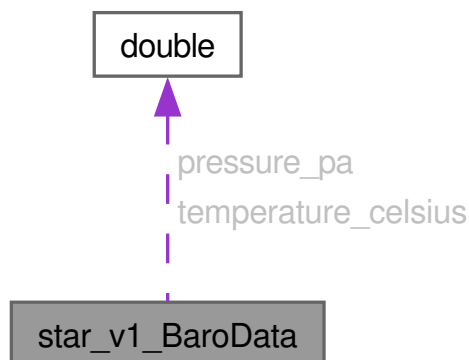
Data Fields

double	accel_x_mps2	
double	accel_y_mps2	
double	accel_z_mps2	
uint32_t	calibration_status	
double	gyro_x_rad_per_s	
double	gyro_y_rad_per_s	
double	gyro_z_rad_per_s	
double	heading_rad	
double	pitch_rad	
double	quat_w	
double	quat_x	
double	quat_y	
double	quat_z	
double	roll_rad	
double	temperature_celsius	
double	yaw_rad	

6.39.1.5 struct star_v1_BaroData

Definition at line 170 of file [telemetry.pb.h](#).

Collaboration diagram for star_v1_BaroData:



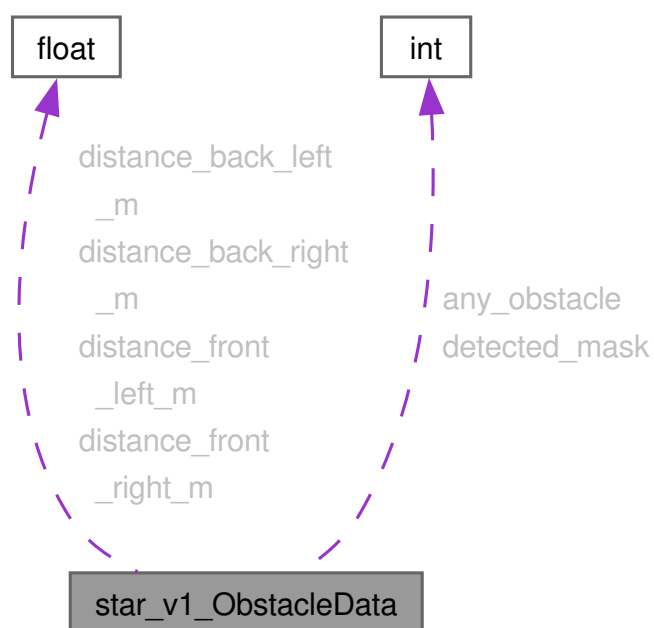
Data Fields

double	pressure_pa	
double	temperature_celsius	

6.39.1.6 struct star_v1_ObstacleData

Definition at line 183 of file [telemetry.pb.h](#).

Collaboration diagram for star_v1_ObstacleData:



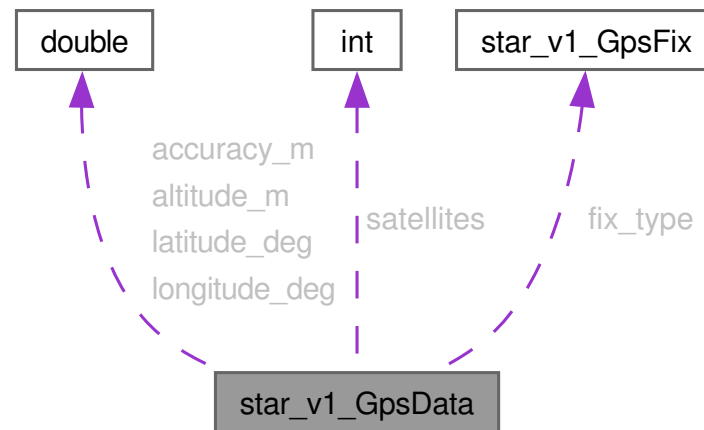
Data Fields

bool	any_obstacle	
uint32_t	detected_mask	
float	distance_back_left_m	
float	distance_back_right_m	
float	distance_front_left_m	
float	distance_front_right_m	

6.39.1.7 struct star_v1_GpsData

Definition at line 200 of file [telemetry.pb.h](#).

Collaboration diagram for star_v1_GpsData:



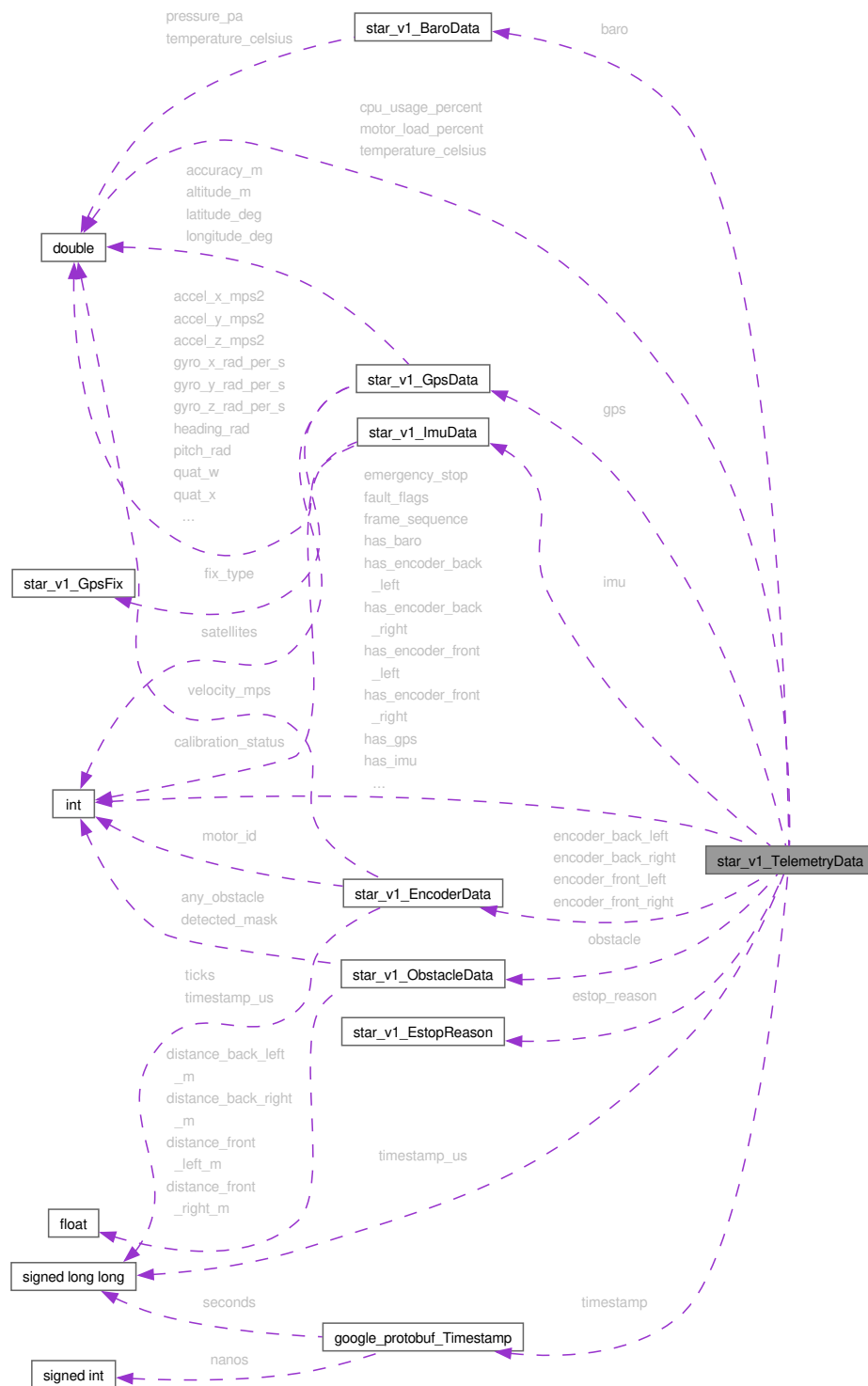
Data Fields

double	accuracy_m	
double	altitude_m	
star_v1_GpsFix	fix_type	
double	latitude_deg	
double	longitude_deg	
int32_t	satellites	

6.39.1.8 struct star_v1_TelemetryData

Definition at line 218 of file [telemetry.pb.h](#).

Collaboration diagram for star_v1_TelemetryData:



Data Fields

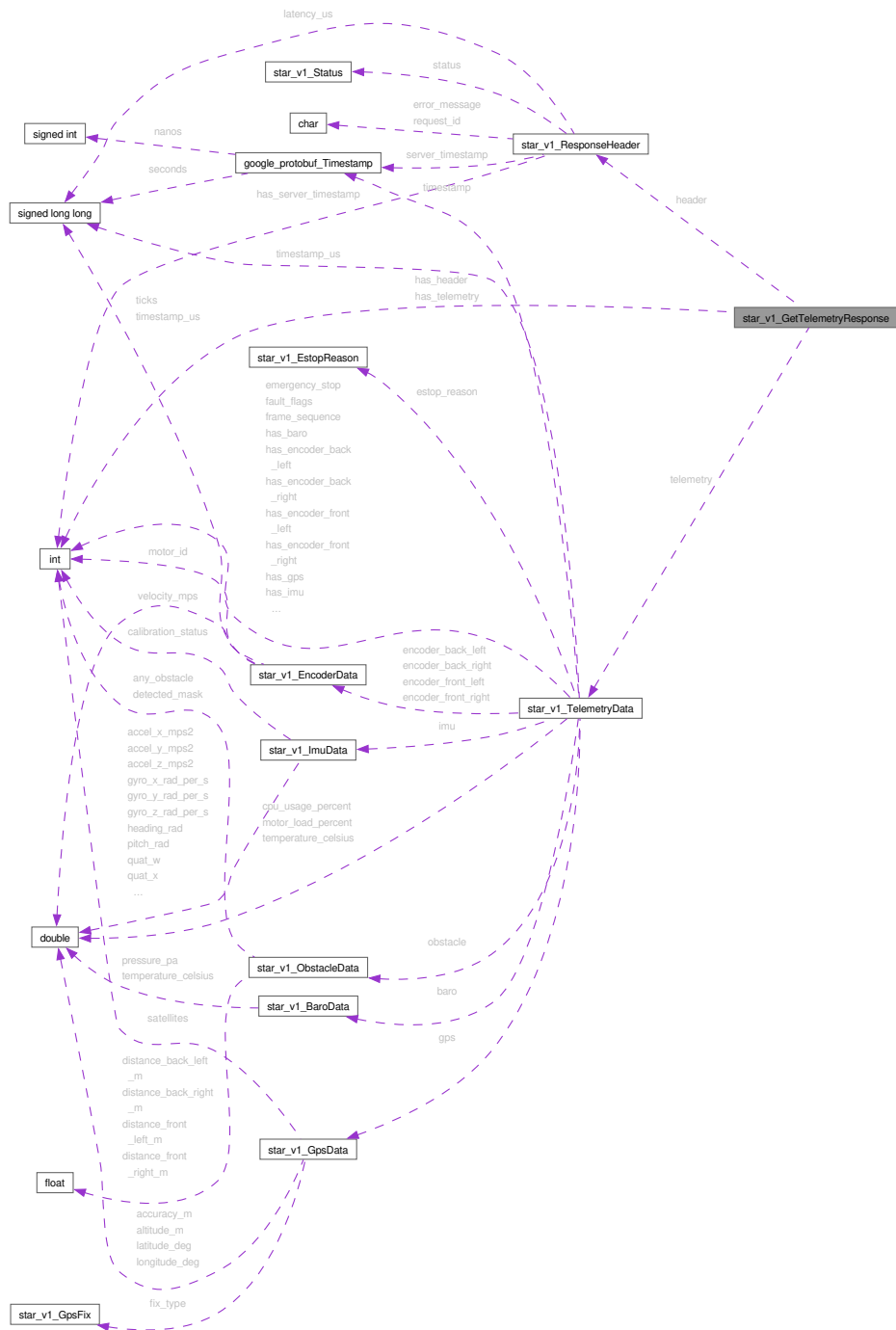
star_v1_Barodata	baro	
double	cpu_usage_percent	
bool	emergency_stop	
star_v1_EncoderData	encoder_back_left	
star_v1_EncoderData	encoder_back_right	
star_v1_EncoderData	encoder_front_left	

star_v1_EncoderData	encoder_front_right	
star_v1_EstopReason	estop_reason	
uint32_t	fault_flags	
uint32_t	frame_sequence	
star_v1_GpsData	gps	
bool	has_baro	
bool	has_encoder_back_left	
bool	has_encoder_back_right	
bool	has_encoder_front_left	
bool	has_encoder_front_right	
bool	has_gps	
bool	has_imu	
bool	has_obstacle	
bool	has_timestamp	
star_v1_ImuData	imu	
double	motor_load_percent	
star_v1_ObstacleData	obstacle	
double	temperature_celsius	
google_protobuf_Timestamp	timestamp	
int64_t	timestamp_us	
int32_t	wifi_signal_dbm	

6.39.1.9 struct star_v1_GetTelemetryResponse

Definition at line 278 of file [telemetry.pb.h](#).

Collaboration diagram for star_v1_GetTelemetryResponse:



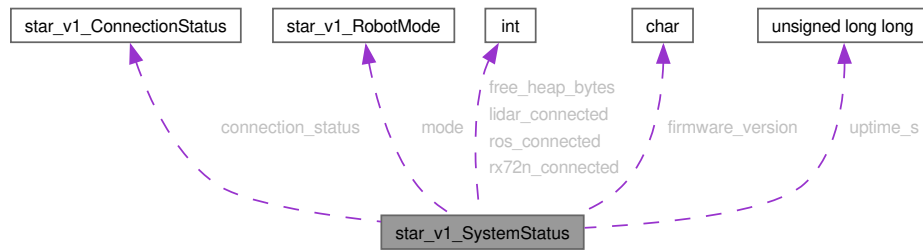
Data Fields

	bool	has_header	
	bool	has_telemetry	
star_v1_ResponseHeader		header	
star_v1_TelemetryData		telemetry	

6.39.1.10 struct star_v1_SystemStatus

Definition at line 288 of file [telemetry.pb.h](#).

Collaboration diagram for `star_v1_SystemStatus`:



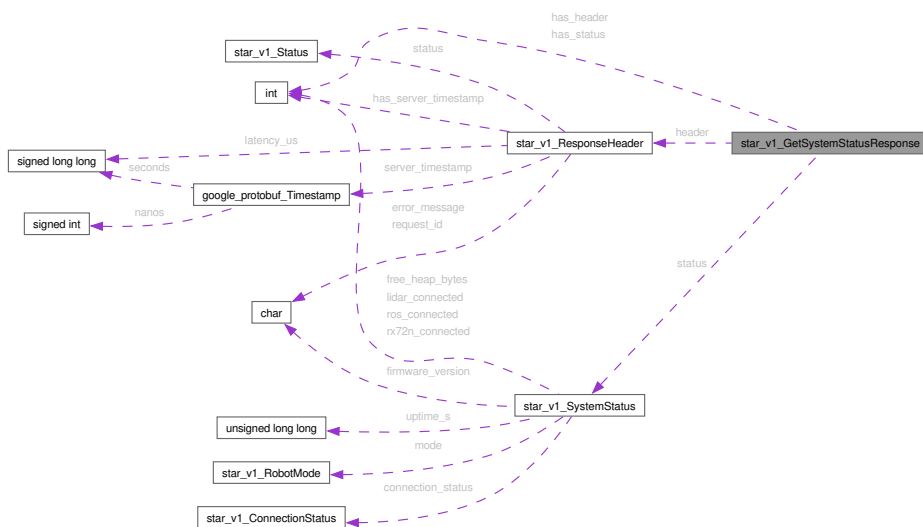
Data Fields

<code>star_v1_ConnectionStatus</code>	<code>connection_status</code>	
<code>char</code>	<code>firmware_version[32]</code>	
<code>uint32_t</code>	<code>free_heap_bytes</code>	
<code>bool</code>	<code>lidar_connected</code>	
<code>star_v1_RobotMode</code>	<code>mode</code>	
<code>bool</code>	<code>ros_connected</code>	
<code>bool</code>	<code>rx72n_connected</code>	
<code>uint64_t</code>	<code>uptime_s</code>	

6.39.1.11 struct `star_v1_GetSystemStatusResponse`

Definition at line 308 of file [telemetry.pb.h](#).

Collaboration diagram for `star_v1_GetSystemStatusResponse`:



Data Fields

<code>bool</code>	<code>has_header</code>	
-------------------	-------------------------	--

bool	has_status	
star_v1_ResponseHeader	header	
star_v1_SystemStatus	status	

6.39.2 Macro Definition Documentation

6.39.2.1 __star_v1_ConnectionStatus_ARRAYSIZE

```
#define __star_v1_ConnectionStatus_ARRAYSIZE ((star_v1_ConnectionStatus)(star_v1_ConnectionStatus_CONNECTION_STATUS_I
```

Definition at line 333 of file [telemetry.pb.h](#).

6.39.2.2 __star_v1_ConnectionStatus_MAX

```
#define __star_v1_ConnectionStatus_MAX star_v1_ConnectionStatus_CONNECTION_STATUS_ERROR
```

Definition at line 332 of file [telemetry.pb.h](#).

6.39.2.3 __star_v1_ConnectionStatus_MIN

```
#define __star_v1_ConnectionStatus_MIN star_v1_ConnectionStatus_CONNECTION_STATUS_UNKNOWN
```

Definition at line 331 of file [telemetry.pb.h](#).

6.39.2.4 __star_v1_EstopReason_ARRAYSIZE

```
#define __star_v1_EstopReason_ARRAYSIZE ((star_v1_EstopReason)(star_v1_EstopReason_ESTOP_REASON_OVERCURRENT+1))
```

Definition at line 325 of file [telemetry.pb.h](#).

6.39.2.5 __star_v1_EstopReason_MAX

```
#define __star_v1_EstopReason_MAX star_v1_EstopReason_ESTOP_REASON_OVERCURRENT
```

Definition at line 324 of file [telemetry.pb.h](#).

6.39.2.6 __star_v1_EstopReason_MIN

```
#define __star_v1_EstopReason_MIN star_v1_EstopReason_ESTOP_REASON_UNKNOWN
```

Definition at line 323 of file [telemetry.pb.h](#).

6.39.2.7 __star_v1_GpsFix_ARRAYSIZE

```
#define __star_v1_GpsFix_ARRAYSIZE ((star_v1_GpsFix)(star_v1_GpsFix_GPS_FIX_RTK_FIXED+1))
```

Definition at line 329 of file [telemetry.pb.h](#).

6.39.2.8 __star_v1_GpsFix_MAX

```
#define __star_v1_GpsFix_MAX star_v1_GpsFix_GPS_FIX_RTK_FIXED
```

Definition at line 328 of file [telemetry.pb.h](#).

6.39.2.9 __star_v1_GpsFix_MIN

```
#define __star_v1_GpsFix_MIN star_v1_GpsFix_GPS_FIX_UNKNOWN
```

Definition at line 327 of file [telemetry.pb.h](#).

6.39.2.10 __star_v1_RobotMode_ARRAYSIZE

```
#define __star_v1_RobotMode_ARRAYSIZE ((star_v1_RobotMode)(star_v1_RobotMode_ROBOT_MODE_EMERGENCY_STOP+1))
```

Definition at line 337 of file [telemetry.pb.h](#).

6.39.2.11 __star_v1_RobotMode_MAX

```
#define __star_v1_RobotMode_MAX star_v1_RobotMode_ROBOT_MODE_EMERGENCY_STOP
```

Definition at line 336 of file [telemetry.pb.h](#).

6.39.2.12 __star_v1_RobotMode_MIN

```
#define __star_v1_RobotMode_MIN star_v1_RobotMode_ROBOT_MODE_UNKNOWN
```

Definition at line 335 of file [telemetry.pb.h](#).

6.39.2.13 star_v1_BaroData_CALLBACK

```
#define star_v1_BaroData_CALLBACK NULL
```

Definition at line 537 of file [telemetry.pb.h](#).

6.39.2.14 star_v1_BaroData_DEFAULT

```
#define star_v1_BaroData_DEFAULT NULL
```

Definition at line 538 of file [telemetry.pb.h](#).

6.39.2.15 star_v1_BaroData_FIELDLIST

```
#define star_v1_BaroData_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 1) \  
X(a, STATIC, SINGULAR, DOUBLE, pressure_pa, 2)
```

Definition at line 534 of file [telemetry.pb.h](#).

```
00534 #define star_v1_BaroData_FIELDLIST(X, a) \  
00535 X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 1) \  
00536 X(a, STATIC, SINGULAR, DOUBLE, pressure_pa, 2)
```

6.39.2.16 star_v1_BaroData_fields

```
#define star_v1_BaroData_fields &star_v1_BaroData_msg
```

Definition at line 592 of file [telemetry.pb.h](#).

6.39.2.17 star_v1_BaroData_init_default

```
#define star_v1_BaroData_init_default {0, 0}
```

Definition at line 363 of file [telemetry.pb.h](#).

6.39.2.18 star_v1_BaroData_init_zero

```
#define star_v1_BaroData_init_zero {0, 0}
```

Definition at line 374 of file [telemetry.pb.h](#).

6.39.2.19 star_v1_BaroData_pressure_pa_tag

```
#define star_v1_BaroData_pressure_pa_tag 2
```

Definition at line 402 of file [telemetry.pb.h](#).

6.39.2.20 star_v1_BaroData_size

```
#define star_v1_BaroData_size 18
```

Definition at line 599 of file [telemetry.pb.h](#).

6.39.2.21 star_v1_BaroData_temperature_celsius_tag

```
#define star_v1_BaroData_temperature_celsius_tag 1
```

Definition at line 401 of file [telemetry.pb.h](#).

6.39.2.22 star_v1_GetSystemStatusRequest_CALLBACK

```
#define star_v1_GetSystemStatusRequest_CALLBACK NULL
```

Definition at line 471 of file [telemetry.pb.h](#).

6.39.2.23 star_v1_GetSystemStatusRequest_DEFAULT

```
#define star_v1_GetSystemStatusRequest_DEFAULT NULL
```

Definition at line 472 of file [telemetry.pb.h](#).

6.39.2.24 star_v1_GetSystemStatusRequest_FIELDLIST

```
#define star_v1_GetSystemStatusRequest_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

Definition at line 469 of file [telemetry.pb.h](#).

```
00469 #define star_v1_GetSystemStatusRequest_FIELDLIST(X, a) \  
00470 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

6.39.2.25 star_v1_GetSystemStatusRequest_fields

```
#define star_v1_GetSystemStatusRequest_fields &star_v1_GetSystemStatusRequest_msg
```

Definition at line 588 of file [telemetry.pb.h](#).

6.39.2.26 star_v1_GetSystemStatusRequest_header_MSGTYPE

```
#define star_v1_GetSystemStatusRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 473 of file [telemetry.pb.h](#).

6.39.2.27 star_v1_GetSystemStatusRequest_header_tag

```
#define star_v1_GetSystemStatusRequest_header_tag 1
```

Definition at line 384 of file [telemetry.pb.h](#).

6.39.2.28 star_v1_GetSystemStatusRequest_init_default

```
#define star_v1_GetSystemStatusRequest_init_default {false, star_v1_RequestHeader_init_default}
```

Definition at line 359 of file [telemetry.pb.h](#).

6.39.2.29 `star_v1_GetSystemStatusRequest_init_zero`

```
#define star_v1_GetSystemStatusRequest_init_zero {false, star_v1_RequestHeader_init_zero}
```

Definition at line 370 of file [telemetry.pb.h](#).

6.39.2.30 `star_v1_GetSystemStatusRequest_size`

```
#define star_v1_GetSystemStatusRequest_size 149
```

Definition at line 600 of file [telemetry.pb.h](#).

6.39.2.31 `star_v1_GetSystemStatusResponse_CALLBACK`

```
#define star_v1_GetSystemStatusResponse_CALLBACK NULL
```

Definition at line 478 of file [telemetry.pb.h](#).

6.39.2.32 `star_v1_GetSystemStatusResponse_DEFAULT`

```
#define star_v1_GetSystemStatusResponse_DEFAULT NULL
```

Definition at line 479 of file [telemetry.pb.h](#).

6.39.2.33 `star_v1_GetSystemStatusResponse_FIELDLIST`

```
#define star_v1_GetSystemStatusResponse_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
X(a, STATIC, OPTIONAL, MESSAGE, status, 2)
```

Definition at line 475 of file [telemetry.pb.h](#).

```
00475 #define star_v1_GetSystemStatusResponse_FIELDLIST(X, a) \  
00476 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
00477 X(a, STATIC, OPTIONAL, MESSAGE, status, 2)
```

6.39.2.34 `star_v1_GetSystemStatusResponse_fields`

```
#define star_v1_GetSystemStatusResponse_fields &star_v1_GetSystemStatusResponse_msg
```

Definition at line 589 of file [telemetry.pb.h](#).

6.39.2.35 `star_v1_GetSystemStatusResponse_header_MSGTYPE`

```
#define star_v1_GetSystemStatusResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 480 of file [telemetry.pb.h](#).

6.39.2.36 star_v1_GetSystemStatusResponse_header_tag

```
#define star_v1_GetSystemStatusResponse_header_tag 1
```

Definition at line 443 of file [telemetry.pb.h](#).

6.39.2.37 star_v1_GetSystemStatusResponse_init_default

```
#define star_v1_GetSystemStatusResponse_init_default {false, star_v1_ResponseHeader_init_default, false, star_v1_SystemStatus_init_default}
```

Definition at line 360 of file [telemetry.pb.h](#).

6.39.2.38 star_v1_GetSystemStatusResponse_init_zero

```
#define star_v1_GetSystemStatusResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false, star_v1_SystemStatus_init_zero}
```

Definition at line 371 of file [telemetry.pb.h](#).

6.39.2.39 star_v1_GetSystemStatusResponse_size

```
#define star_v1_GetSystemStatusResponse_size 425
```

Definition at line 601 of file [telemetry.pb.h](#).

6.39.2.40 star_v1_GetSystemStatusResponse_status_MSGTYPE

```
#define star_v1_GetSystemStatusResponse_status_MSGTYPE star_v1_SystemStatus
```

Definition at line 481 of file [telemetry.pb.h](#).

6.39.2.41 star_v1_GetSystemStatusResponse_status_tag

```
#define star_v1_GetSystemStatusResponse_status_tag 2
```

Definition at line 444 of file [telemetry.pb.h](#).

6.39.2.42 star_v1_GetTelemetryRequest_CALLBACK

```
#define star_v1_GetTelemetryRequest_CALLBACK NULL
```

Definition at line 449 of file [telemetry.pb.h](#).

6.39.2.43 star_v1_GetTelemetryRequest_DEFAULT

```
#define star_v1_GetTelemetryRequest_DEFAULT NULL
```

Definition at line 450 of file [telemetry.pb.h](#).

6.39.2.44 star_v1_GetTelemetryRequest_FIELDLIST

```
#define star_v1_GetTelemetryRequest_FIELDLIST(
    X,
    a)
```

Value:

X(a, STATIC, OPTIONAL, MESSAGE, header, 1)

Definition at line 447 of file [telemetry.pb.h](#).

```
00447 #define star_v1_GetTelemetryRequest_FIELDLIST(X, a) \
00448 X(a, STATIC, OPTIONAL, MESSAGE, header, 1)
```

6.39.2.45 star_v1_GetTelemetryRequest_fields

```
#define star_v1_GetTelemetryRequest_fields &star_v1_GetTelemetryRequest_msg
```

Definition at line 585 of file [telemetry.pb.h](#).

6.39.2.46 star_v1_GetTelemetryRequest_header_MSGTYPE

```
#define star_v1_GetTelemetryRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 451 of file [telemetry.pb.h](#).

6.39.2.47 star_v1_GetTelemetryRequest_header_tag

```
#define star_v1_GetTelemetryRequest_header_tag 1
```

Definition at line 380 of file [telemetry.pb.h](#).

6.39.2.48 star_v1_GetTelemetryRequest_init_default

```
#define star_v1_GetTelemetryRequest_init_default {false, star_v1_RequestHeader_init_default}
```

Definition at line 356 of file [telemetry.pb.h](#).

6.39.2.49 star_v1_GetTelemetryRequest_init_zero

```
#define star_v1_GetTelemetryRequest_init_zero {false, star_v1_RequestHeader_init_zero}
```

Definition at line 367 of file [telemetry.pb.h](#).

6.39.2.50 star_v1_GetTelemetryRequest_size

```
#define star_v1_GetTelemetryRequest_size 149
```

Definition at line 602 of file [telemetry.pb.h](#).

6.39.2.51 `star_v1_GetTelemetryResponse_CALLBACK`

```
#define star_v1_GetTelemetryResponse_CALLBACK NULL
```

Definition at line 456 of file [telemetry.pb.h](#).

6.39.2.52 `star_v1_GetTelemetryResponse_DEFAULT`

```
#define star_v1_GetTelemetryResponse_DEFAULT NULL
```

Definition at line 457 of file [telemetry.pb.h](#).

6.39.2.53 `star_v1_GetTelemetryResponse_FIELDLIST`

```
#define star_v1_GetTelemetryResponse_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
X(a, STATIC, OPTIONAL, MESSAGE, telemetry, 2)
```

Definition at line 453 of file [telemetry.pb.h](#).

```
00453 #define star_v1_GetTelemetryResponse_FIELDLIST(X, a) \  
00454 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \  
00455 X(a, STATIC, OPTIONAL, MESSAGE, telemetry, 2)
```

6.39.2.54 `star_v1_GetTelemetryResponse_fields`

```
#define star_v1_GetTelemetryResponse_fields &star_v1_GetTelemetryResponse_msg
```

Definition at line 586 of file [telemetry.pb.h](#).

6.39.2.55 `star_v1_GetTelemetryResponse_header_MSGTYPE`

```
#define star_v1_GetTelemetryResponse_header_MSGTYPE star_v1_ResponseHeader
```

Definition at line 458 of file [telemetry.pb.h](#).

6.39.2.56 `star_v1_GetTelemetryResponse_header_tag`

```
#define star_v1_GetTelemetryResponse_header_tag 1
```

Definition at line 433 of file [telemetry.pb.h](#).

6.39.2.57 `star_v1_GetTelemetryResponse_init_default`

```
#define star_v1_GetTelemetryResponse_init_default {false, star_v1_ResponseHeader_init_default, false, star_v1_TelemetryData_init_default}
```

Definition at line 357 of file [telemetry.pb.h](#).

6.39.2.58 star_v1_GetTelemetryResponse__init__zero

```
#define star_v1_GetTelemetryResponse__init__zero {false, star_v1_ResponseHeader__init__zero, false, star_v1_TelemetryData__init__zero}
```

Definition at line 368 of file [telemetry.pb.h](#).

6.39.2.59 star_v1_GetTelemetryResponse__size

```
#define star_v1_GetTelemetryResponse__size 881
```

Definition at line 603 of file [telemetry.pb.h](#).

6.39.2.60 star_v1_GetTelemetryResponse__telemetry__MSGTYPE

```
#define star_v1_GetTelemetryResponse__telemetry__MSGTYPE star_v1_TelemetryData
```

Definition at line 459 of file [telemetry.pb.h](#).

6.39.2.61 star_v1_GetTelemetryResponse__telemetry__tag

```
#define star_v1_GetTelemetryResponse__telemetry__tag 2
```

Definition at line 434 of file [telemetry.pb.h](#).

6.39.2.62 star_v1_GpsData__accuracy__m__tag

```
#define star_v1_GpsData__accuracy__m__tag 4
```

Definition at line 412 of file [telemetry.pb.h](#).

6.39.2.63 star_v1_GpsData__altitude__m__tag

```
#define star_v1_GpsData__altitude__m__tag 3
```

Definition at line 411 of file [telemetry.pb.h](#).

6.39.2.64 star_v1_GpsData__CALLBACK

```
#define star_v1_GpsData__CALLBACK NULL
```

Definition at line 557 of file [telemetry.pb.h](#).

6.39.2.65 star_v1_GpsData__DEFAULT

```
#define star_v1_GpsData__DEFAULT NULL
```

Definition at line 558 of file [telemetry.pb.h](#).

6.39.2.66 star_v1_GpsData_FIELDLIST

```
#define star_v1_GpsData_FIELDLIST(
    X,
    a)
```

Value:

```
X(a, STATIC, SINGULAR, DOUBLE, latitude_deg, 1) \
X(a, STATIC, SINGULAR, DOUBLE, longitude_deg, 2) \
X(a, STATIC, SINGULAR, DOUBLE, altitude_m, 3) \
X(a, STATIC, SINGULAR, DOUBLE, accuracy_m, 4) \
X(a, STATIC, SINGULAR, INT32, satellites, 5) \
X(a, STATIC, SINGULAR, UENUM, fix_type, 6)
```

Definition at line 550 of file [telemetry.pb.h](#).

```
00550 #define star_v1_GpsData_FIELDLIST(X, a) \
00551 X(a, STATIC, SINGULAR, DOUBLE, latitude_deg, 1) \
00552 X(a, STATIC, SINGULAR, DOUBLE, longitude_deg, 2) \
00553 X(a, STATIC, SINGULAR, DOUBLE, altitude_m, 3) \
00554 X(a, STATIC, SINGULAR, DOUBLE, accuracy_m, 4) \
00555 X(a, STATIC, SINGULAR, INT32, satellites, 5) \
00556 X(a, STATIC, SINGULAR, UENUM, fix_type, 6)
```

6.39.2.67 star_v1_GpsData_fields

```
#define star_v1_GpsData_fields &star_v1_GpsData_msg
```

Definition at line 594 of file [telemetry.pb.h](#).

6.39.2.68 star_v1_GpsData_fix_type_ENUMTYPE

```
#define star_v1_GpsData_fix_type_ENUMTYPE star_v1_GpsFix
```

Definition at line 349 of file [telemetry.pb.h](#).

6.39.2.69 star_v1_GpsData_fix_type_tag

```
#define star_v1_GpsData_fix_type_tag 6
```

Definition at line 414 of file [telemetry.pb.h](#).

6.39.2.70 star_v1_GpsData_init_default

```
#define star_v1_GpsData_init_default {0, 0, 0, 0, 0, __star_v1_GpsFix_MIN}
```

Definition at line 365 of file [telemetry.pb.h](#).

6.39.2.71 star_v1_GpsData_init_zero

```
#define star_v1_GpsData_init_zero {0, 0, 0, 0, 0, __star_v1_GpsFix_MIN}
```

Definition at line 376 of file [telemetry.pb.h](#).

6.39.2.72 star_v1_GpsData_latitude_deg_tag

```
#define star_v1_GpsData_latitude_deg_tag 1
```

Definition at line 409 of file [telemetry.pb.h](#).

6.39.2.73 star_v1_GpsData_longitude_deg_tag

```
#define star_v1_GpsData_longitude_deg_tag 2
```

Definition at line 410 of file [telemetry.pb.h](#).

6.39.2.74 star_v1_GpsData_satellites_tag

```
#define star_v1_GpsData_satellites_tag 5
```

Definition at line 413 of file [telemetry.pb.h](#).

6.39.2.75 star_v1_GpsData_size

```
#define star_v1_GpsData_size 49
```

Definition at line 604 of file [telemetry.pb.h](#).

6.39.2.76 star_v1_ImuData_accel_x_mps2_tag

```
#define star_v1_ImuData_accel_x_mps2_tag 4
```

Definition at line 388 of file [telemetry.pb.h](#).

6.39.2.77 star_v1_ImuData_accel_y_mps2_tag

```
#define star_v1_ImuData_accel_y_mps2_tag 5
```

Definition at line 389 of file [telemetry.pb.h](#).

6.39.2.78 star_v1_ImuData_accel_z_mps2_tag

```
#define star_v1_ImuData_accel_z_mps2_tag 6
```

Definition at line 390 of file [telemetry.pb.h](#).

6.39.2.79 star_v1_ImuData_calibration_status_tag

```
#define star_v1_ImuData_calibration_status_tag 15
```

Definition at line 399 of file [telemetry.pb.h](#).

6.39.2.80 star_v1_ImuData_CALLBACK

```
#define star_v1_ImuData_CALLBACK NULL
```

Definition at line 531 of file [telemetry.pb.h](#).

6.39.2.81 star_v1_ImuData_DEFAULT

```
#define star_v1_ImuData_DEFAULT NULL
```

Definition at line 532 of file [telemetry.pb.h](#).

6.39.2.82 star_v1_ImuData_FIELDLIST

```
#define star_v1_ImuData_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, DOUBLE, pitch_rad, 1) \  
X(a, STATIC, SINGULAR, DOUBLE, roll_rad, 2) \  
X(a, STATIC, SINGULAR, DOUBLE, yaw_rad, 3) \  
X(a, STATIC, SINGULAR, DOUBLE, accel_x_mps2, 4) \  
X(a, STATIC, SINGULAR, DOUBLE, accel_y_mps2, 5) \  
X(a, STATIC, SINGULAR, DOUBLE, accel_z_mps2, 6) \  
X(a, STATIC, SINGULAR, DOUBLE, gyro_x_rad_per_s, 7) \  
X(a, STATIC, SINGULAR, DOUBLE, gyro_y_rad_per_s, 8) \  
X(a, STATIC, SINGULAR, DOUBLE, gyro_z_rad_per_s, 9) \  
X(a, STATIC, SINGULAR, DOUBLE, heading_rad, 10) \  
X(a, STATIC, SINGULAR, DOUBLE, quat_w, 11) \  
X(a, STATIC, SINGULAR, DOUBLE, quat_x, 12) \  
X(a, STATIC, SINGULAR, DOUBLE, quat_y, 13) \  
X(a, STATIC, SINGULAR, DOUBLE, quat_z, 14) \  
X(a, STATIC, SINGULAR, UINT32, calibration_status, 15) \  
X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 16)
```

Definition at line 514 of file [telemetry.pb.h](#).

```
00514 #define star_v1_ImuData_FIELDLIST(X, a) \  
00515 X(a, STATIC, SINGULAR, DOUBLE, pitch_rad, 1) \  
00516 X(a, STATIC, SINGULAR, DOUBLE, roll_rad, 2) \  
00517 X(a, STATIC, SINGULAR, DOUBLE, yaw_rad, 3) \  
00518 X(a, STATIC, SINGULAR, DOUBLE, accel_x_mps2, 4) \  
00519 X(a, STATIC, SINGULAR, DOUBLE, accel_y_mps2, 5) \  
00520 X(a, STATIC, SINGULAR, DOUBLE, accel_z_mps2, 6) \  
00521 X(a, STATIC, SINGULAR, DOUBLE, gyro_x_rad_per_s, 7) \  
00522 X(a, STATIC, SINGULAR, DOUBLE, gyro_y_rad_per_s, 8) \  
00523 X(a, STATIC, SINGULAR, DOUBLE, gyro_z_rad_per_s, 9) \  
00524 X(a, STATIC, SINGULAR, DOUBLE, heading_rad, 10) \  
00525 X(a, STATIC, SINGULAR, DOUBLE, quat_w, 11) \  
00526 X(a, STATIC, SINGULAR, DOUBLE, quat_x, 12) \  
00527 X(a, STATIC, SINGULAR, DOUBLE, quat_y, 13) \  
00528 X(a, STATIC, SINGULAR, DOUBLE, quat_z, 14) \  
00529 X(a, STATIC, SINGULAR, UINT32, calibration_status, 15) \  
00530 X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 16)
```

6.39.2.83 star_v1_ImuData_fields

```
#define star_v1_ImuData_fields &star_v1_ImuData_msg
```

Definition at line 591 of file [telemetry.pb.h](#).

6.39.2.84 `star_v1_ImuData_gyro_x_rad_per_s_tag`

```
#define star_v1_ImuData_gyro_x_rad_per_s_tag 7
```

Definition at line 391 of file [telemetry.pb.h](#).

6.39.2.85 `star_v1_ImuData_gyro_y_rad_per_s_tag`

```
#define star_v1_ImuData_gyro_y_rad_per_s_tag 8
```

Definition at line 392 of file [telemetry.pb.h](#).

6.39.2.86 `star_v1_ImuData_gyro_z_rad_per_s_tag`

```
#define star_v1_ImuData_gyro_z_rad_per_s_tag 9
```

Definition at line 393 of file [telemetry.pb.h](#).

6.39.2.87 `star_v1_ImuData_heading_rad_tag`

```
#define star_v1_ImuData_heading_rad_tag 10
```

Definition at line 394 of file [telemetry.pb.h](#).

6.39.2.88 `star_v1_ImuData_init_default`

```
#define star_v1_ImuData_init_default {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
```

Definition at line 362 of file [telemetry.pb.h](#).

6.39.2.89 `star_v1_ImuData_init_zero`

```
#define star_v1_ImuData_init_zero {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
```

Definition at line 373 of file [telemetry.pb.h](#).

6.39.2.90 `star_v1_ImuData_pitch_rad_tag`

```
#define star_v1_ImuData_pitch_rad_tag 1
```

Definition at line 385 of file [telemetry.pb.h](#).

6.39.2.91 `star_v1_ImuData_quat_w_tag`

```
#define star_v1_ImuData_quat_w_tag 11
```

Definition at line 395 of file [telemetry.pb.h](#).

6.39.2.92 star_v1_ImuData_quat_x_tag

```
#define star_v1_ImuData_quat_x_tag 12
```

Definition at line 396 of file [telemetry.pb.h](#).

6.39.2.93 star_v1_ImuData_quat_y_tag

```
#define star_v1_ImuData_quat_y_tag 13
```

Definition at line 397 of file [telemetry.pb.h](#).

6.39.2.94 star_v1_ImuData_quat_z_tag

```
#define star_v1_ImuData_quat_z_tag 14
```

Definition at line 398 of file [telemetry.pb.h](#).

6.39.2.95 star_v1_ImuData_roll_rad_tag

```
#define star_v1_ImuData_roll_rad_tag 2
```

Definition at line 386 of file [telemetry.pb.h](#).

6.39.2.96 star_v1_ImuData_size

```
#define star_v1_ImuData_size 142
```

Definition at line 605 of file [telemetry.pb.h](#).

6.39.2.97 star_v1_ImuData_temperature_celsius_tag

```
#define star_v1_ImuData_temperature_celsius_tag 16
```

Definition at line 400 of file [telemetry.pb.h](#).

6.39.2.98 star_v1_ImuData_yaw_rad_tag

```
#define star_v1_ImuData_yaw_rad_tag 3
```

Definition at line 387 of file [telemetry.pb.h](#).

6.39.2.99 star_v1_ObstacleData_any_obstacle_tag

```
#define star_v1_ObstacleData_any_obstacle_tag 5
```

Definition at line 407 of file [telemetry.pb.h](#).

6.39.2.100 `star_v1_ObstacleData_CALLBACK`

```
#define star_v1_ObstacleData_CALLBACK NULL
```

Definition at line 547 of file [telemetry.pb.h](#).

6.39.2.101 `star_v1_ObstacleData_DEFAULT`

```
#define star_v1_ObstacleData_DEFAULT NULL
```

Definition at line 548 of file [telemetry.pb.h](#).

6.39.2.102 `star_v1_ObstacleData_detected_mask_tag`

```
#define star_v1_ObstacleData_detected_mask_tag 6
```

Definition at line 408 of file [telemetry.pb.h](#).

6.39.2.103 `star_v1_ObstacleData_distance_back_left_m_tag`

```
#define star_v1_ObstacleData_distance_back_left_m_tag 3
```

Definition at line 405 of file [telemetry.pb.h](#).

6.39.2.104 `star_v1_ObstacleData_distance_back_right_m_tag`

```
#define star_v1_ObstacleData_distance_back_right_m_tag 4
```

Definition at line 406 of file [telemetry.pb.h](#).

6.39.2.105 `star_v1_ObstacleData_distance_front_left_m_tag`

```
#define star_v1_ObstacleData_distance_front_left_m_tag 1
```

Definition at line 403 of file [telemetry.pb.h](#).

6.39.2.106 `star_v1_ObstacleData_distance_front_right_m_tag`

```
#define star_v1_ObstacleData_distance_front_right_m_tag 2
```

Definition at line 404 of file [telemetry.pb.h](#).

6.39.2.107 star_v1_ObstacleData_FIELDLIST

```
#define star_v1_ObstacleData_FIELDLIST(
    X,
    a)
```

Value:

```
X(a, STATIC, SINGULAR, FLOAT, distance_front_left_m, 1) \
X(a, STATIC, SINGULAR, FLOAT, distance_front_right_m, 2) \
X(a, STATIC, SINGULAR, FLOAT, distance_back_left_m, 3) \
X(a, STATIC, SINGULAR, FLOAT, distance_back_right_m, 4) \
X(a, STATIC, SINGULAR, BOOL, any_obstacle, 5) \
X(a, STATIC, SINGULAR, UINT32, detected_mask, 6)
```

Definition at line 540 of file [telemetry.pb.h](#).

```
00540 #define star_v1_ObstacleData_FIELDLIST(X, a) \
00541 X(a, STATIC, SINGULAR, FLOAT, distance_front_left_m, 1) \
00542 X(a, STATIC, SINGULAR, FLOAT, distance_front_right_m, 2) \
00543 X(a, STATIC, SINGULAR, FLOAT, distance_back_left_m, 3) \
00544 X(a, STATIC, SINGULAR, FLOAT, distance_back_right_m, 4) \
00545 X(a, STATIC, SINGULAR, BOOL, any_obstacle, 5) \
00546 X(a, STATIC, SINGULAR, UINT32, detected_mask, 6)
```

6.39.2.108 star_v1_ObstacleData_fields

```
#define star_v1_ObstacleData_fields &star_v1_ObstacleData_msg
```

Definition at line 593 of file [telemetry.pb.h](#).

6.39.2.109 star_v1_ObstacleData_init_default

```
#define star_v1_ObstacleData_init_default {0, 0, 0, 0, 0, 0}
```

Definition at line 364 of file [telemetry.pb.h](#).

6.39.2.110 star_v1_ObstacleData_init_zero

```
#define star_v1_ObstacleData_init_zero {0, 0, 0, 0, 0, 0}
```

Definition at line 375 of file [telemetry.pb.h](#).

6.39.2.111 star_v1_ObstacleData_size

```
#define star_v1_ObstacleData_size 28
```

Definition at line 606 of file [telemetry.pb.h](#).

6.39.2.112 STAR_V1_STAR_V1_TELEMETRY_PB_H_MAX_SIZE

```
#define STAR_V1_STAR_V1_TELEMETRY_PB_H_MAX_SIZE star_v1_GetTelemetryResponse_size
```

Definition at line 598 of file [telemetry.pb.h](#).

6.39.2.113 `star_v1_StreamTelemetryRequest_CALLBACK`

```
#define star_v1_StreamTelemetryRequest_CALLBACK NULL
```

Definition at line 465 of file [telemetry.pb.h](#).

6.39.2.114 `star_v1_StreamTelemetryRequest_DEFAULT`

```
#define star_v1_StreamTelemetryRequest_DEFAULT NULL
```

Definition at line 466 of file [telemetry.pb.h](#).

6.39.2.115 `star_v1_StreamTelemetryRequest_FIELDLIST`

```
#define star_v1_StreamTelemetryRequest_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, header,      1) \  
X(a, STATIC, SINGULAR, INT32, rate_hz,      2) \  
X(a, STATIC, REPEATED, STRING, fields,      3)
```

Definition at line 461 of file [telemetry.pb.h](#).

```
00461 #define star_v1_StreamTelemetryRequest_FIELDLIST(X, a) \  
00462 X(a, STATIC, OPTIONAL, MESSAGE, header,      1) \  
00463 X(a, STATIC, SINGULAR, INT32, rate_hz,      2) \  
00464 X(a, STATIC, REPEATED, STRING, fields,      3)
```

6.39.2.116 `star_v1_StreamTelemetryRequest_fields`

```
#define star_v1_StreamTelemetryRequest_fields &star_v1_StreamTelemetryRequest_msg
```

Definition at line 587 of file [telemetry.pb.h](#).

6.39.2.117 `star_v1_StreamTelemetryRequest_fields_tag`

```
#define star_v1_StreamTelemetryRequest_fields_tag 3
```

Definition at line 383 of file [telemetry.pb.h](#).

6.39.2.118 `star_v1_StreamTelemetryRequest_header_MSGTYPE`

```
#define star_v1_StreamTelemetryRequest_header_MSGTYPE star_v1_RequestHeader
```

Definition at line 467 of file [telemetry.pb.h](#).

6.39.2.119 `star_v1_StreamTelemetryRequest_header_tag`

```
#define star_v1_StreamTelemetryRequest_header_tag 1
```

Definition at line 381 of file [telemetry.pb.h](#).

6.39.2.120 star_v1_StreamTelemetryRequest_init_default

```
#define star_v1_StreamTelemetryRequest_init_default {false, star_v1_RequestHeader_init_default, 0, 0, {"", "", "", "", "", "", "", "", "", "", ""}}
```

Definition at line 358 of file [telemetry.pb.h](#).

6.39.2.121 star_v1_StreamTelemetryRequest_init_zero

```
#define star_v1_StreamTelemetryRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0, 0, {"", "", "", "", "", "", "", "", "", "", ""}}
```

Definition at line 369 of file [telemetry.pb.h](#).

6.39.2.122 star_v1_StreamTelemetryRequest_rate_hz_tag

```
#define star_v1_StreamTelemetryRequest_rate_hz_tag 2
```

Definition at line 382 of file [telemetry.pb.h](#).

6.39.2.123 star_v1_StreamTelemetryRequest_size

```
#define star_v1_StreamTelemetryRequest_size 688
```

Definition at line 607 of file [telemetry.pb.h](#).

6.39.2.124 star_v1_SystemStatus_CALLBACK

```
#define star_v1_SystemStatus_CALLBACK NULL
```

Definition at line 569 of file [telemetry.pb.h](#).

6.39.2.125 star_v1_SystemStatus_connection_status_ENUMTYPE

```
#define star_v1_SystemStatus_connection_status_ENUMTYPE star_v1_ConnectionStatus
```

Definition at line 351 of file [telemetry.pb.h](#).

6.39.2.126 star_v1_SystemStatus_connection_status_tag

```
#define star_v1_SystemStatus_connection_status_tag 1
```

Definition at line 435 of file [telemetry.pb.h](#).

6.39.2.127 star_v1_SystemStatus_DEFAULT

```
#define star_v1_SystemStatus_DEFAULT NULL
```

Definition at line 570 of file [telemetry.pb.h](#).

6.39.2.128 star_v1_SystemStatus_FIELDLIST

```
#define star_v1_SystemStatus_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, UENUM, connection_status, 1) \  
X(a, STATIC, SINGULAR, UENUM, mode, 2) \  
X(a, STATIC, SINGULAR, BOOL, rx72n_connected, 3) \  
X(a, STATIC, SINGULAR, BOOL, lidar_connected, 4) \  
X(a, STATIC, SINGULAR, BOOL, ros_connected, 5) \  
X(a, STATIC, SINGULAR, STRING, firmware_version, 6) \  
X(a, STATIC, SINGULAR, UINT64, uptime_s, 7) \  
X(a, STATIC, SINGULAR, UINT32, free_heap_bytes, 8)
```

Definition at line 560 of file [telemetry.pb.h](#).

```
00560 #define star_v1_SystemStatus_FIELDLIST(X, a) \  
00561 X(a, STATIC, SINGULAR, UENUM, connection_status, 1) \  
00562 X(a, STATIC, SINGULAR, UENUM, mode, 2) \  
00563 X(a, STATIC, SINGULAR, BOOL, rx72n_connected, 3) \  
00564 X(a, STATIC, SINGULAR, BOOL, lidar_connected, 4) \  
00565 X(a, STATIC, SINGULAR, BOOL, ros_connected, 5) \  
00566 X(a, STATIC, SINGULAR, STRING, firmware_version, 6) \  
00567 X(a, STATIC, SINGULAR, UINT64, uptime_s, 7) \  
00568 X(a, STATIC, SINGULAR, UINT32, free_heap_bytes, 8)
```

6.39.2.129 star_v1_SystemStatus_fields

```
#define star_v1_SystemStatus_fields &star_v1_SystemStatus_msg
```

Definition at line 595 of file [telemetry.pb.h](#).

6.39.2.130 star_v1_SystemStatus_firmware_version_tag

```
#define star_v1_SystemStatus_firmware_version_tag 6
```

Definition at line 440 of file [telemetry.pb.h](#).

6.39.2.131 star_v1_SystemStatus_free_heap_bytes_tag

```
#define star_v1_SystemStatus_free_heap_bytes_tag 8
```

Definition at line 442 of file [telemetry.pb.h](#).

6.39.2.132 star_v1_SystemStatus_init_default

```
#define star_v1_SystemStatus_init_default {_star_v1_ConnectionStatus_MIN, _star_v1_RobotMode_MIN, 0, 0, 0,  
    "", 0, 0}
```

Definition at line 366 of file [telemetry.pb.h](#).

6.39.2.133 star_v1_SystemStatus_init_zero

```
#define star_v1_SystemStatus_init_zero {__star_v1_ConnectionStatus_MIN, __star_v1_RobotMode_MIN, 0, 0, 0, "", 0, 0}
```

Definition at line 377 of file [telemetry.pb.h](#).

6.39.2.134 star_v1_SystemStatus_lidar_connected_tag

```
#define star_v1_SystemStatus_lidar_connected_tag 4
```

Definition at line 438 of file [telemetry.pb.h](#).

6.39.2.135 star_v1_SystemStatus_mode_ENUMTYPE

```
#define star_v1_SystemStatus_mode_ENUMTYPE star_v1_RobotMode
```

Definition at line 352 of file [telemetry.pb.h](#).

6.39.2.136 star_v1_SystemStatus_mode_tag

```
#define star_v1_SystemStatus_mode_tag 2
```

Definition at line 436 of file [telemetry.pb.h](#).

6.39.2.137 star_v1_SystemStatus_ros_connected_tag

```
#define star_v1_SystemStatus_ros_connected_tag 5
```

Definition at line 439 of file [telemetry.pb.h](#).

6.39.2.138 star_v1_SystemStatus_rx72n_connected_tag

```
#define star_v1_SystemStatus_rx72n_connected_tag 3
```

Definition at line 437 of file [telemetry.pb.h](#).

6.39.2.139 star_v1_SystemStatus_size

```
#define star_v1_SystemStatus_size 60
```

Definition at line 608 of file [telemetry.pb.h](#).

6.39.2.140 star_v1_SystemStatus_uptime_s_tag

```
#define star_v1_SystemStatus_uptime_s_tag 7
```

Definition at line 441 of file [telemetry.pb.h](#).

6.39.2.141 `star_v1_TelemetryData_baro_MSGTYPE`

```
#define star_v1_TelemetryData_baro_MSGTYPE star_v1_BaroData
```

Definition at line 508 of file [telemetry.pb.h](#).

6.39.2.142 `star_v1_TelemetryData_baro_tag`

```
#define star_v1_TelemetryData_baro_tag 12
```

Definition at line 426 of file [telemetry.pb.h](#).

6.39.2.143 `star_v1_TelemetryData_CALLBACK`

```
#define star_v1_TelemetryData_CALLBACK NULL
```

Definition at line 502 of file [telemetry.pb.h](#).

6.39.2.144 `star_v1_TelemetryData_cpu_usage_percent_tag`

```
#define star_v1_TelemetryData_cpu_usage_percent_tag 5
```

Definition at line 419 of file [telemetry.pb.h](#).

6.39.2.145 `star_v1_TelemetryData_DEFAULT`

```
#define star_v1_TelemetryData_DEFAULT NULL
```

Definition at line 503 of file [telemetry.pb.h](#).

6.39.2.146 `star_v1_TelemetryData_emergency_stop_tag`

```
#define star_v1_TelemetryData_emergency_stop_tag 10
```

Definition at line 424 of file [telemetry.pb.h](#).

6.39.2.147 `star_v1_TelemetryData_encoder_back_left_MSGTYPE`

```
#define star_v1_TelemetryData_encoder_back_left_MSGTYPE star_v1_EncoderData
```

Definition at line 511 of file [telemetry.pb.h](#).

6.39.2.148 `star_v1_TelemetryData_encoder_back_left_tag`

```
#define star_v1_TelemetryData_encoder_back_left_tag 17
```

Definition at line 430 of file [telemetry.pb.h](#).

6.39.2.149 star_v1_TelemetryData_encoder_back_right_MSGTYPE

```
#define star_v1_TelemetryData_encoder_back_right_MSGTYPE star_v1_EncoderData
```

Definition at line 512 of file [telemetry.pb.h](#).

6.39.2.150 star_v1_TelemetryData_encoder_back_right_tag

```
#define star_v1_TelemetryData_encoder_back_right_tag 18
```

Definition at line 431 of file [telemetry.pb.h](#).

6.39.2.151 star_v1_TelemetryData_encoder_front_left_MSGTYPE

```
#define star_v1_TelemetryData_encoder_front_left_MSGTYPE star_v1_EncoderData
```

Definition at line 509 of file [telemetry.pb.h](#).

6.39.2.152 star_v1_TelemetryData_encoder_front_left_tag

```
#define star_v1_TelemetryData_encoder_front_left_tag 15
```

Definition at line 428 of file [telemetry.pb.h](#).

6.39.2.153 star_v1_TelemetryData_encoder_front_right_MSGTYPE

```
#define star_v1_TelemetryData_encoder_front_right_MSGTYPE star_v1_EncoderData
```

Definition at line 510 of file [telemetry.pb.h](#).

6.39.2.154 star_v1_TelemetryData_encoder_front_right_tag

```
#define star_v1_TelemetryData_encoder_front_right_tag 16
```

Definition at line 429 of file [telemetry.pb.h](#).

6.39.2.155 star_v1_TelemetryData_estop_reason_ENUMTYPE

```
#define star_v1_TelemetryData_estop_reason_ENUMTYPE star_v1_EstopReason
```

Definition at line 344 of file [telemetry.pb.h](#).

6.39.2.156 star_v1_TelemetryData_estop_reason_tag

```
#define star_v1_TelemetryData_estop_reason_tag 9
```

Definition at line 423 of file [telemetry.pb.h](#).

6.39.2.157 star_v1_TelemetryData_fault_flags_tag

```
#define star_v1_TelemetryData_fault_flags_tag 11
```

Definition at line 425 of file [telemetry.pb.h](#).

6.39.2.158 star_v1_TelemetryData_FIELDLIST

```
#define star_v1_TelemetryData_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, OPTIONAL, MESSAGE, imu, 1) \
X(a, STATIC, OPTIONAL, MESSAGE, gps, 2) \
X(a, STATIC, OPTIONAL, MESSAGE, obstacle, 3) \
X(a, STATIC, SINGULAR, INT32, wifi_signal_dbm, 4) \
X(a, STATIC, SINGULAR, DOUBLE, cpu_usage_percent, 5) \
X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 6) \
X(a, STATIC, SINGULAR, DOUBLE, motor_load_percent, 7) \
X(a, STATIC, OPTIONAL, MESSAGE, timestamp, 8) \
X(a, STATIC, SINGULAR, UENUM, estop_reason, 9) \
X(a, STATIC, SINGULAR, BOOL, emergency_stop, 10) \
X(a, STATIC, SINGULAR, UINT32, fault_flags, 11) \
X(a, STATIC, OPTIONAL, MESSAGE, baro, 12) \
X(a, STATIC, SINGULAR, INT64, timestamp_us, 14) \
X(a, STATIC, OPTIONAL, MESSAGE, encoder_front_left, 15) \
X(a, STATIC, OPTIONAL, MESSAGE, encoder_front_right, 16) \
X(a, STATIC, OPTIONAL, MESSAGE, encoder_back_left, 17) \
X(a, STATIC, OPTIONAL, MESSAGE, encoder_back_right, 18) \
X(a, STATIC, SINGULAR, UINT32, frame_sequence, 19)
```

Definition at line 483 of file [telemetry.pb.h](#).

```
00483 #define star_v1_TelemetryData_FIELDLIST(X, a) \
00484 X(a, STATIC, OPTIONAL, MESSAGE, imu, 1) \
00485 X(a, STATIC, OPTIONAL, MESSAGE, gps, 2) \
00486 X(a, STATIC, OPTIONAL, MESSAGE, obstacle, 3) \
00487 X(a, STATIC, SINGULAR, INT32, wifi_signal_dbm, 4) \
00488 X(a, STATIC, SINGULAR, DOUBLE, cpu_usage_percent, 5) \
00489 X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 6) \
00490 X(a, STATIC, SINGULAR, DOUBLE, motor_load_percent, 7) \
00491 X(a, STATIC, OPTIONAL, MESSAGE, timestamp, 8) \
00492 X(a, STATIC, SINGULAR, UENUM, estop_reason, 9) \
00493 X(a, STATIC, SINGULAR, BOOL, emergency_stop, 10) \
00494 X(a, STATIC, SINGULAR, UINT32, fault_flags, 11) \
00495 X(a, STATIC, OPTIONAL, MESSAGE, baro, 12) \
00496 X(a, STATIC, SINGULAR, INT64, timestamp_us, 14) \
00497 X(a, STATIC, OPTIONAL, MESSAGE, encoder_front_left, 15) \
00498 X(a, STATIC, OPTIONAL, MESSAGE, encoder_front_right, 16) \
00499 X(a, STATIC, OPTIONAL, MESSAGE, encoder_back_left, 17) \
00500 X(a, STATIC, OPTIONAL, MESSAGE, encoder_back_right, 18) \
00501 X(a, STATIC, SINGULAR, UINT32, frame_sequence, 19)
```

6.39.2.159 star_v1_TelemetryData_fields

```
#define star_v1_TelemetryData_fields &star_v1_TelemetryData_msg
```

Definition at line 590 of file [telemetry.pb.h](#).

6.39.2.160 star_v1_TelemetryData_frame_sequence_tag

```
#define star_v1_TelemetryData_frame_sequence_tag 19
```

Definition at line 432 of file [telemetry.pb.h](#).

6.39.2.161 star_v1_TelemetryData_gps_MSGTYPE

```
#define star_v1_TelemetryData_gps_MSGTYPE star_v1_GpsData
```

Definition at line 505 of file [telemetry.pb.h](#).

6.39.2.162 star_v1_TelemetryData_gps_tag

```
#define star_v1_TelemetryData_gps_tag 2
```

Definition at line 416 of file [telemetry.pb.h](#).

6.39.2.163 star_v1_TelemetryData_imu_MSGTYPE

```
#define star_v1_TelemetryData_imu_MSGTYPE star_v1_ImuData
```

Definition at line 504 of file [telemetry.pb.h](#).

6.39.2.164 star_v1_TelemetryData_imu_tag

```
#define star_v1_TelemetryData_imu_tag 1
```

Definition at line 415 of file [telemetry.pb.h](#).

6.39.2.165 star_v1_TelemetryData_init_default

```
#define star_v1_TelemetryData_init_default {false, star_v1_ImuData_init_default, false, star_v1_GpsData_init_default,  
false, star_v1_ObstacleData_init_default, 0, 0, 0, 0, false, google_protobuf_Timestamp_init_default, _star_v1_EstopReason_MIN,  
0, 0, false, star_v1_BaroData_init_default, 0, false, star_v1_EncoderData_init_default, false, star_v1_EncoderData_init_default,  
false, star_v1_EncoderData_init_default, false, star_v1_EncoderData_init_default, 0}
```

Definition at line 361 of file [telemetry.pb.h](#).

6.39.2.166 star_v1_TelemetryData_init_zero

```
#define star_v1_TelemetryData_init_zero {false, star_v1_ImuData_init_zero, false, star_v1_GpsData_init_zero,  
false, star_v1_ObstacleData_init_zero, 0, 0, 0, 0, false, google_protobuf_Timestamp_init_zero, _star_v1_EstopReason_MIN,  
0, 0, false, star_v1_BaroData_init_zero, 0, false, star_v1_EncoderData_init_zero, false, star_v1_EncoderData_init_zero,  
false, star_v1_EncoderData_init_zero, false, star_v1_EncoderData_init_zero, 0}
```

Definition at line 372 of file [telemetry.pb.h](#).

6.39.2.167 star_v1_TelemetryData_motor_load_percent_tag

```
#define star_v1_TelemetryData_motor_load_percent_tag 7
```

Definition at line 421 of file [telemetry.pb.h](#).

6.39.2.168 `star_v1_TelemetryData_obstacle_MSGTYPE`

```
#define star_v1_TelemetryData_obstacle_MSGTYPE star\_v1\_ObstacleData
```

Definition at line [506](#) of file [telemetry.pb.h](#).

6.39.2.169 `star_v1_TelemetryData_obstacle_tag`

```
#define star_v1_TelemetryData_obstacle_tag 3
```

Definition at line [417](#) of file [telemetry.pb.h](#).

6.39.2.170 `star_v1_TelemetryData_size`

```
#define star_v1_TelemetryData_size 515
```

Definition at line [609](#) of file [telemetry.pb.h](#).

6.39.2.171 `star_v1_TelemetryData_temperature_celsius_tag`

```
#define star_v1_TelemetryData_temperature_celsius_tag 6
```

Definition at line [420](#) of file [telemetry.pb.h](#).

6.39.2.172 `star_v1_TelemetryData_timestamp_MSGTYPE`

```
#define star_v1_TelemetryData_timestamp_MSGTYPE google\_protobuf\_Timestamp
```

Definition at line [507](#) of file [telemetry.pb.h](#).

6.39.2.173 `star_v1_TelemetryData_timestamp_tag`

```
#define star_v1_TelemetryData_timestamp_tag 8
```

Definition at line [422](#) of file [telemetry.pb.h](#).

6.39.2.174 `star_v1_TelemetryData_timestamp_us_tag`

```
#define star_v1_TelemetryData_timestamp_us_tag 14
```

Definition at line [427](#) of file [telemetry.pb.h](#).

6.39.2.175 `star_v1_TelemetryData_wifi_signal_dbm_tag`

```
#define star_v1_TelemetryData_wifi_signal_dbm_tag 4
```

Definition at line [418](#) of file [telemetry.pb.h](#).

6.39.3 Enumeration Type Documentation

6.39.3.1 star_v1_ConnectionStatus

enum [star_v1_ConnectionStatus](#)

Enumerator

star_v1_ConnectionStatus_CONNECTION_STATUS_UNKNOWN	
star_v1_ConnectionStatus_CONNECTION_STATUS_CONNECTED	
star_v1_ConnectionStatus_CONNECTION_STATUS_DISCONNECTED	
star_v1_ConnectionStatus_CONNECTION_STATUS_CONNECTING	
star_v1_ConnectionStatus_CONNECTION_STATUS_ERROR	

Definition at line 52 of file [telemetry.pb.h](#).

```

00052     {
00053     /* Unknown connection state. */
00054     star_v1_ConnectionStatus_CONNECTION_STATUS_UNKNOWN = 0,
00055     /* Connected and communicating. */
00056     star_v1_ConnectionStatus_CONNECTION_STATUS_CONNECTED = 1,
00057     /* Not connected. */
00058     star_v1_ConnectionStatus_CONNECTION_STATUS_DISCONNECTED = 2,
00059     /* Connection in progress. */
00060     star_v1_ConnectionStatus_CONNECTION_STATUS_CONNECTING = 3,
00061     /* Connection error occurred. */
00062     star_v1_ConnectionStatus_CONNECTION_STATUS_ERROR = 4
00063 } star_v1_ConnectionStatus;

```

6.39.3.2 star_v1_EstopReason

enum [star_v1_EstopReason](#)

Enumerator

star_v1_EstopReason_ESTOP_REASON_UNKNOWN	
star_v1_EstopReason_ESTOP_REASON_MANUAL	
star_v1_EstopReason_ESTOP_REASON_FAULT	
star_v1_EstopReason_ESTOP_REASON_COMM_TIMEOUT	
star_v1_EstopReason_ESTOP_REASON_OBSTACLE	
star_v1_EstopReason_ESTOP_REASON_OVERCURRENT	

Definition at line 18 of file [telemetry.pb.h](#).

```

00018     {
00019     /* Unknown or no emergency stop active. */
00020     star_v1_EstopReason_ESTOP_REASON_UNKNOWN = 0,
00021     /* Manual emergency stop requested via software command. */
00022     star_v1_EstopReason_ESTOP_REASON_MANUAL = 1,
00023     /* Motor driver hardware fault detected (nFAULT signal asserted). */
00024     star_v1_EstopReason_ESTOP_REASON_FAULT = 2,
00025     /* Communication timeout: no command received within watchdog window. */
00026     star_v1_EstopReason_ESTOP_REASON_COMM_TIMEOUT = 3,
00027     /* Obstacle detected too close by ultrasonic sensors. */
00028     star_v1_EstopReason_ESTOP_REASON_OBSTACLE = 4,
00029     /* Motor overcurrent condition detected. */
00030     star_v1_EstopReason_ESTOP_REASON_OVERCURRENT = 5
00031 } star_v1_EstopReason;

```

6.39.3.3 star_v1_GpsFix

enum [star_v1_GpsFix](#)

Enumerator

star_v1_GpsFix_GPS_FIX_UNKNOWN	
star_v1_GpsFix_GPS_FIX_NONE	
star_v1_GpsFix_GPS_FIX_2D	
star_v1_GpsFix_GPS_FIX_3D	
star_v1_GpsFix_GPS_FIX_DGPS	
star_v1_GpsFix_GPS_FIX_RTK_FLOAT	
star_v1_GpsFix_GPS_FIX_RTK_FIXED	

Definition at line 34 of file [telemetry.pb.h](#).

```

00034     {
00035     /* Unknown fix type. */
00036     star_v1_GpsFix_GPS_FIX_UNKNOWN = 0,
00037     /* No fix available. */
00038     star_v1_GpsFix_GPS_FIX_NONE = 1,
00039     /* 2D fix (latitude/longitude only). */
00040     star_v1_GpsFix_GPS_FIX_2D = 2,
00041     /* 3D fix (latitude/longitude/altitude). */
00042     star_v1_GpsFix_GPS_FIX_3D = 3,
00043     /* Differential GPS fix. */
00044     star_v1_GpsFix_GPS_FIX_DGPS = 4,
00045     /* RTK float solution. */
00046     star_v1_GpsFix_GPS_FIX_RTK_FLOAT = 5,
00047     /* RTK fixed solution. */
00048     star_v1_GpsFix_GPS_FIX_RTK_FIXED = 6
00049 } star_v1_GpsFix;

```

6.39.3.4 star_v1_RobotMode

enum [star_v1_RobotMode](#)

Enumerator

star_v1_RobotMode_ROBOT_MODE_UNKNOWN	
star_v1_RobotMode_ROBOT_MODE_IDLE	
star_v1_RobotMode_ROBOT_MODE_MANUAL	
star_v1_RobotMode_ROBOT_MODE_AUTONOMOUS	
star_v1_RobotMode_ROBOT_MODE_MAPPING	
star_v1_RobotMode_ROBOT_MODE_EMERGENCY_STOP	

Definition at line 66 of file [telemetry.pb.h](#).

```

00066     {
00067     /* Unknown operating mode. */
00068     star_v1_RobotMode_ROBOT_MODE_UNKNOWN = 0,
00069     /* Robot is idle (motors off). */
00070     star_v1_RobotMode_ROBOT_MODE_IDLE = 1,
00071     /* Manual control mode (joystick/UI). */
00072     star_v1_RobotMode_ROBOT_MODE_MANUAL = 2,
00073     /* Autonomous navigation mode (Nav2). */
00074     star_v1_RobotMode_ROBOT_MODE_AUTONOMOUS = 3,
00075     /* SLAM mapping mode. */
00076     star_v1_RobotMode_ROBOT_MODE_MAPPING = 4,
00077     /* Emergency stop active. */
00078     star_v1_RobotMode_ROBOT_MODE_EMERGENCY_STOP = 5
00079 } star_v1_RobotMode;

```

6.39.4 Variable Documentation

6.39.4.1 star_v1_BaroData_msg

const pb_msgdesc_t star_v1_BaroData_msg [extern]

6.39.4.2 star_v1_GetSystemStatusRequest_msg

const pb_msgdesc_t star_v1_GetSystemStatusRequest_msg [extern]

6.39.4.3 star_v1_GetSystemStatusResponse_msg

const pb_msgdesc_t star_v1_GetSystemStatusResponse_msg [extern]

6.39.4.4 star_v1_GetTelemetryRequest_msg

const pb_msgdesc_t star_v1_GetTelemetryRequest_msg [extern]

6.39.4.5 star_v1_GetTelemetryResponse_msg

const pb_msgdesc_t star_v1_GetTelemetryResponse_msg [extern]

6.39.4.6 star_v1_GpsData_msg

const pb_msgdesc_t star_v1_GpsData_msg [extern]

6.39.4.7 star_v1_ImuData_msg

const pb_msgdesc_t star_v1_ImuData_msg [extern]

6.39.4.8 star_v1_ObstacleData_msg

const pb_msgdesc_t star_v1_ObstacleData_msg [extern]

6.39.4.9 star_v1_StreamTelemetryRequest_msg

const pb_msgdesc_t star_v1_StreamTelemetryRequest_msg [extern]

6.39.4.10 star_v1_SystemStatus_msg

const pb_msgdesc_t star_v1_SystemStatus_msg [extern]

6.39.4.11 star_v1_TelemetryData_msg

```
const pb_msgdesc_t star_v1_TelemetryData_msg [extern]
```

6.40 telemetry.pb.h

[Go to the documentation of this file.](#)

```
00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_STAR_V1_STAR_V1_TELEMETRY_PB_H_INCLUDED
00005 #define PB_STAR_V1_STAR_V1_TELEMETRY_PB_H_INCLUDED
00006 #include <pb.h>
00007 #include "google/protobuf/timestamp.pb.h"
00008 #include "star/v1/common.pb.h"
00009 #include "star/v1/motor_control.pb.h"
00010
00011 #if PB_PROTO_HEADER_VERSION != 40
00012 #error Regenerate this file with the current version of nanopb generator.
00013 #endif
00014
00015 /* Enum definitions */
00016 /* Emergency stop reason enumeration.
00017 Identifies the cause of an active emergency stop condition on the RX72N. */
00018 typedef enum _star_v1_EstopReason {
00019     /* Unknown or no emergency stop active. */
00020     star_v1_EstopReason_ESTOP_REASON_UNKNOWN = 0,
00021     /* Manual emergency stop requested via software command. */
00022     star_v1_EstopReason_ESTOP_REASON_MANUAL = 1,
00023     /* Motor driver hardware fault detected (nFAULT signal asserted). */
00024     star_v1_EstopReason_ESTOP_REASON_FAULT = 2,
00025     /* Communication timeout: no command received within watchdog window. */
00026     star_v1_EstopReason_ESTOP_REASON_COMM_TIMEOUT = 3,
00027     /* Obstacle detected too close by ultrasonic sensors. */
00028     star_v1_EstopReason_ESTOP_REASON_OBSTACLE = 4,
00029     /* Motor overcurrent condition detected. */
00030     star_v1_EstopReason_ESTOP_REASON_OVERCURRENT = 5
00031 } star_v1_EstopReason;
00032
00033 /* GPS fix type. */
00034 typedef enum _star_v1_GpsFix {
00035     /* Unknown fix type. */
00036     star_v1_GpsFix_GPS_FIX_UNKNOWN = 0,
00037     /* No fix available. */
00038     star_v1_GpsFix_GPS_FIX_NONE = 1,
00039     /* 2D fix (latitude/longitude only). */
00040     star_v1_GpsFix_GPS_FIX_2D = 2,
00041     /* 3D fix (latitude/longitude/altitude). */
00042     star_v1_GpsFix_GPS_FIX_3D = 3,
00043     /* Differential GPS fix. */
00044     star_v1_GpsFix_GPS_FIX_DGPS = 4,
00045     /* RTK float solution. */
00046     star_v1_GpsFix_GPS_FIX_RTK_FLOAT = 5,
00047     /* RTK fixed solution. */
00048     star_v1_GpsFix_GPS_FIX_RTK_FIXED = 6
00049 } star_v1_GpsFix;
00050
00051 /* Connection state enumeration. */
00052 typedef enum _star_v1_ConnectionStatus {
00053     /* Unknown connection state. */
00054     star_v1_ConnectionStatus_CONNECTION_STATUS_UNKNOWN = 0,
00055     /* Connected and communicating. */
00056     star_v1_ConnectionStatus_CONNECTION_STATUS_CONNECTED = 1,
00057     /* Not connected. */
00058     star_v1_ConnectionStatus_CONNECTION_STATUS_DISCONNECTED = 2,
00059     /* Connection in progress. */
00060     star_v1_ConnectionStatus_CONNECTION_STATUS_CONNECTING = 3,
00061     /* Connection error occurred. */
00062     star_v1_ConnectionStatus_CONNECTION_STATUS_ERROR = 4
00063 } star_v1_ConnectionStatus;
00064
00065 /* Robot operating mode enumeration. */
00066 typedef enum _star_v1_RobotMode {
00067     /* Unknown operating mode. */
00068     star_v1_RobotMode_ROBOT_MODE_UNKNOWN = 0,
00069     /* Robot is idle (motors off). */
00070     star_v1_RobotMode_ROBOT_MODE_IDLE = 1,
00071     /* Manual control mode (joystick/UI). */
00072     star_v1_RobotMode_ROBOT_MODE_MANUAL = 2,
```

```

00073  /* Autonomous navigation mode (Nav2). */
00074  star_v1_RobotMode_ROBOT_MODE_AUTONOMOUS = 3,
00075  /* SLAM mapping mode. */
00076  star_v1_RobotMode_ROBOT_MODE_MAPPING = 4,
00077  /* Emergency stop active. */
00078  star_v1_RobotMode_ROBOT_MODE_EMERGENCY_STOP = 5
00079 } star_v1_RobotMode;
00080
00081 /* Struct definitions */
00082 /* Request for telemetry snapshot. */
00083 typedef struct _star_v1_GetTelemetryRequest {
00084     /* Standard request header. */
00085     bool has_header;
00086     star_v1_RequestHeader header;
00087 } star_v1_GetTelemetryRequest;
00088
00089 /* Request to stream telemetry. */
00090 typedef struct _star_v1_StreamTelemetryRequest {
00091     /* Standard request header. */
00092     bool has_header;
00093     star_v1_RequestHeader header;
00094     /* Desired update rate in Hz.
00095     Valid range: 1 to 100 Hz. */
00096     int32_t rate_hz;
00097     /* Fields to include (empty = all fields). */
00098     pb_size_t fields_count;
00099     char fields[16][32];
00100 } star_v1_StreamTelemetryRequest;
00101
00102 /* Request for system status. */
00103 typedef struct _star_v1_GetSystemStatusRequest {
00104     /* Standard request header. */
00105     bool has_header;
00106     star_v1_RequestHeader header;
00107 } star_v1_GetSystemStatusRequest;
00108
00109 /* IMU sensor data from BNO055 NDOF fusion mode.
00110 Euler angles use MKS units (radians) for ROS2 compatibility.
00111 Quaternion components provided for 3D orientation without gimbal lock. */
00112 typedef struct _star_v1_ImuData {
00113     /* Pitch angle in radians.
00114     Range: -pi/2 to pi/2. */
00115     double pitch_rad;
00116     /* Roll angle in radians.
00117     Range: -pi to pi. */
00118     double roll_rad;
00119     /* Yaw angle in radians.
00120     Range: -pi to pi. */
00121     double yaw_rad;
00122     /* Linear acceleration X in meters per second squared.
00123     Gravity-compensated (BNO055 LINEAR_ACCEL register). */
00124     double accel_x_mps2;
00125     /* Linear acceleration Y in meters per second squared.
00126     Gravity-compensated (BNO055 LINEAR_ACCEL register). */
00127     double accel_y_mps2;
00128     /* Linear acceleration Z in meters per second squared.
00129     Gravity-compensated (BNO055 LINEAR_ACCEL register).
00130     At rest, should read approximately 0.0 m/s^2 (gravity removed). */
00131     double accel_z_mps2;
00132     /* Angular velocity X (roll rate) in radians per second. */
00133     double gyro_x_rad_per_s;
00134     /* Angular velocity Y (pitch rate) in radians per second. */
00135     double gyro_y_rad_per_s;
00136     /* Angular velocity Z (yaw rate) in radians per second. */
00137     double gyro_z_rad_per_s;
00138     /* Compass heading (Euler heading from BNO055 NDOF fusion) in radians.
00139     Range: 0.0 to 2*pi radians (0 to 6.283185). 0 = North, pi/2 = East.
00140     Derived from heading_deg16 register value / 16.0 * (pi / 180.0). */
00141     double heading_rad;
00142     /* Quaternion W component (scalar part). Range: -1.0 to 1.0.
00143     Derived from BNO055 QUA_DATA register / 16384.0.
00144     Convention: BNO055 outputs Hamilton-convention quaternions (scalar-first
00145     internally). Maps to ROS2 geometry_msgs/Quaternion as follows:
00146     geometry_msgs::Quaternion.x = quat_x
00147     geometry_msgs::Quaternion.y = quat_y
00148     geometry_msgs::Quaternion.z = quat_z
00149     geometry_msgs::Quaternion.w = quat_w */
00150     double quat_w;
00151     /* Quaternion X component. Range: -1.0 to 1.0. */
00152     double quat_x;
00153     /* Quaternion Y component. Range: -1.0 to 1.0. */
00154     double quat_y;
00155     /* Quaternion Z component. Range: -1.0 to 1.0. */
00156     double quat_z;
00157     /* BNO055 calibration status byte (raw CALIB_STAT register 0x35).
00158     Bits [7:6] = SYS calibration (0=uncal, 3=fully cal).
00159     Bits [5:4] = Gyroscope calibration.

```

```

00160 Bits [3:2] = Accelerometer calibration.
00161 Bits [1:0] = Magnetometer calibration. */
00162 uint32_t calibration_status;
00163 /* BNO055 on-chip temperature in degrees Celsius.
00164 Range: -40.0 to +85.0 degC. */
00165 double temperature_celsius;
00166 } star_v1_ImuData;
00167
00168 /* Barometric pressure and temperature data from BMP280.
00169 Provides atmospheric pressure for altitude estimation and ambient temperature. */
00170 typedef struct _star_v1_BaroData {
00171     /* Compensated temperature in degrees Celsius.
00172     Range: -40.0 to +85.0 degC.
00173     Derived from BMP280 temp_centi_degc / 100.0. */
00174     double temperature_celsius;
00175     /* Compensated atmospheric pressure in Pascals.
00176     Typical range: 30000 to 110000 Pa (300 to 1100 hPa).
00177     Derived from BMP280 press_pa_256 / 256.0. */
00178     double pressure_pa;
00179 } star_v1_BaroData;
00180
00181 /* Obstacle detection data from ultrasonic sensors (RX72N specific).
00182 Reports distance and detection state from 4 HC-SR04 sensors. */
00183 typedef struct _star_v1_ObstacleData {
00184     /* Distance from sensor 0 in meters. 0.0 if sensor invalid. */
00185     float distance_front_left_m;
00186     /* Distance from sensor 1 in meters. 0.0 if sensor invalid. */
00187     float distance_front_right_m;
00188     /* Distance from sensor 2 in meters. 0.0 if sensor invalid. */
00189     float distance_back_left_m;
00190     /* Distance from sensor 3 in meters. 0.0 if sensor invalid. */
00191     float distance_back_right_m;
00192     /* Any sensor has detected an obstacle within threshold distance. */
00193     bool any_obstacle;
00194     /* Per-sensor detection bitmask. Bit N set = sensor N detected obstacle.
00195     Bit 0: sensor 0, Bit 1: sensor 1, Bit 2: sensor 2, Bit 3: sensor 3. */
00196     uint32_t detected_mask;
00197 } star_v1_ObstacleData;
00198
00199 /* GPS position data. */
00200 typedef struct _star_v1_GpsData {
00201     /* WGS84 latitude in degrees.
00202     Range: -90 to 90. */
00203     double latitude_deg;
00204     /* WGS84 longitude in degrees.
00205     Range: -180 to 180. */
00206     double longitude_deg;
00207     /* Altitude above sea level in meters. */
00208     double altitude_m;
00209     /* Horizontal accuracy estimate in meters. */
00210     double accuracy_m;
00211     /* Number of satellites in view. */
00212     int32_t satellites;
00213     /* GPS fix type. */
00214     star_v1_GpsFix fix_type;
00215 } star_v1_GpsData;
00216
00217 /* Complete telemetry snapshot from all sensors. */
00218 typedef struct _star_v1_TelemetryData {
00219     /* IMU sensor data. */
00220     bool has_imu;
00221     star_v1_ImuData imu;
00222     /* GPS data (if available). */
00223     bool has_gps;
00224     star_v1_GpsData gps;
00225     /* Obstacle detection data from ultrasonic sensors (RX72N specific). */
00226     bool has_obstacle;
00227     star_v1_ObstacleData obstacle;
00228     /* WiFi signal strength in dBm.
00229     Typical range: -100 to -30 dBm. */
00230     int32_t wifi_signal_dbm;
00231     /* CPU usage percentage.
00232     Range: 0 to 100 percent. */
00233     double cpu_usage_percent;
00234     /* Temperature in Celsius (from DS18B20 sensor). */
00235     double temperature_celsius;
00236     /* Motor load percentage (average across motors).
00237     Range: 0 to 100 percent. */
00238     double motor_load_percent;
00239     /* Telemetry timestamp. */
00240     bool has_timestamp;
00241     google_protobuf_Timestamp timestamp;
00242     /* Emergency stop reason (RX72N specific). Populated when emergency_stop is true. */
00243     star_v1_EstopReason estop_reason;
00244     /* Emergency stop state (RX72N specific). */
00245     bool emergency_stop;
00246     /* Motor fault flags bitfield (RX72N specific).

```

```

00247 Bit 0: Motor 0 fault (DRV8263 nFAULT; see encoder[0] for position)
00248 Bit 1: Motor 1 fault (DRV8263 nFAULT; see encoder[1] for position)
00249 Bit 2: Motor 2 fault (DRV8263 nFAULT; see encoder[2] for position)
00250 Bit 3: Motor 3 fault (DRV8263 nFAULT; see encoder[3] for position)
00251 Bits 4-31: Reserved, always zero. */
00252 uint32_t fault_flags;
00253 /* Barometric pressure and temperature from BMP280 (RX72N specific). */
00254 bool has_baro;
00255 star_v1_BaroData baro;
00256 /* Timestamp in microseconds since boot (RX72N specific). */
00257 int64_t timestamp_us;
00258 /* Encoder data from motors (RX72N specific - fixed size for embedded).
00259 Front left motor encoder data. */
00260 bool has_encoder_front_left;
00261 star_v1_EncoderData encoder_front_left;
00262 /* Front right motor encoder data. */
00263 bool has_encoder_front_right;
00264 star_v1_EncoderData encoder_front_right;
00265 /* Back left motor encoder data (reserved for future 4-motor support). */
00266 bool has_encoder_back_left;
00267 star_v1_EncoderData encoder_back_left;
00268 /* Back right motor encoder data (reserved for future 4-motor support). */
00269 bool has_encoder_back_right;
00270 star_v1_EncoderData encoder_back_right;
00271 /* Frame sequence number from transport layer for drop detection.
00272 Increments with each transmitted frame. Used by diagnostics to detect
00273 missing frames in the telemetry stream. */
00274 uint32_t frame_sequence;
00275 } star_v1_TelemetryData;
00276
00277 /* Response with telemetry snapshot. */
00278 typedef struct _star_v1_GetTelemetryResponse {
00279 /* Standard response header. */
00280 bool has_header;
00281 star_v1_ResponseHeader header;
00282 /* Current telemetry data. */
00283 bool has_telemetry;
00284 star_v1_TelemetryData telemetry;
00285 } star_v1_GetTelemetryResponse;
00286
00287 /* Complete system status information. */
00288 typedef struct _star_v1_SystemStatus {
00289 /* Overall connection state. */
00290 star_v1_ConnectionStatus connection_status;
00291 /* Current operating mode. */
00292 star_v1_RobotMode mode;
00293 /* RX72N motor controller connected. */
00294 bool rx72n_connected;
00295 /* LiDAR sensor connected. */
00296 bool lidar_connected;
00297 /* ROS2 bridge connected. */
00298 bool ros_connected;
00299 /* RX72N firmware version string. */
00300 char firmware_version[32];
00301 /* System uptime in seconds. */
00302 uint64_t uptime_s;
00303 /* Free heap memory in bytes. */
00304 uint32_t free_heap_bytes;
00305 } star_v1_SystemStatus;
00306
00307 /* Response with system status. */
00308 typedef struct _star_v1_GetSystemStatusResponse {
00309 /* Standard response header. */
00310 bool has_header;
00311 star_v1_ResponseHeader header;
00312 /* System status information. */
00313 bool has_status;
00314 star_v1_SystemStatus status;
00315 } star_v1_GetSystemStatusResponse;
00316
00317
00318 #ifdef __cplusplus
00319 extern "C" {
00320 #endif
00321
00322 /* Helper constants for enums */
00323 #define _star_v1_EstopReason_MIN star_v1_EstopReason_ESTOP_REASON_UNKNOWN
00324 #define _star_v1_EstopReason_MAX star_v1_EstopReason_ESTOP_REASON_OVERCURRENT
00325 #define _star_v1_EstopReason_ARRAYSIZE
00326 ((star_v1_EstopReason)(star_v1_EstopReason_ESTOP_REASON_OVERCURRENT+1))
00327
00328 #define _star_v1_GpsFix_MIN star_v1_GpsFix_GPS_FIX_UNKNOWN
00329 #define _star_v1_GpsFix_MAX star_v1_GpsFix_GPS_FIX_RTK_FIXED
00330 #define _star_v1_GpsFix_ARRAYSIZE ((star_v1_GpsFix)(star_v1_GpsFix_GPS_FIX_RTK_FIXED+1))
00331
00332 #define _star_v1_ConnectionStatus_MIN star_v1_ConnectionStatus_CONNECTION_STATUS_UNKNOWN
00333 #define _star_v1_ConnectionStatus_MAX star_v1_ConnectionStatus_CONNECTION_STATUS_ERROR

```

```

00333 #define __star_v1_ConnectionStatus_ARRAYSIZE
00334 ((star_v1_ConnectionStatus)(star_v1_ConnectionStatus_CONNECTION_STATUS_ERROR+1))
00335 #define __star_v1_RobotMode_MIN star_v1_RobotMode_ROBOT_MODE_UNKNOWN
00336 #define __star_v1_RobotMode_MAX star_v1_RobotMode_ROBOT_MODE_EMERGENCY_STOP
00337 #define __star_v1_RobotMode_ARRAYSIZE
00338 ((star_v1_RobotMode)(star_v1_RobotMode_ROBOT_MODE_EMERGENCY_STOP+1))
00339
00340
00341
00342
00343
00344 #define star_v1_TelemetryData_estop_reason_ENUMTYPE star_v1_EstopReason
00345
00346
00347
00348
00349 #define star_v1_GpsData_fix_type_ENUMTYPE star_v1_GpsFix
00350
00351 #define star_v1_SystemStatus_connection_status_ENUMTYPE star_v1_ConnectionStatus
00352 #define star_v1_SystemStatus_mode_ENUMTYPE star_v1_RobotMode
00353
00354
00355 /* Initializer values for message structs */
00356 #define star_v1_GetTelemetryRequest_init_default {false, star_v1_RequestHeader_init_default}
00357 #define star_v1_GetTelemetryResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
00358 star_v1_TelemetryData_init_default}
00359 #define star_v1_StreamTelemetryRequest_init_default {false, star_v1_RequestHeader_init_default, 0, 0, {"", "", "",
00360 "", "", "", "", "", "", "", "", "", ""}}
00361 #define star_v1_GetSystemStatusRequest_init_default {false, star_v1_RequestHeader_init_default}
00362 #define star_v1_GetSystemStatusResponse_init_default {false, star_v1_ResponseHeader_init_default, false,
00363 star_v1_SystemStatus_init_default}
00364 #define star_v1_TelemetryData_init_default {false, star_v1_ImuData_init_default, false,
00365 star_v1_GpsData_init_default, false, star_v1_ObstacleData_init_default, 0, 0, 0, 0, false,
00366 google_protobuf_Timestamp_init_default, star_v1_EstopReason_MIN, 0, 0, false, star_v1_BaroData_init_default, 0,
00367 false, star_v1_EncoderData_init_default, false, star_v1_EncoderData_init_default, false,
00368 star_v1_EncoderData_init_default, false, star_v1_EncoderData_init_default, 0}
00369 #define star_v1_ImuData_init_default {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
00370 #define star_v1_BaroData_init_default {0, 0}
00371 #define star_v1_ObstacleData_init_default {0, 0, 0, 0, 0}
00372 #define star_v1_GpsData_init_default {0, 0, 0, 0, star_v1_GpsFix_MIN}
00373 #define star_v1_SystemStatus_init_default {star_v1_ConnectionStatus_MIN, star_v1_RobotMode_MIN, 0, 0,
00374 0, "", 0, 0}
00375 #define star_v1_GetTelemetryRequest_init_zero {false, star_v1_RequestHeader_init_zero}
00376 #define star_v1_GetTelemetryResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false,
00377 star_v1_TelemetryData_init_zero}
00378 #define star_v1_StreamTelemetryRequest_init_zero {false, star_v1_RequestHeader_init_zero, 0, 0, {"", "", "", "", "",
00379 "", "", "", "", "", "", ""}}
00380 #define star_v1_GetSystemStatusRequest_init_zero {false, star_v1_RequestHeader_init_zero}
00381 #define star_v1_GetSystemStatusResponse_init_zero {false, star_v1_ResponseHeader_init_zero, false,
00382 star_v1_SystemStatus_init_zero}
00383 #define star_v1_TelemetryData_init_zero {false, star_v1_ImuData_init_zero, false, star_v1_GpsData_init_zero,
00384 false, star_v1_ObstacleData_init_zero, 0, 0, 0, 0, false, google_protobuf_Timestamp_init_zero,
00385 star_v1_EstopReason_MIN, 0, 0, false, star_v1_BaroData_init_zero, 0, false, star_v1_EncoderData_init_zero, false,
00386 star_v1_EncoderData_init_zero, false, star_v1_EncoderData_init_zero, false, star_v1_EncoderData_init_zero, 0}
00387 #define star_v1_ImuData_init_zero {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
00388 #define star_v1_BaroData_init_zero {0, 0}
00389 #define star_v1_ObstacleData_init_zero {0, 0, 0, 0, 0}
00390 #define star_v1_GpsData_init_zero {0, 0, 0, 0, star_v1_GpsFix_MIN}
00391 #define star_v1_SystemStatus_init_zero {star_v1_ConnectionStatus_MIN, star_v1_RobotMode_MIN, 0, 0,
00392 0, "", 0, 0}
00393
00394 /* Field tags (for use in manual encoding/decoding) */
00395 #define star_v1_GetTelemetryRequest_header_tag 1
00396 #define star_v1_StreamTelemetryRequest_header_tag 1
00397 #define star_v1_StreamTelemetryRequest_rate_hz_tag 2
00398 #define star_v1_StreamTelemetryRequest_fields_tag 3
00399 #define star_v1_GetSystemStatusRequest_header_tag 1
00400 #define star_v1_ImuData_pitch_rad_tag 1
00401 #define star_v1_ImuData_roll_rad_tag 2
00402 #define star_v1_ImuData_yaw_rad_tag 3
00403 #define star_v1_ImuData_accel_x_mps2_tag 4
00404 #define star_v1_ImuData_accel_y_mps2_tag 5
00405 #define star_v1_ImuData_accel_z_mps2_tag 6
00406 #define star_v1_ImuData_gyro_x_rad_per_s_tag 7
00407 #define star_v1_ImuData_gyro_y_rad_per_s_tag 8
00408 #define star_v1_ImuData_gyro_z_rad_per_s_tag 9
00409 #define star_v1_ImuData_heading_rad_tag 10
00410 #define star_v1_ImuData_quat_w_tag 11
00411 #define star_v1_ImuData_quat_x_tag 12
00412 #define star_v1_ImuData_quat_y_tag 13
00413 #define star_v1_ImuData_quat_z_tag 14
00414 #define star_v1_ImuData_calibration_status_tag 15
00415 #define star_v1_ImuData_temperature_celsius_tag 16
00416 #define star_v1_BaroData_temperature_celsius_tag 1
00417 #define star_v1_BaroData_pressure_pa_tag 2

```



```

00403 #define star_v1_ObstacleData_distance_front_left_m_tag 1
00404 #define star_v1_ObstacleData_distance_front_right_m_tag 2
00405 #define star_v1_ObstacleData_distance_back_left_m_tag 3
00406 #define star_v1_ObstacleData_distance_back_right_m_tag 4
00407 #define star_v1_ObstacleData_any_obstacle_tag 5
00408 #define star_v1_ObstacleData_detected_mask_tag 6
00409 #define star_v1_GpsData_latitude_deg_tag 1
00410 #define star_v1_GpsData_longitude_deg_tag 2
00411 #define star_v1_GpsData_altitude_m_tag 3
00412 #define star_v1_GpsData_accuracy_m_tag 4
00413 #define star_v1_GpsData_satellites_tag 5
00414 #define star_v1_GpsData_fix_type_tag 6
00415 #define star_v1_TelemetryData_imu_tag 1
00416 #define star_v1_TelemetryData_gps_tag 2
00417 #define star_v1_TelemetryData_obstacle_tag 3
00418 #define star_v1_TelemetryData_wifi_signal_dbm_tag 4
00419 #define star_v1_TelemetryData_cpu_usage_percent_tag 5
00420 #define star_v1_TelemetryData_temperature_celsius_tag 6
00421 #define star_v1_TelemetryData_motor_load_percent_tag 7
00422 #define star_v1_TelemetryData_timestamp_tag 8
00423 #define star_v1_TelemetryData_estop_reason_tag 9
00424 #define star_v1_TelemetryData_emergency_stop_tag 10
00425 #define star_v1_TelemetryData_fault_flags_tag 11
00426 #define star_v1_TelemetryData_baro_tag 12
00427 #define star_v1_TelemetryData_timestamp_us_tag 14
00428 #define star_v1_TelemetryData_encoder_front_left_tag 15
00429 #define star_v1_TelemetryData_encoder_front_right_tag 16
00430 #define star_v1_TelemetryData_encoder_back_left_tag 17
00431 #define star_v1_TelemetryData_encoder_back_right_tag 18
00432 #define star_v1_TelemetryData_frame_sequence_tag 19
00433 #define star_v1_GetTelemetryResponse_header_tag 1
00434 #define star_v1_GetTelemetryResponse_telemetry_tag 2
00435 #define star_v1_SystemStatus_connection_status_tag 1
00436 #define star_v1_SystemStatus_mode_tag 2
00437 #define star_v1_SystemStatus_rx72n_connected_tag 3
00438 #define star_v1_SystemStatus_lidar_connected_tag 4
00439 #define star_v1_SystemStatus_ros_connected_tag 5
00440 #define star_v1_SystemStatus_firmware_version_tag 6
00441 #define star_v1_SystemStatus_uptime_s_tag 7
00442 #define star_v1_SystemStatus_free_heap_bytes_tag 8
00443 #define star_v1_GetSystemStatusResponse_header_tag 1
00444 #define star_v1_GetSystemStatusResponse_status_tag 2
00445
00446 /* Struct field encoding specification for nanopb */
00447 #define star_v1_GetTelemetryRequest_FIELDLIST(X, a) \
00448 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00449 #define star_v1_GetTelemetryRequest_CALLBACK NULL
00450 #define star_v1_GetTelemetryRequest_DEFAULT NULL
00451 #define star_v1_GetTelemetryRequest_header_MSGTYPE star_v1_RequestHeader
00452
00453 #define star_v1_GetTelemetryResponse_FIELDLIST(X, a) \
00454 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00455 X(a, STATIC, OPTIONAL, MESSAGE, telemetry, 2) \
00456 #define star_v1_GetTelemetryResponse_CALLBACK NULL
00457 #define star_v1_GetTelemetryResponse_DEFAULT NULL
00458 #define star_v1_GetTelemetryResponse_header_MSGTYPE star_v1_ResponseHeader
00459 #define star_v1_GetTelemetryResponse_telemetry_MSGTYPE star_v1_TelemetryData
00460
00461 #define star_v1_StreamTelemetryRequest_FIELDLIST(X, a) \
00462 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00463 X(a, STATIC, SINGULAR, INT32, rate_hz, 2) \
00464 X(a, STATIC, REPEATED, STRING, fields, 3) \
00465 #define star_v1_StreamTelemetryRequest_CALLBACK NULL
00466 #define star_v1_StreamTelemetryRequest_DEFAULT NULL
00467 #define star_v1_StreamTelemetryRequest_header_MSGTYPE star_v1_RequestHeader
00468
00469 #define star_v1_GetSystemStatusRequest_FIELDLIST(X, a) \
00470 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00471 #define star_v1_GetSystemStatusRequest_CALLBACK NULL
00472 #define star_v1_GetSystemStatusRequest_DEFAULT NULL
00473 #define star_v1_GetSystemStatusRequest_header_MSGTYPE star_v1_RequestHeader
00474
00475 #define star_v1_GetSystemStatusResponse_FIELDLIST(X, a) \
00476 X(a, STATIC, OPTIONAL, MESSAGE, header, 1) \
00477 X(a, STATIC, OPTIONAL, MESSAGE, status, 2) \
00478 #define star_v1_GetSystemStatusResponse_CALLBACK NULL
00479 #define star_v1_GetSystemStatusResponse_DEFAULT NULL
00480 #define star_v1_GetSystemStatusResponse_header_MSGTYPE star_v1_ResponseHeader
00481 #define star_v1_GetSystemStatusResponse_status_MSGTYPE star_v1_SystemStatus
00482
00483 #define star_v1_TelemetryData_FIELDLIST(X, a) \
00484 X(a, STATIC, OPTIONAL, MESSAGE, imu, 1) \
00485 X(a, STATIC, OPTIONAL, MESSAGE, gps, 2) \
00486 X(a, STATIC, OPTIONAL, MESSAGE, obstacle, 3) \
00487 X(a, STATIC, SINGULAR, INT32, wifi_signal_dbm, 4) \
00488 X(a, STATIC, SINGULAR, DOUBLE, cpu_usage_percent, 5) \
00489 X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 6) \

```

```

00490 X(a, STATIC, SINGULAR, DOUBLE, motor_load_percent, 7) \
00491 X(a, STATIC, OPTIONAL, MESSAGE, timestamp, 8) \
00492 X(a, STATIC, SINGULAR, UENUM, estop_reason, 9) \
00493 X(a, STATIC, SINGULAR, BOOL, emergency_stop, 10) \
00494 X(a, STATIC, SINGULAR, UINT32, fault_flags, 11) \
00495 X(a, STATIC, OPTIONAL, MESSAGE, baro, 12) \
00496 X(a, STATIC, SINGULAR, INT64, timestamp_us, 14) \
00497 X(a, STATIC, OPTIONAL, MESSAGE, encoder_front_left, 15) \
00498 X(a, STATIC, OPTIONAL, MESSAGE, encoder_front_right, 16) \
00499 X(a, STATIC, OPTIONAL, MESSAGE, encoder_back_left, 17) \
00500 X(a, STATIC, OPTIONAL, MESSAGE, encoder_back_right, 18) \
00501 X(a, STATIC, SINGULAR, UINT32, frame_sequence, 19)
00502 #define star_v1_TelemetryData_CALLBACK NULL
00503 #define star_v1_TelemetryData_DEFAULT NULL
00504 #define star_v1_TelemetryData_imu_MSGTYPE star_v1_ImuData
00505 #define star_v1_TelemetryData_gps_MSGTYPE star_v1_GpsData
00506 #define star_v1_TelemetryData_obstacle_MSGTYPE star_v1_ObstacleData
00507 #define star_v1_TelemetryData_timestamp_MSGTYPE google_protobuf_Timestamp
00508 #define star_v1_TelemetryData_baro_MSGTYPE star_v1_BaroData
00509 #define star_v1_TelemetryData_encoder_front_left_MSGTYPE star_v1_EncoderData
00510 #define star_v1_TelemetryData_encoder_front_right_MSGTYPE star_v1_EncoderData
00511 #define star_v1_TelemetryData_encoder_back_left_MSGTYPE star_v1_EncoderData
00512 #define star_v1_TelemetryData_encoder_back_right_MSGTYPE star_v1_EncoderData
00513
00514 #define star_v1_ImuData_FIELDLIST(X, a) \
00515 X(a, STATIC, SINGULAR, DOUBLE, pitch_rad, 1) \
00516 X(a, STATIC, SINGULAR, DOUBLE, roll_rad, 2) \
00517 X(a, STATIC, SINGULAR, DOUBLE, yaw_rad, 3) \
00518 X(a, STATIC, SINGULAR, DOUBLE, accel_x_mps2, 4) \
00519 X(a, STATIC, SINGULAR, DOUBLE, accel_y_mps2, 5) \
00520 X(a, STATIC, SINGULAR, DOUBLE, accel_z_mps2, 6) \
00521 X(a, STATIC, SINGULAR, DOUBLE, gyro_x_rad_per_s, 7) \
00522 X(a, STATIC, SINGULAR, DOUBLE, gyro_y_rad_per_s, 8) \
00523 X(a, STATIC, SINGULAR, DOUBLE, gyro_z_rad_per_s, 9) \
00524 X(a, STATIC, SINGULAR, DOUBLE, heading_rad, 10) \
00525 X(a, STATIC, SINGULAR, DOUBLE, quat_w, 11) \
00526 X(a, STATIC, SINGULAR, DOUBLE, quat_x, 12) \
00527 X(a, STATIC, SINGULAR, DOUBLE, quat_y, 13) \
00528 X(a, STATIC, SINGULAR, DOUBLE, quat_z, 14) \
00529 X(a, STATIC, SINGULAR, UINT32, calibration_status, 15) \
00530 X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 16)
00531 #define star_v1_ImuData_CALLBACK NULL
00532 #define star_v1_ImuData_DEFAULT NULL
00533
00534 #define star_v1_BaroData_FIELDLIST(X, a) \
00535 X(a, STATIC, SINGULAR, DOUBLE, temperature_celsius, 1) \
00536 X(a, STATIC, SINGULAR, DOUBLE, pressure_pa, 2)
00537 #define star_v1_BaroData_CALLBACK NULL
00538 #define star_v1_BaroData_DEFAULT NULL
00539
00540 #define star_v1_ObstacleData_FIELDLIST(X, a) \
00541 X(a, STATIC, SINGULAR, FLOAT, distance_front_left_m, 1) \
00542 X(a, STATIC, SINGULAR, FLOAT, distance_front_right_m, 2) \
00543 X(a, STATIC, SINGULAR, FLOAT, distance_back_left_m, 3) \
00544 X(a, STATIC, SINGULAR, FLOAT, distance_back_right_m, 4) \
00545 X(a, STATIC, SINGULAR, BOOL, any_obstacle, 5) \
00546 X(a, STATIC, SINGULAR, UINT32, detected_mask, 6)
00547 #define star_v1_ObstacleData_CALLBACK NULL
00548 #define star_v1_ObstacleData_DEFAULT NULL
00549
00550 #define star_v1_GpsData_FIELDLIST(X, a) \
00551 X(a, STATIC, SINGULAR, DOUBLE, latitude_deg, 1) \
00552 X(a, STATIC, SINGULAR, DOUBLE, longitude_deg, 2) \
00553 X(a, STATIC, SINGULAR, DOUBLE, altitude_m, 3) \
00554 X(a, STATIC, SINGULAR, DOUBLE, accuracy_m, 4) \
00555 X(a, STATIC, SINGULAR, INT32, satellites, 5) \
00556 X(a, STATIC, SINGULAR, UENUM, fix_type, 6)
00557 #define star_v1_GpsData_CALLBACK NULL
00558 #define star_v1_GpsData_DEFAULT NULL
00559
00560 #define star_v1_SystemStatus_FIELDLIST(X, a) \
00561 X(a, STATIC, SINGULAR, UENUM, connection_status, 1) \
00562 X(a, STATIC, SINGULAR, UENUM, mode, 2) \
00563 X(a, STATIC, SINGULAR, BOOL, rx72n_connected, 3) \
00564 X(a, STATIC, SINGULAR, BOOL, lidar_connected, 4) \
00565 X(a, STATIC, SINGULAR, BOOL, ros_connected, 5) \
00566 X(a, STATIC, SINGULAR, STRING, firmware_version, 6) \
00567 X(a, STATIC, SINGULAR, UINT64, uptime_s, 7) \
00568 X(a, STATIC, SINGULAR, UINT32, free_heap_bytes, 8)
00569 #define star_v1_SystemStatus_CALLBACK NULL
00570 #define star_v1_SystemStatus_DEFAULT NULL
00571
00572 extern const pb_msgdesc_t star_v1_GetTelemetryRequest_msg;
00573 extern const pb_msgdesc_t star_v1_GetTelemetryResponse_msg;
00574 extern const pb_msgdesc_t star_v1_StreamTelemetryRequest_msg;
00575 extern const pb_msgdesc_t star_v1_GetSystemStatusRequest_msg;
00576 extern const pb_msgdesc_t star_v1_GetSystemStatusResponse_msg;

```

```

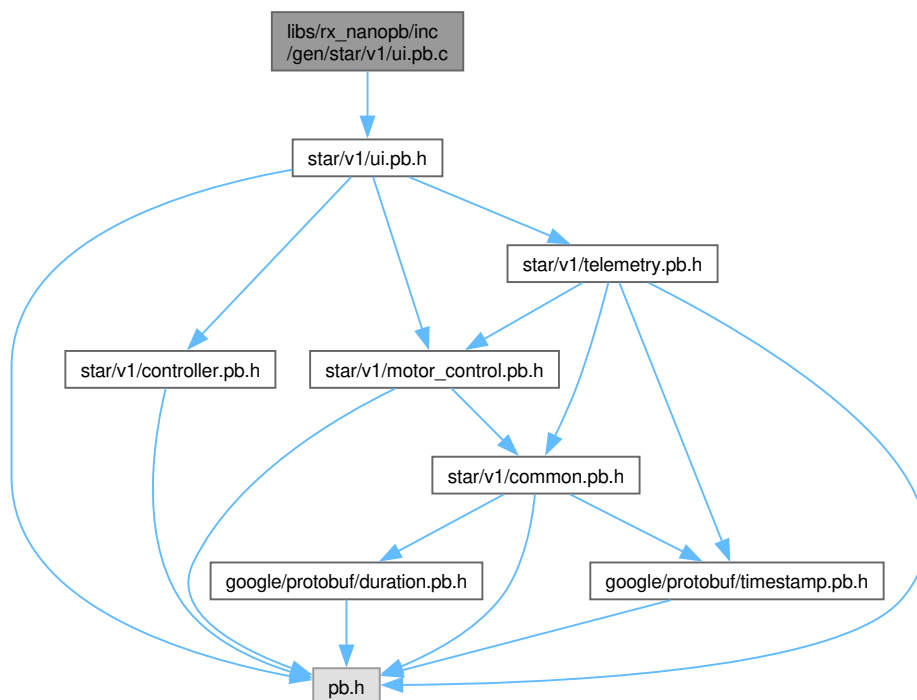
00577 extern const pb_msgdesc_t star_v1_TelemetryData_msg;
00578 extern const pb_msgdesc_t star_v1_ImuData_msg;
00579 extern const pb_msgdesc_t star_v1_BaroData_msg;
00580 extern const pb_msgdesc_t star_v1_ObstacleData_msg;
00581 extern const pb_msgdesc_t star_v1_GpsData_msg;
00582 extern const pb_msgdesc_t star_v1_SystemStatus_msg;
00583
00584 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00585 #define star_v1_GetTelemetryRequest_fields &star_v1_GetTelemetryRequest_msg
00586 #define star_v1_GetTelemetryResponse_fields &star_v1_GetTelemetryResponse_msg
00587 #define star_v1_StreamTelemetryRequest_fields &star_v1_StreamTelemetryRequest_msg
00588 #define star_v1_GetSystemStatusRequest_fields &star_v1_GetSystemStatusRequest_msg
00589 #define star_v1_GetSystemStatusResponse_fields &star_v1_GetSystemStatusResponse_msg
00590 #define star_v1_TelemetryData_fields &star_v1_TelemetryData_msg
00591 #define star_v1_ImuData_fields &star_v1_ImuData_msg
00592 #define star_v1_BaroData_fields &star_v1_BaroData_msg
00593 #define star_v1_ObstacleData_fields &star_v1_ObstacleData_msg
00594 #define star_v1_GpsData_fields &star_v1_GpsData_msg
00595 #define star_v1_SystemStatus_fields &star_v1_SystemStatus_msg
00596
00597 /* Maximum encoded size of messages (where known) */
00598 #define STAR_V1_STAR_V1_TELEMETRY_PB_H_MAX_SIZE star_v1_GetTelemetryResponse_size
00599 #define star_v1_BaroData_size 18
00600 #define star_v1_GetSystemStatusRequest_size 149
00601 #define star_v1_GetSystemStatusResponse_size 425
00602 #define star_v1_GetTelemetryRequest_size 149
00603 #define star_v1_GetTelemetryResponse_size 881
00604 #define star_v1_GpsData_size 49
00605 #define star_v1_ImuData_size 142
00606 #define star_v1_ObstacleData_size 28
00607 #define star_v1_StreamTelemetryRequest_size 688
00608 #define star_v1_SystemStatus_size 60
00609 #define star_v1_TelemetryData_size 515
00610
00611 #ifdef __cplusplus
00612 } /* extern "C" */
00613 #endif
00614
00615 #endif

```

6.41 libs/rx_nanopb/inc/gen/star/v1/ui.pb.c File Reference

#include "star/v1/ui.pb.h"

Include dependency graph for ui.pb.c:



6.42 ui.pb.c

[Go to the documentation of this file.](#)

```

00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "star/v1/ui.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40
00006 #error Regenerate this file with the current version of nanopb generator.
00007 #endif
00008
00009 PB_BIND(star_v1_STAREnvelope, star_v1_STAREnvelope, 4)
00010
00011
00012 PB_BIND(star_v1_MotorStatusList, star_v1_MotorStatusList, AUTO)
00013
00014
00015 PB_BIND(star_v1_OdometryData, star_v1_OdometryData, AUTO)
00016
00017
00018 PB_BIND(star_v1_LidarScan, star_v1_LidarScan, 2)
00019
00020
00021 PB_BIND(star_v1_Alert, star_v1_Alert, AUTO)
00022
00023
00024 PB_BIND(star_v1_EStopCommand, star_v1_EStopCommand, AUTO)
00025
00026
00027
00028
00029
00030 #ifndef PB_CONVERT_DOUBLE_FLOAT
00031 /* On some platforms (such as AVR), double is really float.
00032  * To be able to encode/decode double on these platforms, you need
00033  * to define PB_CONVERT_DOUBLE_FLOAT in pb.h or compiler command line.
00034  */
00035 PB_STATIC_ASSERT(sizeof(double) == 8, DOUBLE_MUST_BE_8_BYTES)
00036 #endif
00037

```

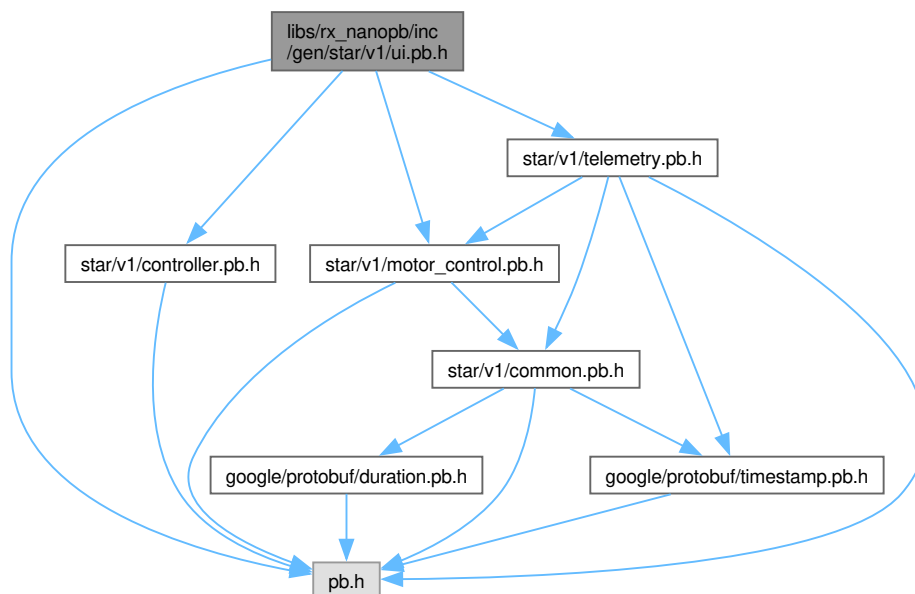
6.43 libs/rx_nanopb/inc/gen/star/v1/ui.pb.h File Reference

```

#include <pb.h>
#include "star/v1/controller.pb.h"
#include "star/v1/motor_control.pb.h"
#include "star/v1/telemetry.pb.h"

```

Include dependency graph for ui.pb.h:



- `#define star_v1_STAREnvelope_payload_motors_MSGTYPE star_v1_MotorStatusList`
- `#define star_v1_STAREnvelope_payload_odometry_MSGTYPE star_v1_OdometryData`
- `#define star_v1_STAREnvelope_payload_lidar_MSGTYPE star_v1_LidarScan`
- `#define star_v1_STAREnvelope_payload_alert_MSGTYPE star_v1_Alert`
- `#define star_v1_STAREnvelope_payload_system_MSGTYPE star_v1_SystemStatus`
- `#define star_v1_STAREnvelope_payload_controller_MSGTYPE star_v1_ControllerState`
- `#define star_v1_STAREnvelope_payload_estop_MSGTYPE star_v1_EStopCommand`
- `#define star_v1_MotorStatusList_FIELDLIST(X, a)`
- `#define star_v1_MotorStatusList_CALLBACK NULL`
- `#define star_v1_MotorStatusList_DEFAULT NULL`
- `#define star_v1_MotorStatusList_motors_MSGTYPE star_v1_MotorStatus`
- `#define star_v1_OdometryData_FIELDLIST(X, a)`
- `#define star_v1_OdometryData_CALLBACK NULL`
- `#define star_v1_OdometryData_DEFAULT NULL`
- `#define star_v1_LidarScan_FIELDLIST(X, a)`
- `#define star_v1_LidarScan_CALLBACK NULL`
- `#define star_v1_LidarScan_DEFAULT NULL`
- `#define star_v1_Alert_FIELDLIST(X, a)`
- `#define star_v1_Alert_CALLBACK NULL`
- `#define star_v1_Alert_DEFAULT NULL`
- `#define star_v1_EStopCommand_FIELDLIST(X, a)`
- `#define star_v1_EStopCommand_CALLBACK NULL`
- `#define star_v1_EStopCommand_DEFAULT NULL`
- `#define star_v1_STAREnvelope_fields &star_v1_STAREnvelope_msg`
- `#define star_v1_MotorStatusList_fields &star_v1_MotorStatusList_msg`
- `#define star_v1_OdometryData_fields &star_v1_OdometryData_msg`
- `#define star_v1_LidarScan_fields &star_v1_LidarScan_msg`
- `#define star_v1_Alert_fields &star_v1_Alert_msg`
- `#define star_v1_EStopCommand_fields &star_v1_EStopCommand_msg`
- `#define STAR_V1_STAR_V1_UI_PB_H_MAX_SIZE star_v1_STAREnvelope_size`
- `#define star_v1_Alert_size 196`
- `#define star_v1_EStopCommand_size 79`
- `#define star_v1_LidarScan_size 7511`
- `#define star_v1_MotorStatusList_size 264`
- `#define star_v1_OdometryData_size 56`
- `#define star_v1_STAREnvelope_size 7536`

Enumerations

- `enum star_v1_AlertLevel {`
`star_v1_AlertLevel_ALERT_LEVEL_UNKNOWN = 0 , star_v1_AlertLevel_ALERT_LEVEL_INFO`
`= 1 , star_v1_AlertLevel_ALERT_LEVEL_WARN = 2 , star_v1_AlertLevel_ALERT_LEVEL_ERROR`
`= 3 ,`
`star_v1_AlertLevel_ALERT_LEVEL_CRITICAL = 4 }`

Variables

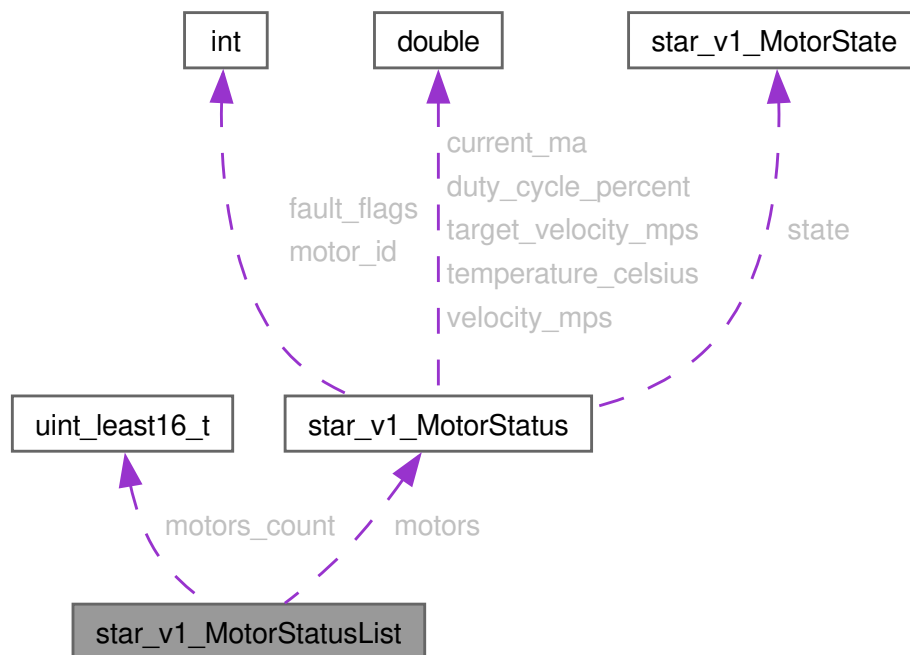
- `const pb_msgdesc_t star_v1_STAREnvelope_msg`
- `const pb_msgdesc_t star_v1_MotorStatusList_msg`
- `const pb_msgdesc_t star_v1_OdometryData_msg`
- `const pb_msgdesc_t star_v1_LidarScan_msg`
- `const pb_msgdesc_t star_v1_Alert_msg`
- `const pb_msgdesc_t star_v1_EStopCommand_msg`

6.43.1 Data Structure Documentation

6.43.1.1 struct star_v1_MotorStatusList

Definition at line 32 of file [ui.pb.h](#).

Collaboration diagram for star_v1_MotorStatusList:



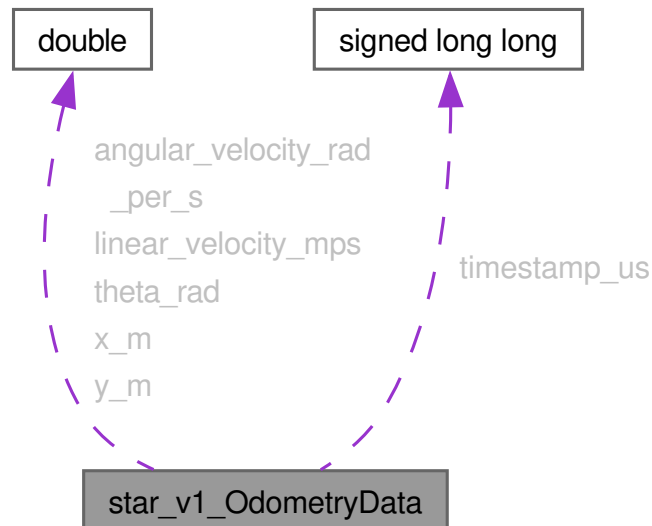
Data Fields

star_v1_MotorStatus	<code>motors[4]</code>	
<code>pb_size_t</code>	<code>motors_count</code>	

6.43.1.2 struct star_v1_OdometryData

Definition at line 40 of file [ui.pb.h](#).

Collaboration diagram for star_v1_OdometryData:



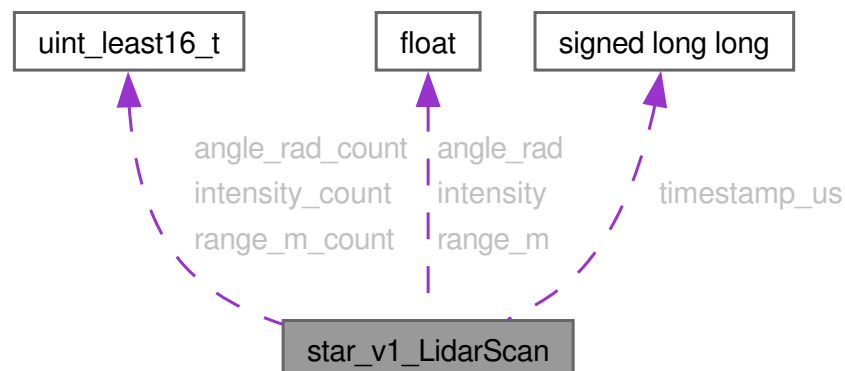
Data Fields

double	angular_velocity_rad_per_s	
double	linear_velocity_mps	
double	theta_rad	
int64_t	timestamp_us	
double	x_m	
double	y_m	

6.43.1.3 struct star_v1_LidarScan

Definition at line 59 of file [ui.pb.h](#).

Collaboration diagram for `star_v1_LidarScan`:



Data Fields

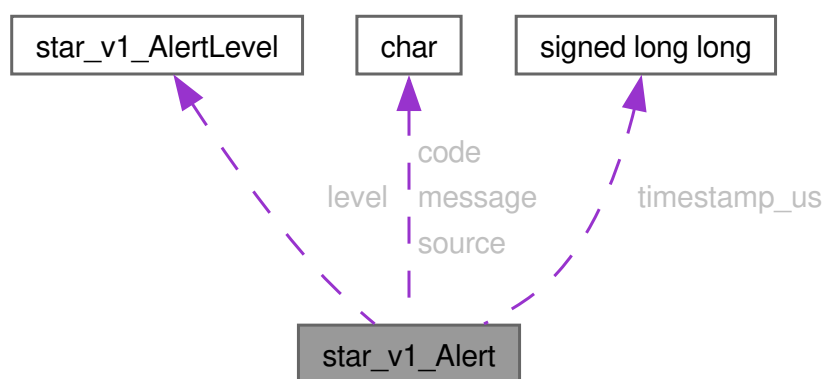
float	angle_rad[500]	
-------	----------------	--

pb_size_t	angle_rad_count	
float	intensity[500]	
pb_size_t	intensity_count	
float	range_m[500]	
pb_size_t	range_m_count	
int64_t	timestamp_us	

6.43.1.4 struct star_v1_Alert

Definition at line 77 of file [ui.pb.h](#).

Collaboration diagram for star_v1_Alert:



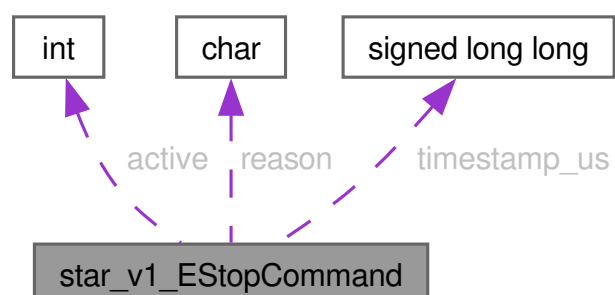
Data Fields

char	code[17]	
star_v1_AlertLevel	level	
char	message[129]	
char	source[33]	
int64_t	timestamp_us	

6.43.1.5 struct star_v1_EStopCommand

Definition at line 91 of file [ui.pb.h](#).

Collaboration diagram for star_v1_EStopCommand:



Data Fields

bool	active	
char	reason[65]	
int64_t	timestamp_us	

6.43.1.6 union star_v1_STAREnvelope.payload

Definition at line 106 of file [ui.pb.h](#).

Data Fields

star_v1_Alert	alert	
star_v1_ControllerState	controller	
star_v1_EStopCommand	estop	
star_v1_LidarScan	lidar	
star_v1_MotorStatusList	motors	
star_v1_OdometryData	odometry	
star_v1_SystemStatus	system	
star_v1_TelemetryData	telemetry	

6.43.2 Macro Definition Documentation

6.43.2.1 __star_v1_AlertLevel_ARRAYSIZE

```
#define __star_v1_AlertLevel_ARRAYSIZE ((star_v1_AlertLevel)(star_v1_AlertLevel_ALERT_LEVEL_CRITICAL+1))
```

Definition at line 138 of file [ui.pb.h](#).

6.43.2.2 __star_v1_AlertLevel_MAX

```
#define __star_v1_AlertLevel_MAX star_v1_AlertLevel_ALERT_LEVEL_CRITICAL
```

Definition at line 137 of file [ui.pb.h](#).

6.43.2.3 __star_v1_AlertLevel_MIN

```
#define __star_v1_AlertLevel_MIN star_v1_AlertLevel_ALERT_LEVEL_UNKNOWN
```

Definition at line 136 of file [ui.pb.h](#).

6.43.2.4 star_v1_Alert_CALLBACK

```
#define star_v1_Alert_CALLBACK NULL
```

Definition at line 246 of file [ui.pb.h](#).

6.43.2.5 star_v1_Alert_code_tag

```
#define star_v1_Alert_code_tag 2
```

Definition at line 175 of file [ui.pb.h](#).

6.43.2.6 star_v1_Alert_DEFAULT

```
#define star_v1_Alert_DEFAULT NULL
```

Definition at line 247 of file [ui.pb.h](#).

6.43.2.7 star_v1_Alert_FIELDLIST

```
#define star_v1_Alert_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, UENUM, level, 1) \
X(a, STATIC, SINGULAR, STRING, code, 2) \
X(a, STATIC, SINGULAR, STRING, message, 3) \
X(a, STATIC, SINGULAR, STRING, source, 4) \
X(a, STATIC, SINGULAR, INT64, timestamp_us, 5)
```

Definition at line 240 of file [ui.pb.h](#).

```
00240 #define star_v1_Alert_FIELDLIST(X, a) \
00241 X(a, STATIC, SINGULAR, UENUM, level, 1) \
00242 X(a, STATIC, SINGULAR, STRING, code, 2) \
00243 X(a, STATIC, SINGULAR, STRING, message, 3) \
00244 X(a, STATIC, SINGULAR, STRING, source, 4) \
00245 X(a, STATIC, SINGULAR, INT64, timestamp_us, 5)
```

6.43.2.8 star_v1_Alert_fields

```
#define star_v1_Alert_fields &star_v1_Alert_msg
```

Definition at line 268 of file [ui.pb.h](#).

6.43.2.9 star_v1_Alert_init_default

```
#define star_v1_Alert_init_default {__star_v1_AlertLevel_MIN, "", "", "", 0}
```

Definition at line 153 of file [ui.pb.h](#).

6.43.2.10 star_v1_Alert_init_zero

```
#define star_v1_Alert_init_zero {__star_v1_AlertLevel_MIN, "", "", "", 0}
```

Definition at line 159 of file [ui.pb.h](#).

6.43.2.11 star_v1_Alert_level_ENUMTYPE

```
#define star_v1_Alert_level_ENUMTYPE star\_v1\_AlertLevel
```

Definition at line [144](#) of file [ui.pb.h](#).

6.43.2.12 star_v1_Alert_level_tag

```
#define star_v1_Alert_level_tag 1
```

Definition at line [174](#) of file [ui.pb.h](#).

6.43.2.13 star_v1_Alert_message_tag

```
#define star_v1_Alert_message_tag 3
```

Definition at line [176](#) of file [ui.pb.h](#).

6.43.2.14 star_v1_Alert_size

```
#define star_v1_Alert_size 196
```

Definition at line [273](#) of file [ui.pb.h](#).

6.43.2.15 star_v1_Alert_source_tag

```
#define star_v1_Alert_source_tag 4
```

Definition at line [177](#) of file [ui.pb.h](#).

6.43.2.16 star_v1_Alert_timestamp_us_tag

```
#define star_v1_Alert_timestamp_us_tag 5
```

Definition at line [178](#) of file [ui.pb.h](#).

6.43.2.17 star_v1_EStopCommand_active_tag

```
#define star_v1_EStopCommand_active_tag 1
```

Definition at line [179](#) of file [ui.pb.h](#).

6.43.2.18 star_v1_EStopCommand_CALLBACK

```
#define star_v1_EStopCommand_CALLBACK NULL
```

Definition at line [253](#) of file [ui.pb.h](#).

6.43.2.19 star_v1_EStopCommand_DEFAULT

```
#define star_v1_EStopCommand_DEFAULT NULL
```

Definition at line 254 of file [ui.pb.h](#).

6.43.2.20 star_v1_EStopCommand_FIELDLIST

```
#define star_v1_EStopCommand_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, BOOL, active, 1) \
X(a, STATIC, SINGULAR, STRING, reason, 2) \
X(a, STATIC, SINGULAR, INT64, timestamp_us, 3)
```

Definition at line 249 of file [ui.pb.h](#).

```
00249 #define star_v1_EStopCommand_FIELDLIST(X, a) \
00250 X(a, STATIC, SINGULAR, BOOL, active, 1) \
00251 X(a, STATIC, SINGULAR, STRING, reason, 2) \
00252 X(a, STATIC, SINGULAR, INT64, timestamp_us, 3)
```

6.43.2.21 star_v1_EStopCommand_fields

```
#define star_v1_EStopCommand_fields &star_v1_EStopCommand_msg
```

Definition at line 269 of file [ui.pb.h](#).

6.43.2.22 star_v1_EStopCommand_init_default

```
#define star_v1_EStopCommand_init_default {0, "", 0}
```

Definition at line 154 of file [ui.pb.h](#).

6.43.2.23 star_v1_EStopCommand_init_zero

```
#define star_v1_EStopCommand_init_zero {0, "", 0}
```

Definition at line 160 of file [ui.pb.h](#).

6.43.2.24 star_v1_EStopCommand_reason_tag

```
#define star_v1_EStopCommand_reason_tag 2
```

Definition at line 180 of file [ui.pb.h](#).

6.43.2.25 star_v1_EStopCommand_size

```
#define star_v1_EStopCommand_size 79
```

Definition at line 274 of file [ui.pb.h](#).

6.43.2.26 star_v1_EStopCommand_timestamp_us_tag

```
#define star_v1_EStopCommand_timestamp_us_tag 3
```

Definition at line 181 of file [ui.pb.h](#).

6.43.2.27 star_v1_LidarScan_angle_rad_tag

```
#define star_v1_LidarScan_angle_rad_tag 1
```

Definition at line 170 of file [ui.pb.h](#).

6.43.2.28 star_v1_LidarScan_CALLBACK

```
#define star_v1_LidarScan_CALLBACK NULL
```

Definition at line 237 of file [ui.pb.h](#).

6.43.2.29 star_v1_LidarScan_DEFAULT

```
#define star_v1_LidarScan_DEFAULT NULL
```

Definition at line 238 of file [ui.pb.h](#).

6.43.2.30 star_v1_LidarScan_FIELDLIST

```
#define star_v1_LidarScan_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, REPEATED, FLOAT, angle_rad, 1) \  
X(a, STATIC, REPEATED, FLOAT, range_m, 2) \  
X(a, STATIC, REPEATED, FLOAT, intensity, 3) \  
X(a, STATIC, SINGULAR, INT64, timestamp_us, 4)
```

Definition at line 232 of file [ui.pb.h](#).

```
00232 #define star_v1_LidarScan_FIELDLIST(X, a) \  
00233 X(a, STATIC, REPEATED, FLOAT, angle_rad, 1) \  
00234 X(a, STATIC, REPEATED, FLOAT, range_m, 2) \  
00235 X(a, STATIC, REPEATED, FLOAT, intensity, 3) \  
00236 X(a, STATIC, SINGULAR, INT64, timestamp_us, 4)
```

6.43.2.31 star_v1_LidarScan_fields

```
#define star_v1_LidarScan_fields &star_v1_LidarScan_msg
```

Definition at line 267 of file [ui.pb.h](#).

6.43.2.39 star_v1_MotorStatusList_DEFAULT

```
#define star_v1_MotorStatusList_DEFAULT NULL
```

Definition at line 219 of file [ui.pb.h](#).

6.43.2.40 star_v1_MotorStatusList_FIELDLIST

```
#define star_v1_MotorStatusList_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, REPEATED, MESSAGE, motors, 1)
```

Definition at line 216 of file [ui.pb.h](#).

```
00216 #define star_v1_MotorStatusList_FIELDLIST(X, a) \  
00217 X(a, STATIC, REPEATED, MESSAGE, motors, 1)
```

6.43.2.41 star_v1_MotorStatusList_fields

```
#define star_v1_MotorStatusList_fields &star_v1_MotorStatusList_msg
```

Definition at line 265 of file [ui.pb.h](#).

6.43.2.42 star_v1_MotorStatusList_init_default

```
#define star_v1_MotorStatusList_init_default {0, {star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default,  
star_v1_MotorStatus_init_default, star_v1_MotorStatus_init_default}}
```

Definition at line 150 of file [ui.pb.h](#).

6.43.2.43 star_v1_MotorStatusList_init_zero

```
#define star_v1_MotorStatusList_init_zero {0, {star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero,  
star_v1_MotorStatus_init_zero, star_v1_MotorStatus_init_zero}}
```

Definition at line 156 of file [ui.pb.h](#).

6.43.2.44 star_v1_MotorStatusList_motors_MSGTYPE

```
#define star_v1_MotorStatusList_motors_MSGTYPE star_v1_MotorStatus
```

Definition at line 220 of file [ui.pb.h](#).

6.43.2.45 star_v1_MotorStatusList_motors_tag

```
#define star_v1_MotorStatusList_motors_tag 1
```

Definition at line 163 of file [ui.pb.h](#).

6.43.2.46 star_v1_MotorStatusList_size

```
#define star_v1_MotorStatusList_size 264
```

Definition at line 276 of file [ui.pb.h](#).

6.43.2.47 star_v1_OdometryData_angular_velocity_rad_per_s_tag

```
#define star_v1_OdometryData_angular_velocity_rad_per_s_tag 5
```

Definition at line 168 of file [ui.pb.h](#).

6.43.2.48 star_v1_OdometryData_CALLBACK

```
#define star_v1_OdometryData_CALLBACK NULL
```

Definition at line 229 of file [ui.pb.h](#).

6.43.2.49 star_v1_OdometryData_DEFAULT

```
#define star_v1_OdometryData_DEFAULT NULL
```

Definition at line 230 of file [ui.pb.h](#).

6.43.2.50 star_v1_OdometryData_FIELDLIST

```
#define star_v1_OdometryData_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, DOUBLE, x_m, 1) \  
X(a, STATIC, SINGULAR, DOUBLE, y_m, 2) \  
X(a, STATIC, SINGULAR, DOUBLE, theta_rad, 3) \  
X(a, STATIC, SINGULAR, DOUBLE, linear_velocity_mps, 4) \  
X(a, STATIC, SINGULAR, DOUBLE, angular_velocity_rad_per_s, 5) \  
X(a, STATIC, SINGULAR, INT64, timestamp_us, 6)
```

Definition at line 222 of file [ui.pb.h](#).

```
00222 #define star_v1_OdometryData_FIELDLIST(X, a) \  
00223 X(a, STATIC, SINGULAR, DOUBLE, x_m, 1) \  
00224 X(a, STATIC, SINGULAR, DOUBLE, y_m, 2) \  
00225 X(a, STATIC, SINGULAR, DOUBLE, theta_rad, 3) \  
00226 X(a, STATIC, SINGULAR, DOUBLE, linear_velocity_mps, 4) \  
00227 X(a, STATIC, SINGULAR, DOUBLE, angular_velocity_rad_per_s, 5) \  
00228 X(a, STATIC, SINGULAR, INT64, timestamp_us, 6)
```

6.43.2.51 star_v1_OdometryData_fields

```
#define star_v1_OdometryData_fields &star_v1_OdometryData_msg
```

Definition at line 266 of file [ui.pb.h](#).

6.43.2.52 `star_v1_OdometryData_init_default`

```
#define star_v1_OdometryData_init_default {0, 0, 0, 0, 0, 0}
```

Definition at line 151 of file [ui.pb.h](#).

6.43.2.53 `star_v1_OdometryData_init_zero`

```
#define star_v1_OdometryData_init_zero {0, 0, 0, 0, 0, 0}
```

Definition at line 157 of file [ui.pb.h](#).

6.43.2.54 `star_v1_OdometryData_linear_velocity_mps_tag`

```
#define star_v1_OdometryData_linear_velocity_mps_tag 4
```

Definition at line 167 of file [ui.pb.h](#).

6.43.2.55 `star_v1_OdometryData_size`

```
#define star_v1_OdometryData_size 56
```

Definition at line 277 of file [ui.pb.h](#).

6.43.2.56 `star_v1_OdometryData_theta_rad_tag`

```
#define star_v1_OdometryData_theta_rad_tag 3
```

Definition at line 166 of file [ui.pb.h](#).

6.43.2.57 `star_v1_OdometryData_timestamp_us_tag`

```
#define star_v1_OdometryData_timestamp_us_tag 6
```

Definition at line 169 of file [ui.pb.h](#).

6.43.2.58 `star_v1_OdometryData_x_m_tag`

```
#define star_v1_OdometryData_x_m_tag 1
```

Definition at line 164 of file [ui.pb.h](#).

6.43.2.59 `star_v1_OdometryData_y_m_tag`

```
#define star_v1_OdometryData_y_m_tag 2
```

Definition at line 165 of file [ui.pb.h](#).

6.43.2.60 STAR_V1_STAR_V1_UI_PB_H_MAX_SIZE

```
#define STAR_V1_STAR_V1_UI_PB_H_MAX_SIZE star_v1_STAREnvelope_size
```

Definition at line 272 of file [ui.pb.h](#).

6.43.2.61 star_v1_STAREnvelope_alert_tag

```
#define star_v1_STAREnvelope_alert_tag 6
```

Definition at line 186 of file [ui.pb.h](#).

6.43.2.62 star_v1_STAREnvelope_CALLBACK

```
#define star_v1_STAREnvelope_CALLBACK NULL
```

Definition at line 205 of file [ui.pb.h](#).

6.43.2.63 star_v1_STAREnvelope_controller_tag

```
#define star_v1_STAREnvelope_controller_tag 8
```

Definition at line 188 of file [ui.pb.h](#).

6.43.2.64 star_v1_STAREnvelope_DEFAULT

```
#define star_v1_STAREnvelope_DEFAULT NULL
```

Definition at line 206 of file [ui.pb.h](#).

6.43.2.65 star_v1_STAREnvelope_estop_tag

```
#define star_v1_STAREnvelope_estop_tag 9
```

Definition at line 189 of file [ui.pb.h](#).

6.43.2.66 star_v1_STAREnvelope_FIELDLIST

```
#define star_v1_STAREnvelope_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC,  ONEOF,  MESSAGE, (payload,telemetry,payload.telemetry),  1) \  
X(a, STATIC,  ONEOF,  MESSAGE, (payload,motors,payload.motors),    2) \  
X(a, STATIC,  ONEOF,  MESSAGE, (payload,odometry,payload.odometry),  4) \  
X(a, STATIC,  ONEOF,  MESSAGE, (payload,lidar,payload.lidar),    5) \  
X(a, STATIC,  ONEOF,  MESSAGE, (payload,alert,payload.alert),    6) \  
X(a, STATIC,  ONEOF,  MESSAGE, (payload,system,payload.system),    7) \  
X(a, STATIC,  ONEOF,  MESSAGE, (payload,controller,payload.controller), 8) \  
X(a, STATIC,  ONEOF,  MESSAGE, (payload,estop,payload.estop),    9) \  
X(a, STATIC,  SINGULAR, UINT64, seq,          14) \  
X(a, STATIC,  SINGULAR, INT64,  ts_ms,        15)
```

Definition at line 194 of file [ui.pb.h](#).

```
00194 #define star_v1_STAREnvelope_FIELDLIST(X, a) \  
00195 X(a, STATIC,  ONEOF,  MESSAGE, (payload,telemetry,payload.telemetry),  1) \  
00196 X(a, STATIC,  ONEOF,  MESSAGE, (payload,motors,payload.motors),    2) \  
00197 X(a, STATIC,  ONEOF,  MESSAGE, (payload,odometry,payload.odometry),  4) \  
00198 X(a, STATIC,  ONEOF,  MESSAGE, (payload,lidar,payload.lidar),    5) \  
00199 X(a, STATIC,  ONEOF,  MESSAGE, (payload,alert,payload.alert),    6) \  
00200 X(a, STATIC,  ONEOF,  MESSAGE, (payload,system,payload.system),    7) \  
00201 X(a, STATIC,  ONEOF,  MESSAGE, (payload,controller,payload.controller), 8) \  
00202 X(a, STATIC,  ONEOF,  MESSAGE, (payload,estop,payload.estop),    9) \  
00203 X(a, STATIC,  SINGULAR, UINT64, seq,          14) \  
00204 X(a, STATIC,  SINGULAR, INT64,  ts_ms,        15)
```

6.43.2.67 star_v1_STAREnvelope_fields

```
#define star_v1_STAREnvelope_fields &star_v1_STAREnvelope_msg
```

Definition at line 264 of file [ui.pb.h](#).

6.43.2.68 star_v1_STAREnvelope_init_default

```
#define star_v1_STAREnvelope_init_default {0, {star_v1_TelemetryData_init_default}, 0, 0}
```

Definition at line 149 of file [ui.pb.h](#).

6.43.2.69 star_v1_STAREnvelope_init_zero

```
#define star_v1_STAREnvelope_init_zero {0, {star_v1_TelemetryData_init_zero}, 0, 0}
```

Definition at line 155 of file [ui.pb.h](#).

6.43.2.70 star_v1_STAREnvelope_lidar_tag

```
#define star_v1_STAREnvelope_lidar_tag 5
```

Definition at line 185 of file [ui.pb.h](#).

6.43.2.71 star_v1_STAREnvelope_motors_tag

```
#define star_v1_STAREnvelope_motors_tag 2
```

Definition at line 183 of file [ui.pb.h](#).

6.43.2.72 star_v1_STAREnvelope_odometry_tag

```
#define star_v1_STAREnvelope_odometry_tag 4
```

Definition at line 184 of file [ui.pb.h](#).

6.43.2.73 star_v1_STAREnvelope_payload_alert_MSGTYPE

```
#define star_v1_STAREnvelope_payload_alert_MSGTYPE star_v1_Alert
```

Definition at line 211 of file [ui.pb.h](#).

6.43.2.74 star_v1_STAREnvelope_payload_controller_MSGTYPE

```
#define star_v1_STAREnvelope_payload_controller_MSGTYPE star_v1_ControllerState
```

Definition at line 213 of file [ui.pb.h](#).

6.43.2.75 `star_v1_STAREnvelope_payload_estop_MSGTYPE`

```
#define star_v1_STAREnvelope_payload_estop_MSGTYPE star_v1_EStopCommand
```

Definition at line 214 of file [ui.pb.h](#).

6.43.2.76 `star_v1_STAREnvelope_payload_lidar_MSGTYPE`

```
#define star_v1_STAREnvelope_payload_lidar_MSGTYPE star_v1_LidarScan
```

Definition at line 210 of file [ui.pb.h](#).

6.43.2.77 `star_v1_STAREnvelope_payload_motors_MSGTYPE`

```
#define star_v1_STAREnvelope_payload_motors_MSGTYPE star_v1_MotorStatusList
```

Definition at line 208 of file [ui.pb.h](#).

6.43.2.78 `star_v1_STAREnvelope_payload_odometry_MSGTYPE`

```
#define star_v1_STAREnvelope_payload_odometry_MSGTYPE star_v1_OdometryData
```

Definition at line 209 of file [ui.pb.h](#).

6.43.2.79 `star_v1_STAREnvelope_payload_system_MSGTYPE`

```
#define star_v1_STAREnvelope_payload_system_MSGTYPE star_v1_SystemStatus
```

Definition at line 212 of file [ui.pb.h](#).

6.43.2.80 `star_v1_STAREnvelope_payload_telemetry_MSGTYPE`

```
#define star_v1_STAREnvelope_payload_telemetry_MSGTYPE star_v1_TelemetryData
```

Definition at line 207 of file [ui.pb.h](#).

6.43.2.81 `star_v1_STAREnvelope_seq_tag`

```
#define star_v1_STAREnvelope_seq_tag 14
```

Definition at line 190 of file [ui.pb.h](#).

6.43.2.82 `star_v1_STAREnvelope_size`

```
#define star_v1_STAREnvelope_size 7536
```

Definition at line 278 of file [ui.pb.h](#).

6.43.2.83 star_v1_STAREnvelope_system_tag

```
#define star_v1_STAREnvelope_system_tag 7
```

Definition at line 187 of file [ui.pb.h](#).

6.43.2.84 star_v1_STAREnvelope_telemetry_tag

```
#define star_v1_STAREnvelope_telemetry_tag 1
```

Definition at line 182 of file [ui.pb.h](#).

6.43.2.85 star_v1_STAREnvelope_ts_ms_tag

```
#define star_v1_STAREnvelope_ts_ms_tag 15
```

Definition at line 191 of file [ui.pb.h](#).

6.43.3 Enumeration Type Documentation

6.43.3.1 star_v1_AlertLevel

```
enum star\_v1\_AlertLevel
```

Enumerator

star_v1_AlertLevel_ALERT_LEVEL_UNKNOWN	
star_v1_AlertLevel_ALERT_LEVEL_INFO	
star_v1_AlertLevel_ALERT_LEVEL_WARN	
star_v1_AlertLevel_ALERT_LEVEL_ERROR	
star_v1_AlertLevel_ALERT_LEVEL_CRITICAL	

Definition at line 17 of file [ui.pb.h](#).

```
00017     {
00018     /* Unknown or unset alert level. */
00019     star_v1_AlertLevel_ALERT_LEVEL_UNKNOWN = 0,
00020     /* Informational alert. */
00021     star_v1_AlertLevel_ALERT_LEVEL_INFO = 1,
00022     /* Warning alert requiring attention. */
00023     star_v1_AlertLevel_ALERT_LEVEL_WARN = 2,
00024     /* Error alert indicating degraded operation. */
00025     star_v1_AlertLevel_ALERT_LEVEL_ERROR = 3,
00026     /* Critical alert requiring immediate action. */
00027     star_v1_AlertLevel_ALERT_LEVEL_CRITICAL = 4
00028 } star_v1_AlertLevel;
```

6.43.4 Variable Documentation

6.43.4.1 star_v1_Alert_msg

```
const pb_msgdesc_t star_v1_Alert_msg [extern]
```

6.43.4.2 star_v1_EStopCommand_msg

```
const pb_msgdesc_t star_v1_EStopCommand_msg [extern]
```

6.43.4.3 star_v1_LidarScan_msg

```
const pb_msgdesc_t star_v1_LidarScan_msg [extern]
```

6.43.4.4 star_v1_MotorStatusList_msg

```
const pb_msgdesc_t star_v1_MotorStatusList_msg [extern]
```

6.43.4.5 star_v1_OdometryData_msg

```
const pb_msgdesc_t star_v1_OdometryData_msg [extern]
```

6.43.4.6 star_v1_STAREnvelope_msg

```
const pb_msgdesc_t star_v1_STAREnvelope_msg [extern]
```

6.44 ui.pb.h

[Go to the documentation of this file.](#)

```
00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_STAR_V1_STAR_V1_UI_PB_H_INCLUDED
00005 #define PB_STAR_V1_STAR_V1_UI_PB_H_INCLUDED
00006 #include <pb.h>
00007 #include "star/v1/controller.pb.h"
00008 #include "star/v1/motor_control.pb.h"
00009 #include "star/v1/telemetry.pb.h"
00010
00011 #if PB_PROTO_HEADER_VERSION != 40
00012 #error Regenerate this file with the current version of nanopb generator.
00013 #endif
00014
00015 /* Enum definitions */
00016 /* Alert severity level. */
00017 typedef enum _star_v1_AlertLevel {
00018     /* Unknown or unset alert level. */
00019     star_v1_AlertLevel_ALERT_LEVEL_UNKNOWN = 0,
00020     /* Informational alert. */
00021     star_v1_AlertLevel_ALERT_LEVEL_INFO = 1,
00022     /* Warning alert requiring attention. */
00023     star_v1_AlertLevel_ALERT_LEVEL_WARN = 2,
00024     /* Error alert indicating degraded operation. */
00025     star_v1_AlertLevel_ALERT_LEVEL_ERROR = 3,
00026     /* Critical alert requiring immediate action. */
00027     star_v1_AlertLevel_ALERT_LEVEL_CRITICAL = 4
00028 } star_v1_AlertLevel;
00029
00030 /* Struct definitions */
00031 /* MotorStatusList bundles the status of all drive motors into one envelope payload. */
00032 typedef struct _star_v1_MotorStatusList {
00033     /* Per-motor status for each drive motor.
00034     nanopb max_count:4 (4-motor robot configuration, see ui.options). */
00035     pb_size_t motors_count;
00036     star_v1_MotorStatus motors[4];
00037 } star_v1_MotorStatusList;
00038
00039 /* OdometryData reports the robot's estimated pose and velocity from wheel odometry. */
```

```

00040 typedef struct _star_v1_OdometryData {
00041     /* X position in metres, world frame. */
00042     double x_m;
00043     /* Y position in metres, world frame. */
00044     double y_m;
00045     /* Heading angle in radians [-pi, pi]. */
00046     double theta_rad;
00047     /* Forward linear velocity in m/s. */
00048     double linear_velocity_mps;
00049     /* Angular velocity in rad/s. */
00050     double angular_velocity_rad_per_s;
00051     /* Source timestamp in microseconds (CLOCK_MONOTONIC). */
00052     int64_t timestamp_us;
00053 } star_v1_OdometryData;
00054
00055 /* LidarScan is a single 360deg scan from the RPLiDAR C1.
00056 angle_rad, range_m, and intensity arrays are parallel; all have the same length.
00057 Maximum samples per revolution: 500 (RPLiDAR C1 @ 10 Hz: 5 kHz sample rate / 10 Hz = 500 samples/rev).
00058 nanopb static bound: max_count:500 for all three arrays (see ui.options). */
00059 typedef struct _star_v1_LidarScan {
00060     /* Angle in radians for each sample [0, 2*pi).
00061 nanopb max_count:500 (RPLiDAR C1 samples/rev; see ui.options). */
00062     pb_size_t angle_rad_count;
00063     float angle_rad[500];
00064     /* Range in metres for each sample (0 = invalid/out-of-range).
00065 nanopb max_count:500 (RPLiDAR C1 samples/rev; see ui.options). */
00066     pb_size_t range_m_count;
00067     float range_m[500];
00068     /* Reflectance intensity in arbitrary sensor units (dimensionless, sensor-specific scale).
00069 nanopb max_count:500 (RPLiDAR C1 samples/rev; see ui.options). */
00070     pb_size_t intensity_count;
00071     float intensity[500];
00072     /* Scan start timestamp in microseconds (CLOCK_MONOTONIC, same epoch as other telemetry). */
00073     int64_t timestamp_us;
00074 } star_v1_LidarScan;
00075
00076 /* Alert is a structured diagnostic notification from any subsystem. */
00077 typedef struct _star_v1_Alert {
00078     /* Severity of the alert. */
00079     star_v1_AlertLevel level;
00080     /* Short machine-readable alert code (max 16 chars). */
00081     char code[17];
00082     /* Human-readable description (max 128 chars). */
00083     char message[129];
00084     /* Subsystem that raised the alert (max 32 chars). */
00085     char source[33];
00086     /* Alert creation timestamp in microseconds (CLOCK_MONOTONIC). */
00087     int64_t timestamp_us;
00088 } star_v1_Alert;
00089
00090 /* EStopCommand activates or clears the robot emergency stop. */
00091 typedef struct _star_v1_EStopCommand {
00092     /* true = stop active, false = stop cleared. */
00093     bool active;
00094     /* Reason for the stop (max 64 chars). */
00095     char reason[65];
00096     /* Command timestamp in microseconds (CLOCK_MONOTONIC). */
00097     int64_t timestamp_us;
00098 } star_v1_EStopCommand;
00099
00100 /* STAREnvelope is a single multiplexed message type for the /ws WebSocket.
00101 Field number layout:
00102 Payload variants: 1-9 (single-byte encoding, most frequent)
00103 Metadata: 14-15 (present on every message but single-byte range preserved for payload) */
00104 typedef struct _star_v1_STAREnvelope {
00105     pb_size_t which_payload;
00106     union {
00107         /* Robot telemetry sensor readings. */
00108         star_v1_TelemetryData telemetry;
00109         /* Motor status list for all drive motors. */
00110         star_v1_MotorStatusList motors;
00111         /* Wheel odometry pose and velocity estimate. */
00112         star_v1_OdometryData odometry;
00113         /* Single 360-degree lidar scan. */
00114         star_v1_LidarScan lidar;
00115         /* Diagnostic alert from a subsystem. */
00116         star_v1_Alert alert;
00117         /* System health and resource status. */
00118         star_v1_SystemStatus system;
00119         /* Gamepad or joystick controller state. */
00120         star_v1_ControllerState controller;
00121         /* Emergency stop command or status. */
00122         star_v1_EStopCommand estop;
00123     } payload;
00124     /* Monotonic sequence stamped by gateway on outbound messages. */
00125     uint64_t seq;
00126     /* Gateway wall-clock timestamp in milliseconds (Unix epoch) at send time. */

```



```

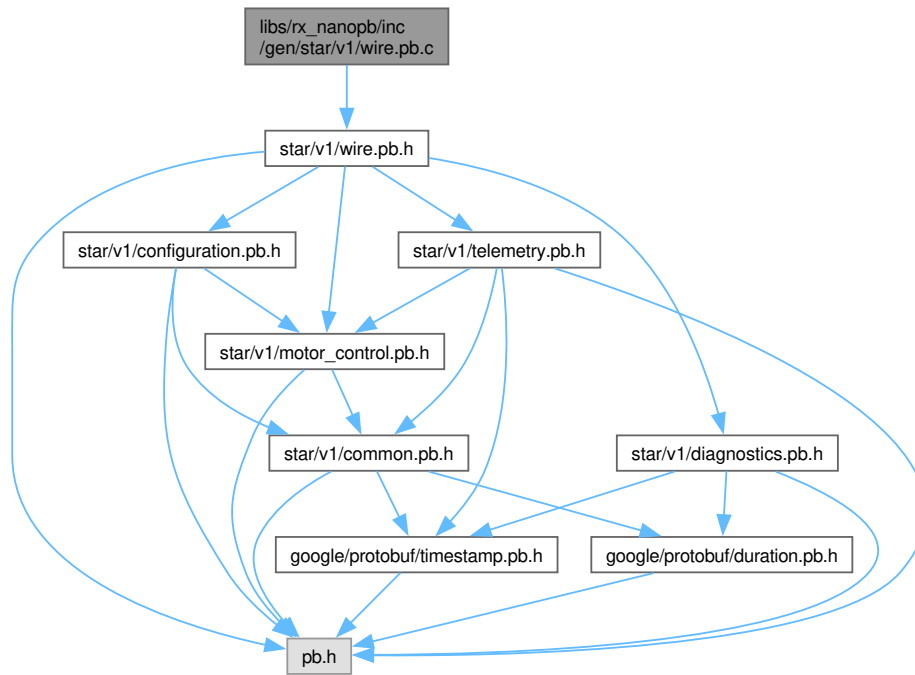
00236 X(a, STATIC, SINGULAR, INT64, timestamp_us, 4)
00237 #define star_v1_LidarScan_CALLBACK NULL
00238 #define star_v1_LidarScan_DEFAULT NULL
00239
00240 #define star_v1_Alert_FIELDLIST(X, a) \
00241 X(a, STATIC, SINGULAR, UENUM, level, 1) \
00242 X(a, STATIC, SINGULAR, STRING, code, 2) \
00243 X(a, STATIC, SINGULAR, STRING, message, 3) \
00244 X(a, STATIC, SINGULAR, STRING, source, 4) \
00245 X(a, STATIC, SINGULAR, INT64, timestamp_us, 5)
00246 #define star_v1_Alert_CALLBACK NULL
00247 #define star_v1_Alert_DEFAULT NULL
00248
00249 #define star_v1_EStopCommand_FIELDLIST(X, a) \
00250 X(a, STATIC, SINGULAR, BOOL, active, 1) \
00251 X(a, STATIC, SINGULAR, STRING, reason, 2) \
00252 X(a, STATIC, SINGULAR, INT64, timestamp_us, 3)
00253 #define star_v1_EStopCommand_CALLBACK NULL
00254 #define star_v1_EStopCommand_DEFAULT NULL
00255
00256 extern const pb_msgdesc_t star_v1_STAREnvelope_msg;
00257 extern const pb_msgdesc_t star_v1_MotorStatusList_msg;
00258 extern const pb_msgdesc_t star_v1_OdometryData_msg;
00259 extern const pb_msgdesc_t star_v1_LidarScan_msg;
00260 extern const pb_msgdesc_t star_v1_Alert_msg;
00261 extern const pb_msgdesc_t star_v1_EStopCommand_msg;
00262
00263 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00264 #define star_v1_STAREnvelope_fields &star_v1_STAREnvelope_msg
00265 #define star_v1_MotorStatusList_fields &star_v1_MotorStatusList_msg
00266 #define star_v1_OdometryData_fields &star_v1_OdometryData_msg
00267 #define star_v1_LidarScan_fields &star_v1_LidarScan_msg
00268 #define star_v1_Alert_fields &star_v1_Alert_msg
00269 #define star_v1_EStopCommand_fields &star_v1_EStopCommand_msg
00270
00271 /* Maximum encoded size of messages (where known) */
00272 #define STAR_V1_STAR_V1_UI_PB_H_MAX_SIZE star_v1_STAREnvelope_size
00273 #define star_v1_Alert_size 196
00274 #define star_v1_EStopCommand_size 79
00275 #define star_v1_LidarScan_size 7511
00276 #define star_v1_MotorStatusList_size 264
00277 #define star_v1_OdometryData_size 56
00278 #define star_v1_STAREnvelope_size 7536
00279
00280 #ifdef __cplusplus
00281 } /* extern "C" */
00282 #endif
00283
00284 #endif

```

6.45 libs/rx_nanopb/inc/gen/star/v1/wire.pb.c File Reference

#include "star/v1/wire.pb.h"

Include dependency graph for wire.pb.c:



6.46 wire.pb.c

[Go to the documentation of this file.](#)

```

00001 /* Automatically generated nanopb constant definitions */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #include "star/v1/wire.pb.h"
00005 #if PB_PROTO_HEADER_VERSION != 40
00006 #error Regenerate this file with the current version of nanopb generator.
00007 #endif
00008
00009 PB_BIND(star_v1_WireMessage, star_v1_WireMessage, 2)
00010
00011
00012 PB_BIND(star_v1_EmergencyStopCommand, star_v1_EmergencyStopCommand, AUTO)
00013
00014
00015

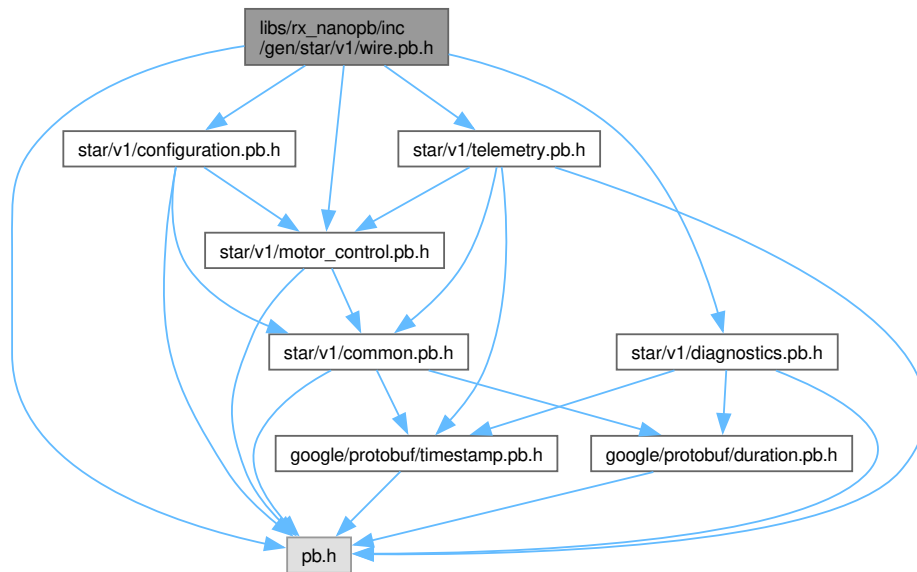
```

6.47 libs/rx_nanopb/inc/gen/star/v1/wire.pb.h File Reference

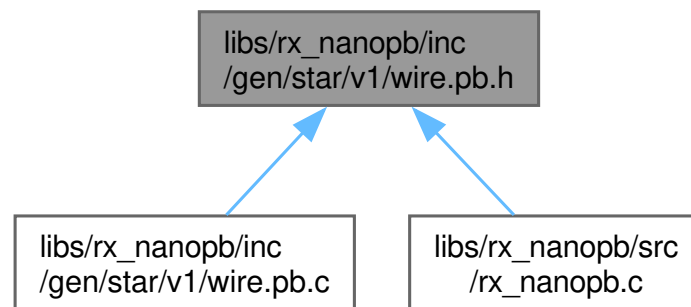
```

#include <pb.h>
#include "star/v1/configuration.pb.h"
#include "star/v1/diagnostics.pb.h"
#include "star/v1/motor_control.pb.h"
#include "star/v1/telemetry.pb.h"
Include dependency graph for wire.pb.h:

```



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `star_v1_EmergencyStopCommand`
- struct `star_v1_WireMessage`
- union `star_v1_WireMessage.payload`

Macros

- `#define star_v1_WireMessage_init_default {0, {star_v1_VelocityCommand_init_default}}`
- `#define star_v1_EmergencyStopCommand_init_default { "", 0, 0 }`
- `#define star_v1_WireMessage_init_zero {0, {star_v1_VelocityCommand_init_zero}}`
- `#define star_v1_EmergencyStopCommand_init_zero { "", 0, 0 }`
- `#define star_v1_EmergencyStopCommand_reason_tag 1`
- `#define star_v1_EmergencyStopCommand_engage_hardware_stop_tag 2`
- `#define star_v1_EmergencyStopCommand_timestamp_us_tag 3`
- `#define star_v1_WireMessage_velocity_command_tag 1`
- `#define star_v1_WireMessage_emergency_stop_command_tag 2`
- `#define star_v1_WireMessage_motor_power_command_tag 3`
- `#define star_v1_WireMessage_telemetry_data_tag 10`
- `#define star_v1_WireMessage_encoder_data_tag 11`
- `#define star_v1_WireMessage_pid_config_tag 30`

- `#define star_v1_WireMessage_retransmit_config_tag 31`
- `#define star_v1_WireMessage_transport_diagnostics_tag 50`
- `#define star_v1_WireMessage_FIELDLIST(X, a)`
- `#define star_v1_WireMessage_CALLBACK NULL`
- `#define star_v1_WireMessage_DEFAULT NULL`
- `#define star_v1_WireMessage_payload_velocity_command_MSGTYPE star_v1_VelocityCommand`
- `#define star_v1_WireMessage_payload_emergency_stop_command_MSGTYPE star_v1_EmergencyStopCommand`
- `#define star_v1_WireMessage_payload_motor_power_command_MSGTYPE star_v1_MotorPowerCommand`
- `#define star_v1_WireMessage_payload_telemetry_data_MSGTYPE star_v1_TelemetryData`
- `#define star_v1_WireMessage_payload_encoder_data_MSGTYPE star_v1_EncoderData`
- `#define star_v1_WireMessage_payload_pid_config_MSGTYPE star_v1_PidConfig`
- `#define star_v1_WireMessage_payload_retransmit_config_MSGTYPE star_v1_RetransmitConfig`
- `#define star_v1_WireMessage_payload_transport_diagnostics_MSGTYPE star_v1_TransportDiagnostics`
- `#define star_v1_EmergencyStopCommand_FIELDLIST(X, a)`
- `#define star_v1_EmergencyStopCommand_CALLBACK NULL`
- `#define star_v1_EmergencyStopCommand_DEFAULT NULL`
- `#define star_v1_WireMessage_fields &star_v1_WireMessage_msg`
- `#define star_v1_EmergencyStopCommand_fields &star_v1_EmergencyStopCommand_msg`
- `#define STAR_V1_STAR_V1_WIRE_PB_H_MAX_SIZE star_v1_WireMessage_size`
- `#define star_v1_EmergencyStopCommand_size 143`
- `#define star_v1_WireMessage_size 1403`

Variables

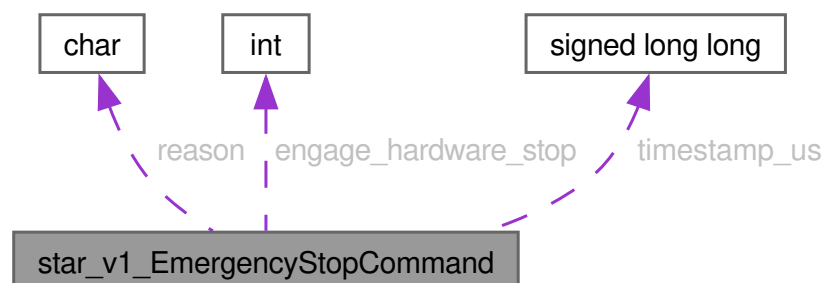
- `const pb_msgdesc_t star_v1_WireMessage_msg`
- `const pb_msgdesc_t star_v1_EmergencyStopCommand_msg`

6.47.1 Data Structure Documentation

6.47.1.1 struct star_v1_EmergencyStopCommand

Definition at line 21 of file [wire.pb.h](#).

Collaboration diagram for star_v1_EmergencyStopCommand:



Data Fields

<code>bool</code>	<code>engage_hardware_stop</code>	
<code>char</code>	<code>reason[128]</code>	

int64_t	timestamp_us	
-------------------------	--------------	--

6.47.1.2 union star_v1_WireMessage.payload

Definition at line 56 of file [wire.pb.h](#).

Data Fields

star_v1_EmergencyStopCommand	emergency_stop_command	
star_v1_EncoderData	encoder_data	
star_v1_MotorPowerCommand	motor_power_command	
star_v1_PidConfig	pid_config	
star_v1_RetransmitConfig	retransmit_config	
star_v1_TelemetryData	telemetry_data	
star_v1_TransportDiagnostics	transport_diagnostics	
star_v1_VelocityCommand	velocity_command	

6.47.2 Macro Definition Documentation

6.47.2.1 star_v1_EmergencyStopCommand_CALLBACK

```
#define star_v1_EmergencyStopCommand_CALLBACK NULL
```

Definition at line 125 of file [wire.pb.h](#).

6.47.2.2 star_v1_EmergencyStopCommand_DEFAULT

```
#define star_v1_EmergencyStopCommand_DEFAULT NULL
```

Definition at line 126 of file [wire.pb.h](#).

6.47.2.3 star_v1_EmergencyStopCommand_engage_hardware_stop_tag

```
#define star_v1_EmergencyStopCommand_engage_hardware_stop_tag 2
```

Definition at line 89 of file [wire.pb.h](#).

6.47.2.4 star_v1_EmergencyStopCommand_FIELDLIST

```
#define star_v1_EmergencyStopCommand_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, SINGULAR, STRING, reason, 1) \
X(a, STATIC, SINGULAR, BOOL, engage_hardware_stop, 2) \
X(a, STATIC, SINGULAR, INT64, timestamp_us, 3)
```

Definition at line 121 of file [wire.pb.h](#).

```
00121 #define star_v1_EmergencyStopCommand_FIELDLIST(X, a) \
00122 X(a, STATIC, SINGULAR, STRING, reason, 1) \
00123 X(a, STATIC, SINGULAR, BOOL, engage_hardware_stop, 2) \
00124 X(a, STATIC, SINGULAR, INT64, timestamp_us, 3)
```

6.47.2.5 star_v1_EmergencyStopCommand_fields

```
#define star_v1_EmergencyStopCommand_fields &star_v1_EmergencyStopCommand_msg
```

Definition at line 133 of file [wire.pb.h](#).

6.47.2.6 star_v1_EmergencyStopCommand_init_default

```
#define star_v1_EmergencyStopCommand_init_default {"" , 0, 0}
```

Definition at line 83 of file [wire.pb.h](#).

6.47.2.7 star_v1_EmergencyStopCommand_init_zero

```
#define star_v1_EmergencyStopCommand_init_zero {"" , 0, 0}
```

Definition at line 85 of file [wire.pb.h](#).

6.47.2.8 star_v1_EmergencyStopCommand_reason_tag

```
#define star_v1_EmergencyStopCommand_reason_tag 1
```

Definition at line 88 of file [wire.pb.h](#).

6.47.2.9 star_v1_EmergencyStopCommand_size

```
#define star_v1_EmergencyStopCommand_size 143
```

Definition at line 137 of file [wire.pb.h](#).

6.47.2.10 star_v1_EmergencyStopCommand_timestamp_us_tag

```
#define star_v1_EmergencyStopCommand_timestamp_us_tag 3
```

Definition at line 90 of file [wire.pb.h](#).

6.47.2.11 STAR_V1_STAR_V1_WIRE_PB_H_MAX_SIZE

```
#define STAR_V1_STAR_V1_WIRE_PB_H_MAX_SIZE star_v1_WireMessage_size
```

Definition at line 136 of file [wire.pb.h](#).

6.47.2.12 star_v1_WireMessage_CALLBACK

```
#define star_v1_WireMessage_CALLBACK NULL
```

Definition at line 110 of file [wire.pb.h](#).

6.47.2.13 star_v1_WireMessage_DEFAULT

```
#define star_v1_WireMessage_DEFAULT NULL
```

Definition at line 111 of file [wire.pb.h](#).

6.47.2.14 star_v1_WireMessage_emergency_stop_command_tag

```
#define star_v1_WireMessage_emergency_stop_command_tag 2
```

Definition at line 92 of file [wire.pb.h](#).

6.47.2.15 star_v1_WireMessage_encoder_data_tag

```
#define star_v1_WireMessage_encoder_data_tag 11
```

Definition at line 95 of file [wire.pb.h](#).

6.47.2.16 star_v1_WireMessage_FIELDLIST

```
#define star_v1_WireMessage_FIELDLIST(  
    X,  
    a)
```

Value:

```
X(a, STATIC, ONEOF, MESSAGE, (payload,velocity_command,payload.velocity_command), 1) \
X(a, STATIC, ONEOF, MESSAGE, (payload,emergency_stop_command,payload.emergency_stop_command), 2) \
X(a, STATIC, ONEOF, MESSAGE, (payload,motor_power_command,payload.motor_power_command), 3) \
X(a, STATIC, ONEOF, MESSAGE, (payload,telemetry_data,payload.telemetry_data), 10) \
X(a, STATIC, ONEOF, MESSAGE, (payload,encoder_data,payload.encoder_data), 11) \
X(a, STATIC, ONEOF, MESSAGE, (payload,pid_config,payload.pid_config), 30) \
X(a, STATIC, ONEOF, MESSAGE, (payload,retransmit_config,payload.retransmit_config), 31) \
X(a, STATIC, ONEOF, MESSAGE, (payload,transport_diagnostics,payload.transport_diagnostics), 50)
```

Definition at line 101 of file [wire.pb.h](#).

```
00101 #define star_v1_WireMessage_FIELDLIST(X, a) \
00102 X(a, STATIC, ONEOF, MESSAGE, (payload,velocity_command,payload.velocity_command), 1) \
00103 X(a, STATIC, ONEOF, MESSAGE, (payload,emergency_stop_command,payload.emergency_stop_command), 2) \
00104 X(a, STATIC, ONEOF, MESSAGE, (payload,motor_power_command,payload.motor_power_command), 3) \
00105 X(a, STATIC, ONEOF, MESSAGE, (payload,telemetry_data,payload.telemetry_data), 10) \
00106 X(a, STATIC, ONEOF, MESSAGE, (payload,encoder_data,payload.encoder_data), 11) \
00107 X(a, STATIC, ONEOF, MESSAGE, (payload,pid_config,payload.pid_config), 30) \
00108 X(a, STATIC, ONEOF, MESSAGE, (payload,retransmit_config,payload.retransmit_config), 31) \
00109 X(a, STATIC, ONEOF, MESSAGE, (payload,transport_diagnostics,payload.transport_diagnostics), 50)
```

6.47.2.17 star_v1_WireMessage_fields

```
#define star_v1_WireMessage_fields &star_v1_WireMessage_msg
```

Definition at line 132 of file [wire.pb.h](#).

Referenced by [rx_nanopb_encode_telemetry\(\)](#).

6.47.2.18 `star_v1_WireMessage_init_default`

```
#define star_v1_WireMessage_init_default {0, {star_v1_VelocityCommand_init_default}}
```

Definition at line 82 of file [wire.pb.h](#).

6.47.2.19 `star_v1_WireMessage_init_zero`

```
#define star_v1_WireMessage_init_zero {0, {star_v1_VelocityCommand_init_zero}}
```

Definition at line 84 of file [wire.pb.h](#).

6.47.2.20 `star_v1_WireMessage_motor_power_command_tag`

```
#define star_v1_WireMessage_motor_power_command_tag 3
```

Definition at line 93 of file [wire.pb.h](#).

6.47.2.21 `star_v1_WireMessage_payload_emergency_stop_command_MSGTYPE`

```
#define star_v1_WireMessage_payload_emergency_stop_command_MSGTYPE star_v1_EmergencyStopCommand
```

Definition at line 113 of file [wire.pb.h](#).

6.47.2.22 `star_v1_WireMessage_payload_encoder_data_MSGTYPE`

```
#define star_v1_WireMessage_payload_encoder_data_MSGTYPE star_v1_EncoderData
```

Definition at line 116 of file [wire.pb.h](#).

6.47.2.23 `star_v1_WireMessage_payload_motor_power_command_MSGTYPE`

```
#define star_v1_WireMessage_payload_motor_power_command_MSGTYPE star_v1_MotorPowerCommand
```

Definition at line 114 of file [wire.pb.h](#).

6.47.2.24 `star_v1_WireMessage_payload_pid_config_MSGTYPE`

```
#define star_v1_WireMessage_payload_pid_config_MSGTYPE star_v1_PidConfig
```

Definition at line 117 of file [wire.pb.h](#).

6.47.2.25 `star_v1_WireMessage_payload_retransmit_config_MSGTYPE`

```
#define star_v1_WireMessage_payload_retransmit_config_MSGTYPE star_v1_RetransmitConfig
```

Definition at line 118 of file [wire.pb.h](#).

6.47.2.26 `star_v1_WireMessage_payload_telemetry_data_MSGTYPE`

```
#define star_v1_WireMessage_payload_telemetry_data_MSGTYPE star\_v1\_TelemetryData
```

Definition at line [115](#) of file [wire.pb.h](#).

6.47.2.27 `star_v1_WireMessage_payload_transport_diagnostics_MSGTYPE`

```
#define star_v1_WireMessage_payload_transport_diagnostics_MSGTYPE star\_v1\_TransportDiagnostics
```

Definition at line [119](#) of file [wire.pb.h](#).

6.47.2.28 `star_v1_WireMessage_payload_velocity_command_MSGTYPE`

```
#define star_v1_WireMessage_payload_velocity_command_MSGTYPE star\_v1\_VelocityCommand
```

Definition at line [112](#) of file [wire.pb.h](#).

6.47.2.29 `star_v1_WireMessage_pid_config_tag`

```
#define star_v1_WireMessage_pid_config_tag 30
```

Definition at line [96](#) of file [wire.pb.h](#).

6.47.2.30 `star_v1_WireMessage_retransmit_config_tag`

```
#define star_v1_WireMessage_retransmit_config_tag 31
```

Definition at line [97](#) of file [wire.pb.h](#).

6.47.2.31 `star_v1_WireMessage_size`

```
#define star_v1_WireMessage_size 1403
```

Definition at line [138](#) of file [wire.pb.h](#).

6.47.2.32 `star_v1_WireMessage_telemetry_data_tag`

```
#define star_v1_WireMessage_telemetry_data_tag 10
```

Definition at line [94](#) of file [wire.pb.h](#).

Referenced by [rx_nanopb_encode_telemetry\(\)](#).

6.47.2.33 star_v1_WireMessage_transport_diagnostics_tag

```
#define star_v1_WireMessage_transport_diagnostics_tag 50
```

Definition at line 98 of file [wire.pb.h](#).

6.47.2.34 star_v1_WireMessage_velocity_command_tag

```
#define star_v1_WireMessage_velocity_command_tag 1
```

Definition at line 91 of file [wire.pb.h](#).

6.47.3 Variable Documentation

6.47.3.1 star_v1_EmergencyStopCommand_msg

```
const pb_msgdesc_t star_v1_EmergencyStopCommand_msg [extern]
```

6.47.3.2 star_v1_WireMessage_msg

```
const pb_msgdesc_t star_v1_WireMessage_msg [extern]
```

6.48 wire.pb.h

[Go to the documentation of this file.](#)

```
00001 /* Automatically generated nanopb header */
00002 /* Generated by nanopb-0.4.9.1 */
00003
00004 #ifndef PB_STAR_V1_STAR_V1_WIRE_PB_H_INCLUDED
00005 #define PB_STAR_V1_STAR_V1_WIRE_PB_H_INCLUDED
00006 #include <pb.h>
00007 #include "star/v1/configuration.pb.h"
00008 #include "star/v1/diagnostics.pb.h"
00009 #include "star/v1/motor_control.pb.h"
00010 #include "star/v1/telemetry.pb.h"
00011
00012 #if PB_PROTO_HEADER_VERSION != 40
00013 #error Regenerate this file with the current version of nanopb generator.
00014 #endif
00015
00016 /* Struct definitions */
00017 /* EmergencyStopCommand triggers immediate motor stop.
00018 This is a dedicated command (not a zero-velocity) to enable
00019 the firmware to engage hardware safety features and enter
00020 MOTOR_STATE_ESTOP as defined in motor_control.proto. */
00021 typedef struct _star_v1_EmergencyStopCommand {
00022     /* Reason for emergency stop (for logging and diagnostics). */
00023     char reason[128];
00024     /* Request hardware E-Stop engagement if available.
00025 When true, firmware should:
00026 - Enter MOTOR_STATE_ESTOP
00027 - Disable PWM outputs via hardware
00028 - Engage brakes if available
00029 - Require explicit reset to exit E-Stop state */
00030     bool engage_hardware_stop;
00031     /* Client timestamp in microseconds for latency tracking. */
00032     int64_t timestamp_us;
00033 } star_v1_EmergencyStopCommand;
00034
00035 /* WireMessage wraps all possible message types for wire-level multiplexing.
00036 The receiver can unmarshal this wrapper and use the oneof discriminator
00037 to determine which concrete message type to process.
```

```

00038
00039 Usage (sender):
00040     wrapper := &WireMessage{
00041         Payload: &WireMessage_VelocityCommand{VelocityCommand: cmd},
00042     }
00043     bytes, _ := proto.Marshal(wrapper)
00044
00045 Usage (receiver):
00046     var wrapper WireMessage
00047     proto.Unmarshal(bytes, &wrapper)
00048     switch msg := wrapper.Payload.(type) {
00049     case *WireMessage_VelocityCommand:
00050         handleVelocity(msg.VelocityCommand)
00051     case *WireMessage_TelemetryData:
00052         handleTelemetry(msg.TelemetryData)
00053     } /*
00054 typedef struct _star_v1_WireMessage {
00055     pb_size_t which_payload;
00056     union {
00057         /* Motor control commands (RPI5 -> RX72N) */
00058         star_v1_VelocityCommand velocity_command;
00059         /* Emergency stop command triggers immediate motor halt with hardware safety engagement. */
00060         star_v1_EmergencyStopCommand emergency_stop_command;
00061         /* Motor power command sets raw PWM duty cycle (bypasses PID control). */
00062         star_v1_MotorPowerCommand motor_power_command;
00063         /* Telemetry data (RX72N -> RPi5) */
00064         star_v1_TelemetryData telemetry_data;
00065         /* Encoder data contains quadrature encoder readings and velocity estimates. */
00066         star_v1_EncoderData encoder_data;
00067         /* Configuration (RPI5 -> RX72N) */
00068         star_v1_PidConfig pid_config;
00069         /* Retransmit configuration (RPI5 -> RX72N) */
00070         star_v1_RetransmitConfig retransmit_config;
00071         /* Transport diagnostics (RPI5 -> UI or bidirectional) */
00072         star_v1_TransportDiagnostics transport_diagnostics;
00073     } payload;
00074 } star_v1_WireMessage;
00075
00076
00077 #ifdef __cplusplus
00078 extern "C" {
00079 #endif
00080
00081 /* Initializer values for message structs */
00082 #define star_v1_WireMessage_init_default {0, {star_v1_VelocityCommand_init_default}}
00083 #define star_v1_EmergencyStopCommand_init_default {"", 0, 0}
00084 #define star_v1_WireMessage_init_zero {0, {star_v1_VelocityCommand_init_zero}}
00085 #define star_v1_EmergencyStopCommand_init_zero {"", 0, 0}
00086
00087 /* Field tags (for use in manual encoding/decoding) */
00088 #define star_v1_EmergencyStopCommand_reason_tag 1
00089 #define star_v1_EmergencyStopCommand_engage_hardware_stop_tag 2
00090 #define star_v1_EmergencyStopCommand_timestamp_us_tag 3
00091 #define star_v1_WireMessage_velocity_command_tag 1
00092 #define star_v1_WireMessage_emergency_stop_command_tag 2
00093 #define star_v1_WireMessage_motor_power_command_tag 3
00094 #define star_v1_WireMessage_telemetry_data_tag 10
00095 #define star_v1_WireMessage_encoder_data_tag 11
00096 #define star_v1_WireMessage_pid_config_tag 30
00097 #define star_v1_WireMessage_retransmit_config_tag 31
00098 #define star_v1_WireMessage_transport_diagnostics_tag 50
00099
00100 /* Struct field encoding specification for nanopb */
00101 #define star_v1_WireMessage_FIELDLIST(X, a) \
00102 X(a, STATIC, ONEOF, MESSAGE, (payload,velocity_command,payload.velocity_command), 1) \
00103 X(a, STATIC, ONEOF, MESSAGE, (payload,emergency_stop_command,payload.emergency_stop_command), 2) \
00104 X(a, STATIC, ONEOF, MESSAGE, (payload,motor_power_command,payload.motor_power_command), 3) \
00105 X(a, STATIC, ONEOF, MESSAGE, (payload,telemetry_data,payload.telemetry_data), 10) \
00106 X(a, STATIC, ONEOF, MESSAGE, (payload,encoder_data,payload.encoder_data), 11) \
00107 X(a, STATIC, ONEOF, MESSAGE, (payload,pid_config,payload.pid_config), 30) \
00108 X(a, STATIC, ONEOF, MESSAGE, (payload,retransmit_config,payload.retransmit_config), 31) \
00109 X(a, STATIC, ONEOF, MESSAGE, (payload,transport_diagnostics,payload.transport_diagnostics), 50)
00110 #define star_v1_WireMessage_CALLBACK NULL
00111 #define star_v1_WireMessage_DEFAULT NULL
00112 #define star_v1_WireMessage_payload_velocity_command_MSGTYPE star_v1_VelocityCommand
00113 #define star_v1_WireMessage_payload_emergency_stop_command_MSGTYPE star_v1_EmergencyStopCommand
00114 #define star_v1_WireMessage_payload_motor_power_command_MSGTYPE star_v1_MotorPowerCommand
00115 #define star_v1_WireMessage_payload_telemetry_data_MSGTYPE star_v1_TelemetryData
00116 #define star_v1_WireMessage_payload_encoder_data_MSGTYPE star_v1_EncoderData
00117 #define star_v1_WireMessage_payload_pid_config_MSGTYPE star_v1_PidConfig
00118 #define star_v1_WireMessage_payload_retransmit_config_MSGTYPE star_v1_RetransmitConfig
00119 #define star_v1_WireMessage_payload_transport_diagnostics_MSGTYPE star_v1_TransportDiagnostics
00120
00121 #define star_v1_EmergencyStopCommand_FIELDLIST(X, a) \
00122 X(a, STATIC, SINGULAR, STRING, reason, 1) \
00123 X(a, STATIC, SINGULAR, BOOL, engage_hardware_stop, 2) \
00124 X(a, STATIC, SINGULAR, INT64, timestamp_us, 3)

```



```

00125 #define star_v1_EmergencyStopCommand_CALLBACK NULL
00126 #define star_v1_EmergencyStopCommand_DEFAULT NULL
00127
00128 extern const pb_msgdesc_t star_v1_WireMessage_msg;
00129 extern const pb_msgdesc_t star_v1_EmergencyStopCommand_msg;
00130
00131 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00132 #define star_v1_WireMessage_fields &star_v1_WireMessage_msg
00133 #define star_v1_EmergencyStopCommand_fields &star_v1_EmergencyStopCommand_msg
00134
00135 /* Maximum encoded size of messages (where known) */
00136 #define STAR_V1_STAR_V1_WIRE_PB_H_MAX_SIZE      star_v1_WireMessage_size
00137 #define star_v1_EmergencyStopCommand_size      143
00138 #define star_v1_WireMessage_size              1403
00139
00140 #ifdef __cplusplus
00141 } /* extern "C" */
00142 #endif
00143
00144 #endif

```

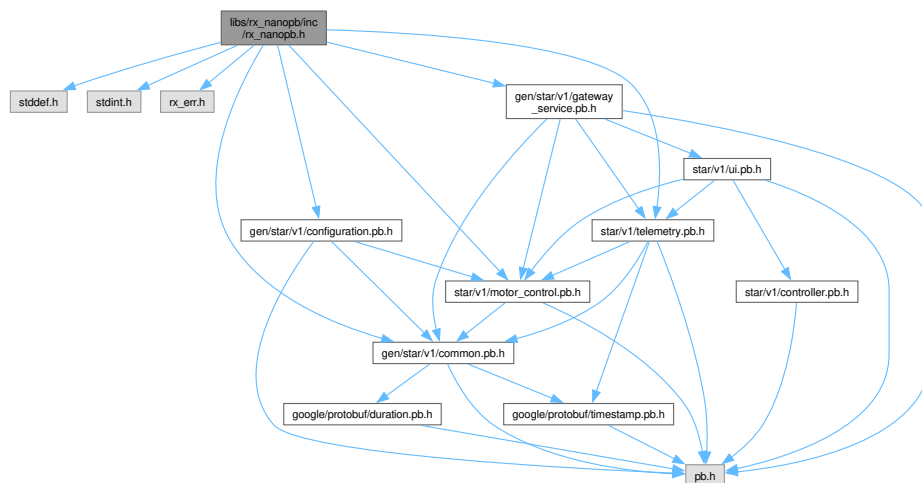
6.49 libs/rx_nanopb/inc/rx_nanopb.h File Reference

nanopb Integration Wrapper for RX72N

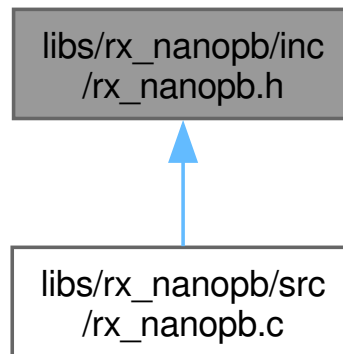
```

#include <stddef.h>
#include <stdint.h>
#include "rx_err.h"
#include "gen/star/v1/common.pb.h"
#include "gen/star/v1/configuration.pb.h"
#include "gen/star/v1/gateway_service.pb.h"
#include "gen/star/v1/motor_control.pb.h"
#include "gen/star/v1/telemetry.pb.h"
Include dependency graph for rx_nanopb.h:

```



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `rx_velocity_command_params_t`
VelocityCommand parameters for 4 independent motors.
- struct `rx_velocity_diff_drive_params_t`
VelocityCommand parameters for differential drive mode.

Functions

- `rx_err_t rx_nanopb_init` (void)
Initialize nanopb wrapper module.
- `rx_err_t rx_nanopb_encode_velocity_request` (const `star_v1_SetVelocityRequest` *msg, `uint8_t` *buffer, `uint32_t` buffer_size, `uint32_t` *len)
Encode SetVelocityRequest message to Protocol Buffer bytes.
- `rx_err_t rx_nanopb_decode_velocity_request` (const `uint8_t` *buffer, `uint32_t` len, `star_v1_SetVelocityRequest` *msg)
Decode SetVelocityRequest message from Protocol Buffer bytes.
- `rx_err_t rx_nanopb_encode_velocity_response` (const `star_v1_SetVelocityResponse` *msg, `uint8_t` *buffer, `uint32_t` buffer_size, `uint32_t` *len)
Encode SetVelocityResponse message to Protocol Buffer bytes.
- `rx_err_t rx_nanopb_decode_estop_request` (const `uint8_t` *buffer, `uint32_t` len, `star_v1_EmergencyStopRequest` *msg)
Decode EmergencyStopRequest message from Protocol Buffer bytes.
- `rx_err_t rx_nanopb_encode_estop_response` (const `star_v1_EmergencyStopResponse` *msg, `uint8_t` *buffer, `uint32_t` buffer_size, `uint32_t` *len)
Encode EmergencyStopResponse message to Protocol Buffer bytes.
- `rx_err_t rx_nanopb_decode_pid_gains_request` (const `uint8_t` *buffer, `uint32_t` len, `star_v1_SetPidGainsRequest` *msg)
Decode SetPidGainsRequest message from Protocol Buffer bytes.
- `rx_err_t rx_nanopb_encode_pid_gains_response` (const `star_v1_SetPidGainsResponse` *msg, `uint8_t` *buffer, `uint32_t` buffer_size, `uint32_t` *len)
Encode SetPidGainsResponse message to Protocol Buffer bytes.
- `rx_err_t rx_nanopb_decode_retransmit_config_request` (const `uint8_t` *buffer, `uint32_t` len, `star_v1_SetRetransmitConfigRequest` *msg)
Decode SetRetransmitConfigRequest from Protocol Buffer bytes.
- `rx_err_t rx_nanopb_encode_telemetry` (const `star_v1_TelemetryData` *msg, `uint8_t` *buffer, `uint32_t` buffer_size, `uint32_t` *len)
Encode TelemetryData message to Protocol Buffer bytes.
- `rx_err_t rx_nanopb_create_velocity_command` (`star_v1_VelocityCommand` *cmd, const `rx_velocity_command_params_t` *params)

- Create VelocityCommand for 4 independent motors.
- `rx_err_t rx_nanopb_create_velocity_command_diff_drive (star_v1_VelocityCommand *cmd, const rx_velocity_diff_drive_params_t *params)`
Create VelocityCommand in differential drive mode.
- `void rx_nanopb_create_response_header (star_v1_ResponseHeader *header, star_v1_Status status, const char *request_id)`
Create ResponseHeader with status.
- `void rx_nanopb_test_reset_state (void)`
Reset module state for unit testing.

Variables

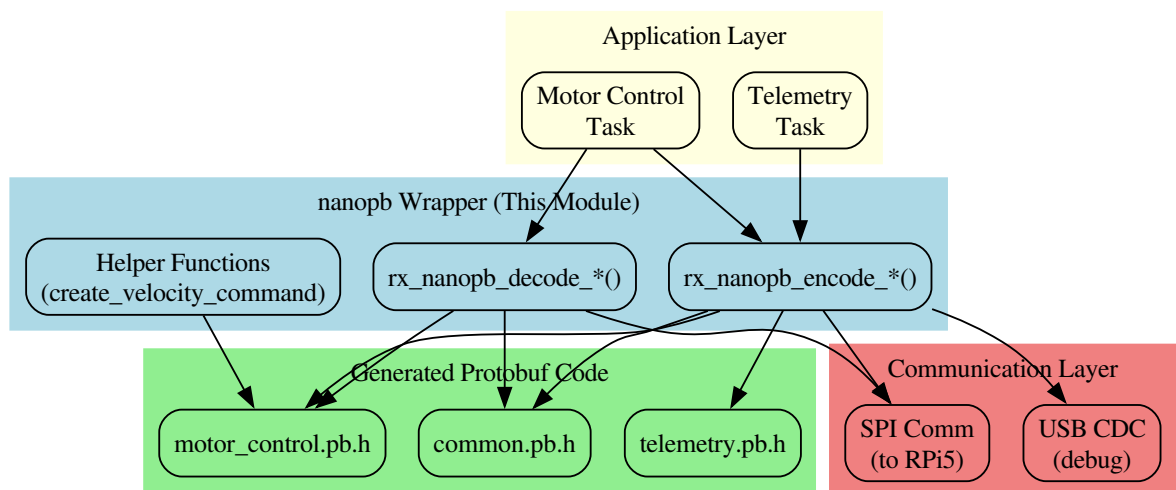
- static const `uint16_t s_nanopb_buffer_size = 512U`
Maximum encode/decode buffer size in bytes.

6.49.1 Detailed Description

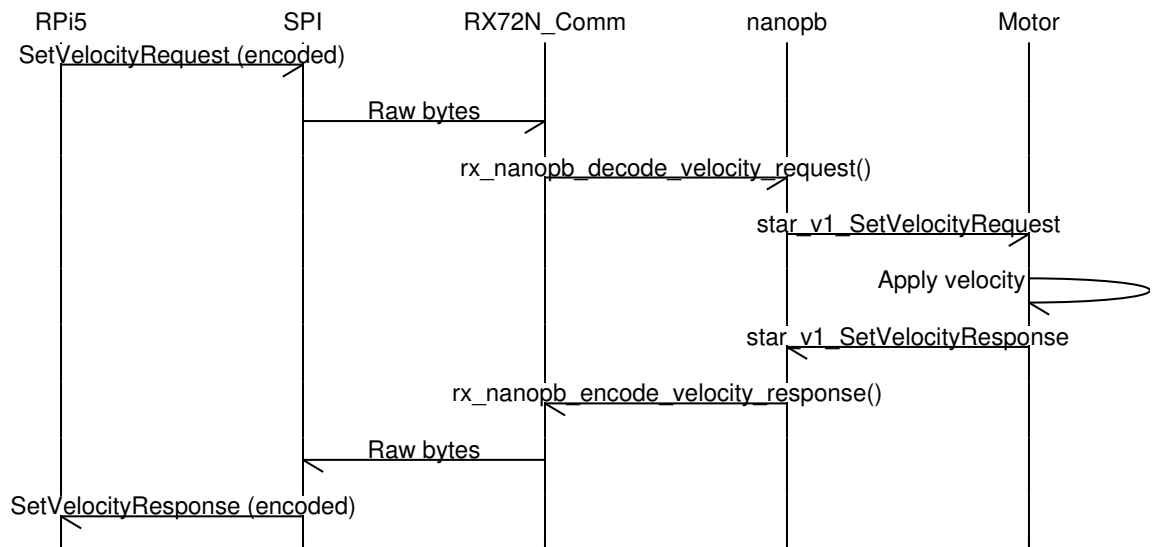
nanopb Integration Wrapper for RX72N

Provides simplified encode/decode functions with static buffers for Protocol Buffer messages used in the STAR project. Wraps the nanopb library with RX72N-friendly APIs that handle memory constraints and error reporting.

System Architecture



Message Flow



Supported Message Types

Message	Direction	Description
SetVelocityRequest	RPi5 -> RX72N	Motor velocity commands
SetVelocityResponse	RX72N -> RPi5	Command acknowledgment
EmergencyStopRequest	RPi5 -> RX72N	Emergency stop command
EmergencyStopResponse	RX72N -> RPi5	E-stop acknowledgment
TelemetryData	RX72N -> RPi5	Sensor/motor telemetry

Buffer Configuration

Parameter	Value	Notes
Max buffer size	512 bytes	Static allocation
Max message size	~256 bytes	Largest single message
Alignment	4-byte	For DMA compatibility

Memory Usage

Component	Size	Notes
Code (.text)	~3 KB	All encode/decode functions
Static buffer	512 bytes	Shared encode buffer
Stack usage	~128 bytes	Per encode/decode call

Component	Size	Notes
Message struct	64-128 bytes	On caller stack

Performance Characteristics

Operation	Time @ 240 MHz	Notes
Encode velocity	~20 us	4-motor command
Decode velocity	~15 us	4-motor command
Encode telemetry	~30 us	Full telemetry packet
Buffer copy	~5 us	256 bytes

Wire Format

nanopb uses standard Protocol Buffer encoding:

- Variable-length integers (varint)
- Length-delimited fields for strings/bytes
- Little-endian for fixed-width types
- No alignment padding

Error Handling

Error	Cause	Recovery
k_rx_err_invalid_size	Buffer too small	Use larger buffer
k_rx_err_protocol_error	Malformed message	Discard, request retransmit
k_rx_err_not_initialized	Module not init'd	Call rx_nanopb_init()

Thread Safety

- Encode/decode functions: NOT thread-safe (shared buffer)
- Helper functions: Thread-safe (no shared state)
- Call from single task or protect with mutex

Module Dependencies

- nanopb: Core protobuf library (git submodule)
- gen/star/v1/<*>.pb.h: Generated message headers
- rx_err.h: Error code definitions

NASA Power of 10 Compliance

- Rule 1: [OK] No goto, setjmp, or recursion
- Rule 2: [OK] All loops bounded by message field counts
- Rule 3: [OK] No dynamic memory (static buffers only)
- Rule 4: [OK] Functions under 60 lines
- Rule 5: [OK] Input validation on all parameters
- Rule 6: [OK] Variables at smallest scope
- Rule 7: [OK] All return values checked
- Rule 8: [OK] Limited preprocessor (nanopb macros only)
- Rule 9: [OK] Callbacks only for nanopb field encoding
- Rule 10: [OK] Compiled with -Wall -Wextra -Werror

SOLID Principles

- S: Single Responsibility - Protobuf encode/decode only
- O: Open/Closed - Extensible by adding new message types
- L: Liskov Substitution - N/A (no inheritance)
- I: Interface Segregation - Separate function per message type
- D: Dependency Inversion - Depends on generated message headers

See also

star-proto/proto/star/v1/ Protocol Buffer schema definitions

<https://jpa.kapsi.fi/nanopb/> nanopb documentation

Version

1.0.0

Date

2026-01-01

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Definition in file [rx_nanopb.h](#).

6.49.2 Data Structure Documentation

6.49.2.1 struct rx_velocity_command_params_t

VelocityCommand parameters for 4 independent motors.

Convenience structure for creating velocity commands with all four motors independently controlled. Maps directly to the protobuf VelocityCommand.

Motor Layout (Top View)

```

*      FRONT
*  +---+---+
*  | FL | FR |
*  |   |   |
*  +---+---+
*  | BL | BR |
*  |   |   |
*  +---+---+
*      BACK
*

```

Velocity Convention

- Positive: Forward motion
- Negative: Reverse motion
- Range: -2.0 to +2.0 m/s (robot max speed)

See also

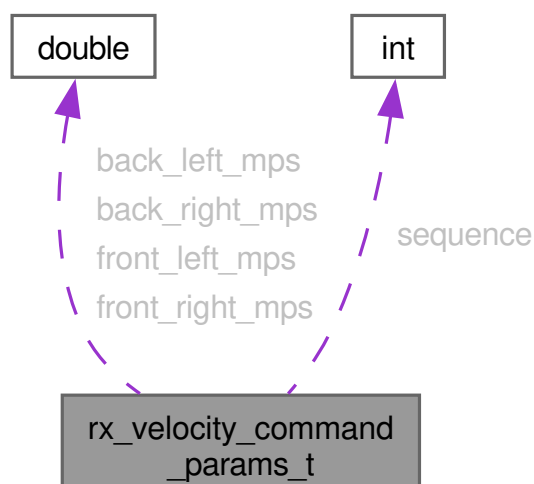
[rx_nanopb_create_velocity_command\(\)](#) Create command from params

Since

Version 1.0.0

Definition at line 761 of file [rx_nanopb.h](#).

Collaboration diagram for rx_velocity_command_params_t:



Data Fields

double	back_left_mps	Back left motor velocity (m/s), [-2.0, +2.0]
double	back_right_mps	Back right motor velocity (m/s), [-2.0, +2.0]
double	front_left_mps	Front left motor velocity (m/s), [-2.0, +2.0]
double	front_right_mps	Front right motor velocity (m/s), [-2.0, +2.0]
uint32_t	sequence	Command sequence number (monotonically increasing)

6.49.2.2 struct rx_velocity_diff_drive_params_t

VelocityCommand parameters for differential drive mode.

Simplified parameters for differential drive robots where left and right sides are controlled together. Front/back motors on same side receive identical velocities.

Differential Drive Mapping

- left_mps -> front_left_mps = back_left_mps
- right_mps -> front_right_mps = back_right_mps

Motion Examples

left_mps	right_mps	Motion
+1.0	+1.0	Forward
-1.0	-1.0	Reverse
+1.0	-1.0	Rotate CW
-1.0	+1.0	Rotate CCW
+1.0	+0.5	Turn right

See also

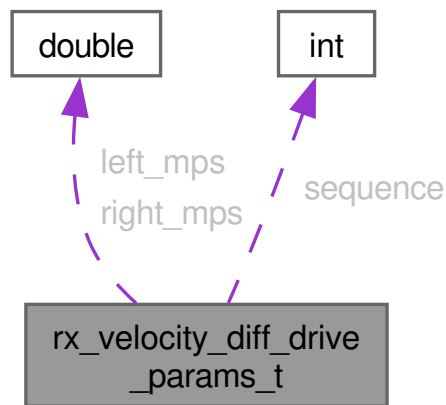
[rx_nanopb_create_velocity_command_diff_drive\(\)](#) Create diff-drive command

Since

Version 1.0.0

Definition at line 794 of file [rx_nanopb.h](#).

Collaboration diagram for rx_velocity_diff_drive_params_t:



Data Fields

double	left_mps	Left side velocity (m/s), applied to FL and BL
double	right_mps	Right side velocity (m/s), applied to FR and BR
uint32_t	sequence	Command sequence number

6.49.3 Function Documentation

6.49.3.1 rx_nanopb_create_response_header()

```

void rx_nanopb_create_response_header (
    star_v1_ResponseHeader * header,
    const star_v1_Status status,
    const char * request_id)

```

Create ResponseHeader with status.

Populates a ResponseHeader structure for response messages. Sets status and optionally echoes the request ID from the original request.

Parameters

out	header	Output header structure to populate
in	status	Response status code - star_v1_Status_STATUS_OK: Success - star_v1_Status_STATUS_INVALID_ARGUMENT: Bad request - star_v1_Status_STATUS_INTERNAL_ERROR: Processing error
in	request_id	Original request ID to echo (can be NULL) - If NULL, request_id field left empty - If provided, copied to header.request_id

Precondition

header must not be nullptr

Postcondition

header populated with status and request_id (if provided)

Note

Thread-safe (no shared state)

request_id is copied, not referenced

Example:

```
star_v1_SetVelocityResponse response;  
rx_nanopb_create_response_header(&response.header,  
                                star_v1_Status_STATUS_OK,  
                                request.header.request_id);
```

Since

Version 1.0.0

Create ResponseHeader with status.

Populates a ResponseHeader structure for response messages. Sets the status code and optionally echoes the request ID from the original request for correlation purposes.

Algorithm Steps

1. Validate header pointer (early return if nullptr)
2. Validate request_id length if provided (early return if too long)
3. Initialize header to zero
4. Set status field
5. If request_id provided, set up nanopb callback for encoding

Callback Setup

When request_id is provided, this function sets up a nanopb callback function (internal_encode_string_callback) that will be invoked during [pb_encode\(\)](#) to serialize the string field.

Precondition

header must not be nullptr (silently returns if nullptr)

request_id, if provided, must remain valid until encoding completes

Postcondition

header->status == status

If request_id != nullptr: callback configured for encoding

Note

Thread-safe (no shared state)

No return value - silently ignores nullptr header

Warning

request_id pointer must remain valid until [pb_encode\(\)](#) is called. Do not pass stack-allocated strings that may go out of scope.

Performance

- Execution time: ~ 1 us @ 240 MHz
- Stack usage: ~ 8 bytes

Example - Success Response:

```
star_v1_SetVelocityResponse response;

// After successfully processing velocity command
rx_nanopb_create_response_header(&response.header,
                                star_v1_Status_STATUS_OK,
                                request.header.request_id);

// Encode and send
rx_nanopb_encode_velocity_response(&response, buffer, sizeof(buffer), &len);
```

Example - Error Response:

```
// After detecting invalid velocity value
rx_nanopb_create_response_header(&response.header,
                                star_v1_Status_STATUS_INVALID_ARGUMENT,
                                request.header.request_id);
```

See also

[star_v1_Status](#) Status enumeration values

[internal_encode_string_callback\(\)](#) nanopb callback for string encoding

Since

Version 1.0.0

Definition at line 1542 of file [rx_nanopb.c](#).

```
01545 {
01546     if (header == nullptr) {
01547         return;
01548     }
01549     if (request_id != nullptr) {
01550         size_t req_id_len = 0;
01551         while (request_id[req_id_len] != '\0') {
01552             req_id_len++;
01553         }
01554         if (req_id_len > s_nanopb_buffer_size) {
01555             return;
01556         }
01557     }
01558 }
01559
01560 *header = (star_v1_ResponseHeader)star_v1_ResponseHeader_init_zero;
01561 header->status = status;
```

```

01562
01563  /* Copy request_id string if provided (fixed-size field in generated code) */
01564  if (request_id != nullptr) {
01565      const size_t max_len = sizeof(header->request_id) - 1U;
01566      size_t      idx      = 0;
01567      while (idx < max_len && request_id[idx] != '\0') {
01568          header->request_id[idx] = request_id[idx];
01569          idx++;
01570      }
01571      header->request_id[idx] = '\0';
01572  }
01573  }

```

References [star_v1_ResponseHeader::request_id](#), [s_nanopb_buffer_size](#), [star_v1_ResponseHeader_init_zero](#), and [star_v1_ResponseHeader::status](#).

6.49.3.2 rx_nanopb_create_velocity_command()

```

rx_err_t rx_nanopb_create_velocity_command (
    star_v1_VelocityCommand * cmd,
    const rx_velocity_command_params_t * params) [nodiscard]

```

Create VelocityCommand for 4 independent motors.

Populates a VelocityCommand structure from parameters. Use for mecanum or omnidirectional robots where each motor can have independent velocity.

Parameters

out	cmd	Output command structure to populate - All velocity fields set from params - sequence field set from params
in	params	Velocity command parameters - All velocities in m/s - sequence should be monotonically increasing

Returns

rx_err_t Error code indicating result

Return values

k_rx_ok	Success, cmd populated
k_rx_err_null_ptr	nullptr in cmd or params

Precondition

cmd and params must not be nullptr

Postcondition

cmd fully populated with velocity values

Note

Thread-safe (no shared state)

Example:

```
rx_velocity_command_params_t params = {
    .front_left_mps = 1.0,
    .front_right_mps = 1.0,
    .back_left_mps = 1.0,
    .back_right_mps = 1.0,
    .sequence = g_command_sequence++,
};
star_v1_VelocityCommand cmd;
rx_nanopb_create_velocity_command(&cmd, &params);
```

See also

[rx_velocity_command_params_t](#) Parameter structure

Since

Version 1.0.0

Create VelocityCommand for 4 independent motors.

Populates a VelocityCommand structure from parameters. Use for mecanum or omnidirectional drive robots where each motor requires independent velocity control. Performs range validation on all velocity values.

Algorithm Steps

1. Validate cmd pointer (not nullptr)
2. Validate params pointer (not nullptr)
3. Validate all velocities in range [k_velocity_mps_min, k_velocity_mps_max]
4. Initialize cmd to zero (clear previous state)
5. Copy velocity values from params to cmd
6. Copy sequence number
7. Set timestamp to 0 (caller sets if needed)

Precondition

cmd must not be nullptr

params must not be nullptr

All velocity values must be in valid range

Postcondition

cmd fully populated with velocity values from params
 cmd->timestamp_us == 0

Note

Thread-safe (no shared state)

Performance

- Execution time: ~2 us @ 240 MHz
- Stack usage: ~16 bytes

Example - Basic Usage:

```
rx_velocity_command_params_t params = {
    .front_left_mps = 1.0,
    .front_right_mps = 1.0,
    .back_left_mps = 1.0,
    .back_right_mps = 1.0,
    .sequence = g_command_sequence++,
};

star_v1_VelocityCommand cmd;
rx_err_t err = rx_nanopb_create_velocity_command(&cmd, &params);
if (err == k_rx_ok) {
    // Use cmd in SetVelocityRequest
    request.command = cmd;
}
```

See also

[rx_velocity_command_params_t](#) Parameter structure

[rx_nanopb_create_velocity_command_diff_drive\(\)](#) For differential drive

Since

Version 1.0.0

Definition at line 1362 of file rx_nanopb.c.

```
01364 {
01365     /* Use bitwise | to evaluate both sides without short-circuit branches */
01366     const bool cmd_null = (bool)(cmd == nullptr);
01367     const bool params_null = (bool)(params == nullptr);
01368     if ((bool)((int)cmd_null | (int)params_null)) {
01369         return k_rx_err_invalid_arg;
01370     }
01371
01372     /* Use bitwise | to evaluate all velocity bounds without short-circuit branches */
01373     const bool fl_lo = (bool)(params->front_left_mps < k_velocity_mps_min);
01374     const bool fl_hi = (bool)(params->front_left_mps > k_velocity_mps_max);
01375     const bool fr_lo = (bool)(params->front_right_mps < k_velocity_mps_min);
01376     const bool fr_hi = (bool)(params->front_right_mps > k_velocity_mps_max);
01377     const bool bl_lo = (bool)(params->back_left_mps < k_velocity_mps_min);
01378     const bool bl_hi = (bool)(params->back_left_mps > k_velocity_mps_max);
01379     const bool br_lo = (bool)(params->back_right_mps < k_velocity_mps_min);
01380     const bool br_hi = (bool)(params->back_right_mps > k_velocity_mps_max);
01381     if ((bool)((int)fl_lo | (int)fl_hi | (int)fr_lo | (int)fr_hi | (int)bl_lo | (int)bl_hi |
01382             (int)br_lo | (int)br_hi)) {
01383         return k_rx_err_invalid_arg;
01384     }
01385
01386     *cmd = (star_v1_VelocityCommand)star_v1_VelocityCommand_init_zero;
01387     cmd->front_left_velocity_mps = params->front_left_mps;
```

```

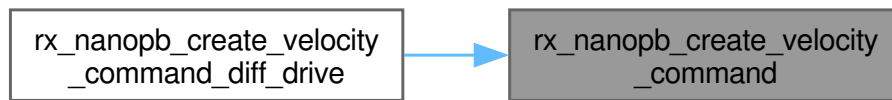
01388 cmd->front_right_velocity_mps = params->front_right_mps;
01389 cmd->back_left_velocity_mps   = params->back_left_mps;
01390 cmd->back_right_velocity_mps  = params->back_right_mps;
01391 cmd->sequence                 = params->sequence;
01392 cmd->timestamp_us             = 0; /* Set by caller if needed */
01393 return k_rx_ok;
01394 }

```

References [rx_velocity_command_params_t::back_left_mps](#), [star_v1_VelocityCommand::back_left_velocity_mps](#), [rx_velocity_command_params_t::back_right_mps](#), [star_v1_VelocityCommand::back_right_velocity_mps](#), [rx_velocity_command_params_t::front_left_mps](#), [star_v1_VelocityCommand::front_left_velocity_mps](#), [rx_velocity_command_params_t::front_right_mps](#), [star_v1_VelocityCommand::front_right_velocity_mps](#), [k_velocity_mps_max](#), [k_velocity_mps_min](#), [rx_velocity_command_params_t::sequence](#), [star_v1_VelocityCommand](#), [star_v1_VelocityCommand_init_zero](#), and [star_v1_VelocityCommand::timestamp_us](#).

Referenced by [rx_nanopb_create_velocity_command_diff_drive\(\)](#).

Here is the caller graph for this function:



6.49.3.3 rx_nanopb_create_velocity_command_diff_drive()

```

rx_err_t rx_nanopb_create_velocity_command_diff_drive (
    star_v1_VelocityCommand * cmd,
    const rx_velocity_diff_drive_params_t * params)

```

Create VelocityCommand in differential drive mode.

Populates a VelocityCommand for differential drive robots. Left and right parameters are applied to both front and back motors on each side.

Velocity Mapping

- `params->left_mps -> cmd->front_left_mps AND cmd->back_left_mps`
- `params->right_mps -> cmd->front_right_mps AND cmd->back_right_mps`

Parameters

out	cmd	Output command structure to populate
in	params	Differential drive parameters • <code>left_mps</code>: Both left motors • <code>right_mps</code>: Both right motors

Returns

`rx_err_t` Error code indicating result

Return values

<code>k_rx_ok</code>	Success, cmd populated
<code>k_rx_err_null_ptr</code>	nullptr in cmd or params

Precondition

cmd and params must not be nullptr

Postcondition

cmd populated with left/right velocities duplicated

Note

Thread-safe (no shared state)

Example:

```
// Turn left: right side faster than left
rx_velocity_diff_drive_params_t params = {
    .left_mps = 0.5,
    .right_mps = 1.0,
    .sequence = g_command_sequence++,
};
star_v1_VelocityCommand cmd;
rx_nanopb_create_velocity_command_diff_drive(&cmd, &params);
// Result: FL=BL=0.5, FR=BR=1.0
```

See also

[rx_velocity_diff_drive_params_t](#) Parameter structure

Since

Version 1.0.0

Populates a VelocityCommand for differential drive robots where left and right sides are controlled together. Front and back motors on the same side receive identical velocity values.

Velocity Mapping

```
params->left_mps -> cmd->front_left_velocity_mps
               -> cmd->back_left_velocity_mps

params->right_mps -> cmd->front_right_velocity_mps
                -> cmd->back_right_velocity_mps
```


Motion Examples

left_mps	right_mps	Robot Motion
+1.0	+1.0	Forward
-1.0	-1.0	Reverse
+1.0	-1.0	Rotate clockwise
-1.0	+1.0	Rotate counter-clockwise
+1.0	+0.5	Curve right (forward)
+0.5	+1.0	Curve left (forward)

Precondition

cmd must not be nullptr

params must not be nullptr

Velocities must be in valid range [-1000.0, +1000.0] m/s

Postcondition

cmd->front_left_velocity_mps == cmd->back_left_velocity_mps

cmd->front_right_velocity_mps == cmd->back_right_velocity_mps

Note

Thread-safe (no shared state)

Internally calls [rx_nanopb_create_velocity_command\(\)](#)

Performance

- Execution time: ~3 us @ 240 MHz
- Stack usage: ~48 bytes

Example - Turn Left:

```
// Turn left by making right side faster than left
rx_velocity_diff_drive_params_t params = {
    .left_mps = 0.5, // Slower
    .right_mps = 1.0, // Faster
    .sequence = g_sequence++,
};

star_v1_VelocityCommand cmd;
rx_err_t err = rx_nanopb_create_velocity_command_diff_drive(&cmd, &params);
// Result: FL=BL=0.5, FR=BR=1.0 (robot curves left)
```

See also

[rx_velocity_diff_drive_params_t](#) Parameter structure

[rx_nanopb_create_velocity_command\(\)](#) 4-motor independent control

Since

Version 1.0.0

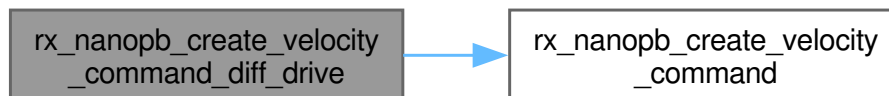
Definition at line 1458 of file [rx_nanopb.c](#).

```

01460 {
01461   rx_velocity_command_params_t params_t;
01462   if (cmd == nullptr || params == nullptr) {
01463     return k_rx_err_invalid_arg;
01464   }
01465 }
01466
01467 /* Differential drive mode: Lock left 2 motors together, right 2 motors together
01468  * Front Left & Back Left = Left side
01469  * Front Right & Back Right = Right side */
01470 params_t.front_left_mps = params->left_mps;
01471 params_t.front_right_mps = params->right_mps;
01472 params_t.back_left_mps = params->left_mps;
01473 params_t.back_right_mps = params->right_mps;
01474 params_t.sequence = params->sequence;
01475
01476 return rx_nanopb_create_velocity_command(cmd, &params_t);
01477 }
```

References [rx_velocity_command_params_t::back_left_mps](#), [rx_velocity_command_params_t::back_right_mps](#), [rx_velocity_command_params_t::front_left_mps](#), [rx_velocity_command_params_t::front_right_mps](#), [rx_velocity_diff_drive_params_t::left_mps](#), [rx_velocity_diff_drive_params_t::right_mps](#), [rx_nanopb_create_velocity_command](#), [rx_velocity_command_params_t::sequence](#), and [rx_velocity_diff_drive_params_t::sequence](#).

Here is the call graph for this function:



6.49.3.4 rx_nanopb_decode_estop_request()

```

rx_err_t rx_nanopb_decode_estop_request (
    const uint8_t * buffer,
    const uint32_t len,
    star_v1_EmergencyStopRequest * msg) [nodiscard]
```

Decode EmergencyStopRequest message from Protocol Buffer bytes.

Deserializes an emergency stop command from RPi5. This is a safety-critical message that must be processed with highest priority.

E-Stop Processing

1. Decode message (this function)
2. Immediately stop all motors
3. Enter safe state
4. Send EmergencyStopResponse

Parameters

in	buffer	Input buffer containing encoded message
in	len	Buffer length in bytes
out	msg	Decoded message structure

Returns

`rx_err_t` Error code indicating result

Return values

<code>k_rx_ok</code>	Success, msg contains decoded values
<code>k_rx_err_invalid_arg</code>	nullptr or len == 0
<code>k_rx_err_not_initialized</code>	Module not initialized
<code>k_rx_err_protocol_error</code>	Malformed message

Warning

E-stop must be processed immediately regardless of return value

Note

Consider stopping motors first, then decode for response

Performance: ~10 us @ 240 MHz (minimal message)

See also

[rx_nanopb_encode_estop_response\(\)](#) Send acknowledgment

Since

Version 1.0.0

Decode EmergencyStopRequest message from Protocol Buffer bytes.

Deserializes an emergency stop command received from RPi5. This is a safety-critical message that triggers immediate motor shutdown.

Safety-Critical Processing

Warning

E-Stop commands must be processed immediately. Consider stopping motors BEFORE attempting to decode if message type is identified.

Algorithm Steps

1. Validate buffer pointer (not nullptr)
2. Validate msg pointer (not nullptr)
3. Validate len (not zero, not too large)
4. Verify module is initialized
5. Initialize msg to zero
6. Create nanopb input stream from buffer
7. Decode message using generated field descriptors

Precondition

`rx_nanopb_init()` must have been called
buffer contains valid Protocol Buffer data

Postcondition

msg fully populated with E-Stop request fields

Note

Not thread-safe

Warning

Always stop motors regardless of decode result

Performance

- Execution time: ~10 us @ 240 MHz
- Stack usage: ~32 bytes

Example - E-Stop Handling:

```
void handle_estop_message(const uint8_t* buffer, uint32_t len) {
    // FIRST: Stop motors immediately (safety-critical)
    motor_emergency_stop_all();

    // THEN: Decode message for response
    star_v1_EmergencyStopRequest estop_req;
    rx_err_t err = rx_nanopb_decode_estop_request(buffer, len, &estop_req);

    // Send response (motors already stopped)
    star_v1_EmergencyStopResponse response = {};
    rx_nanopb_create_response_header(&response.header,
                                    star_v1_Status_STATUS_OK,
                                    estop_req.header.request_id);
    // ... encode and send response
}
```

See also

[rx_nanopb_encode_estop_response\(\)](#) Send acknowledgment

Since

Version 1.0.0

Definition at line 863 of file [rx_nanopb.c](#).

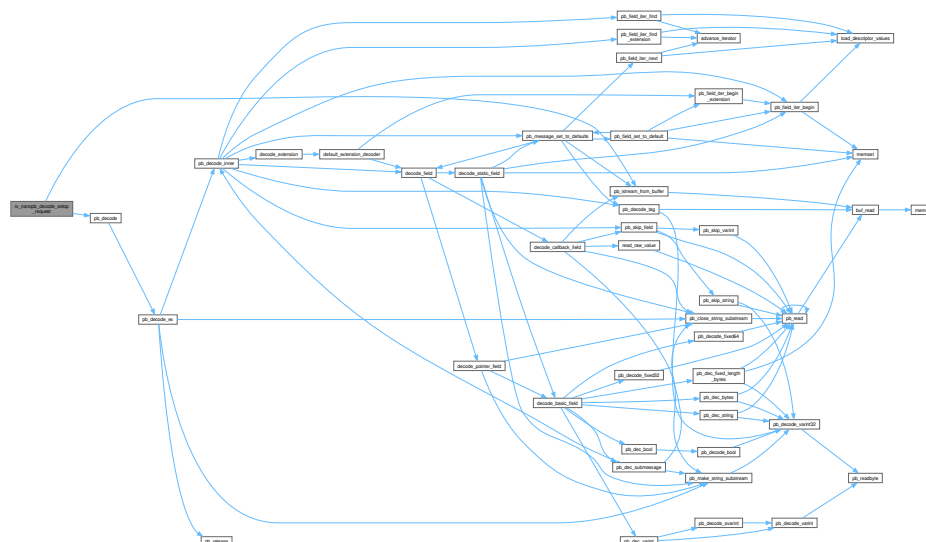
```

00866 {
00867     /* Pre-condition 1: nullptr pointer checks */
00868     if (buffer == nullptr || msg == nullptr) {
00869         return k_rx_err_invalid_arg;
00870     }
00871
00872     /* Pre-condition 2: Length validation */
00873     if (len == 0 || len > s_nanopb_buffer_size) {
00874         return k_rx_err_invalid_arg;
00875     }
00876
00877     /* Pre-condition 3: Module initialized */
00878     if (!s_initialized) {
00879         return k_rx_err_not_initialized;
00880     }
00881
00882     /* Initialize message to default values */
00883     *msg = (star_v1_EmergencyStopRequest)star_v1_EmergencyStopRequest_init_zero;
00884
00885     pb_istream_t stream = pb_istream_from_buffer(buffer, len);
00886
00887     if (!pb_decode(&stream, star_v1_EmergencyStopRequest_fields, msg)) {
00888         return k_rx_err_protocol_error;
00889     }
00890
00891     return k_rx_ok;
00892 }

```

References [pb_decode\(\)](#), [pb_istream_from_buffer\(\)](#), [s_initialized](#), [s_nanopb_buffer_size](#), [star_v1_EmergencyStopRequest_init_zero](#) and [star_v1_EmergencyStopRequest_init_zero](#).

Here is the call graph for this function:



6.49.3.5 rx_nanopb_decode_pid_gains_request()

```
rx_err_t rx_nanopb_decode_pid_gains_request (
    const uint8_t * buffer,
    const uint32_t len,
    star_v1_SetPidGainsRequest * msg) [nodiscard]
```

Decode SetPidGainsRequest message from Protocol Buffer bytes.

Deserializes a PID gains update command from RPi5 over SPI. This message contains new PID controller gains (Kp, Ki, Kd) and output/integral limits to be applied to one or more motors.

PID Configuration Structure

The decoded message contains:

- Kp (proportional gain)
- Ki (integral gain)
- Kd (derivative gain)
- output_min_percent (minimum PWM duty cycle %)
- output_max_percent (maximum PWM duty cycle %)
- integral_min (anti-windup minimum)
- integral_max (anti-windup maximum)
- motor_id (0-3 for specific motor, -1 for all motors)

Decoding Process

1. Validate buffer pointer and length
2. Create nanopb input stream from buffer
3. Decode header and pid_config fields
4. Extract motor_id for targeting

Parameters

in	buffer	Input buffer containing encoded message • Wire-format Protocol Buffer data • Received from SPI or USB
in	len	Buffer length in bytes • Must match actual encoded message size
out	msg	Decoded message structure • Will be fully populated on success • PID gains in standard units

Returns

`rx_err_t` Error code indicating result

Return values

<code>k_rx_ok</code>	Success, msg contains decoded values
<code>k_rx_err_invalid_arg</code>	nullptr or len == 0
<code>k_rx_err_not_initialized</code>	Module not initialized
<code>k_rx_err_protocol_error</code>	Malformed message or invalid field

Precondition

`rx_nanopb_init()` must be called first
buffer contains valid wire-format data

Postcondition

msg fully populated with decoded PID configuration

Note

Not thread-safe (uses shared internal state)

Performance: ~18 us @ 240 MHz

Example:

```
star_v1_SetPidGainsRequest request;
rx_err_t err = rx_nanopb_decode_pid_gains_request(spi_buffer, spi_len, &request);
if (err == k_rx_ok && request.has_pid_config) {
    // Apply gains to motors
    pid_gains_t gains = {
        .kp = (float)request.pid_config.kp,
        .ki = (float)request.pid_config.ki,
        .kd = (float)request.pid_config.kd,
        .output_min = (float)request.pid_config.output_min_percent,
        .output_max = (float)request.pid_config.output_max_percent,
        .integral_min = (float)request.pid_config.integral_min,
        .integral_max = (float)request.pid_config.integral_max,
        .update_pending = true
    };
    shared_data_set_pid_gains(&gains);
} else if (err == k_rx_err_protocol_error) {
    // Request retransmit
    send_nack();
}
```

See also

`rx_nanopb_encode_pid_gains_response()` Send acknowledgment
`shared_data_set_pid_gains()` Apply gains to motor controllers

Since

Version 1.0.0

Decode SetPidGainsRequest message from Protocol Buffer bytes.

Deserializes a PID gains update command from RPi5. This message contains new PID controller configuration (gains and limits) to be applied to one or more motors.

Message Processing

1. Validate input parameters (buffer, length, message pointer)
2. Check module initialization state
3. Initialize message structure to zero
4. Create nanopb input stream from buffer
5. Decode using `star_v1_SetPidGainsRequest_fields` descriptor
6. Return success or protocol error

PID Configuration Fields

The decoded message contains:

- header: Standard RequestHeader with `request_id`
- pid_config: PidConfig structure with:
 - kp: Proportional gain
 - ki: Integral gain
 - kd: Derivative gain
 - output_min_percent: Minimum PWM duty cycle (%)
 - output_max_percent: Maximum PWM duty cycle (%)
 - integral_min: Anti-windup lower bound
 - integral_max: Anti-windup upper bound
- motor_id: Target motor (0-3 for specific motor, -1 for all motors)

Precondition

`rx_nanopb_init()` must be called first
buffer must point to valid wire-format data
len must match actual encoded message size

Postcondition

msg fully populated with decoded PID gains
`msg->has_pid_config == true` if gains present

Invariant

msg initialized to zero before decode (prevents stale data)

Note

Thread Safety: Not thread-safe (uses shared nanopb state)
 Re-entrancy: NOT reentrant. Single-threaded decode only.
 Performance: ~18 us @ 240 MHz (lightweight message)

Warning

Gain Validation: This function does NOT validate gain ranges. Caller must validate that Kp, Ki, Kd are non-negative and within safe operating bounds before applying to PID controllers.
 Motor ID: motor_id == -1 means apply to ALL motors. Caller must handle this case explicitly.

Example - Single Motor Update:

```
star_v1_SetPidGainsRequest request;
rx_err_t err = rx_nanopb_decode_pid_gains_request(spi_buffer, spi_len, &request);

if (err == k_rx_ok && request.has_pid_config) {
    // Convert protobuf gains to firmware structure
    pid_gains_t gains = {
        .kp = (float)request.pid_config.kp,
        .ki = (float)request.pid_config.ki,
        .kd = (float)request.pid_config.kd,
        .output_min = (float)request.pid_config.output_min_percent,
        .output_max = (float)request.pid_config.output_max_percent,
        .integral_min = (float)request.pid_config.integral_min,
        .integral_max = (float)request.pid_config.integral_max,
        .update_pending = true
    };

    // Apply to target motor(s)
    if (request.motor_id == -1) {
        // Apply to all motors
        shared_data_set_pid_gains(&gains);
    } else if (request.motor_id >= 0 && request.motor_id < 4) {
        // Apply to specific motor (motor_id 0-3)
        // Note: Current firmware applies to all motors via shared_data
        shared_data_set_pid_gains(&gains);
    }
} else if (err == k_rx_err_protocol_error) {
    // Malformed message - request retransmit
    rx_log_error("NANOPB", "Failed to decode PID gains request");
    comm_send_nack();
}
```

Example - All Motors Update:

```
// RPi5 sends motor_id = -1 to update all 4 motors
star_v1_SetPidGainsRequest request;
rx_err_t err = rx_nanopb_decode_pid_gains_request(buffer, len, &request);

if (err == k_rx_ok && request.motor_id == -1) {
    rx_log_info("COMM", "Applying PID gains to ALL motors");
    // shared_data_set_pid_gains() applies to all 4 PID controllers
    shared_data_set_pid_gains(&converted_gains);
}
```

See also

[rx_nanopb_encode_pid_gains_response\(\)](#) Send acknowledgment
[shared_data_set_pid_gains\(\)](#) Apply gains to motor controllers
[internal_apply_pid_updates\(\)](#) Motor task applies pending gains
[star_v1_SetPidGainsRequest_fields](#) nanopb field descriptors

- Rule 5: [PASS] 3 preconditions, 2 postconditions documented
- Rule 7: [PASS] `pb_decode()` return value checked

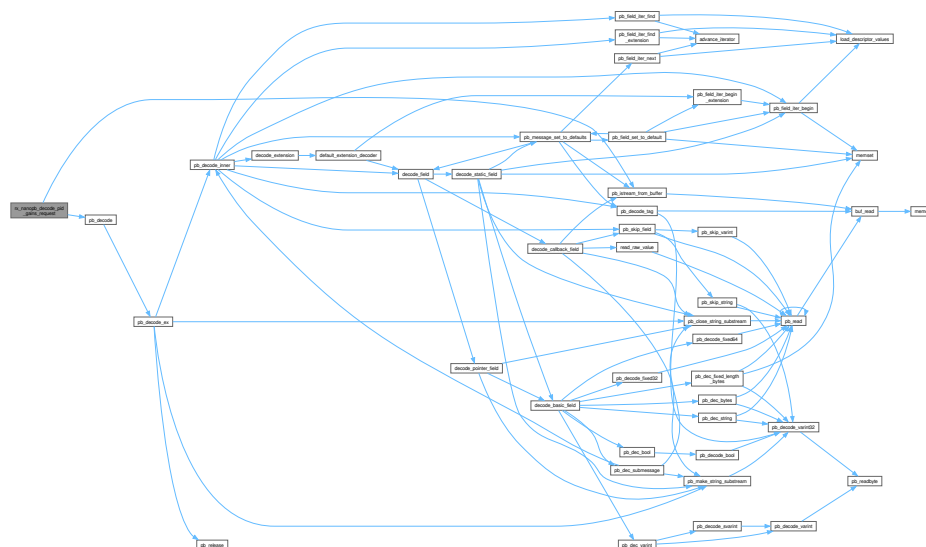
Version 1.0.0

```

01067 {
01068     /* Pre-condition 1: nullptr pointer checks */
01069     if (buffer == nullptr || msg == nullptr) {
01070         return k_rx_err_invalid_arg;
01071     }
01072
01073     /* Pre-condition 2: Length validation */
01074     if (len == 0 || len > s_nanopb_buffer_size) {
01075         return k_rx_err_invalid_arg;
01076     }
01077
01078     /* Pre-condition 3: Module initialized */
01079     if (!s_initialized) {
01080         return k_rx_err_not_initialized;
01081     }
01082
01083     /* Initialize message to default values (NASA Rule 5 - prevent stale data) */
01084     *msg = (star_v1_SetPidGainsRequest)star_v1_SetPidGainsRequest_init_zero;
01085
01086     pb_istream_t stream = pb_istream_from_buffer(buffer, len);
01087
01088     if (!pb_decode(&stream, star_v1_SetPidGainsRequest_fields, msg)) {
01089         return k_rx_err_protocol_error;
01090     }
01091
01092     return k_rx_ok;
01093 }

```

Here is the call graph for this function:



6.49.3.6 rx_nanopb_decode_retransmit_config_request()

```
rx_err_t rx_nanopb_decode_retransmit_config_request (
    const uint8_t * buffer,
    uint32_t len,
    star_v1_SetRetransmitConfigRequest * msg) [nodiscard]
```

Decode SetRetransmitConfigRequest from Protocol Buffer bytes.

Decodes a serialized SetRetransmitConfigRequest message from a byte buffer. Used to process RPi5 commands that configure SPI retransmission parameters.

Parameters

in	buffer	Raw protobuf bytes to decode - Valid range: Non-nullptr to protobuf-encoded data - Max size: s_nanopb_buffer_size (1024 bytes)
in	len	Length of buffer in bytes Valid range: [1, s_nanopb_buffer_size]
out	msg	Decoded message output - Valid range: Non-nullptr to star_v1_SetRetransmitConfigRequest - Output: Zero-initialized then populated from buffer

Returns

rx_err_t Error code

Return values

k_rx_ok	Decode successful
k_rx_err_invalid_arg	nullptr or invalid length
k_rx_err_not_initialized	Module not initialized
k_rx_err_protocol_error	Protobuf decode failed

Precondition

buffer and msg must be non-NULL
[rx_nanopb_init\(\)](#) must have been called

Postcondition

msg populated with decoded fields on success

6.49.3.7 rx_nanopb_decode_velocity_request()

```
rx_err_t rx_nanopb_decode_velocity_request (
    const uint8_t * buffer,
    const uint32_t len,
    star_v1_SetVelocityRequest * msg) [nodiscard]
```

Decode SetVelocityRequest message from Protocol Buffer bytes.

Deserializes a SetVelocityRequest message received from RPi5 over SPI. This is the primary command message for motor velocity control.

Decoding Process

1. Validate buffer pointer and length
2. Create nanopb input stream from buffer
3. Decode header and command fields
4. Validate decoded values are within range

Parameters

in	buffer	Input buffer containing encoded message • Wire-format Protocol Buffer data • Received from SPI or USB
in	len	Buffer length in bytes • Must match actual encoded message size
out	msg	Decoded message structure • Will be fully populated on success • Velocities in m/s

Returns

rx_err_t Error code indicating result

Return values

k_rx_ok	Success, msg contains decoded values
k_rx_err_invalid_arg	nullptr or len == 0
k_rx_err_not_initialized	Module not initialized
k_rx_err_protocol_error	Malformed message, CRC error, or invalid field

Precondition

`rx_nanopb_init()` must be called first
buffer contains valid wire-format data

Postcondition

msg fully populated with decoded values

Note

Not thread-safe (uses shared internal state)

Performance: ~15 us @ 240 MHz

Example:

```
star_v1_SetVelocityRequest request;
rx_err_t err = rx_nanopb_decode_velocity_request(spi_buffer, spi_len, &request);
if (err == k_rx_ok) {
    // Apply velocities to motors
    motor_set_velocity(0, request.command.front_left_mps);
    motor_set_velocity(1, request.command.front_right_mps);
    // ...
} else if (err == k_rx_err_protocol_error) {
    // Request retransmit
    send_nack();
}
```

See also

[rx_nanopb_encode_velocity_request\(\)](#) Encode counterpart

Since

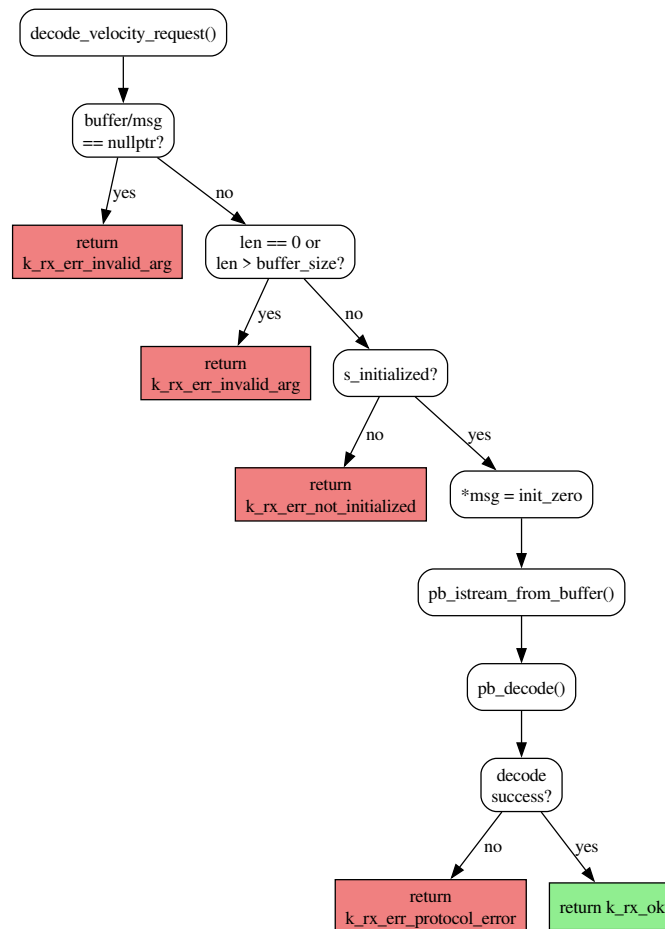
Version 1.0.0

Decode SetVelocityRequest message from Protocol Buffer bytes.

Deserializes a SetVelocityRequest message received from RPi5 over SPI. This is the primary command message for motor velocity control in the STAR system.

Algorithm Steps

1. Validate buffer pointer (not nullptr)
2. Validate msg pointer (not nullptr)
3. Validate len (not zero, not too large)
4. Verify module is initialized
5. Initialize msg to zero (clear previous state)
6. Create nanopb input stream from buffer
7. Decode message using generated field descriptors
8. Return decoded message in *msg



Precondition

- `rx_nanopb_init()` must have been called successfully
- buffer must contain valid Protocol Buffer wire-format data
- len must match actual encoded message size

Postcondition

- msg fully populated with decoded field values
- Unset fields have default/zero values

Note

- Not thread-safe - protect with mutex if called from multiple tasks

Performance

- Execution time: ~15 us @ 240 MHz
- Stack usage: ~96 bytes

Example - Basic Usage:

```
// Receive encoded message over SPI
uint8_t rx_buffer[512];
uint32_t rx_len;
spi_receive(rx_buffer, sizeof(rx_buffer), &rx_len);

// Decode the message
star_v1_SetVelocityRequest request;
rx_err_t err = rx_nanopb_decode_velocity_request(rx_buffer, rx_len, &request);

if (err == k_rx_ok) {
    // Apply velocities to motors
    motor_set_velocity(MOTOR_FL, request.command.front_left_velocity_mps);
    motor_set_velocity(MOTOR_FR, request.command.front_right_velocity_mps);
    motor_set_velocity(MOTOR_BL, request.command.back_left_velocity_mps);
    motor_set_velocity(MOTOR_BR, request.command.back_right_velocity_mps);
} else if (err == k_rx_err_protocol_error) {
    // Request retransmit
    comm_send_nack();
}
```

See also

[rx_nanopb_encode_velocity_request\(\)](#) Encode counterpart
[rx_nanopb_encode_velocity_response\(\)](#) Send response after decoding

NASA Power of 10 Compliance

- Rule 5: [OK] 3 pre-conditions
- Rule 7: [OK] [pb_decode\(\)](#) return value checked

Since

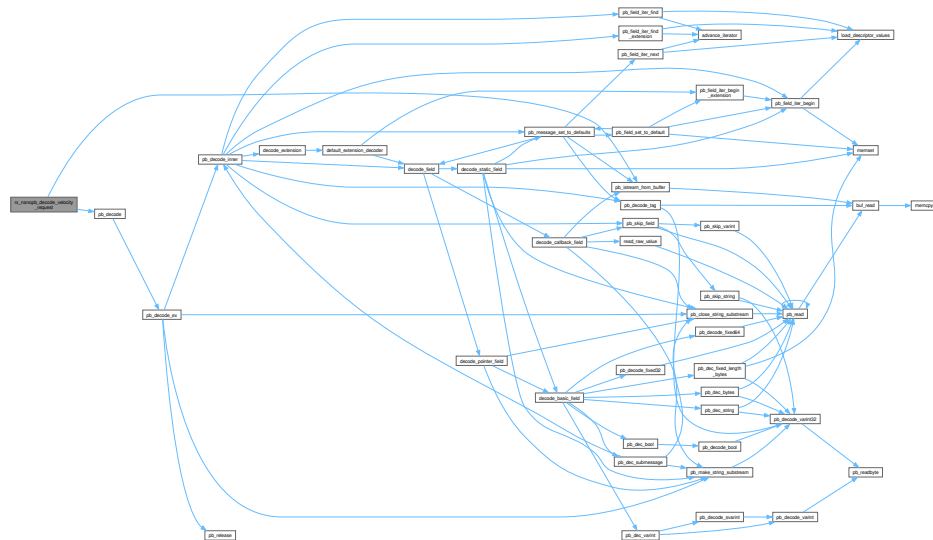
Version 1.0.0

Definition at line 674 of file [rx_nanopb.c](#).

```
00677 {
00678     /* Pre-condition 1: nullptr pointer checks */
00679     if (buffer == nullptr || msg == nullptr) {
00680         return k_rx_err_invalid_arg;
00681     }
00682
00683     /* Pre-condition 2: Length validation */
00684     if (len == 0 || len > s_nanopb_buffer_size) {
00685         return k_rx_err_invalid_arg;
00686     }
00687
00688     /* Pre-condition 3: Module initialized */
00689     if (!s_initialized) {
00690         return k_rx_err_not_initialized;
00691     }
00692
00693     /* Initialize message to default values */
00694     *msg = (star_v1_SetVelocityRequest)star_v1_SetVelocityRequest_init_zero;
00695
00696     pb_istream_t stream = pb_istream_from_buffer(buffer, len);
00697
00698     if (!pb_decode(&stream, star_v1_SetVelocityRequest_fields, msg)) {
00699         return k_rx_err_protocol_error;
00700     }
00701
00702     return k_rx_ok;
00703 }
```

References [pb_decode\(\)](#), [pb_istream_from_buffer\(\)](#), [s_initialized](#), [s_nanopb_buffer_size](#), [star_v1_SetVelocityRequest](#) and [star_v1_SetVelocityRequest_init_zero](#).

Here is the call graph for this function:



6.49.3.8 rx_nanopb_encode_estop_response()

```
rx_err_t rx_nanopb_encode_estop_response (
    const star_v1_EmergencyStopResponse * msg,
    uint8_t * buffer,
    const uint32_t buffer_size,
    uint32_t * len) [nodiscard]
```

Encode EmergencyStopResponse message to Protocol Buffer bytes.

Serializes an emergency stop response as acknowledgment. Confirms that the RX72N has processed the E-stop and entered safe state.

Parameters

in	msg	Message to encode - header.status: Indicates if E-stop was successful - Typically always returns success (motors stopped)
out	buffer	Output buffer for encoded bytes
in	buffer_size	Size of output buffer ($\geq$ s_nanopb_buffer_size)
out	len	Actual encoded message length

Returns

rx_err_t Error code indicating result

Return values

k_rx_ok	Success, buffer contains encoded response
k_rx_err_invalid_arg	nullptr in parameters

<code>k_rx_err_not_initialized</code>	Module not initialized
<code>k_rx_err_invalid_size</code>	Buffer too small

Performance: ~10 us @ 240 MHz

See also

[rx_nanopb_decode_estop_request\(\)](#) Decode incoming E-stop

Since

Version 1.0.0

Encode EmergencyStopResponse message to Protocol Buffer bytes.

Serializes an EmergencyStopResponse message to confirm E-Stop processing. This response informs RPi5 that the RX72N has received the E-Stop command and has entered safe state (motors stopped).

Precondition

[rx_nanopb_init\(\)](#) must have been called

Motors should already be stopped before sending response

Postcondition

buffer contains valid Protocol Buffer wire-format data

Note

Not thread-safe

Performance

- Execution time: ~10 us @ 240 MHz
- Stack usage: ~32 bytes

See also

[rx_nanopb_decode_estop_request\(\)](#) Decode incoming E-Stop first

Since

Version 1.0.0

Definition at line 919 of file [rx_nanopb.c](#).

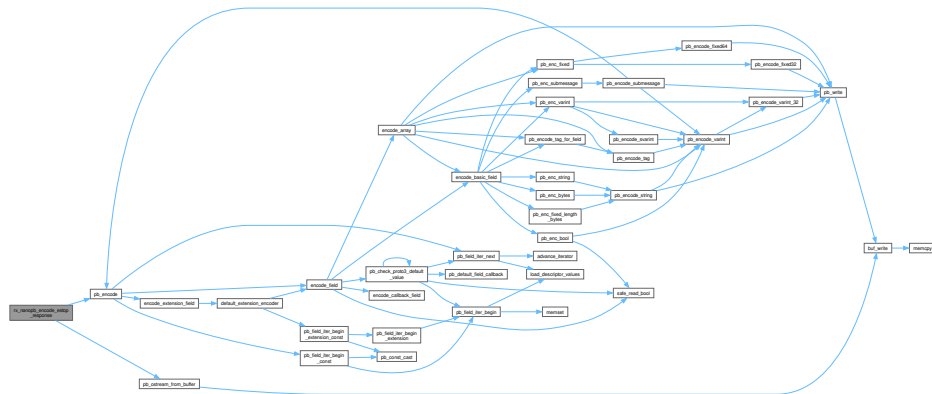
```

00923 {
00924     /* Pre-condition 1: nullptr pointer checks */
00925     if (msg == nullptr || buffer == nullptr || len == nullptr) {
00926         return k_rx_err_invalid_arg;
00927     }
00928
00929     /* Pre-condition 2: Module initialized */
00930     if (!s_initialized) {
00931         return k_rx_err_not_initialized;
00932     }
00933
00934     /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
00935     if (buffer_size < s_nanopb_buffer_size) {
00936         return k_rx_err_invalid_size;
00937     }
00938
00939     pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
00940
00941     /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
00942     (void)pb_encode(&stream, star_v1_EmergencyStopResponse_fields, msg);
00943     *len = stream.bytes_written;
00944
00945     return k_rx_ok;
00946 }

```

References [pb_encode\(\)](#), [pb_ostream_from_buffer\(\)](#), [s_initialized](#), [s_nanopb_buffer_size](#), and [star_v1_EmergencyStopResponse_fields](#).

Here is the call graph for this function:



6.49.3.9 rx_nanopb_encode_pid_gains_response()

```

rx_err_t rx_nanopb_encode_pid_gains_response (
    const star_v1_SetPidGainsResponse * msg,
    uint8_t * buffer,
    const uint32_t buffer_size,
    uint32_t * len) [nodiscard]

```

Encode SetPidGainsResponse message to Protocol Buffer bytes.

Serializes a SetPidGainsResponse message as acknowledgment to a PID gains update command. Sent back to RPi5 after processing a SetPidGainsRequest.

The response indicates whether the gains were successfully written to `shared_data` for the Motor Control Task to apply. The optional human-readable message field is left empty (NULL callback); the success boolean is sufficient for programmatic use.

Parameters

in	msg	Message to encode (must not be nullptr) • has_header: must be true for header to be encoded • header.status: k_star_v1_status_ok or error status • header.request_id: echoed from the original request • success: true if gains were written to shared_data
out	buffer	Output buffer for encoded bytes (must not be nullptr)
in	buffer_size	Size of output buffer in bytes • Must be $\geq$ s_nanopb_buffer_size (512 bytes)
out	len	Actual encoded message length in bytes (must not be nullptr)

Returns

rx_err_t Error code indicating result

Return values

k_rx_ok	Success, buffer contains wire-format response
k_rx_err_invalid_arg	nullptr in msg, buffer, or len
k_rx_err_not_initialized	Module not initialized via rx_nanopb_init()
k_rx_err_invalid_size	buffer_size too small or encoded length overflow

Precondition

[rx_nanopb_init\(\)](#) must be called before this function

buffer must point to at least buffer_size bytes of writable memory

Postcondition

On k_rx_ok: buffer[0..*len-1] contains valid wire-format response

On k_rx_ok: *len $\leq$ s_nanopb_buffer_size

Note

Not thread-safe; call only from Communication Task context

Performance: ~ 15 us @ 240 MHz

See also

[rx_nanopb_decode_pid_gains_request\(\)](#) Decode the incoming request
[rx_nanopb_create_response_header\(\)](#) Create header with status and request_id

Since

Version 1.0.0

Serializes a SetPidGainsResponse message as acknowledgment to a PID gains update command. Sent back to RPi5 after processing a SetPidGainsRequest.

The optional pb_callback_t message field is skipped when its encode callback is NULL (init_zero default). The success boolean and response header status are sufficient for programmatic acknowledgment.

Precondition

[rx_nanopb_init\(\)](#) must be called before this function
buffer must point to at least buffer_size bytes of writable memory

Postcondition

On k_rx_ok: buffer[0..*len-1] contains valid wire-format response
On k_rx_ok: *len <= s_nanopb_buffer_size

Note

Not thread-safe; call only from Communication Task context

Performance: ~15 us @ 240 MHz

See also

[rx_nanopb_decode_pid_gains_request\(\)](#) Decode the incoming request
[rx_nanopb_create_response_header\(\)](#) Create header with status and request_id

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: [PASS] 3 preconditions, 2 postconditions documented
- Rule 7: [PASS] [pb_encode\(\)](#) return value checked

Definition at line 1125 of file [rx_nanopb.c](#).

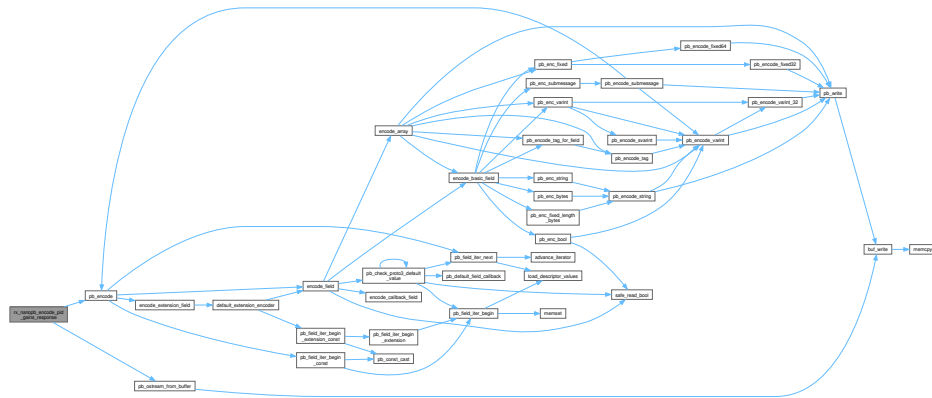
```

01129 {
01130     /* Pre-condition 1: nullptr pointer checks */
01131     if (msg == nullptr || buffer == nullptr || len == nullptr) {
01132         return k_rx_err_invalid_arg;
01133     }
01134
01135     /* Pre-condition 2: Module initialized */
01136     if (!s_initialized) {
01137         return k_rx_err_not_initialized;
01138     }
01139
01140     /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
01141     if (buffer_size < s_nanopb_buffer_size) {
01142         return k_rx_err_invalid_size;
01143     }
01144
01145     pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
01146
01147     /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
01148     (void)pb_encode(&stream, star_v1_SetPidGainsResponse_fields, msg);
01149     *len = stream.bytes_written;
01150
01151     return k_rx_ok;
01152 }

```

References [pb_encode\(\)](#), [pb_ostream_from_buffer\(\)](#), [s_initialized](#), [s_nanopb_buffer_size](#), and [star_v1_SetPidGainsResponse_fields](#).

Here is the call graph for this function:



6.49.3.10 rx_nanopb_encode_telemetry()

```

rx_err_t rx_nanopb_encode_telemetry (
    const star_v1_TelemetryData * msg,
    uint8_t * buffer,
    const uint32_t buffer_size,
    uint32_t * len) [nodiscard]

```

Encode TelemetryData message to Protocol Buffer bytes.

Serializes telemetry data for transmission to RPi5. Contains motor states, encoder readings, and sensor data. Sent periodically (typically 10-100 Hz).

Telemetry Contents

Field	Type	Units	Update Rate
Motor velocities	double x 4	m/s	100 Hz
Motor currents	double x 4	A	100 Hz
Encoder positions	int32 x 4	ticks	100 Hz
Temperature	double	degC	1 Hz

Parameters

in	msg	Message to encode (fully populated telemetry)
out	buffer	Output buffer for encoded bytes
in	buffer_size	Size of output buffer ($\geq$ s_nanopb_buffer_size)
out	len	Actual encoded message length (typically 150-200 bytes)

Returns

rx_err_t Error code indicating result

Return values

k_rx_ok	Success, buffer contains encoded telemetry
k_rx_err_invalid_arg	nullptr in parameters
k_rx_err_not_initialized	Module not initialized
k_rx_err_invalid_size	Buffer too small

Performance: ~ 30 us @ 240 MHz (full telemetry)

Example:

```

star_v1_TelemetryData telemetry = {};

// Populate motor data
telemetry.motor_front_left_velocity_mps = motor_get_velocity(0);
telemetry.motor_front_left_current_a = motor_get_current(0);
// ... (populate all fields)

uint8_t buffer[512];
uint32_t len;
rx_err_t err = rx_nanopb_encode_telemetry(&telemetry, buffer, sizeof(buffer), &len);
if (err == k_rx_ok) {
    spi_send(buffer, len);
}

```

Since

Version 1.0.0

Encode TelemetryData message to Protocol Buffer bytes.

Serializes telemetry data for periodic transmission to RPi5. Contains motor states, encoder readings, and sensor data. Typically called at 10-100 Hz depending on telemetry requirements.

Telemetry Contents

Field Category	Fields	Update Rate
Motor velocities	4 x double (m/s)	100 Hz
Motor currents	4 x double (A)	100 Hz
Encoder positions	4 x int32 (ticks)	100 Hz
Temperature	system temp (degC)	1 Hz

Precondition

[rx_nanopb_init\(\)](#) must have been called
 msg should contain current sensor/motor data

Postcondition

buffer contains valid Protocol Buffer wire-format telemetry

Note

Not thread-safe - protect with mutex if called from multiple tasks

Performance

- Execution time: ~30 us @ 240 MHz
- Stack usage: ~128 bytes

Example - Periodic Telemetry:

```
void telemetry_task(ULONG arg) {
    while (1) {
        // Gather telemetry data
        star_v1_TelemetryData telemetry = {};
        telemetry.motor_front_left_velocity_mps = motor_get_velocity(0);
        telemetry.motor_front_left_current_a = motor_get_current(0);
        // ... populate all fields
        telemetry.timestamp_us = rx_time_get_us();

        // Encode and send
        uint8_t buffer[512];
        uint32_t len;
        rx_err_t err = rx_nanopb_encode_telemetry(&telemetry, buffer, sizeof(buffer), &len);
        if (err == k_rx_ok) {
            spi_send(buffer, len);
        }

        // Sleep until next telemetry period (10ms = 100Hz)
        tx_thread_sleep(TX_TIMER_TICKS_PER_SECOND / 100);
    }
}
```

See also

[star_v1_TelemetryData](#) Generated telemetry message structure

NASA Power of 10 Compliance

- Rule 5: [OK] 3 pre-conditions, 1 post-condition

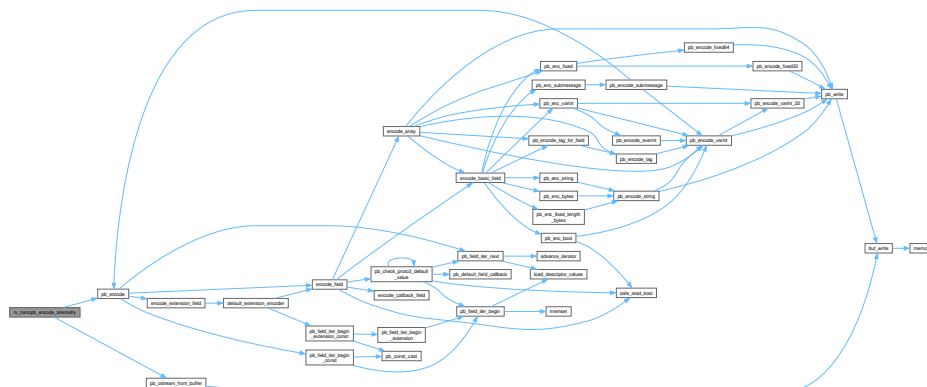
Version 1.0.0

```

01258 {
01259     /* Pre-condition 1: nullptr pointer checks */
01260     if (msg == nullptr || buffer == nullptr || len == nullptr) {
01261         return k_rx_err_invalid_arg;
01262     }
01263 }
01264
01265 /* Pre-condition 2: Module initialized */
01266 if (!s_initialized) {
01267     return k_rx_err_not_initialized;
01268 }
01269
01270 /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
01271 if (buffer_size < s_nanopb_buffer_size) {
01272     return k_rx_err_invalid_size;
01273 }
01274
01275 /* Wrap TelemetryData inside a WireMessage so the payload on the wire
01276  * matches the envelope the gateway dispatcher unmarshals: the Go
01277  * dispatcher (star_gateway/internal/dispatcher/dispatcher.go) calls
01278  * proto.Unmarshal into a &star_v1.WireMessage{} and routes on the
01279  * oneof. Without the wrap the dispatcher drops every frame silently
01280  * at debug level with "failed to unmarshal wire message" and nothing
01281  * reaches ROS2.
01282  */
01283 /* s_wire is file-scope (static) rather than a stack local: the union
01284  * in star_v1_WireMessage spans every oneof variant (TelemetryData,
01285  * TransportDiagnostics, PidConfig, ...) so sizeof(star_v1_WireMessage)
01286  * can be hundreds of bytes. Putting it on the telemetry_task 2 KB
01287  * stack alongside s_telem_buffer exhausts headroom. Single-writer
01288  * (telemetry_task is the only producer), so the static has no
01289  * reentrancy concern. */
01290 static star_v1_WireMessage s_wire;
01291 s_wire.which_payload = star_v1_WireMessage_telemetry_data_tag;
01292 s_wire.payload.telemetry_data = *msg;
01293
01294 pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
01295
01296 /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
01297 (void)pb_encode(&stream, star_v1_WireMessage_fields, &s_wire);
01298 *len = stream.bytes_written;
01299
01300 return k_rx_ok;
01301 }

```

Here is the call graph for this function:



6.49.3.11 rx_nanopb_encode_velocity_request()

```
rx_err_t rx_nanopb_encode_velocity_request (
    const star_v1_SetVelocityRequest * msg,
    uint8_t * buffer,
    const uint32_t buffer_size,
    uint32_t * len) [nodiscard]
```

Encode SetVelocityRequest message to Protocol Buffer bytes.

Serializes a SetVelocityRequest message containing motor velocity commands into Protocol Buffer wire format. Used when RX72N needs to forward or echo velocity commands.

Message Structure

```
message SetVelocityRequest {
    RequestHeader header = 1;
    VelocityCommand command = 2;
}

message VelocityCommand {
    double front_left_mps = 1;
    double front_right_mps = 2;
    double back_left_mps = 3;
    double back_right_mps = 4;
    uint32 sequence = 5;
}
```

Parameters

in	msg	Message to encode - Must be fully populated - Velocities in m/s (typically -2.0 to +2.0)
out	buffer	Output buffer for encoded bytes - Must be at least s_nanopb_buffer_size bytes - Receives wire-format data
in	buffer_size	Size of output buffer in bytes Must be >= s_nanopb_buffer_size (512)
out	len	Actual encoded message length Typically 40-60 bytes for velocity command

Returns

rx_err_t Error code indicating result

Return values

k_rx_ok	Success, buffer contains encoded message
---------	--

<code>k_rx_err_invalid_arg</code>	nullptr in msg, buffer, or len
<code>k_rx_err_not_initialized</code>	Module not initialized
<code>k_rx_err_invalid_size</code>	Buffer too small or encoding failed

Precondition

`rx_nanopb_init()` must be called first
msg must be fully populated
buffer_size >= s_nanopb_buffer_size

Postcondition

buffer contains wire-format Protocol Buffer data
len contains actual encoded length

Note

Not thread-safe (uses shared internal buffer)

Performance: ~20 us @ 240 MHz

See also

`rx_nanopb_decode_velocity_request()` Decode counterpart

Since

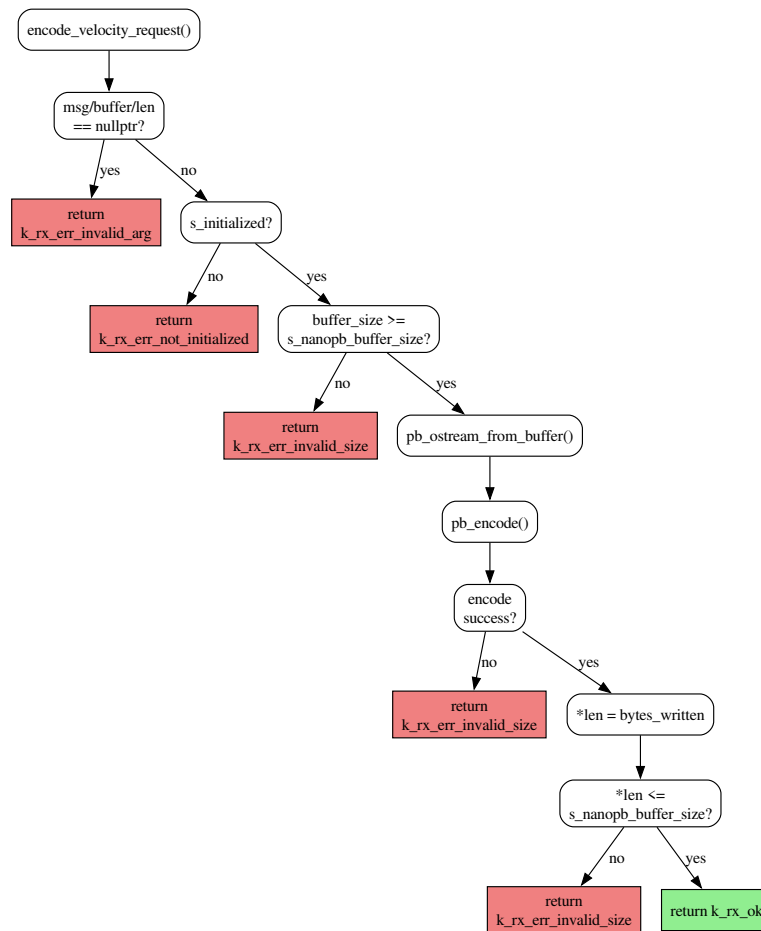
Version 1.0.0

Encode SetVelocityRequest message to Protocol Buffer bytes.

Serializes a SetVelocityRequest message containing motor velocity commands into Protocol Buffer binary format for transmission over SPI or USB.

Algorithm Steps

1. Validate msg pointer (not nullptr)
2. Validate buffer pointer (not nullptr)
3. Validate len pointer (not nullptr)
4. Verify module is initialized
5. Verify buffer size is adequate
6. Create nanopb output stream from buffer
7. Encode message using generated field descriptors
8. Return encoded length in *len
9. Verify encoded length is within bounds (post-condition)



Precondition

`rx_nanopb_init()` must have been called successfully
 msg must be fully populated with valid data
 buffer_size >= s_nanopb_buffer_size (512)

Postcondition

buffer[0..*len-1] contains valid Protocol Buffer wire-format data
 *len <= s_nanopb_buffer_size

Note

Not thread-safe - protect with mutex if called from multiple tasks

Performance

- Execution time: ~20 us @ 240 MHz
- Stack usage: ~96 bytes

Example - Basic Usage:

```
star_v1_SetVelocityRequest request = star_v1_SetVelocityRequest_init_zero;

// Populate command
request.command.front_left_velocity_mps = 1.0;
request.command.front_right_velocity_mps = 1.0;
request.command.back_left_velocity_mps = 1.0;
request.command.back_right_velocity_mps = 1.0;
request.command.sequence = g_sequence++;

uint8_t buffer[512];
uint32_t encoded_len;

rx_err_t err = rx_nanopb_encode_velocity_request(&request, buffer, sizeof(buffer), &encoded_len);
if (err == k_rx_ok) {
    spi_send(buffer, encoded_len);
}
```

See also

[rx_nanopb_decode_velocity_request\(\)](#) Decode counterpart
[star_v1_SetVelocityRequest_fields](#) nanopb field descriptors

NASA Power of 10 Compliance

- Rule 5: [OK] 3 pre-conditions, 1 post-condition
- Rule 7: [OK] [pb_encode\(\)](#) return value checked

Since

Version 1.0.0

Definition at line 546 of file [rx_nanopb.c](#).

```
00550 {
00551     /* Pre-condition 1: nullptr pointer checks */
00552     if (msg == nullptr || buffer == nullptr || len == nullptr) {
00553         return k_rx_err_invalid_arg;
00554     }
00555
00556     /* Pre-condition 2: Module initialized */
00557     if (!s_initialized) {
00558         return k_rx_err_not_initialized;
00559     }
00560
00561     /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
00562     if (buffer_size < s_nanopb_buffer_size) {
00563         return k_rx_err_invalid_size;
00564     }
00565
00566     pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
00567
00568     /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
00569     (void)pb_encode(&stream, star_v1_SetVelocityRequest_fields, msg);
00570     *len = stream.bytes_written;
00571
00572     return k_rx_ok;
00573 }
```

References [pb_encode\(\)](#), [pb_ostream_from_buffer\(\)](#), [s_initialized](#), [s_nanopb_buffer_size](#), and [star_v1_SetVelocityRequest_fields](#).

Here is the call graph for this function:

Returns

`rx_err_t` Error code indicating result

Return values

<code>k_rx_ok</code>	Success, buffer contains encoded message
<code>k_rx_err_invalid_arg</code>	nullptr in parameters
<code>k_rx_err_not_initialized</code>	Module not initialized
<code>k_rx_err_invalid_size</code>	Buffer too small

Precondition

`rx_nanopb_init()` must be called first

Postcondition

buffer contains wire-format response

Performance: ~15 us @ 240 MHz

See also

`rx_nanopb_decode_velocity_request()` Decode the incoming request

`rx_nanopb_create_response_header()` Create header with status

Since

Version 1.0.0

Encode SetVelocityResponse message to Protocol Buffer bytes.

Serializes a SetVelocityResponse message as acknowledgment to a velocity command. This response is sent back to RPi5 after processing a SetVelocityRequest to confirm receipt and indicate success/failure.

Algorithm Steps

1. Validate msg pointer (not nullptr)
2. Validate buffer pointer (not nullptr)
3. Validate len pointer (not nullptr)
4. Verify module is initialized
5. Verify buffer size is adequate
6. Create nanopb output stream from buffer
7. Encode message using generated field descriptors
8. Return encoded length in *len
9. Verify encoded length is within bounds (post-condition)

Precondition

`rx_nanopb_init()` must have been called successfully
msg should be populated via `rx_nanopb_create_response_header()`
`buffer_size` $\geq$ `s_nanopb_buffer_size` (512)

Postcondition

`buffer[0..*len-1]` contains valid Protocol Buffer wire-format data
`*len` $\leq$ `s_nanopb_buffer_size`

Note

Not thread-safe - protect with mutex if called from multiple tasks

Performance

- Execution time: ~ 15 us @ 240 MHz
- Stack usage: ~ 48 bytes

Example - Responding to Velocity Command:

```
// After successfully processing velocity command
star_v1_SetVelocityResponse response = star_v1_SetVelocityResponse_init_zero;

// Create header with success status
rx_nanopb_create_response_header(&response.header,
                                star_v1_Status_STATUS_OK,
                                request.header.request_id);

// Encode and send
uint8_t buffer[512];
uint32_t len;
rx_err_t err = rx_nanopb_encode_velocity_response(&response, buffer, sizeof(buffer), &len);
if (err == k_rx_ok) {
    spi_send(buffer, len);
}
```

See also

`rx_nanopb_decode_velocity_request()` Decode incoming request first
`rx_nanopb_create_response_header()` Create header with status

NASA Power of 10 Compliance

- Rule 5: [OK] 3 pre-conditions, 1 post-condition
- Rule 7: [OK] `pb_encode()` return value checked

Since

Version 1.0.0

Definition at line 772 of file rx_nanopb.c.

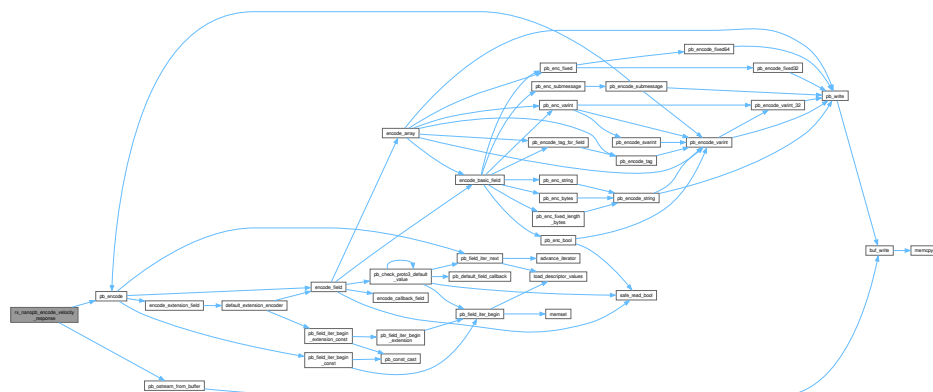
```

00776 {
00777     /* Pre-condition 1: nullptr pointer checks */
00778     if (msg == nullptr || buffer == nullptr || len == nullptr) {
00779         return k_rx_err_invalid_arg;
00780     }
00781
00782     /* Pre-condition 2: Module initialized */
00783     if (!s_initialized) {
00784         return k_rx_err_not_initialized;
00785     }
00786
00787     /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
00788     if (buffer_size < s_nanopb_buffer_size) {
00789         return k_rx_err_invalid_size;
00790     }
00791
00792     pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
00793
00794     /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
00795     (void)pb_encode(&stream, star_v1_SetVelocityResponse_fields, msg);
00796     *len = stream.bytes_written;
00797
00798     return k_rx_ok;
00799 }

```

References `pb_encode()`, `pb_ostream_from_buffer()`, `s_initialized`, `s_nanopb_buffer_size`, and `star_v1_SetVelocityResponse` fields.

Here is the call graph for this function:



6.49.3.13 rx_nanopb_init()

```
rx_err_t rx_nanopb_init (
    void ) [nodiscard]
```

Initialize nanopb wrapper module.

Initializes the nanopb wrapper state and verifies the module is ready for encode/decode operations. Must be called once before any encode/decode function. Typically called during system startup.

Initialization steps:

1. Clear internal state
2. Initialize static buffers
3. Verify nanopb library version
4. Set initialized flag

Returns

`rx_err_t` Error code indicating result

Return values

<code>k_rx_ok</code>	Success, module ready for use
<code>k_rx_err_invalid_state</code>	Already initialized (call rx_nanopb_test_reset_state() first)
<code>k_rx_err_generic</code>	Internal verification failed

Precondition

None (first function to call)

Postcondition

Module ready for encode/decode operations

Note

Thread-safe (initialization is idempotent after first call)

Safe to call multiple times (returns `k_rx_err_invalid_state`)

Example:

```
void system_init(void) {
    rx_err_t err = rx_nanopb_init();
    if (err != k_rx_ok) {
        // Handle initialization failure
        system_halt("nanopb init failed");
    }
    // Now safe to use encode/decode functions
}
```

See also

[rx_nanopb_test_reset_state\(\)](#) Reset for unit testing

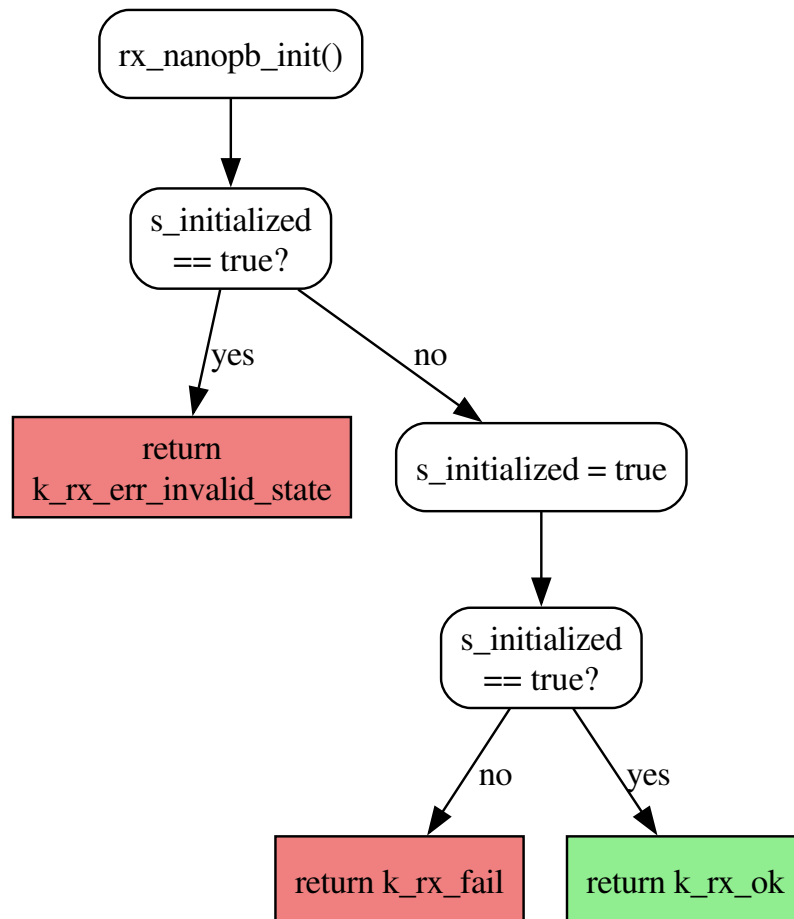
Since

Version 1.0.0

Initializes the nanopb wrapper module state and prepares it for encode/decode operations. This function must be called once during system startup before any message encoding or decoding.

Algorithm Steps

1. Check if module is already initialized (pre-condition)
2. Set initialization flag to true
3. Verify flag was set correctly (post-condition)
4. Return success



Precondition

- Module must not be already initialized
- System startup must be in progress (single-threaded context)

Postcondition

- `s_initialized == true`
- All encode/decode functions can be called

Invariant

- After successful init, `s_initialized` remains true until reset

Note

Not thread-safe - call from main thread during startup
 Idempotent after first call (returns error, no side effects)

Performance: ~1 us @ 240 MHz

Stack Usage: 8 bytes

Example - System Initialization:

```
void app_main_init(void) {
    rx_err_t err;

    // Initialize logging first
    err = rx_log_init();
    RX_ASSERT(err == k_rx_ok);

    // Initialize nanopb wrapper
    err = rx_nanopb_init();
    if (err != k_rx_ok) {
        rx_log_error("NANOPB", "Init failed: %d", err);
        // Handle critical error - cannot communicate
        system_halt();
    }

    // Now safe to use encode/decode functions
    rx_log_info("NANOPB", "Module initialized");
}
```

Example - Error Handling:

```
rx_err_t err = rx_nanopb_init();
switch (err) {
    case k_rx_ok:
        // Success - proceed normally
        break;
    case k_rx_err_invalid_state:
        // Already initialized - may be OK depending on context
        rx_log_warn("NANOPB", "Already initialized");
        break;
    default:
        // Unexpected error - critical failure
        rx_log_error("NANOPB", "Init error: %d", err);
        return err;
}
```

See also

[rx_nanopb_test_reset_state\(\)](#) Reset for unit testing
[s_initialized](#) Module state flag

NASA Power of 10 Compliance

- Rule 5: [OK] 1 pre-condition check, 1 post-condition verification
- Rule 7: [OK] Return value indicates success/failure

Since

Version 1.0.0

Definition at line 357 of file [rx_nanopb.c](#).

```
00358 {
00359     /* Pre-condition: Check not already initialized */
00360     if (s_initialized) {
00361         return k_rx_err_invalid_state;
00362     }
00363
00364     s_initialized = true;
00365
00366     /* Post-condition: Verify initialization succeeded */
00367
00368     return k_rx_ok;
00369 }
```

References [s_initialized](#).

6.49.3.14 rx_nanopb_test_reset_state()

```
void rx_nanopb_test_reset_state (
    void )
```

Reset module state for unit testing.

Resets the nanopb wrapper to uninitialized state. Used in unit tests to verify initialization behavior and ensure clean state between tests.

Warning

FOR UNIT TESTING ONLY - Do not call in production code

Precondition

None

Postcondition

Module state reset to uninitialized

[rx_nanopb_init\(\)](#) must be called before encode/decode functions

Note

This function only exists in test builds

Example - Test Setup:

```
void setUp(void) {
    rx_nanopb_test_reset_state(); // Clean slate
    rx_nanopb_init();             // Re-initialize
}

void test_encode_without_init(void) {
    rx_nanopb_test_reset_state(); // Don't call init

    // Verify encode fails without initialization
    rx_err_t err = rx_nanopb_encode_telemetry(&msg, buf, 512, &len);
    TEST_ASSERT_EQUAL(k_rx_err_not_initialized, err);
}
```

Since

Version 1.0.0

Resets the nanopb wrapper module to its uninitialized state. This function is intended exclusively for unit testing to ensure clean test isolation.

Algorithm Steps

1. Set s_initialized to false

Warning

FOR UNIT TESTING ONLY - Do not call in production code. Calling this while encode/decode operations are in progress will cause undefined behavior.

Precondition

None

Postcondition

s_initialized == false
[rx_nanopb_init\(\)](#) must be called before encode/decode functions

Note

Not thread-safe
No return value - always succeeds

Performance: ~100 ns @ 240 MHz

Stack Usage: 0 bytes (inline capable)

Example - Test Fixture Setup:

```
// Unity test framework setup
void setUp(void) {
    // Reset to clean state before each test
    rx_nanopb_test_reset_state();

    // Re-initialize for tests that need it
    rx_err_t err = rx_nanopb_init();
    TEST_ASSERT_EQUAL(k_rx_ok, err);
}

void tearDown(void) {
    // Clean up after each test
    rx_nanopb_test_reset_state();
}
```

Example - Testing Uninitialized Behavior:

```
void test_encode_fails_when_not_initialized(void) {
    // Ensure module is NOT initialized
    rx_nanopb_test_reset_state();

    star_v1_SetVelocityRequest msg = {};
    uint8_t buffer[512];
    uint32_t len;

    rx_err_t err = rx_nanopb_encode_velocity_request(&msg, buffer, 512, &len);

    // Should fail with not initialized error
    TEST_ASSERT_EQUAL(k_rx_err_not_initialized, err);
}
```

See also

[rx_nanopb_init\(\)](#) Initialize module after reset
[s_initialized](#) Module state flag

Since

Version 1.0.0

Definition at line 436 of file [rx_nanopb.c](#).

```
00437 {
00438     s_initialized = false;
00439 }
```

References [s_initialized](#).

6.49.4 Variable Documentation

6.49.4.1 s_nanopb_buffer_size

```
const uint16_t s_nanopb_buffer_size = 512U    [static]
```

Maximum encode/decode buffer size in bytes.

Defines the static buffer size used for Protocol Buffer encoding/decoding. All messages must fit within this buffer. Size chosen based on:

- Largest message: TelemetryData (~200 bytes encoded)
- Margin for future expansion
- Memory constraints on RX72N

Size Rationale

Message Type	Typical Size	Max Size
SetVelocityRequest	48 bytes	64 bytes

Message Type	Typical Size	Max Size
SetVelocityResponse	24 bytes	32 bytes
TelemetryData	180 bytes	256 bytes
EmergencyStop*	16 bytes	24 bytes

Note

512 bytes provides 2x margin over largest message

Warning

Do not reduce below 256 bytes

Since

Version 1.0.0

Definition at line 211 of file [rx_nanopb.h](#).

Referenced by [rx_nanopb_create_response_header\(\)](#), [rx_nanopb_decode_estop_request\(\)](#), [rx_nanopb_decode_pid_gains_request\(\)](#), [rx_nanopb_decode_retransmit_config_request\(\)](#), [rx_nanopb_decode_velocity_request\(\)](#), [rx_nanopb_encode_estop_request\(\)](#), [rx_nanopb_encode_pid_gains_response\(\)](#), [rx_nanopb_encode_telemetry\(\)](#), [rx_nanopb_encode_velocity_request\(\)](#), and [rx_nanopb_encode_velocity_response\(\)](#).

6.50 rx_nanopb.h

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_nanopb.h
00003  * @brief nanopb Integration Wrapper for RX72N
00004  *
00005  * @details
00006  * Provides simplified encode/decode functions with static buffers for Protocol
00007  * Buffer messages used in the STAR project. Wraps the nanopb library with
00008  * RX72N-friendly APIs that handle memory constraints and error reporting.
00009  *
00010  * @par System Architecture
00011  * @dot
00012  * digraph nanopb_architecture {
00013  *   rankdir=TB;
00014  *   node [shape=box, style=rounded];
00015  *
00016  *   subgraph cluster_app {
00017  *     label="Application Layer";
00018  *     style=filled;
00019  *     color=lightyellow;
00020  *     motor [label="Motor Control\nTask"];
00021  *     telemetry [label="Telemetry\nTask"];
00022  *   }
00023  *
00024  *   subgraph cluster_nanopb {
00025  *     label="nanopb Wrapper (This Module)";
00026  *     style=filled;
00027  *     color=lightblue;
00028  *     encode [label="rx_nanopb_encode_*()"];
00029  *     decode [label="rx_nanopb_decode_*()"];
00030  *     helpers [label="Helper Functions\n(create_velocity_command)"];
00031  *   }
00032  *
00033  *   subgraph cluster_proto {
00034  *     label="Generated Protobuf Code";
00035  *     style=filled;
00036  *     color=lightgreen;

```



```

00037 *   common [label="common.pb.h"];
00038 *   motor_pb [label="motor_control.pb.h"];
00039 *   telem_pb [label="telemetry.pb.h"];
00040 * }
00041 *
00042 * subgraph cluster_comm {
00043 *   label="Communication Layer";
00044 *   style=filled;
00045 *   color=lightcoral;
00046 *   spi [label="SPI Comm\n(to RPi5)"];
00047 *   usb [label="USB CDC\n(debug)"];
00048 * }
00049 *
00050 * motor -> encode;
00051 * motor -> decode;
00052 * telemetry -> encode;
00053 * encode -> common;
00054 * encode -> motor_pb;
00055 * encode -> telem_pb;
00056 * decode -> common;
00057 * decode -> motor_pb;
00058 * helpers -> motor_pb;
00059 * encode -> spi;
00060 * decode -> spi;
00061 * encode -> usb;
00062 * }
00063 * @enddot
00064 *
00065 * @par Message Flow
00066 * @msc
00067 * RPi5, SPI, RX72N_Comm, nanopb, Motor;
00068 *
00069 * RPi5 -> SPI [label="SetVelocityRequest (encoded)"];
00070 * SPI -> RX72N_Comm [label="Raw bytes"];
00071 * RX72N_Comm -> nanopb [label="rx_nanopb_decode_velocity_request()"];
00072 * nanopb -> Motor [label="star_v1_SetVelocityRequest"];
00073 * Motor -> Motor [label="Apply velocity"];
00074 * Motor -> nanopb [label="star_v1_SetVelocityResponse"];
00075 * nanopb -> RX72N_Comm [label="rx_nanopb_encode_velocity_response()"];
00076 * RX72N_Comm -> SPI [label="Raw bytes"];
00077 * SPI -> RPi5 [label="SetVelocityResponse (encoded)"];
00078 * @endmsc
00079 *
00080 * @par Supported Message Types
00081 * | Message | Direction | Description |
00082 * |-----|-----|-----|
00083 * | SetVelocityRequest | RPi5 -> RX72N | Motor velocity commands |
00084 * | SetVelocityResponse | RX72N -> RPi5 | Command acknowledgment |
00085 * | EmergencyStopRequest | RPi5 -> RX72N | Emergency stop command |
00086 * | EmergencyStopResponse | RX72N -> RPi5 | E-stop acknowledgment |
00087 * | TelemetryData | RX72N -> RPi5 | Sensor/motor telemetry |
00088 *
00089 * @par Buffer Configuration
00090 * | Parameter | Value | Notes |
00091 * |-----|-----|-----|
00092 * | Max buffer size | 512 bytes | Static allocation |
00093 * | Max message size | ~256 bytes | Largest single message |
00094 * | Alignment | 4-byte | For DMA compatibility |
00095 *
00096 * @par Memory Usage
00097 * | Component | Size | Notes |
00098 * |-----|-----|-----|
00099 * | Code (.text) | ~3 KB | All encode/decode functions |
00100 * | Static buffer | 512 bytes | Shared encode buffer |
00101 * | Stack usage | ~128 bytes | Per encode/decode call |
00102 * | Message struct | 64-128 bytes | On caller stack |
00103 *
00104 * @par Performance Characteristics
00105 * | Operation | Time @ 240 MHz | Notes |
00106 * |-----|-----|-----|
00107 * | Encode velocity | ~20 us | 4-motor command |
00108 * | Decode velocity | ~15 us | 4-motor command |
00109 * | Encode telemetry | ~30 us | Full telemetry packet |
00110 * | Buffer copy | ~5 us | 256 bytes |
00111 *
00112 * @par Wire Format
00113 * nanopb uses standard Protocol Buffer encoding:
00114 * - Variable-length integers (varint)
00115 * - Length-delimited fields for strings/bytes
00116 * - Little-endian for fixed-width types
00117 * - No alignment padding
00118 *
00119 * @par Error Handling
00120 * | Error | Cause | Recovery |
00121 * |-----|-----|-----|
00122 * | k_rx_err_invalid_size | Buffer too small | Use larger buffer |
00123 * | k_rx_err_protocol_error | Malformed message | Discard, request retransmit |

```

```

00124 * | k_rx_err_not_initialized | Module not init'd | Call rx_nanopb_init() |
00125 *
00126 * @par Thread Safety
00127 * - Encode/decode functions: **NOT thread-safe** (shared buffer)
00128 * - Helper functions: Thread-safe (no shared state)
00129 * - Call from single task or protect with mutex
00130 *
00131 * @par Module Dependencies
00132 * - nanopb: Core protobuf library (git submodule)
00133 * - gen/star/v1/<*>.pb.h: Generated message headers
00134 * - rx_err.h: Error code definitions
00135 *
00136 * @par NASA Power of 10 Compliance
00137 * - Rule 1: [OK] No goto, setjmp, or recursion
00138 * - Rule 2: [OK] All loops bounded by message field counts
00139 * - Rule 3: [OK] No dynamic memory (static buffers only)
00140 * - Rule 4: [OK] Functions under 60 lines
00141 * - Rule 5: [OK] Input validation on all parameters
00142 * - Rule 6: [OK] Variables at smallest scope
00143 * - Rule 7: [OK] All return values checked
00144 * - Rule 8: [OK] Limited preprocessor (nanopb macros only)
00145 * - Rule 9: [OK] Callbacks only for nanopb field encoding
00146 * - Rule 10: [OK] Compiled with -Wall -Wextra -Werror
00147 *
00148 * @par SOLID Principles
00149 * - **S**: Single Responsibility - Protobuf encode/decode only
00150 * - **O**: Open/Closed - Extensible by adding new message types
00151 * - **I**: Liskov Substitution - N/A (no inheritance)
00152 * - **I**: Interface Segregation - Separate function per message type
00153 * - **D**: Dependency Inversion - Depends on generated message headers
00154 *
00155 * @see star-proto/proto/star/v1/ Protocol Buffer schema definitions
00156 * @see https://jpa.kapsi.fi/nanopb/ nanopb documentation
00157 *
00158 * @version 1.0.0
00159 * @date 2026-01-01
00160 * @copyright Copyright (c) 2026 Locked Inc.
00161 * SPDX-License-Identifier: MIT
00162 */
00163
00164 #pragma once
00165
00166 #include <stddef.h>
00167 #include <stdint.h>
00168
00169 #include "rx_err.h"
00170
00171 /* Include generated protobuf headers */
00172 #include "gen/star/v1/common.pb.h"
00173 #include "gen/star/v1/configuration.pb.h"
00174 #include "gen/star/v1/gateway_service.pb.h"
00175 #include "gen/star/v1/motor_control.pb.h"
00176 #include "gen/star/v1/telemetry.pb.h"
00177
00178 #ifdef __cplusplus
00179 extern "C" {
00180 #endif
00181
00182 /*
=====
00183 * Buffer Configuration
00184 *
=====
00185 */
00186
00187 /**
00188 * @var s_nanopb_buffer_size
00189 * @brief Maximum encode/decode buffer size in bytes
00190 *
00191 * @details
00192 * Defines the static buffer size used for Protocol Buffer encoding/decoding.
00193 * All messages must fit within this buffer. Size chosen based on:
00194 * - Largest message: TelemetryData (~200 bytes encoded)
00195 * - Margin for future expansion
00196 * - Memory constraints on RX72N
00197 *
00198 * @par Size Rationale
00199 * | Message Type | Typical Size | Max Size |
00200 * |-----|-----|-----|
00201 * | SetVelocityRequest | 48 bytes | 64 bytes |
00202 * | SetVelocityResponse | 24 bytes | 32 bytes |
00203 * | TelemetryData | 180 bytes | 256 bytes |
00204 * | EmergencyStop* | 16 bytes | 24 bytes |
00205 *
00206 * @note 512 bytes provides 2x margin over largest message
00207 * @warning Do not reduce below 256 bytes
00208 */

```

```

00209 * @since Version 1.0.0
00210 */
00211 static const uint16_t s_nanopb_buffer_size = 512U;
00212
00213 /*
=====
00214 * Initialization
00215 *
=====
00216 */
00217 /**
00218 * @brief Initialize nanopb wrapper module
00219 *
00220 * @details
00221 * Initializes the nanopb wrapper state and verifies the module is ready
00222 * for encode/decode operations. Must be called once before any encode/decode
00223 * function. Typically called during system startup.
00224 *
00225 * Initialization steps:
00226 * 1. Clear internal state
00227 * 2. Initialize static buffers
00228 * 3. Verify nanopb library version
00229 * 4. Set initialized flag
00230 *
00231 * @return rx_err_t Error code indicating result
00232 * @retval k_rx_ok Success, module ready for use
00233 * @retval k_rx_err_invalid_state Already initialized (call rx_nanopb_test_reset_state() first)
00234 * @retval k_rx_err_generic Internal verification failed
00235 *
00236 * @pre None (first function to call)
00237 * @post Module ready for encode/decode operations
00238 *
00239 * @note Thread-safe (initialization is idempotent after first call)
00240 * @note Safe to call multiple times (returns k_rx_err_invalid_state)
00241 *
00242 * @par Example:
00243 * @code
00244 * void system_init(void) {
00245 *     rx_err_t err = rx_nanopb_init();
00246 *     if (err != k_rx_ok) {
00247 *         // Handle initialization failure
00248 *         system_halt("nanopb init failed");
00249 *     }
00250 *     // Now safe to use encode/decode functions
00251 * }
00252 * @endcode
00253 *
00254 * @see rx_nanopb_test_reset_state() Reset for unit testing
00255 * @since Version 1.0.0
00256 */
00257 [[nodiscard]] rx_err_t rx_nanopb_init(void);
00258
00259 /*
=====
00260 * SetVelocityRequest Encode/Decode
00261 *
=====
00262 */
00263 /**
00264 *
00265 */
00266 * @brief Encode SetVelocityRequest message to Protocol Buffer bytes
00267 *
00268 * @details
00269 * Serializes a SetVelocityRequest message containing motor velocity commands
00270 * into Protocol Buffer wire format. Used when RX72N needs to forward or
00271 * echo velocity commands.
00272 *
00273 * @par Message Structure
00274 * ```protobuf
00275 * message SetVelocityRequest {
00276 *     RequestHeader header = 1;
00277 *     VelocityCommand command = 2;
00278 * }
00279 *
00280 * message VelocityCommand {
00281 *     double front_left_mps = 1;
00282 *     double front_right_mps = 2;
00283 *     double back_left_mps = 3;
00284 *     double back_right_mps = 4;
00285 *     uint32 sequence = 5;
00286 * }
00287 * ```
00288 *
00289 * @param[in] msg Message to encode
00290 * - Must be fully populated
00291 * - Velocities in m/s (typically -2.0 to +2.0)

```

```

00292 * @param[out] buffer Output buffer for encoded bytes
00293 * - Must be at least s_nanopb_buffer_size bytes
00294 * - Receives wire-format data
00295 * @param[in] buffer_size Size of output buffer in bytes
00296 * - Must be >= s_nanopb_buffer_size (512)
00297 * @param[out] len Actual encoded message length
00298 * - Typically 40-60 bytes for velocity command
00299 *
00300 * @return rx_err_t Error code indicating result
00301 * @retval k_rx_ok Success, buffer contains encoded message
00302 * @retval k_rx_err_invalid_arg nullptr in msg, buffer, or len
00303 * @retval k_rx_err_not_initialized Module not initialized
00304 * @retval k_rx_err_invalid_size Buffer too small or encoding failed
00305 *
00306 * @pre rx_nanopb_init() must be called first
00307 * @pre msg must be fully populated
00308 * @pre buffer_size >= s_nanopb_buffer_size
00309 *
00310 * @post buffer contains wire-format Protocol Buffer data
00311 * @post len contains actual encoded length
00312 *
00313 * @note Not thread-safe (uses shared internal buffer)
00314 *
00315 * @par Performance: ~20 us @ 240 MHz
00316 *
00317 * @see rx_nanopb_decode_velocity_request() Decode counterpart
00318 * @since Version 1.0.0
00319 */
00320 [[nodiscard]] rx_err_t rx_nanopb_encode_velocity_request(const star_v1_SetVelocityRequest* msg,
00321                                                         uint8_t* buffer,
00322                                                         uint32_t buffer_size,
00323                                                         uint32_t* len);
00324
00325 /**
00326 * @brief Decode SetVelocityRequest message from Protocol Buffer bytes
00327 *
00328 * @details
00329 * Deserializes a SetVelocityRequest message received from RPi5 over SPI.
00330 * This is the primary command message for motor velocity control.
00331 *
00332 * @par Decoding Process
00333 * 1. Validate buffer pointer and length
00334 * 2. Create nanopb input stream from buffer
00335 * 3. Decode header and command fields
00336 * 4. Validate decoded values are within range
00337 *
00338 * @param[in] buffer Input buffer containing encoded message
00339 * - Wire-format Protocol Buffer data
00340 * - Received from SPI or USB
00341 * @param[in] len Buffer length in bytes
00342 * - Must match actual encoded message size
00343 * @param[out] msg Decoded message structure
00344 * - Will be fully populated on success
00345 * - Velocities in m/s
00346 *
00347 * @return rx_err_t Error code indicating result
00348 * @retval k_rx_ok Success, msg contains decoded values
00349 * @retval k_rx_err_invalid_arg nullptr or len == 0
00350 * @retval k_rx_err_not_initialized Module not initialized
00351 * @retval k_rx_err_protocol_error Malformed message, CRC error, or invalid field
00352 *
00353 * @pre rx_nanopb_init() must be called first
00354 * @pre buffer contains valid wire-format data
00355 *
00356 * @post msg fully populated with decoded values
00357 *
00358 * @note Not thread-safe (uses shared internal state)
00359 *
00360 * @par Performance: ~15 us @ 240 MHz
00361 *
00362 * @par Example:
00363 * @code
00364 * star_v1_SetVelocityRequest request;
00365 * rx_err_t err = rx_nanopb_decode_velocity_request(spi_buffer, spi_len, &request);
00366 * if (err == k_rx_ok) {
00367 *     // Apply velocities to motors
00368 *     motor_set_velocity(0, request.command.front_left_mps);
00369 *     motor_set_velocity(1, request.command.front_right_mps);
00370 *     // ...
00371 * } else if (err == k_rx_err_protocol_error) {
00372 *     // Request retransmit
00373 *     send_nack();
00374 * }
00375 * @endcode
00376 *
00377 * @see rx_nanopb_encode_velocity_request() Encode counterpart
00378 * @since Version 1.0.0

```

```

00379 */
00380 [[nodiscard]] rx_err_t rx_nanopb_decode_velocity_request(const uint8_t*      buffer,
00381                                                         uint32_t      len,
00382                                                         star_v1_SetVelocityRequest* msg);
00383
00384 /*
=====
00385 * SetVelocityResponse Encode/Decode
00386 *
=====
00387 */
00388 /**
00389 * @brief Encode SetVelocityResponse message to Protocol Buffer bytes
00390 *
00391 * @details
00392 * Serializes a SetVelocityResponse message as acknowledgment to a velocity
00393 * command. Sent back to RPi5 after processing SetVelocityRequest.
00394 *
00395 * @par Message Structure
00396 * ```protobuf
00397 * message SetVelocityResponse {
00398 *   ResponseHeader header = 1;
00399 *   // Status in header indicates success/failure
00400 * }
00401 * ```
00402 *
00403 * @param[in] msg Message to encode
00404 *   - header.status: k_star_v1_status_ok or error code
00405 *   - header.request_id: Echo of request ID
00406 * @param[out] buffer Output buffer for encoded bytes
00407 * @param[in] buffer_size Size of output buffer (>= s_nanopb_buffer_size)
00408 * @param[out] len Actual encoded message length (typically 20-30 bytes)
00409 *
00410 * @return rx_err_t Error code indicating result
00411 * @retval k_rx_ok Success, buffer contains encoded message
00412 * @retval k_rx_err_invalid_arg nullptr in parameters
00413 * @retval k_rx_err_not_initialized Module not initialized
00414 * @retval k_rx_err_invalid_size Buffer too small
00415 *
00416 * @pre rx_nanopb_init() must be called first
00417 * @post buffer contains wire-format response
00418 *
00419 * @par Performance: ~15 us @ 240 MHz
00420 *
00421 * @see rx_nanopb_decode_velocity_request() Decode the incoming request
00422 * @see rx_nanopb_create_response_header() Create header with status
00423 * @since Version 1.0.0
00424 */
00425
00426 [[nodiscard]] rx_err_t rx_nanopb_encode_velocity_response(const star_v1_SetVelocityResponse* msg,
00427                                                         uint8_t*      buffer,
00428                                                         uint32_t      buffer_size,
00429                                                         uint32_t* len);
00430
00431 /*
=====
00432 * EmergencyStopRequest Encode/Decode
00433 *
=====
00434 */
00435 /**
00436 * @brief Decode EmergencyStopRequest message from Protocol Buffer bytes
00437 *
00438 * @details
00439 * Deserializes an emergency stop command from RPi5. This is a safety-critical
00440 * message that must be processed with highest priority.
00441 *
00442 * @par E-Stop Processing
00443 * 1. Decode message (this function)
00444 * 2. Immediately stop all motors
00445 * 3. Enter safe state
00446 * 4. Send EmergencyStopResponse
00447 *
00448 * @param[in] buffer Input buffer containing encoded message
00449 * @param[in] len Buffer length in bytes
00450 * @param[out] msg Decoded message structure
00451 *
00452 * @return rx_err_t Error code indicating result
00453 * @retval k_rx_ok Success, msg contains decoded values
00454 * @retval k_rx_err_invalid_arg nullptr or len == 0
00455 * @retval k_rx_err_not_initialized Module not initialized
00456 * @retval k_rx_err_protocol_error Malformed message
00457 *
00458 * @warning E-stop must be processed immediately regardless of return value
00459 * @note Consider stopping motors first, then decode for response
00460 */
00461

```

```

00462 * @par Performance: ~10 us @ 240 MHz (minimal message)
00463 *
00464 * @see rx_nanopb_encode_estop_response() Send acknowledgment
00465 * @since Version 1.0.0
00466 */
00467 [[nodiscard]] rx_err_t rx_nanopb_decode_estop_request(const uint8_t*          buffer,
00468                                                       uint32_t          len,
00469                                                       star_v1_EmergencyStopRequest* msg);
00470
00471 /**
00472 * @brief Encode EmergencyStopResponse message to Protocol Buffer bytes
00473 *
00474 * @details
00475 * Serializes an emergency stop response as acknowledgment. Confirms
00476 * that the RX72N has processed the E-stop and entered safe state.
00477 *
00478 * @param[in] msg Message to encode
00479 *   - header.status: Indicates if E-stop was successful
00480 *   - Typically always returns success (motors stopped)
00481 * @param[out] buffer Output buffer for encoded bytes
00482 * @param[in] buffer_size Size of output buffer (>= s_nanopb_buffer_size)
00483 * @param[out] len Actual encoded message length
00484 *
00485 * @return rx_err_t Error code indicating result
00486 * @retval k_rx_ok Success, buffer contains encoded response
00487 * @retval k_rx_err_invalid_arg nullptr in parameters
00488 * @retval k_rx_err_not_initialized Module not initialized
00489 * @retval k_rx_err_invalid_size Buffer too small
00490 *
00491 * @par Performance: ~10 us @ 240 MHz
00492 *
00493 * @see rx_nanopb_decode_estop_request() Decode incoming E-stop
00494 * @since Version 1.0.0
00495 */
00496 [[nodiscard]] rx_err_t rx_nanopb_encode_estop_response(const star_v1_EmergencyStopResponse* msg,
00497                                                         uint8_t*          buffer,
00498                                                         uint32_t buffer_size,
00499                                                         uint32_t* len);
00500
00501 /*
=====
00502 * SetPidGainsRequest Encode/Decode
00503 *
=====
00504 */
00505
00506 /**
00507 * @brief Decode SetPidGainsRequest message from Protocol Buffer bytes
00508 *
00509 * @details
00510 * Deserializes a PID gains update command from RPi5 over SPI. This message
00511 * contains new PID controller gains (Kp, Ki, Kd) and output/integral limits
00512 * to be applied to one or more motors.
00513 *
00514 * @par PID Configuration Structure
00515 * The decoded message contains:
00516 *   - Kp (proportional gain)
00517 *   - Ki (integral gain)
00518 *   - Kd (derivative gain)
00519 *   - output_min_percent (minimum PWM duty cycle %)
00520 *   - output_max_percent (maximum PWM duty cycle %)
00521 *   - integral_min (anti-windup minimum)
00522 *   - integral_max (anti-windup maximum)
00523 *   - motor_id (0-3 for specific motor, -1 for all motors)
00524 *
00525 * @par Decoding Process
00526 * 1. Validate buffer pointer and length
00527 * 2. Create nanopb input stream from buffer
00528 * 3. Decode header and pid_config fields
00529 * 4. Extract motor_id for targeting
00530 *
00531 * @param[in] buffer Input buffer containing encoded message
00532 *   - Wire-format Protocol Buffer data
00533 *   - Received from SPI or USB
00534 * @param[in] len Buffer length in bytes
00535 *   - Must match actual encoded message size
00536 * @param[out] msg Decoded message structure
00537 *   - Will be fully populated on success
00538 *   - PID gains in standard units
00539 *
00540 * @return rx_err_t Error code indicating result
00541 * @retval k_rx_ok Success, msg contains decoded values
00542 * @retval k_rx_err_invalid_arg nullptr or len == 0
00543 * @retval k_rx_err_not_initialized Module not initialized
00544 * @retval k_rx_err_protocol_error Malformed message or invalid field
00545 *
00546 * @pre rx_nanopb_init() must be called first

```

```

00547 * @pre buffer contains valid wire-format data
00548 *
00549 * @post msg fully populated with decoded PID configuration
00550 *
00551 * @note Not thread-safe (uses shared internal state)
00552 *
00553 * @par Performance: ~18 us @ 240 MHz
00554 *
00555 * @par Example:
00556 * @code
00557 * star_v1_SetPidGainsRequest request;
00558 * rx_err_t err = rx_nanopb_decode_pid_gains_request(spi_buffer, spi_len, &request);
00559 * if (err == k_rx_ok && request.has_pid_config) {
00560 *     // Apply gains to motors
00561 *     pid_gains_t gains = {
00562 *         .kp = (float)request.pid_config.kp,
00563 *         .ki = (float)request.pid_config.ki,
00564 *         .kd = (float)request.pid_config.kd,
00565 *         .output_min = (float)request.pid_config.output_min_percent,
00566 *         .output_max = (float)request.pid_config.output_max_percent,
00567 *         .integral_min = (float)request.pid_config.integral_min,
00568 *         .integral_max = (float)request.pid_config.integral_max,
00569 *         .update_pending = true
00570 *     };
00571 *     shared_data_set_pid_gains(&gains);
00572 * } else if (err == k_rx_err_protocol_error) {
00573 *     // Request retransmit
00574 *     send_nack();
00575 * }
00576 * @endcode
00577 *
00578 * @see rx_nanopb_encode_pid_gains_response() Send acknowledgment
00579 * @see shared_data_set_pid_gains() Apply gains to motor controllers
00580 * @since Version 1.0.0
00581 */
00582 [[nodiscard]] rx_err_t rx_nanopb_decode_pid_gains_request(const uint8_t* buffer,
00583                                                         uint32_t len,
00584                                                         star_v1_SetPidGainsRequest* msg);
00585
00586 /**
00587 * @brief Encode SetPidGainsResponse message to Protocol Buffer bytes
00588 *
00589 * @details
00590 * Serializes a SetPidGainsResponse message as acknowledgment to a PID gains
00591 * update command. Sent back to RPi5 after processing a SetPidGainsRequest.
00592 *
00593 * The response indicates whether the gains were successfully written to
00594 * shared_data for the Motor Control Task to apply. The optional human-readable
00595 * message field is left empty (NULL callback); the success boolean is
00596 * sufficient for programmatic use.
00597 *
00598 * @param[in] msg Message to encode (must not be nullptr)
00599 * - has_header: must be true for header to be encoded
00600 * - header.status: k_star_v1_status_ok or error status
00601 * - header.request_id: echoed from the original request
00602 * - success: true if gains were written to shared_data
00603 * @param[out] buffer Output buffer for encoded bytes (must not be nullptr)
00604 * @param[in] buffer_size Size of output buffer in bytes
00605 * - Must be >= s_nanopb_buffer_size (512 bytes)
00606 * @param[out] len Actual encoded message length in bytes (must not be nullptr)
00607 *
00608 * @return rx_err_t Error code indicating result
00609 * @retval k_rx_ok Success, buffer contains wire-format response
00610 * @retval k_rx_err_invalid_arg nullptr in msg, buffer, or len
00611 * @retval k_rx_err_not_initialized Module not initialized via rx_nanopb_init()
00612 * @retval k_rx_err_invalid_size buffer_size too small or encoded length overflow
00613 *
00614 * @pre rx_nanopb_init() must be called before this function
00615 * @pre buffer must point to at least buffer_size bytes of writable memory
00616 * @post On k_rx_ok: buffer[0..*len-1] contains valid wire-format response
00617 * @post On k_rx_ok: *len <= s_nanopb_buffer_size
00618 *
00619 * @note Not thread-safe; call only from Communication Task context
00620 *
00621 * @par Performance: ~15 us @ 240 MHz
00622 *
00623 * @see rx_nanopb_decode_pid_gains_request() Decode the incoming request
00624 * @see rx_nanopb_create_response_header() Create header with status and request_id
00625 * @since Version 1.0.0
00626 */
00627 [[nodiscard]] rx_err_t rx_nanopb_encode_pid_gains_response(const star_v1_SetPidGainsResponse* msg,
00628                                                         uint8_t* buffer,
00629                                                         uint32_t buffer_size,
00630                                                         uint32_t* len);
00631
00632 /**
00633 * @brief Decode SetRetransmitConfigRequest from Protocol Buffer bytes

```



```

00634 *
00635 * @details
00636 * Decodes a serialized SetRetransmitConfigRequest message from a byte buffer.
00637 * Used to process RPi5 commands that configure SPI retransmission parameters.
00638 *
00639 * @param[in] buffer Raw protobuf bytes to decode
00640 * - **Valid range**: Non-nullptr to protobuf-encoded data
00641 * - **Max size**: s_nanopb_buffer_size (1024 bytes)
00642 *
00643 * @param[in] len Length of buffer in bytes
00644 * - **Valid range**: [1, s_nanopb_buffer_size]
00645 *
00646 * @param[out] msg Decoded message output
00647 * - **Valid range**: Non-nullptr to star_v1_SetRetransmitConfigRequest
00648 * - **Output**: Zero-initialized then populated from buffer
00649 *
00650 * @return rx_err_t Error code
00651 * @retval k_rx_ok Decode successful
00652 * @retval k_rx_err_invalid_arg nullptr or invalid length
00653 * @retval k_rx_err_not_initialized Module not initialized
00654 * @retval k_rx_err_protocol_error Protobuf decode failed
00655 *
00656 * @pre buffer and msg must be non-NULL
00657 * @pre rx_nanopb_init() must have been called
00658 * @post msg populated with decoded fields on success
00659 *
00660 * @see rx_spi_comm_set_auto_retransmit() Apply decoded config
00661 *
00662 * @since Version 1.0.0
00663 */
00664 [[nodiscard]] rx_err_t
00665 rx_nanopb_decode_retransmit_config_request(const uint8_t* buffer,
00666                                           uint32_t len,
00667                                           star_v1_SetRetransmitConfigRequest* msg);
00668
00669 /*
=====
00670 * Telemetry Encode/Decode
00671 *
=====
00672 */
00673
00674 /**
00675 * @brief Encode TelemetryData message to Protocol Buffer bytes
00676 *
00677 * @details
00678 * Serializes telemetry data for transmission to RPi5. Contains motor states,
00679 * encoder readings, and sensor data. Sent periodically
00680 * (typically 10-100 Hz).
00681 *
00682 * @par Telemetry Contents
00683 * | Field | Type | Units | Update Rate |
00684 * |-----|-----|-----|-----|
00685 * | Motor velocities | double x 4 | m/s | 100 Hz |
00686 * | Motor currents | double x 4 | A | 100 Hz |
00687 * | Encoder positions | int32 x 4 | ticks | 100 Hz |
00688 * | Temperature | double | degC | 1 Hz |
00689 *
00690 * @param[in] msg Message to encode (fully populated telemetry)
00691 * @param[out] buffer Output buffer for encoded bytes
00692 * @param[in] buffer_size Size of output buffer (>= s_nanopb_buffer_size)
00693 * @param[out] len Actual encoded message length (typically 150-200 bytes)
00694 *
00695 * @return rx_err_t Error code indicating result
00696 * @retval k_rx_ok Success, buffer contains encoded telemetry
00697 * @retval k_rx_err_invalid_arg nullptr in parameters
00698 * @retval k_rx_err_not_initialized Module not initialized
00699 * @retval k_rx_err_invalid_size Buffer too small
00700 *
00701 * @par Performance: ~30 us @ 240 MHz (full telemetry)
00702 *
00703 * @par Example:
00704 * @code
00705 * star_v1_TelemetryData telemetry = {};
00706 *
00707 * // Populate motor data
00708 * telemetry.motor_front_left_velocity_mps = motor_get_velocity(0);
00709 * telemetry.motor_front_left_current_a = motor_get_current(0);
00710 * // ... (populate all fields)
00711 *
00712 * uint8_t buffer[512];
00713 * uint32_t len;
00714 * rx_err_t err = rx_nanopb_encode_telemetry(&telemetry, buffer, sizeof(buffer), &len);
00715 * if (err == k_rx_ok) {
00716 *     spi_send(buffer, len);
00717 * }
00718 * @endcode

```



```

00719 *
00720 * @since Version 1.0.0
00721 */
00722 [[nodiscard]] rx_err_t rx_nanopb_encode_telemetry(const star_v1_TelemetryData* msg,
00723                                                    uint8_t* buffer,
00724                                                    uint32_t buffer_size,
00725                                                    uint32_t* len);
00726
00727 /*
=====
00728 * Helper Functions
00729 *
=====
00730 */
00731
00732 /**
00733 * @struct rx_velocity_command_params_t
00734 * @brief VelocityCommand parameters for 4 independent motors
00735 *
00736 * @details
00737 * Convenience structure for creating velocity commands with all four motors
00738 * independently controlled. Maps directly to the protobuf VelocityCommand.
00739 *
00740 * @par Motor Layout (Top View)
00741 * @verbatim
00742 *      FRONT
00743 *      +-----+-----+
00744 *      | FL  | FR  |
00745 *      |    |    |
00746 *      +-----+-----+
00747 *      | BL  | BR  |
00748 *      |    |    |
00749 *      +-----+-----+
00750 *      BACK
00751 * @endverbatim
00752 *
00753 * @par Velocity Convention
00754 * - Positive: Forward motion
00755 * - Negative: Reverse motion
00756 * - Range: -2.0 to +2.0 m/s (robot max speed)
00757 *
00758 * @see rx_nanopb_create_velocity_command() Create command from params
00759 * @since Version 1.0.0
00760 */
00761 typedef struct {
00762     double front_left_mps; /**< Front left motor velocity (m/s), [-2.0, +2.0] */
00763     double front_right_mps; /**< Front right motor velocity (m/s), [-2.0, +2.0] */
00764     double back_left_mps; /**< Back left motor velocity (m/s), [-2.0, +2.0] */
00765     double back_right_mps; /**< Back right motor velocity (m/s), [-2.0, +2.0] */
00766     uint32_t sequence; /**< Command sequence number (monotonically increasing) */
00767 } rx_velocity_command_params_t;
00768
00769 /**
00770 * @struct rx_velocity_diff_drive_params_t
00771 * @brief VelocityCommand parameters for differential drive mode
00772 *
00773 * @details
00774 * Simplified parameters for differential drive robots where left and right
00775 * sides are controlled together. Front/back motors on same side receive
00776 * identical velocities.
00777 *
00778 * @par Differential Drive Mapping
00779 * - left_mps -> front_left_mps = back_left_mps
00780 * - right_mps -> front_right_mps = back_right_mps
00781 *
00782 * @par Motion Examples
00783 * | left_mps | right_mps | Motion |
00784 * |-----|-----|-----|
00785 * | +1.0 | +1.0 | Forward |
00786 * | -1.0 | -1.0 | Reverse |
00787 * | +1.0 | -1.0 | Rotate CW |
00788 * | -1.0 | +1.0 | Rotate CCW |
00789 * | +1.0 | +0.5 | Turn right |
00790 *
00791 * @see rx_nanopb_create_velocity_command_diff_drive() Create diff-drive command
00792 * @since Version 1.0.0
00793 */
00794 typedef struct {
00795     double left_mps; /**< Left side velocity (m/s), applied to FL and BL */
00796     double right_mps; /**< Right side velocity (m/s), applied to FR and BR */
00797     uint32_t sequence; /**< Command sequence number */
00798 } rx_velocity_diff_drive_params_t;
00799
00800 /**
00801 * @brief Create VelocityCommand for 4 independent motors
00802 *
00803 * @details

```

```

00804 * Populates a VelocityCommand structure from parameters. Use for mecanum
00805 * or omnidirectional robots where each motor can have independent velocity.
00806 *
00807 * @param[out] cmd Output command structure to populate
00808 * - All velocity fields set from params
00809 * - sequence field set from params
00810 * @param[in] params Velocity command parameters
00811 * - All velocities in m/s
00812 * - sequence should be monotonically increasing
00813 *
00814 * @return rx_err_t Error code indicating result
00815 * @retval k_rx_ok Success, cmd populated
00816 * @retval k_rx_err_null_ptr nullptr in cmd or params
00817 *
00818 * @pre cmd and params must not be nullptr
00819 * @post cmd fully populated with velocity values
00820 *
00821 * @note Thread-safe (no shared state)
00822 *
00823 * @par Example:
00824 * @code
00825 * rx_velocity_command_params_t params = {
00826 *     .front_left_mps = 1.0,
00827 *     .front_right_mps = 1.0,
00828 *     .back_left_mps = 1.0,
00829 *     .back_right_mps = 1.0,
00830 *     .sequence = g_command_sequence++,
00831 * };
00832 * star_v1_VelocityCommand cmd;
00833 * rx_nanopb_create_velocity_command(&cmd, &params);
00834 * @endcode
00835 *
00836 * @see rx_velocity_command_params_t Parameter structure
00837 * @since Version 1.0.0
00838 */
00839 [[nodiscard]] rx_err_t
00840 rx_nanopb_create_velocity_command(star_v1_VelocityCommand* cmd,
00841                                 const rx_velocity_command_params_t* params);
00842
00843 /**
00844 * @brief Create VelocityCommand in differential drive mode
00845 *
00846 * @details
00847 * Populates a VelocityCommand for differential drive robots. Left and right
00848 * parameters are applied to both front and back motors on each side.
00849 *
00850 * @par Velocity Mapping
00851 * - params->left_mps -> cmd->front_left_mps AND cmd->back_left_mps
00852 * - params->right_mps -> cmd->front_right_mps AND cmd->back_right_mps
00853 *
00854 * @param[out] cmd Output command structure to populate
00855 * @param[in] params Differential drive parameters
00856 * - left_mps: Both left motors
00857 * - right_mps: Both right motors
00858 *
00859 * @return rx_err_t Error code indicating result
00860 * @retval k_rx_ok Success, cmd populated
00861 * @retval k_rx_err_null_ptr nullptr in cmd or params
00862 *
00863 * @pre cmd and params must not be nullptr
00864 * @post cmd populated with left/right velocities duplicated
00865 *
00866 * @note Thread-safe (no shared state)
00867 *
00868 * @par Example:
00869 * @code
00870 * // Turn left: right side faster than left
00871 * rx_velocity_diff_drive_params_t params = {
00872 *     .left_mps = 0.5,
00873 *     .right_mps = 1.0,
00874 *     .sequence = g_command_sequence++,
00875 * };
00876 * star_v1_VelocityCommand cmd;
00877 * rx_nanopb_create_velocity_command_diff_drive(&cmd, &params);
00878 * // Result: FL=BL=0.5, FR=BR=1.0
00879 * @endcode
00880 *
00881 * @see rx_velocity_diff_drive_params_t Parameter structure
00882 * @since Version 1.0.0
00883 */
00884 rx_err_t
00885 rx_nanopb_create_velocity_command_diff_drive(star_v1_VelocityCommand* cmd,
00886                                             const rx_velocity_diff_drive_params_t* params);
00887
00888 /**
00889 * @brief Create ResponseHeader with status
00890 *

```

```

00891 * @details
00892 * Populates a ResponseHeader structure for response messages. Sets status
00893 * and optionally echoes the request ID from the original request.
00894 *
00895 * @param[out] header Output header structure to populate
00896 * @param[in] status Response status code
00897 * - star_v1_Status_STATUS_OK: Success
00898 * - star_v1_Status_STATUS_INVALID_ARGUMENT: Bad request
00899 * - star_v1_Status_STATUS_INTERNAL_ERROR: Processing error
00900 * @param[in] request_id Original request ID to echo (can be NULL)
00901 * - If NULL, request_id field left empty
00902 * - If provided, copied to header.request_id
00903 *
00904 * @pre header must not be nullptr
00905 * @post header populated with status and request_id (if provided)
00906 *
00907 * @note Thread-safe (no shared state)
00908 * @note request_id is copied, not referenced
00909 *
00910 * @par Example:
00911 * @code
00912 * star_v1_SetVelocityResponse response;
00913 * rx_nanopb_create_response_header(&response.header,
00914 *                                 star_v1_Status_STATUS_OK,
00915 *                                 request.header.request_id);
00916 * @endcode
00917 *
00918 * @since Version 1.0.0
00919 */
00920 void rx_nanopb_create_response_header(star_v1_ResponseHeader* header,
00921                                     star_v1_Status status,
00922                                     const char* request_id);
00923
00924 /*
=====
00925 * Test Helpers
00926 *
=====
00927 */
00928 /**
00929 * @brief Reset module state for unit testing
00930 *
00931 * @details
00932 * Resets the nanopb wrapper to uninitialized state. Used in unit tests
00933 * to verify initialization behavior and ensure clean state between tests.
00934 *
00935 * @warning FOR UNIT TESTING ONLY - Do not call in production code
00936 *
00937 * @pre None
00938 * @post Module state reset to uninitialized
00939 * @post rx_nanopb_init() must be called before encode/decode functions
00940 *
00941 * @note This function only exists in test builds
00942 *
00943 * @par Example - Test Setup:
00944 * @code
00945 * void setUp(void) {
00946 *     rx_nanopb_test_reset_state(); // Clean slate
00947 *     rx_nanopb_init();             // Re-initialize
00948 * }
00949 *
00950 * void test_encode_without_init(void) {
00951 *     rx_nanopb_test_reset_state(); // Don't call init
00952 *
00953 *     // Verify encode fails without initialization
00954 *     rx_err_t err = rx_nanopb_encode_telemetry(&msg, buf, 512, &len);
00955 *     TEST_ASSERT_EQUAL(k_rx_err_not_initialized, err);
00956 * }
00957 * @endcode
00958 *
00959 * @since Version 1.0.0
00960 */
00961 void rx_nanopb_test_reset_state(void);
00962
00963 #ifdef __cplusplus
00964 }
00965 #endif

```

6.51 libs/rx_nanopb/nanopb/extra/pb_syshdr.h File Reference

Macros

- `#define offsetof(st, m)`
- `#define NULL 0`
- `#define false 0`
- `#define true 1`
- `#define CHAR\_BIT 8`

Typedefs

- `typedef signed char int8\_t`
- `typedef unsigned char uint8\_t`
- `typedef signed short int16\_t`
- `typedef unsigned short uint16\_t`
- `typedef signed int int32\_t`
- `typedef unsigned int uint32\_t`
- `typedef signed long long int64\_t`
- `typedef unsigned long long uint64\_t`
- `typedef int8\_t int\_least8\_t`
- `typedef uint8\_t uint\_least8\_t`
- `typedef uint8\_t uint\_fast8\_t`
- `typedef int16\_t int\_least16\_t`
- `typedef uint16\_t uint\_least16\_t`
- `typedef uint32\_t size\_t`
- `typedef int bool`

Functions

- `static size\_t strlen (const char *s)`
- `static void * memcpy (void *s1, const void *s2, size\_t n)`
- `static void * memset (void *s, int c, size\_t n)`

6.51.1 Macro Definition Documentation

6.51.1.1 [CHAR_BIT](#)

```
#define CHAR\_BIT 8
```

Definition at line [117](#) of file [pb_syshdr.h](#).

6.51.1.2 [false](#)

```
#define false 0
```

Definition at line [58](#) of file [pb_syshdr.h](#).

6.51.1.3 NULL

```
#define NULL 0
```

Definition at line 46 of file [pb_syshdr.h](#).

Referenced by [buf_read\(\)](#), [decode_callback_field\(\)](#), [decode_extension\(\)](#), [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [encode_array\(\)](#), [encode_callback_field\(\)](#), [load_descriptor_values\(\)](#), [pb_check_proto3_default_v](#), [pb_close_string_substream\(\)](#), [pb_dec_submessage\(\)](#), [pb_decode_inner\(\)](#), [pb_decode_tag\(\)](#), [pb_default_field_callback\(\)](#), [pb_enc_bytes\(\)](#), [pb_enc_string\(\)](#), [pb_enc_submessage\(\)](#), [pb_encode_submessage\(\)](#), [pb_field_set_to_default\(\)](#), [pb_istream_from_buffer\(\)](#), [pb_ostream_from_buffer\(\)](#), [pb_read\(\)](#), [pb_skip_field\(\)](#), [pb_skip_string\(\)](#), and [pb_write\(\)](#).

6.51.1.4 offsetof

```
#define offsetof(  
    st,  
    m)
```

Value:

```
((size_t)(amp((st *)0)->m))
```

Definition at line 43 of file [pb_syshdr.h](#).

Referenced by [pb_enc_bytes\(\)](#).

6.51.1.5 true

```
#define true 1
```

Definition at line 59 of file [pb_syshdr.h](#).

6.51.2 Typedef Documentation

6.51.2.1 bool

```
typedef int bool
```

Definition at line 57 of file [pb_syshdr.h](#).

6.51.2.2 int16_t

```
typedef signed short int16\_t
```

Definition at line 21 of file [pb_syshdr.h](#).

6.51.2.3 int32_t

```
typedef signed int int32\_t
```

Definition at line 23 of file [pb_syshdr.h](#).

6.51.2.4 int64_t

typedef signed long long [int64_t](#)

Definition at line 25 of file [pb_syshdr.h](#).

6.51.2.5 int8_t

typedef signed char [int8_t](#)

Definition at line 19 of file [pb_syshdr.h](#).

6.51.2.6 int_least16_t

typedef [int16_t](#) [int_least16_t](#)

Definition at line 33 of file [pb_syshdr.h](#).

6.51.2.7 int_least8_t

typedef [int8_t](#) [int_least8_t](#)

Definition at line 30 of file [pb_syshdr.h](#).

6.51.2.8 size_t

typedef [uint32_t](#) [size_t](#)

Definition at line 42 of file [pb_syshdr.h](#).

6.51.2.9 uint16_t

typedef unsigned short [uint16_t](#)

Definition at line 22 of file [pb_syshdr.h](#).

6.51.2.10 uint32_t

typedef unsigned int [uint32_t](#)

Definition at line 24 of file [pb_syshdr.h](#).

6.51.2.11 uint64_t

typedef unsigned long long [uint64_t](#)

Definition at line 26 of file [pb_syshdr.h](#).

6.51.2.12 uint8_t

typedef unsigned char [uint8_t](#)

Definition at line 20 of file [pb_syshdr.h](#).

6.51.2.13 uint_fast8_t

typedef [uint8_t](#) [uint_fast8_t](#)

Definition at line 32 of file [pb_syshdr.h](#).

6.51.2.14 uint_least16_t

typedef [uint16_t](#) [uint_least16_t](#)

Definition at line 34 of file [pb_syshdr.h](#).

6.51.2.15 uint_least8_t

typedef [uint8_t](#) [uint_least8_t](#)

Definition at line 31 of file [pb_syshdr.h](#).

6.51.3 Function Documentation

6.51.3.1 memcpy()

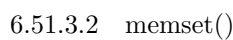
```
void * memcpy (  
    void * s1,  
    const void * s2,  
    size\_t n) [static]
```

Definition at line 90 of file [pb_syshdr.h](#).

```
00091 {  
00092     char * dest = (char *) s1;  
00093     const char * src = (const char *) s2;  
00094     while ( n-- )  
00095     {  
00096         *dest++ = *src++;  
00097     }  
00098     return s1;  
00099 }
```

Referenced by [buf_read\(\)](#), and [buf_write\(\)](#).

Here is the caller graph for this function:



Definition at line 101 of file pb_syshdr.h.

Referenced by `decode_static_field()`, `pb_dec_fixed_length_bytes()`, `pb_field_iter_begin()`, and `pb_field_set_to_default()`.

The diagram illustrates the state transitions for the 'decode' process. Key states and transitions include:

- Initial State:** `decode_partner_field` transitions to `decode_basic_field` on the `decode` action.
- Basic Field Decoding:** `decode_basic_field` transitions to `pb_decode_submessage` on the `pb_decode` action.
- Submessage Decoding:** `pb_decode_submessage` transitions to `pb_decode_extn` on the `pb_decode_extn` action.
- Extension Decoding:** `pb_decode_extn` transitions to `decode_extn` on the `decode_extn` action.
- Extension Encoding:** `decode_extn` transitions to `pb_encode_extn` on the `pb_encode_extn` action.
- Encoding States:** `pb_encode_extn` transitions to `pb_encode_submessage` on the `pb_encode` action.
- Final State:** `pb_encode_submessage` transitions to `pb_encode_extn` on the `pb_encode_extn` action.

The diagram also shows various intermediate states and transitions for handling submessages and extensions, such as `pb_decode_submessage`, `pb_decode_extn`, `decode_extn`, `pb_encode_extn`, and `pb_encode_submessage`.

6.51.3.3 strlen()

```
size_t strlen (
    const char * s) [static]
```

Definition at line 80 of file pb_syshdr.h.

```
00081 {
00082     size_t rc = 0;
00083     while ( s[rc] )
00084     {
00085         ++rc;
00086     }
00087     return rc;
00088 }
```

6.52 pb_syshdr.h

[Go to the documentation of this file.](#)

```
00001 /* This is an example of a header file for platforms/compilers that do
00002  * not come with stdint.h/stddef.h/stdbool.h/string.h. To use it, define
00003  * PB_SYSTEM_HEADER as "pb_syshdr.h", including the quotes, and add the
00004  * extra folder to your include path.
00005  *
00006  * It is very likely that you will need to customize this file to suit
00007  * your platform. For any compiler that supports C99, this file should
00008  * not be necessary.
00009  */
00010
00011 #ifndef __PB_SYSHDR_H_
00012 #define __PB_SYSHDR_H_
00013
00014 /* stdint.h subset */
00015 #ifdef HAVE_STDINT_H
00016 #include <stdint.h>
00017 #else
00018 /* You will need to modify these to match the word size of your platform. */
00019 typedef signed char int8_t;
00020 typedef unsigned char uint8_t;
00021 typedef signed short int16_t;
00022 typedef unsigned short uint16_t;
00023 typedef signed int int32_t;
00024 typedef unsigned int uint32_t;
00025 typedef signed long long int64_t;
00026 typedef unsigned long long uint64_t;
00027
00028 /* These are ok for most platforms, unless uint8_t is actually not available,
00029  * in which case you should give the smallest available type. */
00030 typedef int8_t int_least8_t;
00031 typedef uint8_t uint_least8_t;
00032 typedef int8_t int_fast8_t;
00033 typedef int16_t int_least16_t;
00034 typedef uint16_t uint_least16_t;
00035 #endif
00036
00037 /* stddef.h subset */
00038 #ifdef HAVE_STDDEF_H
00039 #include <stddef.h>
00040 #else
00041
00042 typedef uint32_t size_t;
00043 #define offsetof(st, m) ((size_t)(&((st *)0)->m))
00044
00045 #ifndef NULL
00046 #define NULL 0
00047 #endif
00048
00049 #endif
00050
00051 /* stdbool.h subset */
00052 #ifdef HAVE_STDBOOL_H
00053 #include <stdbool.h>
00054 #else
00055
00056 #ifndef __cplusplus
00057 typedef int bool;
00058 #define false 0
00059 #define true 1
00060 #endif
00061 #endif
```

```

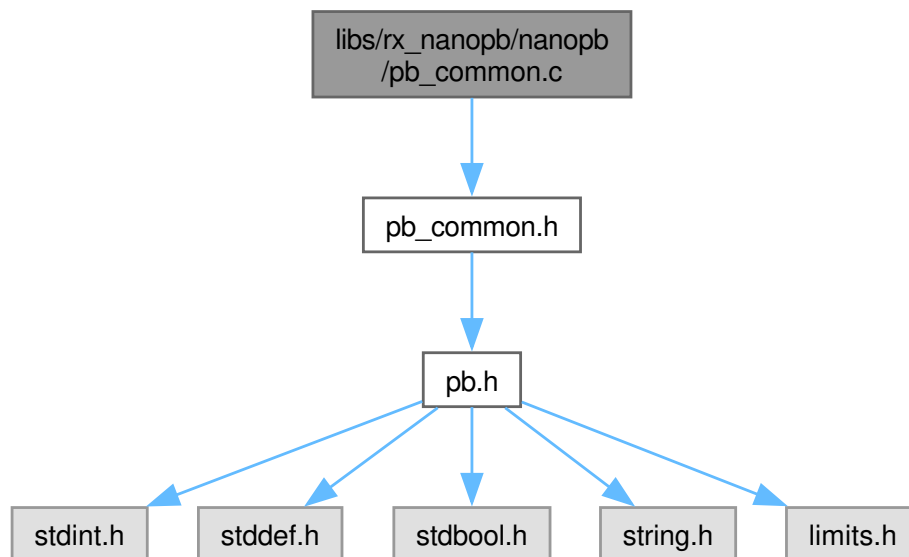
00060 #endif
00061
00062 #endif
00063
00064 /* stdlib.h subset */
00065 #ifdef PB_ENABLE_MALLOC
00066 #ifdef HAVE_STDLIB_H
00067 #include <stdlib.h>
00068 #else
00069 void *realloc(void *ptr, size_t size);
00070 void free(void *ptr);
00071 #endif
00072 #endif
00073
00074 /* string.h subset */
00075 #ifdef HAVE_STRING_H
00076 #include <string.h>
00077 #else
00078
00079 /* Implementations are from the Public Domain C Library (PDCLib). */
00080 static size_t strlen( const char * s )
00081 {
00082     size_t rc = 0;
00083     while ( s[rc] )
00084     {
00085         ++rc;
00086     }
00087     return rc;
00088 }
00089
00090 static void * memcpy( void *s1, const void *s2, size_t n )
00091 {
00092     char * dest = (char *) s1;
00093     const char * src = (const char *) s2;
00094     while ( n-- )
00095     {
00096         *dest++ = *src++;
00097     }
00098     return s1;
00099 }
00100
00101 static void * memset( void * s, int c, size_t n )
00102 {
00103     unsigned char * p = (unsigned char *) s;
00104     while ( n-- )
00105     {
00106         *p++ = (unsigned char) c;
00107     }
00108     return s;
00109 }
00110 #endif
00111
00112
00113 /* limits.h subset */
00114 #ifdef HAVE_LIMITS_H
00115 #include <limits.h>
00116 #else
00117 #define CHAR_BIT 8
00118 #endif
00119
00120 #endif

```

6.53 libs/rx_nanopb/nanopb/pb_common.c File Reference

#include "pb_common.h"

Include dependency graph for pb_common.c:



Functions

- static [bool load_descriptor_values](#) (pb_field_iter_t *iter)
- static void [advance_iterator](#) (pb_field_iter_t *iter)
- [bool pb_field_iter_begin](#) (pb_field_iter_t *iter, const pb_msgdesc_t *desc, void *message)
- [bool pb_field_iter_begin_extension](#) (pb_field_iter_t *iter, pb_extension_t *extension)
- [bool pb_field_iter_next](#) (pb_field_iter_t *iter)
- [bool pb_field_iter_find](#) (pb_field_iter_t *iter, [uint32_t](#) tag)
- [bool pb_field_iter_find_extension](#) (pb_field_iter_t *iter)
- static void * [pb_const_cast](#) (const void *p)
- [bool pb_field_iter_begin_const](#) (pb_field_iter_t *iter, const pb_msgdesc_t *desc, const void *message)
- [bool pb_field_iter_begin_extension_const](#) (pb_field_iter_t *iter, const pb_extension_t *extension)
- [bool pb_default_field_callback](#) (pb_istream_t *istream, pb_ostream_t *ostream, const pb_field_t *field)

6.53.1 Function Documentation

6.53.1.1 [advance_iterator\(\)](#)

```
void advance_iterator (
    pb_field_iter_t * iter) [static]
```

Definition at line 122 of file [pb_common.c](#).

```

00123 {
00124     iter->index++;
00125     if (iter->index >= iter->descriptor->field_count)
00126     {
00127         /* Restart */
00128         iter->index = 0;
00129         iter->field_info_index = 0;
00130         iter->submessage_index = 0;
00131         iter->required_field_index = 0;
00132     }
00133     else
00134     {
00135         /* Increment indexes based on previous field type.
00136          * All field info formats have the following fields:
00137
```



```

00029     break;
00030 }
00031
00032 case 1: {
00033     /* 2-word format */
00034     uint32_t word1 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 1]);
00035
00036     iter->array_size = (pb_size_t)((word0 » 16) & 0x0FFF);
00037     iter->tag = (pb_size_t)((((word0 » 2) & 0x3F) | ((word1 » 28) « 6)));
00038     size_offset = (int_least8_t)((word0 » 28) & 0x0F);
00039     data_offset = word1 & 0xFFFF;
00040     iter->data_size = (pb_size_t)((word1 » 16) & 0x0FFF);
00041     break;
00042 }
00043
00044 case 2: {
00045     /* 4-word format */
00046     uint32_t word1 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 1]);
00047     uint32_t word2 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 2]);
00048     uint32_t word3 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 3]);
00049
00050     iter->array_size = (pb_size_t)(word0 » 16);
00051     iter->tag = (pb_size_t)((((word0 » 2) & 0x3F) | ((word1 » 8) « 6)));
00052     size_offset = (int_least8_t)(word1 & 0xFF);
00053     data_offset = word2;
00054     iter->data_size = (pb_size_t)word3;
00055     break;
00056 }
00057
00058 default: {
00059     /* 8-word format */
00060     uint32_t word1 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 1]);
00061     uint32_t word2 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 2]);
00062     uint32_t word3 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 3]);
00063     uint32_t word4 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 4]);
00064
00065     iter->array_size = (pb_size_t)word4;
00066     iter->tag = (pb_size_t)((((word0 » 2) & 0x3F) | ((word1 » 8) « 6)));
00067     size_offset = (int_least8_t)(word1 & 0xFF);
00068     data_offset = word2;
00069     iter->data_size = (pb_size_t)word3;
00070     break;
00071 }
00072 }
00073
00074 if (iter->message)
00075 {
00076     /* Avoid doing arithmetic on null pointers, it is undefined */
00077     iter->pField = NULL;
00078     iter->pSize = NULL;
00079 }
00080 else
00081 {
00082     iter->pField = (char*)iter->message + data_offset;
00083
00084     if (size_offset)
00085     {
00086         iter->pSize = (char*)iter->pField - size_offset;
00087     }
00088     else if (PB_HTYPE(iter->type) == PB_HTYPE_REPEATED &&
00089              (PB_ATYPE(iter->type) == PB_ATYPE_STATIC ||
00090               PB_ATYPE(iter->type) == PB_ATYPE_POINTER))
00091     {
00092         /* Fixed count array */
00093         iter->pSize = &iter->array_size;
00094     }
00095     else
00096     {
00097         iter->pSize = NULL;
00098     }
00099
00100     if (PB_ATYPE(iter->type) == PB_ATYPE_POINTER && iter->pField != NULL)
00101     {
00102         iter->pData = *(void**)iter->pField;
00103     }
00104     else
00105     {
00106         iter->pData = iter->pField;
00107     }
00108 }
00109
00110 if (PB_LTYPE_IS_SUBMSG(iter->type))
00111 {
00112     iter->submsg_desc = iter->descriptor->submsg_info[iter->submessage_index];
00113 }
00114 else
00115 {

```


6.53.1.4 pb_default_field_callback()

```
bool pb_default_field_callback (
    pb_istream_t * istream,
    pb_ostream_t * ostream,
    const pb_field_t * field)
```

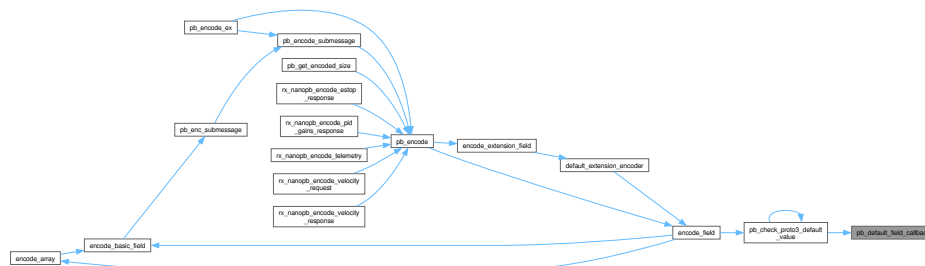
Definition at line 300 of file [pb_common.c](#).

```
00301 {
00302     if (field->data_size == sizeof(pb_callback_t))
00303     {
00304         pb_callback_t *pCallback = (pb_callback_t*)field->pData;
00305
00306         if (pCallback != NULL)
00307         {
00308             if (istream != NULL && pCallback->funcs.decode != NULL)
00309             {
00310                 return pCallback->funcs.decode(istream, field, &pCallback->arg);
00311             }
00312
00313             if (ostream != NULL && pCallback->funcs.encode != NULL)
00314             {
00315                 return pCallback->funcs.encode(ostream, field, &pCallback->arg);
00316             }
00317         }
00318     }
00319     return true; /* Success, but didn't do anything */
00320 }
00321
00322 }
```

References [NULL](#).

Referenced by [pb_check_proto3_default_value\(\)](#).

Here is the caller graph for this function:



6.53.1.5 pb_field_iter_begin()

```
bool pb_field_iter_begin (
    pb_field_iter_t * iter,
    const pb_msgdesc_t * desc,
    void * message)
```

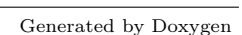
Definition at line 156 of file [pb_common.c](#).

```
00157 {
00158     memset(iter, 0, sizeof(*iter));
00159
00160     iter->descriptor = desc;
00161     iter->message = message;
00162
00163     return load_descriptor_values(iter);
00164 }
```

References [load_descriptor_values\(\)](#), and [memset\(\)](#).

Referenced by [decode_static_field\(\)](#), [pb_check_proto3_default_value\(\)](#), [pb_decode_inner\(\)](#), [pb_field_iter_begin_const\(\)](#), [pb_field_iter_begin_extension\(\)](#), and [pb_field_set_to_default\(\)](#).

Here is the call graph for this function:




```
bool pb_field_iter_begin_extension (
    pb_field_iter_t * iter,
    pb_extension_t * extension)
```

```

00167 {
00168     const pb_msgdesc_t *msg = (const pb_msgdesc_t*)extension->type->arg;
00169     bool status;
00170
00171     uint32_t word0 = PB_PROGMEM_READU32(msg->field_info[0]);
00172     if (PB_ATYPE(word0 » 8) == PB_ATYPE_POINTER)
00173     {
00174         /* For pointer extensions, the pointer is stored directly
00175          * in the extension structure. This avoids having an extra
00176          * indirection. */
00177         status = pb_field_iter_begin(iter, msg, &extension->dest);
00178     }
00179     else
00180     {
00181         status = pb_field_iter_begin(iter, msg, extension->dest);
00182     }
00183
00184     iter->pSize = &extension->found;
00185     return status;
00186 }

```

Referenced by `default_extension_decoder()`, `pb_field_iter_begin_extension_const()`, and `pb_field_set_to_default()`.

```
graph LR; A[pb_field_iter_begin_extension] --> B[pb_field_iter_begin]; B --> C[load_descriptor_values]; B --> D[memset];
```

6.53.1.8 pb_field_iter_begin_extension_const()

```
bool pb_field_iter_begin_extension_const (
    pb_field_iter_t * iter,
    const pb_extension_t * extension)
```

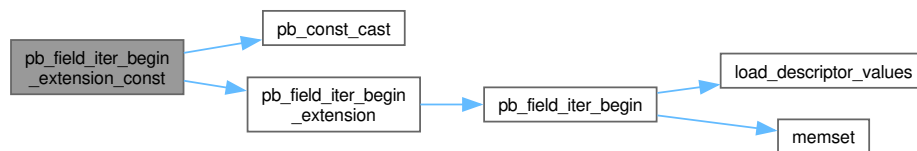
Definition at line 295 of file [pb_common.c](#).

```
00296 {
00297     return pb_field_iter_begin_extension(iter, (pb_extension_t*)pb_const_cast(extension));
00298 }
```

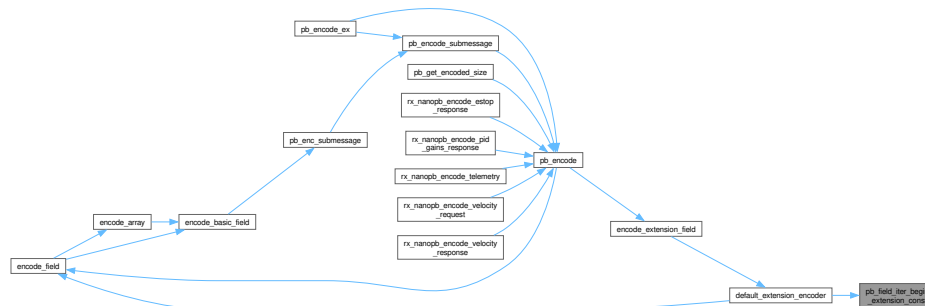
References [pb_const_cast\(\)](#), and [pb_field_iter_begin_extension\(\)](#).

Referenced by [default_extension_encoder\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.53.1.9 pb_field_iter_find()

```
bool pb_field_iter_find (
    pb_field_iter_t * iter,
    uint32_t tag)
```

Definition at line 195 of file [pb_common.c](#).

```
00196 {
00197     if (iter->tag == tag)
00198     {
00199         return true; /* Nothing to do, correct field already. */
00200     }
00201     else if (tag > iter->descriptor->largest_tag)
00202     {
00203         return false;
00204     }
00205     else
00206     {
00207         pb_size_t start = iter->index;
00208         uint32_t fieldinfo;
00209         if (tag < iter->tag)
00210         {

```

```

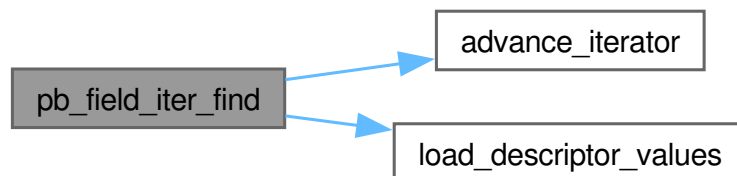
00212     /* Fields are in tag number order, so we know that tag is between
00213     * 0 and our start position. Setting index to end forces
00214     * advance_iterator() call below to restart from beginning. */
00215     iter->index = iter->descriptor->field_count;
00216 }
00217
00218 do
00219 {
00220     /* Advance iterator but don't load values yet */
00221     advance_iterator(iter);
00222
00223     /* Do fast check for tag number match */
00224     fieldinfo = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index]);
00225
00226     if (((fieldinfo >> 2) & 0x3F) == (tag & 0x3F))
00227     {
00228         /* Good candidate, check further */
00229         (void)load_descriptor_values(iter);
00230
00231         if (iter->tag == tag &&
00232             PB_LTYPE(iter->type) != PB_LTYPE_EXTENSION)
00233         {
00234             /* Found it */
00235             return true;
00236         }
00237     }
00238 } while (iter->index != start);
00239
00240 /* Searched all the way back to start, and found nothing. */
00241 (void)load_descriptor_values(iter);
00242 return false;
00243 }
00244 }

```

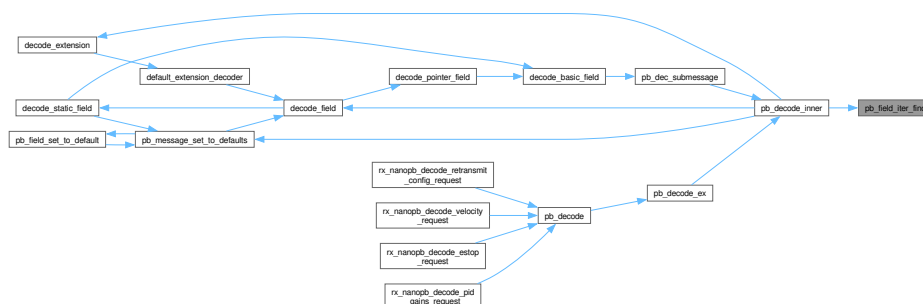
References [advance_iterator\(\)](#), [load_descriptor_values\(\)](#), [PB_LTYPE](#), [PB_LTYPE_EXTENSION](#), and [PB_PROGMEM_READU32](#).

Referenced by [pb_decode_inner\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.53.1.10 pb_field_iter_find_extension()

```
bool pb_field_iter_find_extension (
    pb_field_iter_t * iter)
```

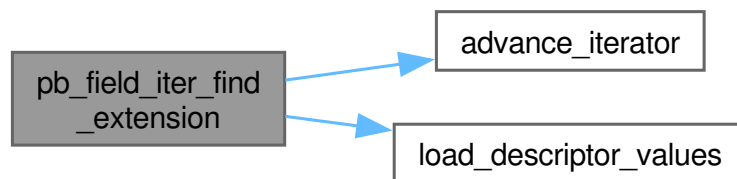
Definition at line 246 of file [pb_common.c](#).

```
00247 {
00248     if (PB_LTYPE(iter->type) == PB_LTYPE_EXTENSION)
00249     {
00250         return true;
00251     }
00252     else
00253     {
00254         pb_size_t start = iter->index;
00255         uint32_t fieldinfo;
00256
00257         do
00258         {
00259             /* Advance iterator but don't load values yet */
00260             advance_iterator(iter);
00261
00262             /* Do fast check for field type */
00263             fieldinfo = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index]);
00264
00265             if (PB_LTYPE((fieldinfo » 8) & 0xFF) == PB_LTYPE_EXTENSION)
00266             {
00267                 return load_descriptor_values(iter);
00268             }
00269         } while (iter->index != start);
00270
00271         /* Searched all the way back to start, and found nothing. */
00272         (void)load_descriptor_values(iter);
00273         return false;
00274     }
00275 }
```

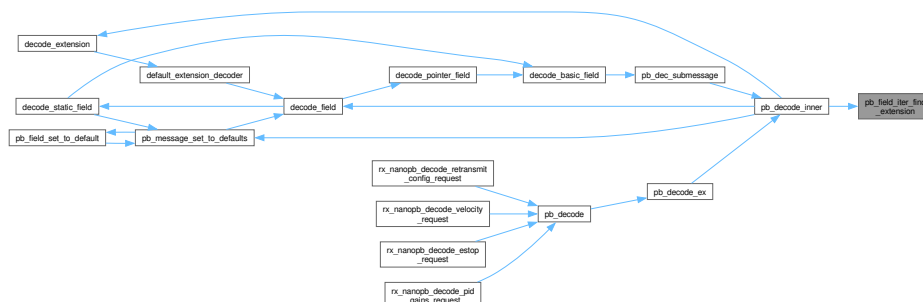
References [advance_iterator\(\)](#), [load_descriptor_values\(\)](#), [PB_LTYPE](#), [PB_LTYPE_EXTENSION](#), and [PB_PROGMEM_READU32](#).

Referenced by [pb_decode_inner\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:




```

00015     return false;
00016
00017     word0 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index]);
00018     iter->type = (pb_type_t)((word0 » 8) & 0xFF);
00019
00020     switch(word0 & 3)
00021     {
00022     case 0: {
00023         /* 1-word format */
00024         iter->array_size = 1;
00025         iter->tag = (pb_size_t)((word0 » 2) & 0x3F);
00026         size_offset = (int_least8_t)((word0 » 24) & 0x0F);
00027         data_offset = (word0 » 16) & 0xFF;
00028         iter->data_size = (pb_size_t)((word0 » 28) & 0x0F);
00029         break;
00030     }
00031
00032     case 1: {
00033         /* 2-word format */
00034         uint32_t word1 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 1]);
00035
00036         iter->array_size = (pb_size_t)((word0 » 16) & 0xFFFF);
00037         iter->tag = (pb_size_t)(((word0 » 2) & 0x3F) | ((word1 » 28) « 6));
00038         size_offset = (int_least8_t)((word0 » 28) & 0x0F);
00039         data_offset = word1 & 0xFFFF;
00040         iter->data_size = (pb_size_t)((word1 » 16) & 0xFFFF);
00041         break;
00042     }
00043
00044     case 2: {
00045         /* 4-word format */
00046         uint32_t word1 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 1]);
00047         uint32_t word2 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 2]);
00048         uint32_t word3 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 3]);
00049
00050         iter->array_size = (pb_size_t)(word0 » 16);
00051         iter->tag = (pb_size_t)(((word0 » 2) & 0x3F) | ((word1 » 8) « 6));
00052         size_offset = (int_least8_t)(word1 & 0xFF);
00053         data_offset = word2;
00054         iter->data_size = (pb_size_t)word3;
00055         break;
00056     }
00057
00058     default: {
00059         /* 8-word format */
00060         uint32_t word1 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 1]);
00061         uint32_t word2 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 2]);
00062         uint32_t word3 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 3]);
00063         uint32_t word4 = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index + 4]);
00064
00065         iter->array_size = (pb_size_t)word4;
00066         iter->tag = (pb_size_t)(((word0 » 2) & 0x3F) | ((word1 » 8) « 6));
00067         size_offset = (int_least8_t)(word1 & 0xFF);
00068         data_offset = word2;
00069         iter->data_size = (pb_size_t)word3;
00070         break;
00071     }
00072 }
00073
00074 if (iter->message)
00075 {
00076     /* Avoid doing arithmetic on null pointers, it is undefined */
00077     iter->pField = NULL;
00078     iter->pSize = NULL;
00079 }
00080 else
00081 {
00082     iter->pField = (char*)iter->message + data_offset;
00083
00084     if (size_offset)
00085     {
00086         iter->pSize = (char*)iter->pField - size_offset;
00087     }
00088     else if (PB_HTYPE(iter->type) == PB_HTYPE_REPEATED &&
00089              (PB_ATYPE(iter->type) == PB_ATYPE_STATIC ||
00090               PB_ATYPE(iter->type) == PB_ATYPE_POINTER))
00091     {
00092         /* Fixed count array */
00093         iter->pSize = &iter->array_size;
00094     }
00095     else
00096     {
00097         iter->pSize = NULL;
00098     }
00099
00100     if (PB_ATYPE(iter->type) == PB_ATYPE_POINTER && iter->pField != NULL)
00101     {

```

```

00102     iter->pData = *(void**)iter->pField;
00103 }
00104 else
00105 {
00106     iter->pData = iter->pField;
00107 }
00108 }
00109
00110 if (PB_LTYPE_IS_SUBMSG(iter->type))
00111 {
00112     iter->submsg_desc = iter->descriptor->submsg_info[iter->submessage_index];
00113 }
00114 else
00115 {
00116     iter->submsg_desc = NULL;
00117 }
00118
00119 return true;
00120 }
00121
00122 static void advance_iterator(pb_field_iter_t *iter)
00123 {
00124     iter->index++;
00125
00126     if (iter->index >= iter->descriptor->field_count)
00127     {
00128         /* Restart */
00129         iter->index = 0;
00130         iter->field_info_index = 0;
00131         iter->submessage_index = 0;
00132         iter->required_field_index = 0;
00133     }
00134     else
00135     {
00136         /* Increment indexes based on previous field type.
00137          * All field info formats have the following fields:
00138          * - lowest 2 bits tell the amount of words in the descriptor (2^n words)
00139          * - bits 2..7 give the lowest bits of tag number.
00140          * - bits 8..15 give the field type.
00141          */
00142         uint32_t prev_descriptor = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index]);
00143         pb_type_t prev_type = (prev_descriptor » 8) & 0xFF;
00144         pb_size_t descriptor_len = (pb_size_t)(1 « (prev_descriptor & 3));
00145
00146         /* Add to fields.
00147          * The cast to pb_size_t is needed to avoid -Wconversion warning.
00148          * Because the data is constants from generator, there is no danger of overflow.
00149          */
00150         iter->field_info_index = (pb_size_t)(iter->field_info_index + descriptor_len);
00151         iter->required_field_index = (pb_size_t)(iter->required_field_index + (PB_HTYPE(prev_type) ==
PB_HTYPE_REQUIRED));
00152         iter->submessage_index = (pb_size_t)(iter->submessage_index + PB_LTYPE_IS_SUBMSG(prev_type));
00153     }
00154 }
00155
00156 bool pb_field_iter_begin(pb_field_iter_t *iter, const pb_msgdesc_t *desc, void *message)
00157 {
00158     memset(iter, 0, sizeof(*iter));
00159
00160     iter->descriptor = desc;
00161     iter->message = message;
00162
00163     return load_descriptor_values(iter);
00164 }
00165
00166 bool pb_field_iter_begin_extension(pb_field_iter_t *iter, pb_extension_t *extension)
00167 {
00168     const pb_msgdesc_t *msg = (const pb_msgdesc_t*)extension->type->arg;
00169     bool status;
00170
00171     uint32_t word0 = PB_PROGMEM_READU32(msg->field_info[0]);
00172     if (PB_ATYPE(word0 » 8) == PB_ATYPE_POINTER)
00173     {
00174         /* For pointer extensions, the pointer is stored directly
00175          * in the extension structure. This avoids having an extra
00176          * indirection. */
00177         status = pb_field_iter_begin(iter, msg, &extension->dest);
00178     }
00179     else
00180     {
00181         status = pb_field_iter_begin(iter, msg, extension->dest);
00182     }
00183
00184     iter->pSize = &extension->found;
00185     return status;
00186 }
00187

```

```

00188 bool pb_field_iter_next(pb_field_iter_t *iter)
00189 {
00190     advance_iterator(iter);
00191     (void)load_descriptor_values(iter);
00192     return iter->index != 0;
00193 }
00194
00195 bool pb_field_iter_find(pb_field_iter_t *iter, uint32_t tag)
00196 {
00197     if (iter->tag == tag)
00198     {
00199         return true; /* Nothing to do, correct field already. */
00200     }
00201     else if (tag > iter->descriptor->largest_tag)
00202     {
00203         return false;
00204     }
00205     else
00206     {
00207         pb_size_t start = iter->index;
00208         uint32_t fieldinfo;
00209
00210         if (tag < iter->tag)
00211         {
00212             /* Fields are in tag number order, so we know that tag is between
00213              * 0 and our start position. Setting index to end forces
00214              * advance_iterator() call below to restart from beginning. */
00215             iter->index = iter->descriptor->field_count;
00216         }
00217
00218         do
00219         {
00220             /* Advance iterator but don't load values yet */
00221             advance_iterator(iter);
00222
00223             /* Do fast check for tag number match */
00224             fieldinfo = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index]);
00225
00226             if (((fieldinfo » 2) & 0x3F) == (tag & 0x3F))
00227             {
00228                 /* Good candidate, check further */
00229                 (void)load_descriptor_values(iter);
00230
00231                 if (iter->tag == tag &&
00232                     PB_LTYPE(iter->type) != PB_LTYPE_EXTENSION)
00233                 {
00234                     /* Found it */
00235                     return true;
00236                 }
00237             }
00238             } while (iter->index != start);
00239
00240             /* Searched all the way back to start, and found nothing. */
00241             (void)load_descriptor_values(iter);
00242             return false;
00243         }
00244     }
00245
00246 bool pb_field_iter_find_extension(pb_field_iter_t *iter)
00247 {
00248     if (PB_LTYPE(iter->type) == PB_LTYPE_EXTENSION)
00249     {
00250         return true;
00251     }
00252     else
00253     {
00254         pb_size_t start = iter->index;
00255         uint32_t fieldinfo;
00256
00257         do
00258         {
00259             /* Advance iterator but don't load values yet */
00260             advance_iterator(iter);
00261
00262             /* Do fast check for field type */
00263             fieldinfo = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index]);
00264
00265             if (PB_LTYPE((fieldinfo » 8) & 0xFF) == PB_LTYPE_EXTENSION)
00266             {
00267                 return load_descriptor_values(iter);
00268             }
00269             } while (iter->index != start);
00270
00271             /* Searched all the way back to start, and found nothing. */
00272             (void)load_descriptor_values(iter);
00273             return false;
00274         }
00275     }

```



```

00275 }
00276
00277 static void *pb_const_cast(const void *p)
00278 {
00279     /* Note: this casts away const, in order to use the common field iterator
00280      * logic for both encoding and decoding. The cast is done using union
00281      * to avoid spurious compiler warnings. */
00282     union {
00283         void *p1;
00284         const void *p2;
00285     } t;
00286     t.p2 = p;
00287     return t.p1;
00288 }
00289
00290 bool pb_field_iter_begin_const(pb_field_iter_t *iter, const pb_msgdesc_t *desc, const void *message)
00291 {
00292     return pb_field_iter_begin(iter, desc, pb_const_cast(message));
00293 }
00294
00295 bool pb_field_iter_begin_extension_const(pb_field_iter_t *iter, const pb_extension_t *extension)
00296 {
00297     return pb_field_iter_begin_extension(iter, (pb_extension_t*)pb_const_cast(extension));
00298 }
00299
00300 bool pb_default_field_callback(pb_istream_t *istream, pb_ostream_t *ostream, const pb_field_t *field)
00301 {
00302     if (field->data_size == sizeof(pb_callback_t))
00303     {
00304         pb_callback_t *pCallback = (pb_callback_t*)field->pData;
00305
00306         if (pCallback != NULL)
00307         {
00308             if (istream != NULL && pCallback->funcs.decode != NULL)
00309             {
00310                 return pCallback->funcs.decode(istream, field, &pCallback->arg);
00311             }
00312
00313             if (ostream != NULL && pCallback->funcs.encode != NULL)
00314             {
00315                 return pCallback->funcs.encode(ostream, field, &pCallback->arg);
00316             }
00317         }
00318     }
00319
00320     return true; /* Success, but didn't do anything */
00321 }
00322 }
00323
00324 #ifdef PB_VALIDATE_UTF8
00325
00326 /* This function checks whether a string is valid UTF-8 text.
00327 *
00328 * Algorithm is adapted from https://www.cl.cam.ac.uk/~mgk25/ucs/utf8_check.c
00329 * Original copyright: Markus Kuhn <http://www.cl.cam.ac.uk/~mgk25/> 2005-03-30
00330 * Licensed under "Short code license", which allows use under MIT license or
00331 * any compatible with it.
00332 */
00333
00334 bool pb_validate_utf8(const char *str)
00335 {
00336     const pb_byte_t *s = (const pb_byte_t*)str;
00337     while (*s)
00338     {
00339         if (*s < 0x80)
00340         {
00341             /* 0xxxxxxx */
00342             s++;
00343         }
00344         else if ((s[0] & 0xe0) == 0xc0)
00345         {
00346             /* 110XXXXx 10xxxxxx */
00347             if ((s[1] & 0xc0) != 0x80 ||
00348                 (s[0] & 0xfe) == 0xc0) /* overlong? */
00349                 return false;
00350             else
00351                 s += 2;
00352         }
00353         else if ((s[0] & 0xf0) == 0xe0)
00354         {
00355             /* 1110XXXX 10XXXXxx 10xxxxxx */
00356             if ((s[1] & 0xc0) != 0x80 ||
00357                 (s[2] & 0xc0) != 0x80 ||
00358                 (s[0] == 0xe0 && (s[1] & 0xe0) == 0x80) || /* overlong? */
00359                 (s[0] == 0xed && (s[1] & 0xe0) == 0xa0) || /* surrogate? */
00360                 (s[0] == 0xef && s[1] == 0xbf &&
00361                  (s[2] & 0xfe) == 0xbe)) /* U+FFFE or U+FFFF? */

```

```

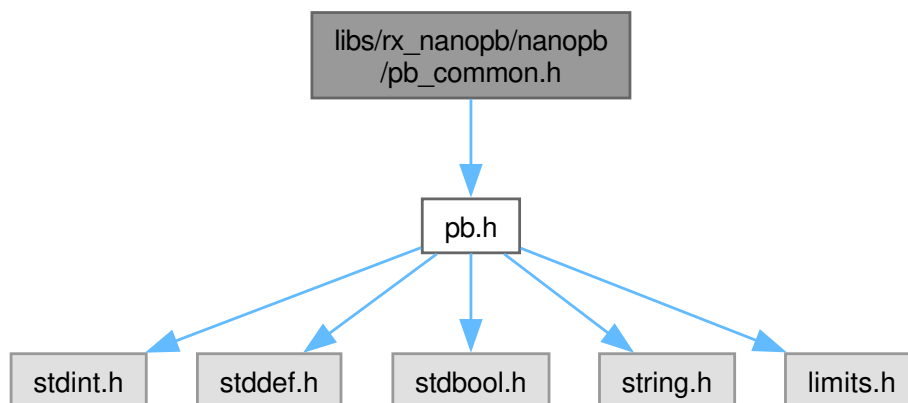
00362         return false;
00363     else
00364         s += 3;
00365     }
00366     else if ((s[0] & 0xf8) == 0xf0)
00367     {
00368         /* 11110XXX 10XXxxxx 10xxxxxx 10xxxxxx */
00369         if ((s[1] & 0xc0) != 0x80 ||
00370             (s[2] & 0xc0) != 0x80 ||
00371             (s[3] & 0xc0) != 0x80 ||
00372             (s[0] == 0xf0 && (s[1] & 0xf0) == 0x80) || /* overlong? */
00373             (s[0] == 0xf4 && s[1] > 0x8f) || s[0] > 0xf4) /* > U+10FFFF? */
00374             return false;
00375     else
00376         s += 4;
00377     }
00378     else
00379     {
00380         return false;
00381     }
00382 }
00383
00384 return true;
00385 }
00386
00387 #endif
00388

```

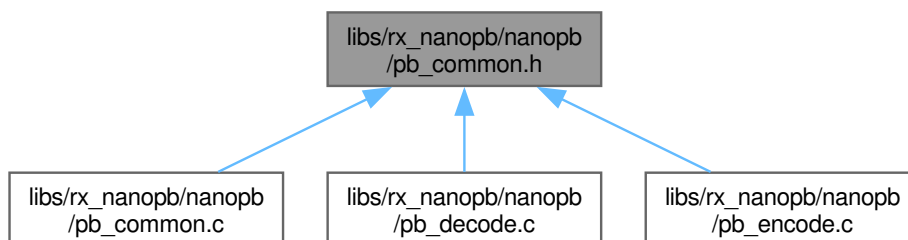
6.55 libs/rx_nanopb/nanopb/pb_common.h File Reference

#include "pb.h"

Include dependency graph for pb_common.h:



This graph shows which files directly or indirectly include this file:



Functions

- [bool pb_field_iter_begin](#) (pb_field_iter_t *iter, const pb_msgdesc_t *desc, void *message)
- [bool pb_field_iter_begin_extension](#) (pb_field_iter_t *iter, pb_extension_t *extension)
- [bool pb_field_iter_begin_const](#) (pb_field_iter_t *iter, const pb_msgdesc_t *desc, const void *message)
- [bool pb_field_iter_begin_extension_const](#) (pb_field_iter_t *iter, const pb_extension_t *extension)
- [bool pb_field_iter_next](#) (pb_field_iter_t *iter)
- [bool pb_field_iter_find](#) (pb_field_iter_t *iter, uint32_t tag)
- [bool pb_field_iter_find_extension](#) (pb_field_iter_t *iter)

6.55.1 Function Documentation

6.55.1.1 pb_field_iter_begin()

```
bool pb_field_iter_begin (
    pb_field_iter_t * iter,
    const pb_msgdesc_t * desc,
    void * message)
```

Definition at line 156 of file [pb_common.c](#).

```
00157 {
00158     memset(iter, 0, sizeof(*iter));
00159
00160     iter->descriptor = desc;
00161     iter->message = message;
00162
00163     return load_descriptor_values(iter);
00164 }
```

6.55.1.2 pb_field_iter_begin_const()

```
bool pb_field_iter_begin_const (
    pb_field_iter_t * iter,
    const pb_msgdesc_t * desc,
    const void * message)
```

Definition at line 290 of file [pb_common.c](#).

```
00291 {
00292     return pb_field_iter_begin(iter, desc, pb_const_cast(message));
00293 }
```

6.55.1.3 pb_field_iter_begin_extension()

```
bool pb_field_iter_begin_extension (
    pb_field_iter_t * iter,
    pb_extension_t * extension)
```

Definition at line 166 of file [pb_common.c](#).

```
00167 {
00168     const pb_msgdesc_t *msg = (const pb_msgdesc_t*)extension->type->arg;
00169     bool status;
00170
00171     uint32_t word0 = PB_PROGMEM_READU32(msg->field_info[0]);
00172     if (PB_ATYPE(word0 » 8) == PB_ATYPE_POINTER)
00173     {
00174         /* For pointer extensions, the pointer is stored directly
```

```

00175     * in the extension structure. This avoids having an extra
00176     * indirection. */
00177     status = pb_field_iter_begin(iter, msg, &extension->dest);
00178 }
00179 else
00180 {
00181     status = pb_field_iter_begin(iter, msg, extension->dest);
00182 }
00183
00184 iter->pSize = &extension->found;
00185 return status;
00186 }

```

6.55.1.4 pb_field_iter_begin_extension_const()

```

bool pb_field_iter_begin_extension_const (
    pb_field_iter_t * iter,
    const pb_extension_t * extension)

```

Definition at line 295 of file [pb_common.c](#).

```

00296 {
00297     return pb_field_iter_begin_extension(iter, (pb_extension_t*)pb_const_cast(extension));
00298 }

```

6.55.1.5 pb_field_iter_find()

```

bool pb_field_iter_find (
    pb_field_iter_t * iter,
    uint32_t tag)

```

Definition at line 195 of file [pb_common.c](#).

```

00196 {
00197     if (iter->tag == tag)
00198     {
00199         return true; /* Nothing to do, correct field already. */
00200     }
00201     else if (tag > iter->descriptor->largest_tag)
00202     {
00203         return false;
00204     }
00205     else
00206     {
00207         pb_size_t start = iter->index;
00208         uint32_t fieldinfo;
00209
00210         if (tag < iter->tag)
00211         {
00212             /* Fields are in tag number order, so we know that tag is between
00213              * 0 and our start position. Setting index to end forces
00214              * advance_iterator() call below to restart from beginning. */
00215             iter->index = iter->descriptor->field_count;
00216         }
00217
00218         do
00219         {
00220             /* Advance iterator but don't load values yet */
00221             advance_iterator(iter);
00222
00223             /* Do fast check for tag number match */
00224             fieldinfo = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index]);
00225
00226             if (((fieldinfo » 2) & 0x3F) == (tag & 0x3F))
00227             {
00228                 /* Good candidate, check further */
00229                 (void)load_descriptor_values(iter);
00230
00231                 if (iter->tag == tag &&
00232                     PB_LTYPE(iter->type) != PB_LTYPE_EXTENSION)
00233                 {
00234                     /* Found it */
00235                     return true;
00236                 }
00237             }
00238         } while (1);
00239     }
00240 }

```

```

00238     } while (iter->index != start);
00239
00240     /* Searched all the way back to start, and found nothing. */
00241     (void)load_descriptor_values(iter);
00242     return false;
00243 }
00244 }

```

6.55.1.6 pb_field_iter_find_extension()

```

bool pb_field_iter_find_extension (
    pb_field_iter_t * iter)

```

Definition at line 246 of file [pb_common.c](#).

```

00247 {
00248     if (PB_LTYPE(iter->type) == PB_LTYPE_EXTENSION)
00249     {
00250         return true;
00251     }
00252     else
00253     {
00254         pb_size_t start = iter->index;
00255         uint32_t fieldinfo;
00256
00257         do
00258         {
00259             /* Advance iterator but don't load values yet */
00260             advance_iterator(iter);
00261
00262             /* Do fast check for field type */
00263             fieldinfo = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index]);
00264
00265             if (PB_LTYPE((fieldinfo » 8) & 0xFF) == PB_LTYPE_EXTENSION)
00266             {
00267                 return load_descriptor_values(iter);
00268             }
00269         } while (iter->index != start);
00270
00271         /* Searched all the way back to start, and found nothing. */
00272         (void)load_descriptor_values(iter);
00273         return false;
00274     }
00275 }

```

6.55.1.7 pb_field_iter_next()

```

bool pb_field_iter_next (
    pb_field_iter_t * iter)

```

Definition at line 188 of file [pb_common.c](#).

```

00189 {
00190     advance_iterator(iter);
00191     (void)load_descriptor_values(iter);
00192     return iter->index != 0;
00193 }

```

6.56 pb_common.h

[Go to the documentation of this file.](#)

```

00001 /* pb_common.h: Common support functions for pb_encode.c and pb_decode.c.
00002  * These functions are rarely needed by applications directly.
00003  */
00004
00005 #ifndef PB_COMMON_H_INCLUDED
00006 #define PB_COMMON_H_INCLUDED
00007
00008 #include "pb.h"
00009

```

```

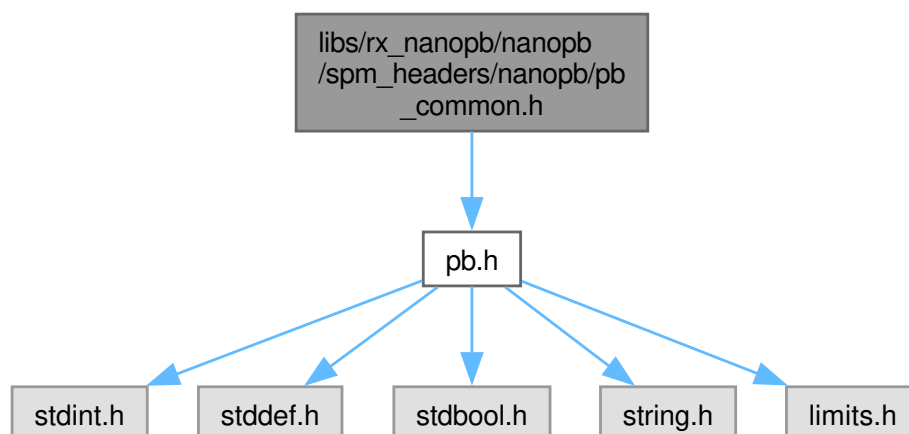
00010 #ifdef __cplusplus
00011 extern "C" {
00012 #endif
00013
00014 /* Initialize the field iterator structure to beginning.
00015  * Returns false if the message type is empty. */
00016 bool pb_field_iter_begin(pb_field_iter_t *iter, const pb_msgdesc_t *desc, void *message);
00017
00018 /* Get a field iterator for extension field. */
00019 bool pb_field_iter_begin_extension(pb_field_iter_t *iter, pb_extension_t *extension);
00020
00021 /* Same as pb_field_iter_begin(), but for const message pointer.
00022  * Note that the pointers in pb_field_iter_t will be non-const but shouldn't
00023  * be written to when using these functions. */
00024 bool pb_field_iter_begin_const(pb_field_iter_t *iter, const pb_msgdesc_t *desc, const void *message);
00025 bool pb_field_iter_begin_extension_const(pb_field_iter_t *iter, const pb_extension_t *extension);
00026
00027 /* Advance the iterator to the next field.
00028  * Returns false when the iterator wraps back to the first field. */
00029 bool pb_field_iter_next(pb_field_iter_t *iter);
00030
00031 /* Advance the iterator until it points at a field with the given tag.
00032  * Returns false if no such field exists. */
00033 bool pb_field_iter_find(pb_field_iter_t *iter, uint32_t tag);
00034
00035 /* Find a field with type PB_LTYPE_EXTENSION, or return false if not found.
00036  * There can be only one extension range field per message. */
00037 bool pb_field_iter_find_extension(pb_field_iter_t *iter);
00038
00039 #ifdef PB_VALIDATE_UTF8
00040 /* Validate UTF-8 text string */
00041 bool pb_validate_utf8(const char *s);
00042 #endif
00043
00044 #ifdef __cplusplus
00045 } /* extern "C" */
00046 #endif
00047
00048 #endif
00049

```

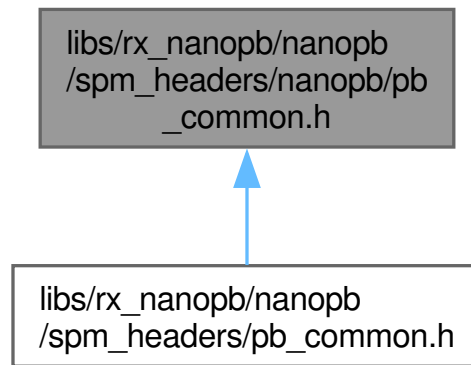
6.57 libs/rx_nanopb/nanopb/spm_headers/nanopb/pb_common.h File Reference

#include "pb.h"

Include dependency graph for pb_common.h:



This graph shows which files directly or indirectly include this file:



Functions

- [bool pb_field_iter_begin](#) (pb_field_iter_t *iter, const pb_msgdesc_t *desc, void *message)
- [bool pb_field_iter_begin_extension](#) (pb_field_iter_t *iter, pb_extension_t *extension)
- [bool pb_field_iter_begin_const](#) (pb_field_iter_t *iter, const pb_msgdesc_t *desc, const void *message)
- [bool pb_field_iter_begin_extension_const](#) (pb_field_iter_t *iter, const pb_extension_t *extension)
- [bool pb_field_iter_next](#) (pb_field_iter_t *iter)
- [bool pb_field_iter_find](#) (pb_field_iter_t *iter, [uint32_t](#) tag)
- [bool pb_field_iter_find_extension](#) (pb_field_iter_t *iter)

6.57.1 Function Documentation

6.57.1.1 pb_field_iter_begin()

```

bool pb_field_iter_begin (
    pb_field_iter_t * iter,
    const pb_msgdesc_t * desc,
    void * message)
  
```

Definition at line 156 of file [pb_common.c](#).

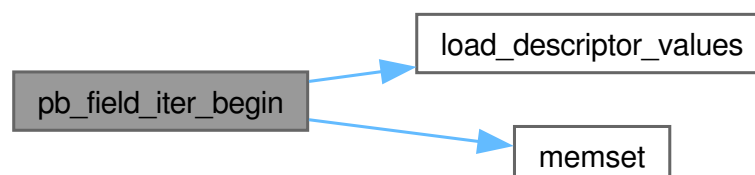
```

00157 {
00158     memset(iter, 0, sizeof(*iter));
00159     iter->descriptor = desc;
00160     iter->message = message;
00161     return load_descriptor_values(iter);
00162 }
00163
00164 }
  
```

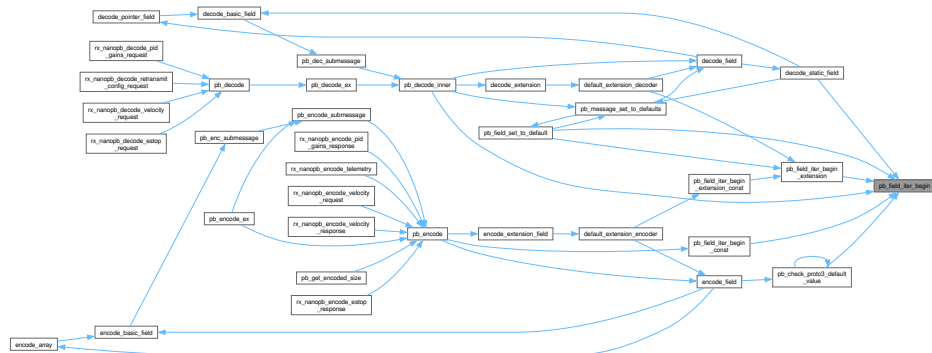
References [load_descriptor_values\(\)](#), and [memset\(\)](#).

Referenced by [decode_static_field\(\)](#), [pb_check_proto3_default_value\(\)](#), [pb_decode_inner\(\)](#), [pb_field_iter_begin_const\(\)](#), [pb_field_iter_begin_extension\(\)](#), and [pb_field_set_to_default\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.57.1.2 pb_field_iter_begin_const()

```
bool pb_field_iter_begin_const (
    pb_field_iter_t * iter,
    const pb_msgdesc_t * desc,
    const void * message)
```

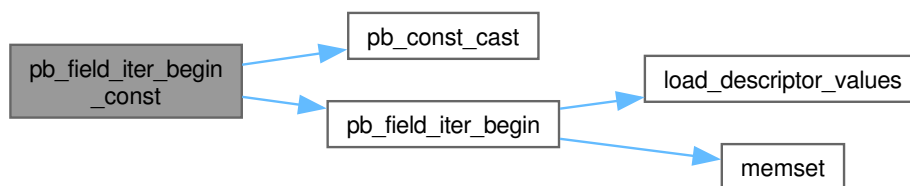
Definition at line 290 of file [pb_common.c](#).

```
00291 {
00292     return pb_field_iter_begin(iter, desc, pb_const_cast(message));
00293 }
```

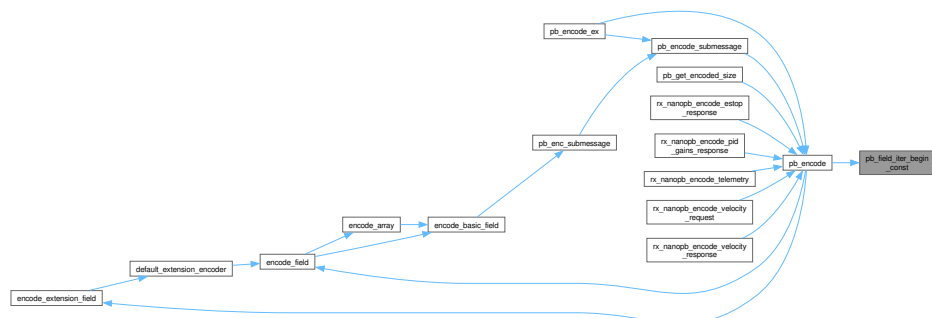
References [pb_const_cast\(\)](#), and [pb_field_iter_begin\(\)](#).

Referenced by [pb_encode\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:




```
bool pb_field_iter_begin_extension (
    pb_field_iter_t * iter,
    pb_extension_t * extension)
```

```

00167 {
00168     const pb_msgdesc_t *msg = (const pb_msgdesc_t*)extension->type->arg;
00169     bool status;
00170
00171     uint32_t word0 = PB_PROGMEM_READU32(msg->field_info[0]);
00172     if (PB_ATYPE(word0 » 8) == PB_ATYPE_POINTER)
00173     {
00174         /* For pointer extensions, the pointer is stored directly
00175          * in the extension structure. This avoids having an extra
00176          * indirection. */
00177         status = pb_field_iter_begin(iter, msg, &extension->dest);
00178     }
00179     else
00180     {
00181         status = pb_field_iter_begin(iter, msg, extension->dest);
00182     }
00183
00184     iter->pSize = &extension->found;
00185     return status;
00186 }

```

Referenced by `default_extension_decoder()`, `pb_field_iter_begin_extension_const()`, and `pb_field_set_to_default()`.

```
graph LR; A[pb_field_iter_begin_extension] --> B[pb_field_iter_begin]; B --> C[load_descriptor_values]; B --> D[memset];
```

The diagram illustrates a complex dependency graph for a protocol, showing the relationships between various fields and messages. The graph is divided into two main sections: the top section for decoding (decode) and the bottom section for encoding (encode).

Top Section (Decode):

- Inputs:** `rx_message_decode_req`, `rx_message_decode_req`, `rx_message_decode_req`, `rx_message_decode_req`, `rx_message_decode_req`.
- Intermediate Nodes:** `pb_decode`, `pb_decode_ext`, `pb_decode_ext_error`.
- Fields:** `decode_static_field`, `decode_field`, `decode_pointer_field`, `decode_base_field`, `pb_ext_submessage`, `pb_ext_error`, `decode_extension`, `pb_ext_submessage`, `pb_ext_error`.
- Default Values:** `pb_ext_submessage`, `pb_ext_error`.
- Output:** `pb_ext_submessage`.

Bottom Section (Encode):

- Inputs:** `tx_message_encode_req`, `tx_message_encode_req`, `tx_message_encode_req`, `tx_message_encode_req`, `tx_message_encode_req`.
- Intermediate Nodes:** `pb_encode`, `pb_encode_ext`, `pb_encode_ext_error`.
- Fields:** `encode_static_field`, `encode_field`, `encode_pointer_field`, `encode_base_field`, `pb_ext_submessage`, `pb_ext_error`, `encode_extension`, `pb_ext_submessage`, `pb_ext_error`.
- Default Values:** `pb_ext_submessage`, `pb_ext_error`.
- Output:** `pb_ext_submessage`.

The graph shows a high degree of connectivity, with many fields having multiple dependencies and being used in various ways. The top section (decode) and bottom section (encode) are highly symmetrical, reflecting the dual nature of the protocol.

6.57.1.4 pb_field_iter_begin_extension_const()

```
bool pb_field_iter_begin_extension_const (
    pb_field_iter_t * iter,
    const pb_extension_t * extension)
```

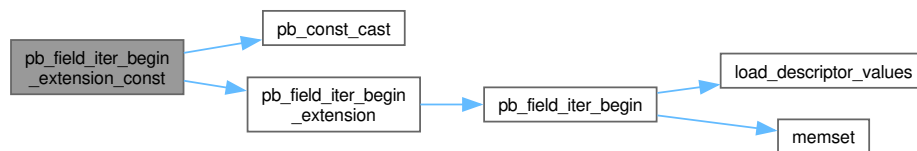
Definition at line 295 of file [pb_common.c](#).

```
00296 {
00297     return pb_field_iter_begin_extension(iter, (pb_extension_t*)pb_const_cast(extension));
00298 }
```

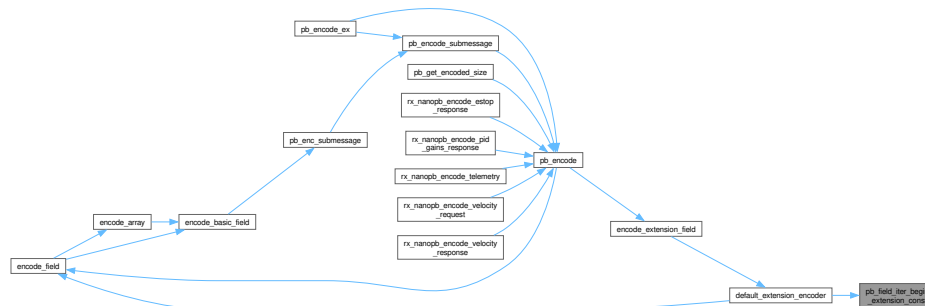
References [pb_const_cast\(\)](#), and [pb_field_iter_begin_extension\(\)](#).

Referenced by [default_extension_encoder\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.57.1.5 pb_field_iter_find()

```
bool pb_field_iter_find (
    pb_field_iter_t * iter,
    uint32_t tag)
```

Definition at line 195 of file [pb_common.c](#).

```
00196 {
00197     if (iter->tag == tag)
00198     {
00199         return true; /* Nothing to do, correct field already. */
00200     }
00201     else if (tag > iter->descriptor->largest_tag)
00202     {
00203         return false;
00204     }
00205     else
00206     {
00207         pb_size_t start = iter->index;
00208         uint32_t fieldinfo;
00209         if (tag < iter->tag)
00210         {

```

```

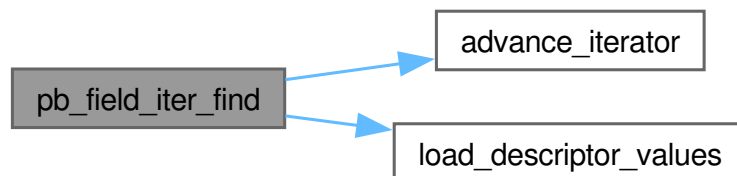
00212     /* Fields are in tag number order, so we know that tag is between
00213     * 0 and our start position. Setting index to end forces
00214     * advance_iterator() call below to restart from beginning. */
00215     iter->index = iter->descriptor->field_count;
00216 }
00217
00218 do
00219 {
00220     /* Advance iterator but don't load values yet */
00221     advance_iterator(iter);
00222
00223     /* Do fast check for tag number match */
00224     fieldinfo = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index]);
00225
00226     if (((fieldinfo >> 2) & 0x3F) == (tag & 0x3F))
00227     {
00228         /* Good candidate, check further */
00229         (void)load_descriptor_values(iter);
00230
00231         if (iter->tag == tag &&
00232             PB_LTYPE(iter->type) != PB_LTYPE_EXTENSION)
00233         {
00234             /* Found it */
00235             return true;
00236         }
00237     }
00238 } while (iter->index != start);
00239
00240 /* Searched all the way back to start, and found nothing. */
00241 (void)load_descriptor_values(iter);
00242 return false;
00243 }
00244 }

```

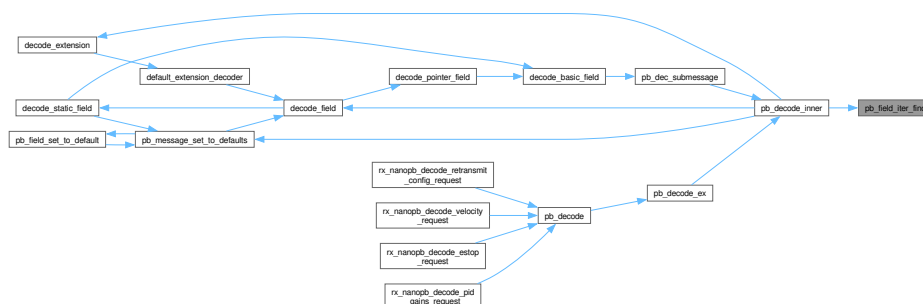
References [advance_iterator\(\)](#), [load_descriptor_values\(\)](#), [PB_LTYPE](#), [PB_LTYPE_EXTENSION](#), and [PB_PROGMEM_READU32](#).

Referenced by [pb_decode_inner\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.57.1.6 pb_field_iter_find_extension()

```
bool pb_field_iter_find_extension (
    pb_field_iter_t * iter)
```

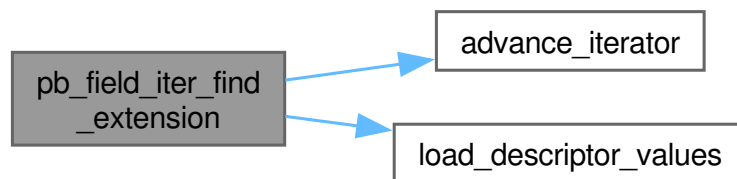
Definition at line 246 of file [pb_common.c](#).

```
00247 {
00248     if (PB_LTYPE(iter->type) == PB_LTYPE_EXTENSION)
00249     {
00250         return true;
00251     }
00252     else
00253     {
00254         pb_size_t start = iter->index;
00255         uint32_t fieldinfo;
00256
00257         do
00258         {
00259             /* Advance iterator but don't load values yet */
00260             advance_iterator(iter);
00261
00262             /* Do fast check for field type */
00263             fieldinfo = PB_PROGMEM_READU32(iter->descriptor->field_info[iter->field_info_index]);
00264
00265             if (PB_LTYPE((fieldinfo » 8) & 0xFF) == PB_LTYPE_EXTENSION)
00266             {
00267                 return load_descriptor_values(iter);
00268             }
00269         } while (iter->index != start);
00270
00271         /* Searched all the way back to start, and found nothing. */
00272         (void)load_descriptor_values(iter);
00273         return false;
00274     }
00275 }
```

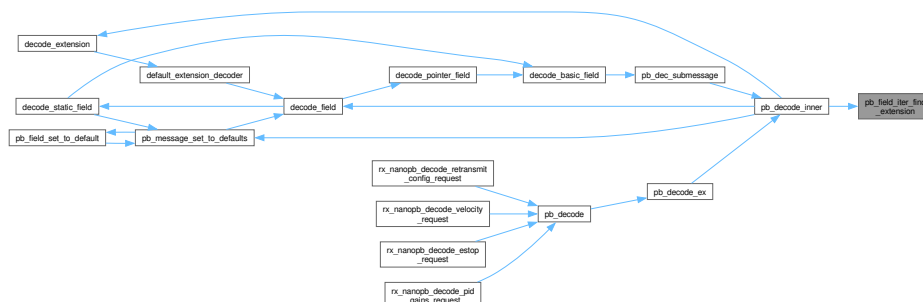
References [advance_iterator\(\)](#), [load_descriptor_values\(\)](#), [PB_LTYPE](#), [PB_LTYPE_EXTENSION](#), and [PB_PROGMEM_READU32](#).

Referenced by [pb_decode_inner\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.57.1.7 pb_field_iter_next()

```
bool pb_field_iter_next (
    pb_field_iter_t * iter)
```

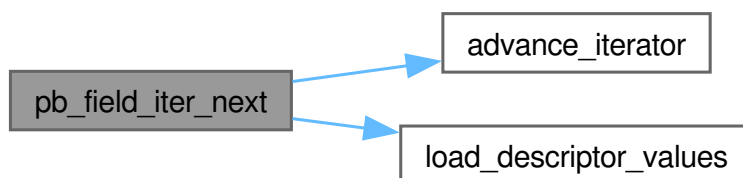
Definition at line 188 of file pb_common.c.

```
00189 {
00190     advance_iterator(iter);
00191     (void)load_descriptor_values(iter);
00192     return iter->index != 0;
00193 }
```

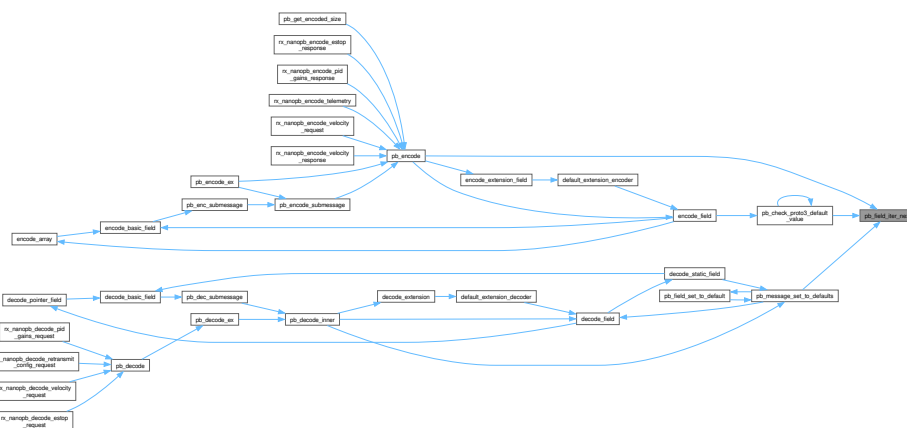
References `advance_iterator()`, and `load_descriptor_values()`.

Referenced by `pb_check_proto3_default_value()`, `pb_encode()`, and `pb_message_set_to_defaults()`.

Here is the call graph for this function:



Here is the caller graph for this function:



6.58 pb_common.h

[Go to the documentation of this file.](#)

```
00001 /* pb_common.h: Common support functions for pb_encode.c and pb_decode.c.
00002  * These functions are rarely needed by applications directly.
00003  */
00004
00005 #ifndef PB_COMMON_H_INCLUDED
00006 #define PB_COMMON_H_INCLUDED
00007
00008 #include "pb.h"
00009
00010 #ifdef __cplusplus
00011 extern "C" {
00012 #endif
00013
00014 /* Initialize the field iterator structure to beginning.
```

```

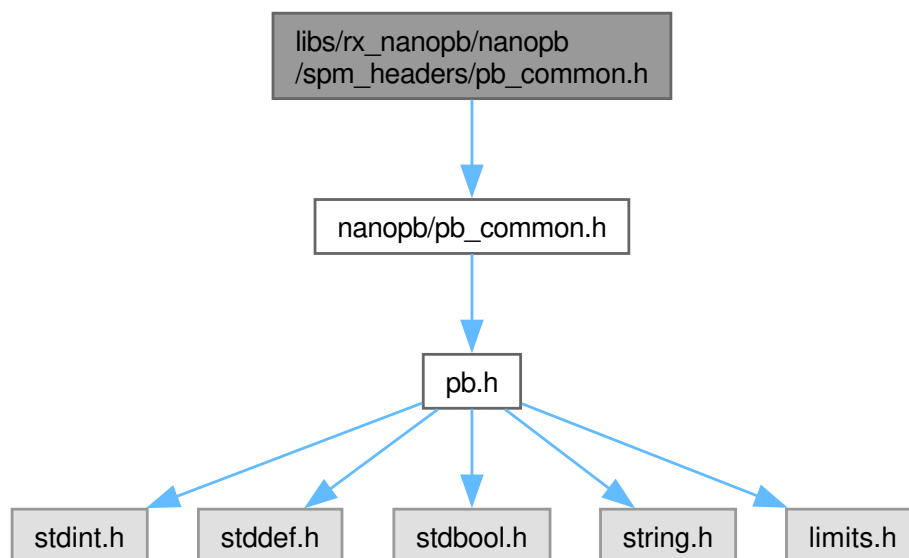
00015  * Returns false if the message type is empty. */
00016  bool pb_field_iter_begin(pb_field_iter_t *iter, const pb_msgdesc_t *desc, void *message);
00017
00018  /* Get a field iterator for extension field. */
00019  bool pb_field_iter_begin_extension(pb_field_iter_t *iter, pb_extension_t *extension);
00020
00021  /* Same as pb_field_iter_begin(), but for const message pointer.
00022  * Note that the pointers in pb_field_iter_t will be non-const but shouldn't
00023  * be written to when using these functions. */
00024  bool pb_field_iter_begin_const(pb_field_iter_t *iter, const pb_msgdesc_t *desc, const void *message);
00025  bool pb_field_iter_begin_extension_const(pb_field_iter_t *iter, const pb_extension_t *extension);
00026
00027  /* Advance the iterator to the next field.
00028  * Returns false when the iterator wraps back to the first field. */
00029  bool pb_field_iter_next(pb_field_iter_t *iter);
00030
00031  /* Advance the iterator until it points at a field with the given tag.
00032  * Returns false if no such field exists. */
00033  bool pb_field_iter_find(pb_field_iter_t *iter, uint32_t tag);
00034
00035  /* Find a field with type PB_LTYPE_EXTENSION, or return false if not found.
00036  * There can be only one extension range field per message. */
00037  bool pb_field_iter_find_extension(pb_field_iter_t *iter);
00038
00039  #ifdef PB_VALIDATE_UTF8
00040  /* Validate UTF-8 text string */
00041  bool pb_validate_utf8(const char *s);
00042  #endif
00043
00044  #ifdef __cplusplus
00045  } /* extern "C" */
00046  #endif
00047
00048  #endif
00049

```

6.59 libs/rx_nanopb/nanopb/spm_headers/pb_common.h File Reference

#include "nanopb/pb_common.h"

Include dependency graph for pb_common.h:



6.60 pb_common.h

[Go to the documentation of this file.](#)

```
00001 #include "nanopb/pb_common.h"
```

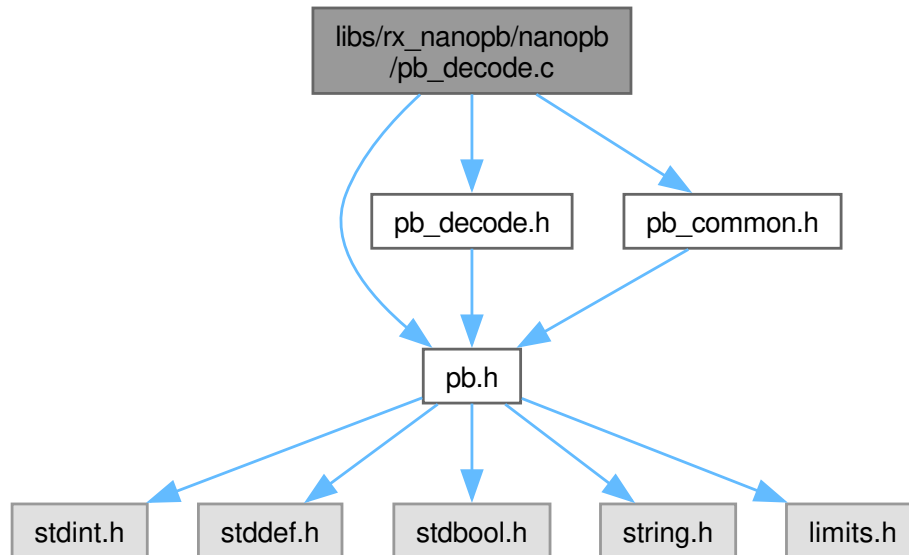
6.61 libs/rx_nanopb/nanopb/pb_decode.c File Reference

```
#include "pb.h"
```

```
#include "pb_decode.h"
```

```
#include "pb_common.h"
```

Include dependency graph for pb_decode.c:



Data Structures

- struct [pb_fields_seen_t](#)

Macros

- `#define` [checkreturn](#)
- `#define` [pb_int64_t](#) [int64_t](#)
- `#define` [pb_uint64_t](#) [uint64_t](#)

Functions

- static [bool](#) [buf_read](#) ([pb_istream_t](#) *stream, [pb_byte_t](#) *buf, [size_t](#) count)
- static [bool](#) [read_raw_value](#) ([pb_istream_t](#) *stream, [pb_wire_type_t](#) wire_type, [pb_byte_t](#) *buf, [size_t](#) *size)
- static [bool](#) [decode_basic_field](#) ([pb_istream_t](#) *stream, [pb_wire_type_t](#) wire_type, [pb_field_iter_t](#) *field)
- static [bool](#) [decode_static_field](#) ([pb_istream_t](#) *stream, [pb_wire_type_t](#) wire_type, [pb_field_iter_t](#) *field)
- static [bool](#) [decode_pointer_field](#) ([pb_istream_t](#) *stream, [pb_wire_type_t](#) wire_type, [pb_field_iter_t](#) *field)
- static [bool](#) [decode_callback_field](#) ([pb_istream_t](#) *stream, [pb_wire_type_t](#) wire_type, [pb_field_iter_t](#) *field)
- static [bool](#) [decode_field](#) ([pb_istream_t](#) *stream, [pb_wire_type_t](#) wire_type, [pb_field_iter_t](#) *field)
- static [bool](#) [default_extension_decoder](#) ([pb_istream_t](#) *stream, [pb_extension_t](#) *extension, [uint32_t](#) tag, [pb_wire_type_t](#) wire_type)

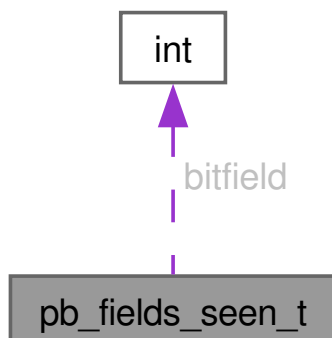
- static [bool decode_extension](#) (pb_istream_t *stream, [uint32_t](#) tag, [pb_wire_type_t](#) wire_type, pb_extension_t *extension)
- static [bool pb_field_set_to_default](#) (pb_field_iter_t *field)
- static [bool pb_message_set_to_defaults](#) (pb_field_iter_t *iter)
- static [bool pb_dec_bool](#) (pb_istream_t *stream, const pb_field_iter_t *field)
- static [bool pb_dec_varint](#) (pb_istream_t *stream, const pb_field_iter_t *field)
- static [bool pb_dec_bytes](#) (pb_istream_t *stream, const pb_field_iter_t *field)
- static [bool pb_dec_string](#) (pb_istream_t *stream, const pb_field_iter_t *field)
- static [bool pb_dec_submessage](#) (pb_istream_t *stream, const pb_field_iter_t *field)
- static [bool pb_dec_fixed_length_bytes](#) (pb_istream_t *stream, const pb_field_iter_t *field)
- static [bool pb_skip_varint](#) (pb_istream_t *stream)
- static [bool pb_skip_string](#) (pb_istream_t *stream)
- [bool pb_read](#) (pb_istream_t *stream, [pb_byte_t](#) *buf, [size_t](#) count)
- static [bool pb_readbyte](#) (pb_istream_t *stream, [pb_byte_t](#) *buf)
- pb_istream_t [pb_istream_from_buffer](#) (const [pb_byte_t](#) *buf, [size_t](#) msglen)
- [bool pb_decode_varint32](#) (pb_istream_t *stream, [uint32_t](#) *dest)
- [bool pb_decode_varint](#) (pb_istream_t *stream, [uint64_t](#) *dest)
- [bool pb_decode_tag](#) (pb_istream_t *stream, [pb_wire_type_t](#) *wire_type, [uint32_t](#) *tag, [bool](#) *eof)
- [bool pb_skip_field](#) (pb_istream_t *stream, [pb_wire_type_t](#) wire_type)
- [bool pb_make_string_substream](#) (pb_istream_t *stream, pb_istream_t *substream)
- [bool pb_close_string_substream](#) (pb_istream_t *stream, pb_istream_t *substream)
- static [bool pb_decode_inner](#) (pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct, unsigned int flags)
- [bool pb_decode_ex](#) (pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct, unsigned int flags)
- [bool pb_decode](#) (pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct)
- void [pb_release](#) (const pb_msgdesc_t *fields, void *dest_struct)
- [bool pb_decode_bool](#) (pb_istream_t *stream, [bool](#) *dest)
- [bool pb_decode_svarint](#) (pb_istream_t *stream, [int64_t](#) *dest)
- [bool pb_decode_fixed32](#) (pb_istream_t *stream, void *dest)
- [bool pb_decode_fixed64](#) (pb_istream_t *stream, void *dest)

6.61.1 Data Structure Documentation

6.61.1.1 struct pb_fields_seen_t

Definition at line 60 of file [pb_decode.c](#).

Collaboration diagram for pb_fields_seen_t:



Data Fields

uint32_t	bitfield[(64+31)/32]	
--------------------------	----------------------	--

6.61.2 Macro Definition Documentation

6.61.2.1 checkreturn

```
#define checkreturn
```

Definition at line 14 of file [pb_decode.c](#).

Referenced by [buf_read\(\)](#), [buf_write\(\)](#), [decode_basic_field\(\)](#), [decode_callback_field\(\)](#), [decode_extension\(\)](#), [decode_field\(\)](#), [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [default_extension_decoder\(\)](#), [default_extension_encoder\(\)](#), [encode_array\(\)](#), [encode_basic_field\(\)](#), [encode_callback_field\(\)](#), [encode_extension_field\(\)](#), [encode_field\(\)](#), [pb_check_proto3_default_value\(\)](#), [pb_close_string_substream\(\)](#), [pb_dec_bool\(\)](#), [pb_dec_bytes\(\)](#), [pb_dec_fixed_length_bytes\(\)](#), [pb_dec_string\(\)](#), [pb_dec_submessage\(\)](#), [pb_dec_varint\(\)](#), [pb_decode\(\)](#), [pb_decode_ex\(\)](#), [pb_decode_inner\(\)](#), [pb_decode_tag\(\)](#), [pb_decode_varint\(\)](#), [pb_decode_varint32\(\)](#), [pb_enc_bool\(\)](#), [pb_enc_bytes\(\)](#), [pb_enc_fixed\(\)](#), [pb_enc_fixed_length_bytes\(\)](#), [pb_enc_string\(\)](#), [pb_enc_submessage\(\)](#), [pb_enc_varint\(\)](#), [pb_encode\(\)](#), [pb_encode_ex\(\)](#), [pb_encode_fixed32\(\)](#), [pb_encode_fixed64\(\)](#), [pb_encode_string\(\)](#), [pb_encode_submessage\(\)](#), [pb_encode_svarint\(\)](#), [pb_encode_tag\(\)](#), [pb_encode_varint\(\)](#), [pb_encode_varint_32\(\)](#), [pb_make_string_substream\(\)](#), [pb_read\(\)](#), [pb_readbyte\(\)](#), [pb_skip_field\(\)](#), [pb_skip_string\(\)](#), [pb_skip_varint\(\)](#), [pb_write\(\)](#), and [read_raw_value\(\)](#).

6.61.2.2 pb_int64_t

```
#define pb_int64_t int64_t
```

Definition at line 56 of file [pb_decode.c](#).

Referenced by [pb_dec_varint\(\)](#), [pb_decode_svarint\(\)](#), [pb_enc_varint\(\)](#), and [pb_encode_svarint\(\)](#).

6.61.2.3 pb_uint64_t

```
#define pb_uint64_t uint64_t
```

Definition at line 57 of file [pb_decode.c](#).

Referenced by [encode_array\(\)](#), [pb_dec_varint\(\)](#), [pb_decode_svarint\(\)](#), [pb_enc_varint\(\)](#), [pb_encode_string\(\)](#), [pb_encode_submessage\(\)](#), [pb_encode_svarint\(\)](#), [pb_encode_tag\(\)](#), and [pb_encode_varint\(\)](#).

6.61.3.2 decode_basic_field()

```
bool decode_basic_field (
    pb_istream_t * stream,
    pb_wire_type_t wire_type,
    pb_field_iter_t * field) [static]
```

Definition at line 417 of file [pb_decode.c](#).

```
00418 {
00419     switch (PB_LTYPE(field->type))
00420     {
00421     case PB_LTYPE_BOOL:
00422         if (wire_type != PB_WT_VARINT && wire_type != PB_WT_PACKED)
00423             PB_RETURN_ERROR(stream, "wrong wire type");
00424
00425         return pb_dec_bool(stream, field);
00426
00427     case PB_LTYPE_VARINT:
00428     case PB_LTYPE_UVARINT:
00429     case PB_LTYPE_SVARINT:
00430         if (wire_type != PB_WT_VARINT && wire_type != PB_WT_PACKED)
00431             PB_RETURN_ERROR(stream, "wrong wire type");
00432
00433         return pb_dec_varint(stream, field);
00434
00435     case PB_LTYPE_FIXED32:
00436         if (wire_type != PB_WT_32BIT && wire_type != PB_WT_PACKED)
00437             PB_RETURN_ERROR(stream, "wrong wire type");
00438
00439         return pb_decode_fixed32(stream, field->pData);
00440
00441     case PB_LTYPE_FIXED64:
00442         if (wire_type != PB_WT_64BIT && wire_type != PB_WT_PACKED)
00443             PB_RETURN_ERROR(stream, "wrong wire type");
00444
00445     #ifdef PB_CONVERT_DOUBLE_FLOAT
00446         if (field->data_size == sizeof(float))
00447         {
00448             return pb_decode_double_as_float(stream, (float*)field->pData);
00449         }
00450     #endif
00451
00452     #ifdef PB_WITHOUT_64BIT
00453         PB_RETURN_ERROR(stream, "invalid data_size");
00454     #else
00455         return pb_decode_fixed64(stream, field->pData);
00456     #endif
00457
00458     case PB_LTYPE_BYTES:
00459         if (wire_type != PB_WT_STRING)
00460             PB_RETURN_ERROR(stream, "wrong wire type");
00461
00462         return pb_dec_bytes(stream, field);
00463
00464     case PB_LTYPE_STRING:
00465         if (wire_type != PB_WT_STRING)
00466             PB_RETURN_ERROR(stream, "wrong wire type");
00467
00468         return pb_dec_string(stream, field);
00469
00470     case PB_LTYPE_SUBMESSAGE:
00471     case PB_LTYPE_SUBMSG_W_CB:
00472         if (wire_type != PB_WT_STRING)
00473             PB_RETURN_ERROR(stream, "wrong wire type");
00474
00475         return pb_dec_submessage(stream, field);
00476
00477     case PB_LTYPE_FIXED_LENGTH_BYTES:
00478         if (wire_type != PB_WT_STRING)
00479             PB_RETURN_ERROR(stream, "wrong wire type");
00480
00481         return pb_dec_fixed_length_bytes(stream, field);
00482
00483     default:
00484         PB_RETURN_ERROR(stream, "invalid field type");
00485     }
00486 }
```

References [checkreturn](#), [pb_dec_bool\(\)](#), [pb_dec_bytes\(\)](#), [pb_dec_fixed_length_bytes\(\)](#), [pb_dec_string\(\)](#), [pb_dec_submessage\(\)](#), [pb_dec_varint\(\)](#), [pb_decode_fixed32\(\)](#), [pb_decode_fixed64\(\)](#), [PB_LTYPE](#), [PB_LTYPE_BOOL](#), [PB_LTYPE_BYTES](#), [PB_LTYPE_FIXED32](#), [PB_LTYPE_FIXED64](#),


```

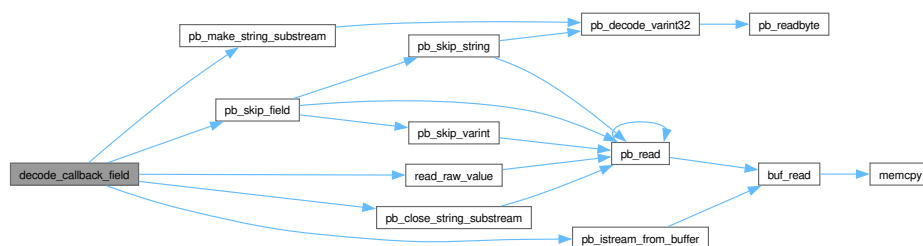
00791     callback = (pb_callback_t*)field->pSize - 1;
00792
00793     if (callback->funcs.decode)
00794     {
00795         if (!callback->funcs.decode(&substream, field, &callback->arg)) {
00796             PB_SET_ERROR(stream, substream.errmsg ? substream.errmsg : "submsg callback failed");
00797             return false;
00798         }
00799     }
00800 }
00801
00802 do
00803 {
00804     prev_bytes_left = substream.bytes_left;
00805     if (!field->descriptor->field_callback(&substream, NULL, field))
00806     {
00807         PB_SET_ERROR(stream, substream.errmsg ? substream.errmsg : "callback failed");
00808         return false;
00809     }
00810 } while (substream.bytes_left > 0 && substream.bytes_left < prev_bytes_left);
00811
00812 if (!pb_close_string_substream(stream, &substream))
00813     return false;
00814
00815 return true;
00816 }
00817 else
00818 {
00819     /* Copy the single scalar value to stack.
00820     * This is required so that we can limit the stream length,
00821     * which in turn allows to use same callback for packed and
00822     * not-packed fields. */
00823     pb_istream_t substream;
00824     pb_byte_t buffer[10];
00825     size_t size = sizeof(buffer);
00826
00827     if (!read_raw_value(stream, wire_type, buffer, &size))
00828         return false;
00829     substream = pb_istream_from_buffer(buffer, size);
00830
00831     return field->descriptor->field_callback(&substream, NULL, field);
00832 }
00833 }

```

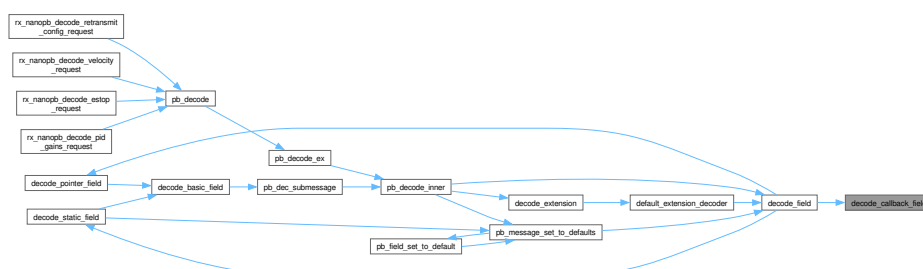
References [checkreturn](#), [NULL](#), [pb_close_string_substream\(\)](#), [pb_istream_from_buffer\(\)](#), [PB_LTYPE](#), [PB_LTYPE_SUBMSG_W_CB](#), [pb_make_string_substream\(\)](#), [PB_SET_ERROR](#), [pb_skip_field\(\)](#), [PB_WT_STRING](#), and [read_raw_value\(\)](#).

Referenced by [decode_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.5 decode_field()

```
bool decode_field (
    pb_istream_t * stream,
    pb_wire_type_t wire_type,
    pb_field_iter_t * field)  [static]
```

Definition at line 835 of file pb_decode.c.

```

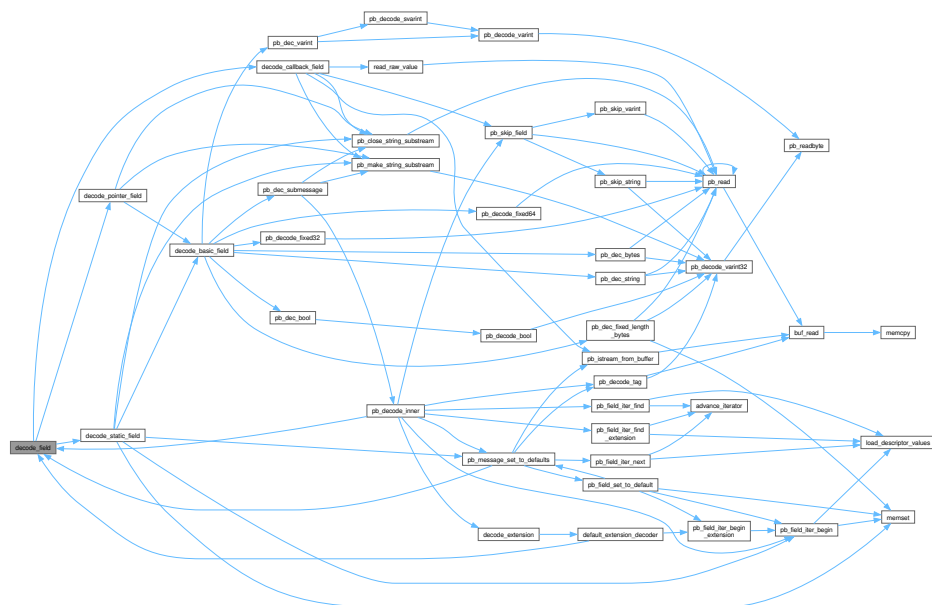
00836 {
00837     #ifndef PB_ENABLE_MALLOC
00838         /* When decoding an oneof field, check if there is old data that must be
00839          * released first. */
00840         if (PB_HTYPE(field->type) == PB_HTYPE_ONEOF)
00841         {
00842             if (!pb_release_union_field(stream, field))
00843                 return false;
00844         }
00845     #endif
00846
00847     switch (PB_ATYPE(field->type))
00848     {
00849         case PB_ATYPE_STATIC:
00850             return decode_static_field(stream, wire_type, field);
00851
00852         case PB_ATYPE_POINTER:
00853             return decode_pointer_field(stream, wire_type, field);
00854
00855         case PB_ATYPE_CALLBACK:
00856             return decode_callback_field(stream, wire_type, field);
00857
00858         default:
00859             PB_RETURN_ERROR(stream, "invalid field type");
00860     }
00861 }

```

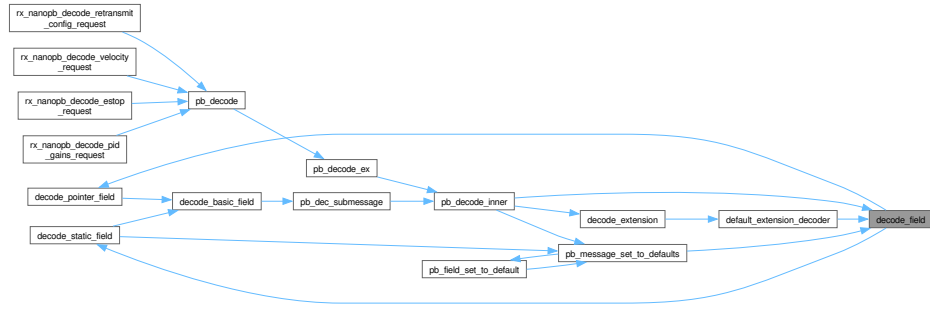
References `checkreturn`, `decode_callback_field()`, `decode_pointer_field()`, `decode_static_field()`, `PB_ATYPE`, `PB_ATYPE_CALLBACK`, `PB_ATYPE_POINTER`, `PB_ATYPE_STATIC`, `PB_HTYPE`, `PB_HTYPE_ONEOF`, and `PB_RETURN_ERROR`.

Referenced by `default_extension_decoder()`, `pb_decode_inner()`, and `pb_message_set_to_defaults()`.

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.6 decode_pointer_field()

```
bool decode_pointer_field (
    pb_istream_t * stream,
    pb_wire_type_t wire_type,
    pb_field_iter_t * field) [static]
```

Definition at line 645 of file [pb_decode.c](#).

```
00646 {
00647 #ifndef PB_ENABLE_MALLOC
00648     PB_UNUSED(wire_type);
00649     PB_UNUSED(field);
00650     PB_RETURN_ERROR(stream, "no malloc support");
00651 #else
00652     switch (PB_HTYPE(field->type))
00653     {
00654     case PB_HTYPE_REQUIRED:
00655     case PB_HTYPE_OPTIONAL:
00656     case PB_HTYPE_ONEOF:
00657         if (PB_LTYPE_IS_SUBMSG(field->type) && *(void**)field->pField != NULL)
00658         {
00659             /* Duplicate field, have to release the old allocation first. */
00660             /* FIXME: Does this work correctly for oneofs? */
00661             pb_release_single_field(field);
00662         }
00663
00664         if (PB_HTYPE(field->type) == PB_HTYPE_ONEOF)
00665         {
00666             *(pb_size_t*)field->pSize = field->tag;
00667         }
00668
00669         if (PB_LTYPE(field->type) == PB_LTYPE_STRING ||
00670             PB_LTYPE(field->type) == PB_LTYPE_BYTES)
00671         {
00672             /* pb_dec_string and pb_dec_bytes handle allocation themselves */
00673             field->pData = field->pField;
00674             return decode_basic_field(stream, wire_type, field);
00675         }
00676         else
00677         {
00678             if (!allocate_field(stream, field->pField, field->data_size, 1))
00679                 return false;
00680
00681             field->pData = *(void**)field->pField;
00682             initialize_pointer_field(field->pData, field);
00683             return decode_basic_field(stream, wire_type, field);
00684         }
00685
00686     case PB_HTYPE_REPEATED:
00687         if (wire_type == PB_WT_STRING
00688             && PB_LTYPE(field->type) <= PB_LTYPE_LAST_PACKABLE)
00689         {
00690             /* Packed array, multiple items come in at once. */
00691             bool status = true;
00692             pb_size_t *size = (pb_size_t*)field->pSize;
00693             size_t allocated_size = *size;
00694             pb_istream_t substream;
00695
00696             if (!pb_make_string_substream(stream, &substream))
00697                 return false;
00698
00699             while (substream.bytes_left)
00700             {
```



```

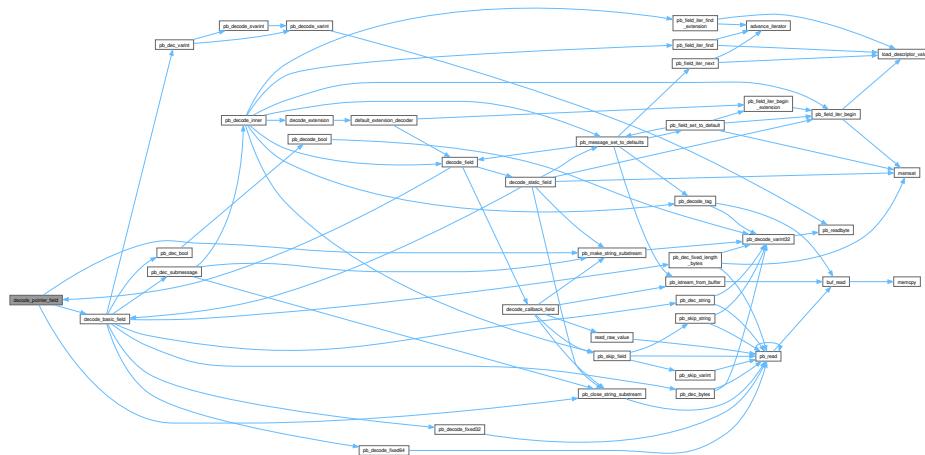
00701         if (*size == PB_SIZE_MAX)
00702         {
00703             #ifndef PB_NO_ERRMSG
00704                 stream->errmsg = "too many array entries";
00705             #endif
00706             status = false;
00707             break;
00708         }
00709
00710         if ((size_t)*size + 1 > allocated_size)
00711         {
00712             /* Allocate more storage. This tries to guess the
00713              * number of remaining entries. Round the division
00714              * upwards. */
00715             size_t remain = (substream.bytes_left - 1) / field->data_size + 1;
00716             if (remain < PB_SIZE_MAX - allocated_size)
00717                 allocated_size += remain;
00718             else
00719                 allocated_size += 1;
00720
00721             if (!allocate_field(&substream, field->pField, field->data_size, allocated_size))
00722             {
00723                 status = false;
00724                 break;
00725             }
00726         }
00727
00728         /* Decode the array entry */
00729         field->pData = *(char**)field->pField + field->data_size * (*size);
00730         if (field->pData == NULL)
00731         {
00732             /* Shouldn't happen, but satisfies static analyzers */
00733             status = false;
00734             break;
00735         }
00736         initialize_pointer_field(field->pData, field);
00737         if (!decode_basic_field(&substream, PB_WT_PACKED, field))
00738         {
00739             status = false;
00740             break;
00741         }
00742         (*size)++;
00743     }
00744     if (!pb_close_string_substream(stream, &substream))
00745         return false;
00746     return status;
00747 }
00748
00749 else
00750 {
00751     /* Normal repeated field, i.e. only one item at a time. */
00752     pb_size_t *size = (pb_size_t*)field->pSize;
00753
00754     if (*size == PB_SIZE_MAX)
00755         PB_RETURN_ERROR(stream, "too many array entries");
00756
00757     if (!allocate_field(stream, field->pField, field->data_size, (size_t)(*size + 1)))
00758         return false;
00759
00760     field->pData = *(char**)field->pField + field->data_size * (*size);
00761     (*size)++;
00762     initialize_pointer_field(field->pData, field);
00763     return decode_basic_field(stream, wire_type, field);
00764 }
00765
00766 default:
00767     PB_RETURN_ERROR(stream, "invalid field type");
00768 }
00769 #endif
00770 }
00771 }

```

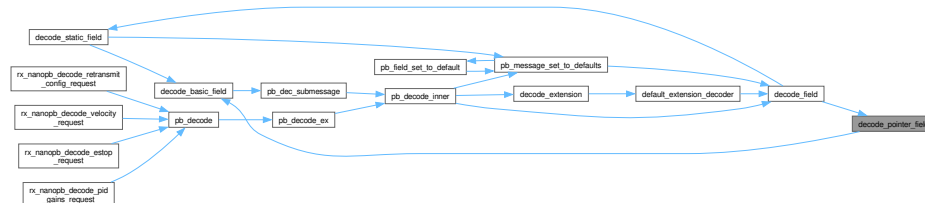
References [checkreturn](#), [decode_basic_field\(\)](#), [NULL](#), [pb_close_string_substream\(\)](#), [PB_HTYPE](#), [PB_HTYPE_ONEOF](#), [PB_HTYPE_OPTIONAL](#), [PB_HTYPE_REPEATED](#), [PB_HTYPE_REQUIRED](#), [PB_LTYPE](#), [PB_LTYPE_BYTES](#), [PB_LTYPE_IS_SUBMSG](#), [PB_LTYPE_LAST_PACKABLE](#), [PB_LTYPE_STRING](#), [pb_make_string_substream\(\)](#), [PB_RETURN_ERROR](#), [PB_SIZE_MAX](#), [PB_UNUSED](#), [PB_WT_PACKED](#), and [PB_WT_STRING](#).

Referenced by [decode_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.7 decode static field()

```
bool decode_static_field (
    pb_istream_t * stream,
    pb_wire_type_t wire_type,
    pb_field_iter t * field) [static]
```

Definition at line 488 of file pb_decode.c.

```

00489 {
00490     switch (PB_HTYPE(field->type))
00491     {
00492         case PB_HTYPE_REQUIRED:
00493             return decode_basic_field(stream, wire_type, field);
00494
00495         case PB_HTYPE_OPTIONAL:
00496             if (field->pSize != NULL)
00497                 *(bool*)&field->pSize = true;
00498             return decode_basic_field(stream, wire_type, field);
00499
00500         case PB_HTYPE_REPEATED:
00501             if (wire_type == PB_WT_STRING
00502                 && PB_LTYPE(field->type) <= PB_LTYPE_LAST_PACKABLE)
00503             {
00504                 /* Packed array */
00505                 bool status = true;
00506                 pb_istream_t substream;
00507                 pb_size_t *size = (pb_size_t*)&field->pSize;
00508                 field->pData = (char*)&field->pField + field->data_size * (*size);
00509
00510                 if (!pb_make_string_substream(stream, &substream))
00511                     return false;
00512
00513                 while (substream.bytes_left > 0 && *size < field->array_size)
00514                 {
00515                     if (!decode_basic_field(&substream, PB_WT_PACKED, field))
00516                     {
00517                         status = false;
00518                         break;
00519                     }
00520                     (*size)++;

```

```

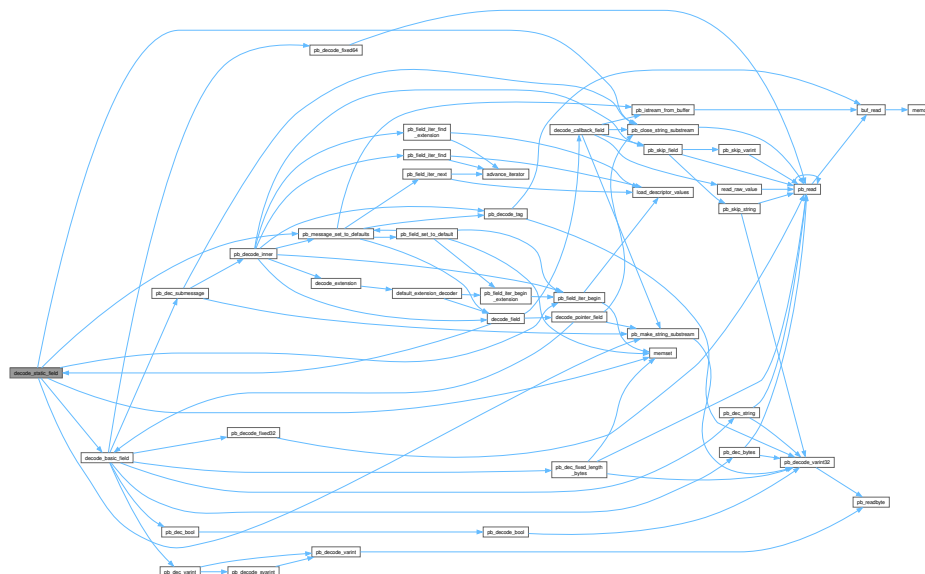
00521         field->pData = (char*)field->pData + field->data_size;
00522     }
00523
00524     if (substream.bytes_left != 0)
00525         PB_RETURN_ERROR(stream, "array overflow");
00526     if (!pb_close_string_substream(stream, &substream))
00527         return false;
00528
00529     return status;
00530 }
00531 else
00532 {
00533     /* Repeated field */
00534     pb_size_t *size = (pb_size_t*)field->pSize;
00535     field->pData = (char*)field->pField + field->data_size * (*size);
00536
00537     if ((*size)++ >= field->array_size)
00538         PB_RETURN_ERROR(stream, "array overflow");
00539
00540     return decode_basic_field(stream, wire_type, field);
00541 }
00542
00543 case PB_HTYPE_ONEOF:
00544     if (PB_LTYPE_IS_SUBMSG(field->type) &&
00545         *(pb_size_t*)field->pSize != field->tag)
00546     {
00547         /* We memset to zero so that any callbacks are set to NULL.
00548          * This is because the callbacks might otherwise have values
00549          * from some other union field.
00550          * If callbacks are needed inside oneof field, use .proto
00551          * option submsg_callback to have a separate callback function
00552          * that can set the fields before submessage is decoded.
00553          * pb_dec_submessage() will set any default values. */
00554         memset(field->pData, 0, (size_t)field->data_size);
00555
00556         /* Set default values for the submessage fields. */
00557         if (field->submsg_desc->default_value != NULL ||
00558             field->submsg_desc->field_callback != NULL ||
00559             field->submsg_desc->submsg_info[0] != NULL)
00560         {
00561             pb_field_iter_t submsg_iter;
00562             if (pb_field_iter_begin(&submsg_iter, field->submsg_desc, field->pData))
00563             {
00564                 if (!pb_message_set_to_defaults(&submsg_iter))
00565                     PB_RETURN_ERROR(stream, "failed to set defaults");
00566             }
00567         }
00568     }
00569     *(pb_size_t*)field->pSize = field->tag;
00570
00571     return decode_basic_field(stream, wire_type, field);
00572
00573 default:
00574     PB_RETURN_ERROR(stream, "invalid field type");
00575 }
00576 }

```

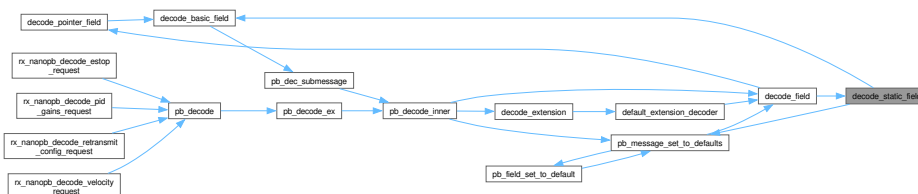
References [checkreturn](#), [decode_basic_field\(\)](#), [memset\(\)](#), [NULL](#), [pb_close_string_substream\(\)](#), [pb_field_iter_begin\(\)](#), [PB_HTYPE](#), [PB_HTYPE_ONEOF](#), [PB_HTYPE_OPTIONAL](#), [PB_HTYPE_REPEATED](#), [PB_HTYPE_REQUIRED](#), [PB_LTYPE](#), [PB_LTYPE_IS_SUBMSG](#), [PB_LTYPE_LAST_PACKABLE](#), [pb_make_string_substream\(\)](#), [pb_message_set_to_defaults\(\)](#), [PB_RETURN_ERROR](#), [PB_WT_PACKED](#), and [PB_WT_STRING](#).

Referenced by [decode_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.8 default_extension_decoder()

```
bool default_extension_decoder (
    pb_istream_t * stream,
    pb_extension_t * extension,
    uint32_t tag,
    pb_wire_type_t wire_type) [static]
```

Definition at line 866 of file pb_decode.c.

```

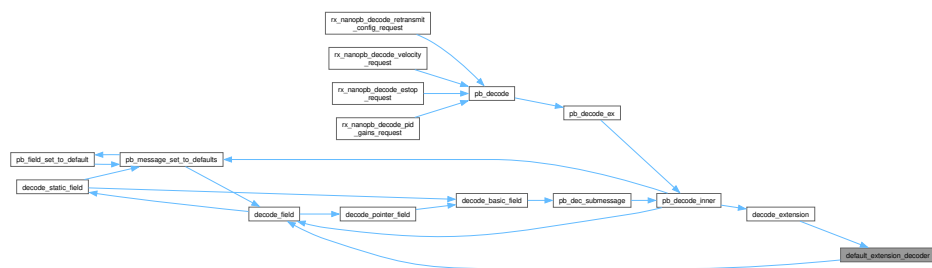
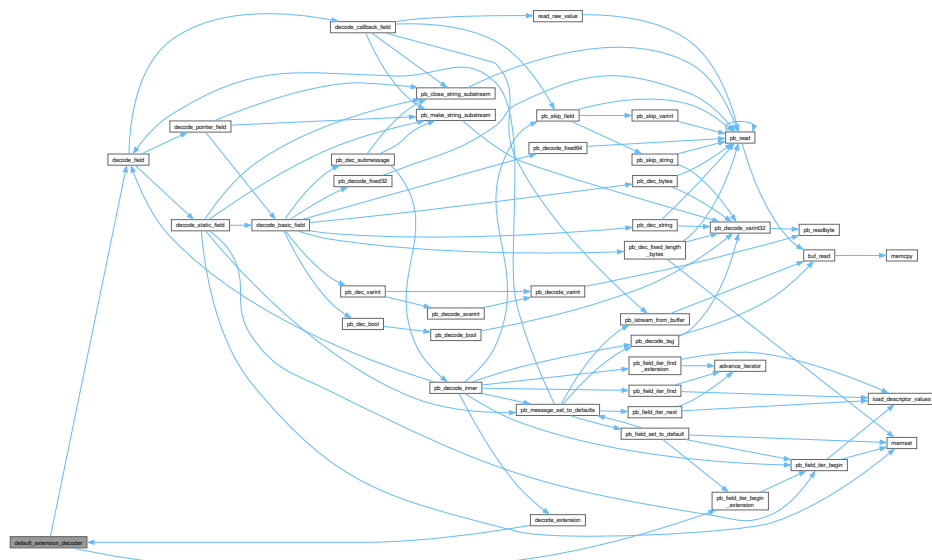
00868 {
00869     pb_field_iter_t iter;
00870
00871     if (!pb_field_iter_begin_extension(&iter, extension))
00872         PB_RETURN_ERROR(stream, "invalid extension");
00873
00874     if (iter.tag != tag || !iter.message)
00875         return true;
00876
00877     extension->found = true;
00878     return decode_field(stream, wire_type, &iter);
00879 }

```

References [checkreturn](#), [decode_field\(\)](#), [pb_field_iter_begin_extension\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [decode_extension\(\)](#).

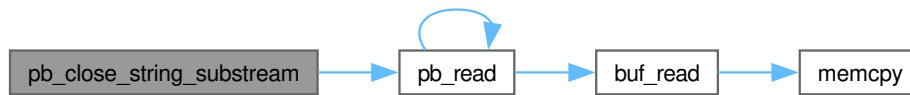
Here is the call graph for this function:



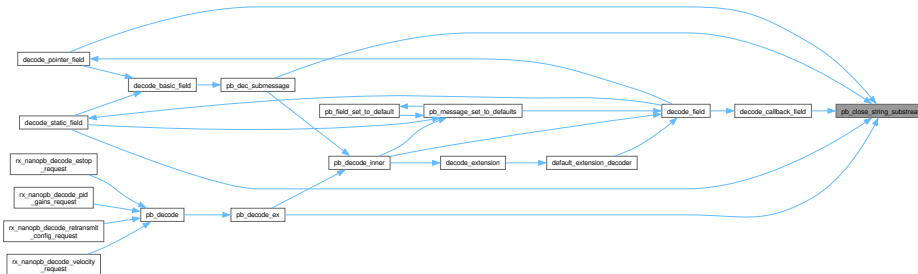
```
00399 {
00400     if (substream->bytes_left) {
```

References `checkreturn`, `NULL`, and `pb_read()`.

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.10 pb_dec_bool()

```

bool pb_dec_bool (
    pb_istream_t * stream,
    const pb_field_iter_t * field) [static]
  
```

Definition at line 1455 of file [pb_decode.c](#).

```

01456 {
01457     return pb_decode_bool(stream, (bool*)field->pData);
01458 }
  
```

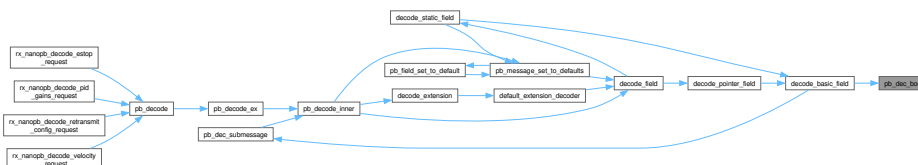
References [checkreturn](#), and [pb_decode_bool\(\)](#).

Referenced by [decode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.11 pb_dec_bytes()

```
bool pb_dec_bytes (
    pb_istream_t * stream,
    const pb_field_iter_t * field) [static]
```

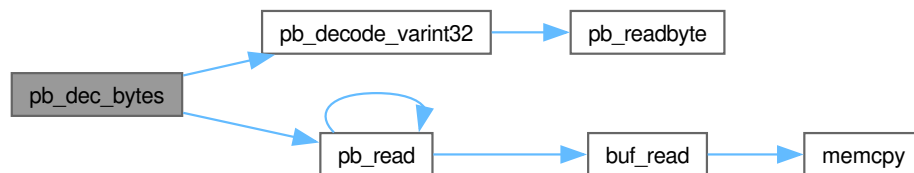
Definition at line 1532 of file [pb_decode.c](#).

```
01533 {
01534     uint32_t size;
01535     size_t alloc_size;
01536     pb_bytes_array_t *dest;
01537
01538     if (!pb_decode_varint32(stream, &size))
01539         return false;
01540
01541     if (size > PB_SIZE_MAX)
01542         PB_RETURN_ERROR(stream, "bytes overflow");
01543
01544     alloc_size = PB_BYTES_ARRAY_T_ALLOCSIZE(size);
01545     if (size > alloc_size)
01546         PB_RETURN_ERROR(stream, "size too large");
01547
01548     if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
01549     {
01550 #ifndef PB_ENABLE_MALLOC
01551         PB_RETURN_ERROR(stream, "no malloc support");
01552     #else
01553         if (stream->bytes_left < size)
01554             PB_RETURN_ERROR(stream, "end-of-stream");
01555
01556         if (!allocate_field(stream, field->pData, alloc_size, 1))
01557             return false;
01558         dest = *(pb_bytes_array_t**)field->pData;
01559     #endif
01560     }
01561     else
01562     {
01563         if (alloc_size > field->data_size)
01564             PB_RETURN_ERROR(stream, "bytes overflow");
01565         dest = (pb_bytes_array_t*)field->pData;
01566     }
01567
01568     dest->size = (pb_size_t)size;
01569     return pb_read(stream, dest->bytes, (size_t)size);
01570 }
```

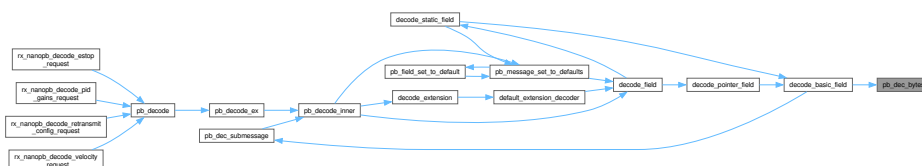
References [checkreturn](#), [PB_ATYPE](#), [PB_ATYPE_POINTER](#), [PB_BYTES_ARRAY_T_ALLOCSIZE](#), [pb_decode_varint32\(\)](#), [pb_read\(\)](#), [PB_RETURN_ERROR](#), and [PB_SIZE_MAX](#).

Referenced by [decode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.12 pb_dec_fixed_length_bytes()

```
bool pb_dec_fixed_length_bytes (
    pb_istream_t * stream,
    const pb_field_iter_t * field)  [static]
```

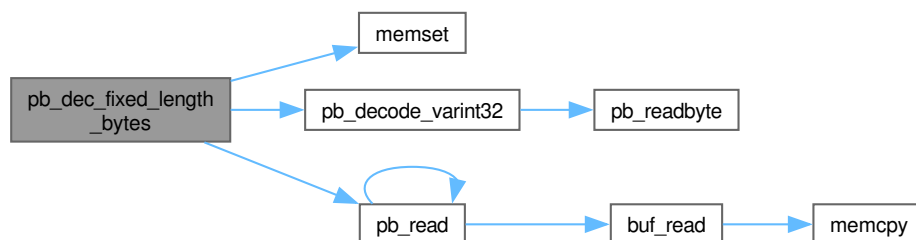
Definition at line 1674 of file [pb_decode.c](#).

```
01675 {
01676     uint32_t size;
01677
01678     if (!pb_decode_varint32(stream, &size))
01679         return false;
01680
01681     if (size > PB_SIZE_MAX)
01682         PB_RETURN_ERROR(stream, "bytes overflow");
01683
01684     if (size == 0)
01685     {
01686         /* As a special case, treat empty bytes string as all zeros for fixed_length_bytes. */
01687         memset(field->pData, 0, (size_t)field->data_size);
01688         return true;
01689     }
01690
01691     if (size != field->data_size)
01692         PB_RETURN_ERROR(stream, "incorrect fixed length bytes size");
01693
01694     return pb_read(stream, (pb_byte_t*)field->pData, (size_t)field->data_size);
01695 }
```

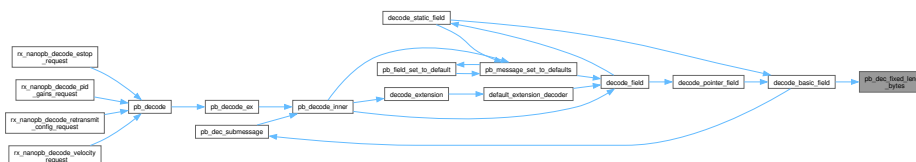
References [checkreturn](#), [memset\(\)](#), [pb_decode_varint32\(\)](#), [pb_read\(\)](#), [PB_RETURN_ERROR](#), and [PB_SIZE_MAX](#).

Referenced by [decode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.13 pb_dec_string()

```
bool pb_dec_string (
    pb_istream_t * stream,
    const pb_field_iter_t * field) [static]
```

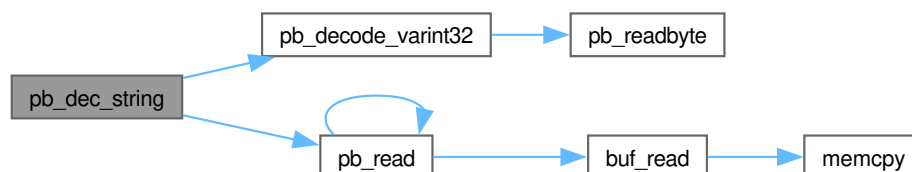
Definition at line 1572 of file [pb_decode.c](#).

```
01573 {
01574     uint32_t size;
01575     size_t alloc_size;
01576     pb_byte_t *dest = (pb_byte_t*)field->pData;
01577
01578     if (!pb_decode_varint32(stream, &size))
01579         return false;
01580
01581     if (size == (uint32_t)-1)
01582         PB_RETURN_ERROR(stream, "size too large");
01583
01584     /* Space for null terminator */
01585     alloc_size = (size_t)(size + 1);
01586
01587     if (alloc_size < size)
01588         PB_RETURN_ERROR(stream, "size too large");
01589
01590     if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
01591     {
01592         #ifndef PB_ENABLE_MALLOC
01593         PB_RETURN_ERROR(stream, "no malloc support");
01594         #else
01595         if (stream->bytes_left < size)
01596             PB_RETURN_ERROR(stream, "end-of-stream");
01597
01598         if (!allocate_field(stream, field->pData, alloc_size, 1))
01599             return false;
01600         dest = *(pb_byte_t**)field->pData;
01601     #endif
01602     }
01603     else
01604     {
01605         if (alloc_size > field->data_size)
01606             PB_RETURN_ERROR(stream, "string overflow");
01607     }
01608
01609     dest[size] = 0;
01610
01611     if (!pb_read(stream, dest, (size_t)size))
01612         return false;
01613
01614     #ifdef PB_VALIDATE_UTF8
01615     if (!pb_validate_utf8((const char*)dest))
01616         PB_RETURN_ERROR(stream, "invalid utf8");
01617     #endif
01618
01619     return true;
01620 }
```

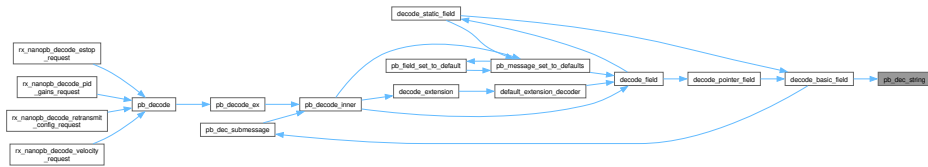
References [checkreturn](#), [PB_ATYPE](#), [PB_ATYPE_POINTER](#), [pb_decode_varint32\(\)](#), [pb_read\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [decode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.14 pb_dec_submessage()

```
bool pb_dec_submessage (
    pb_istream_t * stream,
    const pb_field_iter_t * field) [static]
```

Definition at line 1622 of file [pb_decode.c](#).

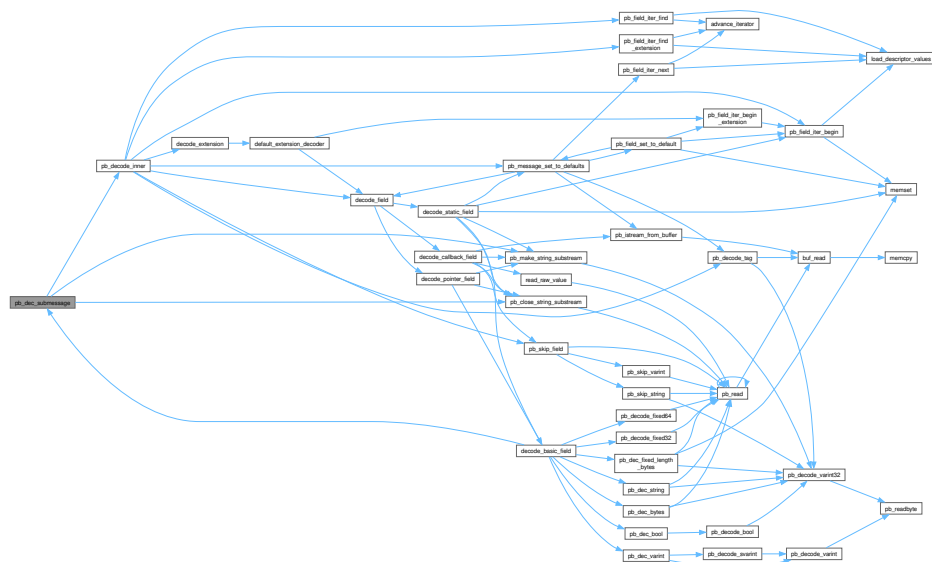
```

1623 {
1624     bool status = true;
1625     bool submsg_consumed = false;
1626     pb_istream_t substream;
1627
1628     if (!pb_make_string_substream(stream, &substream))
1629         return false;
1630
1631     if (field->submsg_desc == NULL)
1632         PB_RETURN_ERROR(stream, "invalid field descriptor");
1633
1634     /* Submessages can have a separate message-level callback that is called
1635     * before decoding the message. Typically it is used to set callback fields
1636     * inside oneofs. */
1637     if (PB_LTYPE(field->type) == PB_LTYPE_SUBMSG_W_CB && field->pSize != NULL)
1638     {
1639         /* Message callback is stored right before pSize. */
1640         pb_callback_t *callback = (pb_callback_t*)field->pSize - 1;
1641         if (callback->funcs.decode)
1642         {
1643             status = callback->funcs.decode(&substream, field, &callback->arg);
1644
1645             if (substream.bytes_left == 0)
1646             {
1647                 submsg_consumed = true;
1648             }
1649         }
1650     }
1651
1652     /* Now decode the submessage contents */
1653     if (status && !submsg_consumed)
1654     {
1655         unsigned int flags = 0;
1656
1657         /* Static required/optional fields are already initialized by top-level
1658         * pb_decode(), no need to initialize them again. */
1659         if (PB_ATYPE(field->type) == PB_ATYPE_STATIC &&
1660             PB_HTYPE(field->type) != PB_HTYPE_REPEATED)
1661         {
1662             flags = PB_DECODE_NOINIT;
1663         }
1664
1665         status = pb_decode_inner(&substream, field->submsg_desc, field->pData, flags);
1666     }
1667
1668     if (!pb_close_string_substream(stream, &substream))
1669         return false;
1670
1671     return status;
1672 }
```

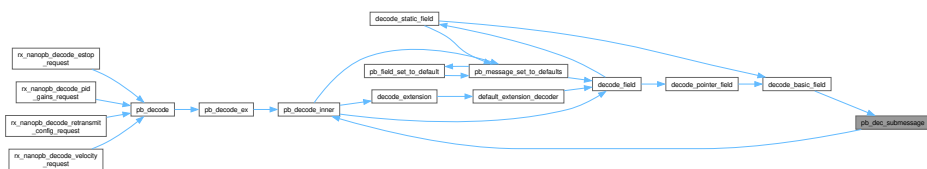
References [checkreturn](#), [NULL](#), [PB_ATYPE](#), [PB_ATYPE_STATIC](#), [pb_close_string_substream\(\)](#), [pb_decode_inner\(\)](#), [PB_DECODE_NOINIT](#), [PB_HTYPE](#), [PB_HTYPE_REPEATED](#), [PB_LTYPE](#), [PB_LTYPE_SUBMSG_W_CB](#), [pb_make_string_substream\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [decode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.15 pb_dec_varint()

```
bool pb_dec_varint (
    pb_istream_t * stream,
    const pb_field_iter_t * field) [static]
```

Definition at line 1460 of file pb_decode.c.

```

01461 {
01462     if (PB_LTYPE(field->type) == PB_LTYPE_UVARINT)
01463     {
01464         pb_uint64_t value, clamped;
01465         if (!pb_decode_varint(stream, &value))
01466             return false;
01467
01468         /* Cast to the proper field size, while checking for overflows */
01469         if (field->data_size == sizeof(pb_uint64_t))
01470             clamped = *(pb_uint64_t*)field->pData = value;
01471         else if (field->data_size == sizeof(uint32_t))
01472             clamped = *(uint32_t*)field->pData = (uint32_t)value;
01473         else if (field->data_size == sizeof(uint_least16_t))
01474             clamped = *(uint_least16_t*)field->pData = (uint_least16_t)value;
01475         else if (field->data_size == sizeof(uint_least8_t))
01476             clamped = *(uint_least8_t*)field->pData = (uint_least8_t)value;
01477         else
01478             PB_RETURN_ERROR(stream, "invalid data_size");
01479
01480         if (clamped != value)
01481             PB_RETURN_ERROR(stream, "integer too large");
01482
01483         return true;
01484     }
01485     else
01486     {
01487         pb_uint64_t value;
01488         pb_int64_t svalue;
01489         pb_int64_t clamped;
01490

```

```

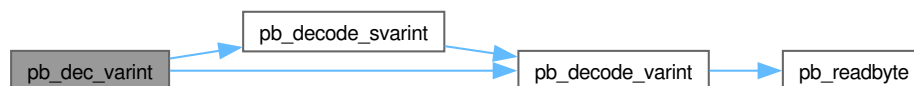
01491     if (PB_LTYPE(field->type) == PB_LTYPE_SVARINT)
01492     {
01493         if (!pb_decode_svarint(stream, &svalue))
01494             return false;
01495     }
01496     else
01497     {
01498         if (!pb_decode_varint(stream, &value))
01499             return false;
01500
01501         /* See issue 97: Google's C++ protobuf allows negative varint values to
01502          * be cast as int32_t, instead of the int64_t that should be used when
01503          * encoding. Nanopb versions before 0.2.5 had a bug in encoding. In order to
01504          * not break decoding of such messages, we cast <=32 bit fields to
01505          * int32_t first to get the sign correct.
01506          */
01507         if (field->data_size == sizeof(pb_int64_t))
01508             svalue = (pb_int64_t)value;
01509         else
01510             svalue = (int32_t)value;
01511     }
01512
01513     /* Cast to the proper field size, while checking for overflows */
01514     if (field->data_size == sizeof(pb_int64_t))
01515         clamped = *(pb_int64_t*)field->pData = svalue;
01516     else if (field->data_size == sizeof(int32_t))
01517         clamped = *(int32_t*)field->pData = (int32_t)svalue;
01518     else if (field->data_size == sizeof(int_least16_t))
01519         clamped = *(int_least16_t*)field->pData = (int_least16_t)svalue;
01520     else if (field->data_size == sizeof(int_least8_t))
01521         clamped = *(int_least8_t*)field->pData = (int_least8_t)svalue;
01522     else
01523         PB_RETURN_ERROR(stream, "invalid data_size");
01524
01525     if (clamped != svalue)
01526         PB_RETURN_ERROR(stream, "integer too large");
01527
01528     return true;
01529 }
01530 }

```

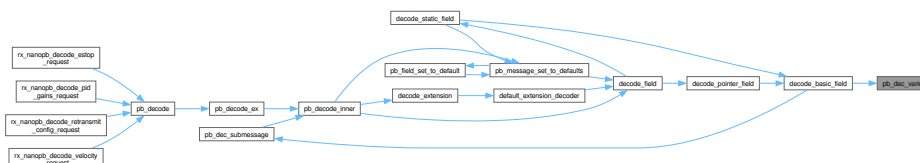
References [checkreturn](#), [pb_decode_svarint\(\)](#), [pb_decode_varint\(\)](#), [pb_int64_t](#), [PB_LTYPE](#), [PB_LTYPE_SVARINT](#), [PB_LTYPE_UVARINT](#), [PB_RETURN_ERROR](#), and [pb_uint64_t](#).

Referenced by [decode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.16 pb_decode()

```

bool pb_decode (
    pb_istream_t * stream,
    const pb_msgdesc_t * fields,
    void * dest_struct)

```

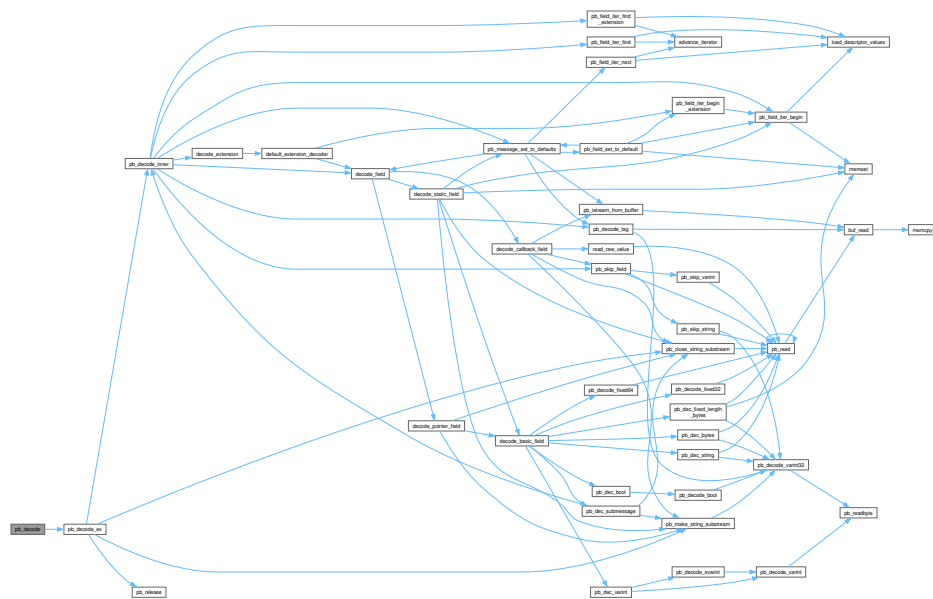
Definition at line 1226 of file [pb_decode.c](#).

```
01227 {
01228     return pb_decode_ex(stream, fields, dest_struct, 0);
01229 }
```

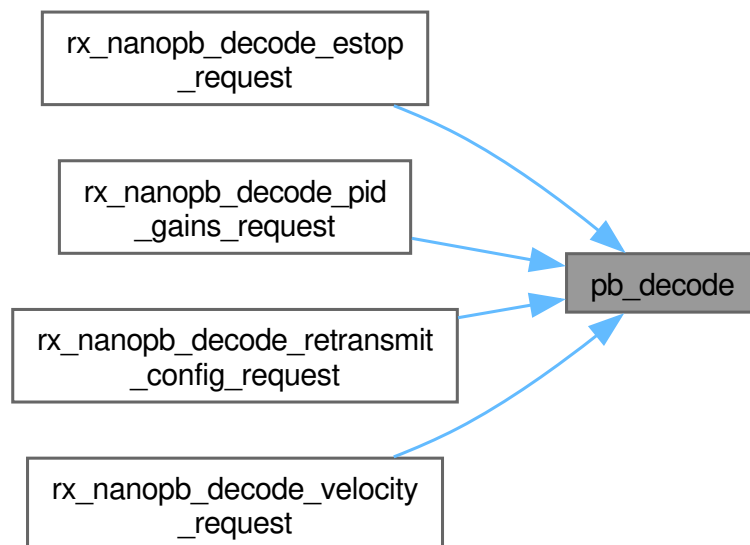
References [checkreturn](#), and [pb_decode_ex\(\)](#).

Referenced by [rx_nanopb_decode_estop_request\(\)](#), [rx_nanopb_decode_pid_gains_request\(\)](#), [rx_nanopb_decode_retransmit_config_request\(\)](#), and [rx_nanopb_decode_velocity_request\(\)](#).

Here is the call graph for this function:

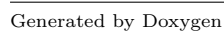
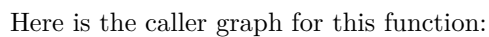


Here is the caller graph for this function:



References [checkreturn](#), [pb_close_string_substream\(\)](#), [PB_DECODE_DELIMITED](#), [pb_decode_inner\(\)](#), [pb_make_string_substream\(\)](#), and [pb_release\(\)](#).

Here is the call graph for this function:




```

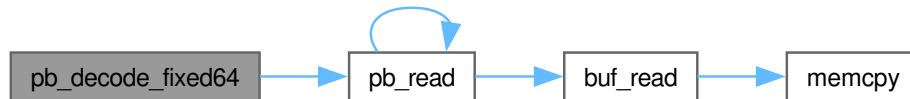
01437
01438 #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
01439 /* fast path - if we know that we're on little endian, assign directly */
01440 *(uint64_t*)dest = u.fixed64;
01441 #else
01442 *(uint64_t*)dest = ((uint64_t)u.bytes[0] « 0) |
01443                     ((uint64_t)u.bytes[1] « 8) |
01444                     ((uint64_t)u.bytes[2] « 16) |
01445                     ((uint64_t)u.bytes[3] « 24) |
01446                     ((uint64_t)u.bytes[4] « 32) |
01447                     ((uint64_t)u.bytes[5] « 40) |
01448                     ((uint64_t)u.bytes[6] « 48) |
01449                     ((uint64_t)u.bytes[7] « 56);
01450 #endif
01451 return true;
01452 }

```

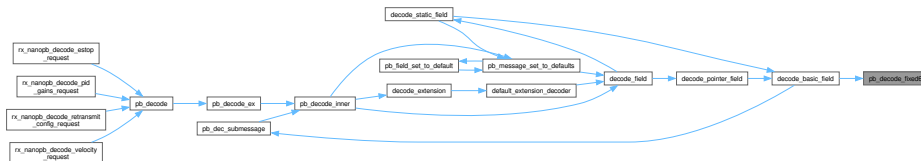
References [pb_read\(\)](#).

Referenced by [decode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.21 pb_decode_inner()

```

bool pb_decode_inner (
    pb_istream_t * stream,
    const pb_msgdesc_t * fields,
    void * dest_struct,
    unsigned int flags) [static]

```

Definition at line 1027 of file [pb_decode.c](#).

```

01028 {
01029 /* If the message contains extension fields, the extension handlers
01030  * are called when tag number is >= extension_range_start. This precheck
01031  * is just for speed, and the handlers will check for precise match.
01032  */
01033 uint32_t extension_range_start = 0;
01034 pb_extension_t *extensions = NULL;
01035
01036 /* 'fixed_count_field' and 'fixed_count_size' track position of a repeated fixed
01037  * count field. This can only handle _one_ repeated fixed count field that
01038  * is unpacked and unordered among other (non repeated fixed count) fields.
01039  */
01040 pb_size_t fixed_count_field = PB_SIZE_MAX;
01041 pb_size_t fixed_count_size = 0;
01042 pb_size_t fixed_count_total_size = 0;
01043
01044 /* Tag and wire type of next field from the input stream */
01045 uint32_t tag;
01046 pb_wire_type_t wire_type;
01047 bool eof;
01048

```

```

01049  /* Track presence of required fields */
01050  pb_fields_seen_t fields_seen = {{0, 0}};
01051  const uint32_t allbits = ~(uint32_t)0;
01052
01053  /* Descriptor for the structure field matching the tag decoded from stream */
01054  pb_field_iter_t iter;
01055
01056  if (pb_field_iter_begin(&iter, fields, dest_struct))
01057  {
01058      if ((flags & PB_DECODE_NOINIT) == 0)
01059      {
01060          if (!pb_message_set_to_defaults(&iter))
01061              PB_RETURN_ERROR(stream, "failed to set defaults");
01062      }
01063  }
01064
01065  while (pb_decode_tag(stream, &wire_type, &tag, &eof))
01066  {
01067      if (tag == 0)
01068      {
01069          if (flags & PB_DECODE_NULLTERMINATED)
01070          {
01071              eof = true;
01072              break;
01073          }
01074          else
01075          {
01076              PB_RETURN_ERROR(stream, "zero tag");
01077          }
01078      }
01079
01080      if (!pb_field_iter_find(&iter, tag) || PB_LTYPE(iter.type) == PB_LTYPE_EXTENSION)
01081      {
01082          /* No match found, check if it matches an extension. */
01083          if (extension_range_start == 0)
01084          {
01085              if (pb_field_iter_find_extension(&iter))
01086              {
01087                  extensions = *(pb_extension_t* const *)iter.pData;
01088                  extension_range_start = iter.tag;
01089              }
01090
01091              if (!extensions)
01092              {
01093                  extension_range_start = (uint32_t)-1;
01094              }
01095          }
01096
01097          if (tag >= extension_range_start)
01098          {
01099              size_t pos = stream->bytes_left;
01100
01101              if (!decode_extension(stream, tag, wire_type, extensions))
01102                  return false;
01103
01104              if (pos != stream->bytes_left)
01105              {
01106                  /* The field was handled */
01107                  continue;
01108              }
01109          }
01110
01111          /* No match found, skip data */
01112          if (!pb_skip_field(stream, wire_type))
01113              return false;
01114          continue;
01115      }
01116
01117      /* If a repeated fixed count field was found, get size from
01118       * 'fixed_count_field' as there is no counter contained in the struct.
01119       */
01120      if (PB_HTYPE(iter.type) == PB_HTYPE_REPEATED && iter.pSize == &iter.array_size)
01121      {
01122          if (fixed_count_field != iter.index) {
01123              /* If the new fixed count field does not match the previous one,
01124               * check that the previous one is NULL or that it finished
01125               * receiving all the expected data.
01126               */
01127              if (fixed_count_field != PB_SIZE_MAX &&
01128                  fixed_count_size != fixed_count_total_size)
01129              {
01130                  PB_RETURN_ERROR(stream, "wrong size for fixed count field");
01131              }
01132
01133              fixed_count_field = iter.index;
01134              fixed_count_size = 0;
01135              fixed_count_total_size = iter.array_size;

```

```

01136     }
01137
01138     iter.pSize = &fixed_count_size;
01139 }
01140
01141 if (PB_HTYPE(iter.type) == PB_HTYPE_REQUIRED
01142     && iter.required_field_index < PB_MAX_REQUIRED_FIELDS)
01143 {
01144     uint32_t tmp = ((uint32_t)1 << (iter.required_field_index & 31));
01145     fields_seen.bitfield[iter.required_field_index >> 5] |= tmp;
01146 }
01147
01148 if (!decode_field(stream, wire_type, &iter))
01149     return false;
01150 }
01151
01152 if (!eof)
01153 {
01154     /* pb_decode_tag() returned error before end of stream */
01155     return false;
01156 }
01157
01158 /* Check that all elements of the last decoded fixed count field were present. */
01159 if (fixed_count_field != PB_SIZE_MAX &&
01160     fixed_count_size != fixed_count_total_size)
01161 {
01162     PB_RETURN_ERROR(stream, "wrong size for fixed count field");
01163 }
01164
01165 /* Check that all required fields were present. */
01166 {
01167     pb_size_t req_field_count = iter.descriptor->required_field_count;
01168
01169     if (req_field_count > 0)
01170     {
01171         pb_size_t i;
01172
01173         if (req_field_count > PB_MAX_REQUIRED_FIELDS)
01174             req_field_count = PB_MAX_REQUIRED_FIELDS;
01175
01176         /* Check the whole words */
01177         for (i = 0; i < (req_field_count >> 5); i++)
01178         {
01179             if (fields_seen.bitfield[i] != allbits)
01180                 PB_RETURN_ERROR(stream, "missing required field");
01181         }
01182
01183         /* Check the remaining bits (if any) */
01184         if ((req_field_count & 31) != 0)
01185         {
01186             if (fields_seen.bitfield[req_field_count >> 5] !=
01187                 (allbits >> (uint_least8_t)(32 - (req_field_count & 31))))
01188             {
01189                 PB_RETURN_ERROR(stream, "missing required field");
01190             }
01191         }
01192     }
01193 }
01194
01195 return true;
01196 }

```

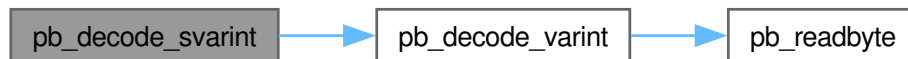
References [pb_fields_seen_t::bitfield](#), [checkreturn](#), [decode_extension\(\)](#), [decode_field\(\)](#), [NULL](#), [PB_DECODE_NOINIT](#), [PB_DECODE_NULLTERMINATED](#), [pb_decode_tag\(\)](#), [pb_field_iter_begin\(\)](#), [pb_field_iter_find\(\)](#), [pb_field_iter_find_extension\(\)](#), [PB_HTYPE](#), [PB_HTYPE_REPEATED](#), [PB_HTYPE_REQUIRED](#), [PB_LTYPE](#), [PB_LTYPE_EXTENSION](#), [PB_MAX_REQUIRED_FIELDS](#), [pb_message_set_to_defaults\(\)](#), [PB_RETURN_ERROR](#), [PB_SIZE_MAX](#), and [pb_skip_field\(\)](#).

Referenced by [pb_dec_submessage\(\)](#), and [pb_decode_ex\(\)](#).

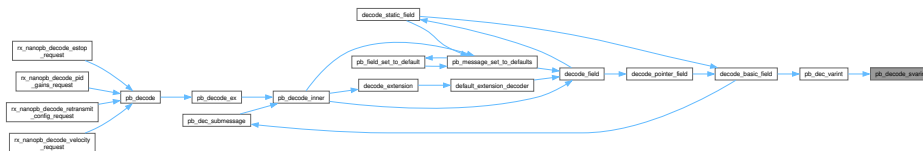
Here is the call graph for this function:



Generated by Doxygen



Here is the caller graph for this function:



6.61.3.23 pb_decode_tag()

```

bool pb_decode_tag (
    pb_istream_t * stream,
    pb_wire_type_t * wire_type,
    uint32_t * tag,
    bool * eof)

```

Definition at line 278 of file [pb_decode.c](#).

```

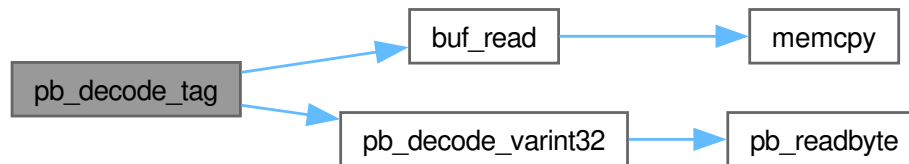
00279 {
00280     uint32_t temp;
00281     *eof = false;
00282     *wire_type = (pb_wire_type_t) 0;
00283     *tag = 0;
00284
00285     if (stream->bytes_left == 0)
00286     {
00287         *eof = true;
00288         return false;
00289     }
00290
00291     if (!pb_decode_varint32(stream, &temp))
00292     {
00293         #ifndef PB_BUFFER_ONLY
00294             /* Workaround for issue #1017
00295              *
00296              * Callback streams don't set bytes_left to 0 on eof until after being called by pb_decode_varint32,
00297              * which results in "io error" being raised. This contrasts the behavior of buffer streams who raise
00298              * no error on eof as bytes_left is already 0 on entry. This causes legitimate errors (e.g. missing
00299              * required fields) to be incorrectly reported by callback streams.
00300              */
00301             if (stream->callback != buf_read && stream->bytes_left == 0)
00302             {
00303                 #ifndef PB_NO_ERRMSG
00304                     if (strcmp(stream->errmsg, "io error") == 0)
00305                         stream->errmsg = NULL;
00306                 #endif
00307                 *eof = true;
00308             }
00309             #endif
00310             return false;
00311         }
00312
00313         *tag = temp >> 3;
00314         *wire_type = (pb_wire_type_t)(temp & 7);
00315         return true;
00316     }

```

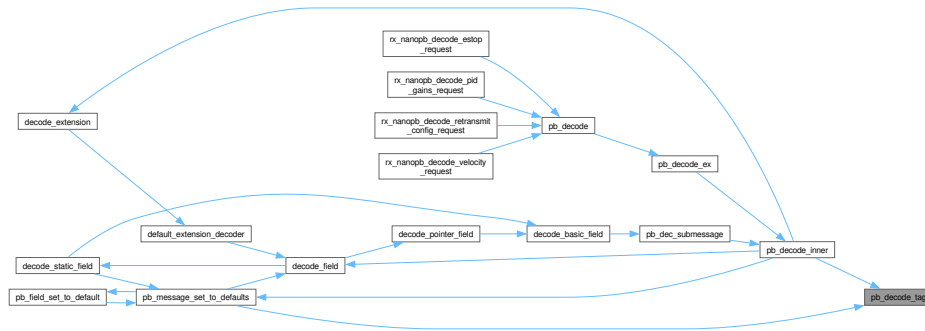
References [buf_read\(\)](#), [checkreturn](#), [NULL](#), and [pb_decode_varint32\(\)](#).

Referenced by [pb_decode_inner\(\)](#), and [pb_message_set_to_defaults\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.24 pb_decode_varint()

```

bool pb_decode_varint (
    pb_istream_t * stream,
    uint64_t * dest)
  
```

Definition at line 230 of file [pb_decode.c](#).

```

00231 {
00232     pb_byte_t byte;
00233     uint_fast8_t bitpos = 0;
00234     uint64_t result = 0;
00235
00236     do
00237     {
00238         if (!pb_readbyte(stream, &byte))
00239             return false;
00240
00241         if (bitpos >= 63 && (byte & 0xFE) != 0)
00242             PB_RETURN_ERROR(stream, "varint overflow");
00243
00244         result |= (uint64_t)(byte & 0x7F) << bitpos;
00245         bitpos = (uint_fast8_t)(bitpos + 7);
00246     } while (byte & 0x80);
00247
00248     *dest = result;
00249     return true;
00250 }
  
```

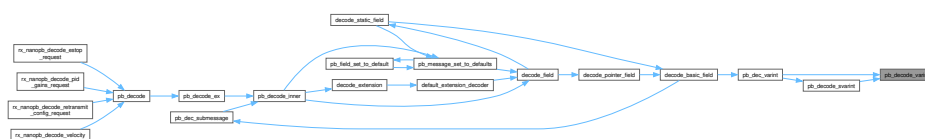
References [checkreturn](#), [pb_readbyte\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [pb_dec_varint\(\)](#), and [pb_decode_svarint\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.25 pb_decode_varint32()

```
bool pb_decode_varint32 (
    pb_istream_t * stream,
    uint32_t * dest)
```

Definition at line 171 of file [pb_decode.c](#).

```
00172 {
00173     pb_byte_t byte;
00174     uint32_t result;
00175
00176     if (!pb_readbyte(stream, &byte))
00177     {
00178         return false;
00179     }
00180
00181     if ((byte & 0x80) == 0)
00182     {
00183         /* Quick case, 1 byte value */
00184         result = byte;
00185     }
00186     else
00187     {
00188         /* Multibyte case */
00189         uint_fast8_t bitpos = 7;
00190         result = byte & 0x7F;
00191
00192         do
00193         {
00194             if (!pb_readbyte(stream, &byte))
00195                 return false;
00196
00197             if (bitpos >= 32)
00198             {
00199                 /* Note: The varint could have trailing 0x80 bytes, or 0xFF for negative. */
00200                 pb_byte_t sign_extension = (bitpos < 63) ? 0xFF : 0x01;
00201                 bool valid_extension = ((byte & 0x7F) == 0x00 ||
00202                                         ((result » 31) != 0 && byte == sign_extension));
00203
00204                 if (bitpos >= 64 || !valid_extension)
00205                 {
00206                     PB_RETURN_ERROR(stream, "varint overflow");
00207                 }
00208             }
00209             else if (bitpos == 28)
00210             {
00211                 if ((byte & 0x70) != 0 && (byte & 0x78) != 0x78)
00212                 {
00213                     PB_RETURN_ERROR(stream, "varint overflow");
00214                 }
00215                 result |= (uint32_t)(byte & 0x0F) « bitpos;
00216             }
00217             else
00218             {
00219                 result |= (uint32_t)(byte & 0x7F) « bitpos;
00220             }
00221             bitpos = (uint_fast8_t)(bitpos + 7);
00222         } while (byte & 0x80);
00223     }
00224
00225     *dest = result;
00226     return true;
00227 }
```

References [checkreturn](#), [pb_readbyte\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [pb_dec_bytes\(\)](#), [pb_dec_fixed_length_bytes\(\)](#), [pb_dec_string\(\)](#), [pb_decode_bool\(\)](#), [pb_decode_tag\(\)](#), [pb_make_string_substream\(\)](#), and [pb_skip_string\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

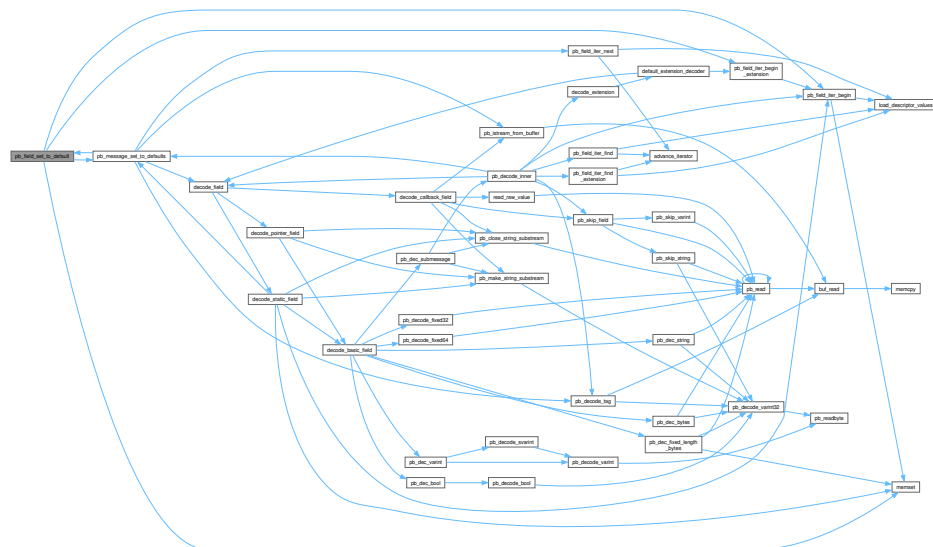


Definition at line 906 of file pb_decode.c.

Generated by Doxygen

References `memset()`, `NULL`, `PB_ATYPE`, `PB_ATYPE_CALLBACK`, `PB_ATYPE_POINTER`, `PB_ATYPE_STATIC`, `pb_field_iter_begin()`, `pb_field_iter_begin_extension()`, `PB_HTYPE`, `PB_HTYPE_ONEOF`, `PB_HTYPE_OPTIONAL`, `PB_HTYPE_REPEATED`, `PB_LTYPE`, `PB_LTYPE_EXTENSION`, `PB_LTYPE_IS_SUBMSG`, and `pb_message_set_to_defaults()`.

Here is the call graph for this function:



The diagram illustrates the ROS2 C++ decoder architecture. It shows a flow from various request types (e.g., `rx_nanopb_decode_estop_request`, `rx_nanopb_decode_pid_getme_request`, `rx_nanopb_decode_retransmit_config_request`, `rx_nanopb_decode_velocity_request`) through decoders (e.g., `pb_decode`, `pb_decode_ext`, `decode_field`, `decode_static_field`, `decode_pointer_field`, `decode_basic_field`, `pb_disc_submessage`, `pb_decode_inner`) to message sets (e.g., `pb_message_set_to_defaults`, `pb_field_set_to_defaults`). The architecture is modular, allowing for the addition of new decoders and message sets.

6.61.3.27 pb_istream_from_buffer()

```
pb_istream_t pb_istream_from_buffer (
    const pb_byte_t * buf,
    size_t msglen)
```

Definition at line 142 of file [pb_decode.c](#).

```
00143 {
00144     pb_istream_t stream;
00145     /* Cast away the const from buf without a compiler error. We are
00146      * careful to use it only in a const manner in the callbacks.
00147      */
00148     union {
00149         void *state;
00150         const void *c_state;
00151     } state;
00152     #ifndef PB_BUFFER_ONLY
00153     stream.callback = NULL;
00154     #else
00155     stream.callback = &buf_read;
00156     #endif
00157     state.c_state = buf;
00158     stream.state = state.state;
00159     stream.bytes_left = msglen;
00160     #ifndef PB_NO_ERRMSG
00161     stream.errmsg = NULL;
00162     #endif
00163     return stream;
00164 }
```

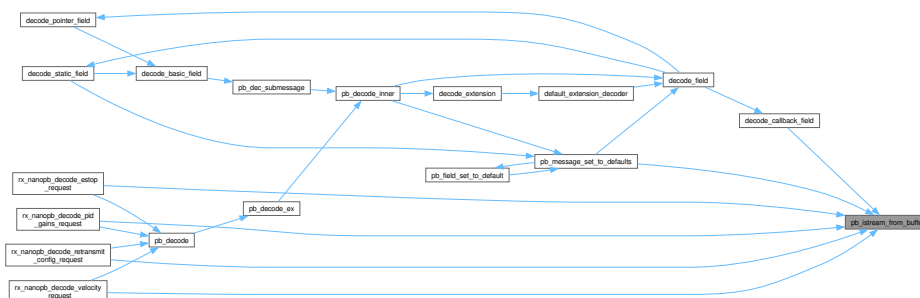
References [buf_read\(\)](#), and [NULL](#).

Referenced by [decode_callback_field\(\)](#), [pb_message_set_to_defaults\(\)](#), [rx_nanopb_decode_estop_request\(\)](#), [rx_nanopb_decode_pid_gains_request\(\)](#), [rx_nanopb_decode_retransmit_config_request\(\)](#), and [rx_nanopb_decode_velocity_request\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.28 pb_make_string_substream()

```
bool pb_make_string_substream (
    pb_istream_t * stream,
    pb_istream_t * substream)
```

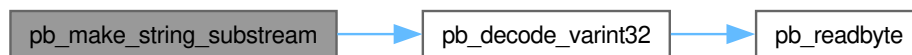
Definition at line 383 of file [pb_decode.c](#).

```
00384 {
00385     uint32_t size;
00386     if (!pb_decode_varint32(stream, &size))
00387         return false;
00388
00389     *substream = *stream;
00390     if (substream->bytes_left < size)
00391         PB_RETURN_ERROR(stream, "parent stream too short");
00392
00393     substream->bytes_left = (size_t)size;
00394     stream->bytes_left -= (size_t)size;
00395     return true;
00396 }
```

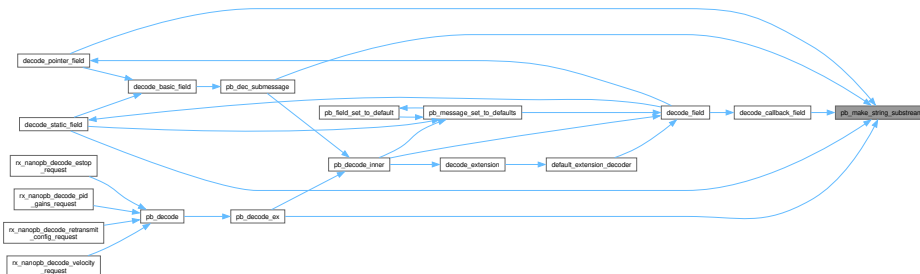
References [checkreturn](#), [pb_decode_varint32\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [decode_callback_field\(\)](#), [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [pb_dec_submessage\(\)](#), and [pb_decode_ex\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.29 pb_message_set_to_defaults()

```
bool pb_message_set_to_defaults (
    pb_field_iter_t * iter) [static]
```

Definition at line 988 of file [pb_decode.c](#).

```
00989 {
00990     pb_istream_t defstream = PB_ISTREAM_EMPTY;
00991     uint32_t tag = 0;
00992     pb_wire_type_t wire_type = PB_WT_VARINT;
00993     bool eof;
00994
00995     if (iter->descriptor->default_value)
00996     {
00997         defstream = pb_istream_from_buffer(iter->descriptor->default_value, (size_t)-1);
00998         if (!pb_decode_tag(&defstream, &wire_type, &tag, &eof))
00999             return false;
01000     }
```


6.61.3.30 pb_read()

```
bool pb_read (
    pb_istream_t * stream,
    pb_byte_t * buf,
    size_t count)
```

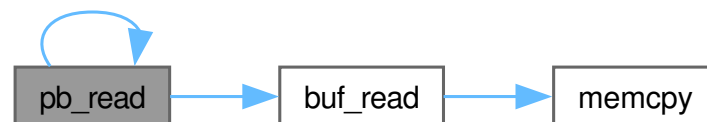
Definition at line 81 of file [pb_decode.c](#).

```
00082 {
00083     if (count == 0)
00084         return true;
00085
00086 #ifndef PB_BUFFER_ONLY
00087     if (buf == NULL && stream->callback != buf_read)
00088     {
00089         /* Skip input bytes */
00090         pb_byte_t tmp[16];
00091         while (count > 16)
00092         {
00093             if (!pb_read(stream, tmp, 16))
00094                 return false;
00095
00096             count -= 16;
00097         }
00098
00099         return pb_read(stream, tmp, count);
00100     }
00101 #endif
00102
00103     if (stream->bytes_left < count)
00104         PB_RETURN_ERROR(stream, "end-of-stream");
00105
00106 #ifndef PB_BUFFER_ONLY
00107     if (!stream->callback(stream, buf, count))
00108         PB_RETURN_ERROR(stream, "io error");
00109 #else
00110     if (!buf_read(stream, buf, count))
00111         return false;
00112 #endif
00113
00114     if (stream->bytes_left < count)
00115         stream->bytes_left = 0;
00116     else
00117         stream->bytes_left -= count;
00118
00119     return true;
00120 }
```

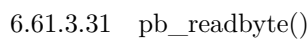
References [buf_read\(\)](#), [checkreturn](#), [NULL](#), [pb_read\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [pb_close_string_substream\(\)](#), [pb_dec_bytes\(\)](#), [pb_dec_fixed_length_bytes\(\)](#), [pb_dec_string\(\)](#), [pb_decode_fixed32\(\)](#), [pb_decode_fixed64\(\)](#), [pb_read\(\)](#), [pb_skip_field\(\)](#), [pb_skip_string\(\)](#), [pb_skip_varint\(\)](#), and [read_raw_value\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

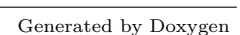


Definition at line 124 of file pb_decode.c.

References [checkreturn](#), and [PB_RETURN_ERROR](#).

Referenced by `pb_decode_varint()`, and `pb_decode_varint32()`.

Here is the caller graph for this function:



6.61.3.32 pb_release()

```
void pb_release (
    const pb_msgdesc_t * fields,
    void * dest_struct)
```

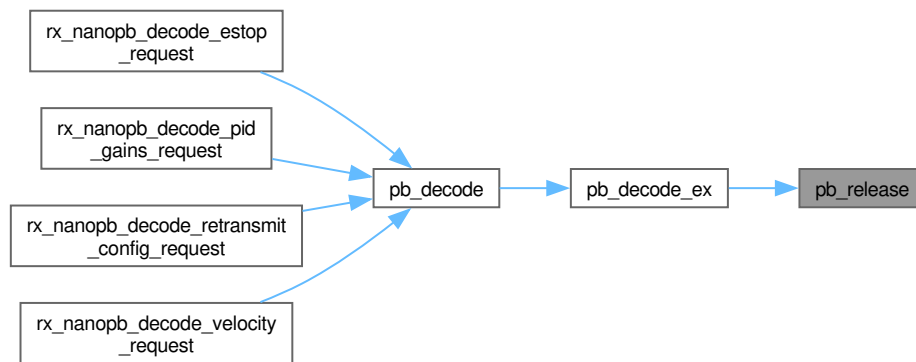
Definition at line 1371 of file [pb_decode.c](#).

```
01372 {
01373     /* Nothing to release without PB_ENABLE_MALLOC. */
01374     PB_UNUSED(fields);
01375     PB_UNUSED(dest_struct);
01376 }
```

References [PB_UNUSED](#).

Referenced by [pb_decode_ex\(\)](#).

Here is the caller graph for this function:



6.61.3.33 pb_skip_field()

```
bool pb_skip_field (
    pb_istream_t * stream,
    pb_wire_type_t wire_type)
```

Definition at line 318 of file [pb_decode.c](#).

```
00319 {
00320     switch (wire_type)
00321     {
00322         case PB_WT_VARINT: return pb_skip_varint(stream);
00323         case PB_WT_64BIT: return pb_read(stream, NULL, 8);
00324         case PB_WT_STRING: return pb_skip_string(stream);
00325         case PB_WT_32BIT: return pb_read(stream, NULL, 4);
00326         case PB_WT_PACKED:
00327             /* Calling pb_skip_field with a PB_WT_PACKED is an error.
00328              * Explicitly handle this case and fallthrough to default to avoid
00329              * compiler warnings.
00330              */
00331             default: PB_RETURN_ERROR(stream, "invalid wire_type");
00332     }
00333 }
```

References [checkreturn](#), [NULL](#), [pb_read\(\)](#), [PB_RETURN_ERROR](#), [pb_skip_string\(\)](#), [pb_skip_varint\(\)](#), [PB_WT_32BIT](#), [PB_WT_64BIT](#), [PB_WT_PACKED](#), [PB_WT_STRING](#), and [PB_WT_VARINT](#).

Referenced by [decode_callback_field\(\)](#), and [pb_decode_inner\(\)](#).

Here is the call graph for this function:

6.61.3.35 pb_skip_varint()

```
bool pb_skip_varint (
    pb_istream_t * stream)  [static]
```

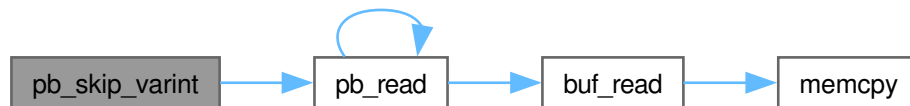
Definition at line 253 of file [pb_decode.c](#).

```
00254 {
00255     pb_byte_t byte;
00256     do
00257     {
00258         if (!pb_read(stream, &byte, 1))
00259             return false;
00260     } while (byte & 0x80);
00261     return true;
00262 }
```

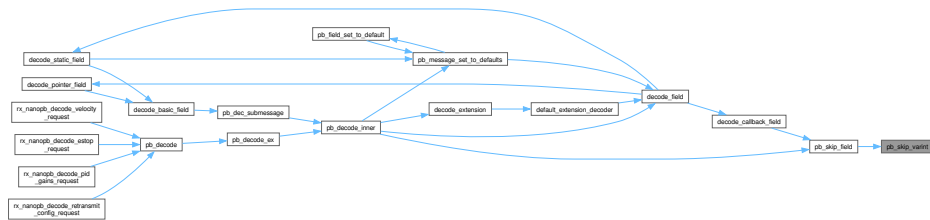
References [checkreturn](#), and [pb_read\(\)](#).

Referenced by [pb_skip_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.61.3.36 read_raw_value()

```
bool read_raw_value (
    pb_istream_t * stream,
    pb_wire_type_t wire_type,
    pb_byte_t * buf,
    size_t * size)  [static]
```

Definition at line 338 of file [pb_decode.c](#).

```
00339 {
00340     size_t max_size = *size;
00341     switch (wire_type)
00342     {
00343         case PB_WT_VARINT:
00344             *size = 0;
00345             do
00346             {
00347                 (*size)++;
00348                 if (*size > max_size)
00349                     PB_RETURN_ERROR(stream, "varint overflow");
00350                 if (!pb_read(stream, buf, 1))
00351                     return false;
00352             } while ((*buf)++ & 0x80);
00353     }
```

```

00354         return true;
00355
00356     case PB_WT_64BIT:
00357         *size = 8;
00358         return pb_read(stream, buf, 8);
00359
00360     case PB_WT_32BIT:
00361         *size = 4;
00362         return pb_read(stream, buf, 4);
00363
00364     case PB_WT_STRING:
00365         /* Calling read_raw_value with a PB_WT_STRING is an error.
00366          * Explicitly handle this case and fallthrough to default to avoid
00367          * compiler warnings.
00368          */
00369
00370     case PB_WT_PACKED:
00371         /* Calling read_raw_value with a PB_WT_PACKED is an error.
00372          * Explicitly handle this case and fallthrough to default to avoid
00373          * compiler warnings.
00374          */
00375
00376     default: PB_RETURN_ERROR(stream, "invalid wire_type");
00377 }
00378 }

```

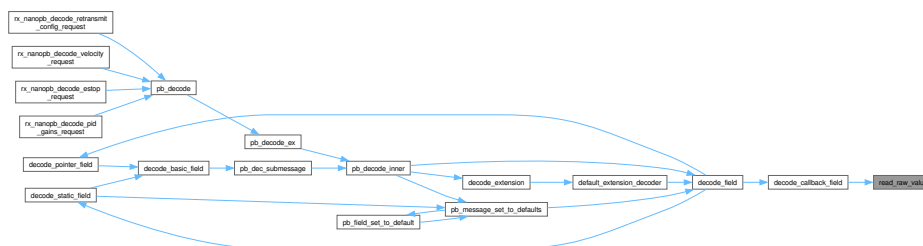
References [checkreturn](#), [pb_read\(\)](#), [PB_RETURN_ERROR](#), [PB_WT_32BIT](#), [PB_WT_64BIT](#), [PB_WT_PACKED](#), [PB_WT_STRING](#), and [PB_WT_VARINT](#).

Referenced by [decode_callback_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.62 pb_decode.c

[Go to the documentation of this file.](#)

```

00001 /* pb_decode.c -- decode a protobuf using minimal resources
00002  *
00003  * 2011 Petteri Aimonen <jpa@kapsi.fi>
00004  */
00005
00006 /* Use the GCC warn_unused_result attribute to check that all return values
00007  * are propagated correctly. On other compilers, gcc before 3.4.0 and iar
00008  * before 9.40.1 just ignore the annotation.
00009  */
00010 #if (defined(__GNUC__) && ((__GNUC__ > 3) || (__GNUC__ == 3 && __GNUC_MINOR__ >= 4))) || \
00011     (defined(__IAR_SYSTEMS_ICC__) && (__VER__ >= 9040001))
00012 #define checkreturn __attribute__((warn_unused_result))
00013 #else
00014 #define checkreturn
00015 #endif

```

```

00016
00017 #include "pb.h"
00018 #include "pb_decode.h"
00019 #include "pb_common.h"
00020
00021 /*****
00022  * Declarations internal to this file *
00023  *****/
00024
00025 static bool checkreturn buf_read(pb_istream_t *stream, pb_byte_t *buf, size_t count);
00026 static bool checkreturn read_raw_value(pb_istream_t *stream, pb_wire_type_t wire_type, pb_byte_t *buf, size_t
    *size);
00027 static bool checkreturn decode_basic_field(pb_istream_t *stream, pb_wire_type_t wire_type, pb_field_iter_t *field);
00028 static bool checkreturn decode_static_field(pb_istream_t *stream, pb_wire_type_t wire_type, pb_field_iter_t *field);
00029 static bool checkreturn decode_pointer_field(pb_istream_t *stream, pb_wire_type_t wire_type, pb_field_iter_t *field);
00030 static bool checkreturn decode_callback_field(pb_istream_t *stream, pb_wire_type_t wire_type, pb_field_iter_t *field);
00031 static bool checkreturn decode_field(pb_istream_t *stream, pb_wire_type_t wire_type, pb_field_iter_t *field);
00032 static bool checkreturn default_extension_decoder(pb_istream_t *stream, pb_extension_t *extension, uint32_t tag,
    pb_wire_type_t wire_type);
00033 static bool checkreturn decode_extension(pb_istream_t *stream, uint32_t tag, pb_wire_type_t wire_type,
    pb_extension_t *extension);
00034 static bool pb_field_set_to_default(pb_field_iter_t *field);
00035 static bool pb_message_set_to_defaults(pb_field_iter_t *iter);
00036 static bool checkreturn pb_dec_bool(pb_istream_t *stream, const pb_field_iter_t *field);
00037 static bool checkreturn pb_dec_varint(pb_istream_t *stream, const pb_field_iter_t *field);
00038 static bool checkreturn pb_dec_bytes(pb_istream_t *stream, const pb_field_iter_t *field);
00039 static bool checkreturn pb_dec_string(pb_istream_t *stream, const pb_field_iter_t *field);
00040 static bool checkreturn pb_dec_submessage(pb_istream_t *stream, const pb_field_iter_t *field);
00041 static bool checkreturn pb_dec_fixed_length_bytes(pb_istream_t *stream, const pb_field_iter_t *field);
00042 static bool checkreturn pb_skip_varint(pb_istream_t *stream);
00043 static bool checkreturn pb_skip_string(pb_istream_t *stream);
00044
00045 #ifndef PB_ENABLE_MALLOC
00046 static bool checkreturn allocate_field(pb_istream_t *stream, void *pData, size_t data_size, size_t array_size);
00047 static void initialize_pointer_field(void *pItem, pb_field_iter_t *field);
00048 static bool checkreturn pb_release_union_field(pb_istream_t *stream, pb_field_iter_t *field);
00049 static void pb_release_single_field(pb_field_iter_t *field);
00050 #endif
00051
00052 #ifndef PB_WITHOUT_64BIT
00053 #define pb_int64_t int32_t
00054 #define pb_uint64_t uint32_t
00055 #else
00056 #define pb_int64_t int64_t
00057 #define pb_uint64_t uint64_t
00058 #endif
00059
00060 typedef struct {
00061     uint32_t bitfield[(PB_MAX_REQUIRED_FIELDS + 31) / 32];
00062 } pb_fields_seen_t;
00063
00064 /*****
00065  * pb_istream_t implementation *
00066  *****/
00067
00068 static bool checkreturn buf_read(pb_istream_t *stream, pb_byte_t *buf, size_t count)
00069 {
00070     const pb_byte_t *source = (const pb_byte_t *)stream->state;
00071     stream->state = (pb_byte_t *)stream->state + count;
00072
00073     if (buf != NULL)
00074     {
00075         memcpy(buf, source, count * sizeof(pb_byte_t));
00076     }
00077
00078     return true;
00079 }
00080
00081 bool checkreturn pb_read(pb_istream_t *stream, pb_byte_t *buf, size_t count)
00082 {
00083     if (count == 0)
00084         return true;
00085
00086 #ifndef PB_BUFFER_ONLY
00087     if (buf == NULL && stream->callback != buf_read)
00088     {
00089         /* Skip input bytes */
00090         pb_byte_t tmp[16];
00091         while (count > 16)
00092         {
00093             if (!pb_read(stream, tmp, 16))
00094                 return false;
00095
00096             count -= 16;
00097         }
00098
00099         return pb_read(stream, tmp, count);
00099     }

```

```

00100     }
00101 #endif
00102
00103     if (stream->bytes_left < count)
00104         PB_RETURN_ERROR(stream, "end-of-stream");
00105
00106 #ifndef PB_BUFFER_ONLY
00107     if (!stream->callback(stream, buf, count))
00108         PB_RETURN_ERROR(stream, "io error");
00109 #else
00110     if (!buf_read(stream, buf, count))
00111         return false;
00112 #endif
00113
00114     if (stream->bytes_left < count)
00115         stream->bytes_left = 0;
00116     else
00117         stream->bytes_left -= count;
00118
00119     return true;
00120 }
00121
00122 /* Read a single byte from input stream. buf may not be NULL.
00123  * This is an optimization for the varint decoding. */
00124 static bool checkreturn pb_readbyte(pb_istream_t *stream, pb_byte_t *buf)
00125 {
00126     if (stream->bytes_left == 0)
00127         PB_RETURN_ERROR(stream, "end-of-stream");
00128
00129 #ifndef PB_BUFFER_ONLY
00130     if (!stream->callback(stream, buf, 1))
00131         PB_RETURN_ERROR(stream, "io error");
00132 #else
00133     *buf = *(const pb_byte_t*)stream->state;
00134     stream->state = (pb_byte_t*)stream->state + 1;
00135 #endif
00136
00137     stream->bytes_left--;
00138
00139     return true;
00140 }
00141
00142 pb_istream_t pb_istream_from_buffer(const pb_byte_t *buf, size_t msglen)
00143 {
00144     pb_istream_t stream;
00145     /* Cast away the const from buf without a compiler error. We are
00146      * careful to use it only in a const manner in the callbacks.
00147      */
00148     union {
00149         void *state;
00150         const void *c_state;
00151     } state;
00152 #ifdef PB_BUFFER_ONLY
00153     stream.callback = NULL;
00154 #else
00155     stream.callback = &buf_read;
00156 #endif
00157     state.c_state = buf;
00158     stream.state = state.state;
00159     stream.bytes_left = msglen;
00160 #ifndef PB_NO_ERRMSG
00161     stream.errmsg = NULL;
00162 #endif
00163     return stream;
00164 }
00165
00166
00167 /*****
00168  * Helper functions *
00169  *****/
00170
00171 bool checkreturn pb_decode_varint32(pb_istream_t *stream, uint32_t *dest)
00172 {
00173     pb_byte_t byte;
00174     uint32_t result;
00175
00176     if (!pb_readbyte(stream, &byte))
00177     {
00178         return false;
00179     }
00180
00181     if ((byte & 0x80) == 0)
00182     {
00183         /* Quick case, 1 byte value */
00184         result = byte;
00185     }
00186     else

```

```

00187 {
00188     /* Multibyte case */
00189     uint_fast8_t bitpos = 7;
00190     result = byte & 0x7F;
00191
00192     do
00193     {
00194         if (!pb_readbyte(stream, &byte))
00195             return false;
00196
00197         if (bitpos >= 32)
00198         {
00199             /* Note: The varint could have trailing 0x80 bytes, or 0xFF for negative. */
00200             pb_byte_t sign_extension = (bitpos < 63) ? 0xFF : 0x01;
00201             bool valid_extension = ((byte & 0x7F) == 0x00 ||
00202                                     ((result » 31) != 0 && byte == sign_extension));
00203
00204             if (bitpos >= 64 || !valid_extension)
00205             {
00206                 PB_RETURN_ERROR(stream, "varint overflow");
00207             }
00208         }
00209         else if (bitpos == 28)
00210         {
00211             if ((byte & 0x70) != 0 && (byte & 0x78) != 0x78)
00212             {
00213                 PB_RETURN_ERROR(stream, "varint overflow");
00214             }
00215             result |= (uint32_t)(byte & 0x0F) « bitpos;
00216         }
00217         else
00218         {
00219             result |= (uint32_t)(byte & 0x7F) « bitpos;
00220         }
00221         bitpos = (uint_fast8_t)(bitpos + 7);
00222     } while (byte & 0x80);
00223 }
00224
00225 *dest = result;
00226 return true;
00227 }
00228
00229 #ifndef PB_WITHOUT_64BIT
00230 bool checkreturn pb_decode_varint(pb_istream_t *stream, uint64_t *dest)
00231 {
00232     pb_byte_t byte;
00233     uint_fast8_t bitpos = 0;
00234     uint64_t result = 0;
00235
00236     do
00237     {
00238         if (!pb_readbyte(stream, &byte))
00239             return false;
00240
00241         if (bitpos >= 63 && (byte & 0xFE) != 0)
00242             PB_RETURN_ERROR(stream, "varint overflow");
00243
00244         result |= (uint64_t)(byte & 0x7F) « bitpos;
00245         bitpos = (uint_fast8_t)(bitpos + 7);
00246     } while (byte & 0x80);
00247
00248     *dest = result;
00249     return true;
00250 }
00251 #endif
00252
00253 bool checkreturn pb_skip_varint(pb_istream_t *stream)
00254 {
00255     pb_byte_t byte;
00256     do
00257     {
00258         if (!pb_read(stream, &byte, 1))
00259             return false;
00260     } while (byte & 0x80);
00261     return true;
00262 }
00263
00264 bool checkreturn pb_skip_string(pb_istream_t *stream)
00265 {
00266     uint32_t length;
00267     if (!pb_decode_varint32(stream, &length))
00268         return false;
00269
00270     if ((size_t)length != length)
00271     {
00272         PB_RETURN_ERROR(stream, "size too large");
00273     }

```

```

00274
00275     return pb_read(stream, NULL, (size_t)length);
00276 }
00277
00278 bool checkreturn pb_decode_tag(pb_istream_t *stream, pb_wire_type_t *wire_type, uint32_t *tag, bool *eof)
00279 {
00280     uint32_t temp;
00281     *eof = false;
00282     *wire_type = (pb_wire_type_t) 0;
00283     *tag = 0;
00284
00285     if (stream->bytes_left == 0)
00286     {
00287         *eof = true;
00288         return false;
00289     }
00290
00291     if (!pb_decode_varint32(stream, &temp))
00292     {
00293 #ifndef PB_BUFFER_ONLY
00294         /* Workaround for issue #1017
00295          *
00296          * Callback streams don't set bytes_left to 0 on eof until after being called by pb_decode_varint32,
00297          * which results in "io error" being raised. This contrasts the behavior of buffer streams who raise
00298          * no error on eof as bytes_left is already 0 on entry. This causes legitimate errors (e.g. missing
00299          * required fields) to be incorrectly reported by callback streams.
00300          */
00301         if (stream->callback != buf_read && stream->bytes_left == 0)
00302         {
00303 #ifndef PB_NO_ERRMSG
00304             if (strcmp(stream->errmsg, "io error") == 0)
00305                 stream->errmsg = NULL;
00306 #endif
00307             *eof = true;
00308         }
00309 #endif
00310         return false;
00311     }
00312
00313     *tag = temp >> 3;
00314     *wire_type = (pb_wire_type_t)(temp & 7);
00315     return true;
00316 }
00317
00318 bool checkreturn pb_skip_field(pb_istream_t *stream, pb_wire_type_t wire_type)
00319 {
00320     switch (wire_type)
00321     {
00322         case PB_WT_VARINT: return pb_skip_varint(stream);
00323         case PB_WT_64BIT: return pb_read(stream, NULL, 8);
00324         case PB_WT_STRING: return pb_skip_string(stream);
00325         case PB_WT_32BIT: return pb_read(stream, NULL, 4);
00326         case PB_WT_PACKED:
00327             /* Calling pb_skip_field with a PB_WT_PACKED is an error.
00328              * Explicitly handle this case and fallthrough to default to avoid
00329              * compiler warnings.
00330              */
00331             default: PB_RETURN_ERROR(stream, "invalid wire_type");
00332     }
00333 }
00334
00335 /* Read a raw value to buffer, for the purpose of passing it to callback as
00336  * a substream. Size is maximum size on call, and actual size on return.
00337  */
00338 static bool checkreturn read_raw_value(pb_istream_t *stream, pb_wire_type_t wire_type, pb_byte_t *buf, size_t
*size)
00339 {
00340     size_t max_size = *size;
00341     switch (wire_type)
00342     {
00343         case PB_WT_VARINT:
00344             *size = 0;
00345             do
00346             {
00347                 (*size)++;
00348                 if (*size > max_size)
00349                     PB_RETURN_ERROR(stream, "varint overflow");
00350
00351                 if (!pb_read(stream, buf, 1))
00352                     return false;
00353             } while ((*buf++ & 0x80));
00354             return true;
00355
00356         case PB_WT_64BIT:
00357             *size = 8;
00358             return pb_read(stream, buf, 8);
00359

```

```

00360     case PB_WT_32BIT:
00361         *size = 4;
00362         return pb_read(stream, buf, 4);
00363
00364     case PB_WT_STRING:
00365         /* Calling read_raw_value with a PB_WT_STRING is an error.
00366          * Explicitly handle this case and fallthrough to default to avoid
00367          * compiler warnings.
00368          */
00369
00370     case PB_WT_PACKED:
00371         /* Calling read_raw_value with a PB_WT_PACKED is an error.
00372          * Explicitly handle this case and fallthrough to default to avoid
00373          * compiler warnings.
00374          */
00375
00376     default: PB_RETURN_ERROR(stream, "invalid wire_type");
00377 }
00378 }
00379
00380 /* Decode string length from stream and return a substream with limited length.
00381  * Remember to close the substream using pb_close_string_substream().
00382  */
00383 bool checkreturn pb_make_string_substream(pb_istream_t *stream, pb_istream_t *substream)
00384 {
00385     uint32_t size;
00386     if (!pb_decode_varint32(stream, &size))
00387         return false;
00388
00389     *substream = *stream;
00390     if (substream->bytes_left < size)
00391         PB_RETURN_ERROR(stream, "parent stream too short");
00392
00393     substream->bytes_left = (size_t)size;
00394     stream->bytes_left -= (size_t)size;
00395     return true;
00396 }
00397
00398 bool checkreturn pb_close_string_substream(pb_istream_t *stream, pb_istream_t *substream)
00399 {
00400     if (substream->bytes_left) {
00401         if (!pb_read(substream, NULL, substream->bytes_left))
00402             return false;
00403     }
00404
00405     stream->state = substream->state;
00406
00407 #ifndef PB_NO_ERRMSG
00408     stream->errmsg = substream->errmsg;
00409 #endif
00410     return true;
00411 }
00412
00413 /*****
00414  * Decode a single field *
00415  *****/
00416
00417 static bool checkreturn decode_basic_field(pb_istream_t *stream, pb_wire_type_t wire_type, pb_field_iter_t *field)
00418 {
00419     switch (PB_LTYPE(field->type))
00420     {
00421     case PB_LTYPE_BOOL:
00422         if (wire_type != PB_WT_VARINT && wire_type != PB_WT_PACKED)
00423             PB_RETURN_ERROR(stream, "wrong wire type");
00424
00425         return pb_dec_bool(stream, field);
00426
00427     case PB_LTYPE_VARINT:
00428     case PB_LTYPE_UVARINT:
00429     case PB_LTYPE_SVARINT:
00430         if (wire_type != PB_WT_VARINT && wire_type != PB_WT_PACKED)
00431             PB_RETURN_ERROR(stream, "wrong wire type");
00432
00433         return pb_dec_varint(stream, field);
00434
00435     case PB_LTYPE_FIXED32:
00436         if (wire_type != PB_WT_32BIT && wire_type != PB_WT_PACKED)
00437             PB_RETURN_ERROR(stream, "wrong wire type");
00438
00439         return pb_decode_fixed32(stream, field->pData);
00440
00441     case PB_LTYPE_FIXED64:
00442         if (wire_type != PB_WT_64BIT && wire_type != PB_WT_PACKED)
00443             PB_RETURN_ERROR(stream, "wrong wire type");
00444
00445 #ifndef PB_CONVERT_DOUBLE_FLOAT
00446         if (field->data_size == sizeof(float))

```

```

00447     {
00448         return pb_decode_double_as_float(stream, (float*)field->pData);
00449     }
00450 #endif
00451
00452 #ifdef PB_WITHOUT_64BIT
00453     PB_RETURN_ERROR(stream, "invalid data_size");
00454 #else
00455     return pb_decode_fixed64(stream, field->pData);
00456 #endif
00457
00458     case PB_LTYPE_BYTES:
00459         if (wire_type != PB_WT_STRING)
00460             PB_RETURN_ERROR(stream, "wrong wire type");
00461
00462         return pb_dec_bytes(stream, field);
00463
00464     case PB_LTYPE_STRING:
00465         if (wire_type != PB_WT_STRING)
00466             PB_RETURN_ERROR(stream, "wrong wire type");
00467
00468         return pb_dec_string(stream, field);
00469
00470     case PB_LTYPE_SUBMESSAGE:
00471     case PB_LTYPE_SUBMSG_W_CB:
00472         if (wire_type != PB_WT_STRING)
00473             PB_RETURN_ERROR(stream, "wrong wire type");
00474
00475         return pb_dec_submessage(stream, field);
00476
00477     case PB_LTYPE_FIXED_LENGTH_BYTES:
00478         if (wire_type != PB_WT_STRING)
00479             PB_RETURN_ERROR(stream, "wrong wire type");
00480
00481         return pb_dec_fixed_length_bytes(stream, field);
00482
00483     default:
00484         PB_RETURN_ERROR(stream, "invalid field type");
00485 }
00486 }
00487
00488 static bool checkreturn decode_static_field(pb_istream_t *stream, pb_wire_type_t wire_type, pb_field_iter_t *field)
00489 {
00490     switch (PB_HTYPE(field->type))
00491     {
00492     case PB_HTYPE_REQUIRED:
00493         return decode_basic_field(stream, wire_type, field);
00494
00495     case PB_HTYPE_OPTIONAL:
00496         if (field->pSize != NULL)
00497             *(bool*)field->pSize = true;
00498         return decode_basic_field(stream, wire_type, field);
00499
00500     case PB_HTYPE_REPEATED:
00501         if (wire_type == PB_WT_STRING
00502             && PB_LTYPE(field->type) <= PB_LTYPE_LAST_PACKABLE)
00503         {
00504             /* Packed array */
00505             bool status = true;
00506             pb_istream_t substream;
00507             pb_size_t *size = (pb_size_t*)field->pSize;
00508             field->pData = (char*)field->pField + field->data_size * (*size);
00509
00510             if (!pb_make_string_substream(stream, &substream))
00511                 return false;
00512
00513             while (substream.bytes_left > 0 && *size < field->array_size)
00514             {
00515                 if (!decode_basic_field(&substream, PB_WT_PACKED, field))
00516                 {
00517                     status = false;
00518                     break;
00519                 }
00520                 (*size)++;
00521                 field->pData = (char*)field->pData + field->data_size;
00522             }
00523
00524             if (substream.bytes_left != 0)
00525                 PB_RETURN_ERROR(stream, "array overflow");
00526             if (!pb_close_string_substream(stream, &substream))
00527                 return false;
00528
00529             return status;
00530         }
00531         else
00532         {
00533             /* Repeated field */

```



```

00534     pb_size_t *size = (pb_size_t*)field->pSize;
00535     field->pData = (char*)field->pField + field->data_size * (*size);
00536
00537     if ((*size)++ >= field->array_size)
00538         PB_RETURN_ERROR(stream, "array overflow");
00539
00540     return decode_basic_field(stream, wire_type, field);
00541 }
00542
00543 case PB_HTYPE_ONEOF:
00544     if (PB_LTYPE_IS_SUBMSG(field->type) &&
00545         *(pb_size_t*)field->pSize != field->tag)
00546     {
00547         /* We memset to zero so that any callbacks are set to NULL.
00548          * This is because the callbacks might otherwise have values
00549          * from some other union field.
00550          * If callbacks are needed inside oneof field, use .proto
00551          * option submsg_callback to have a separate callback function
00552          * that can set the fields before submessage is decoded.
00553          * pb_dec_submessage() will set any default values. */
00554         memset(field->pData, 0, (size_t)field->data_size);
00555
00556         /* Set default values for the submessage fields. */
00557         if (field->submsg_desc->default_value != NULL ||
00558             field->submsg_desc->field_callback != NULL ||
00559             field->submsg_desc->submsg_info[0] != NULL)
00560         {
00561             pb_field_iter_t submsg_iter;
00562             if (pb_field_iter_begin(&submsg_iter, field->submsg_desc, field->pData))
00563             {
00564                 if (!pb_message_set_to_defaults(&submsg_iter))
00565                     PB_RETURN_ERROR(stream, "failed to set defaults");
00566             }
00567         }
00568     }
00569     *(pb_size_t*)field->pSize = field->tag;
00570
00571     return decode_basic_field(stream, wire_type, field);
00572
00573 default:
00574     PB_RETURN_ERROR(stream, "invalid field type");
00575 }
00576 }
00577
00578 #ifndef PB_ENABLE_MALLOC
00579 /* Allocate storage for the field and store the pointer at iter->pData.
00580  * array_size is the number of entries to reserve in an array.
00581  * Zero size is not allowed, use pb_free() for releasing.
00582  */
00583 static bool checkreturn allocate_field(pb_istream_t *stream, void **pData, size_t data_size, size_t array_size)
00584 {
00585     void *ptr = *(void**)pData;
00586
00587     if (data_size == 0 || array_size == 0)
00588         PB_RETURN_ERROR(stream, "invalid size");
00589
00590 #ifdef __AVR__
00591     /* Workaround for AVR libc bug 53284: http://savannah.nongnu.org/bugs/?53284
00592      * Realloc to size of 1 byte can cause corruption of the malloc structures.
00593      */
00594     if (data_size == 1 && array_size == 1)
00595     {
00596         data_size = 2;
00597     }
00598 #endif
00599
00600     /* Check for multiplication overflows.
00601     * This code avoids the costly division if the sizes are small enough.
00602     * Multiplication is safe as long as only half of bits are set
00603     * in either multiplicand.
00604     */
00605     {
00606         const size_t check_limit = (size_t)1 << (sizeof(size_t) * 4);
00607         if (data_size >= check_limit || array_size >= check_limit)
00608         {
00609             const size_t size_max = (size_t)-1;
00610             if (size_max / array_size < data_size)
00611             {
00612                 PB_RETURN_ERROR(stream, "size too large");
00613             }
00614         }
00615     }
00616
00617     /* Allocate new or expand previous allocation */
00618     /* Note: on failure the old pointer will remain in the structure,
00619     * the message must be freed by caller also on error return. */
00620     ptr = pb_realloc(ptr, array_size * data_size);

```

```

00621     if (ptr == NULL)
00622         PB_RETURN_ERROR(stream, "realloc failed");
00623
00624     *(void**)pData = ptr;
00625     return true;
00626 }
00627
00628 /* Clear a newly allocated item in case it contains a pointer, or is a submessage. */
00629 static void initialize_pointer_field(void *pItem, pb_field_iter_t *field)
00630 {
00631     if (PB_LTYPE(field->type) == PB_LTYPE_STRING ||
00632         PB_LTYPE(field->type) == PB_LTYPE_BYTES)
00633     {
00634         *(void**)pItem = NULL;
00635     }
00636     else if (PB_LTYPE_IS_SUBMSG(field->type))
00637     {
00638         /* We memset to zero so that any callbacks are set to NULL.
00639          * Default values will be set by pb_dec_submessage(). */
00640         memset(pItem, 0, field->data_size);
00641     }
00642 }
00643 #endif
00644
00645 static bool checkreturn decode_pointer_field(pb_istream_t *stream, pb_wire_type_t wire_type, pb_field_iter_t *field)
00646 {
00647     #ifndef PB_ENABLE_MALLOC
00648         PB_UNUSED(wire_type);
00649         PB_UNUSED(field);
00650         PB_RETURN_ERROR(stream, "no malloc support");
00651     #else
00652         switch (PB_HTYPE(field->type))
00653         {
00654             case PB_HTYPE_REQUIRED:
00655             case PB_HTYPE_OPTIONAL:
00656             case PB_HTYPE_ONEOF:
00657                 if (PB_LTYPE_IS_SUBMSG(field->type) && *(void**)field->pField != NULL)
00658                 {
00659                     /* Duplicate field, have to release the old allocation first. */
00660                     /* FIXME: Does this work correctly for oneofs? */
00661                     pb_release_single_field(field);
00662                 }
00663
00664                 if (PB_HTYPE(field->type) == PB_HTYPE_ONEOF)
00665                 {
00666                     *(pb_size_t*)field->pSize = field->tag;
00667                 }
00668
00669                 if (PB_LTYPE(field->type) == PB_LTYPE_STRING ||
00670                     PB_LTYPE(field->type) == PB_LTYPE_BYTES)
00671                 {
00672                     /* pb_dec_string and pb_dec_bytes handle allocation themselves */
00673                     field->pData = field->pField;
00674                     return decode_basic_field(stream, wire_type, field);
00675                 }
00676                 else
00677                 {
00678                     if (!allocate_field(stream, field->pField, field->data_size, 1))
00679                         return false;
00680
00681                     field->pData = *(void**)field->pField;
00682                     initialize_pointer_field(field->pData, field);
00683                     return decode_basic_field(stream, wire_type, field);
00684                 }
00685
00686             case PB_HTYPE_REPEATED:
00687                 if (wire_type == PB_WT_STRING
00688                     && PB_LTYPE(field->type) <= PB_LTYPE_LAST_PACKABLE)
00689                 {
00690                     /* Packed array, multiple items come in at once. */
00691                     bool status = true;
00692                     pb_size_t *size = (pb_size_t*)field->pSize;
00693                     size_t allocated_size = *size;
00694                     pb_istream_t substream;
00695
00696                     if (!pb_make_string_substream(stream, &substream))
00697                         return false;
00698
00699                     while (substream.bytes_left)
00700                     {
00701                         if (*size == PB_SIZE_MAX)
00702                         {
00703                             #ifndef PB_NO_ERRMSG
00704                                 stream->errmsg = "too many array entries";
00705                             #endif
00706                             status = false;
00707                             break;

```

```

00708     }
00709
00710     if ((size_t)*size + 1 > allocated_size)
00711     {
00712         /* Allocate more storage. This tries to guess the
00713          * number of remaining entries. Round the division
00714          * upwards. */
00715         size_t remain = (substream.bytes_left - 1) / field->data_size + 1;
00716         if (remain < PB_SIZE_MAX - allocated_size)
00717             allocated_size += remain;
00718         else
00719             allocated_size += 1;
00720
00721         if (!allocate_field(&substream, field->pField, field->data_size, allocated_size))
00722         {
00723             status = false;
00724             break;
00725         }
00726     }
00727
00728     /* Decode the array entry */
00729     field->pData = *(char**)field->pField + field->data_size * (*size);
00730     if (field->pData == NULL)
00731     {
00732         /* Shouldn't happen, but satisfies static analyzers */
00733         status = false;
00734         break;
00735     }
00736     initialize_pointer_field(field->pData, field);
00737     if (!decode_basic_field(&substream, PB_WT_PACKED, field))
00738     {
00739         status = false;
00740         break;
00741     }
00742     (*size)++;
00743 }
00744
00745 if (!pb_close_string_substream(stream, &substream))
00746     return false;
00747
00748 return status;
00749 }
00750 else
00751 {
00752     /* Normal repeated field, i.e. only one item at a time. */
00753     pb_size_t *size = (pb_size_t*)field->pSize;
00754
00755     if (*size == PB_SIZE_MAX)
00756         PB_RETURN_ERROR(stream, "too many array entries");
00757
00758     if (!allocate_field(stream, field->pField, field->data_size, (size_t)(*size + 1)))
00759         return false;
00760
00761     field->pData = *(char**)field->pField + field->data_size * (*size);
00762     (*size)++;
00763     initialize_pointer_field(field->pData, field);
00764     return decode_basic_field(stream, wire_type, field);
00765 }
00766
00767 default:
00768     PB_RETURN_ERROR(stream, "invalid field type");
00769 }
00770 #endif
00771 }
00772
00773 static bool checkreturn decode_callback_field(pb_istream_t *stream, pb_wire_type_t wire_type, pb_field_iter_t *field)
00774 {
00775     if (!field->descriptor->field_callback)
00776         return pb_skip_field(stream, wire_type);
00777
00778     if (wire_type == PB_WT_STRING)
00779     {
00780         pb_istream_t substream;
00781         size_t prev_bytes_left;
00782
00783         if (!pb_make_string_substream(stream, &substream))
00784             return false;
00785
00786         /* If the callback field is inside a submsg, first call the submsg_callback which
00787          * should set the decoder for the callback field. */
00788         if (PB_LTYPE(field->type) == PB_LTYPE_SUBMSG_W_CB && field->pSize != NULL) {
00789             pb_callback_t* callback;
00790             *(pb_size_t*)field->pSize = field->tag;
00791             callback = (pb_callback_t*)field->pSize - 1;
00792
00793             if (callback->funcs.decode)
00794             {

```

```

00795         if (!callback->funcs.decode(&substream, field, &callback->arg)) {
00796             PB_SET_ERROR(stream, substream.errmsg ? substream.errmsg : "submsg callback failed");
00797             return false;
00798         }
00799     }
00800 }
00801
00802 do
00803 {
00804     prev_bytes_left = substream.bytes_left;
00805     if (!field->descriptor->field_callback(&substream, NULL, field))
00806     {
00807         PB_SET_ERROR(stream, substream.errmsg ? substream.errmsg : "callback failed");
00808         return false;
00809     }
00810 } while (substream.bytes_left > 0 && substream.bytes_left < prev_bytes_left);
00811
00812 if (!pb_close_string_substream(stream, &substream))
00813     return false;
00814
00815 return true;
00816 }
00817 else
00818 {
00819     /* Copy the single scalar value to stack.
00820     * This is required so that we can limit the stream length,
00821     * which in turn allows to use same callback for packed and
00822     * not-packed fields. */
00823     pb_istream_t substream;
00824     pb_byte_t buffer[10];
00825     size_t size = sizeof(buffer);
00826
00827     if (!read_raw_value(stream, wire_type, buffer, &size))
00828         return false;
00829     substream = pb_istream_from_buffer(buffer, size);
00830
00831     return field->descriptor->field_callback(&substream, NULL, field);
00832 }
00833 }
00834
00835 static bool checkreturn decode_field(pb_istream_t *stream, pb_wire_type_t wire_type, pb_field_iter_t *field)
00836 {
00837 #ifdef PB_ENABLE_MALLOC
00838     /* When decoding an oneof field, check if there is old data that must be
00839     * released first. */
00840     if (PB_HTYPE(field->type) == PB_HTYPE_ONEOF)
00841     {
00842         if (!pb_release_union_field(stream, field))
00843             return false;
00844     }
00845 #endif
00846
00847     switch (PB_ATYPE(field->type))
00848     {
00849     case PB_ATYPE_STATIC:
00850         return decode_static_field(stream, wire_type, field);
00851
00852     case PB_ATYPE_POINTER:
00853         return decode_pointer_field(stream, wire_type, field);
00854
00855     case PB_ATYPE_CALLBACK:
00856         return decode_callback_field(stream, wire_type, field);
00857
00858     default:
00859         PB_RETURN_ERROR(stream, "invalid field type");
00860     }
00861 }
00862
00863 /* Default handler for extension fields. Expects to have a pb_msgdesc_t
00864 * pointer in the extension->type->arg field, pointing to a message with
00865 * only one field in it. */
00866 static bool checkreturn default_extension_decoder(pb_istream_t *stream,
00867 pb_extension_t *extension, uint32_t tag, pb_wire_type_t wire_type)
00868 {
00869     pb_field_iter_t iter;
00870
00871     if (!pb_field_iter_begin_extension(&iter, extension))
00872         PB_RETURN_ERROR(stream, "invalid extension");
00873
00874     if (iter.tag != tag || !iter.message)
00875         return true;
00876
00877     extension->found = true;
00878     return decode_field(stream, wire_type, &iter);
00879 }
00880
00881 /* Try to decode an unknown field as an extension field. Tries each extension

```

```

00882 /* decoder in turn, until one of them handles the field or loop ends. */
00883 static bool checkreturn decode_extension(pb_istream_t *stream,
00884     uint32_t tag, pb_wire_type_t wire_type, pb_extension_t *extension)
00885 {
00886     size_t pos = stream->bytes_left;
00887
00888     while (extension != NULL && pos == stream->bytes_left)
00889     {
00890         bool status;
00891         if (extension->type->decode)
00892             status = extension->type->decode(stream, extension, tag, wire_type);
00893         else
00894             status = default_extension_decoder(stream, extension, tag, wire_type);
00895
00896         if (!status)
00897             return false;
00898
00899         extension = extension->next;
00900     }
00901
00902     return true;
00903 }
00904
00905 /* Initialize message fields to default values, recursively */
00906 static bool pb_field_set_to_default(pb_field_iter_t *field)
00907 {
00908     pb_type_t type;
00909     type = field->type;
00910
00911     if (PB_LTYPE(type) == PB_LTYPE_EXTENSION)
00912     {
00913         pb_extension_t *ext = *(pb_extension_t* const *)field->pData;
00914         while (ext != NULL)
00915         {
00916             pb_field_iter_t ext_iter;
00917             if (pb_field_iter_begin_extension(&ext_iter, ext))
00918             {
00919                 ext->found = false;
00920                 if (!pb_message_set_to_defaults(&ext_iter))
00921                     return false;
00922             }
00923             ext = ext->next;
00924         }
00925     }
00926     else if (PB_ATYPE(type) == PB_ATYPE_STATIC)
00927     {
00928         bool init_data = true;
00929         if (PB_HTYPE(type) == PB_HTYPE_OPTIONAL && field->pSize != NULL)
00930         {
00931             /* Set has_field to false. Still initialize the optional field
00932              * itself also. */
00933             *(bool*)field->pSize = false;
00934         }
00935         else if (PB_HTYPE(type) == PB_HTYPE_REPEATED ||
00936             PB_HTYPE(type) == PB_HTYPE_ONEOF)
00937         {
00938             /* REPEATED: Set array count to 0, no need to initialize contents.
00939              * ONEOF: Set which_field to 0. */
00940             *(pb_size_t*)field->pSize = 0;
00941             init_data = false;
00942         }
00943
00944         if (init_data)
00945         {
00946             if (PB_LTYPE_IS_SUBMSG(field->type) &&
00947                 (field->submsg_desc->default_value != NULL ||
00948                 field->submsg_desc->field_callback != NULL ||
00949                 field->submsg_desc->submsg_info[0] != NULL))
00950             {
00951                 /* Initialize submessage to defaults.
00952                  * Only needed if it has default values
00953                  * or callback/submessage fields. */
00954                 pb_field_iter_t submsg_iter;
00955                 if (pb_field_iter_begin(&submsg_iter, field->submsg_desc, field->pData))
00956                 {
00957                     if (!pb_message_set_to_defaults(&submsg_iter))
00958                         return false;
00959                 }
00960             }
00961             else
00962             {
00963                 /* Initialize to zeros */
00964                 memset(field->pData, 0, (size_t)field->data_size);
00965             }
00966         }
00967     }
00968     else if (PB_ATYPE(type) == PB_ATYPE_POINTER)

```

```

00969 {
00970     /* Initialize the pointer to NULL. */
00971     *(void**)field->pField = NULL;
00972
00973     /* Initialize array count to 0. */
00974     if (PB_HTYPE(type) == PB_HTYPE_REPEATED ||
00975         PB_HTYPE(type) == PB_HTYPE_ONEOF)
00976     {
00977         *(pb_size_t*)field->pSize = 0;
00978     }
00979 }
00980 else if (PB_ATYPE(type) == PB_ATYPE_CALLBACK)
00981 {
00982     /* Don't overwrite callback */
00983 }
00984
00985 return true;
00986 }
00987
00988 static bool pb_message_set_to_defaults(pb_field_iter_t *iter)
00989 {
00990     pb_istream_t defstream = PB_ISTREAM_EMPTY;
00991     uint32_t tag = 0;
00992     pb_wire_type_t wire_type = PB_WT_VARINT;
00993     bool eof;
00994
00995     if (iter->descriptor->default_value)
00996     {
00997         defstream = pb_istream_from_buffer(iter->descriptor->default_value, (size_t)-1);
00998         if (!pb_decode_tag(&defstream, &wire_type, &tag, &eof))
00999             return false;
01000     }
01001
01002     do
01003     {
01004         if (!pb_field_set_to_default(iter))
01005             return false;
01006
01007         if (tag != 0 && iter->tag == tag)
01008         {
01009             /* We have a default value for this field in the defstream */
01010             if (!decode_field(&defstream, wire_type, iter))
01011                 return false;
01012             if (!pb_decode_tag(&defstream, &wire_type, &tag, &eof))
01013                 return false;
01014
01015             if (iter->pSize)
01016                 *(bool*)iter->pSize = false;
01017         }
01018     } while (pb_field_iter_next(iter));
01019
01020     return true;
01021 }
01022
01023 /*****
01024  * Decode all fields *
01025  *****/
01026
01027 static bool checkreturn pb_decode_inner(pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct, unsigned
int flags)
01028 {
01029     /* If the message contains extension fields, the extension handlers
01030     * are called when tag number is >= extension_range_start. This precheck
01031     * is just for speed, and the handlers will check for precise match.
01032     */
01033     uint32_t extension_range_start = 0;
01034     pb_extension_t *extensions = NULL;
01035
01036     /* 'fixed_count_field' and 'fixed_count_size' track position of a repeated fixed
01037     * count field. This can only handle _one_ repeated fixed count field that
01038     * is unpacked and unordered among other (non repeated fixed count) fields.
01039     */
01040     pb_size_t fixed_count_field = PB_SIZE_MAX;
01041     pb_size_t fixed_count_size = 0;
01042     pb_size_t fixed_count_total_size = 0;
01043
01044     /* Tag and wire type of next field from the input stream */
01045     uint32_t tag;
01046     pb_wire_type_t wire_type;
01047     bool eof;
01048
01049     /* Track presence of required fields */
01050     pb_fields_seen_t fields_seen = {{0, 0}};
01051     const uint32_t allbits = ~(uint32_t)0;
01052
01053     /* Descriptor for the structure field matching the tag decoded from stream */
01054     pb_field_iter_t iter;

```

```

01055
01056 if (pb_field_iter_begin(&iter, fields, dest_struct))
01057 {
01058     if ((flags & PB_DECODE_NOINIT) == 0)
01059     {
01060         if (!pb_message_set_to_defaults(&iter))
01061             PB_RETURN_ERROR(stream, "failed to set defaults");
01062     }
01063 }
01064
01065 while (pb_decode_tag(stream, &wire_type, &tag, &eof))
01066 {
01067     if (tag == 0)
01068     {
01069         if (flags & PB_DECODE_NULLTERMINATED)
01070         {
01071             eof = true;
01072             break;
01073         }
01074         else
01075         {
01076             PB_RETURN_ERROR(stream, "zero tag");
01077         }
01078     }
01079
01080     if (!pb_field_iter_find(&iter, tag) || PB_LTYPE(iter.type) == PB_LTYPE_EXTENSION)
01081     {
01082         /* No match found, check if it matches an extension. */
01083         if (extension_range_start == 0)
01084         {
01085             if (pb_field_iter_find_extension(&iter))
01086             {
01087                 extensions = *(pb_extension_t* const *)iter.pData;
01088                 extension_range_start = iter.tag;
01089             }
01090
01091             if (!extensions)
01092             {
01093                 extension_range_start = (uint32_t)-1;
01094             }
01095         }
01096
01097         if (tag >= extension_range_start)
01098         {
01099             size_t pos = stream->bytes_left;
01100
01101             if (!decode_extension(stream, tag, wire_type, extensions))
01102                 return false;
01103
01104             if (pos != stream->bytes_left)
01105             {
01106                 /* The field was handled */
01107                 continue;
01108             }
01109         }
01110
01111         /* No match found, skip data */
01112         if (!pb_skip_field(stream, wire_type))
01113             return false;
01114         continue;
01115     }
01116
01117     /* If a repeated fixed count field was found, get size from
01118      * 'fixed_count_field' as there is no counter contained in the struct.
01119      */
01120     if (PB_HTYPE(iter.type) == PB_HTYPE_REPEATED && iter.pSize == &iter.array_size)
01121     {
01122         if (fixed_count_field != iter.index) {
01123             /* If the new fixed count field does not match the previous one,
01124              * check that the previous one is NULL or that it finished
01125              * receiving all the expected data.
01126              */
01127             if (fixed_count_field != PB_SIZE_MAX &&
01128                 fixed_count_size != fixed_count_total_size)
01129             {
01130                 PB_RETURN_ERROR(stream, "wrong size for fixed count field");
01131             }
01132
01133             fixed_count_field = iter.index;
01134             fixed_count_size = 0;
01135             fixed_count_total_size = iter.array_size;
01136         }
01137
01138         iter.pSize = &fixed_count_size;
01139     }
01140
01141     if (PB_HTYPE(iter.type) == PB_HTYPE_REQUIRED

```

```

01142         && iter.required_field_index < PB_MAX_REQUIRED_FIELDS)
01143     {
01144         uint32_t tmp = ((uint32_t)1 << (iter.required_field_index & 31));
01145         fields_seen.bitfield[iter.required_field_index >> 5] |= tmp;
01146     }
01147
01148     if (!decode_field(stream, wire_type, &iter))
01149         return false;
01150 }
01151
01152 if (!eof)
01153 {
01154     /* pb_decode_tag() returned error before end of stream */
01155     return false;
01156 }
01157
01158 /* Check that all elements of the last decoded fixed count field were present. */
01159 if (fixed_count_field != PB_SIZE_MAX &&
01160     fixed_count_size != fixed_count_total_size)
01161 {
01162     PB_RETURN_ERROR(stream, "wrong size for fixed count field");
01163 }
01164
01165 /* Check that all required fields were present. */
01166 {
01167     pb_size_t req_field_count = iter.descriptor->required_field_count;
01168
01169     if (req_field_count > 0)
01170     {
01171         pb_size_t i;
01172
01173         if (req_field_count > PB_MAX_REQUIRED_FIELDS)
01174             req_field_count = PB_MAX_REQUIRED_FIELDS;
01175
01176         /* Check the whole words */
01177         for (i = 0; i < (req_field_count >> 5); i++)
01178         {
01179             if (fields_seen.bitfield[i] != allbits)
01180                 PB_RETURN_ERROR(stream, "missing required field");
01181         }
01182
01183         /* Check the remaining bits (if any) */
01184         if ((req_field_count & 31) != 0)
01185         {
01186             if (fields_seen.bitfield[req_field_count >> 5] !=
01187                 (allbits >> (uint_least8_t)(32 - (req_field_count & 31))))
01188             {
01189                 PB_RETURN_ERROR(stream, "missing required field");
01190             }
01191         }
01192     }
01193 }
01194
01195 return true;
01196 }
01197
01198 bool checkreturn pb_decode_ex(pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct, unsigned int flags)
01199 {
01200     bool status;
01201
01202     if ((flags & PB_DECODE_DELIMITED) == 0)
01203     {
01204         status = pb_decode_inner(stream, fields, dest_struct, flags);
01205     }
01206     else
01207     {
01208         pb_istream_t substream;
01209         if (!pb_make_string_substream(stream, &substream))
01210             return false;
01211
01212         status = pb_decode_inner(&substream, fields, dest_struct, flags);
01213
01214         if (!pb_close_string_substream(stream, &substream))
01215             status = false;
01216     }
01217
01218 #ifdef PB_ENABLE_MALLOC
01219     if (!status)
01220         pb_release(fields, dest_struct);
01221 #endif
01222     return status;
01223 }
01224
01225 bool checkreturn pb_decode(pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct)
01226 {
01227     return pb_decode_ex(stream, fields, dest_struct, 0);
01228 }

```



```

01229 }
01230
01231 #ifdef PB_ENABLE_MALLOC
01232 /* Given an oneof field, if there has already been a field inside this oneof,
01233  * release it before overwriting with a different one. */
01234 static bool pb_release_union_field(pb_istream_t *stream, pb_field_iter_t *field)
01235 {
01236     pb_field_iter_t old_field = *field;
01237     pb_size_t old_tag = *(pb_size_t*)field->pSize; /* Previous which_value */
01238     pb_size_t new_tag = field->tag; /* New which_value */
01239
01240     if (old_tag == 0)
01241         return true; /* Ok, no old data in union */
01242
01243     if (old_tag == new_tag)
01244         return true; /* Ok, old data is of same type => merge */
01245
01246     /* Release old data. The find can fail if the message struct contains
01247     * invalid data. */
01248     if (!pb_field_iter_find(&old_field, old_tag))
01249         PB_RETURN_ERROR(stream, "invalid union tag");
01250
01251     pb_release_single_field(&old_field);
01252
01253     if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
01254     {
01255         /* Initialize the pointer to NULL to make sure it is valid
01256         * even in case of error return. */
01257         *(void**)field->pField = NULL;
01258         field->pData = NULL;
01259     }
01260
01261     return true;
01262 }
01263
01264 static void pb_release_single_field(pb_field_iter_t *field)
01265 {
01266     pb_type_t type;
01267     type = field->type;
01268
01269     if (PB_HTYPE(type) == PB_HTYPE_ONEOF)
01270     {
01271         if (*(pb_size_t*)field->pSize != field->tag)
01272             return; /* This is not the current field in the union */
01273     }
01274
01275     /* Release anything contained inside an extension or submsg.
01276     * This has to be done even if the submsg itself is statically
01277     * allocated. */
01278     if (PB_LTYPE(type) == PB_LTYPE_EXTENSION)
01279     {
01280         /* Release fields from all extensions in the linked list */
01281         pb_extension_t *ext = *(pb_extension_t**)field->pData;
01282         while (ext != NULL)
01283         {
01284             pb_field_iter_t ext_iter;
01285             if (pb_field_iter_begin_extension(&ext_iter, ext))
01286             {
01287                 pb_release_single_field(&ext_iter);
01288             }
01289             ext = ext->next;
01290         }
01291     }
01292     else if (PB_LTYPE_IS_SUBMSG(type) && PB_ATYPE(type) != PB_ATYPE_CALLBACK)
01293     {
01294         /* Release fields in submessage or submsg array */
01295         pb_size_t count = 1;
01296
01297         if (PB_ATYPE(type) == PB_ATYPE_POINTER)
01298         {
01299             field->pData = *(void**)field->pField;
01300         }
01301         else
01302         {
01303             field->pData = field->pField;
01304         }
01305
01306         if (PB_HTYPE(type) == PB_HTYPE_REPEATED)
01307         {
01308             count = *(pb_size_t*)field->pSize;
01309
01310             if (PB_ATYPE(type) == PB_ATYPE_STATIC && count > field->array_size)
01311             {
01312                 /* Protect against corrupted __count fields */
01313                 count = field->array_size;
01314             }
01315         }
01316     }

```

```

01316
01317     if (field->pData)
01318     {
01319         for (; count > 0; count--)
01320         {
01321             pb_release(field->submsg_desc, field->pData);
01322             field->pData = (char*)field->pData + field->data_size;
01323         }
01324     }
01325 }
01326
01327 if (PB_ATYPE(type) == PB_ATYPE_POINTER)
01328 {
01329     if (PB_HTYPE(type) == PB_HTYPE_REPEATED &&
01330         (PB_LTYPE(type) == PB_LTYPE_STRING ||
01331          PB_LTYPE(type) == PB_LTYPE_BYTES))
01332     {
01333         /* Release entries in repeated string or bytes array */
01334         void **pItem = *(void**)field->pField;
01335         pb_size_t count = *(pb_size_t*)field->pSize;
01336         for (; count > 0; count--)
01337         {
01338             pb_free(*pItem);
01339             *pItem++ = NULL;
01340         }
01341     }
01342
01343     if (PB_HTYPE(type) == PB_HTYPE_REPEATED)
01344     {
01345         /* We are going to release the array, so set the size to 0 */
01346         *(pb_size_t*)field->pSize = 0;
01347     }
01348
01349     /* Release main pointer */
01350     pb_free(*(void**)field->pField);
01351     *(void**)field->pField = NULL;
01352 }
01353 }
01354
01355 void pb_release(const pb_msgdesc_t *fields, void *dest_struct)
01356 {
01357     pb_field_iter_t iter;
01358
01359     if (!dest_struct)
01360         return; /* Ignore NULL pointers, similar to free() */
01361
01362     if (!pb_field_iter_begin(&iter, fields, dest_struct))
01363         return; /* Empty message type */
01364
01365     do
01366     {
01367         pb_release_single_field(&iter);
01368     } while (pb_field_iter_next(&iter));
01369 }
01370 #else
01371 void pb_release(const pb_msgdesc_t *fields, void *dest_struct)
01372 {
01373     /* Nothing to release without PB_ENABLE_MALLOC. */
01374     PB_UNUSED(fields);
01375     PB_UNUSED(dest_struct);
01376 }
01377 #endif
01378
01379 /* Field decoders */
01380
01381 bool pb_decode_bool(pb_istream_t *stream, bool *dest)
01382 {
01383     uint32_t value;
01384     if (!pb_decode_varint32(stream, &value))
01385         return false;
01386
01387     *(bool*)dest = (value != 0);
01388     return true;
01389 }
01390
01391 bool pb_decode_svarint(pb_istream_t *stream, pb_int64_t *dest)
01392 {
01393     pb_uint64_t value;
01394     if (!pb_decode_varint(stream, &value))
01395         return false;
01396
01397     if (value & 1)
01398         *dest = (pb_int64_t)(~(value » 1));
01399     else
01400         *dest = (pb_int64_t)(value » 1);
01401
01402     return true;

```

```

01403 }
01404
01405 bool pb_decode_fixed32(pb_istream_t *stream, void *dest)
01406 {
01407     union {
01408         uint32_t fixed32;
01409         pb_byte_t bytes[4];
01410     } u;
01411
01412     if (!pb_read(stream, u.bytes, 4))
01413         return false;
01414
01415     #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
01416         /* fast path - if we know that we're on little endian, assign directly */
01417         *(uint32_t*)dest = u.fixed32;
01418     #else
01419         *(uint32_t*)dest = ((uint32_t)u.bytes[0] << 0) |
01420             ((uint32_t)u.bytes[1] << 8) |
01421             ((uint32_t)u.bytes[2] << 16) |
01422             ((uint32_t)u.bytes[3] << 24);
01423     #endif
01424     return true;
01425 }
01426
01427 #ifndef PB_WITHOUT_64BIT
01428 bool pb_decode_fixed64(pb_istream_t *stream, void *dest)
01429 {
01430     union {
01431         uint64_t fixed64;
01432         pb_byte_t bytes[8];
01433     } u;
01434
01435     if (!pb_read(stream, u.bytes, 8))
01436         return false;
01437
01438     #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
01439         /* fast path - if we know that we're on little endian, assign directly */
01440         *(uint64_t*)dest = u.fixed64;
01441     #else
01442         *(uint64_t*)dest = ((uint64_t)u.bytes[0] << 0) |
01443             ((uint64_t)u.bytes[1] << 8) |
01444             ((uint64_t)u.bytes[2] << 16) |
01445             ((uint64_t)u.bytes[3] << 24) |
01446             ((uint64_t)u.bytes[4] << 32) |
01447             ((uint64_t)u.bytes[5] << 40) |
01448             ((uint64_t)u.bytes[6] << 48) |
01449             ((uint64_t)u.bytes[7] << 56);
01450     #endif
01451     return true;
01452 }
01453 #endif
01454
01455 static bool checkreturn pb_dec_bool(pb_istream_t *stream, const pb_field_iter_t *field)
01456 {
01457     return pb_decode_bool(stream, (bool*)field->pData);
01458 }
01459
01460 static bool checkreturn pb_dec_varint(pb_istream_t *stream, const pb_field_iter_t *field)
01461 {
01462     if (PB_LTYPE(field->type) == PB_LTYPE_UVARINT)
01463     {
01464         pb_uint64_t value, clamped;
01465         if (!pb_decode_varint(stream, &value))
01466             return false;
01467
01468         /* Cast to the proper field size, while checking for overflows */
01469         if (field->data_size == sizeof(pb_uint64_t))
01470             clamped = *(pb_uint64_t*)field->pData = value;
01471         else if (field->data_size == sizeof(uint32_t))
01472             clamped = *(uint32_t*)field->pData = (uint32_t)value;
01473         else if (field->data_size == sizeof(uint_least16_t))
01474             clamped = *(uint_least16_t*)field->pData = (uint_least16_t)value;
01475         else if (field->data_size == sizeof(uint_least8_t))
01476             clamped = *(uint_least8_t*)field->pData = (uint_least8_t)value;
01477         else
01478             PB_RETURN_ERROR(stream, "invalid data_size");
01479
01480         if (clamped != value)
01481             PB_RETURN_ERROR(stream, "integer too large");
01482
01483         return true;
01484     }
01485     else
01486     {
01487         pb_uint64_t value;
01488         pb_int64_t svalue;
01489         pb_int64_t clamped;

```

```

01490
01491     if (PB_LTYPE(field->type) == PB_LTYPE_SVARINT)
01492     {
01493         if (!pb_decode_svarint(stream, &svalue))
01494             return false;
01495     }
01496     else
01497     {
01498         if (!pb_decode_varint(stream, &svalue))
01499             return false;
01500
01501         /* See issue 97: Google's C++ protobuf allows negative varint values to
01502          * be cast as int32_t, instead of the int64_t that should be used when
01503          * encoding. Nanopb versions before 0.2.5 had a bug in encoding. In order to
01504          * not break decoding of such messages, we cast <=32 bit fields to
01505          * int32_t first to get the sign correct.
01506          */
01507         if (field->data_size == sizeof(pb_int64_t))
01508             svalue = (pb_int64_t)svalue;
01509         else
01510             svalue = (int32_t)svalue;
01511     }
01512
01513     /* Cast to the proper field size, while checking for overflows */
01514     if (field->data_size == sizeof(pb_int64_t))
01515         clamped = *(pb_int64_t*)field->pData = svalue;
01516     else if (field->data_size == sizeof(int32_t))
01517         clamped = *(int32_t*)field->pData = (int32_t)svalue;
01518     else if (field->data_size == sizeof(int_least16_t))
01519         clamped = *(int_least16_t*)field->pData = (int_least16_t)svalue;
01520     else if (field->data_size == sizeof(int_least8_t))
01521         clamped = *(int_least8_t*)field->pData = (int_least8_t)svalue;
01522     else
01523         PB_RETURN_ERROR(stream, "invalid data_size");
01524
01525     if (clamped != svalue)
01526         PB_RETURN_ERROR(stream, "integer too large");
01527
01528     return true;
01529 }
01530 }
01531
01532 static bool checkreturn pb_dec_bytes(pb_istream_t *stream, const pb_field_iter_t *field)
01533 {
01534     uint32_t size;
01535     size_t alloc_size;
01536     pb_bytes_array_t *dest;
01537
01538     if (!pb_decode_varint32(stream, &size))
01539         return false;
01540
01541     if (size > PB_SIZE_MAX)
01542         PB_RETURN_ERROR(stream, "bytes overflow");
01543
01544     alloc_size = PB_BYTES_ARRAY_T_ALLOCSIZE(size);
01545     if (size > alloc_size)
01546         PB_RETURN_ERROR(stream, "size too large");
01547
01548     if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
01549     {
01550 #ifndef PB_ENABLE_MALLOC
01551         PB_RETURN_ERROR(stream, "no malloc support");
01552 #else
01553         if (stream->bytes_left < size)
01554             PB_RETURN_ERROR(stream, "end-of-stream");
01555
01556         if (!allocate_field(stream, field->pData, alloc_size, 1))
01557             return false;
01558         dest = *(pb_bytes_array_t**)field->pData;
01559 #endif
01560     }
01561     else
01562     {
01563         if (alloc_size > field->data_size)
01564             PB_RETURN_ERROR(stream, "bytes overflow");
01565         dest = (pb_bytes_array_t*)field->pData;
01566     }
01567
01568     dest->size = (pb_size_t)size;
01569     return pb_read(stream, dest->bytes, (size_t)size);
01570 }
01571
01572 static bool checkreturn pb_dec_string(pb_istream_t *stream, const pb_field_iter_t *field)
01573 {
01574     uint32_t size;
01575     size_t alloc_size;
01576     pb_byte_t *dest = (pb_byte_t*)field->pData;

```

```

01577
01578     if (!pb_decode_varint32(stream, &size))
01579         return false;
01580
01581     if (size == (uint32_t)-1)
01582         PB_RETURN_ERROR(stream, "size too large");
01583
01584     /* Space for null terminator */
01585     alloc_size = (size_t)(size + 1);
01586
01587     if (alloc_size < size)
01588         PB_RETURN_ERROR(stream, "size too large");
01589
01590     if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
01591     {
01592 #ifndef PB_ENABLE_MALLOC
01593         PB_RETURN_ERROR(stream, "no malloc support");
01594 #else
01595         if (stream->bytes_left < size)
01596             PB_RETURN_ERROR(stream, "end-of-stream");
01597
01598         if (!allocate_field(stream, field->pData, alloc_size, 1))
01599             return false;
01600         dest = *(pb_byte_t**)field->pData;
01601 #endif
01602     }
01603     else
01604     {
01605         if (alloc_size > field->data_size)
01606             PB_RETURN_ERROR(stream, "string overflow");
01607     }
01608
01609     dest[size] = 0;
01610
01611     if (!pb_read(stream, dest, (size_t)size))
01612         return false;
01613
01614 #ifdef PB_VALIDATE_UTF8
01615     if (!pb_validate_utf8((const char*)dest))
01616         PB_RETURN_ERROR(stream, "invalid utf8");
01617 #endif
01618
01619     return true;
01620 }
01621
01622 static bool checkreturn pb_dec_submessage(pb_istream_t *stream, const pb_field_iter_t *field)
01623 {
01624     bool status = true;
01625     bool submsg_consumed = false;
01626     pb_istream_t substream;
01627
01628     if (!pb_make_string_substream(stream, &substream))
01629         return false;
01630
01631     if (field->submsg_desc == NULL)
01632         PB_RETURN_ERROR(stream, "invalid field descriptor");
01633
01634     /* Submessages can have a separate message-level callback that is called
01635     * before decoding the message. Typically it is used to set callback fields
01636     * inside oneofs. */
01637     if (PB_LTYPE(field->type) == PB_LTYPE_SUBMSG_W_CB && field->pSize != NULL)
01638     {
01639         /* Message callback is stored right before pSize. */
01640         pb_callback_t *callback = (pb_callback_t*)field->pSize - 1;
01641         if (callback->funcs.decode)
01642         {
01643             status = callback->funcs.decode(&substream, field, &callback->arg);
01644
01645             if (substream.bytes_left == 0)
01646             {
01647                 submsg_consumed = true;
01648             }
01649         }
01650     }
01651
01652     /* Now decode the submessage contents */
01653     if (status && !submsg_consumed)
01654     {
01655         unsigned int flags = 0;
01656
01657         /* Static required/optional fields are already initialized by top-level
01658         * pb_decode(), no need to initialize them again. */
01659         if (PB_ATYPE(field->type) == PB_ATYPE_STATIC &&
01660             PB_HTYPE(field->type) != PB_HTYPE_REPEATED)
01661         {
01662             flags = PB_DECODE_NOINIT;
01663         }
01664     }

```

```

01664
01665     status = pb_decode_inner(&substream, field->submsg_desc, field->pData, flags);
01666 }
01667
01668 if (!pb_close_string_substream(stream, &substream))
01669     return false;
01670
01671 return status;
01672 }
01673
01674 static bool checkreturn pb_dec_fixed_length_bytes(pb_istream_t *stream, const pb_field_iter_t *field)
01675 {
01676     uint32_t size;
01677
01678     if (!pb_decode_varint32(stream, &size))
01679         return false;
01680
01681     if (size > PB_SIZE_MAX)
01682         PB_RETURN_ERROR(stream, "bytes overflow");
01683
01684     if (size == 0)
01685     {
01686         /* As a special case, treat empty bytes string as all zeros for fixed_length_bytes. */
01687         memset(field->pData, 0, (size_t)field->data_size);
01688         return true;
01689     }
01690
01691     if (size != field->data_size)
01692         PB_RETURN_ERROR(stream, "incorrect fixed length bytes size");
01693
01694     return pb_read(stream, (pb_byte_t*)field->pData, (size_t)field->data_size);
01695 }
01696
01697 #ifdef PB_CONVERT_DOUBLE_FLOAT
01698 bool pb_decode_double_as_float(pb_istream_t *stream, float *dest)
01699 {
01700     uint_least8_t sign;
01701     int exponent;
01702     uint32_t mantissa;
01703     uint64_t value;
01704     union { float f; uint32_t i; } out;
01705
01706     if (!pb_decode_fixed64(stream, &value))
01707         return false;
01708
01709     /* Decompose input value */
01710     sign = (uint_least8_t)((value >> 63) & 1);
01711     exponent = (int)((value >> 52) & 0x7FFF) - 1023;
01712     mantissa = (value >> 28) & 0xFFFFF; /* Highest 24 bits */
01713
01714     /* Figure if value is in range representable by floats. */
01715     if (exponent == 1024)
01716     {
01717         /* Special value */
01718         exponent = 128;
01719         mantissa >>= 1;
01720     }
01721     else
01722     {
01723         if (exponent > 127)
01724         {
01725             /* Too large, convert to infinity */
01726             exponent = 128;
01727             mantissa = 0;
01728         }
01729         else if (exponent < -150)
01730         {
01731             /* Too small, convert to zero */
01732             exponent = -127;
01733             mantissa = 0;
01734         }
01735         else if (exponent < -126)
01736         {
01737             /* Denormalized */
01738             mantissa |= 0x1000000;
01739             mantissa >>= (-126 - exponent);
01740             exponent = -127;
01741         }
01742
01743         /* Round off mantissa */
01744         mantissa = (mantissa + 1) >> 1;
01745
01746         /* Check if mantissa went over 2.0 */
01747         if (mantissa & 0x800000)
01748         {
01749             exponent += 1;
01750             mantissa &= 0x7FFFFF;
01751         }
01752     }
01753
01754     out.i = (sign < 0 ? 0xFFFFFFFF : 0) | (exponent < 0 ? 0 : (exponent + 127) << 23) | mantissa << 9;
01755     *dest = out.f;
01756 }

```

```

01751     mantissa »= 1;
01752   }
01753 }
01754
01755 /* Combine fields */
01756 out.i = mantissa;
01757 out.i |= (uint32_t)(exponent + 127) « 23;
01758 out.i |= (uint32_t)sign « 31;
01759
01760 *dest = out.f;
01761 return true;
01762 }
01763 #endif

```

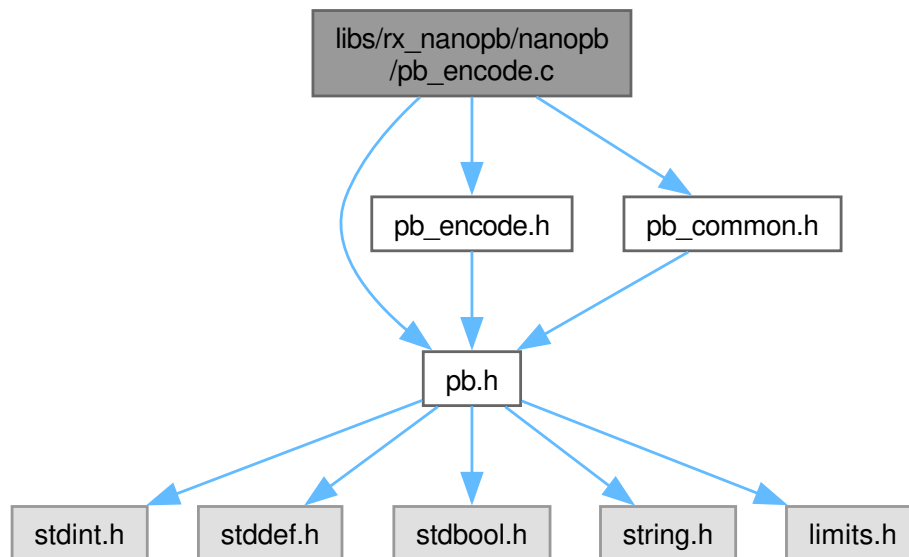
6.63 libs/rx_nanopb/nanopb/pb_encode.c File Reference

```

#include "pb.h"
#include "pb_encode.h"
#include "pb_common.h"

```

Include dependency graph for pb_encode.c:



Macros

- #define [checkreturn](#)
- #define [pb_int64_t](#) int64_t
- #define [pb_uint64_t](#) uint64_t

Functions

- static [bool](#) [buf_write](#) (pb_ostream_t *stream, const [pb_byte_t](#) *buf, [size_t](#) count)
- static [bool](#) [encode_array](#) (pb_ostream_t *stream, pb_field_iter_t *field)
- static [bool](#) [pb_check_proto3_default_value](#) (const pb_field_iter_t *field)
- static [bool](#) [encode_basic_field](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- static [bool](#) [encode_callback_field](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- static [bool](#) [encode_field](#) (pb_ostream_t *stream, pb_field_iter_t *field)
- static [bool](#) [encode_extension_field](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- static [bool](#) [default_extension_encoder](#) (pb_ostream_t *stream, const pb_extension_t *extension)

- static [bool pb_encode_varint_32](#) (pb_ostream_t *stream, [uint32_t](#) low, [uint32_t](#) high)
- static [bool pb_enc_bool](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- static [bool pb_enc_varint](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- static [bool pb_enc_fixed](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- static [bool pb_enc_bytes](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- static [bool pb_enc_string](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- static [bool pb_enc_submessage](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- static [bool pb_enc_fixed_length_bytes](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- pb_ostream_t [pb_ostream_from_buffer](#) ([pb_byte_t](#) *buf, [size_t](#) bufsize)
- [bool pb_write](#) (pb_ostream_t *stream, const [pb_byte_t](#) *buf, [size_t](#) count)
- static [bool safe_read_bool](#) (const void *pSize)
- [bool pb_encode](#) (pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct)
- [bool pb_encode_ex](#) (pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct, unsigned int flags)
- [bool pb_get_encoded_size](#) ([size_t](#) *size, const pb_msgdesc_t *fields, const void *src_struct)
- [bool pb_encode_varint](#) (pb_ostream_t *stream, [uint64_t](#) value)
- [bool pb_encode_svarint](#) (pb_ostream_t *stream, [int64_t](#) value)
- [bool pb_encode_fixed32](#) (pb_ostream_t *stream, const void *value)
- [bool pb_encode_fixed64](#) (pb_ostream_t *stream, const void *value)
- [bool pb_encode_tag](#) (pb_ostream_t *stream, [pb_wire_type_t](#) wiretype, [uint32_t](#) field_number)
- [bool pb_encode_tag_for_field](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- [bool pb_encode_string](#) (pb_ostream_t *stream, const [pb_byte_t](#) *buffer, [size_t](#) size)
- [bool pb_encode_submessage](#) (pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct)

6.63.1 Macro Definition Documentation

6.63.1.1 checkreturn

```
#define checkreturn
```

Definition at line 18 of file [pb_encode.c](#).

6.63.1.2 pb_int64_t

```
#define pb_int64_t int64_t
```

Definition at line 45 of file [pb_encode.c](#).

6.63.1.3 pb_uint64_t

```
#define pb_uint64_t uint64_t
```

Definition at line 46 of file [pb_encode.c](#).

6.63.2 Function Documentation

6.63.2.1 buf_write()

```
bool buf_write (
    pb_ostream_t * stream,
    const pb_byte_t * buf,
    size_t count) [static]
```

Definition at line 53 of file pb_encode.c.

```
00054 {
00055     pb_byte_t *dest = (pb_byte_t*)stream->state;
00056     stream->state = dest + count;
00057
00058     memcpy(dest, buf, count * sizeof(pb_byte_t));
00059
00060     return true;
00061 }
```

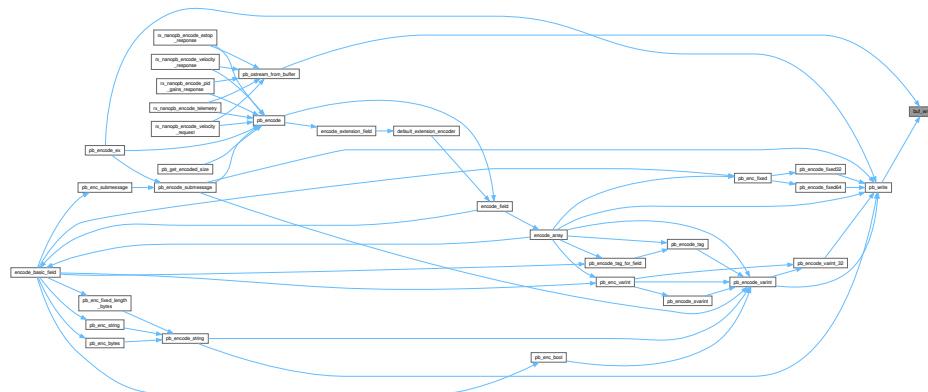
References [checkreturn](#), and [memcpy\(\)](#).

Referenced by `pb_ostream_from_buffer()`, and `pb_write()`.

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.2 default_extension_encoder()

```
blue default_extension_encoder (
    pb_ostream_t * stream,
    const pb_extension_t * extension) [static]
```

Definition at line 474 of file pb_encode.c.

```

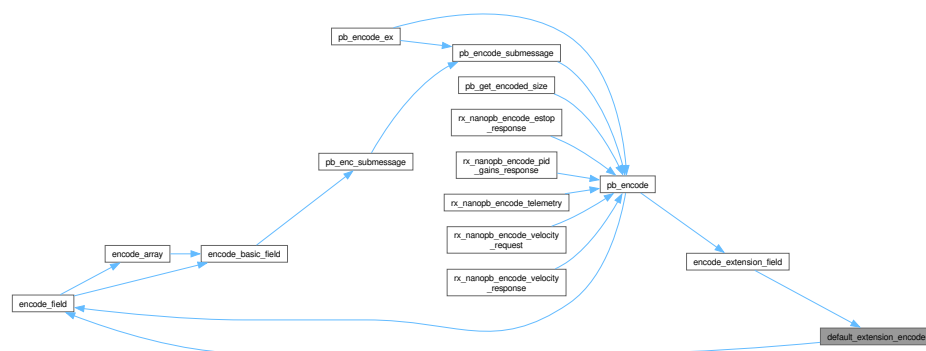
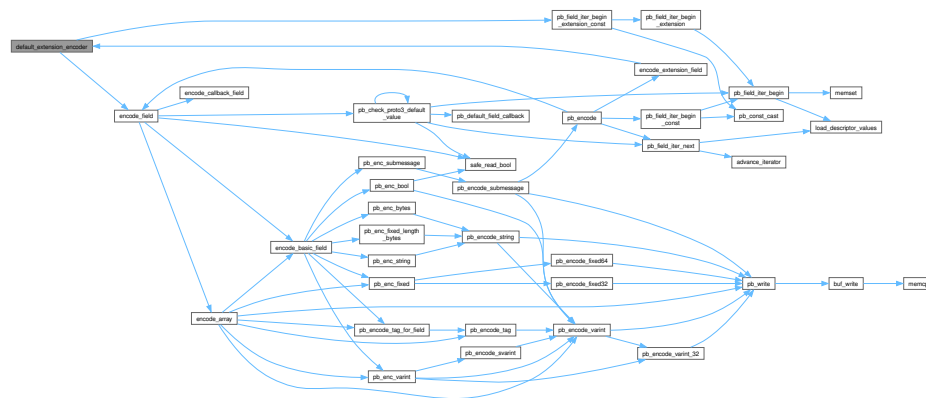
00475 {
00476     pb_field_iter_t iter;
00477
00478     if (!pb_field_iter_begin_extension_const(&iter, extension))
00479         PB_RETURN_ERROR(stream, "invalid extension");
00480
00481     return encode_field(stream, &iter);
00482 }

```

References [checkreturn](#), [encode_field\(\)](#), [pb_field_iter_begin_extension_const\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [encode_extension_field\(\)](#).

Here is the call graph for this function:



```
bool encode_array (
    pb_ostream_t * stream,
    pb_field_iter_t * field) [static]
```

Definition at line 128 of file pb_encode.c.

```

00129 {
00130     pb_size_t i;
00131     pb_size_t count;
00132     #ifndef PB_ENCODE_ARRAYS_UNPACKED
00133         size_t size;
00134     #endif
00135
00136     count = *(pb_size_t*)field->pSize;
00137
00138     if (count == 0)
00139         return true;
00140
00141     if (PB_ATYPE(field->type) != PB_ATYPE_POINTER && count > field->array_size)
00142         PB_RETURN_ERROR(stream, "array max size exceeded");
00143
00144     #ifndef PB_ENCODE_ARRAYS_UNPACKED
00145         /* We always pack arrays if the datatype allows it. */
00146         if (PB_LTYPE(field->type) <= PB_LTYPE_LAST_PACKABLE)
00147         {
00148             if (!pb_encode_tag(stream, PB_WT_STRING, field->tag))
00149                 return false;
00150
00151             /* Determine the total size of packed array. */
00152             if (PB_LTYPE(field->type) == PB_LTYPE_FIXED32)
00153             {
00154                 size = 4 * (size_t)count;
00155             }
00156             else if (PB_LTYPE(field->type) == PB_LTYPE_FIXED64)
00157             {
00158                 size = 8 * (size_t)count;
00159             }
00160         }
00161     #endif
00162
00163     if (count > 0)
00164     {
00165         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00166             field->array = (void**)calloc(count, sizeof(void*));
00167         if (!field->array)
00168             PB_RETURN_ERROR(stream, "calloc failed");
00169         for (i = 0; i < count; i++)
00170             field->array[i] = *(void**)field->data;
00171     }
00172
00173     if (count > 0)
00174     {
00175         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00176             field->array = (void**)calloc(count, sizeof(void*));
00177         if (!field->array)
00178             PB_RETURN_ERROR(stream, "calloc failed");
00179         for (i = 0; i < count; i++)
00180             field->array[i] = *(void**)field->data;
00181     }
00182
00183     if (count > 0)
00184     {
00185         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00186             field->array = (void**)calloc(count, sizeof(void*));
00187         if (!field->array)
00188             PB_RETURN_ERROR(stream, "calloc failed");
00189         for (i = 0; i < count; i++)
00190             field->array[i] = *(void**)field->data;
00191     }
00192
00193     if (count > 0)
00194     {
00195         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00196             field->array = (void**)calloc(count, sizeof(void*));
00197         if (!field->array)
00198             PB_RETURN_ERROR(stream, "calloc failed");
00199         for (i = 0; i < count; i++)
00200             field->array[i] = *(void**)field->data;
00201     }
00202
00203     if (count > 0)
00204     {
00205         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00206             field->array = (void**)calloc(count, sizeof(void*));
00207         if (!field->array)
00208             PB_RETURN_ERROR(stream, "calloc failed");
00209         for (i = 0; i < count; i++)
00210             field->array[i] = *(void**)field->data;
00211     }
00212
00213     if (count > 0)
00214     {
00215         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00216             field->array = (void**)calloc(count, sizeof(void*));
00217         if (!field->array)
00218             PB_RETURN_ERROR(stream, "calloc failed");
00219         for (i = 0; i < count; i++)
00220             field->array[i] = *(void**)field->data;
00221     }
00222
00223     if (count > 0)
00224     {
00225         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00226             field->array = (void**)calloc(count, sizeof(void*));
00227         if (!field->array)
00228             PB_RETURN_ERROR(stream, "calloc failed");
00229         for (i = 0; i < count; i++)
00230             field->array[i] = *(void**)field->data;
00231     }
00232
00233     if (count > 0)
00234     {
00235         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00236             field->array = (void**)calloc(count, sizeof(void*));
00237         if (!field->array)
00238             PB_RETURN_ERROR(stream, "calloc failed");
00239         for (i = 0; i < count; i++)
00240             field->array[i] = *(void**)field->data;
00241     }
00242
00243     if (count > 0)
00244     {
00245         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00246             field->array = (void**)calloc(count, sizeof(void*));
00247         if (!field->array)
00248             PB_RETURN_ERROR(stream, "calloc failed");
00249         for (i = 0; i < count; i++)
00250             field->array[i] = *(void**)field->data;
00251     }
00252
00253     if (count > 0)
00254     {
00255         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00256             field->array = (void**)calloc(count, sizeof(void*));
00257         if (!field->array)
00258             PB_RETURN_ERROR(stream, "calloc failed");
00259         for (i = 0; i < count; i++)
00260             field->array[i] = *(void**)field->data;
00261     }
00262
00263     if (count > 0)
00264     {
00265         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00266             field->array = (void**)calloc(count, sizeof(void*));
00267         if (!field->array)
00268             PB_RETURN_ERROR(stream, "calloc failed");
00269         for (i = 0; i < count; i++)
00270             field->array[i] = *(void**)field->data;
00271     }
00272
00273     if (count > 0)
00274     {
00275         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00276             field->array = (void**)calloc(count, sizeof(void*));
00277         if (!field->array)
00278             PB_RETURN_ERROR(stream, "calloc failed");
00279         for (i = 0; i < count; i++)
00280             field->array[i] = *(void**)field->data;
00281     }
00282
00283     if (count > 0)
00284     {
00285         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00286             field->array = (void**)calloc(count, sizeof(void*));
00287         if (!field->array)
00288             PB_RETURN_ERROR(stream, "calloc failed");
00289         for (i = 0; i < count; i++)
00290             field->array[i] = *(void**)field->data;
00291     }
00292
00293     if (count > 0)
00294     {
00295         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00296             field->array = (void**)calloc(count, sizeof(void*));
00297         if (!field->array)
00298             PB_RETURN_ERROR(stream, "calloc failed");
00299         for (i = 0; i < count; i++)
00300             field->array[i] = *(void**)field->data;
00301     }
00302
00303     if (count > 0)
00304     {
00305         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00306             field->array = (void**)calloc(count, sizeof(void*));
00307         if (!field->array)
00308             PB_RETURN_ERROR(stream, "calloc failed");
00309         for (i = 0; i < count; i++)
00310             field->array[i] = *(void**)field->data;
00311     }
00312
00313     if (count > 0)
00314     {
00315         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00316             field->array = (void**)calloc(count, sizeof(void*));
00317         if (!field->array)
00318             PB_RETURN_ERROR(stream, "calloc failed");
00319         for (i = 0; i < count; i++)
00320             field->array[i] = *(void**)field->data;
00321     }
00322
00323     if (count > 0)
00324     {
00325         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00326             field->array = (void**)calloc(count, sizeof(void*));
00327         if (!field->array)
00328             PB_RETURN_ERROR(stream, "calloc failed");
00329         for (i = 0; i < count; i++)
00330             field->array[i] = *(void**)field->data;
00331     }
00332
00333     if (count > 0)
00334     {
00335         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00336             field->array = (void**)calloc(count, sizeof(void*));
00337         if (!field->array)
00338             PB_RETURN_ERROR(stream, "calloc failed");
00339         for (i = 0; i < count; i++)
00340             field->array[i] = *(void**)field->data;
00341     }
00342
00343     if (count > 0)
00344     {
00345         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00346             field->array = (void**)calloc(count, sizeof(void*));
00347         if (!field->array)
00348             PB_RETURN_ERROR(stream, "calloc failed");
00349         for (i = 0; i < count; i++)
00350             field->array[i] = *(void**)field->data;
00351     }
00352
00353     if (count > 0)
00354     {
00355         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00356             field->array = (void**)calloc(count, sizeof(void*));
00357         if (!field->array)
00358             PB_RETURN_ERROR(stream, "calloc failed");
00359         for (i = 0; i < count; i++)
00360             field->array[i] = *(void**)field->data;
00361     }
00362
00363     if (count > 0)
00364     {
00365         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00366             field->array = (void**)calloc(count, sizeof(void*));
00367         if (!field->array)
00368             PB_RETURN_ERROR(stream, "calloc failed");
00369         for (i = 0; i < count; i++)
00370             field->array[i] = *(void**)field->data;
00371     }
00372
00373     if (count > 0)
00374     {
00375         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00376             field->array = (void**)calloc(count, sizeof(void*));
00377         if (!field->array)
00378             PB_RETURN_ERROR(stream, "calloc failed");
00379         for (i = 0; i < count; i++)
00380             field->array[i] = *(void**)field->data;
00381     }
00382
00383     if (count > 0)
00384     {
00385         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00386             field->array = (void**)
```

```

00159     }
00160     else
00161     {
00162         pb_ostream_t sizestream = PB_OSTREAM_SIZING;
00163         void *pData_orig = field->pData;
00164         for (i = 0; i < count; i++)
00165         {
00166             if (!pb_enc_varint(&sizestream, field))
00167                 PB_RETURN_ERROR(stream, PB_GET_ERROR(&sizestream));
00168             field->pData = (char*)field->pData + field->data_size;
00169         }
00170         field->pData = pData_orig;
00171         size = sizestream.bytes_written;
00172     }
00173
00174     if (!pb_encode_varint(stream, (pb_uint64_t)size))
00175         return false;
00176
00177     if (stream->callback == NULL)
00178         return pb_write(stream, NULL, size); /* Just sizing.. */
00179
00180     /* Write the data */
00181     for (i = 0; i < count; i++)
00182     {
00183         if (PB_LTYPE(field->type) == PB_LTYPE_FIXED32 || PB_LTYPE(field->type) == PB_LTYPE_FIXED64)
00184         {
00185             if (!pb_enc_fixed(stream, field))
00186                 return false;
00187         }
00188         else
00189         {
00190             if (!pb_enc_varint(stream, field))
00191                 return false;
00192         }
00193
00194         field->pData = (char*)field->pData + field->data_size;
00195     }
00196 }
00197 else /* Unpacked fields */
00198 #endif
00199 {
00200     for (i = 0; i < count; i++)
00201     {
00202         /* Normally the data is stored directly in the array entries, but
00203          * for pointer-type string and bytes fields, the array entries are
00204          * actually pointers themselves also. So we have to dereference once
00205          * more to get to the actual data. */
00206         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER &&
00207             (PB_LTYPE(field->type) == PB_LTYPE_STRING ||
00208              PB_LTYPE(field->type) == PB_LTYPE_BYTES))
00209         {
00210             bool status;
00211             void *pData_orig = field->pData;
00212             field->pData = *(void* const*)field->pData;
00213
00214             if (!field->pData)
00215             {
00216                 /* Null pointer in array is treated as empty string / bytes */
00217                 status = pb_encode_tag_for_field(stream, field) &&
00218                     pb_encode_varint(stream, 0);
00219             }
00220             else
00221             {
00222                 status = encode_basic_field(stream, field);
00223             }
00224
00225             field->pData = pData_orig;
00226
00227             if (!status)
00228                 return false;
00229         }
00230         else
00231         {
00232             if (!encode_basic_field(stream, field))
00233                 return false;
00234         }
00235         field->pData = (char*)field->pData + field->data_size;
00236     }
00237 }
00238
00239 return true;
00240 }

```

References [checkreturn](#), [encode_basic_field\(\)](#), [NULL](#), [PB_ATYPE](#), [PB_ATYPE_POINTER](#), [pb_enc_fixed\(\)](#), [pb_enc_varint\(\)](#), [pb_encode_tag\(\)](#), [pb_encode_tag_for_field\(\)](#), [pb_encode_varint\(\)](#), [PB_GET_ERROR](#), [PB_LTYPE](#), [PB_LTYPE_BYTES](#), [PB_LTYPE_FIXED32](#), [PB_LTYPE_FIXED64](#),

Referenced by [encode_field\(\)](#).

[illegible]

```

graph LR
    pb_encode_ex[pb_encode_ex] --> pb_encode_submessage[pb_encode_submessage]
    pb_encode_submessage --> pb_encode[pb_encode]
    pb_encode_submessage --> encode_basic_field[encode_basic_field]
    pb_get_encoded_size[pb_get_encoded_size] --> pb_encode
    rx_nanopb_encode_estop_response[rx_nanopb_encode_estop_response] --> pb_encode
    rx_nanopb_encode_pid_gains_response[rx_nanopb_encode_pid_gains_response] --> pb_encode
    rx_nanopb_encode_telemetry[rx_nanopb_encode_telemetry] --> pb_encode
    rx_nanopb_encode_velocity_request[rx_nanopb_encode_velocity_request] --> pb_encode
    rx_nanopb_encode_velocity_response[rx_nanopb_encode_velocity_response] --> pb_encode
    pb_encode --> encode_extension_field[encode_extension_field]
    encode_extension_field --> default_extension_encoder[default_extension_encoder]
    pb_encode --> encode_field[encode_field]
    encode_field --> encode_arm[encode_arm]
    pb_encode --> encode_basic_field
    encode_basic_field --> encode_arm

```

```
bool encode_basic_field (
    pb_ostream_t * stream,
    const pb_field_iter_t * field) [static]
```

```

00363 {
00364     if (!field->pData)
00365     {
00366         /* Missing pointer field */
00367         return true;
00368     }
00369
00370     if (!pb_encode_tag_for_field(stream, field))
00371         return false;
00372
00373     switch (PB_LTYPE(field->type))
00374     {
00375         case PB_LTYPE_BOOL:
00376             return pb_enc_bool(stream, field);
00377
00378         case PB_LTYPE_VARINT:
00379         case PB_LTYPE_UVARINT:
00380         case PB_LTYPE_SVARINT:
00381             return pb_enc_varint(stream, field);
00382
00383         case PB_LTYPE_FIXED32:
00384         case PB_LTYPE_FIXED64:

```

```

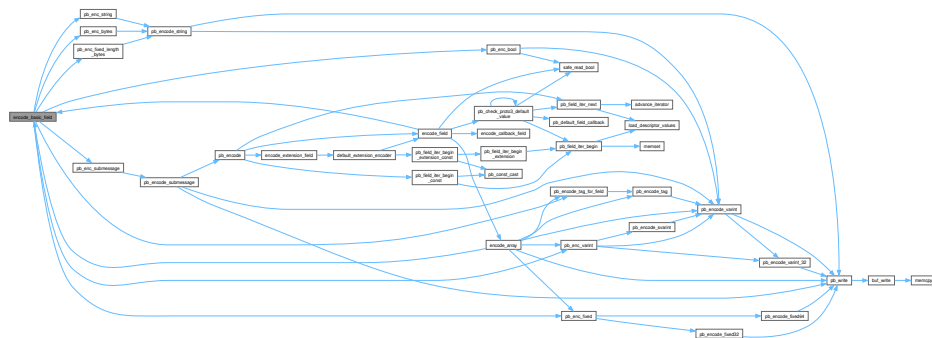
00385         return pb_enc_fixed(stream, field);
00386
00387     case PB_LTYPE_BYTES:
00388         return pb_enc_bytes(stream, field);
00389
00390     case PB_LTYPE_STRING:
00391         return pb_enc_string(stream, field);
00392
00393     case PB_LTYPE_SUBMESSAGE:
00394     case PB_LTYPE_SUBMSG_W_CB:
00395         return pb_enc_submessage(stream, field);
00396
00397     case PB_LTYPE_FIXED_LENGTH_BYTES:
00398         return pb_enc_fixed_length_bytes(stream, field);
00399
00400     default:
00401         PB_RETURN_ERROR(stream, "invalid field type");
00402 }
00403 }

```

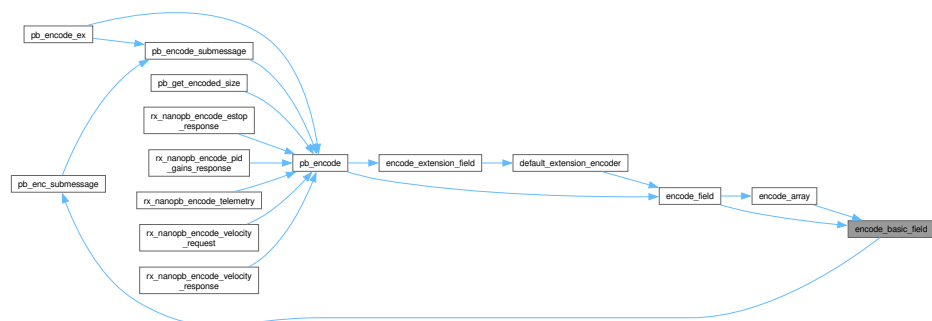
References [checkreturn](#), [pb_enc_bool\(\)](#), [pb_enc_bytes\(\)](#), [pb_enc_fixed\(\)](#), [pb_enc_fixed_length_bytes\(\)](#), [pb_enc_string\(\)](#), [pb_enc_submessage\(\)](#), [pb_enc_varint\(\)](#), [pb_encode_tag_for_field\(\)](#), [PB_LTYPE](#), [PB_LTYPE_BOOL](#), [PB_LTYPE_BYTES](#), [PB_LTYPE_FIXED32](#), [PB_LTYPE_FIXED64](#), [PB_LTYPE_FIXED_LENGTH_BYTES](#), [PB_LTYPE_STRING](#), [PB_LTYPE_SUBMESSAGE](#), [PB_LTYPE_SUBMSG_W_CB](#), [PB_LTYPE_SVARINT](#), [PB_LTYPE_UVARINT](#), [PB_LTYPE_VARINT](#), and [PB_RETURN_ERROR](#).

Referenced by [encode_array\(\)](#), and [encode_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.5 `encode_callback_field()`

```
bool encode_callback_field (
    pb_ostream_t * stream,
    const pb_field_iter_t * field) [static]
```

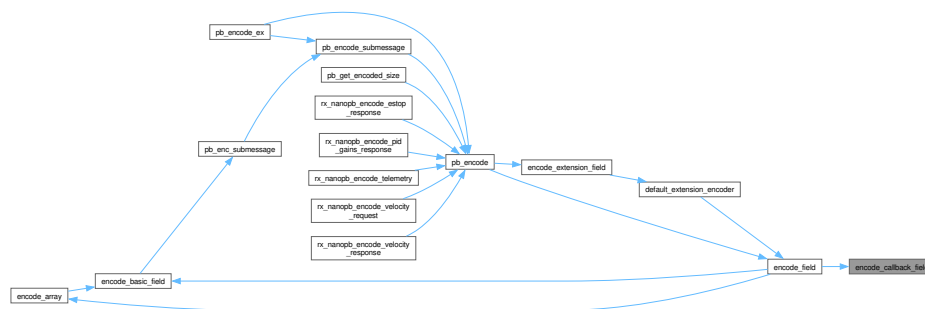
Definition at line 407 of file [pb_encode.c](#).

```
00408 {
00409     if (field->descriptor->field_callback != NULL)
00410     {
00411         if (!field->descriptor->field_callback(NULL, stream, field))
00412             PB_RETURN_ERROR(stream, "callback error");
00413     }
00414     return true;
00415 }
```

References [checkreturn](#), [NULL](#), and [PB_RETURN_ERROR](#).

Referenced by [encode_field\(\)](#).

Here is the caller graph for this function:

6.63.2.6 `encode_extension_field()`

```
bool encode_extension_field (
    pb_ostream_t * stream,
    const pb_field_iter_t * field) [static]
```

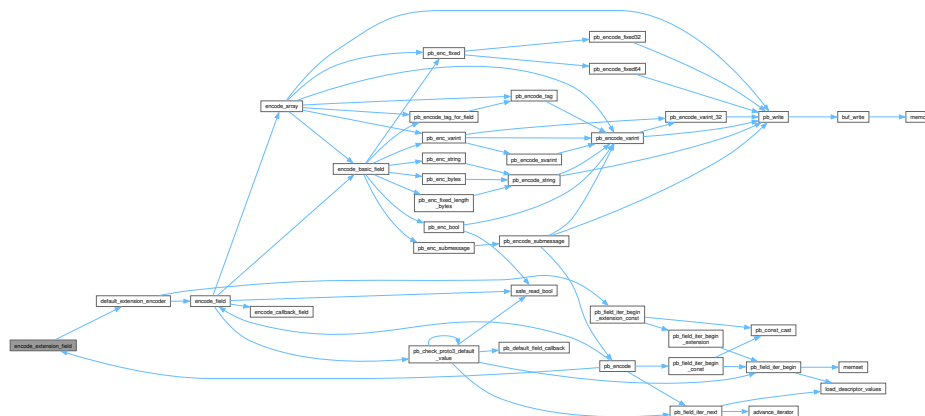
Definition at line 487 of file [pb_encode.c](#).

```
00488 {
00489     const pb_extension_t *extension = *(const pb_extension_t* const *)field->pData;
00490
00491     while (extension)
00492     {
00493         bool status;
00494         if (extension->type->encode)
00495             status = extension->type->encode(stream, extension);
00496         else
00497             status = default_extension_encoder(stream, extension);
00498
00499         if (!status)
00500             return false;
00501
00502         extension = extension->next;
00503     }
00504     return true;
00505 }
00506 }
```

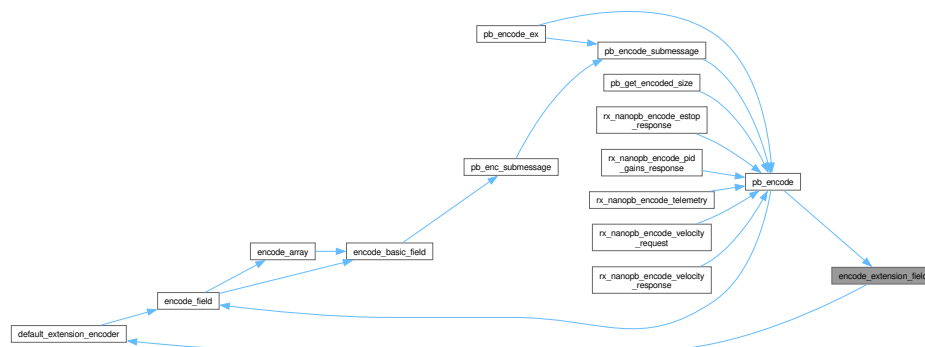
References [checkreturn](#), [default_extension_encoder\(\)](#), and [pb_noinline](#).

Referenced by [pb_encode\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.7 encode_field()

```
bool encode_field (
    pb_ostream_t * stream,
    pb_field_iter t * field) [static]
```

Definition at line 418 of file pb_encode.c.

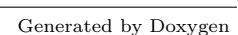
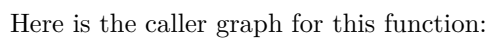
```

00419 {
00420     /* Check field presence */
00421     if (PB_HTYPE(field->type) == PB_HTYPE_ONEOF)
00422     {
00423         if (*(const pb_size_t*)field->pSize != field->tag)
00424         {
00425             /* Different type oneof field */
00426             return true;
00427         }
00428     }
00429     else if (PB_HTYPE(field->type) == PB_HTYPE_OPTIONAL)
00430     {
00431         if (field->pSize)
00432         {
00433             if (safe_read_bool(field->pSize) == false)
00434             {
00435                 /* Missing optional field */
00436                 return true;
00437             }
00438         }
00439         else if (PB_ATYPE(field->type) == PB_ATYPE_STATIC)
00440         {
00441             /* Proto3 singular field */
00442             if (pb_check_proto3_default_value(field))
00443                 return true;
00444         }
00445     }
00446 }
00447 if (!field->pData)

```

References `checkreturn`, `encode_array()`, `encode_basic_field()`, `encode_callback_field()`, `PB_ATYPE`, `PB_ATYPE_CALLBACK`, `PB_ATYPE_STATIC`, `pb_check_proto3_default_value()`, `PB_HTYPE`, `PB_HTYPE_ONEOF`, `PB_HTYPE_OPTIONAL`, `PB_HTYPE_REPEATED`, `PB_HTYPE_REQUIRED`, `PB_RETURN_ERROR`, and `safe_read_bool()`.

Here is the call graph for this function:



6.63.2.8 pb_check_proto3_default_value()

```
bool pb_check_proto3_default_value (
    const pb_field_iter_t * field) [static]
```

Definition at line 244 of file [pb_encode.c](#).

```
00245 {
00246     pb_type_t type = field->type;
00247
00248     if (PB_ATYPE(type) == PB_ATYPE_STATIC)
00249     {
00250         if (PB_HTYPE(type) == PB_HTYPE_REQUIRED)
00251         {
00252             /* Required proto2 fields inside proto3 submessage, pretty rare case */
00253             return false;
00254         }
00255         else if (PB_HTYPE(type) == PB_HTYPE_REPEATED)
00256         {
00257             /* Repeated fields inside proto3 submessage: present if count != 0 */
00258             return *(const pb_size_t*)field->pSize == 0;
00259         }
00260         else if (PB_HTYPE(type) == PB_HTYPE_ONEOF)
00261         {
00262             /* Oneof fields */
00263             return *(const pb_size_t*)field->pSize == 0;
00264         }
00265         else if (PB_HTYPE(type) == PB_HTYPE_OPTIONAL && field->pSize != NULL)
00266         {
00267             /* Proto2 optional fields inside proto3 message, or proto3
00268              * submessage fields. */
00269             return safe_read_bool(field->pSize) == false;
00270         }
00271         else if (field->descriptor->default_value)
00272         {
00273             /* Proto3 messages do not have default values, but proto2 messages
00274              * can contain optional fields without has_fields (generator option 'proto3').
00275              * In this case they must always be encoded, to make sure that the
00276              * non-zero default value is overwritten.
00277              */
00278             return false;
00279         }
00280
00281         /* Rest is proto3 singular fields */
00282         if (PB_LTYPE(type) <= PB_LTYPE_LAST_PACKABLE)
00283         {
00284             /* Simple integer / float fields */
00285             pb_size_t i;
00286             const char *p = (const char*)field->pData;
00287             for (i = 0; i < field->data_size; i++)
00288             {
00289                 if (p[i] != 0)
00290                 {
00291                     return false;
00292                 }
00293             }
00294
00295             return true;
00296         }
00297         else if (PB_LTYPE(type) == PB_LTYPE_BYTES)
00298         {
00299             const pb_bytes_array_t *bytes = (const pb_bytes_array_t*)field->pData;
00300             return bytes->size == 0;
00301         }
00302         else if (PB_LTYPE(type) == PB_LTYPE_STRING)
00303         {
00304             return *(const char*)field->pData == '\0';
00305         }
00306         else if (PB_LTYPE(type) == PB_LTYPE_FIXED_LENGTH_BYTES)
00307         {
00308             /* Fixed length bytes is only empty if its length is fixed
00309              * as 0. Which would be pretty strange, but we can check
00310              * it anyway. */
00311             return field->data_size == 0;
00312         }
00313         else if (PB_LTYPE_IS_SUBMSG(type))
00314         {
00315             /* Check all fields in the submessage to find if any of them
00316              * are non-zero. The comparison cannot be done byte-per-byte
00317              * because the C struct may contain padding bytes that must
00318              * be skipped. Note that usually proto3 submessages have
00319              * a separate has_field that is checked earlier in this if.
00320              */
00321             pb_field_iter_t iter;
00322             if (pb_field_iter_begin(&iter, field->submsg_desc, field->pData))
```

```

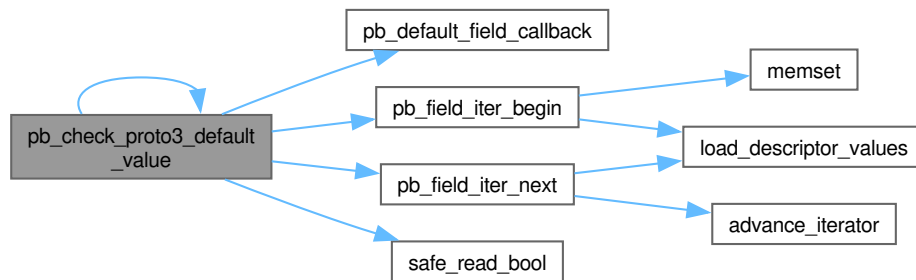
00323     {
00324         do
00325         {
00326             if (!pb_check_proto3_default_value(&iter))
00327             {
00328                 return false;
00329             }
00330         } while (pb_field_iter_next(&iter));
00331     }
00332     return true;
00333 }
00334 }
00335 else if (PB_ATYPE(type) == PB_ATYPE_POINTER)
00336 {
00337     return field->pData == NULL;
00338 }
00339 else if (PB_ATYPE(type) == PB_ATYPE_CALLBACK)
00340 {
00341     if (PB_LTYPE(type) == PB_LTYPE_EXTENSION)
00342     {
00343         const pb_extension_t *extension = *(const pb_extension_t* const *)field->pData;
00344         return extension == NULL;
00345     }
00346     else if (field->descriptor->field_callback == pb_default_field_callback)
00347     {
00348         pb_callback_t *pCallback = (pb_callback_t*)field->pData;
00349         return pCallback->funcs.encode == NULL;
00350     }
00351     else
00352     {
00353         return field->descriptor->field_callback == NULL;
00354     }
00355 }
00356
00357 return false; /* Not typically reached, safe default for weird special cases. */
00358 }

```

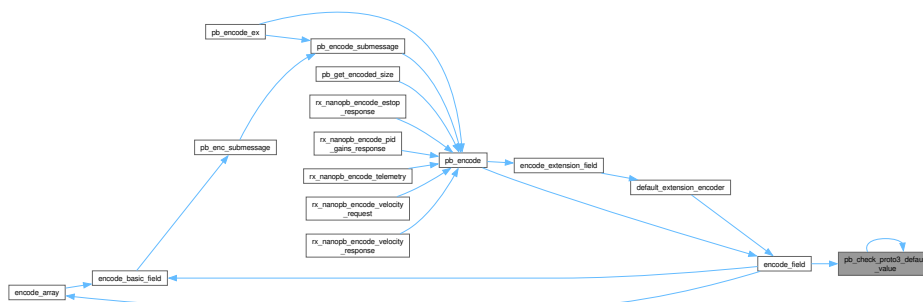
References [checkreturn](#), [NULL](#), [PB_ATYPE](#), [PB_ATYPE_CALLBACK](#), [PB_ATYPE_POINTER](#), [PB_ATYPE_STATIC](#), [pb_check_proto3_default_value\(\)](#), [pb_default_field_callback\(\)](#), [pb_field_iter_begin\(\)](#), [pb_field_iter_next\(\)](#), [PB_HTYPE](#), [PB_HTYPE_ONEOF](#), [PB_HTYPE_OPTIONAL](#), [PB_HTYPE_REPEATED](#), [PB_HTYPE_REQUIRED](#), [PB_LTYPE](#), [PB_LTYPE_BYTES](#), [PB_LTYPE_EXTENSION](#), [PB_LTYPE_FIXED_LENGTH_BYTES](#), [PB_LTYPE_IS_SUBMSG](#), [PB_LTYPE_LAST_PACKABLE](#), [PB_LTYPE_STRING](#), and [safe_read_bool\(\)](#).

Referenced by [encode_field\(\)](#), and [pb_check_proto3_default_value\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.9 pb_enc_bool()

```
bool pb_enc_bool (
    pb_ostream_t * stream,
    const pb_field_iter_t * field) [static]
```

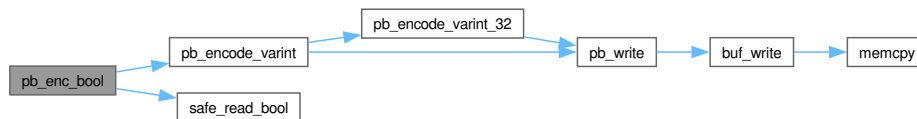
Definition at line 781 of file [pb_encode.c](#).

```
00782 {
00783     uint32_t value = safe_read_bool(field->pData) ? 1 : 0;
00784     PB_UNUSED(field);
00785     return pb_encode_varint(stream, value);
00786 }
```

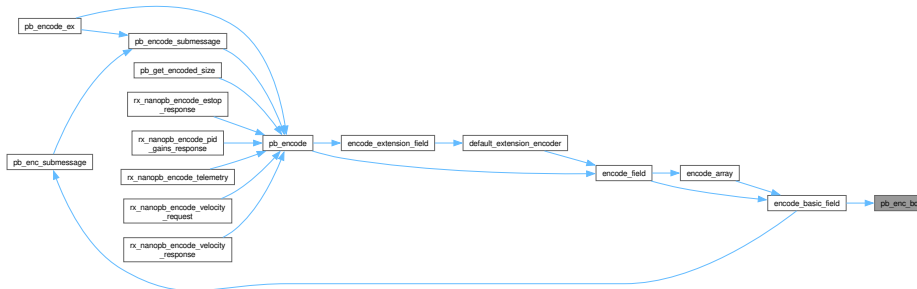
References [checkreturn](#), [pb_encode_varint\(\)](#), [PB_UNUSED](#), and [safe_read_bool\(\)](#).

Referenced by [encode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.10 pb_enc_bytes()

```
bool pb_enc_bytes (
    pb_ostream_t * stream,
    const pb_field_iter_t * field) [static]
```

Definition at line 861 of file [pb_encode.c](#).

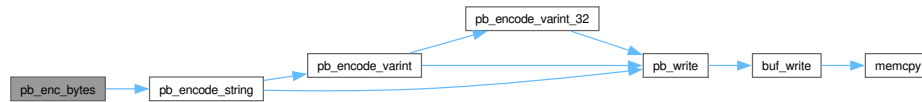
```
00862 {
00863     const pb_bytes_array_t *bytes = NULL;
00864
00865     bytes = (const pb_bytes_array_t*)field->pData;
00866     if (bytes == NULL)
00867     {
00868         /* Treat null pointer as an empty bytes field */
00869         return pb_encode_string(stream, NULL, 0);
00870     }
00871
00872     if (PB_ATYPE(field->type) == PB_ATYPE_STATIC &&
00873         bytes->size > field->data_size - offsetof(pb_bytes_array_t, bytes))
00874     {
00875         PB_RETURN_ERROR(stream, "bytes size exceeded");
00876     }
00877
00878 }
```

```
00879     return pb_encode_string(stream, bytes->bytes, (size_t)bytes->size);
00880 }
```

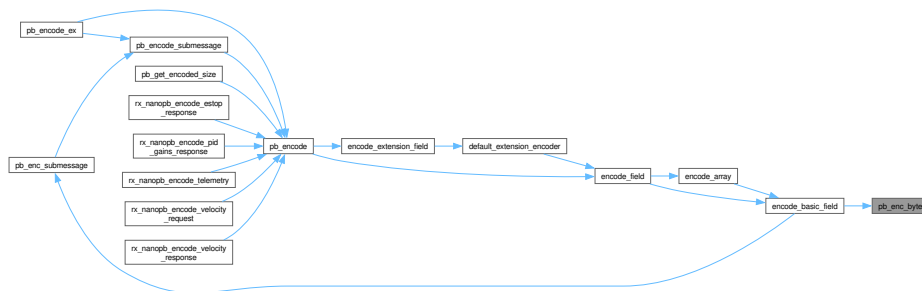
References [checkreturn](#), [NULL](#), [offsetof](#), [PB_ATYPE](#), [PB_ATYPE_STATIC](#), [pb_encode_string\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [encode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.11 pb_enc_fixed()

```
bool pb_enc_fixed (
    pb_ostream_t * stream,
    const pb_field_iter_t * field) [static]
```

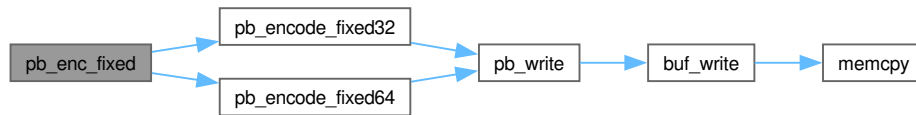
Definition at line 836 of file [pb_encode.c](#).

```
00837 {
00838 #ifdef PB_CONVERT_DOUBLE_FLOAT
00839     if (field->data_size == sizeof(float) && PB_LTYPE(field->type) == PB_LTYPE_FIXED64)
00840     {
00841         return pb_encode_float_as_double(stream, *(float*)field->pData);
00842     }
00843 #endif
00844     if (field->data_size == sizeof(uint32_t))
00845     {
00846         return pb_encode_fixed32(stream, field->pData);
00847     }
00848 #ifndef PB_WITHOUT_64BIT
00849     else if (field->data_size == sizeof(uint64_t))
00850     {
00851         return pb_encode_fixed64(stream, field->pData);
00852     }
00853 #endif
00854     else
00855     {
00856         PB_RETURN_ERROR(stream, "invalid data_size");
00857     }
00858 }
00859 }
```

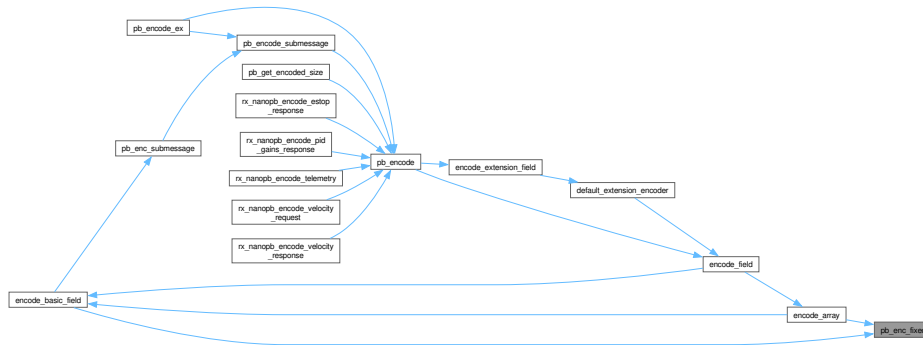
References [checkreturn](#), [pb_encode_fixed32\(\)](#), [pb_encode_fixed64\(\)](#), [PB_LTYPE](#), [PB_LTYPE_FIXED64](#), and [PB_RETURN_ERROR](#).

Referenced by [encode_array\(\)](#), and [encode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.12 pb_enc_fixed_length_bytes()

```

bool pb_enc_fixed_length_bytes (
    pb_ostream_t * stream,
    const pb_field_iter_t * field) [static]
  
```

Definition at line 954 of file [pb_encode.c](#).

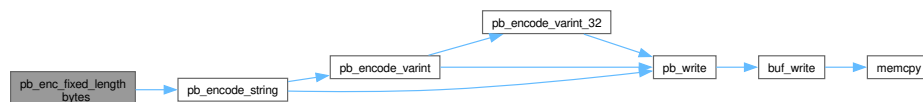
```

00955 {
00956     return pb_encode_string(stream, (const pb_byte_t*)field->pData, (size_t)field->data_size);
00957 }
  
```

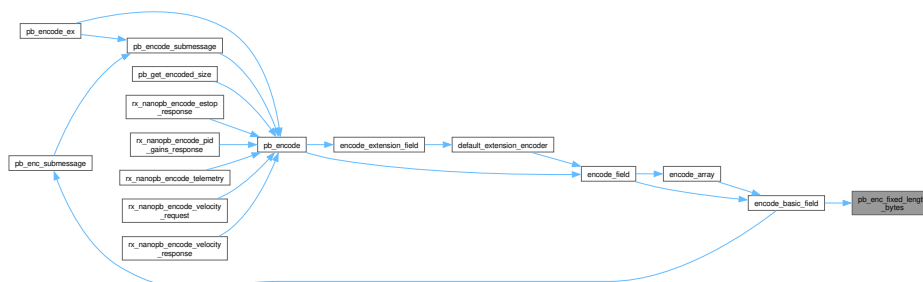
References [checkreturn](#), and [pb_encode_string\(\)](#).

Referenced by [encode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.13 pb_enc_string()

```
bool pb_enc_string (
    pb_ostream_t * stream,
    const pb_field_iter_t * field) [static]
```

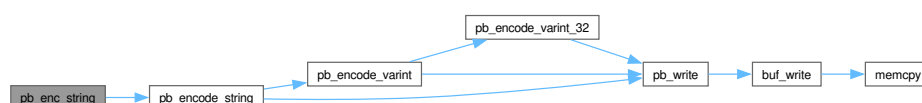
Definition at line 882 of file [pb_encode.c](#).

```
00883 {
00884     size_t size = 0;
00885     size_t max_size = (size_t)field->data_size;
00886     const char *str = (const char*)field->pData;
00887
00888     if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00889     {
00890         max_size = (size_t)-1;
00891     }
00892     else
00893     {
00894         /* pb_dec_string() assumes string fields end with a null
00895          * terminator when the type isn't PB_ATYPE_POINTER, so we
00896          * shouldn't allow more than max-1 bytes to be written to
00897          * allow space for the null terminator.
00898          */
00899         if (max_size == 0)
00900             PB_RETURN_ERROR(stream, "zero-length string");
00901
00902         max_size -= 1;
00903     }
00904
00905     if (str == NULL)
00906     {
00907         size = 0; /* Treat null pointer as an empty string */
00908     }
00909     else
00910     {
00911         const char *p = str;
00912
00913         /* strlen() is not always available, so just use a loop */
00914         while (size < max_size && *p != '\0')
00915         {
00916             size++;
00917             p++;
00918         }
00919
00920         if (*p != '\0')
00921         {
00922             PB_RETURN_ERROR(stream, "unterminated string");
00923         }
00924     }
00925 }
00926
00927 #ifdef PB_VALIDATE_UTF8
00928     if (!pb_validate_utf8(str))
00929         PB_RETURN_ERROR(stream, "invalid utf8");
00930 #endif
00931
00932     return pb_encode_string(stream, (const pb_byte_t*)str, size);
00933 }
```

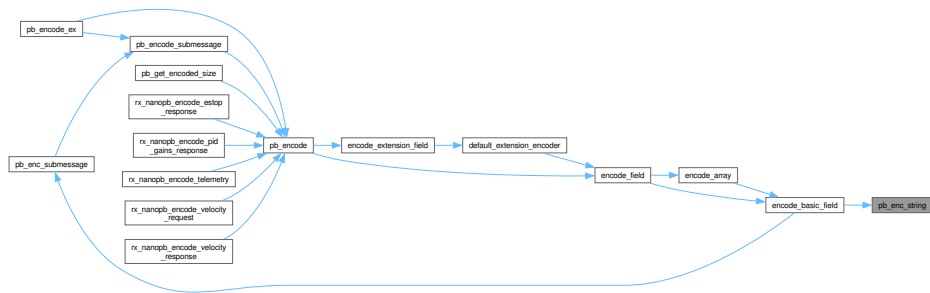
References [checkreturn](#), [NULL](#), [PB_ATYPE](#), [PB_ATYPE_POINTER](#), [pb_encode_string\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [encode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.14 pb_enc_submessage()

```
bool pb_enc_submessage (
    pb_ostream_t * stream,
    const pb_field_iter_t * field) [static]
```

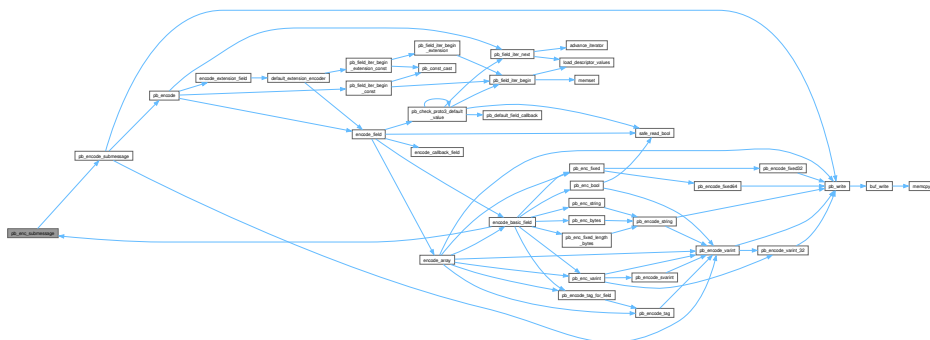
Definition at line 935 of file `pb_encode.c`.

```
00936 {
00937     if (field->submsg_desc == NULL)
00938         PB_RETURN_ERROR(stream, "invalid field descriptor");
00939
00940     if (PB_LTYPE(field->type) == PB_LTYPE_SUBMSG_W_CB && field->pSize != NULL)
00941     {
00942         /* Message callback is stored right before pSize. */
00943         pb_callback_t *callback = (pb_callback_t*)field->pSize - 1;
00944         if (callback->funcs.encode)
00945         {
00946             if (!callback->funcs.encode(stream, field, &callback->arg))
00947                 return false;
00948         }
00949     }
00950
00951     return pb_encode_submessage(stream, field->submsg_desc, field->pData);
00952 }
```

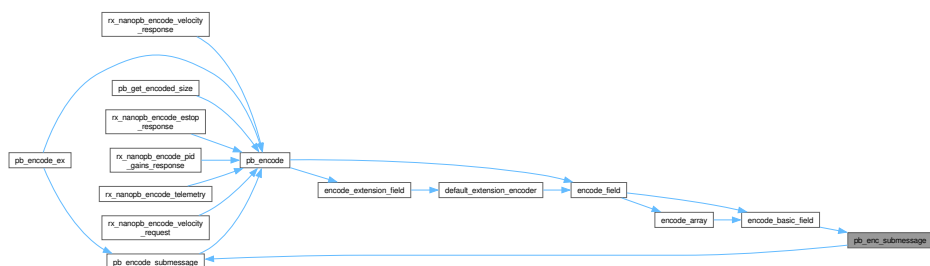
References `checkreturn`, `NULL`, `pb_encode_submessage()`, `PB_LTYPE`, `PB_LTYPE_SUBMSG_W_CB`, and `PB_RETURN_ERROR`.

Referenced by `encode_basic_field()`.

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.15 pb_enc_varint()

```
bool pb_enc_varint (
    pb_ostream_t * stream,
    const pb_field_iter_t * field) [static]
```

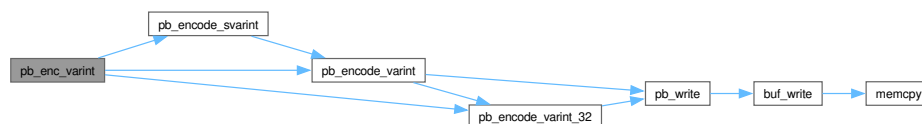
Definition at line 788 of file [pb_encode.c](#).

```
00789 {
00790     if (PB_LTYPE(field->type) == PB_LTYPE_UVARINT)
00791     {
00792         /* Perform unsigned integer extension */
00793         pb_uint64_t value = 0;
00794
00795         if (field->data_size == sizeof(uint_least8_t))
00796             value = *(const uint_least8_t*)field->pData;
00797         else if (field->data_size == sizeof(uint_least16_t))
00798             value = *(const uint_least16_t*)field->pData;
00799         else if (field->data_size == sizeof(uint32_t))
00800             value = *(const uint32_t*)field->pData;
00801         else if (field->data_size == sizeof(pb_uint64_t))
00802             value = *(const pb_uint64_t*)field->pData;
00803         else
00804             PB_RETURN_ERROR(stream, "invalid data_size");
00805
00806         return pb_encode_varint(stream, value);
00807     }
00808     else
00809     {
00810         /* Perform signed integer extension */
00811         pb_int64_t value = 0;
00812
00813         if (field->data_size == sizeof(int_least8_t))
00814             value = *(const int_least8_t*)field->pData;
00815         else if (field->data_size == sizeof(int_least16_t))
00816             value = *(const int_least16_t*)field->pData;
00817         else if (field->data_size == sizeof(int32_t))
00818             value = *(const int32_t*)field->pData;
00819         else if (field->data_size == sizeof(pb_int64_t))
00820             value = *(const pb_int64_t*)field->pData;
00821         else
00822             PB_RETURN_ERROR(stream, "invalid data_size");
00823
00824         if (PB_LTYPE(field->type) == PB_LTYPE_SVARINT)
00825             return pb_encode_svarint(stream, value);
00826         #ifdef PB_WITHOUT_64BIT
00827         else if (value < 0)
00828             return pb_encode_varint_32(stream, (uint32_t)value, (uint32_t)-1);
00829         #endif
00830         else
00831             return pb_encode_varint(stream, (pb_uint64_t)value);
00832     }
00833 }
00834 }
```

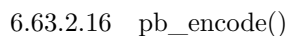
References [checkreturn](#), [pb_encode_svarint\(\)](#), [pb_encode_varint\(\)](#), [pb_encode_varint_32\(\)](#), [pb_int64_t](#), [PB_LTYPE](#), [PB_LTYPE_SVARINT](#), [PB_LTYPE_UVARINT](#), [PB_RETURN_ERROR](#), and [pb_uint64_t](#).

Referenced by [encode_array\(\)](#), and [encode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



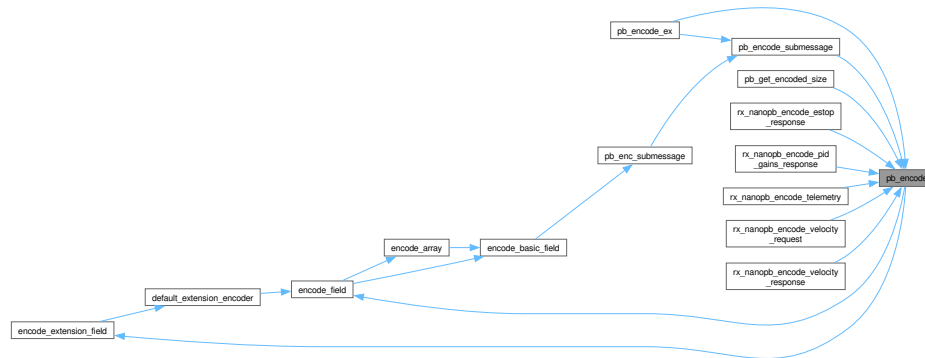
Definition at line 512 of file pb_encode.c.

References `checkreturn`, `encode_extension_field()`, `encode_field()`, `pb_field_iter_begin_const()`, `pb_field_iter_next()`, `PB_LTYPE`, and `PB_LTYPE_EXTENSION`.

Referenced by `pb_encode_ex()`, `pb_encode_submessage()`, `pb_get_encoded_size()`, `rx_nanopb_encode_estop_response()`, `rx_nanopb_encode_pid_gains_response()`, `rx_nanopb_encode_telemetry()`, `rx_nanopb_encode_velocity_request()`, and `rx_nanopb_encode_velocity_response()`.

[illegible]

Here is the caller graph for this function:



6.63.2.17 pb_encode_ex()

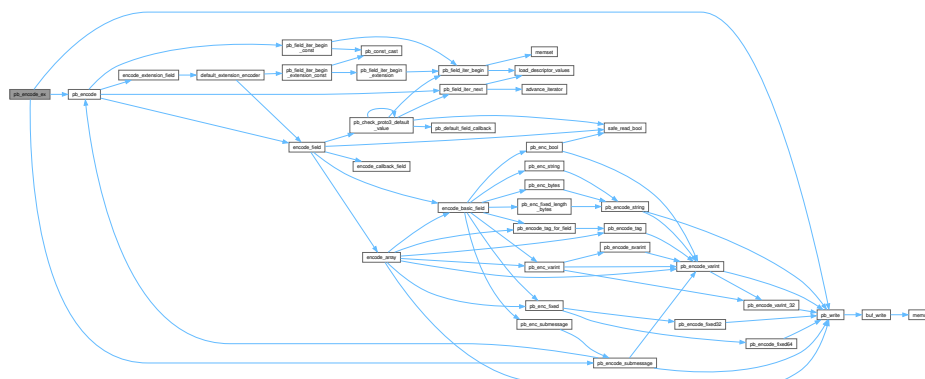
```
bool pb_encode_ex (
    pb_ostream_t * stream,
    const pb_msgdesc_t * fields,
    const void * src_struct,
    unsigned int flags)
```

Definition at line 536 of file pb_encode.c.

```
00537 {
00538     if ((flags & PB_ENCODE_DELIMITED) != 0)
00539     {
00540         return pb_encode_submessage(stream, fields, src_struct);
00541     }
00542     else if ((flags & PB_ENCODE_NULLTERMINATED) != 0)
00543     {
00544         const pb_byte_t zero = 0;
00545
00546         if (!pb_encode(stream, fields, src_struct))
00547             return false;
00548
00549         return pb_write(stream, &zero, 1);
00550     }
00551     else
00552     {
00553         return pb_encode(stream, fields, src_struct);
00554     }
00555 }
```

References [checkreturn](#), [pb_encode\(\)](#), [PB_ENCODE_DELIMITED](#), [PB_ENCODE_NULLTERMINATED](#), [pb_encode_submessage\(\)](#), and [pb_write\(\)](#).

Here is the call graph for this function:



6.63.2.18 pb_encode_fixed32()

```
bool pb_encode_fixed32 (
    pb_ostream_t * stream,
    const void * value)
```

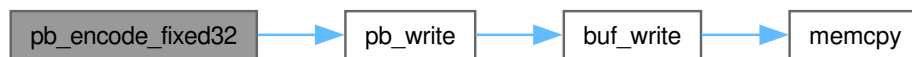
Definition at line 637 of file [pb_encode.c](#).

```
00638 {
00639 #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
00640     /* Fast path if we know that we're on little endian */
00641     return pb_write(stream, (const pb_byte_t*)value, 4);
00642 #else
00643     uint32_t val = *(const uint32_t*)value;
00644     pb_byte_t bytes[4];
00645     bytes[0] = (pb_byte_t)(val & 0xFF);
00646     bytes[1] = (pb_byte_t)((val >> 8) & 0xFF);
00647     bytes[2] = (pb_byte_t)((val >> 16) & 0xFF);
00648     bytes[3] = (pb_byte_t)((val >> 24) & 0xFF);
00649     return pb_write(stream, bytes, 4);
00650 #endif
00651 }
```

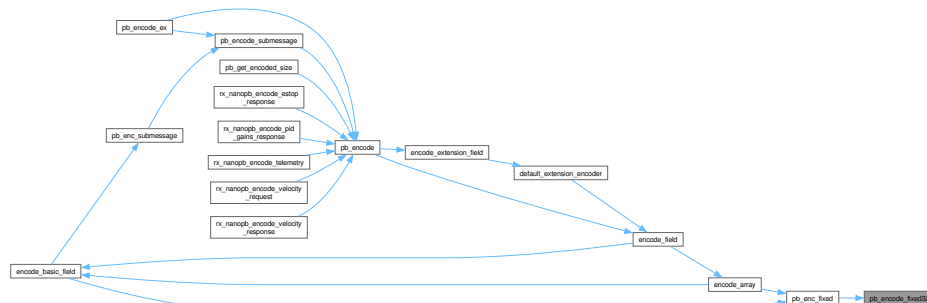
References [checkreturn](#), and [pb_write\(\)](#).

Referenced by [pb_enc_fixed\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.19 pb_encode_fixed64()

```
bool pb_encode_fixed64 (
    pb_ostream_t * stream,
    const void * value)
```

Definition at line 654 of file [pb_encode.c](#).

```
00655 {
00656 #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
00657     /* Fast path if we know that we're on little endian */
00658     return pb_write(stream, (const pb_byte_t*)value, 8);
00659 #else
00660     uint64_t val = *(const uint64_t*)value;
00661     pb_byte_t bytes[8];
00662     bytes[0] = (pb_byte_t)(val & 0xFF);
00663     bytes[1] = (pb_byte_t)((val >> 8) & 0xFF);
00664     bytes[2] = (pb_byte_t)((val >> 16) & 0xFF);
```

```

00665 bytes[3] = (pb_byte_t)((val » 24) & 0xFF);
00666 bytes[4] = (pb_byte_t)((val » 32) & 0xFF);
00667 bytes[5] = (pb_byte_t)((val » 40) & 0xFF);
00668 bytes[6] = (pb_byte_t)((val » 48) & 0xFF);
00669 bytes[7] = (pb_byte_t)((val » 56) & 0xFF);
00670 return pb_write(stream, bytes, 8);
00671 #endif
00672 }

```

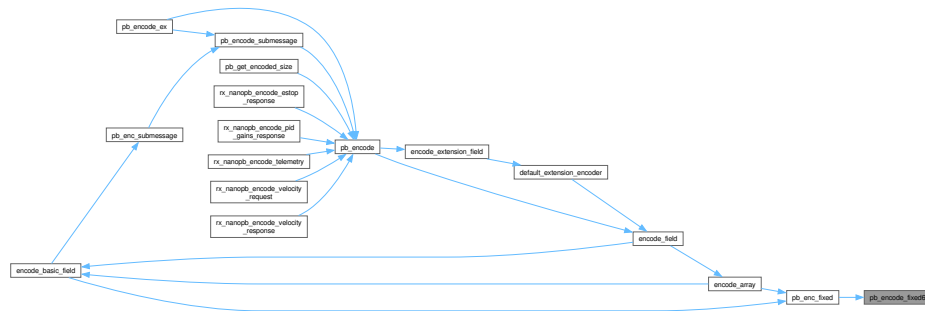
References [checkreturn](#), and [pb_write\(\)](#).

Referenced by [pb_enc_fixed\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.20 pb_encode_string()

```

bool pb_encode_string (
    pb_ostream_t * stream,
    const pb_byte_t * buffer,
    size_t size)

```

Definition at line 716 of file [pb_encode.c](#).

```

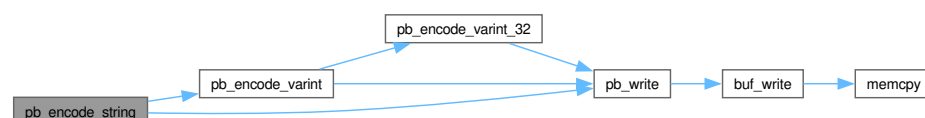
00717 {
00718     if (!pb_encode_varint(stream, (pb_uint64_t)size))
00719         return false;
00720     return pb_write(stream, buffer, size);
00721 }

```

References [checkreturn](#), [pb_encode_varint\(\)](#), [pb_uint64_t](#), and [pb_write\(\)](#).

Referenced by [pb_enc_bytes\(\)](#), [pb_enc_fixed_length_bytes\(\)](#), and [pb_enc_string\(\)](#).

Here is the call graph for this function:



```

graph LR
    pb_encode_ext[pb_encode_ext] --> pb_encode_submessage[pb_encode_submessage]
    pb_encode_submessage --> pb_encode[pb_encode]
    pb_get_encoded_size[pb_get_encoded_size] --> pb_encode
    rx_nanopb_encode_setup_response[rx_nanopb_encode_setup_response] --> pb_encode
    rx_nanopb_encode_get_gains_response[rx_nanopb_encode_get_gains_response] --> pb_encode
    rx_nanopb_encode_telemetry[rx_nanopb_encode_telemetry] --> pb_encode
    rx_nanopb_encode_velocity_request[rx_nanopb_encode_velocity_request] --> pb_encode
    rx_nanopb_encode_velocity_response[rx_nanopb_encode_velocity_response] --> pb_encode
    pb_encode --> encode_extension_field[encode_extension_field]
    encode_extension_field --> default_extension_encoder[default_extension_encoder]
    default_extension_encoder --> encode_field[encode_field]
    encode_field --> encode_array[encode_array]
    encode_array --> encode_basic_field[encode_basic_field]
    encode_basic_field --> pb_encode_bytes[pb_encode_bytes]
    encode_basic_field --> pb_encode_fixed_length_bytes[pb_encode_fixed_length_bytes]
    pb_encode_bytes --> pb_encode_string[pb_encode_string]
    pb_encode_fixed_length_bytes --> pb_encode_string
    pb_encode_string --> pb_encode_ext
    pb_encode_ext --> pb_encode_ext
  
```

```
bool pb_encode_submessage (
    pb_ostream_t * stream,
    const pb_msgdesc_t * fields,
    const void * src_struct)
```

```

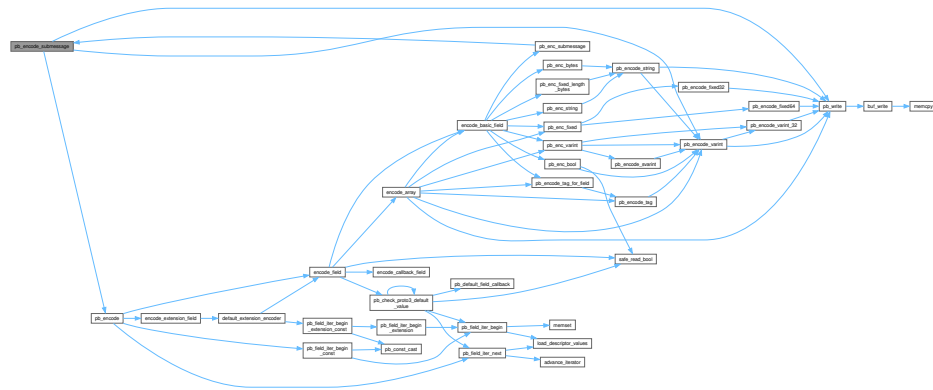
00725 {
00726     /* First calculate the message size using a non-writing substream. */
00727     pb_ostream_t substream = PB_OSTREAM_SIZING;
00728     #if !defined(PB_NO_ENCODE_SIZE_CHECK) || PB_NO_ENCODE_SIZE_CHECK == 0
00729         bool status;
00730         size_t size;
00731     #endif
00732
00733     if (!pb_encode(&substream, fields, src_struct))
00734     {
00735         #ifndef PB_NO_ERRMSG
00736             stream->errmsg = substream.errmsg;
00737         #endif
00738         return false;
00739     }
00740
00741     if (!pb_encode_varint(stream, (pb_uint64_t)substream.bytes_written))
00742         return false;
00743
00744     if (stream->callback == NULL)
00745         return pb_write(stream, NULL, substream.bytes_written); /* Just sizing */
00746
00747     if (stream->bytes_written + substream.bytes_written > stream->max_size)
00748         PB_RETURN_ERROR(stream, "stream full");
00749
00750     #if defined(PB_NO_ENCODE_SIZE_CHECK) && PB_NO_ENCODE_SIZE_CHECK == 1
00751         return pb_encode(stream, fields, src_struct);
00752     #else
00753         size = substream.bytes_written;
00754         /* Use a substream to verify that a callback doesn't write more than
00755          * what it did the first time. */
00756         substream.callback = stream->callback;
00757         substream.state = stream->state;
00758         substream.max_size = size;
00759         substream.bytes_written = 0;
00760         #ifndef PB_NO_ERRMSG
00761             substream.errmsg = NULL;
00762         #endif
00763
00764         status = pb_encode(&substream, fields, src_struct);
00765
00766         stream->bytes_written += substream.bytes_written;
00767         stream->state = substream.state;
00768         #ifndef PB_NO_ERRMSG
00769             stream->errmsg = substream.errmsg;
00770         #endif
00771
00772         if (substream.bytes_written != size)
00773             PB_RETURN_ERROR(stream, "submsg size changed");
00774
00775         return status;
00776     #endif
00777 }

```

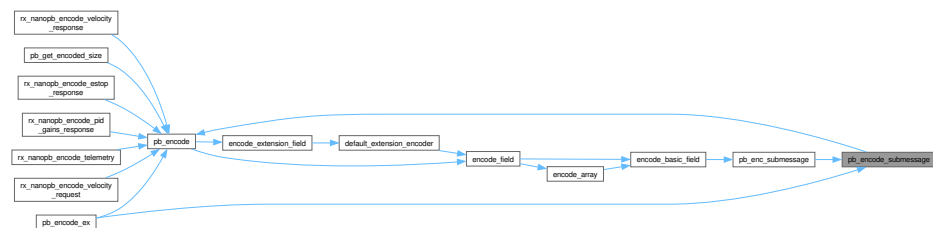
References [checkreturn](#), [NULL](#), [pb_encode\(\)](#), [pb_encode_varint\(\)](#), [PB_OSTREAM_SIZING](#), [PB_RETURN_ERROR](#), [pb_uint64_t](#), and [pb_write\(\)](#).

Referenced by [pb_enc_submessage\(\)](#), and [pb_encode_ex\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.22 pb_encode_svarint()

```
bool pb_encode_svarint (
    pb_ostream_t * stream,
    int64_t value)
```

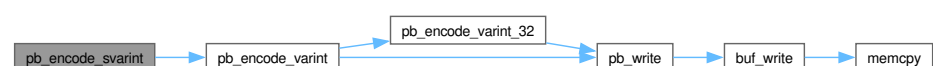
Definition at line 625 of file [pb_encode.c](#).

```
00626 {
00627     pb_uint64_t zigzagged;
00628     pb_uint64_t mask = ((pb_uint64_t)-1) >> 1; /* Satisfy clang -fsanitize=integer */
00629     if (value < 0)
00630         zigzagged = ~(((pb_uint64_t)value & mask) << 1);
00631     else
00632         zigzagged = (pb_uint64_t)value << 1;
00633
00634     return pb_encode_varint(stream, zigzagged);
00635 }
```

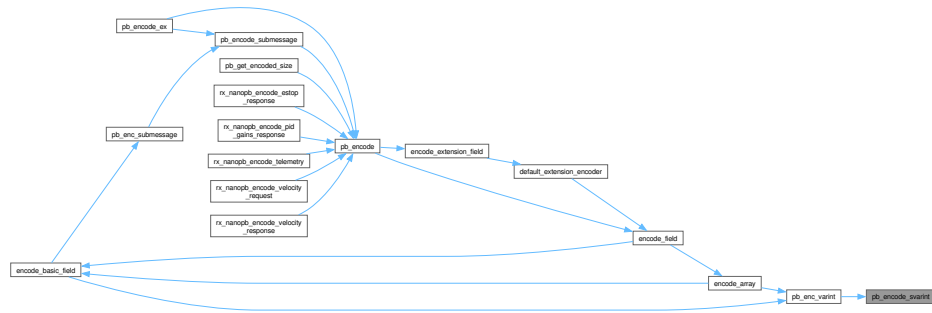
References [checkreturn](#), [pb_encode_varint\(\)](#), [pb_int64_t](#), and [pb_uint64_t](#).

Referenced by [pb_enc_varint\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.23 pb_encode_tag()

```
bool pb_encode_tag (
    pb_ostream_t * stream,
    pb_wire_type_t wiretype,
    uint32_t field_number)
```

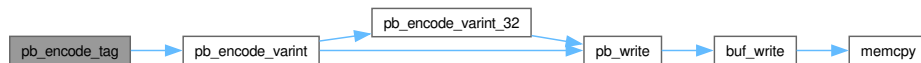
Definition at line 675 of file [pb_encode.c](#).

```
00676 {
00677     pb_uint64_t tag = ((pb_uint64_t)field_number « 3) | wiretype;
00678     return pb_encode_varint(stream, tag);
00679 }
```

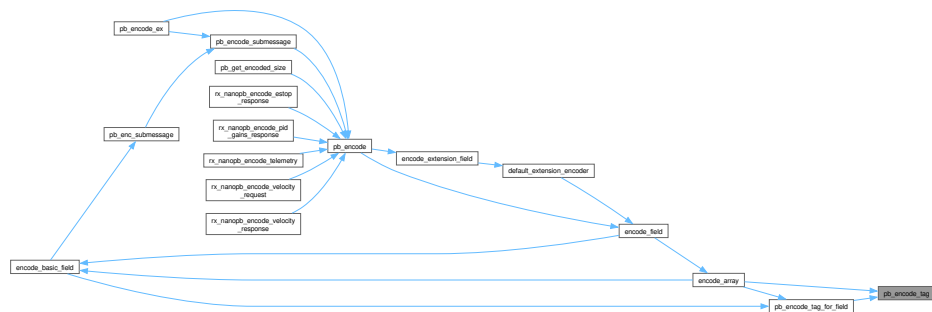
References [checkreturn](#), [pb_encode_varint\(\)](#), and [pb_uint64_t](#).

Referenced by [encode_array\(\)](#), and [pb_encode_tag_for_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:




```
bool pb_encode_varint (
    pb_ostream_t * stream,
    uint64_t value)
```

```
00608 {
00609     if (value <= 0x7F)
00610     {
00611         /* Fast path: single byte */
00612         pb_byte_t byte = (pb_byte_t)value;
00613         return pb_write(stream, &byte, 1);
00614     }
00615     else
00616     {
00617         #ifdef PB_WITHOUT_64BIT
00618             return pb_encode_varint_32(stream, value, 0);
00619         #else
00620             return pb_encode_varint_32(stream, (uint32_t)value, (uint32_t)(value » 32));
00621         #endif
00622     }
00623 }
```

Referenced by `encode_array()`, `pb_enc_bool()`, `pb_enc_varint()`, `pb_encode_string()`, `pb_encode_submessage()`, `pb_encode_svarint()`, and `pb_encode_tag()`.

```

graph LR
    A[pb_encode_varint] --> B[pb_encode_varint_32]
    A --> C[pb_write]
    B --> C
    C --> D[buf_write]
    D --> E[memcpy]
  
```

[illegible]

6.63.2.26 pb_encode_varint_32()

```
bool pb_encode_varint_32 (
    pb_ostream_t * stream,
    uint32_t low,
    uint32_t high) [static]
```

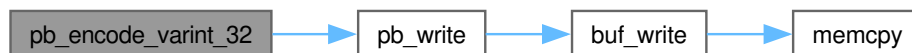
Definition at line 573 of file [pb_encode.c](#).

```
00574 {
00575     size_t i = 0;
00576     pb_byte_t buffer[10];
00577     pb_byte_t byte = (pb_byte_t)(low & 0x7F);
00578     low >>= 7;
00579
00580     while (i < 4 && (low != 0 || high != 0))
00581     {
00582         byte |= 0x80;
00583         buffer[i++] = byte;
00584         byte = (pb_byte_t)(low & 0x7F);
00585         low >>= 7;
00586     }
00587
00588     if (high)
00589     {
00590         byte = (pb_byte_t)(byte | ((high & 0x07) << 4));
00591         high >>= 3;
00592
00593         while (high)
00594         {
00595             byte |= 0x80;
00596             buffer[i++] = byte;
00597             byte = (pb_byte_t)(high & 0x7F);
00598             high >>= 7;
00599         }
00600     }
00601     buffer[i++] = byte;
00602
00603     return pb_write(stream, buffer, i);
00604 }
00605 }
```

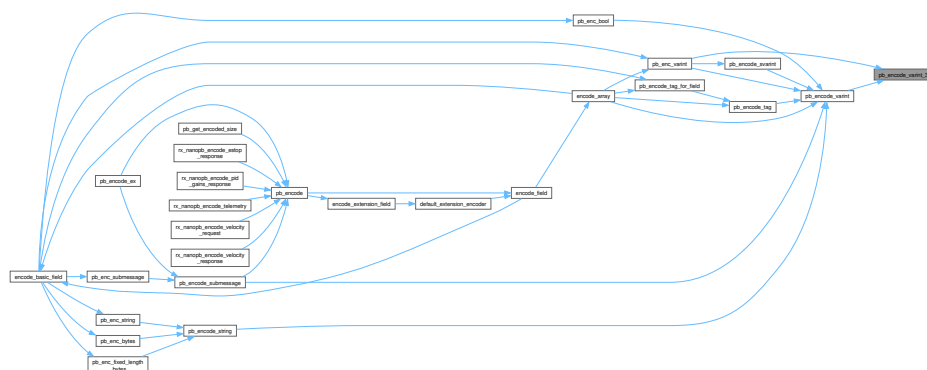
References [checkreturn](#), and [pb_write\(\)](#).

Referenced by [pb_enc_varint\(\)](#), and [pb_encode_varint\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.27 pb_get_encoded_size()

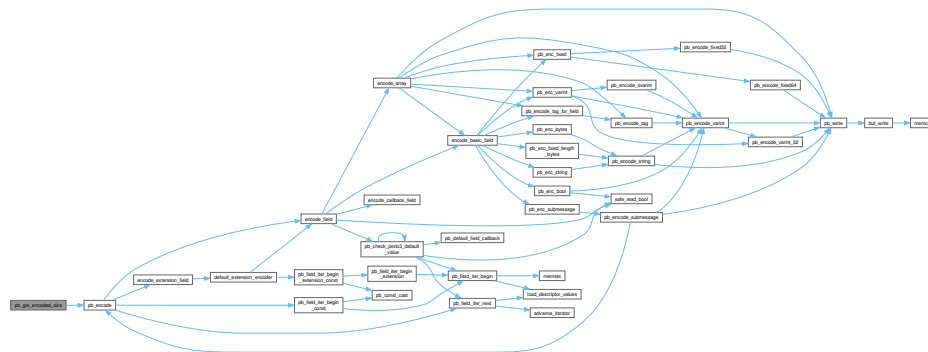
```
bool pb_get_encoded_size (
    size_t * size,
    const pb_msgdesc_t * fields,
    const void * src_struct)
```

Definition at line 557 of file pb_encode.c.

```
00558 {
00559     pb_ostream_t stream = PB_OSTREAM_SIZING;
00560
00561     if (!pb_encode(&stream, fields, src_struct))
00562         return false;
00563
00564     *size = stream.bytes_written;
00565     return true;
00566 }
```

References [pb_encode\(\)](#), and [PB_OSTREAM_SIZING](#).

Here is the call graph for this function:



6.63.2.28 pb_ostream_from_buffer()

```
pb_ostream_t pb_ostream_from_buffer (
    pb_byte_t * buf,
    size_t bufsize)
```

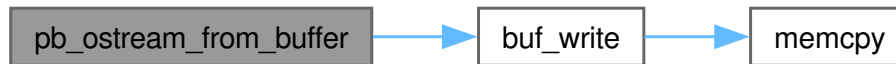
Definition at line 63 of file pb_encode.c.

```
00064 {
00065     pb_ostream_t stream;
00066     #ifdef PB_BUFFER_ONLY
00067         /* In PB_BUFFER_ONLY configuration the callback pointer is just int*.
00068          * NULL pointer marks a sizing field, so put a non-NULL value to mark a buffer stream.
00069          */
00070         static const int marker = 0;
00071         stream.callback = &marker;
00072     #else
00073         stream.callback = &buf_write;
00074     #endif
00075     stream.state = buf;
00076     stream.max_size = bufsize;
00077     stream.bytes_written = 0;
00078     #ifndef PB_NO_ERRMSG
00079         stream.errmsg = NULL;
00080     #endif
00081     return stream;
00082 }
```

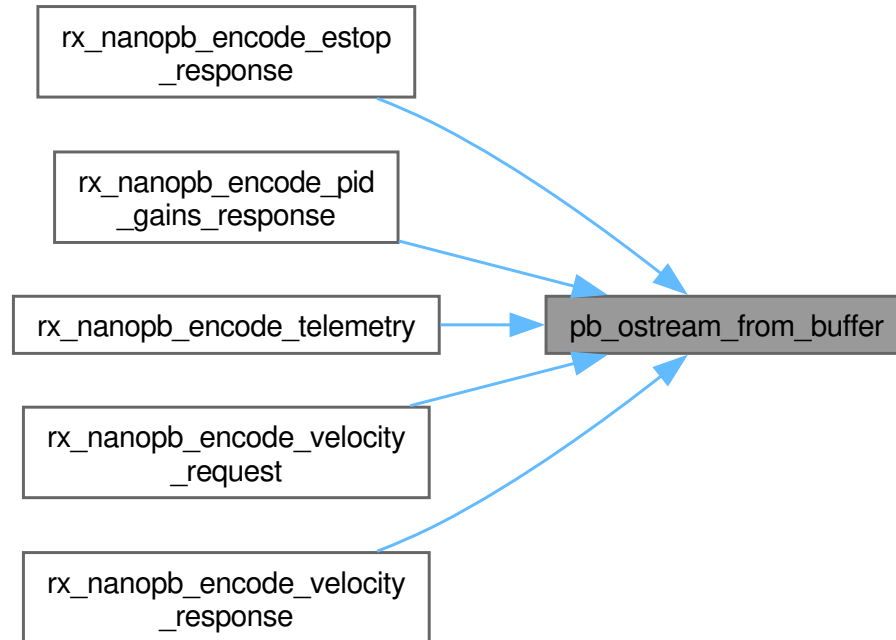
References [buf_write\(\)](#), and [NULL](#).

Referenced by `rx_nanopb_encode_estop_response()`, `rx_nanopb_encode_pid_gains_response()`, `rx_nanopb_encode_telemetry()`, `rx_nanopb_encode_velocity_request()`, and `rx_nanopb_encode_velocity_response()`.

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.29 pb_write()

```

bool pb_write (
    pb_ostream_t * stream,
    const pb_byte_t * buf,
    size_t count)
  
```

Definition at line 84 of file [pb_encode.c](#).

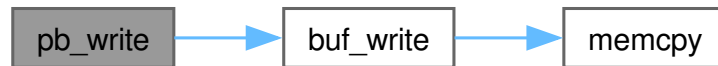
```

00085 {
00086     if (count > 0 && stream->callback != NULL)
00087     {
00088         if (stream->bytes_written + count < stream->bytes_written ||
00089             stream->bytes_written + count > stream->max_size)
00090         {
00091             PB_RETURN_ERROR(stream, "stream full");
00092         }
00093     }
00094     #ifndef PB_BUFFER_ONLY
00095     if (!buf_write(stream, buf, count))
00096         PB_RETURN_ERROR(stream, "io error");
00097     #else
00098     if (!stream->callback(stream, buf, count))
00099         PB_RETURN_ERROR(stream, "io error");
00100     #endif
00101 }
00102
00103 stream->bytes_written += count;
00104 return true;
00105 }
  
```

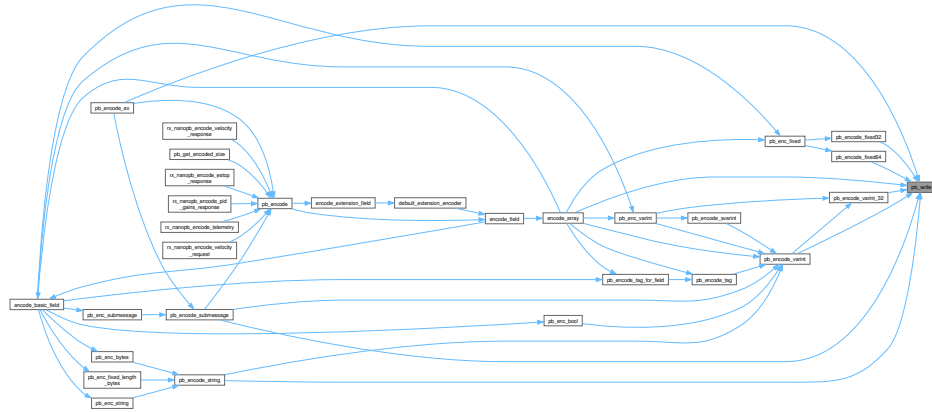
References [buf_write\(\)](#), [checkreturn](#), [NULL](#), and [PB_RETURN_ERROR](#).

Referenced by [encode_array\(\)](#), [pb_encode_ex\(\)](#), [pb_encode_fixed32\(\)](#), [pb_encode_fixed64\(\)](#), [pb_encode_string\(\)](#), [pb_encode_submessage\(\)](#), [pb_encode_varint\(\)](#), and [pb_encode_varint_32\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.63.2.30 safe_read_bool()

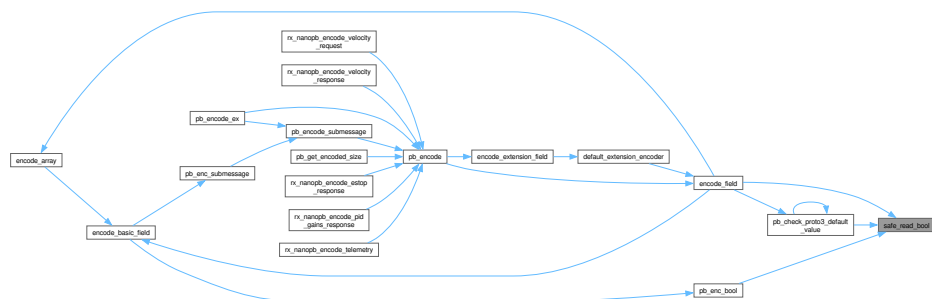
```
bool safe_read_bool (
    const void * pSize) [static]
```

Definition at line 115 of file [pb_encode.c](#).

```
00116 {
00117     const char *p = (const char *)pSize;
00118     size_t i;
00119     for (i = 0; i < sizeof(bool); i++)
00120     {
00121         if (p[i] != 0)
00122             return true;
00123     }
00124     return false;
00125 }
```

Referenced by [encode_field\(\)](#), [pb_check_proto3_default_value\(\)](#), and [pb_enc_bool\(\)](#).

Here is the caller graph for this function:



6.64 pb_encode.c

[Go to the documentation of this file.](#)

```

00001 /* pb_encode.c -- encode a protobuf using minimal resources
00002  *
00003  * 2011 Petteri Aimonen <jpa@kapsi.fi>
00004  */
00005
00006 #include "pb.h"
00007 #include "pb_encode.h"
00008 #include "pb_common.h"
00009
00010 /* Use the GCC warn_unused_result attribute to check that all return values
00011  * are propagated correctly. On other compilers, gcc before 3.4.0 and iar
00012  * before 9.40.1 just ignore the annotation.
00013  */
00014 #if (defined(__GNUC__) && ((__GNUC__ > 3) || (__GNUC__ == 3 && __GNUC_MINOR__ >= 4))) || \
00015     (defined(__IAR_SYSTEMS_ICC__) && (__VER__ >= 9040001))
00016     #define checkreturn __attribute__((warn_unused_result))
00017 #else
00018     #define checkreturn
00019 #endif
00020
00021 /*****
00022  * Declarations internal to this file *
00023  *****/
00024 static bool checkreturn buf_write(pb_ostream_t *stream, const pb_byte_t *buf, size_t count);
00025 static bool checkreturn encode_array(pb_ostream_t *stream, pb_field_iter_t *field);
00026 static bool checkreturn pb_check_proto3_default_value(const pb_field_iter_t *field);
00027 static bool checkreturn encode_basic_field(pb_ostream_t *stream, const pb_field_iter_t *field);
00028 static bool checkreturn encode_callback_field(pb_ostream_t *stream, const pb_field_iter_t *field);
00029 static bool checkreturn encode_field(pb_ostream_t *stream, pb_field_iter_t *field);
00030 static pb_noinline bool checkreturn encode_extension_field(pb_ostream_t *stream, const pb_field_iter_t *field);
00031 static bool checkreturn default_extension_encoder(pb_ostream_t *stream, const pb_extension_t *extension);
00032 static bool checkreturn pb_encode_varint_32(pb_ostream_t *stream, uint32_t low, uint32_t high);
00033 static bool checkreturn pb_enc_bool(pb_ostream_t *stream, const pb_field_iter_t *field);
00034 static bool checkreturn pb_enc_varint(pb_ostream_t *stream, const pb_field_iter_t *field);
00035 static bool checkreturn pb_enc_fixed(pb_ostream_t *stream, const pb_field_iter_t *field);
00036 static bool checkreturn pb_enc_bytes(pb_ostream_t *stream, const pb_field_iter_t *field);
00037 static bool checkreturn pb_enc_string(pb_ostream_t *stream, const pb_field_iter_t *field);
00038 static bool checkreturn pb_enc_submessage(pb_ostream_t *stream, const pb_field_iter_t *field);
00039 static bool checkreturn pb_enc_fixed_length_bytes(pb_ostream_t *stream, const pb_field_iter_t *field);
00040
00041 #ifdef PB_WITHOUT_64BIT
00042 #define pb_int64_t int32_t
00043 #define pb_uint64_t uint32_t
00044 #else
00045 #define pb_int64_t int64_t
00046 #define pb_uint64_t uint64_t
00047 #endif
00048
00049 /*****
00050  * pb_ostream_t implementation *
00051  *****/
00052
00053 static bool checkreturn buf_write(pb_ostream_t *stream, const pb_byte_t *buf, size_t count)
00054 {
00055     pb_byte_t *dest = (pb_byte_t*)stream->state;
00056     stream->state = dest + count;
00057
00058     memcpy(dest, buf, count * sizeof(pb_byte_t));
00059
00060     return true;
00061 }
00062
00063 pb_ostream_t pb_ostream_from_buffer(pb_byte_t *buf, size_t bufsize)
00064 {
00065     pb_ostream_t stream;
00066 #ifdef PB_BUFFER_ONLY
00067     /* In PB_BUFFER_ONLY configuration the callback pointer is just int*.
00068      * NULL pointer marks a sizing field, so put a non-NULL value to mark a buffer stream.
00069      */
00070     static const int marker = 0;
00071     stream.callback = &marker;
00072 #else
00073     stream.callback = &buf_write;
00074 #endif
00075     stream.state = buf;
00076     stream.max_size = bufsize;
00077     stream.bytes_written = 0;
00078 #ifdef PB_NO_ERRMSG
00079     stream.errmsg = NULL;
00080 #endif
00081     return stream;
00082 }

```

```

00083
00084 bool checkreturn pb_write(pb_ostream_t *stream, const pb_byte_t *buf, size_t count)
00085 {
00086     if (count > 0 && stream->callback != NULL)
00087     {
00088         if (stream->bytes_written + count < stream->bytes_written ||
00089             stream->bytes_written + count > stream->max_size)
00090         {
00091             PB_RETURN_ERROR(stream, "stream full");
00092         }
00093     }
00094 #ifndef PB_BUFFER_ONLY
00095     if (!buf_write(stream, buf, count))
00096         PB_RETURN_ERROR(stream, "io error");
00097 #else
00098     if (!stream->callback(stream, buf, count))
00099         PB_RETURN_ERROR(stream, "io error");
00100 #endif
00101 }
00102
00103 stream->bytes_written += count;
00104 return true;
00105 }
00106
00107 /*****
00108  * Encode a single field *
00109  *****/
00110
00111 /* Read a bool value without causing undefined behavior even if the value
00112  * is invalid. See issue #434 and
00113  * https://stackoverflow.com/questions/27661768/weird-results-for-conditional
00114  */
00115 static bool safe_read_bool(const void *pSize)
00116 {
00117     const char *p = (const char *)pSize;
00118     size_t i;
00119     for (i = 0; i < sizeof(bool); i++)
00120     {
00121         if (p[i] != 0)
00122             return true;
00123     }
00124     return false;
00125 }
00126
00127 /* Encode a static array. Handles the size calculations and possible packing. */
00128 static bool checkreturn encode_array(pb_ostream_t *stream, pb_field_iter_t *field)
00129 {
00130     pb_size_t i;
00131     pb_size_t count;
00132 #ifndef PB_ENCODE_ARRAYS_UNPACKED
00133     size_t size;
00134 #endif
00135
00136     count = *(pb_size_t*)field->pSize;
00137
00138     if (count == 0)
00139         return true;
00140
00141     if (PB_ATYPE(field->type) != PB_ATYPE_POINTER && count > field->array_size)
00142         PB_RETURN_ERROR(stream, "array max size exceeded");
00143
00144 #ifndef PB_ENCODE_ARRAYS_UNPACKED
00145     /* We always pack arrays if the datatype allows it. */
00146     if (PB_LTYPE(field->type) <= PB_LTYPE_LAST_PACKABLE)
00147     {
00148         if (!pb_encode_tag(stream, PB_WT_STRING, field->tag))
00149             return false;
00150
00151         /* Determine the total size of packed array. */
00152         if (PB_LTYPE(field->type) == PB_LTYPE_FIXED32)
00153         {
00154             size = 4 * (size_t)count;
00155         }
00156         else if (PB_LTYPE(field->type) == PB_LTYPE_FIXED64)
00157         {
00158             size = 8 * (size_t)count;
00159         }
00160         else
00161         {
00162             pb_ostream_t sizestream = PB_OSTREAM_SIZING;
00163             void *pData_orig = field->pData;
00164             for (i = 0; i < count; i++)
00165             {
00166                 if (!pb_enc_varint(&sizestream, field))
00167                     PB_RETURN_ERROR(stream, PB_GET_ERROR(&sizestream));
00168                 field->pData = (char*)field->pData + field->data_size;
00169             }

```

```

00170         field->pData = pData_orig;
00171         size = sizestream.bytes_written;
00172     }
00173
00174     if (!pb_encode_varint(stream, (pb_uint64_t)size))
00175         return false;
00176
00177     if (stream->callback == NULL)
00178         return pb_write(stream, NULL, size); /* Just sizing.. */
00179
00180     /* Write the data */
00181     for (i = 0; i < count; i++)
00182     {
00183         if (PB_LTYPE(field->type) == PB_LTYPE_FIXED32 || PB_LTYPE(field->type) == PB_LTYPE_FIXED64)
00184         {
00185             if (!pb_enc_fixed(stream, field))
00186                 return false;
00187         }
00188         else
00189         {
00190             if (!pb_enc_varint(stream, field))
00191                 return false;
00192         }
00193     }
00194     field->pData = (char*)field->pData + field->data_size;
00195 }
00196 }
00197 else /* Unpacked fields */
00198 #endif
00199 {
00200     for (i = 0; i < count; i++)
00201     {
00202         /* Normally the data is stored directly in the array entries, but
00203          * for pointer-type string and bytes fields, the array entries are
00204          * actually pointers themselves also. So we have to dereference once
00205          * more to get to the actual data. */
00206         if (PB_ATYPE(field->type) == PB_ATYPE_POINTER &&
00207             (PB_LTYPE(field->type) == PB_LTYPE_STRING ||
00208              PB_LTYPE(field->type) == PB_LTYPE_BYTES))
00209         {
00210             bool status;
00211             void *pData_orig = field->pData;
00212             field->pData = *(void* const*)field->pData;
00213
00214             if (!field->pData)
00215             {
00216                 /* Null pointer in array is treated as empty string / bytes */
00217                 status = pb_encode_tag_for_field(stream, field) &&
00218                     pb_encode_varint(stream, 0);
00219             }
00220             else
00221             {
00222                 status = encode_basic_field(stream, field);
00223             }
00224
00225             field->pData = pData_orig;
00226
00227             if (!status)
00228                 return false;
00229         }
00230         else
00231         {
00232             if (!encode_basic_field(stream, field))
00233                 return false;
00234         }
00235         field->pData = (char*)field->pData + field->data_size;
00236     }
00237 }
00238
00239 return true;
00240 }
00241
00242 /* In proto3, all fields are optional and are only encoded if their value is "non-zero".
00243  * This function implements the check for the zero value. */
00244 static bool checkreturn pb_check_proto3_default_value(const pb_field_iter_t *field)
00245 {
00246     pb_type_t type = field->type;
00247
00248     if (PB_ATYPE(type) == PB_ATYPE_STATIC)
00249     {
00250         if (PB_HTYPE(type) == PB_HTYPE_REQUIRED)
00251         {
00252             /* Required proto2 fields inside proto3 submessage, pretty rare case */
00253             return false;
00254         }
00255         else if (PB_HTYPE(type) == PB_HTYPE_REPEATED)
00256         {

```



```

00257     /* Repeated fields inside proto3 submessage: present if count != 0 */
00258     return *(const pb_size_t*)field->pSize == 0;
00259 }
00260 else if (PB_HTYPE(type) == PB_HTYPE_ONEOF)
00261 {
00262     /* Oneof fields */
00263     return *(const pb_size_t*)field->pSize == 0;
00264 }
00265 else if (PB_HTYPE(type) == PB_HTYPE_OPTIONAL && field->pSize != NULL)
00266 {
00267     /* Proto2 optional fields inside proto3 message, or proto3
00268      * submessage fields. */
00269     return safe_read_bool(field->pSize) == false;
00270 }
00271 else if (field->descriptor->default_value)
00272 {
00273     /* Proto3 messages do not have default values, but proto2 messages
00274      * can contain optional fields without has_fields (generator option 'proto3').
00275      * In this case they must always be encoded, to make sure that the
00276      * non-zero default value is overwritten.
00277      */
00278     return false;
00279 }
00280
00281 /* Rest is proto3 singular fields */
00282 if (PB_LTYPE(type) <= PB_LTYPE_LAST_PACKABLE)
00283 {
00284     /* Simple integer / float fields */
00285     pb_size_t i;
00286     const char *p = (const char*)field->pData;
00287     for (i = 0; i < field->data_size; i++)
00288     {
00289         if (p[i] != 0)
00290         {
00291             return false;
00292         }
00293     }
00294     return true;
00295 }
00296 else if (PB_LTYPE(type) == PB_LTYPE_BYTES)
00297 {
00298     const pb_bytes_array_t *bytes = (const pb_bytes_array_t*)field->pData;
00299     return bytes->size == 0;
00300 }
00301 else if (PB_LTYPE(type) == PB_LTYPE_STRING)
00302 {
00303     return *(const char*)field->pData == '\0';
00304 }
00305 else if (PB_LTYPE(type) == PB_LTYPE_FIXED_LENGTH_BYTES)
00306 {
00307     /* Fixed length bytes is only empty if its length is fixed
00308      * as 0. Which would be pretty strange, but we can check
00309      * it anyway. */
00310     return field->data_size == 0;
00311 }
00312 else if (PB_LTYPE_IS_SUBMSG(type))
00313 {
00314     /* Check all fields in the submessage to find if any of them
00315      * are non-zero. The comparison cannot be done byte-per-byte
00316      * because the C struct may contain padding bytes that must
00317      * be skipped. Note that usually proto3 submessages have
00318      * a separate has_field that is checked earlier in this if.
00319      */
00320     pb_field_iter_t iter;
00321     if (pb_field_iter_begin(&iter, field->submsg_desc, field->pData))
00322     {
00323         do
00324         {
00325             if (!pb_check_proto3_default_value(&iter))
00326             {
00327                 return false;
00328             }
00329         } while (pb_field_iter_next(&iter));
00330     }
00331     return true;
00332 }
00333 }
00334 else if (PB_ATYPE(type) == PB_ATYPE_POINTER)
00335 {
00336     return field->pData == NULL;
00337 }
00338 else if (PB_ATYPE(type) == PB_ATYPE_CALLBACK)
00339 {
00340     if (PB_LTYPE(type) == PB_LTYPE_EXTENSION)
00341     {
00342         const pb_extension_t *extension = *(const pb_extension_t* const *)field->pData;

```

```

00344         return extension == NULL;
00345     }
00346     else if (field->descriptor->field_callback == pb_default_field_callback)
00347     {
00348         pb_callback_t *pCallback = (pb_callback_t*)field->pData;
00349         return pCallback->funcs.encode == NULL;
00350     }
00351     else
00352     {
00353         return field->descriptor->field_callback == NULL;
00354     }
00355 }
00356
00357 return false; /* Not typically reached, safe default for weird special cases. */
00358 }
00359
00360 /* Encode a field with static or pointer allocation, i.e. one whose data
00361  * is available to the encoder directly. */
00362 static bool checkreturn encode_basic_field(pb_ostream_t *stream, const pb_field_iter_t *field)
00363 {
00364     if (!field->pData)
00365     {
00366         /* Missing pointer field */
00367         return true;
00368     }
00369
00370     if (!pb_encode_tag_for_field(stream, field))
00371         return false;
00372
00373     switch (PB_LTYPE(field->type))
00374     {
00375         case PB_LTYPE_BOOL:
00376             return pb_enc_bool(stream, field);
00377
00378         case PB_LTYPE_VARINT:
00379         case PB_LTYPE_UVARINT:
00380         case PB_LTYPE_SVARINT:
00381             return pb_enc_varint(stream, field);
00382
00383         case PB_LTYPE_FIXED32:
00384         case PB_LTYPE_FIXED64:
00385             return pb_enc_fixed(stream, field);
00386
00387         case PB_LTYPE_BYTES:
00388             return pb_enc_bytes(stream, field);
00389
00390         case PB_LTYPE_STRING:
00391             return pb_enc_string(stream, field);
00392
00393         case PB_LTYPE_SUBMESSAGE:
00394         case PB_LTYPE_SUBMSG_W_CB:
00395             return pb_enc_submessage(stream, field);
00396
00397         case PB_LTYPE_FIXED_LENGTH_BYTES:
00398             return pb_enc_fixed_length_bytes(stream, field);
00399
00400         default:
00401             PB_RETURN_ERROR(stream, "invalid field type");
00402     }
00403 }
00404
00405 /* Encode a field with callback semantics. This means that a user function is
00406  * called to provide and encode the actual data. */
00407 static bool checkreturn encode_callback_field(pb_ostream_t *stream, const pb_field_iter_t *field)
00408 {
00409     if (field->descriptor->field_callback != NULL)
00410     {
00411         if (!field->descriptor->field_callback(NULL, stream, field))
00412             PB_RETURN_ERROR(stream, "callback error");
00413     }
00414     return true;
00415 }
00416
00417 /* Encode a single field of any callback, pointer or static type. */
00418 static bool checkreturn encode_field(pb_ostream_t *stream, pb_field_iter_t *field)
00419 {
00420     /* Check field presence */
00421     if (PB_HTYPE(field->type) == PB_HTYPE_ONEOF)
00422     {
00423         if (*(const pb_size_t*)field->pSize != field->tag)
00424         {
00425             /* Different type oneof field */
00426             return true;
00427         }
00428     }
00429     else if (PB_HTYPE(field->type) == PB_HTYPE_OPTIONAL)
00430     {

```

```

00431     if (field->pSize)
00432     {
00433         if (safe_read_bool(field->pSize) == false)
00434         {
00435             /* Missing optional field */
00436             return true;
00437         }
00438     }
00439     else if (PB_ATYPE(field->type) == PB_ATYPE_STATIC)
00440     {
00441         /* Proto3 singular field */
00442         if (pb_check_proto3_default_value(field))
00443             return true;
00444     }
00445 }
00446
00447 if (!field->pData)
00448 {
00449     if (PB_HTYPE(field->type) == PB_HTYPE_REQUIRED)
00450         PB_RETURN_ERROR(stream, "missing required field");
00451
00452     /* Pointer field set to NULL */
00453     return true;
00454 }
00455
00456 /* Then encode field contents */
00457 if (PB_ATYPE(field->type) == PB_ATYPE_CALLBACK)
00458 {
00459     return encode_callback_field(stream, field);
00460 }
00461 else if (PB_HTYPE(field->type) == PB_HTYPE_REPEATED)
00462 {
00463     return encode_array(stream, field);
00464 }
00465 else
00466 {
00467     return encode_basic_field(stream, field);
00468 }
00469 }
00470
00471 /* Default handler for extension fields. Expects to have a pb_msgdesc_t
00472 * pointer in the extension->type->arg field, pointing to a message with
00473 * only one field in it. */
00474 static bool checkreturn default_extension_encoder(pb_ostream_t *stream, const pb_extension_t *extension)
00475 {
00476     pb_field_iter_t iter;
00477
00478     if (!pb_field_iter_begin_extension_const(&iter, extension))
00479         PB_RETURN_ERROR(stream, "invalid extension");
00480
00481     return encode_field(stream, &iter);
00482 }
00483
00484
00485 /* Walk through all the registered extensions and give them a chance
00486 * to encode themselves. */
00487 static pb_noinline bool checkreturn encode_extension_field(pb_ostream_t *stream, const pb_field_iter_t *field)
00488 {
00489     const pb_extension_t *extension = *(const pb_extension_t* const *)field->pData;
00490
00491     while (extension)
00492     {
00493         bool status;
00494         if (extension->type->encode)
00495             status = extension->type->encode(stream, extension);
00496         else
00497             status = default_extension_encoder(stream, extension);
00498
00499         if (!status)
00500             return false;
00501
00502         extension = extension->next;
00503     }
00504
00505     return true;
00506 }
00507
00508 /*****
00509 * Encode all fields *
00510 *****/
00511
00512 bool checkreturn pb_encode(pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct)
00513 {
00514     pb_field_iter_t iter;
00515     if (!pb_field_iter_begin_const(&iter, fields, src_struct))
00516         return true; /* Empty message type */
00517

```

```

00518     do {
00519         if (PB_LTYPE(iter.type) == PB_LTYPE_EXTENSION)
00520         {
00521             /* Special case for the extension field placeholder */
00522             if (!encode_extension_field(stream, &iter))
00523                 return false;
00524         }
00525         else
00526         {
00527             /* Regular field */
00528             if (!encode_field(stream, &iter))
00529                 return false;
00530         }
00531     } while (pb_field_iter_next(&iter));
00532
00533     return true;
00534 }
00535
00536 bool checkreturn pb_encode_ex(pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct, unsigned int
    flags)
00537 {
00538     if ((flags & PB_ENCODE_DELIMITED) != 0)
00539     {
00540         return pb_encode_submessage(stream, fields, src_struct);
00541     }
00542     else if ((flags & PB_ENCODE_NULLTERMINATED) != 0)
00543     {
00544         const pb_byte_t zero = 0;
00545         if (!pb_encode(stream, fields, src_struct))
00546             return false;
00547         return pb_write(stream, &zero, 1);
00548     }
00549     else
00550     {
00551         return pb_encode(stream, fields, src_struct);
00552     }
00553 }
00554
00555 bool pb_get_encoded_size(size_t *size, const pb_msgdesc_t *fields, const void *src_struct)
00556 {
00557     pb_ostream_t stream = PB_OSTREAM_SIZING;
00558     if (!pb_encode(&stream, fields, src_struct))
00559         return false;
00560     *size = stream.bytes_written;
00561     return true;
00562 }
00563
00564 /* Helper functions */
00565
00566 /* This function avoids 64-bit shifts as they are quite slow on many platforms. */
00567 static bool checkreturn pb_encode_varint_32(pb_ostream_t *stream, uint32_t low, uint32_t high)
00568 {
00569     size_t i = 0;
00570     pb_byte_t buffer[10];
00571     pb_byte_t byte = (pb_byte_t)(low & 0x7F);
00572     low >>= 7;
00573
00574     while (i < 4 && (low != 0 || high != 0))
00575     {
00576         byte |= 0x80;
00577         buffer[i++] = byte;
00578         byte = (pb_byte_t)(low & 0x7F);
00579         low >>= 7;
00580     }
00581
00582     if (high)
00583     {
00584         byte = (pb_byte_t)(byte | ((high & 0x07) << 4));
00585         high >>= 3;
00586
00587         while (high)
00588         {
00589             byte |= 0x80;
00590             buffer[i++] = byte;
00591             byte = (pb_byte_t)(high & 0x7F);
00592             high >>= 7;
00593         }
00594     }
00595
00596     buffer[i++] = byte;
00597
00598     return true;
00599 }
00600
00601 buffer[i++] = byte;
00602
00603

```

```

00604     return pb_write(stream, buffer, i);
00605 }
00606
00607 bool checkreturn pb_encode_varint(pb_ostream_t *stream, pb_uint64_t value)
00608 {
00609     if (value <= 0x7F)
00610     {
00611         /* Fast path: single byte */
00612         pb_byte_t byte = (pb_byte_t)value;
00613         return pb_write(stream, &byte, 1);
00614     }
00615     else
00616     {
00617 #ifndef PB_WITHOUT_64BIT
00618         return pb_encode_varint_32(stream, value, 0);
00619 #else
00620         return pb_encode_varint_32(stream, (uint32_t)value, (uint32_t)(value >> 32));
00621 #endif
00622     }
00623 }
00624
00625 bool checkreturn pb_encode_svarint(pb_ostream_t *stream, pb_int64_t value)
00626 {
00627     pb_uint64_t zigzagged;
00628     pb_uint64_t mask = ((pb_uint64_t)-1) >> 1; /* Satisfy clang -fsanitize=integer */
00629     if (value < 0)
00630         zigzagged = ~(((pb_uint64_t)value & mask) << 1);
00631     else
00632         zigzagged = (pb_uint64_t)value << 1;
00633
00634     return pb_encode_varint(stream, zigzagged);
00635 }
00636
00637 bool checkreturn pb_encode_fixed32(pb_ostream_t *stream, const void *value)
00638 {
00639 #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
00640     /* Fast path if we know that we're on little endian */
00641     return pb_write(stream, (const pb_byte_t*)value, 4);
00642 #else
00643     uint32_t val = *(const uint32_t*)value;
00644     pb_byte_t bytes[4];
00645     bytes[0] = (pb_byte_t)(val & 0xFF);
00646     bytes[1] = (pb_byte_t)((val >> 8) & 0xFF);
00647     bytes[2] = (pb_byte_t)((val >> 16) & 0xFF);
00648     bytes[3] = (pb_byte_t)((val >> 24) & 0xFF);
00649     return pb_write(stream, bytes, 4);
00650 #endif
00651 }
00652
00653 #ifndef PB_WITHOUT_64BIT
00654 bool checkreturn pb_encode_fixed64(pb_ostream_t *stream, const void *value)
00655 {
00656 #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
00657     /* Fast path if we know that we're on little endian */
00658     return pb_write(stream, (const pb_byte_t*)value, 8);
00659 #else
00660     uint64_t val = *(const uint64_t*)value;
00661     pb_byte_t bytes[8];
00662     bytes[0] = (pb_byte_t)(val & 0xFF);
00663     bytes[1] = (pb_byte_t)((val >> 8) & 0xFF);
00664     bytes[2] = (pb_byte_t)((val >> 16) & 0xFF);
00665     bytes[3] = (pb_byte_t)((val >> 24) & 0xFF);
00666     bytes[4] = (pb_byte_t)((val >> 32) & 0xFF);
00667     bytes[5] = (pb_byte_t)((val >> 40) & 0xFF);
00668     bytes[6] = (pb_byte_t)((val >> 48) & 0xFF);
00669     bytes[7] = (pb_byte_t)((val >> 56) & 0xFF);
00670     return pb_write(stream, bytes, 8);
00671 #endif
00672 }
00673 #endif
00674
00675 bool checkreturn pb_encode_tag(pb_ostream_t *stream, pb_wire_type_t wiretype, uint32_t field_number)
00676 {
00677     pb_uint64_t tag = ((pb_uint64_t)field_number << 3) | wiretype;
00678     return pb_encode_varint(stream, tag);
00679 }
00680
00681 bool pb_encode_tag_for_field ( pb_ostream_t* stream, const pb_field_iter_t* field )
00682 {
00683     pb_wire_type_t wiretype;
00684     switch (PB_LTYPE(field->type))
00685     {
00686     case PB_LTYPE_BOOL:
00687     case PB_LTYPE_VARINT:
00688     case PB_LTYPE_UVARINT:
00689     case PB_LTYPE_SVARINT:
00690         wiretype = PB_WT_VARINT;

```

```

00691         break;
00692
00693     case PB_LTYPE_FIXED32:
00694         wiretype = PB_WT_32BIT;
00695         break;
00696
00697     case PB_LTYPE_FIXED64:
00698         wiretype = PB_WT_64BIT;
00699         break;
00700
00701     case PB_LTYPE_BYTES:
00702     case PB_LTYPE_STRING:
00703     case PB_LTYPE_SUBMESSAGE:
00704     case PB_LTYPE_SUBMSG_W_CB:
00705     case PB_LTYPE_FIXED_LENGTH_BYTES:
00706         wiretype = PB_WT_STRING;
00707         break;
00708
00709     default:
00710         PB_RETURN_ERROR(stream, "invalid field type");
00711 }
00712
00713 return pb_encode_tag(stream, wiretype, field->tag);
00714 }
00715
00716 bool checkreturn pb_encode_string(pb_ostream_t *stream, const pb_byte_t *buffer, size_t size)
00717 {
00718     if (!pb_encode_varint(stream, (pb_uint64_t)size))
00719         return false;
00720
00721     return pb_write(stream, buffer, size);
00722 }
00723
00724 bool checkreturn pb_encode_submessage(pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct)
00725 {
00726     /* First calculate the message size using a non-writing substream. */
00727     pb_ostream_t substream = PB_OSTREAM_SIZING;
00728     #if !defined(PB_NO_ENCODE_SIZE_CHECK) || PB_NO_ENCODE_SIZE_CHECK == 0
00729         bool status;
00730         size_t size;
00731     #endif
00732
00733     if (!pb_encode(&substream, fields, src_struct))
00734     {
00735         #ifndef PB_NO_ERRMSG
00736         stream->errmsg = substream.errmsg;
00737         #endif
00738         return false;
00739     }
00740
00741     if (!pb_encode_varint(stream, (pb_uint64_t)substream.bytes_written))
00742         return false;
00743
00744     if (stream->callback == NULL)
00745         return pb_write(stream, NULL, substream.bytes_written); /* Just sizing */
00746
00747     if (stream->bytes_written + substream.bytes_written > stream->max_size)
00748         PB_RETURN_ERROR(stream, "stream full");
00749
00750     #if defined(PB_NO_ENCODE_SIZE_CHECK) && PB_NO_ENCODE_SIZE_CHECK == 1
00751     return pb_encode(stream, fields, src_struct);
00752     #else
00753     size = substream.bytes_written;
00754     /* Use a substream to verify that a callback doesn't write more than
00755      * what it did the first time. */
00756     substream.callback = stream->callback;
00757     substream.state = stream->state;
00758     substream.max_size = size;
00759     substream.bytes_written = 0;
00760     #ifndef PB_NO_ERRMSG
00761     substream.errmsg = NULL;
00762     #endif
00763
00764     status = pb_encode(&substream, fields, src_struct);
00765
00766     stream->bytes_written += substream.bytes_written;
00767     stream->state = substream.state;
00768     #ifndef PB_NO_ERRMSG
00769     stream->errmsg = substream.errmsg;
00770     #endif
00771
00772     if (substream.bytes_written != size)
00773         PB_RETURN_ERROR(stream, "submsg size changed");
00774
00775     return status;
00776     #endif
00777 }

```

```

00778
00779 /* Field encoders */
00780
00781 static bool checkreturn pb_enc_bool(pb_ostream_t *stream, const pb_field_iter_t *field)
00782 {
00783     uint32_t value = safe_read_bool(field->pData) ? 1 : 0;
00784     PB_UNUSED(field);
00785     return pb_encode_varint(stream, value);
00786 }
00787
00788 static bool checkreturn pb_enc_varint(pb_ostream_t *stream, const pb_field_iter_t *field)
00789 {
00790     if (PB_LTYPE(field->type) == PB_LTYPE_UVARINT)
00791     {
00792         /* Perform unsigned integer extension */
00793         pb_uint64_t value = 0;
00794
00795         if (field->data_size == sizeof(uint_least8_t))
00796             value = *(const uint_least8_t*)field->pData;
00797         else if (field->data_size == sizeof(uint_least16_t))
00798             value = *(const uint_least16_t*)field->pData;
00799         else if (field->data_size == sizeof(uint32_t))
00800             value = *(const uint32_t*)field->pData;
00801         else if (field->data_size == sizeof(pb_uint64_t))
00802             value = *(const pb_uint64_t*)field->pData;
00803         else
00804             PB_RETURN_ERROR(stream, "invalid data_size");
00805
00806         return pb_encode_varint(stream, value);
00807     }
00808     else
00809     {
00810         /* Perform signed integer extension */
00811         pb_int64_t value = 0;
00812
00813         if (field->data_size == sizeof(int_least8_t))
00814             value = *(const int_least8_t*)field->pData;
00815         else if (field->data_size == sizeof(int_least16_t))
00816             value = *(const int_least16_t*)field->pData;
00817         else if (field->data_size == sizeof(int32_t))
00818             value = *(const int32_t*)field->pData;
00819         else if (field->data_size == sizeof(pb_int64_t))
00820             value = *(const pb_int64_t*)field->pData;
00821         else
00822             PB_RETURN_ERROR(stream, "invalid data_size");
00823
00824         if (PB_LTYPE(field->type) == PB_LTYPE_SVARINT)
00825             return pb_encode_svarint(stream, value);
00826         #ifdef PB_WITHOUT_64BIT
00827         else if (value < 0)
00828             return pb_encode_varint_32(stream, (uint32_t)value, (uint32_t)-1);
00829         #endif
00830         else
00831             return pb_encode_varint(stream, (pb_uint64_t)value);
00832     }
00833 }
00834 }
00835
00836 static bool checkreturn pb_enc_fixed(pb_ostream_t *stream, const pb_field_iter_t *field)
00837 {
00838     #ifdef PB_CONVERT_DOUBLE_FLOAT
00839     if (field->data_size == sizeof(float) && PB_LTYPE(field->type) == PB_LTYPE_FIXED64)
00840     {
00841         return pb_encode_float_as_double(stream, *(float*)field->pData);
00842     }
00843     #endif
00844
00845     if (field->data_size == sizeof(uint32_t))
00846     {
00847         return pb_encode_fixed32(stream, field->pData);
00848     }
00849     #ifdef PB_WITHOUT_64BIT
00850     else if (field->data_size == sizeof(uint64_t))
00851     {
00852         return pb_encode_fixed64(stream, field->pData);
00853     }
00854     #endif
00855     else
00856     {
00857         PB_RETURN_ERROR(stream, "invalid data_size");
00858     }
00859 }
00860
00861 static bool checkreturn pb_enc_bytes(pb_ostream_t *stream, const pb_field_iter_t *field)
00862 {
00863     const pb_bytes_array_t *bytes = NULL;
00864

```

```

00865 bytes = (const pb_bytes_array_t*)field->pData;
00866
00867 if (bytes == NULL)
00868 {
00869     /* Treat null pointer as an empty bytes field */
00870     return pb_encode_string(stream, NULL, 0);
00871 }
00872
00873 if (PB_ATYPE(field->type) == PB_ATYPE_STATIC &&
00874     bytes->size > field->data_size - offsetof(pb_bytes_array_t, bytes))
00875 {
00876     PB_RETURN_ERROR(stream, "bytes size exceeded");
00877 }
00878
00879 return pb_encode_string(stream, bytes->bytes, (size_t)bytes->size);
00880 }
00881
00882 static bool checkreturn pb_enc_string(pb_ostream_t *stream, const pb_field_iter_t *field)
00883 {
00884     size_t size = 0;
00885     size_t max_size = (size_t)field->data_size;
00886     const char *str = (const char*)field->pData;
00887
00888     if (PB_ATYPE(field->type) == PB_ATYPE_POINTER)
00889     {
00890         max_size = (size_t)-1;
00891     }
00892     else
00893     {
00894         /* pb_dec_string() assumes string fields end with a null
00895          * terminator when the type isn't PB_ATYPE_POINTER, so we
00896          * shouldn't allow more than max-1 bytes to be written to
00897          * allow space for the null terminator.
00898          */
00899         if (max_size == 0)
00900             PB_RETURN_ERROR(stream, "zero-length string");
00901
00902         max_size -= 1;
00903     }
00904
00905     if (str == NULL)
00906     {
00907         size = 0; /* Treat null pointer as an empty string */
00908     }
00909     else
00910     {
00911         const char *p = str;
00912
00913         /* strlen() is not always available, so just use a loop */
00914         while (size < max_size && *p != '\0')
00915         {
00916             size++;
00917             p++;
00918         }
00919
00920         if (*p != '\0')
00921         {
00922             PB_RETURN_ERROR(stream, "unterminated string");
00923         }
00924     }
00925 }
00926
00927 #ifdef PB_VALIDATE_UTF8
00928     if (!pb_validate_utf8(str))
00929         PB_RETURN_ERROR(stream, "invalid utf8");
00930 #endif
00931
00932 return pb_encode_string(stream, (const pb_byte_t*)str, size);
00933 }
00934
00935 static bool checkreturn pb_enc_submessage(pb_ostream_t *stream, const pb_field_iter_t *field)
00936 {
00937     if (field->submsg_desc == NULL)
00938         PB_RETURN_ERROR(stream, "invalid field descriptor");
00939
00940     if (PB_LTYPE(field->type) == PB_LTYPE_SUBMSG_W_CB && field->pSize != NULL)
00941     {
00942         /* Message callback is stored right before pSize. */
00943         pb_callback_t *callback = (pb_callback_t*)field->pSize - 1;
00944         if (callback->funcs.encode)
00945         {
00946             if (!callback->funcs.encode(stream, field, &callback->arg))
00947                 return false;
00948         }
00949     }
00950
00951     return pb_encode_submessage(stream, field->submsg_desc, field->pData);

```



```

00952 }
00953
00954 static bool checkreturn pb_enc_fixed_length_bytes(pb_ostream_t *stream, const pb_field_iter_t *field)
00955 {
00956     return pb_encode_string(stream, (const pb_byte_t*)field->pData, (size_t)field->data_size);
00957 }
00958
00959 #ifndef PB_CONVERT_DOUBLE_FLOAT
00960 bool pb_encode_float_as_double(pb_ostream_t *stream, float value)
00961 {
00962     union { float f; uint32_t i; } in;
00963     uint_least8_t sign;
00964     int exponent;
00965     uint64_t mantissa;
00966
00967     in.f = value;
00968
00969     /* Decompose input value */
00970     sign = (uint_least8_t)((in.i » 31) & 1);
00971     exponent = (int)((in.i » 23) & 0xFF) - 127;
00972     mantissa = in.i & 0x7FFFFFFF;
00973
00974     if (exponent == 128)
00975     {
00976         /* Special value (NaN etc.) */
00977         exponent = 1024;
00978     }
00979     else if (exponent == -127)
00980     {
00981         if (!mantissa)
00982         {
00983             /* Zero */
00984             exponent = -1023;
00985         }
00986         else
00987         {
00988             /* Denormalized */
00989             mantissa «= 1;
00990             while (!(mantissa & 0x800000))
00991             {
00992                 mantissa «= 1;
00993                 exponent--;
00994             }
00995             mantissa &= 0x7FFFFFFF;
00996         }
00997     }
00998
00999     /* Combine fields */
01000     mantissa «= 29;
01001     mantissa |= (uint64_t)(exponent + 1023) « 52;
01002     mantissa |= (uint64_t)sign « 63;
01003
01004     return pb_encode_fixed64(stream, &mantissa);
01005 }
01006 #endif

```

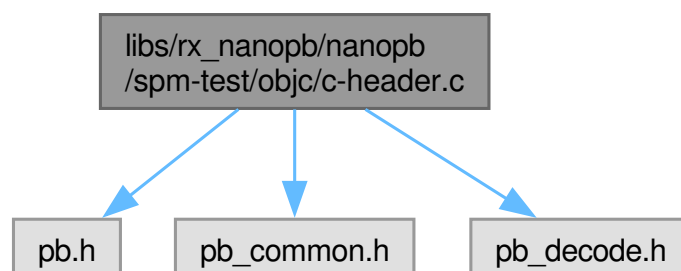
6.65 libs/rx_nanopb/nanopb/spm-test/objc/c-header.c File Reference

```

#include "pb.h"
#include "pb_common.h"
#include "pb_decode.h"

```

Include dependency graph for c-header.c:



6.66 c-header.c

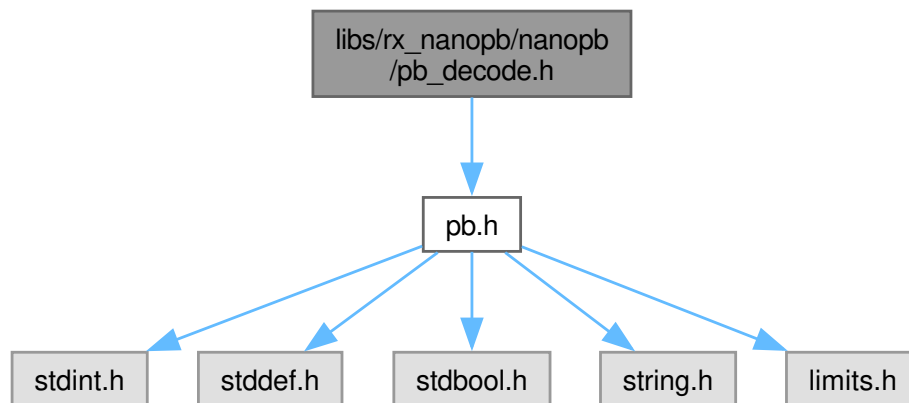
[Go to the documentation of this file.](#)

```
00001 #include "pb.h"
00002 #include "pb_common.h"
00003 #include "pb_decode.h"
```

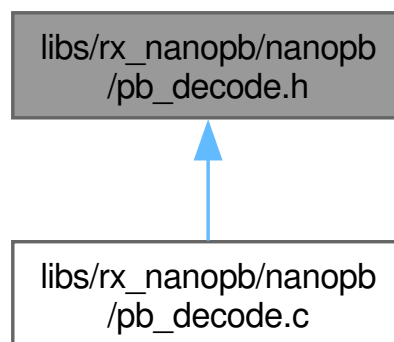
6.67 libs/rx_nanopb/nanopb/pb_decode.h File Reference

```
#include "pb.h"
```

Include dependency graph for pb_decode.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [pb_istream_t](#)

Macros

- `#define` [PB_ISTREAM_EMPTY](#) {0,0,0,0}
- `#define` [PB_DECODE_NOINIT](#) 0x01U
- `#define` [PB_DECODE_DELIMITED](#) 0x02U
- `#define` [PB_DECODE_NULLTERMINATED](#) 0x04U
- `#define` [pb_decode_noinit](#)(s, f, d)
- `#define` [pb_decode_delimited](#)(s, f, d)
- `#define` [pb_decode_delimited_noinit](#)(s, f, d)
- `#define` [pb_decode_nullterminated](#)(s, f, d)

Functions

- [bool pb_decode](#) (pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct)
- [bool pb_decode_ex](#) (pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct, unsigned int flags)
- void [pb_release](#) (const pb_msgdesc_t *fields, void *dest_struct)
- pb_istream_t [pb_istream_from_buffer](#) (const [pb_byte_t](#) *buf, [size_t](#) msglen)
- [bool pb_read](#) (pb_istream_t *stream, [pb_byte_t](#) *buf, [size_t](#) count)
- [bool pb_decode_tag](#) (pb_istream_t *stream, [pb_wire_type_t](#) *wire_type, [uint32_t](#) *tag, [bool](#) *eof)
- [bool pb_skip_field](#) (pb_istream_t *stream, [pb_wire_type_t](#) wire_type)
- [bool pb_decode_varint](#) (pb_istream_t *stream, [uint64_t](#) *dest)
- [bool pb_decode_varint32](#) (pb_istream_t *stream, [uint32_t](#) *dest)
- [bool pb_decode_bool](#) (pb_istream_t *stream, [bool](#) *dest)
- [bool pb_decode_svarint](#) (pb_istream_t *stream, [int64_t](#) *dest)
- [bool pb_decode_fixed32](#) (pb_istream_t *stream, void *dest)
- [bool pb_decode_fixed64](#) (pb_istream_t *stream, void *dest)
- [bool pb_make_string_substream](#) (pb_istream_t *stream, pb_istream_t *substream)
- [bool pb_close_string_substream](#) (pb_istream_t *stream, pb_istream_t *substream)

6.67.1 Macro Definition Documentation

6.67.1.1 PB_DECODE_DELIMITED

```
#define PB_DECODE_DELIMITED 0x02U
```

Definition at line 111 of file [pb_decode.h](#).

Referenced by [pb_decode_ex\(\)](#).

6.67.1.2 pb_decode_delimited

```
#define pb_decode_delimited(  
    s,  
    f,  
    d)
```

Value:

```
pb_decode_ex(s,f,d, PB_DECODE_DELIMITED)
```

Definition at line 117 of file [pb_decode.h](#).

6.67.1.3 pb_decode_delimited_noinit

```
#define pb_decode_delimited_noinit(  
    s,  
    f,  
    d)
```

Value:

```
pb_decode_ex(s,f,d, PB_DECODE_DELIMITED | PB_DECODE_NOINIT)
```

Definition at line 118 of file [pb_decode.h](#).

6.67.1.4 PB_DECODE_NOINIT

```
#define PB_DECODE_NOINIT 0x01U
```

Definition at line 110 of file [pb_decode.h](#).

Referenced by [pb_dec_submessage\(\)](#), and [pb_decode_inner\(\)](#).

6.67.1.5 pb_decode_noinit

```
#define pb_decode_noinit(  
    s,  
    f,  
    d)
```

Value:

```
pb_decode_ex(s,f,d, PB_DECODE_NOINIT)
```

Definition at line 116 of file [pb_decode.h](#).

6.67.1.6 PB_DECODE_NULLTERMINATED

```
#define PB_DECODE_NULLTERMINATED 0x04U
```

Definition at line 112 of file [pb_decode.h](#).

Referenced by [pb_decode_inner\(\)](#).

6.67.1.7 pb_decode_nullterminated

```
#define pb_decode_nullterminated(  
    s,  
    f,  
    d)
```

Value:

```
pb_decode_ex(s,f,d, PB_DECODE_NULLTERMINATED)
```

Definition at line 119 of file [pb_decode.h](#).

6.67.1.8 PB_ISTREAM_EMPTY

```
#define PB_ISTREAM_EMPTY {0,0,0,0}
```

Definition at line 60 of file [pb_decode.h](#).

Referenced by [pb_message_set_to_defaults\(\)](#).

6.67.2 Function Documentation

6.67.2.1 pb_close_string_substream()

```
bool pb_close_string_substream (  
    pb_istream_t * stream,  
    pb_istream_t * substream)
```

Definition at line 398 of file [pb_decode.c](#).

```
00399 {  
00400     if (substream->bytes_left) {  
00401         if (!pb_read(substream, NULL, substream->bytes_left))  
00402             return false;  
00403     }  
00404     stream->state = substream->state;  
00405     stream->errormsg = substream->errormsg;  
00406     #ifndef PB_NO_ERRMSG  
00407     stream->errormsg = substream->errormsg;  
00408     #endif  
00409     return true;  
00410 }  
00411 }
```

6.67.2.2 pb_decode()

```
bool pb_decode (  
    pb_istream_t * stream,  
    const pb_msgdesc_t * fields,  
    void * dest_struct)
```

Definition at line 1226 of file [pb_decode.c](#).

```
01227 {  
01228     return pb_decode_ex(stream, fields, dest_struct, 0);  
01229 }
```

6.67.2.3 pb_decode_bool()

```
bool pb_decode_bool (  
    pb_istream_t * stream,  
    bool * dest)
```

Definition at line 1381 of file [pb_decode.c](#).

```
01382 {  
01383     uint32_t value;  
01384     if (!pb_decode_varint32(stream, &value))  
01385         return false;  
01386     *(bool*)dest = (value != 0);  
01387     return true;  
01388 }  
01389 }
```

6.67.2.4 pb_decode_ex()

```
bool pb_decode_ex (
    pb_istream_t * stream,
    const pb_msgdesc_t * fields,
    void * dest_struct,
    unsigned int flags)
```

Definition at line 1198 of file [pb_decode.c](#).

```
01199 {
01200     bool status;
01201
01202     if ((flags & PB_DECODE_DELIMITED) == 0)
01203     {
01204         status = pb_decode_inner(stream, fields, dest_struct, flags);
01205     }
01206     else
01207     {
01208         pb_istream_t substream;
01209         if (!pb_make_string_substream(stream, &substream))
01210             return false;
01211
01212         status = pb_decode_inner(&substream, fields, dest_struct, flags);
01213
01214         if (!pb_close_string_substream(stream, &substream))
01215             status = false;
01216     }
01217
01218     #ifdef PB_ENABLE_MALLOC
01219     if (!status)
01220         pb_release(fields, dest_struct);
01221     #endif
01222
01223     return status;
01224 }
```

6.67.2.5 pb_decode_fixed32()

```
bool pb_decode_fixed32 (
    pb_istream_t * stream,
    void * dest)
```

Definition at line 1405 of file [pb_decode.c](#).

```
01406 {
01407     union {
01408         uint32_t fixed32;
01409         pb_byte_t bytes[4];
01410     } u;
01411
01412     if (!pb_read(stream, u.bytes, 4))
01413         return false;
01414
01415     #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
01416         /* fast path - if we know that we're on little endian, assign directly */
01417         *(uint32_t*)dest = u.fixed32;
01418     #else
01419         *(uint32_t*)dest = ((uint32_t)u.bytes[0] << 0) |
01420             ((uint32_t)u.bytes[1] << 8) |
01421             ((uint32_t)u.bytes[2] << 16) |
01422             ((uint32_t)u.bytes[3] << 24);
01423     #endif
01424     return true;
01425 }
```

6.67.2.6 pb_decode_fixed64()

```
bool pb_decode_fixed64 (
    pb_istream_t * stream,
    void * dest)
```

Definition at line 1428 of file [pb_decode.c](#).

```

01429 {
01430     union {
01431         uint64_t fixed64;
01432         pb_byte_t bytes[8];
01433     } u;
01434
01435     if (!pb_read(stream, u.bytes, 8))
01436         return false;
01437
01438     #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
01439         /* fast path - if we know that we're on little endian, assign directly */
01440         *(uint64_t*)dest = u.fixed64;
01441     #else
01442         *(uint64_t*)dest = ((uint64_t)u.bytes[0] « 0) |
01443             ((uint64_t)u.bytes[1] « 8) |
01444             ((uint64_t)u.bytes[2] « 16) |
01445             ((uint64_t)u.bytes[3] « 24) |
01446             ((uint64_t)u.bytes[4] « 32) |
01447             ((uint64_t)u.bytes[5] « 40) |
01448             ((uint64_t)u.bytes[6] « 48) |
01449             ((uint64_t)u.bytes[7] « 56);
01450     #endif
01451     return true;
01452 }
```

6.67.2.7 pb_decode_svarint()

```

bool pb_decode_svarint (
    pb_istream_t * stream,
    int64_t * dest)
```

Definition at line 1391 of file [pb_decode.c](#).

```

01392 {
01393     pb_uint64_t value;
01394     if (!pb_decode_varint(stream, &value))
01395         return false;
01396
01397     if (value & 1)
01398         *dest = (pb_int64_t)(~(value » 1));
01399     else
01400         *dest = (pb_int64_t)(value » 1);
01401
01402     return true;
01403 }
```

6.67.2.8 pb_decode_tag()

```

bool pb_decode_tag (
    pb_istream_t * stream,
    pb_wire_type_t * wire_type,
    uint32_t * tag,
    bool * eof)
```

Definition at line 278 of file [pb_decode.c](#).

```

00279 {
00280     uint32_t temp;
00281     *eof = false;
00282     *wire_type = (pb_wire_type_t) 0;
00283     *tag = 0;
00284
00285     if (stream->bytes_left == 0)
00286     {
00287         *eof = true;
00288         return false;
00289     }
00290
00291     if (!pb_decode_varint32(stream, &temp))
00292     {
00293         #ifndef PB_BUFFER_ONLY
00294             /* Workaround for issue #1017
00295              *
```

```

00296     * Callback streams don't set bytes_left to 0 on eof until after being called by pb_decode_varint32,
00297     * which results in "io error" being raised. This contrasts the behavior of buffer streams who raise
00298     * no error on eof as bytes_left is already 0 on entry. This causes legitimate errors (e.g. missing
00299     * required fields) to be incorrectly reported by callback streams.
00300     */
00301     if (stream->callback != buf_read && stream->bytes_left == 0)
00302     {
00303 #ifndef PB_NO_ERRMSG
00304         if (strcmp(stream->errmsg, "io error") == 0)
00305             stream->errmsg = NULL;
00306 #endif
00307         *eof = true;
00308     }
00309 #endif
00310     return false;
00311 }
00312
00313 *tag = temp >> 3;
00314 *wire_type = (pb_wire_type_t)(temp & 7);
00315 return true;
00316 }

```

6.67.2.9 pb_decode_varint()

```

bool pb_decode_varint (
    pb_istream_t * stream,
    uint64_t * dest)

```

Definition at line 230 of file [pb_decode.c](#).

```

00231 {
00232     pb_byte_t byte;
00233     uint_fast8_t bitpos = 0;
00234     uint64_t result = 0;
00235
00236     do
00237     {
00238         if (!pb_readbyte(stream, &byte))
00239             return false;
00240
00241         if (bitpos >= 63 && (byte & 0xFE) != 0)
00242             PB_RETURN_ERROR(stream, "varint overflow");
00243
00244         result |= (uint64_t)(byte & 0x7F) << bitpos;
00245         bitpos = (uint_fast8_t)(bitpos + 7);
00246     } while (byte & 0x80);
00247
00248     *dest = result;
00249     return true;
00250 }

```

6.67.2.10 pb_decode_varint32()

```

bool pb_decode_varint32 (
    pb_istream_t * stream,
    uint32_t * dest)

```

Definition at line 171 of file [pb_decode.c](#).

```

00172 {
00173     pb_byte_t byte;
00174     uint32_t result;
00175
00176     if (!pb_readbyte(stream, &byte))
00177     {
00178         return false;
00179     }
00180
00181     if ((byte & 0x80) == 0)
00182     {
00183         /* Quick case, 1 byte value */
00184         result = byte;
00185     }
00186     else
00187     {

```



```

00188     /* Multibyte case */
00189     uint_fast8_t bitpos = 7;
00190     result = byte & 0x7F;
00191
00192     do
00193     {
00194         if (!pb_readbyte(stream, &byte))
00195             return false;
00196
00197         if (bitpos >= 32)
00198         {
00199             /* Note: The varint could have trailing 0x80 bytes, or 0xFF for negative. */
00200             pb_byte_t sign_extension = (bitpos < 63) ? 0xFF : 0x01;
00201             bool valid_extension = ((byte & 0x7F) == 0x00 ||
00202                 ((result » 31) != 0 && byte == sign_extension));
00203
00204             if (bitpos >= 64 || !valid_extension)
00205             {
00206                 PB_RETURN_ERROR(stream, "varint overflow");
00207             }
00208         }
00209         else if (bitpos == 28)
00210         {
00211             if ((byte & 0x70) != 0 && (byte & 0x78) != 0x78)
00212             {
00213                 PB_RETURN_ERROR(stream, "varint overflow");
00214             }
00215             result |= (uint32_t)(byte & 0x0F) « bitpos;
00216         }
00217         else
00218         {
00219             result |= (uint32_t)(byte & 0x7F) « bitpos;
00220         }
00221         bitpos = (uint_fast8_t)(bitpos + 7);
00222     } while (byte & 0x80);
00223 }
00224
00225 *dest = result;
00226 return true;
00227 }

```

6.67.2.11 pb_istream_from_buffer()

```

pb_istream_t pb_istream_from_buffer (
    const pb_byte_t * buf,
    size_t msglen)

```

Definition at line 142 of file [pb_decode.c](#).

```

00143 {
00144     pb_istream_t stream;
00145     /* Cast away the const from buf without a compiler error. We are
00146      * careful to use it only in a const manner in the callbacks.
00147      */
00148     union {
00149         void *state;
00150         const void *c_state;
00151     } state;
00152     #ifdef PB_BUFFER_ONLY
00153     stream.callback = NULL;
00154     #else
00155     stream.callback = &buf_read;
00156     #endif
00157     state.c_state = buf;
00158     stream.state = state.state;
00159     stream.bytes_left = msglen;
00160     #ifndef PB_NO_ERRMSG
00161     stream.errmsg = NULL;
00162     #endif
00163     return stream;
00164 }

```

6.67.2.12 pb_make_string_substream()

```

bool pb_make_string_substream (
    pb_istream_t * stream,
    pb_istream_t * substream)

```

Definition at line 383 of file [pb_decode.c](#).

```

00384 {
00385     uint32_t size;
00386     if (!pb_decode_varint32(stream, &size))
00387         return false;
00388
00389     *substream = *stream;
00390     if (substream->bytes_left < size)
00391         PB_RETURN_ERROR(stream, "parent stream too short");
00392
00393     substream->bytes_left = (size_t)size;
00394     stream->bytes_left -= (size_t)size;
00395     return true;
00396 }
```

6.67.2.13 pb_read()

```

bool pb_read (
    pb_istream_t * stream,
    pb_byte_t * buf,
    size_t count)
```

Definition at line 81 of file [pb_decode.c](#).

```

00082 {
00083     if (count == 0)
00084         return true;
00085
00086     #ifndef PB_BUFFER_ONLY
00087     if (buf == NULL && stream->callback != buf_read)
00088     {
00089         /* Skip input bytes */
00090         pb_byte_t tmp[16];
00091         while (count > 16)
00092         {
00093             if (!pb_read(stream, tmp, 16))
00094                 return false;
00095
00096             count -= 16;
00097         }
00098
00099         return pb_read(stream, tmp, count);
00100     }
00101     #endif
00102
00103     if (stream->bytes_left < count)
00104         PB_RETURN_ERROR(stream, "end-of-stream");
00105
00106     #ifndef PB_BUFFER_ONLY
00107     if (!stream->callback(stream, buf, count))
00108         PB_RETURN_ERROR(stream, "io error");
00109     #else
00110     if (!buf_read(stream, buf, count))
00111         return false;
00112     #endif
00113
00114     if (stream->bytes_left < count)
00115         stream->bytes_left = 0;
00116     else
00117         stream->bytes_left -= count;
00118
00119     return true;
00120 }
```

6.67.2.14 pb_release()

```

void pb_release (
    const pb_msgdesc_t * fields,
    void * dest_struct)
```

Definition at line 1371 of file [pb_decode.c](#).

```

01372 {
01373     /* Nothing to release without PB_ENABLE_MALLOC. */
01374     PB_UNUSED(fields);
01375     PB_UNUSED(dest_struct);
01376 }
```

6.67.2.15 pb_skip_field()

```
bool pb_skip_field (
    pb_istream_t * stream,
    pb_wire_type_t wire_type)
```

Definition at line 318 of file [pb_decode.c](#).

```
00319 {
00320     switch (wire_type)
00321     {
00322         case PB_WT_VARINT: return pb_skip_varint(stream);
00323         case PB_WT_64BIT: return pb_read(stream, NULL, 8);
00324         case PB_WT_STRING: return pb_skip_string(stream);
00325         case PB_WT_32BIT: return pb_read(stream, NULL, 4);
00326         case PB_WT_PACKED:
00327             /* Calling pb_skip_field with a PB_WT_PACKED is an error.
00328              * Explicitly handle this case and fallthrough to default to avoid
00329              * compiler warnings.
00330              */
00331             default: PB_RETURN_ERROR(stream, "invalid wire_type");
00332     }
00333 }
```

6.68 pb_decode.h

[Go to the documentation of this file.](#)

```
00001 /* pb_decode.h: Functions to decode protocol buffers. Depends on pb_decode.c.
00002  * The main function is pb_decode. You also need an input stream, and the
00003  * field descriptions created by nanopb_generator.py.
00004  */
00005
00006 #ifndef PB_DECODE_H_INCLUDED
00007 #define PB_DECODE_H_INCLUDED
00008
00009 #include "pb.h"
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00015 /* Structure for defining custom input streams. You will need to provide
00016  * a callback function to read the bytes from your storage, which can be
00017  * for example a file or a network socket.
00018  *
00019  * The callback must conform to these rules:
00020  *
00021  * 1) Return false on IO errors. This will cause decoding to abort.
00022  * 2) You can use state to store your own data (e.g. buffer pointer),
00023  *    and rely on pb_read to verify that no-body reads past bytes_left.
00024  * 3) Your callback may be used with substreams, in which case bytes_left
00025  *    is different than from the main stream. Don't use bytes_left to compute
00026  *    any pointers.
00027  */
00028 struct pb_istream_s
00029 {
00030     #ifdef PB_BUFFER_ONLY
00031         /* Callback pointer is not used in buffer-only configuration.
00032          * Having an int pointer here allows binary compatibility but
00033          * gives an error if someone tries to assign callback function.
00034          */
00035         int *callback;
00036     #else
00037         bool (*callback)(pb_istream_t *stream, pb_byte_t *buf, size_t count);
00038     #endif
00039
00040     /* state is a free field for use of the callback function defined above.
00041      * Note that when pb_istream_from_buffer() is used, it reserves this field
00042      * for its own use.
00043      */
00044     void *state;
00045
00046     /* Maximum number of bytes left in this stream. Callback can report
00047      * EOF before this limit is reached. Setting a limit is recommended
00048      * when decoding directly from file or network streams to avoid
00049      * denial-of-service by excessively long messages.
00050      */
00051     size_t bytes_left;
```

```

00052
00053 #ifndef PB_NO_ERRMSG
00054     /* Pointer to constant (ROM) string when decoding function returns error */
00055     const char *errmsg;
00056 #endif
00057 };
00058
00059 #ifndef PB_NO_ERRMSG
00060 #define PB_ISTREAM_EMPTY {0,0,0}
00061 #else
00062 #define PB_ISTREAM_EMPTY {0,0,0}
00063 #endif
00064
00065 /*****
00066  * Main decoding functions *
00067  *****/
00068
00069 /* Decode a single protocol buffers message from input stream into a C structure.
00070  * Returns true on success, false on any failure.
00071  * The actual struct pointed to by dest must match the description in fields.
00072  * Callback fields of the destination structure must be initialized by caller.
00073  * All other fields will be initialized by this function.
00074  *
00075  * Example usage:
00076  *   MyMessage msg = {};
00077  *   uint8_t buffer[64];
00078  *   pb_istream_t stream;
00079  *
00080  *   // ... read some data into buffer ...
00081  *
00082  *   stream = pb_istream_from_buffer(buffer, count);
00083  *   pb_decode(&stream, MyMessage_fields, &msg);
00084  */
00085 bool pb_decode(pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct);
00086
00087 /* Extended version of pb_decode, with several options to control
00088  * the decoding process:
00089  *
00090  * PB_DECODE_NOINIT:      Do not initialize the fields to default values.
00091  *                        This is slightly faster if you do not need the default
00092  *                        values and instead initialize the structure to 0 using
00093  *                        e.g. memset(). This can also be used for merging two
00094  *                        messages, i.e. combine already existing data with new
00095  *                        values.
00096  *
00097  * PB_DECODE_DELIMITED:   Input message starts with the message size as varint.
00098  *                        Corresponds to parseDelimitedFrom() in Google's
00099  *                        protobuf API.
00100  *
00101  * PB_DECODE_NULLTERMINATED: Stop reading when field tag is read as 0. This allows
00102  *                        reading null terminated messages.
00103  *                        NOTE: Until nanopb-0.4.0, pb_decode() also allows
00104  *                        null-termination. This behaviour is not supported in
00105  *                        most other protobuf implementations, so PB_DECODE_DELIMITED
00106  *                        is a better option for compatibility.
00107  *
00108  * Multiple flags can be combined with bitwise or (| operator)
00109  */
00110 #define PB_DECODE_NOINIT      0x01U
00111 #define PB_DECODE_DELIMITED  0x02U
00112 #define PB_DECODE_NULLTERMINATED 0x04U
00113 bool pb_decode_ex(pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct, unsigned int flags);
00114
00115 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00116 #define pb_decode_noinit(s,f,d) pb_decode_ex(s,f,d, PB_DECODE_NOINIT)
00117 #define pb_decode_delimited(s,f,d) pb_decode_ex(s,f,d, PB_DECODE_DELIMITED)
00118 #define pb_decode_delimited_noinit(s,f,d) pb_decode_ex(s,f,d, PB_DECODE_DELIMITED | PB_DECODE_NOINIT)
00119 #define pb_decode_nullterminated(s,f,d) pb_decode_ex(s,f,d, PB_DECODE_NULLTERMINATED)
00120
00121 /* Release any allocated pointer fields. If you use dynamic allocation, you should
00122  * call this for any successfully decoded message when you are done with it. If
00123  * pb_decode() returns with an error, the message is already released.
00124  */
00125 void pb_release(const pb_msgdesc_t *fields, void *dest_struct);
00126
00127 /*****
00128  * Functions for manipulating streams *
00129  *****/
00130
00131 /* Create an input stream for reading from a memory buffer.
00132  *
00133  * msglen should be the actual length of the message, not the full size of
00134  * allocated buffer.
00135  *
00136  * Alternatively, you can use a custom stream that reads directly from e.g.
00137  * a file or a network socket.
00138  */

```

```

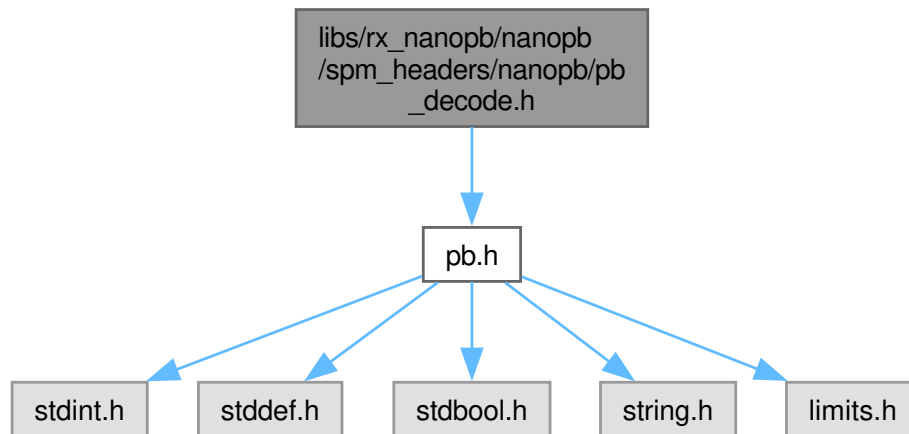
00139 pb_istream_t pb_istream_from_buffer(const pb_byte_t *buf, size_t msglen);
00140
00141 /* Function to read from a pb_istream_t. You can use this if you need to
00142  * read some custom header data, or to read data in field callbacks.
00143  */
00144 bool pb_read(pb_istream_t *stream, pb_byte_t *buf, size_t count);
00145
00146
00147 /*****
00148  * Helper functions for writing field callbacks *
00149  *****/
00150
00151 /* Decode the tag for the next field in the stream. Gives the wire type and
00152  * field tag. At end of the message, returns false and sets eof to true. */
00153 bool pb_decode_tag(pb_istream_t *stream, pb_wire_type_t *wire_type, uint32_t *tag, bool *eof);
00154
00155 /* Skip the field payload data, given the wire type. */
00156 bool pb_skip_field(pb_istream_t *stream, pb_wire_type_t wire_type);
00157
00158 /* Decode an integer in the varint format. This works for enum, int32,
00159  * int64, uint32 and uint64 field types. */
00160 #ifndef PB_WITHOUT_64BIT
00161 bool pb_decode_varint(pb_istream_t *stream, uint64_t *dest);
00162 #else
00163 #define pb_decode_varint pb_decode_varint32
00164 #endif
00165
00166 /* Decode an integer in the varint format. This works for enum, int32,
00167  * and uint32 field types. */
00168 bool pb_decode_varint32(pb_istream_t *stream, uint32_t *dest);
00169
00170 /* Decode a bool value in varint format. */
00171 bool pb_decode_bool(pb_istream_t *stream, bool *dest);
00172
00173 /* Decode an integer in the zig-zagged svarint format. This works for sint32
00174  * and sint64. */
00175 #ifndef PB_WITHOUT_64BIT
00176 bool pb_decode_svarint(pb_istream_t *stream, int64_t *dest);
00177 #else
00178 bool pb_decode_svarint(pb_istream_t *stream, int32_t *dest);
00179 #endif
00180
00181 /* Decode a fixed32, sfixed32 or float value. You need to pass a pointer to
00182  * a 4-byte wide C variable. */
00183 bool pb_decode_fixed32(pb_istream_t *stream, void *dest);
00184
00185 #ifndef PB_WITHOUT_64BIT
00186 /* Decode a fixed64, sfixed64 or double value. You need to pass a pointer to
00187  * a 8-byte wide C variable. */
00188 bool pb_decode_fixed64(pb_istream_t *stream, void *dest);
00189 #endif
00190
00191 #ifdef PB_CONVERT_DOUBLE_FLOAT
00192 /* Decode a double value into float variable. */
00193 bool pb_decode_double_as_float(pb_istream_t *stream, float *dest);
00194 #endif
00195
00196 /* Make a limited-length substream for reading a PB_WT_STRING field. */
00197 bool pb_make_string_substream(pb_istream_t *stream, pb_istream_t *substream);
00198 bool pb_close_string_substream(pb_istream_t *stream, pb_istream_t *substream);
00199
00200 #ifdef __cplusplus
00201 } /* extern "C" */
00202 #endif
00203
00204 #endif

```

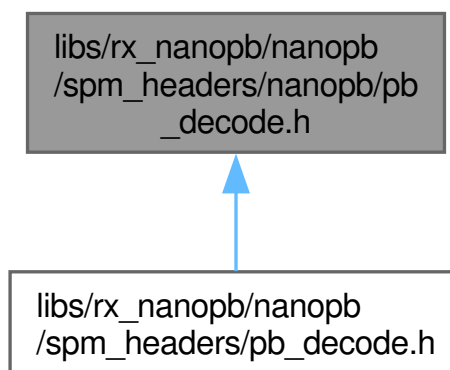
6.69 libs/rx_nanopb/nanopb/spm_headers/nanopb/pb_decode.h File Reference

#include "pb.h"

Include dependency graph for pb_decode.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [pb_istream_t](#)

Macros

- `#define` [PB_ISTREAM_EMPTY](#) {0,0,0,0}
- `#define` [PB_DECODE_NOINIT](#) 0x01U
- `#define` [PB_DECODE_DELIMITED](#) 0x02U
- `#define` [PB_DECODE_NULLTERMINATED](#) 0x04U
- `#define` [pb_decode_noinit](#)(s, f, d)
- `#define` [pb_decode_delimited](#)(s, f, d)
- `#define` [pb_decode_delimited_noinit](#)(s, f, d)
- `#define` [pb_decode_nullterminated](#)(s, f, d)

Functions

- [bool pb_decode](#) (pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct)
- [bool pb_decode_ex](#) (pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct, unsigned int flags)
- void [pb_release](#) (const pb_msgdesc_t *fields, void *dest_struct)
- pb_istream_t [pb_istream_from_buffer](#) (const [pb_byte_t](#) *buf, [size_t](#) msglen)

- [bool pb_read](#) (pb_istream_t *stream, [pb_byte_t](#) *buf, [size_t](#) count)
- [bool pb_decode_tag](#) (pb_istream_t *stream, [pb_wire_type_t](#) *wire_type, [uint32_t](#) *tag, [bool](#) *eof)
- [bool pb_skip_field](#) (pb_istream_t *stream, [pb_wire_type_t](#) wire_type)
- [bool pb_decode_varint](#) (pb_istream_t *stream, [uint64_t](#) *dest)
- [bool pb_decode_varint32](#) (pb_istream_t *stream, [uint32_t](#) *dest)
- [bool pb_decode_bool](#) (pb_istream_t *stream, [bool](#) *dest)
- [bool pb_decode_svarint](#) (pb_istream_t *stream, [int64_t](#) *dest)
- [bool pb_decode_fixed32](#) (pb_istream_t *stream, void *dest)
- [bool pb_decode_fixed64](#) (pb_istream_t *stream, void *dest)
- [bool pb_make_string_substream](#) (pb_istream_t *stream, pb_istream_t *substream)
- [bool pb_close_string_substream](#) (pb_istream_t *stream, pb_istream_t *substream)

6.69.1 Macro Definition Documentation

6.69.1.1 PB_DECODE_DELIMITED

```
#define PB_DECODE_DELIMITED 0x02U
```

Definition at line 111 of file [pb_decode.h](#).

6.69.1.2 pb_decode_delimited

```
#define pb_decode_delimited(  
    s,  
    f,  
    d)
```

Value:

```
pb_decode_ex(s,f,d, PB_DECODE_DELIMITED)
```

Definition at line 117 of file [pb_decode.h](#).

6.69.1.3 pb_decode_delimited_noinit

```
#define pb_decode_delimited_noinit(  
    s,  
    f,  
    d)
```

Value:

```
pb_decode_ex(s,f,d, PB_DECODE_DELIMITED | PB_DECODE_NOINIT)
```

Definition at line 118 of file [pb_decode.h](#).

6.69.1.4 PB_DECODE_NOINIT

```
#define PB_DECODE_NOINIT 0x01U
```

Definition at line 110 of file [pb_decode.h](#).

6.69.1.5 pb_decode_noinit

```
#define pb_decode_noinit(
    s,
    f,
    d)
```

Value:

pb_decode_ex(s,f,d, PB_DECODE_NOINIT)

Definition at line 116 of file [pb_decode.h](#).

6.69.1.6 PB_DECODE_NULLTERMINATED

```
#define PB_DECODE_NULLTERMINATED 0x04U
```

Definition at line 112 of file [pb_decode.h](#).

6.69.1.7 pb_decode_nullterminated

```
#define pb_decode_nullterminated(
    s,
    f,
    d)
```

Value:

pb_decode_ex(s,f,d, PB_DECODE_NULLTERMINATED)

Definition at line 119 of file [pb_decode.h](#).

6.69.1.8 PB_ISTREAM_EMPTY

```
#define PB_ISTREAM_EMPTY {0,0,0,0}
```

Definition at line 60 of file [pb_decode.h](#).

6.69.2 Function Documentation

6.69.2.1 pb_close_string_substream()

```
bool pb_close_string_substream (
    pb_istream_t * stream,
    pb_istream_t * substream)
```

Definition at line 398 of file [pb_decode.c](#).

```
00399 {
00400     if (substream->bytes_left) {
00401         if (!pb_read(substream, NULL, substream->bytes_left))
00402             return false;
00403     }
00404
00405     stream->state = substream->state;
00406
00407     #ifndef PB_NO_ERRMSG
```



```

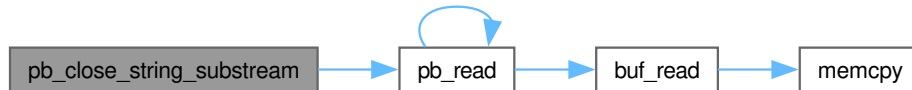
00408     stream->errmsg = substream->errmsg;
00409 #endif
00410     return true;
00411 }

```

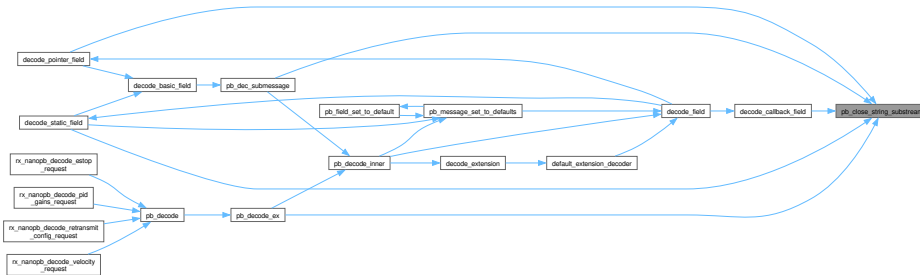
References [checkreturn](#), [NULL](#), and [pb_read\(\)](#).

Referenced by [decode_callback_field\(\)](#), [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [pb_dec_submessage\(\)](#), and [pb_decode_ex\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.69.2.2 pb_decode()

```

bool pb_decode (
    pb_istream_t * stream,
    const pb_msgdesc_t * fields,
    void * dest_struct)

```

Definition at line 1226 of file [pb_decode.c](#).

```

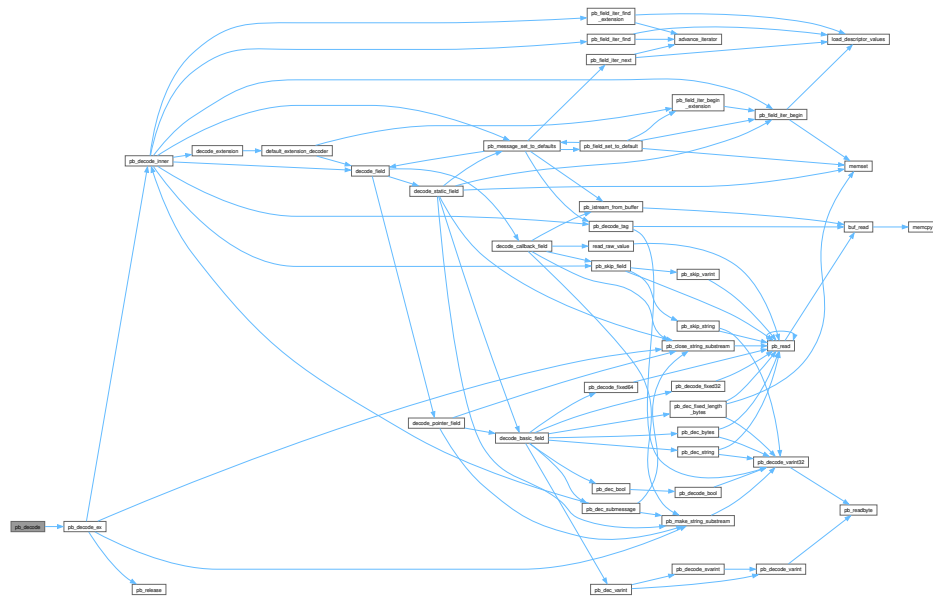
01227 {
01228     return pb_decode_ex(stream, fields, dest_struct, 0);
01229 }

```

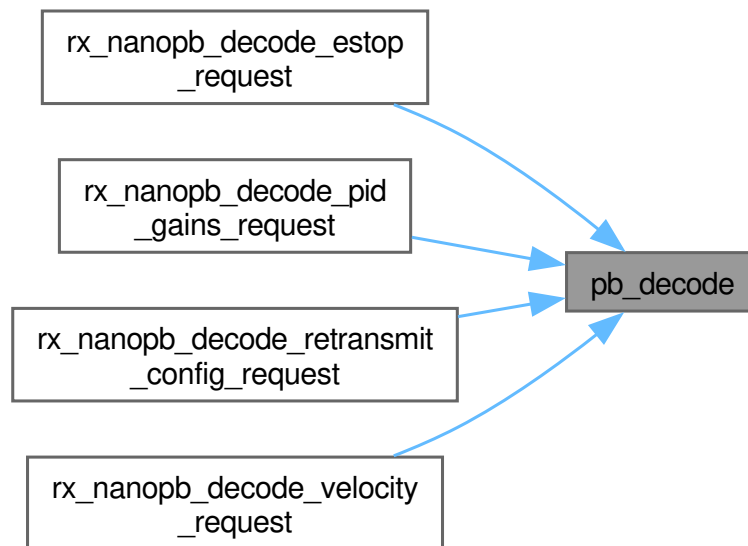
References [checkreturn](#), and [pb_decode_ex\(\)](#).

Referenced by [rx_nanopb_decode_estop_request\(\)](#), [rx_nanopb_decode_pid_gains_request\(\)](#), [rx_nanopb_decode_retransmit_config_request\(\)](#), and [rx_nanopb_decode_velocity_request\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.69.2.3 pb_decode_bool()

```

bool pb_decode_bool (
    pb_istream_t * stream,
    bool * dest)

```

Definition at line 1381 of file [pb_decode.c](#).

```

01382 {
01383     uint32_t value;
01384     if (!pb_decode_varint32(stream, &value))
01385         return false;
01386     *(bool*)dest = (value != 0);
01387     return true;
01388 }
01389

```

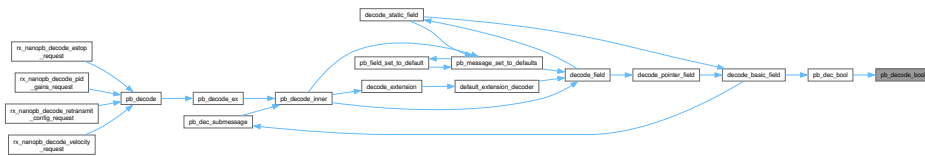
References [pb_decode_varint32\(\)](#).

Referenced by [pb_dec_bool\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.69.2.4 pb_decode_ex()

```

bool pb_decode_ex (
    pb_istream_t * stream,
    const pb_msgdesc_t * fields,
    void * dest_struct,
    unsigned int flags)
  
```

Definition at line 1198 of file [pb_decode.c](#).

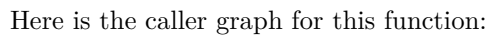
```

01199 {
01200     bool status;
01201
01202     if ((flags & PB_DECODE_DELIMITED) == 0)
01203     {
01204         status = pb_decode_inner(stream, fields, dest_struct, flags);
01205     }
01206     else
01207     {
01208         pb_istream_t substream;
01209         if (!pb_make_string_substream(stream, &substream))
01210             return false;
01211
01212         status = pb_decode_inner(&substream, fields, dest_struct, flags);
01213
01214         if (!pb_close_string_substream(stream, &substream))
01215             status = false;
01216     }
01217
01218     #ifdef PB_ENABLE_MALLOC
01219     if (!status)
01220         pb_release(fields, dest_struct);
01221     #endif
01222
01223     return status;
01224 }
  
```

References [checkreturn](#), [pb_close_string_substream\(\)](#), [PB_DECODE_DELIMITED](#), [pb_decode_inner\(\)](#), [pb_make_string_substream\(\)](#), and [pb_release\(\)](#).

Referenced by [pb_decode\(\)](#).

Here is the call graph for this function:



Definition at line 1405 of file pb_decode.c.

Generated by Doxygen

```

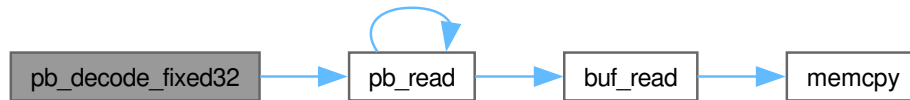
01419  *(uint32_t*)dest = ((uint32_t)u.bytes[0] « 0) |
01420                      ((uint32_t)u.bytes[1] « 8) |
01421                      ((uint32_t)u.bytes[2] « 16) |
01422                      ((uint32_t)u.bytes[3] « 24);
01423  #endif
01424  return true;
01425  }

```

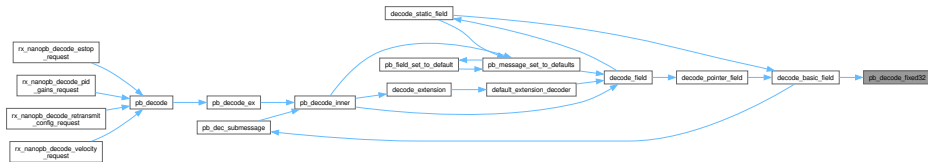
References [pb_read\(\)](#).

Referenced by [decode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.69.2.6 pb_decode_fixed64()

```

bool pb_decode_fixed64 (
    pb_istream_t * stream,
    void * dest)

```

Definition at line 1428 of file [pb_decode.c](#).

```

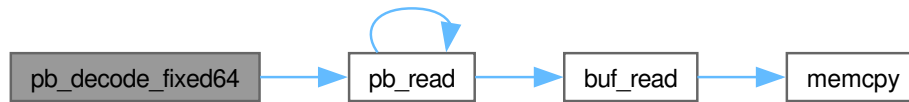
01429 {
01430     union {
01431         uint64_t fixed64;
01432         pb_byte_t bytes[8];
01433     } u;
01434     if (!pb_read(stream, u.bytes, 8))
01435         return false;
01437     #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
01438     /* fast path - if we know that we're on little endian, assign directly */
01439     *(uint64_t*)dest = u.fixed64;
01440     #else
01441     *(uint64_t*)dest = ((uint64_t)u.bytes[0] « 0) |
01442                       ((uint64_t)u.bytes[1] « 8) |
01443                       ((uint64_t)u.bytes[2] « 16) |
01444                       ((uint64_t)u.bytes[3] « 24) |
01445                       ((uint64_t)u.bytes[4] « 32) |
01446                       ((uint64_t)u.bytes[5] « 40) |
01447                       ((uint64_t)u.bytes[6] « 48) |
01448                       ((uint64_t)u.bytes[7] « 56);
01449     #endif
01450     return true;
01451 }
01452 }

```

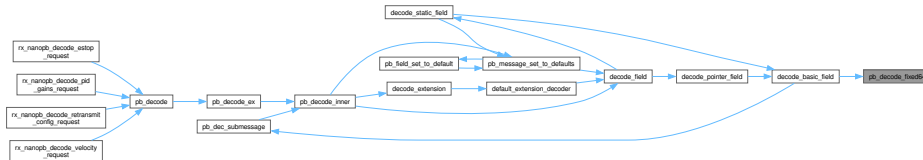
References [pb_read\(\)](#).

Referenced by [decode_basic_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.69.2.7 pb_decode_svarint()

```

bool pb_decode_svarint (
    pb_istream_t * stream,
    int64_t * dest)
  
```

Definition at line 1391 of file `pb_decode.c`.

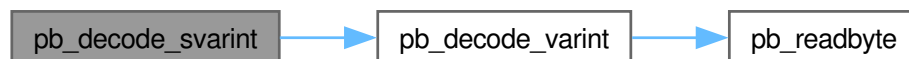
```

01392 {
01393     pb_uint64_t value;
01394     if (!pb_decode_varint(stream, &value))
01395         return false;
01396     if (value & 1)
01397         *dest = (pb_int64_t)(~(value » 1));
01398     else
01399         *dest = (pb_int64_t)(value » 1);
01400     return true;
01401 }
01402
01403 }
  
```

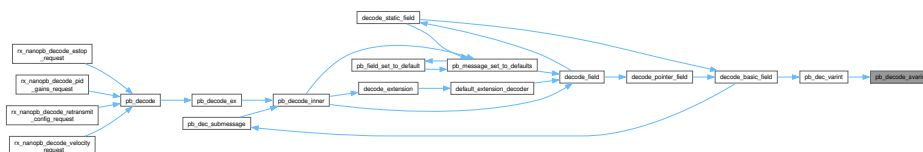
References `pb_decode_varint()`, `pb_int64_t`, and `pb_uint64_t`.

Referenced by `pb_dec_varint()`.

Here is the call graph for this function:



Here is the caller graph for this function:



6.69.2.8 pb_decode_tag()

```
bool pb_decode_tag (
    pb_istream_t * stream,
    pb_wire_type_t * wire_type,
    uint32_t * tag,
    bool * eof)
```

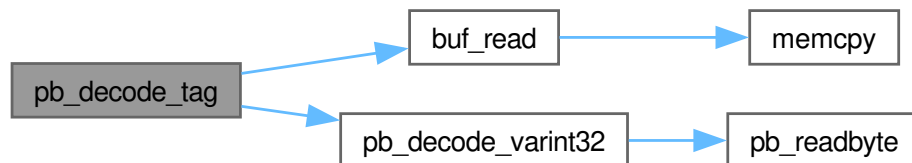
Definition at line 278 of file [pb_decode.c](#).

```
00279 {
00280     uint32_t temp;
00281     *eof = false;
00282     *wire_type = (pb_wire_type_t) 0;
00283     *tag = 0;
00284
00285     if (stream->bytes_left == 0)
00286     {
00287         *eof = true;
00288         return false;
00289     }
00290
00291     if (!pb_decode_varint32(stream, &temp))
00292     {
00293         #ifndef PB_BUFFER_ONLY
00294             /* Workaround for issue #1017
00295              *
00296              * Callback streams don't set bytes_left to 0 on eof until after being called by pb_decode_varint32,
00297              * which results in "io error" being raised. This contrasts the behavior of buffer streams who raise
00298              * no error on eof as bytes_left is already 0 on entry. This causes legitimate errors (e.g. missing
00299              * required fields) to be incorrectly reported by callback streams.
00300              */
00301             if (stream->callback != buf_read && stream->bytes_left == 0)
00302             {
00303                 #ifndef PB_NO_ERRMSG
00304                     if (strcmp(stream->errmsg, "io error") == 0)
00305                         stream->errmsg = NULL;
00306                 #endif
00307                 *eof = true;
00308             }
00309             #endif
00310             return false;
00311         }
00312
00313         *tag = temp >> 3;
00314         *wire_type = (pb_wire_type_t)(temp & 7);
00315         return true;
00316     }
```

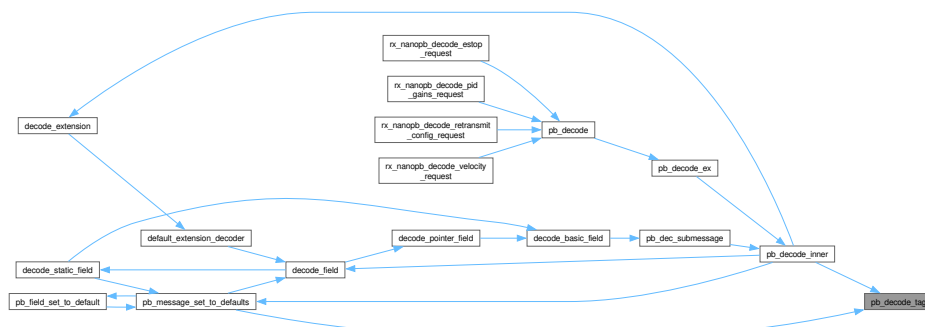
References [buf_read\(\)](#), [checkreturn](#), [NULL](#), and [pb_decode_varint32\(\)](#).

Referenced by [pb_decode_inner\(\)](#), and [pb_message_set_to_defaults\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.69.2.9 pb_decode_varint()

```
bool pb_decode_varint (
    pb_istream_t * stream,
    uint64_t * dest)
```

Definition at line 230 of file [pb_decode.c](#).

```
00231 {
00232     pb_byte_t byte;
00233     uint_fast8_t bitpos = 0;
00234     uint64_t result = 0;
00235
00236     do
00237     {
00238         if (!pb_readbyte(stream, &byte))
00239             return false;
00240
00241         if (bitpos >= 63 && (byte & 0xFE) != 0)
00242             PB_RETURN_ERROR(stream, "varint overflow");
00243
00244         result |= (uint64_t)(byte & 0x7F) << bitpos;
00245         bitpos = (uint_fast8_t)(bitpos + 7);
00246     } while (byte & 0x80);
00247
00248     *dest = result;
00249     return true;
00250 }
```

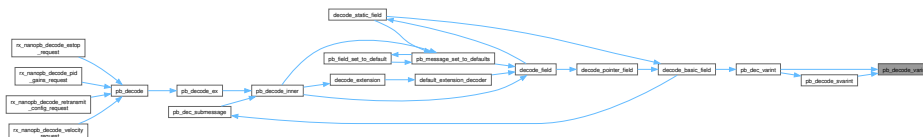
References [checkreturn](#), [pb_readbyte\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [pb_dec_varint\(\)](#), and [pb_decode_svarint\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.69.2.10 pb_decode_varint32()

```
bool pb_decode_varint32 (
    pb_istream_t * stream,
    uint32_t * dest)
```

Definition at line 171 of file [pb_decode.c](#).

```
00172 {
00173     pb_byte_t byte;
00174     uint32_t result;
00175
00176     if (!pb_readbyte(stream, &byte))
00177     {
00178         return false;
00179     }
00180
00181     if ((byte & 0x80) == 0)
00182     {
```


6.69.2.11 pb_istream_from_buffer()

```
pb_istream_t pb_istream_from_buffer (
    const pb_byte_t * buf,
    size_t msglen)
```

Definition at line 142 of file [pb_decode.c](#).

```
00143 {
00144     pb_istream_t stream;
00145     /* Cast away the const from buf without a compiler error. We are
00146      * careful to use it only in a const manner in the callbacks.
00147      */
00148     union {
00149         void *state;
00150         const void *c_state;
00151     } state;
00152     #ifdef PB_BUFFER_ONLY
00153     stream.callback = NULL;
00154     #else
00155     stream.callback = &buf_read;
00156     #endif
00157     state.c_state = buf;
00158     stream.state = state.state;
00159     stream.bytes_left = msglen;
00160     #ifdef PB_NO_ERRMSG
00161     stream.errmsg = NULL;
00162     #endif
00163     return stream;
00164 }
```

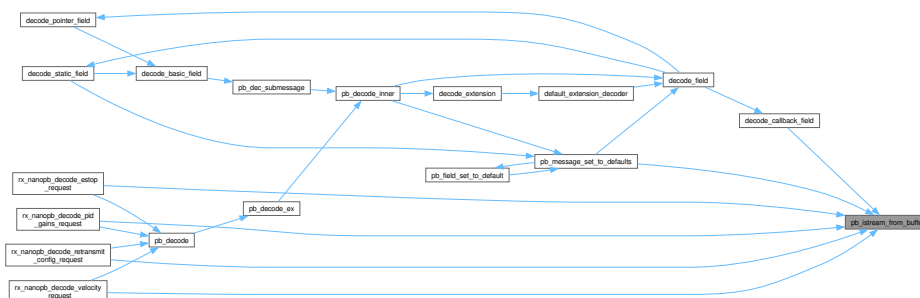
References [buf_read\(\)](#), and [NULL](#).

Referenced by [decode_callback_field\(\)](#), [pb_message_set_to_defaults\(\)](#), [rx_nanopb_decode_estop_request\(\)](#), [rx_nanopb_decode_pid_gains_request\(\)](#), [rx_nanopb_decode_retransmit_config_request\(\)](#), and [rx_nanopb_decode_velocity_request\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.69.2.12 pb_make_string_substream()

```
bool pb_make_string_substream (
    pb_istream_t * stream,
    pb_istream_t * substream)
```

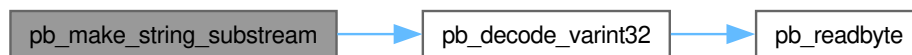
Definition at line 383 of file [pb_decode.c](#).

```
00384 {
00385     uint32_t size;
00386     if (!pb_decode_varint32(stream, &size))
00387         return false;
00388
00389     *substream = *stream;
00390     if (substream->bytes_left < size)
00391         PB_RETURN_ERROR(stream, "parent stream too short");
00392
00393     substream->bytes_left = (size_t)size;
00394     stream->bytes_left -= (size_t)size;
00395     return true;
00396 }
```

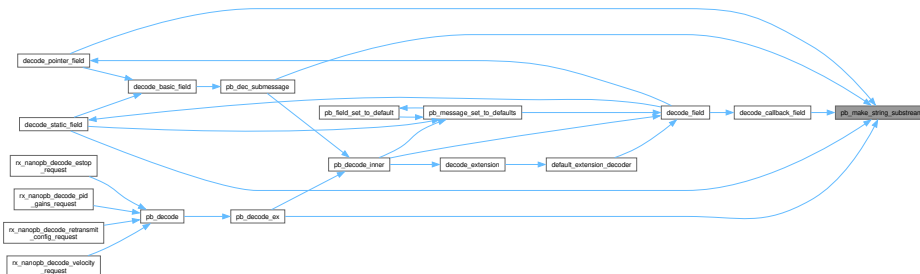
References [checkreturn](#), [pb_decode_varint32\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [decode_callback_field\(\)](#), [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [pb_dec_submessage\(\)](#), and [pb_decode_ex\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.69.2.13 pb_read()

```
bool pb_read (
    pb_istream_t * stream,
    pb_byte_t * buf,
    size_t count)
```

Definition at line 81 of file [pb_decode.c](#).

```
00082 {
00083     if (count == 0)
00084         return true;
00085
00086     #ifndef PB_BUFFER_ONLY
00087     if (buf == NULL && stream->callback != buf_read)
00088     {
00089         /* Skip input bytes */
00090         pb_byte_t tmp[16];
```

```

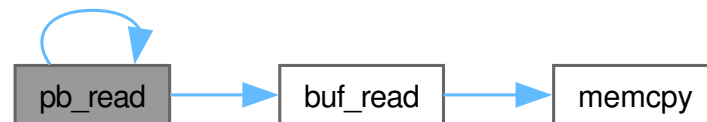
00091     while (count > 16)
00092     {
00093         if (!pb_read(stream, tmp, 16))
00094             return false;
00095
00096         count -= 16;
00097     }
00098
00099     return pb_read(stream, tmp, count);
00100 }
00101 #endif
00102
00103 if (stream->bytes_left < count)
00104     PB_RETURN_ERROR(stream, "end-of-stream");
00105
00106 #ifndef PB_BUFFER_ONLY
00107 if (!stream->callback(stream, buf, count))
00108     PB_RETURN_ERROR(stream, "io error");
00109 #else
00110 if (!buf_read(stream, buf, count))
00111     return false;
00112 #endif
00113
00114 if (stream->bytes_left < count)
00115     stream->bytes_left = 0;
00116 else
00117     stream->bytes_left -= count;
00118
00119 return true;
00120 }

```

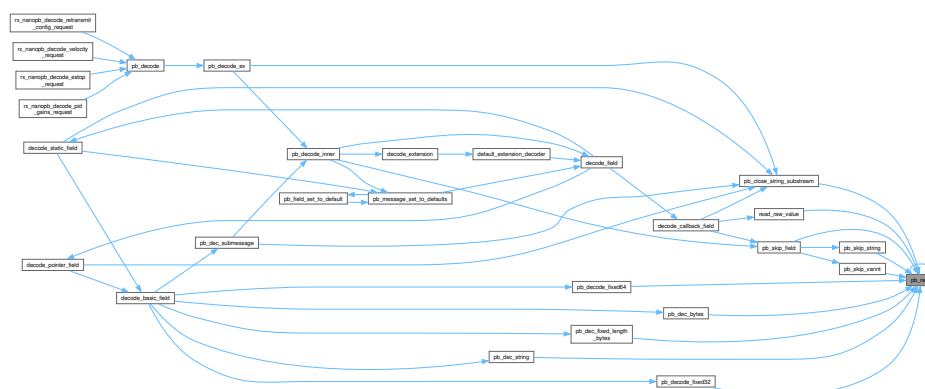
References [buf_read\(\)](#), [checkreturn](#), [NULL](#), [pb_read\(\)](#), and [PB_RETURN_ERROR](#).

Referenced by [pb_close_string_substream\(\)](#), [pb_dec_bytes\(\)](#), [pb_dec_fixed_length_bytes\(\)](#), [pb_dec_string\(\)](#), [pb_decode_fixed32\(\)](#), [pb_decode_fixed64\(\)](#), [pb_read\(\)](#), [pb_skip_field\(\)](#), [pb_skip_string\(\)](#), [pb_skip_varint\(\)](#), and [read_raw_value\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.69.2.14 pb_release()

```
void pb_release (
    const pb_msgdesc_t * fields,
    void * dest_struct)
```

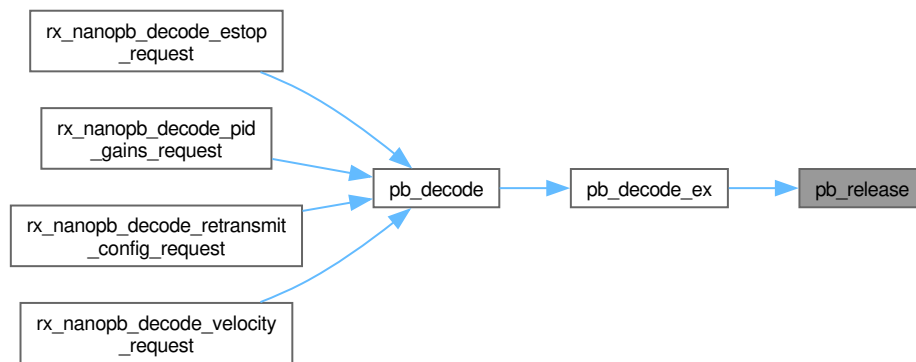
Definition at line 1371 of file [pb_decode.c](#).

```
01372 {
01373     /* Nothing to release without PB_ENABLE_MALLOC. */
01374     PB_UNUSED(fields);
01375     PB_UNUSED(dest_struct);
01376 }
```

References [PB_UNUSED](#).

Referenced by [pb_decode_ex\(\)](#).

Here is the caller graph for this function:



6.69.2.15 pb_skip_field()

```
bool pb_skip_field (
    pb_istream_t * stream,
    pb_wire_type_t wire_type)
```

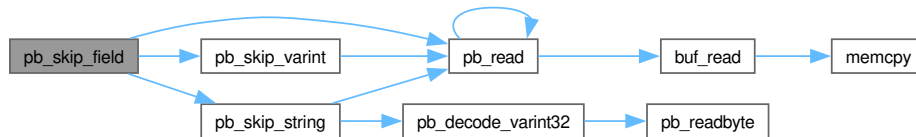
Definition at line 318 of file [pb_decode.c](#).

```
00319 {
00320     switch (wire_type)
00321     {
00322         case PB_WT_VARINT: return pb_skip_varint(stream);
00323         case PB_WT_64BIT: return pb_read(stream, NULL, 8);
00324         case PB_WT_STRING: return pb_skip_string(stream);
00325         case PB_WT_32BIT: return pb_read(stream, NULL, 4);
00326         case PB_WT_PACKED:
00327             /* Calling pb_skip_field with a PB_WT_PACKED is an error.
00328              * Explicitly handle this case and fallthrough to default to avoid
00329              * compiler warnings.
00330              */
00331             default: PB_RETURN_ERROR(stream, "invalid wire_type");
00332     }
00333 }
```

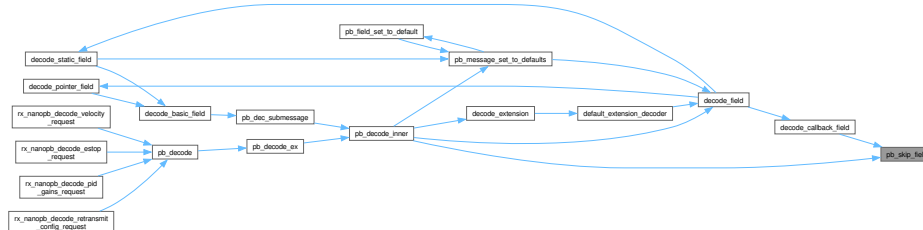
References [checkreturn](#), [NULL](#), [pb_read\(\)](#), [PB_RETURN_ERROR](#), [pb_skip_string\(\)](#), [pb_skip_varint\(\)](#), [PB_WT_32BIT](#), [PB_WT_64BIT](#), [PB_WT_PACKED](#), [PB_WT_STRING](#), and [PB_WT_VARINT](#).

Referenced by [decode_callback_field\(\)](#), and [pb_decode_inner\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.70 pb_decode.h

[Go to the documentation of this file.](#)

```

00001 /* pb_decode.h: Functions to decode protocol buffers. Depends on pb_decode.c.
00002  * The main function is pb_decode. You also need an input stream, and the
00003  * field descriptions created by nanopb_generator.py.
00004  */
00005
00006 #ifndef PB_DECODE_H_INCLUDED
00007 #define PB_DECODE_H_INCLUDED
00008
00009 #include "pb.h"
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00015 /* Structure for defining custom input streams. You will need to provide
00016  * a callback function to read the bytes from your storage, which can be
00017  * for example a file or a network socket.
00018  *
00019  * The callback must conform to these rules:
00020  *
00021  * 1) Return false on IO errors. This will cause decoding to abort.
00022  * 2) You can use state to store your own data (e.g. buffer pointer),
00023  *    and rely on pb_read to verify that no-body reads past bytes_left.
00024  * 3) Your callback may be used with substreams, in which case bytes_left
00025  *    is different than from the main stream. Don't use bytes_left to compute
00026  *    any pointers.
00027  */
00028 struct pb_istream_s
00029 {
00030 #ifdef PB_BUFFER_ONLY
00031     /* Callback pointer is not used in buffer-only configuration.
00032      * Having an int pointer here allows binary compatibility but
00033      * gives an error if someone tries to assign callback function.
00034      */
00035     int *callback;
00036 #else
00037     bool (*callback)(pb_istream_t *stream, pb_byte_t *buf, size_t count);
00038 #endif
00039
00040     /* state is a free field for use of the callback function defined above.
00041      * Note that when pb_istream_from_buffer() is used, it reserves this field
00042      * for its own use.
00043      */
00044     void *state;
00045
00046     /* Maximum number of bytes left in this stream. Callback can report
00047      * EOF before this limit is reached. Setting a limit is recommended
00048      * when decoding directly from file or network streams to avoid
00049      * denial-of-service by excessively long messages.
00050      */
00051     size_t bytes_left;
00052

```

```

00053 #ifndef PB_NO_ERRMSG
00054     /* Pointer to constant (ROM) string when decoding function returns error */
00055     const char *errmsg;
00056 #endif
00057 };
00058
00059 #ifndef PB_NO_ERRMSG
00060 #define PB_ISTREAM_EMPTY {0,0,0}
00061 #else
00062 #define PB_ISTREAM_EMPTY {0,0,0}
00063 #endif
00064
00065 /*****
00066  * Main decoding functions *
00067  *****/
00068
00069 /* Decode a single protocol buffers message from input stream into a C structure.
00070  * Returns true on success, false on any failure.
00071  * The actual struct pointed to by dest must match the description in fields.
00072  * Callback fields of the destination structure must be initialized by caller.
00073  * All other fields will be initialized by this function.
00074  *
00075  * Example usage:
00076  *   MyMessage msg = {};
00077  *   uint8_t buffer[64];
00078  *   pb_istream_t stream;
00079  *
00080  *   // ... read some data into buffer ...
00081  *
00082  *   stream = pb_istream_from_buffer(buffer, count);
00083  *   pb_decode(&stream, MyMessage_fields, &msg);
00084  */
00085 bool pb_decode(pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct);
00086
00087 /* Extended version of pb_decode, with several options to control
00088  * the decoding process:
00089  *
00090  * PB_DECODE_NOINIT:      Do not initialize the fields to default values.
00091  *                        This is slightly faster if you do not need the default
00092  *                        values and instead initialize the structure to 0 using
00093  *                        e.g. memset(). This can also be used for merging two
00094  *                        messages, i.e. combine already existing data with new
00095  *                        values.
00096  *
00097  * PB_DECODE_DELIMITED:   Input message starts with the message size as varint.
00098  *                        Corresponds to parseDelimitedFrom() in Google's
00099  *                        protobuf API.
00100  *
00101  * PB_DECODE_NULLTERMINATED: Stop reading when field tag is read as 0. This allows
00102  *                        reading null terminated messages.
00103  *                        NOTE: Until nanopb-0.4.0, pb_decode() also allows
00104  *                        null-termination. This behaviour is not supported in
00105  *                        most other protobuf implementations, so PB_DECODE_DELIMITED
00106  *                        is a better option for compatibility.
00107  *
00108  * Multiple flags can be combined with bitwise or (| operator)
00109  */
00110 #define PB_DECODE_NOINIT      0x01U
00111 #define PB_DECODE_DELIMITED  0x02U
00112 #define PB_DECODE_NULLTERMINATED 0x04U
00113 bool pb_decode_ex(pb_istream_t *stream, const pb_msgdesc_t *fields, void *dest_struct, unsigned int flags);
00114
00115 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00116 #define pb_decode_noinit(s,f,d) pb_decode_ex(s,f,d, PB_DECODE_NOINIT)
00117 #define pb_decode_delimited(s,f,d) pb_decode_ex(s,f,d, PB_DECODE_DELIMITED)
00118 #define pb_decode_delimited_noinit(s,f,d) pb_decode_ex(s,f,d, PB_DECODE_DELIMITED | PB_DECODE_NOINIT)
00119 #define pb_decode_nullterminated(s,f,d) pb_decode_ex(s,f,d, PB_DECODE_NULLTERMINATED)
00120
00121 /* Release any allocated pointer fields. If you use dynamic allocation, you should
00122  * call this for any successfully decoded message when you are done with it. If
00123  * pb_decode() returns with an error, the message is already released.
00124  */
00125 void pb_release(const pb_msgdesc_t *fields, void *dest_struct);
00126
00127 /*****
00128  * Functions for manipulating streams *
00129  *****/
00130
00131 /* Create an input stream for reading from a memory buffer.
00132  *
00133  * msglen should be the actual length of the message, not the full size of
00134  * allocated buffer.
00135  *
00136  * Alternatively, you can use a custom stream that reads directly from e.g.
00137  * a file or a network socket.
00138  */
00139 pb_istream_t pb_istream_from_buffer(const pb_byte_t *buf, size_t msglen);

```

```

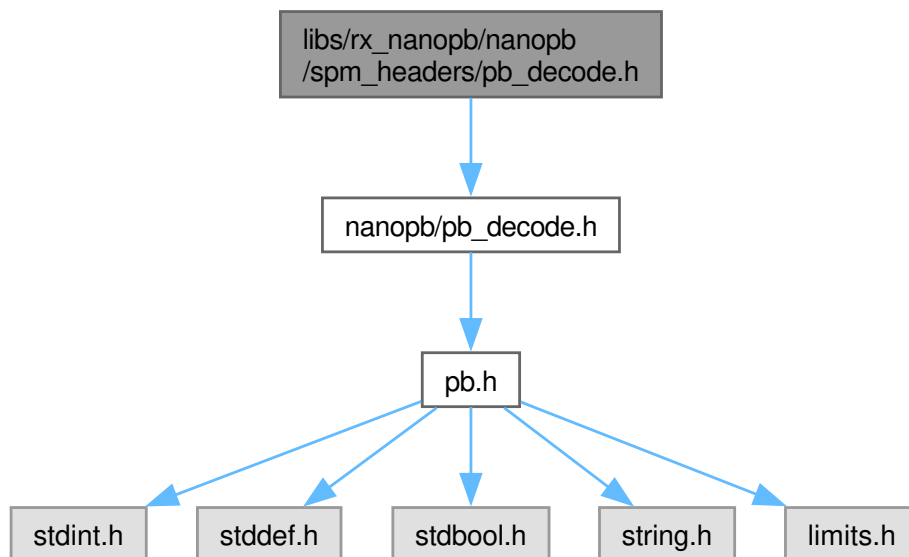
00140
00141 /* Function to read from a pb_istream_t. You can use this if you need to
00142 * read some custom header data, or to read data in field callbacks.
00143 */
00144 bool pb_read(pb_istream_t *stream, pb_byte_t *buf, size_t count);
00145
00146
00147 /*****
00148 * Helper functions for writing field callbacks *
00149 *****/
00150
00151 /* Decode the tag for the next field in the stream. Gives the wire type and
00152 * field tag. At end of the message, returns false and sets eof to true. */
00153 bool pb_decode_tag(pb_istream_t *stream, pb_wire_type_t *wire_type, uint32_t *tag, bool *eof);
00154
00155 /* Skip the field payload data, given the wire type. */
00156 bool pb_skip_field(pb_istream_t *stream, pb_wire_type_t wire_type);
00157
00158 /* Decode an integer in the varint format. This works for enum, int32,
00159 * int64, uint32 and uint64 field types. */
00160 #ifndef PB_WITHOUT_64BIT
00161 bool pb_decode_varint(pb_istream_t *stream, uint64_t *dest);
00162 #else
00163 #define pb_decode_varint pb_decode_varint32
00164 #endif
00165
00166 /* Decode an integer in the varint format. This works for enum, int32,
00167 * and uint32 field types. */
00168 bool pb_decode_varint32(pb_istream_t *stream, uint32_t *dest);
00169
00170 /* Decode a bool value in varint format. */
00171 bool pb_decode_bool(pb_istream_t *stream, bool *dest);
00172
00173 /* Decode an integer in the zig-zagged svarint format. This works for sint32
00174 * and sint64. */
00175 #ifndef PB_WITHOUT_64BIT
00176 bool pb_decode_svarint(pb_istream_t *stream, int64_t *dest);
00177 #else
00178 bool pb_decode_svarint(pb_istream_t *stream, int32_t *dest);
00179 #endif
00180
00181 /* Decode a fixed32, sfixed32 or float value. You need to pass a pointer to
00182 * a 4-byte wide C variable. */
00183 bool pb_decode_fixed32(pb_istream_t *stream, void *dest);
00184
00185 #ifndef PB_WITHOUT_64BIT
00186 /* Decode a fixed64, sfixed64 or double value. You need to pass a pointer to
00187 * a 8-byte wide C variable. */
00188 bool pb_decode_fixed64(pb_istream_t *stream, void *dest);
00189 #endif
00190
00191 #ifdef PB_CONVERT_DOUBLE_FLOAT
00192 /* Decode a double value into float variable. */
00193 bool pb_decode_double_as_float(pb_istream_t *stream, float *dest);
00194 #endif
00195
00196 /* Make a limited-length substream for reading a PB_WT_STRING field. */
00197 bool pb_make_string_substream(pb_istream_t *stream, pb_istream_t *substream);
00198 bool pb_close_string_substream(pb_istream_t *stream, pb_istream_t *substream);
00199
00200 #ifdef __cplusplus
00201 } /* extern "C" */
00202 #endif
00203
00204 #endif

```

6.71 libs/rx_nanopb/nanopb/spm_headers/pb_decode.h File Reference

#include "nanopb/pb_decode.h"

Include dependency graph for pb_decode.h:



6.72 pb_decode.h

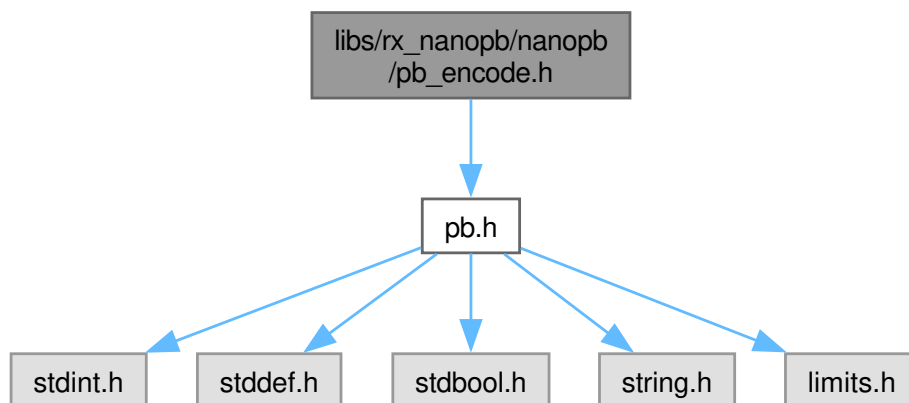
[Go to the documentation of this file.](#)

```
00001 #include "nanopb/pb_decode.h"
```

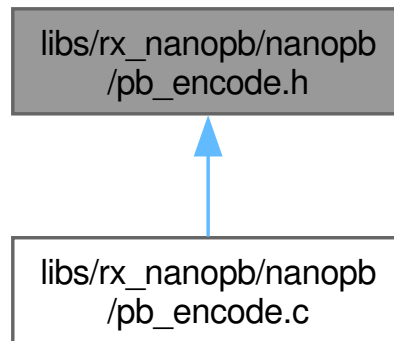
6.73 libs/rx_nanopb/nanopb/pb_encode.h File Reference

```
#include "pb.h"
```

Include dependency graph for `pb_encode.h`:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [pb_ostream_t](#)

Macros

- `#define PB_ENCODE_DELIMITED 0x02U`
- `#define PB_ENCODE_NULLTERMINATED 0x04U`
- `#define pb_encode_delimited(s, f, d)`
- `#define pb_encode_nullterminated(s, f, d)`
- `#define PB_OSTREAM_SIZING {0,0,0,0,0}`

Functions

- `bool pb_encode (pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct)`
- `bool pb_encode_ex (pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct, unsigned int flags)`
- `bool pb_get_encoded_size (size_t *size, const pb_msgdesc_t *fields, const void *src_struct)`
- `pb_ostream_t pb_ostream_from_buffer (pb_byte_t *buf, size_t bufsize)`
- `bool pb_write (pb_ostream_t *stream, const pb_byte_t *buf, size_t count)`
- `bool pb_encode_tag_for_field (pb_ostream_t *stream, const pb_field_iter_t *field)`
- `bool pb_encode_tag (pb_ostream_t *stream, pb_wire_type_t wiretype, uint32_t field_number)`
- `bool pb_encode_varint (pb_ostream_t *stream, uint64_t value)`
- `bool pb_encode_svarint (pb_ostream_t *stream, int64_t value)`
- `bool pb_encode_string (pb_ostream_t *stream, const pb_byte_t *buffer, size_t size)`
- `bool pb_encode_fixed32 (pb_ostream_t *stream, const void *value)`
- `bool pb_encode_fixed64 (pb_ostream_t *stream, const void *value)`
- `bool pb_encode_submessage (pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct)`

6.73.1 Macro Definition Documentation

6.73.1.1 PB_ENCODE_DELIMITED

```
#define PB_ENCODE_DELIMITED 0x02U
```

Definition at line 91 of file [pb_encode.h](#).

Referenced by [pb_encode_ex\(\)](#).

6.73.1.2 pb_encode_delimited

```
#define pb_encode_delimited(  
    s,  
    f,  
    d)
```

Value:

`pb_encode_ex(s,f,d, PB_ENCODE_DELIMITED)`

Definition at line 96 of file [pb_encode.h](#).

6.73.1.3 PB_ENCODE_NULLTERMINATED

```
#define PB_ENCODE_NULLTERMINATED 0x04U
```

Definition at line 92 of file [pb_encode.h](#).

Referenced by [pb_encode_ex\(\)](#).

6.73.1.4 pb_encode_nullterminated

```
#define pb_encode_nullterminated(  
    s,  
    f,  
    d)
```

Value:

`pb_encode_ex(s,f,d, PB_ENCODE_NULLTERMINATED)`

Definition at line 97 of file [pb_encode.h](#).

6.73.1.5 PB_OSTREAM_SIZING

```
#define PB_OSTREAM_SIZING {0,0,0,0,0}
```

Definition at line 126 of file [pb_encode.h](#).

Referenced by [encode_array\(\)](#), [pb_encode_submessage\(\)](#), and [pb_get_encoded_size\(\)](#).

6.73.2 Function Documentation

6.73.2.1 pb_encode()

```
bool pb_encode (
    pb_ostream_t * stream,
    const pb_msgdesc_t * fields,
    const void * src_struct)
```

Definition at line 512 of file [pb_encode.c](#).

```
00513 {
00514     pb_field_iter_t iter;
00515     if (!pb_field_iter_begin_const(&iter, fields, src_struct))
00516         return true; /* Empty message type */
00517
00518     do {
00519         if (PB_LTYPE(iter.type) == PB_LTYPE_EXTENSION)
00520         {
00521             /* Special case for the extension field placeholder */
00522             if (!encode_extension_field(stream, &iter))
00523                 return false;
00524         }
00525         else
00526         {
00527             /* Regular field */
00528             if (!encode_field(stream, &iter))
00529                 return false;
00530         }
00531     } while (pb_field_iter_next(&iter));
00532
00533     return true;
00534 }
```

6.73.2.2 pb_encode_ex()

```
bool pb_encode_ex (
    pb_ostream_t * stream,
    const pb_msgdesc_t * fields,
    const void * src_struct,
    unsigned int flags)
```

Definition at line 536 of file [pb_encode.c](#).

```
00537 {
00538     if ((flags & PB_ENCODE_DELIMITED) != 0)
00539     {
00540         return pb_encode_submessage(stream, fields, src_struct);
00541     }
00542     else if ((flags & PB_ENCODE_NULLTERMINATED) != 0)
00543     {
00544         const pb_byte_t zero = 0;
00545
00546         if (!pb_encode(stream, fields, src_struct))
00547             return false;
00548
00549         return pb_write(stream, &zero, 1);
00550     }
00551     else
00552     {
00553         return pb_encode(stream, fields, src_struct);
00554     }
00555 }
```

6.73.2.3 pb_encode_fixed32()

```
bool pb_encode_fixed32 (
    pb_ostream_t * stream,
    const void * value)
```

Definition at line 637 of file [pb_encode.c](#).

```
00638 {
00639 #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
00640     /* Fast path if we know that we're on little endian */
00641     return pb_write(stream, (const pb_byte_t*)value, 4);
00642 #else
00643     uint32_t val = *(const uint32_t*)value;
00644     pb_byte_t bytes[4];
00645     bytes[0] = (pb_byte_t)(val & 0xFF);
00646     bytes[1] = (pb_byte_t)((val >> 8) & 0xFF);
00647     bytes[2] = (pb_byte_t)((val >> 16) & 0xFF);
00648     bytes[3] = (pb_byte_t)((val >> 24) & 0xFF);
00649     return pb_write(stream, bytes, 4);
00650 #endif
00651 }
```

6.73.2.4 pb_encode_fixed64()

```
bool pb_encode_fixed64 (
    pb_ostream_t * stream,
    const void * value)
```

Definition at line 654 of file [pb_encode.c](#).

```
00655 {
00656 #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
00657     /* Fast path if we know that we're on little endian */
00658     return pb_write(stream, (const pb_byte_t*)value, 8);
00659 #else
00660     uint64_t val = *(const uint64_t*)value;
00661     pb_byte_t bytes[8];
00662     bytes[0] = (pb_byte_t)(val & 0xFF);
00663     bytes[1] = (pb_byte_t)((val >> 8) & 0xFF);
00664     bytes[2] = (pb_byte_t)((val >> 16) & 0xFF);
00665     bytes[3] = (pb_byte_t)((val >> 24) & 0xFF);
00666     bytes[4] = (pb_byte_t)((val >> 32) & 0xFF);
00667     bytes[5] = (pb_byte_t)((val >> 40) & 0xFF);
00668     bytes[6] = (pb_byte_t)((val >> 48) & 0xFF);
00669     bytes[7] = (pb_byte_t)((val >> 56) & 0xFF);
00670     return pb_write(stream, bytes, 8);
00671 #endif
00672 }
```

6.73.2.5 pb_encode_string()

```
bool pb_encode_string (
    pb_ostream_t * stream,
    const pb_byte_t * buffer,
    size_t size)
```

Definition at line 716 of file [pb_encode.c](#).

```
00717 {
00718     if (!pb_encode_varint(stream, (pb_uint64_t)size))
00719         return false;
00720     return pb_write(stream, buffer, size);
00722 }
```

6.73.2.6 pb_encode_submessage()

```
bool pb_encode_submessage (
    pb_ostream_t * stream,
    const pb_msgdesc_t * fields,
    const void * src_struct)
```

Definition at line 724 of file [pb_encode.c](#).

```

00725 {
00726     /* First calculate the message size using a non-writing substream. */
00727     pb_ostream_t substream = PB_OSTREAM_SIZING;
00728     #if !defined(PB_NO_ENCODE_SIZE_CHECK) || PB_NO_ENCODE_SIZE_CHECK == 0
00729         bool status;
00730         size_t size;
00731     #endif
00732
00733     if (!pb_encode(&substream, fields, src_struct))
00734     {
00735         #ifndef PB_NO_ERRMSG
00736             stream->errmsg = substream.errmsg;
00737         #endif
00738         return false;
00739     }
00740
00741     if (!pb_encode_varint(stream, (pb_uint64_t)substream.bytes_written))
00742         return false;
00743
00744     if (stream->callback == NULL)
00745         return pb_write(stream, NULL, substream.bytes_written); /* Just sizing */
00746
00747     if (stream->bytes_written + substream.bytes_written > stream->max_size)
00748         PB_RETURN_ERROR(stream, "stream full");
00749
00750     #if defined(PB_NO_ENCODE_SIZE_CHECK) && PB_NO_ENCODE_SIZE_CHECK == 1
00751         return pb_encode(stream, fields, src_struct);
00752     #else
00753         size = substream.bytes_written;
00754         /* Use a substream to verify that a callback doesn't write more than
00755          * what it did the first time. */
00756         substream.callback = stream->callback;
00757         substream.state = stream->state;
00758         substream.max_size = size;
00759         substream.bytes_written = 0;
00760         #ifndef PB_NO_ERRMSG
00761             substream.errmsg = NULL;
00762         #endif
00763
00764         status = pb_encode(&substream, fields, src_struct);
00765
00766         stream->bytes_written += substream.bytes_written;
00767         stream->state = substream.state;
00768         #ifndef PB_NO_ERRMSG
00769             stream->errmsg = substream.errmsg;
00770         #endif
00771
00772         if (substream.bytes_written != size)
00773             PB_RETURN_ERROR(stream, "submsg size changed");
00774
00775         return status;
00776     #endif
00777 }

```

6.73.2.7 pb_encode_svarint()

```

bool pb_encode_svarint (
    pb_ostream_t * stream,
    int64_t value)

```

Definition at line 625 of file [pb_encode.c](#).

```

00626 {
00627     pb_uint64_t zigzagged;
00628     pb_uint64_t mask = ((pb_uint64_t)-1) » 1; /* Satisfy clang -fsanitize=integer */
00629     if (value < 0)
00630         zigzagged = ~(((pb_uint64_t)value & mask) « 1);
00631     else
00632         zigzagged = (pb_uint64_t)value « 1;
00633
00634     return pb_encode_varint(stream, zigzagged);
00635 }

```

6.73.2.8 pb_encode_tag()

```

bool pb_encode_tag (
    pb_ostream_t * stream,

```

```

    pb_wire_type_t wiretype,
    uint32_t field_number)

```

Definition at line 675 of file [pb_encode.c](#).

```

00676 {
00677     pb_uint64_t tag = ((pb_uint64_t)field_number « 3) | wiretype;
00678     return pb_encode_varint(stream, tag);
00679 }

```

6.73.2.9 pb_encode_tag_for_field()

```

bool pb_encode_tag_for_field (
    pb_ostream_t * stream,
    const pb_field_iter_t * field)

```

Definition at line 681 of file [pb_encode.c](#).

```

00682 {
00683     pb_wire_type_t wiretype;
00684     switch (PB_LTYPE(field->type))
00685     {
00686         case PB_LTYPE_BOOL:
00687         case PB_LTYPE_VARINT:
00688         case PB_LTYPE_UVARINT:
00689         case PB_LTYPE_SVARINT:
00690             wiretype = PB_WT_VARINT;
00691             break;
00692         case PB_LTYPE_FIXED32:
00693             wiretype = PB_WT_32BIT;
00694             break;
00695         case PB_LTYPE_FIXED64:
00696             wiretype = PB_WT_64BIT;
00697             break;
00698         case PB_LTYPE_BYTES:
00699         case PB_LTYPE_STRING:
00700         case PB_LTYPE_SUBMESSAGE:
00701         case PB_LTYPE_SUBMSG_W_CB:
00702         case PB_LTYPE_FIXED_LENGTH_BYTES:
00703             wiretype = PB_WT_STRING;
00704             break;
00705         default:
00706             PB_RETURN_ERROR(stream, "invalid field type");
00707     }
00708     return pb_encode_tag(stream, wiretype, field->tag);
00709 }

```

6.73.2.10 pb_encode_varint()

```

bool pb_encode_varint (
    pb_ostream_t * stream,
    uint64_t value)

```

Definition at line 607 of file [pb_encode.c](#).

```

00608 {
00609     if (value <= 0x7F)
00610     {
00611         /* Fast path: single byte */
00612         pb_byte_t byte = (pb_byte_t)value;
00613         return pb_write(stream, &byte, 1);
00614     }
00615     else
00616     {
00617         #ifdef PB_WITHOUT_64BIT
00618             return pb_encode_varint_32(stream, value, 0);
00619         #else
00620             return pb_encode_varint_32(stream, (uint32_t)value, (uint32_t)(value » 32));
00621         #endif
00622     }
00623 }

```

6.73.2.11 pb_get_encoded_size()

```
bool pb_get_encoded_size (
    size_t * size,
    const pb_msgdesc_t * fields,
    const void * src_struct)
```

Definition at line 557 of file [pb_encode.c](#).

```
00558 {
00559     pb_ostream_t stream = PB_OSTREAM_SIZING;
00560
00561     if (!pb_encode(&stream, fields, src_struct))
00562         return false;
00563
00564     *size = stream.bytes_written;
00565     return true;
00566 }
```

6.73.2.12 pb_ostream_from_buffer()

```
pb_ostream_t pb_ostream_from_buffer (
    pb_byte_t * buf,
    size_t bufsize)
```

Definition at line 63 of file [pb_encode.c](#).

```
00064 {
00065     pb_ostream_t stream;
00066     #ifdef PB_BUFFER_ONLY
00067         /* In PB_BUFFER_ONLY configuration the callback pointer is just int*.
00068          * NULL pointer marks a sizing field, so put a non-NULL value to mark a buffer stream.
00069          */
00070         static const int marker = 0;
00071         stream.callback = &marker;
00072     #else
00073         stream.callback = &buf_write;
00074     #endif
00075     stream.state = buf;
00076     stream.max_size = bufsize;
00077     stream.bytes_written = 0;
00078     #ifndef PB_NO_ERRMSG
00079     stream.errmsg = NULL;
00080     #endif
00081     return stream;
00082 }
```

6.73.2.13 pb_write()

```
bool pb_write (
    pb_ostream_t * stream,
    const pb_byte_t * buf,
    size_t count)
```

Definition at line 84 of file [pb_encode.c](#).

```
00085 {
00086     if (count > 0 && stream->callback != NULL)
00087     {
00088         if (stream->bytes_written + count < stream->bytes_written ||
00089             stream->bytes_written + count > stream->max_size)
00090         {
00091             PB_RETURN_ERROR(stream, "stream full");
00092         }
00093     }
00094     #ifdef PB_BUFFER_ONLY
00095     if (!buf_write(stream, buf, count))
00096         PB_RETURN_ERROR(stream, "io error");
00097     #else
00098     if (!stream->callback(stream, buf, count))
00099         PB_RETURN_ERROR(stream, "io error");
00100     #endif
00101 }
00102
00103 stream->bytes_written += count;
00104 return true;
00105 }
```


6.74 pb_encode.h

[Go to the documentation of this file.](#)

```

00001 /* pb_encode.h: Functions to encode protocol buffers. Depends on pb_encode.c.
00002  * The main function is pb_encode. You also need an output stream, and the
00003  * field descriptions created by nanopb_generator.py.
00004  */
00005
00006 #ifndef PB_ENCODE_H_INCLUDED
00007 #define PB_ENCODE_H_INCLUDED
00008
00009 #include "pb.h"
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00015 /* Structure for defining custom output streams. You will need to provide
00016  * a callback function to write the bytes to your storage, which can be
00017  * for example a file or a network socket.
00018  *
00019  * The callback must conform to these rules:
00020  *
00021  * 1) Return false on IO errors. This will cause encoding to abort.
00022  * 2) You can use state to store your own data (e.g. buffer pointer).
00023  * 3) pb_write will update bytes_written after your callback runs.
00024  * 4) Substreams will modify max_size and bytes_written. Don't use them
00025  *    to calculate any pointers.
00026  */
00027 struct pb_ostream_s
00028 {
00029     #ifdef PB_BUFFER_ONLY
00030         /* Callback pointer is not used in buffer-only configuration.
00031          * Having an int pointer here allows binary compatibility but
00032          * gives an error if someone tries to assign callback function.
00033          * Also, NULL pointer marks a 'sizing stream' that does not
00034          * write anything.
00035          */
00036         const int *callback;
00037     #else
00038         bool (*callback)(pb_ostream_t *stream, const pb_byte_t *buf, size_t count);
00039     #endif
00040
00041     /* state is a free field for use of the callback function defined above.
00042      * Note that when pb_ostream_from_buffer() is used, it reserves this field
00043      * for its own use.
00044      */
00045     void *state;
00046
00047     /* Limit number of output bytes written. Can be set to SIZE_MAX. */
00048     size_t max_size;
00049
00050     /* Number of bytes written so far. */
00051     size_t bytes_written;
00052
00053     #ifndef PB_NO_ERRMSG
00054         /* Pointer to constant (ROM) string when decoding function returns error */
00055         const char *errmsg;
00056     #endif
00057 };
00058
00059 /*****
00060  * Main encoding functions *
00061  *****/
00062
00063 /* Encode a single protocol buffers message from C structure into a stream.
00064  * Returns true on success, false on any failure.
00065  * The actual struct pointed to by src_struct must match the description in fields.
00066  * All required fields in the struct are assumed to have been filled in.
00067  *
00068  * Example usage:
00069  *   MyMessage msg = {};
00070  *   uint8_t buffer[64];
00071  *   pb_ostream_t stream;
00072  *
00073  *   msg.field1 = 42;
00074  *   stream = pb_ostream_from_buffer(buffer, sizeof(buffer));
00075  *   pb_encode(&stream, MyMessage_fields, &msg);
00076  */
00077 bool pb_encode(pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct);
00078
00079 /* Extended version of pb_encode, with several options to control the
00080  * encoding process:
00081  *
00082  * PB_ENCODE_DELIMITED:    Prepend the length of message as a varint.

```

```

00083 *           Corresponds to writeDelimitedTo() in Google's
00084 *           protobuf API.
00085 *
00086 * PB_ENCODE_NULLTERMINATED: Append a null byte to the message for termination.
00087 *           NOTE: This behaviour is not supported in most other
00088 *           protobuf implementations, so PB_ENCODE_DELIMITED
00089 *           is a better option for compatibility.
00090 */
00091 #define PB_ENCODE_DELIMITED    0x02U
00092 #define PB_ENCODE_NULLTERMINATED 0x04U
00093 bool pb_encode_ex(pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct, unsigned int flags);
00094
00095 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00096 #define pb_encode_delimited(s,f,d) pb_encode_ex(s,f,d, PB_ENCODE_DELIMITED)
00097 #define pb_encode_nullterminated(s,f,d) pb_encode_ex(s,f,d, PB_ENCODE_NULLTERMINATED)
00098
00099 /* Encode the message to get the size of the encoded data, but do not store
00100 * the data. */
00101 bool pb_get_encoded_size(size_t *size, const pb_msgdesc_t *fields, const void *src_struct);
00102
00103 /*****
00104 * Functions for manipulating streams *
00105 *****/
00106
00107 /* Create an output stream for writing into a memory buffer.
00108 * The number of bytes written can be found in stream.bytes_written after
00109 * encoding the message.
00110 *
00111 * Alternatively, you can use a custom stream that writes directly to e.g.
00112 * a file or a network socket.
00113 */
00114 pb_ostream_t pb_ostream_from_buffer(pb_byte_t *buf, size_t bufsize);
00115
00116 /* Pseudo-stream for measuring the size of a message without actually storing
00117 * the encoded data.
00118 *
00119 * Example usage:
00120 *     MyMessage msg = {};
00121 *     pb_ostream_t stream = PB_OSTREAM_SIZING;
00122 *     pb_encode(&stream, MyMessage_fields, &msg);
00123 *     printf("Message size is %d\n", stream.bytes_written);
00124 */
00125 #ifndef PB_NO_ERRMSG
00126 #define PB_OSTREAM_SIZING {0,0,0,0}
00127 #else
00128 #define PB_OSTREAM_SIZING {0,0,0,0}
00129 #endif
00130
00131 /* Function to write into a pb_ostream_t stream. You can use this if you need
00132 * to append or prepend some custom headers to the message.
00133 */
00134 bool pb_write(pb_ostream_t *stream, const pb_byte_t *buf, size_t count);
00135
00136 /*****
00137 * Helper functions for writing field callbacks *
00138 *****/
00139
00140
00141 /* Encode field header based on type and field number defined in the field
00142 * structure. Call this from the callback before writing out field contents. */
00143 bool pb_encode_tag_for_field(pb_ostream_t *stream, const pb_field_iter_t *field);
00144
00145 /* Encode field header by manually specifying wire type. You need to use this
00146 * if you want to write out packed arrays from a callback field. */
00147 bool pb_encode_tag(pb_ostream_t *stream, pb_wire_type_t wiretype, uint32_t field_number);
00148
00149 /* Encode an integer in the varint format.
00150 * This works for bool, enum, int32, int64, uint32 and uint64 field types. */
00151 #ifndef PB_WITHOUT_64BIT
00152 bool pb_encode_varint(pb_ostream_t *stream, uint64_t value);
00153 #else
00154 bool pb_encode_varint(pb_ostream_t *stream, uint32_t value);
00155 #endif
00156
00157 /* Encode an integer in the zig-zagged svarint format.
00158 * This works for sint32 and sint64. */
00159 #ifndef PB_WITHOUT_64BIT
00160 bool pb_encode_svarint(pb_ostream_t *stream, int64_t value);
00161 #else
00162 bool pb_encode_svarint(pb_ostream_t *stream, int32_t value);
00163 #endif
00164
00165 /* Encode a string or bytes type field. For strings, pass strlen(s) as size. */
00166 bool pb_encode_string(pb_ostream_t *stream, const pb_byte_t *buffer, size_t size);
00167
00168 /* Encode a fixed32, sfixed32 or float value.
00169 * You need to pass a pointer to a 4-byte wide C variable. */

```

```

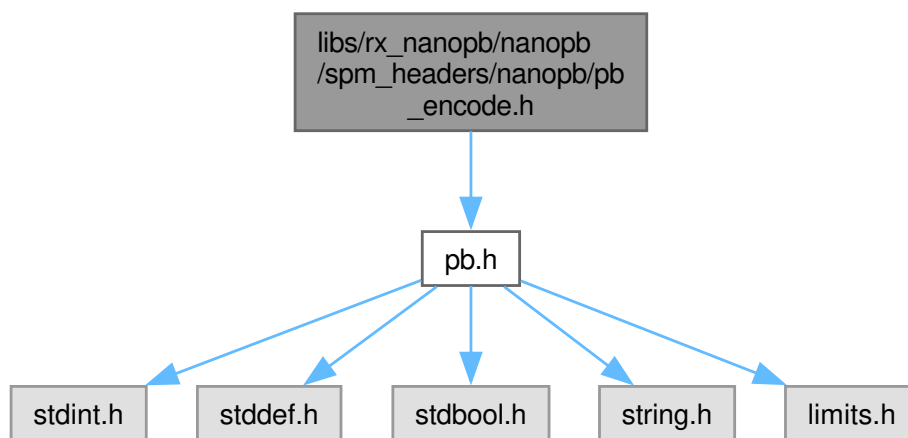
00170 bool pb_encode_fixed32(pb_ostream_t *stream, const void *value);
00171
00172 #ifndef PB_WITHOUT_64BIT
00173 /* Encode a fixed64, sfixed64 or double value.
00174 * You need to pass a pointer to a 8-byte wide C variable. */
00175 bool pb_encode_fixed64(pb_ostream_t *stream, const void *value);
00176 #endif
00177
00178 #ifndef PB_CONVERT_DOUBLE_FLOAT
00179 /* Encode a float value so that it appears like a double in the encoded
00180 * message. */
00181 bool pb_encode_float_as_double(pb_ostream_t *stream, float value);
00182 #endif
00183
00184 /* Encode a submessage field.
00185 * You need to pass the pb_field_t array and pointer to struct, just like
00186 * with pb_encode(). This internally encodes the submessage twice, first to
00187 * calculate message size and then to actually write it out.
00188 */
00189 bool pb_encode_submessage(pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct);
00190
00191 #ifdef __cplusplus
00192 } /* extern "C" */
00193 #endif
00194
00195 #endif

```

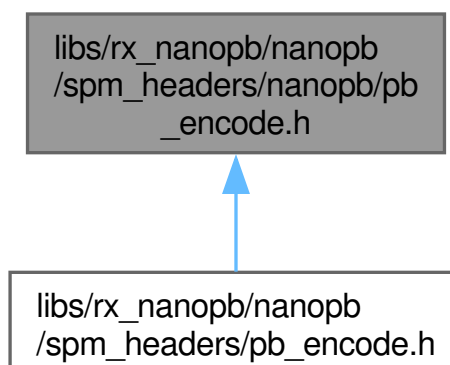
6.75 libs/rx_nanopb/nanopb/spm_headers/nanopb/pb_encode.h File Reference

#include "pb.h"

Include dependency graph for pb_encode.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [pb_ostream_t](#)

Macros

- `#define PB_ENCODE_DELIMITED 0x02U`
- `#define PB_ENCODE_NULLTERMINATED 0x04U`
- `#define pb_encode_delimited(s, f, d)`
- `#define pb_encode_nullterminated(s, f, d)`
- `#define PB_OSTREAM_SIZING {0,0,0,0,0}`

Functions

- [bool pb_encode](#) (pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct)
- [bool pb_encode_ex](#) (pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct, unsigned int flags)
- [bool pb_get_encoded_size](#) (size_t *size, const pb_msgdesc_t *fields, const void *src_struct)
- pb_ostream_t [pb_ostream_from_buffer](#) (pb_byte_t *buf, size_t bufsize)
- [bool pb_write](#) (pb_ostream_t *stream, const pb_byte_t *buf, size_t count)
- [bool pb_encode_tag_for_field](#) (pb_ostream_t *stream, const pb_field_iter_t *field)
- [bool pb_encode_tag](#) (pb_ostream_t *stream, pb_wire_type_t wiretype, uint32_t field_number)
- [bool pb_encode_varint](#) (pb_ostream_t *stream, uint64_t value)
- [bool pb_encode_svarint](#) (pb_ostream_t *stream, int64_t value)
- [bool pb_encode_string](#) (pb_ostream_t *stream, const pb_byte_t *buffer, size_t size)
- [bool pb_encode_fixed32](#) (pb_ostream_t *stream, const void *value)
- [bool pb_encode_fixed64](#) (pb_ostream_t *stream, const void *value)
- [bool pb_encode_submessage](#) (pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct)

6.75.1 Macro Definition Documentation

6.75.1.1 PB_ENCODE_DELIMITED

```
#define PB_ENCODE_DELIMITED 0x02U
```

Definition at line 91 of file [pb_encode.h](#).

6.75.1.2 pb_encode_delimited

```
#define pb_encode_delimited(  
    s,  
    f,  
    d)
```

Value:

```
pb_encode_ex(s,f,d, PB_ENCODE_DELIMITED)
```

Definition at line 96 of file [pb_encode.h](#).

6.75.1.3 PB_ENCODE_NULLTERMINATED

```
#define PB_ENCODE_NULLTERMINATED 0x04U
```

Definition at line 92 of file [pb_encode.h](#).

6.75.1.4 pb_encode_nullterminated

```
#define pb_encode_nullterminated(  
    s,  
    f,  
    d)
```

Value:

```
pb_encode_ex(s,f,d, PB_ENCODE_NULLTERMINATED)
```

Definition at line 97 of file [pb_encode.h](#).

6.75.1.5 PB_OSTREAM_SIZING

```
#define PB_OSTREAM_SIZING {0,0,0,0,0}
```

Definition at line 126 of file [pb_encode.h](#).

6.75.2 Function Documentation

6.75.2.1 pb_encode()

```
bool pb_encode (  
    pb_ostream_t * stream,  
    const pb_msgdesc_t * fields,  
    const void * src_struct)
```

Definition at line 512 of file [pb_encode.c](#).

```
00513 {  
00514     pb_field_iter_t iter;  
00515     if (!pb_field_iter_begin_const(&iter, fields, src_struct))  
00516         return true; /* Empty message type */  
00517  
00518     do {  
00519         if (PB_LTYPE(iter.type) == PB_LTYPE_EXTENSION)  
00520         {  
00521             /* Special case for the extension field placeholder */  
00522             if (!encode_extension_field(stream, &iter))  
00523                 return false;  
00524         }  
00525         else  
00526         {  
00527             /* Regular field */  
00528             if (!encode_field(stream, &iter))  
00529                 return false;  
00530         }  
00531     } while (pb_field_iter_next(&iter));  
00532  
00533     return true;  
00534 }
```

References [checkreturn](#), [encode_extension_field\(\)](#), [encode_field\(\)](#), [pb_field_iter_begin_const\(\)](#), [pb_field_iter_next\(\)](#), [PB_LTYPE](#), and [PB_LTYPE_EXTENSION](#).

Referenced by [pb_encode_ex\(\)](#), [pb_encode_submessage\(\)](#), [pb_get_encoded_size\(\)](#), [rx_nanopb_encode_estop_response\(\)](#), [rx_nanopb_encode_pid_gains_response\(\)](#), [rx_nanopb_encode_telemetry\(\)](#), [rx_nanopb_encode_velocity_request\(\)](#), and [rx_nanopb_encode_velocity_response\(\)](#).

Here is the call graph for this function:



6.75.2.2 pb_encode_ex()

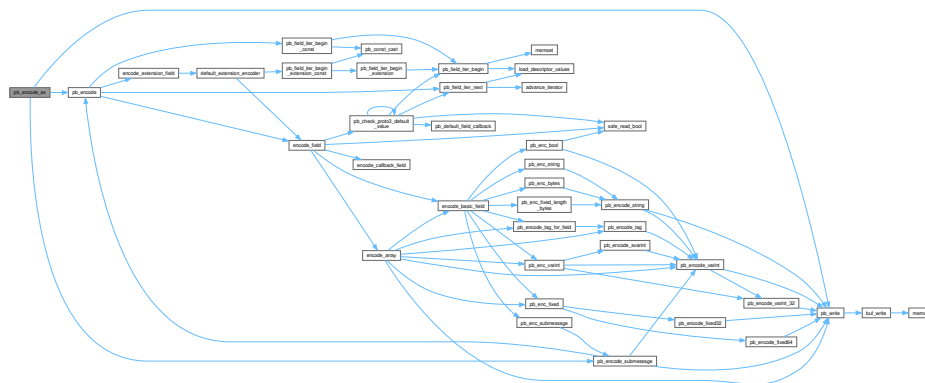
```
bool pb_encode_ex (
    pb_ostream_t * stream,
    const pb_msgdesc_t * fields,
    const void * src_struct,
    unsigned int flags)
```

Definition at line 536 of file pb_encode.c.

```
00537 {
00538     if ((flags & PB_ENCODE_DELIMITED) != 0)
00539     {
00540         return pb_encode_submessage(stream, fields, src_struct);
00541     }
00542     else if ((flags & PB_ENCODE_NULLTERMINATED) != 0)
00543     {
00544         const pb_byte_t zero = 0;
00545
00546         if (!pb_encode(stream, fields, src_struct))
00547             return false;
00548
00549         return pb_write(stream, &zero, 1);
00550     }
00551     else
00552     {
00553         return pb_encode(stream, fields, src_struct);
00554     }
00555 }
```

References [checkreturn](#), [pb_encode\(\)](#), [PB_ENCODE_DELIMITED](#), [PB_ENCODE_NULLTERMINATED](#), [pb_encode_submessage\(\)](#), and [pb_write\(\)](#).

Here is the call graph for this function:



```
bool pb_encode_fixed32 (
    pb_ostream_t * stream,
    const void * value)
```

```

00638 {
00639     /*#if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
00640     /* Fast path if we know that we're on little endian */
00641     return pb_write(stream, (const pb_byte_t*)value, 4);
00642 */else
00643     uint32_t val = *(const uint32_t*)value;
00644     pb_byte_t bytes[4];
00645     bytes[0] = (pb_byte_t)(val & 0xFF);
00646     bytes[1] = (pb_byte_t)((val >> 8) & 0xFF);
00647     bytes[2] = (pb_byte_t)((val >> 16) & 0xFF);
00648     bytes[3] = (pb_byte_t)((val >> 24) & 0xFF);
00649     return pb_write(stream, bytes, 4);
00650 */endif
00651 }

```

Referenced by [pb_enc_fixed\(\)](#).

```
graph LR; A[pb_encode_fixed32] --> B[pb_write]; B --> C[buf_write]; C --> D[memcpy]
```

```

graph LR
    pb_encode_ext[pb_encode_ext] --> pb_encode_ext_size[pb_encode_ext_size]
    pb_encode_ext_size --> pb_encode_ext
    rx_nanopb_encode_ext_stop_response[rx_nanopb_encode_ext_stop_response] --> pb_encode
    rx_nanopb_encode_ext_pid_getter_response[rx_nanopb_encode_ext_pid_getter_response] --> pb_encode
    rx_nanopb_encode_ext_telemetry[rx_nanopb_encode_ext_telemetry] --> pb_encode
    rx_nanopb_encode_ext_velocity_request[rx_nanopb_encode_ext_velocity_request] --> pb_encode
    rx_nanopb_encode_ext_velocity_response[rx_nanopb_encode_ext_velocity_response] --> pb_encode
    pb_encode_ext_submessage[pb_encode_ext_submessage] --> pb_encode
    pb_encode_ext_submessage --> pb_encode_ext_size
    pb_encode --> encode_extension_field[encode_extension_field]
    encode_extension_field --> encode_field[encode_field]
    encode_field --> encode_array[encode_array]
    encode_array --> pb_encode_ext_size
    encode_array --> encode_basic_field[encode_basic_field]
    encode_basic_field --> pb_encode_ext_size
    encode_array --> pb_encode_ext_size
  
```

6.75.2.4 pb_encode_fixed64()

```
bool pb_encode_fixed64 (
    pb_ostream_t * stream,
    const void * value)
```

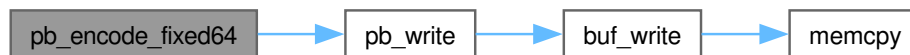
Definition at line 654 of file [pb_encode.c](#).

```
00655 {
00656     #if defined(PB_LITTLE_ENDIAN_8BIT) && PB_LITTLE_ENDIAN_8BIT == 1
00657         /* Fast path if we know that we're on little endian */
00658         return pb_write(stream, (const pb_byte_t*)value, 8);
00659     #else
00660         uint64_t val = *(const uint64_t*)value;
00661         pb_byte_t bytes[8];
00662         bytes[0] = (pb_byte_t)(val & 0xFF);
00663         bytes[1] = (pb_byte_t)((val >> 8) & 0xFF);
00664         bytes[2] = (pb_byte_t)((val >> 16) & 0xFF);
00665         bytes[3] = (pb_byte_t)((val >> 24) & 0xFF);
00666         bytes[4] = (pb_byte_t)((val >> 32) & 0xFF);
00667         bytes[5] = (pb_byte_t)((val >> 40) & 0xFF);
00668         bytes[6] = (pb_byte_t)((val >> 48) & 0xFF);
00669         bytes[7] = (pb_byte_t)((val >> 56) & 0xFF);
00670         return pb_write(stream, bytes, 8);
00671     #endif
00672 }
```

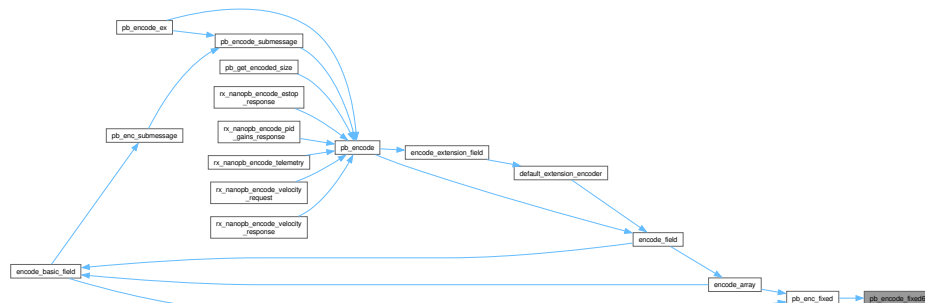
References [checkreturn](#), and [pb_write\(\)](#).

Referenced by [pb_enc_fixed\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.75.2.5 pb_encode_string()

```
bool pb_encode_string (
    pb_ostream_t * stream,
    const pb_byte_t * buffer,
    size_t size)
```

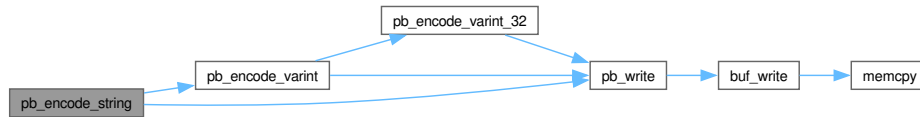
Definition at line 716 of file [pb_encode.c](#).

```
00717 {
00718     if (!pb_encode_varint(stream, (pb_uint64_t)size))
00719         return false;
00720     return pb_write(stream, buffer, size);
00722 }
```

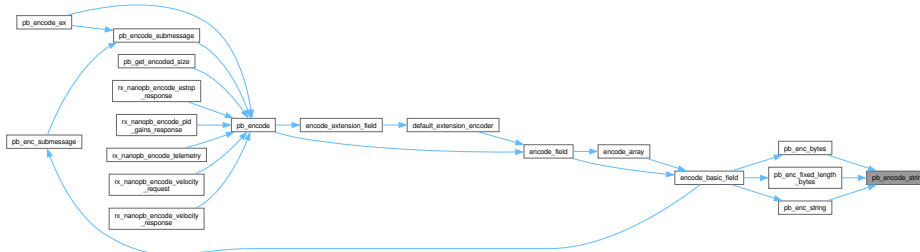
References [checkreturn](#), [pb_encode_varint\(\)](#), [pb_uint64_t](#), and [pb_write\(\)](#).

Referenced by [pb_enc_bytes\(\)](#), [pb_enc_fixed_length_bytes\(\)](#), and [pb_enc_string\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.75.2.6 pb_encode_submessage()

```

bool pb_encode_submessage (
    pb_ostream_t * stream,
    const pb_msgdesc_t * fields,
    const void * src_struct)

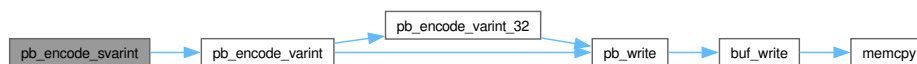
```

Definition at line 724 of file [pb_encode.c](#).

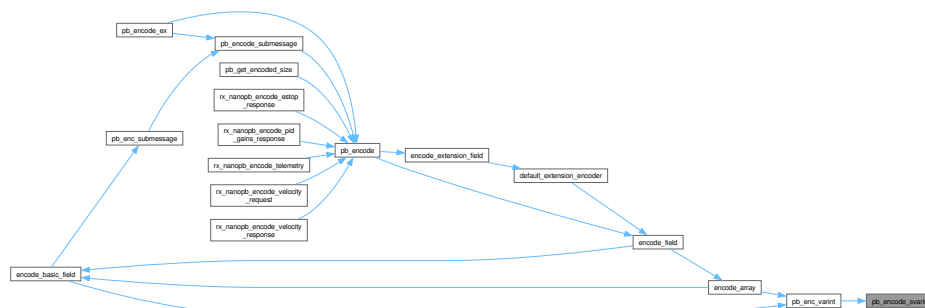
```

00725 {
00726     /* First calculate the message size using a non-writing substream. */
00727     pb_ostream_t substream = PB_OSTREAM_SIZING;
00728     #if !defined(PB_NO_ENCODE_SIZE_CHECK) || PB_NO_ENCODE_SIZE_CHECK == 0
00729     bool status;
00730     size_t size;
00731     #endif
00732
00733     if (!pb_encode(&substream, fields, src_struct))
00734     {
00735     #ifndef PB_NO_ERRMSG
00736         stream->errmsg = substream.errmsg;
00737     #endif
00738         return false;
00739     }
00740
00741     if (!pb_encode_varint(stream, (pb_uint64_t)substream.bytes_written))
00742         return false;
00743
00744     if (stream->callback == NULL)
00745         return pb_write(stream, NULL, substream.bytes_written); /* Just sizing */
00746
00747     if (stream->bytes_written + substream.bytes_written > stream->max_size)
00748         PB_RETURN_ERROR(stream, "stream full");
00749
00750     #if defined(PB_NO_ENCODE_SIZE_CHECK) && PB_NO_ENCODE_SIZE_CHECK == 1
00751     return pb_encode(stream, fields, src_struct);
00752     #else
00753     size = substream.bytes_written;
00754     /* Use a substream to verify that a callback doesn't write more than
00755      * what it did the first time. */
00756     substream.callback = stream->callback;
00757     substream.state = stream->state;
00758     substream.max_size = size;
00759     substream.bytes_written = 0;
00760     #ifndef PB_NO_ERRMSG
00761     substream.errmsg = NULL;
00762     #endif
00763
00764     status = pb_encode(&substream, fields, src_struct);
00765
00766     stream->bytes_written += substream.bytes_written;
00767     stream->state = substream.state;
00768     #ifndef PB_NO_ERRMSG
00769     stream->errmsg = substream.errmsg;

```

Here is the caller graph for this function:



6.75.2.8 pb_encode_tag()

```

bool pb_encode_tag (
    pb_ostream_t * stream,
    pb_wire_type_t wiretype,
    uint32_t field_number)
  
```

Definition at line 675 of file [pb_encode.c](#).

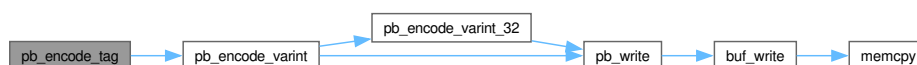
```

00676 {
00677     pb_uint64_t tag = ((pb_uint64_t)field_number << 3) | wiretype;
00678     return pb_encode_varint(stream, tag);
00679 }
  
```

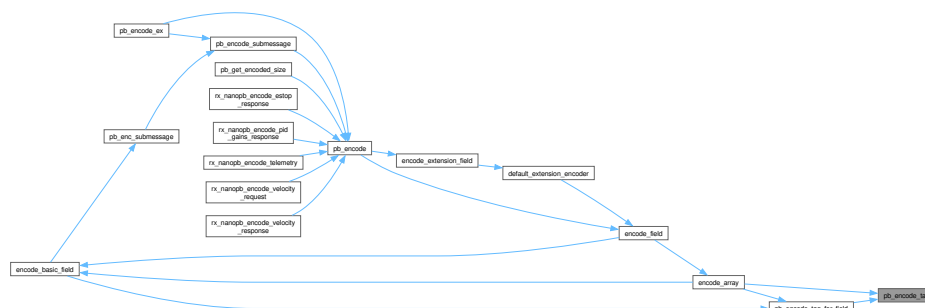
References [checkreturn](#), [pb_encode_varint\(\)](#), and [pb_uint64_t](#).

Referenced by [encode_array\(\)](#), and [pb_encode_tag_for_field\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:




```
bool pb_encode_varint (
    pb_ostream_t * stream,
    uint64_t value)
```

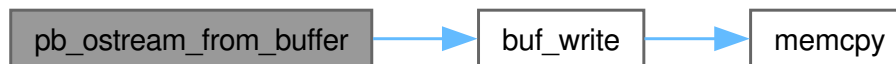
```
00608 {
00609     if (value <= 0x7F)
00610     {
00611         /* Fast path: single byte */
00612         pb_byte_t byte = (pb_byte_t)value;
00613         return pb_write(stream, &byte, 1);
00614     }
00615     else
00616     {
00617         #ifdef PB_WITHOUT_64BIT
00618             return pb_encode_varint_32(stream, value, 0);
00619         #else
00620             return pb_encode_varint_32(stream, (uint32_t)value, (uint32_t)(value » 32));
00621         #endif
00622     }
00623 }
```

Referenced by `encode_array()`, `pb_enc_bool()`, `pb_enc_varint()`, `pb_encode_string()`, `pb_encode_submessage()`, `pb_encode_svarint()`, and `pb_encode_tag()`.

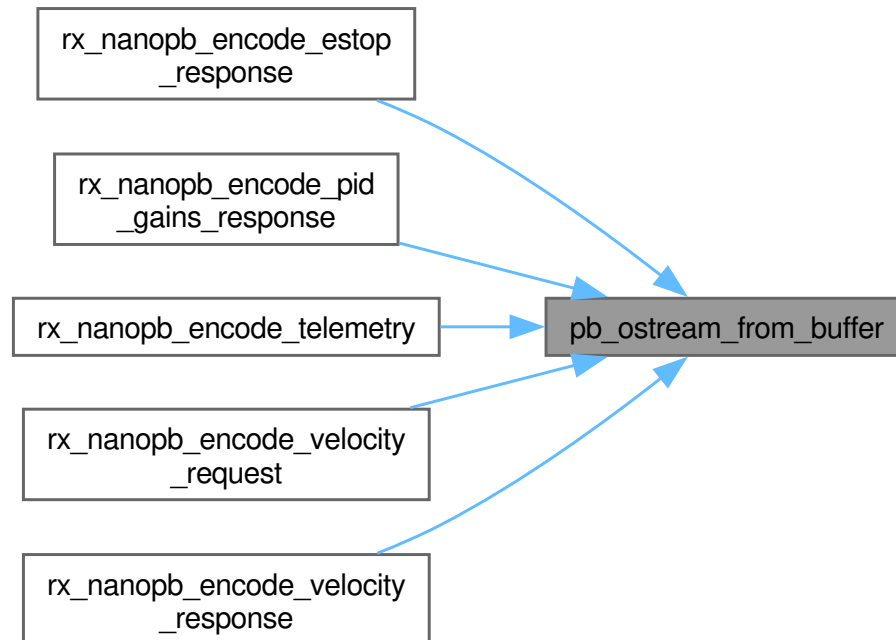
```

graph LR
    A[pb_encode_varint] --> B[pb_encode_varint_32]
    A --> C[pb_write]
    B --> C
    C --> D[buf_write]
    D --> E[memcpy]
  
```

[illegible]



Here is the caller graph for this function:



6.75.2.13 pb_write()

```

bool pb_write (
    pb_ostream_t * stream,
    const pb_byte_t * buf,
    size_t count)
  
```

Definition at line 84 of file [pb_encode.c](#).

```

00085 {
00086     if (count > 0 && stream->callback != NULL)
00087     {
00088         if (stream->bytes_written + count < stream->bytes_written ||
00089             stream->bytes_written + count > stream->max_size)
00090         {
00091             PB_RETURN_ERROR(stream, "stream full");
00092         }
00093     }
00094     #ifndef PB_BUFFER_ONLY
00095     if (!buf_write(stream, buf, count))
00096         PB_RETURN_ERROR(stream, "io error");
00097     #else
00098     if (!stream->callback(stream, buf, count))
00099         PB_RETURN_ERROR(stream, "io error");
00100     #endif
00101 }
00102
00103 stream->bytes_written += count;
00104 return true;
00105 }
  
```

References [buf_write\(\)](#), [checkreturn](#), [NULL](#), and [PB_RETURN_ERROR](#).

Referenced by [encode_array\(\)](#), [pb_encode_ex\(\)](#), [pb_encode_fixed32\(\)](#), [pb_encode_fixed64\(\)](#), [pb_encode_string\(\)](#), [pb_encode_submessage\(\)](#), [pb_encode_varint\(\)](#), and [pb_encode_varint_32\(\)](#).

Here is the call graph for this function:


```

00048     size_t max_size;
00049
00050     /* Number of bytes written so far. */
00051     size_t bytes_written;
00052
00053 #ifndef PB_NO_ERRMSG
00054     /* Pointer to constant (ROM) string when decoding function returns error */
00055     const char *errmsg;
00056 #endif
00057 };
00058
00059 /*****
00060  * Main encoding functions *
00061  *****/
00062
00063 /* Encode a single protocol buffers message from C structure into a stream.
00064  * Returns true on success, false on any failure.
00065  * The actual struct pointed to by src_struct must match the description in fields.
00066  * All required fields in the struct are assumed to have been filled in.
00067  *
00068  * Example usage:
00069  *   MyMessage msg = {};
00070  *   uint8_t buffer[64];
00071  *   pb_ostream_t stream;
00072  *
00073  *   msg.field1 = 42;
00074  *   stream = pb_ostream_from_buffer(buffer, sizeof(buffer));
00075  *   pb_encode(&stream, MyMessage_fields, &msg);
00076  */
00077 bool pb_encode(pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct);
00078
00079 /* Extended version of pb_encode, with several options to control the
00080  * encoding process:
00081  *
00082  * PB_ENCODE_DELIMITED: Prepend the length of message as a varint.
00083  *                       Corresponds to writeDelimitedTo() in Google's
00084  *                       protobuf API.
00085  *
00086  * PB_ENCODE_NULLTERMINATED: Append a null byte to the message for termination.
00087  *                           NOTE: This behaviour is not supported in most other
00088  *                           protobuf implementations, so PB_ENCODE_DELIMITED
00089  *                           is a better option for compatibility.
00090  */
00091 #define PB_ENCODE_DELIMITED 0x02U
00092 #define PB_ENCODE_NULLTERMINATED 0x04U
00093 bool pb_encode_ex(pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct, unsigned int flags);
00094
00095 /* Defines for backwards compatibility with code written before nanopb-0.4.0 */
00096 #define pb_encode_delimited(s,f,d) pb_encode_ex(s,f,d, PB_ENCODE_DELIMITED)
00097 #define pb_encode_nullterminated(s,f,d) pb_encode_ex(s,f,d, PB_ENCODE_NULLTERMINATED)
00098
00099 /* Encode the message to get the size of the encoded data, but do not store
00100  * the data. */
00101 bool pb_get_encoded_size(size_t *size, const pb_msgdesc_t *fields, const void *src_struct);
00102
00103 /*****
00104  * Functions for manipulating streams *
00105  *****/
00106
00107 /* Create an output stream for writing into a memory buffer.
00108  * The number of bytes written can be found in stream.bytes_written after
00109  * encoding the message.
00110  *
00111  * Alternatively, you can use a custom stream that writes directly to e.g.
00112  * a file or a network socket.
00113  */
00114 pb_ostream_t pb_ostream_from_buffer(pb_byte_t *buf, size_t bufsize);
00115
00116 /* Pseudo-stream for measuring the size of a message without actually storing
00117  * the encoded data.
00118  *
00119  * Example usage:
00120  *   MyMessage msg = {};
00121  *   pb_ostream_t stream = PB_OSTREAM_SIZING;
00122  *   pb_encode(&stream, MyMessage_fields, &msg);
00123  *   printf("Message size is %d\n", stream.bytes_written);
00124  */
00125 #ifndef PB_NO_ERRMSG
00126 #define PB_OSTREAM_SIZING {0,0,0,0}
00127 #else
00128 #define PB_OSTREAM_SIZING {0,0,0,0}
00129 #endif
00130
00131 /* Function to write into a pb_ostream_t stream. You can use this if you need
00132  * to append or prepend some custom headers to the message.
00133  */
00134 bool pb_write(pb_ostream_t *stream, const pb_byte_t *buf, size_t count);

```

```

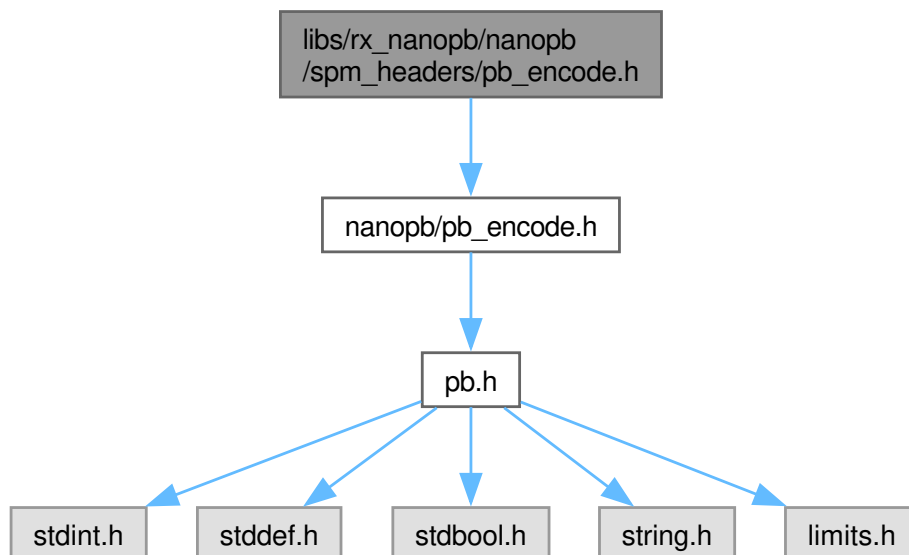
00135
00136
00137 /*****
00138  * Helper functions for writing field callbacks *
00139  *****/
00140
00141 /* Encode field header based on type and field number defined in the field
00142  * structure. Call this from the callback before writing out field contents. */
00143 bool pb_encode_tag_for_field(pb_ostream_t *stream, const pb_field_iter_t *field);
00144
00145 /* Encode field header by manually specifying wire type. You need to use this
00146  * if you want to write out packed arrays from a callback field. */
00147 bool pb_encode_tag(pb_ostream_t *stream, pb_wire_type_t wiretype, uint32_t field_number);
00148
00149 /* Encode an integer in the varint format.
00150  * This works for bool, enum, int32, int64, uint32 and uint64 field types. */
00151 #ifndef PB_WITHOUT_64BIT
00152 bool pb_encode_varint(pb_ostream_t *stream, uint64_t value);
00153 #else
00154 bool pb_encode_varint(pb_ostream_t *stream, uint32_t value);
00155 #endif
00156
00157 /* Encode an integer in the zig-zagged svarint format.
00158  * This works for sint32 and sint64. */
00159 #ifndef PB_WITHOUT_64BIT
00160 bool pb_encode_svarint(pb_ostream_t *stream, int64_t value);
00161 #else
00162 bool pb_encode_svarint(pb_ostream_t *stream, int32_t value);
00163 #endif
00164
00165 /* Encode a string or bytes type field. For strings, pass strlen(s) as size. */
00166 bool pb_encode_string(pb_ostream_t *stream, const pb_byte_t *buffer, size_t size);
00167
00168 /* Encode a fixed32, sfixed32 or float value.
00169  * You need to pass a pointer to a 4-byte wide C variable. */
00170 bool pb_encode_fixed32(pb_ostream_t *stream, const void *value);
00171
00172 #ifndef PB_WITHOUT_64BIT
00173 /* Encode a fixed64, sfixed64 or double value.
00174  * You need to pass a pointer to a 8-byte wide C variable. */
00175 bool pb_encode_fixed64(pb_ostream_t *stream, const void *value);
00176 #endif
00177
00178 #ifdef PB_CONVERT_DOUBLE_FLOAT
00179 /* Encode a float value so that it appears like a double in the encoded
00180  * message. */
00181 bool pb_encode_float_as_double(pb_ostream_t *stream, float value);
00182 #endif
00183
00184 /* Encode a submessage field.
00185  * You need to pass the pb_field_t array and pointer to struct, just like
00186  * with pb_encode(). This internally encodes the submessage twice, first to
00187  * calculate message size and then to actually write it out.
00188  */
00189 bool pb_encode_submessage(pb_ostream_t *stream, const pb_msgdesc_t *fields, const void *src_struct);
00190
00191 #ifdef __cplusplus
00192 } /* extern "C" */
00193 #endif
00194
00195 #endif

```

6.77 libs/rx_nanopb/nanopb/spm_headers/pb_encode.h File Reference

#include "nanopb/pb_encode.h"

Include dependency graph for pb_encode.h:



6.78 pb_encode.h

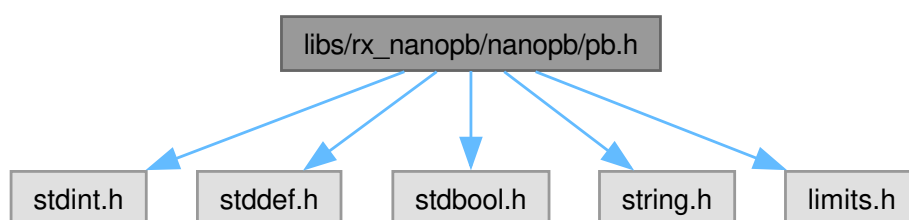
[Go to the documentation of this file.](#)

```
00001 #include "nanopb/pb_encode.h"
```

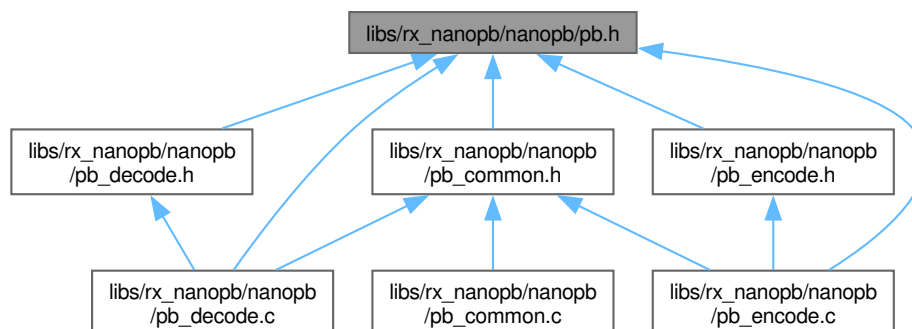
6.79 libs/rx_nanopb/nanopb/pb.h File Reference

```
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>
#include <string.h>
#include <limits.h>
```

Include dependency graph for `pb.h`:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [pb_msgdesc_t](#)
- struct [pb_field_iter_t](#)
- struct [pb_bytes_array_t](#)
- struct [pb_callback_t](#)
- struct [pb_extension_type_t](#)
- struct [pb_extension_t](#)
- union [pb_callback_s.funcs](#)

Macros

- [#define NANOPB_VERSION](#) "nanopb-1.0.0-dev"
- [#define PB_PACKED_STRUCT_START](#)
- [#define PB_PACKED_STRUCT_END](#)
- [#define pb_packed](#)
- [#define pb_noinline](#)
- [#define PB_UNUSED\(x\)](#)
- [#define PB_PROGMEM](#)
- [#define PB_PROGMEM_READU32\(x\)](#)
- [#define PB_STATIC_ASSERT\(COND, MSG\)](#)
- [#define PB_MAX_REQUIRED_FIELDS](#) 64
- [#define PB_LTYPE_BOOL](#) 0x00U /* bool */
- [#define PB_LTYPE_VARINT](#) 0x01U /* int32, int64, enum, bool */
- [#define PB_LTYPE_UVARINT](#) 0x02U /* uint32, uint64 */
- [#define PB_LTYPE_SVARINT](#) 0x03U /* sint32, sint64 */
- [#define PB_LTYPE_FIXED32](#) 0x04U /* fixed32, sfixed32, float */
- [#define PB_LTYPE_FIXED64](#) 0x05U /* fixed64, sfixed64, double */
- [#define PB_LTYPE_LAST_PACKABLE](#) 0x05U
- [#define PB_LTYPE_BYTES](#) 0x06U
- [#define PB_LTYPE_STRING](#) 0x07U
- [#define PB_LTYPE_SUBMESSAGE](#) 0x08U
- [#define PB_LTYPE_SUBMSG_W_CB](#) 0x09U
- [#define PB_LTYPE_EXTENSION](#) 0x0AU
- [#define PB_LTYPE_FIXED_LENGTH_BYTES](#) 0x0BU
- [#define PB_LTYPES_COUNT](#) 0x0CU
- [#define PB_LTYPE_MASK](#) 0x0FU
- [#define PB_HTYPE_REQUIRED](#) 0x00U
- [#define PB_HTYPE_OPTIONAL](#) 0x10U
- [#define PB_HTYPE_SINGULAR](#) 0x10U
- [#define PB_HTYPE_REPEATED](#) 0x20U
- [#define PB_HTYPE_FIXARRAY](#) 0x20U
- [#define PB_HTYPE_ONEOF](#) 0x30U
- [#define PB_HTYPE_MASK](#) 0x30U
- [#define PB_ATYPE_STATIC](#) 0x00U
- [#define PB_ATYPE_POINTER](#) 0x80U
- [#define PB_ATYPE_CALLBACK](#) 0x40U
- [#define PB_ATYPE_MASK](#) 0xC0U
- [#define PB_ATYPE\(x\)](#)
- [#define PB_HTYPE\(x\)](#)
- [#define PB_LTYPE\(x\)](#)
- [#define PB_LTYPE_IS_SUBMSG\(x\)](#)
- [#define PB_SIZE_MAX](#) ((pb_size_t)-1)
- [#define PB_BYTES_ARRAY_T\(n\)](#)

- #define [PB_BYTES_ARRAY_T_ALLOCSIZE](#)(n)
- #define [pb_extension_init_zero](#) {NULL,NULL,NULL,false}
- #define [PB_PROTO_HEADER_VERSION](#) 40
- #define [pb_membersize](#)(st, m)
- #define [pb_arraysize](#)(st, m)
- #define [pb_delta](#)(st, m1, m2)
- #define [PB_EXPAND](#)(x)
- #define [PB_BIND](#)(msgname, structname, width)
- #define [PB_GEN_FIELD_COUNT](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_GEN_REQ_FIELD_COUNT](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_GEN_LARGEST_TAG](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_GEN_FIELD_INFO_1](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_GEN_FIELD_INFO_2](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_GEN_FIELD_INFO_4](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_GEN_FIELD_INFO_8](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_GEN_FIELD_INFO_AUTO](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_FIELDINFO_AUTO2](#)(width, tag, type, data_offset, data_size, size_offset, array_size)
- #define [PB_FIELDINFO_AUTO3](#)(width, tag, type, data_offset, data_size, size_offset, array_size)
- #define [PB_GEN_FIELD_INFO_ASSERT_1](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_GEN_FIELD_INFO_ASSERT_2](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_GEN_FIELD_INFO_ASSERT_4](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_GEN_FIELD_INFO_ASSERT_8](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_GEN_FIELD_INFO_ASSERT_AUTO](#)(structname, atype, htype, ltype, fieldname, tag)
- #define [PB_FIELDINFO_ASSERT_AUTO2](#)(width, tag, type, data_offset, data_size, size_offset, array_size)
- #define [PB_FIELDINFO_ASSERT_AUTO3](#)(width, tag, type, data_offset, data_size, size_offset, array_size)
- #define [PB_DATA_OFFSET_STATIC](#)(htype, structname, fieldname)
- #define [PB_DATA_OFFSET_POINTER](#)(htype, structname, fieldname)
- #define [PB_DATA_OFFSET_CALLBACK](#)(htype, structname, fieldname)
- #define [PB_DO_PB_HTYPE_REQUIRED](#)(structname, fieldname)
- #define [PB_DO_PB_HTYPE_SINGULAR](#)(structname, fieldname)
- #define [PB_DO_PB_HTYPE_ONEOF](#)(structname, fieldname)
- #define [PB_DO_PB_HTYPE_OPTIONAL](#)(structname, fieldname)
- #define [PB_DO_PB_HTYPE_REPEATED](#)(structname, fieldname)
- #define [PB_DO_PB_HTYPE_FIXARRAY](#)(structname, fieldname)
- #define [PB_SIZE_OFFSET_STATIC](#)(htype, structname, fieldname)
- #define [PB_SIZE_OFFSET_POINTER](#)(htype, structname, fieldname)
- #define [PB_SIZE_OFFSET_CALLBACK](#)(htype, structname, fieldname)
- #define [PB_SO_PB_HTYPE_REQUIRED](#)(structname, fieldname)
- #define [PB_SO_PB_HTYPE_SINGULAR](#)(structname, fieldname)
- #define [PB_SO_PB_HTYPE_ONEOF](#)(structname, fieldname)
- #define [PB_SO_PB_HTYPE_ONEOF2](#)(structname, fullname, unionname)
- #define [PB_SO_PB_HTYPE_ONEOF3](#)(structname, fullname, unionname)
- #define [PB_SO_PB_HTYPE_OPTIONAL](#)(structname, fieldname)
- #define [PB_SO_PB_HTYPE_REPEATED](#)(structname, fieldname)
- #define [PB_SO_PB_HTYPE_FIXARRAY](#)(structname, fieldname)
- #define [PB_SO_PTR_PB_HTYPE_REQUIRED](#)(structname, fieldname)
- #define [PB_SO_PTR_PB_HTYPE_SINGULAR](#)(structname, fieldname)
- #define [PB_SO_PTR_PB_HTYPE_ONEOF](#)(structname, fieldname)
- #define [PB_SO_PTR_PB_HTYPE_OPTIONAL](#)(structname, fieldname)
- #define [PB_SO_PTR_PB_HTYPE_REPEATED](#)(structname, fieldname)

- `#define PB_SO_PTR_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_ARRAY_SIZE_STATIC(htype, structname, fieldname)`
- `#define PB_ARRAY_SIZE_POINTER(htype, structname, fieldname)`
- `#define PB_ARRAY_SIZE_CALLBACK(htype, structname, fieldname)`
- `#define PB_AS_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_AS_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_AS_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_AS_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_AS_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_AS_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_DATA_SIZE_STATIC(htype, structname, fieldname)`
- `#define PB_DATA_SIZE_POINTER(htype, structname, fieldname)`
- `#define PB_DATA_SIZE_CALLBACK(htype, structname, fieldname)`
- `#define PB_DS_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_DS_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_DS_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_DS_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_DS_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_DS_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_ONEOF_NAME(type, tuple)`
- `#define PB_ONEOF_NAME_UNION(unionname, membername, fullname)`
- `#define PB_ONEOF_NAME_MEMBER(unionname, membername, fullname)`
- `#define PB_ONEOF_NAME_FULL(unionname, membername, fullname)`
- `#define PB_GEN_SUBMSG_INFO(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_SUBMSG_INFO_REQUIRED(ltype, structname, fieldname)`
- `#define PB_SUBMSG_INFO_SINGULAR(ltype, structname, fieldname)`
- `#define PB_SUBMSG_INFO_OPTIONAL(ltype, structname, fieldname)`
- `#define PB_SUBMSG_INFO_ONEOF(ltype, structname, fieldname)`
- `#define PB_SUBMSG_INFO_ONEOF2(ltype, structname, unionname, membername)`
- `#define PB_SUBMSG_INFO_ONEOF3(ltype, structname, unionname, membername)`
- `#define PB_SUBMSG_INFO_REPEATED(ltype, structname, fieldname)`

- `#define PB_SUBMSG_INFO_FIXARRAY(ltype, structname, fieldname)`
- `#define PB_SI_PB_LTYPE_BOOL(t)`
- `#define PB_SI_PB_LTYPE_BYTES(t)`
- `#define PB_SI_PB_LTYPE_DOUBLE(t)`
- `#define PB_SI_PB_LTYPE_ENUM(t)`
- `#define PB_SI_PB_LTYPE_UENUM(t)`
- `#define PB_SI_PB_LTYPE_FIXED32(t)`
- `#define PB_SI_PB_LTYPE_FIXED64(t)`
- `#define PB_SI_PB_LTYPE_FLOAT(t)`
- `#define PB_SI_PB_LTYPE_INT32(t)`
- `#define PB_SI_PB_LTYPE_INT64(t)`
- `#define PB_SI_PB_LTYPE_MESSAGE(t)`
- `#define PB_SI_PB_LTYPE_MSG_W_CB(t)`
- `#define PB_SI_PB_LTYPE_SFIXED32(t)`
- `#define PB_SI_PB_LTYPE_SFIXED64(t)`
- `#define PB_SI_PB_LTYPE_SINT32(t)`
- `#define PB_SI_PB_LTYPE_SINT64(t)`
- `#define PB_SI_PB_LTYPE_STRING(t)`
- `#define PB_SI_PB_LTYPE_UINT32(t)`
- `#define PB_SI_PB_LTYPE_UINT64(t)`
- `#define PB_SI_PB_LTYPE_EXTENSION(t)`
- `#define PB_SI_PB_LTYPE_FIXED_LENGTH_BYTES(t)`
- `#define PB_SUBMSG_DESCRIPTOR(t)`
- `#define PB_FIELDINFO_1(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_2(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_4(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_8(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FITS(value, bits)`
- `#define PB_FIELDINFO_ASSERT_1(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_ASSERT_2(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_ASSERT_4(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_ASSERT_8(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_WIDTH_AUTO(atype, htype, ltype)`
- `#define PB_FI_WIDTH_PB_ATYPE_STATIC(htype, ltype)`
- `#define PB_FI_WIDTH_PB_ATYPE_POINTER(htype, ltype)`
- `#define PB_FI_WIDTH_PB_ATYPE_CALLBACK(htype, ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_REQUIRED(ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_SINGULAR(ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_OPTIONAL(ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_ONEOF(ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_REPEATED(ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_FIXARRAY(ltype)`
- `#define PB_FI_WIDTH_PB_LTYPE_BOOL 1`
- `#define PB_FI_WIDTH_PB_LTYPE_BYTES 2`
- `#define PB_FI_WIDTH_PB_LTYPE_DOUBLE 1`
- `#define PB_FI_WIDTH_PB_LTYPE_ENUM 1`
- `#define PB_FI_WIDTH_PB_LTYPE_UENUM 1`
- `#define PB_FI_WIDTH_PB_LTYPE_FIXED32 1`
- `#define PB_FI_WIDTH_PB_LTYPE_FIXED64 1`
- `#define PB_FI_WIDTH_PB_LTYPE_FLOAT 1`
- `#define PB_FI_WIDTH_PB_LTYPE_INT32 1`
- `#define PB_FI_WIDTH_PB_LTYPE_INT64 1`
- `#define PB_FI_WIDTH_PB_LTYPE_MESSAGE 2`
- `#define PB_FI_WIDTH_PB_LTYPE_MSG_W_CB 2`
- `#define PB_FI_WIDTH_PB_LTYPE_SFIXED32 1`

- `#define PB_FI_WIDTH_PB_LTYPE_SFIXED64 1`
- `#define PB_FI_WIDTH_PB_LTYPE_SINT32 1`
- `#define PB_FI_WIDTH_PB_LTYPE_SINT64 1`
- `#define PB_FI_WIDTH_PB_LTYPE_STRING 2`
- `#define PB_FI_WIDTH_PB_LTYPE_UINT32 1`
- `#define PB_FI_WIDTH_PB_LTYPE_UINT64 1`
- `#define PB_FI_WIDTH_PB_LTYPE_EXTENSION 1`
- `#define PB_FI_WIDTH_PB_LTYPE_FIXED_LENGTH_BYTES 2`
- `#define PB_LTYPE_MAP_BOOL PB_LTYPE_BOOL`
- `#define PB_LTYPE_MAP_BYTES PB_LTYPE_BYTES`
- `#define PB_LTYPE_MAP_DOUBLE PB_LTYPE_FIXED64`
- `#define PB_LTYPE_MAP_ENUM PB_LTYPE_VARINT`
- `#define PB_LTYPE_MAP_UENUM PB_LTYPE_UVARINT`
- `#define PB_LTYPE_MAP_FIXED32 PB_LTYPE_FIXED32`
- `#define PB_LTYPE_MAP_FIXED64 PB_LTYPE_FIXED64`
- `#define PB_LTYPE_MAP_FLOAT PB_LTYPE_FIXED32`
- `#define PB_LTYPE_MAP_INT32 PB_LTYPE_VARINT`
- `#define PB_LTYPE_MAP_INT64 PB_LTYPE_VARINT`
- `#define PB_LTYPE_MAP_MESSAGE PB_LTYPE_SUBMESSAGE`
- `#define PB_LTYPE_MAP_MSG_W_CB PB_LTYPE_SUBMSG_W_CB`
- `#define PB_LTYPE_MAP_SFIXED32 PB_LTYPE_FIXED32`
- `#define PB_LTYPE_MAP_SFIXED64 PB_LTYPE_FIXED64`
- `#define PB_LTYPE_MAP_SINT32 PB_LTYPE_SVARINT`
- `#define PB_LTYPE_MAP_SINT64 PB_LTYPE_SVARINT`
- `#define PB_LTYPE_MAP_STRING PB_LTYPE_STRING`
- `#define PB_LTYPE_MAP_UINT32 PB_LTYPE_UVARINT`
- `#define PB_LTYPE_MAP_UINT64 PB_LTYPE_UVARINT`
- `#define PB_LTYPE_MAP_EXTENSION PB_LTYPE_EXTENSION`
- `#define PB_LTYPE_MAP_FIXED_LENGTH_BYTES PB_LTYPE_FIXED_LENGTH_BYTES`
- `#define PB_SET_ERROR(stream, msg)`
- `#define PB_GET_ERROR(stream)`
- `#define PB_RETURN_ERROR(stream, msg)`

Typedefs

- `typedef uint_least8_t pb_byte_t`
- `typedef pb_byte_t pb_type_t`
- `typedef uint_least16_t pb_size_t`
- `typedef int_least16_t pb_ssize_t`
- `typedef pb_field_iter_t pb_field_t`

Enumerations

- `enum pb_wire_type_t {`
`PB_WT_VARINT = 0 , PB_WT_64BIT = 1 , PB_WT_STRING = 2 , PB_WT_32BIT = 5 ,`
`PB_WT_PACKED = 255 }`

Functions

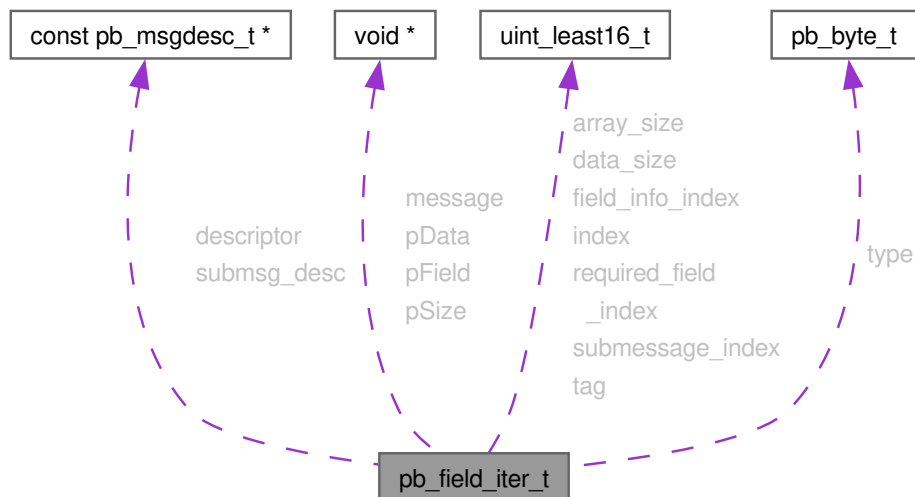
- `bool pb_default_field_callback (pb_istream_t *istream, pb_ostream_t *ostream, const pb_field_t *field)`

6.79.1 Data Structure Documentation

6.79.1.1 struct pb_field_iter_s

Definition at line 363 of file [pb.h](#).

Collaboration diagram for pb_field_iter_t:



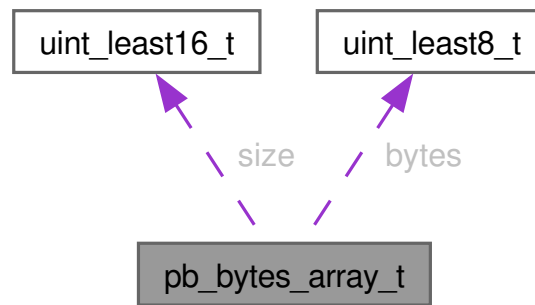
Data Fields

pb_size_t	<code>array_size</code>	
pb_size_t	<code>data_size</code>	
<code>const pb_msgdesc_t *</code>	<code>descriptor</code>	
pb_size_t	<code>field_info_index</code>	
pb_size_t	<code>index</code>	
<code>void *</code>	<code>message</code>	
<code>void *</code>	<code>pData</code>	
<code>void *</code>	<code>pField</code>	
<code>void *</code>	<code>pSize</code>	
pb_size_t	<code>required_field_index</code>	
pb_size_t	<code>submessage_index</code>	
<code>const pb_msgdesc_t *</code>	<code>submsg_desc</code>	
pb_size_t	<code>tag</code>	
pb_type_t	<code>type</code>	

6.79.1.2 struct pb_bytes_array_s

Definition at line 405 of file [pb.h](#).

Collaboration diagram for pb_bytes_array_t:



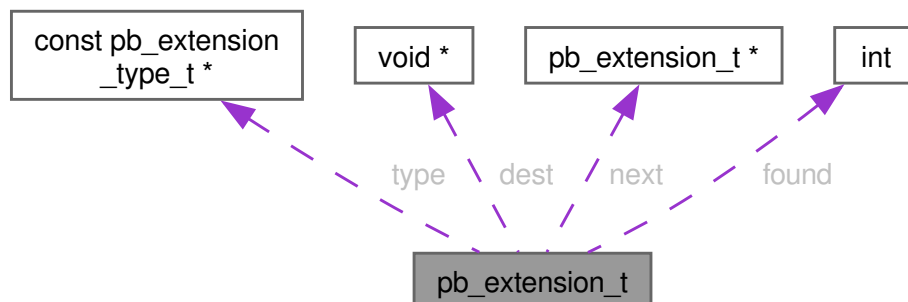
Data Fields

pb_byte_t	bytes[1]	
pb_size_t	size	

6.79.1.3 struct pb_extension_s

Definition at line 484 of file [pb.h](#).

Collaboration diagram for `pb_extension_t`:



Data Fields

	<code>void *</code>	<code>dest</code>	
bool		<code>found</code>	
<code>pb_extension_t *</code>		<code>next</code>	
<code>const pb_extension_type_t *</code>		<code>type</code>	

6.79.2 Macro Definition Documentation

6.79.2.1 NANOPB_VERSION

```
#define NANOPB_VERSION "nanopb-1.0.0-dev"
```

Definition at line 71 of file [pb.h](#).

6.79.2.2 PB_ARRAY_SIZE_CALLBACK

```
#define PB_ARRAY_SIZE_CALLBACK(  
    htype,  
    structname,  
    fieldname)
```

Value:

1

Definition at line 684 of file [pb.h](#).

6.79.2.3 PB_ARRAY_SIZE_POINTER

```
#define PB_ARRAY_SIZE_POINTER(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_AS_PTR ## htype(structname, fieldname)

Definition at line 683 of file [pb.h](#).

6.79.2.4 PB_ARRAY_SIZE_STATIC

```
#define PB_ARRAY_SIZE_STATIC(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_AS ## htype(structname, fieldname)

Definition at line 682 of file [pb.h](#).

6.79.2.5 pb_arraysize

```
#define pb_arraysize(  
    st,  
    m)
```

Value:

(pb_membersize(st, m) / pb_membersize(st, m[0]))

Definition at line 523 of file [pb.h](#).

6.79.2.6 PB_AS_PB_HTYPE_FIXARRAY

```
#define PB_AS_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

`pb_arraysize(structname, fieldname)`

Definition at line 690 of file [pb.h](#).

6.79.2.7 PB_AS_PB_HTYPE_ONEOF

```
#define PB_AS_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 688 of file [pb.h](#).

6.79.2.8 PB_AS_PB_HTYPE_OPTIONAL

```
#define PB_AS_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 687 of file [pb.h](#).

6.79.2.9 PB_AS_PB_HTYPE_REPEATED

```
#define PB_AS_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

`pb_arraysize(structname, fieldname)`

Definition at line 689 of file [pb.h](#).

6.79.2.10 PB_AS_PB_HTYPE_REQUIRED

```
#define PB_AS_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 685 of file [pb.h](#).

6.79.2.11 PB_AS_PB_HTYPE_SINGULAR

```
#define PB_AS_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 686 of file [pb.h](#).

6.79.2.12 PB_AS_PTR_PB_HTYPE_FIXARRAY

```
#define PB_AS_PTR_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

pb_arraysize(structname, fieldname[0])

Definition at line 696 of file [pb.h](#).

6.79.2.13 PB_AS_PTR_PB_HTYPE_ONEOF

```
#define PB_AS_PTR_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 694 of file [pb.h](#).

6.79.2.14 PB_AS_PTR_PB_HTYPE_OPTIONAL

```
#define PB_AS_PTR_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 693 of file [pb.h](#).

6.79.2.15 PB_AS_PTR_PB_HTYPE_REPEATED

```
#define PB_AS_PTR_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 695 of file [pb.h](#).

6.79.2.16 PB_AS_PTR_PB_HTYPE_REQUIRED

```
#define PB_AS_PTR_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 691 of file [pb.h](#).

6.79.2.17 PB_AS_PTR_PB_HTYPE_SINGULAR

```
#define PB_AS_PTR_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 692 of file [pb.h](#).

6.79.2.18 PB_ATYPE

```
#define PB_ATYPE(  
    x)
```

Value:

((x) & PB_ATYPE_MASK)

Definition at line 323 of file [pb.h](#).

Referenced by [decode_field\(\)](#), [encode_array\(\)](#), [encode_field\(\)](#), [load_descriptor_values\(\)](#), [pb_check_proto3_default_value\(\)](#), [pb_dec_bytes\(\)](#), [pb_dec_string\(\)](#), [pb_dec_submessage\(\)](#), [pb_enc_bytes\(\)](#), [pb_enc_string\(\)](#), [pb_field_iter_begin_extension\(\)](#), and [pb_field_set_to_default\(\)](#).

6.79.2.19 PB_ATYPE_CALLBACK

```
#define PB_ATYPE_CALLBACK 0x40U
```

Definition at line 320 of file [pb.h](#).

Referenced by [decode_field\(\)](#), [encode_field\(\)](#), [pb_check_proto3_default_value\(\)](#), and [pb_field_set_to_default\(\)](#).

6.79.2.20 PB_ATYPE_MASK

```
#define PB_ATYPE_MASK 0xC0U
```

Definition at line 321 of file [pb.h](#).

6.79.2.21 PB_ATYPE_POINTER

```
#define PB_ATYPE_POINTER 0x80U
```

Definition at line 319 of file [pb.h](#).

Referenced by [decode_field\(\)](#), [encode_array\(\)](#), [load_descriptor_values\(\)](#), [pb_check_proto3_default_value\(\)](#), [pb_dec_bytes\(\)](#), [pb_dec_string\(\)](#), [pb_enc_string\(\)](#), [pb_field_iter_begin_extension\(\)](#), and [pb_field_set_to_default\(\)](#).

6.79.2.22 PB_ATYPE_STATIC

```
#define PB_ATYPE_STATIC 0x00U
```

Definition at line 318 of file [pb.h](#).

Referenced by [decode_field\(\)](#), [encode_field\(\)](#), [load_descriptor_values\(\)](#), [pb_check_proto3_default_value\(\)](#), [pb_dec_submessage\(\)](#), [pb_enc_bytes\(\)](#), and [pb_field_set_to_default\(\)](#).

6.79.2.23 PB_BIND

```
#define PB_BIND(  
    msgname,  
    structname,  
    width)
```

Value:

```
const uint32_t structname##_field_info[] PB_PROGMEM = \
{ \
    msgname##_FIELDLIST(PB_GEN_FIELD_INFO_ ## width, structname) \
    0 \
}; \
const pb_msgdesc_t* const structname##_submsg_info[] = \
{ \
    msgname##_FIELDLIST(PB_GEN_SUBMSG_INFO, structname) \
    NULL \
}; \
const pb_msgdesc_t structname##_msg = \
{ \
    structname##_field_info, \
    structname##_submsg_info, \
    msgname##_DEFAULT, \
    msgname##_CALLBACK, \
    0 msgname##_FIELDLIST(PB_GEN_FIELD_COUNT, structname), \
    0 msgname##_FIELDLIST(PB_GEN_REQ_FIELD_COUNT, structname), \
    0 msgname##_FIELDLIST(PB_GEN_LARGEST_TAG, structname), \
}; \
msgname##_FIELDLIST(PB_GEN_FIELD_INFO_ASSERT_ ## width, structname)
```

Definition at line 531 of file [pb.h](#).

```
00531 #define PB_BIND(msgname, structname, width) \
00532     const uint32_t structname##_field_info[] PB_PROGMEM = \
00533     { \
00534         msgname##_FIELDLIST(PB_GEN_FIELD_INFO_ ## width, structname) \
00535         0 \
00536     }; \
00537     const pb_msgdesc_t* const structname##_submsg_info[] = \
00538     { \
00539         msgname##_FIELDLIST(PB_GEN_SUBMSG_INFO, structname) \
00540         NULL \
00541     }; \
00542     const pb_msgdesc_t structname##_msg = \
00543     { \
00544         structname##_field_info, \
00545         structname##_submsg_info, \
00546         msgname##_DEFAULT, \
00547         msgname##_CALLBACK, \
00548         0 msgname##_FIELDLIST(PB_GEN_FIELD_COUNT, structname), \
00549         0 msgname##_FIELDLIST(PB_GEN_REQ_FIELD_COUNT, structname), \
00550         0 msgname##_FIELDLIST(PB_GEN_LARGEST_TAG, structname), \
00551     }; \
00552     msgname##_FIELDLIST(PB_GEN_FIELD_INFO_ASSERT_ ## width, structname)
```

6.79.2.24 PB_BYTES_ARRAY_T

```
#define PB_BYTES_ARRAY_T(  
    n)
```

Value:

```
struct { pb_size_t size; pb_byte_t bytes[n]; }
```

Definition at line 402 of file [pb.h](#).

6.79.2.25 PB_BYTES_ARRAY_T_ALLOCSIZE

```
#define PB_BYTES_ARRAY_T_ALLOCSIZE(  
    n)
```

Value:

```
((size_t)n + offsetof(pb_bytes_array_t, bytes))
```

Definition at line 403 of file [pb.h](#).

Referenced by [pb_dec_bytes\(\)](#).

6.79.2.26 PB_DATA_OFFSET_CALLBACK

```
#define PB_DATA_OFFSET_CALLBACK(  
    htype,  
    structname,  
    fieldname)
```

Value:

```
PB_DO ## htype(structname, fieldname)
```

Definition at line 650 of file [pb.h](#).

6.79.2.27 PB_DATA_OFFSET_POINTER

```
#define PB_DATA_OFFSET_POINTER(  
    htype,  
    structname,  
    fieldname)
```

Value:

```
PB_DO ## htype(structname, fieldname)
```

Definition at line 649 of file [pb.h](#).

6.79.2.28 PB_DATA_OFFSET_STATIC

```
#define PB_DATA_OFFSET_STATIC(  
    htype,  
    structname,  
    fieldname)
```

Value:

```
PB_DO ## htype(structname, fieldname)
```

Definition at line 648 of file [pb.h](#).

6.79.2.29 PB_DATA_SIZE_CALLBACK

```
#define PB_DATA_SIZE_CALLBACK(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_DS_CB ## htype(structname, fieldname)

Definition at line 700 of file [pb.h](#).

6.79.2.30 PB_DATA_SIZE_POINTER

```
#define PB_DATA_SIZE_POINTER(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_DS_PTR ## htype(structname, fieldname)

Definition at line 699 of file [pb.h](#).

6.79.2.31 PB_DATA_SIZE_STATIC

```
#define PB_DATA_SIZE_STATIC(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_DS ## htype(structname, fieldname)

Definition at line 698 of file [pb.h](#).

6.79.2.32 pb_delta

```
#define pb_delta(  
    st,  
    m1,  
    m2)
```

Value:

((int)offsetof(st, m1) - (int)offsetof(st, m2))

Definition at line 525 of file [pb.h](#).

6.79.2.33 PB_DO_PB_HTYPE_FIXARRAY

```
#define PB_DO_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, fieldname)`

Definition at line 656 of file [pb.h](#).

6.79.2.34 PB_DO_PB_HTYPE_ONEOF

```
#define PB_DO_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, PB_ONEOF_NAME(FULL, fieldname))`

Definition at line 653 of file [pb.h](#).

6.79.2.35 PB_DO_PB_HTYPE_OPTIONAL

```
#define PB_DO_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, fieldname)`

Definition at line 654 of file [pb.h](#).

6.79.2.36 PB_DO_PB_HTYPE_REPEATED

```
#define PB_DO_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, fieldname)`

Definition at line 655 of file [pb.h](#).

6.79.2.37 PB_DO_PB_HTYPE_REQUIRED

```
#define PB_DO_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, fieldname)`

Definition at line 651 of file [pb.h](#).

6.79.2.38 PB_DO_PB_HTYPE_SINGULAR

```
#define PB_DO_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, fieldname)`

Definition at line 652 of file [pb.h](#).

6.79.2.39 PB_DS_CB_PB_HTYPE_FIXARRAY

```
#define PB_DS_CB_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname)`

Definition at line 718 of file [pb.h](#).

6.79.2.40 PB_DS_CB_PB_HTYPE_ONEOF

```
#define PB_DS_CB_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, PB_ONEOF_NAME(FULL, fieldname))`

Definition at line 716 of file [pb.h](#).

6.79.2.41 PB_DS_CB_PB_HTYPE_OPTIONAL

```
#define PB_DS_CB_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname)`

Definition at line 715 of file [pb.h](#).

6.79.2.42 PB_DS_CB_PB_HTYPE_REPEATED

```
#define PB_DS_CB_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname)`

Definition at line 717 of file [pb.h](#).

6.79.2.43 PB_DS_CB_PB_HTYPE_REQUIRED

```
#define PB_DS_CB_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname)`

Definition at line 713 of file [pb.h](#).

6.79.2.44 PB_DS_CB_PB_HTYPE_SINGULAR

```
#define PB_DS_CB_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname)`

Definition at line 714 of file [pb.h](#).

6.79.2.45 PB_DS_PB_HTYPE_FIXARRAY

```
#define PB_DS_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname[0])`

Definition at line 706 of file [pb.h](#).

6.79.2.46 PB_DS_PB_HTYPE_ONEOF

```
#define PB_DS_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, PB_ONEOF_NAME(FULL, fieldname))`

Definition at line 704 of file [pb.h](#).

6.79.2.47 PB_DS_PB_HTYPE_OPTIONAL

```
#define PB_DS_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname)`

Definition at line 703 of file [pb.h](#).

6.79.2.48 PB_DS_PB_HTYPE_REPEATED

```
#define PB_DS_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname[0])`

Definition at line 705 of file [pb.h](#).

6.79.2.49 PB_DS_PB_HTYPE_REQUIRED

```
#define PB_DS_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname)`

Definition at line 701 of file [pb.h](#).

6.79.2.50 PB_DS_PB_HTYPE_SINGULAR

```
#define PB_DS_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname)`

Definition at line 702 of file [pb.h](#).

6.79.2.51 PB_DS_PTR_PB_HTYPE_FIXARRAY

```
#define PB_DS_PTR_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname[0][0])`

Definition at line 712 of file [pb.h](#).

6.79.2.52 PB_DS_PTR_PB_HTYPE_ONEOF

```
#define PB_DS_PTR_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, PB_ONEOF_NAME(FULL, fieldname)[0])`

Definition at line 710 of file [pb.h](#).

6.79.2.53 PB_DS_PTR_PB_HTYPE_OPTIONAL

```
#define PB_DS_PTR_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname[0])`

Definition at line 709 of file [pb.h](#).

6.79.2.54 PB_DS_PTR_PB_HTYPE_REPEATED

```
#define PB_DS_PTR_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname[0])`

Definition at line 711 of file [pb.h](#).

6.79.2.55 PB_DS_PTR_PB_HTYPE_REQUIRED

```
#define PB_DS_PTR_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname[0])`

Definition at line 707 of file [pb.h](#).

6.79.2.56 PB_DS_PTR_PB_HTYPE_SINGULAR

```
#define PB_DS_PTR_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname[0])`

Definition at line 708 of file [pb.h](#).

6.79.2.57 PB_EXPAND

```
#define PB_EXPAND(  
    x)
```

Value:

`x`

Definition at line 528 of file [pb.h](#).

6.79.2.58 pb_extension_init_zero

```
#define pb_extension_init_zero {NULL,NULL,NULL,false}
```

Definition at line 503 of file [pb.h](#).

6.79.2.59 PB_FI_WIDTH_PB_ATYPE_CALLBACK

```
#define PB_FI_WIDTH_PB_ATYPE_CALLBACK(  
    htype,  
    ltype)
```

Value:

2

Definition at line 848 of file [pb.h](#).

6.79.2.60 PB_FI_WIDTH_PB_ATYPE_POINTER

```
#define PB_FI_WIDTH_PB_ATYPE_POINTER(  
    htype,  
    ltype)
```

Value:

PB_FI_WIDTH ## htype(ltype)

Definition at line 847 of file [pb.h](#).

6.79.2.61 PB_FI_WIDTH_PB_ATYPE_STATIC

```
#define PB_FI_WIDTH_PB_ATYPE_STATIC(  
    htype,  
    ltype)
```

Value:

PB_FI_WIDTH ## htype(ltype)

Definition at line 846 of file [pb.h](#).

6.79.2.62 PB_FI_WIDTH_PB_HTYPE_FIXARRAY

```
#define PB_FI_WIDTH_PB_HTYPE_FIXARRAY(  
    ltype)
```

Value:

2

Definition at line 854 of file [pb.h](#).

6.79.2.63 PB_FI_WIDTH_PB_HTYPE_ONEOF

```
#define PB_FI_WIDTH_PB_HTYPE_ONEOF(  
    ltype)
```

Value:

PB_FI_WIDTH ## ltype

Definition at line 852 of file [pb.h](#).

6.79.2.64 PB_FI_WIDTH_PB_HTYPE_OPTIONAL

```
#define PB_FI_WIDTH_PB_HTYPE_OPTIONAL(  
    ltype)
```

Value:

PB_FI_WIDTH ## ltype

Definition at line 851 of file [pb.h](#).

6.79.2.65 PB_FI_WIDTH_PB_HTYPE_REPEATED

```
#define PB_FI_WIDTH_PB_HTYPE_REPEATED(  
    ltype)
```

Value:

2

Definition at line 853 of file [pb.h](#).

6.79.2.66 PB_FI_WIDTH_PB_HTYPE_REQUIRED

```
#define PB_FI_WIDTH_PB_HTYPE_REQUIRED(  
    ltype)
```

Value:

PB_FI_WIDTH ## ltype

Definition at line 849 of file [pb.h](#).

6.79.2.67 PB_FI_WIDTH_PB_HTYPE_SINGULAR

```
#define PB_FI_WIDTH_PB_HTYPE_SINGULAR(  
    ltype)
```

Value:

PB_FI_WIDTH ## ltype

Definition at line 850 of file [pb.h](#).

6.79.2.68 PB_FI_WIDTH_PB_LTYPE_BOOL

```
#define PB_FI_WIDTH_PB_LTYPE_BOOL 1
```

Definition at line 855 of file [pb.h](#).

6.79.2.69 PB_FI_WIDTH_PB_LTYPE_BYTES

```
#define PB_FI_WIDTH_PB_LTYPE_BYTES 2
```

Definition at line 856 of file [pb.h](#).

6.79.2.70 PB_FI_WIDTH_PB_LTYPE_DOUBLE

```
#define PB_FI_WIDTH_PB_LTYPE_DOUBLE 1
```

Definition at line 857 of file [pb.h](#).

6.79.2.71 PB_FI_WIDTH_PB_LTYPE_ENUM

```
#define PB_FI_WIDTH_PB_LTYPE_ENUM 1
```

Definition at line 858 of file [pb.h](#).

6.79.2.72 PB_FI_WIDTH_PB_LTYPE_EXTENSION

```
#define PB_FI_WIDTH_PB_LTYPE_EXTENSION 1
```

Definition at line 874 of file [pb.h](#).

6.79.2.73 PB_FI_WIDTH_PB_LTYPE_FIXED32

```
#define PB_FI_WIDTH_PB_LTYPE_FIXED32 1
```

Definition at line 860 of file [pb.h](#).

6.79.2.74 PB_FI_WIDTH_PB_LTYPE_FIXED64

```
#define PB_FI_WIDTH_PB_LTYPE_FIXED64 1
```

Definition at line 861 of file [pb.h](#).

6.79.2.75 PB_FI_WIDTH_PB_LTYPE_FIXED_LENGTH_BYTES

```
#define PB_FI_WIDTH_PB_LTYPE_FIXED_LENGTH_BYTES 2
```

Definition at line 875 of file [pb.h](#).

6.79.2.76 PB_FI_WIDTH_PB_LTYPE_FLOAT

```
#define PB_FI_WIDTH_PB_LTYPE_FLOAT 1
```

Definition at line 862 of file [pb.h](#).

6.79.2.77 PB_FI_WIDTH_PB_LTYPE_INT32

```
#define PB_FI_WIDTH_PB_LTYPE_INT32 1
```

Definition at line 863 of file [pb.h](#).

6.79.2.78 PB_FI_WIDTH_PB_LTYPE_INT64

```
#define PB_FI_WIDTH_PB_LTYPE_INT64 1
```

Definition at line 864 of file [pb.h](#).

6.79.2.79 PB_FI_WIDTH_PB_LTYPE_MESSAGE

```
#define PB_FI_WIDTH_PB_LTYPE_MESSAGE 2
```

Definition at line 865 of file [pb.h](#).

6.79.2.80 PB_FI_WIDTH_PB_LTYPE_MSG_W_CB

```
#define PB_FI_WIDTH_PB_LTYPE_MSG_W_CB 2
```

Definition at line 866 of file [pb.h](#).

6.79.2.81 PB_FI_WIDTH_PB_LTYPE_SFIXED32

```
#define PB_FI_WIDTH_PB_LTYPE_SFIXED32 1
```

Definition at line 867 of file [pb.h](#).

6.79.2.82 PB_FI_WIDTH_PB_LTYPE_SFIXED64

```
#define PB_FI_WIDTH_PB_LTYPE_SFIXED64 1
```

Definition at line 868 of file [pb.h](#).

6.79.2.83 PB_FI_WIDTH_PB_LTYPE_SINT32

```
#define PB_FI_WIDTH_PB_LTYPE_SINT32 1
```

Definition at line 869 of file [pb.h](#).

6.79.2.84 PB_FI_WIDTH_PB_LTYPE_SINT64

```
#define PB_FI_WIDTH_PB_LTYPE_SINT64 1
```

Definition at line 870 of file [pb.h](#).

6.79.2.85 PB_FI_WIDTH_PB_LTYPE_STRING

```
#define PB_FI_WIDTH_PB_LTYPE_STRING 2
```

Definition at line 871 of file [pb.h](#).

6.79.2.86 PB_FI_WIDTH_PB_LTYPE_UENUM

```
#define PB_FI_WIDTH_PB_LTYPE_UENUM 1
```

Definition at line 859 of file [pb.h](#).

6.79.2.87 PB_FI_WIDTH_PB_LTYPE_UINT32

```
#define PB_FI_WIDTH_PB_LTYPE_UINT32 1
```

Definition at line 872 of file [pb.h](#).

6.79.2.88 PB_FI_WIDTH_PB_LTYPE_UINT64

```
#define PB_FI_WIDTH_PB_LTYPE_UINT64 1
```

Definition at line 873 of file [pb.h](#).

6.79.2.89 PB_FIELDINFO_1

```
#define PB_FIELDINFO_1(  
    tag,  
    type,  
    data_offset,  
    data_size,  
    size_offset,  
    array_size)
```

Value:

```
(0 | (((uint32_t)(tag) << 2) & 0xFF) | ((type) << 8) | (((uint32_t)(data_offset) & 0xFF) << 16) | \  
(((uint32_t)(size_offset) & 0x0F) << 24) | (((uint32_t)(data_size) & 0x0F) << 28)),
```

Definition at line 787 of file [pb.h](#).

```
00787 #define PB_FIELDINFO_1(tag, type, data_offset, data_size, size_offset, array_size) \  
00788     (0 | (((uint32_t)(tag) << 2) & 0xFF) | ((type) << 8) | (((uint32_t)(data_offset) & 0xFF) << 16) | \  
00789     (((uint32_t)(size_offset) & 0x0F) << 24) | (((uint32_t)(data_size) & 0x0F) << 28)),
```

6.79.2.90 PB_FIELDINFO_2

```
#define PB_FIELDINFO_2(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
(1 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(array_size) & 0xFFF) « 16) | (((uint32_t)(size_offset) & 0x0F)
« 28)), \
(((uint32_t)(data_offset) & 0xFFFF) | (((uint32_t)(data_size) & 0xFFF) « 16) | (((uint32_t)(tag) & 0x3c0) « 22)),
```

Definition at line 791 of file [pb.h](#).

```
00791 #define PB_FIELDINFO_2(tag, type, data_offset, data_size, size_offset, array_size) \
00792     (1 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(array_size) & 0xFFF) « 16) | (((uint32_t)(size_offset)
& 0x0F) « 28)), \
00793     (((uint32_t)(data_offset) & 0xFFFF) | (((uint32_t)(data_size) & 0xFFF) « 16) | (((uint32_t)(tag) & 0x3c0) « 22)),
```

6.79.2.91 PB_FIELDINFO_4

```
#define PB_FIELDINFO_4(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
(2 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(array_size) & 0xFFFF) « 16)), \
((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) « 2) & 0xFFFFFFF0)), \
(data_offset), (data_size),
```

Definition at line 795 of file [pb.h](#).

```
00795 #define PB_FIELDINFO_4(tag, type, data_offset, data_size, size_offset, array_size) \
00796     (2 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(array_size) & 0xFFFF) « 16)), \
00797     ((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) « 2) & 0xFFFFFFF0)), \
00798     (data_offset), (data_size),
```

6.79.2.92 PB_FIELDINFO_8

```
#define PB_FIELDINFO_8(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
(3 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8)), \
((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) « 2) & 0xFFFFFFF0)), \
(data_offset), (data_size), (array_size), 0, 0, 0,
```

Definition at line 800 of file [pb.h](#).

```
00800 #define PB_FIELDINFO_8(tag, type, data_offset, data_size, size_offset, array_size) \
00801     (3 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8)), \
00802     ((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) « 2) & 0xFFFFFFF0)), \
00803     (data_offset), (data_size), (array_size), 0, 0, 0,
```

6.79.2.93 PB_FIELDINFO_ASSERT_1

```
#define PB_FIELDINFO_ASSERT_1(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
PB_STATIC_ASSERT(PB_FITS(tag,6) && PB_FITS(data_offset,8) && PB_FITS(size_offset,4) && PB_FITS(data_size,4)
    && PB_FITS(array_size,1), FIELDINFO_DOES_NOT_FIT_width1_field ## tag)
```

Definition at line 813 of file [pb.h](#).

```
00813 #define PB_FIELDINFO_ASSERT_1(tag, type, data_offset, data_size, size_offset, array_size) \
00814     PB_STATIC_ASSERT(PB_FITS(tag,6) && PB_FITS(data_offset,8) && PB_FITS(size_offset,4) &&
    PB_FITS(data_size,4) && PB_FITS(array_size,1), FIELDINFO_DOES_NOT_FIT_width1_field ## tag)
```

6.79.2.94 PB_FIELDINFO_ASSERT_2

```
#define PB_FIELDINFO_ASSERT_2(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
PB_STATIC_ASSERT(PB_FITS(tag,10) && PB_FITS(data_offset,16) && PB_FITS(size_offset,4) &&
    PB_FITS(data_size,12) && PB_FITS(array_size,12), FIELDINFO_DOES_NOT_FIT_width2_field ## tag)
```

Definition at line 816 of file [pb.h](#).

```
00816 #define PB_FIELDINFO_ASSERT_2(tag, type, data_offset, data_size, size_offset, array_size) \
00817     PB_STATIC_ASSERT(PB_FITS(tag,10) && PB_FITS(data_offset,16) && PB_FITS(size_offset,4) &&
    PB_FITS(data_size,12) && PB_FITS(array_size,12), FIELDINFO_DOES_NOT_FIT_width2_field ## tag)
```

6.79.2.95 PB_FIELDINFO_ASSERT_4

```
#define PB_FIELDINFO_ASSERT_4(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
PB_STATIC_ASSERT(PB_FITS(tag,16) && PB_FITS(data_offset,16) && PB_FITS((int_least8_t)size_offset,8) &&
    PB_FITS(data_size,16) && PB_FITS(array_size,16), FIELDINFO_DOES_NOT_FIT_width4_field ## tag)
```

Definition at line 821 of file [pb.h](#).

```
00821 #define PB_FIELDINFO_ASSERT_4(tag, type, data_offset, data_size, size_offset, array_size) \
00822     PB_STATIC_ASSERT(PB_FITS(tag,16) && PB_FITS(data_offset,16) && PB_FITS((int_least8_t)size_offset,8)
    && PB_FITS(data_size,16) && PB_FITS(array_size,16), FIELDINFO_DOES_NOT_FIT_width4_field ## tag)
```

6.79.2.96 PB_FIELDINFO_ASSERT_8

```
#define PB_FIELDINFO_ASSERT_8(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
PB_STATIC_ASSERT(PB_FITS(tag,16) && PB_FITS(data_offset,16) && PB_FITS((int_least8_t)size_offset,8) &&
    PB_FITS(data_size,16) && PB_FITS(array_size,16), FIELDINFO_DOES_NOT_FIT_width8_field ## tag)
```

Definition at line 824 of file [pb.h](#).

```
00824 #define PB_FIELDINFO_ASSERT_8(tag, type, data_offset, data_size, size_offset, array_size) \
00825     PB_STATIC_ASSERT(PB_FITS(tag,16) && PB_FITS(data_offset,16) && PB_FITS((int_least8_t)size_offset,8)
    && PB_FITS(data_size,16) && PB_FITS(array_size,16), FIELDINFO_DOES_NOT_FIT_width8_field ## tag)
```

6.79.2.97 PB_FIELDINFO_ASSERT_AUTO2

```
#define PB_FIELDINFO_ASSERT_AUTO2(
    width,
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
PB_FIELDINFO_ASSERT_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)
```

Definition at line 642 of file [pb.h](#).

```
00642 #define PB_FIELDINFO_ASSERT_AUTO2(width, tag, type, data_offset, data_size, size_offset, array_size) \
00643     PB_FIELDINFO_ASSERT_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)
```

6.79.2.98 PB_FIELDINFO_ASSERT_AUTO3

```
#define PB_FIELDINFO_ASSERT_AUTO3(
    width,
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
PB_FIELDINFO_ASSERT_ ## width(tag, type, data_offset, data_size, size_offset, array_size)
```

Definition at line 645 of file [pb.h](#).

```
00645 #define PB_FIELDINFO_ASSERT_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size) \
00646     PB_FIELDINFO_ASSERT_ ## width(tag, type, data_offset, data_size, size_offset, array_size)
```

6.79.2.99 PB_FIELDINFO_AUTO2

```
#define PB_FIELDINFO_AUTO2(
    width,
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

PB_FIELDINFO_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)

Definition at line 597 of file [pb.h](#).

```
00597 #define PB_FIELDINFO_AUTO2(width, tag, type, data_offset, data_size, size_offset, array_size) \
00598     PB_FIELDINFO_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)
```

6.79.2.100 PB_FIELDINFO_AUTO3

```
#define PB_FIELDINFO_AUTO3(
    width,
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

PB_FIELDINFO_ ## width(tag, type, data_offset, data_size, size_offset, array_size)

Definition at line 600 of file [pb.h](#).

```
00600 #define PB_FIELDINFO_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size) \
00601     PB_FIELDINFO_ ## width(tag, type, data_offset, data_size, size_offset, array_size)
```

6.79.2.101 PB_FIELDINFO_WIDTH_AUTO

```
#define PB_FIELDINFO_WIDTH_AUTO(
    atype,
    htype,
    ltype)
```

Value:

PB_FI_WIDTH ## atype(htype, ltype)

Definition at line 845 of file [pb.h](#).

6.79.2.102 PB_FITS

```
#define PB_FITS(
    value,
    bits)
```

Value:

((uint32_t)(value) < ((uint32_t)1«bits))

Definition at line 812 of file [pb.h](#).

6.79.2.103 PB_GEN_FIELD_COUNT

```
#define PB_GEN_FIELD_COUNT(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

+1

Definition at line 554 of file [pb.h](#).

6.79.2.104 PB_GEN_FIELD_INFO_1

```
#define PB_GEN_FIELD_INFO_1(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 561 of file [pb.h](#).

```
00561 #define PB_GEN_FIELD_INFO_1(structname, atype, htype, ltype, fieldname, tag) \
00562     PB_FIELDINFO_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00563         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00564         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00565         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00566         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.79.2.105 PB_GEN_FIELD_INFO_2

```
#define PB_GEN_FIELD_INFO_2(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 568 of file [pb.h](#).

```
00568 #define PB_GEN_FIELD_INFO_2(structname, atype, htype, ltype, fieldname, tag) \
00569     PB_FIELDINFO_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00570         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00571         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00572         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00573         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```


6.79.2.106 PB_GEN_FIELD_INFO_4

```
#define PB_GEN_FIELD_INFO_4(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 575 of file [pb.h](#).

```
00575 #define PB_GEN_FIELD_INFO_4(structname, atype, htype, ltype, fieldname, tag) \
00576     PB_FIELDINFO_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00577         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00578         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00579         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00580         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.79.2.107 PB_GEN_FIELD_INFO_8

```
#define PB_GEN_FIELD_INFO_8(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 582 of file [pb.h](#).

```
00582 #define PB_GEN_FIELD_INFO_8(structname, atype, htype, ltype, fieldname, tag) \
00583     PB_FIELDINFO_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00584         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00585         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00586         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00587         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.79.2.108 PB_GEN_FIELD_INFO_ASSERT_1

```
#define PB_GEN_FIELD_INFO_ASSERT_1(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_ASSERT_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 606 of file [pb.h](#).

```
00606 #define PB_GEN_FIELD_INFO_ASSERT_1(structname, atype, htype, ltype, fieldname, tag) \
00607     PB_FIELDINFO_ASSERT_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ##
    ltype, \
00608         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00609         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00610         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00611         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.79.2.109 PB_GEN_FIELD_INFO_ASSERT_2

```
#define PB_GEN_FIELD_INFO_ASSERT_2(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_ASSERT_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 613 of file [pb.h](#).

```
00613 #define PB_GEN_FIELD_INFO_ASSERT_2(structname, atype, htype, ltype, fieldname, tag) \
00614     PB_FIELDINFO_ASSERT_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ##
    ltype, \
00615         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00616         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00617         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00618         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.79.2.110 PB_GEN_FIELD_INFO_ASSERT_4

```
#define PB_GEN_FIELD_INFO_ASSERT_4(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_ASSERT_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 620 of file [pb.h](#).

```
00620 #define PB_GEN_FIELD_INFO_ASSERT_4(structname, atype, htype, ltype, fieldname, tag) \
00621     PB_FIELDINFO_ASSERT_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ##
    ltype, \
00622         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00623         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00624         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00625         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.79.2.111 PB_GEN_FIELD_INFO_ASSERT_8

```
#define PB_GEN_FIELD_INFO_ASSERT_8(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_ASSERT_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 627 of file [pb.h](#).

```
00627 #define PB_GEN_FIELD_INFO_ASSERT_8(structname, atype, htype, ltype, fieldname, tag) \
00628     PB_FIELDINFO_ASSERT_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ##
    ltype, \
00629         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00630         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00631         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00632         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.79.2.112 PB_GEN_FIELD_INFO_ASSERT_AUTO

```
#define PB_GEN_FIELD_INFO_ASSERT_AUTO(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_ASSERT_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_ ##
    htype, _PB_LTYPE_ ## ltype), \
    tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 634 of file [pb.h](#).

```
00634 #define PB_GEN_FIELD_INFO_ASSERT_AUTO(structname, atype, htype, ltype, fieldname, tag) \
00635     PB_FIELDINFO_ASSERT_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_
    ## htype, _PB_LTYPE_ ## ltype), \
00636         tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00637         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00638         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00639         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00640         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.79.2.113 PB_GEN_FIELD_INFO_AUTO

```
#define PB_GEN_FIELD_INFO_AUTO(
    structname,
    atype,
    htype,
    ltype,
```

```

    fieldname,
    tag)

```

Value:

```

PB_FIELDINFO_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_ ## htype,
    _PB_LTYPE_ ## ltype), \
    tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))

```

Definition at line 589 of file [pb.h](#).

```

00589 #define PB_GEN_FIELD_INFO_AUTO(structname, atype, htype, ltype, fieldname, tag) \
00590     PB_FIELDINFO_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_ ## htype,
    _PB_LTYPE_ ## ltype), \
00591         tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00592         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00593         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00594         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00595         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))

```

6.79.2.114 PB_GEN_LARGEST_TAG

```

#define PB_GEN_LARGEST_TAG(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)

```

Value:

```
* 0 + tag
```

Definition at line 557 of file [pb.h](#).

```

00557 #define PB_GEN_LARGEST_TAG(structname, atype, htype, ltype, fieldname, tag) \
00558     * 0 + tag

```

6.79.2.115 PB_GEN_REQ_FIELD_COUNT

```

#define PB_GEN_REQ_FIELD_COUNT(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)

```

Value:

```
+ (PB_HTYPE_ ## htype == PB_HTYPE_REQUIRED)
```

Definition at line 555 of file [pb.h](#).

```

00555 #define PB_GEN_REQ_FIELD_COUNT(structname, atype, htype, ltype, fieldname, tag) \
00556     + (PB_HTYPE_ ## htype == PB_HTYPE_REQUIRED)

```

6.79.2.116 PB_GEN_SUBMSG_INFO

```
#define PB_GEN_SUBMSG_INFO(  
    structname,  
    atype,  
    htype,  
    ltype,  
    fieldname,  
    tag)
```

Value:

PB_SUBMSG_INFO_ ## htype(_PB_LTYPE_ ## ltype, structname, fieldname)

Definition at line 725 of file [pb.h](#).

```
00725 #define PB_GEN_SUBMSG_INFO(structname, atype, htype, ltype, fieldname, tag) \  
00726     PB_SUBMSG_INFO_ ## htype(_PB_LTYPE_ ## ltype, structname, fieldname)
```

6.79.2.117 PB_GET_ERROR

```
#define PB_GET_ERROR(  
    stream)
```

Value:

((stream)->errmsg ? (stream)->errmsg : "(none)")

Definition at line 917 of file [pb.h](#).

Referenced by [encode_array\(\)](#).

6.79.2.118 PB_HTYPE

```
#define PB_HTYPE(  
    x)
```

Value:

((x) & PB_HTYPE_MASK)

Definition at line 324 of file [pb.h](#).

Referenced by [advance_iterator\(\)](#), [decode_field\(\)](#), [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [encode_field\(\)](#), [load_descriptor_values\(\)](#), [pb_check_proto3_default_value\(\)](#), [pb_dec_submessage\(\)](#), [pb_decode_inner\(\)](#), and [pb_field_set_to_default\(\)](#).

6.79.2.119 PB_HTYPE_FIXARRAY

```
#define PB_HTYPE_FIXARRAY 0x20U
```

Definition at line 312 of file [pb.h](#).

6.79.2.120 PB_HTYPE_MASK

```
#define PB_HTYPE_MASK 0x30U
```

Definition at line 314 of file [pb.h](#).

6.79.2.121 PB_HTYPE_ONEOF

```
#define PB_HTYPE_ONEOF 0x30U
```

Definition at line 313 of file [pb.h](#).

Referenced by [decode_field\(\)](#), [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [encode_field\(\)](#), [pb_check_proto3_default_value\(\)](#), and [pb_field_set_to_default\(\)](#).

6.79.2.122 PB_HTYPE_OPTIONAL

```
#define PB_HTYPE_OPTIONAL 0x10U
```

Definition at line 309 of file [pb.h](#).

Referenced by [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [encode_field\(\)](#), [pb_check_proto3_default_value\(\)](#), and [pb_field_set_to_default\(\)](#).

6.79.2.123 PB_HTYPE_REPEATED

```
#define PB_HTYPE_REPEATED 0x20U
```

Definition at line 311 of file [pb.h](#).

Referenced by [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [encode_field\(\)](#), [load_descriptor_values\(\)](#), [pb_check_proto3_default_value\(\)](#), [pb_dec_submessage\(\)](#), [pb_decode_inner\(\)](#), and [pb_field_set_to_default\(\)](#).

6.79.2.124 PB_HTYPE_REQUIRED

```
#define PB_HTYPE_REQUIRED 0x00U
```

Definition at line 308 of file [pb.h](#).

Referenced by [advance_iterator\(\)](#), [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [encode_field\(\)](#), [pb_check_proto3_default_value\(\)](#), and [pb_decode_inner\(\)](#).

6.79.2.125 PB_HTYPE_SINGULAR

```
#define PB_HTYPE_SINGULAR 0x10U
```

Definition at line 310 of file [pb.h](#).

6.79.2.126 PB_LTYPE

```
#define PB_LTYPE(  
    x)
```

Value:

((x) & PB_LTYPE_MASK)

Definition at line 325 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [decode_callback_field\(\)](#), [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [encode_array\(\)](#), [encode_basic_field\(\)](#), [pb_check_proto3_default_value\(\)](#), [pb_dec_submessage\(\)](#), [pb_dec_varint\(\)](#), [pb_decode_inner\(\)](#), [pb_enc_fixed\(\)](#), [pb_enc_submessage\(\)](#), [pb_enc_varint\(\)](#), [pb_encode\(\)](#), [pb_encode_tag_for_field\(\)](#), [pb_field_iter_find\(\)](#), [pb_field_iter_find_extension\(\)](#), and [pb_field_set_to_default\(\)](#).

6.79.2.127 PB_LTYPE_BOOL

```
#define PB_LTYPE_BOOL 0x00U /* bool */
```

Definition at line 265 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [encode_basic_field\(\)](#), and [pb_encode_tag_for_field\(\)](#).

6.79.2.128 PB_LTYPE_BYTES

```
#define PB_LTYPE_BYTES 0x06U
```

Definition at line 277 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [decode_pointer_field\(\)](#), [encode_array\(\)](#), [encode_basic_field\(\)](#), [pb_check_proto3_default_value\(\)](#), and [pb_encode_tag_for_field\(\)](#).

6.79.2.129 PB_LTYPE_EXTENSION

```
#define PB_LTYPE_EXTENSION 0x0AU
```

Definition at line 294 of file [pb.h](#).

Referenced by [pb_check_proto3_default_value\(\)](#), [pb_decode_inner\(\)](#), [pb_encode\(\)](#), [pb_field_iter_find\(\)](#), [pb_field_iter_find_extension\(\)](#), and [pb_field_set_to_default\(\)](#).

6.79.2.130 PB_LTYPE_FIXED32

```
#define PB_LTYPE_FIXED32 0x04U /* fixed32, sfixed32, float */
```

Definition at line 269 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [encode_array\(\)](#), [encode_basic_field\(\)](#), and [pb_encode_tag_for_field\(\)](#).

6.79.2.131 PB_LTYPE_FIXED64

```
#define PB_LTYPE_FIXED64 0x05U /* fixed64, sfixed64, double */
```

Definition at line 270 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [encode_array\(\)](#), [encode_basic_field\(\)](#), [pb_enc_fixed\(\)](#), and [pb_encode_tag_for_field\(\)](#).

6.79.2.132 PB_LTYPE_FIXED_LENGTH_BYTES

```
#define PB_LTYPE_FIXED_LENGTH_BYTES 0x0BU
```

Definition at line 300 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [encode_basic_field\(\)](#), [pb_check_proto3_default_value\(\)](#), and [pb_encode_tag_for_field\(\)](#).

6.79.2.133 PB_LTYPE_IS_SUBMSG

```
#define PB_LTYPE_IS_SUBMSG(  
    x)
```

Value:

```
(PB_LTYPE(x) == PB_LTYPE_SUBMESSAGE || \  
PB_LTYPE(x) == PB_LTYPE_SUBMSG_W_CB)
```

Definition at line 326 of file [pb.h](#).

```
00326 #define PB_LTYPE_IS_SUBMSG(x) (PB_LTYPE(x) == PB_LTYPE_SUBMESSAGE || \  
00327     PB_LTYPE(x) == PB_LTYPE_SUBMSG_W_CB)
```

Referenced by [advance_iterator\(\)](#), [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [load_descriptor_values\(\)](#), [pb_check_proto3_default_value\(\)](#), and [pb_field_set_to_default\(\)](#).

6.79.2.134 PB_LTYPE_LAST_PACKABLE

```
#define PB_LTYPE_LAST_PACKABLE 0x05U
```

Definition at line 273 of file [pb.h](#).

Referenced by [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [encode_array\(\)](#), and [pb_check_proto3_default_value\(\)](#).

6.79.2.135 PB_LTYPE_MAP_BOOL

```
#define PB_LTYPE_MAP_BOOL PB\_LTYPE\_BOOL
```

Definition at line 878 of file [pb.h](#).

6.79.2.136 PB_LTYPE_MAP_BYTES

```
#define PB_LTYPE_MAP_BYTES PB\_LTYPE\_BYTES
```

Definition at line 879 of file [pb.h](#).

6.79.2.137 PB_LTYPE_MAP_DOUBLE

```
#define PB_LTYPE_MAP_DOUBLE PB\_LTYPE\_FIXED64
```

Definition at line 880 of file [pb.h](#).

6.79.2.138 PB_LTYPE_MAP_ENUM

```
#define PB_LTYPE_MAP_ENUM PB\_LTYPE\_VARINT
```

Definition at line 881 of file [pb.h](#).

6.79.2.139 PB_LTYPE_MAP_EXTENSION

```
#define PB_LTYPE_MAP_EXTENSION PB_LTYPE_EXTENSION
```

Definition at line 897 of file [pb.h](#).

6.79.2.140 PB_LTYPE_MAP_FIXED32

```
#define PB_LTYPE_MAP_FIXED32 PB_LTYPE_FIXED32
```

Definition at line 883 of file [pb.h](#).

6.79.2.141 PB_LTYPE_MAP_FIXED64

```
#define PB_LTYPE_MAP_FIXED64 PB_LTYPE_FIXED64
```

Definition at line 884 of file [pb.h](#).

6.79.2.142 PB_LTYPE_MAP_FIXED_LENGTH_BYTES

```
#define PB_LTYPE_MAP_FIXED_LENGTH_BYTES PB_LTYPE_FIXED_LENGTH_BYTES
```

Definition at line 898 of file [pb.h](#).

6.79.2.143 PB_LTYPE_MAP_FLOAT

```
#define PB_LTYPE_MAP_FLOAT PB_LTYPE_FIXED32
```

Definition at line 885 of file [pb.h](#).

6.79.2.144 PB_LTYPE_MAP_INT32

```
#define PB_LTYPE_MAP_INT32 PB_LTYPE_VARINT
```

Definition at line 886 of file [pb.h](#).

6.79.2.145 PB_LTYPE_MAP_INT64

```
#define PB_LTYPE_MAP_INT64 PB_LTYPE_VARINT
```

Definition at line 887 of file [pb.h](#).

6.79.2.146 PB_LTYPE_MAP_MESSAGE

```
#define PB_LTYPE_MAP_MESSAGE PB_LTYPE_SUBMESSAGE
```

Definition at line 888 of file [pb.h](#).

6.79.2.147 PB_LTYPE_MAP_MSG_W_CB

```
#define PB_LTYPE_MAP_MSG_W_CB PB_LTYPE_SUBMSG_W_CB
```

Definition at line 889 of file [pb.h](#).

6.79.2.148 PB_LTYPE_MAP_SF32

```
#define PB_LTYPE_MAP_SF32 PB_LTYPE_FIXED32
```

Definition at line 890 of file [pb.h](#).

6.79.2.149 PB_LTYPE_MAP_SF64

```
#define PB_LTYPE_MAP_SF64 PB_LTYPE_FIXED64
```

Definition at line 891 of file [pb.h](#).

6.79.2.150 PB_LTYPE_MAP_SINT32

```
#define PB_LTYPE_MAP_SINT32 PB_LTYPE_SVARINT
```

Definition at line 892 of file [pb.h](#).

6.79.2.151 PB_LTYPE_MAP_SINT64

```
#define PB_LTYPE_MAP_SINT64 PB_LTYPE_SVARINT
```

Definition at line 893 of file [pb.h](#).

6.79.2.152 PB_LTYPE_MAP_STRING

```
#define PB_LTYPE_MAP_STRING PB_LTYPE_STRING
```

Definition at line 894 of file [pb.h](#).

6.79.2.153 PB_LTYPE_MAP_UENUM

```
#define PB_LTYPE_MAP_UENUM PB_LTYPE_UVARINT
```

Definition at line 882 of file [pb.h](#).

6.79.2.154 PB_LTYPE_MAP_UINT32

```
#define PB_LTYPE_MAP_UINT32 PB_LTYPE_UVARINT
```

Definition at line 895 of file [pb.h](#).

6.79.2.155 PB_LTYPE_MAP_UINT64

```
#define PB_LTYPE_MAP_UINT64 PB_LTYPE_UVARINT
```

Definition at line 896 of file [pb.h](#).

6.79.2.156 PB_LTYPE_MASK

```
#define PB_LTYPE_MASK 0x0FU
```

Definition at line 304 of file [pb.h](#).

6.79.2.157 PB_LTYPE_STRING

```
#define PB_LTYPE_STRING 0x07U
```

Definition at line 281 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [decode_pointer_field\(\)](#), [encode_array\(\)](#), [encode_basic_field\(\)](#), [pb_check_proto3_default_value\(\)](#), and [pb_encode_tag_for_field\(\)](#).

6.79.2.158 PB_LTYPE_SUBMESSAGE

```
#define PB_LTYPE_SUBMESSAGE 0x08U
```

Definition at line 285 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [encode_basic_field\(\)](#), and [pb_encode_tag_for_field\(\)](#).

6.79.2.159 PB_LTYPE_SUBMSG_W_CB

```
#define PB_LTYPE_SUBMSG_W_CB 0x09U
```

Definition at line 290 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [decode_callback_field\(\)](#), [encode_basic_field\(\)](#), [pb_dec_submessage\(\)](#), [pb_enc_submessage\(\)](#), and [pb_encode_tag_for_field\(\)](#).

6.79.2.160 PB_LTYPE_SVARINT

```
#define PB_LTYPE_SVARINT 0x03U /* sint32, sint64 */
```

Definition at line 268 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [encode_basic_field\(\)](#), [pb_dec_varint\(\)](#), [pb_enc_varint\(\)](#), and [pb_encode_tag_for_field\(\)](#).

6.79.2.161 PB_LTYPE_UVARINT

```
#define PB_LTYPE_UVARINT 0x02U /* uint32, uint64 */
```

Definition at line 267 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [encode_basic_field\(\)](#), [pb_dec_varint\(\)](#), [pb_enc_varint\(\)](#), and [pb_encode_tag_for_field\(\)](#).

6.79.2.162 PB_LTYPE_VARINT

```
#define PB_LTYPE_VARINT 0x01U /* int32, int64, enum, bool */
```

Definition at line 266 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [encode_basic_field\(\)](#), and [pb_encode_tag_for_field\(\)](#).

6.79.2.163 PB_LTYPES_COUNT

```
#define PB_LTYPES_COUNT 0x0CU
```

Definition at line 303 of file [pb.h](#).

6.79.2.164 PB_MAX_REQUIRED_FIELDS

```
#define PB_MAX_REQUIRED_FIELDS 64
```

Definition at line 230 of file [pb.h](#).

Referenced by [pb_decode_inner\(\)](#).

6.79.2.165 pb_membersize

```
#define pb_membersize(  
    st,  
    m)
```

Value:

```
(sizeof ((st*)0)->m)
```

Definition at line 521 of file [pb.h](#).

6.79.2.166 pb_noinline

```
#define pb_noinline
```

Definition at line 147 of file [pb.h](#).

Referenced by [encode_extension_field\(\)](#).

6.79.2.167 PB_ONEOF_NAME

```
#define PB_ONEOF_NAME(  
    type,  
    tuple)
```

Value:

PB_EXPAND(PB_ONEOF_NAME_ ## type tuple)

Definition at line 720 of file [pb.h](#).

6.79.2.168 PB_ONEOF_NAME_FULL

```
#define PB_ONEOF_NAME_FULL(  
    unionname,  
    membername,  
    fullname)
```

Value:

fullname

Definition at line 723 of file [pb.h](#).

6.79.2.169 PB_ONEOF_NAME_MEMBER

```
#define PB_ONEOF_NAME_MEMBER(  
    unionname,  
    membername,  
    fullname)
```

Value:

membername

Definition at line 722 of file [pb.h](#).

6.79.2.170 PB_ONEOF_NAME_UNION

```
#define PB_ONEOF_NAME_UNION(  
    unionname,  
    membername,  
    fullname)
```

Value:

unionname

Definition at line 721 of file [pb.h](#).

6.79.2.171 pb_packed

```
#define pb_packed
```

Definition at line 129 of file [pb.h](#).

6.79.2.172 PB_PACKED_STRUCT_END

```
#define PB_PACKED_STRUCT_END
```

Definition at line 128 of file [pb.h](#).

6.79.2.173 PB_PACKED_STRUCT_START

```
#define PB_PACKED_STRUCT_START
```

Definition at line 127 of file [pb.h](#).

6.79.2.174 PB_PROGMEM

```
#define PB_PROGMEM
```

Definition at line 180 of file [pb.h](#).

6.79.2.175 PB_PROGMEM_READU32

```
#define PB_PROGMEM_READU32(  
    x)
```

Value:

(x)

Definition at line 181 of file [pb.h](#).

Referenced by [advance_iterator\(\)](#), [load_descriptor_values\(\)](#), [pb_field_iter_begin_extension\(\)](#), [pb_field_iter_find\(\)](#), and [pb_field_iter_find_extension\(\)](#).

6.79.2.176 PB_PROTO_HEADER_VERSION

```
#define PB_PROTO_HEADER_VERSION 40
```

Definition at line 517 of file [pb.h](#).

6.79.2.177 PB_RETURN_ERROR

```
#define PB_RETURN_ERROR(  
    stream,  
    msg)
```

Value:

```
return PB_SET_ERROR(stream, msg), false
```

Definition at line 920 of file [pb.h](#).

Referenced by [decode_basic_field\(\)](#), [decode_field\(\)](#), [decode_pointer_field\(\)](#), [decode_static_field\(\)](#), [default_extension_decoder\(\)](#), [default_extension_encoder\(\)](#), [encode_array\(\)](#), [encode_basic_field\(\)](#), [encode_callback_field\(\)](#), [encode_field\(\)](#), [pb_dec_bytes\(\)](#), [pb_dec_fixed_length_bytes\(\)](#), [pb_dec_string\(\)](#), [pb_dec_submessage\(\)](#), [pb_dec_varint\(\)](#), [pb_decode_inner\(\)](#), [pb_decode_varint\(\)](#), [pb_decode_varint32\(\)](#), [pb_enc_bytes\(\)](#), [pb_enc_fixed\(\)](#), [pb_enc_string\(\)](#), [pb_enc_submessage\(\)](#), [pb_enc_varint\(\)](#), [pb_encode_submessage\(\)](#), [pb_encode_tag_for_field\(\)](#), [pb_make_string_substream\(\)](#), [pb_read\(\)](#), [pb_readbyte\(\)](#), [pb_skip_field\(\)](#), [pb_skip_string\(\)](#), [pb_write\(\)](#), and [read_raw_value\(\)](#).

6.79.2.178 PB_SET_ERROR

```
#define PB_SET_ERROR(  
    stream,  
    msg)
```

Value:

(stream->errmsg = (stream->errmsg ? (stream->errmsg : (msg)))

Definition at line 916 of file [pb.h](#).

Referenced by [decode_callback_field\(\)](#).

6.79.2.179 PB_SI_PB_LTYPE_BOOL

```
#define PB_SI_PB_LTYPE_BOOL(  
    t)
```

Definition at line 736 of file [pb.h](#).

6.79.2.180 PB_SI_PB_LTYPE_BYTES

```
#define PB_SI_PB_LTYPE_BYTES(  
    t)
```

Definition at line 737 of file [pb.h](#).

6.79.2.181 PB_SI_PB_LTYPE_DOUBLE

```
#define PB_SI_PB_LTYPE_DOUBLE(  
    t)
```

Definition at line 738 of file [pb.h](#).

6.79.2.182 PB_SI_PB_LTYPE_ENUM

```
#define PB_SI_PB_LTYPE_ENUM(  
    t)
```

Definition at line 739 of file [pb.h](#).

6.79.2.183 PB_SI_PB_LTYPE_EXTENSION

```
#define PB_SI_PB_LTYPE_EXTENSION(  
    t)
```

Definition at line 755 of file [pb.h](#).

6.79.2.184 PB_SI_PB_LTYPE_FIXED32

```
#define PB_SI_PB_LTYPE_FIXED32(  
    t)
```

Definition at line 741 of file [pb.h](#).

6.79.2.185 PB_SI_PB_LTYPE_FIXED64

```
#define PB_SI_PB_LTYPE_FIXED64(  
    t)
```

Definition at line 742 of file [pb.h](#).

6.79.2.186 PB_SI_PB_LTYPE_FIXED_LENGTH_BYTES

```
#define PB_SI_PB_LTYPE_FIXED_LENGTH_BYTES(  
    t)
```

Definition at line 756 of file [pb.h](#).

6.79.2.187 PB_SI_PB_LTYPE_FLOAT

```
#define PB_SI_PB_LTYPE_FLOAT(  
    t)
```

Definition at line 743 of file [pb.h](#).

6.79.2.188 PB_SI_PB_LTYPE_INT32

```
#define PB_SI_PB_LTYPE_INT32(  
    t)
```

Definition at line 744 of file [pb.h](#).

6.79.2.189 PB_SI_PB_LTYPE_INT64

```
#define PB_SI_PB_LTYPE_INT64(  
    t)
```

Definition at line 745 of file [pb.h](#).

6.79.2.190 PB_SI_PB_LTYPE_MESSAGE

```
#define PB_SI_PB_LTYPE_MESSAGE(  
    t)
```

Value:

PB_SUBMSG_DESCRIPTOR(t)

Definition at line 746 of file [pb.h](#).

6.79.2.191 PB_SI_PB_LTYPE_MSG_W_CB

```
#define PB_SI_PB_LTYPE_MSG_W_CB(  
    t)
```

Value:

PB_SUBMSG_DESCRIPTOR(t)

Definition at line 747 of file [pb.h](#).

6.79.2.192 PB_SI_PB_LTYPE_SFIXED32

```
#define PB_SI_PB_LTYPE_SFIXED32(  
    t)
```

Definition at line 748 of file [pb.h](#).

6.79.2.193 PB_SI_PB_LTYPE_SFIXED64

```
#define PB_SI_PB_LTYPE_SFIXED64(  
    t)
```

Definition at line 749 of file [pb.h](#).

6.79.2.194 PB_SI_PB_LTYPE_SINT32

```
#define PB_SI_PB_LTYPE_SINT32(  
    t)
```

Definition at line 750 of file [pb.h](#).

6.79.2.195 PB_SI_PB_LTYPE_SINT64

```
#define PB_SI_PB_LTYPE_SINT64(  
    t)
```

Definition at line 751 of file [pb.h](#).

6.79.2.196 PB_SI_PB_LTYPE_STRING

```
#define PB_SI_PB_LTYPE_STRING(  
    t)
```

Definition at line 752 of file [pb.h](#).

6.79.2.197 PB_SI_PB_LTYPE_UENUM

```
#define PB_SI_PB_LTYPE_UENUM(  
    t)
```

Definition at line 740 of file [pb.h](#).

6.79.2.198 PB_SI_PB_LTYPE_UINT32

```
#define PB_SI_PB_LTYPE_UINT32(  
    t)
```

Definition at line 753 of file [pb.h](#).

6.79.2.199 PB_SI_PB_LTYPE_UINT64

```
#define PB_SI_PB_LTYPE_UINT64(  
    t)
```

Definition at line 754 of file [pb.h](#).

6.79.2.200 PB_SIZE_MAX

```
#define PB_SIZE_MAX ((pb\_size\_t)-1)
```

Definition at line 339 of file [pb.h](#).

Referenced by [decode_pointer_field\(\)](#), [pb_dec_bytes\(\)](#), [pb_dec_fixed_length_bytes\(\)](#), and [pb_decode_inner\(\)](#).

6.79.2.201 PB_SIZE_OFFSET_CALLBACK

```
#define PB_SIZE_OFFSET_CALLBACK(  
    htype,  
    structname,  
    fieldname)
```

Value:

```
PB_SO_CB ## htype(structname, fieldname)
```

Definition at line 660 of file [pb.h](#).

6.79.2.202 PB_SIZE_OFFSET_POINTER

```
#define PB_SIZE_OFFSET_POINTER(  
    htype,  
    structname,  
    fieldname)
```

Value:

```
PB_SO_PTR ## htype(structname, fieldname)
```

Definition at line 659 of file [pb.h](#).

6.79.2.203 PB_SIZE_OFFSET_STATIC

```
#define PB_SIZE_OFFSET_STATIC(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_SO ## htype(structname, fieldname)

Definition at line 658 of file [pb.h](#).

6.79.2.204 PB_SO_CB_PB_HTYPE_FIXARRAY

```
#define PB_SO_CB_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 680 of file [pb.h](#).

6.79.2.205 PB_SO_CB_PB_HTYPE_ONEOF

```
#define PB_SO_CB_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

PB_SO_PB_HTYPE_ONEOF(structname, fieldname)

Definition at line 677 of file [pb.h](#).

6.79.2.206 PB_SO_CB_PB_HTYPE_OPTIONAL

```
#define PB_SO_CB_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 678 of file [pb.h](#).

6.79.2.207 PB_SO_CB_PB_HTYPE_REPEATED

```
#define PB_SO_CB_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 679 of file [pb.h](#).

6.79.2.208 PB_SO_CB_PB_HTYPE_REQUIRED

```
#define PB_SO_CB_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 675 of file [pb.h](#).

6.79.2.209 PB_SO_CB_PB_HTYPE_SINGULAR

```
#define PB_SO_CB_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 676 of file [pb.h](#).

6.79.2.210 PB_SO_PB_HTYPE_FIXARRAY

```
#define PB_SO_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 668 of file [pb.h](#).

6.79.2.211 PB_SO_PB_HTYPE_ONEOF

```
#define PB_SO_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

PB_SO_PB_HTYPE_ONEOF2(structname, PB_ONEOF_NAME(FULL, fieldname), PB_ONEOF_NAME(UNION, fieldname))

Definition at line 663 of file [pb.h](#).

6.79.2.212 PB_SO_PB_HTYPE_ONEOF2

```
#define PB_SO_PB_HTYPE_ONEOF2(  
    structname,  
    fullname,  
    unionname)
```

Value:

PB_SO_PB_HTYPE_ONEOF3(structname, fullname, unionname)

Definition at line 664 of file [pb.h](#).

6.79.2.213 PB_SO_PB_HTYPE_ONEOF3

```
#define PB_SO_PB_HTYPE_ONEOF3(  
    structname,  
    fullname,  
    unionname)
```

Value:

pb_delta(structname, fullname, which_ ## unionname)

Definition at line 665 of file [pb.h](#).

6.79.2.214 PB_SO_PB_HTYPE_OPTIONAL

```
#define PB_SO_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

pb_delta(structname, fieldname, has_ ## fieldname)

Definition at line 666 of file [pb.h](#).

6.79.2.215 PB_SO_PB_HTYPE_REPEATED

```
#define PB_SO_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

pb_delta(structname, fieldname, fieldname ## _count)

Definition at line 667 of file [pb.h](#).

6.79.2.216 PB_SO_PB_HTYPE_REQUIRED

```
#define PB_SO_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 661 of file [pb.h](#).

6.79.2.217 PB_SO_PB_HTYPE_SINGULAR

```
#define PB_SO_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 662 of file [pb.h](#).

6.79.2.218 PB_SO_PTR_PB_HTYPE_FIXARRAY

```
#define PB_SO_PTR_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 674 of file [pb.h](#).

6.79.2.219 PB_SO_PTR_PB_HTYPE_ONEOF

```
#define PB_SO_PTR_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

PB_SO_PB_HTYPE_ONEOF(structname, fieldname)

Definition at line 671 of file [pb.h](#).

6.79.2.220 PB_SO_PTR_PB_HTYPE_OPTIONAL

```
#define PB_SO_PTR_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 672 of file [pb.h](#).

6.79.2.221 PB_SO_PTR_PB_HTYPE_REPEATED

```
#define PB_SO_PTR_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

PB_SO_PB_HTYPE_REPEATED(structname, fieldname)

Definition at line 673 of file [pb.h](#).

6.79.2.222 PB_SO_PTR_PB_HTYPE_REQUIRED

```
#define PB_SO_PTR_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 669 of file [pb.h](#).

6.79.2.223 PB_SO_PTR_PB_HTYPE_SINGULAR

```
#define PB_SO_PTR_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 670 of file [pb.h](#).

6.79.2.224 PB_STATIC_ASSERT

```
#define PB_STATIC_ASSERT(  
    COND,  
    MSG)
```

Value:

```
__Static_assert(COND, #MSG);
```

Definition at line 212 of file [pb.h](#).

6.79.2.225 PB_SUBMSG_DESCRIPTOR

```
#define PB_SUBMSG_DESCRIPTOR(  
    t)
```

Value:

```
&(t ## _msg),
```

Definition at line 757 of file [pb.h](#).

6.79.2.226 PB_SUBMSG_INFO_FIXARRAY

```
#define PB_SUBMSG_INFO_FIXARRAY(  
    ltype,  
    structname,  
    fieldname)
```

Value:

```
PB_SI ## ltype(structname ## _ ## fieldname ## _MSGTYPE)
```

Definition at line 735 of file [pb.h](#).

6.79.2.227 PB_SUBMSG_INFO_ONEOF

```
#define PB_SUBMSG_INFO_ONEOF(  
    ltype,  
    structname,  
    fieldname)
```

Value:

```
PB_SUBMSG_INFO_ONEOF2(ltype, structname, PB_ONEOF_NAME(UNION, fieldname), PB_ONEOF_NAME(MEMBER,  
    fieldname))
```

Definition at line 731 of file [pb.h](#).

6.79.2.228 PB_SUBMSG_INFO_ONEOF2

```
#define PB_SUBMSG_INFO_ONEOF2(  
    ltype,  
    structname,  
    unionname,  
    membername)
```

Value:

```
PB_SUBMSG_INFO_ONEOF3(ltype, structname, unionname, membername)
```

Definition at line 732 of file [pb.h](#).

6.79.2.229 PB_SUBMSG_INFO_ONEOF3

```
#define PB_SUBMSG_INFO_ONEOF3(  
    ltype,  
    structname,  
    unionname,  
    membername)
```

Value:

```
PB_SI ## ltype(structname ## _ ## unionname ## _ ## membername ## _MSGTYPE)
```

Definition at line 733 of file [pb.h](#).

6.79.2.230 PB_SUBMSG_INFO_OPTIONAL

```
#define PB_SUBMSG_INFO_OPTIONAL(  
    ltype,  
    structname,  
    fieldname)
```

Value:

```
PB_SI ## ltype(structname ## _ ## fieldname ## _MSGTYPE)
```

Definition at line 730 of file [pb.h](#).

6.79.2.231 PB_SUBMSG_INFO_REPEATED

```
#define PB_SUBMSG_INFO_REPEATED(  
    ltype,  
    structname,  
    fieldname)
```

Value:

```
PB_SI ## ltype(structname ## _ ## fieldname ## _MSGTYPE)
```

Definition at line 734 of file [pb.h](#).

6.79.2.232 PB_SUBMSG_INFO_REQUIRED

```
#define PB_SUBMSG_INFO_REQUIRED(  
    ltype,  
    structname,  
    fieldname)
```

Value:

PB_SI ## ltype(structname ## _ ## fieldname ## _MSGTYPE)

Definition at line 728 of file [pb.h](#).

6.79.2.233 PB_SUBMSG_INFO_SINGULAR

```
#define PB_SUBMSG_INFO_SINGULAR(  
    ltype,  
    structname,  
    fieldname)
```

Value:

PB_SI ## ltype(structname ## _ ## fieldname ## _MSGTYPE)

Definition at line 729 of file [pb.h](#).

6.79.2.234 PB_UNUSED

```
#define PB_UNUSED(  
    x)
```

Value:

(void)(x)

Definition at line 169 of file [pb.h](#).

Referenced by [decode_pointer_field\(\)](#), [pb_enc_bool\(\)](#), and [pb_release\(\)](#).

6.79.3 Typedef Documentation

6.79.3.1 pb_byte_t

```
typedef uint_least8_t pb_byte_t
```

Definition at line 253 of file [pb.h](#).

6.79.3.2 pb_field_t

```
typedef pb_field_iter_t pb_field_t
```

Definition at line 385 of file [pb.h](#).

6.79.3.3 pb_size_t

```
typedef uint_least16_t pb_size_t
```

Definition at line 336 of file [pb.h](#).

6.79.3.4 pb_ssize_t

```
typedef int_least16_t pb_ssize_t
```

Definition at line 337 of file [pb.h](#).

6.79.3.5 pb_type_t

```
typedef pb_byte_t pb_type_t
```

Definition at line 260 of file [pb.h](#).

6.79.4 Enumeration Type Documentation

6.79.4.1 pb_wire_type_t

```
enum pb_wire_type_t
```

Enumerator

PB_WT_VARINT	
PB_WT_64BIT	
PB_WT_STRING	
PB_WT_32BIT	
PB_WT_PACKED	

Definition at line 446 of file [pb.h](#).

```
00446     {
00447     PB_WT_VARINT = 0,
00448     PB_WT_64BIT  = 1,
00449     PB_WT_STRING = 2,
00450     PB_WT_32BIT  = 5,
00451     PB_WT_PACKED = 255 /* PB_WT_PACKED is internal marker for packed arrays. */
00452 } pb_wire_type_t;
```

6.79.5 Function Documentation

6.79.5.1 pb_default_field_callback()

```
bool pb_default_field_callback (
    pb_istream_t * istream,
    pb_ostream_t * ostream,
    const pb_field_t * field) [extern]
```

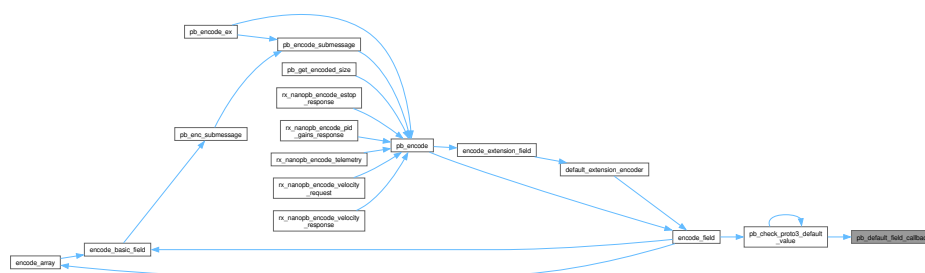
Definition at line 300 of file pb_common.c.

```
00301 {
00302     if (field->data_size == sizeof(pb_callback_t))
00303     {
00304         pb_callback_t *pCallback = (pb_callback_t*)field->pData;
00305
00306         if (pCallback != NULL)
00307         {
00308             if (istream != NULL && pCallback->funcs.decode != NULL)
00309             {
00310                 return pCallback->funcs.decode(istream, field, &pCallback->arg);
00311             }
00312
00313             if (ostream != NULL && pCallback->funcs.encode != NULL)
00314             {
00315                 return pCallback->funcs.encode(ostream, field, &pCallback->arg);
00316             }
00317         }
00318     }
00319
00320     return true; /* Success, but didn't do anything */
00321 }
00322 }
```

References NULL.

Referenced by `pb_check_proto3_default_value()`.

Here is the caller graph for this function:



6.80 pb.h

[Go to the documentation of this file.](#)

```

00001 /* Common parts of the nanopb library. Most of these are quite low-level
00002  * stuff. For the high-level interface, see pb_encode.h and pb_decode.h.
00003  */
00004
00005 #ifndef PB_H_INCLUDED
00006 #define PB_H_INCLUDED
00007
00008 /*****
00009  * Nanopb compilation time options. You can change these here by *
00010  * uncommenting the lines, or on the compiler command line.          *
00011  *****/
00012
00013 /* Enable support for dynamically allocated fields */
00014 /* #define PB_ENABLE_MALLOC 1 */

```

```

00015
00016 /* Define this if your CPU / compiler combination does not support
00017 * unaligned memory access to packed structures. Note that packed
00018 * structures are only used when requested in .proto options. */
00019 /* #define PB_NO_PACKED_STRUCTS 1 */
00020
00021 /* Increase the number of required fields that are tracked.
00022 * A compiler warning will tell if you need this. */
00023 /* #define PB_MAX_REQUIRED_FIELDS 256 */
00024
00025 /* Add support for tag numbers > 65536 and fields larger than 65536 bytes. */
00026 /* #define PB_FIELD_32BIT 1 */
00027
00028 /* Disable support for error messages in order to save some code space. */
00029 /* #define PB_NO_ERRMSG 1 */
00030
00031 /* Disable checks to ensure sub-message encoded size is consistent when re-run. */
00032 /* #define PB_NO_ENCODE_SIZE_CHECK 1 */
00033
00034 /* Disable support for custom streams (support only memory buffers). */
00035 /* #define PB_BUFFER_ONLY 1 */
00036
00037 /* Disable support for 64-bit datatypes, for compilers without int64_t
00038 * or to save some code space. */
00039 /* #define PB_WITHOUT_64BIT 1 */
00040
00041 /* Don't encode scalar arrays as packed. This is only to be used when
00042 * the decoder on the receiving side cannot process packed scalar arrays.
00043 * Such example is older protobuf.js. */
00044 /* #define PB_ENCODE_ARRAYS_UNPACKED 1 */
00045
00046 /* Enable conversion of doubles to floats for platforms that do not
00047 * support 64-bit doubles. Most commonly AVR. */
00048 /* #define PB_CONVERT_DOUBLE_FLOAT 1 */
00049
00050 /* Check whether incoming strings are valid UTF-8 sequences. Slows down
00051 * the string processing slightly and slightly increases code size. */
00052 /* #define PB_VALIDATE_UTF8 1 */
00053
00054 /* This can be defined if the platform is little-endian and has 8-bit bytes.
00055 * Normally it is automatically detected based on __BYTE_ORDER__ macro. */
00056 /* #define PB_LITTLE_ENDIAN_8BIT 1 */
00057
00058 /* Configure static assert mechanism. Instead of changing these, set your
00059 * compiler to C11 standard mode if possible. */
00060 /* #define PB_C99_STATIC_ASSERT 1 */
00061 /* #define PB_NO_STATIC_ASSERT 1 */
00062
00063 /*****
00064 * You usually don't need to change anything below this line.
00065 * Feel free to look around and use the defined macros, though.
00066 *****/
00067
00068
00069 /* Version of the nanopb library. Just in case you want to check it in
00070 * your own program. */
00071 #define NANOPB_VERSION "nanopb-1.0.0-dev"
00072
00073 /* Include all the system headers needed by nanopb. You will need the
00074 * definitions of the following:
00075 * - strlen, memcpy, memset functions
00076 * - [u]int_least8_t, uint_fast8_t, [u]int_least16_t, [u]int32_t, [u]int64_t
00077 * - size_t
00078 * - bool
00079 *
00080 * If you don't have the standard header files, you can instead provide
00081 * a custom header that defines or includes all this. In that case,
00082 * define PB_SYSTEM_HEADER to the path of this file.
00083 */
00084 #ifndef PB_SYSTEM_HEADER
00085 #include PB_SYSTEM_HEADER
00086 #else
00087 #include <stdint.h>
00088 #include <stddef.h>
00089 #include <stdbool.h>
00090 #include <string.h>
00091 #include <limits.h>
00092
00093 #ifdef PB_ENABLE_MALLOC
00094 #include <stdlib.h>
00095 #endif
00096 #endif
00097
00098 #ifdef __cplusplus
00099 extern "C" {
00100 #endif
00101

```

```

00102 /* Macro for defining packed structures (compiler dependent).
00103 * This just reduces memory requirements, but is not required.
00104 */
00105 #if defined(PB_NO_PACKED_STRUCTS)
00106 /* Disable struct packing */
00107 # define PB_PACKED_STRUCT_START
00108 # define PB_PACKED_STRUCT_END
00109 # define pb_packed
00110 #elif defined(__GNUC__) || defined(__clang__)
00111 /* For GCC and clang */
00112 # define PB_PACKED_STRUCT_START
00113 # define PB_PACKED_STRUCT_END
00114 # define pb_packed __attribute__((packed))
00115 #elif defined(__ICCARM__) || defined(__CC_ARM)
00116 /* For IAR ARM and Keil MDK-ARM compilers */
00117 # define PB_PACKED_STRUCT_START _Pragma("pack(push, 1)")
00118 # define PB_PACKED_STRUCT_END _Pragma("pack(pop)")
00119 # define pb_packed
00120 #elif defined(_MSC_VER) && (_MSC_VER >= 1500)
00121 /* For Microsoft Visual C++ */
00122 # define PB_PACKED_STRUCT_START __pragma(pack(push, 1))
00123 # define PB_PACKED_STRUCT_END __pragma(pack(pop))
00124 # define pb_packed
00125 #else
00126 /* Unknown compiler */
00127 # define PB_PACKED_STRUCT_START
00128 # define PB_PACKED_STRUCT_END
00129 # define pb_packed
00130 #endif
00131
00132 /* Define for explicitly not inlining a given function */
00133 #ifndef pb_noinline
00134 #if defined(__GNUC__) || defined(__clang__)
00135 /* For GCC and clang */
00136 # if defined(noinline)
00137 # define pb_noinline noinline
00138 # else
00139 # define pb_noinline __attribute__((noinline))
00140 # endif
00141 #elif defined(__ICCARM__) || defined(__CC_ARM)
00142 /* For IAR ARM and Keil MDK-ARM compilers */
00143 # define pb_noinline
00144 #elif defined(_MSC_VER) && (_MSC_VER >= 1500)
00145 # define pb_noinline __declspec(noinline)
00146 #else
00147 # define pb_noinline
00148 #endif
00149 #endif
00150
00151 /* Detect endianness */
00152 #if !defined(CHAR_BIT) && defined(__CHAR_BIT__)
00153 #define CHAR_BIT __CHAR_BIT__
00154 #endif
00155
00156 #ifndef PB_LITTLE_ENDIAN_8BIT
00157 #if ((defined(__BYTE_ORDER) && __BYTE_ORDER == __LITTLE_ENDIAN) || \
00158 (defined(__BYTE_ORDER) && __BYTE_ORDER == __ORDER_LITTLE_ENDIAN)) || \
00159 (defined(__LITTLE_ENDIAN__) || defined(__ARMEL__) || \
00160 (defined(__THUMBEL__) || defined(__AARCH64EL__) || defined(__MIPSEL) || \
00161 (defined(__M_X86) || defined(__M_X64) || defined(__M_ARM)) \
00162 && defined(CHAR_BIT) && CHAR_BIT == 8)
00163 #define PB_LITTLE_ENDIAN_8BIT 1
00164 #endif
00165 #endif
00166
00167 /* Handy macro for suppressing unreferenced-parameter compiler warnings. */
00168 #ifndef PB_UNUSED
00169 #define PB_UNUSED(x) (void)(x)
00170 #endif
00171
00172 /* Harvard-architecture processors may need special attributes for storing
00173 * field information in program memory. */
00174 #ifndef PB_PROGMEM
00175 #if defined __AVR__
00176 #include <avr/pgmspace.h>
00177 #define PB_PROGMEM PROGMEM
00178 #define PB_PROGMEM_READU32(x) pgm_read_dword(&x)
00179 #else
00180 #define PB_PROGMEM
00181 #define PB_PROGMEM_READU32(x) (x)
00182 #endif
00183 #endif
00184
00185 /* Compile-time assertion, used for checking compatible compilation options.
00186 * If this does not work properly on your compiler, use
00187 * #define PB_NO_STATIC_ASSERT to disable it.
00188 */

```

```

00189 * But before doing that, check carefully the error message / place where it
00190 * comes from to see if the error has a real cause. Unfortunately the error
00191 * message is not always very clear to read, but you can see the reason better
00192 * in the place where the PB_STATIC_ASSERT macro was called.
00193 */
00194 #ifndef PB_NO_STATIC_ASSERT
00195 # if defined(PB_STATIC_ASSERT)
00196 #   if defined(__ICCARM__)
00197     /* IAR has static_assert keyword but no _Static_assert */
00198     # define PB_STATIC_ASSERT(COND,MSG) static_assert(COND,#MSG);
00199 #   elif defined(__MSC_VER) && (!defined(__STDC_VERSION__) || __STDC_VERSION__ < 201112)
00200     /* MSVC in C89 mode supports static_assert() keyword anyway */
00201     # define PB_STATIC_ASSERT(COND,MSG) static_assert(COND,#MSG);
00202 #   elif defined(PB_C99_STATIC_ASSERT)
00203     /* Classic negative-size-array static assert mechanism */
00204     # define PB_STATIC_ASSERT(COND,MSG) typedef char PB_STATIC_ASSERT_MSG(MSG, __LINE__,
00205     #   __COUNTER__)[(COND)?1:-1];
00206 #   define PB_STATIC_ASSERT_MSG(MSG, LINE, COUNTER) PB_STATIC_ASSERT_MSG_(MSG, LINE,
00207     #   COUNTER)
00208 #   define PB_STATIC_ASSERT_MSG_(MSG, LINE, COUNTER)
00209 #   pb_static_assertion_##MSG##_##LINE##_##COUNTER
00210 #   elif defined(__cplusplus)
00211     /* C++11 standard static_assert mechanism */
00212     # define PB_STATIC_ASSERT(COND,MSG) static_assert(COND,#MSG);
00213 #   else
00214     /* C11 standard _Static_assert mechanism */
00215     # define PB_STATIC_ASSERT(COND,MSG) _Static_assert(COND,#MSG);
00216 #   endif
00217 # endif
00218 #endif
00219
00220 /* Test that PB_STATIC_ASSERT works
00221 * If you get errors here, you may need to do one of these:
00222 * - Enable C11 standard support in your compiler
00223 * - Define PB_C99_STATIC_ASSERT to enable C99 standard support
00224 * - Define PB_NO_STATIC_ASSERT to disable static asserts altogether
00225 */
00226 PB_STATIC_ASSERT(1, STATIC_ASSERT_IS_NOT_WORKING)
00227
00228 /* Number of required fields to keep track of. */
00229 #ifndef PB_MAX_REQUIRED_FIELDS
00230 #define PB_MAX_REQUIRED_FIELDS 64
00231 #endif
00232
00233 #if PB_MAX_REQUIRED_FIELDS < 64
00234 #error You should not lower PB_MAX_REQUIRED_FIELDS from the default value (64).
00235 #endif
00236
00237 #ifndef PB_WITHOUT_64BIT
00238 #ifndef PB_CONVERT_DOUBLE_FLOAT
00239 /* Cannot use doubles without 64-bit types */
00240 #undef PB_CONVERT_DOUBLE_FLOAT
00241 #endif
00242 #endif
00243
00244 /* Data type for storing encoded data and other byte streams.
00245 * This typedef exists to support platforms where uint8_t does not exist.
00246 * You can regard it as equivalent on uint8_t on other platforms.
00247 */
00248 #if defined(PB_BYTE_T_OVERRIDE)
00249 typedef PB_BYTE_T_OVERRIDE pb_byte_t;
00250 #elif defined(UINT8_MAX)
00251 typedef uint8_t pb_byte_t;
00252 #else
00253 typedef uint_least8_t pb_byte_t;
00254 #endif
00255
00256 /* List of possible field types. These are used in the autogenerated code.
00257 * Least-significant 4 bits tell the scalar type
00258 * Most-significant 4 bits specify repeated/required/packed etc.
00259 */
00260 typedef pb_byte_t pb_type_t;
00261
00262 **** Field data types ****
00263
00264 /* Numeric types */
00265 #define PB_LTYPE_BOOL 0x00U /* bool */
00266 #define PB_LTYPE_VARINT 0x01U /* int32, int64, enum, bool */
00267 #define PB_LTYPE_UVARINT 0x02U /* uint32, uint64 */
00268 #define PB_LTYPE_SVARINT 0x03U /* sint32, sint64 */
00269 #define PB_LTYPE_FIXED32 0x04U /* fixed32, sfixed32, float */
00270 #define PB_LTYPE_FIXED64 0x05U /* fixed64, sfixed64, double */
00271
00272 /* Marker for last packable field type. */

```

```

00273 #define PB_LTYPE_LAST_PACKABLE 0x05U
00274
00275 /* Byte array with pre-allocated buffer.
00276 * data_size is the length of the allocated PB_BYTES_ARRAY structure. */
00277 #define PB_LTYPE_BYTES 0x06U
00278
00279 /* String with pre-allocated buffer.
00280 * data_size is the maximum length. */
00281 #define PB_LTYPE_STRING 0x07U
00282
00283 /* Submessage
00284 * submsg_fields is pointer to field descriptions */
00285 #define PB_LTYPE_SUBMESSAGE 0x08U
00286
00287 /* Submessage with pre-decoding callback
00288 * The pre-decoding callback is stored as pb_callback_t right before pSize.
00289 * submsg_fields is pointer to field descriptions */
00290 #define PB_LTYPE_SUBMSG_W_CB 0x09U
00291
00292 /* Extension pseudo-field
00293 * The field contains a pointer to pb_extension_t */
00294 #define PB_LTYPE_EXTENSION 0x0AU
00295
00296 /* Byte array with inline, pre-allocated byffer.
00297 * data_size is the length of the inline, allocated buffer.
00298 * This differs from PB_LTYPE_BYTES by defining the element as
00299 * pb_byte_t[data_size] rather than pb_bytes_array_t. */
00300 #define PB_LTYPE_FIXED_LENGTH_BYTES 0x0BU
00301
00302 /* Number of declared LTYPES */
00303 #define PB_LTYPES_COUNT 0x0CU
00304 #define PB_LTYPE_MASK 0x0FU
00305
00306 /**** Field repetition rules ****/
00307
00308 #define PB_HTYPE_REQUIRED 0x00U
00309 #define PB_HTYPE_OPTIONAL 0x10U
00310 #define PB_HTYPE_SINGULAR 0x10U
00311 #define PB_HTYPE_REPEATED 0x20U
00312 #define PB_HTYPE_FIXARRAY 0x20U
00313 #define PB_HTYPE_ONEOF 0x30U
00314 #define PB_HTYPE_MASK 0x30U
00315
00316 /**** Field allocation types ****/
00317
00318 #define PB_ATYPE_STATIC 0x00U
00319 #define PB_ATYPE_POINTER 0x80U
00320 #define PB_ATYPE_CALLBACK 0x40U
00321 #define PB_ATYPE_MASK 0xC0U
00322
00323 #define PB_ATYPE(x) ((x) & PB_ATYPE_MASK)
00324 #define PB_HTYPE(x) ((x) & PB_HTYPE_MASK)
00325 #define PB_LTYPE(x) ((x) & PB_LTYPE_MASK)
00326 #define PB_LTYPE_IS_SUBMSG(x) (PB_LTYPE(x) == PB_LTYPE_SUBMESSAGE || \
00327     PB_LTYPE(x) == PB_LTYPE_SUBMSG_W_CB)
00328
00329 /* Data type used for storing sizes of struct fields
00330 * and array counts.
00331 */
00332 #if defined(PB_FIELD_32BIT)
00333     typedef uint32_t pb_size_t;
00334     typedef int32_t pb_ssize_t;
00335 #else
00336     typedef uint_least16_t pb_size_t;
00337     typedef int_least16_t pb_ssize_t;
00338 #endif
00339 #define PB_SIZE_MAX ((pb_size_t)-1)
00340
00341 /* Forward declaration of struct types */
00342 typedef struct pb_istream_s pb_istream_t;
00343 typedef struct pb_ostream_s pb_ostream_t;
00344 typedef struct pb_field_iter_s pb_field_iter_t;
00345
00346 /* This structure is used in auto-generated constants
00347 * to specify struct fields.
00348 */
00349 typedef struct pb_msgdesc_s pb_msgdesc_t;
00350 struct pb_msgdesc_s {
00351     const uint32_t *field_info;
00352     const pb_msgdesc_t * const * submsg_info;
00353     const pb_byte_t *default_value;
00354
00355     bool (*field_callback)(pb_istream_t *istream, pb_ostream_t *ostream, const pb_field_iter_t *field);
00356
00357     pb_size_t field_count;
00358     pb_size_t required_field_count;
00359     pb_size_t largest_tag;

```

```

00360 };
00361
00362 /* Iterator for message descriptor */
00363 struct pb_field_iter_s {
00364     const pb_msgdesc_t *descriptor; /* Pointer to message descriptor constant */
00365     void *message; /* Pointer to start of the structure */
00366
00367     pb_size_t index; /* Index of the field */
00368     pb_size_t field_info_index; /* Index to descriptor->field_info array */
00369     pb_size_t required_field_index; /* Index that counts only the required fields */
00370     pb_size_t submessage_index; /* Index that counts only submessages */
00371
00372     pb_size_t tag; /* Tag of current field */
00373     pb_size_t data_size; /* sizeof() of a single item */
00374     pb_size_t array_size; /* Number of array entries */
00375     pb_type_t type; /* Type of current field */
00376
00377     void *pField; /* Pointer to current field in struct */
00378     void *pData; /* Pointer to current data contents. Different than pField for arrays and pointers. */
00379     void *pSize; /* Pointer to count/has field */
00380
00381     const pb_msgdesc_t *submsg_desc; /* For submessage fields, pointer to field descriptor for the submessage. */
00382 };
00383
00384 /* For compatibility with legacy code */
00385 typedef pb_field_iter_t pb_field_t;
00386
00387 /* Make sure that the standard integer types are of the expected sizes.
00388  * Otherwise fixed32/fixed64 fields can break.
00389  */
00390 /* If you get errors here, it probably means that your stdint.h is not
00391  * correct for your platform.
00392  */
00393 #ifndef PB_WITHOUT_64BIT
00394 PB_STATIC_ASSERT(sizeof(int64_t) == 2 * sizeof(int32_t), INT64_T_WRONG_SIZE)
00395 PB_STATIC_ASSERT(sizeof(uint64_t) == 2 * sizeof(uint32_t), UINT64_T_WRONG_SIZE)
00396 #endif
00397
00398 /* This structure is used for 'bytes' arrays.
00399  * It has the number of bytes in the beginning, and after that an array.
00400  * Note that actual structs used will have a different length of bytes array.
00401  */
00402 #define PB_BYTES_ARRAY_T(n) struct { pb_size_t size; pb_byte_t bytes[n]; }
00403 #define PB_BYTES_ARRAY_T_ALLOCSIZE(n) ((size_t)n + offsetof(pb_bytes_array_t, bytes))
00404
00405 struct pb_bytes_array_s {
00406     pb_size_t size;
00407     pb_byte_t bytes[1];
00408 };
00409 typedef struct pb_bytes_array_s pb_bytes_array_t;
00410
00411 /* This structure is used for giving the callback function.
00412  * It is stored in the message structure and filled in by the method that
00413  * calls pb_decode.
00414  */
00415 /* The decoding callback will be given a limited-length stream
00416  * If the wire type was string, the length is the length of the string.
00417  * If the wire type was a varint/fixed32/fixed64, the length is the length
00418  * of the actual value.
00419  * The function may be called multiple times (especially for repeated types,
00420  * but also otherwise if the message happens to contain the field multiple
00421  * times.)
00422  */
00423 /* The encoding callback will receive the actual output stream.
00424  * It should write all the data in one call, including the field tag and
00425  * wire type. It can write multiple fields.
00426  */
00427 /* The callback can be null if you want to skip a field.
00428  */
00429 typedef struct pb_callback_s pb_callback_t;
00430 struct pb_callback_s {
00431     /* Callback functions receive a pointer to the arg field.
00432     * You can access the value of the field as *arg, and modify it if needed.
00433     */
00434     union {
00435         bool (*decode)(pb_istream_t *stream, const pb_field_t *field, void **arg);
00436         bool (*encode)(pb_ostream_t *stream, const pb_field_t *field, void *const *arg);
00437     } funcs;
00438
00439     /* Free arg for use by callback */
00440     void *arg;
00441 };
00442
00443 extern bool pb_default_field_callback(pb_istream_t *istream, pb_ostream_t *ostream, const pb_field_t *field);
00444
00445 /* Wire types. Library user needs these only in encoder callbacks. */
00446 typedef enum {

```



```

00447 PB_WT_VARINT = 0,
00448 PB_WT_64BIT = 1,
00449 PB_WT_STRING = 2,
00450 PB_WT_32BIT = 5,
00451 PB_WT_PACKED = 255 /* PB_WT_PACKED is internal marker for packed arrays. */
00452 } pb_wire_type_t;
00453
00454 /* Structure for defining the handling of unknown/extension fields.
00455 * Usually the pb_extension_type_t structure is automatically generated,
00456 * while the pb_extension_t structure is created by the user. However,
00457 * if you want to catch all unknown fields, you can also create a custom
00458 * pb_extension_type_t with your own callback.
00459 */
00460 typedef struct pb_extension_type_s pb_extension_type_t;
00461 typedef struct pb_extension_s pb_extension_t;
00462 struct pb_extension_type_s {
00463     /* Called for each unknown field in the message.
00464     * If you handle the field, read off all of its data and return true.
00465     * If you do not handle the field, do not read anything and return false.
00466     * If you run into an error, return false.
00467     * Set to NULL for default handler.
00468     */
00469     bool (*decode)(pb_istream_t *stream, pb_extension_t *extension,
00470                   uint32_t tag, pb_wire_type_t wire_type);
00471
00472     /* Called once after all regular fields have been encoded.
00473     * If you have something to write, do so and return true.
00474     * If you do not have anything to write, just return true.
00475     * If you run into an error, return false.
00476     * Set to NULL for default handler.
00477     */
00478     bool (*encode)(pb_ostream_t *stream, const pb_extension_t *extension);
00479
00480     /* Free field for use by the callback. */
00481     const void *arg;
00482 };
00483
00484 struct pb_extension_s {
00485     /* Type describing the extension field. Usually you'll initialize
00486     * this to a pointer to the automatically generated structure. */
00487     const pb_extension_type_t *type;
00488
00489     /* Destination for the decoded data. This must match the datatype
00490     * of the extension field. */
00491     void *dest;
00492
00493     /* Pointer to the next extension handler, or NULL.
00494     * If this extension does not match a field, the next handler is
00495     * automatically called. */
00496     pb_extension_t *next;
00497
00498     /* The decoder sets this to true if the extension was found.
00499     * Ignored for encoding. */
00500     bool found;
00501 };
00502
00503 #define pb_extension_init_zero {NULL,NULL,NULL,false}
00504
00505 /* Memory allocation functions to use. You can define pb_realloc and
00506 * pb_free to custom functions if you want. */
00507 #ifdef PB_ENABLE_MALLOC
00508 #   ifndef pb_realloc
00509 #       define pb_realloc(ptr, size) realloc(ptr, size)
00510 #   endif
00511 #   ifndef pb_free
00512 #       define pb_free(ptr) free(ptr)
00513 #   endif
00514 #endif
00515
00516 /* This is used to inform about need to regenerate .pb.h/.pb.c files. */
00517 #define PB_PROTO_HEADER_VERSION 40
00518
00519 /* These macros are used to declare pb_field_t's in the constant array. */
00520 /* Size of a structure member, in bytes. */
00521 #define pb_membersize(st, m) (sizeof ((st*)0)->m)
00522 /* Number of entries in an array. */
00523 #define pb_arraysize(st, m) (pb_membersize(st, m) / pb_membersize(st, m[0]))
00524 /* Delta from start of one member to the start of another member. */
00525 #define pb_delta(st, m1, m2) ((int)offsetof(st, m1) - (int)offsetof(st, m2))
00526
00527 /* Force expansion of macro value */
00528 #define PB_EXPAND(x) x
00529
00530 /* Binding of a message field set into a specific structure */
00531 #define PB_BIND(msgname, structname, width) \
00532     const uint32_t structname##_field_info[] PB_PROGMEM = \
00533     { \

```

```

00534     msgname ## _FIELDLIST(PB_GEN_FIELD_INFO_ ## width, structname) \
00535     0 \
00536 }; \
00537 const pb_msgdesc_t* const structname ## _submsg_info[] = \
00538 { \
00539     msgname ## _FIELDLIST(PB_GEN_SUBMSG_INFO, structname) \
00540     NULL \
00541 }; \
00542 const pb_msgdesc_t structname ## _msg = \
00543 { \
00544     structname ## _field_info, \
00545     structname ## _submsg_info, \
00546     msgname ## _DEFAULT, \
00547     msgname ## _CALLBACK, \
00548     0 msgname ## _FIELDLIST(PB_GEN_FIELD_COUNT, structname), \
00549     0 msgname ## _FIELDLIST(PB_GEN_REQ_FIELD_COUNT, structname), \
00550     0 msgname ## _FIELDLIST(PB_GEN_LARGEST_TAG, structname), \
00551 }; \
00552 msgname ## _FIELDLIST(PB_GEN_FIELD_INFO_ASSERT_ ## width, structname)
00553
00554 #define PB_GEN_FIELD_COUNT(structname, atype, htype, ltype, fieldname, tag) +1
00555 #define PB_GEN_REQ_FIELD_COUNT(structname, atype, htype, ltype, fieldname, tag) \
00556     + (PB_HTYPE_ ## htype == PB_HTYPE_REQUIRED)
00557 #define PB_GEN_LARGEST_TAG(structname, atype, htype, ltype, fieldname, tag) \
00558     * 0 + tag
00559
00560 /* X-macro for generating the entries in struct_field_info[] array. */
00561 #define PB_GEN_FIELD_INFO_1(structname, atype, htype, ltype, fieldname, tag) \
00562     PB_FIELDINFO_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00563     PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00564     PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00565     PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00566     PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00567
00568 #define PB_GEN_FIELD_INFO_2(structname, atype, htype, ltype, fieldname, tag) \
00569     PB_FIELDINFO_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00570     PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00571     PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00572     PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00573     PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00574
00575 #define PB_GEN_FIELD_INFO_4(structname, atype, htype, ltype, fieldname, tag) \
00576     PB_FIELDINFO_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00577     PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00578     PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00579     PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00580     PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00581
00582 #define PB_GEN_FIELD_INFO_8(structname, atype, htype, ltype, fieldname, tag) \
00583     PB_FIELDINFO_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00584     PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00585     PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00586     PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00587     PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00588
00589 #define PB_GEN_FIELD_INFO_AUTO(structname, atype, htype, ltype, fieldname, tag) \
00590     PB_FIELDINFO_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_ ## htype, \
00591     _PB_LTYPE_ ## ltype), \
00592     tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00593     PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00594     PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00595     PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00596     PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00597 #define PB_FIELDINFO_AUTO2(width, tag, type, data_offset, data_size, size_offset, array_size) \
00598     PB_FIELDINFO_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)
00599
00600 #define PB_FIELDINFO_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size) \
00601     PB_FIELDINFO_ ## width(tag, type, data_offset, data_size, size_offset, array_size)
00602
00603 /* X-macro for generating asserts that entries fit in struct_field_info[] array.
00604 * The structure of macros here must match the structure above in PB_GEN_FIELD_INFO_x(),
00605 * but it is not easily reused because of how macro substitutions work. */
00606 #define PB_GEN_FIELD_INFO_ASSERT_1(structname, atype, htype, ltype, fieldname, tag) \
00607     PB_FIELDINFO_ASSERT_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## \
00608     ltype, \
00609     PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00610     PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00611     PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00612     PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00613 #define PB_GEN_FIELD_INFO_ASSERT_2(structname, atype, htype, ltype, fieldname, tag) \
00614     PB_FIELDINFO_ASSERT_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## \
00615     ltype, \
00616     PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00617     PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00618     PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \

```

```

00618         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00619
00620 #define PB_GEN_FIELD_INFO_ASSERT_4(structname, atype, htype, ltype, fieldname, tag) \
00621     PB_FIELDINFO_ASSERT_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ##
ltype, \
00622         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00623         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00624         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00625         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00626
00627 #define PB_GEN_FIELD_INFO_ASSERT_8(structname, atype, htype, ltype, fieldname, tag) \
00628     PB_FIELDINFO_ASSERT_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ##
ltype, \
00629         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00630         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00631         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00632         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00633
00634 #define PB_GEN_FIELD_INFO_ASSERT_AUTO(structname, atype, htype, ltype, fieldname, tag) \
00635     PB_FIELDINFO_ASSERT_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_
## htype, _PB_LTYPE_ ## ltype), \
00636         tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00637         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00638         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00639         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00640         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00641
00642 #define PB_FIELDINFO_ASSERT_AUTO2(width, tag, type, data_offset, data_size, size_offset, array_size) \
00643     PB_FIELDINFO_ASSERT_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)
00644
00645 #define PB_FIELDINFO_ASSERT_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size) \
00646     PB_FIELDINFO_ASSERT_ ## width(tag, type, data_offset, data_size, size_offset, array_size)
00647
00648 #define PB_DATA_OFFSET_STATIC(htype, structname, fieldname) PB_DO ## htype(structname, fieldname)
00649 #define PB_DATA_OFFSET_POINTER(htype, structname, fieldname) PB_DO ## htype(structname, fieldname)
00650 #define PB_DATA_OFFSET_CALLBACK(htype, structname, fieldname) PB_DO ## htype(structname, fieldname)
00651 #define PB_DO_PB_HTYPE_REQUIRED(structname, fieldname) offsetof(structname, fieldname)
00652 #define PB_DO_PB_HTYPE_SINGULAR(structname, fieldname) offsetof(structname, fieldname)
00653 #define PB_DO_PB_HTYPE_ONEOF(structname, fieldname) offsetof(structname, PB_ONEOF_NAME(FULL,
fieldname))
00654 #define PB_DO_PB_HTYPE_OPTIONAL(structname, fieldname) offsetof(structname, fieldname)
00655 #define PB_DO_PB_HTYPE_REPEATED(structname, fieldname) offsetof(structname, fieldname)
00656 #define PB_DO_PB_HTYPE_FIXARRAY(structname, fieldname) offsetof(structname, fieldname)
00657
00658 #define PB_SIZE_OFFSET_STATIC(htype, structname, fieldname) PB_SO ## htype(structname, fieldname)
00659 #define PB_SIZE_OFFSET_POINTER(htype, structname, fieldname) PB_SO_PTR ## htype(structname, fieldname)
00660 #define PB_SIZE_OFFSET_CALLBACK(htype, structname, fieldname) PB_SO_CB ## htype(structname, fieldname)
00661 #define PB_SO_PB_HTYPE_REQUIRED(structname, fieldname) 0
00662 #define PB_SO_PB_HTYPE_SINGULAR(structname, fieldname) 0
00663 #define PB_SO_PB_HTYPE_ONEOF(structname, fieldname) PB_SO_PB_HTYPE_ONEOF2(structname,
PB_ONEOF_NAME(FULL, fieldname), PB_ONEOF_NAME(UNION, fieldname))
00664 #define PB_SO_PB_HTYPE_ONEOF2(structname, fullname, unionname)
PB_SO_PB_HTYPE_ONEOF3(structname, fullname, unionname)
00665 #define PB_SO_PB_HTYPE_ONEOF3(structname, fullname, unionname) pb_delta(structname, fullname, which_ ##
unionname)
00666 #define PB_SO_PB_HTYPE_OPTIONAL(structname, fieldname) pb_delta(structname, fieldname, has_ ## fieldname)
00667 #define PB_SO_PB_HTYPE_REPEATED(structname, fieldname) pb_delta(structname, fieldname, fieldname ##
_count)
00668 #define PB_SO_PB_HTYPE_FIXARRAY(structname, fieldname) 0
00669 #define PB_SO_PTR_PB_HTYPE_REQUIRED(structname, fieldname) 0
00670 #define PB_SO_PTR_PB_HTYPE_SINGULAR(structname, fieldname) 0
00671 #define PB_SO_PTR_PB_HTYPE_ONEOF(structname, fieldname) PB_SO_PB_HTYPE_ONEOF(structname,
fieldname)
00672 #define PB_SO_PTR_PB_HTYPE_OPTIONAL(structname, fieldname) 0
00673 #define PB_SO_PTR_PB_HTYPE_REPEATED(structname, fieldname)
PB_SO_PB_HTYPE_REPEATED(structname, fieldname)
00674 #define PB_SO_PTR_PB_HTYPE_FIXARRAY(structname, fieldname) 0
00675 #define PB_SO_CB_PB_HTYPE_REQUIRED(structname, fieldname) 0
00676 #define PB_SO_CB_PB_HTYPE_SINGULAR(structname, fieldname) 0
00677 #define PB_SO_CB_PB_HTYPE_ONEOF(structname, fieldname) PB_SO_PB_HTYPE_ONEOF(structname,
fieldname)
00678 #define PB_SO_CB_PB_HTYPE_OPTIONAL(structname, fieldname) 0
00679 #define PB_SO_CB_PB_HTYPE_REPEATED(structname, fieldname) 0
00680 #define PB_SO_CB_PB_HTYPE_FIXARRAY(structname, fieldname) 0
00681
00682 #define PB_ARRAY_SIZE_STATIC(htype, structname, fieldname) PB_AS ## htype(structname, fieldname)
00683 #define PB_ARRAY_SIZE_POINTER(htype, structname, fieldname) PB_AS_PTR ## htype(structname, fieldname)
00684 #define PB_ARRAY_SIZE_CALLBACK(htype, structname, fieldname) 1
00685 #define PB_AS_PB_HTYPE_REQUIRED(structname, fieldname) 1
00686 #define PB_AS_PB_HTYPE_SINGULAR(structname, fieldname) 1
00687 #define PB_AS_PB_HTYPE_OPTIONAL(structname, fieldname) 1
00688 #define PB_AS_PB_HTYPE_ONEOF(structname, fieldname) 1
00689 #define PB_AS_PB_HTYPE_REPEATED(structname, fieldname) pb_arraysize(structname, fieldname)
00690 #define PB_AS_PB_HTYPE_FIXARRAY(structname, fieldname) pb_arraysize(structname, fieldname)
00691 #define PB_AS_PTR_PB_HTYPE_REQUIRED(structname, fieldname) 1
00692 #define PB_AS_PTR_PB_HTYPE_SINGULAR(structname, fieldname) 1
00693 #define PB_AS_PTR_PB_HTYPE_OPTIONAL(structname, fieldname) 1

```

```

00694 #define PB_AS_PTR_PB_HTYPE_ONEOF(structname, fieldname) 1
00695 #define PB_AS_PTR_PB_HTYPE_REPEATED(structname, fieldname) 1
00696 #define PB_AS_PTR_PB_HTYPE_FIXARRAY(structname, fieldname) pb_arraysize(structname, fieldname[0])
00697
00698 #define PB_DATA_SIZE_STATIC(htype, structname, fieldname) PB_DS_ ## htype(structname, fieldname)
00699 #define PB_DATA_SIZE_POINTER(htype, structname, fieldname) PB_DS_PTR_ ## htype(structname, fieldname)
00700 #define PB_DATA_SIZE_CALLBACK(htype, structname, fieldname) PB_DS_CB_ ## htype(structname, fieldname)
00701 #define PB_DS_PB_HTYPE_REQUIRED(structname, fieldname) pb_membersize(structname, fieldname)
00702 #define PB_DS_PB_HTYPE_SINGULAR(structname, fieldname) pb_membersize(structname, fieldname)
00703 #define PB_DS_PB_HTYPE_OPTIONAL(structname, fieldname) pb_membersize(structname, fieldname)
00704 #define PB_DS_PB_HTYPE_ONEOF(structname, fieldname) pb_membersize(structname, PB_ONEOF_NAME(FULL,
    fieldname))
00705 #define PB_DS_PB_HTYPE_REPEATED(structname, fieldname) pb_membersize(structname, fieldname[0])
00706 #define PB_DS_PB_HTYPE_FIXARRAY(structname, fieldname) pb_membersize(structname, fieldname[0])
00707 #define PB_DS_PTR_PB_HTYPE_REQUIRED(structname, fieldname) pb_membersize(structname, fieldname[0])
00708 #define PB_DS_PTR_PB_HTYPE_SINGULAR(structname, fieldname) pb_membersize(structname, fieldname[0])
00709 #define PB_DS_PTR_PB_HTYPE_OPTIONAL(structname, fieldname) pb_membersize(structname, fieldname[0])
00710 #define PB_DS_PTR_PB_HTYPE_ONEOF(structname, fieldname) pb_membersize(structname,
    PB_ONEOF_NAME(FULL, fieldname)[0])
00711 #define PB_DS_PTR_PB_HTYPE_REPEATED(structname, fieldname) pb_membersize(structname, fieldname[0])
00712 #define PB_DS_PTR_PB_HTYPE_FIXARRAY(structname, fieldname) pb_membersize(structname, fieldname[0][0])
00713 #define PB_DS_CB_PB_HTYPE_REQUIRED(structname, fieldname) pb_membersize(structname, fieldname)
00714 #define PB_DS_CB_PB_HTYPE_SINGULAR(structname, fieldname) pb_membersize(structname, fieldname)
00715 #define PB_DS_CB_PB_HTYPE_OPTIONAL(structname, fieldname) pb_membersize(structname, fieldname)
00716 #define PB_DS_CB_PB_HTYPE_ONEOF(structname, fieldname) pb_membersize(structname,
    PB_ONEOF_NAME(FULL, fieldname))
00717 #define PB_DS_CB_PB_HTYPE_REPEATED(structname, fieldname) pb_membersize(structname, fieldname)
00718 #define PB_DS_CB_PB_HTYPE_FIXARRAY(structname, fieldname) pb_membersize(structname, fieldname)
00719
00720 #define PB_ONEOF_NAME(type, tuple) PB_EXPAND(PB_ONEOF_NAME_ ## type tuple)
00721 #define PB_ONEOF_NAME_UNION(unionname, membername, fullname) unionname
00722 #define PB_ONEOF_NAME_MEMBER(unionname, membername, fullname) membername
00723 #define PB_ONEOF_NAME_FULL(unionname, membername, fullname) fullname
00724
00725 #define PB_GEN_SUBMSG_INFO(structname, atype, htype, ltype, fieldname, tag) \
00726     PB_SUBMSG_INFO_ ## htype(_PB_LTYPE_ ## ltype, structname, fieldname)
00727
00728 #define PB_SUBMSG_INFO_REQUIRED(ltype, structname, fieldname) PB_SI_ ## ltype(structname ## _ ##
    fieldname ## _MSGTYPE)
00729 #define PB_SUBMSG_INFO_SINGULAR(ltype, structname, fieldname) PB_SI_ ## ltype(structname ## _ ##
    fieldname ## _MSGTYPE)
00730 #define PB_SUBMSG_INFO_OPTIONAL(ltype, structname, fieldname) PB_SI_ ## ltype(structname ## _ ##
    fieldname ## _MSGTYPE)
00731 #define PB_SUBMSG_INFO_ONEOF(ltype, structname, fieldname) PB_SUBMSG_INFO_ONEOF2(ltype, structname,
    PB_ONEOF_NAME(UNION, fieldname), PB_ONEOF_NAME(MEMBER, fieldname))
00732 #define PB_SUBMSG_INFO_ONEOF2(ltype, structname, unionname, membername)
    PB_SUBMSG_INFO_ONEOF3(ltype, structname, unionname, membername)
00733 #define PB_SUBMSG_INFO_ONEOF3(ltype, structname, unionname, membername) PB_SI_ ## ltype(structname ##
    _ ## unionname ## _ ## membername ## _MSGTYPE)
00734 #define PB_SUBMSG_INFO_REPEATED(ltype, structname, fieldname) PB_SI_ ## ltype(structname ## _ ##
    fieldname ## _MSGTYPE)
00735 #define PB_SUBMSG_INFO_FIXARRAY(ltype, structname, fieldname) PB_SI_ ## ltype(structname ## _ ##
    fieldname ## _MSGTYPE)
00736 #define PB_SI_PB_LTYPE_BOOL(t)
00737 #define PB_SI_PB_LTYPE_BYTES(t)
00738 #define PB_SI_PB_LTYPE_DOUBLE(t)
00739 #define PB_SI_PB_LTYPE_ENUM(t)
00740 #define PB_SI_PB_LTYPE_UENUM(t)
00741 #define PB_SI_PB_LTYPE_FIXED32(t)
00742 #define PB_SI_PB_LTYPE_FIXED64(t)
00743 #define PB_SI_PB_LTYPE_FLOAT(t)
00744 #define PB_SI_PB_LTYPE_INT32(t)
00745 #define PB_SI_PB_LTYPE_INT64(t)
00746 #define PB_SI_PB_LTYPE_MESSAGE(t) PB_SUBMSG_DESCRIPTOR(t)
00747 #define PB_SI_PB_LTYPE_MSG_W_CB(t) PB_SUBMSG_DESCRIPTOR(t)
00748 #define PB_SI_PB_LTYPE_SFIXED32(t)
00749 #define PB_SI_PB_LTYPE_SFIXED64(t)
00750 #define PB_SI_PB_LTYPE_SINT32(t)
00751 #define PB_SI_PB_LTYPE_SINT64(t)
00752 #define PB_SI_PB_LTYPE_STRING(t)
00753 #define PB_SI_PB_LTYPE_UINT32(t)
00754 #define PB_SI_PB_LTYPE_UINT64(t)
00755 #define PB_SI_PB_LTYPE_EXTENSION(t)
00756 #define PB_SI_PB_LTYPE_FIXED_LENGTH_BYTES(t)
00757 #define PB_SUBMSG_DESCRIPTOR(t)    &(t ## _msg),
00758
00759 /* The field descriptors use a variable width format, with width of either
00760 * 1, 2, 4 or 8 of 32-bit words. The two lowest bytes of the first byte always
00761 * encode the descriptor size, 6 lowest bits of field tag number, and 8 bits
00762 * of the field type.
00763 *
00764 * Descriptor size is encoded as 0 = 1 word, 1 = 2 words, 2 = 4 words, 3 = 8 words.
00765 *
00766 * Formats, listed starting with the least significant bit of the first word.
00767 * 1 word: [2-bit len] [6-bit tag] [8-bit type] [8-bit data_offset] [4-bit size_offset] [4-bit data_size]
00768 *
00769 * 2 words: [2-bit len] [6-bit tag] [8-bit type] [12-bit array_size] [4-bit size_offset]

```



```

00770 *      [16-bit data_offset] [12-bit data_size] [4-bit tag]»6]
00771 *
00772 * 4 words: [2-bit len] [6-bit tag] [8-bit type] [16-bit array_size]
00773 *      [8-bit size_offset] [24-bit tag]»6]
00774 *      [32-bit data_offset]
00775 *      [32-bit data_size]
00776 *
00777 * 8 words: [2-bit len] [6-bit tag] [8-bit type] [16-bit reserved]
00778 *      [8-bit size_offset] [24-bit tag]»6]
00779 *      [32-bit data_offset]
00780 *      [32-bit data_size]
00781 *      [32-bit array_size]
00782 *      [32-bit reserved]
00783 *      [32-bit reserved]
00784 *      [32-bit reserved]
00785 */
00786
00787 #define PB_FIELDINFO_1(tag, type, data_offset, data_size, size_offset, array_size) \
00788     (0 | (((uint32_t)(tag) < 2) & 0xFF) | ((type) < 8) | (((uint32_t)(data_offset) & 0xFF) < 16) | \
00789     (((uint32_t)(size_offset) & 0x0F) < 24) | (((uint32_t)(data_size) & 0x0F) < 28)), \
00790
00791 #define PB_FIELDINFO_2(tag, type, data_offset, data_size, size_offset, array_size) \
00792     (1 | (((uint32_t)(tag) < 2) & 0xFF) | ((type) < 8) | (((uint32_t)(array_size) & 0xFFF) < 16) | (((uint32_t)(size_offset) \
00793     & 0x0F) < 28)), \
00794     (((uint32_t)(data_offset) & 0xFFFF) | (((uint32_t)(data_size) & 0xFFF) < 16) | (((uint32_t)(tag) & 0x3c0) < 22)), \
00795
00796 #define PB_FIELDINFO_4(tag, type, data_offset, data_size, size_offset, array_size) \
00797     (2 | (((uint32_t)(tag) < 2) & 0xFF) | ((type) < 8) | (((uint32_t)(array_size) & 0xFFFF) < 16)), \
00798     ((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) < 2) & 0xFFFFF00)), \
00799     (data_offset), (data_size),
00800
00801 #define PB_FIELDINFO_8(tag, type, data_offset, data_size, size_offset, array_size) \
00802     (3 | (((uint32_t)(tag) < 2) & 0xFF) | ((type) < 8)), \
00803     ((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) < 2) & 0xFFFFF00)), \
00804     (data_offset), (data_size), (array_size), 0, 0, 0,
00805
00806 /* These assertions verify that the field information fits in the allocated space.
00807 * The generator tries to automatically determine the correct width that can fit all
00808 * data associated with a message. These asserts will fail only if there has been a
00809 * problem in the automatic logic - this may be worth reporting as a bug. As a workaround,
00810 * you can increase the descriptor width by defining PB_FIELDINFO_WIDTH or by setting
00811 * descriptorsize option in .options file.
00812 */
00813 #define PB_FITS(value, bits) ((uint32_t)(value) < ((uint32_t)1<bits))
00814 #define PB_FIELDINFO_ASSERT_1(tag, type, data_offset, data_size, size_offset, array_size) \
00815     PB_STATIC_ASSERT(PB_FITS(tag, 6) && PB_FITS(data_offset, 8) && PB_FITS(size_offset, 4) && \
00816     PB_FITS(data_size, 4) && PB_FITS(array_size, 1), FIELDINFO_DOES_NOT_FIT_width1_field ## tag)
00817
00818 #define PB_FIELDINFO_ASSERT_2(tag, type, data_offset, data_size, size_offset, array_size) \
00819     PB_STATIC_ASSERT(PB_FITS(tag, 10) && PB_FITS(data_offset, 16) && PB_FITS(size_offset, 4) && \
00820     PB_FITS(data_size, 12) && PB_FITS(array_size, 12), FIELDINFO_DOES_NOT_FIT_width2_field ## tag)
00821
00822 #ifndef PB_FIELD_32BIT
00823 /* Maximum field sizes are still 16-bit if pb_size_t is 16-bit */
00824 #define PB_FIELDINFO_ASSERT_4(tag, type, data_offset, data_size, size_offset, array_size) \
00825     PB_STATIC_ASSERT(PB_FITS(tag, 16) && PB_FITS(data_offset, 16) && PB_FITS((int_least8_t)size_offset, 8) \
00826     && PB_FITS(data_size, 16) && PB_FITS(array_size, 16), FIELDINFO_DOES_NOT_FIT_width4_field ## tag)
00827
00828 #define PB_FIELDINFO_ASSERT_8(tag, type, data_offset, data_size, size_offset, array_size) \
00829     PB_STATIC_ASSERT(PB_FITS(tag, 16) && PB_FITS(data_offset, 16) && PB_FITS((int_least8_t)size_offset, 8) \
00830     && PB_FITS(data_size, 16) && PB_FITS(array_size, 16), FIELDINFO_DOES_NOT_FIT_width8_field ## tag)
00831
00832 #else
00833 /* Up to 32-bit fields supported.
00834 * Note that the checks are against 31 bits to avoid compiler warnings about shift wider than type in the test.
00835 * I expect that there is no reasonable use for >2GB messages with nanopb anyway.
00836 */
00837 #define PB_FIELDINFO_ASSERT_4(tag, type, data_offset, data_size, size_offset, array_size) \
00838     PB_STATIC_ASSERT(PB_FITS(tag, 30) && PB_FITS(data_offset, 31) && PB_FITS(size_offset, 8) && \
00839     PB_FITS(data_size, 31) && PB_FITS(array_size, 16), FIELDINFO_DOES_NOT_FIT_width4_field ## tag)
00840
00841 #define PB_FIELDINFO_ASSERT_8(tag, type, data_offset, data_size, size_offset, array_size) \
00842     PB_STATIC_ASSERT(PB_FITS(tag, 30) && PB_FITS(data_offset, 31) && PB_FITS(size_offset, 8) && \
00843     PB_FITS(data_size, 31) && PB_FITS(array_size, 31), FIELDINFO_DOES_NOT_FIT_width8_field ## tag)
00844
00845 #endif
00846
00847 /* Automatic picking of FIELDINFO width:
00848 * Uses width 1 when possible, otherwise resorts to width 2.
00849 * This is used when PB_BIND() is called with "AUTO" as the argument.
00850 * The generator will give explicit size argument when it knows that a message
00851 * structure grows beyond 1-word format limits.
00852 */
00853 #define PB_FIELDINFO_WIDTH_AUTO(atype, htype, ltype) PB_FI_WIDTH ## atype(htype, ltype)
00854 #define PB_FI_WIDTH_PB_ATYPE_STATIC(htype, ltype) PB_FI_WIDTH ## htype(ltype)
00855 #define PB_FI_WIDTH_PB_ATYPE_POINTER(htype, ltype) PB_FI_WIDTH ## htype(ltype)
00856 #define PB_FI_WIDTH_PB_ATYPE_CALLBACK(htype, ltype) 2
00857 #define PB_FI_WIDTH_PB_HTYPE_REQUIRED(ltype) PB_FI_WIDTH ## ltype

```

```

00850 #define PB_FI_WIDTH_PB_HTYPE_SINGULAR(ltype) PB_FI_WIDTH ## ltype
00851 #define PB_FI_WIDTH_PB_HTYPE_OPTIONAL(ltype) PB_FI_WIDTH ## ltype
00852 #define PB_FI_WIDTH_PB_HTYPE_ONEOF(ltype) PB_FI_WIDTH ## ltype
00853 #define PB_FI_WIDTH_PB_HTYPE_REPEATED(ltype) 2
00854 #define PB_FI_WIDTH_PB_HTYPE_FIXARRAY(ltype) 2
00855 #define PB_FI_WIDTH_PB_LTYPE_BOOL 1
00856 #define PB_FI_WIDTH_PB_LTYPE_BYTES 2
00857 #define PB_FI_WIDTH_PB_LTYPE_DOUBLE 1
00858 #define PB_FI_WIDTH_PB_LTYPE_ENUM 1
00859 #define PB_FI_WIDTH_PB_LTYPE_UENUM 1
00860 #define PB_FI_WIDTH_PB_LTYPE_FIXED32 1
00861 #define PB_FI_WIDTH_PB_LTYPE_FIXED64 1
00862 #define PB_FI_WIDTH_PB_LTYPE_FLOAT 1
00863 #define PB_FI_WIDTH_PB_LTYPE_INT32 1
00864 #define PB_FI_WIDTH_PB_LTYPE_INT64 1
00865 #define PB_FI_WIDTH_PB_LTYPE_MESSAGE 2
00866 #define PB_FI_WIDTH_PB_LTYPE_MSG_W_CB 2
00867 #define PB_FI_WIDTH_PB_LTYPE_SFIXED32 1
00868 #define PB_FI_WIDTH_PB_LTYPE_SFIXED64 1
00869 #define PB_FI_WIDTH_PB_LTYPE_SINT32 1
00870 #define PB_FI_WIDTH_PB_LTYPE_SINT64 1
00871 #define PB_FI_WIDTH_PB_LTYPE_STRING 2
00872 #define PB_FI_WIDTH_PB_LTYPE_UINT32 1
00873 #define PB_FI_WIDTH_PB_LTYPE_UINT64 1
00874 #define PB_FI_WIDTH_PB_LTYPE_EXTENSION 1
00875 #define PB_FI_WIDTH_PB_LTYPE_FIXED_LENGTH_BYTES 2
00876
00877 /* The mapping from protobuf types to LTYPEs is done using these macros. */
00878 #define PB_LTYPE_MAP_BOOL PB_LTYPE_BOOL
00879 #define PB_LTYPE_MAP_BYTES PB_LTYPE_BYTES
00880 #define PB_LTYPE_MAP_DOUBLE PB_LTYPE_FIXED64
00881 #define PB_LTYPE_MAP_ENUM PB_LTYPE_VARINT
00882 #define PB_LTYPE_MAP_UENUM PB_LTYPE_UVARINT
00883 #define PB_LTYPE_MAP_FIXED32 PB_LTYPE_FIXED32
00884 #define PB_LTYPE_MAP_FIXED64 PB_LTYPE_FIXED64
00885 #define PB_LTYPE_MAP_FLOAT PB_LTYPE_FIXED32
00886 #define PB_LTYPE_MAP_INT32 PB_LTYPE_VARINT
00887 #define PB_LTYPE_MAP_INT64 PB_LTYPE_VARINT
00888 #define PB_LTYPE_MAP_MESSAGE PB_LTYPE_SUBMESSAGE
00889 #define PB_LTYPE_MAP_MSG_W_CB PB_LTYPE_SUBMSG_W_CB
00890 #define PB_LTYPE_MAP_SFIXED32 PB_LTYPE_FIXED32
00891 #define PB_LTYPE_MAP_SFIXED64 PB_LTYPE_FIXED64
00892 #define PB_LTYPE_MAP_SINT32 PB_LTYPE_SVARINT
00893 #define PB_LTYPE_MAP_SINT64 PB_LTYPE_SVARINT
00894 #define PB_LTYPE_MAP_STRING PB_LTYPE_STRING
00895 #define PB_LTYPE_MAP_UINT32 PB_LTYPE_UVARINT
00896 #define PB_LTYPE_MAP_UINT64 PB_LTYPE_UVARINT
00897 #define PB_LTYPE_MAP_EXTENSION PB_LTYPE_EXTENSION
00898 #define PB_LTYPE_MAP_FIXED_LENGTH_BYTES PB_LTYPE_FIXED_LENGTH_BYTES
00899
00900 /* These macros are used for giving out error messages.
00901 * They are mostly a debugging aid; the main error information
00902 * is the true/false return value from functions.
00903 * Some code space can be saved by disabling the error
00904 * messages if not used.
00905 *
00906 * PB_SET_ERROR() sets the error message if none has been set yet.
00907 * msg must be a constant string literal.
00908 * PB_GET_ERROR() always returns a pointer to a string.
00909 * PB_RETURN_ERROR() sets the error and returns false from current
00910 * function.
00911 */
00912 #ifndef PB_NO_ERRMSG
00913 #define PB_SET_ERROR(stream, msg) PB_UNUSED(stream)
00914 #define PB_GET_ERROR(stream) "(errmsg disabled)"
00915 #else
00916 #define PB_SET_ERROR(stream, msg) (stream->errmsg = (stream)->errmsg ? (stream)->errmsg : (msg))
00917 #define PB_GET_ERROR(stream) ((stream)->errmsg ? (stream)->errmsg : "(none)")
00918 #endif
00919
00920 #define PB_RETURN_ERROR(stream, msg) return PB_SET_ERROR(stream, msg), false
00921
00922 #ifdef __cplusplus
00923 } /* extern "C" */
00924 #endif
00925
00926 #ifdef __cplusplus
00927 #if __cplusplus >= 201103L
00928 #define PB_CONSTEXPR constexpr
00929 #else // __cplusplus >= 201103L
00930 #define PB_CONSTEXPR
00931 #endif // __cplusplus >= 201103L
00932
00933 #if __cplusplus >= 201703L
00934 #define PB_INLINE_CONSTEXPR inline constexpr
00935 #else // __cplusplus >= 201703L
00936 #define PB_INLINE_CONSTEXPR PB_CONSTEXPR

```

```

00937 #endif // __cplusplus >= 201703L
00938
00939 extern "C++"
00940 {
00941     namespace nanopb {
00942         // Each type will be partially specialized by the generator.
00943         template <typename GenMessageType> struct MessageDescriptor;
00944     } // namespace nanopb
00945 }
00946 #endif /* __cplusplus */
00947
00948 #endif

```

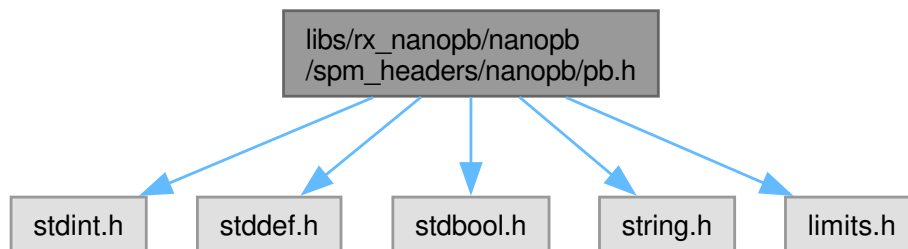
6.81 libs/rx_nanopb/nanopb/spm_headers/nanopb/pb.h File Reference

```

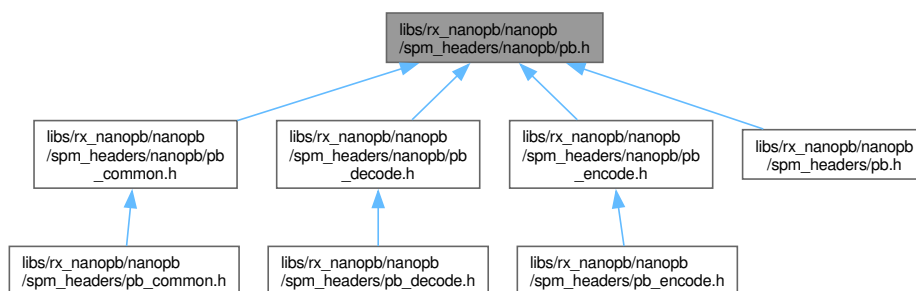
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>
#include <string.h>
#include <limits.h>

```

Include dependency graph for pb.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [pb_msgdesc_t](#)
- struct [pb_field_iter_t](#)
- struct [pb_bytes_array_t](#)
- struct [pb_callback_t](#)
- struct [pb_extension_type_t](#)
- struct [pb_extension_t](#)
- union [pb_callback_s.funcs](#)

Macros

- `#define NANOPB_VERSION "nanopb-1.0.0-dev"`
- `#define PB_PACKED_STRUCT_START`
- `#define PB_PACKED_STRUCT_END`
- `#define pb_packed`
- `#define pb_noinline`
- `#define PB_UNUSED(x)`
- `#define PB_PROGMEM`
- `#define PB_PROGMEM_READU32(x)`
- `#define PB_STATIC_ASSERT(COND, MSG)`
- `#define PB_MAX_REQUIRED_FIELDS 64`
- `#define PB_LTYPE_BOOL 0x00U /* bool */`
- `#define PB_LTYPE_VARINT 0x01U /* int32, int64, enum, bool */`
- `#define PB_LTYPE_UVARINT 0x02U /* uint32, uint64 */`
- `#define PB_LTYPE_SVARINT 0x03U /* sint32, sint64 */`
- `#define PB_LTYPE_FIXED32 0x04U /* fixed32, sfixed32, float */`
- `#define PB_LTYPE_FIXED64 0x05U /* fixed64, sfixed64, double */`
- `#define PB_LTYPE_LAST_PACKABLE 0x05U`
- `#define PB_LTYPE_BYTES 0x06U`
- `#define PB_LTYPE_STRING 0x07U`
- `#define PB_LTYPE_SUBMESSAGE 0x08U`
- `#define PB_LTYPE_SUBMSG_W_CB 0x09U`
- `#define PB_LTYPE_EXTENSION 0x0AU`
- `#define PB_LTYPE_FIXED_LENGTH_BYTES 0x0BU`
- `#define PB_LTYPES_COUNT 0x0CU`
- `#define PB_LTYPE_MASK 0x0FU`
- `#define PB_HTYPE_REQUIRED 0x00U`
- `#define PB_HTYPE_OPTIONAL 0x10U`
- `#define PB_HTYPE_SINGULAR 0x10U`
- `#define PB_HTYPE_REPEATED 0x20U`
- `#define PB_HTYPE_FIXARRAY 0x20U`
- `#define PB_HTYPE_ONEOF 0x30U`
- `#define PB_HTYPE_MASK 0x30U`
- `#define PB_ATYPE_STATIC 0x00U`
- `#define PB_ATYPE_POINTER 0x80U`
- `#define PB_ATYPE_CALLBACK 0x40U`
- `#define PB_ATYPE_MASK 0xC0U`
- `#define PB_ATYPE(x)`
- `#define PB_HTYPE(x)`
- `#define PB_LTYPE(x)`
- `#define PB_LTYPE_IS_SUBMSG(x)`
- `#define PB_SIZE_MAX ((pb_size_t)-1)`
- `#define PB_BYTES_ARRAY_T(n)`
- `#define PB_BYTES_ARRAY_T_ALLOCSIZE(n)`
- `#define pb_extension_init_zero {NULL,NULL,NULL,false}`
- `#define PB_PROTO_HEADER_VERSION 40`
- `#define pb_membersize(st, m)`
- `#define pb_arraysize(st, m)`
- `#define pb_delta(st, m1, m2)`
- `#define PB_EXPAND(x)`
- `#define PB_BIND(msgname, structname, width)`
- `#define PB_GEN_FIELD_COUNT(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_GEN_REQ_FIELD_COUNT(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_GEN_LARGEST_TAG(structname, atype, htype, ltype, fieldname, tag)`

- `#define PB_GEN_FIELD_INFO_1(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_GEN_FIELD_INFO_2(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_GEN_FIELD_INFO_4(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_GEN_FIELD_INFO_8(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_GEN_FIELD_INFO_AUTO(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_FIELDINFO_AUTO2(width, tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_GEN_FIELD_INFO_ASSERT_1(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_GEN_FIELD_INFO_ASSERT_2(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_GEN_FIELD_INFO_ASSERT_4(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_GEN_FIELD_INFO_ASSERT_8(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_GEN_FIELD_INFO_ASSERT_AUTO(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_FIELDINFO_ASSERT_AUTO2(width, tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_ASSERT_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_DATA_OFFSET_STATIC(htype, structname, fieldname)`
- `#define PB_DATA_OFFSET_POINTER(htype, structname, fieldname)`
- `#define PB_DATA_OFFSET_CALLBACK(htype, structname, fieldname)`
- `#define PB_DO_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_DO_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_DO_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_DO_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_DO_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_DO_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_SIZE_OFFSET_STATIC(htype, structname, fieldname)`
- `#define PB_SIZE_OFFSET_POINTER(htype, structname, fieldname)`
- `#define PB_SIZE_OFFSET_CALLBACK(htype, structname, fieldname)`
- `#define PB_SO_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_SO_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_SO_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_SO_PB_HTYPE_ONEOF2(structname, fullname, unionname)`
- `#define PB_SO_PB_HTYPE_ONEOF3(structname, fullname, unionname)`
- `#define PB_SO_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_SO_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_SO_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_SO_PTR_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_SO_PTR_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_SO_PTR_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_SO_PTR_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_SO_PTR_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_SO_PTR_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_SO_CB_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_ARRAY_SIZE_STATIC(htype, structname, fieldname)`
- `#define PB_ARRAY_SIZE_POINTER(htype, structname, fieldname)`
- `#define PB_ARRAY_SIZE_CALLBACK(htype, structname, fieldname)`
- `#define PB_AS_PB_HTYPE_REQUIRED(structname, fieldname)`

- `#define PB_AS_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_AS_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_AS_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_AS_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_AS_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_AS_PTR_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_DATA_SIZE_STATIC(htype, structname, fieldname)`
- `#define PB_DATA_SIZE_POINTER(htype, structname, fieldname)`
- `#define PB_DATA_SIZE_CALLBACK(htype, structname, fieldname)`
- `#define PB_DS_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_DS_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_DS_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_DS_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_DS_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_DS_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_DS_PTR_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_REQUIRED(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_SINGULAR(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_OPTIONAL(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_ONEOF(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_REPEATED(structname, fieldname)`
- `#define PB_DS_CB_PB_HTYPE_FIXARRAY(structname, fieldname)`
- `#define PB_ONEOF_NAME(type, tuple)`
- `#define PB_ONEOF_NAME_UNION(unionname, membername, fullname)`
- `#define PB_ONEOF_NAME_MEMBER(unionname, membername, fullname)`
- `#define PB_ONEOF_NAME_FULL(unionname, membername, fullname)`
- `#define PB_GEN_SUBMSG_INFO(structname, atype, htype, ltype, fieldname, tag)`
- `#define PB_SUBMSG_INFO_REQUIRED(ltype, structname, fieldname)`
- `#define PB_SUBMSG_INFO_SINGULAR(ltype, structname, fieldname)`
- `#define PB_SUBMSG_INFO_OPTIONAL(ltype, structname, fieldname)`
- `#define PB_SUBMSG_INFO_ONEOF(ltype, structname, fieldname)`
- `#define PB_SUBMSG_INFO_ONEOF2(ltype, structname, unionname, membername)`
- `#define PB_SUBMSG_INFO_ONEOF3(ltype, structname, unionname, membername)`
- `#define PB_SUBMSG_INFO_REPEATED(ltype, structname, fieldname)`
- `#define PB_SUBMSG_INFO_FIXARRAY(ltype, structname, fieldname)`
- `#define PB_SI_PB_LTYPE_BOOL(t)`
- `#define PB_SI_PB_LTYPE_BYTES(t)`
- `#define PB_SI_PB_LTYPE_DOUBLE(t)`
- `#define PB_SI_PB_LTYPE_ENUM(t)`
- `#define PB_SI_PB_LTYPE_UENUM(t)`
- `#define PB_SI_PB_LTYPE_FIXED32(t)`
- `#define PB_SI_PB_LTYPE_FIXED64(t)`
- `#define PB_SI_PB_LTYPE_FLOAT(t)`
- `#define PB_SI_PB_LTYPE_INT32(t)`
- `#define PB_SI_PB_LTYPE_INT64(t)`

- `#define PB_SI_PB_LTYPE_MESSAGE(t)`
- `#define PB_SI_PB_LTYPE_MSG_W_CB(t)`
- `#define PB_SI_PB_LTYPE_SFIXED32(t)`
- `#define PB_SI_PB_LTYPE_SFIXED64(t)`
- `#define PB_SI_PB_LTYPE_SINT32(t)`
- `#define PB_SI_PB_LTYPE_SINT64(t)`
- `#define PB_SI_PB_LTYPE_STRING(t)`
- `#define PB_SI_PB_LTYPE_UINT32(t)`
- `#define PB_SI_PB_LTYPE_UINT64(t)`
- `#define PB_SI_PB_LTYPE_EXTENSION(t)`
- `#define PB_SI_PB_LTYPE_FIXED_LENGTH_BYTES(t)`
- `#define PB_SUBMSG_DESCRIPTOR(t)`
- `#define PB_FIELDINFO_1(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_2(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_4(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_8(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FITS(value, bits)`
- `#define PB_FIELDINFO_ASSERT_1(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_ASSERT_2(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_ASSERT_4(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_ASSERT_8(tag, type, data_offset, data_size, size_offset, array_size)`
- `#define PB_FIELDINFO_WIDTH_AUTO(atype, htype, ltype)`
- `#define PB_FI_WIDTH_PB_ATYPE_STATIC(htype, ltype)`
- `#define PB_FI_WIDTH_PB_ATYPE_POINTER(htype, ltype)`
- `#define PB_FI_WIDTH_PB_ATYPE_CALLBACK(htype, ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_REQUIRED(ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_SINGULAR(ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_OPTIONAL(ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_ONEOF(ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_REPEATED(ltype)`
- `#define PB_FI_WIDTH_PB_HTYPE_FIXARRAY(ltype)`
- `#define PB_FI_WIDTH_PB_LTYPE_BOOL 1`
- `#define PB_FI_WIDTH_PB_LTYPE_BYTES 2`
- `#define PB_FI_WIDTH_PB_LTYPE_DOUBLE 1`
- `#define PB_FI_WIDTH_PB_LTYPE_ENUM 1`
- `#define PB_FI_WIDTH_PB_LTYPE_UENUM 1`
- `#define PB_FI_WIDTH_PB_LTYPE_FIXED32 1`
- `#define PB_FI_WIDTH_PB_LTYPE_FIXED64 1`
- `#define PB_FI_WIDTH_PB_LTYPE_FLOAT 1`
- `#define PB_FI_WIDTH_PB_LTYPE_INT32 1`
- `#define PB_FI_WIDTH_PB_LTYPE_INT64 1`
- `#define PB_FI_WIDTH_PB_LTYPE_MESSAGE 2`
- `#define PB_FI_WIDTH_PB_LTYPE_MSG_W_CB 2`
- `#define PB_FI_WIDTH_PB_LTYPE_SFIXED32 1`
- `#define PB_FI_WIDTH_PB_LTYPE_SFIXED64 1`
- `#define PB_FI_WIDTH_PB_LTYPE_SINT32 1`
- `#define PB_FI_WIDTH_PB_LTYPE_SINT64 1`
- `#define PB_FI_WIDTH_PB_LTYPE_STRING 2`
- `#define PB_FI_WIDTH_PB_LTYPE_UINT32 1`
- `#define PB_FI_WIDTH_PB_LTYPE_UINT64 1`
- `#define PB_FI_WIDTH_PB_LTYPE_EXTENSION 1`
- `#define PB_FI_WIDTH_PB_LTYPE_FIXED_LENGTH_BYTES 2`
- `#define PB_LTYPE_MAP_BOOL PB_LTYPE_BOOL`
- `#define PB_LTYPE_MAP_BYTES PB_LTYPE_BYTES`
- `#define PB_LTYPE_MAP_DOUBLE PB_LTYPE_FIXED64`

- `#define PB_LTYPE_MAP_ENUM PB_LTYPE_VARINT`
- `#define PB_LTYPE_MAP_UENUM PB_LTYPE_UVARINT`
- `#define PB_LTYPE_MAP_FIXED32 PB_LTYPE_FIXED32`
- `#define PB_LTYPE_MAP_FIXED64 PB_LTYPE_FIXED64`
- `#define PB_LTYPE_MAP_FLOAT PB_LTYPE_FIXED32`
- `#define PB_LTYPE_MAP_INT32 PB_LTYPE_VARINT`
- `#define PB_LTYPE_MAP_INT64 PB_LTYPE_VARINT`
- `#define PB_LTYPE_MAP_MESSAGE PB_LTYPE_SUBMESSAGE`
- `#define PB_LTYPE_MAP_MSG_W_CB PB_LTYPE_SUBMSG_W_CB`
- `#define PB_LTYPE_MAP_SFIXED32 PB_LTYPE_FIXED32`
- `#define PB_LTYPE_MAP_SFIXED64 PB_LTYPE_FIXED64`
- `#define PB_LTYPE_MAP_SINT32 PB_LTYPE_SVARINT`
- `#define PB_LTYPE_MAP_SINT64 PB_LTYPE_SVARINT`
- `#define PB_LTYPE_MAP_STRING PB_LTYPE_STRING`
- `#define PB_LTYPE_MAP_UINT32 PB_LTYPE_UVARINT`
- `#define PB_LTYPE_MAP_UINT64 PB_LTYPE_UVARINT`
- `#define PB_LTYPE_MAP_EXTENSION PB_LTYPE_EXTENSION`
- `#define PB_LTYPE_MAP_FIXED_LENGTH_BYTES PB_LTYPE_FIXED_LENGTH_BYTES`
- `#define PB_SET_ERROR(stream, msg)`
- `#define PB_GET_ERROR(stream)`
- `#define PB_RETURN_ERROR(stream, msg)`

Enumerations

- `enum pb_wire_type_t {
PB_WT_VARINT = 0 , PB_WT_64BIT = 1 , PB_WT_STRING = 2 , PB_WT_32BIT = 5 ,
PB_WT_PACKED = 255 }`

Functions

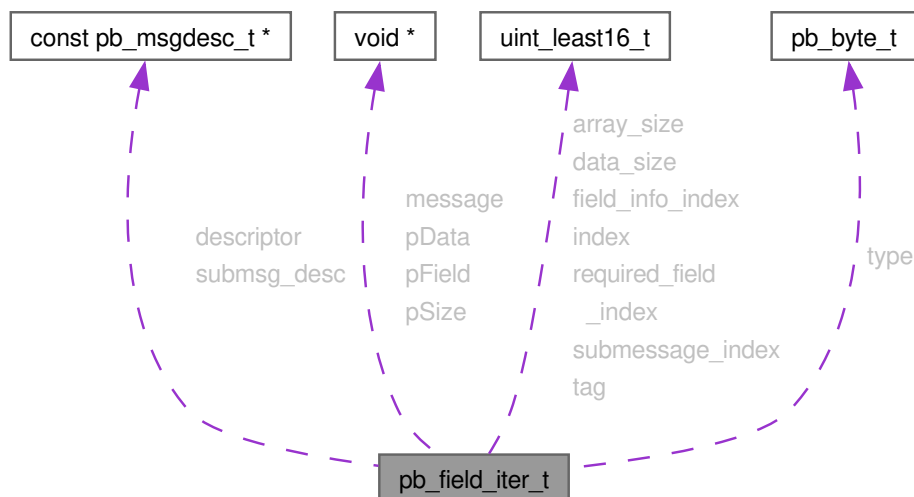
- `bool pb_default_field_callback (pb_istream_t *istream, pb_ostream_t *ostream, const pb_field_t *field)`

6.81.1 Data Structure Documentation

6.81.1.1 struct pb_field_iter_s

Definition at line 363 of file [pb.h](#).

Collaboration diagram for pb_field_iter_t:



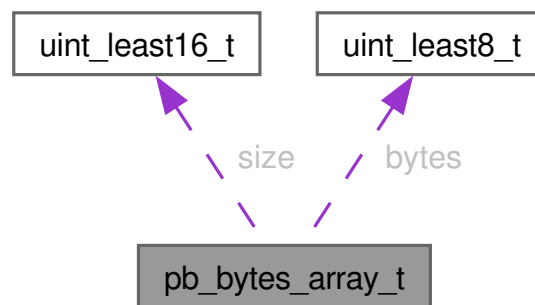
Data Fields

pb_size_t	array_size	
pb_size_t	data_size	
const pb_msgdesc_t *	descriptor	
pb_size_t	field_info_index	
pb_size_t	index	
void *	message	
void *	pData	
void *	pField	
void *	pSize	
pb_size_t	required_field_index	
pb_size_t	submessage_index	
const pb_msgdesc_t *	submsg_desc	
pb_size_t	tag	
pb_type_t	type	

6.81.1.2 struct pb_bytes_array_s

Definition at line 405 of file [pb.h](#).

Collaboration diagram for pb_bytes_array_t:



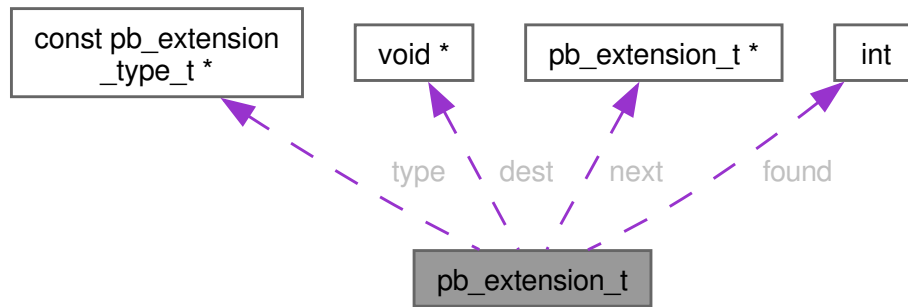
Data Fields

pb_byte_t	bytes[1]	
pb_size_t	size	

6.81.1.3 struct pb_extension_s

Definition at line 484 of file [pb.h](#).

Collaboration diagram for pb_extension_t:



Data Fields

void *	dest	
bool	found	
pb_extension_t *	next	
const pb_extension_type_t *	type	

6.81.2 Macro Definition Documentation

6.81.2.1 NANOPB_VERSION

```
#define NANOPB_VERSION "nanopb-1.0.0-dev"
```

Definition at line 71 of file [pb.h](#).

6.81.2.2 PB_ARRAY_SIZE_CALLBACK

```
#define PB_ARRAY_SIZE_CALLBACK(
    htype,
    structname,
    fieldname)
```

Value:

1

Definition at line 684 of file [pb.h](#).

6.81.2.3 PB_ARRAY_SIZE_POINTER

```
#define PB_ARRAY_SIZE_POINTER(
    htype,
    structname,
    fieldname)
```

Value:

PB_AS_PTR ## htype(structname, fieldname)

Definition at line 683 of file [pb.h](#).

6.81.2.4 PB_ARRAY_SIZE_STATIC

```
#define PB_ARRAY_SIZE_STATIC(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_AS ## htype(structname, fieldname)

Definition at line 682 of file [pb.h](#).

6.81.2.5 pb_arraysize

```
#define pb_arraysize(  
    st,  
    m)
```

Value:

(pb_membersize(st, m) / pb_membersize(st, m[0]))

Definition at line 523 of file [pb.h](#).

6.81.2.6 PB_AS_PB_HTYPE_FIXARRAY

```
#define PB_AS_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

pb_arraysize(structname, fieldname)

Definition at line 690 of file [pb.h](#).

6.81.2.7 PB_AS_PB_HTYPE_ONEOF

```
#define PB_AS_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 688 of file [pb.h](#).

6.81.2.8 PB_AS_PB_HTYPE_OPTIONAL

```
#define PB_AS_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 687 of file [pb.h](#).

6.81.2.9 PB_AS_PB_HTYPE_REPEATED

```
#define PB_AS_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

`pb_arraysize(structname, fieldname)`

Definition at line 689 of file [pb.h](#).

6.81.2.10 PB_AS_PB_HTYPE_REQUIRED

```
#define PB_AS_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 685 of file [pb.h](#).

6.81.2.11 PB_AS_PB_HTYPE_SINGULAR

```
#define PB_AS_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 686 of file [pb.h](#).

6.81.2.12 PB_AS_PTR_PB_HTYPE_FIXARRAY

```
#define PB_AS_PTR_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

`pb_arraysize(structname, fieldname[0])`

Definition at line 696 of file [pb.h](#).

6.81.2.13 PB_AS_PTR_PB_HTYPE_ONEOF

```
#define PB_AS_PTR_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 694 of file [pb.h](#).

6.81.2.14 PB_AS_PTR_PB_HTYPE_OPTIONAL

```
#define PB_AS_PTR_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 693 of file [pb.h](#).

6.81.2.15 PB_AS_PTR_PB_HTYPE_REPEATED

```
#define PB_AS_PTR_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 695 of file [pb.h](#).

6.81.2.16 PB_AS_PTR_PB_HTYPE_REQUIRED

```
#define PB_AS_PTR_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 691 of file [pb.h](#).

6.81.2.17 PB_AS_PTR_PB_HTYPE_SINGULAR

```
#define PB_AS_PTR_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

1

Definition at line 692 of file [pb.h](#).

6.81.2.18 PB_ATYPE

```
#define PB_ATYPE(  
    x)
```

Value:

((x) & PB_ATYPE_MASK)

Definition at line 323 of file [pb.h](#).

6.81.2.19 PB__ATYPE__CALLBACK

```
#define PB__ATYPE__CALLBACK 0x40U
```

Definition at line 320 of file [pb.h](#).

6.81.2.20 PB__ATYPE__MASK

```
#define PB__ATYPE__MASK 0xC0U
```

Definition at line 321 of file [pb.h](#).

6.81.2.21 PB__ATYPE__POINTER

```
#define PB__ATYPE__POINTER 0x80U
```

Definition at line 319 of file [pb.h](#).

6.81.2.22 PB__ATYPE__STATIC

```
#define PB__ATYPE__STATIC 0x00U
```

Definition at line 318 of file [pb.h](#).

6.81.2.23 PB__BIND

```
#define PB__BIND(  
    msgname,  
    structname,  
    width)
```

Value:

```
const uint32_t structname ## _field_info[] PB_PROGMEM = \
{ \
    msgname ## _FIELDLIST(PB_GEN_FIELD_INFO_ ## width, structname) \
    0 \
}; \
const pb_msgdesc_t* const structname ## _submsg_info[] = \
{ \
    msgname ## _FIELDLIST(PB_GEN_SUBMSG_INFO, structname) \
    NULL \
}; \
const pb_msgdesc_t structname ## _msg = \
{ \
    structname ## _field_info, \
    structname ## _submsg_info, \
    msgname ## _DEFAULT, \
    msgname ## _CALLBACK, \
    0 msgname ## _FIELDLIST(PB_GEN_FIELD_COUNT, structname), \
    0 msgname ## _FIELDLIST(PB_GEN_REQ_FIELD_COUNT, structname), \
    0 msgname ## _FIELDLIST(PB_GEN_LARGEST_TAG, structname), \
}; \
msgname ## _FIELDLIST(PB_GEN_FIELD_INFO_ASSERT_ ## width, structname)
```

Definition at line 531 of file [pb.h](#).

```
00531 #define PB__BIND(msgname, structname, width) \
00532     const uint32_t structname ## _field_info[] PB_PROGMEM = \
00533     { \
00534         msgname ## _FIELDLIST(PB_GEN_FIELD_INFO_ ## width, structname) \
00535         0 \
```

```

00536     }; \
00537     const pb_msgdesc_t* const structname ## _submsg_info[] = \
00538     { \
00539         msgname ## _FIELDLIST(PB_GEN_SUBMSG_INFO, structname) \
00540         NULL \
00541     }; \
00542     const pb_msgdesc_t structname ## _msg = \
00543     { \
00544         structname ## _field_info, \
00545         structname ## _submsg_info, \
00546         msgname ## _DEFAULT, \
00547         msgname ## _CALLBACK, \
00548         0 msgname ## _FIELDLIST(PB_GEN_FIELD_COUNT, structname), \
00549         0 msgname ## _FIELDLIST(PB_GEN_REQ_FIELD_COUNT, structname), \
00550         0 msgname ## _FIELDLIST(PB_GEN_LARGEST_TAG, structname), \
00551     }; \
00552     msgname ## _FIELDLIST(PB_GEN_FIELD_INFO_ASSERT_ ## width, structname)

```

6.81.2.24 PB_BYTES_ARRAY_T

```

#define PB_BYTES_ARRAY_T(
    n)

```

Value:

```

struct { pb_size_t size; pb_byte_t bytes[n]; }

```

Definition at line 402 of file [pb.h](#).

6.81.2.25 PB_BYTES_ARRAY_T_ALLOCSIZE

```

#define PB_BYTES_ARRAY_T_ALLOCSIZE(
    n)

```

Value:

```

((size_t)n + offsetof(pb_bytes_array_t, bytes))

```

Definition at line 403 of file [pb.h](#).

6.81.2.26 PB_DATA_OFFSET_CALLBACK

```

#define PB_DATA_OFFSET_CALLBACK(
    htype,
    structname,
    fieldname)

```

Value:

```

PB_DO ## htype(structname, fieldname)

```

Definition at line 650 of file [pb.h](#).

6.81.2.27 PB_DATA_OFFSET_POINTER

```

#define PB_DATA_OFFSET_POINTER(
    htype,
    structname,
    fieldname)

```

Value:

```

PB_DO ## htype(structname, fieldname)

```

Definition at line 649 of file [pb.h](#).

6.81.2.28 PB_DATA_OFFSET_STATIC

```
#define PB_DATA_OFFSET_STATIC(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_DO ## htype(structname, fieldname)

Definition at line 648 of file [pb.h](#).

6.81.2.29 PB_DATA_SIZE_CALLBACK

```
#define PB_DATA_SIZE_CALLBACK(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_DS_CB ## htype(structname, fieldname)

Definition at line 700 of file [pb.h](#).

6.81.2.30 PB_DATA_SIZE_POINTER

```
#define PB_DATA_SIZE_POINTER(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_DS_PTR ## htype(structname, fieldname)

Definition at line 699 of file [pb.h](#).

6.81.2.31 PB_DATA_SIZE_STATIC

```
#define PB_DATA_SIZE_STATIC(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_DS ## htype(structname, fieldname)

Definition at line 698 of file [pb.h](#).

6.81.2.32 pb_delta

```
#define pb_delta(  
    st,  
    m1,  
    m2)
```

Value:

`((int)offsetof(st, m1) - (int)offsetof(st, m2))`

Definition at line 525 of file [pb.h](#).

6.81.2.33 PB_DO_PB_HTYPE_FIXARRAY

```
#define PB_DO_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, fieldname)`

Definition at line 656 of file [pb.h](#).

6.81.2.34 PB_DO_PB_HTYPE_ONEOF

```
#define PB_DO_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, PB_ONEOF_NAME(FULL, fieldname))`

Definition at line 653 of file [pb.h](#).

6.81.2.35 PB_DO_PB_HTYPE_OPTIONAL

```
#define PB_DO_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, fieldname)`

Definition at line 654 of file [pb.h](#).

6.81.2.36 PB_DO_PB_HTYPE_REPEATED

```
#define PB_DO_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, fieldname)`

Definition at line 655 of file [pb.h](#).

6.81.2.37 PB_DO_PB_HTYPE_REQUIRED

```
#define PB_DO_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, fieldname)`

Definition at line 651 of file [pb.h](#).

6.81.2.38 PB_DO_PB_HTYPE_SINGULAR

```
#define PB_DO_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

`offsetof(structname, fieldname)`

Definition at line 652 of file [pb.h](#).

6.81.2.39 PB_DS_CB_PB_HTYPE_FIXARRAY

```
#define PB_DS_CB_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname)`

Definition at line 718 of file [pb.h](#).

6.81.2.40 PB_DS_CB_PB_HTYPE_ONEOF

```
#define PB_DS_CB_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, PB_ONEOF_NAME(FULL, fieldname))`

Definition at line 716 of file [pb.h](#).

6.81.2.41 PB_DS_CB_PB_HTYPE_OPTIONAL

```
#define PB_DS_CB_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

`pb_membersize(structname, fieldname)`

Definition at line 715 of file [pb.h](#).

6.81.2.42 PB_DS_CB_PB_HTYPE_REPEATED

```
#define PB_DS_CB_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname)

Definition at line 717 of file [pb.h](#).

6.81.2.43 PB_DS_CB_PB_HTYPE_REQUIRED

```
#define PB_DS_CB_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname)

Definition at line 713 of file [pb.h](#).

6.81.2.44 PB_DS_CB_PB_HTYPE_SINGULAR

```
#define PB_DS_CB_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname)

Definition at line 714 of file [pb.h](#).

6.81.2.45 PB_DS_PB_HTYPE_FIXARRAY

```
#define PB_DS_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname[0])

Definition at line 706 of file [pb.h](#).

6.81.2.46 PB_DS_PB_HTYPE_ONEOF

```
#define PB_DS_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, PB_ONEOF_NAME(FULL, fieldname))

Definition at line 704 of file [pb.h](#).

6.81.2.47 PB_DS_PB_HTYPE_OPTIONAL

```
#define PB_DS_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname)

Definition at line 703 of file [pb.h](#).

6.81.2.48 PB_DS_PB_HTYPE_REPEATED

```
#define PB_DS_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname[0])

Definition at line 705 of file [pb.h](#).

6.81.2.49 PB_DS_PB_HTYPE_REQUIRED

```
#define PB_DS_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname)

Definition at line 701 of file [pb.h](#).

6.81.2.50 PB_DS_PB_HTYPE_SINGULAR

```
#define PB_DS_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname)

Definition at line 702 of file [pb.h](#).

6.81.2.51 PB_DS_PTR_PB_HTYPE_FIXARRAY

```
#define PB_DS_PTR_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname[0][0])

Definition at line 712 of file [pb.h](#).

6.81.2.52 PB_DS_PTR_PB_HTYPE_ONEOF

```
#define PB_DS_PTR_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, PB_ONEOF_NAME(FULL, fieldname)[0])

Definition at line 710 of file [pb.h](#).

6.81.2.53 PB_DS_PTR_PB_HTYPE_OPTIONAL

```
#define PB_DS_PTR_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname[0])

Definition at line 709 of file [pb.h](#).

6.81.2.54 PB_DS_PTR_PB_HTYPE_REPEATED

```
#define PB_DS_PTR_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname[0])

Definition at line 711 of file [pb.h](#).

6.81.2.55 PB_DS_PTR_PB_HTYPE_REQUIRED

```
#define PB_DS_PTR_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname[0])

Definition at line 707 of file [pb.h](#).

6.81.2.56 PB_DS_PTR_PB_HTYPE_SINGULAR

```
#define PB_DS_PTR_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

pb_membersize(structname, fieldname[0])

Definition at line 708 of file [pb.h](#).

6.81.2.57 PB_EXPAND

```
#define PB_EXPAND(  
    x)
```

Value:

x

Definition at line 528 of file [pb.h](#).

6.81.2.58 pb_extension_init_zero

```
#define pb_extension_init_zero {NULL,NULL,NULL,false}
```

Definition at line 503 of file [pb.h](#).

6.81.2.59 PB_FI_WIDTH_PB_ATYPE_CALLBACK

```
#define PB_FI_WIDTH_PB_ATYPE_CALLBACK(  
    htype,  
    ltype)
```

Value:

2

Definition at line 848 of file [pb.h](#).

6.81.2.60 PB_FI_WIDTH_PB_ATYPE_POINTER

```
#define PB_FI_WIDTH_PB_ATYPE_POINTER(  
    htype,  
    ltype)
```

Value:

PB_FI_WIDTH ## htype(ltype)

Definition at line 847 of file [pb.h](#).

6.81.2.61 PB_FI_WIDTH_PB_ATYPE_STATIC

```
#define PB_FI_WIDTH_PB_ATYPE_STATIC(  
    htype,  
    ltype)
```

Value:

PB_FI_WIDTH ## htype(ltype)

Definition at line 846 of file [pb.h](#).

6.81.2.62 PB_FI_WIDTH_PB_HTYPE_FIXARRAY

```
#define PB_FI_WIDTH_PB_HTYPE_FIXARRAY(  
    ltype)
```

Value:

2

Definition at line 854 of file [pb.h](#).

6.81.2.63 PB_FI_WIDTH_PB_HTYPE_ONEOF

```
#define PB_FI_WIDTH_PB_HTYPE_ONEOF(  
    ltype)
```

Value:

PB_FI_WIDTH ## ltype

Definition at line 852 of file [pb.h](#).

6.81.2.64 PB_FI_WIDTH_PB_HTYPE_OPTIONAL

```
#define PB_FI_WIDTH_PB_HTYPE_OPTIONAL(  
    ltype)
```

Value:

PB_FI_WIDTH ## ltype

Definition at line 851 of file [pb.h](#).

6.81.2.65 PB_FI_WIDTH_PB_HTYPE_REPEATED

```
#define PB_FI_WIDTH_PB_HTYPE_REPEATED(  
    ltype)
```

Value:

2

Definition at line 853 of file [pb.h](#).

6.81.2.66 PB_FI_WIDTH_PB_HTYPE_REQUIRED

```
#define PB_FI_WIDTH_PB_HTYPE_REQUIRED(  
    ltype)
```

Value:

PB_FI_WIDTH ## ltype

Definition at line 849 of file [pb.h](#).

6.81.2.67 PB_FI_WIDTH_PB_HTYPE_SINGULAR

```
#define PB_FI_WIDTH_PB_HTYPE_SINGULAR(  
    ltype)
```

Value:

PB_FI_WIDTH ## ltype

Definition at line 850 of file [pb.h](#).

6.81.2.68 PB_FI_WIDTH_PB_LTYPE_BOOL

```
#define PB_FI_WIDTH_PB_LTYPE_BOOL 1
```

Definition at line 855 of file [pb.h](#).

6.81.2.69 PB_FI_WIDTH_PB_LTYPE_BYTES

```
#define PB_FI_WIDTH_PB_LTYPE_BYTES 2
```

Definition at line 856 of file [pb.h](#).

6.81.2.70 PB_FI_WIDTH_PB_LTYPE_DOUBLE

```
#define PB_FI_WIDTH_PB_LTYPE_DOUBLE 1
```

Definition at line 857 of file [pb.h](#).

6.81.2.71 PB_FI_WIDTH_PB_LTYPE_ENUM

```
#define PB_FI_WIDTH_PB_LTYPE_ENUM 1
```

Definition at line 858 of file [pb.h](#).

6.81.2.72 PB_FI_WIDTH_PB_LTYPE_EXTENSION

```
#define PB_FI_WIDTH_PB_LTYPE_EXTENSION 1
```

Definition at line 874 of file [pb.h](#).

6.81.2.73 PB_FI_WIDTH_PB_LTYPE_FIXED32

```
#define PB_FI_WIDTH_PB_LTYPE_FIXED32 1
```

Definition at line 860 of file [pb.h](#).

6.81.2.74 PB_FI_WIDTH_PB_LTYPE_FIXED64

```
#define PB_FI_WIDTH_PB_LTYPE_FIXED64 1
```

Definition at line 861 of file [pb.h](#).

6.81.2.75 PB_FI_WIDTH_PB_LTYPE_FIXED_LENGTH_BYTES

```
#define PB_FI_WIDTH_PB_LTYPE_FIXED_LENGTH_BYTES 2
```

Definition at line 875 of file [pb.h](#).

6.81.2.76 PB_FI_WIDTH_PB_LTYPE_FLOAT

```
#define PB_FI_WIDTH_PB_LTYPE_FLOAT 1
```

Definition at line 862 of file [pb.h](#).

6.81.2.77 PB_FI_WIDTH_PB_LTYPE_INT32

```
#define PB_FI_WIDTH_PB_LTYPE_INT32 1
```

Definition at line 863 of file [pb.h](#).

6.81.2.78 PB_FI_WIDTH_PB_LTYPE_INT64

```
#define PB_FI_WIDTH_PB_LTYPE_INT64 1
```

Definition at line 864 of file [pb.h](#).

6.81.2.79 PB_FI_WIDTH_PB_LTYPE_MESSAGE

```
#define PB_FI_WIDTH_PB_LTYPE_MESSAGE 2
```

Definition at line 865 of file [pb.h](#).

6.81.2.80 PB_FI_WIDTH_PB_LTYPE_MSG_W_CB

```
#define PB_FI_WIDTH_PB_LTYPE_MSG_W_CB 2
```

Definition at line 866 of file [pb.h](#).

6.81.2.81 PB_FI_WIDTH_PB_LTYPE_SFIXED32

```
#define PB_FI_WIDTH_PB_LTYPE_SFIXED32 1
```

Definition at line 867 of file [pb.h](#).

6.81.2.82 PB_FI_WIDTH_PB_LTYPE_SFIED64

```
#define PB_FI_WIDTH_PB_LTYPE_SFIED64 1
```

Definition at line 868 of file [pb.h](#).

6.81.2.83 PB_FI_WIDTH_PB_LTYPE_SINT32

```
#define PB_FI_WIDTH_PB_LTYPE_SINT32 1
```

Definition at line 869 of file [pb.h](#).

6.81.2.84 PB_FI_WIDTH_PB_LTYPE_SINT64

```
#define PB_FI_WIDTH_PB_LTYPE_SINT64 1
```

Definition at line 870 of file [pb.h](#).

6.81.2.85 PB_FI_WIDTH_PB_LTYPE_STRING

```
#define PB_FI_WIDTH_PB_LTYPE_STRING 2
```

Definition at line 871 of file [pb.h](#).

6.81.2.86 PB_FI_WIDTH_PB_LTYPE_UENUM

```
#define PB_FI_WIDTH_PB_LTYPE_UENUM 1
```

Definition at line 859 of file [pb.h](#).

6.81.2.87 PB_FI_WIDTH_PB_LTYPE_UINT32

```
#define PB_FI_WIDTH_PB_LTYPE_UINT32 1
```

Definition at line 872 of file [pb.h](#).

6.81.2.88 PB_FI_WIDTH_PB_LTYPE_UINT64

```
#define PB_FI_WIDTH_PB_LTYPE_UINT64 1
```

Definition at line 873 of file [pb.h](#).

6.81.2.89 PB_FIELDINFO_1

```
#define PB_FIELDINFO_1(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
(0 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(data_offset) & 0xFF) « 16) | \
(((uint32_t)(size_offset) & 0x0F) « 24) | (((uint32_t)(data_size) & 0x0F) « 28)),
```

Definition at line 787 of file [pb.h](#).

```
00787 #define PB_FIELDINFO_1(tag, type, data_offset, data_size, size_offset, array_size) \
00788     (0 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(data_offset) & 0xFF) « 16) | \
00789     (((uint32_t)(size_offset) & 0x0F) « 24) | (((uint32_t)(data_size) & 0x0F) « 28)),
```

6.81.2.90 PB_FIELDINFO_2

```
#define PB_FIELDINFO_2(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
(1 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(array_size) & 0xFFF) « 16) | (((uint32_t)(size_offset) & 0x0F) « 28)), \
(((uint32_t)(data_offset) & 0xFFFF) | (((uint32_t)(data_size) & 0xFFF) « 16) | (((uint32_t)(tag) & 0x3c0) « 22)),
```

Definition at line 791 of file [pb.h](#).

```
00791 #define PB_FIELDINFO_2(tag, type, data_offset, data_size, size_offset, array_size) \
00792     (1 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(array_size) & 0xFFF) « 16) | (((uint32_t)(size_offset) & 0x0F) « 28)), \
00793     (((uint32_t)(data_offset) & 0xFFFF) | (((uint32_t)(data_size) & 0xFFF) « 16) | (((uint32_t)(tag) & 0x3c0) « 22)),
```

6.81.2.91 PB_FIELDINFO_4

```
#define PB_FIELDINFO_4(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
(2 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(array_size) & 0xFFFF) « 16)), \
((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) « 2) & 0xFFFFFFF0)), \
(data_offset), (data_size),
```

Definition at line 795 of file [pb.h](#).

```
00795 #define PB_FIELDINFO_4(tag, type, data_offset, data_size, size_offset, array_size) \
00796     (2 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(array_size) & 0xFFFF) « 16)), \
00797     ((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) « 2) & 0xFFFFFFF0)), \
00798     (data_offset), (data_size),
```

6.81.2.92 PB_FIELDINFO_8

```
#define PB_FIELDINFO_8(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
(3 | (((uint32_t)(tag) << 2) & 0xFF) | ((type) << 8)), \
((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) << 2) & 0xFFFFFFF0)), \
(data_offset), (data_size), (array_size), 0, 0, 0,
```

Definition at line 800 of file [pb.h](#).

```
00800 #define PB_FIELDINFO_8(tag, type, data_offset, data_size, size_offset, array_size) \
00801     (3 | (((uint32_t)(tag) << 2) & 0xFF) | ((type) << 8)), \
00802     ((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) << 2) & 0xFFFFFFF0)), \
00803     (data_offset), (data_size), (array_size), 0, 0, 0,
```

6.81.2.93 PB_FIELDINFO_ASSERT_1

```
#define PB_FIELDINFO_ASSERT_1(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
PB_STATIC_ASSERT(PB_FITS(tag,6) && PB_FITS(data_offset,8) && PB_FITS(size_offset,4) && PB_FITS(data_size,4)
    && PB_FITS(array_size,1), FIELDINFO_DOES_NOT_FIT_width1_field ## tag)
```

Definition at line 813 of file [pb.h](#).

```
00813 #define PB_FIELDINFO_ASSERT_1(tag, type, data_offset, data_size, size_offset, array_size) \
00814     PB_STATIC_ASSERT(PB_FITS(tag,6) && PB_FITS(data_offset,8) && PB_FITS(size_offset,4) &&
    PB_FITS(data_size,4) && PB_FITS(array_size,1), FIELDINFO_DOES_NOT_FIT_width1_field ## tag)
```

6.81.2.94 PB_FIELDINFO_ASSERT_2

```
#define PB_FIELDINFO_ASSERT_2(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
PB_STATIC_ASSERT(PB_FITS(tag,10) && PB_FITS(data_offset,16) && PB_FITS(size_offset,4) &&
    PB_FITS(data_size,12) && PB_FITS(array_size,12), FIELDINFO_DOES_NOT_FIT_width2_field ## tag)
```

Definition at line 816 of file [pb.h](#).

```
00816 #define PB_FIELDINFO_ASSERT_2(tag, type, data_offset, data_size, size_offset, array_size) \
00817     PB_STATIC_ASSERT(PB_FITS(tag,10) && PB_FITS(data_offset,16) && PB_FITS(size_offset,4) &&
    PB_FITS(data_size,12) && PB_FITS(array_size,12), FIELDINFO_DOES_NOT_FIT_width2_field ## tag)
```


6.81.2.95 PB_FIELDINFO_ASSERT_4

```
#define PB_FIELDINFO_ASSERT_4(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
PB_STATIC_ASSERT(PB_FITS(tag,16) && PB_FITS(data_offset,16) && PB_FITS((int_least8_t)size_offset,8) &&
    PB_FITS(data_size,16) && PB_FITS(array_size,16), FIELDINFO_DOES_NOT_FIT_width4_field ## tag)
```

Definition at line 821 of file [pb.h](#).

```
00821 #define PB_FIELDINFO_ASSERT_4(tag, type, data_offset, data_size, size_offset, array_size) \
00822     PB_STATIC_ASSERT(PB_FITS(tag,16) && PB_FITS(data_offset,16) && PB_FITS((int_least8_t)size_offset,8)
    && PB_FITS(data_size,16) && PB_FITS(array_size,16), FIELDINFO_DOES_NOT_FIT_width4_field ## tag)
```

6.81.2.96 PB_FIELDINFO_ASSERT_8

```
#define PB_FIELDINFO_ASSERT_8(
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
PB_STATIC_ASSERT(PB_FITS(tag,16) && PB_FITS(data_offset,16) && PB_FITS((int_least8_t)size_offset,8) &&
    PB_FITS(data_size,16) && PB_FITS(array_size,16), FIELDINFO_DOES_NOT_FIT_width8_field ## tag)
```

Definition at line 824 of file [pb.h](#).

```
00824 #define PB_FIELDINFO_ASSERT_8(tag, type, data_offset, data_size, size_offset, array_size) \
00825     PB_STATIC_ASSERT(PB_FITS(tag,16) && PB_FITS(data_offset,16) && PB_FITS((int_least8_t)size_offset,8)
    && PB_FITS(data_size,16) && PB_FITS(array_size,16), FIELDINFO_DOES_NOT_FIT_width8_field ## tag)
```

6.81.2.97 PB_FIELDINFO_ASSERT_AUTO2

```
#define PB_FIELDINFO_ASSERT_AUTO2(
    width,
    tag,
    type,
    data_offset,
    data_size,
    size_offset,
    array_size)
```

Value:

```
PB_FIELDINFO_ASSERT_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)
```

Definition at line 642 of file [pb.h](#).

```
00642 #define PB_FIELDINFO_ASSERT_AUTO2(width, tag, type, data_offset, data_size, size_offset, array_size) \
00643     PB_FIELDINFO_ASSERT_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)
```

6.81.2.98 PB_FIELDINFO_ASSERT_AUTO3

```
#define PB_FIELDINFO_ASSERT_AUTO3(  
    width,  
    tag,  
    type,  
    data_offset,  
    data_size,  
    size_offset,  
    array_size)
```

Value:

```
PB_FIELDINFO_ASSERT_ ## width(tag, type, data_offset, data_size, size_offset, array_size)
```

Definition at line 645 of file [pb.h](#).

```
00645 #define PB_FIELDINFO_ASSERT_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size) \  
00646     PB_FIELDINFO_ASSERT_ ## width(tag, type, data_offset, data_size, size_offset, array_size)
```

6.81.2.99 PB_FIELDINFO_AUTO2

```
#define PB_FIELDINFO_AUTO2(  
    width,  
    tag,  
    type,  
    data_offset,  
    data_size,  
    size_offset,  
    array_size)
```

Value:

```
PB_FIELDINFO_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)
```

Definition at line 597 of file [pb.h](#).

```
00597 #define PB_FIELDINFO_AUTO2(width, tag, type, data_offset, data_size, size_offset, array_size) \  
00598     PB_FIELDINFO_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)
```

6.81.2.100 PB_FIELDINFO_AUTO3

```
#define PB_FIELDINFO_AUTO3(  
    width,  
    tag,  
    type,  
    data_offset,  
    data_size,  
    size_offset,  
    array_size)
```

Value:

```
PB_FIELDINFO_ ## width(tag, type, data_offset, data_size, size_offset, array_size)
```

Definition at line 600 of file [pb.h](#).

```
00600 #define PB_FIELDINFO_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size) \  
00601     PB_FIELDINFO_ ## width(tag, type, data_offset, data_size, size_offset, array_size)
```

6.81.2.101 PB_FIELDINFO_WIDTH_AUTO

```
#define PB_FIELDINFO_WIDTH_AUTO(
    atype,
    htype,
    ltype)
```

Value:

PB_FI_WIDTH ## atype(htype, ltype)

Definition at line 845 of file [pb.h](#).

6.81.2.102 PB_FITS

```
#define PB_FITS(
    value,
    bits)
```

Value:

((uint32_t)(value) < ((uint32_t)1«bits))

Definition at line 812 of file [pb.h](#).

6.81.2.103 PB_GEN_FIELD_COUNT

```
#define PB_GEN_FIELD_COUNT(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

+1

Definition at line 554 of file [pb.h](#).

6.81.2.104 PB_GEN_FIELD_INFO_1

```
#define PB_GEN_FIELD_INFO_1(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 561 of file [pb.h](#).

```
00561 #define PB_GEN_FIELD_INFO_1(structname, atype, htype, ltype, fieldname, tag) \
00562     PB_FIELDINFO_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00563         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00564         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00565         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00566         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.81.2.105 PB_GEN_FIELD_INFO_2

```
#define PB_GEN_FIELD_INFO_2(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 568 of file [pb.h](#).

```
00568 #define PB_GEN_FIELD_INFO_2(structname, atype, htype, ltype, fieldname, tag) \
00569     PB_FIELDINFO_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00570         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00571         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00572         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00573         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.81.2.106 PB_GEN_FIELD_INFO_4

```
#define PB_GEN_FIELD_INFO_4(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 575 of file [pb.h](#).

```
00575 #define PB_GEN_FIELD_INFO_4(structname, atype, htype, ltype, fieldname, tag) \
00576     PB_FIELDINFO_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00577         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00578         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00579         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00580         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.81.2.107 PB_GEN_FIELD_INFO_8

```
#define PB_GEN_FIELD_INFO_8(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 582 of file [pb.h](#).

```
00582 #define PB_GEN_FIELD_INFO_8(structname, atype, htype, ltype, fieldname, tag) \
00583     PB_FIELDINFO_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00584         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00585         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00586         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00587         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.81.2.108 PB_GEN_FIELD_INFO_ASSERT_1

```
#define PB_GEN_FIELD_INFO_ASSERT_1(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_ASSERT_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 606 of file [pb.h](#).

```
00606 #define PB_GEN_FIELD_INFO_ASSERT_1(structname, atype, htype, ltype, fieldname, tag) \
00607     PB_FIELDINFO_ASSERT_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ##
    ltype, \
00608         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00609         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00610         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00611         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.81.2.109 PB_GEN_FIELD_INFO_ASSERT_2

```
#define PB_GEN_FIELD_INFO_ASSERT_2(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_ASSERT_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 613 of file [pb.h](#).

```
00613 #define PB_GEN_FIELD_INFO_ASSERT_2(structname, atype, htype, ltype, fieldname, tag) \
00614     PB_FIELDINFO_ASSERT_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ##
    ltype, \
00615         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00616         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00617         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00618         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.81.2.110 PB_GEN_FIELD_INFO_ASSERT_4

```
#define PB_GEN_FIELD_INFO_ASSERT_4(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_ASSERT_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 620 of file [pb.h](#).

```
00620 #define PB_GEN_FIELD_INFO_ASSERT_4(structname, atype, htype, ltype, fieldname, tag) \
00621     PB_FIELDINFO_ASSERT_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ##
    ltype, \
00622         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00623         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00624         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00625         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.81.2.111 PB_GEN_FIELD_INFO_ASSERT_8

```
#define PB_GEN_FIELD_INFO_ASSERT_8(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_ASSERT_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 627 of file [pb.h](#).

```
00627 #define PB_GEN_FIELD_INFO_ASSERT_8(structname, atype, htype, ltype, fieldname, tag) \
00628     PB_FIELDINFO_ASSERT_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ##
    ltype, \
00629         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00630         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00631         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00632         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.81.2.112 PB_GEN_FIELD_INFO_ASSERT_AUTO

```
#define PB_GEN_FIELD_INFO_ASSERT_AUTO(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_ASSERT_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_ ##
    htype, _PB_LTYPE_ ## ltype), \
    tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 634 of file [pb.h](#).

```
00634 #define PB_GEN_FIELD_INFO_ASSERT_AUTO(structname, atype, htype, ltype, fieldname, tag) \
00635     PB_FIELDINFO_ASSERT_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_
    ## htype, _PB_LTYPE_ ## ltype), \
00636         tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00637         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00638         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00639         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00640         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.81.2.113 PB_GEN_FIELD_INFO_AUTO

```
#define PB_GEN_FIELD_INFO_AUTO(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_FIELDINFO_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_ ## htype,
    _PB_LTYPE_ ## ltype), \
    tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
    PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
    PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

Definition at line 589 of file [pb.h](#).

```
00589 #define PB_GEN_FIELD_INFO_AUTO(structname, atype, htype, ltype, fieldname, tag) \
00590     PB_FIELDINFO_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_ ## htype,
    _PB_LTYPE_ ## ltype), \
00591         tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00592         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00593         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00594         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00595         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
```

6.81.2.114 PB_GEN_LARGEST_TAG

```
#define PB_GEN_LARGEST_TAG(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

* 0 + tag

Definition at line 557 of file [pb.h](#).

```
00557 #define PB_GEN_LARGEST_TAG(structname, atype, htype, ltype, fieldname, tag) \
00558     * 0 + tag
```

6.81.2.115 PB_GEN_REQ_FIELD_COUNT

```
#define PB_GEN_REQ_FIELD_COUNT(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
+ (PB_HTYPE_ ## htype == PB_HTYPE_REQUIRED)
```

Definition at line 555 of file [pb.h](#).

```
00555 #define PB_GEN_REQ_FIELD_COUNT(structname, atype, htype, ltype, fieldname, tag) \
00556     + (PB_HTYPE_ ## htype == PB_HTYPE_REQUIRED)
```

6.81.2.116 PB_GEN_SUBMSG_INFO

```
#define PB_GEN_SUBMSG_INFO(
    structname,
    atype,
    htype,
    ltype,
    fieldname,
    tag)
```

Value:

```
PB_SUBMSG_INFO_ ## htype(_PB_LTYPE_ ## ltype, structname, fieldname)
```

Definition at line 725 of file [pb.h](#).

```
00725 #define PB_GEN_SUBMSG_INFO(structname, atype, htype, ltype, fieldname, tag) \
00726     PB_SUBMSG_INFO_ ## htype(_PB_LTYPE_ ## ltype, structname, fieldname)
```

6.81.2.117 PB_GET_ERROR

```
#define PB_GET_ERROR(
    stream)
```

Value:

```
((stream)->errmsg ? (stream)->errmsg : "(none)")
```

Definition at line 917 of file [pb.h](#).

6.81.2.118 PB_HTYPE

```
#define PB_HTYPE(
    x)
```

Value:

```
((x) & PB_HTYPE_MASK)
```

Definition at line 324 of file [pb.h](#).

6.81.2.119 PB_HTYPE_FIXARRAY

```
#define PB_HTYPE_FIXARRAY 0x20U
```

Definition at line 312 of file [pb.h](#).

6.81.2.120 PB_HTYPE_MASK

```
#define PB_HTYPE_MASK 0x30U
```

Definition at line 314 of file [pb.h](#).

6.81.2.121 PB_HTYPE_ONEOF

```
#define PB_HTYPE_ONEOF 0x30U
```

Definition at line 313 of file [pb.h](#).

6.81.2.122 PB_HTYPE_OPTIONAL

```
#define PB_HTYPE_OPTIONAL 0x10U
```

Definition at line 309 of file [pb.h](#).

6.81.2.123 PB_HTYPE_REPEATED

```
#define PB_HTYPE_REPEATED 0x20U
```

Definition at line 311 of file [pb.h](#).

6.81.2.124 PB_HTYPE_REQUIRED

```
#define PB_HTYPE_REQUIRED 0x00U
```

Definition at line 308 of file [pb.h](#).

6.81.2.125 PB_HTYPE_SINGULAR

```
#define PB_HTYPE_SINGULAR 0x10U
```

Definition at line 310 of file [pb.h](#).

6.81.2.126 PB_LTYPE

```
#define PB_LTYPE(  
    x)
```

Value:

((x) & PB_LTYPE_MASK)

Definition at line 325 of file [pb.h](#).

6.81.2.127 PB_LTYPE_BOOL

```
#define PB_LTYPE_BOOL 0x00U /* bool */
```

Definition at line 265 of file [pb.h](#).

6.81.2.128 PB_LTYPE_BYTES

```
#define PB_LTYPE_BYTES 0x06U
```

Definition at line 277 of file [pb.h](#).

6.81.2.129 PB_LTYPE_EXTENSION

```
#define PB_LTYPE_EXTENSION 0x0AU
```

Definition at line 294 of file [pb.h](#).

6.81.2.130 PB_LTYPE_FIXED32

```
#define PB_LTYPE_FIXED32 0x04U /* fixed32, sfixed32, float */
```

Definition at line 269 of file [pb.h](#).

6.81.2.131 PB_LTYPE_FIXED64

```
#define PB_LTYPE_FIXED64 0x05U /* fixed64, sfixed64, double */
```

Definition at line 270 of file [pb.h](#).

6.81.2.132 PB_LTYPE_FIXED_LENGTH_BYTES

```
#define PB_LTYPE_FIXED_LENGTH_BYTES 0x0BU
```

Definition at line 300 of file [pb.h](#).

6.81.2.133 PB_LTYPE_IS_SUBMSG

```
#define PB_LTYPE_IS_SUBMSG(  
    x)
```

Value:

```
(PB_LTYPE(x) == PB_LTYPE_SUBMESSAGE || \  
PB_LTYPE(x) == PB_LTYPE_SUBMSG_W_CB)
```

Definition at line 326 of file [pb.h](#).

```
00326 #define PB_LTYPE_IS_SUBMSG(x) (PB_LTYPE(x) == PB_LTYPE_SUBMESSAGE || \  
00327     PB_LTYPE(x) == PB_LTYPE_SUBMSG_W_CB)
```

6.81.2.134 PB_LTYPE_LAST_PACKABLE

```
#define PB_LTYPE_LAST_PACKABLE 0x05U
```

Definition at line 273 of file [pb.h](#).

6.81.2.135 PB_LTYPE_MAP_BOOL

```
#define PB_LTYPE_MAP_BOOL PB_LTYPE_BOOL
```

Definition at line 878 of file [pb.h](#).

6.81.2.136 PB_LTYPE_MAP_BYTES

```
#define PB_LTYPE_MAP_BYTES PB_LTYPE_BYTES
```

Definition at line 879 of file [pb.h](#).

6.81.2.137 PB_LTYPE_MAP_DOUBLE

```
#define PB_LTYPE_MAP_DOUBLE PB_LTYPE_FIXED64
```

Definition at line 880 of file [pb.h](#).

6.81.2.138 PB_LTYPE_MAP_ENUM

```
#define PB_LTYPE_MAP_ENUM PB_LTYPE_VARINT
```

Definition at line 881 of file [pb.h](#).

6.81.2.139 PB_LTYPE_MAP_EXTENSION

```
#define PB_LTYPE_MAP_EXTENSION PB_LTYPE_EXTENSION
```

Definition at line 897 of file [pb.h](#).

6.81.2.140 PB_LTYPE_MAP_FIXED32

```
#define PB_LTYPE_MAP_FIXED32 PB_LTYPE_FIXED32
```

Definition at line 883 of file [pb.h](#).

6.81.2.141 PB_LTYPE_MAP_FIXED64

```
#define PB_LTYPE_MAP_FIXED64 PB_LTYPE_FIXED64
```

Definition at line 884 of file [pb.h](#).

6.81.2.142 PB_LTYPE_MAP_FIXED_LENGTH_BYTES

```
#define PB_LTYPE_MAP_FIXED_LENGTH_BYTES PB_LTYPE_FIXED_LENGTH_BYTES
```

Definition at line 898 of file [pb.h](#).

6.81.2.143 PB_LTYPE_MAP_FLOAT

```
#define PB_LTYPE_MAP_FLOAT PB_LTYPE_FIXED32
```

Definition at line 885 of file [pb.h](#).

6.81.2.144 PB_LTYPE_MAP_INT32

```
#define PB_LTYPE_MAP_INT32 PB_LTYPE_VARINT
```

Definition at line 886 of file [pb.h](#).

6.81.2.145 PB_LTYPE_MAP_INT64

```
#define PB_LTYPE_MAP_INT64 PB_LTYPE_VARINT
```

Definition at line 887 of file [pb.h](#).

6.81.2.146 PB_LTYPE_MAP_MESSAGE

```
#define PB_LTYPE_MAP_MESSAGE PB_LTYPE_SUBMESSAGE
```

Definition at line 888 of file [pb.h](#).

6.81.2.147 PB_LTYPE_MAP_MSG_W_CB

```
#define PB_LTYPE_MAP_MSG_W_CB PB_LTYPE_SUBMSG_W_CB
```

Definition at line 889 of file [pb.h](#).

6.81.2.148 PB_LTYPE_MAP_SF32

```
#define PB_LTYPE_MAP_SF32 PB_LTYPE_FIXED32
```

Definition at line 890 of file [pb.h](#).

6.81.2.149 PB_LTYPE_MAP_SF64

```
#define PB_LTYPE_MAP_SF64 PB_LTYPE_FIXED64
```

Definition at line 891 of file [pb.h](#).

6.81.2.150 PB_LTYPE_MAP_SINT32

```
#define PB_LTYPE_MAP_SINT32 PB_LTYPE_SVARINT
```

Definition at line 892 of file [pb.h](#).

6.81.2.151 PB_LTYPE_MAP_SINT64

```
#define PB_LTYPE_MAP_SINT64 PB_LTYPE_SVARINT
```

Definition at line 893 of file [pb.h](#).

6.81.2.152 PB_LTYPE_MAP_STRING

```
#define PB_LTYPE_MAP_STRING PB_LTYPE_STRING
```

Definition at line 894 of file [pb.h](#).

6.81.2.153 PB_LTYPE_MAP_UENUM

```
#define PB_LTYPE_MAP_UENUM PB_LTYPE_UVARINT
```

Definition at line 882 of file [pb.h](#).

6.81.2.154 PB_LTYPE_MAP_UINT32

```
#define PB_LTYPE_MAP_UINT32 PB_LTYPE_UVARINT
```

Definition at line 895 of file [pb.h](#).

6.81.2.155 PB_LTYPE_MAP_UINT64

```
#define PB_LTYPE_MAP_UINT64 PB_LTYPE_UVARINT
```

Definition at line 896 of file [pb.h](#).

6.81.2.156 PB_LTYPE_MASK

```
#define PB_LTYPE_MASK 0x0FU
```

Definition at line [304](#) of file [pb.h](#).

6.81.2.157 PB_LTYPE_STRING

```
#define PB_LTYPE_STRING 0x07U
```

Definition at line [281](#) of file [pb.h](#).

6.81.2.158 PB_LTYPE_SUBMESSAGE

```
#define PB_LTYPE_SUBMESSAGE 0x08U
```

Definition at line [285](#) of file [pb.h](#).

6.81.2.159 PB_LTYPE_SUBMSG_W_CB

```
#define PB_LTYPE_SUBMSG_W_CB 0x09U
```

Definition at line [290](#) of file [pb.h](#).

6.81.2.160 PB_LTYPE_SVARINT

```
#define PB_LTYPE_SVARINT 0x03U /* sint32, sint64 */
```

Definition at line [268](#) of file [pb.h](#).

6.81.2.161 PB_LTYPE_UVARINT

```
#define PB_LTYPE_UVARINT 0x02U /* uint32, uint64 */
```

Definition at line [267](#) of file [pb.h](#).

6.81.2.162 PB_LTYPE_VARINT

```
#define PB_LTYPE_VARINT 0x01U /* int32, int64, enum, bool */
```

Definition at line [266](#) of file [pb.h](#).

6.81.2.163 PB_LTYPES_COUNT

```
#define PB_LTYPES_COUNT 0x0CU
```

Definition at line [303](#) of file [pb.h](#).

6.81.2.164 PB_MAX_REQUIRED_FIELDS

```
#define PB_MAX_REQUIRED_FIELDS 64
```

Definition at line 230 of file [pb.h](#).

6.81.2.165 pb_membersize

```
#define pb_membersize(  
    st,  
    m)
```

Value:

```
(sizeof ((st*)0)->m)
```

Definition at line 521 of file [pb.h](#).

6.81.2.166 pb_noinline

```
#define pb_noinline
```

Definition at line 147 of file [pb.h](#).

6.81.2.167 PB_ONEOF_NAME

```
#define PB_ONEOF_NAME(  
    type,  
    tuple)
```

Value:

```
PB_EXPAND(PB_ONEOF_NAME_ ## type tuple)
```

Definition at line 720 of file [pb.h](#).

6.81.2.168 PB_ONEOF_NAME_FULL

```
#define PB_ONEOF_NAME_FULL(  
    unionname,  
    membername,  
    fullname)
```

Value:

```
fullname
```

Definition at line 723 of file [pb.h](#).

6.81.2.169 PB_ONEOF_NAME_MEMBER

```
#define PB_ONEOF_NAME_MEMBER(  
    unionname,  
    membername,  
    fullname)
```

Value:

membername

Definition at line [722](#) of file [pb.h](#).

6.81.2.170 PB_ONEOF_NAME_UNION

```
#define PB_ONEOF_NAME_UNION(  
    unionname,  
    membername,  
    fullname)
```

Value:

unionname

Definition at line [721](#) of file [pb.h](#).

6.81.2.171 pb_packed

```
#define pb_packed
```

Definition at line [129](#) of file [pb.h](#).

6.81.2.172 PB_PACKED_STRUCT_END

```
#define PB_PACKED_STRUCT_END
```

Definition at line [128](#) of file [pb.h](#).

6.81.2.173 PB_PACKED_STRUCT_START

```
#define PB_PACKED_STRUCT_START
```

Definition at line [127](#) of file [pb.h](#).

6.81.2.174 PB_PROGMEM

```
#define PB_PROGMEM
```

Definition at line [180](#) of file [pb.h](#).

6.81.2.175 PB_PROGMEM_READU32

```
#define PB_PROGMEM_READU32(  
    x)
```

Value:

(x)

Definition at line 181 of file [pb.h](#).

6.81.2.176 PB_PROTO_HEADER_VERSION

```
#define PB_PROTO_HEADER_VERSION 40
```

Definition at line 517 of file [pb.h](#).

6.81.2.177 PB_RETURN_ERROR

```
#define PB_RETURN_ERROR(  
    stream,  
    msg)
```

Value:

```
return PB_SET_ERROR(stream, msg), false
```

Definition at line 920 of file [pb.h](#).

6.81.2.178 PB_SET_ERROR

```
#define PB_SET_ERROR(  
    stream,  
    msg)
```

Value:

```
(stream->errmsg = (stream)->errmsg ? (stream)->errmsg : (msg))
```

Definition at line 916 of file [pb.h](#).

6.81.2.179 PB_SI_PB_LTYPE_BOOL

```
#define PB_SI_PB_LTYPE_BOOL(  
    t)
```

Definition at line 736 of file [pb.h](#).

6.81.2.180 PB_SI_PB_LTYPE_BYTES

```
#define PB_SI_PB_LTYPE_BYTES(  
    t)
```

Definition at line 737 of file [pb.h](#).

6.81.2.181 PB_SI_PB_LTYPE_DOUBLE

```
#define PB_SI_PB_LTYPE_DOUBLE(  
    t)
```

Definition at line 738 of file [pb.h](#).

6.81.2.182 PB_SI_PB_LTYPE_ENUM

```
#define PB_SI_PB_LTYPE_ENUM(  
    t)
```

Definition at line 739 of file [pb.h](#).

6.81.2.183 PB_SI_PB_LTYPE_EXTENSION

```
#define PB_SI_PB_LTYPE_EXTENSION(  
    t)
```

Definition at line 755 of file [pb.h](#).

6.81.2.184 PB_SI_PB_LTYPE_FIXED32

```
#define PB_SI_PB_LTYPE_FIXED32(  
    t)
```

Definition at line 741 of file [pb.h](#).

6.81.2.185 PB_SI_PB_LTYPE_FIXED64

```
#define PB_SI_PB_LTYPE_FIXED64(  
    t)
```

Definition at line 742 of file [pb.h](#).

6.81.2.186 PB_SI_PB_LTYPE_FIXED_LENGTH_BYTES

```
#define PB_SI_PB_LTYPE_FIXED_LENGTH_BYTES(  
    t)
```

Definition at line 756 of file [pb.h](#).

6.81.2.187 PB_SI_PB_LTYPE_FLOAT

```
#define PB_SI_PB_LTYPE_FLOAT(  
    t)
```

Definition at line 743 of file [pb.h](#).

6.81.2.188 PB_SI_PB_LTYPE_INT32

```
#define PB_SI_PB_LTYPE_INT32(  
    t)
```

Definition at line 744 of file [pb.h](#).

6.81.2.189 PB_SI_PB_LTYPE_INT64

```
#define PB_SI_PB_LTYPE_INT64(  
    t)
```

Definition at line 745 of file [pb.h](#).

6.81.2.190 PB_SI_PB_LTYPE_MESSAGE

```
#define PB_SI_PB_LTYPE_MESSAGE(  
    t)
```

Value:

PB_SUBMSG_DESCRIPTOR(t)

Definition at line 746 of file [pb.h](#).

6.81.2.191 PB_SI_PB_LTYPE_MSG_W_CB

```
#define PB_SI_PB_LTYPE_MSG_W_CB(  
    t)
```

Value:

PB_SUBMSG_DESCRIPTOR(t)

Definition at line 747 of file [pb.h](#).

6.81.2.192 PB_SI_PB_LTYPE_SFIXED32

```
#define PB_SI_PB_LTYPE_SFIXED32(  
    t)
```

Definition at line 748 of file [pb.h](#).

6.81.2.193 PB_SI_PB_LTYPE_SFIXED64

```
#define PB_SI_PB_LTYPE_SFIXED64(  
    t)
```

Definition at line 749 of file [pb.h](#).

6.81.2.194 PB_SI_PB_LTYPE_SINT32

```
#define PB_SI_PB_LTYPE_SINT32(  
    t)
```

Definition at line 750 of file [pb.h](#).

6.81.2.195 PB_SI_PB_LTYPE_SINT64

```
#define PB_SI_PB_LTYPE_SINT64(  
    t)
```

Definition at line 751 of file [pb.h](#).

6.81.2.196 PB_SI_PB_LTYPE_STRING

```
#define PB_SI_PB_LTYPE_STRING(  
    t)
```

Definition at line 752 of file [pb.h](#).

6.81.2.197 PB_SI_PB_LTYPE_UENUM

```
#define PB_SI_PB_LTYPE_UENUM(  
    t)
```

Definition at line 740 of file [pb.h](#).

6.81.2.198 PB_SI_PB_LTYPE_UINT32

```
#define PB_SI_PB_LTYPE_UINT32(  
    t)
```

Definition at line 753 of file [pb.h](#).

6.81.2.199 PB_SI_PB_LTYPE_UINT64

```
#define PB_SI_PB_LTYPE_UINT64(  
    t)
```

Definition at line 754 of file [pb.h](#).

6.81.2.200 PB_SIZE_MAX

```
#define PB_SIZE_MAX ((pb\_size\_t)-1)
```

Definition at line 339 of file [pb.h](#).

6.81.2.201 PB_SIZE_OFFSET_CALLBACK

```
#define PB_SIZE_OFFSET_CALLBACK(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_SO_CB ## htype(structname, fieldname)

Definition at line 660 of file [pb.h](#).

6.81.2.202 PB_SIZE_OFFSET_POINTER

```
#define PB_SIZE_OFFSET_POINTER(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_SO_PTR ## htype(structname, fieldname)

Definition at line 659 of file [pb.h](#).

6.81.2.203 PB_SIZE_OFFSET_STATIC

```
#define PB_SIZE_OFFSET_STATIC(  
    htype,  
    structname,  
    fieldname)
```

Value:

PB_SO ## htype(structname, fieldname)

Definition at line 658 of file [pb.h](#).

6.81.2.204 PB_SO_CB_PB_HTYPE_FIXARRAY

```
#define PB_SO_CB_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 680 of file [pb.h](#).

6.81.2.205 PB_SO_CB_PB_HTYPE_ONEOF

```
#define PB_SO_CB_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

PB_SO_PB_HTYPE_ONEOF(structname, fieldname)

Definition at line 677 of file [pb.h](#).

6.81.2.206 PB_SO_CB_PB_HTYPE_OPTIONAL

```
#define PB_SO_CB_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 678 of file [pb.h](#).

6.81.2.207 PB_SO_CB_PB_HTYPE_REPEATED

```
#define PB_SO_CB_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 679 of file [pb.h](#).

6.81.2.208 PB_SO_CB_PB_HTYPE_REQUIRED

```
#define PB_SO_CB_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 675 of file [pb.h](#).

6.81.2.209 PB_SO_CB_PB_HTYPE_SINGULAR

```
#define PB_SO_CB_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 676 of file [pb.h](#).

6.81.2.210 PB_SO_PB_HTYPE_FIXARRAY

```
#define PB_SO_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 668 of file [pb.h](#).

6.81.2.211 PB_SO_PB_HTYPE_ONEOF

```
#define PB_SO_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

```
PB_SO_PB_HTYPE_ONEOF2(structname, PB_ONEOF_NAME(FULL, fieldname), PB_ONEOF_NAME(UNION,  
    fieldname))
```

Definition at line 663 of file [pb.h](#).

6.81.2.212 PB_SO_PB_HTYPE_ONEOF2

```
#define PB_SO_PB_HTYPE_ONEOF2(  
    structname,  
    fullname,  
    unionname)
```

Value:

```
PB_SO_PB_HTYPE_ONEOF3(structname, fullname, unionname)
```

Definition at line 664 of file [pb.h](#).

6.81.2.213 PB_SO_PB_HTYPE_ONEOF3

```
#define PB_SO_PB_HTYPE_ONEOF3(  
    structname,  
    fullname,  
    unionname)
```

Value:

```
pb_delta(structname, fullname, which_ ## unionname)
```

Definition at line 665 of file [pb.h](#).

6.81.2.214 PB_SO_PB_HTYPE_OPTIONAL

```
#define PB_SO_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

```
pb_delta(structname, fieldname, has_ ## fieldname)
```

Definition at line 666 of file [pb.h](#).

6.81.2.215 PB_SO_PB_HTYPE_REPEATED

```
#define PB_SO_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

pb_delta(structname, fieldname, fieldname##_count)

Definition at line 667 of file [pb.h](#).

6.81.2.216 PB_SO_PB_HTYPE_REQUIRED

```
#define PB_SO_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 661 of file [pb.h](#).

6.81.2.217 PB_SO_PB_HTYPE_SINGULAR

```
#define PB_SO_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 662 of file [pb.h](#).

6.81.2.218 PB_SO_PTR_PB_HTYPE_FIXARRAY

```
#define PB_SO_PTR_PB_HTYPE_FIXARRAY(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 674 of file [pb.h](#).

6.81.2.219 PB_SO_PTR_PB_HTYPE_ONEOF

```
#define PB_SO_PTR_PB_HTYPE_ONEOF(  
    structname,  
    fieldname)
```

Value:

PB_SO_PB_HTYPE_ONEOF(structname, fieldname)

Definition at line 671 of file [pb.h](#).

6.81.2.220 PB_SO_PTR_PB_HTYPE_OPTIONAL

```
#define PB_SO_PTR_PB_HTYPE_OPTIONAL(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 672 of file [pb.h](#).

6.81.2.221 PB_SO_PTR_PB_HTYPE_REPEATED

```
#define PB_SO_PTR_PB_HTYPE_REPEATED(  
    structname,  
    fieldname)
```

Value:

PB_SO_PB_HTYPE_REPEATED(structname, fieldname)

Definition at line 673 of file [pb.h](#).

6.81.2.222 PB_SO_PTR_PB_HTYPE_REQUIRED

```
#define PB_SO_PTR_PB_HTYPE_REQUIRED(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 669 of file [pb.h](#).

6.81.2.223 PB_SO_PTR_PB_HTYPE_SINGULAR

```
#define PB_SO_PTR_PB_HTYPE_SINGULAR(  
    structname,  
    fieldname)
```

Value:

0

Definition at line 670 of file [pb.h](#).

6.81.2.224 PB_STATIC_ASSERT

```
#define PB_STATIC_ASSERT(  
    COND,  
    MSG)
```

Value:

[__Static_assert](#)(COND, #MSG);

Definition at line 212 of file [pb.h](#).

6.81.2.225 PB_SUBMSG_DESCRIPTOR

```
#define PB_SUBMSG_DESCRIPTOR(  
    t)
```

Value:

```
&(t ## _msg),
```

Definition at line 757 of file [pb.h](#).

6.81.2.226 PB_SUBMSG_INFO_FIXARRAY

```
#define PB_SUBMSG_INFO_FIXARRAY(  
    ltype,  
    structname,  
    fieldname)
```

Value:

```
PB_SI ## ltype(structname ## _ ## fieldname ## _MSGTYPE)
```

Definition at line 735 of file [pb.h](#).

6.81.2.227 PB_SUBMSG_INFO_ONEOF

```
#define PB_SUBMSG_INFO_ONEOF(  
    ltype,  
    structname,  
    fieldname)
```

Value:

```
PB_SUBMSG_INFO_ONEOF2(ltype, structname, PB_ONEOF_NAME(UNION, fieldname), PB_ONEOF_NAME(MEMBER,  
    fieldname))
```

Definition at line 731 of file [pb.h](#).

6.81.2.228 PB_SUBMSG_INFO_ONEOF2

```
#define PB_SUBMSG_INFO_ONEOF2(  
    ltype,  
    structname,  
    unionname,  
    membername)
```

Value:

```
PB_SUBMSG_INFO_ONEOF3(ltype, structname, unionname, membername)
```

Definition at line 732 of file [pb.h](#).

6.81.2.229 PB_SUBMSG_INFO_ONEOF3

```
#define PB_SUBMSG_INFO_ONEOF3(  
    ltype,  
    structname,  
    unionname,  
    membername)
```

Value:

```
PB_SI ## ltype(structname ## _ ## unionname ## _ ## membername ## _MSGTYPE)
```

Definition at line 733 of file [pb.h](#).

6.81.2.230 PB_SUBMSG_INFO_OPTIONAL

```
#define PB_SUBMSG_INFO_OPTIONAL(  
    ltype,  
    structname,  
    fieldname)
```

Value:

```
PB_SI ## ltype(structname ## _ ## fieldname ## _MSGTYPE)
```

Definition at line 730 of file [pb.h](#).

6.81.2.231 PB_SUBMSG_INFO_REPEATED

```
#define PB_SUBMSG_INFO_REPEATED(  
    ltype,  
    structname,  
    fieldname)
```

Value:

```
PB_SI ## ltype(structname ## _ ## fieldname ## _MSGTYPE)
```

Definition at line 734 of file [pb.h](#).

6.81.2.232 PB_SUBMSG_INFO_REQUIRED

```
#define PB_SUBMSG_INFO_REQUIRED(  
    ltype,  
    structname,  
    fieldname)
```

Value:

```
PB_SI ## ltype(structname ## _ ## fieldname ## _MSGTYPE)
```

Definition at line 728 of file [pb.h](#).

6.81.2.233 PB_SUBMSG_INFO_SINGULAR

```
#define PB_SUBMSG_INFO_SINGULAR(  
    ltype,  
    structname,  
    fieldname)
```

Value:

```
PB_SI ## ltype(structname ## _ ## fieldname ## _MSGTYPE)
```

Definition at line 729 of file [pb.h](#).

6.81.2.234 PB_UNUSED

```
#define PB_UNUSED(  
    x)
```

Value:

```
(void)(x)
```

Definition at line 169 of file [pb.h](#).

6.81.3 Enumeration Type Documentation

6.81.3.1 pb_wire_type_t

enum [pb_wire_type_t](#)

Enumerator

PB_WT_VARINT	
PB_WT_64BIT	
PB_WT_STRING	
PB_WT_32BIT	
PB_WT_PACKED	

Definition at line 446 of file [pb.h](#).

```
00446     {  
00447     PB_WT_VARINT = 0,  
00448     PB_WT_64BIT  = 1,  
00449     PB_WT_STRING = 2,  
00450     PB_WT_32BIT  = 5,  
00451     PB_WT_PACKED = 255 /* PB_WT_PACKED is internal marker for packed arrays. */  
00452 } pb_wire_type_t;
```

6.81.4 Function Documentation

6.81.4.1 pb_default_field_callback()

```
bool pb_default_field_callback (
    pb_istream_t * istream,
    pb_ostream_t * ostream,
    const pb_field_t * field) [extern]
```

Definition at line 300 of file [pb_common.c](#).

```
00301 {
00302     if (field->data_size == sizeof(pb_callback_t))
00303     {
00304         pb_callback_t *pCallback = (pb_callback_t*)field->pData;
00305
00306         if (pCallback != NULL)
00307         {
00308             if (istream != NULL && pCallback->funcs.decode != NULL)
00309             {
00310                 return pCallback->funcs.decode(istream, field, &pCallback->arg);
00311             }
00312
00313             if (ostream != NULL && pCallback->funcs.encode != NULL)
00314             {
00315                 return pCallback->funcs.encode(ostream, field, &pCallback->arg);
00316             }
00317         }
00318     }
00319
00320     return true; /* Success, but didn't do anything */
00321 }
00322 }
```

6.82 pb.h

[Go to the documentation of this file.](#)

```
00001 /* Common parts of the nanopb library. Most of these are quite low-level
00002  * stuff. For the high-level interface, see pb_encode.h and pb_decode.h.
00003  */
00004
00005 #ifndef PB_H_INCLUDED
00006 #define PB_H_INCLUDED
00007
00008 /*****
00009  * Nanopb compilation time options. You can change these here by *
00010  * uncommenting the lines, or on the compiler command line. *
00011  *****/
00012
00013 /* Enable support for dynamically allocated fields */
00014 /* #define PB_ENABLE_MALLOC 1 */
00015
00016 /* Define this if your CPU / compiler combination does not support
00017  * unaligned memory access to packed structures. Note that packed
00018  * structures are only used when requested in .proto options. */
00019 /* #define PB_NO_PACKED_STRUCTS 1 */
00020
00021 /* Increase the number of required fields that are tracked.
00022  * A compiler warning will tell if you need this. */
00023 /* #define PB_MAX_REQUIRED_FIELDS 256 */
00024
00025 /* Add support for tag numbers > 65536 and fields larger than 65536 bytes. */
00026 /* #define PB_FIELD_32BIT 1 */
00027
00028 /* Disable support for error messages in order to save some code space. */
00029 /* #define PB_NO_ERRMSG 1 */
00030
00031 /* Disable checks to ensure sub-message encoded size is consistent when re-run. */
00032 /* #define PB_NO_ENCODE_SIZE_CHECK 1 */
00033
00034 /* Disable support for custom streams (support only memory buffers). */
00035 /* #define PB_BUFFER_ONLY 1 */
00036
00037 /* Disable support for 64-bit datatypes, for compilers without int64_t
00038  * or to save some code space. */
```

```

00039 /* #define PB_WITHOUT_64BIT 1 */
00040
00041 /* Don't encode scalar arrays as packed. This is only to be used when
00042 * the decoder on the receiving side cannot process packed scalar arrays.
00043 * Such example is older protobuf.js. */
00044 /* #define PB_ENCODE_ARRAYS_UNPACKED 1 */
00045
00046 /* Enable conversion of doubles to floats for platforms that do not
00047 * support 64-bit doubles. Most commonly AVR. */
00048 /* #define PB_CONVERT_DOUBLE_FLOAT 1 */
00049
00050 /* Check whether incoming strings are valid UTF-8 sequences. Slows down
00051 * the string processing slightly and slightly increases code size. */
00052 /* #define PB_VALIDATE_UTF8 1 */
00053
00054 /* This can be defined if the platform is little-endian and has 8-bit bytes.
00055 * Normally it is automatically detected based on __BYTE_ORDER__ macro. */
00056 /* #define PB_LITTLE_ENDIAN_8BIT 1 */
00057
00058 /* Configure static assert mechanism. Instead of changing these, set your
00059 * compiler to C11 standard mode if possible. */
00060 /* #define PB_C99_STATIC_ASSERT 1 */
00061 /* #define PB_NO_STATIC_ASSERT 1 */
00062
00063 /*****
00064 * You usually don't need to change anything below this line.
00065 * Feel free to look around and use the defined macros, though.
00066 *****/
00067
00068
00069 /* Version of the nanopb library. Just in case you want to check it in
00070 * your own program. */
00071 #define NANOPB_VERSION "nanopb-1.0.0-dev"
00072
00073 /* Include all the system headers needed by nanopb. You will need the
00074 * definitions of the following:
00075 * - strlen, memcpy, memset functions
00076 * - [u]int_least8_t, uint_fast8_t, [u]int_least16_t, [u]int32_t, [u]int64_t
00077 * - size_t
00078 * - bool
00079 *
00080 * If you don't have the standard header files, you can instead provide
00081 * a custom header that defines or includes all this. In that case,
00082 * define PB_SYSTEM_HEADER to the path of this file.
00083 */
00084 #ifdef PB_SYSTEM_HEADER
00085 #include PB_SYSTEM_HEADER
00086 #else
00087 #include <stdint.h>
00088 #include <stddef.h>
00089 #include <stdbool.h>
00090 #include <string.h>
00091 #include <limits.h>
00092
00093 #ifdef PB_ENABLE_MALLOC
00094 #include <stdlib.h>
00095 #endif
00096 #endif
00097
00098 #ifdef __cplusplus
00099 extern "C" {
00100 #endif
00101
00102 /* Macro for defining packed structures (compiler dependent).
00103 * This just reduces memory requirements, but is not required.
00104 */
00105 #if defined(PB_NO_PACKED_STRUCTS)
00106 /* Disable struct packing */
00107 # define PB_PACKED_STRUCT_START
00108 # define PB_PACKED_STRUCT_END
00109 # define pb_packed
00110 #elif defined(__GNUC__) || defined(__clang__)
00111 /* For GCC and clang */
00112 # define PB_PACKED_STRUCT_START
00113 # define PB_PACKED_STRUCT_END
00114 # define pb_packed __attribute__((packed))
00115 #elif defined(__ICCARM__) || defined(__CC_ARM)
00116 /* For IAR ARM and Keil MDK-ARM compilers */
00117 # define PB_PACKED_STRUCT_START _Pragma("pack(push, 1)")
00118 # define PB_PACKED_STRUCT_END _Pragma("pack(pop)")
00119 # define pb_packed
00120 #elif defined(_MSC_VER) && (_MSC_VER >= 1500)
00121 /* For Microsoft Visual C++ */
00122 # define PB_PACKED_STRUCT_START __pragma(pack(push, 1))
00123 # define PB_PACKED_STRUCT_END __pragma(pack(pop))
00124 # define pb_packed
00125 #else

```

```

00126  /* Unknown compiler */
00127 #   define PB_PACKED_STRUCT_START
00128 #   define PB_PACKED_STRUCT_END
00129 #   define pb_packed
00130 #endif
00131
00132 /* Define for explicitly not inlining a given function */
00133 #ifndef pb_noinline
00134 #if defined(__GNUC__) || defined(__clang__)
00135  /* For GCC and clang */
00136 #   if defined(noinline)
00137 #       define pb_noinline noinline
00138 #   else
00139 #       define pb_noinline __attribute__((noinline))
00140 #   endif
00141 #elif defined(__ICCARM__) || defined(__CC_ARM)
00142  /* For IAR ARM and Keil MDK-ARM compilers */
00143 #   define pb_noinline
00144 #elif defined(_MSC_VER) && (_MSC_VER >= 1500)
00145 #   define pb_noinline __declspec(noinline)
00146 #else
00147 #   define pb_noinline
00148 #endif
00149 #endif
00150
00151 /* Detect endianness */
00152 #if !defined(CHAR_BIT) && defined(__CHAR_BIT__)
00153 #define CHAR_BIT __CHAR_BIT__
00154 #endif
00155
00156 #ifndef PB_LITTLE_ENDIAN_8BIT
00157 #if ((defined(__BYTE_ORDER) && __BYTE_ORDER == __LITTLE_ENDIAN) || \
00158     (defined(__BYTE_ORDER) && __BYTE_ORDER == __ORDER_LITTLE_ENDIAN) || \
00159     defined(__LITTLE_ENDIAN__) || defined(__ARMEL__) || \
00160     defined(__THUMBEL__) || defined(__AARCH64EL__) || defined(__MIPSEL) || \
00161     defined(__M_X86) || defined(__M_X64) || defined(__M_ARM)) \
00162     && defined(CHAR_BIT) && CHAR_BIT == 8
00163 #define PB_LITTLE_ENDIAN_8BIT 1
00164 #endif
00165 #endif
00166
00167 /* Handy macro for suppressing unreferenced-parameter compiler warnings. */
00168 #ifndef PB_UNUSED
00169 #define PB_UNUSED(x) (void)(x)
00170 #endif
00171
00172 /* Harvard-architecture processors may need special attributes for storing
00173  * field information in program memory. */
00174 #ifndef PB_PROGMEM
00175 #ifdef __AVR__
00176 #include <avr/pgmspace.h>
00177 #define PB_PROGMEM          PROGMEM
00178 #define PB_PROGMEM_READU32(x) pgm_read_dword(&x)
00179 #else
00180 #define PB_PROGMEM
00181 #define PB_PROGMEM_READU32(x) (x)
00182 #endif
00183 #endif
00184
00185 /* Compile-time assertion, used for checking compatible compilation options.
00186  * If this does not work properly on your compiler, use
00187  * #define PB_NO_STATIC_ASSERT to disable it.
00188  *
00189  * But before doing that, check carefully the error message / place where it
00190  * comes from to see if the error has a real cause. Unfortunately the error
00191  * message is not always very clear to read, but you can see the reason better
00192  * in the place where the PB_STATIC_ASSERT macro was called.
00193  */
00194 #ifndef PB_NO_STATIC_ASSERT
00195 #ifndef PB_STATIC_ASSERT
00196 #   if defined(__ICCARM__)
00197  /* IAR has static_assert keyword but no __Static_assert */
00198 #       define PB_STATIC_ASSERT(COND,MSG) static_assert(COND,#MSG);
00199 #   elif defined(_MSC_VER) && (!defined(__STDC_VERSION__) || __STDC_VERSION__ < 201112)
00200  /* MSVC in C89 mode supports static_assert() keyword anyway */
00201 #       define PB_STATIC_ASSERT(COND,MSG) static_assert(COND,#MSG);
00202 #   elif defined(PB_C99_STATIC_ASSERT)
00203  /* Classic negative-size-array static assert mechanism */
00204 #       define PB_STATIC_ASSERT(COND,MSG) typedef char PB_STATIC_ASSERT_MSG(MSG, __LINE__,
00205                                     __COUNTER__)[(COND)?1:-1];
00206 #       define PB_STATIC_ASSERT_MSG(MSG, LINE, COUNTER) PB_STATIC_ASSERT_MSG_(MSG, LINE,
00207                                     COUNTER)
00208 #       define PB_STATIC_ASSERT_MSG_(MSG, LINE, COUNTER)
00209 #           pb_static_assertion_##MSG##_##LINE##_##COUNTER
00210 #       elif defined(__cplusplus)
00211  /* C++11 standard static_assert mechanism */
00212 #       define PB_STATIC_ASSERT(COND,MSG) static_assert(COND,#MSG);

```

```

00210 #   else
00211     /* C11 standard _Static_assert mechanism */
00212     #define PB_STATIC_ASSERT(COND,MSG) _Static_assert(COND,#MSG);
00213 #   endif
00214 #endif
00215 #else
00216     /* Static asserts disabled by PB_NO_STATIC_ASSERT */
00217     #define PB_STATIC_ASSERT(COND,MSG)
00218 #endif
00219
00220 /* Test that PB_STATIC_ASSERT works
00221 * If you get errors here, you may need to do one of these:
00222 * - Enable C11 standard support in your compiler
00223 * - Define PB_C99_STATIC_ASSERT to enable C99 standard support
00224 * - Define PB_NO_STATIC_ASSERT to disable static asserts altogether
00225 */
00226 PB_STATIC_ASSERT(1, STATIC_ASSERT_IS_NOT_WORKING)
00227
00228 /* Number of required fields to keep track of. */
00229 #ifndef PB_MAX_REQUIRED_FIELDS
00230 #define PB_MAX_REQUIRED_FIELDS 64
00231 #endif
00232
00233 #if PB_MAX_REQUIRED_FIELDS < 64
00234 #error You should not lower PB_MAX_REQUIRED_FIELDS from the default value (64).
00235 #endif
00236
00237 #ifdef PB_WITHOUT_64BIT
00238 #ifdef PB_CONVERT_DOUBLE_FLOAT
00239     /* Cannot use doubles without 64-bit types */
00240 #undef PB_CONVERT_DOUBLE_FLOAT
00241 #endif
00242 #endif
00243
00244 /* Data type for storing encoded data and other byte streams.
00245 * This typedef exists to support platforms where uint8_t does not exist.
00246 * You can regard it as equivalent on uint8_t on other platforms.
00247 */
00248 #if defined(PB_BYTE_T_OVERRIDE)
00249 typedef PB_BYTE_T_OVERRIDE pb_byte_t;
00250 #elif defined(UINT8_MAX)
00251 typedef uint8_t pb_byte_t;
00252 #else
00253 typedef uint_least8_t pb_byte_t;
00254 #endif
00255
00256 /* List of possible field types. These are used in the autogenerated code.
00257 * Least-significant 4 bits tell the scalar type
00258 * Most-significant 4 bits specify repeated/required/packed etc.
00259 */
00260 typedef pb_byte_t pb_type_t;
00261
00262 /**** Field data types ****/
00263
00264 /* Numeric types */
00265 #define PB_LTYPE_BOOL    0x00U /* bool */
00266 #define PB_LTYPE_VARINT  0x01U /* int32, int64, enum, bool */
00267 #define PB_LTYPE_UVARINT 0x02U /* uint32, uint64 */
00268 #define PB_LTYPE_SVARINT 0x03U /* sint32, sint64 */
00269 #define PB_LTYPE_FIXED32 0x04U /* fixed32, sfixed32, float */
00270 #define PB_LTYPE_FIXED64 0x05U /* fixed64, sfixed64, double */
00271
00272 /* Marker for last packable field type. */
00273 #define PB_LTYPE_LAST_PACKABLE 0x05U
00274
00275 /* Byte array with pre-allocated buffer.
00276 * data_size is the length of the allocated PB_BYTES_ARRAY structure. */
00277 #define PB_LTYPE_BYTES 0x06U
00278
00279 /* String with pre-allocated buffer.
00280 * data_size is the maximum length. */
00281 #define PB_LTYPE_STRING 0x07U
00282
00283 /* Submessage
00284 * submsg_fields is pointer to field descriptions */
00285 #define PB_LTYPE_SUBMESSAGE 0x08U
00286
00287 /* Submessage with pre-decoding callback
00288 * The pre-decoding callback is stored as pb_callback_t right before pSize.
00289 * submsg_fields is pointer to field descriptions */
00290 #define PB_LTYPE_SUBMSG_W_CB 0x09U
00291
00292 /* Extension pseudo-field
00293 * The field contains a pointer to pb_extension_t */
00294 #define PB_LTYPE_EXTENSION 0x0AU
00295
00296 /* Byte array with inline, pre-allocated byffer.

```



```

00297 * data_size is the length of the inline, allocated buffer.
00298 * This differs from PB_LTYPE_BYTES by defining the element as
00299 * pb_byte_t[data_size] rather than pb_bytes_array_t. */
00300 #define PB_LTYPE_FIXED_LENGTH_BYTES 0x0BU
00301
00302 /* Number of declared LTYPES */
00303 #define PB_LTYPES_COUNT 0x0CU
00304 #define PB_LTYPE_MASK 0x0FU
00305
00306 /**** Field repetition rules ****/
00307
00308 #define PB_HTYPE_REQUIRED 0x00U
00309 #define PB_HTYPE_OPTIONAL 0x10U
00310 #define PB_HTYPE_SINGULAR 0x10U
00311 #define PB_HTYPE_REPEATED 0x20U
00312 #define PB_HTYPE_FIXARRAY 0x20U
00313 #define PB_HTYPE_ONEOF 0x30U
00314 #define PB_HTYPE_MASK 0x30U
00315
00316 /**** Field allocation types ****/
00317
00318 #define PB_ATYPE_STATIC 0x00U
00319 #define PB_ATYPE_POINTER 0x80U
00320 #define PB_ATYPE_CALLBACK 0x40U
00321 #define PB_ATYPE_MASK 0xC0U
00322
00323 #define PB_ATYPE(x) ((x) & PB_ATYPE_MASK)
00324 #define PB_HTYPE(x) ((x) & PB_HTYPE_MASK)
00325 #define PB_LTYPE(x) ((x) & PB_LTYPE_MASK)
00326 #define PB_LTYPE_IS_SUBMSG(x) (PB_LTYPE(x) == PB_LTYPE_SUBMESSAGE || \
00327     PB_LTYPE(x) == PB_LTYPE_SUBMSG_W_CB)
00328
00329 /* Data type used for storing sizes of struct fields
00330 * and array counts.
00331 */
00332 #if defined(PB_FIELD_32BIT)
00333     typedef uint32_t pb_size_t;
00334     typedef int32_t pb_ssize_t;
00335 #else
00336     typedef uint_least16_t pb_size_t;
00337     typedef int_least16_t pb_ssize_t;
00338 #endif
00339 #define PB_SIZE_MAX ((pb_size_t)-1)
00340
00341 /* Forward declaration of struct types */
00342 typedef struct pb_istream_s pb_istream_t;
00343 typedef struct pb_ostream_s pb_ostream_t;
00344 typedef struct pb_field_iter_s pb_field_iter_t;
00345
00346 /* This structure is used in auto-generated constants
00347 * to specify struct fields.
00348 */
00349 typedef struct pb_msgdesc_s pb_msgdesc_t;
00350 struct pb_msgdesc_s {
00351     const uint32_t *field_info;
00352     const pb_msgdesc_t * const * submsg_info;
00353     const pb_byte_t *default_value;
00354
00355     bool (*field_callback)(pb_istream_t *istream, pb_ostream_t *ostream, const pb_field_iter_t *field);
00356
00357     pb_size_t field_count;
00358     pb_size_t required_field_count;
00359     pb_size_t largest_tag;
00360 };
00361
00362 /* Iterator for message descriptor */
00363 struct pb_field_iter_s {
00364     const pb_msgdesc_t *descriptor; /* Pointer to message descriptor constant */
00365     void *message; /* Pointer to start of the structure */
00366
00367     pb_size_t index; /* Index of the field */
00368     pb_size_t field_info_index; /* Index to descriptor->field_info array */
00369     pb_size_t required_field_index; /* Index that counts only the required fields */
00370     pb_size_t submessage_index; /* Index that counts only submessages */
00371
00372     pb_size_t tag; /* Tag of current field */
00373     pb_size_t data_size; /* sizeof() of a single item */
00374     pb_size_t array_size; /* Number of array entries */
00375     pb_type_t type; /* Type of current field */
00376
00377     void *pField; /* Pointer to current field in struct */
00378     void *pData; /* Pointer to current data contents. Different than pField for arrays and pointers. */
00379     void *pSize; /* Pointer to count/has field */
00380
00381     const pb_msgdesc_t *submsg_desc; /* For submessage fields, pointer to field descriptor for the submessage. */
00382 };
00383

```

```

00384 /* For compatibility with legacy code */
00385 typedef pb_field_iter_t pb_field_t;
00386
00387 /* Make sure that the standard integer types are of the expected sizes.
00388 * Otherwise fixed32/fixed64 fields can break.
00389 *
00390 * If you get errors here, it probably means that your stdint.h is not
00391 * correct for your platform.
00392 */
00393 #ifndef PB_WITHOUT_64BIT
00394 PB_STATIC_ASSERT(sizeof(int64_t) == 2 * sizeof(int32_t), INT64_T_WRONG_SIZE)
00395 PB_STATIC_ASSERT(sizeof(uint64_t) == 2 * sizeof(uint32_t), UINT64_T_WRONG_SIZE)
00396 #endif
00397
00398 /* This structure is used for 'bytes' arrays.
00399 * It has the number of bytes in the beginning, and after that an array.
00400 * Note that actual structs used will have a different length of bytes array.
00401 */
00402 #define PB_BYTES_ARRAY_T(n) struct { pb_size_t size; pb_byte_t bytes[n]; }
00403 #define PB_BYTES_ARRAY_T_ALLOCSIZE(n) ((size_t)n + offsetof(pb_bytes_array_t, bytes))
00404
00405 struct pb_bytes_array_s {
00406     pb_size_t size;
00407     pb_byte_t bytes[1];
00408 };
00409 typedef struct pb_bytes_array_s pb_bytes_array_t;
00410
00411 /* This structure is used for giving the callback function.
00412 * It is stored in the message structure and filled in by the method that
00413 * calls pb_decode.
00414 *
00415 * The decoding callback will be given a limited-length stream
00416 * If the wire type was string, the length is the length of the string.
00417 * If the wire type was a varint/fixed32/fixed64, the length is the length
00418 * of the actual value.
00419 * The function may be called multiple times (especially for repeated types,
00420 * but also otherwise if the message happens to contain the field multiple
00421 * times.)
00422 *
00423 * The encoding callback will receive the actual output stream.
00424 * It should write all the data in one call, including the field tag and
00425 * wire type. It can write multiple fields.
00426 *
00427 * The callback can be null if you want to skip a field.
00428 */
00429 typedef struct pb_callback_s pb_callback_t;
00430 struct pb_callback_s {
00431     /* Callback functions receive a pointer to the arg field.
00432     * You can access the value of the field as *arg, and modify it if needed.
00433     */
00434     union {
00435         bool (*decode)(pb_istream_t *stream, const pb_field_t *field, void **arg);
00436         bool (*encode)(pb_ostream_t *stream, const pb_field_t *field, void *const *arg);
00437     } funcs;
00438
00439     /* Free arg for use by callback */
00440     void *arg;
00441 };
00442
00443 extern bool pb_default_field_callback(pb_istream_t *istream, pb_ostream_t *ostream, const pb_field_t *field);
00444
00445 /* Wire types. Library user needs these only in encoder callbacks. */
00446 typedef enum {
00447     PB_WT_VARINT = 0,
00448     PB_WT_64BIT = 1,
00449     PB_WT_STRING = 2,
00450     PB_WT_32BIT = 5,
00451     PB_WT_PACKED = 255 /* PB_WT_PACKED is internal marker for packed arrays. */
00452 } pb_wire_type_t;
00453
00454 /* Structure for defining the handling of unknown/extension fields.
00455 * Usually the pb_extension_type_t structure is automatically generated,
00456 * while the pb_extension_t structure is created by the user. However,
00457 * if you want to catch all unknown fields, you can also create a custom
00458 * pb_extension_type_t with your own callback.
00459 */
00460 typedef struct pb_extension_type_s pb_extension_type_t;
00461 typedef struct pb_extension_s pb_extension_t;
00462 struct pb_extension_type_s {
00463     /* Called for each unknown field in the message.
00464     * If you handle the field, read off all of its data and return true.
00465     * If you do not handle the field, do not read anything and return true.
00466     * If you run into an error, return false.
00467     * Set to NULL for default handler.
00468     */
00469     bool (*decode)(pb_istream_t *stream, pb_extension_t *extension,
00470         uint32_t tag, pb_wire_type_t wire_type);

```

```

00471
00472 /* Called once after all regular fields have been encoded.
00473  * If you have something to write, do so and return true.
00474  * If you do not have anything to write, just return true.
00475  * If you run into an error, return false.
00476  * Set to NULL for default handler.
00477  */
00478 bool (*encode)(pb_ostream_t *stream, const pb_extension_t *extension);
00479
00480 /* Free field for use by the callback. */
00481 const void *arg;
00482 };
00483
00484 struct pb_extension_s {
00485     /* Type describing the extension field. Usually you'll initialize
00486     * this to a pointer to the automatically generated structure. */
00487     const pb_extension_type_t *type;
00488
00489     /* Destination for the decoded data. This must match the datatype
00490     * of the extension field. */
00491     void *dest;
00492
00493     /* Pointer to the next extension handler, or NULL.
00494     * If this extension does not match a field, the next handler is
00495     * automatically called. */
00496     pb_extension_t *next;
00497
00498     /* The decoder sets this to true if the extension was found.
00499     * Ignored for encoding. */
00500     bool found;
00501 };
00502
00503 #define pb_extension_init_zero {NULL,NULL,NULL,false}
00504
00505 /* Memory allocation functions to use. You can define pb_realloc and
00506  * pb_free to custom functions if you want. */
00507 #ifdef PB_ENABLE_MALLOC
00508 #   ifndef pb_realloc
00509 #       define pb_realloc(ptr, size) realloc(ptr, size)
00510 #   endif
00511 #   ifndef pb_free
00512 #       define pb_free(ptr) free(ptr)
00513 #   endif
00514 #endif
00515
00516 /* This is used to inform about need to regenerate .pb.h/.pb.c files. */
00517 #define PB_PROTO_HEADER_VERSION 40
00518
00519 /* These macros are used to declare pb_field_t's in the constant array. */
00520 /* Size of a structure member, in bytes. */
00521 #define pb_membersize(st, m) (sizeof ((st*)0)->m)
00522 /* Number of entries in an array. */
00523 #define pb_arraysize(st, m) (pb_membersize(st, m) / pb_membersize(st, m[0]))
00524 /* Delta from start of one member to the start of another member. */
00525 #define pb_delta(st, m1, m2) ((int)offsetof(st, m1) - (int)offsetof(st, m2))
00526
00527 /* Force expansion of macro value */
00528 #define PB_EXPAND(x) x
00529
00530 /* Binding of a message field set into a specific structure */
00531 #define PB_BIND(msgname, structname, width) \
00532     const uint32_t structname##_field_info[] PB_PROGMEM = \
00533     { \
00534         msgname##_FIELDLIST(PB_GEN_FIELD_INFO_## width, structname) \
00535         0 \
00536     }; \
00537     const pb_msgdesc_t* const structname##_submsg_info[] = \
00538     { \
00539         msgname##_FIELDLIST(PB_GEN_SUBMSG_INFO, structname) \
00540         NULL \
00541     }; \
00542     const pb_msgdesc_t structname##_msg = \
00543     { \
00544         structname##_field_info, \
00545         structname##_submsg_info, \
00546         msgname##_DEFAULT, \
00547         msgname##_CALLBACK, \
00548         0 msgname##_FIELDLIST(PB_GEN_FIELD_COUNT, structname), \
00549         0 msgname##_FIELDLIST(PB_GEN_REQ_FIELD_COUNT, structname), \
00550         0 msgname##_FIELDLIST(PB_GEN_LARGEST_TAG, structname), \
00551     }; \
00552     msgname##_FIELDLIST(PB_GEN_FIELD_INFO_ASSERT_## width, structname)
00553
00554 #define PB_GEN_FIELD_COUNT(structname, atype, htype, ltype, fieldname, tag) +1
00555 #define PB_GEN_REQ_FIELD_COUNT(structname, atype, htype, ltype, fieldname, tag) \
00556     + (PB_HTYPE_## htype == PB_HTYPE_REQUIRED)
00557 #define PB_GEN_LARGEST_TAG(structname, atype, htype, ltype, fieldname, tag) \

```

```

00558     * 0 + tag
00559
00560 /* X-macro for generating the entries in struct_field_info[] array. */
00561 #define PB_GEN_FIELD_INFO_1(structname, atype, htype, ltype, fieldname, tag) \
00562     PB_FIELDINFO_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00563         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00564         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00565         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00566         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00567
00568 #define PB_GEN_FIELD_INFO_2(structname, atype, htype, ltype, fieldname, tag) \
00569     PB_FIELDINFO_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00570         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00571         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00572         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00573         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00574
00575 #define PB_GEN_FIELD_INFO_4(structname, atype, htype, ltype, fieldname, tag) \
00576     PB_FIELDINFO_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00577         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00578         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00579         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00580         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00581
00582 #define PB_GEN_FIELD_INFO_8(structname, atype, htype, ltype, fieldname, tag) \
00583     PB_FIELDINFO_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00584         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00585         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00586         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00587         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00588
00589 #define PB_GEN_FIELD_INFO_AUTO(structname, atype, htype, ltype, fieldname, tag) \
00590     PB_FIELDINFO_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_ ## htype, \
00591         _PB_LTYPE_ ## ltype), \
00592         tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00593         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00594         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00595         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00596         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00597
00598 #define PB_FIELDINFO_AUTO2(width, tag, type, data_offset, data_size, size_offset, array_size) \
00599     PB_FIELDINFO_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size) \
00600
00601 #define PB_FIELDINFO_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size) \
00602     PB_FIELDINFO_ ## width(tag, type, data_offset, data_size, size_offset, array_size) \
00603
00604 /* X-macro for generating asserts that entries fit in struct_field_info[] array.
00605  * The structure of macros here must match the structure above in PB_GEN_FIELD_INFO_x(),
00606  * but it is not easily reused because of how macro substitutions work. */
00607 #define PB_GEN_FIELD_INFO_ASSERT_1(structname, atype, htype, ltype, fieldname, tag) \
00608     PB_FIELDINFO_ASSERT_1(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## \
00609     ltype, \
00610         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00611         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00612         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00613         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00614
00615 #define PB_GEN_FIELD_INFO_ASSERT_2(structname, atype, htype, ltype, fieldname, tag) \
00616     PB_FIELDINFO_ASSERT_2(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## \
00617     ltype, \
00618         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00619         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00620         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00621         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00622
00623 #define PB_GEN_FIELD_INFO_ASSERT_4(structname, atype, htype, ltype, fieldname, tag) \
00624     PB_FIELDINFO_ASSERT_4(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## \
00625     ltype, \
00626         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00627         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00628         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00629         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00630
00631 #define PB_GEN_FIELD_INFO_ASSERT_8(structname, atype, htype, ltype, fieldname, tag) \
00632     PB_FIELDINFO_ASSERT_8(tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## \
00633     ltype, \
00634         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00635         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00636         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00637         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00638
00639 #define PB_GEN_FIELD_INFO_ASSERT_AUTO(structname, atype, htype, ltype, fieldname, tag) \
00640     PB_FIELDINFO_ASSERT_AUTO2(PB_FIELDINFO_WIDTH_AUTO(_PB_ATYPE_ ## atype, _PB_HTYPE_ ## \
00641     htype, _PB_LTYPE_ ## ltype), \
00642         tag, PB_ATYPE_ ## atype | PB_HTYPE_ ## htype | PB_LTYPE_MAP_ ## ltype, \
00643         PB_DATA_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00644         PB_DATA_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00645         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00646         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))

```

```

00639         PB_SIZE_OFFSET_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname), \
00640         PB_ARRAY_SIZE_ ## atype(_PB_HTYPE_ ## htype, structname, fieldname))
00641
00642 #define PB_FIELDINFO_ASSERT_AUTO2(width, tag, type, data_offset, data_size, size_offset, array_size) \
00643     PB_FIELDINFO_ASSERT_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size)
00644
00645 #define PB_FIELDINFO_ASSERT_AUTO3(width, tag, type, data_offset, data_size, size_offset, array_size) \
00646     PB_FIELDINFO_ASSERT_ ## width(tag, type, data_offset, data_size, size_offset, array_size)
00647
00648 #define PB_DATA_OFFSET_STATIC(htype, structname, fieldname) PB_DO ## htype(structname, fieldname)
00649 #define PB_DATA_OFFSET_POINTER(htype, structname, fieldname) PB_DO ## htype(structname, fieldname)
00650 #define PB_DATA_OFFSET_CALLBACK(htype, structname, fieldname) PB_DO ## htype(structname, fieldname)
00651 #define PB_DO_PB_HTYPE_REQUIRED(structname, fieldname) offsetof(structname, fieldname)
00652 #define PB_DO_PB_HTYPE_SINGULAR(structname, fieldname) offsetof(structname, fieldname)
00653 #define PB_DO_PB_HTYPE_ONEOF(structname, fieldname) offsetof(structname, PB_ONEOF_NAME(FULL,
    fieldname))
00654 #define PB_DO_PB_HTYPE_OPTIONAL(structname, fieldname) offsetof(structname, fieldname)
00655 #define PB_DO_PB_HTYPE_REPEATED(structname, fieldname) offsetof(structname, fieldname)
00656 #define PB_DO_PB_HTYPE_FIXARRAY(structname, fieldname) offsetof(structname, fieldname)
00657
00658 #define PB_SIZE_OFFSET_STATIC(htype, structname, fieldname) PB_SO ## htype(structname, fieldname)
00659 #define PB_SIZE_OFFSET_POINTER(htype, structname, fieldname) PB_SO_PTR ## htype(structname, fieldname)
00660 #define PB_SIZE_OFFSET_CALLBACK(htype, structname, fieldname) PB_SO_CB ## htype(structname, fieldname)
00661 #define PB_SO_PB_HTYPE_REQUIRED(structname, fieldname) 0
00662 #define PB_SO_PB_HTYPE_SINGULAR(structname, fieldname) 0
00663 #define PB_SO_PB_HTYPE_ONEOF(structname, fieldname) PB_SO_PB_HTYPE_ONEOF2(structname,
    PB_ONEOF_NAME(FULL, fieldname), PB_ONEOF_NAME(UNION, fieldname))
00664 #define PB_SO_PB_HTYPE_ONEOF2(structname, fullname, unionname)
    PB_SO_PB_HTYPE_ONEOF3(structname, fullname, unionname)
00665 #define PB_SO_PB_HTYPE_ONEOF3(structname, fullname, unionname) pb_delta(structname, fullname, which_ ##
    unionname)
00666 #define PB_SO_PB_HTYPE_OPTIONAL(structname, fieldname) pb_delta(structname, fieldname, has_ ## fieldname)
00667 #define PB_SO_PB_HTYPE_REPEATED(structname, fieldname) pb_delta(structname, fieldname, fieldname ##
    _count)
00668 #define PB_SO_PB_HTYPE_FIXARRAY(structname, fieldname) 0
00669 #define PB_SO_PTR_PB_HTYPE_REQUIRED(structname, fieldname) 0
00670 #define PB_SO_PTR_PB_HTYPE_SINGULAR(structname, fieldname) 0
00671 #define PB_SO_PTR_PB_HTYPE_ONEOF(structname, fieldname) PB_SO_PB_HTYPE_ONEOF(structname,
    fieldname)
00672 #define PB_SO_PTR_PB_HTYPE_OPTIONAL(structname, fieldname) 0
00673 #define PB_SO_PTR_PB_HTYPE_REPEATED(structname, fieldname)
    PB_SO_PB_HTYPE_REPEATED(structname, fieldname)
00674 #define PB_SO_PTR_PB_HTYPE_FIXARRAY(structname, fieldname) 0
00675 #define PB_SO_CB_PB_HTYPE_REQUIRED(structname, fieldname) 0
00676 #define PB_SO_CB_PB_HTYPE_SINGULAR(structname, fieldname) 0
00677 #define PB_SO_CB_PB_HTYPE_ONEOF(structname, fieldname) PB_SO_PB_HTYPE_ONEOF(structname,
    fieldname)
00678 #define PB_SO_CB_PB_HTYPE_OPTIONAL(structname, fieldname) 0
00679 #define PB_SO_CB_PB_HTYPE_REPEATED(structname, fieldname) 0
00680 #define PB_SO_CB_PB_HTYPE_FIXARRAY(structname, fieldname) 0
00681
00682 #define PB_ARRAY_SIZE_STATIC(htype, structname, fieldname) PB_AS ## htype(structname, fieldname)
00683 #define PB_ARRAY_SIZE_POINTER(htype, structname, fieldname) PB_AS_PTR ## htype(structname, fieldname)
00684 #define PB_ARRAY_SIZE_CALLBACK(htype, structname, fieldname) 1
00685 #define PB_AS_PB_HTYPE_REQUIRED(structname, fieldname) 1
00686 #define PB_AS_PB_HTYPE_SINGULAR(structname, fieldname) 1
00687 #define PB_AS_PB_HTYPE_OPTIONAL(structname, fieldname) 1
00688 #define PB_AS_PB_HTYPE_ONEOF(structname, fieldname) 1
00689 #define PB_AS_PB_HTYPE_REPEATED(structname, fieldname) pb_arraysize(structname, fieldname)
00690 #define PB_AS_PB_HTYPE_FIXARRAY(structname, fieldname) pb_arraysize(structname, fieldname)
00691 #define PB_AS_PTR_PB_HTYPE_REQUIRED(structname, fieldname) 1
00692 #define PB_AS_PTR_PB_HTYPE_SINGULAR(structname, fieldname) 1
00693 #define PB_AS_PTR_PB_HTYPE_OPTIONAL(structname, fieldname) 1
00694 #define PB_AS_PTR_PB_HTYPE_ONEOF(structname, fieldname) 1
00695 #define PB_AS_PTR_PB_HTYPE_REPEATED(structname, fieldname) 1
00696 #define PB_AS_PTR_PB_HTYPE_FIXARRAY(structname, fieldname) pb_arraysize(structname, fieldname[0])
00697
00698 #define PB_DATA_SIZE_STATIC(htype, structname, fieldname) PB_DS ## htype(structname, fieldname)
00699 #define PB_DATA_SIZE_POINTER(htype, structname, fieldname) PB_DS_PTR ## htype(structname, fieldname)
00700 #define PB_DATA_SIZE_CALLBACK(htype, structname, fieldname) PB_DS_CB ## htype(structname, fieldname)
00701 #define PB_DS_PB_HTYPE_REQUIRED(structname, fieldname) pb_membersize(structname, fieldname)
00702 #define PB_DS_PB_HTYPE_SINGULAR(structname, fieldname) pb_membersize(structname, fieldname)
00703 #define PB_DS_PB_HTYPE_OPTIONAL(structname, fieldname) pb_membersize(structname, fieldname)
00704 #define PB_DS_PB_HTYPE_ONEOF(structname, fieldname) pb_membersize(structname, PB_ONEOF_NAME(FULL,
    fieldname))
00705 #define PB_DS_PB_HTYPE_REPEATED(structname, fieldname) pb_membersize(structname, fieldname[0])
00706 #define PB_DS_PB_HTYPE_FIXARRAY(structname, fieldname) pb_membersize(structname, fieldname[0])
00707 #define PB_DS_PTR_PB_HTYPE_REQUIRED(structname, fieldname) pb_membersize(structname, fieldname[0])
00708 #define PB_DS_PTR_PB_HTYPE_SINGULAR(structname, fieldname) pb_membersize(structname, fieldname[0])
00709 #define PB_DS_PTR_PB_HTYPE_OPTIONAL(structname, fieldname) pb_membersize(structname, fieldname[0])
00710 #define PB_DS_PTR_PB_HTYPE_ONEOF(structname, fieldname) pb_membersize(structname,
    PB_ONEOF_NAME(FULL, fieldname)[0])
00711 #define PB_DS_PTR_PB_HTYPE_REPEATED(structname, fieldname) pb_membersize(structname, fieldname[0])
00712 #define PB_DS_PTR_PB_HTYPE_FIXARRAY(structname, fieldname) pb_membersize(structname, fieldname[0][0])
00713 #define PB_DS_CB_PB_HTYPE_REQUIRED(structname, fieldname) pb_membersize(structname, fieldname)
00714 #define PB_DS_CB_PB_HTYPE_SINGULAR(structname, fieldname) pb_membersize(structname, fieldname)
00715 #define PB_DS_CB_PB_HTYPE_OPTIONAL(structname, fieldname) pb_membersize(structname, fieldname)

```



```

00716 #define PB_DS_CB_PB_HTYPE_ONEOF(structname, fieldname) pb_membersize(structname,
PB_ONEOF_NAME(FULL, fieldname))
00717 #define PB_DS_CB_PB_HTYPE_REPEATED(structname, fieldname) pb_membersize(structname, fieldname)
00718 #define PB_DS_CB_PB_HTYPE_FIXARRAY(structname, fieldname) pb_membersize(structname, fieldname)
00719
00720 #define PB_ONEOF_NAME(type, tuple) PB_EXPAND(PB_ONEOF_NAME_ ## type tuple)
00721 #define PB_ONEOF_NAME_UNION(unionname, membername, fullname) unionname
00722 #define PB_ONEOF_NAME_MEMBER(unionname, membername, fullname) membername
00723 #define PB_ONEOF_NAME_FULL(unionname, membername, fullname) fullname
00724
00725 #define PB_GEN_SUBMSG_INFO(structname, atype, htype, ltype, fieldname, tag) \
00726     PB_SUBMSG_INFO_ ## htype(_PB_LTYPE_ ## ltype, structname, fieldname)
00727
00728 #define PB_SUBMSG_INFO_REQUIRED(ltype, structname, fieldname) PB_SI ## ltype(structname ## _ ##
fieldname ## _MSGTYPE)
00729 #define PB_SUBMSG_INFO_SINGULAR(ltype, structname, fieldname) PB_SI ## ltype(structname ## _ ##
fieldname ## _MSGTYPE)
00730 #define PB_SUBMSG_INFO_OPTIONAL(ltype, structname, fieldname) PB_SI ## ltype(structname ## _ ##
fieldname ## _MSGTYPE)
00731 #define PB_SUBMSG_INFO_ONEOF(ltype, structname, fieldname) PB_SUBMSG_INFO_ONEOF2(ltype, structname,
PB_ONEOF_NAME(UNION, fieldname), PB_ONEOF_NAME(MEMBER, fieldname))
00732 #define PB_SUBMSG_INFO_ONEOF2(ltype, structname, unionname, membername)
PB_SUBMSG_INFO_ONEOF3(ltype, structname, unionname, membername)
00733 #define PB_SUBMSG_INFO_ONEOF3(ltype, structname, unionname, membername) PB_SI ## ltype(structname ##
_ ## unionname ## _ ## membername ## _MSGTYPE)
00734 #define PB_SUBMSG_INFO_REPEATED(ltype, structname, fieldname) PB_SI ## ltype(structname ## _ ##
fieldname ## _MSGTYPE)
00735 #define PB_SUBMSG_INFO_FIXARRAY(ltype, structname, fieldname) PB_SI ## ltype(structname ## _ ##
fieldname ## _MSGTYPE)
00736 #define PB_SI_PB_LTYPE_BOOL(t)
00737 #define PB_SI_PB_LTYPE_BYTES(t)
00738 #define PB_SI_PB_LTYPE_DOUBLE(t)
00739 #define PB_SI_PB_LTYPE_ENUM(t)
00740 #define PB_SI_PB_LTYPE_UENUM(t)
00741 #define PB_SI_PB_LTYPE_FIXED32(t)
00742 #define PB_SI_PB_LTYPE_FIXED64(t)
00743 #define PB_SI_PB_LTYPE_FLOAT(t)
00744 #define PB_SI_PB_LTYPE_INT32(t)
00745 #define PB_SI_PB_LTYPE_INT64(t)
00746 #define PB_SI_PB_LTYPE_MESSAGE(t) PB_SUBMSG_DESCRIPTOR(t)
00747 #define PB_SI_PB_LTYPE_MSG_W_CB(t) PB_SUBMSG_DESCRIPTOR(t)
00748 #define PB_SI_PB_LTYPE_SFIED32(t)
00749 #define PB_SI_PB_LTYPE_SFIED64(t)
00750 #define PB_SI_PB_LTYPE_SINT32(t)
00751 #define PB_SI_PB_LTYPE_SINT64(t)
00752 #define PB_SI_PB_LTYPE_STRING(t)
00753 #define PB_SI_PB_LTYPE_UINT32(t)
00754 #define PB_SI_PB_LTYPE_UINT64(t)
00755 #define PB_SI_PB_LTYPE_EXTENSION(t)
00756 #define PB_SI_PB_LTYPE_FIXED_LENGTH_BYTES(t)
00757 #define PB_SUBMSG_DESCRIPTOR(t)    &(t ## _msg),
00758
00759 /* The field descriptors use a variable width format, with width of either
00760 * 1, 2, 4 or 8 of 32-bit words. The two lowest bytes of the first byte always
00761 * encode the descriptor size, 6 lowest bits of field tag number, and 8 bits
00762 * of the field type.
00763 *
00764 * Descriptor size is encoded as 0 = 1 word, 1 = 2 words, 2 = 4 words, 3 = 8 words.
00765 *
00766 * Formats, listed starting with the least significant bit of the first word.
00767 * 1 word: [2-bit len] [6-bit tag] [8-bit type] [8-bit data_offset] [4-bit size_offset] [4-bit data_size]
00768 *
00769 * 2 words: [2-bit len] [6-bit tag] [8-bit type] [12-bit array_size] [4-bit size_offset]
00770 *           [16-bit data_offset] [12-bit data_size] [4-bit tag»6]
00771 *
00772 * 4 words: [2-bit len] [6-bit tag] [8-bit type] [16-bit array_size]
00773 *           [8-bit size_offset] [24-bit tag»6]
00774 *           [32-bit data_offset]
00775 *           [32-bit data_size]
00776 *
00777 * 8 words: [2-bit len] [6-bit tag] [8-bit type] [16-bit reserved]
00778 *           [8-bit size_offset] [24-bit tag»6]
00779 *           [32-bit data_offset]
00780 *           [32-bit data_size]
00781 *           [32-bit array_size]
00782 *           [32-bit reserved]
00783 *           [32-bit reserved]
00784 *           [32-bit reserved]
00785 */
00786
00787 #define PB_FIELDINFO_1(tag, type, data_offset, data_size, size_offset, array_size) \
00788     (0 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(data_offset) & 0xFF) « 16) | \
00789     (((uint32_t)(size_offset) & 0x0F) « 24) | (((uint32_t)(data_size) & 0x0F) « 28)),
00790
00791 #define PB_FIELDINFO_2(tag, type, data_offset, data_size, size_offset, array_size) \
00792     (1 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(array_size) & 0xFFF) « 16) | \
    (((uint32_t)(size_offset) & 0x0F) « 28)), \

```

```

00793     (((uint32_t)(data_offset) & 0xFFFF) | (((uint32_t)(data_size) & 0xFFF) « 16) | (((uint32_t)(tag) & 0x3c0) « 22)),
00794
00795 #define PB_FIELDINFO_4(tag, type, data_offset, data_size, size_offset, array_size) \
00796     (2 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8) | (((uint32_t)(array_size) & 0xFFFF) « 16)), \
00797     ((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) « 2) & 0xFFFFFFF0)), \
00798     (data_offset), (data_size),
00799
00800 #define PB_FIELDINFO_8(tag, type, data_offset, data_size, size_offset, array_size) \
00801     (3 | (((uint32_t)(tag) « 2) & 0xFF) | ((type) « 8)), \
00802     ((uint32_t)(int_least8_t)(size_offset) | (((uint32_t)(tag) « 2) & 0xFFFFFFF0)), \
00803     (data_offset), (data_size), (array_size), 0, 0, 0,
00804
00805 /* These assertions verify that the field information fits in the allocated space.
00806 * The generator tries to automatically determine the correct width that can fit all
00807 * data associated with a message. These asserts will fail only if there has been a
00808 * problem in the automatic logic - this may be worth reporting as a bug. As a workaround,
00809 * you can increase the descriptor width by defining PB_FIELDINFO_WIDTH or by setting
00810 * descriptorsize option in .options file.
00811 */
00812 #define PB_FITS(value,bits) ((uint32_t)(value) < ((uint32_t)1«bits))
00813 #define PB_FIELDINFO_ASSERT_1(tag, type, data_offset, data_size, size_offset, array_size) \
00814     PB_STATIC_ASSERT(PB_FITS(tag,6) && PB_FITS(data_offset,8) && PB_FITS(size_offset,4) &&
00815     PB_FITS(data_size,4) && PB_FITS(array_size,1), FIELDINFO_DOES_NOT_FIT_width1_field ## tag)
00816
00817 #define PB_FIELDINFO_ASSERT_2(tag, type, data_offset, data_size, size_offset, array_size) \
00818     PB_STATIC_ASSERT(PB_FITS(tag,10) && PB_FITS(data_offset,16) && PB_FITS(size_offset,4) &&
00819     PB_FITS(data_size,12) && PB_FITS(array_size,12), FIELDINFO_DOES_NOT_FIT_width2_field ## tag)
00820
00821 #ifndef PB_FIELD_32BIT
00822 /* Maximum field sizes are still 16-bit if pb_size_t is 16-bit */
00823 #define PB_FIELDINFO_ASSERT_4(tag, type, data_offset, data_size, size_offset, array_size) \
00824     PB_STATIC_ASSERT(PB_FITS(tag,16) && PB_FITS(data_offset,16) && PB_FITS((int_least8_t)size_offset,8)
00825     && PB_FITS(data_size,16) && PB_FITS(array_size,16), FIELDINFO_DOES_NOT_FIT_width4_field ## tag)
00826
00827 #define PB_FIELDINFO_ASSERT_8(tag, type, data_offset, data_size, size_offset, array_size) \
00828     PB_STATIC_ASSERT(PB_FITS(tag,16) && PB_FITS(data_offset,16) && PB_FITS((int_least8_t)size_offset,8)
00829     && PB_FITS(data_size,16) && PB_FITS(array_size,16), FIELDINFO_DOES_NOT_FIT_width8_field ## tag)
00830 #else
00831 /* Up to 32-bit fields supported.
00832 * Note that the checks are against 31 bits to avoid compiler warnings about shift wider than type in the test.
00833 * I expect that there is no reasonable use for >2GB messages with nanopb anyway.
00834 */
00835 #define PB_FIELDINFO_ASSERT_4(tag, type, data_offset, data_size, size_offset, array_size) \
00836     PB_STATIC_ASSERT(PB_FITS(tag,30) && PB_FITS(data_offset,31) && PB_FITS(size_offset,8) &&
00837     PB_FITS(data_size,31) && PB_FITS(array_size,16), FIELDINFO_DOES_NOT_FIT_width4_field ## tag)
00838
00839 #define PB_FIELDINFO_ASSERT_8(tag, type, data_offset, data_size, size_offset, array_size) \
00840     PB_STATIC_ASSERT(PB_FITS(tag,30) && PB_FITS(data_offset,31) && PB_FITS(size_offset,8) &&
00841     PB_FITS(data_size,31) && PB_FITS(array_size,31), FIELDINFO_DOES_NOT_FIT_width8_field ## tag)
00842 #endif
00843
00844 /* Automatic picking of FIELDINFO width:
00845 * Uses width 1 when possible, otherwise resorts to width 2.
00846 * This is used when PB_BIND() is called with "AUTO" as the argument.
00847 * The generator will give explicit size argument when it knows that a message
00848 * structure grows beyond 1-word format limits.
00849 */
00850 #define PB_FIELDINFO_WIDTH_AUTO(atype, htype, ltype) PB_FI_WIDTH ## atype(htype, ltype)
00851 #define PB_FI_WIDTH_PB_ATYPE_STATIC(htype, ltype) PB_FI_WIDTH ## htype(ltype)
00852 #define PB_FI_WIDTH_PB_ATYPE_POINTER(htype, ltype) PB_FI_WIDTH ## htype(ltype)
00853 #define PB_FI_WIDTH_PB_ATYPE_CALLBACK(htype, ltype) 2
00854 #define PB_FI_WIDTH_PB_HTYPE_REQUIRED(ltype) PB_FI_WIDTH ## ltype
00855 #define PB_FI_WIDTH_PB_HTYPE_SINGULAR(ltype) PB_FI_WIDTH ## ltype
00856 #define PB_FI_WIDTH_PB_HTYPE_OPTIONAL(ltype) PB_FI_WIDTH ## ltype
00857 #define PB_FI_WIDTH_PB_HTYPE_ONEOF(ltype) PB_FI_WIDTH ## ltype
00858 #define PB_FI_WIDTH_PB_HTYPE_REPEATED(ltype) 2
00859 #define PB_FI_WIDTH_PB_HTYPE_FIXARRAY(ltype) 2
00860 #define PB_FI_WIDTH_PB_LTYPE_BOOL 1
00861 #define PB_FI_WIDTH_PB_LTYPE_BYTES 2
00862 #define PB_FI_WIDTH_PB_LTYPE_DOUBLE 1
00863 #define PB_FI_WIDTH_PB_LTYPE_ENUM 1
00864 #define PB_FI_WIDTH_PB_LTYPE_UENUM 1
00865 #define PB_FI_WIDTH_PB_LTYPE_FIXED32 1
00866 #define PB_FI_WIDTH_PB_LTYPE_FIXED64 1
00867 #define PB_FI_WIDTH_PB_LTYPE_FLOAT 1
00868 #define PB_FI_WIDTH_PB_LTYPE_INT32 1
00869 #define PB_FI_WIDTH_PB_LTYPE_INT64 1
00870 #define PB_FI_WIDTH_PB_LTYPE_MESSAGE 2
00871 #define PB_FI_WIDTH_PB_LTYPE_MSG_W_CB 2
00872 #define PB_FI_WIDTH_PB_LTYPE_SFIXED32 1
00873 #define PB_FI_WIDTH_PB_LTYPE_SFIXED64 1
00874 #define PB_FI_WIDTH_PB_LTYPE_SINT32 1
00875 #define PB_FI_WIDTH_PB_LTYPE_SINT64 1
00876 #define PB_FI_WIDTH_PB_LTYPE_STRING 2
00877 #define PB_FI_WIDTH_PB_LTYPE_UINT32 1
00878 #define PB_FI_WIDTH_PB_LTYPE_UINT64 1

```

```

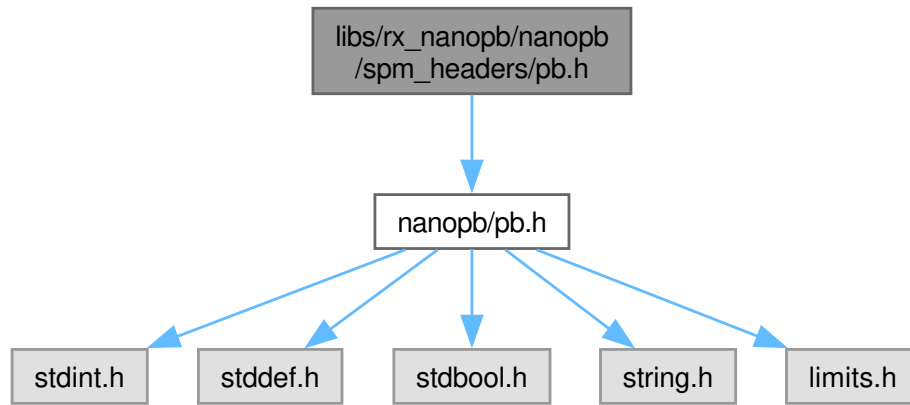
00874 #define PB_FI_WIDTH_PB_LTYPE_EXTENSION 1
00875 #define PB_FI_WIDTH_PB_LTYPE_FIXED_LENGTH_BYTES 2
00876
00877 /* The mapping from protobuf types to LTYPEs is done using these macros. */
00878 #define PB_LTYPE_MAP_BOOL PB_LTYPE_BOOL
00879 #define PB_LTYPE_MAP_BYTES PB_LTYPE_BYTES
00880 #define PB_LTYPE_MAP_DOUBLE PB_LTYPE_FIXED64
00881 #define PB_LTYPE_MAP_ENUM PB_LTYPE_VARINT
00882 #define PB_LTYPE_MAP_UENUM PB_LTYPE_UVARINT
00883 #define PB_LTYPE_MAP_FIXED32 PB_LTYPE_FIXED32
00884 #define PB_LTYPE_MAP_FIXED64 PB_LTYPE_FIXED64
00885 #define PB_LTYPE_MAP_FLOAT PB_LTYPE_FIXED32
00886 #define PB_LTYPE_MAP_INT32 PB_LTYPE_VARINT
00887 #define PB_LTYPE_MAP_INT64 PB_LTYPE_VARINT
00888 #define PB_LTYPE_MAP_MESSAGE PB_LTYPE_SUBMESSAGE
00889 #define PB_LTYPE_MAP_MSG_W_CB PB_LTYPE_SUBMSG_W_CB
00890 #define PB_LTYPE_MAP_SFIXED32 PB_LTYPE_FIXED32
00891 #define PB_LTYPE_MAP_SFIXED64 PB_LTYPE_FIXED64
00892 #define PB_LTYPE_MAP_SINT32 PB_LTYPE_SVARINT
00893 #define PB_LTYPE_MAP_SINT64 PB_LTYPE_SVARINT
00894 #define PB_LTYPE_MAP_STRING PB_LTYPE_STRING
00895 #define PB_LTYPE_MAP_UINT32 PB_LTYPE_UVARINT
00896 #define PB_LTYPE_MAP_UINT64 PB_LTYPE_UVARINT
00897 #define PB_LTYPE_MAP_EXTENSION PB_LTYPE_EXTENSION
00898 #define PB_LTYPE_MAP_FIXED_LENGTH_BYTES PB_LTYPE_FIXED_LENGTH_BYTES
00899
00900 /* These macros are used for giving out error messages.
00901  * They are mostly a debugging aid; the main error information
00902  * is the true/false return value from functions.
00903  * Some code space can be saved by disabling the error
00904  * messages if not used.
00905  *
00906  * PB_SET_ERROR() sets the error message if none has been set yet.
00907  * msg must be a constant string literal.
00908  * PB_GET_ERROR() always returns a pointer to a string.
00909  * PB_RETURN_ERROR() sets the error and returns false from current
00910  * function.
00911  */
00912 #ifndef PB_NO_ERRMSG
00913 #define PB_SET_ERROR(stream, msg) PB_UNUSED(stream)
00914 #define PB_GET_ERROR(stream) "(errmsg disabled)"
00915 #else
00916 #define PB_SET_ERROR(stream, msg) (stream->errmsg = (stream)->errmsg ? (stream)->errmsg : (msg))
00917 #define PB_GET_ERROR(stream) ((stream)->errmsg ? (stream)->errmsg : "(none)")
00918 #endif
00919
00920 #define PB_RETURN_ERROR(stream, msg) return PB_SET_ERROR(stream, msg), false
00921
00922 #ifdef __cplusplus
00923 } /* extern "C" */
00924 #endif
00925
00926 #ifdef __cplusplus
00927 #if __cplusplus >= 201103L
00928 #define PB_CONSTEXPR constexpr
00929 #else // __cplusplus >= 201103L
00930 #define PB_CONSTEXPR
00931 #endif // __cplusplus >= 201103L
00932
00933 #if __cplusplus >= 201703L
00934 #define PB_INLINE_CONSTEXPR inline constexpr
00935 #else // __cplusplus >= 201703L
00936 #define PB_INLINE_CONSTEXPR PB_CONSTEXPR
00937 #endif // __cplusplus >= 201703L
00938
00939 extern "C++"
00940 {
00941 namespace nanopb {
00942 // Each type will be partially specialized by the generator.
00943 template <typename GenMessageType> struct MessageDescriptor;
00944 } // namespace nanopb
00945 }
00946 #endif /* __cplusplus */
00947
00948 #endif

```

6.83 libs/rx_nanopb/nanopb/spm_headers/pb.h File Reference

#include "nanopb/pb.h"

Include dependency graph for pb.h:



6.84 pb.h

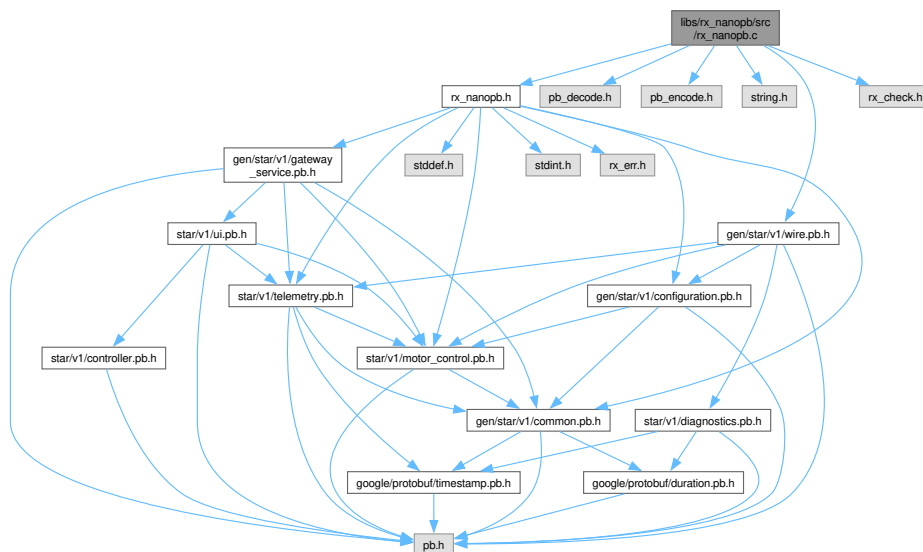
[Go to the documentation of this file.](#)

```
00001 #include "nanopb/pb.h"
```

6.85 libs/rx_nanopb/src/rx_nanopb.c File Reference

nanopb Integration Wrapper Implementation for RX72N

```
#include "rx_nanopb.h"
#include <pb_decode.h>
#include <pb_encode.h>
#include <string.h>
#include "gen/star/v1/wire.pb.h"
#include "rx_check.h"
Include dependency graph for rx_nanopb.c:
```



Functions

- `rx_err_t rx_nanopb_init` (void)
Initialize nanopb wrapper module.
- `void rx_nanopb_test_reset_state` (void)
Reset module state for unit testing.
- `rx_err_t rx_nanopb_encode_velocity_request` (const `star_v1_SetVelocityRequest` *msg, `uint8_t` *buffer, const `uint32_t` buffer_size, `uint32_t` *len)
Encode SetVelocityRequest message to Protocol Buffer wire format.
- `rx_err_t rx_nanopb_decode_velocity_request` (const `uint8_t` *buffer, const `uint32_t` len, `star_v1_SetVelocityRequest` *msg)
Decode SetVelocityRequest message from Protocol Buffer wire format.
- `rx_err_t rx_nanopb_encode_velocity_response` (const `star_v1_SetVelocityResponse` *msg, `uint8_t` *buffer, const `uint32_t` buffer_size, `uint32_t` *len)
Encode SetVelocityResponse message to Protocol Buffer wire format.
- `rx_err_t rx_nanopb_decode_estop_request` (const `uint8_t` *buffer, const `uint32_t` len, `star_v1_EmergencyStopRequest` *msg)
Decode EmergencyStopRequest message from Protocol Buffer wire format.
- `rx_err_t rx_nanopb_encode_estop_response` (const `star_v1_EmergencyStopResponse` *msg, `uint8_t` *buffer, const `uint32_t` buffer_size, `uint32_t` *len)
Encode EmergencyStopResponse message to Protocol Buffer wire format.
- `rx_err_t rx_nanopb_decode_pid_gains_request` (const `uint8_t` *buffer, const `uint32_t` len, `star_v1_SetPidGainsRequest` *msg)
Decode SetPidGainsRequest message from Protocol Buffer wire format.
- `rx_err_t rx_nanopb_encode_pid_gains_response` (const `star_v1_SetPidGainsResponse` *msg, `uint8_t` *buffer, const `uint32_t` buffer_size, `uint32_t` *len)
Encode SetPidGainsResponse message to Protocol Buffer bytes.
- `rx_err_t rx_nanopb_decode_retransmit_config_request` (const `uint8_t` *buffer, const `uint32_t` len, `star_v1_SetRetransmitConfigRequest` *msg)
Decode SetRetransmitConfigRequest from Protocol Buffer bytes.
- `rx_err_t rx_nanopb_encode_telemetry` (const `star_v1_TelemetryData` *msg, `uint8_t` *buffer, const `uint32_t` buffer_size, `uint32_t` *len)
Encode TelemetryData message to Protocol Buffer wire format.
- `rx_err_t rx_nanopb_create_velocity_command` (`star_v1_VelocityCommand` *cmd, const `rx_velocity_command_params_t` *params)
Create VelocityCommand for 4 independent motor velocities.
- `rx_err_t rx_nanopb_create_velocity_command_diff_drive` (`star_v1_VelocityCommand` *cmd, const `rx_velocity_diff_drive_params_t` *params)
Create VelocityCommand in differential drive mode.
- `void rx_nanopb_create_response_header` (`star_v1_ResponseHeader` *header, const `star_v1_Status` status, const char *request_id)
Create ResponseHeader with status and request ID.

Variables

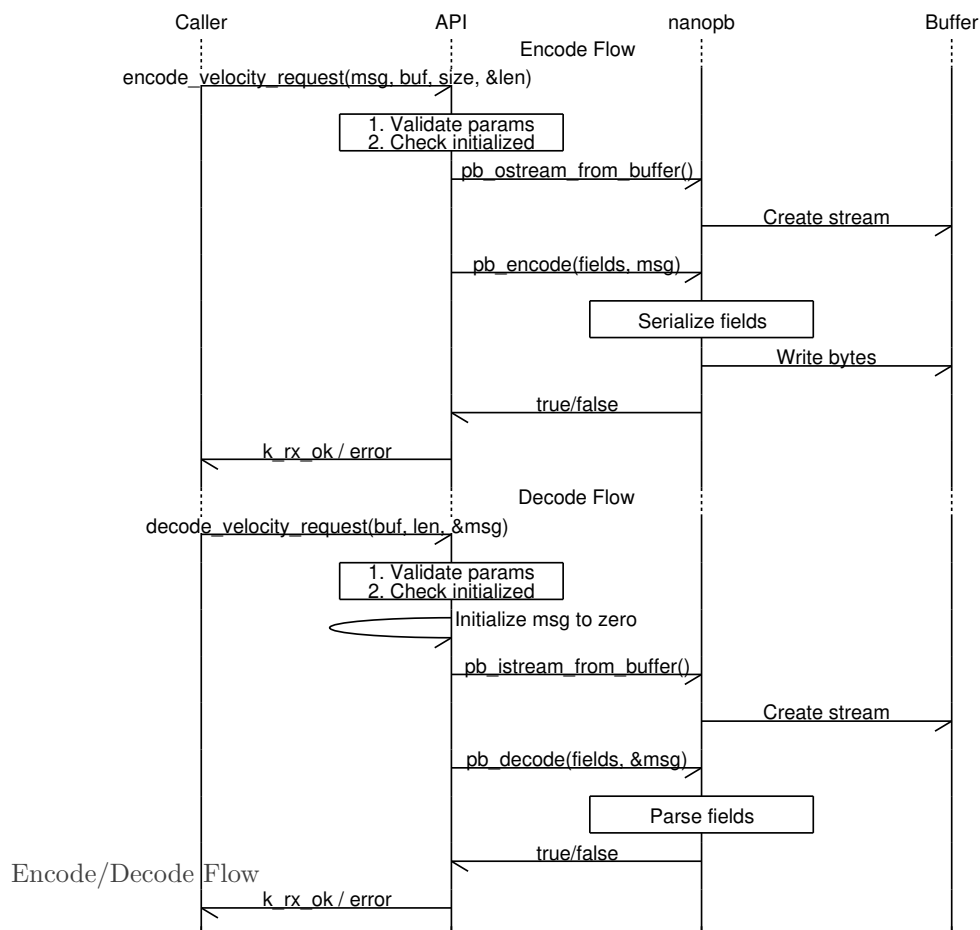
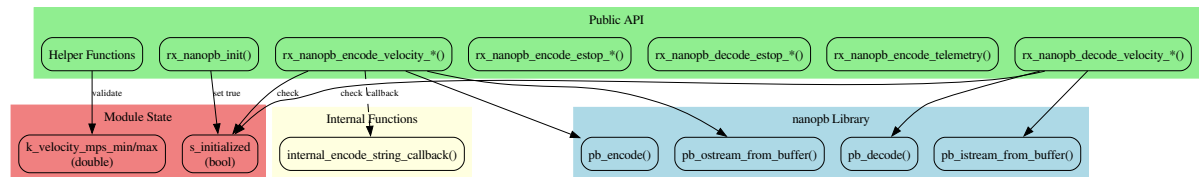
- static `bool s_initialized` = false
Module initialization state flag.
- static const double `k_velocity_mps_min` = -1000.0
Minimum valid motor velocity in meters per second.
- static const double `k_velocity_mps_max` = 1000.0
Maximum valid motor velocity in meters per second.

6.85.1 Detailed Description

nanopb Integration Wrapper Implementation for RX72N

Implements the Protocol Buffer encode/decode wrapper functions for the STAR project's motor control and telemetry messaging system. This module provides a safety-critical interface between the nanopb library and the RX72N firmware application layer.

Implementation Architecture



Memory Layout

Variable	Type	Size	Description
s_initialized	bool	1 byte	Module state flag
k_velocity_mps_min	const double	8 bytes	Min velocity bound
k_velocity_mps_max	const double	8 bytes	Max velocity bound
Total Static		17 bytes	Minimal footprint

Performance Characteristics

Function	Time @ 240 MHz	Stack Usage	Notes
rx_nanopb_init()	~1 us	8 bytes	One-time setup
encode_velocity_request	~20 us	~96 bytes	4 doubles + overhead
decode_velocity_request	~15 us	~96 bytes	4 doubles + overhead
encode_velocity_response	~15 us	~48 bytes	Header only
encode_telemetry	~30 us	~128 bytes	Full telemetry
decode_estop_request	~10 us	~32 bytes	Minimal message
create_velocity_command	~2 us	~16 bytes	Copy operations

Error Handling Strategy

1. Parameter Validation: nullptr checks, size bounds (NASA Rule 5)
2. State Validation: Module must be initialized
3. nanopb Errors: Translate pb_encode/pb_decode failures
4. Post-conditions: Verify output bounds

Thread Safety Analysis

Function	Thread Safe	Notes
rx_nanopb_init()	No	Call once at startup
encode_* functions	No	Protect with mutex
decode_* functions	No	Protect with mutex
create_* helpers	Yes	No shared state
test_reset_state	No	Testing only

NASA Power of 10 Compliance

- Rule 1: [OK] No goto, setjmp, longjmp, or recursion
- Rule 2: [OK] All loops bounded by nanopb field counts (internal)
- Rule 3: [OK] No dynamic memory allocation
- Rule 4: [OK] All functions under 60 lines
- Rule 5: [OK] Input validation with minimum 2 checks per function

- Rule 6: [OK] Variables declared at smallest scope
- Rule 7: [OK] All nanopb return values checked
- Rule 8: [OK] Minimal preprocessor usage
- Rule 9: [OK] Function pointer only for nanopb callback
- Rule 10: [OK] Compiles with -Wall -Wextra -Werror

SOLID Principles Compliance

- S (Single Responsibility): Only protobuf encode/decode operations
- O (Open/Closed): Extensible by adding new message functions
- L (Liskov Substitution): N/A (no inheritance)
- I (Interface Segregation): Separate function per message type
- D (Dependency Inversion): Depends on abstract pb_fields_t

See also

[rx_nanopb.h](#) Public API header with message documentation
gen/star/v1/ Generated protobuf message definitions (.pb.h files)
star-proto/proto/star/v1/ Protocol Buffer schema files

Author

Locked, Inc.

Date

2026-01-27

Version

1.0.0

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Definition in file [rx_nanopb.c](#).

6.85.2 Function Documentation

6.85.2.1 rx_nanopb_create_response_header()

```
void rx_nanopb_create_response_header (  
    star_v1_ResponseHeader * header,  
    const star_v1_Status status,  
    const char * request_id)
```

Create ResponseHeader with status and request ID.

Create ResponseHeader with status.

Populates a ResponseHeader structure for response messages. Sets the status code and optionally echoes the request ID from the original request for correlation purposes.

Algorithm Steps

1. Validate header pointer (early return if nullptr)
2. Validate request_id length if provided (early return if too long)
3. Initialize header to zero
4. Set status field
5. If request_id provided, set up nanopb callback for encoding

Callback Setup

When request_id is provided, this function sets up a nanopb callback function (internal_encode_string_callback) that will be invoked during pb_encode() to serialize the string field.

Precondition

header must not be nullptr (silently returns if nullptr)
request_id, if provided, must remain valid until encoding completes

Postcondition

header->status == status
If request_id != nullptr: callback configured for encoding

Note

Thread-safe (no shared state)
No return value - silently ignores nullptr header

Warning

request_id pointer must remain valid until pb_encode() is called. Do not pass stack-allocated strings that may go out of scope.

Performance

- Execution time: ~ 1 us @ 240 MHz
- Stack usage: ~ 8 bytes

Example - Success Response:

```

star_v1_SetVelocityResponse response;

// After successfully processing velocity command
rx_nanopb_create_response_header(&response.header,
                                star_v1_Status_STATUS_OK,
                                request.header.request_id);

// Encode and send
rx_nanopb_encode_velocity_response(&response, buffer, sizeof(buffer), &len);

```

Example - Error Response:

```

// After detecting invalid velocity value
rx_nanopb_create_response_header(&response.header,
                                star_v1_Status_STATUS_INVALID_ARGUMENT,
                                request.header.request_id);

```

See also

[star_v1_Status](#) Status enumeration values
[internal_encode_string_callback\(\)](#) nanopb callback for string encoding

Since

Version 1.0.0

Definition at line 1542 of file [rx_nanopb.c](#).

```

01545 {
01546     if (header == nullptr) {
01547         return;
01548     }
01549
01550     if (request_id != nullptr) {
01551         size_t req_id_len = 0;
01552         while (request_id[req_id_len] != '\0') {
01553             req_id_len++;
01554         }
01555         if (req_id_len > s_nanopb_buffer_size) {
01556             return;
01557         }
01558     }
01559
01560     *header = (star_v1_ResponseHeader)star_v1_ResponseHeader_init_zero;
01561     header->status = status;
01562
01563     /* Copy request_id string if provided (fixed-size field in generated code) */
01564     if (request_id != nullptr) {
01565         const size_t max_len = sizeof(header->request_id) - 1U;
01566         size_t idx = 0;
01567         while (idx < max_len && request_id[idx] != '\0') {
01568             header->request_id[idx] = request_id[idx];
01569             idx++;
01570         }
01571         header->request_id[idx] = '\0';
01572     }
01573 }

```

References [star_v1_ResponseHeader::request_id](#), [s_nanopb_buffer_size](#), [star_v1_ResponseHeader_init_zero](#), and [star_v1_ResponseHeader::status](#).

6.85.2.2 rx_nanopb_create_velocity_command()

```
rx_err_t rx_nanopb_create_velocity_command (  
    star_v1_VelocityCommand * cmd,  
    const rx_velocity_command_params_t * params)  [nodiscard]
```

Create VelocityCommand for 4 independent motor velocities.

Create VelocityCommand for 4 independent motors.

Populates a VelocityCommand structure from parameters. Use for mecanum or omnidirectional drive robots where each motor requires independent velocity control. Performs range validation on all velocity values.

Algorithm Steps

1. Validate cmd pointer (not nullptr)
2. Validate params pointer (not nullptr)
3. Validate all velocities in range [k_velocity_mps_min, k_velocity_mps_max]
4. Initialize cmd to zero (clear previous state)
5. Copy velocity values from params to cmd
6. Copy sequence number
7. Set timestamp to 0 (caller sets if needed)

Precondition

cmd must not be nullptr
params must not be nullptr
All velocity values must be in valid range

Postcondition

cmd fully populated with velocity values from params
cmd->timestamp_us == 0

Note

Thread-safe (no shared state)

Performance

- Execution time: ~2 us @ 240 MHz
- Stack usage: ~16 bytes

Example - Basic Usage:

```
rx_velocity_command_params_t params = {
    .front_left_mps = 1.0,
    .front_right_mps = 1.0,
    .back_left_mps = 1.0,
    .back_right_mps = 1.0,
    .sequence = g_command_sequence++,
};

star_v1_VelocityCommand cmd;
rx_err_t err = rx_nanopb_create_velocity_command(&cmd, &params);
if (err == k_rx_ok) {
    // Use cmd in SetVelocityRequest
    request.command = cmd;
}
```

See also

[rx_velocity_command_params_t](#) Parameter structure

[rx_nanopb_create_velocity_command_diff_drive\(\)](#) For differential drive

Since

Version 1.0.0

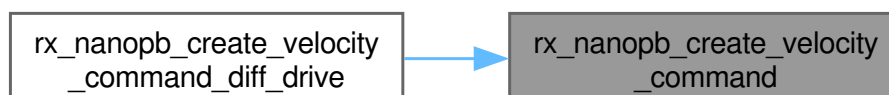
Definition at line 1362 of file [rx_nanopb.c](#).

```
01364 {
01365     /* Use bitwise | to evaluate both sides without short-circuit branches */
01366     const bool cmd_null = (bool)(cmd == nullptr);
01367     const bool params_null = (bool)(params == nullptr);
01368     if ((bool)((int)cmd_null | (int)params_null)) {
01369         return k_rx_err_invalid_arg;
01370     }
01371
01372     /* Use bitwise | to evaluate all velocity bounds without short-circuit branches */
01373     const bool fl_lo = (bool)(params->front_left_mps < k_velocity_mps_min);
01374     const bool fl_hi = (bool)(params->front_left_mps > k_velocity_mps_max);
01375     const bool fr_lo = (bool)(params->front_right_mps < k_velocity_mps_min);
01376     const bool fr_hi = (bool)(params->front_right_mps > k_velocity_mps_max);
01377     const bool bl_lo = (bool)(params->back_left_mps < k_velocity_mps_min);
01378     const bool bl_hi = (bool)(params->back_left_mps > k_velocity_mps_max);
01379     const bool br_lo = (bool)(params->back_right_mps < k_velocity_mps_min);
01380     const bool br_hi = (bool)(params->back_right_mps > k_velocity_mps_max);
01381     if ((bool)((int)fl_lo | (int)fl_hi | (int)fr_lo | (int)fr_hi | (int)bl_lo | (int)bl_hi |
01382             (int)br_lo | (int)br_hi)) {
01383         return k_rx_err_invalid_arg;
01384     }
01385
01386     *cmd = (star_v1_VelocityCommand)star_v1_VelocityCommand_init_zero;
01387     cmd->front_left_velocity_mps = params->front_left_mps;
01388     cmd->front_right_velocity_mps = params->front_right_mps;
01389     cmd->back_left_velocity_mps = params->back_left_mps;
01390     cmd->back_right_velocity_mps = params->back_right_mps;
01391     cmd->sequence = params->sequence;
01392     cmd->timestamp_us = 0; /* Set by caller if needed */
01393     return k_rx_ok;
01394 }
```

References [rx_velocity_command_params_t::back_left_mps](#), [star_v1_VelocityCommand::back_left_velocity_mps](#), [rx_velocity_command_params_t::back_right_mps](#), [star_v1_VelocityCommand::back_right_velocity_mps](#), [rx_velocity_command_params_t::front_left_mps](#), [star_v1_VelocityCommand::front_left_velocity_mps](#), [rx_velocity_command_params_t::front_right_mps](#), [star_v1_VelocityCommand::front_right_velocity_mps](#), [k_velocity_mps_max](#), [k_velocity_mps_min](#), [rx_velocity_command_params_t::sequence](#), [star_v1_VelocityCommand::star_v1_VelocityCommand_init_zero](#), and [star_v1_VelocityCommand::timestamp_us](#).

Referenced by [rx_nanopb_create_velocity_command_diff_drive\(\)](#).

Here is the caller graph for this function:



6.85.2.3 rx_nanopb_create_velocity_command_diff_drive()

```
rx_err_t rx_nanopb_create_velocity_command_diff_drive (
    star_v1_VelocityCommand * cmd,
    const rx_velocity_diff_drive_params_t * params)
```

Create VelocityCommand in differential drive mode.

Populates a VelocityCommand for differential drive robots where left and right sides are controlled together. Front and back motors on the same side receive identical velocity values.

Velocity Mapping

```
params->left_mps -> cmd->front_left_velocity_mps
               -> cmd->back_left_velocity_mps

params->right_mps -> cmd->front_right_velocity_mps
                -> cmd->back_right_velocity_mps
```

Motion Examples

left_mps	right_mps	Robot Motion
+1.0	+1.0	Forward
-1.0	-1.0	Reverse
+1.0	-1.0	Rotate clockwise
-1.0	+1.0	Rotate counter-clockwise
+1.0	+0.5	Curve right (forward)
+0.5	+1.0	Curve left (forward)

Precondition

cmd must not be nullptr
 params must not be nullptr
 Velocities must be in valid range [-1000.0, +1000.0] m/s

Postcondition

```
cmd->front_left_velocity_mps == cmd->back_left_velocity_mps
cmd->front_right_velocity_mps == cmd->back_right_velocity_mps
```

Note

Thread-safe (no shared state)
 Internally calls `rx_nanopb_create_velocity_command()`

Performance

- Execution time: ~3 us @ 240 MHz
- Stack usage: ~48 bytes

Example - Turn Left:

```
// Turn left by making right side faster than left
rx_velocity_diff_drive_params_t params = {
    .left_mps = 0.5, // Slower
    .right_mps = 1.0, // Faster
    .sequence = g_sequence++,
};

star_v1_VelocityCommand cmd;
rx_err_t err = rx_nanopb_create_velocity_command_diff_drive(&cmd, &params);
// Result: FL=BL=0.5, FR=BR=1.0 (robot curves left)
```

See also

[rx_velocity_diff_drive_params_t](#) Parameter structure
[rx_nanopb_create_velocity_command\(\)](#) 4-motor independent control

Since

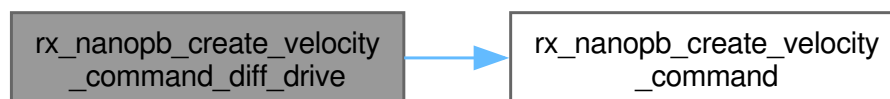
Version 1.0.0

Definition at line 1458 of file [rx_nanopb.c](#).

```
01460 {
01461     rx_velocity_command_params_t params_t;
01462
01463     if (cmd == nullptr || params == nullptr) {
01464         return k_rx_err_invalid_arg;
01465     }
01466
01467     /* Differential drive mode: Lock left 2 motors together, right 2 motors together
01468      * Front Left & Back Left = Left side
01469      * Front Right & Back Right = Right side */
01470     params_t.front_left_mps = params->left_mps;
01471     params_t.front_right_mps = params->right_mps;
01472     params_t.back_left_mps = params->left_mps;
01473     params_t.back_right_mps = params->right_mps;
01474     params_t.sequence = params->sequence;
01475
01476     return rx_nanopb_create_velocity_command(cmd, &params_t);
01477 }
```

References [rx_velocity_command_params_t::back_left_mps](#), [rx_velocity_command_params_t::back_right_mps](#), [rx_velocity_command_params_t::front_left_mps](#), [rx_velocity_command_params_t::front_right_mps](#), [rx_velocity_diff_drive_params_t::left_mps](#), [rx_velocity_diff_drive_params_t::right_mps](#), [rx_nanopb_create_velocity_command\(\)](#), [rx_velocity_command_params_t::sequence](#), and [rx_velocity_diff_drive_params_t::sequence](#).

Here is the call graph for this function:



6.85.2.4 rx_nanopb_decode_estop_request()

```
rx_err_t rx_nanopb_decode_estop_request (
    const uint8_t * buffer,
    const uint32_t len,
    star_v1_EmergencyStopRequest * msg) [nodiscard]
```

Decode EmergencyStopRequest message from Protocol Buffer wire format.

Decode EmergencyStopRequest message from Protocol Buffer bytes.

Deserializes an emergency stop command received from RPi5. This is a safety-critical message that triggers immediate motor shutdown.

Safety-Critical Processing

Warning

E-Stop commands must be processed immediately. Consider stopping motors BEFORE attempting to decode if message type is identified.

Algorithm Steps

1. Validate buffer pointer (not nullptr)
2. Validate msg pointer (not nullptr)
3. Validate len (not zero, not too large)
4. Verify module is initialized
5. Initialize msg to zero
6. Create nanopb input stream from buffer
7. Decode message using generated field descriptors

Precondition

`rx_nanopb_init()` must have been called
buffer contains valid Protocol Buffer data

Postcondition

msg fully populated with E-Stop request fields

Note

Not thread-safe

Warning

Always stop motors regardless of decode result

Performance

- Execution time: ~10 us @ 240 MHz
- Stack usage: ~32 bytes

Example - E-Stop Handling:

```
void handle_estop_message(const uint8_t* buffer, uint32_t len) {  
    // FIRST: Stop motors immediately (safety-critical)  
    motor_emergency_stop_all();  
  
    // THEN: Decode message for response  
    star_v1_EmergencyStopRequest estop_req;  
    rx_err_t err = rx_nanopb_decode_estop_request(buffer, len, &estop_req);  
  
    // Send response (motors already stopped)  
    star_v1_EmergencyStopResponse response = {};  
    rx_nanopb_create_response_header(&response.header,  
                                    star_v1_Status_STATUS_OK,  
                                    estop_req.header.request_id);  
    // ... encode and send response  
}
```

See also

[rx_nanopb_encode_estop_response\(\)](#) Send acknowledgment

Since

Version 1.0.0

Definition at line 863 of file [rx_nanopb.c](#).

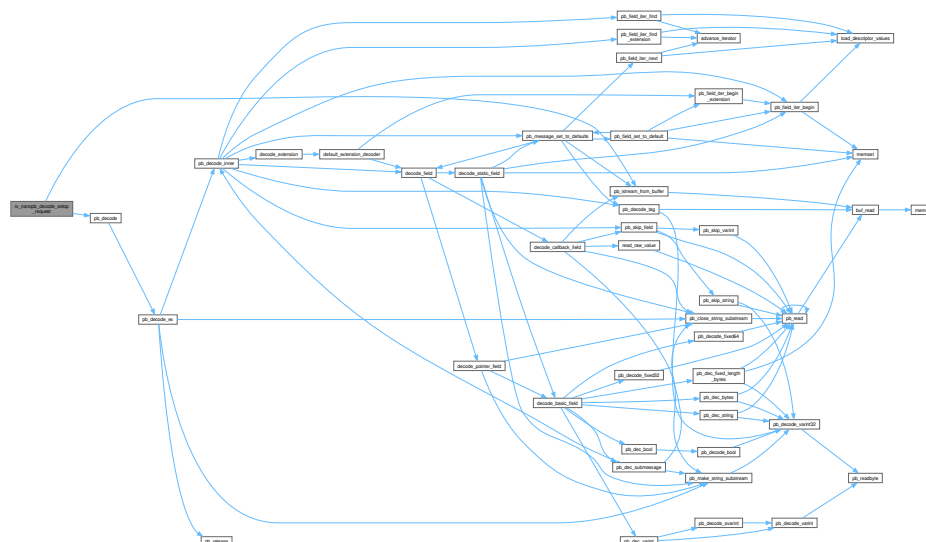
```

00866 {
00867     /* Pre-condition 1: nullptr pointer checks */
00868     if (buffer == nullptr || msg == nullptr) {
00869         return k_rx_err_invalid_arg;
00870     }
00871
00872     /* Pre-condition 2: Length validation */
00873     if (len == 0 || len > s_nanopb_buffer_size) {
00874         return k_rx_err_invalid_arg;
00875     }
00876
00877     /* Pre-condition 3: Module initialized */
00878     if (!s_initialized) {
00879         return k_rx_err_not_initialized;
00880     }
00881
00882     /* Initialize message to default values */
00883     *msg = (star_v1_EmergencyStopRequest)star_v1_EmergencyStopRequest_init_zero;
00884
00885     pb_istream_t stream = pb_istream_from_buffer(buffer, len);
00886
00887     if (!pb_decode(&stream, star_v1_EmergencyStopRequest_fields, msg)) {
00888         return k_rx_err_protocol_error;
00889     }
00890
00891     return k_rx_ok;
00892 }

```

References [pb_decode\(\)](#), [pb_istream_from_buffer\(\)](#), [s_initialized](#), [s_nanopb_buffer_size](#), [star_v1_EmergencyStopRequest_init_zero](#) and [star_v1_EmergencyStopRequest_init_zero](#).

Here is the call graph for this function:



6.85.2.5 rx_nanopb_decode_pid_gains_request()

```
rx_err_t rx_nanopb_decode_pid_gains_request (
    const uint8_t * buffer,
    const uint32_t len,
    star_v1_SetPidGainsRequest * msg) [nodiscard]
```

Decode SetPidGainsRequest message from Protocol Buffer wire format.

Decode SetPidGainsRequest message from Protocol Buffer bytes.

Deserializes a PID gains update command from RPi5. This message contains new PID controller configuration (gains and limits) to be applied to one or more motors.

Message Processing

1. Validate input parameters (buffer, length, message pointer)
2. Check module initialization state
3. Initialize message structure to zero
4. Create nanopb input stream from buffer
5. Decode using star_v1_SetPidGainsRequest_fields descriptor
6. Return success or protocol error

PID Configuration Fields

The decoded message contains:

- header: Standard RequestHeader with request_id
- pid_config: PidConfig structure with:
 - kp: Proportional gain
 - ki: Integral gain
 - kd: Derivative gain
 - output_min_percent: Minimum PWM duty cycle (%)
 - output_max_percent: Maximum PWM duty cycle (%)
 - integral_min: Anti-windup lower bound
 - integral_max: Anti-windup upper bound
- motor_id: Target motor (0-3 for specific motor, -1 for all motors)

Precondition

`rx_nanopb_init()` must be called first
 buffer must point to valid wire-format data
 len must match actual encoded message size

Postcondition

msg fully populated with decoded PID gains
 msg->has_pid_config == true if gains present

Invariant

msg initialized to zero before decode (prevents stale data)

Note

Thread Safety: Not thread-safe (uses shared nanopb state)

Re-entrancy: NOT reentrant. Single-threaded decode only.

Performance: ~18 us @ 240 MHz (lightweight message)

Warning

Gain Validation: This function does NOT validate gain ranges. Caller must validate that Kp, Ki, Kd are non-negative and within safe operating bounds before applying to PID controllers.

Motor ID: motor_id == -1 means apply to ALL motors. Caller must handle this case explicitly.

Example - Single Motor Update:

```
star_v1_SetPidGainsRequest request;
rx_err_t err = rx_nanopb_decode_pid_gains_request(spi_buffer, spi_len, &request);

if (err == k_rx_ok && request.has_pid_config) {
    // Convert protobuf gains to firmware structure
    pid_gains_t gains = {
        .kp = (float)request.pid_config.kp,
        .ki = (float)request.pid_config.ki,
        .kd = (float)request.pid_config.kd,
        .output_min = (float)request.pid_config.output_min_percent,
        .output_max = (float)request.pid_config.output_max_percent,
        .integral_min = (float)request.pid_config.integral_min,
        .integral_max = (float)request.pid_config.integral_max,
        .update_pending = true
    };

    // Apply to target motor(s)
    if (request.motor_id == -1) {
        // Apply to all motors
        shared_data_set_pid_gains(&gains);
    } else if (request.motor_id >= 0 && request.motor_id < 4) {
        // Apply to specific motor (motor_id 0-3)
        // Note: Current firmware applies to all motors via shared_data
        shared_data_set_pid_gains(&gains);
    }
} else if (err == k_rx_err_protocol_error) {
    // Malformed message - request retransmit
    rx_log_error("NANOPB", "Failed to decode PID gains request");
    comm_send_nack();
}
```

Example - All Motors Update:

```
// RPi5 sends motor_id = -1 to update all 4 motors
star_v1_SetPidGainsRequest request;
rx_err_t err = rx_nanopb_decode_pid_gains_request(buffer, len, &request);

if (err == k_rx_ok && request.motor_id == -1) {
    rx_log_info("COMM", "Applying PID gains to ALL motors");
    // shared_data_set_pid_gains() applies to all 4 PID controllers
    shared_data_set_pid_gains(&converted_gains);
}
```

See also

[rx_nanopb_encode_pid_gains_response\(\)](#) Send acknowledgment

[shared_data_set_pid_gains\(\)](#) Apply gains to motor controllers

[internal_apply_pid_updates\(\)](#) Motor task applies pending gains

[star_v1_SetPidGainsRequest_fields](#) nanopb field descriptors

NASA Power of 10 Compliance:

- Rule 5: [PASS] 3 preconditions, 2 postconditions documented
- Rule 7: [PASS] `pb_decode()` return value checked

Since

Version 1.0.0

Definition at line 1064 of file `rx_nanopb.c`.

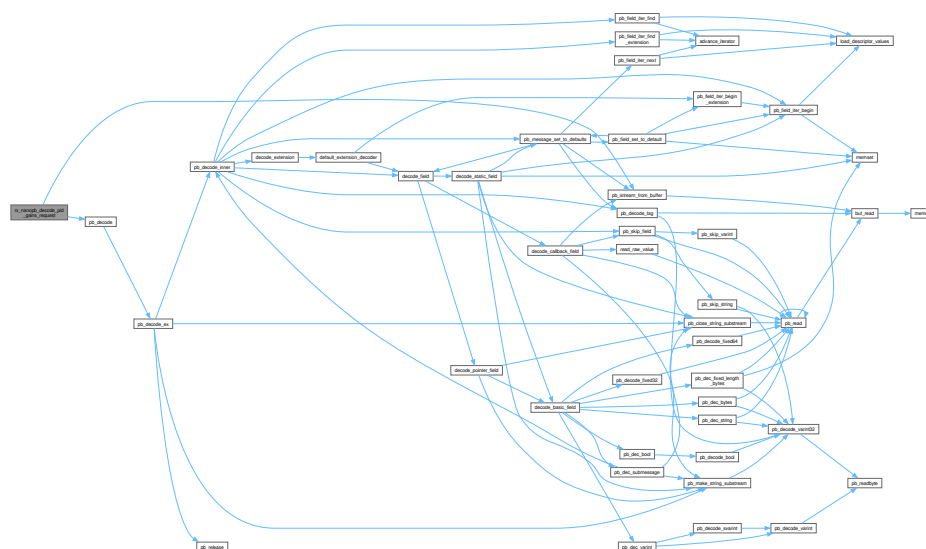
```

01067 {
01068     /* Pre-condition 1: nullptr pointer checks */
01069     if (buffer == nullptr || msg == nullptr) {
01070         return k_rx_err_invalid_arg;
01071     }
01072
01073     /* Pre-condition 2: Length validation */
01074     if (len == 0 || len > s_nanopb_buffer_size) {
01075         return k_rx_err_invalid_arg;
01076     }
01077
01078     /* Pre-condition 3: Module initialized */
01079     if (!s_initialized) {
01080         return k_rx_err_not_initialized;
01081     }
01082
01083     /* Initialize message to default values (NASA Rule 5 - prevent stale data) */
01084     *msg = (star_v1_SetPidGainsRequest)star_v1_SetPidGainsRequest_init_zero;
01085
01086     pb_istream_t stream = pb_istream_from_buffer(buffer, len);
01087
01088     if (!pb_decode(&stream, star_v1_SetPidGainsRequest_fields, msg)) {
01089         return k_rx_err_protocol_error;
01090     }
01091
01092     return k_rx_ok;
01093 }

```

References `pb_decode()`, `pb_istream_from_buffer()`, `s_initialized`, `s_nanopb_buffer_size`, `star_v1_SetPidGainsRequest` and `star_v1_SetPidGainsRequest_init_zero`.

Here is the call graph for this function:



6.85.2.6 rx_nanopb_decode_retransmit_config_request()

```
rx_err_t rx_nanopb_decode_retransmit_config_request (
    const uint8_t * buffer,
    uint32_t len,
    star_v1_SetRetransmitConfigRequest * msg) [nodiscard]
```

Decode SetRetransmitConfigRequest from Protocol Buffer bytes.

Decodes a serialized SetRetransmitConfigRequest message from a byte buffer. Used to process RPi5 commands that configure SPI retransmission parameters.

Parameters

in	buffer	Raw protobuf bytes to decode - Valid range: Non-nullptr to protobuf-encoded data - Max size: s_nanopb_buffer_size (1024 bytes)
in	len	Length of buffer in bytes Valid range: [1, s_nanopb_buffer_size]
out	msg	Decoded message output - Valid range: Non-nullptr to star_v1_SetRetransmitConfigRequest - Output: Zero-initialized then populated from buffer

Returns

rx_err_t Error code

Return values

k_rx_ok	Decode successful
k_rx_err_invalid_arg	nullptr or invalid length
k_rx_err_not_initialized	Module not initialized
k_rx_err_protocol_error	Protobuf decode failed

Precondition

buffer and msg must be non-NULL
[rx_nanopb_init\(\)](#) must have been called

Postcondition

msg populated with decoded fields on success

6.85.2.7 rx_nanopb_decode_velocity_request()

```
rx_err_t rx_nanopb_decode_velocity_request (
    const uint8_t * buffer,
    const uint32_t len,
    star_v1_SetVelocityRequest * msg) [nodiscard]
```

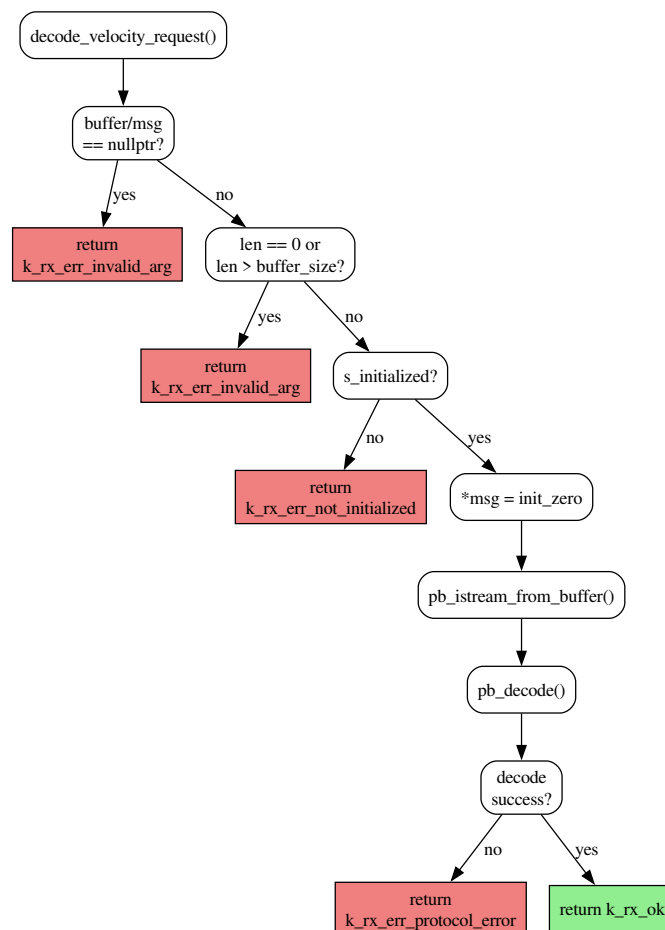
Decode SetVelocityRequest message from Protocol Buffer wire format.

Decode SetVelocityRequest message from Protocol Buffer bytes.

Deserializes a SetVelocityRequest message received from RPi5 over SPI. This is the primary command message for motor velocity control in the STAR system.

Algorithm Steps

1. Validate buffer pointer (not nullptr)
2. Validate msg pointer (not nullptr)
3. Validate len (not zero, not too large)
4. Verify module is initialized
5. Initialize msg to zero (clear previous state)
6. Create nanopb input stream from buffer
7. Decode message using generated field descriptors
8. Return decoded message in *msg



Precondition

`rx_nanopb_init()` must have been called successfully
buffer must contain valid Protocol Buffer wire-format data
len must match actual encoded message size

Postcondition

msg fully populated with decoded field values
Unset fields have default/zero values

Note

Not thread-safe - protect with mutex if called from multiple tasks

Performance

- Execution time: ~15 us @ 240 MHz
- Stack usage: ~96 bytes

Example - Basic Usage:

```
// Receive encoded message over SPI
uint8_t rx_buffer[512];
uint32_t rx_len;
spi_receive(rx_buffer, sizeof(rx_buffer), &rx_len);

// Decode the message
star_v1_SetVelocityRequest request;
rx_err_t err = rx_nanopb_decode_velocity_request(rx_buffer, rx_len, &request);

if (err == k_rx_ok) {
    // Apply velocities to motors
    motor_set_velocity(MOTOR_FL, request.command.front_left_velocity_mps);
    motor_set_velocity(MOTOR_FR, request.command.front_right_velocity_mps);
    motor_set_velocity(MOTOR_BL, request.command.back_left_velocity_mps);
    motor_set_velocity(MOTOR_BR, request.command.back_right_velocity_mps);
} else if (err == k_rx_err_protocol_error) {
    // Request retransmit
    comm_send_nack();
}
```

See also

`rx_nanopb_encode_velocity_request()` Encode counterpart
`rx_nanopb_encode_velocity_response()` Send response after decoding

NASA Power of 10 Compliance

- Rule 5: [OK] 3 pre-conditions
- Rule 7: [OK] `pb_decode()` return value checked

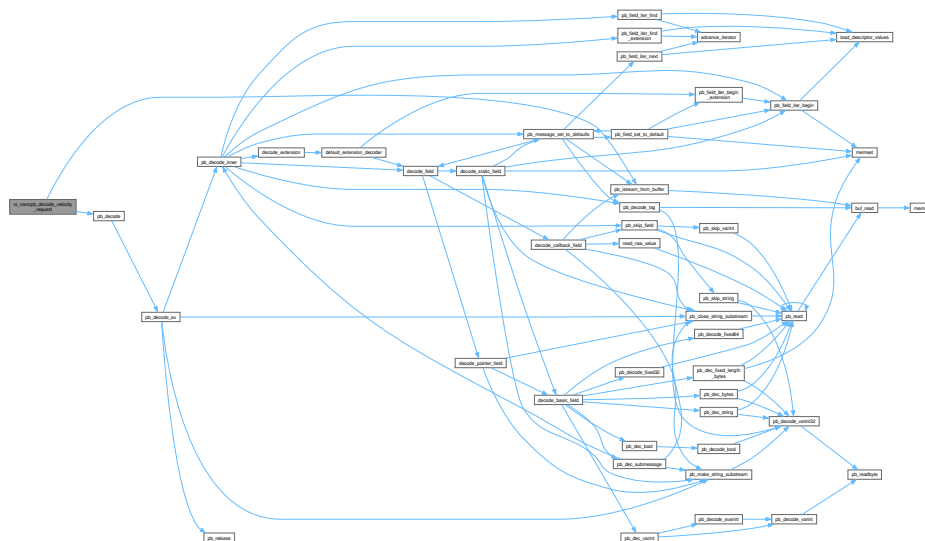
Version 1.0.0

```

00677 {
00678     /* Pre-condition 1: nullptr pointer checks */
00679     if (buffer == nullptr || msg == nullptr) {
00680         return k_rx_err_invalid_arg;
00681     }
00682
00683     /* Pre-condition 2: Length validation */
00684     if (len == 0 || len > s_nanopb_buffer_size) {
00685         return k_rx_err_invalid_arg;
00686     }
00687
00688     /* Pre-condition 3: Module initialized */
00689     if (!s_initialized) {
00690         return k_rx_err_not_initialized;
00691     }
00692
00693     /* Initialize message to default values */
00694     *msg = (star_v1_SetVelocityRequest)star_v1_SetVelocityRequest_init_zero;
00695
00696     pb_istream_t stream = pb_istream_from_buffer(buffer, len);
00697
00698     if (!pb_decode(&stream, star_v1_SetVelocityRequest_fields, msg)) {
00699         return k_rx_err_protocol_error;
00700     }
00701
00702     return k_rx_ok;
00703 }

```

Here is the call graph for this function:



```
rx_err_t rx_nanopb_encode_estop_response (
    const star_v1_EmergencyStopResponse * msg,
    uint8_t * buffer,
    const uint32_t buffer_size,
    uint32_t * len) [nodiscard]
```

Serializes an EmergencyStopResponse message to confirm E-Stop processing. This response informs RPi5 that the RX72N has received the E-Stop command and has entered safe state (motors stopped).

6.85.2.9 rx_nanopb_encode_pid_gains_response()

```
rx_err_t rx_nanopb_encode_pid_gains_response (
    const star\_v1\_SetPidGainsResponse * msg,
    uint8\_t * buffer,
    const uint32\_t buffer_size,
    uint32\_t * len) [nodiscard]
```

Encode SetPidGainsResponse message to Protocol Buffer bytes.

Serializes a SetPidGainsResponse message as acknowledgment to a PID gains update command. Sent back to RPi5 after processing a SetPidGainsRequest.

The optional pb_callback_t message field is skipped when its encode callback is NULL (init_zero default). The success boolean and response header status are sufficient for programmatic acknowledgment.

Precondition

[rx_nanopb_init\(\)](#) must be called before this function
buffer must point to at least buffer_size bytes of writable memory

Postcondition

On k_rx_ok: buffer[0..*len-1] contains valid wire-format response
On k_rx_ok: *len <= s_nanopb_buffer_size

Note

Not thread-safe; call only from Communication Task context

Performance: ~15 us @ 240 MHz

See also

[rx_nanopb_decode_pid_gains_request\(\)](#) Decode the incoming request
[rx_nanopb_create_response_header\(\)](#) Create header with status and request_id

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: [PASS] 3 preconditions, 2 postconditions documented
- Rule 7: [PASS] [pb_encode\(\)](#) return value checked

Definition at line 1125 of file [rx_nanopb.c](#).

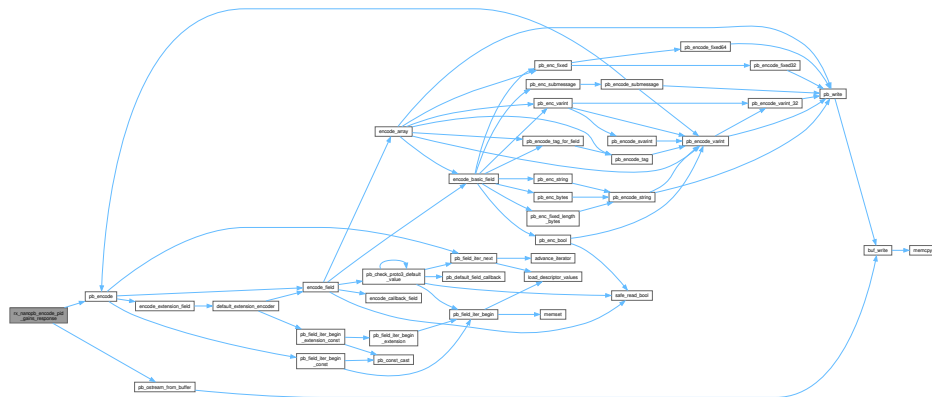
```

01129 {
01130  /* Pre-condition 1: nullptr pointer checks */
01131  if (msg == nullptr || buffer == nullptr || len == nullptr) {
01132    return k_rx_err_invalid_arg;
01133  }
01134
01135  /* Pre-condition 2: Module initialized */
01136  if (!s_initialized) {
01137    return k_rx_err_not_initialized;
01138  }
01139
01140  /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
01141  if (buffer_size < s_nanopb_buffer_size) {
01142    return k_rx_err_invalid_size;
01143  }
01144
01145  pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
01146
01147  /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
01148  (void)pb_encode(&stream, star_v1_SetPidGainsResponse_fields, msg);
01149  *len = stream.bytes_written;
01150
01151  return k_rx_ok;
01152 }

```

References [pb_encode\(\)](#), [pb_ostream_from_buffer\(\)](#), [s_initialized](#), [s_nanopb_buffer_size](#), and [star_v1_SetPidGainsResponse_fields](#).

Here is the call graph for this function:



6.85.2.10 rx_nanopb_encode_telemetry()

```

rx_err_t rx_nanopb_encode_telemetry (
    const star_v1_TelemetryData * msg,
    uint8_t * buffer,
    const uint32_t buffer_size,
    uint32_t * len) [nodiscard]

```

Encode TelemetryData message to Protocol Buffer wire format.

Encode TelemetryData message to Protocol Buffer bytes.

Serializes telemetry data for periodic transmission to RPi5. Contains motor states, encoder readings, and sensor data. Typically called at 10-100 Hz depending on telemetry requirements.

Telemetry Contents

Field Category	Fields	Update Rate
Motor velocities	4 x double (m/s)	100 Hz
Motor currents	4 x double (A)	100 Hz
Encoder positions	4 x int32 (ticks)	100 Hz
Temperature	system temp (degC)	1 Hz

Precondition

[rx_nanopb_init\(\)](#) must have been called
 msg should contain current sensor/motor data

Postcondition

buffer contains valid Protocol Buffer wire-format telemetry

Note

Not thread-safe - protect with mutex if called from multiple tasks

Performance

- Execution time: ~30 us @ 240 MHz
- Stack usage: ~128 bytes

Example - Periodic Telemetry:

```
void telemetry_task(ULONG arg) {
    while (1) {
        // Gather telemetry data
        star_v1_TelemetryData telemetry = {};
        telemetry.motor_front_left_velocity_mps = motor_get_velocity(0);
        telemetry.motor_front_left_current_a = motor_get_current(0);
        // ... populate all fields
        telemetry.timestamp_us = rx_time_get_us();

        // Encode and send
        uint8_t buffer[512];
        uint32_t len;
        rx_err_t err = rx_nanopb_encode_telemetry(&telemetry, buffer, sizeof(buffer), &len);
        if (err == k_rx_ok) {
            spi_send(buffer, len);
        }

        // Sleep until next telemetry period (10ms = 100Hz)
        tx_thread_sleep(TX_TIMER_TICKS_PER_SECOND / 100);
    }
}
```

See also

[star_v1_TelemetryData](#) Generated telemetry message structure

NASA Power of 10 Compliance

- Rule 5: [OK] 3 pre-conditions, 1 post-condition

6.85.2.11 rx_nanopb_encode_velocity_request()

```
rx_err_t rx_nanopb_encode_velocity_request (
    const star_v1_SetVelocityRequest * msg,
    uint8_t * buffer,
    const uint32_t buffer_size,
    uint32_t * len) [nodiscard]
```

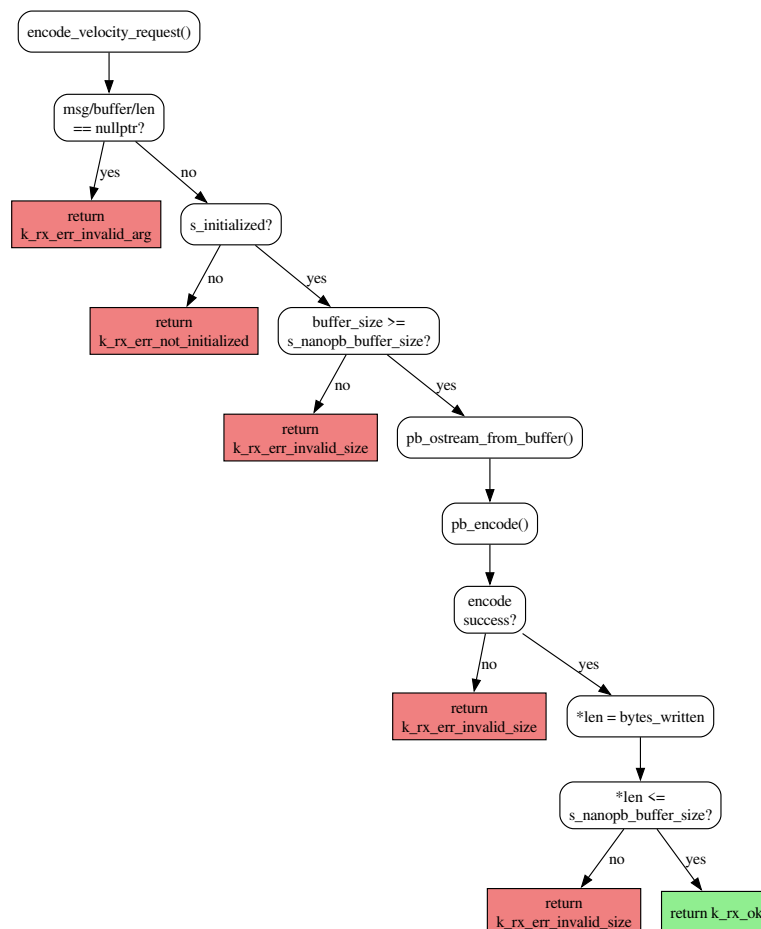
Encode SetVelocityRequest message to Protocol Buffer wire format.

Encode SetVelocityRequest message to Protocol Buffer bytes.

Serializes a SetVelocityRequest message containing motor velocity commands into Protocol Buffer binary format for transmission over SPI or USB.

Algorithm Steps

1. Validate msg pointer (not nullptr)
2. Validate buffer pointer (not nullptr)
3. Validate len pointer (not nullptr)
4. Verify module is initialized
5. Verify buffer size is adequate
6. Create nanopb output stream from buffer
7. Encode message using generated field descriptors
8. Return encoded length in *len
9. Verify encoded length is within bounds (post-condition)



Precondition

`rx_nanopb_init()` must have been called successfully
 msg must be fully populated with valid data
`buffer_size` $\geq$ `s_nanopb_buffer_size` (512)

Postcondition

`buffer[0..*len-1]` contains valid Protocol Buffer wire-format data
`*len` $\leq$ `s_nanopb_buffer_size`

Note

Not thread-safe - protect with mutex if called from multiple tasks

Performance

- Execution time: ~ 20 us @ 240 MHz
- Stack usage: ~ 96 bytes

Example - Basic Usage:

```
star_v1_SetVelocityRequest request = star_v1_SetVelocityRequest_init_zero;

// Populate command
request.command.front_left_velocity_mps = 1.0;
request.command.front_right_velocity_mps = 1.0;
request.command.back_left_velocity_mps = 1.0;
request.command.back_right_velocity_mps = 1.0;
request.command.sequence = g_sequence++;

uint8_t buffer[512];
uint32_t encoded_len;

rx_err_t err = rx_nanopb_encode_velocity_request(&request, buffer, sizeof(buffer), &encoded_len);
if (err == k_rx_ok) {
    spi_send(buffer, encoded_len);
}
```

See also

[rx_nanopb_decode_velocity_request\(\)](#) Decode counterpart
[star_v1_SetVelocityRequest_fields](#) nanopb field descriptors

NASA Power of 10 Compliance

- Rule 5: [OK] 3 pre-conditions, 1 post-condition
- Rule 7: [OK] [pb_encode\(\)](#) return value checked

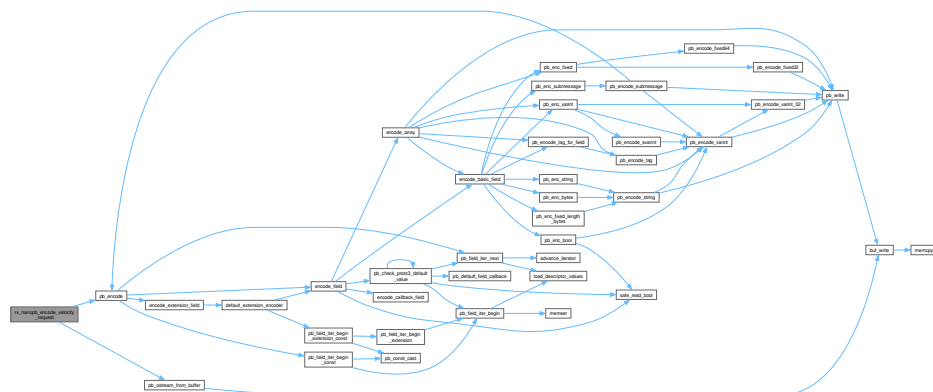
Version 1.0.0

```

00550 {
00551     /* Pre-condition 1: nullptr pointer checks */
00552     if (msg == nullptr || buffer == nullptr || len == nullptr) {
00553         return k_rx_err_invalid_arg;
00554     }
00555
00556     /* Pre-condition 2: Module initialized */
00557     if (!s_initialized) {
00558         return k_rx_err_not_initialized;
00559     }
00560
00561     /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
00562     if (buffer_size < s_nanopb_buffer_size) {
00563         return k_rx_err_invalid_size;
00564     }
00565
00566     pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
00567
00568     /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
00569     (void)pb_encode(&stream, star_v1_SetVelocityRequest_fields, msg);
00570     *len = stream.bytes_written;
00571
00572     return k_rx_ok;
00573 }

```

Here is the call graph for this function:



```
rx_err_t rx_nanopb_encode_velocity_response (
    const star_v1_SetVelocityResponse * msg,
    uint8_t * buffer,
    const uint32_t buffer_size,
    uint32_t * len) [nodiscard]
```

Encode SetVelocityResponse message to Protocol Buffer bytes.

Generated by Doxygen

Algorithm Steps

1. Validate msg pointer (not nullptr)
2. Validate buffer pointer (not nullptr)
3. Validate len pointer (not nullptr)
4. Verify module is initialized
5. Verify buffer size is adequate
6. Create nanopb output stream from buffer
7. Encode message using generated field descriptors
8. Return encoded length in *len
9. Verify encoded length is within bounds (post-condition)

Precondition

`rx_nanopb_init()` must have been called successfully
 msg should be populated via `rx_nanopb_create_response_header()`
`buffer_size >= s_nanopb_buffer_size (512)`

Postcondition

`buffer[0..*len-1]` contains valid Protocol Buffer wire-format data
`*len <= s_nanopb_buffer_size`

Note

Not thread-safe - protect with mutex if called from multiple tasks

Performance

- Execution time: ~15 us @ 240 MHz
- Stack usage: ~48 bytes

Example - Responding to Velocity Command:

```
// After successfully processing velocity command
star_v1_SetVelocityResponse response = star_v1_SetVelocityResponse_init_zero;

// Create header with success status
rx_nanopb_create_response_header(&response.header,
                                star_v1_Status_STATUS_OK,
                                request.header.request_id);

// Encode and send
uint8_t buffer[512];
uint32_t len;
rx_err_t err = rx_nanopb_encode_velocity_response(&response, buffer, sizeof(buffer), &len);
if (err == k_rx_ok) {
    spi_send(buffer, len);
}
```

See also

`rx_nanopb_decode_velocity_request()` Decode incoming request first
`rx_nanopb_create_response_header()` Create header with status

NASA Power of 10 Compliance

- Rule 5: [OK] 3 pre-conditions, 1 post-condition
- Rule 7: [OK] `pb_encode()` return value checked

Since

Version 1.0.0

Definition at line 772 of file `rx_nanopb.c`.

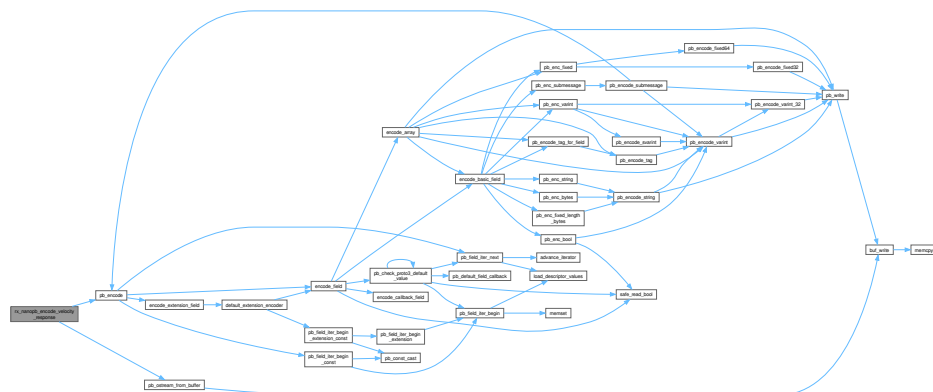
```

00776 {
00777     /* Pre-condition 1: nullptr pointer checks */
00778     if (msg == nullptr || buffer == nullptr || len == nullptr) {
00779         return k_rx_err_invalid_arg;
00780     }
00781
00782     /* Pre-condition 2: Module initialized */
00783     if (!s_initialized) {
00784         return k_rx_err_not_initialized;
00785     }
00786
00787     /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
00788     if (buffer_size < s_nanopb_buffer_size) {
00789         return k_rx_err_invalid_size;
00790     }
00791
00792     pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
00793
00794     /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
00795     (void)pb_encode(&stream, star_v1_SetVelocityResponse_fields, msg);
00796     *len = stream.bytes_written;
00797
00798     return k_rx_ok;
00799 }

```

References `pb_encode()`, `pb_ostream_from_buffer()`, `s_initialized`, `s_nanopb_buffer_size`, and `star_v1_SetVelocityResponse_fields`.

Here is the call graph for this function:



6.85.2.13 rx_nanopb_init()

```

rx_err_t rx_nanopb_init (
    void ) [nodiscard]

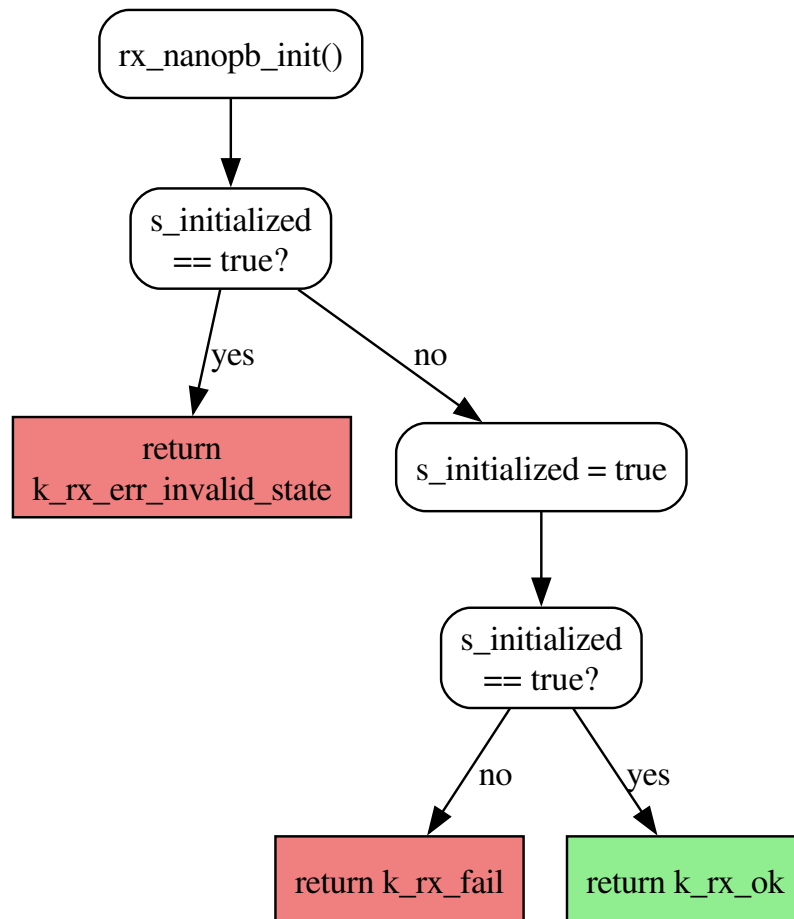
```

Initialize nanopb wrapper module.

Initializes the nanopb wrapper module state and prepares it for encode/decode operations. This function must be called once during system startup before any message encoding or decoding.

Algorithm Steps

1. Check if module is already initialized (pre-condition)
2. Set initialization flag to true
3. Verify flag was set correctly (post-condition)
4. Return success



Precondition

- Module must not be already initialized
- System startup must be in progress (single-threaded context)

Postcondition

- `s_initialized == true`
- All encode/decode functions can be called

Invariant

- After successful init, `s_initialized` remains true until reset

Note

Not thread-safe - call from main thread during startup
Idempotent after first call (returns error, no side effects)

Performance: ~1 us @ 240 MHz

Stack Usage: 8 bytes

Example - System Initialization:

```
void app_main_init(void) {
    rx_err_t err;

    // Initialize logging first
    err = rx_log_init();
    RX_ASSERT(err == k_rx_ok);

    // Initialize nanopb wrapper
    err = rx_nanopb_init();
    if (err != k_rx_ok) {
        rx_log_error("NANOPB", "Init failed: %d", err);
        // Handle critical error - cannot communicate
        system_halt();
    }

    // Now safe to use encode/decode functions
    rx_log_info("NANOPB", "Module initialized");
}
```

Example - Error Handling:

```
rx_err_t err = rx_nanopb_init();
switch (err) {
    case k_rx_ok:
        // Success - proceed normally
        break;
    case k_rx_err_invalid_state:
        // Already initialized - may be OK depending on context
        rx_log_warn("NANOPB", "Already initialized");
        break;
    default:
        // Unexpected error - critical failure
        rx_log_error("NANOPB", "Init error: %d", err);
        return err;
}
```

See also

[rx_nanopb_test_reset_state\(\)](#) Reset for unit testing
[s_initialized](#) Module state flag

NASA Power of 10 Compliance

- Rule 5: [OK] 1 pre-condition check, 1 post-condition verification
- Rule 7: [OK] Return value indicates success/failure

Since

Version 1.0.0

Definition at line 357 of file [rx_nanopb.c](#).

```
00358 {
00359  /* Pre-condition: Check not already initialized */
00360  if (s_initialized) {
00361      return k_rx_err_invalid_state;
00362  }
00363
00364  s_initialized = true;
00365
00366  /* Post-condition: Verify initialization succeeded */
00367
00368  return k_rx_ok;
00369 }
```

References [s_initialized](#).

6.85.2.14 rx_nanopb_test_reset_state()

```
void rx_nanopb_test_reset_state (
    void )
```

Reset module state for unit testing.

Resets the nanopb wrapper module to its uninitialized state. This function is intended exclusively for unit testing to ensure clean test isolation.

Algorithm Steps

1. Set s_initialized to false

Warning

FOR UNIT TESTING ONLY - Do not call in production code. Calling this while encode/decode operations are in progress will cause undefined behavior.

Precondition

None

Postcondition

s_initialized == false

[rx_nanopb_init\(\)](#) must be called before encode/decode functions

Note

Not thread-safe

No return value - always succeeds

Performance: ~100 ns @ 240 MHz

Stack Usage: 0 bytes (inline capable)

Example - Test Fixture Setup:

```
// Unity test framework setup
void setUp(void) {
    // Reset to clean state before each test
    rx_nanopb_test_reset_state();

    // Re-initialize for tests that need it
    rx_err_t err = rx_nanopb_init();
    TEST_ASSERT_EQUAL(k_rx_ok, err);
}

void tearDown(void) {
    // Clean up after each test
    rx_nanopb_test_reset_state();
}
```

Example - Testing Uninitialized Behavior:

```
void test_encode_fails_when_not_initialized(void) {
    // Ensure module is NOT initialized
    rx_nanopb_test_reset_state();

    star_v1_SetVelocityRequest msg = {};
    uint8_t buffer[512];
    uint32_t len;

    rx_err_t err = rx_nanopb_encode_velocity_request(&msg, buffer, 512, &len);

    // Should fail with not initialized error
    TEST_ASSERT_EQUAL(k_rx_err_not_initialized, err);
}
```

See also

[rx_nanopb_init\(\)](#) Initialize module after reset
[s_initialized](#) Module state flag

Since

Version 1.0.0

Definition at line 436 of file [rx_nanopb.c](#).

```
00437 {
00438     s_initialized = false;
00439 }
```

References [s_initialized](#).

6.85.3 Variable Documentation

6.85.3.1 k_velocity_mps_max

```
const double k_velocity_mps_max = 1000.0 [static]
```

Maximum valid motor velocity in meters per second.

Upper bound for velocity command validation. Symmetric with k_velocity_mps_min to allow full bidirectional motor control.

Rationale

- Physical robot max: ~ 2.5 m/s
- Buffer for future high-speed configurations
- Catches NaN/Inf or garbage values

Value: +1000.0 m/s

Note

Memory: 8 bytes in .rodata section (const)

See also

[k_velocity_mps_min](#) Lower bound

[rx_nanopb_create_velocity_command\(\)](#) Uses for validation

Since

Version 1.0.0

Definition at line 250 of file [rx_nanopb.c](#).

Referenced by [rx_nanopb_create_velocity_command\(\)](#).

6.85.3.2 k_velocity_mps_min

```
const double k_velocity_mps_min = -1000.0 [static]
```

Minimum valid motor velocity in meters per second.

Lower bound for velocity command validation. This generous limit (-1000 m/s) allows for a wide range of robot configurations while preventing obviously invalid values from propagating to motor control.

Rationale

- Physical robot max: ~2.5 m/s
- Buffer for future high-speed configurations
- Catches NaN/Inf or garbage values

Value: -1000.0 m/s

Note

Memory: 8 bytes in .rodata section (const)

See also

[k_velocity_mps_max](#) Upper bound

[rx_nanopb_create_velocity_command\(\)](#) Uses for validation

Since

Version 1.0.0

Definition at line 226 of file [rx_nanopb.c](#).

Referenced by [rx_nanopb_create_velocity_command\(\)](#).

6.85.3.3 s_initialized

```
bool s_initialized = false [static]
```

Module initialization state flag.

Tracks whether [rx_nanopb_init\(\)](#) has been called successfully. All encode and decode functions check this flag and return `k_rx_err_not_initialized` if the module has not been properly initialized.

State Transitions

Current	Operation	Next
false	rx_nanopb_init() success	true
true	rx_nanopb_init() called	unchanged (error)
true	rx_nanopb_test_reset_state()	false

Warning

Do not modify directly - use [rx_nanopb_init\(\)](#) and [rx_nanopb_test_reset_state\(\)](#)

Note

Memory: 1 byte in .bss section (zero-initialized)

Since

Version 1.0.0

Definition at line 201 of file [rx_nanopb.c](#).

Referenced by [rx_nanopb_decode_estop_request\(\)](#), [rx_nanopb_decode_pid_gains_request\(\)](#), [rx_nanopb_decode_retransmit_config_request\(\)](#), [rx_nanopb_decode_velocity_request\(\)](#), [rx_nanopb_encode_estop_request\(\)](#), [rx_nanopb_encode_pid_gains_response\(\)](#), [rx_nanopb_encode_telemetry\(\)](#), [rx_nanopb_encode_velocity_request\(\)](#), [rx_nanopb_encode_velocity_response\(\)](#), [rx_nanopb_init\(\)](#), and [rx_nanopb_test_reset_state\(\)](#).

6.86 rx_nanopb.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_nanopb.c
00003  * @brief nanopb Integration Wrapper Implementation for RX72N
00004  *
00005  * @details
00006  * Implements the Protocol Buffer encode/decode wrapper functions for the
00007  * STAR project's motor control and telemetry messaging system. This module
00008  * provides a safety-critical interface between the nanopb library and the
00009  * RX72N firmware application layer.
00010  *
00011  * @par Implementation Architecture
00012  * @dot
00013  * digraph nanopb_impl {
00014  *   rankdir=TB;
00015  *   node [shape=box, style=rounded];
00016  *   edge [fontsize=10];
00017  *
00018  *   subgraph cluster_public {
00019  *     label="Public API";
00020  *     style=filled;
00021  *     color=lightgreen;
00022  *     init [label="rx_nanopb_init()"];
00023  *     encode_vel [label="rx_nanopb_encode_velocity_*()"];
00024  *     decode_vel [label="rx_nanopb_decode_velocity_*()"];
00025  *     encode_estop [label="rx_nanopb_encode_estop_*()"];
00026  *     decode_estop [label="rx_nanopb_decode_estop_*()"];
00027  *     encode_telem [label="rx_nanopb_encode_telemetry()"];
00028  *     helpers [label="Helper Functions"];
00029  *   }
00030  *
00031  *   subgraph cluster_internal {
00032  *     label="Internal Functions";
00033  *     style=filled;
00034  *     color=lightyellow;

```

```

00035 *   str_cb [label="internal_encode_string_callback()"];
00036 * }
00037 *
00038 * subgraph cluster_state {
00039 *   label="Module State";
00040 *   style=filled;
00041 *   color=lightcoral;
00042 *   s_init [label="s_initialized\n(bool)"];
00043 *   vel_limits [label="k_velocity_mps_min/max\n(double)"];
00044 * }
00045 *
00046 * subgraph cluster_nanopb {
00047 *   label="nanopb Library";
00048 *   style=filled;
00049 *   color=lightblue;
00050 *   pb_encode [label="pb_encode()"];
00051 *   pb_decode [label="pb_decode()"];
00052 *   pb_ostream [label="pb_ostream_from_buffer()"];
00053 *   pb_istream [label="pb_istream_from_buffer()"];
00054 * }
00055 *
00056 * init -> s_init [label="set true"];
00057 * encode_vel -> s_init [label="check"];
00058 * encode_vel -> pb_ostream;
00059 * encode_vel -> pb_encode;
00060 * decode_vel -> s_init [label="check"];
00061 * decode_vel -> pb_istream;
00062 * decode_vel -> pb_decode;
00063 * helpers -> vel_limits [label="validate"];
00064 * encode_vel -> str_cb [style=dashed, label="callback"];
00065 * }
00066 * @enddot
00067 *
00068 * @par Encode/Decode Flow
00069 * @msc
00070 * Caller, API, nanopb, Buffer;
00071 *
00072 * ... [label="Encode Flow"];
00073 * Caller -> API [label="encode_velocity_request(msg, buf, size, &len)"];
00074 * API box API [label="1. Validate params\n2. Check initialized"];
00075 * API -> nanopb [label="pb_ostream_from_buffer()"];
00076 * nanopb -> Buffer [label="Create stream"];
00077 * API -> nanopb [label="pb_encode(fields, msg)"];
00078 * nanopb box nanopb [label="Serialize fields"];
00079 * nanopb -> Buffer [label="Write bytes"];
00080 * nanopb -> API [label="true/false"];
00081 * API -> Caller [label="k_rx_ok / error"];
00082 *
00083 * ... [label="Decode Flow"];
00084 * Caller -> API [label="decode_velocity_request(buf, len, &msg)"];
00085 * API box API [label="1. Validate params\n2. Check initialized"];
00086 * API -> API [label="Initialize msg to zero"];
00087 * API -> nanopb [label="pb_istream_from_buffer()"];
00088 * nanopb -> Buffer [label="Create stream"];
00089 * API -> nanopb [label="pb_decode(fields, &msg)"];
00090 * nanopb box nanopb [label="Parse fields"];
00091 * nanopb -> API [label="true/false"];
00092 * API -> Caller [label="k_rx_ok / error"];
00093 * @endmsc
00094 *
00095 * @par Memory Layout
00096 * | Variable | Type | Size | Description |
00097 * |-----|-----|-----|-----|
00098 * | s_initialized | bool | 1 byte | Module state flag |
00099 * | k_velocity_mps_min | const double | 8 bytes | Min velocity bound |
00100 * | k_velocity_mps_max | const double | 8 bytes | Max velocity bound |
00101 * | **Total Static** | | **17 bytes** | Minimal footprint |
00102 *
00103 * @par Performance Characteristics
00104 * | Function | Time @ 240 MHz | Stack Usage | Notes |
00105 * |-----|-----|-----|-----|
00106 * | rx_nanopb_init() | ~1 us | 8 bytes | One-time setup |
00107 * | encode_velocity_request | ~20 us | ~96 bytes | 4 doubles + overhead |
00108 * | decode_velocity_request | ~15 us | ~96 bytes | 4 doubles + overhead |
00109 * | encode_velocity_response | ~15 us | ~48 bytes | Header only |
00110 * | encode_telemetry | ~30 us | ~128 bytes | Full telemetry |
00111 * | decode_estop_request | ~10 us | ~32 bytes | Minimal message |
00112 * | create_velocity_command | ~2 us | ~16 bytes | Copy operations |
00113 *
00114 * @par Error Handling Strategy
00115 * 1. **Parameter Validation**: nullptr checks, size bounds (NASA Rule 5)
00116 * 2. **State Validation**: Module must be initialized
00117 * 3. **nanopb Errors**: Translate pb_encode/pb_decode failures
00118 * 4. **Post-conditions**: Verify output bounds
00119 *
00120 * @par Thread Safety Analysis
00121 * | Function | Thread Safe | Notes |

```

```

00122 * |-----|-----|-----|
00123 * | rx_nanopb_init() | No | Call once at startup |
00124 * | encode_* functions | No | Protect with mutex |
00125 * | decode_* functions | No | Protect with mutex |
00126 * | create_* helpers | Yes | No shared state |
00127 * | test_reset_state | No | Testing only |
00128 *
00129 * @par NASA Power of 10 Compliance
00130 * - Rule 1: [OK] No goto, setjmp, longjmp, or recursion
00131 * - Rule 2: [OK] All loops bounded by nanopb field counts (internal)
00132 * - Rule 3: [OK] No dynamic memory allocation
00133 * - Rule 4: [OK] All functions under 60 lines
00134 * - Rule 5: [OK] Input validation with minimum 2 checks per function
00135 * - Rule 6: [OK] Variables declared at smallest scope
00136 * - Rule 7: [OK] All nanopb return values checked
00137 * - Rule 8: [OK] Minimal preprocessor usage
00138 * - Rule 9: [OK] Function pointer only for nanopb callback
00139 * - Rule 10: [OK] Compiles with -Wall -Wextra -Werror
00140 *
00141 * @par SOLID Principles Compliance
00142 * - **S (Single Responsibility)**: Only protobuf encode/decode operations
00143 * - **O (Open/Closed)**: Extensible by adding new message functions
00144 * - **L (Liskov Substitution)**: N/A (no inheritance)
00145 * - **I (Interface Segregation)**: Separate function per message type
00146 * - **D (Dependency Inversion)**: Depends on abstract pb_fields_t
00147 *
00148 * @see rx_nanopb.h Public API header with message documentation
00149 * @see gen/star/v1/ Generated protobuf message definitions (.pb.h files)
00150 * @see star-proto/proto/star/v1/ Protocol Buffer schema files
00151 *
00152 * @author Locked, Inc.
00153 * @date 2026-01-27
00154 * @version 1.0.0
00155 * @copyright Copyright (c) 2026 Locked Inc.
00156 * SPDX-License-Identifier: MIT
00157 */
00158
00159 #include "rx_nanopb.h"
00160
00161 #include <pb_decode.h>
00162 #include <pb_encode.h>
00163 #include <string.h>
00164
00165 #include "gen/star/v1/wire.pb.h"
00166 #include "rx_check.h"
00167
00168 /* nanopb's pb_ostream_from_buffer() and pb_istream_from_buffer() return structs
00169  * by value, which triggers -Waggregate-return. This is the standard nanopb API
00170  * and cannot be changed, so suppress the warning for this file. */
00171 #pragma GCC diagnostic ignored "-Waggregate-return"
00172
00173 /*
=====
00174 * Module State
00175 *
=====
00176 */
00177 /**
00178  * @var s_initialized
00179  * @brief Module initialization state flag
00180  *
00181  * @details
00182  * Tracks whether rx_nanopb_init() has been called successfully. All encode
00183  * and decode functions check this flag and return k_rx_err_not_initialized
00184  * if the module has not been properly initialized.
00185  *
00186  * @par State Transitions
00187  * | Current | Operation | Next |
00188  * |-----|-----|-----|
00189  * | false | rx_nanopb_init() success | true |
00190  * | true | rx_nanopb_init() called | unchanged (error) |
00191  * | true | rx_nanopb_test_reset_state() | false |
00192  *
00193  * @warning Do not modify directly - use rx_nanopb_init() and
00194  * rx_nanopb_test_reset_state()
00195  *
00196  * @note Memory: 1 byte in .bss section (zero-initialized)
00197  *
00198  * @since Version 1.0.0
00199  */
00200 */
00201 static bool s_initialized = false;
00202
00203 /**
00204  * @var k_velocity_mps_min
00205  * @brief Minimum valid motor velocity in meters per second
00206  */

```



```

00207 * @details
00208 * Lower bound for velocity command validation. This generous limit
00209 * (-1000 m/s) allows for a wide range of robot configurations while
00210 * preventing obviously invalid values from propagating to motor control.
00211 *
00212 * @par Rationale
00213 * - Physical robot max: ~2.5 m/s
00214 * - Buffer for future high-speed configurations
00215 * - Catches NaN/Inf or garbage values
00216 *
00217 * @par Value: -1000.0 m/s
00218 *
00219 * @note Memory: 8 bytes in .rodata section (const)
00220 *
00221 * @see k_velocity_mps_max Upper bound
00222 * @see rx_nanopb_create_velocity_command() Uses for validation
00223 *
00224 * @since Version 1.0.0
00225 */
00226 static const double k_velocity_mps_min = -1000.0;
00227
00228 /**
00229 * @var k_velocity_mps_max
00230 * @brief Maximum valid motor velocity in meters per second
00231 *
00232 * @details
00233 * Upper bound for velocity command validation. Symmetric with
00234 * k_velocity_mps_min to allow full bidirectional motor control.
00235 *
00236 * @par Rationale
00237 * - Physical robot max: ~2.5 m/s
00238 * - Buffer for future high-speed configurations
00239 * - Catches NaN/Inf or garbage values
00240 *
00241 * @par Value: +1000.0 m/s
00242 *
00243 * @note Memory: 8 bytes in .rodata section (const)
00244 *
00245 * @see k_velocity_mps_min Lower bound
00246 * @see rx_nanopb_create_velocity_command() Uses for validation
00247 *
00248 * @since Version 1.0.0
00249 */
00250 static const double k_velocity_mps_max = 1000.0;
00251
00252 /*
=====
00253 * Initialization
00254 *
=====
00255 */
00256
00257 /**
00258 * @brief Initialize nanopb wrapper module
00259 *
00260 * @details
00261 * Initializes the nanopb wrapper module state and prepares it for
00262 * encode/decode operations. This function must be called once during
00263 * system startup before any message encoding or decoding.
00264 *
00265 * @par Algorithm Steps
00266 * 1. Check if module is already initialized (pre-condition)
00267 * 2. Set initialization flag to true
00268 * 3. Verify flag was set correctly (post-condition)
00269 * 4. Return success
00270 *
00271 * @dot
00272 * digraph init_flow {
00273 *   rankdir=TB;
00274 *   node [shape=box, style=rounded];
00275 *
00276 *   start [label="rx_nanopb_init()"];
00277 *   check_init [label="s_initialized\n== true?"];
00278 *   err_state [label="return\nk_rx_err_invalid_state", fillcolor=lightcoral, style=filled];
00279 *   set_flag [label="s_initialized = true"];
00280 *   verify [label="s_initialized\n== true?"];
00281 *   err_fail [label="return k_rx_fail", fillcolor=lightcoral, style=filled];
00282 *   success [label="return k_rx_ok", fillcolor=lightgreen, style=filled];
00283 *
00284 *   start -> check_init;
00285 *   check_init -> err_state [label="yes"];
00286 *   check_init -> set_flag [label="no"];
00287 *   set_flag -> verify;
00288 *   verify -> err_fail [label="no"];
00289 *   verify -> success [label="yes"];
00290 * }
00291 * @enddot

```

```

00292 *
00293 *
00294 * @pre Module must not be already initialized
00295 * @pre System startup must be in progress (single-threaded context)
00296 *
00297 * @post s_initialized == true
00298 * @post All encode/decode functions can be called
00299 *
00300 * @invariant After successful init, s_initialized remains true until reset
00301 *
00302 * @note Not thread-safe - call from main thread during startup
00303 * @note Idempotent after first call (returns error, no side effects)
00304 *
00305 * @par Performance: ~1 us @ 240 MHz
00306 * @par Stack Usage: 8 bytes
00307 *
00308 * @par Example - System Initialization:
00309 * @code
00310 * void app_main_init(void) {
00311 *     rx_err_t err;
00312 *
00313 *     // Initialize logging first
00314 *     err = rx_log_init();
00315 *     RX_ASSERT(err == k_rx_ok);
00316 *
00317 *     // Initialize nanopb wrapper
00318 *     err = rx_nanopb_init();
00319 *     if (err != k_rx_ok) {
00320 *         rx_log_error("NANOPB", "Init failed: %d", err);
00321 *         // Handle critical error - cannot communicate
00322 *         system_halt();
00323 *     }
00324 *
00325 *     // Now safe to use encode/decode functions
00326 *     rx_log_info("NANOPB", "Module initialized");
00327 * }
00328 * @endcode
00329 *
00330 * @par Example - Error Handling:
00331 * @code
00332 * rx_err_t err = rx_nanopb_init();
00333 * switch (err) {
00334 *     case k_rx_ok:
00335 *         // Success - proceed normally
00336 *         break;
00337 *     case k_rx_err_invalid_state:
00338 *         // Already initialized - may be OK depending on context
00339 *         rx_log_warn("NANOPB", "Already initialized");
00340 *         break;
00341 *     default:
00342 *         // Unexpected error - critical failure
00343 *         rx_log_error("NANOPB", "Init error: %d", err);
00344 *         return err;
00345 * }
00346 * @endcode
00347 *
00348 * @see rx_nanopb_test_reset_state() Reset for unit testing
00349 * @see s_initialized Module state flag
00350 *
00351 * @par NASA Power of 10 Compliance
00352 * - Rule 5: [OK] 1 pre-condition check, 1 post-condition verification
00353 * - Rule 7: [OK] Return value indicates success/failure
00354 *
00355 * @since Version 1.0.0
00356 */
00357 rx_err_t rx_nanopb_init(void)
00358 {
00359     /* Pre-condition: Check not already initialized */
00360     if (s_initialized) {
00361         return k_rx_err_invalid_state;
00362     }
00363
00364     s_initialized = true;
00365
00366     /* Post-condition: Verify initialization succeeded */
00367
00368     return k_rx_ok;
00369 }
00370
00371 /**
00372 * @brief Reset module state for unit testing
00373 *
00374 * @details
00375 * Resets the nanopb wrapper module to its uninitialized state. This function
00376 * is intended exclusively for unit testing to ensure clean test isolation.
00377 *
00378 * @par Algorithm Steps

```

```

00379 * 1. Set s_initialized to false
00380 *
00381 * @warning **FOR UNIT TESTING ONLY** - Do not call in production code.
00382 *     Calling this while encode/decode operations are in progress
00383 *     will cause undefined behavior.
00384 *
00385 * @pre None
00386 *
00387 * @post s_initialized == false
00388 * @post rx_nanopb_init() must be called before encode/decode functions
00389 *
00390 * @note Not thread-safe
00391 * @note No return value - always succeeds
00392 *
00393 * @par Performance: ~100 ns @ 240 MHz
00394 * @par Stack Usage: 0 bytes (inline capable)
00395 *
00396 * @par Example - Test Fixture Setup:
00397 * @code
00398 * // Unity test framework setup
00399 * void setUp(void) {
00400 *     // Reset to clean state before each test
00401 *     rx_nanopb_test_reset_state();
00402 *
00403 *     // Re-initialize for tests that need it
00404 *     rx_err_t err = rx_nanopb_init();
00405 *     TEST_ASSERT_EQUAL(k_rx_ok, err);
00406 * }
00407 *
00408 * void tearDown(void) {
00409 *     // Clean up after each test
00410 *     rx_nanopb_test_reset_state();
00411 * }
00412 * @endcode
00413 *
00414 * @par Example - Testing Uninitialized Behavior:
00415 * @code
00416 * void test_encode_fails_when_not_initialized(void) {
00417 *     // Ensure module is NOT initialized
00418 *     rx_nanopb_test_reset_state();
00419 *
00420 *     star_v1_SetVelocityRequest msg = {};
00421 *     uint8_t buffer[512];
00422 *     uint32_t len;
00423 *
00424 *     rx_err_t err = rx_nanopb_encode_velocity_request(&msg, buffer, 512, &len);
00425 *
00426 *     // Should fail with not initialized error
00427 *     TEST_ASSERT_EQUAL(k_rx_err_not_initialized, err);
00428 * }
00429 * @endcode
00430 *
00431 * @see rx_nanopb_init() Initialize module after reset
00432 * @see s_initialized Module state flag
00433 *
00434 * @since Version 1.0.0
00435 */
00436 void rx_nanopb_test_reset_state(void)
00437 {
00438     s_initialized = false;
00439 }
00440
00441 /*
=====
00442 * SetVelocityRequest Encode/Decode
00443 *
=====
00444 */
00445
00446 /**
00447 * @brief Encode SetVelocityRequest message to Protocol Buffer wire format
00448 *
00449 * @details
00450 * Serializes a SetVelocityRequest message containing motor velocity commands
00451 * into Protocol Buffer binary format for transmission over SPI or USB.
00452 *
00453 * @par Algorithm Steps
00454 * 1. Validate msg pointer (not nullptr)
00455 * 2. Validate buffer pointer (not nullptr)
00456 * 3. Validate len pointer (not nullptr)
00457 * 4. Verify module is initialized
00458 * 5. Verify buffer size is adequate
00459 * 6. Create nanopb output stream from buffer
00460 * 7. Encode message using generated field descriptors
00461 * 8. Return encoded length in *len
00462 * 9. Verify encoded length is within bounds (post-condition)
00463 *

```

```

00464 * @dot
00465 * digraph encode_velocity_request {
00466 *   rankdir=TB;
00467 *   node [shape=box, style=rounded];
00468 *
00469 *   start [label="encode_velocity_request()"];
00470 *   check_null [label="msg/buffer/len\n== nullptr?"];
00471 *   err_arg [label="return\nk_rx_err_invalid_arg", fillcolor=lightcoral, style=filled];
00472 *   check_init [label="s_initialized?"];
00473 *   err_init [label="return\nk_rx_err_not_initialized", fillcolor=lightcoral, style=filled];
00474 *   check_size [label="buffer_size >=\ns_nanopb_buffer_size?"];
00475 *   err_size [label="return\nk_rx_err_invalid_size", fillcolor=lightcoral, style=filled];
00476 *   create_stream [label="pb_ostream_from_buffer()"];
00477 *   encode [label="pb_encode()"];
00478 *   check_encode [label="encode\nsuccess?"];
00479 *   err_encode [label="return\nk_rx_err_invalid_size", fillcolor=lightcoral, style=filled];
00480 *   set_len [label="*len = bytes_written"];
00481 *   check_post [label="*len <=\ns_nanopb_buffer_size?"];
00482 *   err_post [label="return\nk_rx_err_invalid_size", fillcolor=lightcoral, style=filled];
00483 *   success [label="return k_rx_ok", fillcolor=lightgreen, style=filled];
00484 *
00485 *   start -> check_null;
00486 *   check_null -> err_arg [label="yes"];
00487 *   check_null -> check_init [label="no"];
00488 *   check_init -> err_init [label="no"];
00489 *   check_init -> check_size [label="yes"];
00490 *   check_size -> err_size [label="no"];
00491 *   check_size -> create_stream [label="yes"];
00492 *   create_stream -> encode;
00493 *   encode -> check_encode;
00494 *   check_encode -> err_encode [label="no"];
00495 *   check_encode -> set_len [label="yes"];
00496 *   set_len -> check_post;
00497 *   check_post -> err_post [label="no"];
00498 *   check_post -> success [label="yes"];
00499 * }
00500 * @enddot
00501 *
00502 *
00503 *
00504 * @pre rx_nanopb_init() must have been called successfully
00505 * @pre msg must be fully populated with valid data
00506 * @pre buffer_size >= s_nanopb_buffer_size (512)
00507 *
00508 * @post buffer[0..*len-1] contains valid Protocol Buffer wire-format data
00509 * @post *len <= s_nanopb_buffer_size
00510 *
00511 * @note Not thread-safe - protect with mutex if called from multiple tasks
00512 *
00513 * @par Performance
00514 * - Execution time: ~20 us @ 240 MHz
00515 * - Stack usage: ~96 bytes
00516 *
00517 * @par Example - Basic Usage:
00518 * @code
00519 * star_v1_SetVelocityRequest request = star_v1_SetVelocityRequest_init_zero;
00520 *
00521 * // Populate command
00522 * request.command.front_left_velocity_mps = 1.0;
00523 * request.command.front_right_velocity_mps = 1.0;
00524 * request.command.back_left_velocity_mps = 1.0;
00525 * request.command.back_right_velocity_mps = 1.0;
00526 * request.command.sequence = g_sequence++;
00527 *
00528 * uint8_t buffer[512];
00529 * uint32_t encoded_len;
00530 *
00531 * rx_err_t err = rx_nanopb_encode_velocity_request(&request, buffer, sizeof(buffer), &encoded_len);
00532 * if (err == k_rx_ok) {
00533 *   spi_send(buffer, encoded_len);
00534 * }
00535 * @endcode
00536 *
00537 * @see rx_nanopb_decode_velocity_request() Decode counterpart
00538 * @see star_v1_SetVelocityRequest_fields nanopb field descriptors
00539 *
00540 * @par NASA Power of 10 Compliance
00541 * - Rule 5: [OK] 3 pre-conditions, 1 post-condition
00542 * - Rule 7: [OK] pb_encode() return value checked
00543 *
00544 * @since Version 1.0.0
00545 */
00546 rx_err_t rx_nanopb_encode_velocity_request(const star_v1_SetVelocityRequest* msg,
00547                                           uint8_t* buffer,
00548                                           const uint32_t buffer_size,
00549                                           uint32_t* len)
00550 {

```

```

00551  /* Pre-condition 1: nullptr pointer checks */
00552  if (msg == nullptr || buffer == nullptr || len == nullptr) {
00553      return k_rx_err_invalid_arg;
00554  }
00555
00556  /* Pre-condition 2: Module initialized */
00557  if (!s_initialized) {
00558      return k_rx_err_not_initialized;
00559  }
00560
00561  /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
00562  if (buffer_size < s_nanopb_buffer_size) {
00563      return k_rx_err_invalid_size;
00564  }
00565
00566  pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
00567
00568  /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
00569  (void)pb_encode(&stream, star_v1_SetVelocityRequest_fields, msg);
00570  *len = stream.bytes_written;
00571
00572  return k_rx_ok;
00573 }
00574
00575 /**
00576  * @brief Decode SetVelocityRequest message from Protocol Buffer wire format
00577  *
00578  * @details
00579  * Deserializes a SetVelocityRequest message received from RPi5 over SPI.
00580  * This is the primary command message for motor velocity control in the
00581  * STAR system.
00582  *
00583  * @par Algorithm Steps
00584  * 1. Validate buffer pointer (not nullptr)
00585  * 2. Validate msg pointer (not nullptr)
00586  * 3. Validate len (not zero, not too large)
00587  * 4. Verify module is initialized
00588  * 5. Initialize msg to zero (clear previous state)
00589  * 6. Create nanopb input stream from buffer
00590  * 7. Decode message using generated field descriptors
00591  * 8. Return decoded message in *msg
00592  *
00593  * @dot
00594  * digraph decode_velocity_request {
00595  *     rankdir=TB;
00596  *     node [shape=box, style=rounded];
00597  *
00598  *     start [label="decode_velocity_request()"];
00599  *     check_null [label="buffer/msg\n== nullptr?"];
00600  *     err_arg1 [label="return\nk_rx_err_invalid_arg", fillcolor=lightcoral, style=filled];
00601  *     check_len [label="len == 0 or\nlen > buffer_size?"];
00602  *     err_arg2 [label="return\nk_rx_err_invalid_arg", fillcolor=lightcoral, style=filled];
00603  *     check_init [label="s_initialized?"];
00604  *     err_init [label="return\nk_rx_err_not_initialized", fillcolor=lightcoral, style=filled];
00605  *     zero_msg [label="*msg = init_zero"];
00606  *     create_stream [label="pb_istream_from_buffer()"];
00607  *     decode [label="pb_decode()"];
00608  *     check_decode [label="decode\nsuccess?"];
00609  *     err_proto [label="return\nk_rx_err_protocol_error", fillcolor=lightcoral, style=filled];
00610  *     success [label="return k_rx_ok", fillcolor=lightgreen, style=filled];
00611  *
00612  *     start -> check_null;
00613  *     check_null -> err_arg1 [label="yes"];
00614  *     check_null -> check_len [label="no"];
00615  *     check_len -> err_arg2 [label="yes"];
00616  *     check_len -> check_init [label="no"];
00617  *     check_init -> err_init [label="no"];
00618  *     check_init -> zero_msg [label="yes"];
00619  *     zero_msg -> create_stream;
00620  *     create_stream -> decode;
00621  *     decode -> check_decode;
00622  *     check_decode -> err_proto [label="no"];
00623  *     check_decode -> success [label="yes"];
00624  * }
00625  * @enddot
00626  *
00627  *
00628  *
00629  * @pre rx_nanopb_init() must have been called successfully
00630  * @pre buffer must contain valid Protocol Buffer wire-format data
00631  * @pre len must match actual encoded message size
00632  *
00633  * @post msg fully populated with decoded field values
00634  * @post Unset fields have default/zero values
00635  *
00636  * @note Not thread-safe - protect with mutex if called from multiple tasks
00637  *

```

```

00638 * @par Performance
00639 * - Execution time: ~15 us @ 240 MHz
00640 * - Stack usage: ~96 bytes
00641 *
00642 * @par Example - Basic Usage:
00643 * @code
00644 * // Receive encoded message over SPI
00645 * uint8_t rx_buffer[512];
00646 * uint32_t rx_len;
00647 * spi_receive(rx_buffer, sizeof(rx_buffer), &rx_len);
00648 *
00649 * // Decode the message
00650 * star_v1_SetVelocityRequest request;
00651 * rx_err_t err = rx_nanopb_decode_velocity_request(rx_buffer, rx_len, &request);
00652 *
00653 * if (err == k_rx_ok) {
00654 *     // Apply velocities to motors
00655 *     motor_set_velocity(MOTOR_FL, request.command.front_left_velocity_mps);
00656 *     motor_set_velocity(MOTOR_FR, request.command.front_right_velocity_mps);
00657 *     motor_set_velocity(MOTOR_BL, request.command.back_left_velocity_mps);
00658 *     motor_set_velocity(MOTOR_BR, request.command.back_right_velocity_mps);
00659 * } else if (err == k_rx_err_protocol_error) {
00660 *     // Request retransmit
00661 *     comm_send_nack();
00662 * }
00663 * @endcode
00664 *
00665 * @see rx_nanopb_encode_velocity_request() Encode counterpart
00666 * @see rx_nanopb_encode_velocity_response() Send response after decoding
00667 *
00668 * @par NASA Power of 10 Compliance
00669 * - Rule 5: [OK] 3 pre-conditions
00670 * - Rule 7: [OK] pb_decode() return value checked
00671 *
00672 * @since Version 1.0.0
00673 */
00674 rx_err_t rx_nanopb_decode_velocity_request(const uint8_t* buffer,
00675                                           const uint32_t len,
00676                                           star_v1_SetVelocityRequest* msg)
00677 {
00678     /* Pre-condition 1: nullptr pointer checks */
00679     if (buffer == nullptr || msg == nullptr) {
00680         return k_rx_err_invalid_arg;
00681     }
00682
00683     /* Pre-condition 2: Length validation */
00684     if (len == 0 || len > s_nanopb_buffer_size) {
00685         return k_rx_err_invalid_arg;
00686     }
00687
00688     /* Pre-condition 3: Module initialized */
00689     if (!s_initialized) {
00690         return k_rx_err_not_initialized;
00691     }
00692
00693     /* Initialize message to default values */
00694     *msg = (star_v1_SetVelocityRequest)star_v1_SetVelocityRequest_init_zero;
00695
00696     pb_istream_t stream = pb_istream_from_buffer(buffer, len);
00697
00698     if (!pb_decode(&stream, star_v1_SetVelocityRequest_fields, msg)) {
00699         return k_rx_err_protocol_error;
00700     }
00701
00702     return k_rx_ok;
00703 }
00704
00705 /*
=====
00706 * SetVelocityResponse Encode/Decode
=====
00707 */
00708 /**
00709 **
00710 ** @brief Encode SetVelocityResponse message to Protocol Buffer wire format
00711 *
00712 *
00713 * @details
00714 * Serializes a SetVelocityResponse message as acknowledgment to a velocity
00715 * command. This response is sent back to RPi5 after processing a
00716 * SetVelocityRequest to confirm receipt and indicate success/failure.
00717 *
00718 * @par Algorithm Steps
00719 * 1. Validate msg pointer (not nullptr)
00720 * 2. Validate buffer pointer (not nullptr)
00721 * 3. Validate len pointer (not nullptr)
00722 * 4. Verify module is initialized

```

```

00723 * 5. Verify buffer size is adequate
00724 * 6. Create nanopb output stream from buffer
00725 * 7. Encode message using generated field descriptors
00726 * 8. Return encoded length in *len
00727 * 9. Verify encoded length is within bounds (post-condition)
00728 *
00729 *
00730 *
00731 * @pre rx_nanopb_init() must have been called successfully
00732 * @pre msg should be populated via rx_nanopb_create_response_header()
00733 * @pre buffer_size >= s_nanopb_buffer_size (512)
00734 *
00735 * @post buffer[0..*len-1] contains valid Protocol Buffer wire-format data
00736 * @post *len <= s_nanopb_buffer_size
00737 *
00738 * @note Not thread-safe - protect with mutex if called from multiple tasks
00739 *
00740 * @par Performance
00741 * - Execution time: ~15 us @ 240 MHz
00742 * - Stack usage: ~48 bytes
00743 *
00744 * @par Example - Responding to Velocity Command:
00745 * @code
00746 * // After successfully processing velocity command
00747 * star_v1_SetVelocityResponse response = star_v1_SetVelocityResponse_init_zero;
00748 *
00749 * // Create header with success status
00750 * rx_nanopb_create_response_header(&response.header,
00751 *                                 star_v1_Status_STATUS_OK,
00752 *                                 request.header.request_id);
00753 *
00754 * // Encode and send
00755 * uint8_t buffer[512];
00756 * uint32_t len;
00757 * rx_err_t err = rx_nanopb_encode_velocity_response(&response, buffer, sizeof(buffer), &len);
00758 * if (err == k_rx_ok) {
00759 *     spi_send(buffer, len);
00760 * }
00761 * @endcode
00762 *
00763 * @see rx_nanopb_decode_velocity_request() Decode incoming request first
00764 * @see rx_nanopb_create_response_header() Create header with status
00765 *
00766 * @par NASA Power of 10 Compliance
00767 * - Rule 5: [OK] 3 pre-conditions, 1 post-condition
00768 * - Rule 7: [OK] pb_encode() return value checked
00769 *
00770 * @since Version 1.0.0
00771 */
00772 rx_err_t rx_nanopb_encode_velocity_response(const star_v1_SetVelocityResponse* msg,
00773                                           uint8_t* buffer,
00774                                           const uint32_t buffer_size,
00775                                           uint32_t* len)
00776 {
00777     /* Pre-condition 1: nullptr pointer checks */
00778     if (msg == nullptr || buffer == nullptr || len == nullptr) {
00779         return k_rx_err_invalid_arg;
00780     }
00781
00782     /* Pre-condition 2: Module initialized */
00783     if (!s_initialized) {
00784         return k_rx_err_not_initialized;
00785     }
00786
00787     /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
00788     if (buffer_size < s_nanopb_buffer_size) {
00789         return k_rx_err_invalid_size;
00790     }
00791
00792     pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
00793
00794     /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
00795     (void)pb_encode(&stream, star_v1_SetVelocityResponse_fields, msg);
00796     *len = stream.bytes_written;
00797
00798     return k_rx_ok;
00799 }
00800
00801 /*
=====
00802 * EmergencyStopRequest Encode/Decode
00803 *
=====
00804 */
00805
00806 /**
00807 * @brief Decode EmergencyStopRequest message from Protocol Buffer wire format

```

```

00808 *
00809 * @details
00810 * Deserializes an emergency stop command received from RPi5. This is a
00811 * safety-critical message that triggers immediate motor shutdown.
00812 *
00813 * @par Safety-Critical Processing
00814 * @warning E-Stop commands must be processed immediately. Consider stopping
00815 *         motors BEFORE attempting to decode if message type is identified.
00816 *
00817 * @par Algorithm Steps
00818 * 1. Validate buffer pointer (not nullptr)
00819 * 2. Validate msg pointer (not nullptr)
00820 * 3. Validate len (not zero, not too large)
00821 * 4. Verify module is initialized
00822 * 5. Initialize msg to zero
00823 * 6. Create nanopb input stream from buffer
00824 * 7. Decode message using generated field descriptors
00825 *
00826 *
00827 *
00828 * @pre rx_nanopb_init() must have been called
00829 * @pre buffer contains valid Protocol Buffer data
00830 *
00831 * @post msg fully populated with E-Stop request fields
00832 *
00833 * @note Not thread-safe
00834 * @warning Always stop motors regardless of decode result
00835 *
00836 * @par Performance
00837 * - Execution time: ~10 us @ 240 MHz
00838 * - Stack usage: ~32 bytes
00839 *
00840 * @par Example - E-Stop Handling:
00841 * @code
00842 * void handle_estop_message(const uint8_t* buffer, uint32_t len) {
00843 *     // FIRST: Stop motors immediately (safety-critical)
00844 *     motor_emergency_stop_all();
00845 *
00846 *     // THEN: Decode message for response
00847 *     star_v1_EmergencyStopRequest estop_req;
00848 *     rx_err_t err = rx_nanopb_decode_estop_request(buffer, len, &estop_req);
00849 *
00850 *     // Send response (motors already stopped)
00851 *     star_v1_EmergencyStopResponse response = {};
00852 *     rx_nanopb_create_response_header(&response.header,
00853 *                                     star_v1_Status_STATUS_OK,
00854 *                                     estop_req.header.request_id);
00855 *     // ... encode and send response
00856 * }
00857 * @endcode
00858 *
00859 * @see rx_nanopb_encode_estop_response() Send acknowledgment
00860 *
00861 * @since Version 1.0.0
00862 */
00863 rx_err_t rx_nanopb_decode_estop_request(const uint8_t*          buffer,
00864                                       const uint32_t          len,
00865                                       star_v1_EmergencyStopRequest* msg)
00866 {
00867     /* Pre-condition 1: nullptr pointer checks */
00868     if (buffer == nullptr || msg == nullptr) {
00869         return k_rx_err_invalid_arg;
00870     }
00871
00872     /* Pre-condition 2: Length validation */
00873     if (len == 0 || len > s_nanopb_buffer_size) {
00874         return k_rx_err_invalid_arg;
00875     }
00876
00877     /* Pre-condition 3: Module initialized */
00878     if (!s_initialized) {
00879         return k_rx_err_not_initialized;
00880     }
00881
00882     /* Initialize message to default values */
00883     *msg = (star_v1_EmergencyStopRequest)star_v1_EmergencyStopRequest_init_zero;
00884
00885     pb_istream_t stream = pb_istream_from_buffer(buffer, len);
00886
00887     if (!pb_decode(&stream, star_v1_EmergencyStopRequest_fields, msg)) {
00888         return k_rx_err_protocol_error;
00889     }
00890
00891     return k_rx_ok;
00892 }
00893
00894 /**

```



```

00895 * @brief Encode EmergencyStopResponse message to Protocol Buffer wire format
00896 *
00897 * @details
00898 * Serializes an EmergencyStopResponse message to confirm E-Stop processing.
00899 * This response informs RPi5 that the RX72N has received the E-Stop command
00900 * and has entered safe state (motors stopped).
00901 *
00902 *
00903 *
00904 * @pre rx_nanopb_init() must have been called
00905 * @pre Motors should already be stopped before sending response
00906 *
00907 * @post buffer contains valid Protocol Buffer wire-format data
00908 *
00909 * @note Not thread-safe
00910 *
00911 * @par Performance
00912 * - Execution time: ~10 us @ 240 MHz
00913 * - Stack usage: ~32 bytes
00914 *
00915 * @see rx_nanopb_decode_estop_request() Decode incoming E-Stop first
00916 *
00917 * @since Version 1.0.0
00918 */
00919 rx_err_t rx_nanopb_encode_estop_response(const star_v1_EmergencyStopResponse* msg,
00920                                         uint8_t* buffer,
00921                                         const uint32_t buffer_size,
00922                                         uint32_t* len)
00923 {
00924     /* Pre-condition 1: nullptr pointer checks */
00925     if (msg == nullptr || buffer == nullptr || len == nullptr) {
00926         return k_rx_err_invalid_arg;
00927     }
00928
00929     /* Pre-condition 2: Module initialized */
00930     if (!s_initialized) {
00931         return k_rx_err_not_initialized;
00932     }
00933
00934     /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
00935     if (buffer_size < s_nanopb_buffer_size) {
00936         return k_rx_err_invalid_size;
00937     }
00938
00939     pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
00940
00941     /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
00942     (void)pb_encode(&stream, star_v1_EmergencyStopResponse_fields, msg);
00943     *len = stream.bytes_written;
00944
00945     return k_rx_ok;
00946 }
00947
00948 /*
=====
00949 * SetPidGainsRequest Encode/Decode
00950 *
=====
00951 */
00952 /**
00953 * @brief Decode SetPidGainsRequest message from Protocol Buffer wire format
00954 *
00955 * @details
00956 * Deserializes a PID gains update command from RPi5. This message contains
00957 * new PID controller configuration (gains and limits) to be applied to one
00958 * or more motors.
00959 *
00960 *
00961 * @par Message Processing
00962 * 1. Validate input parameters (buffer, length, message pointer)
00963 * 2. Check module initialization state
00964 * 3. Initialize message structure to zero
00965 * 4. Create nanopb input stream from buffer
00966 * 5. Decode using star_v1_SetPidGainsRequest_fields descriptor
00967 * 6. Return success or protocol error
00968 *
00969 * @par PID Configuration Fields
00970 * The decoded message contains:
00971 * - header: Standard RequestHeader with request_id
00972 * - pid_config: PidConfig structure with:
00973 *   - kp: Proportional gain
00974 *   - ki: Integral gain
00975 *   - kd: Derivative gain
00976 *   - output_min_percent: Minimum PWM duty cycle (%)
00977 *   - output_max_percent: Maximum PWM duty cycle (%)
00978 *   - integral_min: Anti-windup lower bound
00979 *   - integral_max: Anti-windup upper bound

```

```

00980 * - motor_id: Target motor (0-3 for specific motor, -1 for all motors)
00981 *
00982 *
00983 *
00984 * @pre rx_nanopb_init() must be called first
00985 * @pre buffer must point to valid wire-format data
00986 * @pre len must match actual encoded message size
00987 *
00988 * @post msg fully populated with decoded PID gains
00989 * @post msg->has_pid_config == true if gains present
00990 *
00991 * @invariant msg initialized to zero before decode (prevents stale data)
00992 *
00993 * @note **Thread Safety:** Not thread-safe (uses shared nanopb state)
00994 * @note **Re-entrancy:** NOT reentrant. Single-threaded decode only.
00995 * @note **Performance:** ~18 us @ 240 MHz (lightweight message)
00996 *
00997 * @warning **Gain Validation:** This function does NOT validate gain ranges.
00998 *         Caller must validate that Kp, Ki, Kd are non-negative and within
00999 *         safe operating bounds before applying to PID controllers.
01000 * @warning **Motor ID:** motor_id == -1 means apply to ALL motors. Caller
01001 *         must handle this case explicitly.
01002 *
01003 * @par Example - Single Motor Update:
01004 * @code
01005 * star_v1_SetPidGainsRequest request;
01006 * rx_err_t err = rx_nanopb_decode_pid_gains_request(spi_buffer, spi_len, &request);
01007 *
01008 * if (err == k_rx_ok && request.has_pid_config) {
01009 *     // Convert protobuf gains to firmware structure
01010 *     pid_gains_t gains = {
01011 *         .kp = (float)request.pid_config.kp,
01012 *         .ki = (float)request.pid_config.ki,
01013 *         .kd = (float)request.pid_config.kd,
01014 *         .output_min = (float)request.pid_config.output_min_percent,
01015 *         .output_max = (float)request.pid_config.output_max_percent,
01016 *         .integral_min = (float)request.pid_config.integral_min,
01017 *         .integral_max = (float)request.pid_config.integral_max,
01018 *         .update_pending = true
01019 *     };
01020 *
01021 *     // Apply to target motor(s)
01022 *     if (request.motor_id == -1) {
01023 *         // Apply to all motors
01024 *         shared_data_set_pid_gains(&gains);
01025 *     } else if (request.motor_id >= 0 && request.motor_id < 4) {
01026 *         // Apply to specific motor (motor_id 0-3)
01027 *         // Note: Current firmware applies to all motors via shared_data
01028 *         shared_data_set_pid_gains(&gains);
01029 *     }
01030 * } else if (err == k_rx_err_protocol_error) {
01031 *     // Malformed message - request retransmit
01032 *     rx_log_error("NANOPB", "Failed to decode PID gains request");
01033 *     comm_send_nack();
01034 * }
01035 * @endcode
01036 *
01037 * @par Example - All Motors Update:
01038 * @code
01039 * // RPi5 sends motor_id = -1 to update all 4 motors
01040 * star_v1_SetPidGainsRequest request;
01041 * rx_err_t err = rx_nanopb_decode_pid_gains_request(buffer, len, &request);
01042 *
01043 * if (err == k_rx_ok && request.motor_id == -1) {
01044 *     rx_log_info("COMM", "Applying PID gains to ALL motors");
01045 *     // shared_data_set_pid_gains() applies to all 4 PID controllers
01046 *     shared_data_set_pid_gains(&converted_gains);
01047 * }
01048 * @endcode
01049 *
01050 * @see rx_nanopb_encode_pid_gains_response() Send acknowledgment
01051 * @see shared_data_set_pid_gains() Apply gains to motor controllers
01052 * @see internal_apply_pid_updates() Motor task applies pending gains
01053 * @see star_v1_SetPidGainsRequest_fields nanopb field descriptors
01054 *
01055 * @par NASA Power of 10 Compliance:
01056 * - **Rule 5:** [PASS] 3 preconditions, 2 postconditions documented
01057 * - **Rule 7:** [PASS] pb_decode() return value checked
01058 *
01059 * @since Version 1.0.0
01060 *
01061 * @callgraph
01062 * @callergraph
01063 */
01064 rx_err_t rx_nanopb_decode_pid_gains_request(const uint8_t* buffer,
01065                                           const uint32_t len,
01066                                           star_v1_SetPidGainsRequest* msg)

```

```

01067 {
01068     /* Pre-condition 1: nullptr pointer checks */
01069     if (buffer == nullptr || msg == nullptr) {
01070         return k_rx_err_invalid_arg;
01071     }
01072
01073     /* Pre-condition 2: Length validation */
01074     if (len == 0 || len > s_nanopb_buffer_size) {
01075         return k_rx_err_invalid_arg;
01076     }
01077
01078     /* Pre-condition 3: Module initialized */
01079     if (!s_initialized) {
01080         return k_rx_err_not_initialized;
01081     }
01082
01083     /* Initialize message to default values (NASA Rule 5 - prevent stale data) */
01084     *msg = (star_v1_SetPidGainsRequest)star_v1_SetPidGainsRequest_init_zero;
01085
01086     pb_istream_t stream = pb_istream_from_buffer(buffer, len);
01087
01088     if (!pb_decode(&stream, star_v1_SetPidGainsRequest_fields, msg)) {
01089         return k_rx_err_protocol_error;
01090     }
01091
01092     return k_rx_ok;
01093 }
01094
01095 /**
01096  * @brief Encode SetPidGainsResponse message to Protocol Buffer bytes
01097  *
01098  * @details
01099  * Serializes a SetPidGainsResponse message as acknowledgment to a PID gains
01100  * update command. Sent back to RPi5 after processing a SetPidGainsRequest.
01101  *
01102  * The optional pb_callback_t message field is skipped when its encode callback
01103  * is NULL (init_zero default). The success boolean and response header status
01104  * are sufficient for programmatic acknowledgment.
01105  *
01106  *
01107  *
01108  * @pre rx_nanopb_init() must be called before this function
01109  * @pre buffer must point to at least buffer_size bytes of writable memory
01110  * @post On k_rx_ok: buffer[0..*len-1] contains valid wire-format response
01111  * @post On k_rx_ok: *len <= s_nanopb_buffer_size
01112  *
01113  * @note Not thread-safe; call only from Communication Task context
01114  *
01115  * @par Performance: ~15 us @ 240 MHz
01116  *
01117  * @see rx_nanopb_decode_pid_gains_request() Decode the incoming request
01118  * @see rx_nanopb_create_response_header() Create header with status and request_id
01119  * @since Version 1.0.0
01120  *
01121  * @par NASA Power of 10 Compliance:
01122  * - Rule 5: [PASS] 3 preconditions, 2 postconditions documented
01123  * - Rule 7: [PASS] pb_encode() return value checked
01124  */
01125 rx_err_t rx_nanopb_encode_pid_gains_response(const star_v1_SetPidGainsResponse* msg,
01126                                             uint8_t* buffer,
01127                                             const uint32_t buffer_size,
01128                                             uint32_t* len)
01129 {
01130     /* Pre-condition 1: nullptr pointer checks */
01131     if (msg == nullptr || buffer == nullptr || len == nullptr) {
01132         return k_rx_err_invalid_arg;
01133     }
01134
01135     /* Pre-condition 2: Module initialized */
01136     if (!s_initialized) {
01137         return k_rx_err_not_initialized;
01138     }
01139
01140     /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
01141     if (buffer_size < s_nanopb_buffer_size) {
01142         return k_rx_err_invalid_size;
01143     }
01144
01145     pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
01146
01147     /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
01148     (void)pb_encode(&stream, star_v1_SetPidGainsResponse_fields, msg);
01149     *len = stream.bytes_written;
01150
01151     return k_rx_ok;
01152 }
01153

```

```

01154 /**
01155  * @brief Decode SetRetransmitConfigRequest from Protocol Buffer bytes
01156  */
01157 rx_err_t rx_nanopb_decode_retransmit_config_request(const uint8_t*          buffer,
01158                                                     const uint32_t          len,
01159                                                     star_v1_SetRetransmitConfigRequest* msg)
01160 {
01161     /* Pre-condition 1: nullptr pointer checks */
01162     if (buffer == nullptr || msg == nullptr) {
01163         return k_rx_err_invalid_arg;
01164     }
01165
01166     /* Pre-condition 2: Length validation */
01167     if (len == 0 || len > s_nanopb_buffer_size) {
01168         return k_rx_err_invalid_arg;
01169     }
01170
01171     /* Pre-condition 3: Module initialized */
01172     if (!s_initialized) {
01173         return k_rx_err_not_initialized;
01174     }
01175
01176     /* Initialize message to default values (NASA Rule 5 - prevent stale data) */
01177     *msg = (star_v1_SetRetransmitConfigRequest)star_v1_SetRetransmitConfigRequest_init_zero;
01178
01179     pb_istream_t stream = pb_istream_from_buffer(buffer, len);
01180
01181     if (!pb_decode(&stream, star_v1_SetRetransmitConfigRequest_fields, msg)) {
01182         return k_rx_err_protocol_error;
01183     }
01184
01185     return k_rx_ok;
01186 }
01187
01188 /*
=====
01189 * Telemetry Encode/Decode
01190 *
=====
01191 */
01192
01193 /**
01194  * @brief Encode TelemetryData message to Protocol Buffer wire format
01195  *
01196  * @details
01197  * Serializes telemetry data for periodic transmission to RPi5. Contains
01198  * motor states, encoder readings, and sensor data.
01199  * Typically called at 10-100 Hz depending on telemetry requirements.
01200  *
01201  * @par Telemetry Contents
01202  * | Field Category | Fields | Update Rate |
01203  * |-----|-----|-----|
01204  * | Motor velocities | 4 x double (m/s) | 100 Hz |
01205  * | Motor currents | 4 x double (A) | 100 Hz |
01206  * | Encoder positions | 4 x int32 (ticks) | 100 Hz |
01207  * | Temperature | system temp (degC) | 1 Hz |
01208  *
01209  *
01210  *
01211  * @pre rx_nanopb_init() must have been called
01212  * @pre msg should contain current sensor/motor data
01213  *
01214  * @post buffer contains valid Protocol Buffer wire-format telemetry
01215  *
01216  * @note Not thread-safe - protect with mutex if called from multiple tasks
01217  *
01218  * @par Performance
01219  * - Execution time: ~30 us @ 240 MHz
01220  * - Stack usage: ~128 bytes
01221  *
01222  * @par Example - Periodic Telemetry:
01223  * @code
01224  void telemetry_task(ULONG arg) {
01225     while (1) {
01226         // Gather telemetry data
01227         star_v1_TelemetryData telemetry = {};
01228         telemetry.motor_front_left_velocity_mps = motor_get_velocity(0);
01229         telemetry.motor_front_left_current_a = motor_get_current(0);
01230         // ... populate all fields
01231         telemetry.timestamp_us = rx_time_get_us();
01232
01233         // Encode and send
01234         uint8_t buffer[512];
01235         uint32_t len;
01236         rx_err_t err = rx_nanopb_encode_telemetry(&telemetry, buffer, sizeof(buffer), &len);
01237         if (err == k_rx_ok) {
01238             spi_send(buffer, len);

```

```

01239 *      }
01240 *
01241 *      // Sleep until next telemetry period (10ms = 100Hz)
01242 *      tx_thread_sleep(TX_TIMER_TICKS_PER_SECOND / 100);
01243 *  }
01244 * }
01245 * @endcode
01246 *
01247 * @see star_v1_TelemetryData Generated telemetry message structure
01248 *
01249 * @par NASA Power of 10 Compliance
01250 * - Rule 5: [OK] 3 pre-conditions, 1 post-condition
01251 *
01252 * @since Version 1.0.0
01253 */
01254 rx_err_t rx_nanopb_encode_telemetry(const star_v1_TelemetryData* msg,
01255                                     uint8_t* buffer,
01256                                     const uint32_t buffer_size,
01257                                     uint32_t* len)
01258 {
01259     /* Pre-condition 1: nullptr pointer checks */
01260     if (msg == nullptr || buffer == nullptr || len == nullptr) {
01261         return k_rx_err_invalid_arg;
01262     }
01263
01264     /* Pre-condition 2: Module initialized */
01265     if (!s_initialized) {
01266         return k_rx_err_not_initialized;
01267     }
01268
01269     /* Pre-condition 3: Buffer size validation (NASA Rule 5 - buffer overflow prevention) */
01270     if (buffer_size < s_nanopb_buffer_size) {
01271         return k_rx_err_invalid_size;
01272     }
01273
01274     /* Wrap TelemetryData inside a WireMessage so the payload on the wire
01275     * matches the envelope the gateway dispatcher unmarshals: the Go
01276     * dispatcher (star_gateway/internal/dispatcher/dispatcher.go) calls
01277     * proto.Unmarshal into a &star_v1.WireMessage{} and routes on the
01278     * oneof. Without the wrap the dispatcher drops every frame silently
01279     * at debug level with "failed to unmarshal wire message" and nothing
01280     * reaches ROS2.
01281     */
01282     /* s_wire is file-scope (static) rather than a stack local: the union
01283     * in star_v1_WireMessage spans every oneof variant (TelemetryData,
01284     * TransportDiagnostics, PidConfig, ...) so sizeof(star_v1_WireMessage)
01285     * can be hundreds of bytes. Putting it on the telemetry_task 2 KB
01286     * stack alongside s_telem_buffer exhausts headroom. Single-writer
01287     * (telemetry_task is the only producer), so the static has no
01288     * reentrancy concern. */
01289     static star_v1_WireMessage s_wire;
01290     s_wire.which_payload = star_v1_WireMessage_telemetry_data_tag;
01291     s_wire.payload.telemetry_data = *msg;
01292
01293     pb_ostream_t stream = pb_ostream_from_buffer(buffer, buffer_size);
01294
01295     /* pb_encode always succeeds for valid msg with sufficient buffer (pre-validated above) */
01296     (void)pb_encode(&stream, star_v1_WireMessage_fields, &s_wire);
01297     *len = stream.bytes_written;
01298
01299     return k_rx_ok;
01300 }
01301
01302 /*
=====
01303 * Helper Functions
01304 *
=====
01305 */
01306
01307 /**
01308 * @brief Create VelocityCommand for 4 independent motor velocities
01309 *
01310 * @details
01311 * Populates a VelocityCommand structure from parameters. Use for mecanum
01312 * or omnidirectional drive robots where each motor requires independent
01313 * velocity control. Performs range validation on all velocity values.
01314 *
01315 * @par Algorithm Steps
01316 * 1. Validate cmd pointer (not nullptr)
01317 * 2. Validate params pointer (not nullptr)
01318 * 3. Validate all velocities in range [k_velocity_mps_min, k_velocity_mps_max]
01319 * 4. Initialize cmd to zero (clear previous state)
01320 * 5. Copy velocity values from params to cmd
01321 * 6. Copy sequence number
01322 * 7. Set timestamp to 0 (caller sets if needed)
01323 */

```

```

01324 *
01325 *
01326 * @pre cmd must not be nullptr
01327 * @pre params must not be nullptr
01328 * @pre All velocity values must be in valid range
01329 *
01330 * @post cmd fully populated with velocity values from params
01331 * @post cmd->timestamp_us == 0
01332 *
01333 * @note Thread-safe (no shared state)
01334 *
01335 * @par Performance
01336 * - Execution time: ~2 us @ 240 MHz
01337 * - Stack usage: ~16 bytes
01338 *
01339 * @par Example - Basic Usage:
01340 * @code
01341 * rx_velocity_command_params_t params = {
01342 *     .front_left_mps = 1.0,
01343 *     .front_right_mps = 1.0,
01344 *     .back_left_mps = 1.0,
01345 *     .back_right_mps = 1.0,
01346 *     .sequence = g_command_sequence++,
01347 * };
01348 *
01349 * star_v1_VelocityCommand cmd;
01350 * rx_err_t err = rx_nanopb_create_velocity_command(&cmd, &params);
01351 * if (err == k_rx_ok) {
01352 *     // Use cmd in SetVelocityRequest
01353 *     request.command = cmd;
01354 * }
01355 * @endcode
01356 *
01357 * @see rx_velocity_command_params_t Parameter structure
01358 * @see rx_nanopb_create_velocity_command_diff_drive() For differential drive
01359 *
01360 * @since Version 1.0.0
01361 */
01362 rx_err_t rx_nanopb_create_velocity_command(star_v1_VelocityCommand* cmd,
01363                                           const rx_velocity_command_params_t* params)
01364 {
01365     /* Use bitwise | to evaluate both sides without short-circuit branches */
01366     const bool cmd_null = (bool)(cmd == nullptr);
01367     const bool params_null = (bool)(params == nullptr);
01368     if ((bool)((int)cmd_null | (int)params_null)) {
01369         return k_rx_err_invalid_arg;
01370     }
01371
01372     /* Use bitwise | to evaluate all velocity bounds without short-circuit branches */
01373     const bool fl_lo = (bool)(params->front_left_mps < k_velocity_mps_min);
01374     const bool fl_hi = (bool)(params->front_left_mps > k_velocity_mps_max);
01375     const bool fr_lo = (bool)(params->front_right_mps < k_velocity_mps_min);
01376     const bool fr_hi = (bool)(params->front_right_mps > k_velocity_mps_max);
01377     const bool bl_lo = (bool)(params->back_left_mps < k_velocity_mps_min);
01378     const bool bl_hi = (bool)(params->back_left_mps > k_velocity_mps_max);
01379     const bool br_lo = (bool)(params->back_right_mps < k_velocity_mps_min);
01380     const bool br_hi = (bool)(params->back_right_mps > k_velocity_mps_max);
01381     if ((bool)((int)fl_lo | (int)fl_hi | (int)fr_lo | (int)fr_hi | (int)bl_lo | (int)bl_hi |
01382           (int)br_lo | (int)br_hi)) {
01383         return k_rx_err_invalid_arg;
01384     }
01385
01386     *cmd = (star_v1_VelocityCommand)star_v1_VelocityCommand_init_zero;
01387     cmd->front_left_velocity_mps = params->front_left_mps;
01388     cmd->front_right_velocity_mps = params->front_right_mps;
01389     cmd->back_left_velocity_mps = params->back_left_mps;
01390     cmd->back_right_velocity_mps = params->back_right_mps;
01391     cmd->sequence = params->sequence;
01392     cmd->timestamp_us = 0; /* Set by caller if needed */
01393     return k_rx_ok;
01394 }
01395
01396 /**
01397 * @brief Create VelocityCommand in differential drive mode
01398 *
01399 * @details
01400 * Populates a VelocityCommand for differential drive robots where left and
01401 * right sides are controlled together. Front and back motors on the same
01402 * side receive identical velocity values.
01403 *
01404 * @par Velocity Mapping
01405 * \``
01406 * params->left_mps -> cmd->front_left_velocity_mps
01407 *                  -> cmd->back_left_velocity_mps
01408 *
01409 * params->right_mps -> cmd->front_right_velocity_mps
01410 *                   -> cmd->back_right_velocity_mps

```

```

01411 * ``
01412 *
01413 * @par Motion Examples
01414 * | left_mps | right_mps | Robot Motion |
01415 * |-----|-----|-----|
01416 * | +1.0 | +1.0 | Forward |
01417 * | -1.0 | -1.0 | Reverse |
01418 * | +1.0 | -1.0 | Rotate clockwise |
01419 * | -1.0 | +1.0 | Rotate counter-clockwise |
01420 * | +1.0 | +0.5 | Curve right (forward) |
01421 * | +0.5 | +1.0 | Curve left (forward) |
01422 *
01423 *
01424 *
01425 * @pre cmd must not be nullptr
01426 * @pre params must not be nullptr
01427 * @pre Velocities must be in valid range [-1000.0, +1000.0] m/s
01428 *
01429 * @post cmd->front_left_velocity_mps == cmd->back_left_velocity_mps
01430 * @post cmd->front_right_velocity_mps == cmd->back_right_velocity_mps
01431 *
01432 * @note Thread-safe (no shared state)
01433 * @note Internally calls rx_nanopb_create_velocity_command()
01434 *
01435 * @par Performance
01436 * - Execution time: ~3 us @ 240 MHz
01437 * - Stack usage: ~48 bytes
01438 *
01439 * @par Example - Turn Left:
01440 * @code
01441 * // Turn left by making right side faster than left
01442 * rx_velocity_diff_drive_params_t params = {
01443 *     .left_mps = 0.5, // Slower
01444 *     .right_mps = 1.0, // Faster
01445 *     .sequence = g_sequence++,
01446 * };
01447 *
01448 * star_v1_VelocityCommand cmd;
01449 * rx_err_t err = rx_nanopb_create_velocity_command_diff_drive(&cmd, &params);
01450 * // Result: FL=BL=0.5, FR=BR=1.0 (robot curves left)
01451 * @endcode
01452 *
01453 * @see rx_velocity_diff_drive_params_t Parameter structure
01454 * @see rx_nanopb_create_velocity_command() 4-motor independent control
01455 *
01456 * @since Version 1.0.0
01457 */
01458 rx_err_t rx_nanopb_create_velocity_command_diff_drive(star_v1_VelocityCommand* cmd,
01459               const rx_velocity_diff_drive_params_t* params)
01460 {
01461     rx_velocity_command_params_t params_t;
01462
01463     if (cmd == nullptr || params == nullptr) {
01464         return k_rx_err_invalid_arg;
01465     }
01466
01467     /* Differential drive mode: Lock left 2 motors together, right 2 motors together
01468     * Front Left & Back Left = Left side
01469     * Front Right & Back Right = Right side */
01470     params_t.front_left_mps = params->left_mps;
01471     params_t.front_right_mps = params->right_mps;
01472     params_t.back_left_mps = params->left_mps;
01473     params_t.back_right_mps = params->right_mps;
01474     params_t.sequence = params->sequence;
01475
01476     return rx_nanopb_create_velocity_command(cmd, &params_t);
01477 }
01478
01479 /**
01480 * @brief Create ResponseHeader with status and request ID
01481 *
01482 * @details
01483 * Populates a ResponseHeader structure for response messages. Sets the
01484 * status code and optionally echoes the request ID from the original
01485 * request for correlation purposes.
01486 *
01487 * @par Algorithm Steps
01488 * 1. Validate header pointer (early return if nullptr)
01489 * 2. Validate request_id length if provided (early return if too long)
01490 * 3. Initialize header to zero
01491 * 4. Set status field
01492 * 5. If request_id provided, set up nanopb callback for encoding
01493 *
01494 * @par Callback Setup
01495 * When request_id is provided, this function sets up a nanopb callback
01496 * function (internal_encode_string_callback) that will be invoked during
01497 * pb_encode() to serialize the string field.

```

```

01498 *
01499 *
01500 * @pre header must not be nullptr (silently returns if nullptr)
01501 * @pre request_id, if provided, must remain valid until encoding completes
01502 *
01503 * @post header->status == status
01504 * @post If request_id != nullptr: callback configured for encoding
01505 *
01506 * @note Thread-safe (no shared state)
01507 * @note No return value - silently ignores nullptr header
01508 *
01509 * @warning request_id pointer must remain valid until pb_encode() is called.
01510 *         Do not pass stack-allocated strings that may go out of scope.
01511 *
01512 * @par Performance
01513 * - Execution time: ~1 us @ 240 MHz
01514 * - Stack usage: ~8 bytes
01515 *
01516 * @par Example - Success Response:
01517 * @code
01518 * star_v1_SetVelocityResponse response;
01519 *
01520 * // After successfully processing velocity command
01521 * rx_nanopb_create_response_header(&response.header,
01522 *                                star_v1_Status_STATUS_OK,
01523 *                                request.header.request_id);
01524 *
01525 * // Encode and send
01526 * rx_nanopb_encode_velocity_response(&response, buffer, sizeof(buffer), &len);
01527 * @endcode
01528 *
01529 * @par Example - Error Response:
01530 * @code
01531 * // After detecting invalid velocity value
01532 * rx_nanopb_create_response_header(&response.header,
01533 *                                star_v1_Status_STATUS_INVALID_ARGUMENT,
01534 *                                request.header.request_id);
01535 * @endcode
01536 *
01537 * @see star_v1_Status Status enumeration values
01538 * @see internal_encode_string_callback() nanopb callback for string encoding
01539 *
01540 * @since Version 1.0.0
01541 */
01542 void rx_nanopb_create_response_header(star_v1_ResponseHeader* header,
01543                                     const star_v1_Status status,
01544                                     const char* request_id)
01545 {
01546     if (header == nullptr) {
01547         return;
01548     }
01549     if (request_id != nullptr) {
01550         size_t req_id_len = 0;
01551         while (request_id[req_id_len] != '\0') {
01552             req_id_len++;
01553         }
01554         if (req_id_len > s_nanopb_buffer_size) {
01555             return;
01556         }
01557     }
01558 }
01559
01560 *header = (star_v1_ResponseHeader)star_v1_ResponseHeader_init_zero;
01561 header->status = status;
01562
01563 /* Copy request_id string if provided (fixed-size field in generated code) */
01564 if (request_id != nullptr) {
01565     const size_t max_len = sizeof(header->request_id) - 1U;
01566     size_t idx = 0;
01567     while (idx < max_len && request_id[idx] != '\0') {
01568         header->request_id[idx] = request_id[idx];
01569         idx++;
01570     }
01571     header->request_id[idx] = '\0';
01572 }
01573 }

```